TERCERA EDICIÓN

REDES DE COMPUTADORAS

ANDREW S. TANENBAUM

PEARSON

Redes de Computadoras

Tercera edición

Andrew S. Tanenbaum

Vrije Universiteit
Amsterdam, The Netherlands

TRADUCCIÓN:

David Morales Peake

Traductor Profesional

REVISIÓN TÉCNICA:

Gabriel Guerrero Reyes

Doctor en Informática
Universidad de París VI

UNIVERSIDAD DE L
FACULTAD DE I
DPTO. DE DOCUMENTACI
BIBLIOTECA DE
Ing. Edo. García
MONTEVIDEO -

No. de Entrada 053837
/ 8.01
C-4.

SIBUL

Pearson Educación

MÉXICO • ARGENTINA • BRASIL • COLOMBIA • COSTA RICA • CHILE
ESPAÑA • GUATEMALA • PERÚ • PUERTO RICO • VENEZUELA

EDICIÓN EN ESPAÑOL:

SUPERVISOR DE TRADUCCIÓN: TERESA SANZ FALCÓN
SUPERVISOR DE PRODUCCIÓN: ALEJANDRO A. GÓMEZ RUIZ

EDICIÓN EN INGLÉS:

Editorial/production manager: *Camille Trentacoste*
Interior design and composition: *Andrew S. Tanenbaum*
Cover design director: *Jerry Votta*
Cover designer: *Don Martinetti, DM Graphics, Inc.*
Cover concept: *Andrew S. Tanenbaum, from an idea by Marilyn Tremaine*
Interior graphics: *Hadel Studio*
Manufacturing manager: *Alexis R. Heydt*
Acquisitions editor: *Mary Franz*
Editorial Assistant: *Noreen Regina*

TANENBAUM: REDES DE COMPUTADORAS, 3a. Ed.

Traducido del inglés de la obra: **COMPUTER NETWORKS**, Third Edition.

All rights reserved. Authorized translation from English language edition published by Prentice-Hall, Inc.
A Simon & Schuster Company.

Todos los derechos reservados. Traducción autorizada de la edición en inglés publicada por Prentice-Hall, Inc.
A Simon & Schuster Company.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage and retrieval system, without permission in writing from the publisher.

Prohibida la reproducción total o parcial de esta obra, por cualquier medio o método sin autorización por escrito del editor.

Derechos reservados © 1997 respecto a la segunda edición en español publicada por

Prentice Hall Hispanoamericana, S.A.
Calle 4 Nº 25-2º piso Fracc. Ind. Alce Blanco,
Naucalpan de Juárez, Edo. de México,
C.P. 53370

ISBN 968-880-958-6

Miembro de la Cámara Nacional de la Industria Editorial, Reg. Núm. 1524.

ABR

Original English Language Edition Published by Prentice-Hall, Inc.
A Simon & Schuster Company
Copyright © MCMXCVI
All rights reserved

004.6
T164rE
e.4.

2000

ISBN 0-13-349945-6

IMPRESO EN MÉXICO/PRINTED IN MEXICO

A Suzanne, Barbara, Marvin y el pequeño Bram

CONTENIDO

PREFACIO xv

1 INTRODUCCIÓN 1

1.1 USOS DE LAS REDES DE COMPUTADORAS 3
- 1.1.1 Redes para compañías 3
- 1.1.2 Redes para la gente 4
- 1.1.3 Consideraciones sociales 6

1.2 *HARDWARE* DE RED 7
- 1.2.1 Redes de área local 9
- 1.2.2 Redes de área metropolitana 10
- 1.2.3 Redes de área amplia 11
- 1.2.4 Redes inalámbricas 13
- 1.2.5 Interredes 16

1.3 *SOFTWARE* DE RED 16
- 1.3.1 Jerarquías de protocolos 17
- 1.3.2 Consideraciones de diseño para las capas 21
- 1.3.3 Interfaces y servicios 22
- 1.3.4 Servicios orientados a conexión y sin conexión 23
- 1.3.5 Primitivas de servicios 25
- 1.3.6 La relación entre servicios y protocolos 27

1.4 MODELOS DE REFERENCIA 28
- 1.4.1 El modelo de referencia OSI 28
- 1.4.2 El modelo de referencia TCP/IP 35
- 1.4.3 Comparación de los modelos de referencia OSI y TCP 38
- 1.4.4 Una crítica del modelo y los protocolos OSI 40
- 1.4.5 Una crítica del modelo de referencia TCP/IP 43

1.5 EJEMPLOS DE REDES 44
- 1.5.1 NetWare de Novell 45
- 1.5.2 La ARPANET 47
- 1.5.3 NSFNET 50
- 1.5.4 La Internet 52
- 1.5.5 Plataformas de pruebas de gigabits 54

1.6 EJEMPLOS DE SERVICIOS DE COMUNICACIÓN DE DATOS 56
- 1.6.1 SMDS — Servicio de datos conmutado de multimegabits 57
- 1.6.2 Redes X.25 59
- 1.6.3 *Frame relay* 60
- 1.6.4 ISDN de banda ancha y ATM 61
- 1.6.5 Comparación de los servicios 66

1.7 ESTANDARIZACIÓN DE REDES 66
- 1.7.1 Quién es quién en el mundo de las telecomunicaciones 67
- 1.7.2 Quién es quién en el mundo de los estándares internacionales 69
- 1.7.3 Quién es quién en el mundo de los estándares de Internet 70

1.8 ESQUEMA DEL RESTO DEL LIBRO 72

1.9 RESUMEN 73

2 LA CAPA FÍSICA 77

2.1 BASES TEÓRICAS DE LA COMUNICACIÓN DE DATOS 77
- 2.1.1 Análisis de Fourier 78
- 2.1.2 Señales limitadas por el ancho de banda 78
- 2.1.3 La tasa de envío máxima de un canal 81

2.2 MEDIOS DE TRANSMISIÓN 82
- 2.2.1 Medios magnéticos 82
- 2.2.2 Par trenzado 83
- 2.2.3 Cable coaxial de banda base 84
- 2.2.4 Cable coaxial de banda ancha 85
- 2.2.5 Fibra óptica 87

2.3 TRANSMISIÓN INALÁMBRICA 94
- 2.3.1 El espectro electromagnético 94
- 2.3.2 Radiotransmisión 97
- 2.3.3 Transmisión por microondas 98
- 2.3.4 Ondas infrarrojas y milimétricas 100
- 2.3.5 Transmisión por ondas de luz 100

2.4 EL SISTEMA TELEFÓNICO 102
- 2.4.1 Estructura del sistema telefónico 103
- 2.4.2 La política de los teléfonos 106
- 2.4.3 El lazo local 108
- 2.4.4 Troncales y multiplexión 118
- 2.4.5 Conmutación 130

2.5 ISDN DE BANDA ESTRECHA 139
2.5.1 Servicios ISDN 140
2.5.2 Arquitectura del sistema ISDN 140
2.5.3 La interfaz ISDN 142
2.5.4 Perspectiva de N-ISDN 143

2.6 ISDN DE BANDA ANCHA Y ATM 144
2.6.1 Circuitos virtuales y conmutación de circuitos 145
2.6.2 Transmisión en redes ATM 146
2.6.3 Conmutadores ATM 147

2.7 RADIO CELULAR 155
2.7.1 Sistemas de aviso 155
2.7.2 Teléfonos inalámbricos 157
2.7.3 Teléfonos celulares analógicos 157
2.7.4 Teléfonos celulares digitales 162
2.7.5 Servicios de comunicaciones personales 162

2.8 SATÉLITES DE COMUNICACIONES 163
2.8.1 Satélites geosincrónicos 164
2.8.2 Satélites de órbita baja 167
2.8.3 Satélites y fibra 168

2.9 RESUMEN 170

3 LA CAPA DE ENLACE DE DATOS 175

3.1 CUESTIONES DE DISEÑO DE LA CAPA DE ENLACE DE DATOS 176
3.1.1 Servicios proporcionados a la capa de red 176
3.1.2 Enmarcado 179
3.1.3 Control de errores 182
3.1.4 Control de flujo 183

3.2 DETECCIÓN Y CORRECCIÓN DE ERRORES 183
3.2.1 Códigos de corrección de errores 184
3.2.2 Códigos de detección de errores 186

3.3 PROTOCOLOS ELEMENTALES DE ENLACE DE DATOS 190
3.3.1 Un protocolo símplex sin restricciones 195
3.3.2 Protocolo símplex de parada y espera 195
3.3.3 Protocolo símplex para un canal ruidoso 197

3.4 PROTOCOLO DE VENTANA CORREDIZA 202
3.4.1 Un protocolo de ventana corrediza de un bit 206
3.4.2 Un protocolo que usa regresar n 207
3.4.3 Protocolo usando repetición selectiva 213

3.5 ESPECIFICACIÓN Y VERIFICACIÓN DE LOS PROTOCOLOS 219
3.5.1 Modelos de máquina de estado finito 219
3.5.2 Modelos de red de Petri 223

3.6 EJEMPLOS DE PROTOCOLOS DE ENLACE DE DATOS 225
3.6.1 HDLC — Control de enlace de datos de alto nivel 225
3.6.2 La capa de enlace de datos en Internet 229
3.6.3 La capa de enlace de datos en ATM 235

3.7 RESUMEN 239

4 LA SUBCAPA DE ACCESO AL MEDIO 243

4.1 EL PROBLEMA DE REPARTO DEL CANAL 244
4.1.1 Reparto estático de canal en las LAN y las MAN 244
4.1.2 Reparto dinámico de canales en las LAN y MAN 245

4.2 PROTOCOLOS DE ACCESO MÚLTIPLE 246
4.2.1 ALOHA 246
4.2.2 Protocolos de acceso múltiple con detección de portadora 250
4.2.3 Protocolos libres de colisiones 254
4.2.4 Protocolos de contención limitada 256
4.2.5 Protocolos de acceso múltiple de división en longitud de onda 260
4.2.6 Protocolos de LAN inalámbricas 262
4.2.7 Radio celular digital 266

4.3 ESTÁNDAR IEEE 802 PARA LAN Y MAN 275
4.3.1 Estándar IEEE 802.3 y Ethernet 276
4.3.2 Estándar IEEE 802.4: *token bus* 287
4.3.3 Estándar IEEE 802.5: *token ring* 292
4.3.4 Comparación del 802.3, el 802.4 y el 802.5 299
4.3.5 Estándar IEEE 802.6: *bus* doble de colas distribuidas 301
4.3.6 Estándar IEEE 802.2: control lógico de enlace 302

4.4 PUENTES 304
4.4.1 Puentes de 802.x a 802.y 307
4.4.2 Puentes transparentes 310
4.4.3 Puentes de enrutamiento desde el origen 314
4.4.4 Comparación de los puentes 802 316
4.4.5 Puentes remotos 317

4.5 LAN DE ALTA VELOCIDAD 318
4.5.1 FDDI 319
4.5.2 Ethernet rápido 322
4.5.3 HIPPI — Interfaz paralela de alto desempeño 325
4.5.4 Canal de fibra 326

4.6 REDES SATELITALES 327
4.6.1 Sondeo 328
4.6.2 ALOHA 329
4.6.3 FDM 330
4.6.4 TDM 330
4.6.5 CDMA 333

4.7 RESUMEN 333

5 LA CAPA DE RED **339**

5.1 TEMAS DE DISEÑO DE LA CAPA DE RED 339
5.1.1 Servicios proporcionados a la capa de transporte 340
5.1.2 Organización interna de la capa de red 342
5.1.3 Comparación de las subredes de circuitos virtuales y de datagramas 344

5.2 ALGORITMOS DE ENRUTAMIENTO 345
5.2.1 Principio de optimación 347
5.2.2 Enrutamiento por la trayectoria más corta 348
5.2.3 Inundación 351
5.2.4 Enrutamiento basado en flujo 353
5.2.5 Enrutamiento por vector de distancia 355
5.2.6 Enrutamiento por estado de enlace 359
5.2.7 Enrutamiento jerárquico 365
5.2.8 Enrutamiento para *hosts* móviles 367
5.2.9 Enrutamiento por difusión 370
5.2.10 Enrutamiento por multitransmisión 372

5.3 ALGORITMOS DE CONTROL DE CONGESTIONAMIENTOS 374
5.3.1 Principios generales del control de congestionamiento 376
5.3.2 Políticas de prevención de congestionamientos 378
5.3.3 Conformación de tráfico 379
5.3.4 Especificaciones de flujo 384
5.3.5 Control de congestionamientos en las subredes de circuitos virtuales 386
5.3.6 Paquetes de estrangulamiento 387
5.3.7 Desprendimiento de carga 390
5.3.8 Control de fluctuación 392
5.3.9 Control de congestionamiento para multitransmisión 393

5.4 INTERREDES 396
5.4.1 Diferencias entre las redes 399
5.4.2 Circuitos virtuales concatenados 401
5.4.3 Interredes sin conexiones 402
5.4.4 Proceso de túnel 404
5.4.5 Enrutamiento por interred 405
5.4.6 Fragmentación 406
5.4.7 Muros de seguridad 410

5.5 LA CAPA DE RED EN INTERNET 412
5.5.1 El protocolo IP 413
5.5.2 Direcciones IP 416
5.5.3 Subredes 418
5.5.4 Protocolos de control de Internet 419
5.5.5 Protocolo de enrutamiento de pasarela interior: OSPF 424
5.5.6 Protocolo de enrutamiento de pasarela exterior: BGP 429
5.5.7 Multidifusión Internet 431
5.5.8 IP móvil 432
5.5.9 CIDR — Enrutamiento interdominios sin clases 434
5.5.10 IPv6 437

5.6 LA CAPA DE RED EN LAS REDES ATM 449
5.6.1 Formatos de célula 450
5.6.2 Establecimiento de la conexión 452
5.6.3 Enrutamiento y conmutación 455
5.6.4 Categorías de servicios 458
5.6.5 Calidad del servicio 460
5.6.6 Conformación y vigilancia de tráfico 463
5.6.7 Control de congestionamientos 467
5.6.8 LAN ATM 471

5.7 RESUMEN 473

6 LA CAPA DE TRANSPORTE 479

6.1 EL SERVICIO DE TRANSPORTE 479
- 6.1.1 Servicios proporcionados a las capas superiores 479
- 6.1.2 Calidad del servicio 481
- 6.1.3 Primitivas de servicio de transporte 483

6.2 ELEMENTOS DE LOS PROTOCOLOS DE TRANSPORTE 488
- 6.2.1 Direccionamiento 489
- 6.2.2 Establecimiento de una conexión 493
- 6.2.3 Liberación de una conexión 498
- 6.2.4 Control de flujo y *buffers* 502
- 6.2.5 Multiplexión 506
- 6.2.6 Recuperación de caídas 508

6.3 UN PROTOCOLO DE TRANSPORTE SENCILLO 510
- 6.3.1 Las primitivas de servicio de ejemplo 510
- 6.3.2 La entidad de transporte de ejemplo 512
- 6.3.3 El ejemplo como máquina de estados finitos 519

6.4 LOS PROTOCOLOS DE TRANSPORTE DE INTERNET (TCP Y UDP) 521
- 6.4.1 Modelo de servicio TCP 523
- 6.4.2 El protocolo TCP 524
- 6.4.3 La cabecera de segmento TCP 526
- 6.4.4 Gestión de una conexión TCP 529
- 6.4.5 Política de transmisión del TCP 533
- 6.4.6 Control de congestionamientos con TCP 536
- 6.4.7 Gestión de temporizadores del TCP 539
- 6.4.8 UDP 542
- 6.4.9 TCP y UDP inalámbricos 543

6.5 LOS PROTOCOLOS DE CAPA ATM AAL 545
- 6.5.1 Estructura de la capa de adaptación de ATM 546
- 6.5.2 AAL 1 547
- 6.5.3 AAL 2 549
- 6.5.4 AAL 3/4 550
- 6.5.5 AAL 5 552
- 6.5.6 Comparación de los protocolos AAL 554
- 6.5.7 SSCOP — Protocolo de servicio específico orientado a conexiones 555

6.6 DESEMPEÑO 555
- 6.6.1 Problemas de desempeño en las redes de cómputo 556
- 6.6.2 Medición del desempeño de las redes 559

- 6.6.3 Diseño de sistemas para un mejor desempeño 561
- 6.6.4 Procesamiento rápido de las TPDU 565
- 6.6.5 Protocolos de redes de gigabits 568

6.7 RESUMEN 572

7 LA CAPA DE APLICACIÓN 577

7.1 SEGURIDAD DE LA RED 577
- 7.1.1 Cifrado tradicional 580
- 7.1.2 Dos principios criptográficos fundamentales 585
- 7.1.3 Algoritmos de clave secreta 587
- 7.1.4 Algoritmos de clave pública 597
- 7.1.5 Protocolos de validación de identificación 601
- 7.1.6 Firmas digitales 613
- 7.1.7 Aspectos sociales 620

7.2 DNS — SISTEMA DE NOMBRES DE DOMINIO 622
- 7.2.1 El espacio de nombres del DNS 622
- 7.2.2 Registros de recursos 624
- 7.2.3 Servidores de nombres 628

7.3 SNMP — PROTOCOLO SENCILLO DE ADMINISTRACIÓN DE REDES 630
- 7.3.1 El modelo SNMP 631
- 7.3.2 ASN.1 — Notación de sintaxis abstracta 1 633
- 7.3.3 SMI — Estructura de la información de administración 639
- 7.3.4 La MIB — Base de información de administración 641
- 7.3.5 El protocolo SNMP 642

7.4 CORREO ELECTRÓNICO 643
- 7.4.1 Arquitectura y servicios 645
- 7.4.2 El agente de usuario 646
- 7.4.3 Formatos de mensaje 650
- 7.4.4 Transferencia de mensajes 657
- 7.4.5 Confidencialidad en el correo electrónico 663

7.5 NOTICIAS USENET 669
- 7.5.1 USENET desde el punto de vista del usuario 670
- 7.5.2 Implementación de USENET 675

7.6 LA *WORLD WIDE WEB* 681
7.6.1 El lado del cliente 682
7.6.2 El lado del servidor 685
7.6.3 Escritura de una página de *Web* en HTML 691
7.6.4 Java 706
7.6.5 Localización de información en la *Web* 720

7.7 MULTIMEDIA 723
7.7.1 Audio 724
7.7.2 Vídeo 727
7.7.3 Compresión de datos 730
7.7.4 Vídeo a solicitud 744
7.7.5 MBone — *Backbone* de multidifusión 756

7.8 RESUMEN 760

8 LISTA DE LECTURAS Y BIBLIOGRAFÍA 767

8.1 SUGERENCIAS PARA LECTURA ADICIONAL 767
8.1.1 Introducción y obras generales 768
8.1.2 La capa física 769
8.1.3 La capa de enlace de datos 770
8.1.4 La subcapa de acceso al medio 770
8.1.5 La capa de red 771
8.1.6 La capa de transporte 772
8.1.7 La capa de aplicación 772

8.2 BIBLIOGRAFÍA ALFABÉTICA 775

ÍNDICE 795

UNIVERSIDAD D
FACULTAD I
DEPARTA
DOCUMENTACIC
MONTEVIDE

PREFACIO

Este libro ahora está en su tercera edición. Cada edición ha correspondido a una fase diferente de la manera de uso de las redes de computadoras. Cuando apareció la primera edición en 1980, las redes eran una curiosidad académica. Al aparecer la segunda edición en 1988, las redes se usaban en las universidades y grandes empresas. Y cuando apareció la tercera edición en 1996, las redes de computadoras, en especial la Internet mundial, se han vuelto una realidad diaria para millones de personas.

Además, el *hardware* y *software* de redes han cambiado por completo desde la aparición de la segunda edición. En 1988, casi todas las redes se basaban en alambre de cobre. Ahora, muchas se basan en fibra óptica o comunicación inalámbrica. Las redes patentadas, como la SNA, se han vuelto mucho menos importantes que las redes públicas, especialmente la Internet. Los protocolos OSI han desaparecido calladamente, y el grupo de protocolos TCP/IP se ha hecho dominante. En realidad, todo ha cambiado tanto que el libro ha debido reescribirse prácticamente desde cero.

Aunque el capítulo 1 tiene la misma función introductoria que en la segunda edición, el contenido se ha revisado y actualizado por completo. Por ejemplo, en lugar de basarse en el modelo OSI de siete capas, el libro usa ahora un modelo híbrido de cinco capas (mostrado en la figura 1-21) que se presenta en el capítulo 1. Aunque no es idéntico al modelo TCP/IP, dicho modelo es mucho más cercano espiritualmente a TCP/IP que al modelo OSI usado en la segunda edición. También se introducen aquí los nuevos ejemplos operativos usados en el libro (Internet y las redes ATM), al igual que algunas redes de gigabits y otras redes populares.

05383.

En el capítulo 2, el enfoque ha pasado del alambre de cobre a la fibra óptica y la comunicación inalámbrica, puesto que son las tecnologías del futuro. El sistema telefónico se ha convertido casi por completo en digital durante la última década, así que el material correspondiente se ha reescrito en buena medida, con la adición de nuevo material sobre la ISDN de banda ancha. El material sobre la radio celular ha aumentado considerablemente, y se ha agregado al capítulo material nuevo sobre satélites de órbita baja.

Se ha invertido el orden de estudio de la capa de enlace de datos y la subcapa MAC, puesto que la experiencia con los estudiantes demuestra que entienden mejor la subcapa MAC después de haber estudiado la capa de enlace de datos. Se han conservado los protocolos de ejemplo, pero reescritos en C, puesto que han demostrado ser muy populares. Se ha agregado material nuevo sobre las capas de enlace de datos de Internet y ATM.

Los principios de la subcapa MAC del capítulo 4 se han modificado para reflejar los protocolos nuevos, incluida la multiplexión por división en longitud de onda, las LAN inalámbricas y la radio digital. Se ha revisado también el estudio de los puentes, y se ha agregado material nuevo sobre las LAN de alta velocidad.

La mayoría de los algoritmos de enrutamiento del capítulo 5 se ha reemplazado por algoritmos más modernos, incluidos los enrutamientos por vector de distancia y por estado de enlace. Las secciones sobre control de congestionamiento se han rehecho por completo, y el material que trata los ejemplos operativos, la Internet y el ATM es totalmente nuevo.

El capítulo 6 trata aún la capa de transporte, pero también aquí ha habido cambios sustanciales, principalmente la adición de una gran cantidad de material nuevo sobre Internet, ATM y desempeño de redes.

El capítulo 7, que cubre la capa de aplicación, ahora es el más largo del libro. Se ha duplicado la longitud del material sobre seguridad en las redes, y se ha agregado material nuevo sobre DNS, SNMP, correo electrónico, USENET, la *World Wide Web*, HTML, Java, multimedia, vídeo a solicitud y el MBone.

De las 395 figuras de la tercera edición, 276 (70%) son completamente nuevas y se han actualizado algunas de las otras. De las 370 referencias a la literatura, 281 (76%) son libros y artículos que han aparecido desde la publicación de la segunda edición. De estas referencias, más de 100 son a trabajos publicados durante 1995 y 1996. En síntesis, probablemente el 75% del libro es completamente nuevo, y se han actualizado exhaustivamente partes del 25% restante. Dado que es éste de hecho un libro nuevo, se rediseñó la portada para evitar confusiones con la segunda edición.

Los libros de computación están llenos de siglas, y éste no es la excepción. Al terminar la lectura de este libro, las siguientes siglas deberán serle familiares: AAL, AMPS, ARP, ASN, ATM, BGP, CDMA, CDPD, CSMA, DQDB, DNS, FAQ, FDM, FTP, FTTC, FTTH, GSM, HDLC, HEC, HIPPI, IAB, ICMP, IDEA, IETF, IPv6, ISO, ITU, LATA, MAC, MACA, MAN, MIB, MIME, NAP, NNTP, NSA, NSAP, OSI, OSPF, PCM, PCN, PCS, PEM, PGP, PPP, PSTN, PTT, PVC, QAM, RARP, RFC, RSA, SABME, SAP, SAR, SDH, SDLC, SHA, SMI, SNA, SNMP, SNRME, SPX, TCP, UDP, VHF, VLF, VSAT, WARC, WDM, WWV y WWW. Pero no se preocupe; cada una se definirá cuidadosamente antes de usarla.

Para ayudar a los profesores que usen esta obra como libro de texto de su cátedra, el autor ha preparado tres herramientas de enseñanza:

- Un manual de solución de problemas.
- Archivos PostScript que contienen todas las figuras (para hacer hojas de proyección).
- Un simulador (escrito en C) de los protocolos de ejemplo del capítulo 3.

El manual de soluciones puede pedirse a Prentice Hall (pero sólo profesores). El archivo con las figuras y el simulador están disponibles a través de la *World Wide Web*. Para obtenerlos, por favor vea la página base del autor: *http://www.cs.vu.nl/~ast/*.

La composición del libro se hizo con tipografía Times, usando Troff, que, a pesar del paso de los años, aún es el único camino a seguir. Aunque Troff no está de moda como los sistemas WYSIWYG, se invita al lector a que compare la calidad de la composición de este libro con la de los libros producidos mediante sistemas WYSIWYG. Mi única concesión a la publicación en PC y de escritorio es que, por primera vez, las imágenes se produjeron con Adobe Illustrator, en lugar de dibujarse en papel. También por primera vez, el libro se produjo por completo de manera electrónica. La salida PostScript de Troff se envió por la Internet al impresor, donde se produjo la película para hacer las placas de offset. No se imprimió ni fotografió una copia intermedia en papel, como es normal.

Mucha gente me ayudó durante esta tercera edición. Quisiera agradecer especialmente a Chase Bailey, Saniya Ben Hassen, Nathaniel Borenstein, Ron Cocchi, Dave Crocker, Wiebren de Jonge, Carl Ellison, M. Rasit Eskicioglu, John Evans, Mario Gerla, Mike Goguen, Paul Green, Dick Grune, Wayne Hathaway, Franz Hauck, Jack Holtzman, Gerard Holzmann, Philip Homburg, Peter Honeyman, Raj Jain, Dave Johnson, Charlie Kaufman, Vinay Kumar, Jorg Liebeherr, Paul Mockapetris, Carol Orange, Craig Partridge, Charlie Perkins, Thomas Powell, Greg Sharp, Anne Steegstra, George Swallow, Mark Taylor, Peter van der Linden, Hans van Staveren, Maarten van Steen, Kees Verstoep, Stephen Walters, Michael Weintraub, Joseph Wilkes y Stephen Wolff. Doy gracias especiales a Radia Perlman por sus muchas sugerencias útiles. Mis estudiantes también me han ayudado de muchas maneras. En particular, quisiera mencionar a Martijn Bot, Wilbert de Graaf, Flavio del Pomo y Arnold de Wit por su colaboración.

Mi editor de Prentice Hall, Mary Franz, me proporcionó más material de lectura del que había consumido en los 10 años previos. También fue de gran ayuda en otras tareas, pequeñas, medianas, grandes y jumbo. Mi editor de producción, Camille Trentacoste, me enseñó qué es la gente de nieve, los apartamentos para 8 o más, fax (sic) y otros temas importantes, mientras llevaba a cabo el servicio de asistente para un autor quisquilloso con un itinerario complicado.

Por último llegamos a la gente más importante. Suzanne, Barbara, Marvin, e incluso el pequeño Bram, han pasado por esta rutina antes; la soportan con paciencia infinita y de buen talante. Gracias.

ANDREW S. TANENBAUM

1

INTRODUCCIÓN

Los tres últimos siglos han estado dominados, cada uno de ellos, por una tecnología. El siglo XVIII fue la época de los grandes sistemas mecánicos que acompañaron a la Revolución Industrial. El siglo XIX fue la era de las máquinas de vapor. En el siglo XX, la tecnología clave ha sido la obtención, procesamiento y distribución de la información. Entre otros avances, hemos visto la instalación de redes telefónicas mundiales, la invención del radio y la televisión, el nacimiento y crecimiento sin precedentes de la industria de las computadoras y el lanzamiento de satélites de comunicación.

Debido al rápido progreso de la tecnología, estas áreas están convergiendo rápidamente, y las diferencias entre juntar, transportar, almacenar y procesar información desaparecen con rapidez. Las organizaciones con cientos de oficinas que se extienden sobre una amplia área geográfica esperan ser capaces de examinar la situación, aun de sus más remotos puestos de avanzada, oprimiendo un botón. Al crecer nuestra habilidad para obtener, procesar y distribuir información, también crece la demanda de técnicas de procesamiento de información más avanzadas.

Aunque la industria de la computación es joven comparada con otras industrias (por ejemplo, las de automóviles y transporte aéreo), las computadoras han logrado un progreso espectacular en un tiempo corto. Durante las dos primeras décadas de su existencia, los sistemas de cómputo eran altamente centralizados, por lo general, dentro de un cuarto grande. En muchos casos, este cuarto tenía paredes de vidrio a través de las cuales los visitantes podían asombrarse de la gran maravilla electrónica que se encontraba dentro. Una compañía de tamaño mediano o

una universidad tenía una o dos computadoras, mientras que una institución grande tenía cuando mucho unas cuantas docenas. La idea de que dentro de 20 años se pudieran producir en masa, por millones, computadoras de igual capacidad más pequeñas que las estampillas de correo, era pura ciencia ficción.

La fusión de las computadoras y las comunicaciones ha tenido una profunda influencia en la forma en que los sistemas de cómputo se organizan. El concepto de "centro de cómputo" como cuarto con una gran computadora a la cual los usuarios traían sus trabajos para procesar es ahora totalmente obsoleto. El viejo modelo de una sola computadora que atendía todas las necesidades de computación de la organización ha sido reemplazado por uno en el cual un gran número de computadoras separadas pero interconectadas hacen el trabajo. Estos sistemas se llaman **redes de computadoras**. El diseño y organización de estas redes es el tema de este libro.

A lo largo del libro usaremos el término "red de computadoras" para referirnos a una colección *interconectada* de computadoras *autónomas*. Se dice que dos computadoras están interconectadas si son capaces de intercambiar información. La conexión no tiene que ser por medio de un alambre de cobre; puede usarse fibra óptica, microondas y satélites de comunicación. Al indicar que las computadoras son autónomas, queremos excluir de nuestra definición a los sistemas en los que existe una clara relación amo-esclavo. Si una computadora puede arrancar, parar o controlar otra a voluntad, las computadoras no son autónomas. Un sistema con una unidad de control y muchos esclavos no es una red; tampoco lo es una computadora grande con impresoras y terminales remotas.

Existe en la bibliografía sobre el tema considerable confusión entre la red de computadoras y un **sistema distribuido**. La diferencia radica en que en el sistema distribuido la existencia de múltiples computadoras autónomas es transparente para el usuario (es decir, no es visible). El usuario puede teclear una orden para ejecutar un programa y éste se ejecutará. La tarea de seleccionar el mejor procesador, encontrar y transportar todos los archivos de entrada al procesador y poner los resultados en el lugar apropiado, corresponde al sistema operativo.

En otras palabras, el usuario de un sistema distribuido no está consciente de que haya múltiples procesadores; más bien, ve al sistema como un monoprocesador virtual. La asignación de trabajos a procesadores y de archivos a discos, el movimiento de archivos entre el lugar donde se almacenan y donde son necesarios, y todas las demás funciones del sistema, deben ser automáticas.

En una red, el usuario debe ingresar de forma *explícita* en una máquina, enviar los trabajos remotos *explícitamente*, mover *explícitamente* los archivos y, en general, llevar a cabo de manera personal el manejo de la red. En un sistema distribuido nada se tiene que hacer de forma explícita; el sistema lo hace todo automáticamente sin que el usuario tenga conocimiento de ello.

En efecto, un sistema distribuido es un sistema de *software* construido encima de una red, a la que el *software* confiere un alto grado de cohesión y transparencia. Así, la distinción entre una red y un sistema distribuido tiene que ver con el *software* más que con el *hardware* (especialmente el sistema operativo).

No obstante, los dos temas se superponen de manera considerable. Por ejemplo, tanto un sistema distribuido como una red de computadoras necesitan transferir archivos. La diferencia está en quién invoca la transferencia, el sistema o el usuario. Aunque el enfoque fundamental de este libro es hacia las redes, muchos de los temas también son importantes en sistemas distribuidos. Si se desea mayor información sobre sistemas distribuidos, véase (Coulouris *et al.*, 1994; Mullender, 1993; y Tanenbaum, 1995).

1.1. USOS DE LAS REDES DE COMPUTADORAS

Antes de empezar a examinar los aspectos técnicos en detalle, es importante dedicar algún tiempo a entender por qué la gente está interesada en las redes de computadoras y para qué puede usarlas.

1.1.1. Redes para compañías

Muchas organizaciones tienen una cantidad importante de computadoras en operación, con frecuencia alejadas entre sí. Por ejemplo, una compañía con muchas fábricas puede tener una computadora en cada localidad para llevar el control de los inventarios, vigilar la productividad y pagar la nómina local. Inicialmente, cada una de estas computadoras puede haber trabajado aislada de las otras, pero en algún momento la gerencia decidió conectarlas para poder extraer y correlacionar información acerca de toda la compañía.

En términos más generales, la cuestión aquí es **compartir los recursos** y la meta es hacer que todos los programas, el equipo y especialmente los datos estén disponibles para cualquiera en la red, sin importar la localización física de los recursos y de los usuarios. En otras palabras, el hecho de que un usuario esté a 1000 km de distancia de sus datos no deberá impedirle usar los datos como si fueran locales. Este objetivo puede resumirse diciendo que es un intento por acabar con la "tiranía de la geografía".

Una segunda meta es lograr una **alta confiabilidad** al contar con fuentes alternativas de suministro. Por ejemplo, todos los archivos podrían replicarse en dos o tres máquinas; así, si una de ellas no está disponible (debido a una falla del *hardware*), podrán usarse las otras copias. Además, la existencia de múltiples CPU significa que si una de ellas falla, las otras serán capaces de hacer su trabajo, aunque se reduzca el rendimiento. En aplicaciones militares, bancarias, de control de tráfico aéreo, seguridad de reactores nucleares y muchas otras, la capacidad para continuar operando pese a problemas de *hardware* es de suma importancia.

Otra meta es **ahorrar dinero**. Las computadoras pequeñas tienen una relación precio/rendimiento mucho mejor que las grandes. Las *mainframes* (computadoras del tamaño de un cuarto) son aproximadamente 10 veces más rápidas que las computadoras personales, pero cuestan mil veces más. Este desequilibrio ha ocasionado que muchos diseñadores construyan sistemas compuestos por computadoras personales, una por usuario, con los datos guardados en una o más máquinas **servidoras de archivos** compartidas. En este modelo, los usuarios se denominan **clientes**, y el arreglo completo se llama **modelo cliente-servidor**. Esto se ilustra en la figura 1-1.

En el modelo cliente-servidor, la comunicación generalmente adopta la forma de un mensaje de solicitud del cliente al servidor pidiendo que se efectúe algún trabajo. A continuación, el

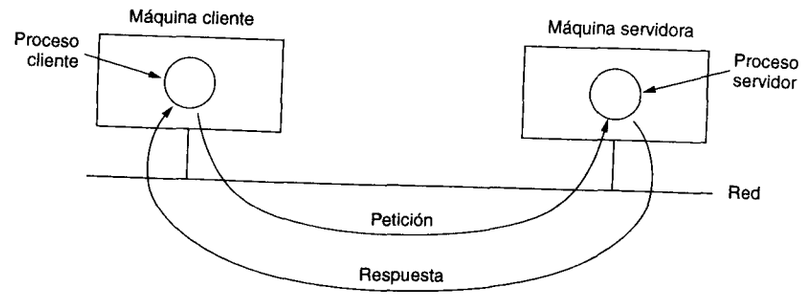


Figura 1-1. El modelo cliente-servidor.

servidor hace el trabajo y devuelve la respuesta. Por lo regular, muchos clientes utilizan un número pequeño de servidores.

Otra meta al establecer redes es la escalabilidad: la capacidad para incrementar el rendimiento del sistema gradualmente cuando la carga de trabajo crece, añadiendo solamente más procesadores. En el caso de *mainframes* centralizadas, cuando el sistema esté lleno hay que reemplazarlo por uno mayor, usualmente más caro, lo que implica largas interrupciones para los usuarios. Con el modelo cliente-servidor se pueden añadir nuevos clientes y nuevos servidores cuando es necesario.

Un objetivo más del establecimiento de una red de computadoras tiene poco que ver con la tecnología. Una red de computadoras puede proporcionar un potente **medio de comunicación** entre empleados que están muy distantes. Al usar una red, es fácil para dos o más personas que viven lejos escribir un informe juntas. Cuando un trabajador hace un cambio a un documento en línea, los demás pueden ver el cambio inmediatamente, sin tener que esperar varios días la llegada de una carta. Tal rapidez hace fácil la cooperación entre grupos de gente muy apartada, cosa que previamente era imposible. A largo plazo, el uso de redes para mejorar la comunicación entre las personas probablemente resultará más importante que las metas técnicas tales como la mejora de la confiabilidad.

1.1.2. Redes para la gente

Todas las motivaciones arriba citadas para construir redes de computadoras son de naturaleza esencialmente económica y tecnológica. Si *mainframes* suficientemente grandes y potentes estuvieran disponibles a precios aceptables, muchas compañías habrían optado por guardar todos sus datos en ellas y proporcionar a sus empleados terminales conectadas a estas máquinas. En la década de 1970 y a principios de la de 1980, casi todas las compañías operaban de esta forma. Las redes de computadoras llegaron a ser populares únicamente cuando las computadoras personales ofrecieron una descomunal ventaja precio/rendimiento sobre las *mainframes*.

Al iniciar la década de 1990, las redes de computadoras comenzaron a prestar servicios a particulares en su hogar. Estos servicios y la motivación para usarlos son muy diferentes del

modelo de "eficiencia corporativa" descrito en la sección anterior. A continuación esbozaremos tres de los más estimulantes aspectos de esta evolución:

1. Acceso a información remota.
2. Comunicación de persona a persona.
3. Entretenimiento interactivo.

El acceso a la información remota vendrá en muchas formas. Un área en la cual ya está sucediendo es el acceso a las instituciones financieras. Mucha gente paga sus facturas, administra sus cuentas bancarias y maneja sus inversiones en forma electrónica. Las compras desde el hogar se están haciendo populares, con la facilidad de inspeccionar los catálogos en línea de miles de compañías. Algunos de estos catálogos pronto ofrecerán un vídeo instantáneo de cualquier producto que se pueda ver con sólo hacer clic en el nombre del producto.

Los periódicos se publicarán en línea y serán personalizados. Podremos decirle al periódico que queremos saber todo lo que haya acerca de los políticos corruptos, los grandes incendios, los escándalos de celebridades y las epidemias, pero nada de fútbol, gracias. En la noche mientras usted duerme, el periódico se bajará al disco de su computadora o se imprimirá en su impresora láser. A pequeña escala, este servicio ya existe. El siguiente paso más allá de los periódicos (y de las revistas y publicaciones científicas) es la biblioteca digital en línea. Dependiendo del costo, tamaño y peso de las computadoras portátiles, los libros impresos quizá lleguen a ser obsoletos. Quienes lo duden deberían tomar nota de las consecuencias de la imprenta sobre los manuscritos medievales iluminados.

Otra aplicación en esta categoría es el acceso a sistemas de información como la actual red mundial (*World Wide Web*), la cual contiene información sobre arte, negocios, cocina, gobierno, salud, historia, aficiones, recreación, ciencia, deportes, viajes y muchos otros temas, demasiado numerosos para mencionarlos aquí.

Todas las aplicaciones antes mencionadas implican la interacción entre una persona y una base de datos remota. La segunda categoría extensa de redes que se usará implica la interacción persona a persona, básicamente la respuesta del siglo XXI al teléfono del siglo XIX. Millones de personas utilizan ya el correo electrónico o **email** y pronto contendrá en forma rutinaria audio y vídeo además de texto. El perfume en los mensajes tardará un poco más en perfeccionarse.

El correo electrónico de tiempo real permitirá a los usuarios remotos comunicarse sin retraso, posiblemente viéndose y escuchándose. Esta tecnología hace posible realizar reuniones virtuales, llamadas **videoconferencias**, entre gente muy alejada. A veces se dice que el transporte y la comunicación están en competencia, y cualquiera que gane hará al otro obsoleto. Las reuniones virtuales podrán servir para recibir enseñanza remota, obtener opiniones médicas de especialistas distantes y otras muchas aplicaciones.

Los grupos de noticias a nivel mundial, con discusiones sobre todos los temas concebibles, son ya comunes entre un grupo selecto de personas, y esto crecerá para incluir a la población en general. Estas discusiones en las cuales una persona pone un mensaje y los demás suscriptores al grupo de noticias pueden leerlo, van desde lo humorístico hasta lo apasionado.

La tercera categoría es el entretenimiento, que es una industria enorme y en crecimiento. La aplicación irresistible aquí (y que puede impulsar a todas las demás) es el vídeo por solicitud. Dentro de aproximadamente una década, será posible seleccionar cualquier película o programa de televisión creado en cualquier país y exhibirlo en la pantalla de forma instantánea. Algunas películas nuevas llegarán a ser interactivas, preguntándose al usuario ocasionalmente qué dirección debe seguir la historia (¿deberá MacBeth asesinar a Duncan o esperar su momento?) con argumentos alternativos para todos los casos. La televisión en vivo también puede llegar a ser interactiva, con el auditorio participando en concursos, escogiendo entre los concursantes, etcétera.

Por otro lado, tal vez la aplicación irresistible no sea la petición de vídeos, sino los juegos. Tenemos ya juegos de simulación en tiempo real multipersonales, como las aventuras en calabozos virtuales, y simuladores de vuelo en los que los jugadores de un equipo tratan de derribar a los del equipo contrario. Si esto se hace con anteojos que muestren imágenes en movimiento con calidad fotográfica en tiempo real tridimensional, tendremos una especie de realidad virtual compartida mundial.

En pocas palabras, la capacidad para combinar información, comunicación y entretenimiento seguramente hará surgir una nueva y enorme industria basada en las redes de computadoras.

1.1.3. Consideraciones sociales

La introducción ampliamente difundida de redes significará nuevos problemas sociales, éticos y políticos (Laudon, 1995). Sólo mencionaremos en forma breve algunos de ellos; un estudio minucioso requiere un libro completo, por lo menos. Una característica popular de muchas redes son los grupos de noticias o quioscos de anuncios en los que la gente puede intercambiar mensajes con individuos de gustos parecidos. Mientras los temas estén restringidos a asuntos técnicos o aficiones como la jardinería, no se presentarán muchos problemas.

El problema surge cuando los grupos de noticias tratan temas que a la gente en verdad le importan, como la política, la religión o el sexo. Las opiniones expresadas en tales grupos pueden ser profundamente ofensivas para algunas personas. Además, los mensajes no necesariamente están limitados al texto. Fotografías a color de alta definición e incluso pequeños *videoclips* pueden transmitirse ahora con facilidad por las redes de computadoras. Algunas personas adoptan una postura de vive y deja vivir pero otras sienten que enviar cierto material (por ejemplo, pornografía infantil) es simplemente inaceptable. Así pues, el debate sigue causando furor.

Hay gente que ha demandado a los operadores de redes, reclamando que son responsables por el contenido de lo que aquéllas acarrean, como los periódicos y revistas. La respuesta inevitable es que una red es como una compañía de teléfonos o como la oficina de correos y no puede esperarse que los operadores vigilen lo que los usuarios dicen. Por otro lado, si se obligara a los operadores de redes a censurar los mensajes, probablemente optarían por eliminar cualquier cosa que tuviera la más leve posibilidad de causar una demanda en su contra y por tanto violarían el derecho de los usuarios a hablar con libertad. Lo más seguro es que este debate continuará durante un tiempo.

Otra área divertida es el conflicto entre los derechos de los empleados y los derechos de los patrones. Muchas personas leen y escriben correo electrónico en su trabajo. Algunos patrones han reclamado el derecho a leer y posiblemente censurar los mensajes de los empleados, incluidos los mensajes enviados desde una terminal casera después de horas de trabajo. No todos los empleados están de acuerdo con esto (Sipior y Ward, 1995).

Aun si los patrones tienen poder sobre los empleados, ¿esta relación también gobierna a universidades y estudiantes? ¿Y qué hay acerca de las preparatorias y sus estudiantes? En 1994, la Universidad Carnegie-Mellon decidió bloquear la entrada de mensajes de algunos grupos de noticias que trataban el sexo porque la universidad sintió que el material era inapropiado para menores (es decir, los pocos estudiantes que tenían menos de 18 años). Las repercusiones de este suceso tardarán años en disiparse.

Las redes de computadoras ofrecen la posibilidad de enviar mensajes anónimos. En algunas situaciones, esta capacidad puede ser deseable. Por ejemplo, proporciona un mecanismo para que estudiantes, militares, empleados y ciudadanos llamen la atención sobre comportamientos ilegales por parte de profesores, oficiales, superiores y políticos sin miedo a represalias. Por otro lado, en Estados Unidos y muchas otras democracias la ley otorga específicamente a una persona acusada el derecho de enfrentar y recusar a su acusador en los tribunales. Las acusaciones anónimas no pueden aceptarse como pruebas.

En pocas palabras, las redes de computadoras, igual que la imprenta hace 500 años, permiten a los ciudadanos comunes distribuir sus puntos de vista en diferentes formas y a diferentes públicos que antes estaban fuera de su alcance. Esta nueva libertad trae consigo muchos problemas sociales, políticos y morales aún no resueltos. La resolución de estos problemas se deja como ejercicio para el lector.

1.2. *HARDWARE* DE RED

Ahora es tiempo de dejar a un lado las aplicaciones y los aspectos sociales de las redes para enfocarnos a los problemas técnicos que implica su diseño. No existe una taxonomía generalmente aceptada dentro de la cual quepan todas las redes de computadoras, pero dos dimensiones sobresalen como importantes: la tecnología de transmisión y la escala. Examinaremos ahora cada una de ellas por turno.

En términos generales, hay dos tipos de tecnología de transmisión:

1. Redes de difusión.
2. Redes punto a punto.

Las **redes de difusión** tienen un solo canal de comunicación compartido por todas las máquinas de la red. Los mensajes cortos (llamados **paquetes** en ciertos contextos) que envía una máquina son recibidos por todas las demás. Un campo de dirección dentro del paquete especifica a quién se dirige. Al recibir un paquete, una máquina verifica el campo de dirección. Si el paquete está dirigido a ella, lo procesa; si está dirigido a alguna otra máquina, lo ignora.

Como analogía, consideremos a una persona de pie e inmóvil al final de un corredor que da acceso a muchos cuartos y que grita: "Watson, ven aquí, te necesito". Aunque en realidad mucha gente puede recibir (oír) el paquete, únicamente Watson responderá; los otros lo ignorarán. Otro ejemplo es un anuncio en el aeropuerto, pidiendo a todos los pasajeros del vuelo 644 que se presenten en la sala 12.

Los sistemas de difusión generalmente también ofrecen la posibilidad de dirigir un paquete a *todos* los destinos colocando un código especial en el campo de dirección. Cuando se transmite un paquete con este código, cada máquina en la red lo recibe y lo procesa. Este modo de operación se llama **difusión** (*broadcasting*). Algunos sistemas de difusión también contemplan la transmisión a un subconjunto de las máquinas, algo conocido como **multidifusión**. Un esquema posible consiste en reservar un bit para indicar multidifusión. Los restantes $n - 1$ bits de dirección pueden contener un número de grupo. Cada máquina se puede "suscribir" a cualquier grupo o a todos. Cuando se envía un paquete a cierto grupo, se entrega a todas las máquinas que se suscribieron a ese grupo.

En contraste, las redes **punto a punto** consisten en muchas conexiones entre pares individuales de máquinas. Para ir del origen al destino, un paquete en este tipo de red puede tener que visitar primero una o más máquinas intermedias. A veces son posibles múltiples rutas de diferentes longitudes, por lo que los algoritmos de ruteo desempeñan un papel importante en las redes punto a punto. Como regla general (aunque hay muchas excepciones), las redes pequeñas geográficamente localizadas tienden a usar la difusión, mientras que las redes más grandes suelen ser punto a punto.

Distancia entre procesadores	Procesadores ubicados en el (la) mismo(a)	Ejemplo
0.1 m	Tarjeta de circuitos	Máquina de flujo de datos
1 m	Sistema	Multicomputadora
10 m	Cuarto	Red de área local
100 m	Edificio	
1 km	Campus	
10 km	Ciudad	Red de área metropolitana
100 km	País	Red de área amplia
1,000 km	Continente	
10,000 km	Planeta	La Internet

Figura 1-2. Clasificación de procesadores interconectados según su escala.

Un criterio alterno para clasificar las redes es su escala. En la figura 1-2 damos una clasificación de los sistemas de múltiples procesadores de acuerdo con su tamaño físico. En la parte superior están las **máquinas de flujo de datos**, computadoras con alto grado de paralelismo y muchas unidades funcionales, todas trabajando en el mismo programa. A continuación vienen las **multicomputadoras**, sistemas que se comunican enviando mensajes por *buses* muy cortos y

rápidos. Más allá de las multicomputadoras están las verdaderas redes, computadoras que se comunican intercambiando mensajes por cables largos. Éstas pueden dividirse en redes locales, metropolitanas y de área amplia. Finalmente, la conexión de dos o más redes es una interred. La red Internet, de alcance mundial, es un ejemplo muy conocido de interred. La distancia es importante como medio de clasificación porque se usan diferentes técnicas a diferentes escalas. En este libro nos ocuparemos únicamente de las redes verdaderas y de su interconexión. A continuación presentamos una breve introducción al tema de *hardware* de red.

1.2.1. Redes de área local

Las **redes de área local**, generalmente llamadas **LAN** (*local area networks*), son redes de propiedad privada dentro de un solo edificio o campus de hasta unos cuantos kilómetros de extensión. Se usan ampliamente para conectar computadoras personales y estaciones de trabajo en oficinas de compañías y fábricas con objeto de compartir recursos (por ejemplo, impresoras) e intercambiar información. Las LAN se distinguen de otro tipo de redes por tres características: (1) su tamaño, (2) su tecnología de transmisión, y (3) su topología.

Las LAN están restringidas en tamaño, lo cual significa que el tiempo de transmisión del peor caso está limitado y se conoce de antemano. Conocer este límite hace posible usar ciertos tipos de diseños que de otra manera no serían prácticos, y también simplifica la administración de la red.

Las LAN a menudo usan una tecnología de transmisión que consiste en un cable sencillo al cual están conectadas todas las máquinas, como las líneas compartidas de la compañía telefónica que solían usarse en áreas rurales. Las LAN tradicionales operan a velocidades de 10 a 100 Mbps, tienen bajo retardo (décimas de microsegundos) y experimentan muy pocos errores. Las LAN más nuevas pueden operar a velocidades muy altas, de hasta cientos de megabits/seg. En este libro nos apegamos a la tradición y medimos la velocidad de las líneas en megabits/seg (Mbps), no megabytes/seg (MB/seg). Un megabit es 1,000,000 bits, no 1,048,576 (2^{20}) bits.

Las LAN de transmisión pueden tener diversas topologías; la figura 1-3 muestra dos de ellas. En una red de *bus* (esto es, un cable lineal), en cualquier instante una computadora es la

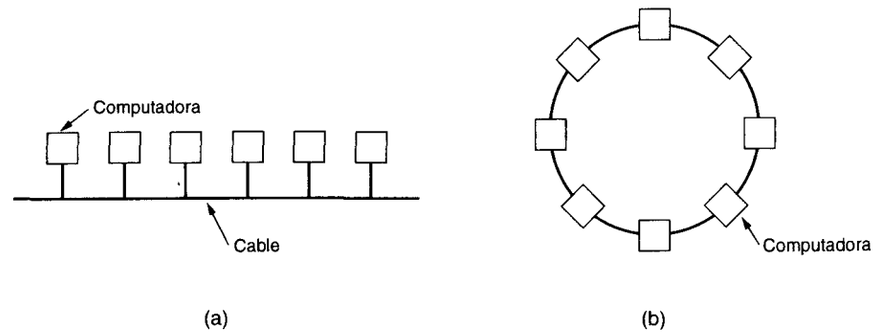


Figura 1-3. Dos redes de difusión. (a) *Bus*. (b) Anillo.

máquina maestra y puede transmitir; se pide a las otras máquinas que se abstengan de enviar mensajes. Es necesario un mecanismo de arbitraje para resolver conflictos cuando dos o más máquinas quieren transmitir simultáneamente. El mecanismo de arbitraje puede ser centralizado o distribuido. Por ejemplo, la IEEE 802.3, popularmente llamada **Ethernet**MR, es una red de transmisión basada en *bus* con control de operación descentralizado a 10 o 100 Mbps. Las computadoras de una Ethernet pueden transmitir cuando quieran; si dos o más paquetes chocan, cada computadora sólo espera un tiempo al azar y lo vuelve a intentar.

Un segundo tipo de sistema de difusión es el anillo. En un anillo, cada bit se propaga por sí mismo, sin esperar al resto del paquete al cual pertenece. Típicamente, cada bit recorre el anillo entero en el tiempo que toma transmitir unos pocos bits, a veces antes de que el paquete completo se haya transmitido. Como en todos los sistemas de difusión, se necesitan reglas para arbitrar el acceso simultáneo al anillo. Se emplean varios métodos que se analizarán más adelante en este libro. La IEEE 802.5 (el *token ring* de IBM), es una popular LAN basada en anillo que opera a 4 y 16 Mbps.

Las redes de difusión se pueden dividir también en estáticas y dinámicas, dependiendo de cómo se asigna el canal. Una asignación estática típica divide el tiempo en intervalos discretos y ejecuta un algoritmo de asignación cíclica, permitiendo a cada máquina transmitir únicamente cuando le llega su turno. La asignación estática desperdicia la capacidad del canal cuando una máquina no tiene nada que decir durante su segmento asignado, por lo que muchos sistemas intentan asignar el canal dinámicamente (es decir, por demanda).

Los métodos de asignación dinámica para un canal común son centralizados o descentralizados. En el método de asignación de canal centralizado hay una sola entidad, por ejemplo una unidad de arbitraje del *bus*, la cual determina quién es el siguiente. Podría hacer esto aceptando peticiones y tomando una decisión de acuerdo con un algoritmo interno. En el método de asignación de canal descentralizado no hay una entidad central; cada máquina debe decidir por sí misma si transmite o no. Podríamos pensar que esto siempre conduce al caos, pero no es así. Más adelante estudiaremos muchos algoritmos diseñados para poner orden en el caos potencial.

El otro tipo de LAN se construye con líneas punto a punto. Las líneas individuales conectan una máquina específica a otra. Una LAN así es realmente una red de área amplia en miniatura. Veremos esto posteriormente.

1.2.2. Redes de área metropolitana

Una **red de área metropolitana**, o **MAN** (*metropolitan area network*) es básicamente una versión más grande de una LAN y normalmente se basa en una tecnología similar. Podría abarcar un grupo de oficinas corporativas cercanas o una ciudad y podría ser privada o pública. Una MAN puede manejar datos y voz, e incluso podría estar relacionada con la red de televisión por cable local. Una MAN sólo tiene uno o dos cables y no contiene elementos de conmutación, los cuales desvían los paquetes por una de varias líneas de salida potenciales. Al no tener que conmutar, se simplifica el diseño.

La principal razón para distinguir las MAN como una categoría especial es que se ha adoptado un estándar para ellas, y este estándar ya se está implementando: se llama **DQDB**

(*distributed queue dual bus*, o ***bus* dual de cola distribuida**) o, para la gente que prefiere números a letras, 802.6 (el número de la norma IEEE que lo define). El DQDB consiste en dos buses (cables) unidireccionales, a los cuales están conectadas todas las computadoras, como se muestra en la figura 1-4. Cada *bus* tiene una cabeza terminal (*head-end*), un dispositivo que inicia la actividad de transmisión. El tráfico destinado a una computadora situada a la derecha del emisor usa el *bus* superior. El tráfico hacia la izquierda usa el de abajo.

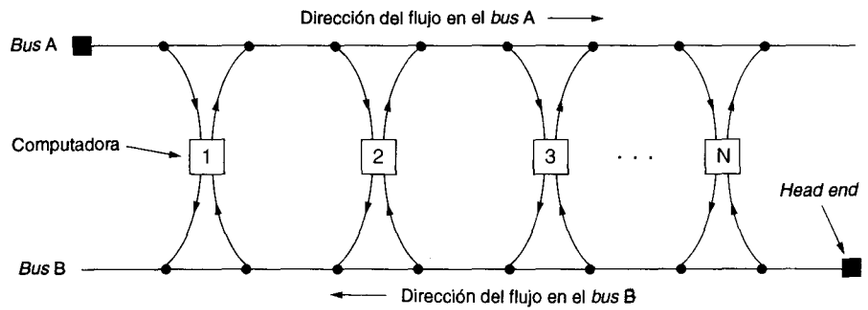


Figura 1-4. Arquitectura de la red de área metropolitana DQDB.

Un aspecto clave de las MAN es que hay un medio de difusión (dos cables, en el caso de la 802.6) al cual se conectan todas las computadoras. Esto simplifica mucho el diseño comparado con otros tipos de redes. Estudiaremos el DQDB con más detalle en el capítulo 4.

1.2.3. Redes de área amplia

Una **red de área amplia**, o **WAN** (*wide area network*), se extiende sobre un área geográfica extensa, a veces un país o un continente; contiene una colección de máquinas dedicadas a ejecutar programas de usuario (es decir, de aplicación). Seguiremos el uso tradicional y llamaremos a estas máquinas ***hosts***. El término **sistema terminal** (*end system*) se utiliza también ocasionalmente en la literatura. Las *hosts* están conectadas por una **subred de comunicación**, o simplemente **subred**. El trabajo de la subred es conducir mensajes de una *host* a otra, así como el sistema telefónico conduce palabras del que habla al que escucha. La separación entre los aspectos exclusivamente de comunicación de la red (la subred) y los aspectos de aplicación (las *hosts*), simplifica enormemente el diseño total de la red.

En muchas redes de área amplia, la subred tiene dos componentes distintos: las líneas de transmisión y los elementos de conmutación. Las líneas de transmisión (también llamadas **circuitos**, **canales** o **troncales**) mueven bits de una máquina a otra.

Los elementos de conmutación son computadoras especializadas que conectan dos o más líneas de transmisión. Cuando los datos llegan por una línea de entrada, el elemento de conmutación debe escoger una línea de salida para reenviarlos. Desafortunadamente, no hay una

terminología estándar para designar estas computadoras; se les denomina **nodos conmutadores de paquetes, sistemas intermedios** y **centrales de conmutación de datos**, entre otras cosas. Como término genérico para las computadoras de conmutación, usaremos la palabra **enrutador**, pero conviene que el lector quede advertido de que no hay consenso sobre la terminología. En este modelo, mostrado en la figura 1-5, cada *host* generalmente está conectada a una LAN en la cual está presente un enrutador, aunque en algunos casos una *host* puede estar conectada directamente a un enrutador. La colección de líneas de comunicación y enrutadores (pero no las *hosts*) forman la subred.

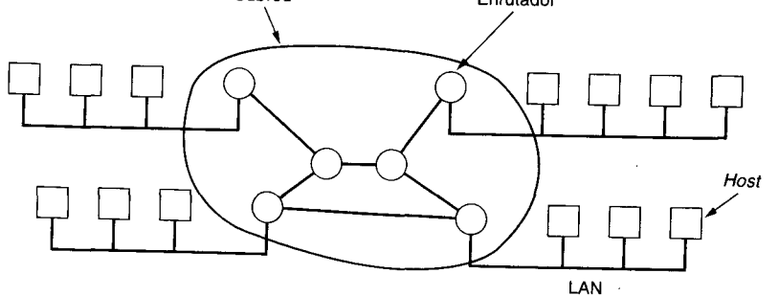


Figura 1-5. Relación entre las *hosts* y la subred.

Es pertinente un comentario al margen acerca del término "subred". Originalmente, sólo significaba la colección de enrutadores y líneas de comunicación que movían los paquetes de la *host* de origen a la *host* de destino. Sin embargo, algunos años después surgió un segundo significado en relación con la identificación de direcciones en la red (lo cual trataremos en el capítulo 5). Así, el término tiene cierta ambigüedad. Desafortunadamente, no existen opciones ampliamente aceptadas para su significado inicial, de modo que muy a pesar nuestro lo usaremos en ambos sentidos. Por el contexto, siempre quedará claro lo que significa la palabra.

En casi todas las WAN, la red contiene numerosos cables o líneas telefónicas, cada una conectada a un par de enrutadores. Si dos enrutadores que no comparten un cable desean comunicarse, deberán hacerlo indirectamente, por medio de otros enrutadores. Cuando se envía un paquete de un enrutador a otro a través de uno o más enrutadores intermedios, el paquete se recibe completo en cada enrutador intermedio, se almacena hasta que la línea de salida requerida está libre, y a continuación se reenvía. Una subred basada en este principio se llama, de **punto a punto**, de **almacenar y reenviar**, o de **paquete conmutado**. Casi todas las redes de área amplia (excepto aquellas que usan satélites) tienen subredes de almacenar y reenviar. Cuando los paquetes son pequeños y el tamaño de todos es el mismo, suelen llamarse **celdas**.

Cuando se usa una subred punto a punto, una consideración de diseño importante es la topología de interconexión del enrutador. La figura 1-6 muestra algunas posibles topologías. Las redes locales que fueron diseñadas como tales usualmente tienen una topología simétrica. En contraste, las redes de área amplia típicamente tienen topologías irregulares.

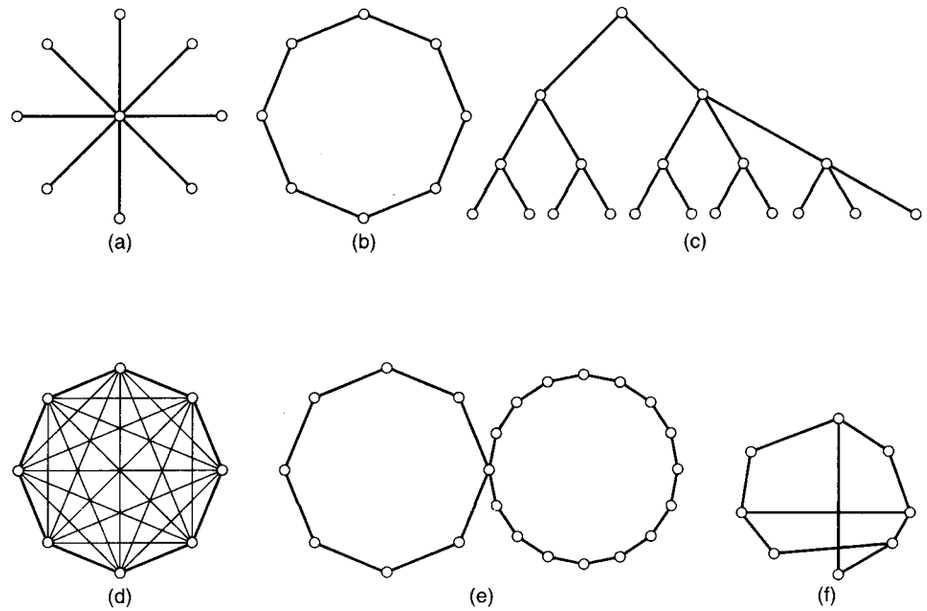


Figura 1-6. Posibles topologías para una subred punto a punto. (a) Estrella. (b) Anillo. (c) Árbol. (d) Completa. (e) Intersección de anillos. (f) Irregular.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE DOCUMENTACION Y BIBLIOTECA

Una segunda posibilidad para una WAN es un sistema de satélite o de radio en tierra. Cada enrutador tiene una antena por medio de la cual puede enviar y recibir. Todos los enrutadores pueden oír las salidas enviadas *desde* el satélite y en algunos casos pueden también oír la transmisión ascendente de los otros enrutadores *hacia* el satélite. Algunas veces los enrutadores están conectados a una subred punto a punto de gran tamaño, y únicamente algunos de ellos tienen una antena de satélite. Por su naturaleza, las redes de satélite son de difusión y son más útiles cuando la propiedad de difusión es importante.

1.2.4. Redes inalámbricas

Las computadoras portátiles, como las *notebooks* y los asistentes personales digitales (PDAs, *personal digital assistants*), son el segmento de más rápido crecimiento de la industria de la computación. Muchos de los dueños de estas computadoras tienen máquinas de escritorio conectadas a LAN y WAN en la oficina y quieren estar conectados a su base de operaciones aun cuando estén lejos de casa o de viaje. Puesto que tener una conexión por cable es imposible en autos y aeroplanos, existe mucho interés en las redes inalámbricas. En esta sección presentaremos brevemente el tema. (Nota: en este libro, consideramos secciones a las denotadas por un número de tres partes, como 1.2.4.)

En realidad, la comunicación inalámbrica digital no es una idea nueva. Ya en 1901 el físico italiano Guglielmo Marconi demostró un telégrafo inalámbrico de barco a costa usando el código Morse (los puntos y rayas son binarios, después de todo). Los sistemas inalámbricos digitales modernos tienen mejor rendimiento, pero la idea básica es la misma. Se puede encontrar información adicional acerca de estos sistemas en (Garg y Wilkes, 1996 y Pahlavan *et al.*, 1995).

Las redes inalámbricas tienen muchos usos. Uno común es la oficina portátil. La gente que viaja con frecuencia quiere usar su equipo electrónico portátil para enviar y recibir llamadas telefónicas, faxes y correo electrónico, leer archivos remotos, entrar en máquinas remotas etc., y hacer esto desde cualquier lugar ya sea en tierra, mar o aire.

Las redes inalámbricas son de gran valor para que las flotillas de camiones, taxis y autobuses, y las personas que hacen reparaciones, mantengan contacto con su base. También pueden usarlas los rescatistas en sitios de desastre (incendios, inundaciones, temblores, etc.) donde se ha dañado el sistema telefónico. Las computadoras pueden enviar mensajes, guardar registros, y muchas otras cosas.

Por último, las redes inalámbricas son importantes para los militares. Si hay necesidad de combatir en cualquier parte del mundo con poco tiempo de aviso, contar con el uso de la infraestructura local de las redes probablemente no sea una buena idea. Es mejor llevar una red propia.

Aunque las redes inalámbricas y las computadoras portátiles con frecuencia están relacionadas, no son idénticas, como lo muestra la figura 1-7. Las computadoras portátiles en ocasiones se conectan a redes alambradas. Por ejemplo, si un viajero conecta una computadora portátil al enchufe telefónico de un hotel, disfruta de movilidad sin una red inalámbrica. Otro ejemplo es cuando se lleva una computadora portátil mientras se inspecciona un tren en busca de problemas técnicos. Aquí el inspector puede arrastrar un cordón largo tras de sí (modelo "aspiradora").

Inalámbrica	Móvil	Aplicaciones
No	No	Estaciones de trabajo estacionarias en oficinas
No	Sí	Uso de una portátil en un hotel; mantenimiento de trenes
Sí	No	LAN en edificios viejos y sin alambrado
Sí	Sí	Oficina portátil; PDA para inventarios

Figura 1-7. Combinación de redes inalámbricas y computación móvil.

Por otro lado, algunas computadoras inalámbricas no son portátiles. Un ejemplo importante es una compañía que posee una construcción vieja, que no tiene instalado cable para red y quiere conectar sus computadoras. Para instalar una LAN inalámbrica sólo necesita comprar una pequeña caja con algo de electrónica y armar algunas antenas. Esta solución puede ser más económica que alambrar el edificio.

Aunque las LAN inalámbricas son fáciles de instalar, también tienen desventajas. Típicamente, su capacidad es de 1 a 2 Mbps, lo cual es mucho más lento que las LAN alambradas. Además las tasas de error son a veces mucho más altas, y las transmisiones desde diferentes computadoras pueden interferirse.

Desde luego, también están las aplicaciones inalámbricas verdaderamente móviles, que van desde la oficina portátil hasta personas que recorren una tienda haciendo el inventario con una PDA. En muchos aeropuertos concurridos, los empleados que reciben carros rentados trabajan en los estacionamientos con computadoras portátiles inalámbricas. Ellos teclean el número de matrícula de los carros devueltos, y su portátil, que tiene una impresora integrada, llama a la computadora principal, obtiene la información de la renta e imprime la factura allí mismo. La verdadera computación móvil se estudia más a fondo en (Forman y Zahorjan, 1994).

Las redes inalámbricas tienen muchas formas. Algunas universidades ya están instalando antenas por todo su campus para permitir a los estudiantes sentarse debajo de los árboles y consultar las tarjetas del catálogo de la biblioteca. Aquí las computadoras se comunican directamente con las LAN inalámbricas en forma digital. Otra posibilidad es usar un teléfono celular (es decir, portátil) con un módem analógico tradicional. El servicio celular digital directo, llamado **CDPD** (*celular digital packet data*, **paquete de datos celular digital**), ya está disponible en muchas ciudades; lo estudiaremos en el capítulo 4.

Finalmente, es posible tener diferentes combinaciones de redes alámbricas e inalámbricas. Por ejemplo, en la figura 1-8(a), se muestra un aeroplano con varias personas usando módems y teléfonos fijos al asiento para llamar a la oficina. Cada llamada es independiente de las otras. Sin embargo, una opción mucho más eficiente es la LAN volante de la figura 1-8(b). Aquí cada asiento viene equipado con una conexión Ethernet a la cual los pasajeros pueden conectar sus computadoras. Un enrutador simple en el avión mantiene un enlace de radio con algún enrutador en tierra, cambiando de enrutador según avanza el vuelo. Esta configuración es una LAN tradicional, excepto que su conexión al mundo externo resulta ser un enlace de radio en lugar de una línea alambrada.

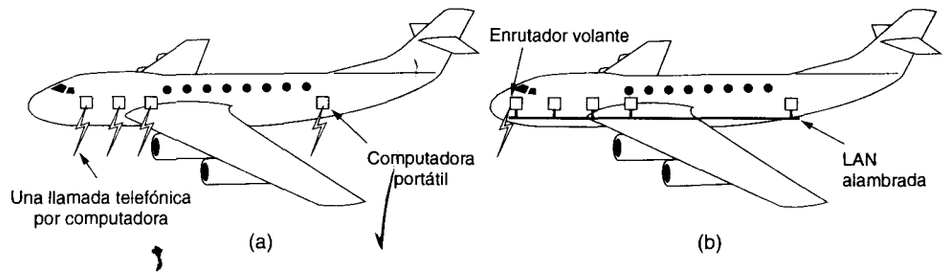


Figura 1-8. (a) Computadoras individuales móviles. (b) Una LAN volante.

Si bien muchas personas creen que las computadoras portátiles son la ola del futuro, se ha oído al menos una voz disidente. Bob Metcalfe, el inventor de la Ethernet, ha escrito: "Las computadoras móviles inalámbricas son como cuartos de baño móviles sin tubería: retretes portátiles. Llegarán a ser comunes en vehículos, en lugares de construcción y en conciertos de rock. Mi consejo es cablear su casa y permanecer ahí" (Metcalfe, 1995). ¿Seguirá la mayoría de la gente la advertencia de Metcalfe? El tiempo lo dirá.

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO

1.2.5. Interredes

Existen muchas redes en el mundo, a veces con diferente *hardware* y *software*. La gente conectada a una red a menudo quiere comunicarse con gente conectada a una red distinta. Esto requiere conectar redes diferentes y con frecuencia incompatibles, algunas veces usando máquinas llamadas **pasarelas** para hacer la conexión y realizar la traducción necesaria, ambas en términos de *hardware* y *software*. Una colección de redes interconectadas se llama **interred**.

Una forma común de interred es una colección de LAN conectadas por una WAN. En efecto, si reemplazamos la etiqueta "subred" en la figura 1.5 por "WAN", nada más en la figura tendría que cambiar. En este caso la única distinción real entre una subred y una WAN es si están o no presentes las *hosts*. Si el sistema dentro de la curva cerrada contiene únicamente enrutadores, es una subred; si contiene tanto enrutadores como *hosts* con sus propios usuarios, es una WAN.

Para evitar confusión, por favor note que la palabra "interred"[1] siempre se usará en este libro en un sentido genérico. Por su parte, la **Internet** (note la I mayúscula) es una red específica mundial que se usa ampliamente para conectar universidades, oficinas de gobierno, compañías y finalmente individuos. Tendremos mucho que decir acerca de las interredes y la Internet más adelante en este libro.

Las subredes, redes e interredes con frecuencia se confunden. La subred tiene su sentido estándar en el contexto de una red de área amplia, donde se refiere a la colección de enrutadores y líneas de comunicación propiedad del operador de la red; por ejemplo, compañías como America Online y CompuServe. Como analogía, el sistema telefónico consiste en centrales telefónicas conectadas unas a otras por líneas de alta velocidad, y a casas y negocios por líneas de baja velocidad. Estas líneas y el equipo, propiedad de la compañía de teléfonos y administrado por ella, forman la subred del sistema telefónico. Los teléfonos por sí mismos (los nodos en esta analogía) no son parte de la subred. La combinación de una subred y sus nodos forma una red. En el caso de una LAN, el cableado y los nodos forman la red; realmente no hay subred.

Se forma una interred cuando se conectan distintas redes entre sí. Desde nuestro punto de vista, al conectar una LAN y una WAN o al conectar dos LAN formamos una interred, pero no hay mucho consenso en la industria acerca de la terminología en esta área.

1.3. *SOFTWARE* DE RED

Las primeras redes de computadoras se diseñaron con el *hardware* como preocupación principal y el *software* como una idea tardía. Esta estrategia ya no funciona; ahora el *software* de la red es altamente estructurado. En las siguientes secciones examinaremos en detalle la técnica de

[1] El comentario del autor se refiere a que el término "interred" corresponde en inglés a la palabra *internet* (naturalmente, con *i* minúscula, y que no debe confundirse con Internet, el nombre de la red mundial) o a *internetwork*. (*N. de supervisor.*)

estructuración del *software*. Los métodos descritos aquí son la piedra angular del libro entero y hablaremos de ellos repetidamente más adelante.

1.3.1. Jerarquías de protocolos

Para reducir la complejidad de su diseño, muchas redes están organizadas como una serie de **capas** o **niveles**, cada una construida sobre la inferior. El número de capas y el nombre, el contenido y la función de cada una difieren de red a red. Sin embargo, en todas las redes el propósito de cada capa es ofrecer ciertos servicios a las capas superiores de modo que no tengan que ocuparse del detalle de la implementación real de los servicios.

La capa *n* de una máquina lleva a cabo una conversación con la capa *n* de otra. Las reglas y convenciones que se siguen en esta conversación se conocen colectivamente como **protocolo** de la capa *n*. Básicamente, un protocolo es un acuerdo entre las partes que se comunican sobre cómo va a proceder la comunicación. Como analogía, cuando una mujer es presentada a un hombre, ella puede elegir entre extender su mano para saludar o no extenderla. Él, a su vez, puede decidir estrechar su mano o besarla, dependiendo, por ejemplo, de si ella es una abogada estadounidense en una reunión de negocios o una princesa europea en un baile de gala. Si se viola el protocolo, la comunicación será más difícil, si no imposible.

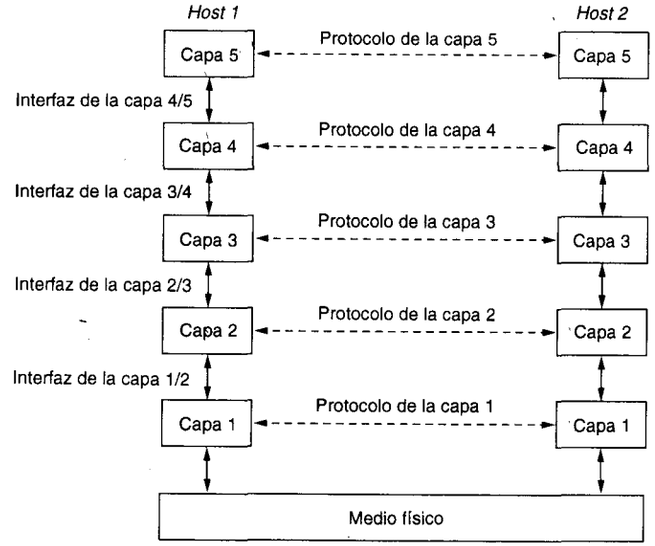


Figura 1-9. Capas, protocolos e interfaces.

En la figura 1-9 se ilustra una red de cinco capas. Las entidades que comprenden las capas correspondientes en las diferentes máquinas se denominan **pares**. En otras palabras, son los pares los que se comunican usando el protocolo.

En realidad, los datos no se transfieren directamente de la capa *n* de una máquina a la capa *n* de otra. Más bien, cada capa pasa datos e información de control a la capa que está inmediatamente debajo de ella, hasta llegar a la capa más baja. Bajo la capa 1 está el **medio físico** a través del cual ocurre la comunicación real. En la figura 1-9, se muestra en líneas punteadas la comunicación virtual y en líneas continuas la comunicación física.

Entre cada par de capas adyacentes hay una **interfaz**. La interfaz define cuáles operaciones y servicios primitivos ofrece la capa inferior a la superior. Cuando los diseñadores de redes deciden cuántas capas incluir en una red y lo que cada una debe hacer, una de las consideraciones más importantes es definir interfaces claras entre las capas. Esto requiere, a su vez, que cada capa ejecute una colección específica de funciones bien conocidas. Además de minimizar la cantidad de información que se debe pasar entre capas, las interfaces bien definidas también simplifican el reemplazo de la implementación de una capa con una implementación completamente diferente (por ejemplo, todas las líneas de teléfonos se reemplazan por canales de satélite), pues todo lo que se requiere de la nueva implementación es que ofrezca a su vecino de arriba exactamente el mismo conjunto de servicios que ofrecía la implementación vieja.

Un conjunto de capas y protocolos recibe el nombre de **arquitectura de red**. La especificación de una arquitectura debe contener información suficiente para que un implementador pueda escribir el programa o construir el *hardware* para cada capa de manera que cada una obedezca en forma correcta el protocolo apropiado. Ni los detalles de la implementación ni la especificación de las interfaces forman parte de la arquitectura porque se encuentran ocultas dentro de las máquinas y no son visibles desde fuera. Ni siquiera es necesario que las interfaces en todas las máquinas de una red sean iguales, siempre que cada máquina pueda usar correctamente todos los protocolos. La lista de protocolos empleados por cierto sistema, con un protocolo por capa, se llama **pila de protocolos**. Los temas de las arquitecturas de red, las pilas de protocolos y los protocolos mismos son la materia principal de este libro.

Una analogía puede ayudar a explicar la idea de la comunicación multicapas. Imagine a dos filósofos (procesos pares de la capa 3), uno de los cuales habla urdu e inglés y el otro habla chino y francés. Ya que no tienen un idioma en común, cada uno contrata un traductor (procesos pares de la capa 2), cada uno de los cuales, a su vez, establece contacto con una secretaria (procesos pares de la capa 1). El filósofo 1 desea comunicar a su par su afecto por el *oryctolagus cuniculus* (el conejo). Para hacerlo, pasa a su traductor un mensaje (en inglés), por conducto de la interfaz 2/3, que dice: "*I like rabbits*", como se ilustra en la figura 1-10. Los traductores acuerdan el uso de un idioma neutral, el holandés, así que el mensaje se convierte en "*Ik hou van konijnen*". El idioma elegido es el protocolo de la capa 2 y la elección corresponde a los procesos pares de la capa 2.

A continuación, el traductor entrega el mensaje a su secretaria para que lo transmita, por ejemplo, por fax (el protocolo de la capa 1). Cuando el mensaje llega, se traduce al francés y se pasa a través de la interfaz 2/3 al filósofo 2. Observe que cada protocolo es independiente por completo de los otros mientras las interfaces no cambien. Los traductores pueden cambiar a voluntad del holandés, por decir, al finlandés, siempre que ambos lo acuerden y que nada cambie su interfaz ya sea con la capa 1 o con la 3. De manera similar, las secretarias pueden cambiar de fax a correo electrónico o a teléfono sin molestar (o incluso sin informar) a las otras capas. Cada

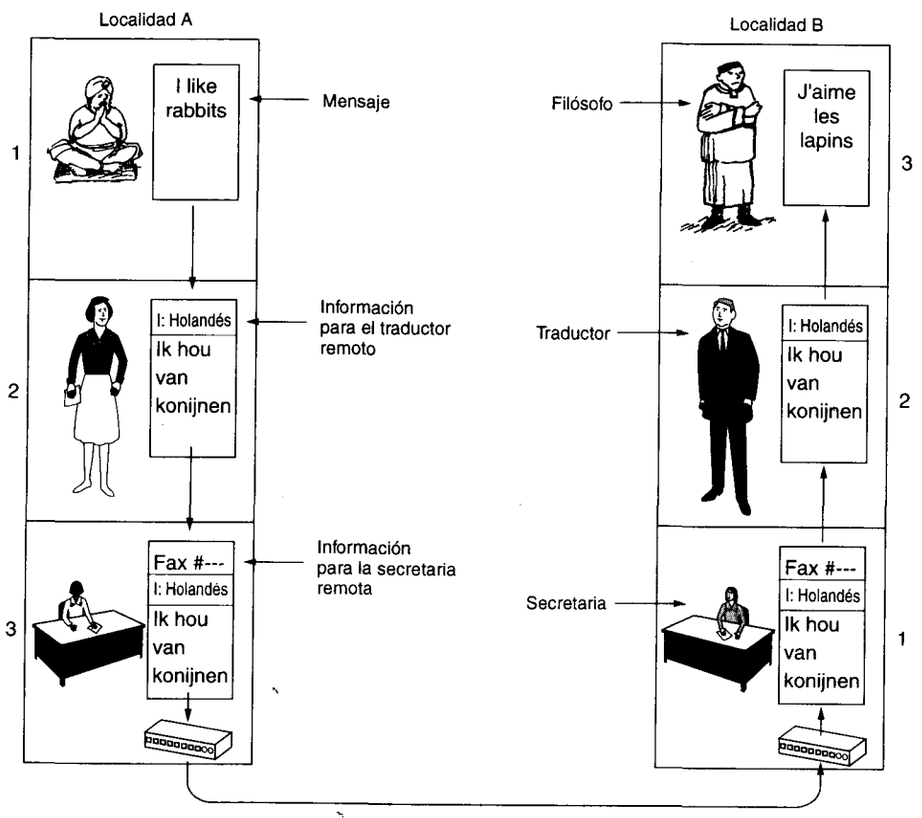


Figura 1-10. La arquitectura filósofo-traductor-secretaria.

proceso puede añadir algo de información dirigida únicamente a su par. Esta información no se pasa a la capa de arriba.

Consideremos ahora un ejemplo más técnico: cómo proveer la comunicación a la capa superior de la red de cinco capas de la figura 1-11. Se produce un mensaje *M* por un proceso de aplicación que se ejecuta en la capa 5 y se entrega a la capa 4 para su transmisión. La capa 4 coloca un **encabezado** al principio del mensaje para identificarlo y pasa el resultado a la capa 3. El encabezado incluye información de control, como números de secuencia, para que la capa 4 en la máquina de destino pueda entregar los mensajes en el orden correcto si las capas inferiores no mantienen la secuencia. En algunas capas, los encabezados contienen también tamaños, horas y otros campos de control.

En muchas redes no hay límite al tamaño de los mensajes que se transmiten en el protocolo de la capa 4, pero casi siempre existe un límite impuesto por el protocolo de la capa 3. En consecuencia, la capa 3 debe dividir los mensajes que le llegan en unidades más pequeñas,

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE
DOCUMENTACIÓN Y BIBLIOTECA

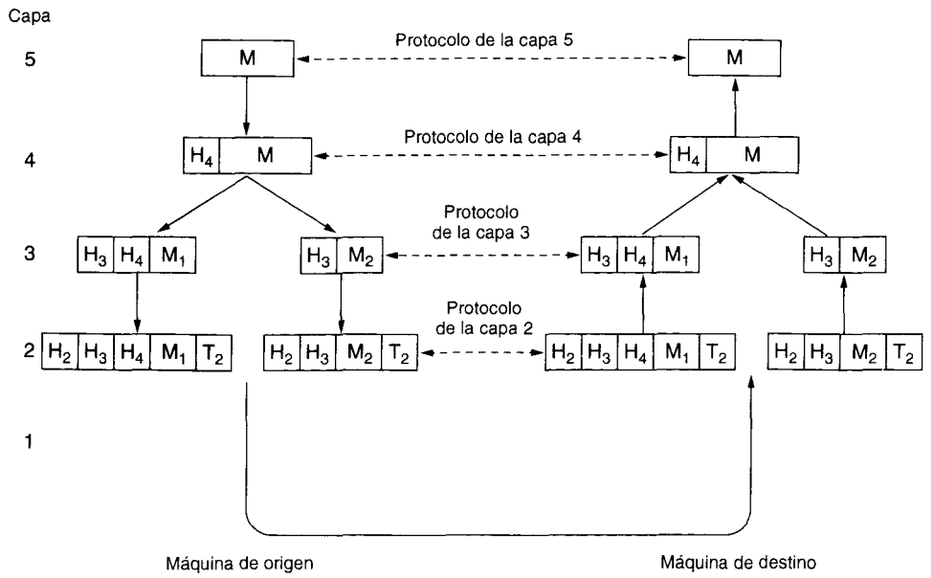


Figura 1-11. Ejemplo del flujo de información que apoya la comunicación virtual en la capa 5.

paquetes, anexando un encabezado de la capa 3 a cada paquete. En este ejemplo, *M* se divide en dos partes, M_1 y M_2.

La capa 3 decide cuál de las líneas que salen usará y pasa los paquetes a la capa 2. La capa 2 no solamente añade un encabezado a cada pieza sino también un apéndice, y entrega la unidad resultante a la capa 1 para su transmisión física. En la máquina receptora el mensaje se mueve hacia arriba, de capa en capa, perdiendo los encabezados conforme avanza. Ninguno de los encabezados para capas inferiores a la *n* pasa hasta la capa *n*.

Lo que es importante recordar de la figura 1-11 es la relación entre la comunicación virtual y la real y la diferencia entre protocolos e interfaces. Por ejemplo, los procesos pares de la capa 4 piensan que su comunicación es "horizontal" empleando el protocolo de la capa 4. Cada uno probablemente tiene un procedimiento llamado algo así como *EnvíaAlOtroLado* y *TomaDelOtroLado,* aunque en realidad estos procedimientos se comunican con capas más bajas a través de la interfaz 2/3, no con el otro lado.

La abstracción de procesos pares es básica para todo diseño de red. Al usarla, la inmanejable tarea de diseñar la red completa se puede dividir en varios problemas de diseño más chicos y manejables, es decir, el diseño de las capas individuales.

Aunque la sección 1.3 se llame "*Software* de red", conviene señalar que las capas más bajas de una jerarquía de protocolos con frecuencia se instrumentan en *hardware* o *firmware*. Sin embargo, implican algoritmos de protocolo complejos, aun si están incluidos (totalmente o en parte) en el *hardware*.

1.3.2. Consideraciones de diseño para las capas

Algunos de los problemas clave en el diseño de redes de computadoras se presentan en varias capas. A continuación mencionaremos en forma breve algunos de los más importantes.

Cada capa necesita un mecanismo para identificar emisores y receptores. Puesto que una red normalmente tiene muchas computadoras, algunas de las cuales tienen múltiples procesos, se requiere un mecanismo para que un proceso en una máquina especifique con quién quiere conversar. Como consecuencia de tener destinos múltiples, se necesita alguna forma de direccionamiento que permita determinar un destino específico.

Otro conjunto de decisiones de diseño concierne a las reglas de la transferencia de datos. En algunos sistemas, los datos viajan solamente en una dirección (**comunicación símplex**). En otros, los datos pueden viajar en cualquier dirección, pero no en forma simultánea (**comunicación semidúplex**). En otras más, los datos viajan en ambas direcciones a la vez (**comunicación dúplex**). El protocolo debe determinar también cuántos canales lógicos corresponden a la conexión y cuáles son sus prioridades. Muchas redes proveen al menos dos canales lógicos por conexión, uno para datos normales y otro para datos urgentes.

El control de errores es una consideración importante porque los circuitos de comunicación física no son perfectos. Se conocen muchos códigos de detección y de corrección de errores, pero ambos extremos de la conexión deben acordar cuál se va a usar. Además, el receptor debe tener alguna forma de indicar al emisor cuáles mensajes se han recibido correctamente y cuáles no.

No todos los canales de comunicación mantienen el orden de los mensajes que se envían. Para lidiar con una posible pérdida de secuencia, el protocolo debe incluir un mecanismo que permita al receptor volver a unir los pedazos en forma apropiada. Una solución obvia es numerar los pedazos, pero esta solución deja abierta la pregunta de qué se debe hacer con los pedazos que llegaron en desorden.

Una consideración importante en todos los niveles es cómo evitar que un emisor rápido sature de datos a un receptor lento. Se han propuesto varias soluciones y se verán más adelante. Algunas de ellas implican una realimentación del receptor al emisor, ya sea en forma directa o indirecta, respecto a la situación actual del receptor. Otras limitan al emisor a una velocidad de transmisión previamente convenida.

Otro problema que se debe resolver en varios niveles es la incapacidad de algunos procesos para aceptar mensajes de longitud arbitraria. Esta propiedad conduce a mecanismos para desensamblar, transmitir y después reensamblar los mensajes. Un problema relacionado es qué hacer cuando los procesos insisten en transmitir datos en unidades tan pequeñas que el envío de cada una por separado es ineficiente. La solución aquí es juntar varios mensajes pequeños dirigidos a un destino común en un solo mensaje largo y desmembrar el mensaje largo en el otro lado.

Cuando no es conveniente o económico establecer una conexión individual para cada par de procesos en comunicación, la capa inferior puede decidir usar la misma conexión para múltiples conversaciones, no relacionadas entre sí. Mientras esta multiplexión y desmultiplexión se haga de manera transparente, se puede usar con cualquier capa. La multiplexión se necesita en la capa

física, por ejemplo, donde se tiene que enviar todo el tráfico para todas las conexiones, cuando mucho, por unos cuantos circuitos físicos.

Cuando entre el origen y el destino hay múltiples trayectorias, se debe elegir una ruta. A veces esta decisión se debe dividir entre dos o más capas. Por ejemplo, para enviar datos de Londres a Roma, podría requerirse una decisión de alto nivel para pasar por Francia o por Alemania basándose en sus leyes de confidencialidad respectivas, y puede que se tuviera que tomar una decisión de bajo nivel para elegir uno de los muchos circuitos disponibles basándose en la carga de tráfico actual.

1.3.3. Interfaces y servicios

La función de cada capa es proporcionar servicios a la capa que está encima de ella. En esta sección veremos precisamente lo que es un servicio con más detalle, pero primero presentaremos algo de terminología.

Los elementos activos de cada capa generalmente se llaman **entidades**. Una entidad puede ser de *software* (como un proceso), o de *hardware* (como un circuito integrado inteligente de entrada/salida). Las entidades de la misma capa en máquinas diferentes se llaman **entidades pares**. Las entidades de la capa n implementan un servicio que usa la capa $n + 1$. En este caso la capa n se llama **proveedor del servicio** y la capa $n + 1$ es el **usuario del servicio**. La capa n puede usar los servicios de la capa $n - 1$ con el fin de proveer su propio servicio; puede ofrecer varias clases de servicio, por ejemplo: comunicación rápida cara y comunicación lenta barata.

Los servicios están disponibles en los **SAP** (*service access points,* **puntos de acceso al servicio**). Los SAP de la capa n son los lugares en los que la capa $n + 1$ puede tener acceso a los servicios ofrecidos. Cada SAP tiene una dirección que lo identifica de manera única. Para aclarar este punto, los SAP del sistema telefónico son los enchufes en los que se pueden conectar los teléfonos modulares, y las direcciones de los SAP son los números telefónicos de estas tomas. Para llamar a alguien necesitamos saber la dirección de SAP de quien debe recibir la llamada. De manera similar, en el sistema postal las direcciones de SAP son las direcciones de calle y de número de apartado postal. Para mandar una carta debemos conocer la dirección de SAP del destinatario.

Para que dos capas intercambien información, tiene que haber un acuerdo sobre el conjunto de reglas relativas a la interfaz. En una interfaz típica, la entidad de la capa $n + 1$ pasa una **IDU** (*interface data unit,* **unidad de datos de la interfaz**) a la entidad de la capa n a través del SAP como se muestra en la figura 1-12. La IDU consiste en una **SDU** (*service data unit,* **unidad de datos de servicio**) y cierta información de control. La SDU es la información que se pasa mediante la red a la entidad par y después hasta la capa $n + 1$. La información de control es necesaria para ayudar a la capa inferior a efectuar su trabajo (por ejemplo, la cantidad de bytes en la SDU) pero no forma parte de los datos mismos.

Para que se transfiera la SDU, la entidad de la capa n puede tener que fragmentarla en varios pedazos, a cada uno de los cuales se le da un encabezado y se envía como una **PDU** (*protocol data unit,* **unidad de datos de protocolo**) independiente, que podría ser un paquete. Las entidades pares usan los encabezados de las PDU para acarrear su protocolo de par. Los encabezados

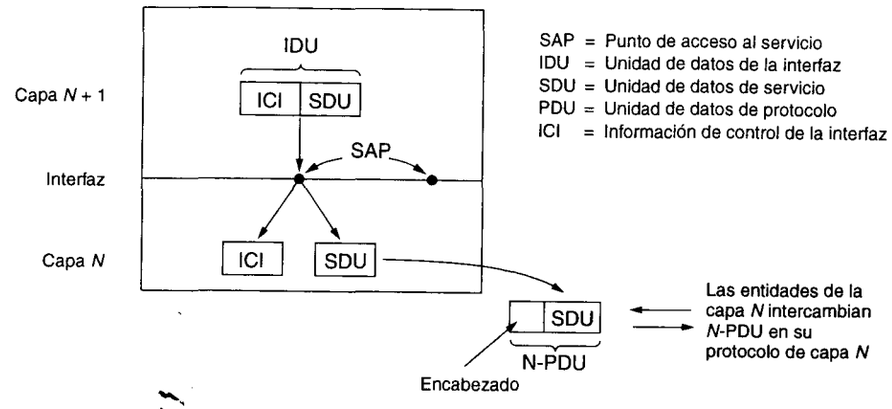


Figura 1-12. Relación entre capas en una interfaz.

indican cuáles PDU contienen datos y cuáles contienen información de control, proveen números de secuencia y cuentas, y otras cosas.

1.3.4. Servicios orientados a conexión y sin conexión

Las capas pueden ofrecer dos tipos diferentes de servicio a las capas que se encuentran sobre ellas, los orientados a la conexión y los que carecen de conexión. En esta sección veremos estos dos tipos y examinaremos las diferencias entre ellos.

El **servicio orientado a la conexión** encuentra su modelo en el sistema telefónico. Para conversar con alguien, descolgamos el teléfono, marcamos el número, hablamos y después colgamos. De manera similar, para usar un servicio de red orientado a la conexión, el usuario del servicio establece primero una conexión, la usa y después la libera. El aspecto esencial de una conexión es que actúa como un tubo: el emisor empuja objetos (bits) por un extremo y el receptor los saca en el mismo orden por el otro extremo.

En contraste, el **servicio sin conexión** toma su modelo del sistema postal. Cada mensaje (carta) lleva la dirección completa de destino, y cada uno se encamina a través del sistema de forma independiente de todos los demás. Normalmente, cuando se envían dos mensajes al mismo destino, el primero que se envió será el primero en llegar. Sin embargo, es posible que el primero que se envió se retrase tanto que el segundo llegue primero. Con un servicio orientado a la conexión, esto es imposible.

Cada servicio se puede caracterizar por una **calidad de servicio**. Algunos servicios son confiables en el sentido de que nunca pierden datos. Usualmente, un servicio confiable se implementa haciendo que el receptor acuse el recibo de cada mensaje, de modo que el emisor esté seguro de que llegó. El proceso de acuse de recibo introduce una sobrecarga y retardos que con frecuencia valen la pena pero que algunas veces son intolerables.

Una situación típica en la que un servicio confiable orientado a la conexión es apropiado es la transferencia de archivos. El propietario del archivo quiere asegurarse de que todos los bits lleguen correctamente y en el mismo orden en que se enviaron. Muy pocos clientes de transferencia de archivos preferirían un servicio que revolviera o perdiera algunos bits ocasionalmente, aun si fuera mucho más rápido.

Los servicios confiables orientados a la conexión tienen dos variantes menores: secuencias de mensajes y corrientes de bytes. En la primera, se mantienen los límites del mensaje. Cuando se envían dos mensajes de 1 kB, llegan como dos mensajes distintos de 1 kB, nunca como un mensaje de 2 kB. (Nota: kB significa kilobytes; kb significa kilobits). En la segunda, la conexión es simplemente una corriente de bytes, sin fronteras entre mensajes. Cuando llegan 2 kB al receptor, no hay forma de saber si se enviaron como un mensaje de 2 kB, dos mensajes de 1 kB o 2048 mensajes de 1 byte. Si las páginas de un libro se enviaran por la red como mensajes separados a una máquina de composición fotográfica, podría ser importante preservar los límites entre los mensajes. Por otra parte, con una terminal que se conecta a un sistema remoto de tiempo compartido, todo lo que se necesita es una corriente de bytes de la terminal a la computadora.

Como se mencionó con anterioridad, en ciertas aplicaciones los retrasos que introducen los acuses de recibo son inaceptables. Una de tales aplicaciones es el tráfico de voz digitalizada. Para los usuarios de teléfonos es preferible oír un poco de ruido en la línea o una palabra confusa de vez en cuando que introducir un retardo para esperar acuses de recibo. De manera similar, al transmitir una videocinta, tener algunos pixeles equivocados no es mayor problema, pero ver la película a jalones conforme el flujo se detiene para corregir errores resulta exasperante.

No todas las aplicaciones requieren conexiones. Por ejemplo, a medida que el correo electrónico se vuelve más común, ¿puede estar muy lejano el correo electrónico chatarra? El que envió el correo electrónico de propaganda probablemente no quiera tomarse la molestia de establecer y después liberar una conexión sólo para enviar un artículo. Tampoco es esencial una entrega 100% confiable, en especial si cuesta más. Todo lo que se necesita es una forma de enviar un solo mensaje que tenga una probabilidad elevada de llegar, aunque no esté garantizada. El servicio sin conexión no confiable (lo que significa sin acuse de recibo) con frecuencia recibe el nombre de **servicio de datagramas**, en analogía con el servicio de telegramas, que tampoco proporciona al emisor un acuse de recibo.

En otras situaciones, es deseable la comodidad de no tener que establecer una conexión para enviar un mensaje corto, pero la confiabilidad es esencial. Para estas aplicaciones se puede proporcionar el **servicio de datagrama con acuse**. Es como mandar una carta certificada y pedir recibo. Cuando el recibo regresa, el remitente tiene la seguridad absoluta de que la carta se entregó a la parte interesada y de que no se perdió en el camino.

Un servicio más es el **servicio de petición y respuesta**. En este servicio el remitente transmite un datagrama sencillo que contiene una petición; la repuesta contiene la contestación. Por ejemplo, una consulta a la biblioteca local para averiguar dónde se habla uighur cae en esta categoría. La petición/respuesta se usa mucho para instrumentar la comunicación en el modelo cliente-servidor; el cliente emite una petición y el servidor le responde. La figura 1-13 resume los tipos de servicio que hemos visto.

	Servicio	Ejemplo
Orientado a la conexión	Flujo de mensaje confiable	Secuencia de páginas
	Flujo de *bytes* confiable	Ingreso remoto
	Conexión no confiable	Voz digitalizada
Sin conexión	Datagrama no confiable	Correo electrónico chatarra
	Datagrama con acuse de recibo	Correo registrado
	Petición/respuesta	Consulta de base de datos

Figura 1-13. Seis tipos diferentes de servicio.

1.3.5. Primitivas de servicios

Un servicio se especifica de manera formal con un conjunto de (operaciones) **primitivas** disponibles para que un usuario u otra entidad acceda al servicio. Estas primitivas ordenan al servicio que ejecute alguna acción o que informe de una acción que haya tomado una entidad par. Una forma de clasificar las primitivas de servicio es dividirlas en cuatro clases según se muestra en la figura 1-14.

Primitiva	Significado
Petición	Una entidad quiere que el servicio haga un trabajo
Indicación	Se le informa a una entidad acerca de un suceso
Respuesta	Una entidad quiere responder a un suceso
Confirmación	Ha llegado la respuesta a una petición anterior

Figura 1-14. Cuatro clases de primitivas de servicio.

Para ilustrar los usos de las primitivas, consideremos cómo se establece y libera una conexión. La entidad que inicia efectúa una petición de conexión *CONNECT.request* que resulta en el envío de un paquete. A continuación, el receptor recibe una indicación de conexión *CONNECT.indication* que le anuncia que en alguna parte una entidad quiere establecer una comunicación con él. Después, la entidad que recibe la indicación usa la primitiva de respuesta a la conexión *CONNECT.response* para indicar si quiere aceptar o rechazar la conexión propuesta. En cualquier caso, la entidad que emite la petición inicial averigua qué sucedió por medio de una primitiva confirmación de conexión *CONNECT.confirm*.

Las primitivas pueden tener parámetros, y la mayor parte de ellas los tiene. Los parámetros de una petición de conexión pueden especificar la máquina a la que se va a conectar, el tipo de servicio deseado y el tamaño máximo de mensaje a usar en la conexión. Los parámetros de una indicación de conexión podrían contener la identidad de quien llama, el tipo de servicio deseado y el tamaño de mensaje máximo propuesto. Si la entidad llamada no está de acuerdo con el tamaño

máximo propuesto, podría presentar una contrapropuesta en su primitiva de *respuesta*, que se pondría a disposición del originador de la llamada en la *confirmación*. Los detalles de esta **negociación** son parte del protocolo. Por ejemplo, en el caso de dos propuestas en conflicto acerca del tamaño máximo del mensaje, el protocolo podría especificar que siempre se elija el más pequeño.

En lo que toca a la terminología, evitaremos cuidadosamente los términos "abrir una conexión" y "cerrar una conexión", porque para los ingenieros en electrónica un "circuito abierto" es uno con una separación o interrupción. La electricidad solamente puede fluir por "circuitos cerrados". Los científicos de la computación nunca estarán de acuerdo en que haya flujo de información por un circuito cerrado. Para pacificar ambos dominios usaremos los términos "establecer una conexión" y "liberar una conexión".

Los servicios pueden ser **confirmados** o **no confirmados**. En un servicio confirmado, existe una *petición,* una *indicación,* una *respuesta* y una *confirmación*. En un servicio no confirmado únicamente hay una *petición* y una *indicación*. El servicio CONNECT (conexión) siempre es un servicio confirmado porque el par remoto debe estar de acuerdo con el establecimiento de una conexión. Por otro lado, la transferencia de datos puede ser confirmada o no confirmada, dependiendo de si el emisor necesita acuse de recibo o no. En las redes se usan ambas clases de servicio.

Para concretar más el concepto de servicio, consideremos como ejemplo un servicio simple orientado a la conexión con las siguientes ocho primitivas de servicio:

1. CONNECT.*request* - Petición para establecer una conexión.
2. CONNECT.*indication* - Envía una señal a la parte llamada.
3. CONNECT.*response* - La usa el receptor para aceptar o rechazar llamadas.
4. CONNECT.*confirm* - Indica al originador si se aceptó o no la llamada.
5. DATA.*request* - Petición de envío de datos.
6. DATA.*indication* - Señal de llegada de los datos.
7. DISCONNECT.*request* - Petición para liberar una conexión.
8. DISCONNECT.*indication* - Indica al par la petición.

En este ejemplo, CONNECT es un servicio confirmado (requiere una respuesta específica), mientras que DISCONNECT es no confirmado (no hay respuesta).

Puede ser de utilidad hacer una analogía con el sistema telefónico para ver cómo se usan estas primitivas. Para esta analogía, considere los pasos que se requieren para llamar por teléfono a la tía Graciela e invitarla a su casa a tomar el té:

1. CONNECT.*request* - Marcar el número de teléfono de la tía Graciela.
2. CONNECT.*indication* - El teléfono de la tía suena.
3. CONNECT.*response* - Ella descuelga el teléfono.
4. CONNECT.*confirm* - Usted escucha que ya no llama.

5. DATA.*request* - Usted la invita a tomar el té.
6. DATA.*indication* - Ella escucha su invitación.
7. DATA.*request* - Ella dice que estaría encantada de ir.
8. DATA.*indication* - Usted escucha que ella acepta.
9. DISCONNECT.*request* - Usted cuelga el teléfono.
10. DISCONNECT.*indication* - Ella escucha y también cuelga el teléfono.

La figura 1-15 muestra esta misma secuencia de pasos como una serie de primitivas de servicio, incluida la confirmación final de desconexión. Cada paso implica una interacción entre dos capas en una de las computadoras. Cada *petición* o *respuesta* causa una *indicación* o *confirmación* en el otro extremo un poco después. En este ejemplo, los usuarios del servicio (usted y la tía Graciela) están en la capa $N + 1$ y el proveedor del servicio (el sistema de teléfonos) está en la capa N.

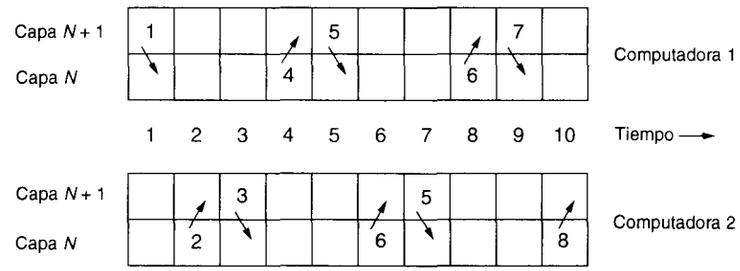


Figura 1-15. Cómo una computadora invitaría al té a la tía Graciela. Los números junto a la cola de cada flecha se refieren a las ocho primitivas de servicio que se vieron en esta sección.

1.3.6. La relación entre servicios y protocolos

Los servicios y los protocolos son conceptos distintos, aunque con frecuencia se les confunde. Sin embargo, esta distinción es tan importante que la subrayamos aquí de nuevo. Un *servicio* es un conjunto de (operaciones) primitivas que ofrece una capa a la que está por encima de ella. El servicio define cuáles son las operaciones que la capa está preparada para ejecutar en beneficio de sus usuarios, pero nada dice respecto de cómo se van a instrumentar estas operaciones. El servicio se refiere a la interfaz entre dos capas, siendo la capa inferior la que provee el servicio y la capa superior la que hace uso de él.

En contraste, un *protocolo* es un conjunto de reglas que gobiernan el formato y el significado de los marcos, paquetes o mensajes que se intercambian entre las entidades pares dentro de una

capa. Las entidades usan protocolos con el fin de instrumentar sus definiciones de servicios; son libres de cambiar sus protocolos a voluntad, siempre que no cambien el servicio visible a sus usuarios. De esta manera el servicio y el protocolo están desacoplados por completo.

Vale la pena hacer una analogía con los lenguajes de programación. Un servicio es como un tipo de datos abstracto o como un objeto en un lenguaje orientado a objetos. Define las operaciones que se pueden ejecutar con un objeto pero no especifica cómo se implementan éstas. Un protocolo se refiere a la *implementación* del servicio y como tal no es visible al usuario del mismo.

Muchos de los protocolos más antiguos no distinguían el servicio del protocolo. En efecto, una capa típica podía haber tenido una primitiva de servicio *ENVIAR PAQUETE* y el usuario proporcionaba un apuntador hacia un paquete totalmente ensamblado. Este arreglo significaba que todos los cambios al protocolo eran visibles de inmediato a los usuarios. La mayoría de diseñadores de redes reconocen ahora tal diseño como un disparate.

1.4. MODELOS DE REFERENCIA

Ya que analizamos en lo abstracto las redes basadas en capas, es hora de ver algunos ejemplos. En las próximas dos secciones veremos dos arquitecturas de red importantes: el modelo de referencia OSI y el modelo de referencia TCP/IP.

1.4.1. El modelo de referencia OSI

El modelo OSI se muestra en la figura 1-16 (menos el medio físico). Este modelo se basa en una propuesta que desarrolló la Organización Internacional de Normas (ISO, por sus siglas en inglés) como primer paso hacia la estandarización internacional de los protocolos que se usan en las diversas capas (Day y Zimmermann, 1983). El modelo se llama **modelo de referencia OSI** (*open systems interconnection,* **interconexión de sistemas abiertos) de la ISO** puesto que se ocupa de la conexión de sistemas abiertos, esto es, sistemas que están abiertos a la comunicación con otros sistemas. Usualmente lo llamaremos sólo modelo OSI para acortar.

El modelo OSI tiene siete capas. Los principios que se aplicaron para llegar a las siete capas son los siguientes:

1. Se debe crear una capa siempre que se necesite un nivel diferente de abstracción.
2. Cada capa debe realizar una función bien definida.
3. La función de cada capa se debe elegir pensando en la definición de protocolos estandarizados internacionalmente.
4. Los límites de las capas deben elegirse a modo de minimizar el flujo de información a través de las interfaces.

5. La cantidad de capas debe ser suficiente para no tener que agrupar funciones distintas en la misma capa y lo bastante pequeña para que la arquitectura no se vuelva inmanejable.

Más adelante estudiaremos cada capa del modelo en orden, empezando por la capa del fondo. Note que el modelo de OSI en sí no es una arquitectura de red porque no especifica los servicios y protocolos exactos que se han de usar en cada capa; sólo dice lo que debe hacer cada capa. Sin embargo, la ISO también ha elaborado estándares para todas las capas, aunque no sean parte del modelo de referencia mismo. Cada uno se ha publicado por separado como norma internacional.

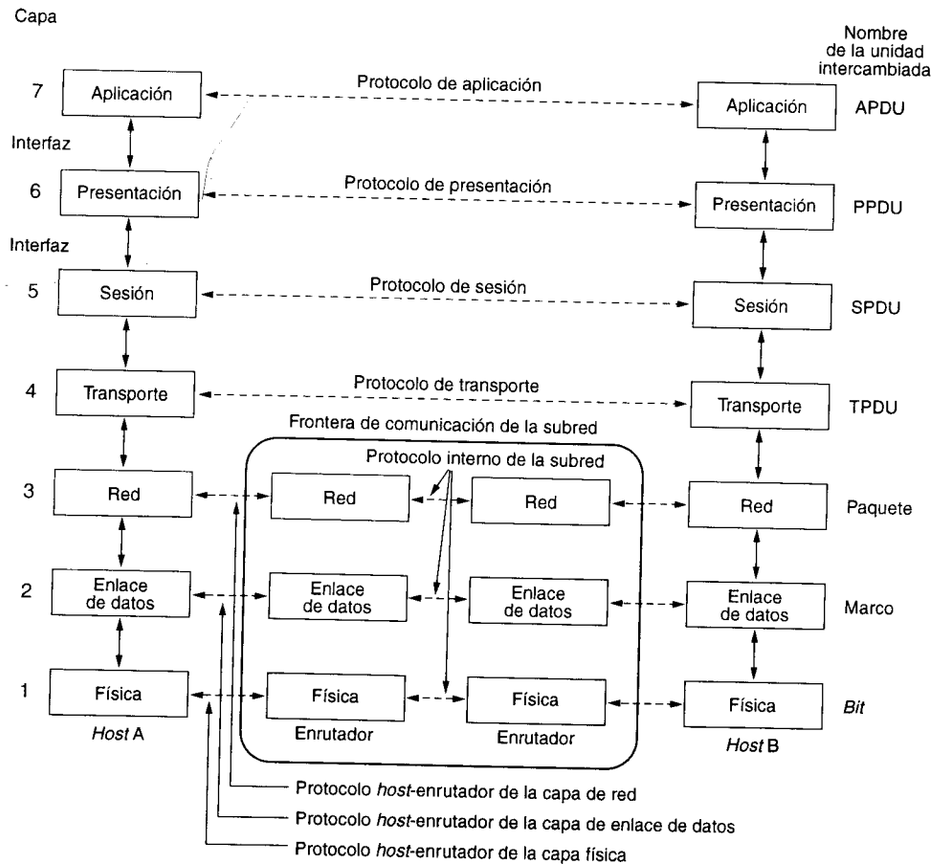


Figura 1-16. El modelo de referencia OSI.

La capa física

La **capa física** tiene que ver con la transmisión de bits por un canal de comunicación. Las consideraciones de diseño tienen que ver con la acción de asegurarse de que cuando un lado envíe un bit 1, se reciba en el otro lado como bit 1, no como bit 0. Las preguntas típicas aquí son: cuántos volts deberán usarse para representar un 1 y cuántos para un 0; cuántos microsegundos dura un bit; si la transmisión se puede efectuar simultáneamente en ambas direcciones o no; cómo se establece la conexión inicial y cómo se interrumpe cuando ambos lados han terminado; y cuántas puntas tiene el conector de la red y para qué sirve cada una. Aquí las consideraciones de diseño tienen mucho que ver con las interfaces mecánica, eléctrica y de procedimientos, y con el medio de transmisión físico que está bajo la capa física.

La capa de enlace de datos

La tarea principal de la **capa de enlace de datos** es tomar un medio de transmisión en bruto y transformarlo en una línea que parezca libre de errores de transmisión no detectados a la capa de red. Esta tarea la cumple al hacer que el emisor divida los datos de entrada en **marcos de datos** (unos cientos o miles de bytes, normalmente), que transmita los marcos en forma secuencial y procese los **marcos de acuse de recibo** que devuelve el receptor. Puesto que la capa física solamente acepta y transmite una corriente de bits sin preocuparse por su significado o su estructura, corresponde a la capa de enlace de datos crear y reconocer los límites de los marcos. Esto se puede lograr añadiendo patrones especiales de bits al principio y al final del marco. Si estos patrones de bits ocurrieran en los datos por accidente, se debe tener cuidado especial para asegurar que estos patrones no se interpreten incorrectamente como delimitadores de marcos.

Una ráfaga de ruido en la línea puede destruir por completo un marco. En este caso, el *software* de la capa de enlace de datos de la máquina fuente puede retransmitir el marco. Sin embargo, las transmisiones repetidas del mismo marco introducen la posibilidad de duplicar marcos. Se podría enviar un marco duplicado si se perdiera el marco del acuse de recibo que el receptor devuelve al emisor. Corresponde a esta capa resolver el problema provocado por los marcos dañados, perdidos y duplicados. La capa de enlace de datos puede ofrecer varias clases de servicio distintas a la capa de la red, cada una con diferente calidad y precio.

Otra consideración que surge en la capa de enlace de datos (y también de la mayor parte de las capas más altas) es cómo evitar que un transmisor veloz sature de datos a un receptor lento. Se debe emplear algún mecanismo de regulación de tráfico para que el transmisor sepa cuánto espacio de almacenamiento temporal (*buffer*) tiene el receptor en ese momento. Con frecuencia esta regulación de flujo y el manejo de errores están integrados.

Si se puede usar la línea para transmitir datos en ambas direcciones, esto introduce una nueva complicación que el *software* de la capa de enlace de datos debe considerar. El problema es que los marcos de acuse de recibo para el tráfico de *A* a *B* compiten por el uso de la línea con marcos de datos para el tráfico de *B* a *A*. Ya se inventó una solución inteligente (plataformas transportadoras) que veremos en detalle más adelante.

Las redes de difusión tienen una consideración adicional en la capa de enlace de datos: cómo controlar el acceso al canal compartido. Una subcapa especial de la capa de enlace de datos se encarga de este problema, la subcapa de acceso al medio.

La capa de red

La **capa de red** se ocupa de controlar el funcionamiento de la subred. Una consideración clave de diseño es determinar cómo se encaminan los paquetes de la fuente a su destino. Las rutas se pueden basar en tablas estáticas que se "alambran" en la red y rara vez cambian. También se pueden determinar al inicio de cada conversación, por ejemplo en una sesión de terminal. Por último, pueden ser altamente dinámicas, determinándose de nuevo con cada paquete para reflejar la carga actual de la red.

Si en la subred se encuentran presentes demasiados paquetes a la vez, se estorbarán mutuamente, formando cuellos de botella. El control de tal congestión pertenece también a la capa de red.

En vista de que los operadores de la subred podrían esperar remuneración por su labor, con frecuencia hay una función de contabilidad integrada a la capa de red. Cuando menos, el *software* debe contar cuántos paquetes o caracteres o bits envía cada cliente para producir información de facturación. Cuando un paquete cruza una frontera nacional, con tarifas diferentes de cada lado, la contabilidad se puede complicar.

Cuando un paquete debe viajar de una red a otra para alcanzar su destino, pueden surgir muchos problemas. El tipo de direcciones que usa la segunda red puede ser diferente del de la primera; puede ser que la segunda no acepte en absoluto el paquete por ser demasiado grande; los protocolos pueden diferir y otras cosas. La capa de red debe resolver todos estos problemas para lograr que se interconecten redes heterogéneas.

En las redes de difusión el problema del ruteo es simple y la capa de red con frecuencia es delgada o incluso inexistente.

La capa de transporte

La función básica de la **capa de transporte** es aceptar datos de la capa de sesión, dividirlos en unidades más pequeñas si es necesario, pasarlos a la capa de red y asegurar que todos los pedazos lleguen correctamente al otro extremo. Además, todo esto se debe hacer de manera eficiente y en forma que aísle a las capas superiores de los cambios inevitables en la tecnología del *hardware*.

En condiciones normales, la capa de transporte crea una conexión de red distinta para cada conexión de transporte que requiera la capa de sesión. Sin embargo, si la conexión de transporte requiere un volumen de transmisión alto, la capa de transporte podría crear múltiples conexiones de red, dividiendo los datos entre las conexiones para aumentar el volumen. Por otro lado, si es costoso crear o mantener una conexión de red, la capa de transporte puede multiplexar varias conexiones de transporte en la misma conexión de red para reducir el costo. En todos los casos, la capa de transporte debe lograr que la multiplexión sea transparente para la capa de sesión.

La capa de transporte determina también qué tipo de servicio proporcionará a la capa de sesión y, finalmente, a los usuarios de la red. El tipo más popular de conexión de transporte es un canal de punto a punto libre de errores que entrega mensajes o bytes en el orden en que se enviaron. Sin embargo, otras posibles clases de servicio de transporte son el transporte de mensajes aislados sin garantía respecto al orden de entrega y la difusión de mensajes a múltiples destinos. El tipo de servicio se determina al establecer la sesión.

La capa de transporte es una verdadera capa de extremo a extremo, del origen al destino. En otras palabras, un programa en la máquina fuente sostiene una conversación con un programa similar en la máquina de destino, haciendo uso de los encabezados de mensajes y de los mensajes de control. En las capas bajas, los protocolos se usan entre cada máquina y sus vecinas inmediatas, y no entre las máquinas de origen y destino, que pueden estar separadas por muchos enrutadores. La diferencia entre las capas 1 a la 3, que están encadenadas, y las capas 4 a la 7, que son de extremo a extremo, se ilustra en la figura 1-16. Muchos nodos están multiprogramados, lo que implica que múltiples conexiones entran y salen de cada nodo. En este caso se necesita una manera de saber cuál mensaje pertenece a cuál conexión. El encabezado de transporte (H_4 en la figura 1-11), es una opción para colocar esta información.

Además de multiplexar varias corrientes de mensajes por un canal, la capa de transporte debe cuidar de establecer y liberar conexiones a través de la red. Esto requiere alguna clase de mecanismo de asignación de nombres, de modo que un proceso en una máquina pueda describir con quién quiere conversar. También debe haber un mecanismo para regular el flujo de información, a fin de que un nodo rápido no pueda saturar a uno lento. Tal mecanismo se llama **control de flujo** y desempeña un papel clave en la capa de transporte (también en otras capas). El control de flujo entre nodos es distinto del control de flujo entre enrutadores, aunque después veremos que se aplican principios similares a ambos.

La capa de sesión

La capa de sesión permite a los usuarios de máquinas diferentes establecer sesiones entre ellos. Una sesión permite el transporte ordinario de datos, como lo hace la capa de transporte, pero también proporciona servicios mejorados que son útiles en algunas aplicaciones. Se podría usar una sesión para que el usuario se conecte a un sistema remoto de tiempo compartido o para transferir un archivo entre dos máquinas.

Uno de los servicios de la capa de sesión es manejar el control del diálogo. Las sesiones pueden permitir que el tráfico vaya en ambas direcciones al mismo tiempo, o sólo en una dirección a la vez. Si el tráfico puede ir únicamente en un sentido a la vez (en analogía con una sola vía de ferrocarril), la capa de sesión puede ayudar a llevar el control de los turnos.

Un servicio de sesión relacionado es el **manejo de fichas**. Para algunos protocolos es esencial que ambos lados no intenten la misma operación al mismo tiempo. A fin de controlar estas actividades, la capa de sesión proporciona fichas que se pueden intercambiar. Solamente el lado que posea la ficha podrá efectuar la operación crítica.

Otro servicio de sesión es la **sincronización**. Considere los problemas que pueden ocurrir cuando se trata de efectuar una transferencia de archivos de 2 horas de duración entre dos

máquinas que tienen un tiempo medio entre rupturas de 1 hora. Cada transferencia, después de abortar, tendría que empezar de nuevo desde el principio y probablemente fallaría también la siguiente vez. Para eliminar este problema, la capa de sesión ofrece una forma de insertar puntos de verificación en la corriente de datos, de modo que después de cada interrupción sólo se deban repetir los datos que se transfirieron después del último punto de verificación.

La capa de presentación

La **capa de presentación** realiza ciertas funciones que se piden con suficiente frecuencia para justificar la búsqueda de una solución general, en lugar de dejar que cada usuario resuelva los problemas. En particular, y a diferencia de todas las capas inferiores que se interesan sólo en mover bits de manera confiable de acá para allá, la capa de presentación se ocupa de la sintaxis y la semántica de la información que se transmite.

Un ejemplo típico de servicio de presentación es la codificación de datos en una forma estándar acordada. La mayor parte de los programas de usuario no intercambian cadenas de bits al azar; intercambian cosas como nombres de personas, fechas, cantidades de dinero y cuentas. Estos elementos se representan como cadenas de caracteres, enteros, cantidades de punto flotante y estructuras de datos compuestas de varios elementos más simples. Las diferentes computadoras tienen códigos diferentes para representar cadenas de caracteres (por ejemplo, ASCII y Unicode), enteros (por ejemplo, en complemento a uno y en complemento a dos), y demás. Con el fin de hacer posible la comunicación entre computadoras con representaciones diferentes, las estructuras de datos por intercambiar se pueden definir en forma abstracta, junto con un código estándar que se use "en el cable". La capa de presentación maneja estas estructuras de datos abstractas y las convierte de la representación que se usa dentro de la computadora a la representación estándar de la red y viceversa.

La capa de aplicación

La **capa de aplicación** contiene varios protocolos que se necesitan con frecuencia. Por ejemplo, existen cientos de tipos de terminales incompatibles en el mundo. Considere la situación de un editor de pantalla completa que debe trabajar en una red con muchos tipos diferentes de terminal, cada uno con formatos diferentes de pantalla, secuencias de escape para insertar y eliminar texto, mover el cursor, etcétera.

Una forma de resolver este problema es definir una **terminal virtual de red** abstracta que los editores y otros programas puedan manejar. Para cada tipo de terminal, se debe escribir un programa para establecer la correspondencia entre las funciones de la terminal virtual de red y las de la terminal real. Por ejemplo, cuando el editor mueva el cursor de la terminal virtual a la esquina superior izquierda de la pantalla, este *software* debe emitir la secuencia apropiada de órdenes a la terminal real para poner su cursor en ese lugar. Todo el *software* de terminal virtual está en la capa de aplicación.

Otra función de la capa de aplicación es la transferencia de archivos. Los diferentes sistemas de archivos tienen convenciones diferentes para nombrar los archivos, formas diferentes de

representar líneas de texto, etc. La transferencia de un archivo entre dos sistemas diferentes requiere la resolución de éstas y otras incompatibilidades. Este trabajo también pertenece a la capa de aplicación, lo mismo que el correo electrónico, la carga remota de trabajos, la búsqueda en directorios y otros recursos de uso general y especial.

Transmisión de datos en el modelo OSI

La figura 1-17 muestra un ejemplo de cómo se pueden transmitir datos empleando el modelo OSI. El proceso remitente tiene algunos datos que quiere enviar al proceso receptor, así que entrega los datos a la capa de aplicación, la cual añade entonces al principio el encabezado de aplicación *AH* (que puede ser nulo) y entrega el elemento resultante a la capa de presentación.

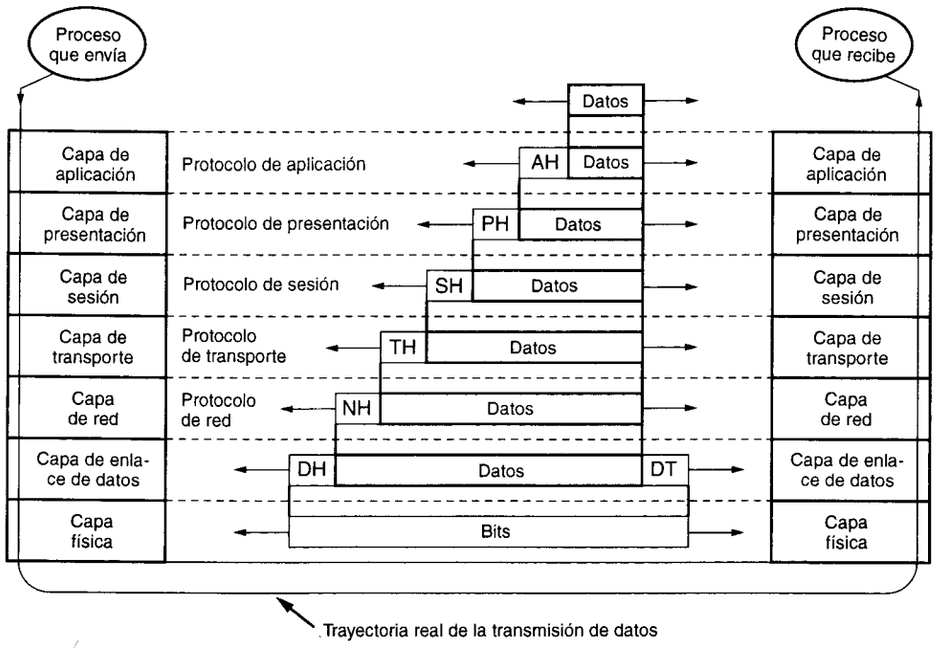


Figura 1-17. Ejemplo de uso del modelo OSI. Algunos de los encabezados pueden ser nulos. (Fuente: H.C. Folts. Usado con autorización.)

La capa de presentación puede transformar este elemento de diferentes maneras y posiblemente añadir al principio un encabezado, entregando el resultado a la capa de sesión. Es importante darse cuenta que la capa de presentación no sabe cuál porción de los datos entregados a ella por la capa de aplicación es la *AH,* si existe, y cuáles son en verdad los datos del usuario.

Este proceso se repite hasta que los datos alcanzan la capa física, donde son transmitidos realmente a la máquina receptora. En esa máquina se retiran los distintos encabezados, uno por

uno, conforme el mensaje se propaga hacia arriba por las capas hasta que por fin llega al proceso receptor.

La idea clave en todo este proceso es que aunque la transmisión real de los datos es vertical en la figura 1-17, cada capa se programa como si fuera horizontal. Por ejemplo, cuando la capa de transporte emisora recibe un mensaje de la capa de sesión, le añade un encabezado de transporte y lo envía a la capa de transporte receptora. Desde su punto de vista, el hecho de que en realidad debe dirigir el mensaje a la capa de red de su propia máquina es un tecnicismo sin importancia. A manera de analogía, cuando un diplomático que habla tagalog se dirige a las Naciones Unidas, piensa que se dirige a los demás diplomáticos de la asamblea. El hecho de que en realidad sólo hable con su traductor se ve como un detalle técnico.

1.4.2. El modelo de referencia TCP/IP

Pasemos ahora del modelo de referencia OSI al modelo que se usa en la abuela de todas las redes de computadoras, la ARPANET, y su sucesora, la Internet mundial. Aunque más adelante presentaremos una breve historia de la ARPANET, es de utilidad mencionar ahora algunos de sus aspectos. La ARPANET era una red de investigación patrocinada por el DoD (Departamento de Defensa de Estados Unidos). Al final conectó a cientos de universidades e instalaciones del gobierno usando líneas telefónicas rentadas. Cuando más tarde se añadieron redes de satélite y radio, los protocolos existentes tuvieron problemas para interactuar con ellas, de modo que se necesitó una arquitectura de referencia nueva. Así, la capacidad de conectar entre sí múltiples redes de manera inconsútil fue uno de los principales objetivos de diseño desde el principio. Esta arquitectura se popularizó después como el **modelo de referencia TCP/IP**, por las iniciales de sus dos protocolos primarios. Este modelo se definió por primera vez en (Cerf y Kahn, 1974). En (Leiner *et al.*, 1985) se da una perspectiva posterior. La metodología de diseño en que se basa el modelo se aborda en (Clark, 1988).

Debido a la preocupación del DoD por que alguno de sus costosos nodos, enrutadores o pasarelas de interredes pudiera ser objeto de un atentado en cualquier momento, otro de los objetivos principales fue que la red fuera capaz de sobrevivir a la pérdida del *hardware* de subred sin que las conversaciones existentes se interrumpieran. En otras palabras, el DoD quería que las conexiones permanecieran intactas mientras las máquinas de origen y destino estuvieran funcionando, aun si alguna de las máquinas o de las líneas de transmisión en el trayecto dejara de funcionar en forma repentina. Es más, se necesitaba una arquitectura flexible, pues se tenía la visión de aplicaciones con requerimientos divergentes, abarcando desde la transferencia de archivos hasta la transmisión de discursos en tiempo real.

La capa de interred

Todos estos requerimientos condujeron a la elección de una red de conmutación de paquetes basada en una capa de interred carente de conexiones. Esta capa, llamada **capa de interred**, es el eje que mantiene unida toda la arquitectura. La misión de esta capa es permitir que los nodos

inyecten paquetes en cualquier red y los hagan viajar de forma independiente a su destino (que podría estar en una red diferente). Los paquetes pueden llegar incluso en un orden diferente a aquel en que se enviaron, en cuyo caso corresponde a las capas superiores reacomodarlos, si se desea la entrega ordenada. Nótese que aquí se usa "interred" en un sentido genérico, aunque esta capa esté presente en la Internet.

Aquí la analogía es con el sistema de correos (lento). Una persona puede depositar una secuencia de cartas internacionales en un buzón en un país, y con un poco de suerte, casi todas se entregarán en la dirección correcta en el país de destino. Es probable que las cartas viajen a través de una o más pasarelas internacionales de correo en el camino, pero esto es transparente para los usuarios. Más aún, los usuarios no necesitan saber que cada país (esto es, cada red), tiene sus propias estampillas, tamaños preferidos de sobres y reglas de entrega.

La capa de interred define un formato de paquete y protocolo oficial llamado **IP** (*Internet protocol,* **protocolo de interred**). El trabajo de la capa de interred es entregar paquetes IP a donde se supone que deben ir. Aquí la consideración más importante es claramente el ruteo de los paquetes, y también evitar la congestión. Por lo anterior es razonable decir que la capa de interred TCP/IP es muy parecida en funcionalidad a la capa de red OSI. La figura 1-18 muestra la correspondencia.

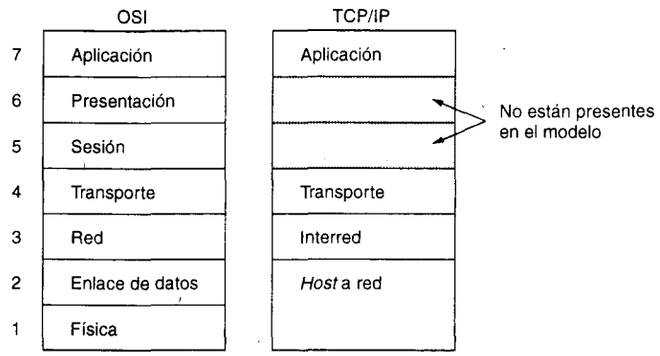


Figura 1-18. El modelo de referencia TCP/IP.

La capa de transporte

La capa que está sobre la capa de interredes en el modelo TCP/IP se llama usualmente ahora **capa de transporte**. Esta capa se diseñó para permitir que las entidades pares en los nodos de origen y destino lleven a cabo una conversación, lo mismo que en la capa de transporte OSI. Aquí se definieron dos protocolos de extremo a extremo. El primero, **TCP** (*transmission control protocol,* **protocolo de control de la transmisión**) es un protocolo confiable orientado a la conexión que permite que una corriente de bytes originada en una máquina se entregue sin errores en cualquier otra máquina de la interred. Este protocolo fragmenta la corriente entrante

de bytes en mensajes discretos y pasa cada uno a la capa de interred. En el destino, el proceso TCP receptor reensambla los mensajes recibidos para formar la corriente de salida. El TCP también se encarga del control de flujo para asegurar que un emisor rápido no pueda abrumar a un receptor lento con más mensajes de los que pueda manejar.

El segundo protocolo de esta capa, el **UDP** (*user datagram protocol,* **protocolo de datagrama de usuario**), es un protocolo sin conexión, no confiable, para aplicaciones que no necesitan la asignación de secuencia ni el control de flujo del TCP y que desean utilizar los suyos propios. Este protocolo también se usa ampliamente para consultas de petición y respuesta de una sola ocasión, del tipo cliente-servidor, y en aplicaciones en las que la entrega pronta es más importante que la entrega precisa, como las transmisiones de voz o vídeo. La relación entre IP, TCP y UDP se muestra en la figura 1-19. Desde que se desarrolló el modelo, el IP se ha implementado en muchas otras redes.

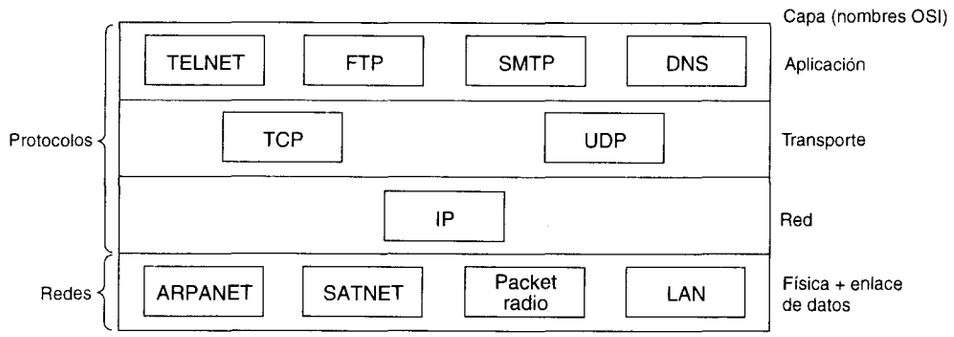


Figura 1-19. Protocolos y redes en el modelo TCP/IP inicial.

La capa de aplicación

El modelo TCP/IP no tiene capas de sesión ni de presentación. No se pensó que fueran necesarias, así que no se incluyeron. La experiencia con el modelo OSI ha comprobado que esta visión fue correcta: se utilizan muy poco en la mayor parte de las aplicaciones.

Encima de la capa de transporte está la **capa de aplicación**, que contiene todos los protocolos de alto nivel. Entre los protocolos más antiguos están el de terminal virtual (TELNET), el de transferencia de archivos (FTP) y el de correo electrónico (SMTP), según se muestra en la figura 1-19. El protocolo de terminal virtual permite que un usuario en una máquina ingrese en una máquina distante y trabaje ahí. El protocolo de transferencia de archivos ofrece un mecanismo para mover datos de una máquina a otra en forma eficiente. El correo electrónico fue en sus orígenes sólo una clase de transferencia de archivos, pero más adelante se desarrolló para él un protocolo especializado; con los años, se le han añadido muchos otros protocolos, como el servicio de nombres de dominio (DNS) para relacionar los nombres de los nodos con sus

direcciones de la red; NNTP, el protocolo que se usa para transferir artículos noticiosos; HTTP, el protocolo que se usa para recuperar páginas en la *World Wide Web* y muchos otros.

La capa del nodo a la red

Bajo la capa de interred está un gran vacío. El modelo de referencia TCP/IP realmente no dice mucho de lo que aquí sucede, fuera de indicar que el nodo se ha de conectar a la red haciendo uso de algún protocolo de modo que pueda enviar por ella paquetes de IP. Este protocolo no está definido y varía de un nodo a otro y de red a red. Los libros y artículos sobre el modelo TCP/IP rara vez hablan de él.

1.4.3. Comparación de los modelos de referencia OSI y TCP

Los modelos de referencia OSI y TCP/IP tienen mucho en común. Ambos se basan en el concepto de un gran número de protocolos independientes. También la funcionalidad de las capas es muy similar. Por ejemplo, en ambos modelos las capas por encima de la de transporte, incluida ésta, están ahí para prestar un servicio de transporte de extremo a extremo, independiente de la red, a los procesos que deseen comunicarse. Estas capas forman el proveedor de transporte. También en ambos modelos, las capas encima de la de transporte son usuarios del servicio de transporte orientados a aplicaciones.

A pesar de estas similitudes fundamentales, los dos modelos tienen también muchas diferencias. En esta sección enfocaremos las diferencias clave entre los dos modelos de referencia. Es importante notar que aquí estamos comparando los *modelos de referencia,* no las *pilas de protocolos* correspondientes. Los protocolos mismos se estudiarán después. Un libro dedicado a comparar y contrastar TCP/IP y OSI es (Piscitello y Chapin, 1993).

En el modelo OSI, tres conceptos son fundamentales:

1. Servicios.
2. Interfaces.
3. Protocolos.

Es probable que la contribución más importante del modelo OSI sea hacer explícita la distinción entre estos tres conceptos. Cada capa presta algunos servicios a la capa que se encuentra sobre ella. La definición de *servicio* dice lo que la capa hace, no cómo es que las entidades superiores tienen acceso a ella o cómo funciona la capa.

La *interfaz* de una capa les dice a los procesos de arriba cómo acceder a ella; especifica cuáles son los parámetros y qué resultados esperar; nada dice tampoco sobre cómo trabaja la capa por dentro.

Finalmente, los *protocolos* pares que se usan en una capa son asunto de la capa. Ésta puede usar los protocolos que quiera, siempre que consiga que se realice el trabajo (esto es, que provea los servicios que ofrece). La capa también puede cambiar los protocolos a voluntad sin afectar el *software* de las capas superiores.

Estas ideas ajustan muy bien con las ideas modernas acerca de la programación orientada a objetos. Al igual que una capa, un objeto tiene un conjunto de métodos (operaciones) que los procesos pueden invocar desde fuera del objeto. La semántica de estos métodos define el conjunto de servicios que ofrece el objeto. Los parámetros y resultados de los métodos forman la interfaz del objeto. El código interno del objeto es su protocolo y no está visible ni es de la incumbencia de las entidades externas al objeto.

El modelo TCP/IP originalmente no distinguía en forma clara entre servicio, interfaz y protocolo, aunque se ha tratado de reajustarlo después a fin de hacerlo más parecido a OSI. Por ejemplo, los únicos servicios reales que ofrece la capa de interred son SENT IP PACKET y RESEIVE IP PACKET para enviar y recibir paquetes de IP, respectivamente.

Como consecuencia, en el modelo OSI se ocultan mejor los protocolos que en el modelo TCP/IP y se pueden reemplazar con relativa facilidad al cambiar la tecnología. La capacidad de efectuar tales cambios es uno de los principales propósitos de tener protocolos por capas en primer lugar.

El modelo de referencia OSI se desarrolló *antes* de que se inventaran los protocolos. Este orden significa que el modelo no se orientó hacia un conjunto específico de protocolos, lo cual lo convirtió en algo muy general. El lado malo de este orden es que los diseñadores no tenían mucha experiencia con el asunto y no supieron bien cuál funcionalidad poner en cuál capa.

Por ejemplo, la capa de enlace de datos originalmente tenía que ver sólo con redes de punto a punto. Cuando llegaron las redes de difusión, se tuvo que insertar una nueva subcapa en el modelo. Cuando la gente empezó a construir redes reales haciendo uso del modelo OSI y de los protocolos existentes, descubrió que no cuadraban con las especificaciones de servicio requeridas (maravilla de maravillas), de modo que se tuvieron que injertar en el modelo subcapas de convergencia que permitieran "tapar" las diferencias. Por último, el comité esperaba originalmente que cada país tuviera una red controlada por el gobierno que usara los protocolos OSI, de manera que no se pensó en la interconexión de redes. Para no hacer el cuento largo, las cosas no salieron como se esperaba.

Lo contrario sucedió con TCP/IP: primero llegaron los protocolos, y el modelo fue en realidad sólo una descripción de los protocolos existentes. No hubo el problema de ajustar los protocolos al modelo; se ajustaban a la perfección. El único problema fue que el *modelo* no se ajustaba a ninguna otra pila de protocolos; en consecuencia, no fue de mucha utilidad para describir otras redes que no fueran del tipo TCP/IP.

Pasando de temas filosóficos a otros más específicos, una diferencia obvia entre los dos modelos es la cantidad de capas: el modelo OSI tiene siete capas y el TCP/IP cuatro. Ambos tienen capas de (inter)red, de transporte y de aplicación, pero las otras capas son diferentes.

Otra diferencia se tiene en el área de la comunicación sin conexión frente a la orientada a la conexión. El modelo OSI apoya la comunicación tanto sin conexión como la orientada a la conexión en la capa de red, pero en la capa de transporte donde es más importante (porque el servicio de transporte es visible a los usuarios) lo hace únicamente con la comunicación orientada a la conexión. El modelo TCP/IP sólo tiene un modo en la capa de red (sin conexión) pero apoya ambos modos en la capa de transporte, con lo que ofrece una alternativa a los usuarios. Esta elección es importante sobre todo para los protocolos simples de petición y respuesta.

1.4.4. Una crítica del modelo y los protocolos OSI

Ni el modelo OSI y sus protocolos ni el modelo TCP/IP y sus protocolos son perfectos. Se puede criticar bastante a ambos, y así se ha hecho. En esta sección y en la siguiente veremos algunas de estas críticas. Empezaremos por OSI y examinaremos TCP/IP más adelante.

Al momento de publicarse la segunda edición de este libro (1989), pareció a los más expertos en este campo que el modelo OSI y sus protocolos iban a conquistar el mundo y a desalojar de su camino todo lo demás. Esto no sucedió. ¿Por qué? Una mirada en retrospectiva a algunas de las lecciones puede ser de utilidad. Estas lecciones se pueden resumir como:

1. Mala sincronización.
2. Mala tecnología.
3. Malas instrumentaciones.
4. Mala política.

Mala sincronización

Veamos primero la razón número uno: la mala sincronización. El momento en el que se establece un estándar es absolutamente crucial para su éxito. David Clark del M.I.T. tiene una teoría de los estándares que llama el *apocalipsis de los dos elefantes,* y que se ilustra en la figura 1-20.

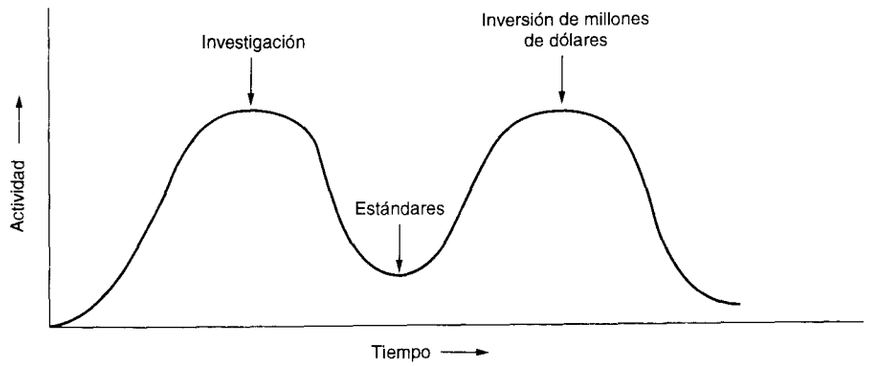


Figura 1-20. El apocalipsis de los dos elefantes.

Esta figura muestra la cantidad de actividad que rodea a un tema nuevo. Cuando se descubre inicialmente el tema, hay un frenesí de actividad de investigación en forma de discusiones, documentos y reuniones. Al cabo de un tiempo, esto disminuye, las corporaciones descubren el tema y llega la ola de inversión de millones de dólares.

Es esencial que se escriban los estándares en el intermedio entre los dos "elefantes". Si se escriben demasiado pronto, antes de que termine la investigación, el tema todavía no se entiende bien, lo que conduce a malos estándares. Si se escriben demasiado tarde, es probable que muchas compañías ya hayan hecho inversiones importantes en diferentes formas de hacer las cosas, de modo que los estándares se ignoran en la práctica. Si el intervalo entre los dos elefantes es muy corto (porque todos tienen prisa por empezar), la gente que desarrolla los estándares puede quedar aplastada.

Parece que ahora los protocolos estándar de OSI han quedado aplastados. Los protocolos competidores de TCP/IP ya se usaban ampliamente en universidades que hacían investigación cuando aparecieron los protocolos de OSI. Antes de que llegara la ola de inversión de millones de dólares, el mercado académico tenía el tamaño suficiente para que los proveedores empezaran a ofrecer con cautela los productos TCP/IP. Cuando el OSI llegó, no quisieron apoyar una segunda pila de protocolos hasta que se les forzó, de modo que no hubo ofertas iniciales. Al estar cada compañía en espera de que otra tomara la iniciativa, ninguna compañía lo hizo y OSI nunca sucedió.

Mala tecnología

La segunda razón por la que OSI nunca prendió es que tanto el modelo como los protocolos son imperfectos. La mayor parte de las explicaciones acerca del modelo de las siete capas da la impresión de que la cantidad y el contenido de las capas finalmente seleccionadas eran el único camino o, al menos, el camino obvio. Esto está lejos de ser verdad. La capa de sesión tiene poco uso en la mayor parte de las aplicaciones, y la capa de presentación casi está vacía. De hecho, la propuesta inglesa a la ISO tenía sólo cinco capas, no siete. En contraste con las capas de sesión y de presentación, las capas de enlace de datos y de red están tan llenas que en trabajos posteriores se dividieron en múltiples subcapas, cada una con funciones distintas.

A pesar de que casi nadie lo admite en público, la verdadera razón de que el modelo OSI tenga siete capas es que en el momento en que se diseñó IBM tenía un protocolo patentado de siete capas llamado **SNA**[MR] (*systems network architecture,* **arquitectura de red de sistemas**). En esa época, IBM dominaba la industria de la computación a tal grado que todo el mundo, incluidas las compañías de teléfonos, las compañías de computadoras de la competencia y hasta los principales gobiernos sentían pánico de que IBM usara su fuerza en el mercado para obligar prácticamente a todos a usar SNA, el cual podría cambiar en el momento que quisiera. Lo que se pretendía con OSI era crear un modelo de referencia y pila de protocolos semejante al de IBM que se pudiera convertir en el estándar mundial y estuviera controlado no por una compañía sino por una organización neutral, la ISO.

El modelo OSI, junto con las correspondientes definiciones y protocolos de servicios, es extraordinariamente completo. Si se apilan, los estándares impresos ocupan una fracción significativa de un metro de papel. También son difíciles de implementar e ineficientes en su operación. En este contexto, viene a la memoria un acertijo de Paul Mockapetris citado en (Rose, 1993):

P: ¿Qué se obtiene cuando se cruza un pandillero con un estándar internacional?
R: Alguien que te hace una oferta que no puedas comprender.

Otro problema con OSI, además de ser incomprensible, es que algunas funciones, como el direccionamiento, el control de flujo y el control de errores reaparecen una y otra vez en cada capa. Por ejemplo, Saltzer *et al.* (1984), han señalado que, para ser eficaz, el control de errores se debe hacer en la capa más alta, de modo que repetirlo una y otra vez en cada una de las capas inferiores con frecuencia es innecesario e ineficiente.

Otra consideración es que la decisión de colocar ciertas funciones en capas particulares no siempre es obvia. El manejo de una terminal virtual (ahora en la capa de aplicación) estuvo en la capa de presentación durante gran parte del desarrollo del estándar. Se pasó a la capa de aplicación porque el comité tuvo problemas para decidir para qué servía la capa de presentación. La seguridad de los datos y el cifrado eran tan polémicos que nadie podía ponerse de acuerdo en cuál capa ponerlos, así que se dejaron fuera. La administración de la red se omitió también del modelo por razones parecidas.

Otra crítica del estándar original es que ignoró por completo los servicios y protocolos carentes de conexión, a pesar de que casi todas las redes de área local trabajan de esa manera. Adiciones subsecuentes (conocidas en el mundo del *software* como parches) corrigieron este problema.

Quizá la crítica más seria es que el modelo está dominado por una mentalidad de comunicaciones. La relación entre la computación y las comunicaciones apenas si se menciona, y muchas de las decisiones tomadas son completamente inapropiadas para la forma en que trabajan las computadoras y el *software*. Como ejemplo, considere las primitivas de OSI, listadas en la figura 1-14. En particular, piense con cuidado en las primitivas y en cómo las podría usar en un lenguaje de programación.

La primitiva CONNECT.*request* es sencilla. Podemos imaginar un procedimiento de biblioteca, *conectar*, que los programas pueden llamar para establecer una conexión. Ahora piense en CONNECT.*indication*. Cuando llega un mensaje, se debe indicar esto al proceso de destino. En efecto, tiene que manejar una interrupción —un concepto difícilmente apropiado para programas escritos en cualquier lenguaje moderno de alto nivel—. Desde luego, en la capa más baja sí ocurre una indicación (interrupción).

Si el programa estuviera esperando una llamada, podría invocar un procedimiento de biblioteca, *recibir*, para bloquearse a sí mismo. Pero si esto es así, ¿por qué no fue la primitiva *recibir* en lugar de *indicación*? *Recibir* se orienta claramente hacia la forma en que trabajan las computadoras, mientras que *indicación* se orienta de forma igualmente clara hacia la forma en que trabajan los teléfonos. Las computadoras son diferentes de los teléfonos. Los teléfonos suenan. Las computadoras no suenan. En resumen, el modelo semántico de un sistema controlado por interrupciones no es una buena idea en lo conceptual y está totalmente en desacuerdo con las ideas modernas de la programación estructurada. Langsford (1984) analiza este problema y otros similares.

Malas instrumentaciones

Dada la enorme complejidad del modelo y los protocolos, no caerá de sorpresa que las implementaciones iniciales fueran enormes, inmanejables y lentas. Todos los que las probaron se arrepintieron. No tuvo que pasar mucho tiempo para que la gente asociara a "OSI" con la

"mala calidad". Mientras los productos mejoraban con el paso del tiempo, la imagen empeoraba.

En contraste, una de las primeras implementaciones de TCP/IP fue parte del UNIX® de Berkeley y era bastante buena (y, además, gratuita). La gente comenzó rápidamente a usarla, lo cual condujo a una gran comunidad de usuarios, lo que condujo a mejoras, lo que condujo a una comunidad todavía mayor. Aquí la espiral era hacia arriba en lugar de hacia abajo.

Mala política

Gracias a la implementación inicial, mucha gente, en especial los académicos, pensaban en TCP/IP como parte de UNIX, y UNIX en la década de 1980 era para los académicos algo así como la tarta de manzana para los estadounidenses comunes.

En cambio, se veía a OSI como una invención de los ministerios europeos de telecomunicaciones, de la Comunidad Europea y, más tarde, del gobierno de Estados Unidos. Esta creencia no era del todo justificada, pero la mera idea de un montón de burócratas tratando de obligar a los pobres investigadores y programadores que estaban en las trincheras desarrollando redes de computadoras reales a que aceptaran un estándar técnicamente inferior no ayudó mucho. Algunos vieron este intento como algo similar a cuando IBM anunció en la década de 1960 que PL/I era el lenguaje del futuro, o cuando DoD corrigió esto más tarde anunciando que en realidad era Ada®.

A pesar de que el modelo y los protocolos de OSI han sido algo menos que un sonado éxito, todavía hay algunas organizaciones interesadas en él, principalmente las PTT europeas que aún tienen un monopolio de las telecomunicaciones. En consecuencia, se ha hecho un esfuerzo débil por actualizar OSI, dando por resultado un modelo revisado que se publicó en 1994. Si desea saber qué se cambió (poco) y qué debió haberse cambiado (y mucho), consulte (Day, 1995).

1.4.5. Una crítica del modelo de referencia TCP/IP

También el modelo y los protocolos de TCP/IP tienen sus problemas. Primero, el modelo no distingue con claridad los conceptos de servicio, interfaz y protocolo. La práctica correcta de la ingeniería de *software* requiere la diferenciación entre las especificaciones y la implementación, algo que OSI hace con mucho cuidado y que TCP/IP no. En consecuencia el modelo de TCP/IP no es una buena guía para diseñar redes nuevas utilizando tecnologías nuevas.

Segundo, el modelo TCP/IP no es general en absoluto y no resulta apropiado para describir cualquier pila de protocolos distinta de TCP/IP. Por ejemplo, tratar de describir SNA mediante el modelo TCP/IP sería casi imposible.

Tercero, la capa de nodo a red en realidad no es una capa en el sentido normal en que se usa el término en el contexto de los protocolos de capas. Es una interfaz (entre la red y las capas de enlace de datos). La distinción entre interfaz y capa es crucial y hay que ser muy minuciosos al respecto.

Cuarto, el modelo TCP/IP no distingue entre la capa física (a la que ni siquiera menciona) y la de enlace de datos. Estas capas son completamente diferentes. La capa física tiene que ver con las características de transmisión del alambre de cobre, la fibra óptica y la comunicación inalámbrica. La tarea de la capa de enlace de datos es delimitar el inicio y el fin de los marcos y transferirlos de un lado a otro con el grado deseado de confiabilidad. Un modelo apropiado debería incluir ambas como capas separadas. El modelo TCP/IP no lo hace.

Por último, aunque los protocolos IP y TCP se pensaron y se implementaron con cuidado, muchos otros protocolos se fueron creando conforme surgía la necesidad, producidos por lo general por un par de estudiantes graduados que trabajaban hasta agotarse. A continuación, las implementaciones de los protocolos se distribuían gratuitamente, lo cual resultó en que se utilizaran con amplitud, atrincherándose y dificultando mucho su reemplazo. Algunos de ellos están ahora en apuros. Por ejemplo, el protocolo de terminal virtual, TELNET, se diseñó para una terminal mecánica Teletype de 10 caracteres por segundo; nada sabe de interfaces gráficas con el usuario ni de ratones. Sin embargo, 25 años después, todavía está en amplio uso.

En síntesis, a pesar de sus problemas, el *modelo* OSI (quitando las capas de sesión y de presentación) ha demostrado ser excepcionalmente útil para estudiar las redes de computadoras. En contraste, los *protocolos* de OSI no se han hecho populares. Lo contrario sucede con TCP/IP: el *modelo* prácticamente es inexistente, pero los *protocolos* se usan mucho. Puesto que los científicos de la computación gustan de tener su pastel y además comérselo, también, en este libro usaremos un modelo OSI modificado pero nos concentraremos principalmente en los protocolos TCP/IP y los que se relacionan con él, así como en los más nuevos, incluidos SMDS, *frame relay*, SONET y ATM. En efecto, en este libro usaremos el modelo híbrido de la figura 1-21 como marco de trabajo.

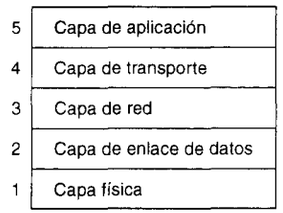


Figura 1-21. El modelo de referencia híbrido que se usa en este libro.

1.5. EJEMPLOS DE REDES

En la actualidad operan en el mundo numerosas redes. Algunas son redes públicas operadas por empresas de comunicaciones o PTT, otras son redes de investigación, otras más son redes cooperativas operadas por sus usuarios y todavía otras son redes comerciales o corporativas. En las siguientes secciones echaremos una mirada a algunas redes actuales e históricas para tener una idea de cómo funcionan (o funcionaban) y en qué difieren unas de otras.

Las redes difieren en su historia, su administración, los recursos que ofrecen, su diseño técnico y sus comunidades de usuarios. La historia y la administración pueden variar desde una red planeada con cuidado por una sola organización con un objetivo bien definido hasta una colección *ad hoc* de máquinas que se han conectado una con otra al paso de los años sin un plan maestro ni una administración central. Los recursos disponibles van desde la comunicación arbitraria entre procesos hasta el correo electrónico, la transferencia de archivos, el ingreso remoto (*login*) y la ejecución remota. Los diseños técnicos pueden diferir en los medios de transmisión empleados, los algoritmos de nomenclatura y ruteo, la cantidad y el contenido de las capas presentes y los protocolos utilizados. Por último, la comunidad de usuarios puede variar desde una sola corporación a todos los computólogos académicos del mundo industrializado.

En las siguientes secciones veremos algunos ejemplos. Éstos son: el popular paquete comercial de LAN, NetWare® de Novell; la Internet mundial (lo que incluye a sus predecesores, ARPANET y NSFNET), y las primeras redes de gigabits.

1.5.1. NetWare de Novell

El sistema de redes más popular en el mundo PC es **NetWare** de **Novell**. Se diseñó para que lo usaran compañías que deseaban cambiar su *mainframe* por una red de PC. En tales sistemas, cada usuario tiene una computadora de escritorio que funciona como cliente. Además, varias PC de alta capacidad operan como servidores para proveer de servicios de archivos, de bases de datos y otros a una colección de clientes. En otras palabras, el NetWare de Novell se basa en el modelo de cliente-servidor.

NetWare usa una pila de protocolos patentada que se ilustra en la figura 1-22 y que se basa en el antiguo Xerox Network System, XNS[MR] pero con varias modificaciones. NetWare de Novell es previo a OSI y no se basa en él. Si acaso, se parece más a TCP/IP que a OSI.

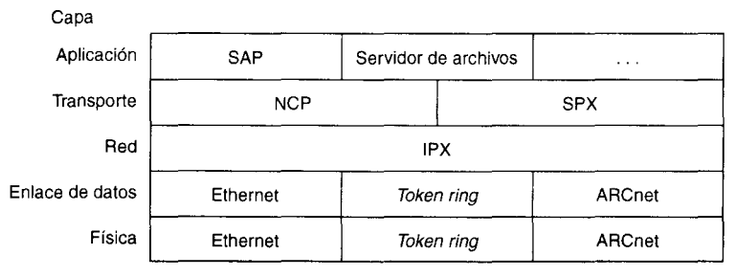


Figura 1-22. El modelo de referencia Novell NetWare.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

Las capas física y de enlace de datos se pueden escoger de entre varios estándares de la industria, lo que incluye Ethernet, el *token ring* de IBM y ARCnet. La capa de red utiliza un

protocolo de interred no confiable, sin conexión, llamado **IPX**. Este protocolo transfiere paquetes del origen al destino en forma transparente, aun si la fuente y el destino se encuentran en redes diferentes. En lo funcional, IPX es similar a IP, excepto que usa direcciones de 10 bytes en lugar de direcciones de 4 bytes. La sabiduría de esta elección se hará evidente en el capítulo 5.

Por encima de IPX está un protocolo de transporte orientado a la conexión que se llama **NCP** (*network core protocol,* **protocolo central de red**). El NCP proporciona otros servicios además del transporte de datos de usuario y en realidad es el corazón de NetWare. También está disponible un segundo protocolo, **SPX**, pero sólo proporciona transporte. Otra opción es TCP. Las aplicaciones pueden seleccionar cualquiera de ellos. Por ejemplo, el sistema de archivos usa NCP y Lotus Notes® usa SPX. Las capas de sesión y de presentación no existen. En la capa de aplicación están presentes varios protocolos de aplicación.

Igual que en TCP/IP, la clave de toda la arquitectura es el paquete de datagrama de interred sobre el cual se construye todo lo demás. En la figura 1-23 se muestra el formato de un paquete IPX. El campo *Suma de verificación* pocas veces se usa, puesto que la capa de enlace subyacente también proporciona una suma de verificación. El campo *Longitud del paquete* indica qué tan grande es el paquete, encabezado más datos. El campo *Control de transporte* cuenta cuántas redes ha atravesado el paquete; cuando se excede un máximo, el paquete se descarta. El campo *Tipo de paquete* sirve para marcar varios paquetes de control. Cada una de las dos direcciones contiene un número de red de 32 bits, un número de máquina de 48 bits (la dirección 802 LAN) y la dirección local (*socket*) de 16 bits en esa máquina. Por último, tenemos los datos que ocupan el resto del paquete, cuyo tamaño máximo está determinado por la capa subyacente.

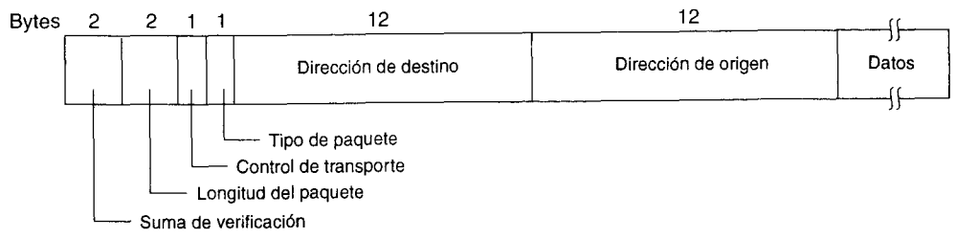


Figura 1-23. Un paquete IPX de Novell NetWare.

Aproximadamente cada minuto, cada servidor difunde un paquete con su dirección que indica cuáles servicios ofrece. Estas difusiones usan el **SAP** (*service advertising protocol,* **protocolo de publicidad del servicio**). Procesos de agentes especiales que se ejecutan en las máquinas enrutadoras que detectan y recopilan los paquetes. Los agentes usan la información contenida en los paquetes para construir bases de datos que indican cuáles servidores se ejecutan y dónde.

Cuando se arranca una máquina cliente, emite una petición en la que pregunta dónde está el servidor más cercano. El agente en la máquina del enrutador local detecta esta solicitud y busca

en su base de datos de servidores cuál es el mejor servidor para su solicitud. A continuación se devuelve al cliente la dirección del mejor servidor a usar. Ahora el cliente puede establecer una conexión NCP con el servidor. Mediante esta conexión, el cliente y el servidor negocian el tamaño máximo de paquete. De aquí en adelante, el cliente puede acceder al sistema de archivos y a otros servicios usando esta conexión. También puede hacer consultas a la base de datos de servidores para buscar otros servidores (más distantes).

1.5.2. La ARPANET

Pasemos ahora de las LAN a las WAN. A mediados de la década de 1960, en la cúspide de la Guerra Fría, el DoD quería una red de comando y control que pudiera sobrevivir a una guerra nuclear. Las redes telefónicas tradicionales de circuito conmutado se consideraban muy vulnerables, puesto que la pérdida de una línea o un conmutador ciertamente terminaría toda conversación que los estuviera usando y podría incluso partir la red. Para resolver este problema, el DoD acudió a su rama de investigación, ARPA o Advanced Research Projects Agency (más tarde DARPA y ahora ARPA de nuevo), es decir, la Agencia (de Defensa, periódicamente) de Proyectos de Investigación Avanzados.

ARPA se creó en respuesta al lanzamiento del Sputnik de la Unión Soviética en 1957 y tuvo la misión de desarrollar tecnologías que pudieran ser útiles a la milicia. ARPA nunca tuvo científicos ni laboratorios, de hecho, no tenía más que una oficina y un presupuesto pequeño (para los estándares del Pentágono). Cumplió con su trabajo al ofrecer financiamiento y contratos a universidades y compañías cuyas ideas le parecían prometedoras.

Varias de las primeras subvenciones se concedieron a las universidades para investigar la idea entonces radical de la conmutación de paquetes, algo que Paul Baran había sugerido en una serie de informes de la RAND Corporation que se publicaron a principios de la década de 1960. Después de discusiones con varios expertos, la ARPA decidió que la red que necesitaba el DoD debía ser una red de paquete conmutado, que consistía en una subred y computadoras *hosts*.

La subred consistiría en minicomputadoras llamadas **IMP** (*interface message processors,* **procesadores interfaz de mensajes**) conectadas por líneas de transmisión. Para lograr alta confiabilidad, cada IMP se conectaría al menos a otras dos. La subred iba a ser una subred de datagrama, de modo que si algunas líneas e IMP resultaban destruidas, los mensajes se podrían reencaminar de forma automática a través de trayectorias alternas.

Cada nodo de la red consistiría en un IMP y una *host* en el mismo cuarto, conectados por un cable corto. Una *host* podría enviar mensajes de hasta 8063 bits a su IMP, que entonces los dividiría en paquetes de 1008 bits a lo sumo y los reenviaría a su destino en forma independiente. Cada paquete se recibía en su totalidad antes de reenviarse, por lo que la subred fue la primera red electrónica de conmutación de paquetes de almacenar y reenviar.

A continuación, la ARPA lanzó un ofrecimiento para construir la subred. Doce compañías licitaron. Después de evaluar todas las propuestas, la ARPA seleccionó a BBN, una firma de consultores de Cambridge, Massachusetts, y en diciembre de 1968 le concedió un contrato para construir la subred y escribir el *software* de la misma. BBN eligió usar como IMP las minicomputadoras DDP-316 de Honeywell, modificadas especialmente, con 12K palabras de

16 bits de memoria central. Los IMP no tenían discos, pues las partes móviles se consideraban no confiables. Los IMP se interconectaron con líneas de 56 kbps rentadas a compañías de teléfonos.

El *software* se dividió en dos partes: subred y *host*. El *software* de la subred consistió en el extremo IMP de la conexión *host*-IMP, el protocolo IMP-IMP y un protocolo de IMP fuente a IMP destino diseñado para mejorar la confiabilidad. El diseño original de la ARPANET se muestra en la figura 1-24.

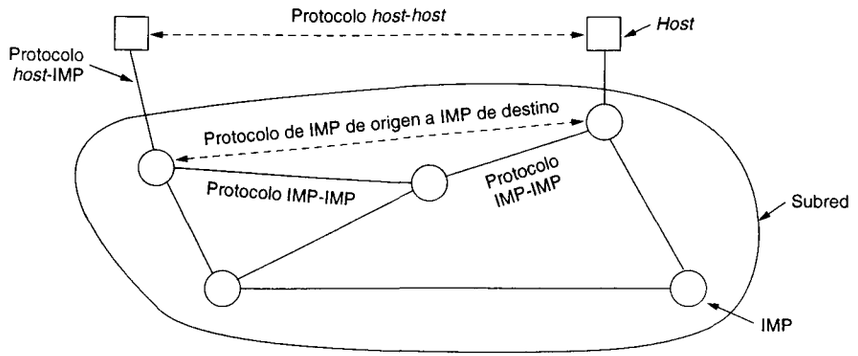


Figura 1-24. El diseño original de la ARPANET.

Fuera de la subred también se necesitaba *software,* a saber, el extremo *host* de la conexión *host*-IMP, el protocolo *host-host* y el *software* de aplicación. Pronto quedó claro que, según BBN, una vez que aceptaba un mensaje por un cable *host*-IMP y lo colocaba en el cable *host*-IMP de su destino, su trabajo estaba hecho.

Para enfrentar el problema del *software* de la *host*, Larry Roberts de ARPA convocó a una reunión de investigadores de redes, la mayoría estudiantes graduados, en Snowbird, Utah, en el verano de 1969. Los estudiantes graduados esperaban que un experto en redes les explicara el diseño de la red y su *software* y luego les asignara el trabajo de escribir una parte del mismo a cada uno de ellos. Quedaron pasmados cuando no hubo experto en redes y tampoco un gran diseño. Tuvieron que descifrar qué hacer por sí mismos.

Sin embargo, de alguna forma, una red experimental entró en funciones en diciembre de 1969 con cuatro nodos en UCLA, UCSB, SRI y la Universidad de Utah. Estas cuatro se eligieron porque todas tenían gran cantidad de contratos con ARPA, y todas tenían computadoras *host* diferentes y completamente incompatibles (sólo para hacerlo más divertido). La red creció rápidamente conforme se fueron entregando e instalando más IMP; pronto abarcó todo Estados Unidos. La figura 1-25 muestra con qué rapidez creció la ARPANET en los primeros tres años.

Más tarde, el *software* de IMP se cambió para permitir que las terminales se conectaran de forma directa a un IMP especial llamado **TIP** (*terminal interface processor,* **procesador de interfaz de terminal**) sin tener que pasar por una *host*. Los cambios subsecuentes incluyeron el tener múltiples *hosts* por cada IMP (para ahorrar dinero), *hosts* que se comunicaban con múltiples IMP (para protegerse de fallas del IMP) y *hosts* e IMP separadas por una gran distancia (para alojar a las *hosts* situadas lejos de la subred).

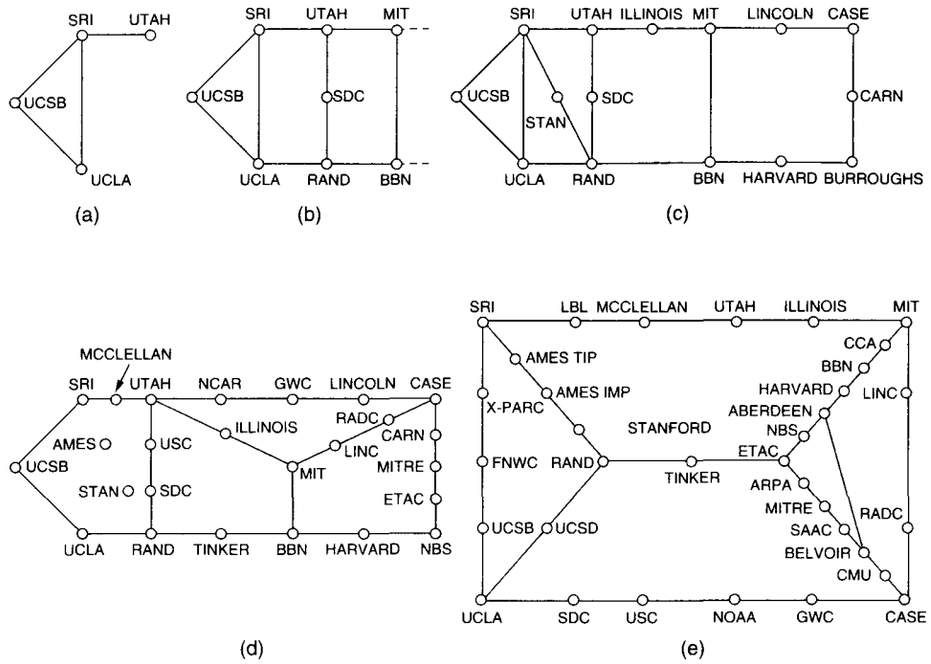


Figura 1-25. Crecimiento de la ARPANET. (a) Dic. 1969. (b) Julio 1970. (c) Marzo 1971. (d) Abril 1972. (e) Sept. 1972.

Además de ayudar al crecimiento de la novel ARPANET, ARPA financió también investigaciones sobre redes de satélites y redes de radio de paquetes móviles. En una demostración famosa, un camión que recorría California usó la red radial de paquetes para enviar mensajes a SRI, que entonces los reenvió por conducto de ARPANET a la Costa Este, desde donde se transmitieron a la University College en Londres mediante una red de satélites. Esto permitió a un investigador en el camión usar una computadora en Londres mientras conducía por California.

Este experimento demostró también que los protocolos de ARPANET existentes no eran apropiados para funcionar en múltiples redes. Esta observación condujo a más investigaciones sobre protocolos, lo que culminó con la invención del modelo y los protocolos TCP/IP (Cerf y Kahn, 1974). TCP/IP se diseñó de manera específica para manejar la comunicación en las interredes, algo que se volvió cada vez más importante al conectarse más y más redes a la ARPANET.

Para fomentar la adopción de estos protocolos nuevos, la ARPA concedió varios contratos a BBN y a la Universidad de California en Berkeley para integrarlos en el UNIX de Berkeley. Los investigadores de Berkeley desarrollaron una interfaz de programa conveniente para la red

(*sockets*) y escribieron muchos programas de aplicación, utilería y administración para facilitar el trabajo con redes.

El momento era ideal. Muchas universidades acababan de adquirir su segunda o tercera computadora VAX y una LAN para conectarlas, pero no tenían *software* de red. Cuando el 4.2BSD hizo su arribo, junto con TCP/IP, los *sockets* y muchas utilerías de red, el paquete completo se adoptó de inmediato. Además, con TCP/IP era fácil que las LAN se conectaran a la ARPANET, y muchas lo hicieron.

Para 1983 la ARPANET gozaba de estabilidad y éxito, con más de 200 IMP y cientos de *hosts*. En este momento, ARPA cedió el manejo de la red a la Agencia de Comunicaciones de la Defensa (DCA), para hacerla funcionar como una red de operaciones. Lo primero que hizo la DCA fue separar la porción militar en una subred independiente, **MILNET** (cerca de 160 IMP, de los cuales 110 estaban en Estados Unidos y 50 en otros países), con pasarelas estrictas entre MILNET y la red de investigación restante.

Durante la década de 1980, se conectaron a la ARPANET redes adicionales, sobre todo LAN. Al aumentar la escala, encontrar *hosts* era algo que se hizo cada vez más costoso, así que se creó el **DNS** (*domain naming system,* **sistema de designación de dominios**) para organizar las máquinas en dominios y establecer correspondencias entre los nombres de las *hosts* y las direcciones de IP. Desde entonces, el DNS se ha convertido en un sistema distribuido y generalizado de bases de datos para almacenar diversa información referente a los nombres. Estudiaremos el DNS con detalle en el capítulo 7.

Para 1990, la ARPANET había sido rebasada por redes más nuevas que ella misma había engendrado, de manera que se clausuró y desmanteló, pero aún vive en los corazones y las mentes de los investigadores de redes en todas partes. En cambio, MILNET continúa operando.

1.5.3. NSFNET

A finales de la década de 1970, la NSF (*National Science Foundation,* **Fundación Nacional de la Ciencia** de Estados Unidos) vio el impacto enorme que había tenido ARPANET en la investigación universitaria al permitir que científicos de todo el país compartieran datos y colaboraran en proyectos de investigación. Sin embargo, para introducirse en la ARPANET, una universidad debía tener un contrato de investigación con el DoD, cosa que muchas no tenían. Esta falta de acceso universal motivó a la NSF a establecer una red virtual, **CSNET**, centrada en una sola máquina en BBN que permitía el uso de líneas de acceso por discado y tenía conexiones con la ARPANET y otras redes. Mediante CSNET, los investigadores académicos podían hacer llamadas y dejar correo electrónico para que otras personas lo recogieran más tarde. Era simple, pero funcionaba.

En 1984, la NSF empezó a diseñar un sucesor de alta velocidad para la ARPANET que se abriría a todos los grupos universitarios de investigación. A fin de tener algo en concreto con lo cual empezar, la NSF decidió construir una red de *backbone* (tronco o columna vertebral) para conectar sus seis centros de supercomputadoras en San Diego, Boulder, Champaign, Pittsburgh, Ithaca y Princeton. A cada supercomputadora se le dio un hermanito que consistía en una

microcomputadora LSI-11 a la que llamaron ***fuzzball***. Las *fuzzballs* se conectaron con líneas rentadas de 56 kbps y formaron la subred, la misma tecnología de *hardware* que usó ARPANET. Sin embargo, la tecnología de *software* era diferente: las *fuzzballs* hablaban TCP/IP desde un principio, convirtiéndose en la primera WAN de TCP/IP.

La NSF financió también algunas redes regionales (finalmente cerca de 20), que se conectaron a la *backbone* para permitir a los usuarios de miles de universidades, laboratorios de investigación, bibliotecas y museos acceder a cualquiera de las supercomputadoras y comunicarse entre sí. La red completa, que incluía la *backbone* y las redes regionales, se llamó **NSFNET**. La NSFNET se conectó a la ARPANET mediante un enlace entre un IMP y una *fuzzball* en el cuarto de máquinas de Carnegie-Mellon. En la figura 1-26 se ilustra la primera *backbone* de NSFNET.

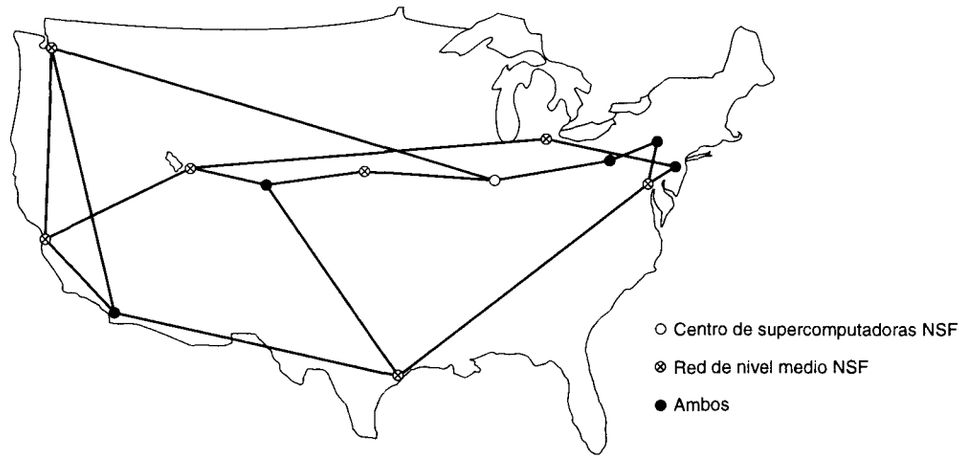


Figura 1-26. La columna vertebral (*backbone*) de NSFNET en 1988.

La NSFNET fue un éxito instantáneo y se sobrecargó desde el primer momento. La NSF comenzó de inmediato a planear a su sucesor y concedió un contrato al consorcio MERIT de Michigan para que lo operara. Se rentaron a la MCI canales de fibra óptica a 448 kbps para establecer la versión 2 de la *backbone*. Como enrutadores se usaron equipos IBM RS6000. También esta *backbone* resultó superada en poco tiempo, y para 1990 se había subido de nivel a la segunda *backbone*, a 1.5 Mbps.

Al continuar el crecimiento, la NSF se dio cuenta que el gobierno no podía seguir financiando eternamente el uso de redes. Más aún, las organizaciones comerciales querían unirse pero los estatutos de la NSF les prohibían usar redes pagadas por la propia Fundación. En consecuencia, la NSF animó a MERIT, MCI e IBM a formar una corporación no lucrativa, **ANS** (**Advanced Network and Services**) como un paso hacia la comercialización. En 1990, ANS tomó la NSFNET y subió el nivel de los enlaces de 1.5 Mbps a 45 Mbps para formar **ANSNET**.

En diciembre de 1991, el Congreso de Estados Unidos aprobó un documento que autorizaba a la **NREN**, la **Red Nacional Educativa y de Investigación**, como sucesora de investigación de la NSFNET, sólo que operando a velocidades de gigabits. La meta era una red nacional que funcionara a 3 Gbps antes del milenio. Esta red iba a actuar como prototipo para la tan discutida supercarretera de la información.

Para 1995, la *backbone* NSFNET ya no era necesaria para interconectar las redes regionales de la NSF porque un gran número de compañías operaba con redes comerciales IP. Cuando se vendió la ANSNET a America Online en 1995, las redes regionales de la NSF tuvieron que comprar el servicio comercial de IP para interconectarse.

Para facilitar la transición y asegurar que cada red regional se pudiera comunicar con todas las demás redes regionales, la NSF otorgó contratos a cuatro operadores de redes diferentes para establecer un **NAP** (*network access point,* **punto de acceso a la red**). Estos operadores eran PacBell (San Francisco), Ameritech (Chicago), MFS (Washington, D.C.) y Sprint (Nueva York, donde, para propósitos de NAP, Pennsauken, N.J. cuenta como la ciudad de Nueva York). Cada operador de red que quisiera proporcionar el servicio de *backbone* a las redes regionales de la NSF debía conectarse a todos los NAP. Este arreglo significaba que un paquete que se originara en cualquier red regional podía elegir entre varias portadoras de *backbone* para ir desde su NAP al NAP de destino. En consecuencia, las portadoras de *backbone* se vieron forzadas a competir para que las redes regionales las escogieran con base en el servicio y el precio, que desde luego era lo que se buscaba. Además de los NAP de la NSF, ya se habían creado varios NAP del gobierno (por ejemplo, FIX-E, FIX-W, MAE-East y MAE-West) y comerciales (por ejemplo, CIX), así que el concepto de una *backbone* solitaria por omisión fue sustituido por una infraestructura competitiva regida por lo comercial.

Otros países y regiones están construyendo redes comparables a NSFNET. En Europa, por ejemplo, EBONE es una *backbone* IP para organizaciones de investigación y EuropaNET es una red más orientada hacia lo comercial. Ambas conectan gran cantidad de ciudades en Europa con líneas de 2 Mbps y se están instalando mejoras de nivel a 34 Mbps. Cada país de Europa tiene una o más redes nacionales, lo cual es comparable en forma aproximada a las redes regionales de la NSF.

1.5.4. La Internet

La cantidad de redes, máquinas y usuarios conectados a la ARPANET creció con rapidez después de que TCP/IP se convirtió en el único protocolo oficial el 1° de enero de 1983. Cuando se interconectaron la NSFNET y la ARPANET, el crecimiento se hizo exponencial; se unieron muchas redes regionales y se hicieron conexiones con redes en Canadá, Europa y el Pacífico.

En algún momento de mediados de la década de 1980, la gente empezó a ver la aglomeración de redes como una interred, y más tarde como la Internet, aunque no hubo dedicatoria oficial con algún político rompiendo una botella de champagne sobre una *fuzzball*.

El crecimiento continuó en forma exponencial, y para 1990 la Internet había crecido a 3000 redes y 200,000 computadoras. En 1992 se adhirió la *host* número un millón. Para 1995, había múltiples redes de *backbones*, cientos de redes de nivel medio (esto es, regionales), decenas de

miles de LAN, millones de *hosts* y decenas de millones de usuarios. El tamaño se duplica aproximadamente cada año (Paxson, 1994).

Gran parte del crecimiento se debe a la conexión de redes existentes a la Internet. En el pasado, éstas incluyeron: SPAN, la red de física del espacio de la NASA, HEPNET, una red de física de alto nivel, BITNET, la red de *mainframes* de IBM, EARN, una red académica europea que ahora se usa ampliamente en Europa del Este, y muchas otras. Se utilizan numerosos enlaces trasatlánticos que operan desde 64 kbps hasta 2 Mbps.

El aglutinante de Internet es el modelo de referencia TCP/IP y la pila de protocolos de TCP/IP. El TCP/IP hace posible el servicio universal y se puede comparar con el sistema de teléfonos o la adopción del ancho de vía universal para los ferrocarriles en el siglo XIX.

¿Qué significa en realidad estar en Internet? Nuestra definición es que una máquina está en Internet si opera con la pila de protocolos de TCP/IP, tiene una dirección de IP y es capaz de enviar paquetes de IP a todas las demás máquinas de Internet. La mera capacidad de enviar y recibir correo electrónico no es suficiente, pues el correo electrónico se distribuye a muchas redes fuera de Internet. Sin embargo, el asunto pierde claridad en cierta forma por el hecho de que muchas computadoras personales tienen la capacidad de llamar a un proveedor de servicios de Internet mediante un módem, recibir la asignación de una dirección de IP temporal y enviar paquetes IP a otras *hosts* de Internet. Tiene sentido considerar que tales máquinas están en Internet mientras están conectadas al enrutador del proveedor de servicios.

Con el crecimiento exponencial, la antigua manera informal de operar la Internet ya no funciona. En enero de 1992 se integró la **Sociedad Internet** para promover el uso de Internet y quizá en algún momento hacerse cargo de su gestión.

Tradicionalmente, Internet ha tenido cuatro aplicaciones principales, que son las siguientes:

1. **Correo electrónico**. La capacidad de redactar, enviar y recibir correo electrónico ha estado disponible desde los primeros días de la ARPANET y es enormemente popular. Mucha gente recibe docenas de mensajes al día y lo considera su forma primaria de interactuar con el mundo externo, dejando muy atrás al teléfono y al correo lento. En estos días, los programas de correo electrónico están disponibles virtualmente en todo tipo de computadoras.

2. **Noticias**. Los grupos de noticias son foros especializados en los que usuarios con un interés común pueden intercambiar mensajes. Existen miles de grupos de noticias, con temas técnicos y no técnicos, lo que incluye computadoras, ciencia, recreación y política. Cada grupo de noticias tiene su propia etiqueta, estilo y costumbres, y ¡ay de cualquiera que los viole!

3. **Sesión remota**. Mediante el uso de Telnet, Rlogin u otros programas, los usuarios en cualquier lugar de la Internet pueden ingresar en cualquier otra máquina en la que tengan una cuenta autorizada.

4. **Transferencia de archivos**. Con el programa FTP, es posible copiar archivos de una máquina en Internet a otra. De esta manera está disponible una vasta cantidad de artículos, bases de datos y otra información.

Hasta casi fines de la década de 1990, la Internet se poblaba en gran medida de investigadores académicos, del gobierno y de la industria. Una aplicación nueva, la **WWW** (*world wide web,* **red mundial**) cambió todo eso y atrajo millones de nuevos usuarios no académicos a la red. Esta aplicación, inventada por el físico del CERN Tim Berners-Lee, no cambió ninguno de los recursos subyacentes pero los hizo más fáciles de usar. Junto con el visor de Mosaic, escrito en el Centro Nacional para Aplicaciones de Supercomputadoras, la WWW hizo posible que una localidad estableciera varias páginas de información conteniendo texto, dibujos, sonido y hasta vídeo con enlaces intercalados a otras páginas. Al accionar el ratón en un enlace, el usuario se ve transportado de inmediato a la página a la que apunta ese enlace. Por ejemplo, muchas compañías tienen una página local con entradas que apuntan a otras páginas que ofrecen información de productos, listas de precios, ventas, apoyo técnico, comunicación con los empleados, información para accionistas y muchas cosas más.

En un tiempo muy corto han aparecido muchos otros tipos de páginas, incluso mapas, tablas del mercado de valores, catálogos de tarjetas de bibliotecas, programas de radio grabados y hasta una página que apunta al texto completo de muchos libros cuyos derechos de autor han expirado (Mark Twain, Charles Dickens, etc.). Mucha gente tiene también páginas personales (páginas locales).

El primer año en que salió a la luz Mosaic, la cantidad de servidores de WWW creció de 100 a 7000. Sin duda el crecimiento en los años por venir continuará siendo enorme, y es probable que sea la fuerza que impulse la tecnología y el uso de Internet hacia el próximo milenio.

Se han escrito muchos libros respecto a la Internet y sus protocolos. Para más información, véase (Black, 1995; Carl-Mitchell y Quarterman, 1993; Comer, 1995, y Santifaller, 1994).

1.5.5. Plataformas de pruebas de gigabits

Las redes de *backbones* de Internet operan a velocidades de megabits, de modo que para la gente que quiere forzar los límites de la tecnología, el próximo paso es el uso de redes de gigabits. Con cada incremento en el ancho de banda de la red, se hacen posibles nuevas aplicaciones, y las redes de gigabits no son la excepción. En esta sección hablaremos primero un poco sobre las aplicaciones de gigabits, mencionaremos dos de ellas y a continuación listaremos algunas plataformas de pruebas de gigabits que se han construido.

Las redes de gigabits proporcionan mayor ancho de banda que las redes de megabits, pero no siempre reducen mucho los retardos. Por ejemplo, enviar un paquete de 1 kB de Nueva York a San Francisco a 1 Mbps toma 1 mseg para sacar los bits y 20 mseg para el retardo transcontinental, dando un total de 21 mseg. Una red de 1 Gbps puede reducir esto a 20.001 mseg. Aunque los bits van saliendo más rápido, el retardo transcontinental no cambia, puesto que la velocidad de la luz en la fibra óptica (o en el alambre de cobre) es cercana a los 200,000 km/seg y es independiente de la velocidad de transmisión de los datos. Así, para aplicaciones de área amplia en las que el retardo es crítico, ir a velocidades más altas no puede ayudar mucho. Por fortuna, para algunas aplicaciones el ancho de banda es lo que cuenta, y éstas son las aplicaciones en las que las redes de gigabits implicarán una gran diferencia.

Una aplicación es la telemedicina. Mucha gente piensa que una forma de reducir los costos médicos es reintroducir los médicos familiares y las clínicas familiares en gran escala, de modo que todo mundo tenga acceso cómodo a atención médica de primera línea. Cuando ocurra un problema médico serio, el médico familiar podrá ordenar análisis e imágenes de laboratorio, tales como rayos X, exploraciones CAT y de MRI. Después, los resultados e imágenes se podrían enviar en forma electrónica a un especialista, quien a continuación daría el diagnóstico.

Por lo general, a los doctores no les gusta hacer diagnósticos a partir de imágenes de computadora, a menos que la calidad de la imagen transmitida sea tan buena como la del original. Este requerimiento significa que las imágenes probablemente necesitarán 4K × 4K pixeles, con 8 bits por pixel (imágenes en blanco y negro) o 24 bits por pixel (imágenes en color). Puesto que muchos exámenes requieren hasta 100 imágenes (por ejemplo, diferentes secciones transversales del órgano en cuestión), una sola serie para un paciente puede generar 40 gigabits. Las imágenes en movimiento (por ejemplo, un corazón que late) generan todavía más datos. La compresión puede ayudar un poco, pero los doctores están recelosos de ella, pues los algoritmos más eficientes reducen la calidad de la imagen. Además, todas las imágenes se deben almacenar durante años, pero podría ser necesario recuperarlas de inmediato en el caso de una emergencia médica. Los hospitales no quieren convertirse en centros de cómputo, de manera que es esencial el almacenamiento fuera de la localidad en combinación con la recuperación electrónica de alto ancho de banda.

Otra aplicación de gigabits es la reunión virtual. Cada sala de juntas contiene una cámara esférica y una o más personas. Las corrientes de bits de cada cámara se combinan en forma electrónica para dar la ilusión de que todos los participantes están en la misma sala. Cada persona ve esta imagen con anteojos de realidad virtual. De esta forma, las reuniones se pueden efectuar sin viajes pero, una vez más, las velocidades de transmisión de datos requeridas son enormes.

A partir de 1989, la ARPA y la NSF acordaron en forma conjunta financiar varias plataformas de pruebas de gigabits en universidades y en la industria y más tarde como parte del proyecto NREN. En algunas de ellas, la velocidad de los datos en cada dirección era de 622 Mbps, así que solamente contando los datos en ambas direcciones se lograba un gigabit. A veces a esta clase de gigabit se le llama "gigabit del gobierno". (Algunos cínicos lo llaman gigabit después de impuestos.) A continuación mencionaremos en forma breve los primeros cinco proyectos que ya han cumplido con su misión y se han clausurado, pero merecen cierto crédito como pioneros, de la misma manera en que lo merece la ARPANET.

1. **Aurora** era una plataforma de pruebas que enlazaba a cuatro localidades del noreste de Estados Unidos: el M.I.T., la Universidad de Pennsylvania, el laboratorio T. J. Watson de IBM y Bellcore (Morristown, N. J.) a 622 Mbps mediante el uso de fibra óptica proporcionada por MCI, Bell Atlantic y NYNEX. Aurora se diseñó principalmente para ayudar a depurar el conmutador Sunshine de Bellcore y el conmutador (patentado) plaNET de IBM mediante el uso de redes paralelas. Las consideraciones de investigación incluyeron la tecnología de conmutación, los protocolos de gigabits, el ruteo, el control de red, la memoria virtual distribuida y la colaboración mediante videoconferencias. Para mayor información, véase (Clark *et al.*, 1993).

2. **Blanca** originalmente fue un proyecto de investigación llamado XUNET en el que intervinieron los Laboratorios Bell de AT&T, Berkeley y la Universidad de Wisconsin. En 1990 se añadieron algunas localidades nuevas (LBL, Cray Research y la Universidad de Illinois) y se logró financiamiento de NSF/ARPA. Una parte operaba a 622 Mbps, pero otras lo hacían a velocidades más bajas. Blanca fue la única plataforma de pruebas de cobertura nacional; el resto fueron regionales. En consecuencia, gran parte de la investigación se ocupaba de los efectos del retardo de la velocidad de la luz. Aquí el interés estaba en los protocolos, en especial los protocolos de control de red, las interfaces de *hosts* y las aplicaciones de gigabits, como las imágenes médicas, los modelos meteorológicos y la radioastronomía. Para más información, véase (Catlett, 1992, y Fraser, 1993).

3. **CASA** estaba dirigido a la investigación en aplicaciones de supercomputadoras, sobre todo aquellas en las que una parte del problema se ejecutaba mejor en un tipo de supercomputadora (por ejemplo, una Cray vectorial), y otra parte se ejecutaba mejor en una clase diferente de supercomputadora (por ejemplo, una paralela). Las aplicaciones que se investigaron incluyeron aplicaciones de geología (mediante el análisis de las imágenes del Landsat), modelado del clima, y comprensión de reacciones químicas. CASA operaba en California y Nuevo México y conectaba a Los Álamos, Cal Tech, JPL y el Centro de Supercomputadoras de San Diego.

4. **Nectar** difirió de las tres plataformas mencionadas anteriormente en que fue una MAN experimental de gigabits que operaba desde la CMU al Centro de Supercomputadoras de Pittsburgh. Los diseñadores estaban interesados en aplicaciones relacionadas con la diagramación de procesos químicos y la investigación de operaciones, así como las herramientas para depurarlos.

5. **VISTAnet** fue una pequeña plataforma de pruebas de gigabits en Research Triangle Park, Carolina del Norte, que conectaba a la Universidad de Carolina del Norte, la Universidad Estatal de Carolina del Norte y MCNC. Aquí el interés estaba en un prototipo para una red pública conmutada de gigabits en la que los conmutadores tenían cientos de líneas de gigabits, lo que significaba que los conmutadores tenían que ser capaces de procesar terabits/seg. La investigación científica se centró en el uso de imágenes en tercera dimensión para planear la terapia con radiaciones de pacientes de cáncer, en la que el oncólogo fuera capaz de variar los parámetros del haz y ver en forma instantánea las dosis de radiación que llegaban al tumor y a los tejidos circundantes (Ransom, 1992).

1.6. EJEMPLOS DE SERVICIOS DE COMUNICACIÓN DE DATOS

Las compañías de teléfonos y de otro tipo ya empezaron a ofrecer servicios de red a cualquier organización que desee suscribirse. La subred es propiedad del operador de la red y proporciona el servicio de comunicación a las *hosts* y terminales del cliente. Tal sistema se llama **red pública**; es análogo al sistema telefónico público y con frecuencia es parte de él. En la figura 1-4 vimos ya en forma breve un servicio nuevo, DQDB. En las siguientes secciones estudiaremos otros cuatro ejemplos de servicios: SMDS, X.25, relevo de marcos (*frame relay*) y la ISDN de banda ancha.

1.6.1. SMDS — Servicio de datos conmutado de multimegabits

El primer servicio que veremos, **SMDS** (*switched multimegabit data service,* **servicio de datos conmutado de multimegabits**) se diseñó para conectar entre sí múltiples LAN, en muchos casos en las sucursales y en las fábricas de una sola compañía. SMDS fue diseñado por Bellcore en la década de 1980 y fue puesto en funciones a principios de la década de 1990 por portadoras regionales y algunas de larga distancia. La meta era producir un servicio de datos de alta velocidad e inaugurarlo con un mínimo de escándalo. SMDS fue el primer servicio conmutado de banda ancha (esto es, de alta velocidad) que se ofreció al público.

Como ejemplo de situación en la que es útil el SMDS, consideremos una compañía con cuatro oficinas en cuatro ciudades diferentes, cada una con su propia LAN. A la compañía le gustaría conectar todas las LAN, de modo que los paquetes puedan ir de una LAN a otra. Una solución sería rentar seis líneas de alta velocidad y conectar por completo las LAN, según se muestra en la figura 1-27(a). Ciertamente, tal solución es posible pero cara.

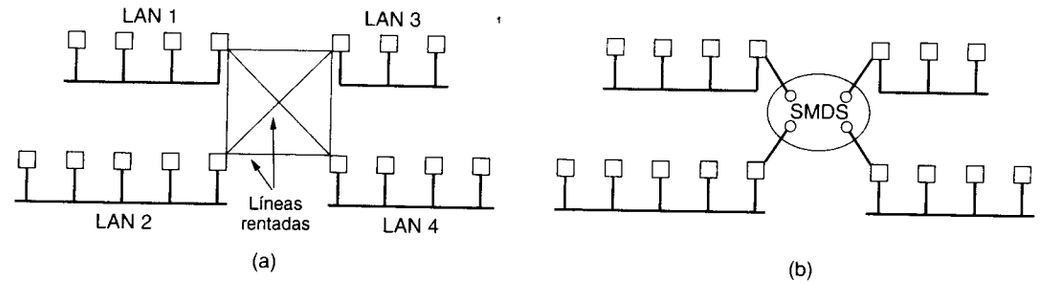


Figura 1-27. (a) Cuatro LAN interconectadas con líneas rentadas. (b) Interconexión mediante SMDS.

Una solución alterna es usar SMDS, como se muestra en la figura 1-27(b). La red SMDS actúa como un *backbone* de LAN de alta velocidad que permite que fluyan los paquetes de una LAN a cualquier otra. Entre las LAN de las oficinas del cliente y la red SMDS en las oficinas de la compañía de teléfonos está una línea de acceso (corta) rentada a la compañía de teléfonos. Por lo general, esta línea es una MAN y usa DQDB, pero también podría haber otras opciones.

Mientras que la mayoría de los servicios de las compañías de teléfonos se diseñan para tráfico continuo, SMDS está diseñado para manejar tráfico en ráfagas. En otras palabras, muy de vez en cuando se tiene que transportar con rapidez un paquete de una LAN a otra, pero gran parte del tiempo no hay tráfico de LAN a LAN. La solución de la línea rentada de la figura 1-27(a) tiene el problema de las altas cuentas mensuales; una vez instaladas, el cliente debe pagar las

líneas, se usen o no en forma continua. Para tráfico intermitente, las líneas rentadas son una solución cara, y SMDS tiene un precio muy competitivo. Con n LAN, una red de líneas rentadas completa requiere rentar $n(n-1)/2$ líneas que podrían ser largas (es decir, caras), mientras que SMDS requiere rentar solamente n líneas cortas de acceso al enrutador de SMDS más cercano.

Puesto que el objetivo de SMDS es transportar tráfico de LAN a LAN, debe tener la suficiente rapidez para efectuar el trabajo. La velocidad estándar es de 45 Mbps, aunque a veces se puede optar por velocidades más bajas. Las MAN también pueden operar a 45 Mbps pero no son conmutadas; esto es, para conectar cuatro LAN mediante una MAN, la compañía de teléfonos tendría que operar un solo cable de la LAN 1 a la LAN 2 a la LAN 3 a la LAN 4, lo que sólo es posible si están en la misma ciudad. Con SMDS, cada LAN se conecta con el conmutador de una compañía de teléfonos que encamina paquetes por conducto de la red SMDS según se necesite para alcanzar el destino, posiblemente atravesando múltiples conmutadores en el proceso.

El servicio básico de SMDS es un simple servicio de entrega de paquetes sin conexión. El formato del paquete se muestra en la figura 1-28 y tiene tres campos: el destino (a dónde va el paquete), la fuente (quién lo envía) y un campo de longitud variable que es la carga útil de hasta 9188 bytes de datos de usuario. En la LAN que envía, la máquina conectada a la línea de acceso pone el paquete en dicha línea, y SMDS hace su mejor esfuerzo por entregarlo en el destino correcto. No se da garantía.

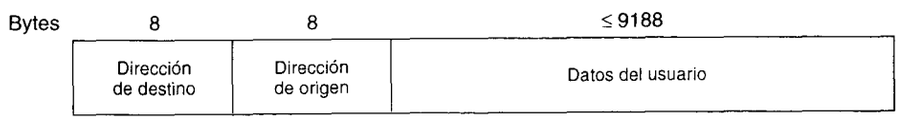


Figura 1-28. Formato del paquete SMDS.

Las direcciones de origen y de destino consisten en un código de 4 bits seguido de un número telefónico de hasta 15 dígitos decimales. Cada dígito se codifica en un campo de 4 bits. Los números telefónicos contienen el código del país, el código de área y el número de suscriptor, de modo que el servicio podría llegar a ofrecerse a nivel internacional. Se pensó que tener números telefónicos decimales como direcciones de red haría que el nuevo servicio pareciera familiar a los usuarios recelosos.

Cuando un paquete llega a la red SMDS, el primer enrutador verifica que la dirección de origen corresponda a la línea entrante, para prevenir fraudes de facturación. Si la dirección es incorrecta el paquete simplemente se descarta; si es correcta, el paquete se envía hacia su destino.

La difusión es una característica útil de SMDS. El cliente puede especificar una lista de números telefónicos de SMDS y pedir que se asigne un número especial a la lista completa. Cualquier paquete que se envíe a ese número se entregará a todos los miembros de la lista. La Asociación Nacional de Comerciantes en Valores de Estados Unidos usa esta característica del servicio SMDS de MCI para difundir los precios de acciones nuevos a todos sus 5000 miembros.

Una función de usuario adicional es la preselección de direcciones de los paquetes tanto salientes como entrantes. Con la preselección saliente, el cliente puede dar una lista de números de teléfono y especificar que no se envíen paquetes a ninguna otra dirección. Con preselección entrante, sólo se aceptarán los paquetes de ciertos números de teléfono preespecificados. Cuando se habilitan ambas funciones, el usuario puede construir efectivamente una red privada sin conexiones SMDS al mundo exterior. Para las compañías con datos confidenciales, esta función es muy valiosa.

La carga útil puede contener cualquier secuencia de bytes que desee el usuario, hasta 9188 bytes; SMDS no la ve. La carga puede contener un paquete Ethernet, un paquete *token ring* de IBM, un paquete IP o cualquier otro. Cualquier cosa presente en el campo de carga útil se transferirá sin modificaciones de la LAN de origen a la de destino.

SMDS maneja el tráfico por ráfagas de la manera siguiente. El enrutador conectado a cada línea de acceso contiene un contador que se incrementa a velocidad constante, digamos una vez cada 10 mseg. Cuando llega un paquete al enrutador, se verifica si el contador es mayor que la longitud del paquete en bytes. Si lo es, el paquete se envía sin retardo y el contador disminuye en un número igual a la longitud del paquete. Si la longitud del paquete es mayor que el contador, el paquete se descarta.

En efecto, con un pulso cada 10 μseg, el usuario puede enviar a una velocidad *promedio* de 100,000 bytes/seg, pero la velocidad de ráfaga puede ser mucho mayor. Por ejemplo, si la línea ha estado desocupada durante 10 mseg, el contador estará en 1000 y se permitirá al usuario que envíe una ráfaga de 1 kB a la velocidad máxima de 45 Mbps, de modo que se transmitirá en aproximadamente 180 μseg. Con una línea rentada de 100,000 bytes/seg, el mismo kilobyte tardaría 10 mseg. Así, SMDS ofrece retardos cortos para ráfagas de datos independientes muy espaciadas, mientras que la velocidad promedio permanece por debajo del valor acordado. Este mecanismo pro-porciona respuesta rápida cuando se necesita pero evita que los usuarios utilicen más ancho de banda del que han acordado pagar.

1.6.2. Redes X.25

Muchas redes públicas antiguas, en especial fuera de Estados Unidos, siguen un estándar llamado **X.25** que la CCITT desarrolló durante la década de 1970 para proveer una interfaz entre las redes públicas de conmutación de paquetes y sus clientes.

El protocolo de la capa física, llamado **X.21**, especifica la interfaz física, eléctrica y de procedimientos entre el *host* y la red. En realidad, muy pocas redes públicas manejan este estándar, pues requiere señalamiento digital en lugar de analógico en las líneas telefónicas. Como medida provisional, se definió una interfaz analógica similar al estándar RS-232, tan conocido.

El estándar de la capa de enlace de datos tiene algunas variaciones (ligeramente incompatibles), todas las cuales se diseñaron para manejar los errores de transmisión en la línea telefónica entre el equipo del usuario (*host* o terminal) y la red pública (enrutador).

El protocolo de la capa de red se ocupa de la asignación de direcciones, el control de flujo, la confirmación de entrega, las interrupciones y otras consideraciones relacionadas. Básicamente, este protocolo permite al usuario establecer circuitos virtuales y después enviar paquetes de

hasta 128 bytes a través de ellos. Estos paquetes se entregan en forma confiable y en orden. La mayor parte de las redes X.25 trabajan a velocidades de hasta 64 kbps, lo cual las hace obsoletas para muchos propósitos. No obstante, su uso aún es extenso, por lo que conviene que los lectores sepan de su existencia.

X.25 está orientado a la conexión y trabaja con circuitos virtuales tanto conmutados como permanentes. Un **circuito virtual conmutado** se crea cuando una computadora envía un paquete a la red y pide que se haga una llamada a una computadora remota. Una vez establecida la conexión, los paquetes se pueden enviar por ella y siempre llegarán en orden. X.25 proporciona control de flujo para asegurar que un emisor rápido no pueda abrumar a un receptor lento u ocupado.

Un **circuito virtual permanente** se usa de la misma forma que uno conmutado pero se establece previamente por un acuerdo entre el cliente y la portadora; siempre está presente y no se requiere una llamada que lo establezca para poder usarlo. Un circuito de este tipo es semejante a una línea rentada.

Puesto que el mundo todavía está lleno de terminales que no hablan X.25, se definió otro grupo de normas que describen cómo una terminal ordinaria (no inteligente) se comunica con una red pública X.25. En efecto, el usuario o el operador de la red instala una "caja negra" a la que se pueden conectar estas terminales. La caja negra se llama **PAD** (*packet assembler disassembler,* **ensamblador-desensamblador de paquetes**) y su función se describe en un documento denominado **X.3**. Se definió un protocolo estándar entre la terminal y el PAD, el **X.28**, y existe otro protocolo estándar entre el PAD y la red, el **X.29**. Estas tres recomendaciones juntas se conocen como **triple X**.

1.6.3. *Frame relay*

El ***frame relay*** (retransmisión de marco) es un servicio para personas que quieren una forma lo más austera posible, orientada a la conexión, para mover bits de *A* a *B* a una velocidad razonable y bajo costo (Smith, 1993). Su existencia se debe a cambios en la tecnología en las últimas dos décadas. Hace 20 años, la comunicación a través de líneas telefónicas era lenta, analógica y no confiable, y las computadoras eran lentas y caras. En consecuencia, se requirieron protocolos complejos para enmascarar los errores, pero las computadoras de los usuarios eran demasiado caras para ponerlas a hacer este trabajo.

La situación ha cambiado en forma radical. Ahora, las líneas telefónicas rentadas son rápidas, digitales y confiables, y las computadoras son rápidas y baratas. Esto sugiere el uso de protocolos simples, con la mayor parte del trabajo realizada por las computadoras de los usuarios en vez de la red. Éste es el ambiente para el que está pensado el *frame relay*.

Se puede pensar en el *frame relay* como una línea virtual rentada. El cliente renta un circuito virtual permanente entre dos puntos y entonces puede enviar marcos o *frames* (es decir, paquetes) de hasta 1600 bytes entre ellos. También es posible rentar circuitos virtuales permanentes entre un lugar determinado y muchas otras localidades, de modo que cada marco lleve un número de 10 bits que le diga cuál circuito virtual usar.

La diferencia entre una línea rentada real y una virtual es que, con una real, el usuario puede enviar tráfico durante todo el día a máxima velocidad. Con una línea virtual se pueden enviar ráfagas de datos a toda velocidad, pero el uso promedio a largo plazo deberá ser inferior a un

nivel predeterminado. A cambio, la portadora cobra mucho menos por una línea virtual que por una física. Además de competir con las líneas rentadas, el *frame relay* también compite con los circuitos virtuales permanentes de X.25, excepto que opera a altas velocidades, usualmente a 1.5 Mbps, y ofrece menos funciones.

El *frame relay* proporciona un servicio mínimo que básicamente es una forma de determinar el inicio y el fin de cada marco y de detectar errores de transmisión. Si se recibe un marco defectuoso, el *frame relay* simplemente lo descarta. Corresponde al usuario descubrir que se perdió un bloque y emprender la acción necesaria para recuperarlo. A diferencia de X.25, *frame relay* no proporciona acuses de recibo ni control de flujo normal. Sin embargo, tiene un bit en el encabezado que un extremo de la conexión puede encender para indicar al otro que hay problemas. El uso de este bit es opción de los usuarios.

1.6.4. ISDN de banda ancha y ATM

Aun si los servicios antes mencionados llegaran a ser populares, las compañías telefónicas enfrentan todavía un problema mucho más fundamental: las redes múltiples. El POTS o *plain old telephone service* (el antiguo servicio telefónico ordinario) y Telex utilizan la red antigua de circuitos conmutados. Todos los nuevos servicios de datos, como SMDS y *frame relay*, emplean sus propias redes de conmutación de paquetes. DQDB es diferente, y la red interna de administración de llamadas de la compañía de teléfonos (SSN 7) es otra red adicional. Mantener todas estas redes individuales es un dolor de cabeza mayúsculo, y existe otra red, la de televisión por cable, que las compañías de teléfonos no controlan pero que les gustaría controlar.

La solución que se percibe es inventar una nueva red única para el futuro que reemplazará a todo el sistema telefónico y a todas las redes especializadas por una sola red integrada para todos los tipos de transferencia de información. Esta nueva red tendrá una velocidad de transmisión muy elevada en comparación con todos servicios y redes existentes y hará posible ofrecer una gran variedad de servicios nuevos. Este proyecto no es pequeño y ciertamente no va a suceder de la noche a la mañana, pero ya está en camino.

El nuevo servicio de área amplia se llama **B-ISDN** (*broadband integrated services digital network,* **red digital de servicios integrados de banda ancha**); ofrecerá vídeo sobre pedido, televisión en vivo de muchas fuentes, correo electrónico en multimedia de movimiento total, música con calidad de disco compacto, interconexión de LAN, transporte de alta velocidad para datos científicos e industriales y muchos otros servicios en los que ni siquiera se ha pensado, todo por la línea telefónica.

La tecnología subyacente que hace posible la B-ISDN se llama **ATM** (*asynchronous transfer mode,* **modo de transferencia asíncrono**) debido a que no es síncrono (atado a un reloj maestro), como lo está la mayor parte de las líneas telefónicas de larga distancia. Cabe señalar que el acrónimo ATM nada tiene que ver aquí con los cajeros automáticos o *automated teller machines* que ofrecen muchos bancos (aunque un cajero automático puede usar una red ATM para hablar con su banco).

Se ha trabajado mucho en ATM y en el sistema B-ISDN que lo usa, aunque todavía hay más por hacer. Para más información sobre este tema, véase (Fischer *et al.*, 1994; Gasman, 1994; Goralski, 1995; Kim *et al.*, 1994; Kyas, 1995; McDysan y Spohn, 1995, y Stallings, 1995a).

La idea en que se basa la ATM consiste en transmitir toda la información en paquetes pequeños de tamaño fijo llamados **células**. Las celdas tienen una longitud de 53 bytes, de los cuales cinco son de encabezado y 48 de carga útil, según se muestra en la figura 1-29. ATM es tanto una tecnología (oculta a los usuarios) como un servicio potencial (visible a los usuarios). A veces se llama al servicio **cell relay**, como analogía con *frame relay*.

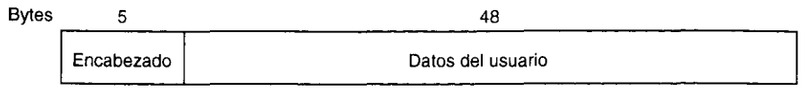


Figura 1-29. Una celda ATM.

El uso de una tecnología de conmutación de celdas es un rompimiento drástico con la tradición centenaria de la conmutación de circuitos (estableciendo una trayectoria de cobre) dentro del sistema de teléfonos. Son muchas las razones por las que se escogió la conmutación de celdas, entre ellas están las siguientes. Primero, la conmutación de celdas es altamente flexible y puede manejar con facilidad tanto tráfico de velocidad constante (audio, vídeo) como variable (datos). Segundo, a las velocidades tan altas que se contemplan (los gigabits por segundo están al alcance de la mano), la conmutación digital de las celdas es más fácil que el empleo de las técnicas tradicionales de multiplexión, en especial si se usa fibra óptica. Tercero, para la distribución de televisión es esencial la difusión; esto lo puede proporcionar la conmutación de celdas pero no la de circuitos.

Las redes ATM son orientadas a la conexión. Para hacer una llamada primero se debe enviar un mensaje para establecer la conexión. Después, todas las celdas subsecuentes siguen la misma trayectoria al destino. La entrega de celdas no está garantizada, pero sí su orden. Si las celdas 1 y 2 se envían en ese orden, y ambas llegan, lo harán en ese orden, nunca la 2 primero y después la 1.

Las redes ATM se organizan como las WAN tradicionales, con líneas y conmutadores (enrutadores). Las velocidades pretendidas para las redes ATM son de 155 Mbps y 622 Mbps, con la posibilidad de tener velocidades de gigabits más adelante. La velocidad de 155 Mbps se escogió porque es cercana a lo que se necesita para transmitir televisión de alta definición. La elección exacta de 155.52 Mbps se hizo por compatibilidad con el sistema de transmisión SONET de AT&T. La velocidad de 622 Mbps se eligió para que se pudieran mandar por ella cuatro canales de 155 Mbps. Ahora debe quedar claro por qué algunas de las plataformas de pruebas de gigabits operaban a 622 Mbps; usaban ATM.

Cuando se propuso ATM, virtualmente toda la discusión (esto es, la propaganda) era acerca del vídeo sobre pedido en cada hogar y el reemplazo del sistema de telefonía, según se describió antes. Desde entonces se han vuelto importantes otros avances. Muchas organizaciones han agotado el ancho de banda en las LAN de sus campus o edificios y se están viendo forzadas a recurrir a alguna clase de sistema de conmutación que tenga más ancho de banda que una sola LAN. También, en la computación cliente-servidor algunas aplicaciones necesitan hablar con ciertos servidores a velocidad elevada. Ciertamente, ATM es un candidato importante para ambos tipos de aplicación. Sin embargo, resulta un poco frustrante pasar de la meta de reemplazar todo el sistema telefónico de baja velocidad por uno digital de alta velocidad, a la meta de

tratar de conectar todas las Ethernets de una universidad. La interconexión de LAN mediante ATM se trata en (Kavak, 1995; Newman, 1994, y Truong *et al.*, 1995).

También vale la pena señalar que las diferentes organizaciones comprometidas con ATM tienen diversos intereses (financieros). Las portadoras de telefonía de larga distancia y las PTT se interesan principalmente en usar ATM para elevar el nivel del sistema telefónico y competir con las compañías de televisión por cable en la distribución electrónica de vídeo. Los vendedores de computadoras ven las LAN de ATM para universidades como el gran negocio (para ellos). Todos estos intereses opuestos no hacen más fácil, rápido ni coherente el proceso de estandarización actual. También, la política y el poder dentro de la organización que estandariza ATM (el ATM Forum) tienen una influencia considerable sobre la dirección que seguirá ATM.

El modelo de referencia B-ISDN ATM

Regresemos ahora a la tecnología de ATM, especialmente su aplicación en el sistema telefónico (futuro). La ISDN de banda ancha con ATM tiene su propio modelo de referencia, diferente del modelo OSI y también del modelo TCP/IP. Este modelo se muestra en la figura 1-30 y consiste en tres capas: la capa física, la capa ATM y la capa de adaptación de ATM, más cualquier cosa que los usuarios quieran poner encima.

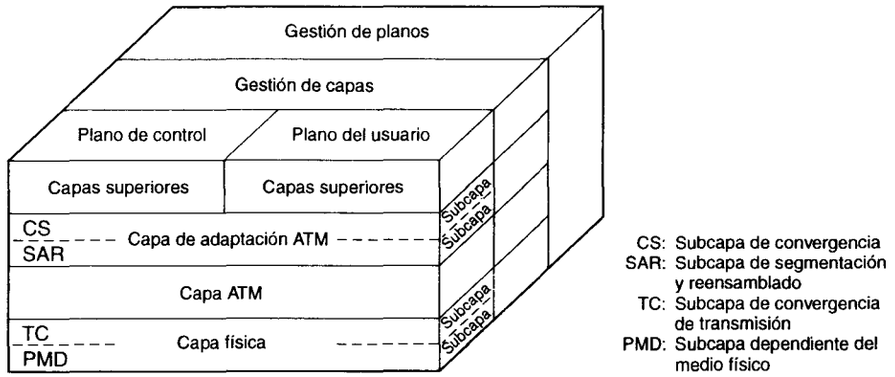


Figura 1-30. El modelo de referencia B-ISDN ATM.

La capa física tiene que ver con el medio físico: voltajes, temporización de bits y varias consideraciones más. ATM no prescribe un conjunto de reglas en particular, pero en cambio dice que las celdas ATM se pueden enviar por sí solas por un cable o fibra o bien se pueden empacar dentro de la carga útil de otros sistemas portadores. En otras palabras, ATM se diseñó para que fuera independiente del medio de transmisión.

La **capa ATM** tiene que ver con las celdas y su transporte; define la organización de las celdas y dice lo que significan los campos del encabezado. Esta capa también tiene que ver con el establecimiento y la liberación de circuitos virtuales y aquí es donde se localiza el control de la congestión.

Se ha definido una capa sobre la capa ATM que permita a los usuarios enviar paquetes mayores que una celda porque la mayor parte de las aplicaciones no quieren trabajar de manera directa con celdas (aunque algunas puedan hacerlo). La interfaz ATM segmenta estos paquetes, transmite las celdas en forma individual y las reensambla en el otro extremo. Esta capa es la **AAL** (*ATM adaptation layer,* **capa de adaptación de ATM**).

A diferencia de los antiguos modelos de referencia bidimensionales, el modelo ATM se define en tres dimensiones, como se muestra en la figura 1-30. El **plano de usuario** se encarga del transporte de los datos, el control de flujo, la corrección de errores y otras funciones de usuario. En contraste, el **plano de control** tiene que ver con la administración de la conexión. Las funciones de gestión de capas y planos se relacionan con la administración de recursos y la coordinación intercapas.

Las capas física y AAL se dividen, cada una, en dos subcapas, una en el fondo que hace el trabajo y una subcapa de convergencia en la parte superior que proporciona la interfaz adecuada con la capa de arriba. En la figura 1-31 se indican las funciones de las capas y subcapas.

Capa OSI	Capa ATM	Subcapa ATM	Funcionalidad
3/4	AAL	CS	Proporciona la interfaz estándar (convergencia)
		SAR	Segmentación y reensamblado
2/3	ATM		Control de flujo Generación/extracción de encabezados de la celda Gestión de circuitos/trayectorias virtuales Multiplexión/desmultiplexión de celdas
2	Física	TC	Desacoplamiento de la velocidad de envío de celdas Generación y comprobación de la suma de verificación del encabezado Generación de celdas Empacado/desempacado de celdas de la envoltura que las encierra Generación de marcos
1		PMD	Temporización de bits Acceso físico a la red

Figura 1-31. Las capas y subcapas de ATM y sus funciones.

La subcapa **PMD** (*physical medium dependent,* **dependiente del medio físico**) establece la interfaz con el cable real; transfiere los bits y controla su temporización. Esta capa es diferente para diferentes portadoras y cables.

La otra subcapa de la capa física es la subcapa **TC** (*transmission convergence,* **convergencia de transmisión**). Cuando se transmiten las celdas, la capa TC las envía como una corriente de bits a la capa PMD, lo cual es fácil de hacer. En el otro extremo, la subcapa TC obtiene una corriente entrante de puros bits de la subcapa PMD; su trabajo es convertir esta corriente de bits

en una corriente de celdas para la capa ATM. La subcapa TC se encarga de todas las consideraciones que se relacionan con determinar dónde empiezan y dónde terminan las celdas en la corriente de bits. En el modelo ATM, esta funcionalidad pertenece a la capa física. En el modelo OSI y en casi todas las demás redes, el trabajo de enmarcar, esto es, de convertir una corriente de bits en bruto en una secuencia de marcos o celdas, es tarea de la capa de enlace de datos. Por esa razón la estudiaremos en este libro junto con la capa de enlace de datos, no con la capa física.

Como mencionamos al principio, la capa ATM maneja celdas, lo que incluye su generación y transporte. Aquí se localiza la mayor parte de los aspectos interesantes de ATM. La capa ATM es una mezcla de las capas de enlace de datos y de red de OSI, pero no se divide en subcapas.

La capa AAL se divide en la subcapa **SAR** (*segmentation and reassembly,* **segmentación y rensamblado**) y la **CS** (*convergence sublayer,* **subcapa de convergencia**). La subcapa inferior divide los paquetes en celdas en el lado de la transmisión y los vuelve a armar de nuevo en el destino. La subcapa superior hace posible tener sistemas ATM que ofrezcan diferentes clases de servicios a diferentes aplicaciones (por ejemplo, la transferencia de archivos y el vídeo sobre pedido tienen diferentes necesidades en lo concerniente a manejo de errores, temporización, etcétera).

Perspectiva de ATM

En gran medida, ATM es un proyecto inventado por la industria telefónica porque después de que se instaló ampliamente Ethernet, la industria de las computadoras nunca apoyó una tecnología de redes de alta velocidad específica para hacerla estándar. Las compañías de teléfonos llenaron este vacío con ATM, aunque en octubre de 1991 muchos proveedores de computadoras se unieron con las compañías de teléfonos para armar el **ATM Forum**, un grupo de industriales que guiará el futuro de ATM.

Aunque ATM promete tener la capacidad de entregar la información en cualquier parte a velocidades que pronto excederán de 1 Gbps, el cumplimiento de esta promesa no será fácil. Básicamente, ATM es un conmutador de paquetes de alta velocidad, una tecnología con la que las compañías de teléfonos tienen poca experiencia. Lo que sí tienen es una inversión enorme en una tecnología diferente (la conmutación de circuitos) que en concepto no ha cambiado desde los días de Alexander Graham Bell. No es necesario decir que esta transición no ocurrirá rápidamente, sobre todo porque es un cambio revolucionario en lugar de ser evolucionario, y las revoluciones nunca ocurren suavemente.

También se deben considerar los aspectos económicos de instalar ATM en todo el mundo. Se tendrá que reemplazar una fracción sustancial del sistema telefónico existente. ¿Quién pagará esto? ¿Cuánto están dispuestos a pagar los usuarios por obtener una película sobre pedido en forma electrónica, cuando pueden obtener una en la tienda local de vídeos por un par de dólares? Finalmente, la pregunta de dónde se proveerán muchos de los servicios avanzados es crucial. Si la red los suministra, las compañías telefónicas se beneficiarán con ellos. Si los proporcionan computadoras conectadas a la red, los fabricantes y operadores de estos aparatos obtendrán las ganancias. Puede que a los usuarios no les importe, pero sin duda a las compañías de teléfonos y a los fabricantes de computadoras sí les importará, y de seguro que esto afectará su interés en lograr que ATM tenga éxito.

1.6.5. Comparación de los servicios

El lector podrá preguntarse por qué existen tantos servicios incompatibles y que se traslapan, incluidos DQDB, SMDS, X.25, *frame relay,* ATM y otros más. La razón fundamental es la decisión tomada en 1984 de dividir la AT&T y fomentar la competencia en la industria de las telecomunicaciones. Ahora, varias compañías con intereses y tecnologías diferentes están en libertad de ofrecer cualquier servicio para el que crean que existe una demanda, y muchas de ellas hacen esto sin contemplaciones.

Para repasar un poco sobre los servicios que hemos mencionado en este capítulo: DQDB es una tecnología de MAN no conmutada que permite enviar celdas de 53 bytes (de los cuales 44 son carga útil) por una línea larga dentro de una ciudad. SMDS es una tecnología de datagramas conmutados para enviar datagramas a cualquier punto de una red a 45 Mbps. X.25 es una tecnología antigua de redes orientada a la conexión para transmitir paquetes pequeños de tamaño variable a 64 kbps. *Frame relay* es un servicio que proporciona líneas rentadas virtuales con velocidades de alrededor de 1.5 Mbps. Finalmente, ATM se diseñó para reemplazar todo el sistema de teléfonos de conmutación de circuitos por uno de conmutación de celdas y poder manejar tanto datos como televisión. En la figura 1-32 se resumen algunas diferencias entre estos competidores.

Aspecto	DQDB	SMDS	X.25	*Frame relay*	ATM AAL
Orientado a la conexión	Sí	No	Sí	Sí	Sí
Velocidad normal (Mbps)	45	45	.064	1.5	155
Conmutado	No	Sí	Sí	No	Sí
Carga útil de tamaño fijo	Sí	No	No	No	No
Carga útil máxima	44	9188	128	1600	Variable
Circuitos virtuales permanentes	No	No	Sí	Sí	Sí
Multidifusión	No	Sí	No	No	Sí

Figura 1-32. Diferentes servicios de redes.

1.7. ESTANDARIZACIÓN DE REDES

Hay muchos proveedores de servicios de red, cada uno con sus propias ideas acerca de cómo deben hacerse las cosas. Sin coordinación, existiría un caos completo, y los usuarios nunca lograrían hacer nada. La única manera es acordar ciertos estándares de redes.

Los estándares no sólo permiten a diferentes computadoras comunicarse, sino que también incrementan el mercado para los productos que se ajustan a la norma, lo cual conduce a la

producción en masa, las economías de escala en la producción, las implementaciones VLSI, y otros beneficios que disminuyen el precio y aumentan la aceptación posterior. En las siguientes secciones veremos rápidamente el importante pero poco conocido mundo de la estandarización internacional.

Las normas se dividen en dos categorías: *de facto* y *de jure*. ***De facto*** (del latín "del hecho") son aquellos estándares que simplemente aparecieron, sin ningún plan formal. La PC de IBM y sus sucesores son normas *de facto* para computadoras pequeñas de oficina porque docenas de fabricantes decidieron copiar las máquinas IBM con mucha exactitud. UNIX es el estándar *de facto* para los sistemas operativos en los departamentos de ciencias de la computación de las universidades.

Los estándares **de jure** (del latín "por ley"), en contraste, son estándares formales y legales adoptados por algún organismo de estandarización autorizado. Las autoridades internacionales de estandarización generalmente se dividen en dos clases: las establecidas por tratados entre los gobiernos de las naciones y las organizaciones voluntarias, no surgidas de un tratado. En el área de las normas para redes de computadoras existen varias organizaciones de cada tipo, las cuales veremos a continuación.

1.7.1. Quién es quién en el mundo de las telecomunicaciones

La situación legal de las compañías telefónicas del mundo varía considerablemente de un país a otro. En un extremo está Estados Unidos, con 1500 compañías telefónicas privadas. Antes de que la AT&T se dividiera en 1984 era la corporación más grande del mundo y dominaba completamente la escena; proporcionaba servicio telefónico a cerca del 80% de los teléfonos instalados en Estados Unidos, distribuidos en la mitad de su área geográfica, y todas las demás compañías combinadas daban servicio a los clientes restantes (en su mayoría rurales). Desde su división, AT&T continúa proporcionando servicio de larga distancia, aunque ahora en competencia con otras compañías. Las siete compañías operadoras Bell regionales que se separaron de AT&T y 1500 independientes proporcionan servicio telefónico local y celular. Algunas de estas compañías independientes, como GTE, son muy grandes.

Las compañías estadounidenses, que proporcionan servicios de comunicación al público se llaman **portadoras comunes**. Los precios y servicios de estas empresas están descritos en un documento llamado **tarifa**, el cual debe ser aprobado por la Comisión Federal de Comunicaciones para el tráfico interestatal e internacional, y por las comisiones de servicios públicos para el tráfico intraestatal.

En el otro extremo se encuentran los países en los que el gobierno nacional tiene un monopolio completo de todas las comunicaciones, incluidos el correo, el telégrafo, el teléfono, y frecuentemente también el radio y la televisión. La mayor parte del mundo pertenece a esta categoría. En algunos casos, la autoridad de telecomunicaciones es una compañía nacionalizada; en otros, es simplemente una rama del gobierno, usualmente conocida como la **PTT** (administración de **correo**, **telégrafo** y **teléfono**). La tendencia mundial es hacia la liberalización y competencia, alejándose del monopolio gubernamental.

Con todos estos proveedores de servicios diferentes, existe una clara necesidad de lograr la compatibilidad a escala mundial para asegurar que las personas (y las computadoras) de un país puedan llamar a sus homólogos en algún otro. En realidad, esta necesidad ha existido desde hace mucho tiempo. En 1865, representantes de muchos gobiernos europeos se reunieron para formar el predecesor de la actual **ITU** (*International Telecom Union,* **Unión Internacional de Telecomunicaciones**). La misión de la ITU fue estandarizar las telecomunicaciones internacionales, lo que en esos días significaba telegrafía. Aun entonces era claro que si una mitad de los países usaba código Morse y la otra usaba algún otro código, se iba a presentar un problema. Cuando el teléfono se convirtió en un servicio internacional, la ITU emprendió la tarea de estandarizar también la telefonía. En 1947 la ITU llegó a ser una agencia de las Naciones Unidas.

La ITU tiene tres sectores principales:

1. Sector de radiocomunicaciones (ITU-R).
2. Sector de estandarización de telecomunicaciones (ITU-T).
3. Sector de desarrollo (ITU-D).

La ITU-R se ocupa de la asignación de frecuencias de radio en todo el mundo a los grupos de interés en competencia. A nosotros nos concierne principalmente la ITU-T, que está relacionada con los sistemas telefónicos y de comunicación de datos. De 1956 a 1993, la ITU-T fue conocida como **CCITT** por las iniciales de su nombre en francés: Comité Consultatif International Télégraphique et Téléphonique. El 1° de marzo de 1993 se reorganizó el CCITT para hacerlo menos burocrático y cambió de nombre para reflejar su nuevo papel. Tanto la ITU-T como el CCITT emitieron recomendaciones en el área de las comunicaciones telefónicas y de datos. Entre las recomendaciones del CCITT hay una que todavía está en uso; tal es la X.25 de CCITT, aunque desde 1993 las recomendaciones llevan la etiqueta ITU-T.

La ITU-T tiene cinco clases de miembros:

1. Administraciones (PTT nacionales).
2. Operadores privados reconocidos (por ejemplo, AT&T, MCI, British Telecom).
3. Organizaciones regionales de telecomunicaciones (por ejemplo, la ETSI europea).
4. Organizaciones comerciales y científicas de telecomunicaciones.
5. Otras organizaciones interesadas (por ejemplo, redes bancarias y de aerolíneas).

La ITU-T tiene cerca de 200 administraciones, 100 operadores privados y varios cientos de miembros más. Únicamente las administraciones pueden votar, pero todos los miembros pueden participar en el trabajo de la ITU-T. Como Estados Unidos no tiene una PTT, alguien más tiene que representarlo en la ITU-T. Esta tarea recayó en el Departamento de Estado, probablemente con la excusa de que ITU-T tenía que ver con países extranjeros, la especialidad del Departamento de Estado.

La tarea de la ITU-T es hacer recomendaciones técnicas acerca de las interfaces de telefonía, telegrafía y comunicación de datos. A menudo estos estándares lograron reconocimiento internacional; por ejemplo, V.24 (también conocida como EIA RS-232 en Estados Unidos), que especifica la disposición y significado de las clavijas en el conector utilizado por la mayor parte de las terminales asíncronas.

Cabe señalar que las recomendaciones de la ITU-T técnicamente sólo son sugerencias que los gobiernos pueden adoptar o ignorar, según lo deseen. En la práctica, un país que desee adoptar un estándar telefónico distinto al del resto del mundo es libre de hacerlo, pero a expensas de aislarse de todos los demás. Esto podría funcionar en Albania, pero en cualquier otro lugar sería un problema real. La ficción de llamar a los estándares de la ITU-T "recomendaciones" fue y es necesaria para mantener apaciguadas a las fuerzas nacionalistas en muchos países.

El trabajo real de la ITU-T se realiza en grupos de estudio, que frecuentemente llegan a incluir hasta 400 personas. Para lograr que se haga algo, los grupos de estudio se dividen en partidas de trabajo, las cuales a su vez se dividen en equipos de expertos, mismos que se subdividen en grupos *ad hoc*. Los burócratas nunca dejarán de serlo.

A pesar de todo esto, la ITU-T ha conseguido que se hagan las cosas. Su producción actual es de cerca de 5000 páginas de recomendaciones al año. Los miembros contribuyen con cuotas para cubrir los gastos de la ITU. Los países grandes y ricos supuestamente pagan hasta 30 unidades de contribución al año; los países pequeños y pobres pueden arreglárselas para pagar sólo 1/16 de una unidad de contribución (que asciende a unos 250,000 dólares). Un testimonio del valor de la ITU-T lo constituye el hecho de que casi todo el mundo paga su contribución justa aun cuando las contribuciones son completamente voluntarias.

Conforme las telecomunicaciones completan la transición iniciada en la década de 1980 de ser enteramente nacionales a ser enteramente globales, las normas se harán cada vez más importantes y más y más organizaciones van a querer participar en su fijación. Para más información acerca de la ITU, véase (Irmer, 1994).

1.7.2. Quién es quién en el mundo de los estándares internacionales

Los estándares internacionales son producidos por la **ISO** (*International Standards Organization*†, **Organización Internacional de Estándares**), una organización voluntaria, no surgida de un tratado, fundada en 1946. Sus miembros son las organizaciones nacionales de estándares de los 89 países miembros. Estos miembros incluyen ANSI (Estados Unidos), BSI (Gran Bretaña), AFNOR (Francia), DIN (Alemania) y otros 85.

La ISO emite estándares sobre un vasto número de temas, que van desde tuercas y pernos (literalmente) al revestimiento de los postes telefónicos. Se han emitido más de 5000 estándares, incluido el estándar OSI. La ISO tiene casi 200 comités técnicos (TC), numerados en el orden de su creación, cada uno de los cuales se hace cargo de un tema específico. El TC1 se ocupa de tuercas y pernos (estandarizando los pasos de la rosca de los tornillos). El TC97 se ocupa de computadoras y procesamiento de información. Cada TC tiene subcomités (SC) divididos en grupos de trabajo (WG).

† Para los puristas, el verdadero nombre de la ISO es International Organization for Standardization.

El trabajo real se hace en gran parte en los WG por más de 100,000 voluntarios en todo el mundo. Muchos de estos "voluntarios" son asignados para trabajar en asuntos de la ISO por sus patrones, cuyos productos se están estandarizando. Otros voluntarios son oficiales gubernamentales interesados en que la forma en que se hacen las cosas en su país llegue a ser el estándar internacional. También participan expertos académicos en muchos de los WG.

En cuestiones de estándares de telecomunicaciones, la ISO y la ITU-T a menudo cooperan para evitar la ironía de tener dos estándares internacionales oficiales y mutuamente incompatibles (la ISO es un miembro de la ITU-T).

El representante de Estados Unidos en la ISO es el **ANSI** (*American National Standards Institute,* **Instituto Nacional Estadounidense de Estándares**), el cual, a pesar de su nombre, es una organización privada, no gubernamental y no lucrativa. Sus miembros son fabricantes, empresas de telecomunicaciones y otros particulares interesados. La ISO a menudo adopta los estándares ANSI como estándares internacionales.

El procedimiento que sigue la ISO para adoptar estándares está diseñado para lograr el mayor consenso posible. El proceso se inicia cuando una de las organizaciones nacionales de estándares siente la necesidad de un estándar internacional en alguna área. A continuación se forma un grupo de trabajo para proponer un **CD** (*committee draft,* **borrador de comité**). Luego se circula el CD a todos los miembros, los cuales tienen seis meses para criticarlo. Si una mayoría considerable aprueba el CD, se produce un documento revisado, llamado **DIS** (*draft international standard,* **borrador de estándar internacional**), y se circula para ser comentado y votado. Con base en los resultados de esta vuelta se prepara el texto final del **IS** (*international standard,* **estándar internacional**), se aprueba y se publica. En áreas de mucha controversia, un CD o un DIS podría pasar por varias versiones antes de obtener suficientes votos, y el proceso completo puede tardar años.

El **NIST** (*National Institute of Standards and Technology,* **Instituto Nacional de Estándares y Tecnología**) es una agencia del Departamento de Comercio de Estados Unidos, antiguamente conocida como Oficina Nacional de Estándares. Este organismo emite estándares que son obligatorios para las compras hechas por el gobierno de Estados Unidos, excepto las del Departamento de la Defensa, que tiene sus propios estándares.

Otro protagonista importante en el mundo de los estándares es el **IEEE** (*Institute of Electrical and Electronics Engineers,* **Instituto de Ingenieros Eléctricos y Electrónicos**), la organización profesional más grande del mundo. Además de publicar revistas y organizar numerosas conferencias cada año, el IEEE tiene un grupo de estandarización que elabora estándares en las áreas de ingeniería eléctrica y computación. El estándar 802 del IEEE para redes de área local es el estándar clave para las LAN, y posteriormente fue adoptado por la ISO como base para el estándar ISO 8802.

1.7.3. Quién es quién en el mundo de los estándares de Internet

El amplio mundo de la Internet tiene sus propios mecanismos de estandarización, muy diferentes de los de la ITU-T y la ISO. La diferencia puede resumirse en forma burda diciendo que la

gente que asiste a las juntas de estandarización de la ITU o la ISO usa traje. La gente que asiste a las juntas de estandarización de la Internet usa ya sea *jeans* o uniformes militares.

Las reuniones de la ITU-T y la ISO están pobladas por oficiales corporativos y burócratas para quienes la estandarización es su trabajo. Ellos consideran la estandarización como algo positivo y dedican sus vidas a ella. Por otro lado, la gente de Internet definitivamente prefiere la anarquía por cuestiones de principios, pero sabe que algunas veces se necesitan acuerdos para lograr que las cosas funcionen. Así pues, los estándares, aunque no debieran existir, son ocasionalmente necesarios.

Cuando se inició la ARPANET, el DoD creó un comité informal para supervisarla. En 1983, el comité fue rebautizado como **IAB** (*Internet Activities Board,* **Consejo de Actividades de Internet**) y se le encomendó una misión un poco más amplia, a saber, mantener a los investigadores que trabajaban con la ARPANET y la Internet apuntando más o menos en la misma dirección, una actividad no muy diferente de controlar una manada de gatos. El significado del acrónimo "IAB" se cambio más tarde a **Consejo de Arquitectura de Internet** (*Internet Architecture Board*).

Cada uno de los aproximadamente 10 miembros del IAB encabezó una fuerza de trabajo sobre algún aspecto de importancia. El IAB se reunía varias veces al año para comentar resultados y realimentar al DoD y a la NSF, quienes proporcionaban la mayor parte de los fondos en esa época. Cuando se necesitaba un estándar (por ejemplo, un algoritmo de ruteo nuevo), los miembros del IAB lo discutían y después anunciaban el cambio para que los estudiantes graduados, quienes eran el corazón de las labores de creación de *software*, pudieran implementarlo. La comunicación era a través de una serie de informes técnicos llamados **RFC** (*request for comments,* **petición de comentarios**). Los RFC se guardan en línea y pueden ser recuperados por cualquier interesado en ellos. Los RFC están numerados en orden cronológico de creación y ya hay cerca de 2000.

En 1989, la Internet había llegado a ser tan grande que este estilo tan informal ya no funcionaba. Para entonces, muchos proveedores ofrecían ya productos TCP/IP y no los querían cambiar sólo porque 10 investigadores habían tenido una mejor idea. En el verano de 1989, el IAB se reorganizó otra vez. Los investigadores pasaron a la **IRTF** (*Internet Research Task Force,* **Fuerza de Trabajo de Investigación sobre Internet**), la cual se hizo subsidiaria de IAB junto con la **IETF** (*Internet Engineering Task Force,* **Fuerza de Trabajo de Ingeniería de Internet**). El IAB se repobló con gente que representaba una gama amplia de organizaciones, no sólo la comunidad de investigación. Inicialmente, el IAB fue un grupo que se perpetuaba a sí mismo, pues sus miembros servían por un término de dos años y los nuevos miembros eran designados por los antiguos. Más tarde fue creada la **Internet Society**, formada por gente interesada en la Internet. Así, la Internet Society es en cierto sentido comparable con la ACM o el IEEE; está gobernada por administradores elegidos quienes designan a los miembros del IAB.

Lo que se buscaba con esta división era tener al IRTF concentrado en investigaciones a largo plazo, mientras que el IETF se encargaba de los problemas de ingeniería a corto plazo. El IETF se dividió en grupos de trabajo, cada uno con un problema específico por resolver. Los presidentes de estos grupos de trabajo inicialmente se reunían como un comité de conducción para dirigir los trabajos de ingeniería. Los temas del grupo de trabajo incluyen nuevas aplicaciones, información de usuarios, integración de OSI, ruteo y direccionamiento, seguridad, administración de

redes y estándares. Se llegaron a formar tantos grupos de trabajo (más de 70) que fue necesario agruparlos en áreas, y los presidentes de área formaron el comité de conducción.

Adicionalmente, se adoptó un proceso de estandarización más formal a imagen del de la ISO. Para convertirse en **propuesta de estándar**, la idea básica se debe explicar completamente en un RFC y debe generar suficiente interés en la comunidad para justificar su consideración. Para avanzar a la etapa de **borrador de estándar**, debe existir una implementación operante que haya sido probada concienzudamente por al menos dos sitios independientes durante cuatro meses. Si el IAB se convence de que la idea es buena y el software funciona, puede declarar que el RFC es un estándar de Internet. Algunos estándares de Internet han llegado a ser estándares del DoD (MIL-STD), volviéndose obligatorios para los proveedores del DoD. David Clark hizo una vez un comentario ahora famoso acerca de que la estandarización de Internet consistía en "consenso aproximado y código en operación".

1.8. ESQUEMA DEL RESTO DEL LIBRO

Este libro trata tanto los principios como la práctica de las redes de computadoras. La mayor parte de los capítulos empieza con una explicación de los principios pertinentes, seguida de varios ejemplos que ilustran estos principios. A lo largo del texto se usan dos redes como ejemplos continuos: las redes Internet y ATM. En cierta forma, las dos son complementarias: ATM tiene que ver en su mayor parte con las capas más bajas y la Internet está relacionada principalmente con las capas superiores. En el futuro, la Internet quizá opere en buena parte sobre una *backbone* de ATM, de modo que las dos podrían coexistir. Se darán otros ejemplos donde sean pertinentes.

El libro se estructura de acuerdo con el modelo híbrido de la figura 1-21. A partir del capítulo 2, comenzaremos a subir por la jerarquía de protocolos, empezando desde los cimientos. El segundo capítulo proporciona antecedentes en el campo de la comunicación de datos; cubre la transmisión analógica y digital, multiplexión, conmutación y el sistema telefónico pasado, presente y futuro. Este material concierne a la capa física, aunque sólo trataremos los aspectos arquitectónicos y no los de *hardware*. También veremos varios ejemplos de la capa física, tales como SONET y el radio celular.

El capítulo 3 se ocupa de la capa de enlace de datos y sus protocolos presentando varios ejemplos de complejidad creciente. También se hará un análisis de estos protocolos. Más adelante estudiaremos algunos protocolos importantes del mundo real, entre ellos: HDLC (que se usa en redes de velocidad baja y media), SLIP, PPP (que se usa en la Internet) y ATM (que se usa en B-ISDN).

El capítulo 4 se dedica a la subcapa de acceso al medio, que es parte de la capa de enlace de datos. La pregunta básica que aborda es cómo determinar quién puede ser el siguiente en usar la red cuando ésta consiste en un solo canal compartido, como en la mayor parte de las LAN y en algunas redes de satélites. Se dan muchos ejemplos del área de las LAN, las redes de fibra óptica y las redes de satélites. También se estudian aquí los puentes, que sirven para conectar entre sí las LAN.

El capítulo 5 trata la capa de red, en especial el ruteo, el control de congestión y la interconexión de redes. Veremos algoritmos tanto estáticos como dinámicos. Se cubre también el ruteo por difusión y se analiza en detalle el efecto del ruteo deficiente y la congestión. Conectar entre sí redes heterogéneas para formar interredes origina gran cantidad de problemas de los que hablaremos aquí. Se hará una amplia cobertura a las capas de red en la Internet y las redes ATM.

El capítulo 6 trata la capa de transporte. Se hará hincapié en los protocolos orientados a la conexión, pues muchas aplicaciones los necesitan. Se verá en detalle un ejemplo de servicio de transporte y su implementación. Estudiaremos en detalle tanto los protocolos de transporte de Internet (TCP y UDP) como los protocolos de transporte de ATM (AAL 1-5).

Las capas de sesión y de presentación de OSI no se verán en este libro pues no se utilizan ampliamente.

El capítulo 7 trata la capa de aplicación, sus protocolos y aplicaciones. Entre las aplicaciones que veremos están la seguridad, el uso de nombres, el correo electrónico, las noticias de red, la administración de redes, la Red Mundial y los multimedios.

El capítulo 8 contiene una lista anotada de lecturas recomendadas ordenadas por capítulo. Se pretende ayudar a aquellos lectores que quisieran continuar de manera más profunda su estudio de las redes. El capítulo contiene también una bibliografía en orden alfabético de todas las referencias citadas en este libro.

1.9. RESUMEN

Las redes de computadoras pueden prestar un gran número de servicios, tanto para compañías como para individuos. Para las compañías, las redes de computadoras personales que usan servidores compartidos ofrecen con frecuencia flexibilidad y una buena relación precio/rendimiento. Para los individuos, las redes ofrecen acceso a una variedad de recursos de información y distracción.

En términos aproximados, las redes se pueden dividir en LAN, MAN, WAN e interredes, cada una con sus características, tecnologías, velocidades y nichos propios. Las LAN abarcan un edificio, las MAN cubren una ciudad y las WAN un país o un continente. Las LAN y las MAN son no conmutadas (es decir, no tienen enrutadores); las WAN son conmutadas.

El *software* de red consiste en protocolos: reglas que rigen la comunicación entre procesos. Los protocolos pueden ser sin conexiones u orientados a la conexión. La mayor parte de las redes manejan las jerarquías de protocolos, en las que cada capa provee servicios a las capas que están sobre ella y las aísla de los detalles de los protocolos empleados en las capas más bajas. Las pilas de protocolos por lo regular se basan ya sea en el modelo OSI o en el TCP/IP. Los dos modelos tienen capas de red, de transporte y de aplicación, pero en las otras capas son diferentes.

Entre las redes más conocidas están NetWare de Novell, la ARPANET (ya difunta), la NSFNET, la Internet y varias plataformas de pruebas de gigabits. Entre los servicios de red han estado DQDB, SMDS, X.25, *frame relay* e ISDN de banda ancha. Todos están comercialmente disponibles de diversos proveedores. El mercado determinará quiénes sobrevivirán y quiénes no.

PROBLEMAS

1. En el futuro, cuando todo mundo tenga una terminal casera conectada a una red de computadoras, serán posibles los referendos públicos instantáneos sobre legislación importante pendiente. En algún momento las legislaturas existentes se podrían eliminar para dejar que se exprese de manera directa el deseo del pueblo. Los aspectos positivos de tal democracia directa son bastante obvios; comente los aspectos negativos.
2. Una alternativa a una LAN es simplemente un sistema grande de tiempo compartido con terminales para todos los usuarios. Cite dos ventajas de un sistema cliente-servidor que utiliza una LAN.
3. Un grupo de cinco enrutadores se va a conectar a una subred punto a punto. Entre cada par de enrutadores los diseñadores pueden colocar una línea de alta velocidad, una de velocidad media, una de baja velocidad o ninguna línea. Si se requieren 100 ms de tiempo de computadora para generar e inspeccionar cada topología, ¿qué tiempo tomará inspeccionarlas todas para determinar cuál es la mejor para la carga esperada?
4. Un grupo de $2^n - 1$ enrutadores se interconecta en un árbol binario centralizado, con un enrutador en cada *host* del árbol. El enrutador i se comunica con el j enviando un mensaje a la raíz del árbol. Luego, la raíz envía hacia abajo el mensaje a j. Deduzca una expresión aproximada de la cantidad media de saltos por mensaje para un valor grande de n suponiendo que todos los pares de enrutadores son igualmente probables.
5. Una desventaja de una subred de difusión es la capacidad desperdiciada cuando múltiples *hosts* intentan acceder al canal al mismo tiempo. Como ejemplo simplista, suponga que el tiempo se dividió en porciones discretas, y cada una de las n *hosts* intenta usar el canal con una probabilidad de p durante cada porción. ¿Qué fracción de las porciones se desperdicia debido a los choques?
6. ¿Cuáles son las direcciones de SAP en la radiodifusión FM?
7. ¿Cuál es la diferencia principal entre la comunicación sin conexión y la orientada a la conexión?
8. Cada una de dos redes proporciona un servicio confiable orientado a la conexión. Una de ellas ofrece una corriente de bytes confiable y la otra ofrece una corriente de mensajes confiable. ¿Son idénticas? Si es así, ¿por qué se hace la distinción? Si no, cite un ejemplo de cómo difieren.
9. ¿Qué diferencia hay entre un servicio confirmado y uno no confirmado? Para cada uno de los siguientes, diga si podría ser un servicio confirmado, uno no confirmado, ambos o ninguno.
   (a) Establecimiento de la conexión.
   (b) Transmisión de datos.
   (c) Liberación de la conexión.
10. ¿Qué significa "negociación" cuando se habla de protocolos de red? Cite un ejemplo.
11. ¿Cite dos razones para usar protocolos en capas?
12. Mencione dos semejanzas entre el modelo de referencia OSI y el TCP/IP. Mencione dos aspectos en los que son diferentes.

13. El presidente de la Specialty Paint Corp. tiene la idea de colaborar con un fabricante de cerveza local con el propósito de producir una lata de cerveza invisible (como medida para reducir la basura). El presidente ordena a su departamento legal que investiguen el asunto, y ellos a su vez piden ayuda al grupo de ingeniería. Como resultado, el ingeniero en jefe llama a su homólogo en la otra compañía para discutir los aspectos técnicos del proyecto. Después, los ingenieros presentan un informe a sus respectivos departamentos legales, quienes entonces conversan por teléfono para arreglar los aspectos legales. Por último, los presidentes corporativos discuten el aspecto financiero del trato. ¿Es éste un ejemplo de protocolo multicapas en el sentido del modelo OSI?
14. En la mayor parte de las redes, la capa de enlace de datos maneja los errores de transmisión pidiendo que se retransmitan los marcos dañados. Si la probabilidad de que un marco se dañe es p, ¿cuál es la cantidad promedio de transmisiones requeridas para enviar un marco si los acuses de recibo nunca se pierden?
15. ¿Cuál de las capas OSI maneja cada uno de los siguientes aspectos?
   (a) Dividir la corriente de bits transmitida en marcos.
   (b) Determinar cuál ruta seguir a través de la subred.
16. ¿Las TPDU encapsulan marcos o los marcos encapsulan las TPDU? Explique.
17. Un sistema tiene una jerarquía de protocolos de n capas. Las aplicaciones generan mensajes de M bytes de longitud. En cada capa se añade una cabecera de h bytes. ¿Qué fracción del ancho de banda de la red se llena con cabeceras?
18. ¿Cuál es la diferencia principal entre TCP y UDP?
19. La arquitectura de NetWare de Novell, ¿se parece más a X.25 o a Internet? Explique su respuesta.
20. La Internet duplica su tamaño aproximadamente cada 18 meses. Aunque nadie lo sabe con certeza, alguien estimó que en enero de 1996 la cantidad de *hosts* que incluía era de 7 millones. Utilice estos datos para calcular la cantidad esperada de *hosts* en Internet en el año 2008.
21. ¿Por qué se diseñó SMDS como una red sin conexión y *frame relay* como una red orientada a la conexión?
22. Imagine que entrenó a Bernie, su perro San Bernardo, para cargar una caja de tres cintas Exabyte de 8 mm en lugar de un barril de brandy. (Cuando su disco se llena, usted considera esto una emergencia.) Cada una de estas cintas contiene 7 gigabytes. El perro puede viajar a su lado, a dondequiera que usted vaya, a 18 km/h. ¿Para qué intervalos de distancia tiene Bernie una velocidad de datos mayor que una línea ATM de 155 Mbps?
23. Al transferir un archivo entre dos computadoras, son posibles (al menos) dos estrategias de acuse de recibo. En la primera, el archivo se divide en paquetes, cuyo recibo acusa de forma individual el receptor, aunque no se acusa el recibo como un todo. En la segunda, no se acusa el recibo de los paquetes individuales, sino el del archivo entero cuando llega. Comente los dos enfoques.
24. Imagine que el paquete SMDS de la figura 1-28 se fuera a incorporar en la jerarquía de protocolos OSI. ¿En cuál capa aparecería?
25. Cite una ventaja y una desventaja del *frame relay* sobre las líneas telefónicas rentadas.

26. ¿Por qué ATM usa celdas pequeñas de longitud fija?

27. Mencione dos ventajas y dos desventajas de tener estándares internacionales para los protocolos de redes.

28. Cuando un sistema tiene una parte permanente y una removible, como una unidad de disquetes y el disquete, es importante que el sistema esté estandarizado, de manera que diferentes compañías puedan fabricar tanto las partes permanentes como las removibles y todo pueda funcionar junto. Cite tres ejemplos fuera de la industria de las computadoras en los que existan tales estándares internacionales. Ahora cite tres áreas fuera de la industria de las computadoras en las que no existan tales normas.

2

LA CAPA FÍSICA

En este capítulo veremos la capa más baja representada en la jerarquía de la figura 1-21. Comenzaremos con un análisis teórico de la transmisión de datos, únicamente para descubrir que la Madre (¿o Padre?) Naturaleza fija límites sobre lo que se puede enviar a través de un canal.

A continuación estudiaremos los medios de transmisión, tanto los guiados (alambre de cobre y fibra óptica) como los no guiados (inalámbricos). Este material proveerá información básica sobre las tecnologías de transmisión clave que se usan en las redes modernas.

El resto del capítulo se dedica a ejemplos de sistemas de comunicación que se valen de estos medios de transmisión subyacentes. Empezaremos con el sistema telefónico, examinando tres versiones diferentes: el sistema actual (parcialmente) analógico, un sistema digital potencial para el futuro cercano (N-ISDN) y un sistema digital probable para el futuro distante (ATM). Después veremos dos sistemas inalámbricos, el radio celular y los satélites de comunicaciones.

2.1. BASES TEÓRICAS DE LA COMUNICACIÓN DE DATOS

La Vfiformación se puede transmitir por cables al variar alguna propiedad física, como el voltaje o la corriente. Si representamos el valor de este voltaje o corriente como una función simple del tiempo, $f(t)$, podemos modelar el comportamiento de la señal y analizarla de manera matemática. Este análisis es el tema de las secciones siguientes.

2.1.1. Análisis de Fourier

A principios del siglo XIX, el matemático francés Jean-Baptiste Fourier demostró que sumando una cantidad (posiblemente infinita) de senos y cosenos se puede construir cualquier función periódica de comportamiento razonable, $g(t)$, con periodo T:

$$g(t) = \frac{1}{2}c + \sum_{n=1}^{\infty} a_n \operatorname{sen}(2\pi nft) + \sum_{n=1}^{\infty} b_n \cos(2\pi nft) \qquad (2\text{-}1)$$

donde $f = 1/T$ es la frecuencia fundamental y a_n y b_n son las amplitudes de seno y coseno del n-ésimo (término) **armónico**. Tal descomposición se llama **serie de Fourier**. La función se puede reconstruir a partir de la serie de Fourier; esto es, si se conoce el periodo T y se dan las amplitudes, se puede encontrar la función original del tiempo efectuando las sumas de la ecuación (2-1).

Una señal de datos que tenga duración finita (lo cual tienen todas) se puede manejar con sólo imaginar que el patrón completo se repite una y otra vez eternamente (es decir, el intervalo de T a $2T$ es el mismo que de 0 a T, etcétera).

Las amplitudes a_n se pueden calcular para cualquier $g(t)$ multiplicando ambos lados de la ecuación (2-1) por sen($2\pi kft$) e integrando de 0 a T. Puesto que:

$$\int_0^T \operatorname{sen}(2\pi kft)\,\operatorname{sen}(2\pi nft)\,dt = \begin{cases} 0 \text{ para } k \neq n \\ T/2 \text{ para } k = n \end{cases}$$

únicamente sobrevive un término de la sumatoria: a_n. La sumatoria de b_n se desvanece por completo. De manera similar, al multiplicar la ecuación (2-1) por cos($2\pi kft$) e integrar entre 0 y T podemos deducir b_n. Con sólo integrar ambos miembros de la ecuación como está, se puede encontrar c. Los resultados de estas operaciones son los siguientes:

$$a_n = \frac{2}{T}\int_0^T g(t)\operatorname{sen}(2\pi ft)dt \qquad b_n = \frac{2}{T}\int_0^T g(t)\cos(2\pi ft)dt \qquad c = \frac{2}{T}\int_0^T g(t)dt$$

2.1.2. Señales limitadas por el ancho de banda

Para ver qué tiene que ver todo esto con la comunicación de datos, consideremos un ejemplo específico: la transmisión del carácter ASCII "b", codificado en un byte de ocho dígitos. El patrón de bits por transmitir es 01100010. La parte izquierda de la figura 2-1(a) muestra la salida de voltaje de la computadora transmisora. El análisis de Fourier de esta señal conduce a los coeficientes:

$$a_n = \frac{1}{\pi n}[\cos(\pi n/4) - \cos(3\pi n/4) + \cos(6\pi n/4) - \cos(7\pi n/4)]$$

$$b_n = \frac{1}{\pi n}[\operatorname{sen}(3\pi n/4) - \operatorname{sen}(\pi n/4) + \operatorname{sen}(7\pi n/4) - \operatorname{sen}(6\pi n/4)]$$

$$c = 3/8$$

En el lado derecho de la figura 2-1(a) se muestran las amplitudes de raíz cuadrada media, $\sqrt{a_n^2 + b_n^2}$, para los primeros términos. Estos valores son de interés porque sus cuadrados son proporcionales a la energía transmitida a la frecuencia correspondiente.

Ninguna instalación transmisora puede enviar señales sin perder cierta potencia en el proceso. Si todos los componentes de Fourier disminuyeran en la misma proporción, la señal resultante se reduciría en amplitud pero no se distorsionaría [esto es, tendría la misma forma cuadrada de la figura 2-1(a)]. Desafortunadamente, todas las instalaciones de transmisión disminuyen los distintos componentes de Fourier en diferente grado, introduciendo así una distorsión. Por lo general, las amplitudes se transmiten sin disminución desde 0 hasta cierta frecuencia f_c [medida en ciclos/s o Hertz (Hz)], y todas las frecuencias que están sobre esta frecuencia de corte se atenúan fuertemente. En ciertos casos, ésta es una propiedad física de los medios de transmisión, pero en otros se introduce intencionalmente un filtro en el circuito para limitar el valioso ancho de banda disponible para cada cliente.

Consideremos ahora cómo luciría la señal de la figura 2-1(a) si el ancho de banda fuera tan bajo que se transmitieran únicamente las frecuencias más bajas [es decir, que la función se aproximara con los primeros términos de la ecuación (2-1)]. La figura 2-1(b) muestra la señal que resulta de un canal que permite el paso únicamente a la primera armónica (la fundamental, f). De manera similar, las figuras 2-1(c) a (e) muestran los espectros y las funciones reconstruidas para canales de mayor ancho de banda.

El tiempo T requerido para transmitir el carácter depende tanto del método de codificación como de la velocidad de señalización [la cantidad de veces por segundo que la señal cambia su valor (por ejemplo, su voltaje)]. La cantidad de cambios por segundo se mide en **bauds**. Una línea de b bauds no necesariamente transmite b bits/seg, pues cada señal podría transportar varios bits. Si se usaran los voltajes 0, 1, 2, 3, 4, 5, 6 y 7, el valor de cada señal podría servir para transportar tres bits, de modo que la velocidad en bits/seg sería tres veces la velocidad en bauds. En nuestro ejemplo, sólo se usan ceros y unos como niveles de señal, así que la velocidad en bits/seg es igual a la velocidad en bauds.

Si se tiene una velocidad de b bits/seg, el tiempo que se requiere para enviar (por ejemplo) 8 bits es de $8/b$ segundos; por tanto, la frecuencia de la primera armónica es $b/8$ Hz. Una línea telefónica común, por lo general llamada **línea de grado de voz**, tiene una frecuencia de corte cercana a los 3000 Hz establecida en forma artificial. Esta restricción significa que el número de la armónica más alta que puede pasar es de $3000/(b/8)$ o $24{,}000/b$, aproximadamente (el corte no es abrupto).

Para algunas tasas de datos, los números son los que se muestran en la figura 2-2. De estos números, queda claro que tratar de transmitir a 9600 bps por una línea telefónica de voz transformará la figura 2-1(a) en algo parecido a la figura 2-1(c), dificultando mucho la recepción exacta de la corriente de bits original. Debe ser obvio que a velocidades de envío de datos mucho mayores que 38.4 kbps no hay la menor esperanza para las señales *binarias*, aun si la instalación de transmisión se encuentra completamente libre de ruidos. En otras palabras, limitar el ancho de banda limita la velocidad de envío de datos, incluso en canales perfectos. Sin embargo, existen esquemas de codificación avanzados que utilizan varios niveles de voltaje y pueden lograr tasas de envío de datos mayores. Éstos los veremos más adelante en este capítulo.

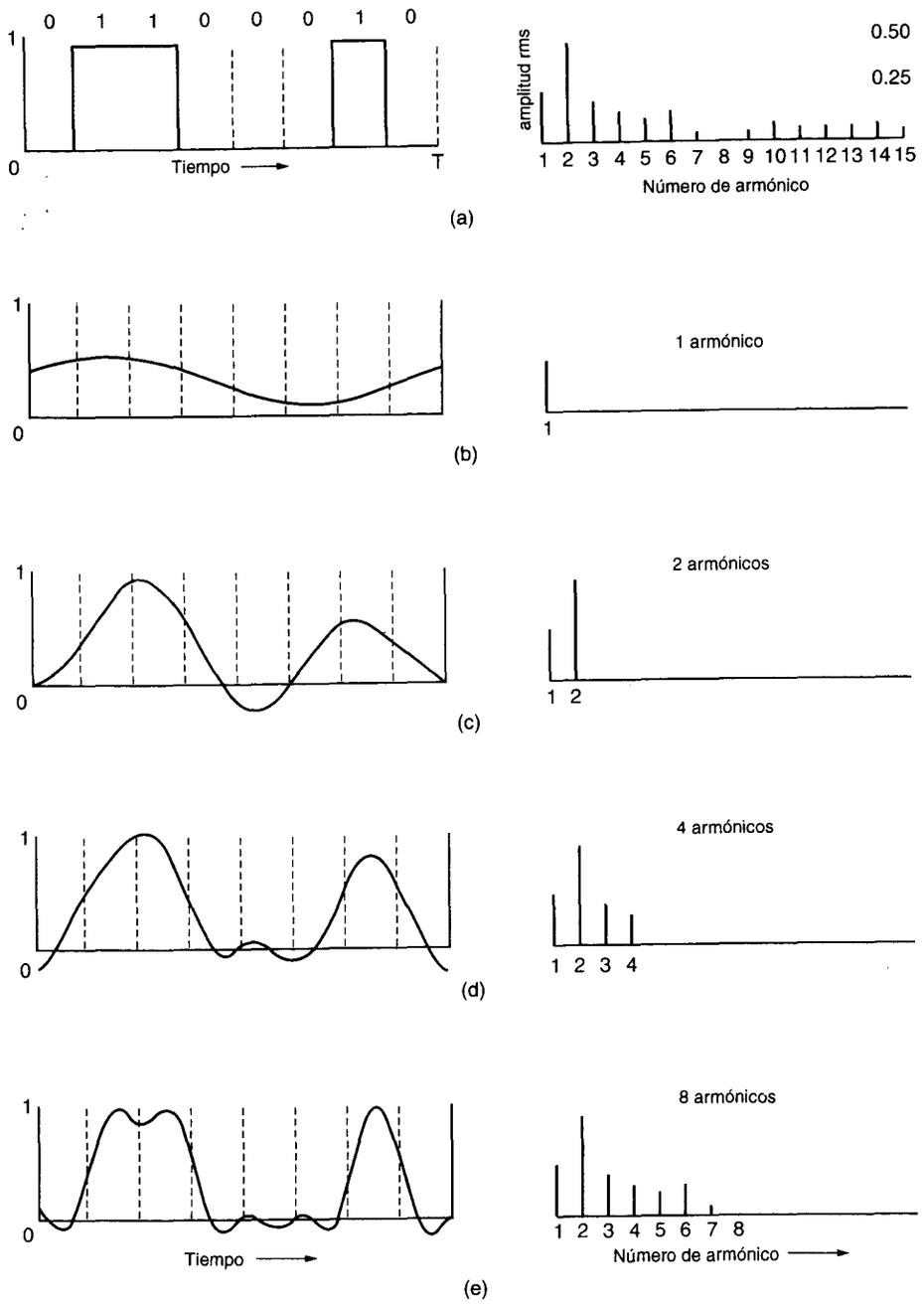


Figura 2-1. (a) Señal binaria y sus amplitudes de raíz cuadrada media de Fourier. (b)-(e) Aproximaciones sucesivas a la señal original.

Bps	T (mseg)	Primer armónico (Hz)	Num. de armónicos enviados
300	26.67	37.5	80
600	13.33	75	40
1200	6.67	150	20
2400	3.33	300	10
4800	1.67	600	5
9600	0.83	1200	2
19200	0.42	2400	1
38400	0.21	4800	0

Figura 2-2. Relación entre tasa de datos y armónicos.

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DOCUMENTACIÓN Y BIBLIOTECA
MONTEVIDEO - URUGUAY

2.1.3. La tasa de envío máxima de un canal

Ya en 1924, H. Nyquist se dio cuenta de la existencia de este límite fundamental y dedujo una ecuación que expresa la tasa de datos máxima para un canal sin ruido de ancho de banda finito. En 1948, Claude Shannon llevó más allá el trabajo de Nyquist y lo extendió al caso de un canal sujeto a ruido aleatorio (esto es, termodinámico) (Shannon, 1948). Aquí sólo resumiremos en forma breve sus resultados que ahora son clásicos.

Nyquist demostró que si se pasa una señal arbitraria a través de un filtro pasa-bajas (*low-pass*) de ancho de banda *H*, la señal filtrada se puede reconstruir por completo tomando sólo $2H$ muestras (exactas) por segundo. Es inútil muestrear la línea a velocidad mayor que $2H$ veces por segundo porque los componentes de frecuencia más alta que tal muestreo podría recuperar ya se han filtrado. Si la señal consiste en *V* niveles discretos, el teorema de Nyquist establece:

$$\text{tasa de datos máxima} = 2H \log_2 V \text{ bits/seg}$$

Por ejemplo, un canal sin ruido de 3 kHz no puede transmitir señales binarias (esto es, de dos niveles) a una velocidad mayor de 6000 bps.

Hasta aquí hemos considerado sólo canales sin ruido. Si está presente un ruido aleatorio, la situación se deteriora rápidamente. La cantidad de ruido térmico presente se mide por la relación entre la potencia de la señal y la potencia del ruido, y se llama **relación señal a ruido**. Si denotamos la potencia de la señal por *S* y la potencia del ruido por *N*, la relación señal a ruido es *S/N*. Generalmente, la relación misma no se cita; en cambio, se usa la cantidad $10 \log_{10} S/N$. A estas unidades se les llama **decibeles** (dB). Una relación *S/N* de 10 es igual a 10 dB, una relación de 100 es 20 dB, una relación de 1000 es 30 dB y así sucesivamente. Los fabricantes de amplificadores estereofónicos a menudo caracterizan el ancho de banda (gama de frecuencias) en el

cual es lineal su producto dando la frecuencia de 3 dB en cada extremo. Éstos son los puntos en los que el factor de amplificación se ha dividido aproximadamente a la mitad.

El principal resultado de Shannon es que la tasa de datos máxima de un canal ruidoso cuyo ancho de banda es H Hz y cuya relación señal a ruido es S/N está dada por:

$$\text{número máximo de bits/seg} = H\log_2(1 + S/N)$$

Por ejemplo, un canal con ancho de banda de 3000 Hz y una relación de señal a ruido térmico de 30 dB (parámetros típicos de la parte analógica de un sistema telefónico) nunca puede transmitir a mucho más que 30,000 bps, no importa cuántos niveles de señal se usen ni qué tan frecuente o infrecuente sea el muestreo. El resultado de Shannon se dedujo aplicando argumentos de la teoría de la información y es válido para cualquier canal sujeto a ruido gaussiano (térmico). Los ejemplos contrarios se deben clasificar en la misma categoría que las máquinas de movimiento perpetuo. Sin embargo, cabe señalar que éste solamente es un límite superior y que los sistemas reales rara vez lo alcanzan.

2.2. MEDIOS DE TRANSMISIÓN

El propósito de la capa física es transportar una corriente de bits en bruto de una máquina a otra. Se pueden usar varios medios físicos para la transmisión real; cada uno con su propio nicho en términos de ancho de banda, retardo, costo y facilidad de instalación y mantenimiento. A grandes rasgos, los medios se agrupan en medios guiados, como el cable de cobre y la fibra óptica, y medios no guiados, como la radio y los láseres a través del aire. Veremos éstos en esta sección y en la siguiente.

2.2.1. Medios magnéticos

Una de las formas más comunes de transportar datos de una computadora a otra es escribirlos en cinta magnética o disquetes, transportar físicamente la cinta o los discos a la máquina de destino y leerlos de nuevo. Si bien este método no es tan avanzado como usar un satélite de comunicación geosíncrono, con frecuencia tiene mucha mayor eficacia de costo, en especial con aplicaciones en las que el ancho de banda o costo por bit transportado es un factor clave.

Un cálculo simple aclarará este punto. Una cinta estándar de vídeo de 8 mm (por ejemplo, Exabyte), puede guardar 7 gigabytes. Una caja de $50 \times 50 \times 50$ cm puede contener cerca de 1000 de estas cintas, para una capacidad total de 7000 gigabytes. Federal Express y otras compañías pueden entregar una caja de cintas en cualquier lugar de Estados Unidos en 24 horas. El ancho de banda efectivo de esta transmisión es de 56 gigabytes/86,400 seg = 648 Mbps, lo cual es ligeramente mejor que la versión de alta velocidad de ATM (622 Mbps). Si el destino está sólo a una hora de camino, el ancho de banda se incrementa a más de 15 Gbps.

Respaldar diariamente los gigabytes de datos de un banco en una segunda máquina (de manera que el banco pueda continuar funcionando aun en caso de un desastre mayor o de un

terremoto) es probable que ninguna otra tecnología de transmisión pueda siquiera acercarse a la cinta magnética en rendimiento.

Si vemos ahora el costo, obtenemos una imagen similar. El costo de 1000 cintas de vídeo quizá sea de 5000 dólares comprando al mayoreo. Una cinta de vídeo se puede reutilizar al menos 10 veces, así que el costo de las cintas puede ser de 500 dólares. Añada a esto otros 200 dólares de embarque y tendrá un costo de aproximadamente 700 dólares para enviar 7000 gigabytes. Esto representa 10 centavos de dólar por cada gigabyte. Ninguna portadora de red en el mundo puede competir con esto. La moraleja de la historia es:

Nunca subestime el ancho de banda de una camioneta
llena de cintas viajando por la carretera.

2.2.2. Par trenzado

Aunque las características de ancho de banda de la cinta magnética son excelentes, las características de retardo son pobres. El tiempo de transmisión se mide en minutos o en horas, no en milisegundos. Para muchas aplicaciones se necesita una conexión en línea. El medio de transmisión más viejo y todavía el más común es el **par trenzado**. Un par trenzado consiste en dos alambres de cobre aislados, por lo regular de 1 mm de grueso. Los alambres se trenzan en forma helicoidal, igual que una molécula de DNA. El propósito de torcer los alambres es reducir la interferencia eléctrica de pares similares cercanos. (Dos alambres paralelos constituyen una antena simple; un par trenzado no.)

La aplicación más común del par trenzado es en el sistema de teléfonos. Casi todos los teléfonos se conectan a la central telefónica por un par trenzado. Se pueden tender varios kilómetros de par trenzado sin necesidad de amplificación, pero se necesitan repetidoras para distancias mayores. Cuando muchos pares entrelazados corren distancias sustanciales en paralelo, como los cables que van de un edificio de departamentos a la central telefónica, se atan en un haz y se forran con una funda que los protege. Los pares de estos haces interferirían unos con otros si no fuera por el entrelazamiento. En algunas partes del mundo donde las líneas telefónicas penden de postes sobre el terreno es común ver haces de varios centímetros de diámetro.

Los pares trenzados se pueden usar tanto para transmisión analógica como digital. El ancho de banda depende del grosor del cable y de la distancia, pero en muchos casos se pueden lograr varios megabits/seg durante algunos kilómetros. Los pares entrelazados se usan ampliamente debido a su rendimiento adecuado y a su bajo costo, y no parece que esto vaya a cambiar durante algunos años.

El cableado de par trenzado tiene algunas variaciones, dos de las cuales son importantes para las redes de computadoras. Los pares entrelazados de la **categoría 3** consisten en dos hilos aislados que se trenzan de manera delicada. Cuatro de estos pares se agrupan por lo regular en una funda de plástico para su protección y para mantener juntos los ocho hilos. Antes de alrededor de 1988, la mayor parte de los edificios de oficinas tenían un cable de categoría 3 en cada oficina que salía de un gabinete de cableado central en cada piso. Este esquema permitía

conectar hasta cuatro teléfonos ordinarios o dos teléfonos de multilínea en cada oficina al equipo de la compañía telefónica en el gabinete de cableado.

Alrededor de 1988 se comenzaron a introducir los pares trenzados más avanzados de la **categoría 5**; son similares a los de la categoría 3, pero con más vueltas por centímetro y con aislamiento de Teflón, lo cual produce menor diafonía y una señal de mejor calidad a distancias más largas, lo que los hace más adecuados para la comunicación de computadoras a alta velocidad. Ambos tipos de cableado con frecuencia reciben el nombre de **UTP** (*unshielded twisted pair,* **par trenzado sin blindaje**), en contraste con los voluminosos y costosos cables de par trenzado blindado que IBM introdujo a principios de la década de 1980 pero que nunca se popularizaron fuera de las instalaciones de IBM.

2.2.3. Cable coaxial de banda base

Otro medio de transmisión común es el **cable coaxial** (conocido por sus muchos amigos simplemente como "coax"). Este cable tiene mejor blindaje que el par trenzado, así que puede abarcar tramos más largos a velocidades mayores. Son dos las clases de cable coaxial más utilizadas. Una clase, el cable de 50 ohms, se usa comúnmente para transmisión digital y es el que estudiaremos en esta sección. La otra clase, el cable de 75 ohms, se usa comúnmente para la transmisión analógica y se describirá en la siguiente sección. Esta distinción se basa en factores históricos y no técnicos (por ejemplo, las primeras antenas de dos polos tenían una impedancia de 300 ohms, y era fácil construir transformadores con igualación de impedancia de 4:1).

Un cable coaxial consiste en un alambre de cobre rígido como núcleo, rodeado por un material aislante. El aislante está forrado con un conductor cilíndrico, que con frecuencia es una malla de tejido fuertemente trenzado. El conductor externo se cubre con una envoltura protectora de plástico. En la figura 2-3 se muestra una vista en corte por capas de un cable coaxial.

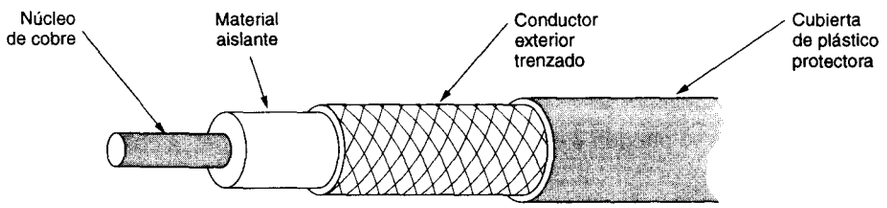


Figura 2-3. Un cable coaxial.

La construcción y el blindaje del cable coaxial le confieren una buena combinación de elevado ancho de banda y excelente inmunidad al ruido. El ancho de banda posible depende de la longitud del cable. En cables de 1 km es factible una velocidad de datos de 1 a 2 Gbps. También se pueden usar cables más largos, pero a velocidades de datos más bajas o con amplificadores periódicos. Los cables coaxiales solían usarse ampliamente en el sistema telefónico, pero ahora

se les ha reemplazado en gran medida por fibra óptica en rutas de largo recorrido. Tan sólo en Estados Unidos se instalan a diario 1000 km de fibra (si se cuenta como 1000 km un haz de 100 km con 10 ramales de fibra). Sprint ya es 100% de fibra, y las otras portadoras principales se acercan muy rápido a esa cifra. Sin embargo, el cable coaxial todavía se utiliza ampliamente para la televisión por cable y algunas redes de área local.

2.2.4. Cable coaxial de banda ancha

El otro tipo de sistema de cable coaxial transporta transmisión analógica en el cableado estándar de la televisión por cable; se le llama de **banda ancha**. Aunque el término "banda ancha" viene del mundo de la telefonía, donde se refiere a cualquier cosa más ancha que 4 kHz, en el mundo de las redes de computadoras "cable de banda ancha" significa cualquier red de cable que utilice transmisión analógica, véase (Cooper, 1986).

Puesto que las redes de banda ancha emplean tecnología estándar de la televisión por cable, los cables se pueden usar hasta 300 MHz (y con frecuencia hasta 450 MHz) y pueden extenderse distancias de cerca de 100 km gracias a la señalización analógica, que es mucho menos crítica que la digital. Para transmitir señales digitales por una red analógica, cada interfaz debe contener circuitos electrónicos para convertir la corriente de bits saliente en una señal analógica y la señal analógica entrante en una corriente de bits. Dependiendo del tipo de estos circuitos, 1 bps puede ocupar aproximadamente 1 Hz de ancho de banda. A frecuencias más altas pueden transmitirse muchos bits por Hz si se emplean técnicas de modulación avanzadas.

Los sistemas de banda ancha se dividen en múltiples canales, con frecuencia los canales de 6 MHz que se usan para la difusión de televisión. Cada canal puede servir para televisión analógica, audio de calidad de disco compacto (1.4 Mbps), o una corriente digital de bits independiente de las otras, digamos a 3 Mbps. La televisión y los datos se pueden mezclar en un cable.

Una diferencia clave entre la banda base y la banda ancha es que los sistemas de banda ancha normalmente cubren un área mayor y, por tanto, necesitan amplificadores analógicos para reforzar la señal en forma periódica. Estos amplificadores solamente pueden transmitir señales en una dirección, de modo que una computadora que esté enviando un paquete no será capaz de comunicarse con una computadora situada "corriente arriba" de ella si entre ellas hay un amplificador. Para superar este problema se han desarrollado dos tipos de sistemas de banda ancha: sistemas de cable dual y de cable sencillo.

Los sistemas de cable dual tienen dos cables idénticos que corren en paralelo, uno junto al otro. Para transmitir datos, la computadora envía los datos por el cable 1, que conduce a un dispositivo llamado ***head-end*** en la raíz del árbol de cables. A continuación, el *head-end* transfiere la señal al cable 2 para transmitirla de regreso por el árbol. Todas las computadoras transmiten por el cable 1 y reciben por el cable 2. En la figura 2-4(a) se muestra un sistema de cable dual.

El otro esquema asigna diferentes bandas de frecuencia para comunicación entrante y saliente en un cable sencillo [véase la figura 2-4(b)]. La banda de frecuencia baja se usa para la comunicación de las computadoras al *head-end*, el cual cambia después la señal a la banda de alta frecuencia y la vuelve a difundir. En el sistema **subdividido** las frecuencias de 5 a 30 MHz se usan para el tráfico entrante y las frecuencias de 40 a 300 MHz se usan para el tráfico saliente.

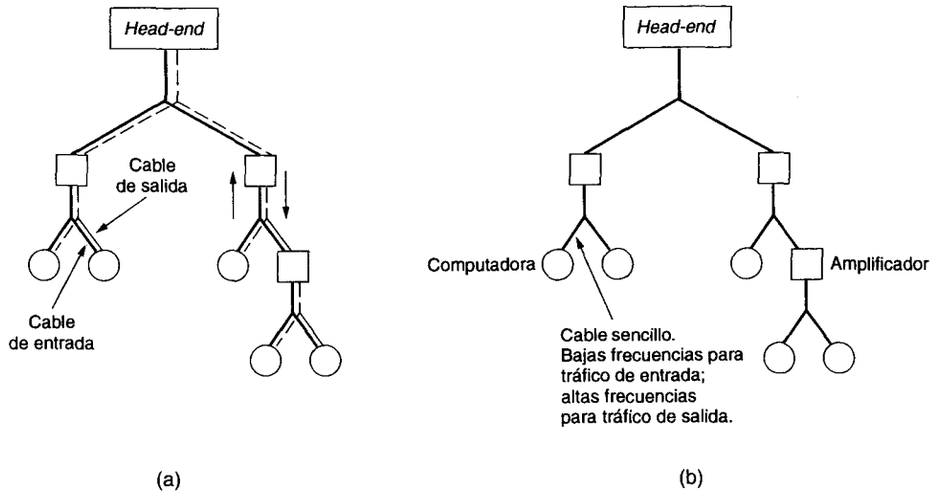


Figura 2-4. Redes de banda ancha. (a) Cable dual. (b) Cable sencillo.

En el sistema **dividido por la mitad**, la banda de entrada es de 5 a 116 MHz y la de salida es de 168 a 300 MHz. La elección de estas bandas de frecuencia es histórica, y tiene que ver con la forma en que la Comisión Federal de Comunicaciones de Estados Unidos asignó frecuencias a la difusión de televisión, para la cual se diseñó la banda ancha. Ambos sistemas divididos requieren un *head-end* activo que acepte señales de entrada por una banda y las vuelva a difundir por otra. Estas técnicas y frecuencias se desarrollaron para la televisión por cable y se han adoptado para las redes sin modificación debido a la disponibilidad de *hardware* confiable y relativamente barato.

La banda ancha se puede usar de varias maneras. Se puede asignar un canal permanente a algunos pares de computadoras para su uso exclusivo. Otras computadoras pueden solicitar un canal para una conexión temporal a través de un canal de control y luego cambiar sus frecuencias a ese canal mientras dura la conexión. Otro arreglo consiste en hacer que todas las computadoras compitan por el acceso a un canal o a un grupo de canales utilizando técnicas que se verán en el capítulo 4.

Técnicamente, el cable de banda ancha es inferior al de banda base (es decir, un solo canal) para enviar datos digitales, pero tiene la ventaja de que ya se ha instalado una gran cantidad de él. Por ejemplo, en Holanda el 90% de los hogares tiene una conexión de televisión por cable. En Estados Unidos, los cables de televisión pasan cerca de más del 80% de los hogares. Alrededor del 60% de éstos tiene realmente una conexión de cable. Con la competencia entre compañías de teléfonos y compañías de televisión por cable en su apogeo, podemos esperar que los sistemas de televisión por cable empiecen a operar como MAN y a ofrecer servicios telefónicos y de otro tipo con frecuencia creciente. Para mayor información en el uso de la televisión por cable como red de computadoras, véase (Karshmer y Thomas, 1992).

2.2.5. Fibra óptica

Mucha gente en la industria de la computación siente un orgullo enorme por la rapidez con que está mejorando la tecnología de las computadoras. En la década de 1970, una computadora rápida (por ejemplo, la CDC 6600) podía ejecutar una instrucción en 100 nseg. Veinte años después, una computadora Cray rápida podía ejecutar una instrucción en 1 nseg, un factor de mejora de 10 por década. No está tan mal.

En el mismo periodo, la comunicación de datos pasó de 56 kbps (la ARPANET) a 1 Gbps (la comunicación óptica moderna), un factor de ganancia de más de 100 por década, mientras que la tasa de errores se redujo de 10^{-5} por bit a casi cero.

Más todavía, las CPU individuales están empezando a acercarse a los límites físicos, como la velocidad de la luz y los problemas de disipación de calor. En contraste, con la tecnología *actual* de fibras, el ancho de banda asequible ciertamente excede los 50,000 Gbps (50 Tbps) y muchos investigadores se están esforzando por encontrar mejores materiales. El límite práctico de señalización actual de cerca de 1 Gbps se debe a nuestra incapacidad para convertir con mayor rapidez las señales eléctricas a ópticas. En el laboratorio, es factible obtener 100 Gbps en transmisiones cortas. Faltan pocos años para que se pueda alcanzar una velocidad de 1 terabit/s. Los sistemas totalmente ópticos, que incluyen entradas y salidas ópticas de la computadora, están al alcance (Miki, 1994a).

En la carrera entre las computadoras y la comunicación, ganó la comunicación. La actual generación de científicos e ingenieros de computadoras enseñados a pensar en términos de los bajos límites de Nyquist y Shannon impuestos por el alambre de cobre todavía no ha comprendido todas las implicaciones del ancho de banda prácticamente infinito (aunque no sin costo). El nuevo acuerdo convencional debería ser que todas las computadoras son irremediablemente lentas y que las redes deben tratar de evitar la computación a toda costa, sin importar qué tanto ancho de banda se desperdicie con eso. Estudiaremos la óptica de fibras en esta sección para ver cómo funciona esa tecnología de transmisión.

Un sistema de transmisión óptico tiene tres componentes: la fuente de luz, el medio transmisor y el detector. Convencionalmente, un pulso de luz indica un bit 1 y la ausencia de luz indica un bit 0. El medio de transmisión es una fibra de vidrio ultradelgada. El detector genera un pulso eléctrico cuando la luz incide en él. Al conectar una fuente de luz en un extremo de una fibra óptica y un detector en el otro, tenemos un sistema de transmisión de datos unidireccional que acepta una señal eléctrica, la convierte y la transmite por pulsos de luz, y después reconvierte la salida a una señal eléctrica en el extremo receptor.

Este sistema de transmisión tendría fugas de luz y sería inútil en la práctica excepto por un principio interesante de la física. Cuando un rayo de luz pasa de un medio a otro, por ejemplo de sílice fundida al aire, el rayo se refracta (se dobla) en la frontera de la sílice y el aire, como se muestra en la figura 2-5. Vemos aquí que un rayo de luz que incide en la frontera con un ángulo α_1 emerge con un ángulo β_1. El grado de refracción depende de las propiedades de los dos medios (en particular, de sus índices de refracción). Para ángulos de incidencia por encima de cierto valor crítico, la luz se refracta de regreso a la sílice; ninguna función escapa hacia el aire. Así, un rayo incidente con un ángulo igual o mayor que el crítico queda atrapado dentro de la

fibra, según se muestra en la figura 2-5(b), y se puede propagar por muchos kilómetros virtualmente sin pérdidas.

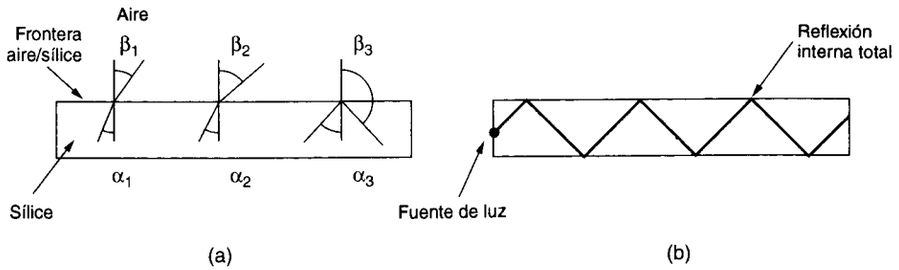


Figura 2-5. Tres ejemplos de rayo de luz procedente del interior de una fibra de sílice que incide sobre la frontera aire/sílice con diferentes ángulos. (b) Luz atrapada por reflexión interna total.

El diagrama de la figura 2-5(b) únicamente muestra un rayo atrapado, pero puesto que cualquier rayo de luz que incida en la frontera con un ángulo mayor que el crítico se reflejará internamente, muchos rayos estarán rebotando dentro de la sílice con ángulos diferentes. Se dice que cada rayo tiene un **modo** diferente, y una fibra que tiene esta propiedad se denomina **fibra multimodal**.

Por otro lado, si se reduce el diámetro de la fibra a unas cuantas longitudes de onda de la luz, la fibra actúa como una guía de ondas y la luz se puede propagar sólo en línea recta, sin rebotar, obteniéndose una **fibra de modo único**. Las fibras de modo único son más caras pero se pueden utilizar en distancias más grandes. Las fibras de modo único disponibles en la actualidad pueden transmitir datos a varios Gbps a una distancia de 30 km. En el laboratorio se han logrado velocidades de datos todavía mayores a distancias más cortas. Los experimentos han demostrado que los láseres potentes pueden impulsar una señal por una fibra a 100 km sin repetidoras, aunque a velocidades más bajas. Las investigaciones sobre fibras contaminadas con erbio prometen alcances todavía mayores sin repetidoras.

Transmisión de la luz a través de fibras

Las fibras ópticas se hacen de vidrio, que a su vez se fabrica con arena, una materia prima de bajo costo disponible en cantidades ilimitadas. La fabricación del vidrio era conocida por los antiguos egipcios, pero su vidrio no podía ser mayor de 1 mm de grueso para que la luz lograra atravesarlo. Durante el Renacimiento se desarrolló un vidrio lo bastante transparente como para ser útil en las ventanas. El vidrio que se utiliza en las fibras ópticas modernas es tan transparente que si los océanos estuvieran llenos de él en lugar de agua el lecho marino sería tan visible desde la superficie como en un día claro lo es el suelo desde un avión.

La atenuación de la luz dentro del vidrio depende de la longitud de onda de la luz. En la figura 2-6 se muestra la atenuación para la clase de vidrio que se usa en las fibras, en decibeles por kilómetro lineal de fibra. La atenuación en decibeles está dada por la fórmula

$$\text{Atenuación en decibeles} = 10 \log_{10} \frac{\text{poder transmitido}}{\text{poder recibido}}$$

Por ejemplo, un factor de pérdida de dos da una atenuación de $10 \log_{10} 2 = 3$ dB. La figura muestra la parte cercana al infrarrojo del espectro, que es la que se usa en la práctica. La luz visible tiene longitudes de onda ligeramente más cortas, de 0.4 a 0.7 micras (1 micra = 10^{-6} metros).

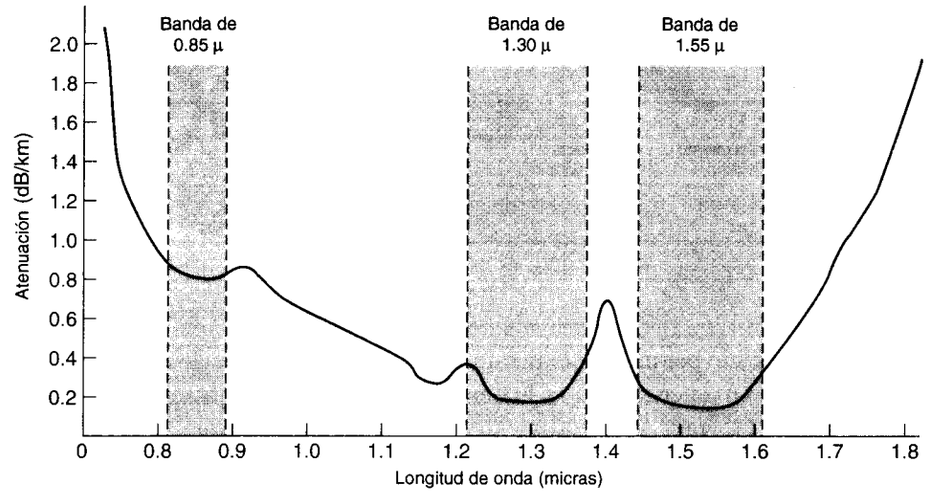


Figura 2-6. Atenuación de la luz dentro de una fibra en la región de infrarrojo.

Para las comunicaciones se utilizan tres bandas de longitud de onda, las cuales se centran respectivamente en 0.85, 1.30 y 1.55 micras. Las últimas dos tienen buenas propiedades de atenuación (una pérdida de menos del 5% por kilómetro). La banda de 0.85 micras tiene una atenuación más alta pero la propiedad conveniente de que a esa longitud de onda los láseres y los componentes electrónicos se pueden fabricar con el mismo material (arsenuro de galio). Las tres bandas tienen una anchura de entre 25,000 y 30,000 GHz.

La longitud de los pulsos de luz transmitidos por una fibra aumenta conforme se propagan. Este fenómeno se llama **dispersión**, y su magnitud depende de la longitud de onda. Una forma de evitar que se encimen los pulsos dispersos es incrementar la distancia entre ellos, pero esto solamente se puede hacer reduciendo la velocidad de emisión de las señales. Por fortuna, se ha descubierto que al dar a los pulsos cierta forma especial relacionada con el recíproco del coseno hiperbólico, todos los efectos de la dispersión se cancelan y puede ser posible enviar pulsos a miles de kilómetros sin una distorsión apreciable de la forma. Estos pulsos se llaman **solitones**. Se está realizando una cantidad considerable de investigaciones para llevar a la práctica las soluciones obtenidas en el laboratorio.

Cables de fibras

Los cables de fibra óptica son similares a los coaxiales, excepto por el trenzado. La figura 2-7(a) muestra una fibra individual vista de lado. El núcleo de vidrio está al centro, y a través de él se propaga la luz. En las fibras multimodales el diámetro es de 50 micras, aproximadamente el grosor de un cabello humano. En las fibras de modo único el núcleo es de 8 a 10 micras.

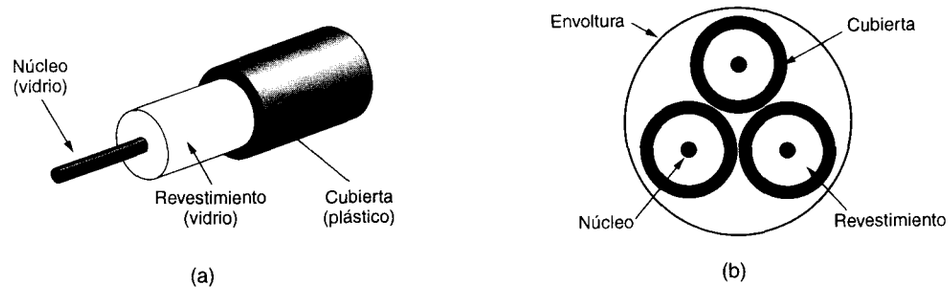


Figura 2-7. (a) Vista de lado de una fibra individual. (b) Vista de extremo de una envoltura con tres fibras.

El núcleo está rodeado por un revestimiento de vidrio con un índice de refracción menor que el del núcleo, a fin de mantener toda la luz en el núcleo. A continuación viene una cubierta plástica delgada para proteger al revestimiento. Las fibras normalmente se agrupan en haces, protegidas por una funda exterior. La figura 2-7(b) muestra una funda con tres fibras.

Los haces de fibras terrestres por lo regular se colocan un metro debajo de la superficie, donde ocasionalmente están sujetos a ataques de azadones o picos. Cerca de la costa, los haces de fibras transoceánicos se entierran en zanjas con una especie de arado marino. En aguas profundas, simplemente descansan en el fondo, donde las redes de pesca por arrastre pueden tropezar con ellas o los tiburones pueden morderlas.

Las fibras se pueden conectar de tres formas diferentes. Primera, pueden terminar en conectores e insertarse en enchufes de fibra. Los conectores pierden casi el 10 o 20% de la luz, pero facilitan la reconfiguración de los sistemas.

Segunda, se pueden empalmar de manera mecánica. Los empalmes mecánicos acomodan dos extremos cortados con cuidado uno junto a otro en una manga especial y los sujetan en su lugar. Se puede mejorar la alineación haciendo pasar luz por la unión y efectuando pequeños ajustes para maximizar la señal. Los empalmes mecánicos toman al personal entrenado cerca de 5 minutos y resultan en una pérdida de luz del 10 por ciento.

Tercera, se pueden fusionar (fundir) dos tramos de fibra para formar una conexión sólida. Un empalme por fusión es casi tan bueno como una fibra de hilado único, pero aun aquí hay un poco de atenuación. Con los tres tipos de empalme pueden ocurrir reflejos en el punto del empalme, y la energía reflejada puede interferir la señal.

Se pueden utilizar dos clases de fuente de luz para producir las señales, LED (diodos emisores de luz) y láseres semiconductores. Estas fuentes tienen propiedades diferentes, como

se muestra en la figura 2-8 y se pueden afinar en lo que respecta a la longitud de onda insertando interferómetros Fabry-Perot o Mach-Zehnder entre la fuente y la fibra. Los interferómetros Fabry-Perot son simples cavidades resonantes que consisten en dos espejos paralelos. La luz incide en los espejos de manera perpendicular. La cavidad separa las longitudes de onda que caben en ella de un número entero de veces. Los interferómetros Mach-Zehnder dividen la luz en dos rayos, los cuales viajan distancias ligeramente distintas. Los rayos se recombinan en un extremo y quedan en fase solamente para ciertas longitudes de onda.

Característica	LED	Semiconductor láser
Velocidad de datos	Baja	Alta
Modo	Multimodo	Multimodo o modo único
Distancia	Corta	Larga
Tiempo de vida	Vida larga	Vida corta
Sensibilidad a la temperatura	Baja	Considerable
Costo	Bajo	Elevado

Figura 2-8. Comparación de diodos semiconductores y LED como fuentes de luz.

El extremo receptor de una fibra óptica consiste en un fotodiodo que emite un pulso eléctrico cuando lo golpea la luz. El tiempo de respuesta normal de los fotodiodos es de 1 ns, lo que limita la velocidad de datos a cerca de 1 Gbps. El ruido térmico es otro inconveniente, por lo que un pulso de luz debe llevar energía suficiente para ser detectable. Si la potencia de los pulsos es suficiente, la relación de errores se puede hacer arbitrariamente pequeña.

Redes de fibra óptica

Las fibras ópticas se pueden utilizar con LAN, así como para transmisión de largo alcance, aunque derivar en ella es más complicado que conectarse a una Ethernet. Una forma de superar el problema es reconocer que una red de anillo es en realidad una colección de enlaces punto a punto, como se muestra en la figura 2-9. La interfaz en cada computadora pasa la corriente de pulsos de luz hacia el siguiente enlace y también sirve como unión T para que la computadora pueda enviar y aceptar mensajes.

Se usan dos tipos de interfaz. Una interfaz pasiva consiste en dos derivaciones fusionadas a la fibra principal. Una derivación tiene un LED o un diodo láser en su extremo (para transmitir) y el otro tiene un fotodiodo (para recibir). La derivación misma es pasiva por completo y por lo mismo es extremadamente confiable pues un LED o fotodiodo descompuesto no romperá el anillo, sólo dejará fuera de línea a una computadora.

El otro tipo de interfaz, mostrado en la figura 2-9, es el **repetidor activo**. La luz entrante se convierte en una señal eléctrica que se regenera a su intensidad completa si se debilitó y se

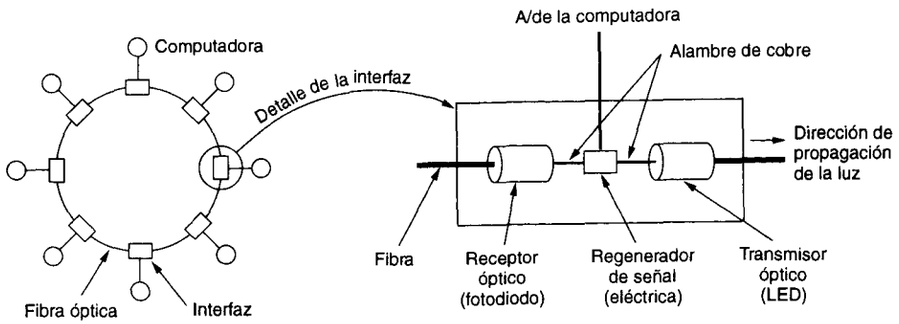


Figura 2-9. Anillo de fibra óptica con repetidores activos.

retransmite como luz. La interfaz con la computadora es un alambre ordinario de cobre que entra en el regenerador de señales. También se usan ahora repetidores puramente ópticos. Estos dispositivos no requieren las conversiones óptica a eléctrica a óptica, lo que significa que pueden operar con anchos de banda extremadamente altos.

Si falla un repetidor activo, el anillo se rompe y la red se cae. Por otro lado, puesto que la señal se regenera en cada interfaz, los enlaces individuales de computadora a computadora pueden tener una longitud de kilómetros, virtualmente sin un límite para el tamaño total del anillo. Las interfaces pasivas pierden luz en cada unión, de modo que la cantidad de computadoras y la longitud total del anillo se restringen considerablemente.

La topología de anillo no es la única manera de construir una LAN con fibra óptica. También es posible tener difusión por *hardware* utilizando la construcción de **estrella pasiva** de la figura 2-10. En este diseño, cada interfaz tiene una fibra que corre desde su transmisor hasta un cilindro de sílice, con las fibras entrantes fusionadas a un extremo del cilindro. De forma similar, las fibras fusionadas en el otro extremo del cilindro corren hacia cada uno de los receptores. Siempre que una interfaz emite un pulso de luz, se difunde dentro de la estrella pasiva para iluminar a todos los receptores, logrando de este modo la difusión. En efecto, la estrella pasiva combina todas las señales que entran y transmite el resultado combinado por todas las líneas. Puesto que la energía entrante se divide entre todas las líneas que salen, la cantidad de nodos en la red está limitada por la sensibilidad de los fotodiodos.

Comparación de la fibra óptica y el alambre de cobre

Es instructivo comparar la fibra con el cobre. La fibra tiene muchas ventajas. Para empezar, puede manejar anchos de banda mucho más grandes que el cobre. Solamente por esto sería indispensable su uso en redes de alto rendimiento. Debido a la baja atenuación, sólo se necesitan repetidoras a cada 30 km aproximadamente en líneas largas, contra casi cada 5 km cuando se usa cobre, lo que implica un ahorro sustancial. La fibra también tiene la ventaja de no ser afectada por las elevaciones en la carga, la interferencia electromagnética o los cortes en el suministro de

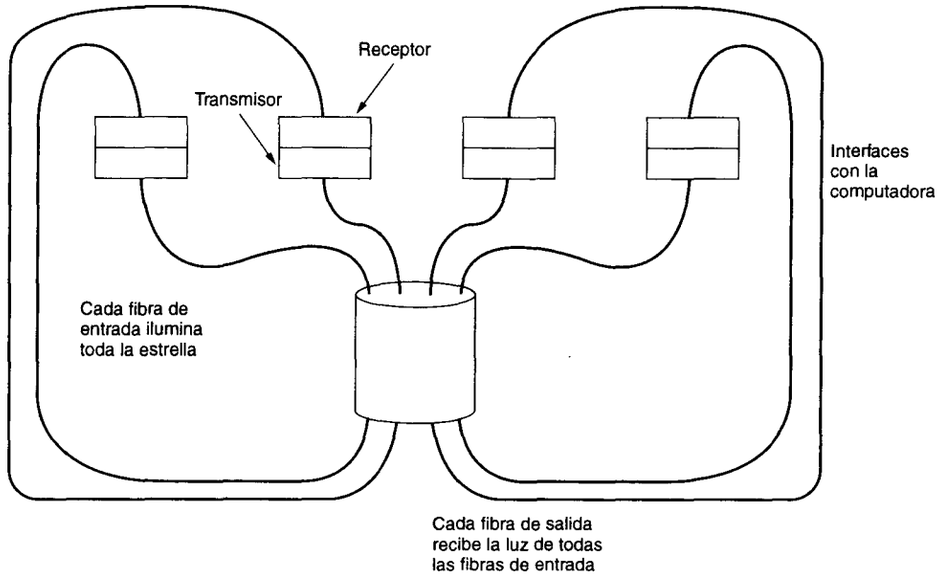


Figura 2-10. Conexión en estrella pasiva en una red de fibra óptica.

energía. Tampoco es afectada por las sustancias corrosivas del ambiente, lo que la hace ideal para ambientes fabriles pesados.

Extrañamente, las compañías de teléfonos son partidarias de la fibra por una razón diferente: es delgada y ligera. Muchos conductos de cables existentes están llenos por completo, de modo que no hay espacio para añadir más capacidad. Al retirar todo el cobre y reemplazarlo por fibras, se vacían los conductos, y el cobre tiene un excelente valor de reventa para las refinerías de cobre, que lo ven como materia prima de alto grado. Además, la fibra es más ligera que el cobre. Mil pares trenzados de 1 km de longitud pesan 8000 kg. Dos fibras tienen mayor capacidad y únicamente pesan 100 kg, lo que reduce en gran medida la necesidad de costosos sistemas mecánicos de apoyo a los que deben dar mantenimiento. Para rutas nuevas, la fibra es la opción obvia debido a su costo de instalación mucho más bajo.

Por último, las fibras no tienen fugas de luz y es difícil intervenirlas. Esto les confiere excelente inmunidad contra espías potenciales.

La razón por la que la fibra es mejor que el cobre es inherente en la física subyacente. Cuando los electrones se desplazan por un alambre, se afectan unos a otros y ellos mismos son afectados por electrones de fuera del alambre. Los fotones en una fibra no se afectan entre sí (no tienen carga eléctrica) y no resultan afectados por haces de fotones externos a la fibra.

Por el lado negativo, la fibra es una tecnología poco familiar que requiere habilidades que la mayoría de los ingenieros no tiene. Como por su naturaleza la transmisión óptica es unidireccional, la comunicación en ambos sentidos requiere ya sea dos fibras o dos bandas de frecuencia en una

fibra. Por último, las interfaces de fibra cuestan más que las de cobre. No obstante, el futuro de toda la comunicación fija de datos a distancias de algo más que unos cuantos metros claramente está en la fibra. Para un análisis detallado de todos los aspectos de las redes de fibra óptica, véase (Green, 1993).

2.3. TRANSMISIÓN INALÁMBRICA

Nuestra era ha dado origen a adictos a la información: gente que necesita estar todo el tiempo en línea. Para estos usuarios móviles, los pares trenzados, los cables coaxiales y la fibra óptica no son útiles; necesitan obtener los datos para sus computadoras laptop, *notebook,* de bolsillo, de mano o de reloj pulsera sin estar atados a la infraestructura de comunicaciones terrestres. Para estos usuarios, la respuesta es la comunicación inalámbrica. En esta sección veremos la comunicación inalámbrica en general, pues tiene muchas otras aplicaciones importantes además de proveer conectividad a los usuarios que quieren leer su correo electrónico en los aviones.

Algunas personas incluso creen que en el futuro sólo habrá dos clases de comunicación: de fibra e inalámbrica. Todos los aparatos fijos (esto es, no móviles): computadoras, teléfonos, faxes y demás, se conectarán con fibra; todos los móviles usarán comunicación inalámbrica.

Sin embargo, la comunicación inalámbrica también tiene ventajas para los dispositivos fijos en ciertas circunstancias. Por ejemplo, si es difícil tender fibras hasta un edificio debido al terreno (montañas, selvas, pantanos, etc.), podría ser preferible un sistema inalámbrico. Es interesante que la comunicación digital inalámbrica moderna se inició en las islas de Hawai, donde largos tramos del Océano Pacífico separaban a los usuarios y el sistema telefónico era inadecuado.

2.3.1. El espectro electromagnético

Cuando los electrones se mueven crean ondas electromagnéticas que se pueden propagar por el espacio libre (aun en el vacío). El físico británico James Clerk Maxwell predijo estas ondas en 1865 y el físico alemán Heinrich Hertz las produjo y observó por primera vez en 1887. La cantidad de oscilaciones por segundo de una onda electromagnética es su **frecuencia**, f, y se mide en **Hz** (en honor de Heinrich Hertz). La distancia entre dos máximos (o mínimos) consecutivos se llama **longitud de onda** y se designa de forma universal con la letra griega λ (lambda).

Al conectarse una antena del tamaño apropiado a un circuito eléctrico, las ondas electromagnéticas se pueden difundir de manera eficiente y captarse por un receptor a cierta distancia. Toda la comunicación inalámbrica se basa en este principio.

En el vacío, todas las ondas electromagnéticas viajan a la misma velocidad, no importa cuál sea su frecuencia. Esta velocidad, usualmente llamada **velocidad de la luz**, c, es de aproximadamente 3×10^8 m/seg, o de casi 1 pie (30 cm) por nanosegundo. En el cobre o en la fibra, la velocidad baja a casi 2/3 de este valor y se vuelve ligeramente dependiente de la frecuencia. La

velocidad de la luz es el límite máximo de velocidad. Ningún objeto o señal puede llegar a ser más rápido que la luz.

La relación fundamental entre f, λ y c (en el vacío) es:

$$\lambda f = c \tag{2-2}$$

Puesto que c es una constante, si conocemos f podemos encontrar λ y viceversa. Por ejemplo, las ondas de 1 MHz tienen una longitud de alrededor de 300 m y las ondas de 1 cm tienen una frecuencia de 30 GHz.

En la figura 2-11 se muestra el espectro electromagnético. Las porciones de radio, microondas, infrarrojo y luz visible del espectro pueden servir para transmitir información modulando la amplitud, la frecuencia o la fase de las ondas. La luz ultravioleta, los rayos X y los rayos gamma serían todavía mejores, debido a sus frecuencias más altas, pero son difíciles de producir y de modular, no se propagan bien entre edificios y son peligrosos para los seres vivos. Las bandas que se listan en la parte inferior de la figura 2-11 son los nombres oficiales de la ITU y se basan en las longitudes de onda, de modo que la banda LF va de 1 a 10 km (aproximadamente 30 a 300 kHz). Los términos LF, MF y HF se refieren a las frecuencias baja, media y alta, respectivamente. Queda claro que, cuando se asignaron los nombres, nadie esperaba rebasar los 10 MHz, así que las bandas más altas se denominaron después bandas de muy, ultra, super, extremadamente y tremendamente alta frecuencia. Más allá de eso ya no hay nombres, pero las designaciones increíblemente, asombrosamente y prodigiosamente alta frecuencia sonarían bien (IHF, AHF y PHF).

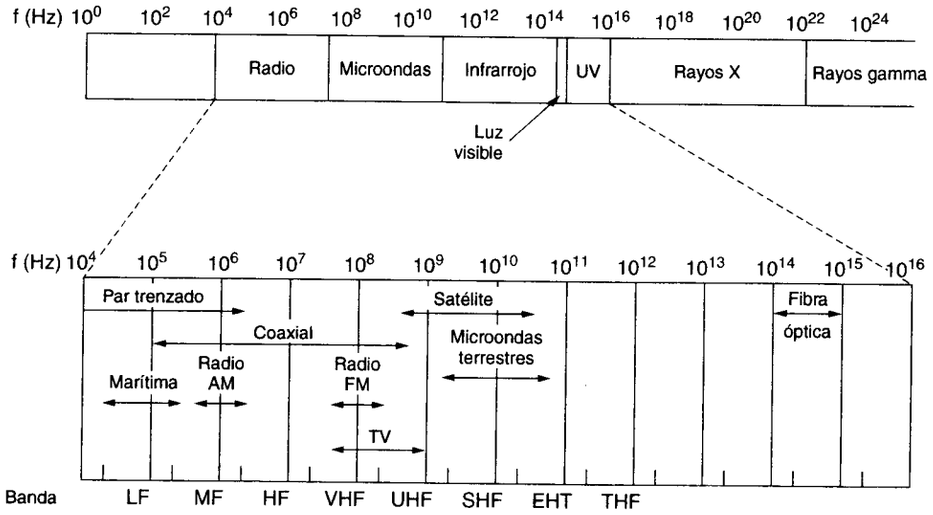


Figura 2-11. El espectro electromagnético y sus usos para comunicaciones.

La cantidad de información que puede llevar una onda electromagnética se relaciona con su ancho de banda. Con la tecnología actual, es posible codificar unos cuantos bits por hertz a frecuencias bajas, pero a frecuencias altas el número puede llegar a 40 en ciertas condiciones, de modo que un cable con un ancho de banda de 500 MHz puede transportar varios gigabits/s. La figura 2-11 debe dejar en claro ahora por qué a la gente de redes le gusta tanto la fibra óptica.

Si resolvemos la ecuación (2-2) para f y la diferenciamos con respecto a λ obtenemos:

$$\frac{df}{d\lambda} = -\frac{c}{\lambda^2}$$

Si ahora usamos diferencias finitas en lugar de diferenciales y sólo consideramos los valores absolutos, obtenemos

$$\Delta f = \frac{c\Delta\lambda}{\lambda^2} \tag{2-3}$$

Establecido de este modo el ancho de una banda de longitud de onda, $\Delta\lambda$, podemos calcular la banda de frecuencia correspondiente Δf, y con ella, la velocidad de transmisión de datos que puede producir la banda. Cuanto más ancha sea la banda, mayor será la velocidad de transmisión de los datos. A modo de ejemplo, considere la banda de 1.30 micras de la figura 2-6. Aquí tenemos $\lambda = 1.3 \times 10^{-6}$ y $\Delta\lambda = 0.17 \times 10^{-6}$, de manera que Δf es de cerca de 30 THz.

Para prevenir el caos total, hay acuerdos nacionales e internacionales acerca de quién puede usar cuáles frecuencias. Puesto que todos quieren una velocidad de datos más alta, todos quieren más espectro. En Estados Unidos, la FCC asigna el espectro para radio AM y FM, televisión y teléfonos celulares, así como a las compañías telefónicas, policía, marina, navegación, ejército, gobierno y muchos otros usuarios competidores. A nivel mundial, una agencia de la ITU-R (WARC), se encarga de esto. Por ejemplo, en la junta de España, en 1991, la WARC asignó parte del espectro a los comunicadores personales portátiles. Desafortunadamente, la FCC, que no está limitada por las recomendaciones de la WARC, escogió una parte diferente (porque la gente que tiene la banda elegida por la WARC en Estados Unidos no quiso dejarla y tuvo el peso político suficiente para conservarla). En consecuencia, los comunicadores personales construidos para el mercado de Estados Unidos no funcionan en Europa o en Asia y viceversa.

La mayor parte de las transmisiones ocupa una banda estrecha de frecuencias (esto es, $\Delta f / f \ll 1$) a fin de obtener la mejor recepción (muchos watts/Hz). Sin embargo, en algunos casos el transmisor salta de frecuencia a frecuencia siguiendo un patrón regular o las transmisiones se dispersan intencionalmente en una banda ancha de frecuencia. Esta técnica se denomina **espectro disperso** (Kohno *et al.*, 1995) y es popular en la comunicación militar porque hace que las transmisiones sean difíciles de detectar y casi imposibles de sabotear. El salto de frecuencias no tiene mucho interés para nosotros (fuera de señalar que la actriz de cine Hedy Lamarr fue su coinventora). El espectro disperso, a veces llamado **espectro disperso de secuencia directa**, está ganando popularidad en el mundo comercial, y regresaremos a él en el capítulo 4. Para una historia fascinante y detallada de las comunicaciones de espectro disperso, véase (Scholtz, 1982).

Por el momento, supondremos que todas las transmisiones usan una banda estrecha de frecuencia. Ahora veremos cómo se usan las distintas partes del espectro, comenzando por la radio.

2.3.2. Radiotransmisión

Las ondas de radio son fáciles de generar, pueden viajar distancias largas y penetrar edificios sin problemas, de modo que se utilizan mucho en la comunicación, tanto en interiores como en exteriores. Las ondas de radio también son omnidireccionales, lo que significa que viajan en todas direcciones desde la fuente, por lo que el transmisor y el receptor no tienen que alinearse con cuidado físicamente.

En ocasiones, la radio omnidireccional es buena, pero otras veces no lo es tanto. En la década de 1970, la General Motors decidió equipar sus Cadillacs nuevos con frenos antibloqueo controlados por computadora. Cuando el conductor pisaba el pedal del freno, la computadora accionaba los frenos en lugar de bloquearlos firmemente. Un buen día, un patrullero de las carreteras de Ohio encendió su nuevo radio móvil para llamar a su cuartel general; de repente, el Cadillac que iba junto a él empezó a comportarse como un potro salvaje. Cuando el oficial hizo parar el carro a un lado del camino, el conductor alegó que él nada había hecho y que el carro se había vuelto loco.

Con el tiempo, empezó a surgir un patrón: los Cadillacs a veces se volvían locos, pero sólo en las principales carreteras de Ohio y sólo cuando la patrulla de caminos iba vigilando. Durante mucho tiempo, la General Motors no pudo comprender por qué los Cadillacs funcionaban bien en todos los demás estados y también en los caminos secundarios de Ohio. Únicamente después de una búsqueda considerable descubrieron que el cableado de los Cadillacs formaba una excelente antena para la frecuencia que usaba el nuevo sistema de radio de las patrullas de caminos de Ohio.

Las propiedades de las ondas de radio dependen de la frecuencia. A bajas frecuencias, las ondas de radio cruzan bien los obstáculos, pero la potencia se reduce drásticamente con la distancia a la fuente, aproximadamente en proporción $1/r^3$ en el aire. A frecuencias altas, las ondas de radio tienden a viajar en línea recta y a rebotar en los obstáculos. También son absorbidas por la lluvia. En todas las frecuencias, las ondas de radio están sujetas a interferencia por los motores y otros equipos eléctricos.

Por la capacidad del radio de viajar distancias largas, la interferencia entre usuarios es un problema. Por esta razón, los gobiernos legislan estrictamente el uso de radiotransmisores, con una excepción (que veremos más adelante).

En las bandas VLF, LF y MF, las ondas de radio siguen el terreno, como se ilustra en la figura 2-12(a). Estas ondas se pueden detectar quizás a 1000 km en las frecuencias más bajas, y a menos en frecuencias más altas. La difusión de radio AM usa la banda MF, y es por ello que las estaciones de radio AM de Boston no se pueden oír con facilidad en Nueva York. Las ondas de radio en estas bandas cruzan con facilidad los edificios, y es por ello que los radios portátiles funcionan en interiores. El problema principal al usar estas bandas para comunicación de datos es el ancho de banda relativamente bajo que ofrecen [véase la ecuación (2-2)].

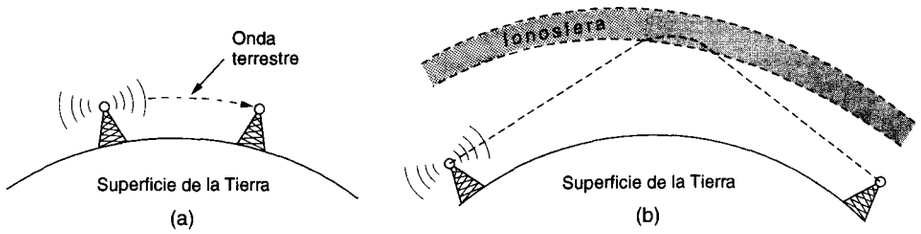


Figura 2-12. (a) En las bandas VLF, VF y MF, las ondas de radio siguen la curvatura de la Tierra. (b) En la banda HF las ondas rebotan en la ionosfera.

En las bandas HF y VHF, las ondas a nivel del suelo tienden a ser absorbidas por la Tierra. Sin embargo, las ondas que alcanzan la ionosfera, una capa de partículas cargadas que rodea a la Tierra a una altura de 100 a 500 km, se refractan y se envían de regreso a nuestro planeta, como se muestra en la figura 2-12(b). En ciertas condiciones atmosféricas, las señales pueden rebotar varias veces. Los operadores de radio aficionados usan estas bandas para conversar a larga distancia. El ejército se comunica también en las bandas HF y VHF.

2.3.3. Transmisión por microondas

Por encima de los 100 MHz las ondas viajan en línea recta y, por tanto, se pueden enfocar en un haz estrecho. Concentrar toda la energía en un haz pequeño con una antena parabólica (como el tan familiar plato de televisión por satélite) produce una señal mucho más alta en relación con el ruido, pero las antenas transmisora y receptora deben estar muy bien alineadas entre sí. Además, esta direccionalidad permite a transmisores múltiples alineados en una fila comunicarse con receptores múltiples en fila, sin interferencia. Antes de la fibra óptica, estas microondas formaron durante décadas el corazón del sistema de transmisión telefónica de larga distancia. De hecho, el nombre de la empresa de telecomunicaciones de larga distancia MCI proviene de Microwave Communications, Inc., porque su sistema entero se basó originalmente en torres de microondas (desde entonces ha modernizado las principales porciones de su red empleando fibras).

Ya que las microondas viajan en línea recta, si las torres están muy separadas, partes de la Tierra estorbarán (piense en un enlace de San Francisco a Amsterdam). En consecuencia, se necesitan repetidoras periódicas. Cuanto más altas sean las torres, más separadas pueden estar. La distancia entre las repetidoras se eleva en forma muy aproximada con la raíz cuadrada de la altura de las torres. Con torres de 100 m de altura, las repetidoras pueden estar espaciadas a 80 km de distancia.

A diferencia de las ondas de radio a frecuencias más bajas, las microondas no atraviesan bien los edificios. Además, aun cuando el haz puede estar bien enfocado en el transmisor, hay

cierta divergencia en el espacio. Algunas ondas pueden refractarse en las capas atmosféricas más bajas y tardar un poco más en llegar que las ondas directas. Las ondas diferidas pueden llegar fuera de fase con la onda directa y cancelar así la señal. Este efecto se llama **desvanecimiento de trayectoria múltiple** y con frecuencia es un problema serio que depende del clima y de la frecuencia. Algunos operadores mantienen el 10% de sus canales inactivos como repuesto para activarlos cuando el desvanecimiento de trayectoria múltiple cancela en forma temporal alguna banda de frecuencia.

La creciente demanda de espectro obliga a mejorar continuamente la tecnología de modo que las transmisiones puedan usar frecuencias todavía más altas. Las bandas de hasta 10 Ghz son ahora de uso rutinario, pero a casi 8 GHz se presenta un nuevo problema: la absorción del agua. Estas ondas sólo tienen unos centímetros de longitud y la lluvia las absorbe. Este efecto sería benéfico si se quisiera construir un enorme horno de microondas para exteriores, pero para la comunicación es un problema grave. Al igual que con el desvanecimiento de trayectoria múltiple, la única solución es apagar los enlaces afectados por la lluvia y enrutarlos dando un rodeo.

En síntesis, la comunicación por microondas se utiliza tanto para la comunicación telefónica de larga distancia, los teléfonos celulares, la distribución de la televisión y otros usos, que el espectro se ha vuelto muy escaso. Esta tecnología tiene varias ventajas significativas respecto a la fibra. La principal es que no se necesita derecho de paso; basta comprar un terreno pequeño cada 50 km y construir en él una torre de microondas para saltarse el sistema telefónico y comunicarse en forma directa. Así es como MCI logró establecerse como una compañía nueva de teléfonos de larga distancia tan rápidamente. (Sprint siguió un camino diferente: la fundó el ferrocarril Southern Pacific Railroad, que ya poseía una gran cantidad de derechos de paso, limitándose a enterrar la fibra junto a las vías.)

Las microondas también son relativamente baratas. Erigir dos torres sencillas (quizá sólo postes grandes con cuatro cables de retén) y poner antenas en cada uno puede costar menos que enterrar 50 km de fibra a través de un área urbana congestionada o sobre una montaña, y también puede ser más económico que rentar la fibra de la compañía de teléfonos, en especial si la compañía de teléfonos aún no paga por completo el cobre que quitó cuando instaló la fibra.

Además de servir para transmisión de larga distancia, las microondas tienen otro uso importante, a saber, las bandas **industriales**, **médicas** y **científicas**. Estas bandas constituyen la excepción a la regla de las licencias: los transmisores que usan estas bandas no requieren licencia del gobierno. Hay una banda asignada mundialmente: de 2.400 a 2.484 GHz. Además, en Estados Unidos y Canadá existen bandas de 902 a 928 MHz y de 5.725 a 5.850 GHz. Estas bandas se utilizan para teléfonos sin cordón, controles electrónicos de puertas de cocheras, altavoces inalámbricos de alta fidelidad, puertas de seguridad, etc. La banda de 900 MHz es la que funciona mejor pero está muy poblada y el equipo para usarla sólo se puede operar en América del Norte. Las bandas más altas requieren circuitos electrónicos más costosos y están sujetas a la interferencia de los hornos de microondas y de las instalaciones de radar. No obstante, estas bandas son populares en varias aplicaciones de redes inalámbricas de corto alcance porque evitan los problemas asociados con las licencias.

2.3.4. Ondas infrarrojas y milimétricas

Las ondas infrarrojas y milimétricas no guiadas se usan mucho para la comunicación de corto alcance. Todos los controles remotos de los televisores, grabadoras de vídeo y estéreos utilizan comunicación infrarroja. Estos controles son relativamente direccionales, baratos y fáciles de construir, pero tienen un inconveniente importante: no atraviesan los objetos sólidos (pruebe a pararse entre su control remoto y su televisor y vea si todavía funciona). En general, conforme pasamos de la radio de onda larga hacia la luz visible, las ondas se comportan cada vez más como la luz y cada vez menos como la radio.

Por otro lado, el hecho de que las ondas infrarrojas no atraviesen bien las paredes sólidas también es una ventaja. Esto significa que un sistema infrarrojo en un cuarto de un edificio no interferirá un sistema similar en cuartos adyacentes. Además, la seguridad de los sistemas infrarrojos contra el espionaje es mejor que la de los sistemas de radio, precisamente por esta razón. Por lo mismo, no es necesario obtener licencia del gobierno para operar un sistema infrarrojo, en contraste con los sistemas de radio, que deben tener licencia.

Estas propiedades han hecho del infrarrojo un candidato interesante para las LAN inalámbricas en interiores. Por ejemplo, las computadoras y las oficinas de un edificio se pueden equipar con transmisores y receptores infrarrojos relativamente desenfocados (es decir, un tanto omnidireccionales). De esta manera, las computadoras portátiles capaces de utilizar infrarrojo pueden estar en la LAN local sin tener que conectarse a ella físicamente. Cuando varias personas se presentan a una reunión con sus máquinas portátiles, sólo tienen que sentarse en la sala de conferencias para estar conectados por completo, sin tener que enchufar. La comunicación con infrarrojo no se puede usar en exteriores porque el sol brilla con igual intensidad en el infrarrojo como en el espectro visible. Para mayor información acerca de la comunicación infrarroja, véase (Adams *et al.*, 1993, y Bantz y Bauchot, 1994).

2.3.5. Transmisión por ondas de luz

La señalización óptica sin guías se ha usado durante siglos. Paul Revere utilizó señalización óptica binaria desde la vieja Iglesia del Norte justo antes de su famoso viaje. Una aplicación más moderna es conectar las LAN de dos edificios por medio de láseres montados en sus azoteas. La señalización óptica coherente con láseres es inherentemente unidireccional, de modo que cada edificio necesita su propio láser y su propio fotodetector. Este esquema ofrece un ancho de banda muy alto y un costo muy bajo. También es relativamente fácil de instalar y, a diferencia de las microondas, no requiere una licencia de la FCC.

La ventaja del láser, un haz muy estrecho, es aquí también una debilidad. Apuntar un rayo láser de 1 mm de anchura a un blanco de 1 mm a 500 metros de distancia requiere la puntería de una Annie Oakley moderna. Por lo general, se añaden lentes al sistema para desenfocar ligeramente el rayo.

Una desventaja es que los rayos láser no pueden penetrar la lluvia ni la niebla densa, pero normalmente funcionan bien en días soleados. Sin embargo, el autor asistió en una ocasión a una conferencia en un moderno hotel de Europa en la que los organizadores tuvieron la atención de

proporcionar un salón lleno de terminales para que los asistentes leyeran su correo electrónico durante las presentaciones aburridas. Puesto que la PTT local no estaba dispuesta a instalar un gran número de líneas telefónicas sólo para tres días, los organizadores colocaron un láser en el techo y lo apuntaron al edificio de ciencias de la computación de su universidad a unos cuantos kilómetros de allí; lo probaron la noche anterior a la conferencia y funcionó a la perfección. A las 9 a.m. del día siguiente, en un día brillante y soleado, el enlace falló por completo y permaneció caído todo el día. Esa noche, los organizadores probaron otra vez el enlace con mucho cuidado y de nuevo funcionó con absoluta perfección. El patrón se repitió durante dos días más de forma idéntica.

Después de la conferencia, los organizadores descubrieron el problema. Durante el día, el calor del sol causaba corrientes de convección que se elevaban desde el techo del edificio, como se muestra en la figura 2-13. Este aire turbulento desviaba el rayo y lo hacía danzar alrededor del detector. Una "vista" atmosférica como ésta hace titilar a las estrellas (y es la razón por la que los astrónomos ponen sus telescopios en las cimas de las montañas, para quedar tan arriba en la atmósfera como sea posible). Este fenómeno es también la causa del aspecto trémulo de las carreteras en un día caluroso y de las imágenes ondulantes cuando se mira sobre un radiador caliente.

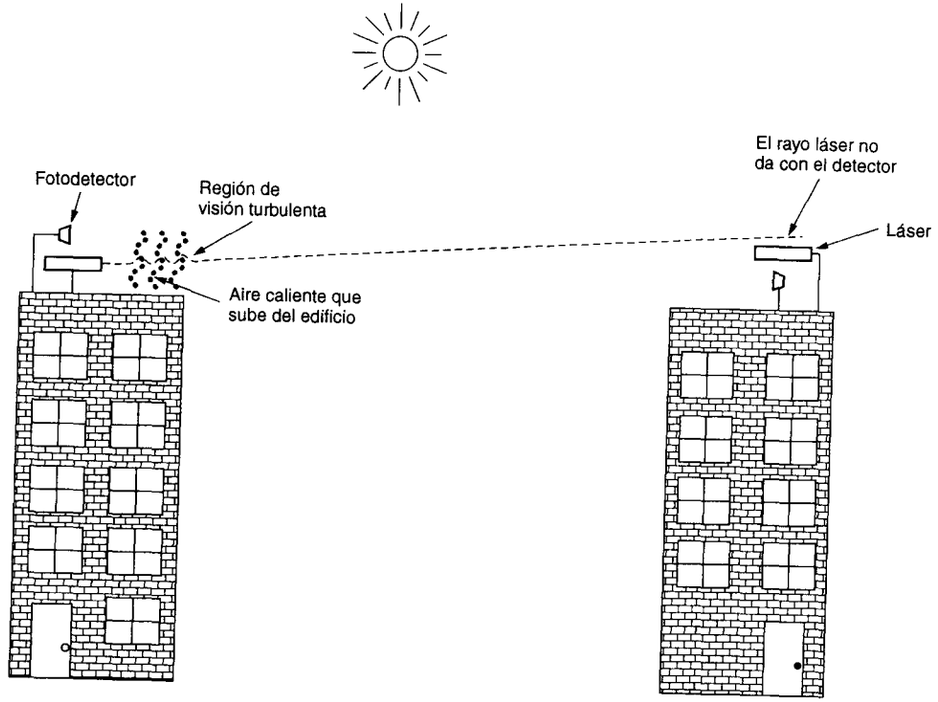


Figura 2-13. Las corrientes de convección pueden interferir los sistemas de comunicación por láser. Aquí se ilustra un sistema bidireccional con dos láseres.

2.4. EL SISTEMA TELEFÓNICO

Cuando dos computadoras propiedad de la misma compañía u organización y ubicadas una cerca de otra se necesitan comunicar, con frecuencia lo más fácil es tender un cable entre ellas. Las LAN trabajan de esta manera. Sin embargo, cuando las distancias son grandes o hay muchas computadoras, o cuando los cables tendrían que pasar a través de un camino público u otro derecho de paso público, los costos de tender cables privados por lo general son prohibitivos. Más aún, en casi todos los países instalar líneas de transmisión privadas a través (o por debajo) de una propiedad pública también es ilegal. En consecuencia, los diseñadores de redes deben apoyarse en las instalaciones de telecomunicaciones existentes.

Estas instalaciones, en particular la **PSTN** (*public switched telephone network,* **red telefónica pública conmutada**), generalmente se diseñaron hace muchos años y con un objetivo totalmente distinto: transmitir la voz humana en una forma más o menos reconocible. Su adaptabilidad para la comunicación de computadora a computadora con frecuencia es marginal en el mejor de los casos, pero la situación está cambiando rápidamente con la introducción de la fibra óptica y de la tecnología digital. En cualquier caso, el sistema telefónico está tan estrechamente ligado a las redes de computadoras (de área amplia) que merece la pena dedicar un tiempo considerable a su estudio.

Para apreciar la magnitud del problema, hagamos una comparación aproximada pero ilustrativa de las propiedades de una conexión típica de computadora a computadora por conducto de un cable local y de una línea telefónica de discado. Un cable tendido entre dos computadoras puede transferir datos a velocidades de memoria, por lo común de 10^7 a 10^8 bps. La tasa de errores generalmente es tan baja que es difícil medirla, pero en la mayor parte de las instalaciones se considera pobre una tasa de un error por día. Un error por día a estas velocidades equivale a un error por 10^{12} o 10^{13} bits enviados.

En contraste, una línea de discado tiene una velocidad máxima de datos del orden de 10^4 bps y una tasa de errores de aproximadamente 1 por 10^5 bits enviados, aunque varía un poco con la edad del equipo de conmutación telefónica en cuestión. El rendimiento combinado de tasa de bits multiplicada por la tasa de errores de un cable local es entonces de 11 órdenes de magnitud mejor que el de una línea telefónica de grado de voz. Para hacer una analogía en el campo del transporte, la relación entre el costo del proyecto Apolo entero, que puso hombres en la Luna, y el costo de un viaje por ómnibus al centro de la ciudad es de casi 11 órdenes de magnitud (en dólares de 1965: 40,000 millones a 0.40).

Desde luego, el problema es que los diseñadores de sistemas de cómputo están acostumbrados a trabajar con sistemas de cómputo, y cuando de repente enfrentan otro sistema cuyo rendimiento (desde su punto de vista) es de 11 órdenes de magnitud peor, no es de sorprender que hayan dedicado mucho tiempo y esfuerzo a encontrar una forma de usarlo de manera eficiente. Por otro lado, las compañías de teléfonos han logrado avances enormes en la década pasada en su esfuerzo por modernizar el equipo y mejorar el servicio en ciertas áreas. En las siguientes secciones describiremos el sistema telefónico y mostraremos lo que solía ser y hacia dónde va. Si desea información adicional acerca de las características internas del sistema telefónico, consulte (Bellamy, 1991).

2.4.1. Estructura del sistema telefónico

Cuando Alexander Graham Bell patentó el teléfono en 1876 (apenas unas horas antes que su rival, Elisha Gray), había una demanda enorme por su invento. El mercado inicial fue para la venta de teléfonos, que venían en pares; era obligación del cliente tender un alambre entre ellos. Los electrones regresaban por tierra. Si el propietario de un teléfono quería hablar con otros n propietarios de teléfonos, tenía que tender alambres individuales a todas las n casas. En el lapso de un año, las ciudades estaban cubiertas de alambres que pasaban sobre las casas y los árboles en una horrible maraña. Pronto fue obvio que no iba a funcionar el modelo de conectar cada teléfono a todos los otros teléfonos, como en la figura 2-14(a).

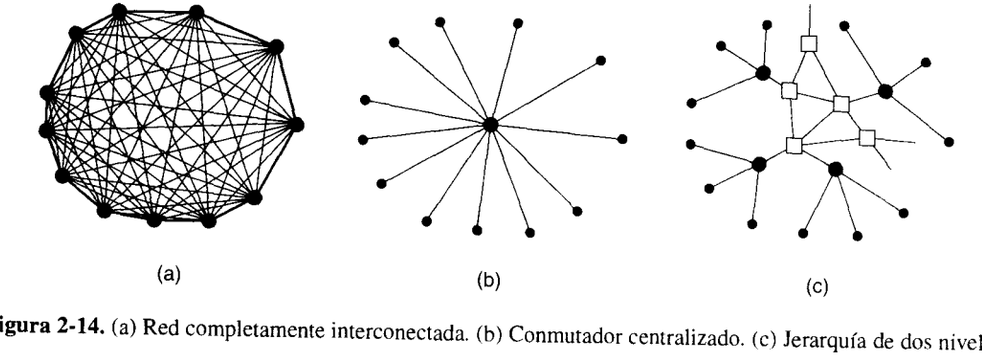


Figura 2-14. (a) Red completamente interconectada. (b) Conmutador centralizado. (c) Jerarquía de dos niveles.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE

Bell tuvo la suficiente visión para darse cuenta de esto y formó la Bell Telephone Company, que abrió su primera oficina de conmutación en 1878 (en New Haven, Connecticut). La compañía tendió un cable a la casa u oficina de cada cliente. Para hacer una llamada, el usuario debía dar vueltas a una manivela en el teléfono, con lo que producía un timbrazo en la oficina de la compañía de teléfonos para atraer la atención de un operador, que a continuación conectaba en forma manual a quien llamaba con quien era llamado mediante un cable puenteador. En la figura 2-14(b) se ilustra el modelo de una sola oficina de conmutación.

Muy pronto surgieron por todas partes oficinas de conmutación del Bell System y la gente quiso hacer llamadas de larga distancia entre ciudades, de modo que el Bell System empezó a conectar las oficinas de conmutación. El problema original pronto reapareció: conectar cada oficina de conmutación con todas las demás por medio de un cable entre ellas pronto dejó de ser práctico, así que se inventaron las oficinas de conmutación de segundo nivel. Al cabo de un rato, se necesitó una multitud de oficinas de segundo nivel, como en el diagrama de la figura 2-14(c). Por último, la jerarquía creció a cinco niveles.

Para 1890, las tres partes principales del sistema telefónico ya estaban en su lugar: las oficinas de conmutación, los cables entre los clientes y las oficinas de conmutación (a estas alturas pares trenzados balanceados y aislados, en lugar de cables abiertos con retorno a tierra) y las conexiones de larga distancia entre las oficinas de conmutación. Aunque desde entonces se

han realizado mejoras en las tres áreas, el modelo básico del Bell System ha permanecido intacto en lo esencial por más de 100 años. Para una historia técnica corta del sistema telefónico véase (Hawley, 1991).

Hoy día, el sistema telefónico se organiza como una jerarquía altamente redundante, de múltiples niveles. La siguiente descripción está sumamente simplificada, pero da la idea esencial. Cada teléfono tiene dos alambres de cobre que salen de él y que van directamente a la **oficina final** más cercana de la compañía de teléfonos (también llamada **oficina central local**). Por lo general, la distancia es de 1 a 10 km, y en las ciudades es menor que en las áreas rurales.

Tan sólo en Estados Unidos hay alrededor de 19,000 oficinas finales. La concatenación del código de área y los tres primeros dígitos del número telefónico especifican de manera única una oficina final, y es por ello que la estructura de tarifas se basa en esta información. Las conexiones de dos hilos entre el teléfono de cada suscriptor y la oficina final se conocen en el negocio como **lazo local**. Si los lazos locales del mundo se unieran por los extremos, se extenderían a la Luna y regresarían 1000 veces.

En cierto momento, el 80% del valor del capital de AT&T fue el cobre en los lazos locales. En efecto, AT&T era entonces la más grande mina de cobre del mundo. Por fortuna, este hecho no era muy conocido en la comunidad inversionista. De haberse sabido, algún pirata corporativo podría haber comprado la AT&T, cancelado todo el servicio telefónico en Estados Unidos, extraído todos los cables, y vendido el cableado a algún refinador de cobre para obtener una retribución rápida.

Si un suscriptor conectado a una oficina final determinada llama a otro suscriptor conectado a la misma oficina final, el mecanismo de conmutación dentro de la oficina establece una conexión eléctrica directa entre los dos lazos locales. Esta conexión permanece intacta mientras dura la llamada.

Si el teléfono al que se llama está conectado a otra oficina final, se tiene que usar un procedimiento diferente. Cada oficina final tiene varias líneas salientes a uno o más centros de conmutación cercanos, llamados **oficinas de cargo** (o, si están dentro de la misma área local, **oficinas tándem**). Estas líneas se llaman **troncales de conexión con cargo**. Si sucede que tanto la oficina final de quien llama como la de quien es llamado tienen una troncal de conexión a la misma oficina de cargo (algo muy probable si no están muy alejadas), la conexión se puede establecer dentro de la oficina de cargo. En la figura 2-14(c) se muestra una red telefónica que consiste únicamente en teléfonos (los puntos pequeños), oficinas finales (los puntos grandes) y oficinas de cargo (los marcos).

Si el que llama y el que es llamado no tienen una oficina de cargo en común, la trayectoria se deberá establecer en un nivel más alto de la jerarquía. Hay oficinas primarias, seccionales y regionales que forman una red que conecta a las oficinas de cargo. Las centrales de cargo, primarias, seccionales y regionales se comunican entre sí mediante **troncales intercargo** de gran ancho de banda (llamadas también **troncales interoficinas**). La cantidad de tipos diferentes de centros de conmutación y su topología varían de país a país dependiendo de su densidad telefónica (por ejemplo, ¿pueden dos oficinas de sección tener una conexión directa o deben pasar por una oficina regional?). La figura 2-15 muestra cómo se podría enrutar una conexión de media distancia.

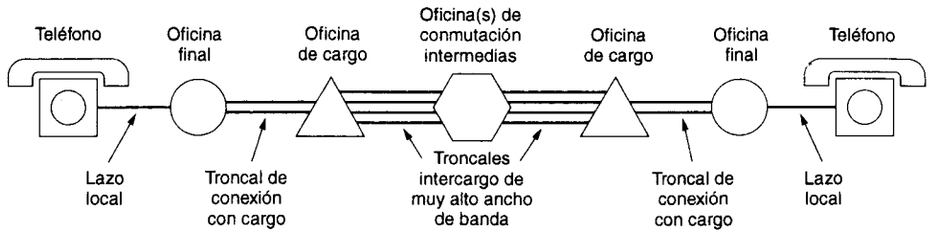


Figura 2-15. Ruta típica de un circuito para una llamada de media distancia.

Para telecomunicaciones se usan diversos medios de transmisión. En nuestros días, los lazos locales consisten en pares trenzados, aunque en los primeros días de la telefonía eran comunes los cables no aislados espaciados a 25 cm en los postes telefónicos. Entre las oficinas de conmutación se usan ampliamente cables coaxiales, microondas y, en especial, fibra óptica.

En el pasado, la señalización en todo el sistema telefónico era analógica, con la señal de voz real transmitida como un voltaje eléctrico del origen al destino. Con el advenimiento de la electrónica digital y las computadoras, se ha hecho posible la señalización digital. En este sistema se permiten sólo dos voltajes, por ejemplo –5 volts y +5 volts.

Este esquema tiene muchas ventajas sobre la señalización analógica. La primera es que aunque la atenuación y la distorsión son más severas cuando se envían señales de dos niveles que cuando se usan módems, es fácil calcular qué tan lejos se puede propagar una onda y todavía ser reconocible. Se puede insertar un regenerador digital en ese punto de la línea para restablecer la señal a su valor original, ya que existen solamente dos posibilidades. Una señal digital puede pasar a través de una cantidad arbitraria de regeneradores sin pérdida de señal y viajar de este modo grandes distancias sin pérdida de información. En contraste, las señales analógicas siempre sufren cierta pérdida de información cuando se amplifican, y esta pérdida es acumulativa. El resultado neto es que se puede lograr que la transmisión digital tenga una tasa de errores baja.

Una segunda ventaja de la transmisión digital es que se pueden intercalar voz, datos, música e imágenes (por ejemplo, televisión, fax y vídeo) para aprovechar de forma más eficiente los circuitos y el equipo. Otra ventaja es que son posibles velocidades de transmisión de datos mucho más altas con las líneas existentes.

Una tercera ventaja es que la transmisión digital es mucho más económica que la analógica, puesto que no es necesario reproducir exactamente una forma de onda analógica después de atravesar tal vez cientos de amplificadores en una llamada transcontinental; basta poder distinguir correctamente un 0 de un 1.

Por último, el mantenimiento de un sistema digital es más fácil que el de uno analógico. Un bit transmitido se recibe bien o no, con lo que se simplifica el rastreo de problemas.

En consecuencia, todas las troncales de larga distancia dentro del sistema telefónico se están convirtiendo rápidamente a tecnología digital. El sistema antiguo utilizaba transmisión analógica por cables de cobre; el nuevo emplea transmisión digital por fibra óptica.

En síntesis, el sistema telefónico consiste en tres componentes principales:

1. Lazos locales (pares trenzados, señalización analógica).
2. Troncales (fibra óptica o microondas, en su mayoría digitales).
3. Oficinas de conmutación.

Después de una breve digresión sobre la política de los teléfonos, regresaremos a cada uno de estos tres componentes en detalle. Para el lazo local, nos interesará cómo enviar datos por él (respuesta rápida: usar un módem). Para las troncales de largo alcance, la principal consideración es cómo reunir múltiples llamadas y enviarlas juntas. Este tema se llama multiplexión, y estudiaremos tres formas diferentes de hacerlo. Por último, existen dos formas fundamentalmente distintas de efectuar la conmutación, así que veremos ambas.

2.4.2. La política de los teléfonos

Durante las décadas anteriores a 1984, el Bell System proporcionó tanto el servicio local como el de larga distancia en casi todo Estados Unidos. En la década de 1970, el gobierno estadounidense se convenció de que éste era un monopolio ilegal y entabló un juicio para dividirlo. El gobierno ganó, y el 1° de enero de 1984 la AT&T se dividió en AT&T Long Lines, 23 **BOC** (*Bell Operating Companies,* **Compañías Operativas de Bell**) y algunas otras partes pequeñas. Las 23 BOC se agruparon en siete BOC regionales (RBOC) para hacerlas económicamente viables. La naturaleza entera de la telecomunicación en Estados Unidos se cambió de la noche a la mañana por orden judicial (*no* por un acto del Congreso).

Los detalles exactos del desmantelamiento se describieron en el llamado **MFJ** (*modified final judgment,* **juicio final modificado**), un contrasentido si alguna vez hubo uno (si el juicio se pudo modificar, obviamente no era final). Este suceso condujo a un aumento en la competencia, mejor servicio y menores precios para los consumidores y las empresas. Muchos otros países consideran ahora la introducción de la competencia por caminos similares.

Para dejar en claro quiénes podrían actuar y cómo, Estados Unidos se dividió en más de 160 **LATA** (*local access and transport areas,* **áreas de acceso local y de transporte**). En forma muy aproximada, una LATA es casi tan grande como el área cubierta por un código de área. Dentro de una LATA normalmente hay una **LEC** (*local exchange carrier,* **portadora de intercambio local**) que tiene un monopolio sobre el servicio tradicional de telefonía dentro de la LATA. Los LEC más importantes son los BOC, aunque algunas LATA contienen una o más de las 1500 compañías telefónicas independientes que operan como LEC. En las LATA geográficamente grandes (principalmente en el Oeste), la LEC puede manejar las llamadas de larga distancia dentro de su propia LATA pero no las que van a una LATA diferente.

Un tipo de compañía diferente maneja todo el tráfico interLATA: una **IXC** (*intereXchange carrier,* **portadora entre centrales**). Originalmente, AT&T Long Lines era la única IXC seria, pero ahora MCI y Sprint son competidores bien establecidos en el negocio de las IXC. Una de las consideraciones durante la división fue asegurar que todas las IXC serían tratadas con

igualdad en términos de calidad de líneas, tarifas y cantidad de dígitos que tendrían que marcar sus clientes para usarlas. La forma como esto se resolvió se ilustra en la figura 2-16. Aquí vemos tres LATA de ejemplo, cada una con varias oficinas finales. Las LATA 2 y 3 tienen también una pequeña jerarquía con oficinas tándem (oficinas de cargo intraLATA).

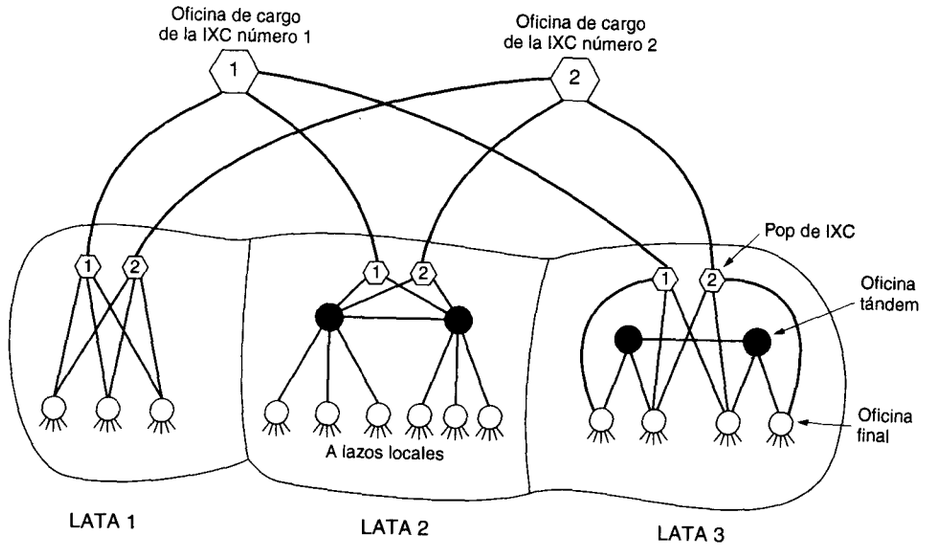


Figura 2-16. Relación entre las LATA, LEC e IXC. Todos los círculos son oficinas de conmutación LEC. Cada hexágono pertenece a la IXC cuyo número contiene.

Cualquier IXC que desee manejar llamadas que se originen en una LATA puede construir allí una oficina de conmutación llamada **POP** (*point of presence,* **punto de presencia**). La LEC es necesaria para conectar cada IXC a cada oficina final, ya sea en forma directa, como en las LATA 1 y 3, o indirecta, como en la LATA 2. Además, los términos de la conexión, tanto técnicos como financieros, deben ser idénticos para todas las IXC. De esta forma, un suscriptor en, digamos, la LATA 1 puede elegir cuál IXC usar para llamar a los suscriptores en la LATA 3.

Como parte del MFJ, se prohibió a las IXC ofrecer servicio telefónico local y a las LAC ofrecer servicio telefónico interLATA, aunque ambas eran libres de participar en otros negocios, como la operación de restaurantes de pollos fritos. En 1984, éste era un dictamen bastante claro. Desafortunadamente, la tecnología tiene la mala costumbre de hacer obsoletas las leyes. Ni la televisión por cable ni los teléfonos celulares estaban cubiertos por el acuerdo. Conforme la televisión por cable pasó de ser unidireccional a bidireccional, y la popularidad de los teléfonos celulares subió como la espuma, tanto las LEC como las IXC comenzaron a comprar o a fusionarse con las operadoras de cable y celulares.

Para 1995, el Congreso vio que tratar de mantener la distinción entre las diversas clases de compañías ya no era sostenible y esbozó una propuesta de ley para permitir a las compañías

de televisión por cable, a las compañías telefónicas locales, a las portadoras de larga distancia y a las operadoras de teléfonos celulares participar en los negocios de unas y otras. La intención era que así cualquier compañía podría ofrecer a sus clientes un solo paquete integrado que contuviera televisión por cable, teléfono, y servicios de información, y que las diferentes compañías compitieran en servicio y precio. La propuesta se convirtió en ley en febrero de 1996. Como resultado, el panorama de las telecomunicaciones en Estados Unidos está pasando por una restructuración radical.

2.4.3. El lazo local

Durante los últimos 100 años la transmisión analógica ha dominado las comunicaciones. En particular, el sistema telefónico originalmente se basó por completo en la señalización analógica. Si bien las troncales de larga distancia ya son predominantemente digitales en los países más avanzados, los lazos locales todavía son analógicos y es probable que permanezcan así por lo menos una década o dos, debido al enorme costo de convertirlos. En consecuencia, cuando una computadora desea enviar datos digitales por una línea de discado, los datos se deben convertir primero a la forma analógica mediante un módem para transmitirse por el lazo local, luego convertirse a la forma digital para transmitirse por las troncales de largo alcance, después reconvertirse a analógicos en el lazo local del extremo receptor y, por último, a digitales con otro módem para almacenarse en la computadora de destino. Este arreglo se muestra en la figura 2-17.

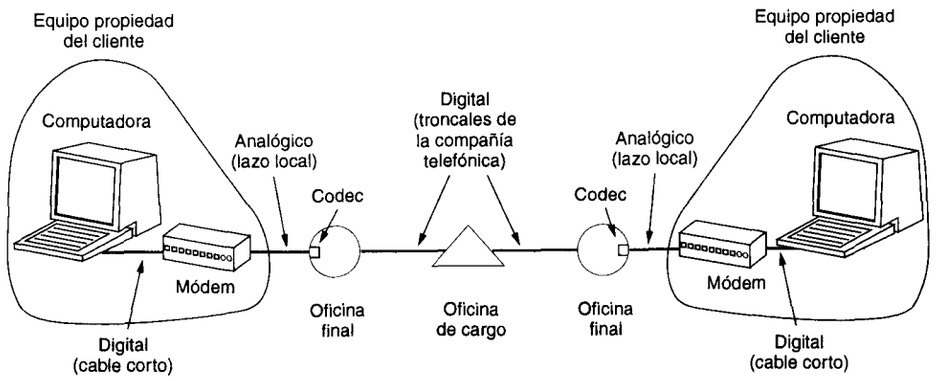


Figura 2-17. Empleo de transmisión tanto analógica como digital para una llamada de computadora a computadora. La conversión la realizan los módems y los codecs.

Si bien esta situación no es exactamente ideal, así es la vida por lo pronto, y los estudiantes de redes deben tener ciertos conocimientos tanto de la transmisión analógica como de la digital, así como de la forma en que funcionan las conversiones en un sentido y en otro. En el caso de las líneas rentadas es posible trabajar en forma digital desde el principio hasta el fin, pero estas líneas son caras y sólo sirven para construir redes privadas intracompañías.

En las secciones siguientes veremos en forma breve qué efectos tiene la transmisión analógica y examinaremos cómo los módems hacen posible transmitir datos digitales por circuitos analógicos. También veremos dos interfaces de módem comunes: RS-232-C y RS-449.

Deterioros en la transmisión

La señalización analógica consiste en la variación del voltaje con el tiempo para representar una corriente de información. Si los medios de transmisión fueran perfectos, el receptor recibiría exactamente la misma señal que envió el transmisor. Desafortunadamente, los medios no son perfectos, así que la señal recibida no es la misma que la transmitida. Si los datos son digitales, esta diferencia puede conducir a errores.

Las líneas de transmisión tienen tres problemas principales: atenuación, distorsión por retardo y ruido. La **atenuación** es la pérdida de energía conforme la señal se propaga hacia su destino. En los medios guiados (por ejemplo, cables y fibra óptica), la señal decae en forma logarítmica con la distancia. La pérdida se expresa en decibeles por kilómetro. La cantidad de energía perdida depende de la frecuencia. Para ver el efecto de esta dependencia de la frecuencia, imagine una señal no como una simple forma de onda, sino como una serie de componentes de Fourier. Cada componente es atenuado en diferente medida, lo que resulta en un espectro de Fourier distinto en el receptor y, por ende, en una señal diferente.

Si la atenuación es mucha, puede ser que el receptor no sea capaz de detectar en absoluto la señal, o la señal puede caer por debajo del nivel de ruido. En muchos casos se conocen las propiedades de atenuación de un medio y se pueden insertar amplificadores para tratar de compensar la atenuación dependiente de la frecuencia. Esta medida ayuda pero nunca puede restaurar la señal a su forma original exacta.

El segundo deterioro de la transmisión es la **distorsión por retardo** causada por el hecho de que los diferentes componentes de Fourier viajan a diferentes velocidades. En el caso de los datos digitales, los componentes rápidos de un bit pueden alcanzar y rebasar a los componentes lentos del bit de adelante, mezclando los dos bits e incrementando la probabilidad de una recepción incorrecta.

El tercer deterioro es el **ruido**, que es energía no deseada de fuentes distintas del transmisor. El movimiento al azar de los electrones en un cable causa el ruido térmico y es inevitable. La diafonía se debe al acoplamiento inductivo entre dos cables que están cerca uno de otro. A veces, cuando se habla por teléfono, se puede oír otra conversación en el fondo. Eso es diafonía. Finalmente, hay ruido de impulso, causado por picos en la línea de suministro de energía o por otros fenómenos. En el caso de datos digitales, el ruido de impulso puede borrar uno o más bits.

Módems

Debido a los problemas antes mencionados, en especial al hecho de que tanto la atenuación como la velocidad de propagación dependen de la frecuencia, es indeseable tener una gama amplia de frecuencias en la señal. Desafortunadamente, las ondas cuadradas, como las de los datos digitales, tienen un espectro amplio y por ello están sujetas a una fuerte atenuación y a

distorsión por retardo. Estos efectos hacen a la señalización de banda base (cc) inadecuada excepto a velocidades bajas y distancias cortas.

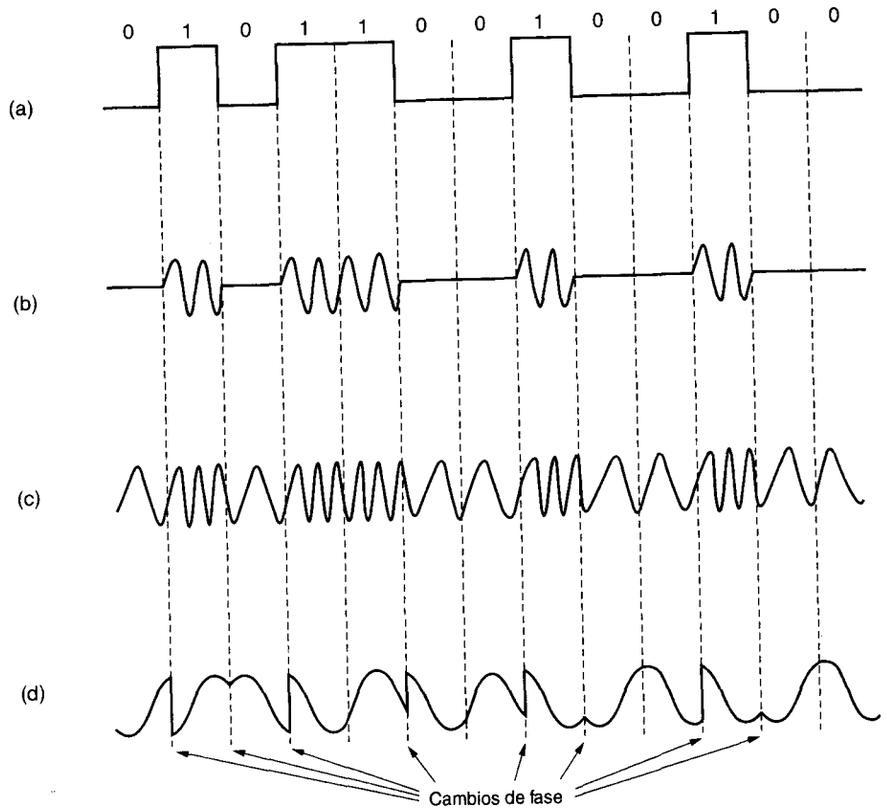


Figura 2-18. (a) Una señal binaria. (b) Modulación de amplitud. (c) Modulación de frecuencia. (d) Modulación de fase.

Para superar los problemas asociados a la señalización de corriente continua, en especial en las líneas telefónicas, se usa la señalización de ca. Se introduce un tono continuo en el intervalo de 1000 a 2000 Hz, llamado **portadora de onda senoidal**, cuya amplitud, frecuencia o fase se puede modular para transmitir la información. En la **modulación de amplitud** se usan dos niveles diferentes de voltaje para representar 0 y 1, respectivamente. En la **modulación de frecuencia**, conocida también como **modulación por desplazamiento de frecuencia**, se usan dos (o más) tonos diferentes. En la forma más simple de la **modulación de fase** la onda portadora se desplaza de modo sistemático 45, 135, 225 o 315 grados a intervalos espaciados de manera uniforme. Cada desplazamiento de fase transmite 2 bits de información. La figura 2-18 ilustra las tres formas de modulación. Un dispositivo que acepta una corriente en serie de bits

como entrada y que produce una portadora modulada como salida (o viceversa) es un **módem** (por modulador-desmodulador). El módem se inserta entre la computadora (digital) y el sistema telefónico (analógico).

Si queremos alcanzar velocidades cada vez más altas, no lo lograremos sólo con incrementar la velocidad de muestreo. El teorema de Nyquist dice que aun con una línea perfecta de 3000 Hz (cosa que decididamente no son las líneas de un teléfono de discado), no tiene caso muestrear con una frecuencia mayor de 6000 Hz. Por ello, las investigaciones para lograr módems más rápidos se enfoca a obtener más bits por muestra (es decir, por baud).

La mayor parte de los módems avanzados se valen de una combinación de técnicas para transmitir múltiples bits por baud. En la figura 2-19(a) vemos puntos a 0, 90, 180 y 270 grados, con dos niveles de amplitud por cambio de fase. La amplitud se indica con la distancia desde el origen. En la figura 2-19(b) vemos un esquema de modulación diferente en el que se usan 16 combinaciones distintas de amplitud y cambio de fase. Así, la figura 2-19(a) tiene ocho combinaciones válidas y puede servir para transmitir 3 bits por baud. En contraste, la figura 2-19(b) tiene 16 combinaciones válidas y por ello puede servir para transmitir 4 bits por baud. El esquema de la figura 2-19(b), cuando se utiliza para transmitir 9600 bps por una línea de 2400 baud, se llama **QAM** (*quadrature amplitude modulation,* **modulación de amplitud en cuadratura**).

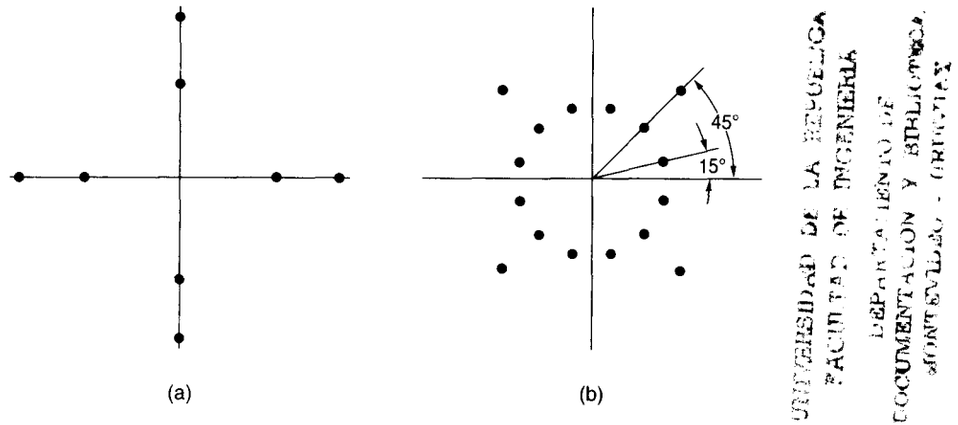


Figura 2-19. (a) Modulación de 3 bits/baud. (b) Modulación de 4 bits/baud.

A los diagramas como los de la figura 2-19, que muestran las combinaciones permitidas de amplitud y fase, se les llama **patrones de constelación**. Cada estándar de módem de alta velocidad tiene su propio patrón de constelación y solamente puede conversar con otros módems que utilicen el mismo modelo (aunque la mayor parte de los módems pueden emular a todos los modelos más lentos). Por ejemplo, el estándar ITU V.32 para módems de 9600 bps usa el patrón de constelación de la figura 2-19(b).

El siguiente escalón arriba de 9600 bps es el de 14,400 bps y se llama **V.32 bis**. Esta velocidad se logra transmitiendo 6 bits por muestra a 2400 baud. El patrón de constelación tiene

64 puntos. Los módem de fax usan esta velocidad para transmitir páginas que se han digitalizado como mapas de bits. Tras el V.32 bis viene el **V.34**, que opera a 28,800 bps.

Con tantos puntos en el patrón de constelación, hasta una pequeña cantidad de ruido en la amplitud o la fase detectada puede resultar en un error y potencialmente, en 6 bits malos. Para reducir la posibilidad de errores, muchos módems añaden un bit de paridad, dando 128 puntos en el patrón de constelación. La codificación de los puntos se hace con cuidado para maximizar la oportunidad de detectar errores. El código que hace esto se llama **codificación enrejada**.

Un enfoque diferente por completo para la transmisión a alta velocidad consiste en dividir el espectro disponible de 3000 Hz en 512 bandas pequeñas y transmitir a, digamos, 20 bps en cada una. Este esquema requiere un procesador sustancial dentro del módem, pero tiene la ventaja de ser capaz de inhabilitar bandas de frecuencia muy ruidosas. Los módems que implementan esta solución normalmente tienen también capacidad de V.32 o V.34, así que pueden conversar con módems estándar.

Muchos módems tienen ahora integradas compresión y corrección de errores. La gran ventaja de este enfoque es que con estas funciones se mejora la velocidad de transmisión de datos efectiva sin necesidad de cambiar el *software* existente. Un esquema de compresión popular es el **MNP 5**, que usa codificación de longitud de series para comprimir series de bytes idénticos. También los módems de fax utilizan codificación de longitud de series, puesto que son muy comunes las series de ceros (papel en blanco). Otro esquema es el **V.42 bis**, que emplea un algoritmo de compresión de Ziv-Lempel, utilizado también en Compress y otros programas (Ziv y Lempel, 1977).

Aun cuando se usan módems, puede presentarse otro problema en las líneas telefónicas: los ecos. En una línea larga, cuando la señal llega a su destino final se puede reflejar parte de la energía, de forma análoga a los ecos acústicos en las montañas. Como ilustración de los ecos electromagnéticos, pruebe a hacer brillar en la noche una linterna desde un cuarto oscuro a través de una ventana cerrada. Verá el reflejo de la luz de la linterna en la ventana (porque parte de la energía se refleja en la unión aire-vidrio y regresa hacia usted). Lo mismo sucede en las líneas de transmisión, en especial en el punto donde termina el lazo local en la oficina final.

El efecto del eco es que una persona que habla por teléfono escucha sus propias palabras después de un lapso corto. Estudios psicológicos han demostrado que esto molesta a muchas personas, con frecuencia haciéndolas tartamudear o confundiéndolas. Para eliminar el problema de los ecos, se instalan supresores de ecos en líneas de más de 2000 km. (En líneas cortas, los ecos regresan tan rápidamente que no molestan a la gente.) Un **supresor de ecos** es un dispositivo que detecta el habla humana procedente de un extremo de la conexión y suprime todas las señales que van en el otro sentido. Básicamente, es un amplificador que puede activarse y desactivarse mediante una señal de control producida por un circuito detector de voz.

Cuando la primera persona deja de hablar y la segunda empieza, el supresor de ecos invierte las direcciones. Un buen supresor de ecos puede reaccionar en un lapso de 2 a 5 ms. Sin embargo, cuando está funcionando, la información solamente puede viajar en una dirección; los ecos no pueden regresar al emisor. La figura 2-20(a) muestra el estado de los supresores de ecos mientras *A* está hablando con *B*. La figura 2-20(b) muestra el estado después de que *B* ha empezado a hablar.

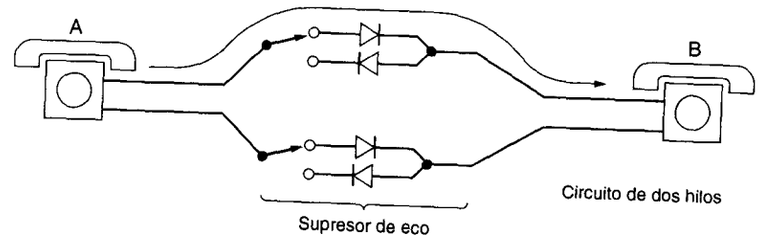


Figura 2-20. (a) *A* hablando con *B*. (b) *B* hablando con *A*.

Los supresores de ecos tienen varias propiedades indeseables para la comunicación de datos. Primera, si no estuvieran presentes, sería posible transmitir en ambas direcciones al mismo tiempo usando una banda de frecuencia diferente para cada dirección. Este funcionamiento se llama transmisión **dúplex**. Con los supresores de ecos la transmisión dúplex resulta imposible. La opción alterna es la transmisión **semidúplex**, en la que la comunicación puede efectuarse en cualquier sentido, pero sólo en uno a la vez. Una vía de tren sencilla es semidúplex. Aunque la transmisión semidúplex sea adecuada, es engorrosa pues el tiempo requerido para la inversión de direcciones puede ser sustancial. Además, los supresores de ecos se diseñan para invertir la dirección al detectar el habla humana, no datos digitales.

Para aliviar estos problemas se ha dispuesto una puerta de escape en los circuitos telefónicos con supresores de ecos. Cuando los supresores detectan un tono puro a 2100 Hz, se desactivan y permanecen inactivos mientras está presente una portadora. Este arreglo es uno de los muchos ejemplos de la **señalización en banda**, así llamada porque las señales de control que activan y desactivan las funciones de control interno están dentro de la banda accesible al usuario. En general, la tendencia actual es evitar la señalización en banda para prevenir que los usuarios interfieran el funcionamiento del sistema. En Estados Unidos se ha retirado en buena medida la señalización en banda, pero en otros países todavía existe.

Una alternativa a los supresores de ecos son los **canceladores de ecos**. Éstos son circuitos que simulan el eco, estiman su magnitud y lo restan a la señal entregada, sin necesidad de relés mecánicos. Cuando se usan canceladores de ecos es posible la operación dúplex. Por esta razón los canceladores de ecos están reemplazando con rapidez a los supresores de ecos en Estados Unidos y otros países grandes.

RS-232-C y RS-449

La interfaz entre la computadora o la terminal y el módem es un ejemplo de protocolo de capa física, que debe especificar con detalle la interfaz mecánica, eléctrica, funcional y de procedimientos. Ahora veremos de cerca dos estándares de capa física muy conocidos: RS-232-C y su sucesor, RS-449.

Empecemos con **RS-232-C**, la tercera revisión del estándar RS-232 original. El estándar lo planeó la Electronic Industries Association, una organización comercial de fabricantes de equipo electrónico, y su nombre correcto es EIA RS-232-C. La versión internacional está en la recomendación **V.24** del CCITT y es similar; sólo difiere ligeramente en algunos de los circuitos que se usan poco. En los estándares, la terminal o computadora se llama de manera oficial **DTE** (*data terminal equipment,* **equipo terminal de datos**), y el módem, **DCE** (*data circuit-terminating equipment,* **equipo de comunicación de datos**).

La especificación mecánica describe un conector de 25 agujas de 47.04 ± .13 mm de ancho (de centro de tornillo a centro de tornillo), con todas las demás dimensiones especificadas con igual precisión. La hilera superior tiene las agujas numeradas de 1 a 13 (de izquierda a derecha); la hilera de abajo tiene las agujas numeradas de 14 a 25 (también de izquierda a derecha).

La especificación eléctrica para RS-232-C es que un voltaje más negativo que −3 volts es un 1 binario y que un voltaje más positivo que +4 volts es un 0 binario. Se permiten velocidades de transmisión de datos de hasta 20 kbps, así como cables de hasta 15 metros.

La especificación funcional dice cuáles circuitos se conectan a cada una de las 25 agujas, y qué significan. La figura 2-21 muestra 9 puntas que casi siempre están implementadas; las restantes con frecuencia se omiten. Cuando la terminal o computadora se enciende, establece (es decir, pone en 1 lógico) la línea Terminal de Datos Preparada (aguja 20). Cuando el módem se enciende, establece la línea Conjunto de Datos Preparado (aguja 6). Cuando el módem detecta una portadora en la línea telefónica establece la línea Detección de Portadora (aguja 8). La Petición de Envío (aguja 4) indica que la terminal quiere enviar datos. Libre para Enviar (aguja 5) indica que el módem está preparado para aceptar datos. Los datos se transmiten por el circuito Transmitir (aguja 2) y se reciben por el circuito Recibir (aguja 3).

Se dispone de otros circuitos para seleccionar la velocidad de transmisión de los datos, probar el módem, sincronizar los datos, detectar señales de llamada y enviar datos en dirección contraria por un canal secundario. Estos circuitos casi nunca se usan en la práctica.

La especificación de procedimientos es el protocolo, esto es, la secuencia permitida de sucesos. El protocolo se basa en pares de acción-reacción. Por ejemplo, cuando la terminal establece Petición de Envío, el módem contesta con Libre para Enviar, si está en condiciones de aceptar datos. También existen pares de acción-reacción similares para otros circuitos.

Es común que dos computadoras deban conectarse usando RS-232-C. Puesto que ninguna de ellas es un módem, existe un problema de interfaz. Este problema se resuelve conectando las computadoras con un aparato llamado **módem nulo**, que conecta la línea transmisora de una máquina con la línea receptora de la otra; también cruza algunas otras líneas de forma similar. Un módem nulo se parece a un cable corto.

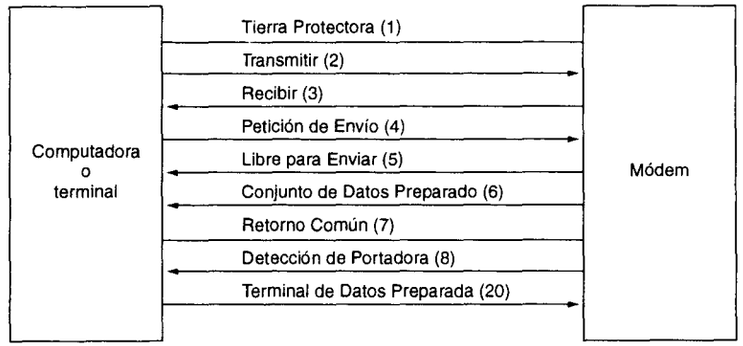


Figura 2-21. Algunos de los principales circuitos de RS-232-C. El número de aguja se da entre paréntesis.

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE
DOCUMENTACIÓN Y BIBLIOTECA
MONTEVIDEO - URUGUAY

El RS-232-C ha estado en circulación desde hace años. Gradualmente, la limitación de la velocidad de transmisión de datos a no más que 20 kbps y la longitud máxima del cable de 15 metros se han vuelto cada vez más fastidiosas. La EIA tuvo un largo debate sobre si debía o no tratar de crear un estándar nuevo que fuera compatible con el viejo (pero técnicamente no muy avanzado) o uno nuevo e incompatible que satisficiera todas las necesidades de los años por venir. Finalmente optaron por elegir ambos.

El estándar nuevo, llamado **RS-449**, en realidad es tres estándares en uno. Las interfaces mecánica, eléctrica y de procedimientos se especifican en RS-449, pero la interfaz eléctrica se especifica en dos estándares diferentes. El primero de ellos, **RS-423-A**, es parecido al RS-232-C en que todos sus circuitos comparten una tierra común. Esta técnica se llama **transmisión desbalanceada**. El segundo estándar eléctrico, **RS-422-A**, usa en contraste la **transmisión balanceada**, en la que cada uno de los circuitos principales requiere dos hilos, sin tierra común. Esto permite usar la RS-422-A a velocidades de hasta 2 Mbps por cables de 60 metros.

En la figura 2-22 se muestran los circuitos que se usan en RS-449. Se añadieron varios circuitos nuevos que no están presentes en RS-232-C. En particular, se han incluido circuitos para probar el módem tanto en forma local como remota. Debido a la inclusión de varios circuitos de dos hilos (cuando se usa RS-422-A), se requieren más agujas en el estándar nuevo, así que el familiar conector de 25 agujas se desechó. En su lugar está un conector de 37 agujas y uno de 9 agujas. El conector de 9 agujas se requiere solamente si se usa el segundo canal (en sentido contrario).

Fibra en el lazo local

Para servicios avanzados futuros, como el vídeo sobre pedido, el canal de 3 kHz que se usa actualmente ya no se usará. Las discusiones acerca de qué hacer al respecto tienden a enfocarse en dos soluciones. La directa, tender una fibra desde la oficina final a las casas de todos, se llama

	RS-232-C			CCITT V .24			RS-449		
	Código	Aguja	Circuito	Código	Aguja	Circuito	Código	Aguja	Circuito
	AA	1	Tierra protectora	101	1	Tierra protectora		1	
	AB	7	Tierra de señal	102	7	Tierra de señal	$\overline{SG}$	19	Tierra de señal
							SC	37	Envío común
							RC	20	Recepción común
	BA	2	Datos transmitidos	103	2	Datos transmitidos	SD	4,22	Enviar datos
	BB	3	Datos recibidos	104	3	Datos recibidos	RD	6,24	Recibir datos
	CA	4	Petición de envío	105	4	Petición de envío	RS	7,25	Petición de envío
	CB	5	Libre para enviar	106	5	Listo para envío	CS	9,27	Libre para enviar
	CC	6	Conjunto de datos listo	107	6	Conjunto de datos listo	DM	11,29	Modo de datos
	CD	20	Terminal de datos lista	108	20	Terminal de datos lista	TR	12,30	Terminal lista
	CE	22	Indicador de campanilla	125	22	Indicador de llamada	IC	15	Llamada que entra
	CF	8	Detector de línea	109	8	Detector de línea	RR	13,31	Receptor listo
	CG	21	Calidad de la señal	110	21	Calidad de la señal	SQ	33	Calidad de la señal
	CH	23	Tasa de DTE	111	23	Tasa de DTE	SR	16	Tasa de señalización
	CI	18	Tasa de DCE	112	18	Tasa de DCE	SI	2	Indicadores de señalización
							IS	28	Terminal en servicio
				136		Señal nueva	NS	34	Señal nueva
				126	11	Selección de frecuencia	SF	16	Selección de frecuencia
	DA	24	Temporización de DTE	113	24	Temporización de DTE	TT	17,25	Temporización de terminal
	DB	15	Temporización de DCE	114	15	Temporización de DCE	ST	5,23	Temporización de envío
	DD	17	Temporización del receptor	115	17	Temporización del receptor	RT	8,26	Temporización de recepción
Canal secundario	SBA	14	Datos transmitidos	118	14	Datos transmitidos	SSD	3	Envío de datos
Canal secundario	SBB	16	Datos recibidos	119	16	Datos recibidos	SRD	4	Recepción de datos
Canal secundario	SCA	19	Petición de envío	120	19	Señal de línea	SRS	7	Petición de envío
Canal secundario	SCB	13	Libre para enviar	121	13	Canal listo	SCS	8	Libre para enviar
Canal secundario	SCF	12	Detector de línea	122	12	Detector de línea	SRR	2	Receptor listo
							LL	10	Prueba local
							RL	14	Prueba remota
							TM	18	Modo de prueba
							SS	32	Selección de espera
							SB	36	Indicador de espera

Figura 2-22. Comparación de RS-232-C, V.24 y RS-449.

FTTH (*fiber to the home,* **fibra a los hogares**). Esta solución se ajusta bien al sistema actual pero en lo económico no será viable durante décadas; simplemente es muy costosa.

Una solución alterna y mucho más barata es la **FTTC** (*fiber to the curb*, **fibra a la banqueta**). En este modelo, la compañía de teléfonos instala una fibra óptica desde cada oficina final hasta cada vecindario (la banqueta) al que da servicio (Paff, 1995). La fibra termina en una caja de empalme a la que llegan todos los lazos locales. Puesto que los lazos locales son ahora mucho más cortos (quizá 100 m en lugar de 3 km), pueden funcionar a velocidades mayores, probablemente a 1 Mbps, lo cual es apenas suficiente para vídeo comprimido. Este diseño se muestra en la figura 2-23(a).

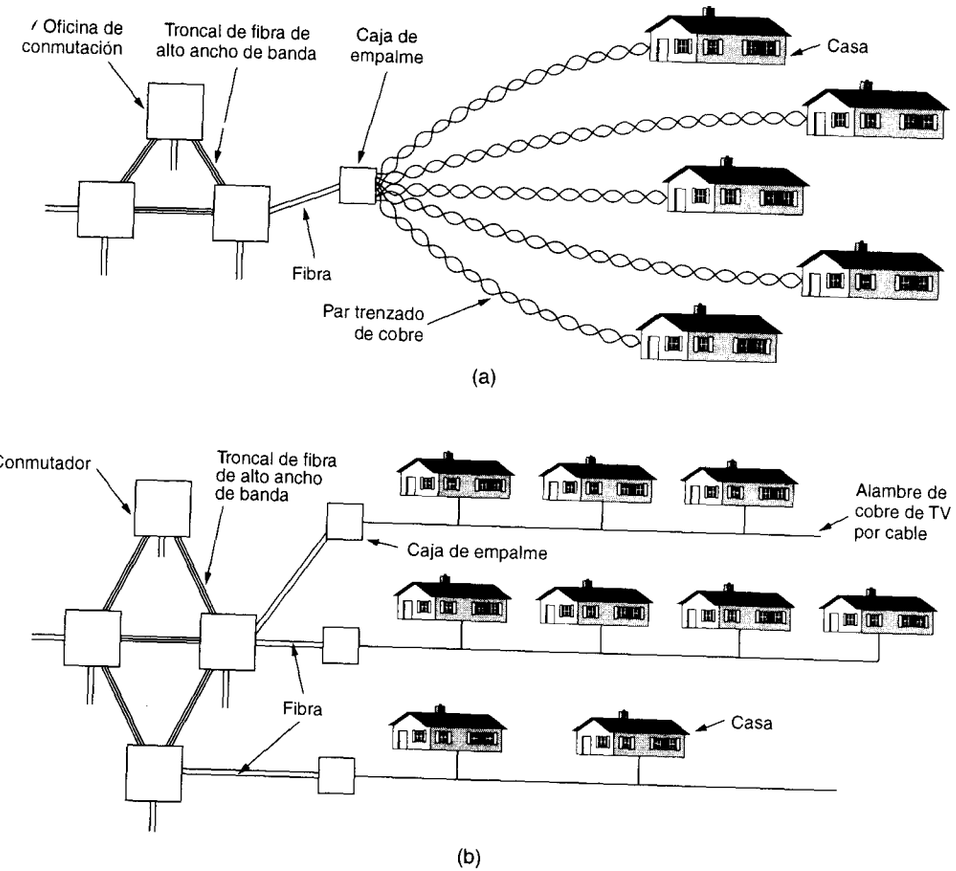


Figura 2-23. Fibra a la banqueta. (a) Usando la red telefónica. (b) Usando la red de televisión por cable.

De esta manera, múltiples vídeos (u otros canales de información) pueden enviarse por la fibra a alta velocidad y dividirse entre los pares trenzados en el extremo. Al compartir una fibra de 1 Gbps entre 100 a 1000 clientes, el costo por cliente se reduce y se puede proporcionar un ancho de banda considerablemente mayor que el de ahora. Las velocidades mayores de 1 Mbps no pueden mantenerse a largas distancias con los pares trenzados existentes, así que, a largo plazo, todos los pares trenzados se tendrán que reemplazar por fibra. La cuestión de que por lo pronto se use la solución intermedia de FTTC o que desde el principio el objetivo sea FTTH es materia de debate dentro de la industria telefónica.

En la figura 2-23(b) se muestra un diseño alternativo que aprovecha la infraestructura existente de la televisión por cable. Aquí se usa un cable de múltiples derivaciones en lugar del sistema punto a punto característico del sistema telefónico. Es probable que tanto la figura 2-23(a) como la figura 2-23(b) coexistan en el futuro, conforme las compañías de teléfonos y las operadoras de televisión por cable se conviertan en competidores directos para prestar servicios de voz, datos, y posiblemente hasta televisión. Si desea más información sobre este tema, consulte (Cook y Stern, 1994; Miki, 1994b, y Mochida, 1994).

2.4.4. Troncales y multiplexión

La economía de escala desempeña un papel importante en el sistema telefónico. Cuesta prácticamente lo mismo instalar y mantener un troncal de alto ancho de banda que uno de bajo ancho de banda entre dos oficinas de conmutación (esto es, el gasto principal es la excavación de zanjas y no el alambre de cobre o la fibra óptica). En consecuencia, las compañías de teléfonos han desarrollado esquemas elaborados para multiplexar muchas conversaciones en un solo troncal físico. Estos esquemas de multiplexión se pueden dividir en dos categorías: **FDM** (*frequency division multiplexing*, **multiplexión por división en frecuencia**) y **TDM** (*time division multiplexing*, **multiplexión por división en tiempo**). En FDM el espectro de frecuencia se divide entre los canales lógicos, con cada usuario en posesión exclusiva de alguna banda de frecuencia. En TDM los usuarios esperan su turno (por asignación cíclica), obteniendo cada uno en forma periódica la banda entera durante un breve lapso de tiempo.

La radiodifusión AM ilustra ambas clases de multiplexión. El espectro asignado es de alrededor de 1 MHz, aproximadamente de 500 a 1500 kHz. A los diferentes canales lógicos (estaciones) se les asigna una frecuencia distinta, y cada uno funciona en una porción del espectro con una separación entre canales lo bastante grande como para prevenir la interferencia. Este sistema es un ejemplo de multiplexión por división en frecuencia. Adicionalmente (en algunos países), las estaciones individuales tienen dos subcanales lógicos, música y publicidad, los cuales se alternan en el tiempo en la misma frecuencia, primero una ráfaga de música y después una ráfaga de publicidad, luego más música y así sucesivamente. Esta situación es multiplexión por división en tiempo.

A continuación examinaremos la multiplexión por división en frecuencia y después veremos cómo es que se puede aplicar FDM a la fibra óptica (multiplexión por división en longitud de onda). Después nos enfocaremos en TDM y terminaremos con un sistema TDM avanzado que se usa para fibra óptica (SONET).

Multiplexión por división en frecuencia

La figura 2-24 muestra cómo multiplexar tres canales telefónicos de grado de voz utilizando FDM. Los filtros limitan el ancho de banda utilizable a cerca de 3000 Hz por canal de grado de voz. Cuando se multiplexan muchos canales juntos, se asigna 4000 Hz a cada canal para mantenerlos bien separados. Primero se eleva la frecuencia de los canales de voz, cada uno en una cantidad diferente, después de lo cual ya se pueden combinar, porque ahora no hay dos canales

que ocupen la misma porción del espectro. Observe que aunque existen separaciones entre los canales (bandas de protección), hay cierta superposición entre canales adyacentes porque los filtros no tienen bordes abruptos. Esta superposición significa que un pico fuerte en el borde de un canal se detectará en el adyacente como ruido no térmico.

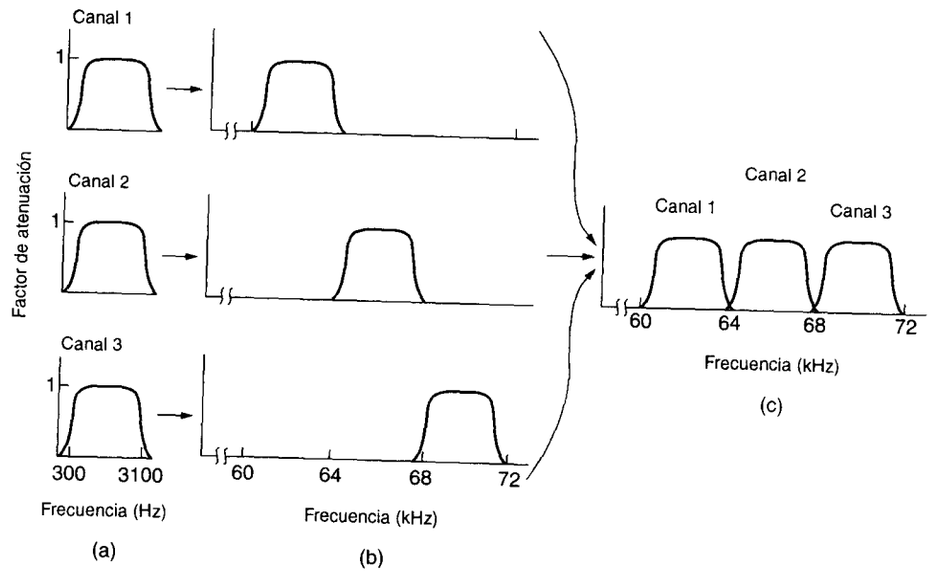


Figura 2-24. Multiplexión por división en frecuencia. (a) Los anchos de banda originales. (b) Los anchos de banda elevados en frecuencia. (c) El canal multiplexado.

Los esquemas de FDM que se emplean en el mundo están normalizados hasta cierto punto. Un estándar muy difundido es el de 12 canales de voz a 4000 Hz (3000 Hz para el usuario más dos bandas de protección de 500 Hz cada una) multiplexados dentro de la banda de 60 a 108 kHz. Esta unidad se llama **grupo**. La banda de 12 a 60 kHz a veces se usa para otro grupo. Muchas portadoras ofrecen un servicio de líneas alquiladas de 48 a 56 kbps que se basan en este grupo. Se pueden multiplexar cinco grupos (60 canales de voz) para formar un **supergrupo**. La siguiente unidad es el **grupo maestro**, que se compone de cinco supergrupos (en el estándar de CCITT) o de 10 supergrupos (en el sistema Bell). Existen también otros estándares que llegan hasta 230,000 canales de voz.

Multiplexión por división en longitud de onda

Para los canales de fibra óptica se utiliza una variante de la multiplexión por división en frecuencia llamada **WDM** (*wavelength division multiplexing*, **multiplexión por división en longitud de onda**). En la figura 2-25 se describe una forma simple de lograr la FDM en fibra. Dos fibras llegan

juntas a un prisma (o, para ser más precisos, a una rejilla de difracción), cada una con su energía en una banda diferente. Los dos haces pasan a través del prisma o rejilla, y se combinan en una misma fibra compartida para ser transmitidos a un destino distante, donde se dividirán de nuevo.

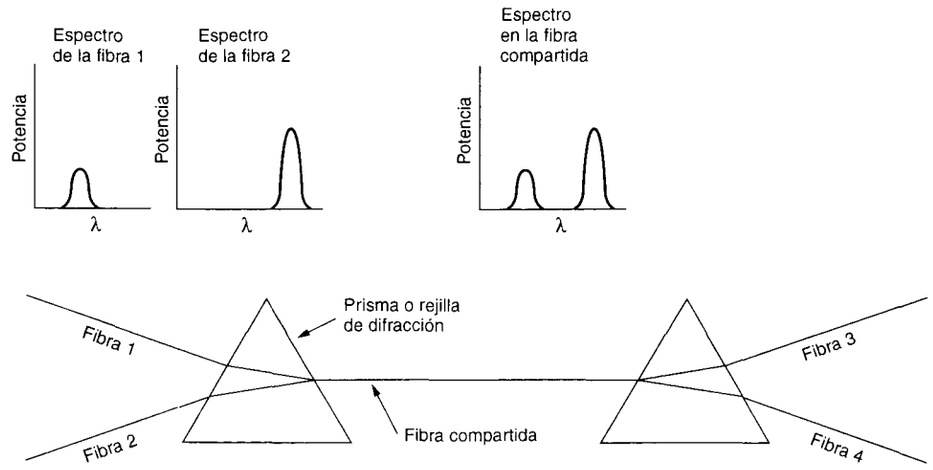


Figura 2-25. Multiplexión por división en longitud de onda.

En realidad, aquí nada es nuevo. Puesto que cada canal tiene su propio intervalo de frecuencia, y todos los intervalos están separados, se les puede multiplexar juntos en la fibra de largo alcance. La única diferencia respecto a la FDM eléctrica es que un sistema óptico que usa una rejilla de difracción es totalmente pasivo y, por ello, altamente confiable.

Cabe señalar que la razón por la cual la WDM es popular es que la energía en una sola fibra normalmente es de sólo unos cuantos gigahertz de ancho porque en la actualidad es imposible convertir con mayor rapidez entre los medios eléctrico y óptico. Puesto que el ancho de banda de una sola banda de fibra es de alrededor de 25,000 GHz (véase la figura 2-6), existe un gran potencial para multiplexar muchos canales juntos en rutas de largo alcance. Sin embargo, una condición necesaria es que los canales de entrada utilicen frecuencias diferentes.

Una aplicación potencial de la WDM está en los sistemas FTTC que se describieron antes. Inicialmente, una compañía de teléfonos podía tender una sola fibra desde una oficina final hasta la caja de empalme de un vecindario, donde se conectaba con los pares trenzados de las casas. Años después, habiendo bajado el costo de la fibra y aumentado su demanda, los pares trenzados se pueden reemplazar por fibra y todos los lazos locales se pueden empalmar a la fibra que llega a la oficina final empleando WDM.

En el ejemplo de la figura 2-25 tenemos un sistema de longitud de onda fija. Los bits de la fibra 1 van a la fibra 3 y los de la fibra 2 van a la fibra 4. No es posible hacer que los bits vayan de la fibra 1 a la fibra 4. Por otro lado, es posible construir sistemas WDM conmutados. Un

dispositivo así tiene muchas fibras de entrada y muchas de salida, y los datos de cualquiera de las fibras de entrada pueden ir a cualquiera de las de salida. Lo normal es que el acoplador sea una estrella pasiva y que con la luz de cada fibra de entrada ilumine la estrella. Aunque la distribución de la energía entre n salidas la diluye por un factor de n, tales sistemas son prácticos para cientos de canales.

Desde luego, si la luz de una de las fibras de entrada está a 1.50206 micras y podría ir a cualquiera de las fibras de salida, todas las fibras de salida necesitan filtros ajustables de modo que la fibra seleccionada se pueda poner ella misma a 1.50206 micras. Tales filtros ópticos ajustables se pueden construir con interferómetros de Fabry-Perot o de Mach-Zehnder. Como alternativa, las fibras de entrada podrían ser ajustables y las de salida fijas. Hacer ambas ajustables es un gasto innecesario y rara vez vale la pena.

Multiplexión por división en el tiempo

Aunque todavía se usa la FDM en cables de cobre o canales de microondas, requiere circuitos analógicos y no es fácil que lo efectúe una computadora. En contraste, la TDM se puede controlar por completo con electrónica digital, de modo que se ha extendido en forma más amplia en años recientes. Desafortunadamente, esta técnica sólo se puede aplicar a datos digitales. Puesto que los lazos locales producen señales analógicas, se requiere una conversión de analógico a digital en la oficina final, donde se reúnen todos los lazos locales individuales para combinarse en troncales de salida. Veremos ahora cómo múltiples señales de voz analógicas se digitalizan y combinan en un solo troncal digital de salida. (Recuerde que los datos de computadora que se envían por un módem también son analógicos cuando llegan a la oficina final.)

Las señales analógicas se digitalizan en la oficina final con un dispositivo llamado **codec** (codificador-decodificador), con lo que producen números de 7 u 8 bits (véase la figura 2-17). El codec toma 8000 muestras por segundo (125 μseg/muestra) porque el teorema de Nyquist dice que esto es suficiente para capturar toda la información del ancho de banda de 4 kHz del canal telefónico. A una velocidad de muestreo menor, la información se perdería; a una mayor, no se ganaría información extra. Esta técnica se llama **PCM** (*pulse code modulation,* **modulación de código de pulso**). La PCM es el corazón del sistema telefónico moderno. En consecuencia, virtualmente todos los intervalos de tiempo dentro del sistema telefónico son múltiplos de 125 μseg.

Cuando la transmisión digital empezó a surgir como una tecnología factible, el CCITT fue incapaz de lograr un acuerdo respecto al estándar internacional para la PCM. En consecuencia, ahora se usan diversos esquemas incompatibles en diferentes países alrededor del mundo. Las conexiones internacionales entre países incompatibles requieren "cajas negras" (con frecuencia caras) para convertir el sistema del país originador al del país receptor.

Un método muy en boga en Estados Unidos y Japón es el de la portadora T1, descrito en la figura 2-26. (Hablando técnicamente, el formato se llama DS1 y la portadora se llama T1, pero aquí no haremos esa sutil distinción.) La portadora T1 consiste en 24 canales de voz que se multiplexan juntos. Por lo común las señales analógicas se muestrean por asignación cíclica, alimentando la corriente analógica resultante al codec en lugar de tener 24 codec y mezclar después la salida digital. Cada uno de los 24 canales, por turno, inserta 8 bits en la corriente de

salida. Siete bits son de datos y uno es de control, obteniéndose $7 \times 8000 = 56{,}000$ bps de datos, y $1 \times 8000 = 8000$ bps de información de señalización por canal.

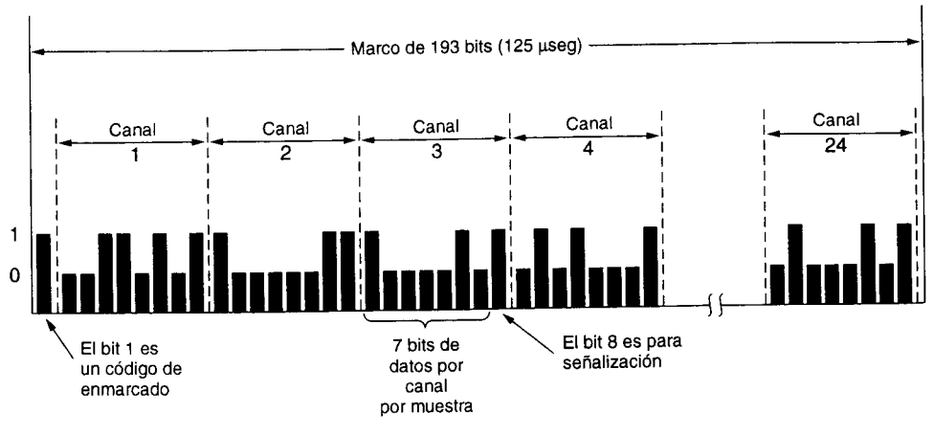


Figura 2-26. La portadora T1 (1.544 Mbps).

Un marco consiste en $24 \times 8 = 192$ bits más un bit extra para enmarcado, lo que da 193 bits cada 125 µseg. Esto produce una tasa de transmisión de datos aproximada de 1.544 Mbps. El bit número 193 se usa para sincronización del marco y sigue el patrón 0101010101... Normalmente, el receptor verifica continuamente este bit para asegurarse de que no ha perdido la sincronización. Si llegara a perder sincronía, el receptor puede esperar hasta detectar otra vez el patrón y volverse a sincronizar. Los clientes analógicos no pueden generar el patrón de bits porque corresponde a una onda senoidal a 4000 Hz, que sería filtrada. Desde luego, los clientes digitales pueden generar este patrón, pero hay poca probabilidad de que esté presente cuando el marco pierde sincronía. Cuando se utiliza un sistema T1 exclusivamente para datos, sólo 23 de los canales llevan datos. El vigésimo cuarto lleva un patrón especial de sincronización que permite la recuperación rápida en caso de que el marco pierda sincronía.

Cuando el CCITT por fin llegó a un acuerdo, sintió que 8000 bps de información de señalización era demasiado, de modo que su estándar de 1.544 Mbps se basa en un elemento de datos de 8 bits en lugar de 7; esto es, la señal analógica se cuantiza en 256 niveles discretos en lugar de 128. Hay dos variantes (incompatibles). En la **señalización de canal común**, el bit extra (que se anexa al final y no al principio del marco de 193 bits) adopta los valores 10101010... en los marcos nones y contiene información de señalización para todos los canales de los marcos pares.

En la otra variante, la **señalización asociada al canal**, cada canal tiene su propio subcanal privado de señalización. Se establece un subcanal privado asignando uno de los ocho bits de usuario de cada sexto marco a funciones de señalización, así que cinco de cada seis muestras tienen 8 bits de ancho y la otra sólo tiene 7 bits de ancho. El CCITT también tiene una recomendación para una portadora PCM a 2.048 Mbps llamada **E1**. Esta portadora empaca 32 muestras

de datos de 8 bits en el marco básico de 125 µseg. Treinta de los canales se usan para información y dos para señalización. Cada grupo de cuatro marcos proporciona 64 bits de señalización, la mitad de los cuales se usa para señalización asociada al canal y el resto se usa para sincronización de marcos o se reserva para que cada país los use como quiera. Fuera de América del Norte y Japón, la portadora de 2.048 Mbps tiene un uso muy extendido.

Una vez que la señal de voz se digitaliza, es tentador tratar de aplicar técnicas estadísticas para reducir la cantidad de bits necesarios por canal. Estas técnicas no sólo son apropiadas para codificar la voz, sino también para digitalizar cualquier señal analógica. Todos los métodos de compactación se basan en el principio de que la señal cambia con relativa lentitud en comparación con la frecuencia de muestreo, de modo que mucha de la información en el nivel digital de 7 u 8 bits es redundante.

Un método llamado **modulación de código de pulso diferencial** consiste en transmitir no la amplitud digitalizada sino la diferencia entre su valor actual y el previo. Puesto que los saltos de ±16 o más en una escala de 128 no son probables, podrían bastar 5 bits en lugar de 7. Si la señal llegara a saltar alocadamente en forma ocasional, la lógica de codificación podría requerir varios periodos de muestreo para "ponerse al día". En el caso de la voz, se puede ignorar el error que se introduce.

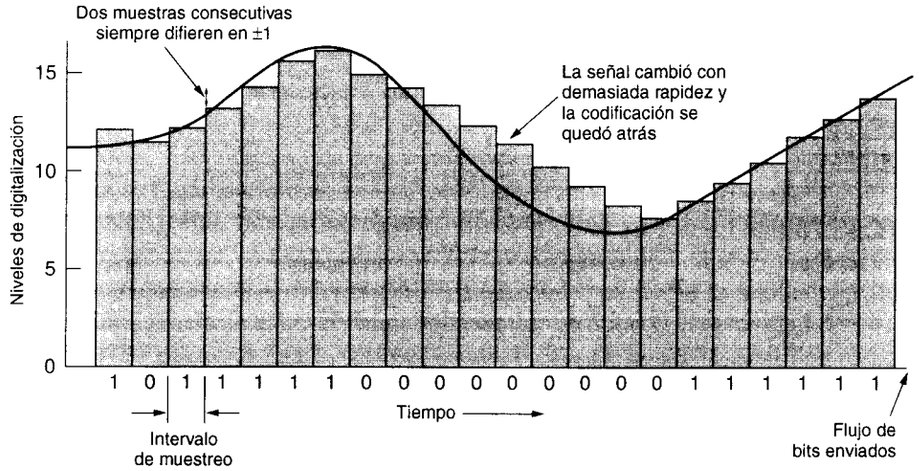


Figura 2-27. Modulación delta.

Una variante de este método de compactación requiere que cada valor muestreado difiera de su predecesor en +1 o −1. Se transmite un solo bit, que indica si la nueva muestra está por arriba o por debajo de la anterior. En la figura 2-27 se ilustra esta técnica, llamada **modulación delta**. Al igual que todas las técnicas de compactación que suponen cambios pequeños de nivel entre muestras consecutivas, la codificación delta se puede meter en problemas si la señal cambia con demasiada rapidez, como se aprecia en la figura. Cuando esto sucede, se pierde información.

Una mejora a la PCM diferencial consiste en extrapolar unos cuantos valores previos para predecir el siguiente valor y codificar a continuación la diferencia entre la señal real y la que se predice. Desde luego, el transmisor y el receptor deben usar el mismo algoritmo de predicción. A tales esquemas se les llama **codificación por predicción**, y son útiles porque reducen al tamaño de los números a codificar y, por tanto, la cantidad de bits a enviar.

Aunque la PCM se utiliza ampliamente en troncales interoficinas, el usuario de una computadora obtiene beneficios relativamente pequeños de ella si todos los datos se han de enviar a la oficina final en forma de onda senoidal analógica modulada a 28.8 kbps. Sería conveniente que la portadora conectara el lazo local en forma directa al sistema PCM troncal, de modo que la computadora pudiera emitir los datos digitales en forma directa por el lazo local a 1.544 o 2.048 Mbps. Desafortunadamente, los lazos locales no pueden operar a estas velocidades a lo largo de distancias muy grandes.

La multiplexión por división en el tiempo permite que se multiplexen varias portadoras T1 en portadoras de orden más alto. La figura 2-28 muestra cómo se puede hacer esto. A la izquierda vemos que se multiplexan cuatro canales T1 en un canal T2. La multiplexión en T2 y superiores se hace bit por bit, en lugar de byte por byte, como en los 24 canales de voz que forman un marco T1. Cuatro corrientes T1 a 1.544 Mbps deberían generar 6.176 Mbps, pero T2 es en realidad de 6.312 Mbps. Los bits extra sirven para enmarcar y para recuperación en caso de que la portadora pierda sincronía.

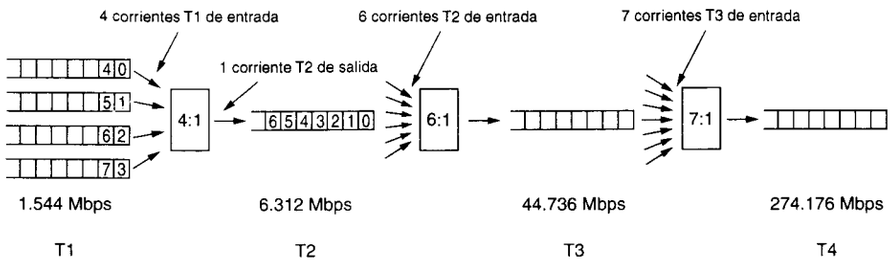


Figura 2-28. Multiplexión de corrientes T1 en portadoras más altas.

En el siguiente nivel se combinan seis corrientes T2 bit por bit para formar una corriente T3. A continuación se unen siete corrientes T3 para formar una corriente T4. En cada paso se añade una pequeña cantidad de bits extra para enmarcado y recuperación.

Así como existe desacuerdo en lo tocante a la portadora básica entre Estados Unidos y el resto del mundo, también hay desacuerdo respecto a cómo es que se ha de multiplexar en portadoras de más alto ancho de banda. El esquema de Estados Unidos de dar pasos de 4, 6 y 7 no pareció lógico a todo el mundo, de modo que el estándar de CCITT prescribe la multiplexión de cuatro corrientes en una en cada nivel. También, los datos de enmarcado y de recuperación son diferentes. La jerarquía del CCITT para 32, 128, 512, 2048 y 8192 canales funciona a velocidades de 2.048, 8.848, 34.304, 139.264 y 565.148 Mbps.

SONET/SDH

En los primeros días de la fibra óptica, cada compañía de teléfonos tenía su propio sistema óptico TDM patentado. Después de que AT&T se dividió en 1984, las compañías de teléfonos locales se tuvieron que conectar a múltiples portadoras de larga distancia, todas con diferentes sistemas ópticos TDM, de modo que se hizo obvia la necesidad de la estandarización. En 1985, Bellcore, la rama de investigación de las RBOC, empezó a trabajar en un estándar llamado **SONET** (*synchronous optical NETwork,* **red óptica sincrónica**). Más tarde el CCITT se unió al esfuerzo, que en 1989 produjo un estándar SONET y un conjunto de recomendaciones paralelas del CCITT (G.707, G.708 y G.709). A las recomendaciones del CCITT se les llama **SDH** (*synchronous digital hierarchy,* **jerarquía digital sincrónica**) pero difieren de SONET únicamente en detalles menores. Virtualmente, todo el tráfico telefónico de larga distancia en Estados Unidos y una buena parte del mismo en los demás países tiene ahora troncales que funcionan con SONET en la capa física. Al hacerse más baratos los circuitos SONET, las tarjetas de interfaz SONET para computadora pueden llegar a tener mayor difusión y se puede hacer más fácil para las compañías conectar sus computadoras en forma directa al corazón de la red telefónica por líneas rentadas especialmente acondicionadas. Más adelante hablaremos en forma breve de las metas y el diseño de SONET. Para información adicional, véase (Bellamy, 1991; y Omidyar y Aldridge, 1993).

El diseño de SONET tuvo cuatro objetivos principales. Antes que nada, SONET tenía que hacer posible la interconexión de redes de diferentes portadoras. El logro de este objetivo requirió que se definiera un estándar de señalización común con respecto a la longitud de onda, la temporización, la estructura de marcos y otras consideraciones.

Segundo, se necesitaron medidas para unificar los sistemas digitales estadounidense, europeo y japonés, todos los cuales se basaban en canales PCM de 64 kbps, pero combinados en formas diferentes (e incompatibles).

Tercero, SONET tenía que proporcionar un mecanismo para multiplexar varios canales digitales. En el momento en que se creó SONET, la portadora digital de mayor velocidad que se usaba ampliamente en Estados Unidos era la T3, a 44.736 Mbps. La T4 se había definido pero no se utilizaba mucho, y nada se había definido todavía por encima de la velocidad de T4. Parte de la misión de SONET era continuar la jerarquía a gigabits/seg y más allá. También se necesitaba una forma estándar de multiplexar canales más lentos en un solo canal SONET.

Cuarto, SONET tenía que proporcionar apoyo para las operaciones, la administración y el mantenimiento (OAM). Los sistemas anteriores no hacían esto muy bien.

Una decisión temprana fue convertir a SONET en un sistema TDM tradicional, con todo el ancho de banda de la fibra dedicado a un canal que contuviera segmentos de tiempo para los distintos subcanales. Como tal, SONET es un sistema sincrónico, controlado por un reloj maestro con una precisión de alrededor de 1 parte en 10^9. En una línea SONET, los bits se envían a intervalos extremadamente precisos, controlados por el reloj maestro.

Cuando posteriormente se propuso que la conmutación de células fuera la base del ISDN de banda ancha, el hecho de que permitiera la llegada de células a intervalos irregulares le confirió la etiqueta de modo de transferencia *asíncrono* (esto es, ATM) para contrastarlo con el funcionamiento sincrónico de SONET.

Un sistema SONET consiste en conmutadores, multiplexores y repetidoras, conectados todos por fibra. En la figura 2-29 se muestra la trayectoria desde un origen hasta un destino con un multiplexor intermedio y una repetidora intermedia. En la terminología de SONET, una fibra que va desde un dispositivo hasta cualquier otro en forma directa, con nada en medio, se llama **sección**. Una conexión entre dos multiplexores (posiblemente con una o más repetidoras en medio) se llama **línea**. Finalmente, la conexión entre la fuente y el destino (posiblemente con uno o más multiplexores y repetidoras) se llama **trayectoria**. La topología de SONET puede ser una malla, pero es frecuente que sea un anillo dual.

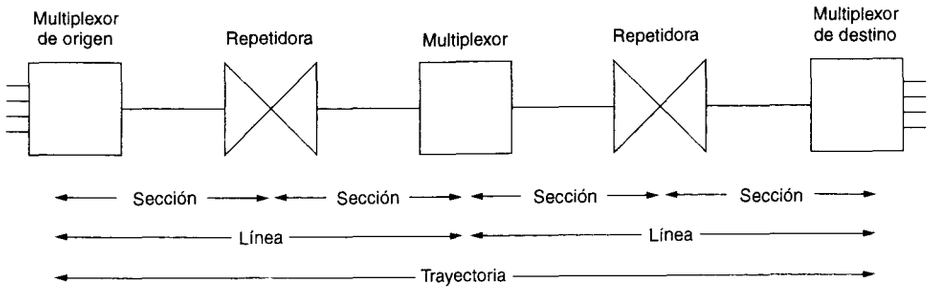


Figura 2-29. Una trayectoria SONET.

El marco básico de SONET es un bloque de 810 bytes que se emite cada 125 µseg. Puesto que SONET es sincrónica, los marcos se emiten haya o no datos útiles que enviar. La velocidad de 8000 marcos/seg coincide perfectamente con la tasa de muestreo de los canales de PCM que se utilizan en todos los sistemas de telefonía digital.

Los marcos de 810 bytes de SONET se pueden describir mejor como un rectángulo de bytes, de 90 columnas de ancho por 9 filas de altura. De este modo, $8 \times 810 = 6480$ bits se transmiten 8000 veces por segundo, dando una tasa de datos aproximada de 51.84 Mbps. Éste es el canal básico de SONET y se llama **STS-1** (*synchronous transport signal-1,* **señal sincrónica de transporte 1**). Todos los troncales de SONET son múltiplos de STS-1.

Las primeras tres columnas de cada marco se reservan para información de administración del sistema, como se ilustra en la figura 2-30. Las primeras tres filas contienen la información extra de la sección; las siguientes seis contienen la información extra de la línea. La primera se genera y verifica al comienzo y al final de cada sección, mientras que la segunda se genera y verifica al comienzo y al final de cada línea.

Las 87 columnas restantes contienen $87 \times 9 \times 8 \times 8000 = 50.112$ Mbps de datos de usuario. Sin embargo, los datos de usuario, llamados **SPE** (*synchronous payload envelope,* **envoltura de carga útil sincrónica**), no siempre empiezan en la fila 1, columna 4. La SPE puede empezar en cualquier parte dentro del marco. La primera fila de la información extra de línea contiene un apuntador al primer byte. La primera columna de la SPE es la información extra de la trayectoria (esto es, la cabecera para el protocolo de subcapa de la trayectoria de extremo a extremo).

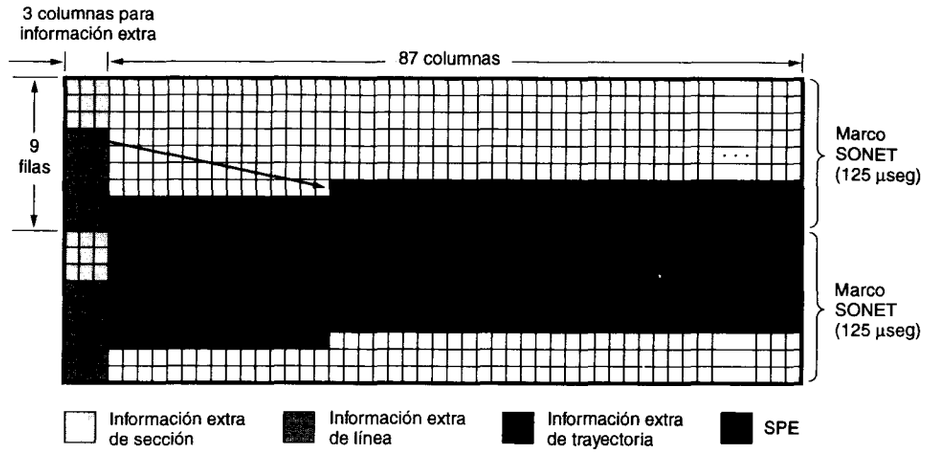


Figura 2-30. Dos marcos SONET consecutivos.

La facultad de que la SPE empiece en cualquier lugar dentro del marco SONET o incluso abarque dos marcos, como se muestra en la figura 2-30, confiere una flexibilidad adicional al sistema. Por ejemplo, si una carga útil llega al origen mientras se está construyendo un marco SONET falso, se puede insertar en el marco actual, en lugar de retenerlo hasta el inicio del siguiente. Esta facultad también es útil cuando la carga útil no se ajusta de manera exacta a un marco, como en el caso de una secuencia de células de ATM de 53 bytes. En tal caso, la primera fila de la información extra de línea puede apuntar al inicio de la primera célula llena y así establecer la sincronización.

La información extra de sección, de línea y de trayectoria contiene una profusión de bytes que se usan para operaciones, administración y mantenimiento. Puesto que cada byte ocurre 8000 veces por segundo, esto representa un canal de PCM. De hecho, tres de éstos se usan para proporcionar canales de voz al personal de mantenimiento de sección, línea y trayectoria. Se emplean otros bytes para encuadrado, paridad, vigilancia de errores, identificaciones, medición del tiempo, sincronización y otras funciones. Bellamy (1991) describe todos los campos en detalle.

La multiplexión de varias corrientes de datos, llamadas **tributarias**, desempeña un papel importante en SONET. La multiplexión se ilustra en la figura 2-31. A la izquierda, comenzamos con varias corrientes de entrada de baja velocidad que se convierten a la velocidad básica STS-1 de SONET, por lo regular añadiendo un relleno para redondear a 51.84 Mbps. A continuación, tres tributarias STS-1 se multiplexan en una corriente de salida STS-3 de 155.52 Mbps. A su vez, esta coriente se multiplexa con otras tres en una corriente final de salida que tiene 12 veces la capacidad de la corriente STS-1. En este punto la señal se revuelve para evitar que las series largas de ceros o unos interfieran la medición del tiempo, y se convierte de señal eléctrica en óptica.

La multiplexión se hace byte por byte. Por ejemplo, cuando tres tributarias STS-1 a 51.84 Mbps se combinan en una corriente STS-3 a 155.52 Mbps, el multiplexor emite primero un byte de la tributaria 1, después uno de la tributaria 2 y, finalmente, uno de la tributaria 3, antes de

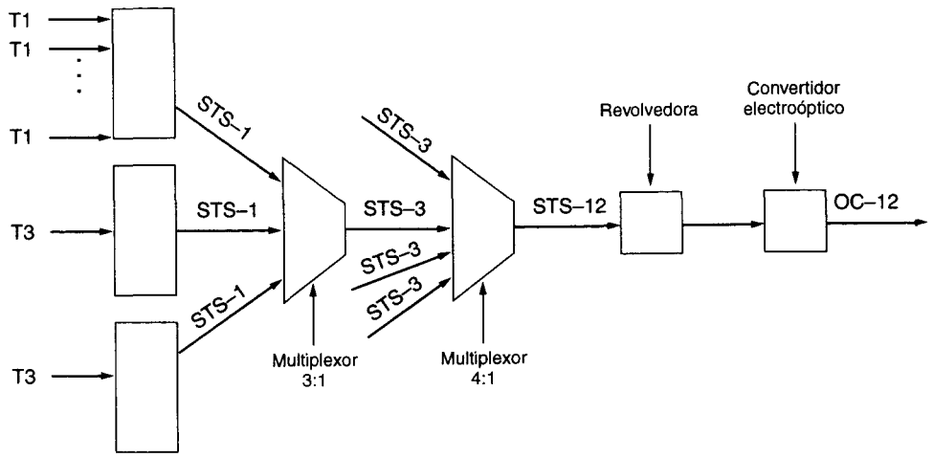


Figura 2-31. Multiplexión en SONET.

regresar a la 1. La figura de STS-3 análoga a la figura 2-30 muestra (de izquierda a derecha) las columnas de las tributarias 1, 2 y 3, en ese orden, después otra tripleta, y así sucesivamente, hasta la columna 270. Uno de estos marcos de 270×9 bytes se envía cada 125 µseg, lo que da una tasa de transmisión de datos de 155.52 Mbps.

En la figura 2-32 se muestra la jerarquía de multiplexión de SONET. Se definieron tasas de STS-1 a STS-48. La portadora óptica que corresponde a cada STS-*n* se llama OC-*n,* pero bit por bit es la misma, excepto por la revoltura que aparece en la figura 2-31. Los nombres de SDH son diferentes y empiezan en OC-3 porque los sistemas basados en CCITT no tienen una tasa de transmisión cercana a los 51.84 Mbps. La portadora OC-9 está presente porque se aproxima mucho a la velocidad de uno de los principales troncales de alta velocidad que se usan en Japón. Las OC-18 y OC-36 se usarán en Japón en el futuro. La tasa bruta de transmisión de datos incluye toda la información extra. La tasa de transmisión de SPE excluye la información extra de línea y de sección. La tasa de transmisión de datos de usuario excluye toda información extra y cuenta solamente las 86 columnas disponibles para la carga útil.

Por cierto, cuando una portadora, como la OC-3, no se multiplexa, sino que conduce datos de una fuente única, la letra *c* (de concatenado), se agrega a la designación, de modo que OC-3 indica una portadora de 155.52 Mbps que consiste en tres portadoras OC-1 independientes, pero OC-3c indica una corriente de datos de una sola fuente a 155.52 Mbps. Las tres corrientes OC-1 dentro de una corriente OC-3c se entrelazan por columnas, primero la columna 1 de la corriente 1, a continuación la columna 1 de la corriente 2, después la columna 1 de la corriente 3 seguida de la columna 2 de la corriente 1, y así sucesivamente, lo que conduce a un marco de 270 columnas de ancho y 9 filas de profundidad.

La cantidad de datos de usuario reales en una corriente OC-3c es un poco más alta que en una OC-3 (149.760 Mbps contra 148.608 Mbps) porque la columna de información extra de trayectoria se incluye dentro de la SPE sólo una vez, en lugar de tres, en el caso de tres corrientes

SONET		SDH	Tasa de datos (Mbps)		
Eléctrico	Óptico	Óptico	Bruta	SPE	De usuario
STS-1	OC-1		51.84	50.112	49.536
STS-3	OC-3	STM-1	155.52	150.336	148.608
STS-9	OC-9	STM-3	466.56	451.008	445.824
STS-12	OC-12	STM-4	622.08	601.344	594.432
STS-18	OC-18	STM-6	933.12	902.016	891.648
STS-24	OC-24	STM-8	1244.16	1202.688	1188.864
STS-36	OC-36	STM-12	1866.24	1804.032	1783.296
STS-48	OC-48	STM-16	2488.32	2405.376	2377.728

Figura 2-32. Tasas de multiplexión de SONET y SDH.

independientes OC-1. En otras palabras, en OC-3c están disponibles para datos de usuario 260 de las 270 columnas, mientras que en OC-3 solamente están disponibles 258 columnas para datos de usuario. También existen marcos concatenados de orden más alto (por ejemplo, OC-12c).

Ahora debe quedar claro por qué ATM funciona a 155 Mbps: la intención es transportar células de ATM por troncales OC-3c de SONET. También debe quedar claro que la cifra tan citada de 155 Mbps es la velocidad de transmisión bruta, que incluye la información extra de SONET. Además, en algún momento alguien redondeó de manera incorrecta los 155.52 Mbps a 155 Mbps en lugar de 156 Mbps y ahora todos los demás lo hacen mal también.

La capa física de SONET se divide hasta en cuatro subcapas, como se muestra en la figura 2-33. La subcapa más baja es la **subcapa fotónica**; tiene que ver con la especificación de las propiedades físicas de la luz y de la fibra que se usarán.

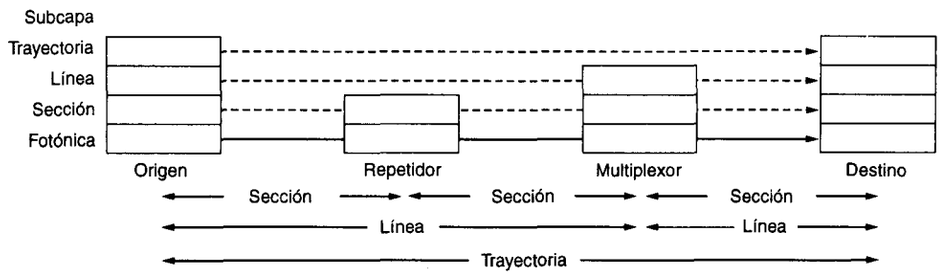


Figura 2-33. La arquitectura SONET.

Las tres subcapas restantes corresponden a las secciones, líneas y trayectorias. La subcapa de sección maneja una sola conexión de fibra de punto a punto, generando un marco estándar en un extremo y procesándolo en el otro. Las secciones pueden comenzar y terminar en repetidoras, que sólo amplifican y regeneran los bits, pero no los cambian o procesan en forma alguna.

La subcapa de línea se encarga de la multiplexión de varias tributarias en una sola línea y de desmultiplexarlas en el otro extremo. Para la subcapa de línea, las repetidoras son transparentes. Cuando un multiplexor envía bits por una fibra, espera que lleguen al siguiente multiplexor sin cambios, no importa por cuántas repetidoras tengan que pasar. Así, el protocolo de la subcapa de línea se usa entre dos multiplexores y tiene que ver con situaciones tales como cuántas entradas se van a multiplexar y cómo se hará. En contraste, la subcapa y el protocolo de trayectoria tienen que ver con emisiones de extremo a extremo.

2.4.5. Conmutación

Desde el punto de vista de un ingeniero de telefonía ordinario, el sistema telefónico se divide en dos partes: planta externa (los lazos locales y troncales, puesto que están fuera de las oficinas de conmutación) y la planta interna (los conmutadores). Sólo hemos visto la planta externa. Ahora es el momento de examinar la planta interna.

Se utilizan dos técnicas de conmutación diferentes dentro del sistema telefónico: conmutación de circuitos y conmutación de paquetes. A continuación presentaremos una breve introducción a cada una de ellas. Después veremos con detalle la conmutación de circuitos, porque así es como trabaja el actual sistema de teléfonos. Más adelante en este mismo capítulo examinaremos a fondo la conmutación de paquetes en el contexto de la siguiente generación del sistema telefónico, la ISDN de banda ancha.

Conmutación de circuitos

Cuando usted o su computadora hacen una llamada telefónica, el equipo de conmutación del sistema telefónico busca una trayectoria física de "cobre" (lo que incluye la fibra y la radio) que vaya desde su teléfono al del receptor. Esta técnica se llama **conmutación de circuitos** y se muestra de manera esquemática en la figura 2-34(a). Cada uno de los seis rectángulos representa una oficina de conmutación de la portadora (oficina final, oficina de cargo, etc.). En este ejemplo, cada oficina tiene tres líneas de entrada y tres de salida. Cuando una llamada pasa por una oficina de conmutación, se establece una conexión física (en forma conceptual) entre la línea por la que llegó la llamada y una de las líneas de salida, como indican las líneas punteadas.

En los primeros días del teléfono, se establecía la conexión cuando el operador conectaba un cable puenteador en los enchufes de entrada y de salida. Por cierto, existe una pequeña y sorprendente historia asociada a la invención del equipo de conmutación automática de circuitos: lo inventó el dueño de una funeraria del siglo XIX, un hombre llamado Almon B. Strowger. Poco después de que se inventara el teléfono, cuando alguien moría, alguno de los deudos llamaba a la operadora del pueblo y decía: "Por favor, comuníqueme con una funeraria". Desafortunadamente para el señor Strowger, había dos funerarias en el pueblo, y la esposa del dueño de la otra era la operadora de teléfonos. Strowger pronto se dio cuenta de que si no inventaba el equipo de conmutación telefónica automática iba a quedar en bancarrota, así que eligió la primera opción. Durante casi 100 años, el equipo de conmutación de circuitos empleado en todo el mundo se conoció como el aparato de Strowger. (La historia no registra si la ahora desempleada

operadora de conmutador obtuvo trabajo como operadora de información, contestando preguntas como: ¿Me da por favor el número telefónico de una funeraria?

Desde luego, el modelo que se muestra en la figura 2-34(a) está altamente simplificado, porque partes de la trayectoria de "cobre" entre los dos teléfonos pueden ser, de hecho, enlaces de microondas en los cuales se multiplexan miles de llamadas. Sin embargo, la idea básica es válida: una vez que se ha establecido una llamada, existe una trayectoria dedicada entre ambos extremos y continuará existiendo hasta que termine la llamada.

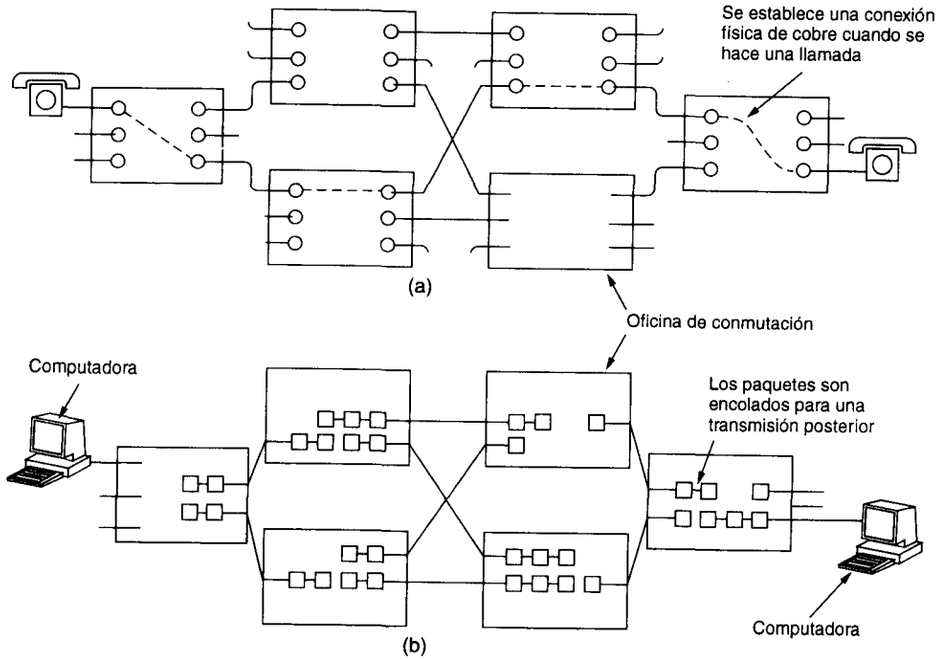


Figura 2-34. (a) Conmutación de circuitos. (b) Conmutación de paquetes.

Una propiedad importante de la conmutación de circuitos es la necesidad de establecer una trayectoria de un extremo a otro *antes* de que se pueda enviar cualquier dato. El tiempo que transcurre entre que se termina de marcar y que el timbre comienza a sonar puede ser fácilmente de 10 seg, y más en las llamadas de larga distancia o internacionales. Durante este intervalo de tiempo, el sistema telefónico está buscando una trayectoria de cobre, como se muestra en la figura 2-35(a). Observe que antes de que pueda comenzar la transmisión de datos, la señal de petición de llamada se debe propagar hasta el destino, y debe ser reconocida. En muchas aplicaciones de computadora (por ejemplo, la verificación de crédito en un punto de venta), son indeseables los tiempos de establecimiento largos.

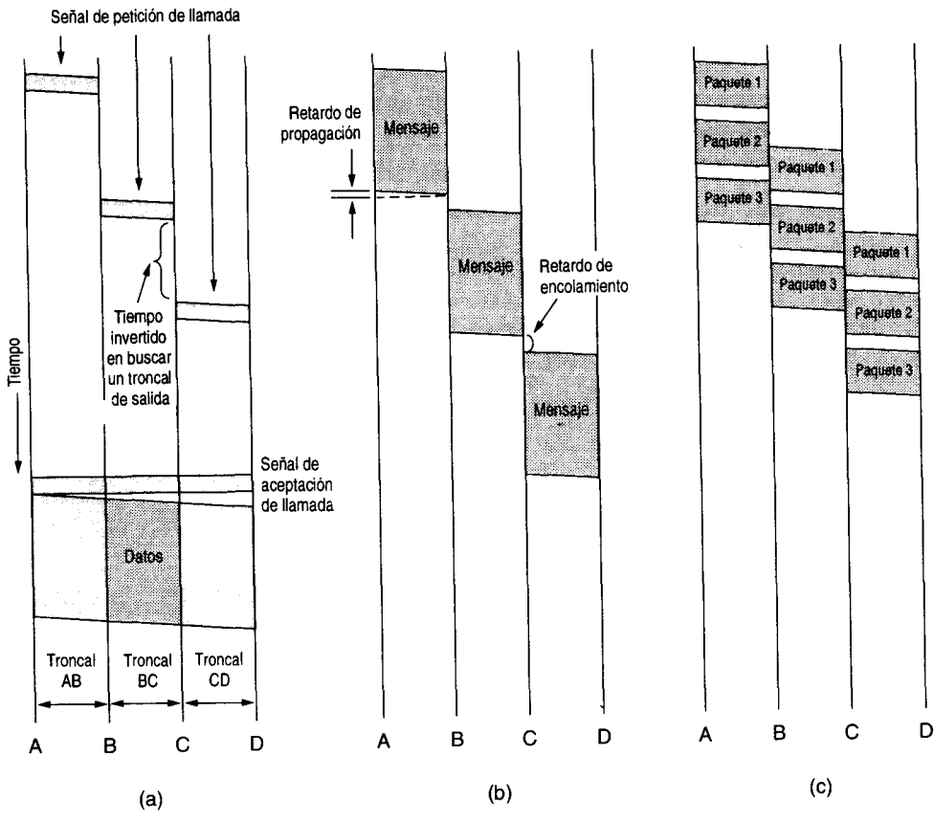


Figura 2-35. Tiempos de los sucesos en (a) conmutación de circuitos, (b) conmutación de mensajes, (c) conmutación de paquetes.

Al existir una trayectoria de cobre entre las partes en comunicación, una vez completado el establecimiento el único retardo de los datos es el tiempo de propagación de la señal electromagnética, alrededor de 5 mseg por cada 1000 km. Otra ventaja de la trayectoria establecida es que no hay peligro de congestión; esto es, una vez que la llamada entró, no hay posibilidad de obtener una señal de ocupado, aunque podría obtener una antes de establecerse la conexión debido a la falta de capacidad de conmutación o de troncal.

Una estrategia de conmutación alterna es la **conmutación de mensajes** que se muestra en la figura 2-35(b). Cuando se usa esta forma de conmutación, no se establece por adelantado una trayectoria de cobre física entre el emisor y el receptor. En cambio, cuando el emisor tiene un marco de datos para enviar, éste se almacena en la primera oficina de conmutación (es decir, enrutador) y después se reenvía, un salto a la vez. Cada marco se recibe en su totalidad, se inspecciona en busca de errores, y después se retransmite. Una red que usa esta técnica se llama red de **almacenar y reenviar**, como se mencionó en el capítulo 1.

Los primeros sistemas de telecomunicación electromecánicos usaban conmutación de circuitos para enviar telegramas. El mensaje se perforaba en cinta de papel fuera de línea en la oficina emisora y después se leía y transmitía por una línea de comunicación a la siguiente oficina a lo largo del recorrido, donde se perforaba en cinta de papel. Allí, una operadora arrancaba la cinta de papel y la insertaba en una de las muchas lectoras de cinta, una por cada troncal de salida. Tal oficina de conmutación se llamaba **oficina de arrancado de cinta de papel**.

Con la conmutación de mensajes, no hay límite para el tamaño de los marcos, lo que significa que los enrutadores (en un sistema moderno) deben tener discos para almacenar en forma temporal los marcos largos. También significa que un solo marco puede acaparar una línea de enrutador a enrutador durante minutos, lo que hace inútil la conmutación de mensajes para el tráfico interactivo. Con el fin de resolver estos problemas se inventó la **conmutación de paquetes**. Las redes de conmutación de paquetes establecen un límite superior al tamaño del marco, lo que permite almacenar los paquetes en la memoria principal del enrutador en lugar de hacerlo en disco. Al asegurarse de que ningún usuario pueda monopolizar una línea de transmisión durante mucho tiempo (milisegundos), las redes de conmutación de paquetes pueden manejar tráfico interactivo. En la figura 2-35(b) y (c) se muestra una ventaja adicional de la conmutación de paquetes sobre la conmutación de mensajes: el primer paquete de un mensaje de varios paquetes se puede reenviar antes de que el segundo haya llegado por completo, lo que reduce el retardo y mejora el rendimiento. Por estas razones, las redes de computadoras normalmente son de conmutación de paquetes, ocasionalmente de conmutación de circuitos y nunca de conmutación de mensajes.

La conmutación de circuitos y la de paquetes difieren en muchos aspectos. La diferencia clave es que la conmutación de circuitos reserva de manera estática por adelantado el ancho de banda requerido, mientras que la conmutación de paquetes lo adquiere y lo libera según se necesita. Con la conmutación de circuitos, cualquier ancho de banda que no se use en un circuito asignado sencillamente se desperdicia. Con la conmutación de paquetes este ancho de banda se puede utilizar para transmitir otros paquetes de fuentes no relacionadas que van a destinos no relacionados porque los circuitos nunca son dedicados. Por otro lado, debido a que no hay circuitos dedicados, una crecida súbita en el tráfico de entrada puede saturar un enrutador, excediendo su capacidad de almacenamiento y provocando que pierda paquetes.

En contraste con la conmutación de circuitos, cuando se usa la conmutación de paquetes resulta sencillo para los enrutadores efectuar conversiones de velocidad y de código. También pueden realizar corrección de errores en cierto grado. Sin embargo, en algunas redes de conmutación de paquetes éstos se pueden entregar a su destino en el orden equivocado, cosa que nunca puede suceder con la conmutación de circuitos.

Otra diferencia es que la conmutación de circuitos es totalmente transparente. El emisor y el receptor pueden usar cualquier velocidad, formato o método de encuadrado de bits que quieran. La portadora no lo sabe ni le interesa. Con la conmutación de paquetes la portadora determina los parámetros básicos. Una analogía burda sería comparar un camino con una vía de tren. En el primero, el usuario determina el tamaño, la velocidad y la naturaleza del vehículo; en la vía del tren esto lo hace el prestador de servicios. Esta transparencia es la que hace posible que coexistan voz, datos y fax dentro del sistema telefónico.

Una diferencia final entre la conmutación de circuitos y la de paquetes es el algoritmo de cobro. Las portadoras de paquetes por lo general basan sus cargos tanto en la cantidad de bytes (o paquetes) transmitidos como en el tiempo de conexión. Además, la distancia de transmisión normalmente no importa, excepto quizás en lo internacional. Con la conmutación de circuitos, el cargo se basa sólo en la distancia y el tiempo, no en el tráfico. Las diferencias se resumen en la figura 2-36.

Elemento	Conmutación de circuitos	Conmutación de paquetes
Trayectoria de "cobre" dedicada	Sí	No
Ancho de banda disponible	Fijo	Dinámico
Puede desperdiciarse ancho de banda	Sí	No
Transmisión de almacenamiento y reenvío	No	Sí
Cada paquete sigue la misma ruta	Sí	No
Establecimiento de llamada	Requerido	No es necesario
Cuándo puede haber congestión	Durante el establecimiento	En cada paquete
Cargos	Por minuto	Por paquete

Figura 2-36. Comparación de las redes de conmutación de circuitos y de conmutación de paquetes.

Tanto la conmutación de circuitos como la de paquetes son tan importantes que regresaremos a ellas dentro de poco y describiremos en detalle las diversas tecnologías que se usan.

La jerarquía de conmutación

Vale la pena decir unas palabras acerca de cómo se efectúa el enrutamiento entre conmutadores dentro del actual sistema telefónico de conmutación de circuitos. Describiremos aquí el sistema de AT&T, pero otras compañías y otros países utilizan los mismos principios generales. El sistema telefónico tiene cinco clases de oficinas de conmutación, como se ilustra en la figura 2-37. Hay 10 oficinas de conmutación regionales plenamente interconectadas mediante 45 troncales de fibra óptica de alto ancho de banda. Abajo de las oficinas regionales están 67 oficinas de sección, 230 oficinas primarias, 1300 oficinas de cargo y 19,000 oficinas finales. Los cuatro niveles más bajos se conectaban originalmente en forma de árbol.

Por lo general, las llamadas se conectan en el nivel más bajo posible. Así, si un suscriptor conectado a la oficina final 1 llama a otro suscriptor conectado a la oficina final 1, la llamada se completará en esa oficina. Por otro lado, una llamada de un cliente conectado a la oficina final 1 de la figura 2-37 a un cliente conectado a la oficina final 2 tendrá que ir a la oficina de cargo 1. En cambio, una llamada de la oficina final 1 a la oficina final 4 tendrá que ascender a la oficina primaria 1, y así sucesivamente. Con un árbol puro sólo existe una ruta mínima, y normalmente es la que se toma.

Después de años de operación, las compañías de teléfonos notaron que algunas rutas estaban más ocupadas que otras. Por ejemplo, había muchas llamadas de Nueva York a Los Ángeles. En

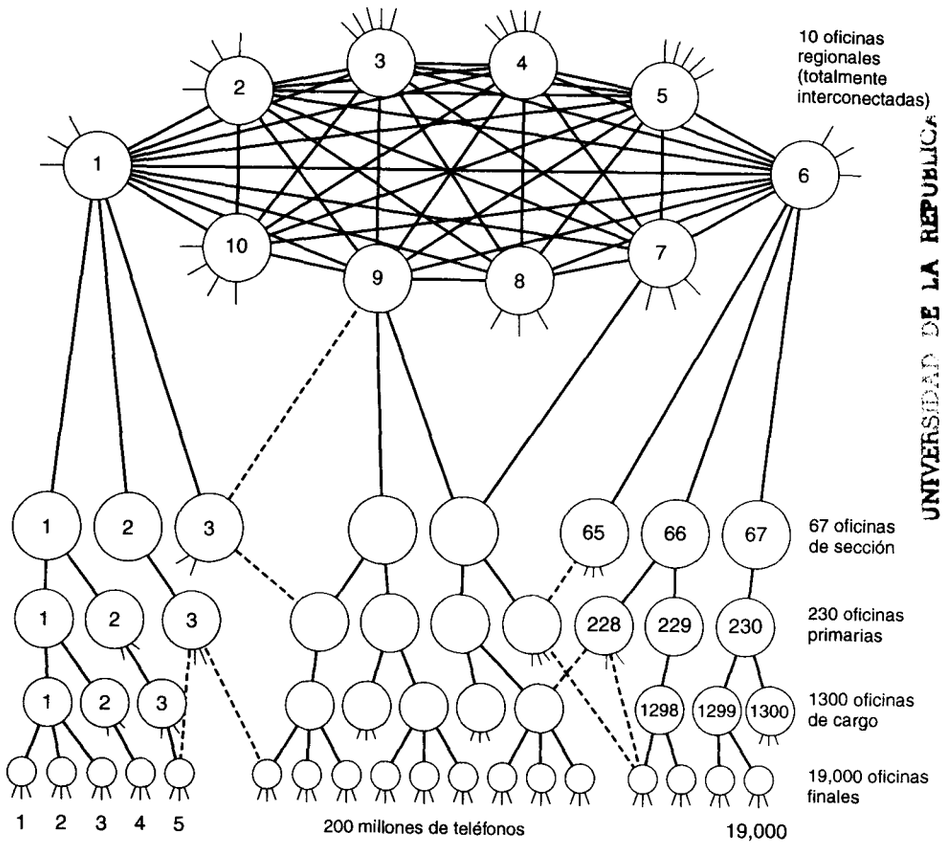


Figura 2-37. La jerarquía telefónica de AT&T. Las líneas punteadas son troncales directos.

lugar de subir hasta lo más alto de la jerarquía, simplemente instalaron **troncales directos** para las rutas ocupadas. En la figura 2-37 se muestran algunos de éstos como líneas punteadas. En consecuencia, ahora muchas llamadas se pueden enrutar por múltiples trayectorias. Por lo general, la ruta que se elige es la más directa, pero si los troncales necesarios están llenos, se elige una opción alterna. Este complejo enrutamiento es posible ahora porque las máquinas conmutadoras, como la ESS de AT&T, de hecho son sólo computadoras de propósito general con una gran cantidad de equipo de E/S muy especializado.

Conmutadores de matriz

Pasemos ahora de cómo se encaminan las llamadas entre conmutadores a cómo funcionan en realidad los conmutadores individuales por dentro. Dentro del sistema telefónico son (o fueron) comunes varias clases de conmutadores. La clase más simple es el **conmutador de matriz** (llamado también **conmutador de puntos de cruce**), el cual se muestra en la figura 2-38. En un

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

conmutador con n líneas de entrada y n líneas de salida (esto es, n líneas dúplex), el conmutador de matriz tiene n^2 intersecciones llamadas **puntos de cruce**, donde se pueden conectar una línea de entrada y una de salida con un conmutador semiconductor, como se aprecia en la figura 2-38(a). En la figura 2-38(b) vemos un ejemplo en el que la línea 0 se conecta a la línea 4, la línea 2-38(a). En la figura 2-38(b) vemos un ejemplo en el que la línea 0 se conecta a la línea 4, la línea 1 se conecta a la 7 y la línea 2 se conecta a la 6. Las líneas 3 y 5 no se conectan. Todos los bits que llegan al conmutador por la línea 4, por ejemplo, se envían de inmediato por la línea 0. De este modo, el conmutador de matriz instrumenta la conmutación de circuitos al efectuar una conexión eléctrica directa, casi como la de los cables puenteadores de los conmutadores de la primera generación, sólo que en forma automática y en microsegundos.

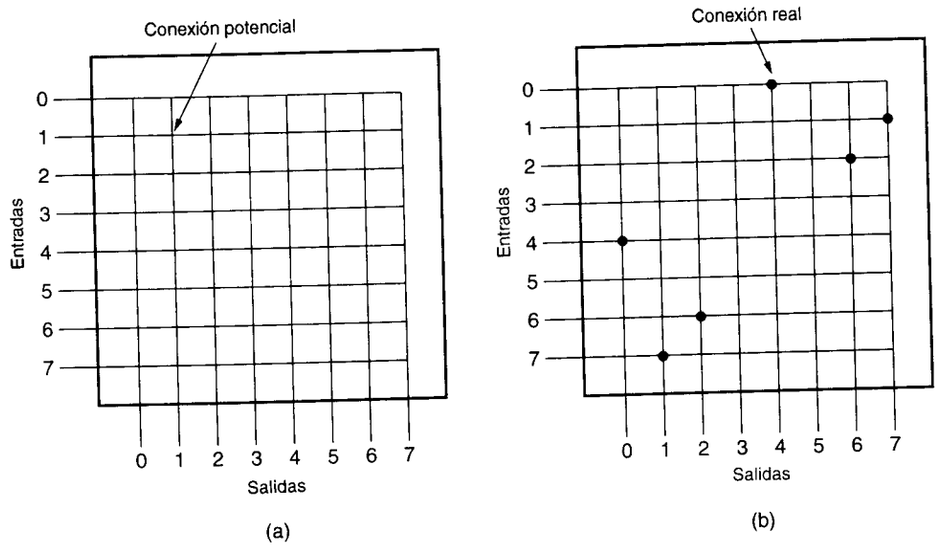


Figura 2-38. (a) Conmutador de matriz sin conexiones. (b) Conmutador de matriz con tres conexiones establecidas: 0 con 4, 1 con 7 y 2 con 6.

El problema con un conmutador de matriz es que la cantidad de cruces crece al ritmo del cuadrado de la cantidad de líneas en el conmutador. Si suponemos que todas las líneas son dúplex y que no existen autoconexiones, solamente se necesitan los puntos de cruce arriba de la diagonal, que son $n(n-1)/2$. Para $n = 1000$, necesitamos 499,500 puntos de cruce. Aunque es posible construir un circuito integrado VLSI con esta cantidad de conmutadores de transistores, no lo es tener 1000 terminales en el circuito integrado. Por ello, un solo conmutador de matriz es útil solamente para oficinas finales relativamente pequeñas.

Conmutadores por división en el espacio

Si dividimos el conmutador de matriz en marcos pequeños y los interconectamos podemos construir conmutadores de múltiples etapas con muchos menos puntos de cruce. Éstos se

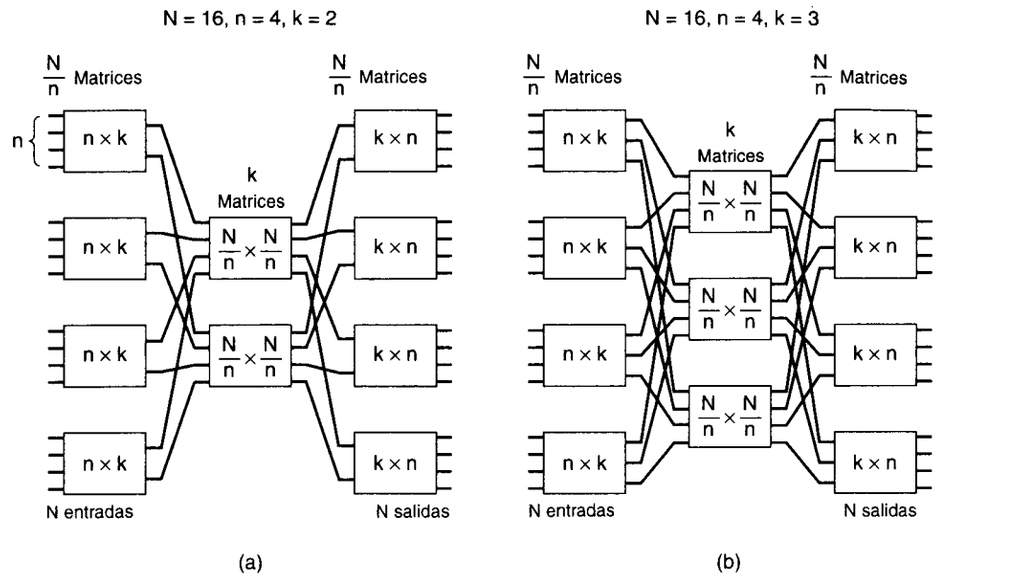


Figura 2-39. Dos conmutadores por división en el espacio con diferentes parámetros.

llaman **conmutadores por división en el espacio**. En la figura 2-39 se ilustran dos configuraciones.

Para no complicar nuestro ejemplo sólo consideraremos conmutadores de tres etapas, pero son posibles conmutadores con más etapas. En estos ejemplos tenemos un total de N entradas y N salidas. En lugar de construir un solo conmutador de matriz $N \times N$, construimos el conmutador con matrices rectangulares más chicas. En la primera etapa, cada matriz tiene n entradas, así que necesitamos N/n de ellas para manejar todas las N líneas que entran.

La segunda etapa tiene k matrices, cada una con N/n entradas y N/n salidas. La tercera etapa es una repetición de la primera, pero invertida de izquierda a derecha. Cada matriz intermedia se conecta a cada matriz de entrada y cada matriz de salida. En consecuencia, es posible conectar cada entrada a cada salida usando ya sea la primera matriz intermedia de la figura 2-39 o bien la segunda. De hecho, existen dos trayectorias disjuntas de cada entrada a cada salida, dependiendo de cuál matriz intermedia se escoja. En la figura 2-39(b) hay tres trayectorias para cada par de entrada/salida. Con k etapas intermedias (k es un parámetro de diseño), existen k trayectorias disjuntas.

Calculemos ahora la cantidad de puntos de cruce necesarios para un conmutador de tres etapas. En la primera etapa hay N/n matrices, cada una con nk puntos de cruce, para un total de Nk. En la segunda etapa hay k matrices, cada una con $(N/n)^2$ puntos de cruce. La tercera etapa es como la primera. Al sumar las tres etapas obtenemos

$$\text{Cantidad de puntos de cruce} = 2kN + k(N/n)^2$$

Para $N = 1000$, $n = 50$ y $k = 10$ solamente necesitamos 24,000 puntos de cruce en lugar de los 499,500 que se requieren en una matriz de una sola etapa de 1000×1000.

Desafortunadamente, como es lo usual, nada es gratis. El conmutador se puede bloquear. Considere de nuevo la figura 2-39(a). La etapa 2 tiene ocho entradas, de modo que se puede conectar un máximo de ocho llamadas al mismo tiempo. Si llega la llamada nueve, tendrá que recibir una señal de ocupado, aunque el destino esté disponible. El conmutador de la figura 2-39(b) es mejor porque puede manejar un máximo de 12 llamadas en lugar de 8, pero tiene más puntos de cruce. Algunas veces, al efectuar una llamada telefónica podemos recibir una señal de ocupado antes de terminar de marcar. Esto probablemente se debió a que se bloqueó parte del recorrido a través de la red.

Debe ser obvio que, cuanto mayor sea k, más costoso será el conmutador y menor la probabilidad de bloqueo. En 1953, Clos demostró que cuando $k = 2n - 1$, el conmutador nunca se bloqueará (Clos, 1953). Otros investigadores han analizado con gran detalle los patrones de llamadas para construir conmutadores que en teoría se puedan encuadrar pero que en la práctica sólo lo hagan rara vez.

Conmutadores por división en el tiempo

Una clase de conmutador totalmente diferente es el **conmutador por división en el tiempo** que se ilustra en la figura 2-40. En la conmutación por división en el tiempo, las n líneas de entrada se examinan en secuencia para construir un marco de entrada con n ranuras. Cada ranura tiene k bits. En los conmutadores T1 las ranuras tienen 8 bits y se procesan 8000 marcos por segundo.

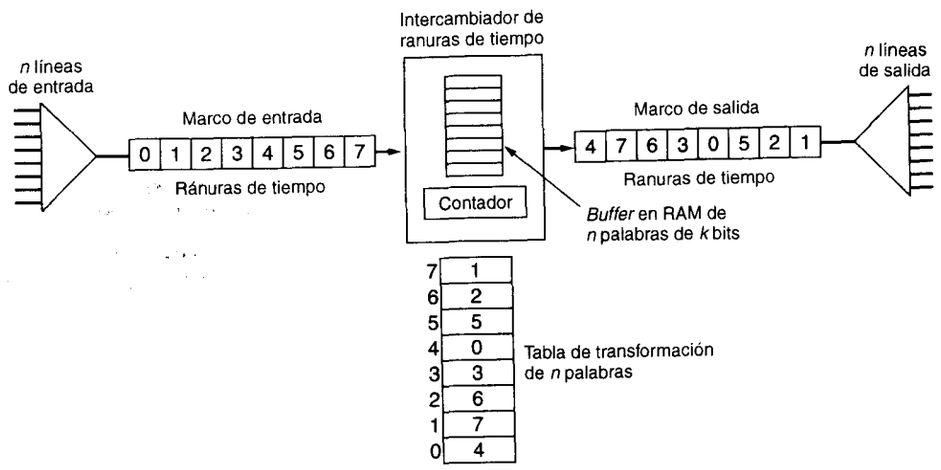


Figura 2-40. Conmutador por división en el tiempo.

El corazón del conmutador por división en el tiempo es el **intercambiador de ranuras de tiempo**, que acepta marcos de entrada y produce marcos de salida en los que se han reordenado

las ranuras de tiempo. En la figura 2-40, la ranura de entrada 4 sale primero, a continuación la 7, y así sucesivamente. Por último, el marco de salida se desmultiplexa y la ranura de salida 0 (ranura de entrada 4) sale por la línea 0, etc. En esencia, el conmutador ha transferido un byte de la línea de entrada 4 a la línea de salida 0, otro byte de la línea de entrada 7 a la línea de salida 1, y así sucesivamente. Visto desde fuera, el arreglo completo es un conmutador de circuitos, aunque no existan conexiones físicas.

El intercambiador de ranuras de tiempo funciona como sigue: cuando un marco de entrada está listo para procesarse, cada ranura (esto es, cada byte del marco de entrada) se escribe en un *buffer* de RAM dentro del intercambiador. Las ranuras se escriben en orden, de modo que la palabra i del *buffer* contiene la ranura i.

Una vez almacenadas en el *buffer* todas las ranuras del marco de entrada, se construye el marco de salida leyendo las palabras del *buffer*, pero esta vez en un orden diferente. Un contador va de 0 a $n - 1$. En el paso j se lee el contenido de la palabra j de una tabla de transformación y se usa para apuntar a una dirección del *buffer* en RAM. Así, si la palabra 0 de la tabla de transformación contiene un 4, se leerá primero la palabra 4 del *buffer* en RAM y la primera ranura del marco de salida será la ranura 4 del marco de entrada. De este modo, el contenido de la tabla de transformación determina cuál permutación del marco de entrada se generará como marco de salida, y por ende cuál línea de entrada se conectará con cuál línea de salida.

Los conmutadores por división en el tiempo usan tablas que son lineales en lugar de cuadráticas en cuanto a la cantidad de líneas, pero tienen otra limitación. Es necesario almacenar n ranuras en la memoria temporal RAM y luego leerlas otra vez en un periodo de marco de 125 µseg. Si cada uno de estos accesos a la memoria tarda T microsegundos, el tiempo necesario para procesar un marco es $2nT$ microsegundos, así que tenemos $2nT = 125$ o $n = 125/2T$. Si la memoria tiene un tiempo de ciclo de 100 nseg, podemos manejar cuando mucho 625 líneas. También podemos invertir esta relación y usarla para determinar el ciclo de memoria requerido para manejar una cantidad determinada de líneas. Al igual que con un conmutador de matriz, es posible idear conmutadores de múltiples etapas que dividan el trabajo en varias partes y después combinen los resultados para poder manejar mayor cantidad de líneas.

UNIVERSIDAD DE
FACULTAD DE
DEPARTAMENTO
DOCUMENTACION
MONTEVIDEO - URUGUAY

2.5. ISDN DE BANDA ESTRECHA

Durante más de un siglo, la principal infraestructura de telecomunicaciones internacional ha sido el sistema telefónico público de conmutación de circuitos. Este sistema se diseñó para la transmisión analógica de voz y es inadecuado para las necesidades de las comunicaciones modernas. Anticipando una demanda considerable por parte de los usuarios de un servicio digital de extremo a extremo (esto es, no como el de la figura 2-17, el cual tiene una parte digital y otra analógica), las compañías de teléfonos del mundo y las PTT se unieron en 1984 bajo los auspicios de la CCITT y estuvieron de acuerdo en construir un sistema de teléfonos de conmutación de circuitos nuevo, completamente digital, para principios del siglo XXI. Este nuevo sistema, llamado **ISDN** (*Integrated Services Digital Network,* **red digital de servicios integrados**), tiene como meta principal la integración de servicios de voz y sin voz. ISDN ya está disponible en muchas lo-

calidades y su uso está creciendo lentamente. En las siguientes secciones describiremos qué hace y cómo trabaja. Para información adicional, véase (Dagdeviren *et al.*, 1994; y Kessler, 1993).

2.5.1. Servicios ISDN

El servicio clave de ISDN continuará siendo la voz, aunque se añadirán muchas características mejoradas. Por ejemplo, muchos gerentes de compañías tienen un botón de intercomunicación en sus teléfonos para llamar a sus secretarias en forma instantánea (sin tiempo de establecimiento de llamada). Una característica de ISDN son los teléfonos con múltiples botones para establecer llamadas inmediatas con teléfonos en cualquier parte del mundo. Otra posibilidad es un teléfono que exhibe el número, nombre y dirección de quien llama en una pantalla mientras el teléfono suena. Una versión más avanzada de este recurso permite que el teléfono se conecte a una computadora para que se exhiba el registro de base de datos de quien llama cuando la llamada entra. Por ejemplo, un corredor de bolsa podría arreglar que cuando se conteste el teléfono la cartera de quien llama esté ya en la pantalla junto con los precios actuales de todas sus acciones. Otros servicios de voz avanzados incluyen el redireccionamiento de llamadas y las llamadas de conferencias en todo el mundo.

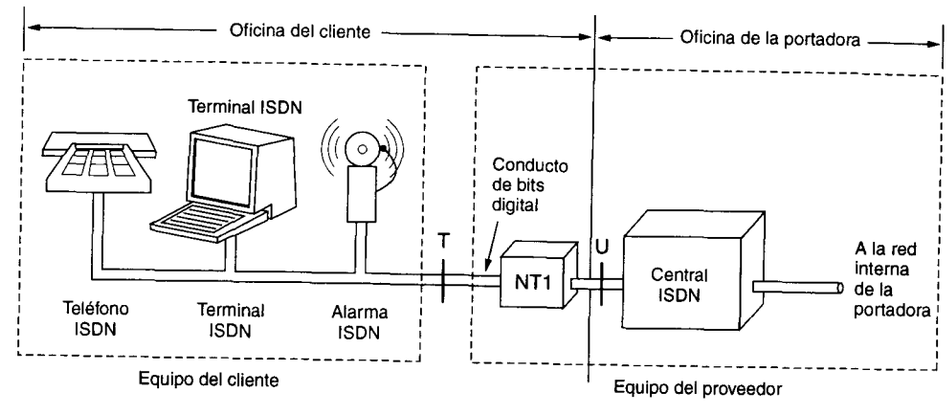
Los servicios avanzados que no son de voz incluyen tomar la lectura del medidor de electricidad en forma remota y alarmas en línea médicas, contra ladrones, y de humo que llaman en forma automática al hospital, a la policía o al departamento de bomberos, respectivamente, y proporcionan la dirección para agilizar la respuesta.

2.5.2. Arquitectura del sistema ISDN

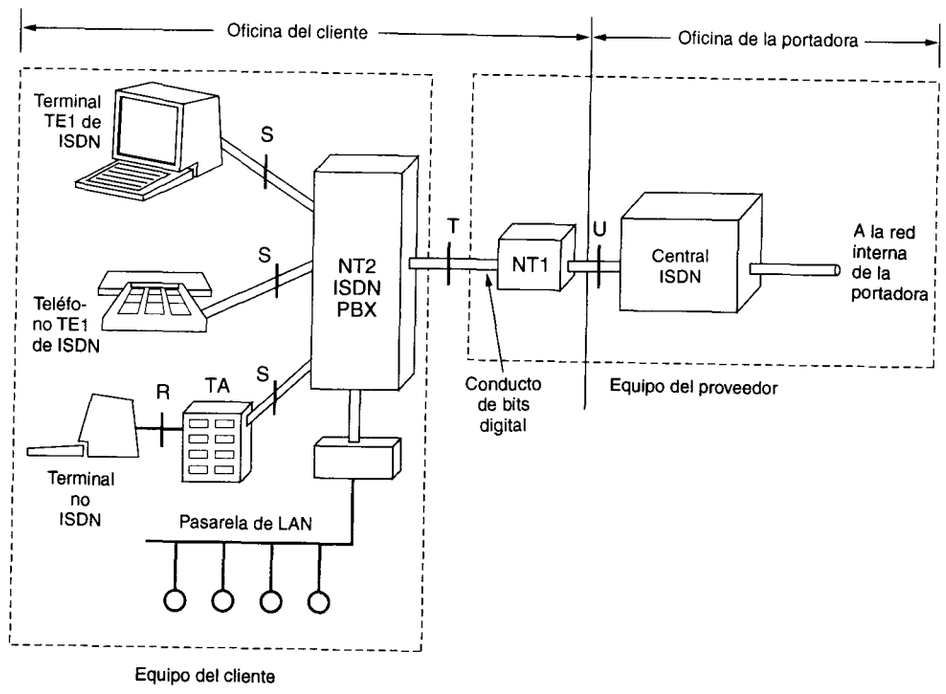
Es el momento de examinar la arquitectura de ISDN en detalle, particularmente el equipo del cliente y la interfaz entre el cliente y la compañía telefónica o PTT. La idea clave en que se basa la ISDN es la del **conducto digital de bits**, un conducto conceptual entre el cliente y la portadora a través del cual fluyen los bits. No importa si los bits se originan en un teléfono digital, una terminal digital, una máquina de fax digital, o algún otro dispositivo. Todo lo que importa es que los bits puedan fluir a través del conducto en ambas direcciones.

El conducto de bits digital puede, y normalmente lo hace, manejar múltiples canales independientes por medio de la multiplexión por división en el tiempo del flujo de bits. El formato exacto del flujo de bits y su multiplexión es una parte cuidadosamente definida de la especificación de la interfaz para el conducto de bits digital. Se han elaborado dos estándares principales para el conducto de bits, un estándar con un ancho de banda bajo para uso del hogar y otro con ancho de banda más alto para uso de empresas que maneja múltiples canales que son idénticos al canal de uso del hogar. Además, las empresas pueden tener múltiples conductos de bits si necesitan capacidad adicional más allá de la que el conducto empresarial estándar puede proporcionar.

En la figura 2-41(a) vemos la configuración normal para un hogar o un negocio pequeño. La portadora instala un dispositivo terminal de la red, **NT1**, en el local del cliente y lo conecta a la central ISDN en la oficina de la portadora, a varios kilómetros de distancia, usando el par trenzado con que antes se conectaba el teléfono. La caja NT1 tiene un conector en el que se



(a)



(b)

Figura 2-41. (a) Ejemplo de sistema ISDN para uso del hogar. (b) Ejemplo de sistema ISDN con PBX para uso en compañías grandes.

puede insertar un cable de *bus* pasivo. Hasta ocho teléfonos, terminales, alarmas y otros dispositivos ISDN se pueden conectar al cable, de forma similar a como los dispositivos se conectan a una LAN. Desde el punto de vista del cliente, la frontera de la red es el conector en NT1.

Para empresas grandes, el modelo de la figura 2-41(a) es inadecuado porque es común tener más conversaciones llevándose a cabo en forma simultánea que las que el *bus* puede manejar. Por tanto, se usa el modelo de la figura 2-41(b). En este modelo tenemos un dispositivo, **NT2**, llamado **PBX** (*private branch eXchange,* **central de ramal privado**), conectado a NT1 y proporcionando la interfaz real para teléfonos, terminales y otro equipo. Una PBX de ISDN no es muy diferente en lo conceptual de un conmutador de ISDN, aunque usualmente es más pequeño y no puede manejar tantas conversaciones al mismo tiempo.

El CCITT definió cuatro **puntos de referencia**, llamados **R**, **S**, **T** y **U**, entre los diferentes dispositivos. Éstos están marcados en la figura 2-41. El punto de referencia U es la conexión entre la central ISDN en la oficina de la portadora y el NT1. Por ahora es un par trenzado de cobre de dos hilos, pero en el futuro podría ser reemplazado por fibra óptica. El punto de referencia T es lo que el conector de NT1 proporciona al cliente. El punto de referencia S es la interfaz entre la PBX de ISDN y las terminales ISDN. El punto de referencia R es la conexión entre el adaptador de terminal y las terminales que no son ISDN. En R se podrán usar muchos tipos diferentes de interfaces.

2.5.3. La interfaz ISDN

El conducto de bits ISDN maneja múltiples canales intercalados mediante multiplexión por división en el tiempo. Se han estandarizado varios tipos de canales:

A - canal analógico telefónico de 4 kHz
B - canal digital PCM de 64 kbps para voz o datos
C - canal digital de 8 o 16 kbps
D - canal digital de 16 kbps para señalización fuera de banda
E - canal digital de 64 kbps para señalización ISDN interna
H - canal digital de 384, 1536 o 1920 kbps

No era intención del CCITT permitir una combinación arbitraria de canales en el conducto digital de bits. Hasta ahora se han estandarizado tres combinaciones:

1. **Velocidad básica**: 2B + 1D.
2. **Velocidad primaria**: 23B + 1D (Estados Unidos y Japón) o 30B + 1D (Europa).
3. **Híbrida**: 1A + 1C.

Los canales de velocidad básica y primaria se ilustran en la figura 2-42.

La velocidad básica debe verse como un reemplazo de **POTS** (*plain old telephone service,* **servicio telefónico ordinario antiguo**) para uso de hogares o negocios pequeños. Cada uno de los canales B de 64 kbps puede manejar un canal de voz PCM sencillo con muestras de 8 bits tomadas 8000 veces por segundo (tenga presente que 64 kbps significa aquí 64,000 y no 65,536). La señalización se efectúa por un canal D de 16 kbps independiente, de modo que los 64 kbps

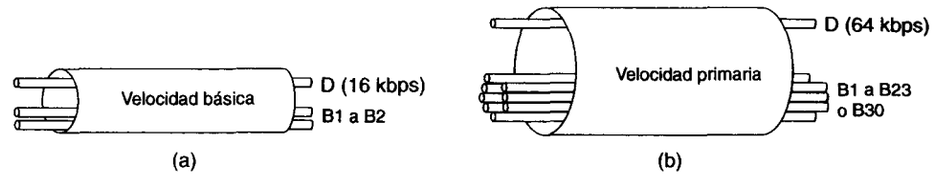


Figura 2-42. (a) Conducto digital de velocidad básica. (b) Conducto digital de velocidad primaria.

completos están disponibles para el usuario (como en el sistema CCITT de 2.048 Mbps y a diferencia del sistema T1 de Estados Unidos y Japón). Como la ISDN se enfoca principalmente a canales de 64 kbps, nos referiremos a ella como **N-ISDN** (*narrowband ISDN,* **ISDN de banda estrecha**) para contrastarlo con la ISDN de banda ancha (ATM), la cual se verá más adelante.

Se pretende que la interfaz de velocidad primaria se use como el punto de referencia T para empresas con una PBX. En Estados Unidos y Japón esta interfaz tiene 23 canales B y un canal D (a 64 kbps) y en Europa tiene 30 canales B y un canal D (a 64 kbps). La elección de 23B + 1D se hizo para permitir que un marco de ISDN se ajuste al sistema T1 de AT&T. La elección 30B + 1D se hizo para permitir que un marco de ISDN se ajuste al sistema CCITT de 2.048 Mbps. La ranura de tiempo de 30 segundos del sistema CCITT se usa para enmarcado y mantenimiento general de la red. Observe que la proporción entre canal D y canal B en la velocidad primaria es mucho menor que en la velocidad básica, pues no se espera que haya muchos datos de telemetría o de paquetes de bajo ancho de banda.

2.5.4. Perspectiva de N-ISDN

N-ISDN fue un intento masivo por sustituir el sistema de teléfonos analógico por uno digital adecuado para tráfico tanto de voz como sin voz. Se suponía que lograr un acuerdo mundial en cuanto al estándar de la interfaz para la velocidad básica conduciría a una gran demanda de los usuarios por equipo ISDN, llevando así a la producción en masa, economías de escala y circuitos integrados VLSI de ISDN de bajo costo. Desafortunadamente, el proceso de estandarización tomó años y la tecnología en esta área avanzó muy rápidamente, de modo que cuando finalmente se tuvo el acuerdo sobre el estándar, éste era obsoleto.

Para uso del hogar, la demanda más grande de nuevos servicios será indudablemente para el vídeo sobre pedido. Desafortunadamente, la velocidad básica ISDN carece del ancho de banda necesario por dos órdenes de magnitud. Para uso de empresas, el panorama es aún más desalentador. Las LAN actuales ofrecen por lo menos 10 Mbps y están siendo reemplazadas por LAN de 100 Mbps. Ofrecer servicio de 64 kbps a los negocios en la década de 1980 era una proposición seria; en la de 1990, resulta una broma.

Curiosamente aún se puede salvar la ISDN, pero con una aplicación totalmente inesperada: el acceso a Internet. Varias compañías venden ahora adaptadores ISDN que combinan los canales 2B + D en un solo canal digital de 144 kbps. Muchos proveedores de servicios de Internet también manejan estos adaptadores. El resultado es que la gente puede acceder a Internet por un

enlace digital de 144 kbps, en lugar de un enlace de módem analógico de 28.8 kbps. Para muchos usuarios de Internet, obtener una mejora de un factor de cinco para descargar páginas completas con gráficos de la *World Wide Web* es un servicio que vale la pena tener. Si bien la B-ISDN a 155 Mbps es todavía mejor, la N-ISDN a 144 kbps está aquí ahora a un precio accesible, y ése puede ser su principal nicho para los próximos años.

2.6. ISDN DE BANDA ANCHA Y ATM

Cuando el CCITT por fin se dio cuenta de que la ISDN de banda angosta no iba a incendiar el mundo, trató de pensar en un nuevo servicio que lo hiciera. El resultado fue la **ISDN de banda ancha** (**B-ISDN**, *broadband ISDN*) básicamente un circuito digital virtual para transferir paquetes de tamaño fijo (células) del origen al destino a 155 Mbps (realmente a 156 Mbps, como mencionamos anteriormente). Puesto que esta velocidad de datos es suficiente para HDTV (no comprimida), probablemente satisfará aun a los más grandes consumidores de ancho de banda, al menos por unos cuantos años.

Mientras que la ISDN de banda angosta fue un tímido primer paso hacia la era digital, la ISDN de banda ancha es un salto temerario hacia lo desconocido. Los beneficios son enormes, como un incremento del ancho de banda respecto a la ISDN de banda angosta por un factor de 2500, pero los retos son igualmente enormes (Armbruster, 1995).

Para comenzar, la ISDN de banda ancha está basada en la tecnología ATM y, como explicamos brevemente en el capítulo 1, ATM es fundamentalmente una tecnología de conmutación de paquetes, no de circuitos (aunque puede emular la conmutación de circuitos bastante bien). En contraste, ambas, la PSTN existente y la ISDN de banda estrecha son tecnologías de conmutación de circuitos. Con el cambio, una cantidad enorme de experiencia de ingeniería en conmutación de circuitos se volverá obsoleta. Pasar de conmutación de circuitos a conmutación de paquetes es un verdadero cambio de paradigma.

Como si eso no fuera suficiente, la ISDN de banda ancha no puede utilizar el alambrado de par trenzado existente excepto a distancias cortas. Esto significa que introducirla requerirá retirar muchos de los lazos locales e instalar ya sea par trenzado de categoría 5 o fibra (Stephens y Banwell, 1995). Además, los conmutadores por división en el espacio y división en el tiempo no sirven para conmutación de paquetes; todos tendrán que ser reemplazados por conmutadores nuevos basados en principios diferentes y que funcionen a velocidades mucho más altas. Las únicas partes que pueden aprovecharse son los troncales de fibra de área extensa.

En pocas palabras, tirar a la basura 100 años de conocimientos acumulados y además hacer una inversión tanto dentro de las plantas como fuera de ellas que asciende a casi un billón de dólares no es exactamente un paso pequeño que se tome a la ligera. No obstante, para las compañías telefónicas está claro que si ellas no lo hacen, las compañías de televisión por cable, pensando en el vídeo sobre pedido, probablemente lo harán. Si bien es probable que tanto la PSTN existente como la ISDN de banda angosta seguirán en funciones por una década o quizá más, el futuro a largo plazo seguramente pertenece a ATM, de modo que lo estudiaremos con gran detalle en este libro, empezando con la capa física en este capítulo.

2.6.1. Circuitos virtuales y conmutación de circuitos

El servicio básico de ISDN de banda ancha es un término medio entre conmutación de circuitos pura y conmutación de paquetes pura. El servicio real ofrecido está orientado a la conexión, pero internamente se implementa con conmutación de paquetes, no con conmutación de circuitos. Se ofrecen dos tipos de conexiones: circuitos virtuales permanentes y circuitos virtuales conmutados. Los **circuitos virtuales permanentes** son solicitados por el cliente en forma manual (por ejemplo, enviando un fax a la portadora) y normalmente permanecen en su lugar durante meses o años. Los **circuitos virtuales conmutados** son como las llamadas telefónicas: se establecen en forma dinámica cuando se necesitan y pueden liberarse inmediatamente después.

En una red de conmutación de circuitos, hacer una conexión realmente significa establecer una trayectoria física del origen al destino a través de la red, sobre todo cuando se usan conmutadores por división en el espacio. (Con conmutadores por división en el tiempo, el concepto de una "trayectoria física" ya no está muy claro.) En una red de circuitos virtuales, como ATM, cuando se establece un circuito, lo que realmente sucede es que se escoge una ruta desde el origen al destino y todos los conmutadores (esto es, los enrutadores) a lo largo del camino crean entradas de tabla para poder enrutar cualquier paquete por ese circuito virtual. Los conmutadores también tienen la oportunidad para reservar recursos para el nuevo circuito. La figura 2-43 muestra un circuito virtual desde el *host* H_1 al *host* H_5 a través de los conmutadores (enrutadores) *A*, *E*, *C* y *D*.

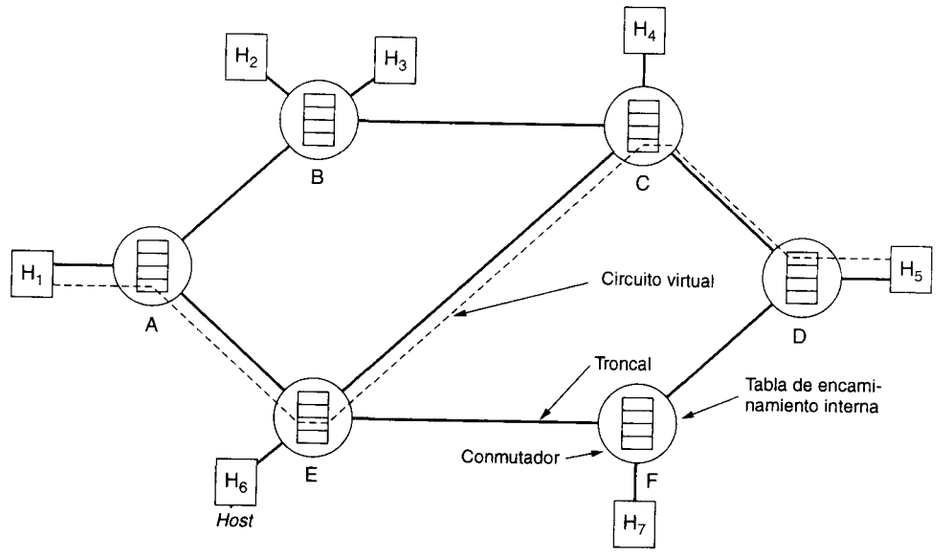


Figura 2-43. La línea punteada muestra un circuito virtual que está definido sencillamente por entradas de tabla dentro de los conmutadores.

Cuando un paquete llega, el conmutador inspecciona el encabezado del paquete para averiguar a cuál circuito virtual pertenece. A continuación, busca ese circuito virtual en sus tablas

para determinar a cuál línea de comunicación debe enviar el paquete. Examinaremos este proceso con más detalle en el capítulo 5.

Ahora debe quedar claro el significado del circuito virtual permanente entre H_1 y H_5 en la figura 2-43: es un convenio entre el cliente y el proveedor de que los conmutadores siempre mantendrán entradas en las tablas para un destino en particular, aunque no haya habido tráfico durante meses. Es obvio que tal acuerdo cuesta recursos (ciertamente espacio de tabla dentro de los conmutadores y posiblemente también reservados ancho de banda y *buffer*), así que siempre hay un cargo mensual por el circuito virtual permanente. La ventaja sobre un circuito virtual conmutado es que no hay tiempo de establecimiento. Los paquetes se pueden transferir en forma instantánea. Para algunas aplicaciones, como las verificaciones de tarjeta de crédito, ahorrar unos pocos segundos en cada transacción fácilmente puede compensar el costo.

En contraste, una línea rentada desde H_1 hasta H_5 en una red de conmutación de circuitos con la topología de la figura 2-43 y conmutadores por división en el espacio mantendrían cerrados en realidad los puntos de cruce durante meses y reservarían de hecho ancho de banda en los troncales de forma permanente, ya sea como bandas de FDM o como ranuras de tiempo (una "línea" rentada puede ser de múltiples saltos si no está disponible una línea directa). Tal arreglo obviamente desperdicia más recursos cuando está ocioso que un circuito virtual.

2.6.2. Transmisión en redes ATM

Como hemos apuntado antes, ATM significa modo de transferencia *asíncrono*. Este modo puede contrastarse con la portadora sincrónica T1 ilustrada en la figura 2-44(a). Un marco T1 se genera precisamente cada 125 μseg. Un reloj maestro gobierna esta velocidad. La ranura k de cada marco contiene 1 byte de datos de la misma fuente. T1 es sincrónico.

En contraste, ATM no exige que las células se alternen de manera rígida entre las diversas fuentes. La figura 2-44(b) muestra células en una línea proveniente de varias fuentes sin un patrón específico. Las células llegan aleatoriamente de varias fuentes.

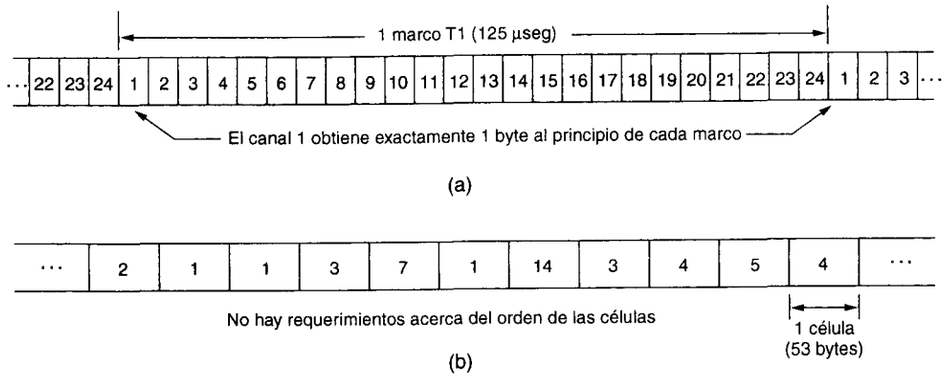


Figura 2-44. (a) Modo de transmisión sincrónico. (b) Modo de transmisión asíncrono.

Además, ni siquiera se exige que el flujo de células que salen de una computadora sea continuo; puede haber huecos entre las células de datos. Tales huecos se llenan con células desocupadas especiales.

ATM no estandariza el formato para transmitir células. En lugar de ello especifica que se permite enviar células individuales, pero también especifica que las células pueden envolverse en una portadora como T1, T3, SONET o FDDI (una LAN de fibra óptica). Para estos ejemplos, existen normas que especifican cómo empacar las células dentro de los marcos que estos sistemas proporcionan.

En el estándar ATM original, la velocidad primaria fue de 155.52 Mbps, con una velocidad adicional cuatro veces mayor (622.08 Mbps). Estas velocidades se escogieron por ser compatibles con SONET, el estándar de enmarcado que se usa en los enlaces de fibra óptica de todo el sistema telefónico. También están previstos ATM sobre T3 (44.736 Mbps) y FDDI (100 Mbps).

El medio de transmisión para ATM es normalmente la fibra óptica, pero en tramos menores de 100 metros, también son aceptables el cable coaxial o la categoría 5 de par trenzado. Los tramos de fibra pueden ser de muchos kilómetros. Cada enlace se tiende entre una computadora y un conmutador ATM o entre dos conmutadores ATM. En otras palabras, todos los enlaces ATM son punto a punto (a diferencia de las LAN, las cuales pueden tener muchos emisores y receptores en el mismo cable). La multidifusión se logra haciendo que una célula entre a un conmutador por una línea y salga por múltiples líneas. Cada enlace punto a punto es unidireccional. Para funcionamiento dúplex se necesitan dos enlaces paralelos, uno para el tráfico en cada sentido.

La subcapa ATM **dependiente del medio físico** se ocupa de meter bits en el cable y sacarlos de él. Se necesita *hardware* diferente para los distintos cables y fibras dependiendo de la rapidez y codificación de la línea. El propósito de la subcapa de convergencia de transmisión es proveer una interfaz uniforme con la capa ATM en ambas direcciones. Hacia el exterior, la capa ATM proporciona una secuencia de células y la subcapa PDM las codifica según sea necesario y las saca como una corriente de bits.

Hacia el interior, la subcapa PDM toma los bits entrantes de la red y entrega un flujo de bits a la subcapa de TC. Los límites de las células no están marcados en forma alguna; corresponde a la subcapa TC descifrar de alguna forma dónde finaliza una célula y principia la siguiente. Este trabajo no sólo es difícil, es teóricamente imposible, así que la subcapa TC enfrenta una tarea descomunal. Debido a que la subcapa TC realiza el enmarcado de células, que es una función de enlace de datos, la veremos en el capítulo 3. Si desea información adicional acerca de la capa física ATM, consulte (Rao y Hatamian, 1995).

2.6.3. Conmutadores ATM

Se han descrito muchos diseños de conmutadores de células ATM en la literatura. Algunos de éstos se han instalado y probado. En esta sección presentaremos una breve introducción a los principios de diseño de conmutadores de células ATM e ilustraremos éstos con algunos ejemplos. Para más información, véase (De Prycker, 1993; Garcia-Haro y Jajszczyk, 1994; Handel *et al.*, 1994, y Partridge, 1994). Para un conmutador ATM optimizado para ejecutar IP sobre ATM, véase (Parulkar *et al.*, 1995).

El modelo general para un conmutador de células ATM se muestra en la figura 2-45. Hay cierto número de líneas de entrada y cierto número de líneas de salida, casi siempre la misma cantidad (porque las líneas son bidireccionales). Los conmutadores ATM generalmente son sincrónicos en el sentido de que, durante un ciclo, se toma una célula de cada línea de entrada (si está una presente), se pasa a la **estructura de conmutación** interna y finalmente se transmite por la línea de salida apropiada.

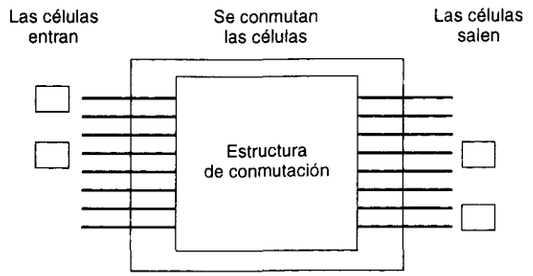


Figura 2-45. Un conmutador ATM genérico.

Los conmutadores se pueden empalmar, esto es, se pueden requerir varios ciclos para que una célula de entrada aparezca en su línea de salida. Las células realmente llegan por las líneas de entrada en forma asíncrona, por lo que hay un reloj maestro que marca el comienzo de cada ciclo. Cualquier célula que llega completa cuando el reloj marca un ciclo es elegible para conmutarse durante ese ciclo. Una célula que no ha llegado completa tiene que esperar hasta el siguiente ciclo.

Las células llegan a la velocidad de ATM, normalmente cerca de 150 Mbps. Esto corresponde a un poco más de 360,000 células/seg, lo cual significa que el tiempo de ciclo del conmutador tiene que ser de cerca de 2.7 μseg. Un conmutador comercial podría tener desde 16 hasta 1024 líneas de entrada, lo cual significa que debe estar preparado para aceptar y comenzar a conmutar un lote de 16 a 1024 células cada 2.7 μseg. A 622 Mbps, se inyecta un lote de células nuevo dentro a la estructura de conmutación cada 700 ns aproximadamente. El hecho de que las células sean de longitud fija y cortas (53 bytes) hace posible construir tales conmutadores. Con paquetes más grandes de longitud variable la conmutación de alta velocidad sería más compleja, y es por esto que ATM maneja células cortas de longitud fija.

Todos los conmutadores de ATM tienen dos metas comunes:

1. Conmutar todas las células con una velocidad de desecho lo más baja posible.
2. Nunca reordenar las células en un circuito virtual.

La meta 1 dice que se permite suprimir células en emergencias, pero que la tasa de pérdida deberá ser lo más pequeña posible. Probablemente sea aceptable una tasa de pérdida de una célula en 10^{12}. En un conmutador grande, esta tasa de pérdida equivale a alrededor de una a dos células por hora. La meta 2 dice que las células que llegan a un circuito virtual en cierto

orden deben salir también en ese orden, sin excepciones. Esta restricción hace que el diseño de conmutadores sea mucho más difícil, pero lo requiere el estándar ATM.

Un problema que se presenta en todos los conmutadores ATM es qué hacer si las células que llegan a dos o más líneas de entrada quieren ir al mismo puerto de salida en el mismo ciclo. Resolver este problema es uno de los aspectos clave del diseño de todos los conmutadores ATM. Una solución falsa consiste en elegir una célula para entregarla y descartar el resto. Puesto que este algoritmo viola la meta 1, no lo podemos usar.

Nuestro siguiente intento es crear una cola para cada línea de entrada. Si dos o más células están en conflicto, se escoge una de ellas para entregarla y el resto se guarda para el siguiente ciclo. La selección puede hacerse en forma aleatoria o cíclica, pero no debe exhibir una predisposición sistemática (por ejemplo, en favor de la línea numerada más baja) para evitar dar mejor servicio a las líneas con números bajos que a las líneas con números altos. La figura 2-46(a) describe la situación al inicio del ciclo 1, en el cual han llegado células por las cuatro líneas de entrada, destinadas para las líneas de salida 2, 0, 2 y 1, respectivamente. Debido a que hay un conflicto para la línea 2, únicamente se puede escoger una de las células. Suponga que se elige la que está en la línea de entrada 0. Al inicio del ciclo 2, mostrado en la figura 2-46(b), han salido tres células pero la célula de la línea 2 ha sido retenida y han llegado dos células más. Es hasta el inicio del ciclo 4 [figura 2-46(d)] que todas las células han dejado el conmutador.

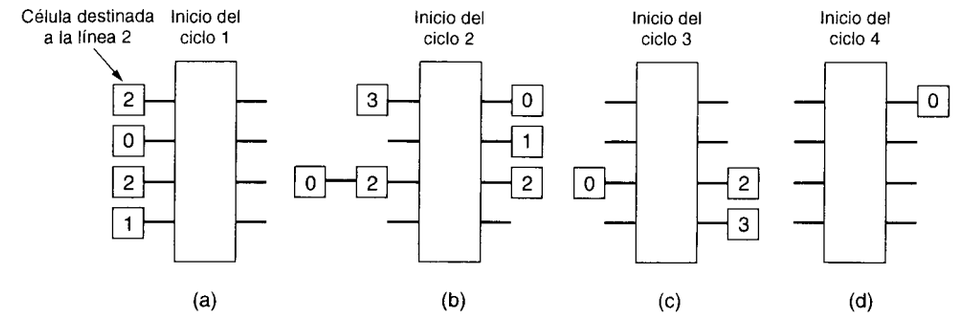


Figura 2-46. Encolamiento de entrada en un conmutador ATM.

El problema con las colas de entrada es que cuando se tiene que retener una célula se bloquea el avance de cualquier célula que venga detrás de ella, aun si ésta se pudiera conmutar a otro lugar. Este efecto se denomina **bloqueo de cabecera de línea** y es algo más complicado que lo que se muestra aquí, pues en un conmutador con 1024 líneas de entrada puede ser que los conflictos no se noten hasta que las células ya han atravesado el conmutador y están peleando por la línea de salida. Mantener una célula en su cola de entrada hasta que llegue una señal de regreso que indique que ya atravesó el conmutador requiere lógica extra, una trayectoria de señalización en el sentido opuesto, y más retardo. Lo que a veces se hace es poner las células que pierden en un *bus* recirculante que las envía de regreso a la entrada, pero el conmutador tiene que ser cuidadoso de dónde las pone, para evitar entregar las células del mismo circuito virtual en desorden.

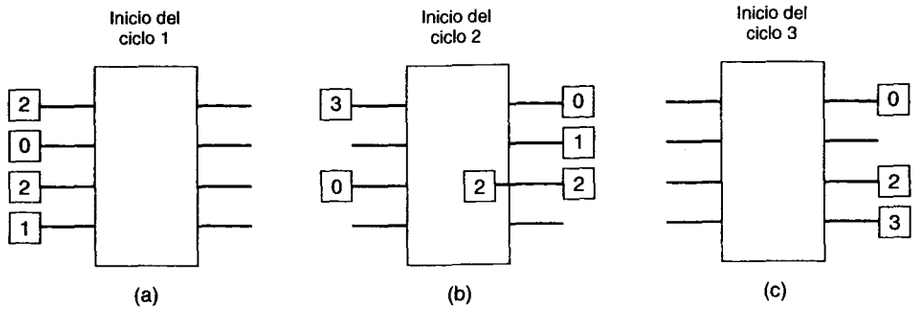


Figura 2-47. Encolamiento de salida en un conmutador ATM.

Un diseño alternativo que no sufre bloqueo de cabecera de línea hace el encolamiento en el extremo de salida, como se muestra en la figura 2-47. Aquí tenemos el mismo patrón de llegada de células, pero ahora cuando dos células quieren ir a la misma línea de salida en el mismo ciclo, ambas pasan a través del conmutador. Una de ellas se pone en la línea de salida, y la otra se encola en la línea de salida, como en la figura 2-47(b). Aquí se requieren únicamente tres ciclos, en lugar de cuatro, para conmutar todos los paquetes. Karol *et al.* (1987) ha demostrado que en general el encolamiento de salida es más eficiente que el de entrada.

El conmutador de eliminación

Veamos ahora con mayor detenimiento el diseño de un conmutador de ATM que utiliza encolamiento de salida: se llama **conmutador de eliminación** (Yeh *et al.*, 1987) y se ilustra en la figura 2-48 para ocho líneas de entrada y ocho de salida. Cada línea de entrada se conecta a un *bus* por el cual las células que están llegando se difunden en el mismo ciclo en el que llegan. Al tener únicamente un manejador de *bus* por cada *bus* el diseño y la temporización se simplifican considerablemente.

Para cada célula que llega, el *hardware* inspecciona el encabezado de la célula para leer su información de circuito virtual, busca la información correspondiente en las tablas de enrutamiento (véase la figura 2-43) y habilita el punto de cruce correcto. A continuación, la célula viaja por su *bus* hasta el punto de cruce habilitado, donde cambia de dirección para dirigirse hacia su línea de salida. Es posible que varias células, o incluso todas ellas, vayan a la misma línea de salida. También es posible que una célula se envíe por multidifusión a varias líneas de salida con sólo habilitar varios puntos de cruce en su *bus* de difusión.

La manera más sencilla de manejar las colisiones sería simplemente almacenar en un *buffer* todas las células en el extremo de salida. Sin embargo, para un conmutador con 1024 líneas, en el peor caso podrían ser necesarios 1024 *buffers* de salida. En la práctica es muy improbable que ocurra esta situación, así que una optimización razonable sería proporcionar muchos menos *buffers* de salida, digamos *n*.

En el poco probable caso de que en un ciclo lleguen más células que las que pueden manejarse, el concentrador de cada línea seleccionaría *n* células para encolarlas, descartando el

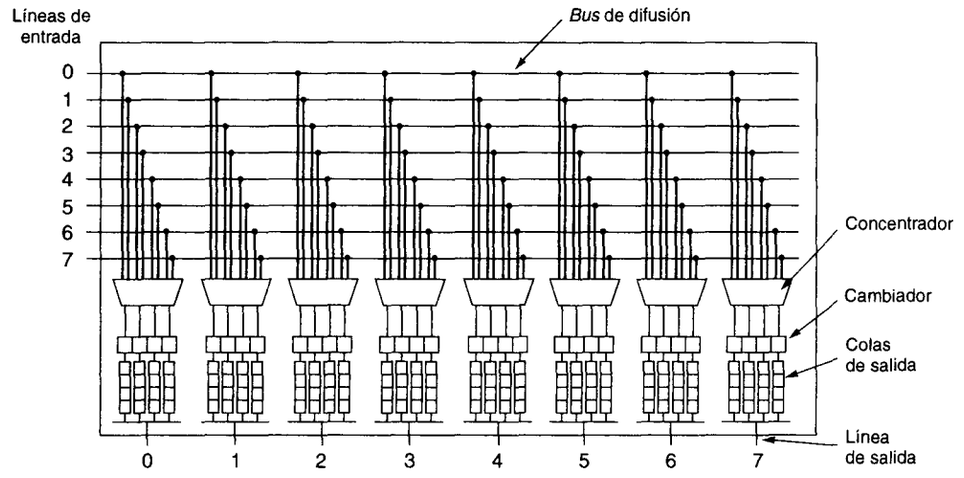


Figura 2-48. Diagrama simplificado del conmutador de eliminación.

resto. El concentrador es un circuito ingenioso que hace esta selección de forma justa, utilizando un esquema de eliminación similar a los cuartos de final, semifinales y finales de muchos torneos deportivos.

Conceptualmente, todas las células elegidas entran en una sola cola de salida (a menos que esté llena, en cuyo caso se descartan células). Sin embargo, en la práctica no es factible meter todas las células en una sola cola en el tiempo asignado, así la cola de salida se simula con múltiples colas. Las células seleccionadas pasan a un cambiador, el cual las distribuye uniformemente entre *n* colas de salida usando un testigo para recordar cuál cola sigue, a fin de mantener la secuencia dentro de cada circuito virtual. Variando *n*, los diseñadores pueden balancear el costo del conmutador y la tasa de pérdida de células esperada.

El conmutador Batcher-banyan

El problema con el conmutador de eliminación es que básicamente es un conmutador de matriz, así que el número de puntos de cruce es el cuadrado del número de líneas. Así como este factor resultó ser un problema en la conmutación de circuitos, también es un problema en la conmutación de paquetes. La solución para la conmutación de circuitos fue la conmutación por división en el espacio, que reduce considerablemente el número de puntos de cruce al costo de requerir un conmutador de múltiples etapas. Existe una solución similar para la conmutación de paquetes.

Esta solución es el **conmutador Batcher-banyan**. Al igual que los conmutadores de eliminación, los Batcher-banyan son asíncronos y aceptan un conjunto de células en cada ciclo (cero o una por cada línea de entrada). Aun un Batcher-banyan sencillo es más complicado que los conmutadores por división en el espacio de la figura 2-39, así que los presentaremos paso por paso. En la figura 2-49(a) tenemos un conmutador banyan de 8×8 de tres etapas, llamado así

porque su alambrado supuestamente se parece a las raíces de un baniano (un tipo de árbol). En todos los conmutadores banyan, sólo existe una trayectoria de cada línea de entrada a cada línea de salida. El enrutamiento se hace buscando la línea de salida para cada célula (con base en la información del circuito virtual y tablas). Este número binario de 3 bits se pone entonces al principio de la célula, pues se usará para el enrutamiento a través del conmutador.

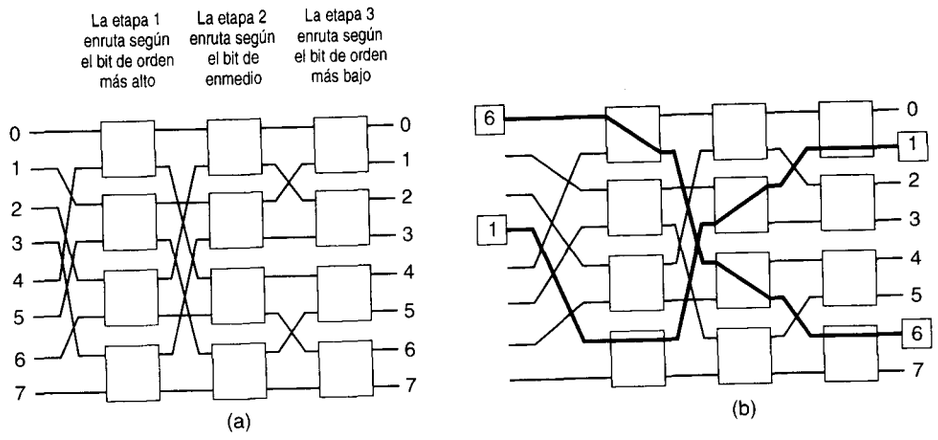


Figura 2-49. (a) Conmutador banyan con ocho líneas de entrada y ocho de salida. (b) Las rutas que dos células siguen a través del conmutador banyan.

Cada uno de los 12 elementos de conmutación del conmutador banyan tiene dos entradas y dos salidas. Cuando una célula llega a un elemento de conmutación, se inspecciona 1 bit del número de línea de salida y, con base en él, la célula se encamina ya sea al puerto 0 (el puerto superior) o el 1 (el puerto inferior). En caso de una colisión, una célula se encamina y la otra se descarta.

El conmutador banyan analiza el número de la línea de salida de izquierda a derecha, así que la etapa 1 examina el bit que está más a la izquierda (esto es, el de orden más alto), la etapa 2 examina el bit de enmedio, y la etapa 3 examina el bit de la derecha (o sea, el de orden más bajo). En la figura 2-49(b) tenemos presentes dos células: una en el encabezado de la línea de entrada 0 dirigida a la línea de salida 6, y una en el encabezado de la línea 3 dirigida a la línea de salida 1. Para la primera célula, la dirección de salida binaria es 110, así que pasa a través de las tres etapas usando los puertos inferior, inferior y superior respectivamente, como se muestra. De manera similar, la otra célula, etiquetada 001 en binario, usa los puertos superior, superior e inferior respectivamente.

Desafortunadamente, cuando en un conmutador banyan dos células que llegan quieren salir de un elemento de conmutación por el mismo puerto y al mismo tiempo, ocurre una colisión. Una serie de tales colisiones se ilustra en la figura 2-50(a). En la etapa 1, las colisiones implican las células dirigidas a los siguientes pares de líneas de salida: (5, 7), (0, 3), (6, 4) y (2, 1). Supongamos que estas colisiones se resuelven en favor de 5, 0, 4 y 1. En la segunda etapa tenemos

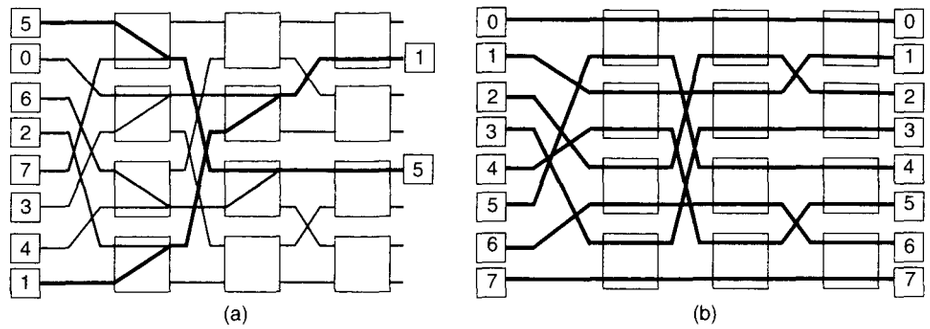


Figura 2-50. (a) Choque de células en un conmutador banyan. (b) Encaminamiento libre de choques a través de un conmutador banyan.

colisiones entre (0, 1) y (5, 4). Aquí dejamos ganar a 1 y 5, que entonces se enrutan a las líneas de salida correctas.

Veamos ahora la figura 2-50(b). Las ocho células atraviesan sin colisiones. La conclusión es: dependiendo de la entrada, el conmutador banyan puede hacer un buen trabajo o un mal trabajo de enrutamiento.

La idea en que se basa el conmutador Batcher-banyan es poner un conmutador antes del conmutador banyan para permutar las células a una configuración que el conmutador banyan pueda manejar sin pérdidas. Por ejemplo, si las células que llegan se ordenan por destino y se presentan en las líneas de entrada 0, 2, 4, 6, 1, 3, 5 y 7, en ese orden hasta donde sea necesario (dependiendo de cuántas células haya), entonces el conmutador banyan no perderá células.

Para ordenar las células de entrada podemos utilizar un conmutador Batcher, inventado por K.E. Batcher (1968). Al igual que los conmutadores banyan y de eliminación, es sincrónico y trabaja con ciclos discretos. Un conmutador Banyan se compone de 2×2 elementos de conmutación, pero éstos funcionan de forma diferente a los del conmutador banyan. Cuando un elemento de conmutación recibe dos células, compara sus direcciones de salida numéricamente (no sólo 1 bit) y enruta la más alta por el puerto al que apunta la flecha, y la más baja por el otro. Si hay únicamente una célula, va al puerto opuesto al que apunta la flecha.

En el lado izquierdo de la figura 2-51 se representa un conmutador Batcher para ocho líneas. La etapa 1 ordena las células que llegan en pares. Las dos etapas siguientes realizan una combinación tetradireccional. Las tres etapas finales efectúan una combinación octodireccional. En general, para n líneas, la complejidad de un conmutador Batcher crece según $n \log^2 n$. Cuando hay k células presentes en las líneas de entrada, el conmutador Batcher las ordena numéricamente en las primeras k líneas de salida.

Después de salir el conmutador Batcher, las células se reacomodan e inyectan a un conmutador banyan. El resultado final es que cada célula aparece en la línea de salida correcta en la salida del conmutador banyan.

En la figura 2-52 se presenta un ejemplo de cómo trabaja la estructura de conmutación Batcher-banyan combinada. Aquí hay células presentes en las líneas de entrada 2, 3, 4 y 5,

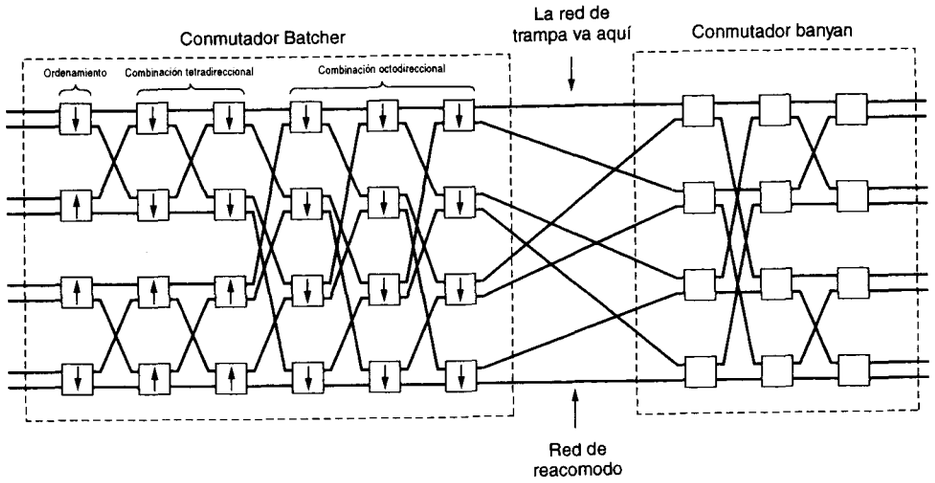


Figura 2-51. Estructura de conmutación de un conmutador Batcher-banyan.

dirigidas a las líneas de salida 6, 5, 1 y 4, respectivamente. Inicialmente, las células para 5 y 6 entran en el mismo elemento de conmutación. La célula 6 tiene la dirección más alta, así que sale en la dirección de la flecha; la célula 5 toma el otro camino. Aquí no hay intercambio. Con las células 1 y 4 sí ocurre un intercambio, pues la 4 entra por la parte inferior pero sale por la superior. Las líneas gruesas muestran todas las trayectorias hasta las salidas.

Observe que a la salida del conmutador Batcher las cuatro células están agrupadas en la parte superior, en orden. Las células pasan entonces a través de una red de reacomodo y se inyectan al conmutador banyan, el cual es capaz de procesarlas sin colisiones.

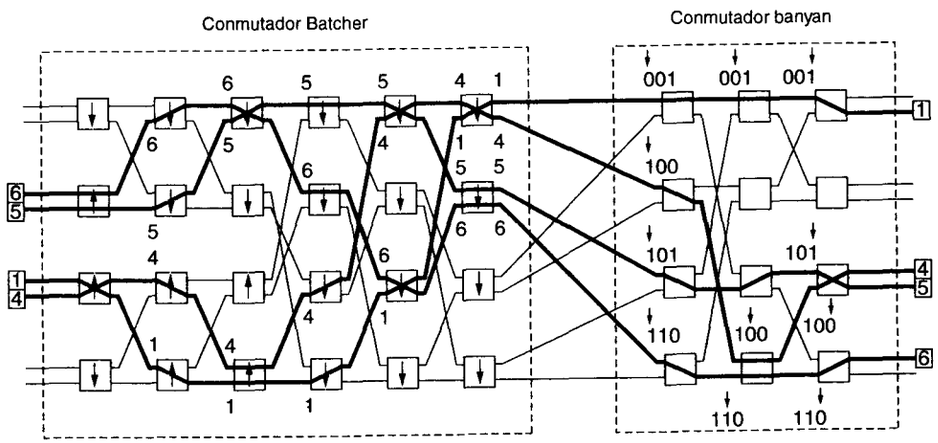


Figura 2-52. Ejemplo con cuatro celdas empleando el conmutador Batcher-banyan.

En principio, el conmutador Batcher-banyan es un excelente conmutador para ATM, pero hay dos complicaciones que hemos puesto a un lado: las colisiones de líneas de salida y la multidifusión. Si dos o más células van a la misma línea de salida, el conmutador Batcher-banyan no puede manejarlas, y tenemos que recurrir a algún tipo de almacenamiento temporal en memoria. Una manera de resolver este problema es insertar una red de trampa entre el conmutador Batcher y el banyan. Esta red filtra los duplicados y los recircula para ciclos subsecuentes al mismo tiempo que mantiene el orden de las células en un circuito virtual. (A estas alturas deberá quedar claro que el requerimiento de ordenar las células es más problemático de lo que parecía a primera vista.) Los conmutadores comerciales también tienen que manejar la multidifusión.

El primer conmutador Batcher-banyan ATM fue diseñado por Huang y Knauer (1984) y se llamó Starlite. A continuación vinieron Moonshine (Hui, 1987) y Sunshine (Giacopelli *et al.*, 1991). Es preciso admitir que estas personas tienen sentido del humor. Starlite, Moonshine y Sunshine difieren principalmente en el circuito trampa y en cómo manejan la multidifusión.

2.7. RADIO CELULAR

El sistema telefónico tradicional (incluso cuando la ISDN de banda ancha entre plenamente en funciones) no será capaz de satisfacer un grupo creciente de usuarios: la gente en movimiento. Por ello, hay una competencia creciente por parte de un sistema que usa ondas de radio en lugar de hilos telefónicos y fibras para la comunicación. Este sistema desempeñará un papel cada vez más importante en la conexión a redes de computadoras portátiles, teléfonos de bolsillo, y asistentes personales digitales en los años por venir. En la siguiente sección examinaremos los avisos por satélite, los teléfonos inalámbricos, los teléfonos celulares y tecnologías similares. Estos sistemas ya se están fusionando, produciendo computadoras portátiles capaces de enviar y recibir llamadas telefónicas, faxes y correo electrónico, así como de buscar información en bases de datos remotas y hacer esto en cualquier lugar de la Tierra.

Tales dispositivos ya están creando un enorme mercado. Muchas compañías en las industrias de computadoras, teléfonos, satélites y otras quieren una parte de la acción. El resultado es un mercado caótico, con numerosos productos y servicios que se traslapan y son incompatibles, que cambian con rapidez, y que normalmente son diferentes en cada país. Sin embargo, las descripciones que daremos a continuación deberán proporcionar al menos un conocimiento básico de las tecnologías fundamentales. Para más información, véase (Bates, 1994; Goodman, 1991; Macario, 1993; Padgett *et al.*, 1995, y Seybold, 1994).

2.7.1. Sistemas de aviso

Los primeros sistemas de aviso (voceo) usaban altavoces dentro de un solo edificio. En un hospital es común oír anuncios en el sistema público de voceo como: "Doctora Irma Aguilar por favor comuníquese a la extensión 4321". Hoy día, la gente que quiere recibir avisos usa pequeños

beepers o radiolocalizadores, usualmente con diminutas pantallas que exhiben los mensajes cortos que llegan.

Una persona que quiera localizar a un usuario de un *beeper* puede llamar a la compañía del *beeper* e introducir un código de seguridad, el número del *beeper* y el número al cual el usuario del *beeper* ha de llamar (o algún otro mensaje corto). La computadora que recibe la petición la transmite entonces por líneas terrestres a una antena en la cima de una colina, desde donde la llamada de aviso se difunde directamente (aviso local) o se envía a un satélite (avisos de larga distancia), el cual a continuación lo vuelve a difundir. Cuando el *beeper* detecta su número único en el flujo de radio que entra, suena y exhibe el número al que se debe llamar. También es posible enviar un aviso a un grupo de gente de forma simultánea con una sola llamada telefónica.

Los sistemas de *beeper* más avanzados se pueden conectar directamente a una computadora y recibir no sólo un número telefónico, sino mensajes largos. La computadora puede entonces procesar los datos conforme llegan. Por ejemplo, una compañía podría mantener actualizadas las listas de precios en las computadoras portátiles de sus vendedores utilizando esta forma de localización.

Muchos sistemas de aviso actuales tienen la propiedad de ser sistemas de una vía, desde una sola computadora hasta un gran número de receptores. No hay problema respecto a quién será el siguiente en hablar, y no hay contención entre muchos usuarios compitiendo por un número pequeño de canales ya que sólo hay un emisor en todo el sistema.

Los sistemas de aviso requieren un ancho de banda pequeño puesto que cada mensaje consiste en una ráfaga de quizá 30 bytes. A esta velocidad de datos, un canal de satélite de 1 Mbps puede manejar más de 240,000 avisos por minuto. Los sistemas de aviso antiguos operan a diversas frecuencias en la banda de 150-174 MHz. Muchos de los sistemas modernos operan en la banda de 930-932 MHz. La figura 2-53(a) muestra la naturaleza unidireccional de un sistema de aviso, con todas las comunicaciones enviadas a una misma frecuencia. Veremos más tarde cómo esta modalidad contrasta con los teléfonos móviles, los cuales son de dos vías y usan dos frecuencias por llamada, empleándose diferentes pares de frecuencias para diferentes llamadas, como se muestra en la figura 2-53(b). Estas diferencias hacen que el sistema de avisos sea más simple y menos costoso de operar.

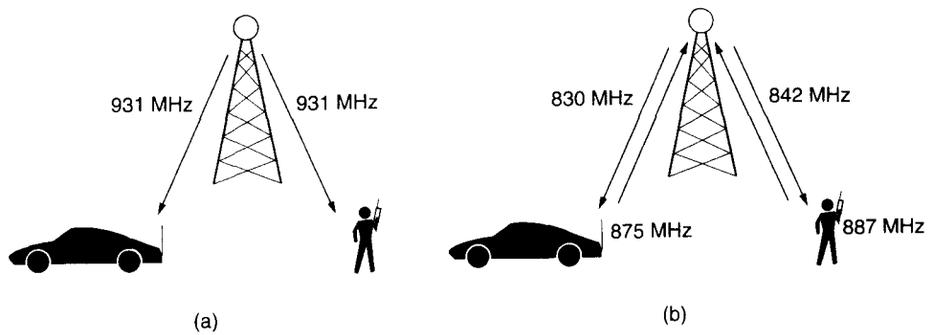


Figura 2-53. Los sistemas de avisos van en un sentido. (b) Los teléfonos móviles son bidireccionales.

2.7.2. Teléfonos inalámbricos

Los teléfonos inalámbricos empezaron como una forma de que la gente pudiera caminar por su casa mientras hablaba por teléfono. Un teléfono inalámbrico consta de dos partes: una estación base y un teléfono, las cuales siempre se venden juntas. La estación base tiene un enchufe telefónico estándar en la parte trasera para que se pueda conectar al sistema telefónico (por medio de un cable). El teléfono se comunica con la estación base por medio de una radio de baja potencia. Normalmente, el alcance es de 100 a 300 metros.

Como en un principio la intención era que los teléfonos inalámbricos se comunicaran solamente con sus propias estaciones base, no había necesidad de estandarización. Algunos de los modelos más baratos usaban una frecuencia fija, seleccionada de fábrica. Si, por casualidad, sucedía que su teléfono inalámbrico usaba la misma frecuencia que el de su vecino, cada uno podía oír en su teléfono las llamadas del otro. Los modelos más caros evitaban este problema permitiendo al usuario seleccionar la frecuencia de la transmisión.

La primera generación de teléfonos inalámbricos, conocida como CT-1 en Estados Unidos y CEPT-1 en Europa, era completamente analógica. Estos teléfonos podían, y a veces lo hacían, ocasionar interferencia con radios y televisores. La recepción pobre y la falta de seguridad obligó a la industria a elaborar un estándar digital, el CT-2, que se originó en Inglaterra. Los primeros dispositivos CT-2 podían hacer llamadas, pero no recibirlas. Sin embargo, tan pronto como se vendió el primero, el fabricante comenzó a recibir quejas y el sistema se rediseñó rápidamente. Al igual que la versión CT-1, cada teléfono tenía que estar a unos pocos cientos de metros de su propia estación base, haciéndolos útiles en las casas y oficinas, pero inútiles en coches o en las calles de la ciudad.

En 1992 se introdujo una tercera generación, la CT-3 o DECT, la cual apoyaba la acción de *roaming* sobre las estaciones base. Esta tecnología está empezando a acercarse a los teléfonos celulares, que describiremos a continuación.

2.7.3. Teléfonos celulares analógicos

Los radioteléfonos móviles se usaron esporádicamente para comunicación marítima y militar durante las primeras décadas del siglo xx. En 1946, el primer sistema de teléfonos instalados en autos se construyó en St. Louis, Estados Unidos. Este sistema utilizaba un solo transmisor grande colocado en la parte superior de un edificio alto y tenía un solo canal que servía para enviar y recibir. Para hablar, el usuario tenía que oprimir un botón que habilitaba el transmisor e inhabilitaba el receptor. Tales sistemas, conocidos como **sistemas de oprimir para hablar**, se instalaron en algunas ciudades desde finales de la década de 1950. El radio de banda civil, los taxis y las patrullas policiacas en programas de televisión a veces usan esta tecnología.

En la década de 1960 se instaló el **IMTS** (*improved mobile telephone system,* **sistema de teléfonos móviles mejorado**). También utilizaba un transmisor de alta potencia (200 watts), en la parte superior de una colina, pero tenía dos frecuencias, una para enviar y otra para recibir, por lo que el botón de oprimir para hablar ya no era necesario. Puesto que toda la comunicación desde los teléfonos móviles entraba por un canal diferente del que recibían los teléfonos emiso-

res, los usuarios móviles no se podían escuchar unos a otros (a diferencia del sistema de oprimir para hablar empleado en los taxis).

IMTS manejaba 23 canales dispersos desde 150 MHz hasta 450 MHz. Debido al número tan pequeño de canales, los usuarios a veces tenían que esperar largo tiempo antes de obtener el tono de marcar. También, debido a la gran potencia del transmisor en la cima de la colina, los sistemas adyacentes tenían que estar alejados varios cientos de kilómetros para evitar la interferencia. Considerando todo, el sistema no era práctico debido a su capacidad limitada.

Sistema avanzado de telefonía móvil

Todo cambió con **AMPS** (*advanced mobile phone system,* **sistema avanzado de telefonía móvil**), inventado por los Laboratorios Bell e instalado por primera vez en Estados Unidos en 1982. El sistema también se usa en Inglaterra, donde se llama TACS, y en Japón, donde se llama MCS-L1. Con AMPS, una región geográfica se divide en **celdas**, normalmente de 10 a 20 km de diámetro, cada una de las cuales utiliza un conjunto de frecuencias. La idea clave que confiere a AMPS más capacidad que todos los sistemas anteriores es utilizar celdas relativamente pequeñas, y reutilizar las frecuencias de transmisión en celdas cercanas (pero no adyacentes). Mientras un sistema IMTS de 100 km de alcance puede tener una llamada en cada frecuencia, un sistema AMPS podría tener 100 celdas de 10 km en la misma área con 5 a 10 llamadas en cada frecuencia en celdas muy separadas. Además, al ser las celdas más pequeñas se necesita menor potencia, lo cual conduce a dispositivos más pequeños y baratos. Los teléfonos manuales tienen una salida de 0.6 watts; los transmisores en los autos normalmente son de 3 watts, el máximo permitido por la FCC.

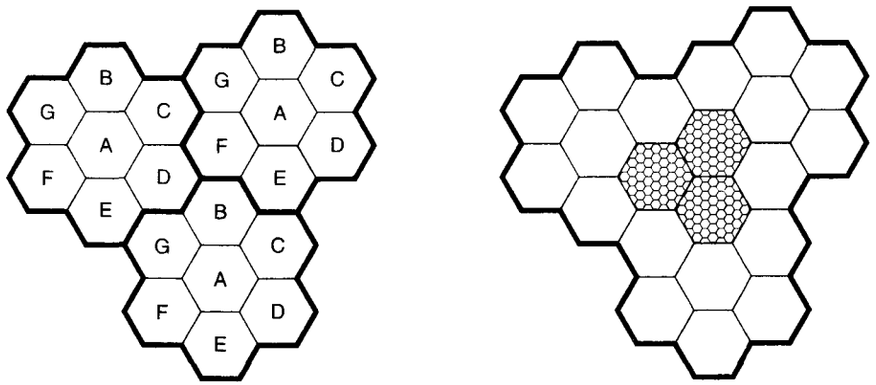


Figura 2-54. (a) Las frecuencias no se reutilizan en celdas adyacentes. (b) Para añadir mas usuarios, se pueden usar celdas más pequeñas.

La idea de reutilizar frecuencias se ilustra en la figura 2-54(a). Las celdas normalmente son casi circulares, pero es más fácil modelarlas como hexágonos. En la figura 2-54(a) las celdas

son del mismo tamaño y están agrupadas en unidades de siete celdas. Cada letra indica un grupo de frecuencias. Observe que para cada conjunto de frecuencias hay un área neutral de alrededor de dos celdas de ancho en la que esa frecuencia no se reutiliza, proporcionando buena separación y baja interferencia.

Encontrar localidades elevadas para colocar antenas de estación base es un problema importante. Este problema ha llevado a algunas portadoras de telecomunicaciones a forjar alianzas con la Iglesia Católica Romana, puesto que ésta posee un número sustancial de sitios potenciales para antenas en todo el mundo, todos convenientemente bajo una administración única.

En un área en la que la cantidad de usuarios ha crecido tanto que el sistema está sobrecargado, la potencia se reduce y las celdas sobrecargadas se dividen en celdas más pequeñas para permitir una mayor reutilización de las frecuencias, como se muestra en la figura 2-54(b). Qué tan grandes deben ser las celdas es un tema complejo, y se trata en (Hac, 1995).

En el centro de cada celda está una estación base a la cual transmiten todos los teléfonos de la celda. La estación base consiste en una computadora y un transmisor/receptor conectado a una antena. En un sistema pequeño, todas las estaciones base se conectan a un mismo dispositivo llamado **MTSO** (*mobile telephone switching office,* **oficina de conmutación de telefonía móvil**) o **MSC** (*mobile switching center,* **centro de conmutación móvil**). En un sistema grande, pueden ser necesarias varias MTSO, todas las cuales se conectan a una MTSO de segundo nivel, y así sucesivamente. Las MTSO son esencialmente oficinas finales como en un sistema telefónico y, en efecto, están conectadas al menos a una oficina final del sistema telefónico. Las MTSO se comunican con las estaciones base, con otras MTSO y con las PSTN mediante una red de conmutación de paquetes.

En cualquier instante, cada teléfono móvil está lógicamente en una celda específica y bajo el control de la estación base de esa celda. Cuando un teléfono móvil sale de una celda, su estación base nota que la señal telefónica se desvanece y pregunta a todas las estaciones base circundantes cuánta potencia están recibiendo de ella. A continuación, la estación base transfiere la posesión a la celda que está recibiendo la señal más fuerte, esto es, la celda donde se localiza ahora el teléfono. Se informa entonces al teléfono cuál es su nueva base y, si está efectuando una llamada, se le pide que cambie a un nuevo canal (porque el antiguo no se reutiliza en ninguna celda adyacente). Este proceso se llama **entrega** (*handoff*) y tarda cerca de 300 mseg. La asignación del canal la efectúa la MTSO, que es el centro nervioso del sistema. Las estaciones base realmente son sólo retransmisoras de radio.

Canales

El sistema AMPS emplea 832 canales dúplex, cada uno compuesto por un par de canales símplex. Hay 832 canales de transmisión símplex desde 824 hasta 849 MHz, y 832 canales de recepción símplex desde 869 hasta 894 MHz. Cada uno de estos canales símplex es de 30 kHz de ancho; por tanto, AMPS usa FDM para separar los canales.

En la banda de 800 MHz, las ondas de radio son de cerca de 40 cm de largo y viajan en línea recta; son absorbidas por árboles y plantas y rebotan en el suelo y los edificios. Es posible que una señal enviada por un teléfono móvil llegue a la estación base por una trayectoria directa,

pero también con un pequeño retardo después de rebotar en el suelo o en un edificio. Esto puede producir un efecto de eco o de distorsión de la señal. A veces, es posible incluso oír una conversación distante que ha rebotado varias veces.

En Estados Unidos, los 832 canales de cada ciudad son asignados por la FCC. De éstos, la mitad se asignan a la compañía de teléfonos local, la **portadora de línea** o **portadora de lado B**. La otra mitad se asigna a un nuevo participante en el negocio celular, la **portadora de lado A**. Lo que se busca es asegurar que existen por lo menos dos proveedores de telefonía celular, a fin de promover la competencia y los precios bajos.

Sin embargo, la distinción entre una compañía de teléfonos y una de telefonía celular ya no es muy clara, pues muchas compañías telefónicas tienen un socio celular, y en 1994 la AT&T se unió con McCaw Cellular, el operador celular más grande. Frecuentemente sucede que una compañía sea una portadora de lado A en algunos mercados y de lado B en otros. Por si fuera poco, una portadora puede vender o canjear cualquiera de sus 416 licencias de canal o todas.

Los 832 canales se dividen en cuatro categorías:

1. Control (base a móvil) para administrar el sistema.
2. Localización (base a móvil) para avisar a usuarios móviles que tienen llamadas.
3. Acceso (bidireccional) para establecimiento de llamadas y asignación de canales.
4. Datos (bidireccional) para voz, fax o datos.

Veintiuno de entre todos los canales se reservan para control, y están fijos dentro de un PROM en cada teléfono. Puesto que las mismas frecuencias no pueden reutilizarse en celdas cercanas, la cantidad real de canales de voz disponibles por célula es mucho menor que 832, normalmente cerca de 45.

Administración de llamadas

Cada teléfono móvil en AMPS tiene un número de serie de 32 bits y un número telefónico de 10 dígitos en su PROM. El número de teléfono se representa como un código de área de 3 dígitos, en 10 bits, y un número de suscriptor de 7 dígitos, en 24 bits. Cuando un teléfono se enciende, examina una lista preprogramada de 21 canales de control para encontrar la señal más potente. Los teléfonos móviles se programan para rastrear únicamente el lado A, únicamente el lado B, el lado A de preferencia o el lado B de preferencia, dependiendo de a cuál(es) servicio(s) se ha suscrito el cliente. Por el canal de control, aprende los números de los canales de acceso y de aviso.

El teléfono difunde entonces su número de serie de 32 bits y su número de teléfono de 34 bits. Al igual que toda la información de control de AMPS, este paquete se envía en forma digital varias veces y con código de corrección de error, aunque los canales de voz mismos son analógicos.

Cuando la estación base oye el anuncio, avisa a la MTSO, la cual registra la existencia de su nuevo cliente y también informa a la MTSO local del cliente de su ubicación actual. Durante el funcionamiento normal, el teléfono móvil se vuelve a registrar cada 15 minutos aproximadamente.

Para hacer una llamada, un usuario móvil enciende el teléfono, teclea el número al que desea llamar y oprime el botón de ENVIAR. El teléfono envía entonces el número por llamar y su propia identidad por el canal de acceso. Si ocurre una colisión, lo intenta otra vez más tarde. Cuando la estación base recibe la petición, informa a la MTSO. Si el que llama es un cliente de la compañía de la MTSO (o de uno de sus socios), la MTSO busca un canal desocupado para la llamada; si encuentra uno, el número de canal se envía de regreso por el canal de control. A continuación, el teléfono móvil se conecta en forma automática con el canal de voz seleccionado y espera hasta que la persona llamada levanta el teléfono.

Las llamadas que llegan funcionan de forma diferente. Para empezar, todos los teléfonos desocupados escuchan continuamente el canal de aviso para detectar mensajes dirigidos a ellos. Cuando se hace una llamada a un teléfono móvil (ya sea desde un teléfono fijo o algún otro teléfono móvil), se envía un paquete a la MTSO local del destinatario de la llamada para averiguar dónde se encuentra. Luego se envía un paquete a la estación base de su celda actual, la cual realiza entonces una difusión por el canal de aviso de la forma: "Unidad 14, ¿está ahí?" El teléfono llamado responde entonces con "Sí" por el canal de control. La base entonces dice algo como: "Unidad 14, tiene llamada por el canal 3". En este punto, el teléfono llamado conmuta al canal 3 y empieza a timbrar.

Consideraciones de seguridad

Los teléfonos celulares analógicos son totalmente inseguros. Cualquiera con un radiorreceptor de toda la banda (rastreador) puede sintonizar y escuchar todo lo que está ocurriendo dentro de una celda. La princesa Diana y su amante fueron atrapados una vez de esta manera, dando pie a titulares en todo el mundo. Puesto que muchos usuarios de teléfonos celulares no se dan cuenta de cuán inseguro es el sistema, a veces proporcionan por esta vía sus números de tarjetas de crédito y otra información que era confidencial.

Otro problema importante es el robo de tiempo al aire. Con un receptor de todas las bandas conectado a una computadora, un ladrón puede vigilar el canal de control y grabar el número de serie de 32 bits y los números telefónicos de 34 bits de todos los teléfonos móviles que oye. Con sólo recorrer la ciudad un par de horas, el delincuente puede construir una gran base de datos; después, puede elegir un número y usarlo para sus llamadas. Este truco funcionará hasta que la víctima reciba su factura, semanas después; para entonces, el ladrón habrá escogido un nuevo número.

Algunos ladrones ofrecen un servicio telefónico de bajo costo haciendo llamadas para sus clientes mediante el uso de los números robados. Otros reprograman teléfonos móviles con números robados y los venden como teléfonos que pueden hacer llamadas gratuitas.

Algunos de estos problemas podrían resolverse mediante cifrado, pero entonces la policía no podría intervenir fácilmente los teléfonos de los delincuentes inalámbricos. Este tema es causa de mucha controversia y se analiza con más detalle en el capítulo 7.

Otro problema en el área general de la seguridad es el vandalismo y daños a las antenas y las estaciones base. Todos estos problemas son bastante graves y ascienden a cientos de millones de dólares al año en pérdidas para la industria celular.

2.7.4. Teléfonos celulares digitales

La primera generación de sistemas celulares fue analógica; la segunda es digital. En Estados Unidos había básicamente un sistema único: AMPS. Cuando llegó el momento de la tecnología digital emergieron tres o cuatro competidores, y comenzó una lucha por sobrevivir. Ahora parece que sobrevivirán dos sistemas. El primero es compatible con el esquema de asignación de frecuencia AMPS y se especifica en los estándares conocidos como IS-54 e IS-135. El otro está basado en el espectro extendido de secuencia directa y se especifica en el estándar IS-95.

El IS-54 es de modo dual (analógico y digital) y utiliza los mismos canales de 30 kHz que AMP; empaca 48.6 kbps en cada canal y lo comparte entre tres usuarios simultáneos. Cada usuario obtiene 13 kbps; el resto es para control y temporización. Las celdas, las estaciones base y las MTSO funcionan igual que en AMPS; sólo la señalización digital y la codificación digital de voz son diferentes. El sistema IS-95 es bastante nuevo. Lo estudiaremos cuando lleguemos a la asignación de canales en el capítulo 4.

En Europa ha sucedido el proceso inverso. Se usaban cinco sistemas analógicos diferentes en diferentes países, a tal grado que una persona con un teléfono británico no podía usarlo en Francia, etc. Esta experiencia llevó a los PTT europeos a convenir en un sistema digital común llamado **GSM** (*global systems for mobile communications,* **sistema global para comunicaciones móviles**) que entró en funciones antes que cualquiera de los sistemas en competencia de Estados Unidos. El sistema japonés es diferente de todos los anteriores.

Puesto que todos los sistemas europeos eran diferentes, fue más sencillo hacerlos de funcionamiento digital puro en una nueva banda de frecuencia (1.8 GHz), además de readaptar la banda de 900 MHz en los casos en que era posible. GSM usa tanto FDM como TDM. El espectro disponible se divide en 50 bandas de 200 kHz cada una. Dentro de cada banda se usa TDM para multiplexar varios usuarios.

Algunos teléfonos GSM usan tarjetas inteligentes, es decir, dispositivos del tamaño de una tarjeta de crédito que contienen una CPU. El número de serie y el número de teléfono se almacenan en la tarjeta, no en el aparato telefónico, con lo que mejora la seguridad física (robar el teléfono sin la tarjeta no sirve para conseguir el número). También se emplea cifrado. Veremos GSM en el capítulo 4.

2.7.5. Servicios de comunicaciones personales

El Santo Grial del mundo de los teléfonos es un pequeño teléfono inalámbrico que se pueda usar en y también llevarse a cualquier parte del mundo; deberá responder al mismo número de teléfono, sin importar dónde esté, así que la gente únicamente tendrá un número de teléfono (con AMPS, el teléfono de su casa y su teléfono móvil tienen diferentes números). Este sistema está actualmente en vigoroso desarrollo (Lipper y Rumsewicz, 1994). En Estados Unidos el sistema se llama **PCS** (*personal communications services,* **servicios de comunicaciones personales**); en todas las demás partes se llama **PCN** (*personal communications network*, **red de comunicaciones personales**). En el mundo de la telefonía, Estados Unidos tiene una especie de tradición

de marchar a un ritmo diferente que los demás. Afortunadamente, muchos de los detalles técnicos son iguales.

Los PCS usarán tecnología celular, pero con microceldas, quizá de 50 a 100 metros de ancho. Esto permite utilizar muy baja potencia (1/4 watt) y en consecuencia construir teléfonos muy pequeños y ligeros. Por otro lado, se requieren muchas más celdas que con AMPS, cuyas celdas abarcan 20 km. Si suponemos que una microcelda tiene 1/200 del diámetro de una celda AMPS, se requieren 40,000 celdas para cubrir la misma área. Aun si estas microceldas son mucho más económicas que las celdas AMPS, queda claro que construir un sistema PCS completo desde cero requerirá una inversión masiva en infraestructura mucho mayor que la de AMPS. Algunas compañías de teléfonos se han dado cuenta de que sus postes telefónicos son lugares excelentes para colocar estaciones base del tamaño de un tostador de pan, puesto que los postes y el cableado ya existen, con lo que se reduce enormemente el costo de instalación. Estas pequeñas estaciones base suelen llamarse **telepuntos**. Cuántos instalar y dónde ponerlos es una cuestión complicada (Steele *et al.*, 1995a, 1995b).

El gobierno de Estados Unidos (específicamente la FCC) está usando los PCS para sacar dinero de la nada: en 1994-95 subastó licencias para usar el espectro de PCS (1.7 a 2.3 GHz), remate que recabó 7700 millones de dólares para el gobierno. Este remate reemplazó el sistema anterior de concesión de bandas de frecuencia por lotería, eliminando así la práctica de ciertas compañías sin intereses en las telecomunicaciones de participar en la lotería. Cualquiera de esas compañías que ganara una frecuencia podía venderla de inmediato a una de las perdedoras por millones de dólares.

Desafortunadamente, nada es gratis, ni siquiera para el gobierno. La banda de 1.7 a 2.3 GHz ya está completamente asignada a otros usuarios, a los cuales se deberá dar otro espectro para que cambien a él. El problema es que el tamaño de la antena depende de la frecuencia, de modo que la reasignación forzada de frecuencia obligará a tirar a la basura una inversión multimillonaria en antenas, transmisores, etc. Hordas de cabilderos han invadido Washington con sugerencias acerca de quién deberá pagar todo esto. El resultado neto es que los PCS podrían no entrar en servicio ampliamente en este milenio. Para una forma más racional de manejar el espectro véase (Youssef *et al.*, 1995).

2.8. SATÉLITES DE COMUNICACIONES

En la década de 1950 y al inicio de la de 1960, se hicieron intentos por establecer sistemas de comunicación rebotando señales en globos meteorológicos metalizados. Desafortunadamente, las señales recibidas eran muy débiles para tener un uso práctico. Después, la Armada de Estados Unidos notó en el cielo una especie de globo meteorológico permanente —la Luna— y construyó un sistema funcional de comunicación entre los barcos y tierra firme rebotando señales en ella.

El progreso en el campo de la comunicación celestial tuvo que esperar hasta el lanzamiento del primer satélite de comunicaciones en 1962. La diferencia clave entre un satélite artificial y uno real es que el artificial puede ampliar las señales antes de devolverlas, convirtiendo una curiosidad en un potente sistema de comunicación.

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA

Los satélites de comunicaciones tienen algunas propiedades interesantes que los hacen atractivos para muchas aplicaciones. Un satélite de comunicaciones se puede ver como una gran repetidora de microondas en el cielo. El satélite contiene varios **transpondedores**, cada uno de los cuales capta alguna porción del espectro, amplifica la señal de entrada y después la redifunde a otra frecuencia para evitar la interferencia con la señal de entrada. Los haces retransmitidos pueden ser amplios y cubrir una fracción sustancial de la superficie de la Tierra, o estrechos y cubrir un área de sólo cientos de kilómetros de diámetro.

2.8.1. Satélites geosincrónicos

De acuerdo con la ley de Kepler, el periodo de la órbita de un satélite varía con el radio de la órbita a la potencia 3/2. Cerca de la superficie de la Tierra, el periodo es cercano a los 90 minutos. Los satélites de comunicaciones a altitudes tan bajas son problemáticos porque están a la vista de una estación terrestre determinada sólo durante un intervalo de tiempo corto.

Sin embargo, a una altitud aproximada de 36,000 km sobre el ecuador, el periodo del satélite es de 24 horas,† de modo que gira a la misma velocidad que la Tierra bajo él. Un observador que mire un satélite en una órbita ecuatorial lo verá suspendido en un punto fijo del cielo, al parecer sin movimiento. Es en extremo deseable tener el satélite fijo en el cielo, porque de otra manera se necesitaría una costosa antena guiable para rastrearlo.

Con la tecnología actual, no es prudente tener a los satélites espaciados menos de dos grados en el plano ecuatorial de 360 grados, para evitar la interferencia. Con el espaciado de dos grados sólo puede haber 360/2 = 180 satélites geosincrónicos de comunicaciones en el cielo al mismo tiempo. Algunos de estos apartados orbitales se reservan para otras clases de usuarios (por ejemplo, difusión de televisión, uso gubernamental y militar, etcétera).

Por fortuna, los satélites que utilizan partes diferentes del espectro no compiten, de modo que cada uno de los 180 satélites posibles podría manejar varias corrientes de datos en ambos sentidos simultáneamente. Como alternativa, dos o más satélites podrían ocupar un apartado orbital si operan a diferentes frecuencias.

Para evitar el caos total en el cielo, ha habido acuerdos internacionales respecto a quién puede usar cuáles apartados orbitales y frecuencias. En la figura 2-55 se listan las principales bandas comerciales. La banda C fue la primera en destinarse al tráfico comercial por satélite; en ella se asignan dos intervalos de frecuencia, el más bajo para tráfico de enlaces descendentes (desde el satélite) y el superior para tráfico de enlaces ascendente (hacia el satélite). Para una conexión dúplex se requiere un canal en cada sentido. Estas bandas ya están sobrepobladas porque también las usan las portadoras comunes para enlaces terrestres de microondas.

La siguiente banda más alta disponible para las portadoras de telecomunicaciones comerciales es la banda Ku. Esta banda no está congestionada (todavía), y a estas frecuencias los satélites pueden estar espaciados tan cerca como 1 grado. Sin embargo, existe un problema: la lluvia. El

† Para los puristas, el periodo de rotación es el día sideral: 23 horas 56 minutos 4.09 segundos.

Banda	Frecuencias	Enlace descendente (GHz)	Enlace ascendente (GHz)	Problemas
C	4/6	3.7–4.2	5.925–6.425	Interferencia terrestre
Ku	11/14	11.7–12.2	14.0–14.5	Lluvia
Ka	20/30	17.7–21.7	27.5–30.5	Lluvia; costo del equipo

Figura 2-55. Las principales bandas de satélite.

agua es un excelente absorbente de estas microondas cortas. Por fortuna, las tormentas fuertes casi nunca abarcan áreas extensas, de modo que con usar varias estaciones terrestres ampliamente separadas en lugar de una sola se puede resolver el problema, a expensas de gastar más en antenas, cables y circuitos electrónicos para conmutar con rapidez entre estaciones. Ya se asignó también ancho de banda en la banda Ka para tráfico comercial por satélite, pero el equipo necesario para aprovecharlo todavía es caro. Además de estas bandas comerciales, existen muchas bandas gubernamentales y militares.

Un satélite normal tiene entre 12 y 20 transpondedores, cada uno con un ancho de banda de 36 a 50 MHz. Se puede usar un transpondedor de 50 Mbps para codificar una sola corriente de datos de 50 Mbps, 800 canales digitales de voz a 64 kbps, o varias combinaciones distintas. Además, dos transpondedores pueden aplicar diferentes polarizaciones a la señal, de modo que puedan utilizar la misma gama de frecuencias sin interferencia. En los primeros satélites, la división de los transpondedores en canales era estática, dividiendo el ancho de banda en bandas de frecuencia fijas (FDM). Hoy día también se usa la multiplexión por división en el tiempo, debido a su mayor flexibilidad.

Los primeros satélites tenían un solo haz espacial que iluminaba la Tierra entera. Con la enorme reducción en precio, tamaño y requerimientos de energía de la microelectrónica se ha hecho posible una estrategia de difusión mucho más compleja. Cada satélite está equipado con múltiples antenas y transpondedores. Cada haz descendente se puede enfocar en un área geográfica pequeña, de modo que pueden tener lugar de manera simultánea múltiples transmisiones ascendentes y descendentes. Los llamados **haces puntuales** normalmente tienen forma elíptica y pueden ser tan pequeños como algunos cientos de kilómetros de diámetro. Un satélite de comunicaciones para Estados Unidos tendría normalmente un haz grande para los 48 estados contiguos más haces puntuales para Alaska y Hawai.

Un avance nuevo en el mundo de la comunicación por satélite es la invención de microestaciones de bajo costo, llamadas a veces **VSAT** (*very small aperture terminals,* **terminales de abertura muy pequeña**) (Ivancic *et al.*, 1994). Estas diminutas terminales tienen antenas de 1 metro y salidas de cerca de 1 watt de potencia. El enlace ascendente por lo general llega a 19.2 kbps, pero el descendente es más rápido, con frecuencia de 512 kbps. En muchos sistemas VSAT, las microestaciones no tienen suficiente energía para comunicarse en forma directa unas con otras (desde luego, por la vía del satélite). Para ello se necesita una estación terrena especial, el **eje**, con una antena grande de ganancia alta para retransmitir el tráfico entre VSAT, como se muestra en la figura 2-56. En este modo de operación, ya sea el emisor o el receptor tiene una antena

grande y un amplificador potente. El costo de tener estaciones de usuario final más baratas es un mayor retardo.

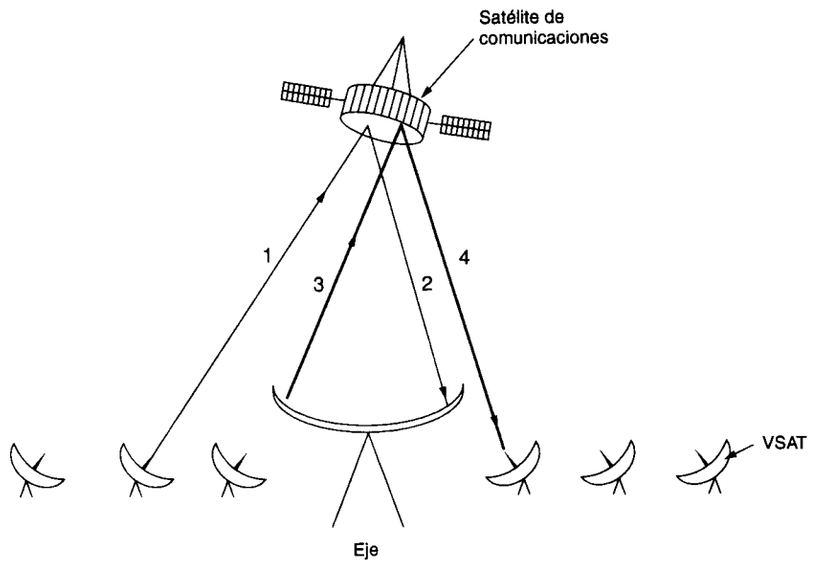


Figura 2-56. VSAT con un eje.

Los satélites de comunicaciones tienen varias propiedades que son radicalmente diferentes de los enlaces terrestres punto a punto. Para empezar, aunque las señales hacia y desde un satélite viajan a la velocidad de la luz (cerca de 300,000 km/seg), la gran distancia del viaje redondo introduce un retardo sustancial. Dependiendo de la distancia entre el usuario y la estación terrena y de la elevación del satélite sobre el horizonte, el tiempo de tránsito de extremo a extremo es de 250 a 300 mseg. Una cifra común es 270 mseg (540 para un sistema de VSAT con un eje).

Como base de comparación, los enlaces terrestres de microondas tienen un retardo de propagación de casi 3 µseg/km y los enlaces de cable coaxial o fibra óptica tienen un retardo de aproximadamente 5 µseg/km (las señales electromagnéticas viajan más rápidamente en el aire que en los materiales sólidos).

Otra propiedad importante de los satélites es que por su naturaleza son medios de difusión. No cuesta más mandar un mensaje a miles de estaciones dentro del alcance de un transpondedor que mandarlo a una sola. En algunas aplicaciones, esta propiedad es muy útil. Aun cuando la difusión se puede simular mediante líneas punto a punto, la difusión por satélite puede ser mucho más económica. Por otro lado, desde el punto de vista de la seguridad y confidencialidad, los satélites son un desastre completo: todos pueden oír todo. El cifrado es esencial cuando se requiere seguridad.

Los satélites también tienen la propiedad de que el costo de transmitir un mensaje es independiente de la distancia recorrida. Una llamada al otro lado del océano no cuesta más en cuanto a servicio que una llamada al otro lado de la calle. Los satélites tienen también excelentes tasas de errores y se pueden instalar en forma casi instantánea, una consideración importante para la comunicación militar.

2.8.2. Satélites de órbita baja

Durante los primeros 30 años de la era de los satélites, los de órbita baja casi no se usaron en comunicaciones porque aparecían y desaparecían con demasiada rapidez. En 1990, Motorola abrió nuevo camino al solicitar a la FCC permiso para lanzar 77 satélites de órbita baja para el proyecto Iridio (el iridio es el elemento 77). Más tarde se modificó el plan para usar solamente 66 satélites, de manera que el proyecto debió renombrarse Disprosio (elemento 66), aunque es probable que eso pareciera a muchos el nombre de una enfermedad. La idea era que tan pronto como un satélite se perdiera de vista, otro lo reemplazaría. Esta propuesta provocó un frenesí de actividad entre otras compañías de comunicaciones. De repente, todo mundo quiso lanzar una cadena de satélites de órbita baja. Aquí describiremos en forma breve el sistema Iridio, pues los otros son similares.

La meta básica de Iridio es proporcionar servicio mundial de telecomunicaciones usando aparatos manuales que se comunican con los satélites Iridio directamente. El sistema proporciona servicio de voz, datos, avisos, fax y navegación en todos los rincones del planeta. Este servicio compite de frente con PCS/PCN y hace obsoleto al segundo.

El sistema utiliza ideas del radio celular, pero con un giro. Normalmente las celdas están fijas mientras que los usuarios son móviles. Aquí, cada satélite tiene una cantidad considerable de haces puntuales que barren la Tierra mientras el satélite se mueve. Así, en este sistema tanto las celdas como los usuarios son móviles, pero las técnicas de relevo que se usan para el radio celular son igualmente aplicables al caso de una celda que abandona a un usuario como lo son al caso de un usuario que sale de una celda.

Los satélites se posicionan a una altura de 750 km en órbitas polares circulares. Los satélites se dispondrían en collares norte-sur, con un satélite cada 32 grados de latitud. La Tierra entera se cubriría con seis collares de satélites, como lo sugiere la figura 2-57(a). La gente que no sepa mucho de química puede visualizar este arreglo como un átomo muy, muy grande de disprosio, con la Tierra como núcleo y los satélites como electrones.

Cada satélite tendría un máximo de 48 haces de puntuales, con un total de 1628 celdas sobre la superficie de la Tierra, como se muestra en la figura 2-57(b). Las frecuencias se podrían reutilizar cada tres células, como en el radio celular convencional. Cada celda tendría 174 canales dúplex, para un total de 283,272 canales en todo el mundo. Algunos de éstos serían para avisos y navegación, cosa que no requiere mucho ancho de banda. (Los aparatos de avisos que se contemplan exhibirían dos líneas de texto alfanumérico.)

Los enlaces ascendentes y descendentes funcionarían en la banda L, a 1.6 GHz, con lo que harían posible comunicarse con el satélite empleando un pequeño aparato alimentado por pilas. Los mensajes recibidos por un satélite pero destinados a uno remoto serían retransmitidos por

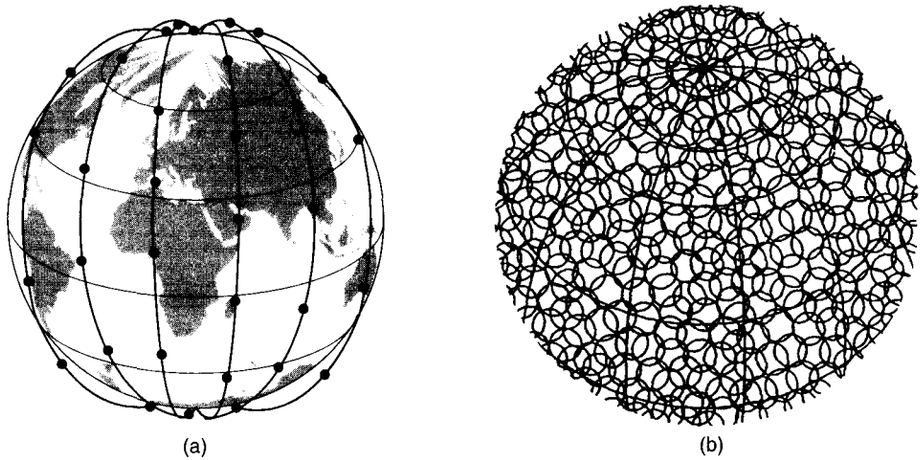


Figura 2-57. Los satélites Iridio forman seis collares alrededor de la Tierra. (b) 1628 celdas móviles cubren la Tierra.

satélites en la banda Ka. En el espacio exterior hay suficiente ancho de banda disponible para los enlaces entre satélites. El factor limitante serían los segmentos de los enlaces ascendentes y descendentes. Motorola estima que 200 MHz serían suficientes para todo el sistema.

El costo proyectado para el usuario final es de casi 3 dólares por minuto. Si esta tecnología puede proporcionar servicio universal en cualquier lugar de la Tierra por ese precio, es improbable que el proyecto muera por falta de clientes. Los viajeros de negocios y otros que quieran mantenerse en contacto todo el tiempo, aun en áreas no desarrolladas, se conectarán en manada. Sin embargo, en las áreas desarrolladas Iridio enfrentará una dura competencia de PCS/PCN con sus telepuntos de "tostador en un poste".

2.8.3. Satélites y fibra

Una comparación entre la comunicación por satélite y la comunicación terrestre es instructiva. Apenas hace 20 años, se podía apostar que el futuro de la comunicación estaba en los satélites de comunicaciones. Después de todo, el sistema telefónico había cambiado poco en los pasados 100 años y no mostraba señales de cambiar en los próximos 100. Este movimiento glacial lo provocó, en buena medida, el ambiente de reglamentación en el que se esperaba que las compañías de teléfonos proporcionaran un buen servicio de voz a precios razonables (lo cual hicieron) y tuvieran a cambio utilidades garantizadas sobre su inversión. Para quienes tenían datos que transmitir estaban disponibles módems de 1200 bps. Eso era prácticamente todo lo que había.

La introducción en 1984 de la competencia en Estados Unidos y poco después en Europa cambió todo eso de forma radical. Las compañías de teléfonos comenzaron a reemplazar por fibra sus redes de largo alcance e introdujeron servicios de alto ancho de banda como SMDS y

B-ISDN. También cesaron su ya añeja práctica de cobrar precios artificialmente altos a los usuarios de larga distancia para subsidiar el servicio local.

De repente, las conexiones terrestres de fibra parecían el ganador a largo plazo. No obstante, los satélites de comunicaciones tienen algunos nichos de mercado importantes en los que la fibra no tiene presencia (y que a veces no puede atender). Ahora veremos algunos de éstos.

Mientras que, en principio, una sola fibra tiene más ancho de banda potencial que todos los satélites lanzados hasta ahora, este ancho de banda no está disponible para la mayoría de los usuarios. Las fibras que se están instalando ahora se usan dentro del sistema telefónico para manejar muchas llamadas de larga distancia al mismo tiempo, no para proporcionar a los usuarios individuales ancho de banda alto. Además, pocos usuarios tienen siquiera acceso a un canal de fibra porque el viejo lazo local de par trenzado se interpone. Hacer una llamada a la oficina final de la compañía de teléfonos local a 28.8 kbps nunca dará un ancho de banda mayor que 28.8 kbps, no importa qué tan amplio sea el enlace intermedio. Con los satélites, resulta práctico para el usuario erigir una antena en el techo del edificio y saltarse por completo el sistema de telefonía. Para muchos usuarios, saltarse el lazo local es una motivación importante.

Una opción para usuarios que (ocasionalmente) necesitan 40 o 50 Mbps es rentar una portadora T3 (de 44.736 Mbps). Sin embargo, ésta es una solución cara. Si ese ancho de banda se necesita sólo de forma intermitente, una solución adecuada puede ser SMDS, pero no está disponible en todas partes, y el servicio de satélite sí.

Un segundo nicho es la comunicación móvil. En nuestros días, mucha gente se quiere comunicar mientras trota, maneja, navega y vuela. Los enlaces terrestres de fibra óptica no les sirven, pero los enlaces de satélite pueden ser una solución. Sin embargo, es posible que una combinación de radio celular y fibra será más que suficiente para la mayoría de los usuarios (aunque probablemente no para aquellos que están en el aire o en el mar).

Un tercer nicho es las situaciones en las que la difusión es esencial. Un mensaje que se envía por satélite se puede recibir en miles de estaciones terrestres al mismo tiempo. Por ejemplo, para una organización que transmite una corriente de precios de acciones, bonos o mercancías a miles de corredores, un sistema de satélite podría ser mucho más económico que simular la difusión en tierra.

Un cuarto nicho es la comunicación en lugares con terreno escabroso o con una infraestructura terrestre poco desarrollada. Por ejemplo, Indonesia tiene su propio satélite para tráfico telefónico doméstico. Lanzar un solo satélite fue mucho más fácil que tender miles de cables submarinos entre todas las islas del archipiélago.

Un quinto nicho de mercado para los satélites es en los casos en que obtener el derecho de paso para la fibra tendida por el piso es difícil o excesivamente costoso. En sexto lugar, cuando la instalación rápida es crítica, como en los sistemas militares de comunicación en tiempo de guerra, los satélites ganan con facilidad.

En síntesis, parece que la corriente principal en la comunicación del futuro será la fibra óptica terrestre combinada con radio celular, pero para algunos usos especializados los satélites son mejores. Sin embargo, hay un factor que puede cambiar todo esto: la economía. Aunque la fibra ofrece mayor ancho de banda, ciertamente es posible que la comunicación terrestre y por satélite compitan en forma agresiva con sus precios. Si los avances en la tecnología reducen de

forma radical el costo de lanzar un satélite (por ejemplo, algún futuro transbordador espacial podría lanzar docenas de satélites en un viaje), o si los satélites de órbita baja se popularizan, no hay certeza de que la fibra gane en todos los mercados.

2.9. RESUMEN

La capa física es la base de todas las redes. La naturaleza impone dos límites fundamentales a todos los canales, y esto determina su ancho de banda. Estos límites son el de Nyquist que tiene que ver con los canales sin ruido y el de Shannon, para canales ruidosos.

Los medios de transmisión pueden ser guiados o no guiados. Los principales medios guiados son el par trenzado, el cable coaxial y la fibra óptica. Los medios no guiados incluyen la radio, las microondas, el infrarrojo y los láseres a través del aire.

Un elemento clave de la mayor parte de las redes de área amplia es el sistema telefónico. Sus componentes principales son los lazos locales, troncales y conmutadores. Los lazos locales son circuitos de par trenzado, analógicos, que requieren módems para transmitir datos digitales. Los troncales son digitales y se pueden multiplexar de varias formas, incluidas FDM, TDM y WDM. Los conmutadores incluyen los de matriz, los de división en el espacio y los de división en el tiempo. Tanto la conmutación de circuitos como la de paquetes son importantes.

En el futuro, el sistema telefónico será digital de un extremo al otro y llevará tráfico tanto de voz como sin voz por las mismas líneas. Se están presentando dos variantes de este nuevo sistema, llamado ISDN. La ISDN de banda estrecha es un sistema digital de conmutación de circuitos, que representa una mejora incremental respecto al sistema actual. En contraste, la ISDN de banda ancha representa un cambio del paradigma, ya que se basa en tecnología ATM de conmutación de células. Existen varias clases de conmutadores ATM, incluidos el conmutador de eliminación y el de Batcher-banyan.

Para aplicaciones móviles, no es adecuado el sistema telefónico alambrado. Las alternativas del sistema telefónico incluyen la radio celular y los satélites de comunicaciones. La radio celular ya se usa ahora ampliamente para teléfonos portátiles, pero pronto será común también para el tráfico de datos. La generación actual de sistemas celulares (por ejemplo, AMPS) es analógica, pero la generación que sigue (por ejemplo, PCS/PCN) será completamente digital. Los satélites de comunicación tradicionales son geosincrónicos, pero ha surgido mucho interés por los sistemas de satélites de órbita baja, como Iridio.

PROBLEMAS

1. Calcule los coeficientes de Fourier para la función $f(t) = t\ (0 \leq t \leq 1)$.
2. Un canal sin ruido de 4 kHz se muestrea cada 1 mseg. ¿Cuál es la máxima velocidad de transferencia de datos?
3. Los canales de televisión tienen un ancho de 6 Mhz. ¿Cuántos bits/seg se pueden enviar si se usan señales digitales de cuatro niveles? Suponga que el canal es sin ruido.

4. Si se envía una señal binaria por un canal de 3 kHz cuya relación de señal a ruido es de 20 dB, ¿cuál es la máxima velocidad de transmisión de datos que se puede obtener?
5. ¿Qué relación de señal a ruido se necesita para poner una portadora T1 en una línea de 50 kHz?
6. ¿Qué diferencia hay entre una estrella pasiva y una repetidora activa en una red de fibra óptica?
7. ¿Cuánto ancho de banda existe en 0.1 micras de espectro a una longitud de onda de 1 micra?
8. Se desea enviar una secuencia de imágenes de pantalla de computadora por una fibra óptica. La pantalla es de 480 × 640 pixeles y cada pixel ocupa 24 bits. Hay 60 imágenes de pantalla por segundo. ¿Cuánto ancho de banda se necesita y cuántas micras de longitud de onda se necesitan para esta banda a 1.30 micras?
9. ¿Se cumple el teorema de Nyquist para la fibra óptica o solamente para el alambre de cobre?
10. En la figura 2-6 la banda de la izquierda es más angosta que las otras. ¿Por qué?
11. A menudo las antenas de radio funcionan mejor cuando el diámetro de la antena es igual a la longitud de la onda de radio. Las antenas prácticas tienen diámetros desde 1 cm hasta 5 m de diámetro. ¿Qué gama de frecuencias cubre esto?
12. El desvanecimiento de múltiples trayectorias alcanza un máximo cuando los dos haces llegan desfasados 180 grados. ¿Qué tan diferentes deben ser las trayectorias para que el desvanecimiento sea máximo para un enlace de microondas de 1 GHz de 50 km de largo?
13. Un rayo láser de 1 mm de diámetro se apunta a un detector de 1 mm de diámetro a 100 m en el techo de un edificio. ¿Cuánta desviación angular deberá tener el láser antes de que pierda al detector?
14. Un sistema telefónico simple consiste en dos oficinas finales y una oficina de cargo a la que está conectada cada oficina final por un troncal dúplex de 1 MHz. En promedio, cada teléfono se usa para hacer cuatro llamadas por cada jornada de 8 horas. La duración media de las llamadas es de 6 minutos. El 10% de las llamadas son de larga distancia (esto es, pasan por la oficina de cargo). ¿Cuál es la cantidad máxima de teléfonos que puede manejar una oficina final? (Suponga que hay 4 kHz por circuito.)
15. Una compañía de teléfonos regional tiene 10 millones de suscriptores. Cada uno de sus teléfonos está conectado a una oficina central por un par trenzado de cobre. La longitud promedio de estos pares trenzados es de 10 km. ¿Cuánto vale el cobre de los lazos locales? Suponga que la sección transversal de cada filamento es un círculo de 1 mm de diámetro, que el peso específico relativo del cobre es 9.0 y que el cobre se vende a 3 dólares por kilogramo.
16. El costo de un microprocesador potente se ha reducido a tal grado que ahora es posible incluir uno en cada módem. ¿Cómo afecta esto el manejo de errores en las líneas telefónicas?
17. Un diagrama de constelación de módem, similar al de la figura 2-19 tiene puntos de datos en las siguientes coordenadas: (1, 1), (1, −1), (−1, 1) y (−1, −1). ¿Cuántos bps puede lograr un módem a 1200 baud con estos parámetros?
18. Un diagrama de constelación de módem, similar al de la figura 2-19 tiene puntos de datos en (0, 1) y (0, 2). ¿El módem usa modulación de fase o modulación de amplitud?

19. ¿Embona FTTH en el modelo de compañía de teléfonos de oficinas finales, oficinas de cargo, etc., o se debe cambiar el modelo de manera fundamental? Explique su respuesta.

20. En el nivel más bajo, el sistema de teléfonos tiene forma de estrella, con todos los lazos locales de un vecindario convergiendo en una oficina final. En contraste, la televisión por cable consiste en un solo cable largo que pasa por todas las casas del mismo vecindario. Suponga que una televisión futura por cable fuera de fibra de 10 Gbps en lugar de cobre. ¿Se podría usar para simular el modelo telefónico en que todo mundo tiene su propia línea privada a la oficina final? Si es así, ¿cuántas casas de un teléfono se podrían conectar a una sola fibra?

21. Un sistema de televisión por cable tiene 100 canales comerciales, todos los cuales alternan sus programas con anuncios. ¿Se parece más esto a TDM o a FDM?

22. ¿Por qué se fijó el tiempo de muestreo de PCM en 125 μseg?

23. ¿Cuál es el porcentaje de información extra en una portadora T1?; esto es, ¿qué porcentaje de los 1.544 Mbps no se entrega al usuario final?

24. Compare la velocidad de transmisión de datos máxima de un canal sin ruido de 4 kHz que utiliza:
(a) Codificación analógica con 2 bits por muestra.
(b) El sistema T1 de PCM.

25. Si un sistema de portadora T1 pierde la pista de dónde está, trata de resincronizarse con base en el primer bit de cada marco. ¿Cuántos marcos se tendrían que inspeccionar en promedio para resincronizarse con una probabilidad de 0.001 de estar en un error?

26. ¿Cuál es la diferencia, si la hay, entre la parte desmoduladora de un módem y la parte codificadora de un codec? (Después de todo, ambas convierten señales analógicas a digitales.)

27. Una señal se transmite en forma digital por un canal sin ruido de 4 kHz, con una muestra cada 125 μseg. ¿Cuántos bits por segundo se envían realmente con cada uno de los siguientes métodos de codificación?
(a) CCITT, 2.048 Mbps estándar.
(b) DPCM con un valor de señal relativo de 4 bits.
(c) Modulación delta.

28. Se codifica una onda senoidal pura de amplitud A usando modulación delta, con x muestras/seg. Una salida de +1 corresponde a un cambio de señal de $+A/8$, y una señal de salida de −1 corresponde a un cambio de señal de $-A/8$. ¿Cuál es la frecuencia más alta que se puede rastrear sin error acumulativo?

29. Los relojes de SONET tienen una tasa de deriva de casi 1 parte en 10^9. ¿Cuánto tiempo tomará para que la deriva iguale el ancho de 1 bit? ¿Cuáles son las implicaciones de este cálculo?

30. En la figura 2-32 se establece que la velocidad de transmisión de datos del usuario para OC-3 es de 148.608 Mbps. Demuestre cómo se puede deducir esta cantidad de los parámetros de OC-3 de SONET.

31. ¿Cuál es el ancho de banda disponible para el usuario en una conexión OC-12c?

32. Tres redes de conmutación de paquetes contienen n nodos cada una. La primera red tiene una topología de estrella con un conmutador central, la segunda es un anillo (bidireccional) y la tercera está interconectada por completo, con un alambre de cada nodo en cada uno de los otros nodos. ¿Cuáles son las trayectorias de transmisión óptima, media y de peor caso en saltos?

33. Compare el retardo al enviar un mensaje de x bits por una trayectoria de k saltos en una red de conmutación de circuitos y en una red de conmutación de paquetes (con carga ligera). El tiempo de establecimiento de circuito es de s segundos, el retraso de propagación es de d segundos por salto, el tamaño del paquete es de p bits y la relación de transmisión de datos es de b bps. ¿En qué condiciones tiene un retardo menor la red de paquetes?

34. Suponga que se van a transmitir x bits de datos de usuario por una trayectoria de k saltos en una red de conmutación de paquetes como una serie de paquetes, cada uno contiene p bits de datos y h bits de cabecera, donde $x \gg p + h$. La tasa de bits de las líneas es de b bps y el retardo de propagación es nulo. ¿Qué valor de p minimiza el retardo total?

35. ¿Cuántos puntos de cruce tienen los conmutadores de las figuras 2-39(a) y 2-39(b)? Compare esto con un conmutador completo de matriz de una sola etapa de 16×16.

36. En el conmutador por división en el espacio de la figura 2-39(a), ¿cuál es la cantidad más pequeña de conexiones existentes que pueden encuadrar una llamada de salida nueva?

37. Un diseño alterno al de la figura 2-39(a) es uno en el cual las 16 líneas se dividen en dos bloques de ocho, en lugar de cuatro bloques de cuatro (esto es, $n = 8$ en lugar de $n = 4$). Tal diseño ahorraría costos de *hardware* puesto que sólo se necesitarían dos concentradores en los extremos de entrada y salida. ¿Cuál es el argumento más fuerte contra esta alternativa?

38. ¿Cuántas líneas puede manejar un conmutador por división en el tiempo si el tiempo de acceso a RAM es de 50 nseg?

39. ¿Cuántos bits de memoria temporal en RAM necesita un intercambiador conmutador de tiempo si las muestras de la línea de entrada son de 10 bits y hay 80 líneas de entrada?

40. ¿Es forzoso que la conmutación por división en el tiempo introduzca un retardo mínimo en cada etapa de conmutación? ¿Si es así, de cuánto?

41. ¿Cuánto tiempo toma transmitir una imagen facsimilar de 8 pulgadas por 10 pulgadas por un canal B de ISDN? El facsímil digitaliza la imagen con 300 pixeles por pulgada y asigna 4 bits por pixel. Las máquinas de fax actuales operan a velocidades más altas por líneas telefónicas. ¿Cómo cree que lo logran?

42. Cite una ventaja y una desventaja de NT12 (en oposición a NT1 y NT2) en una red ISDN.

43. En la figura 2-50(a) vimos colisiones entre células que viajan a través de un conmutador *banyan*. Estas colisiones ocurrieron en las etapas primera y segunda. ¿Pueden ocurrir también colisiones en la tercera etapa? Si es así, ¿en qué condiciones?

44. Para este problema es necesario enrutar paso a paso varias células por un conmutador Batcher-banyan de ATM. Están presentes cuatro células en las líneas de entrada 0 a 3 dirigidas a 3, 5, 2 y 1 respectivamente. Para cada una de las seis etapas del conmutador Batcher y los cuatro pasos del conmutador banyan (lo que incluye la entrada y la salida), liste las células presentes como óctuplas (célula en la línea 0, célula en la línea 1 y así sucesivamente). Indique las líneas sin célula con –.

45. Repita el problema anterior empezando con (7, –, 6, –, 5, –, 4, –).

46. Un conmutador ATM tiene 1024 líneas de entrada y 1024 líneas de salida. Las líneas operan a la velocidad de SONET de 622 Mbps, dando una velocidad de usuario de 594 Mbps aproximadamente.

¿Qué ancho de banda total necesita el conmutador para manejar la carga? ¿Cuántas células por segundo debe ser capaz de procesar?

47. En un sistema común de telefonía celular con celdas hexagonales está prohibido reutilizar una banda de frecuencia en una celda adyacente. Si están disponibles 840 frecuencias en total, ¿cuántas se pueden usar en una celda determinada?

48. Haga un cálculo aproximado de la cantidad de microceldas de PCS de 100 m de diámetro que se necesitarían para cubrir San Francisco (120 km^2).

49. A veces, cuando un usuario de teléfono celular cruza la frontera de una celda a otra, la llamada actual se corta de manera abrupta, aunque todos los transmisores y receptores estén funcionando a la perfección. ¿Por qué?

50. Los 66 satélites de órbita baja del proyecto Iridio se dividen en seis collares alrededor de la Tierra. A la altitud a la que están, el periodo es de 90 minutos. ¿Cuál es el intervalo medio entre relevos de un transmisor estacionario?

3

LA CAPA DE ENLACE DE DATOS

En este capítulo estudiaremos el diseño de la capa 2, la capa de enlace de datos. Este estudio tiene que ver con los algoritmos para lograr una comunicación confiable y eficiente entre dos máquinas adyacentes en la capa de enlace de datos. Por adyacente, queremos decir que las dos máquinas están conectadas físicamente por un canal de comunicaciones que actúa conceptualmente como un alambre (por ejemplo, un cable coaxial o una línea telefónica). La propiedad esencial de un canal que lo hace asemejarse a un alambre es que los bits se entregan con exactitud en el mismo orden en que fueron enviados.

A primera vista podría pensarse que este problema es tan trivial que no hay ningún *software* que estudiar: la máquina *A* sólo pone los bits en el alambre y la máquina *B* simplemente los toma. Desafortunadamente, los circuitos de comunicación cometen errores ocasionales. Además, tienen una tasa de datos finita y hay un retardo de propagación diferente de cero entre el momento en que se envía un bit y el momento en que se recibe. Estas limitaciones tienen implicaciones importantes para la eficiencia de la transferencia de datos. Los protocolos usados para comunicaciones deben considerar todos estos factores. Dichos protocolos son el tema de este capítulo.

Tras una introducción a los aspectos de diseño presentes en la capa de enlace de datos, comenzaremos nuestro estudio de sus protocolos observando la naturaleza de los errores, sus causas y la manera en que se pueden detectar y corregir. Después estudiaremos una serie de protocolos de complejidad creciente, cada uno de los cuales resuelve una cantidad mayor de los problemas presentes en esta capa. Por último, concluiremos con un estudio del modelado y la corrección de los protocolos y daremos algunos ejemplos de protocolos de enlace de datos.

3.1. CUESTIONES DE DISEÑO DE LA CAPA DE ENLACE DE DATOS

La capa de enlace de datos tiene que desempeñar varias funciones específicas que incluyen proporcionar una interfaz de servicio bien definida con la capa de red, determinar la manera en que los bits de la capa física se agrupan en marcos, manejar los errores de transmisión y regular el flujo de marcos para que los receptores lentos no sean abrumados por los transmisores rápidos. En las siguientes secciones examinaremos, por turno, cada uno de estos puntos.

3.1.1. Servicios proporcionados a la capa de red

La función de la capa de enlace de datos es suministrar servicios a la capa de red. El servicio principal es la transferencia de datos de la capa de red en la máquina de origen a la capa de red en la máquina de destino. En la máquina de origen hay una entidad, llámese proceso, en la capa de red que entrega algunos bits a la capa de enlace de datos para su transmisión a la máquina de destino. El trabajo de la capa de enlace de datos es transmitir los bits a la máquina de destino, para que puedan ser entregados a su capa de red, como se muestra en la figura 3-1(a). La transmisión real sigue la trayectoria de la figura 3-1(b), pero es más fácil pensar en términos de dos procesos de capa de enlace de datos que se comunican usando un protocolo de enlace de datos. Por esta razón, a lo largo de este capítulo usaremos implícitamente el modelo de la figura 3-1(a).

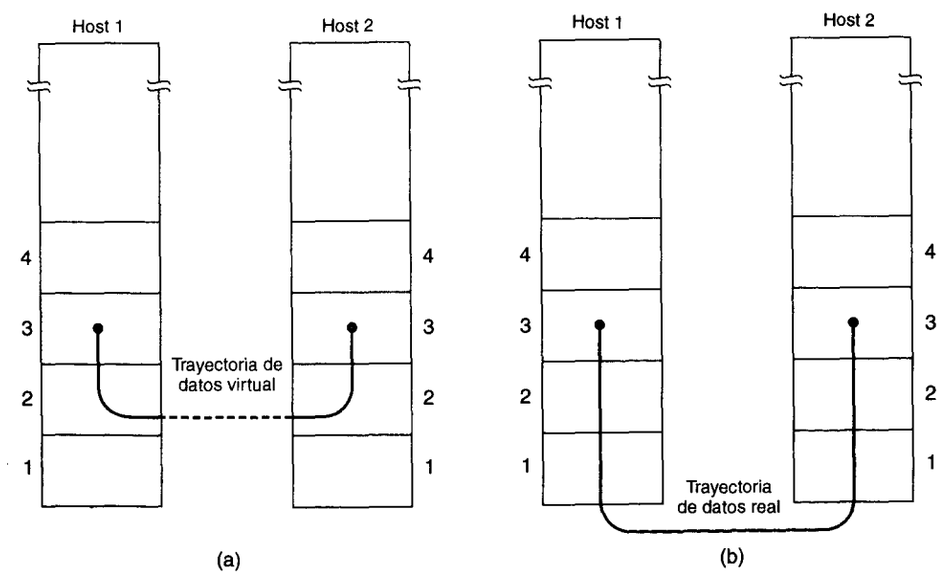


Figura 3-1. (a) Comunicación virtual. (b) Comunicación real.

La capa de enlace de datos puede diseñarse para ofrecer varios servicios. Los servicios reales ofrecidos pueden variar de sistema en sistema. Tres posibilidades razonables que normalmente se proporcionan son:

1. Servicio sin acuse sin conexión.
2. Servicio con acuse sin conexión.
3. Servicio con acuse orientado a la conexión.

Consideremos cada uno de ellos por separado.

El servicio sin acuse sin conexión consiste en hacer que la máquina de origen envíe marcos independientes a la máquina de destino sin pedir que ésta los reconozca o acuse su recibo. No se establece conexión de antemano ni se libera después. Si se pierde un marco debido a ruido en la línea, no se intenta recuperarlo en la capa de enlace de datos. Esta clase de servicio es apropiada cuando la tasa de errores es muy baja, por lo que la recuperación se deja a las capas más altas. También es apropiada para el tráfico en tiempo real, por ejemplo de voz, en el que la llegada retrasada de datos es peor que los errores de datos. La mayoría de las LAN usan servicios sin acuse sin conexión en la capa de enlace de datos.

El siguiente paso hacia arriba en cuanto a confiabilidad es el servicio con acuse sin conexión. Cuando se ofrece este servicio, aún no se usan conexiones, pero cada marco enviado es reconocido individualmente. De esta manera, el transmisor sabe si el marco ha llegado bien o no. Si no ha llegado en un tiempo especificado, puede enviarse nuevamente. Este servicio es útil en canales inestables, como los de los sistemas inalámbricos.

Tal vez valga la pena poner énfasis en que proporcionar acuses de recibo en la capa de enlace de datos sólo es una optimización, nunca un requisito. La capa de transporte siempre puede enviar un mensaje y esperar que sea reconocido. Si el acuse no llega antes de terminar el temporizador, el transmisor puede volver a enviar el mensaje. El problema con esta estrategia es que, si el mensaje promedio se divide en, digamos, 10 marcos, y se pierde el 20% de todos los marcos enviados, el mensaje puede tardar mucho tiempo en pasar. Si los marcos individuales se reconocen y retransmiten, los mensajes completos pasan con mayor rapidez. En los canales confiables, como la fibra, el gasto extra que implica el uso de un protocolo de enlace de datos muy robusto puede ser innecesario, pero en canales inalámbricos bien vale la pena el costo debido a su inestabilidad inherente.

Regresando a nuestros servicios, el servicio más elaborado que puede proporcionar la capa de enlace de datos a la capa de red es el servicio orientado a la conexión. Con este servicio, las máquinas de origen y de destino establecen una conexión antes de transferir datos. Cada marco enviado a través de la conexión está numerado, y la capa de enlace de datos garantiza que cada marco enviado llegará a su destino. Es más, se garantiza que cada marco será recibido exactamente una vez y que todos los marcos sean recibidos en el orden adecuado. En contraste, con el servicio sin conexión es concebible que un acuse de recibo perdido cause el envío de un marco varias veces, y por tanto sea recibido varias veces. El servicio orientado a la conexión, por su parte, proporciona a los procesos de la capa de red el equivalente de un flujo de bits confiable.

Al usarse un servicio orientado a conexión, las transferencias tienen tres fases distintas. En la primera fase, la conexión se establece haciendo que ambos lados inicialicen las variables y contadores necesarios para seguir la pista de los marcos que han sido recibidos y los que no. En la segunda fase se transmiten uno o más marcos. En la tercera fase, la final, la conexión se cierra, liberando las variables, los *buffers* y otros recursos utilizados para mantener la conexión.

Considere un ejemplo típico: una subred de WAN que consiste en enrutadores conectados por medio de líneas telefónicas arrendadas punto a punto. Cuando llega un marco a un enrutador, el *hardware* verifica la suma de comprobación, y pasa el marco al *software* de la capa de enlace de datos (que podría estar integrado en un *chip* de la tarjeta adaptadora de red). La capa de enlace de datos comprueba si éste es el marco esperado y, de ser así, entrega el paquete contenido en el campo de carga útil al *software* del enrutador. El *software* del enrutador escoge la línea de salida adecuada y pasa el paquete de regreso al *software* de la capa de enlace de datos, que luego lo transmite. El flujo a través de los enrutadores se muestra en la figura 3-2.

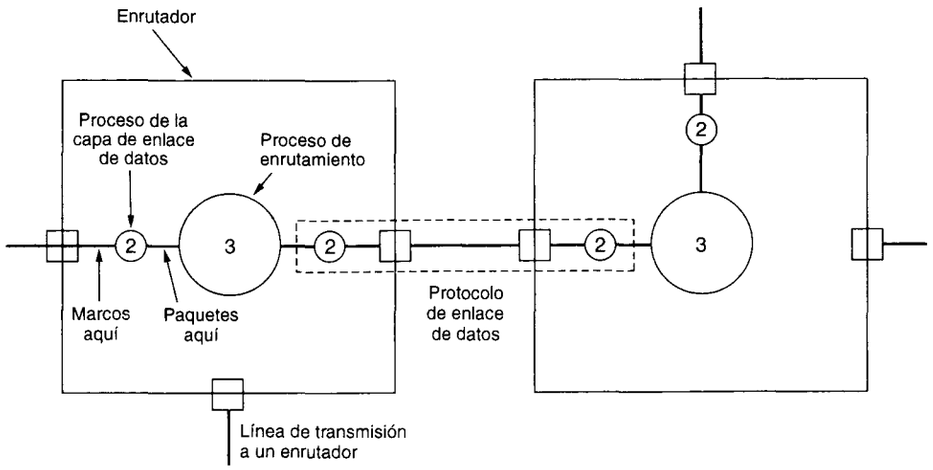


Figura 3-2. Ubicación del protocolo de enlace de datos.

El código de enrutamiento con frecuencia quiere que el trabajo se haga bien, es decir, que haya conexiones estables y ordenadas en cada una de las líneas punto a punto. No quiere que se le moleste frecuentemente con paquetes que se perdieron en el camino. Es responsabilidad del protocolo de enlace de datos, mostrado en el rectángulo punteado, hacer que las líneas de comunicación no estables parezcan perfectas o, cuando menos, bastante buenas. Esta propiedad es especialmente importante para los enlaces inalámbricos, que inherentemente son muy poco confiables. Como información adicional, aunque hemos mostrado múltiples copias del *software* de la capa de enlace de datos en cada enrutador, de hecho una sola copia maneja todas las líneas, con diferentes tablas y estructuras de datos para cada una.

Aunque este capítulo trata específicamente la capa de enlace de datos y los protocolos de enlace de datos, muchos de los principios que estudiaremos aquí, como el control de errores y el control de flujo, también se encuentran en el protocolo de transporte, así como en otros protocolos.

3.1.2. Enmarcado

A fin de proporcionar servicios a la capa de red, la capa de enlace de datos debe usar los servicios proporcionados a él por la capa física. Lo que hace la capa física es aceptar un flujo de bits en bruto e intentar entregarlo al destino. No se garantiza que este flujo de bits esté libre de errores. El número de bits recibidos puede ser menor, igual o mayor que el número de bits transmitidos, y pueden tener diferentes valores. Es responsabilidad de la capa de enlace de datos detectar y, de ser necesario, corregir los errores.

El enfoque común es que la capa de enlace de datos divida el flujo de bits en marcos discretos y que calcule la suma de comprobación para cada marco. (Posteriormente en este capítulo se analizarán los algoritmos de suma de comprobación.) Cuando un marco llega al destino, se recalcula la suma de comprobación. Si la nueva suma de comprobación calculada es distinta de la contenida en el marco, la capa de enlace de datos sabe que ha ocurrido un error y toma medidas para manejarlo (por ejemplo, descartando el marco malo y enviando de regreso un informe de error).

La división en marcos del flujo de bits es más difícil de lo que parece a primera vista. Una manera de lograr esta división en marcos es introducir intervalos de tiempo entre los marcos, a semejanza de los espacios entre las palabras en el texto común. Sin embargo, las redes pocas veces ofrecen garantías sobre la temporización, por lo que es posible que estos intervalos sean eliminados o que puedan introducirse otros intervalos durante la transmisión.

Dado que es demasiado riesgoso depender de la temporización para marcar el inicio y el final de cada marco, se han diseñado otros métodos. En esta sección veremos cuatro métodos.

1. Conteo de caracteres.
2. Caracteres de inicio y fin, con relleno de caracteres.
3. Indicadores de inicio y fin, con relleno de bits.
4. Violaciones de codificación de la capa física.

El primer método de enmarcado se vale de un campo del encabezado para especificar el número de caracteres en el marco. Cuando la capa de enlace de datos del destino ve la cuenta de caracteres, sabe cuántos caracteres siguen, y por tanto dónde está el fin del marco. Esta técnica se muestra en la figura 3-3(a) para cuatro marcos de 5, 5, 8 y 8 caracteres de longitud, respectivamente.

El problema con este algoritmo es que la cuenta puede alterarse por un error de transmisión. Por ejemplo, si la cuenta de caracteres de 5 en el segundo marco de la figura 3-3(b) se vuelve un 7, el destino perderá la sincronía y será incapaz de localizar el inicio del siguiente marco. Aún si la suma de comprobación es incorrecta, de manera que el destino sabe que el marco está mal, no tiene forma de saber dónde comienza el siguiente marco. El envío de un marco de regreso a la

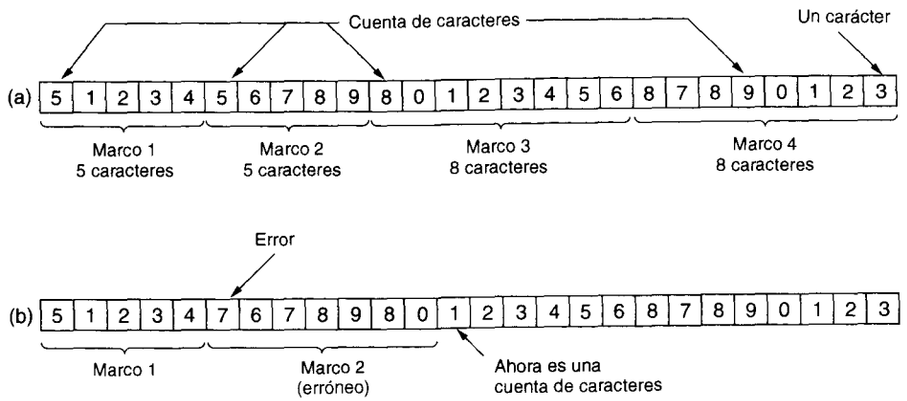


Figura 3-3. Una corriente de caracteres. (a) Sin errores. (b) Con un error.

fuente solicitando una retransmisión tampoco ayuda, ya que el destino no sabe cuántos caracteres tiene que saltar para llegar al inicio de la retransmisión. Por esta razón, el método de conteo de caracteres ya casi no se usa en la actualidad.

El segundo método de enmarcado supera el problema de resincronización tras un error al hacer que cada marco comience con la secuencia de caracteres ASCII DLE STX y termine con la secuencia DLE ETX. (DLE es *Data Link Escape*, escape de enlace de datos; STX es *Start of TeXt*, inicio de texto, y ETX es *End of TeXt*, fin de texto.) De esta manera, si el destino llega a perder la pista de los límites del marco, todo lo que tiene que hacer es buscar los caracteres DLE STX o DLE ETX para determinarlos.

Hay un problema importante con este método cuando se transmiten datos binarios, como programas objeto o números de punto flotante. Puede ocurrir fácilmente que los caracteres correspondientes a DLE STX o a DLE ETX ocurran en los datos, lo que interferirá en el enmarcado. Una forma de resolver este problema es hacer que la capa de enlace de datos inserte un carácter ASCII DLE justo antes de cada carácter DLE "accidental" de los datos. La capa de enlace de datos del lado receptor quita el DLE antes de entregar los datos a la capa de red. Esta técnica se llama **relleno de caracteres**. Por tanto, un DLE STX o DLE ETX de enmarcado puede distinguirse de uno en los datos por la ausencia o presencia de un solo DLE. Los DLE en los datos siempre se doblan. En la figura 3-4 se da un ejemplo de flujo de datos antes del relleno de caracteres, después del relleno y después de quitar el relleno.

Una desventaja importante del uso de esta técnica de enmarcado es que está fuertemente atada a los caracteres de 8 bits en general y al código ASCII en particular. A medida que se desarrollaron las redes, las desventajas de incorporar el código de caracteres en el mecanismo de enmarcado se volvieron más obvias, por lo que tuvo que desarrollarse una técnica nueva que permitiera caracteres de tamaño arbitrario.

La nueva técnica permite que los marcos de datos contengan un número arbitrario de bits y admite códigos de caracteres con un número arbitrario de bits por carácter. La técnica funciona

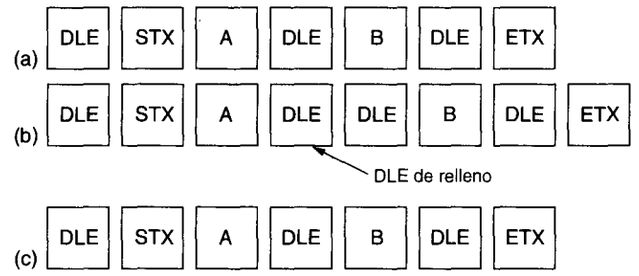


Figura 3-4. (a) Datos enviados por la capa de red. (b) Datos después del relleno de caracteres por la capa de enlace de datos. (c) Datos pasados a la capa de red del lado receptor.

de la siguiente manera. Cada marco comienza y termina con un patrón especial de bits, 01111110, llamado byte **indicador**. Cada vez que la capa de enlace de datos del transistor encuentra cinco unos consecutivos en los datos, automáticamente inserta un bit 0 en la cadena de bits. Este **relleno de bits** es análogo al relleno de caracteres, en el cual se rellena la cadena de datos de salida con un DLE entes de otro DLE en los datos.

Cuando el receptor ve cinco bits 1 de entrada consecutivos, seguidos de un bit 0, automáticamente quita (es decir, borra) el bit 0 de relleno. Así como el relleno de caracteres es completamente transparente para la capa de red en ambas computadoras, también lo es el relleno de bits. Si los datos de usuario contienen el patrón indicador, 01111110, esta bandera se transmite como 011111010, pero se almacena en la memoria del receptor como 01111110. En la figura 3-5 se da un ejemplo de relleno de bits.

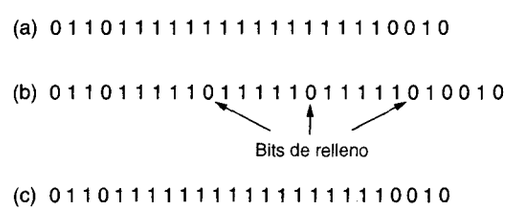


Figura 3-5. Relleno de bits. (a) Los datos originales. (b) Los datos, según aparecen en la línea. (c) Los datos, como se guardan en la memoria del receptor tras remover el relleno.

Con el relleno de bits, el límite entre los dos marcos puede ser reconocido sin ambigüedades mediante el patrón indicador. Si el receptor pierde la pista de dónde está, todo lo que tiene que hacer es explorar la entrada en busca de secuencias indicadoras, pues sólo pueden ocurrir en los límites de los marcos y nunca en los datos.

El último método de enmarcado sólo se aplica a las redes en las que la codificación en el medio físico contiene cierta redundancia. Por ejemplo, algunas LAN codifican un bit de datos

usando 2 bits físicos. Normalmente, un bit 1 es un par alto-bajo, y un bit 0 es un par bajo-alto. Las combinaciones alto-alto y bajo-bajo no se usan para datos. El esquema implica que cada bit de datos tiene una transición a medio camino, lo que hace fácil para el receptor localizar los límites de los bits. Este uso de los códigos físicos no válidos es parte del estándar LAN 802 que estudiaremos en el capítulo 4.

Como nota final sobre el enmarcado, muchos protocolos de enlace de datos usan, por seguridad, una combinación de cuenta de caracteres con uno de los otros métodos. Cuando llega un marco, se usa el campo de cuenta para localizar el final del marco. Sólo si está presente el delimitador apropiado en esa posición y la suma de comprobación es correcta, se acepta como válido el marco. De otra manera, se explora la cadena entrada en búsqueda del siguiente delimitador.

3.1.3. Control de errores

Habiendo resuelto el problema de marcar el inicio y el final de cada marco, llegamos al siguiente problema: cómo asegurar que todos los marcos sean entregados finalmente a la capa de red en el destino, en el orden apropiado. Suponga que el transmisor se dedicó a enviar marcos sin importarle si estaban llegando adecuadamente. Esto podría estar bien para un servicio sin acuse sin conexión, pero ciertamente no será correcto para un servicio confiable orientado a la conexión.

La manera normal de asegurar la entrega confiable de datos es proporcionar al transmisor realimentación sobre lo que está ocurriendo en el otro lado de la línea. Típicamente, el protocolo exige que el transmisor envíe de regreso marcos de control especiales que contengan acuses positivos o negativos de los marcos de entrada. Si el transmisor recibe un acuse positivo de un marco, sabe que el marco llegó correctamente. Por otra parte, un acuse negativo significa que algo falló y el marco debe transmitirse otra vez.

Una complicación adicional surge de la posibilidad de que problemas de *hardware* pueden causar la desaparición de un marco completo (por ejemplo, por una ráfaga de ruido). En este caso, el receptor no reaccionará en absoluto, ya que no tiene razón para reaccionar. Debe quedar claro que un protocolo en el cual el transmisor envía un marco y luego espera un acuse, positivo o negativo, se quedaría esperando eternamente si se pierde por completo un marco debido a una falla de *hardware*.

Esta posibilidad se maneja introduciendo temporizadores en la capa de enlace de datos. Cuando el transmisor envía un marco, generalmente también arranca un temporizador. El temporizador se ajusta de modo que termine cuando haya transcurrido un intervalo suficiente para que el marco llegue a su destino, se procese ahí y el acuse se propague de regreso al transmisor. Normalmente, el marco se recibirá correctamente y el acuse llegará antes de que el temporizador termine, en cuyo caso se cancelará.

Sin embargo, si el marco o el acuse se pierden, el temporizador terminará, alertando al transmisor sobre un problema potencial. La solución obvia es simplemente transmitir de nuevo el marco. Sin embargo, aun cuando los marcos pueden transmitirse muchas veces, hay el peligro de que el receptor acepte el mismo marco dos o más veces y que lo pase a la capa de red más de una

vez. Para evitar que ocurra esto, generalmente es necesario asignar números de secuencia a los marcos de salida, para que el receptor pueda distinguir las retransmisiones de los originales.

El asunto de la administración de temporizadores y números de secuencia para asegurar que cada marco llegue finalmente a la capa de red en el destino una sola vez, ni más ni menos, es una parte importante de las tareas de la capa de enlace de datos. Posteriormente en este capítulo estudiaremos la manera en que se lleva a cabo esta administración, observando una serie de ejemplos de complejidad creciente.

3.1.4. Control de flujo

Otro importante tema de diseño que se presenta en la capa de enlace de datos (y también en las capas superiores) es qué hacer con un transmisor que sistemáticamente quiere transmitir marcos a mayor velocidad que aquella con que puede aceptarlos el receptor. Esta situación puede ocurrir fácilmente cuando el transmisor opera en una computadora rápida (o con baja carga) y el receptor opera en una máquina lenta (o sobrecarga). El transmisor envía los marcos a alta velocidad hasta que satura por completo al receptor. Aun si la transmisión está libre de errores, en cierto punto el receptor simplemente no será capaz de manejar los marcos según van llegando y comenzará a perder algunos. Es obvio que algo tiene que hacerse para evitar esta situación.

La solución común es introducir un **control de flujo** para controlar la velocidad del transmisor de modo que no envíe a mayor velocidad que la que puede manejar el receptor. Este control de velocidad generalmente requiere algún mecanismo de realimentación, para que el transmisor pueda enterarse si el receptor es capaz de mantener el ritmo o no.

Se conocen varios esquemas de control de flujo, pero la mayoría se basa en el mismo principio. El protocolo contiene reglas bien definidas respecto al momento en que un transmisor puede enviar el siguiente marco. Estas reglas con frecuencia prohíben el envío de marcos hasta que el receptor lo haya autorizado, implícita o explícitamente. Por ejemplo, cuando se establece una conexión, el receptor podría decir: "Puedes enviarme *n* marcos ahora, pero tras transmitirlos, no envíes más hasta que te haya indicado que continúes". En este capítulo estudiaremos varios mecanismos de control de flujo basados en este principio. En los capítulos siguientes estudiaremos otros mecanismos.

3.2. DETECCIÓN Y CORRECCIÓN DE ERRORES

Como vimos en el capítulo 2, el sistema telefónico tiene tres partes: los conmutadores, las líneas de interconexión entre oficinas y los lazos locales. Los primeros dos son ahora casi enteramente digitales en Estados Unidos y en algunos otros países. Los lazos locales aún son pares trenzados de cobre analógicos en todos lados y continuarán siéndolo así durante décadas debido al enorme costo de su reemplazo. Aunque los errores son raros en la parte digital, aún son comunes en los lazos locales. Es más, la comunicación inalámbrica se está volviendo más común, y las tasas

de errores aquí son de magnitud mucho mayor que en las líneas de interconexión de fibra entre oficinas. La conclusión es: los errores de transmisión van a ser inevitables durante muchos años más.

Como resultado de los procesos físicos que los generan, los errores en algunos medios (por ejemplo, la radio) tienden a aparecer en ráfagas, no individualmente. El que los errores lleguen en ráfaga tiene tanto ventajas como desventajas respecto a los errores aislados de un solo bit. Por el lado de las ventajas, los datos de computadora siempre se envían en bloques de bits. Suponga que el tamaño de bloque es de 1000 bits y la tasa de errores es de 0.001 por bit. Si los errores fueran independientes, la mayor parte de los bloques contendría un error. Sin embargo, si los errores llegan en ráfagas de 100, en promedio sólo uno o dos bloques de cada 100 serán afectados. La desventaja de los errores en ráfaga es que son mucho más difíciles de detectar y corregir que los errores aislados.

3.2.1. Códigos de corrección de errores

Los diseñadores de redes han desarrollado dos estrategias básicas para manejar los errores. Una es incluir suficiente información redundante en cada bloque de datos transmitido para que el receptor pueda deducir lo que debió ser el carácter transmitido. La otra estrategia es incluir sólo suficiente redundancia para que el receptor sepa que ha ocurrido un error (pero no qué error) y entonces solicite una retransmisión. La primera estrategia usa **códigos de corrección de errores**; la segunda usa **códigos de detección de errores**.

Para entender la manera en que pueden manejarse los errores es necesario estudiar de cerca lo que es en realidad un error. Normalmente, un marco consiste en m bits de datos (es decir, de mensaje) y r bits redundantes o de comprobación. Sea la longitud total n (es decir, $n = m + r$). A una unidad de n bits que contiene datos y bits de comprobación se le conoce como **palabra código** de n bits.

Dadas dos palabras código cualesquiera, digamos 10001001 y 10110001, es posible determinar cuántos bits correspondientes difieren. En este caso, difieren tres bits. Para determinar la cantidad de bits diferentes basta aplicar una operación OR EXCLUSIVO a las dos palabras código y contar la cantidad de bits 1 en el resultado. La cantidad de posiciones de bit en la que difieren dos palabras código se llama **distancia de Hamming** (Hamming, 1950). Su significado es que, si dos palabras código están separadas una distancia de Hamming d, se requerirán d errores de un bit para convertir una en la otra.

En la mayoría de las aplicaciones de transmisión de datos, todos los 2^m mensajes de datos posibles son legales, pero debido a la manera en que se calculan los bits de comprobación no se usan todas las 2^n palabras código posibles. Dado el algoritmo de cálculo de los bits de comprobación, es posible construir una lista completa de palabras código legales y, en esta lista, encontrar las dos palabras código cuya distancia de Hamming es mínima. Esta distancia es la distancia de Hamming de todo el código.

Las propiedades de detección y corrección de errores de un código dependen de su distancia de Hamming. Para detectar d errores se necesita un código con distancia $d + 1$, pues con tal

código no hay manera de que d errores de un bit puedan cambiar una palabra código válida a otra. Cuando el receptor ve una palabra código no válida, sabe que ha ocurrido un error de transmisión. De manera parecida, para corregir d errores se necesita un código de distancia $2d + 1$, pues así las palabras código legales están tan separadas que, aun con d cambios, la palabra código original sigue estando más cercana que cualquier otra palabra código, por lo que puede determinarse de manera única.

Como ejemplo sencillo de código de detección de errores, considere un código en el que se agrega un solo **bit de paridad** a los datos. El bit de paridad se escoge de manera que la cantidad de bits 1 en la palabra código sea par (o impar). Por ejemplo, cuando se envía 10110101 con paridad par añadiendo un bit al final, se vuelve 101101011, pero 10110001 se vuelve 101100010 con paridad par. Un código con un solo bit de paridad tiene una distancia de 2, pues cualquier error de un bit produce una palabra código con la paridad equivocada. Este sistema puede usarse para detectar errores individuales.

Como ejemplo sencillo de código de corrección de errores, considere un código con sólo cuatro palabras código válidas:

0000000000, 0000011111, 1111100000 y 1111111111

Este código tiene una distancia de 5, lo que significa que puede corregir errores dobles. Si llega la palabra código 0000000111, el receptor sabe que el original debió ser 0000011111. Sin embargo, si un triple error cambia 0000000000 a 0000000111, el error no se corregirá adecuadamente.

Imagine que queremos diseñar un código con m bits de mensaje y r bits de comprobación que permitirá la corrección de todos los errores individuales. Cada uno de los 2^m mensajes legales tiene n palabras código ilegales a una distancia 1 de él. Éstas se forman invirtiendo sistemáticamente cada uno de los n bits de la palabra código de n bits formada a partir de él. Por tanto, cada uno de los 2^m mensajes legales requiere $n + 1$ patrones de bits dedicados a él. Dado que la cantidad de patrones de bits es 2^n, debemos tener $(n + 1)2^m \leq 2^n$. Usando $n = m + r$, este requisito se vuelve $(m + r + 1) \leq 2^r$. Dado m, esto impone un límite inferior a la cantidad de bits de comprobación necesarios para corregir errores individuales.

De hecho, este límite inferior teórico puede lograrse usando un método debido a Hamming (1950). Los bits de la palabra código se numeran consecutivamente, comenzando por el bit 1 a la izquierda. Los bits que son potencias de 2 (1, 2, 4, 8, 16, etc.) son bits de comprobación. El resto (3, 5, 6, 7, 9, etc.) se rellenan con los m bits de datos. Cada bit de comprobación obliga a que la paridad de un grupo de bits, incluyéndolo a él mismo, sea par (o impar). Un bit puede estar incluido en varios cálculos de paridad. Para ver a qué bits de comprobación contribuye el bit de datos en la posición k, reescriba k como una suma de potencias de 2. Por ejemplo, $11 = 1 + 2 + 8$ y $29 = 1 + 4 + 8 + 16$. Se comprueba un bit solamente por los bits de comprobación que ocurren en su expansión (por ejemplo, el bit 11 es comprobado por los bits 1, 2 y 8).

Al llegar una palabra código, el receptor inicializa a cero un contador y luego examina cada bit de comprobación, k ($k = 1, 2, 4, 8, \ldots$) para ver si tiene la paridad correcta. Si no, suma k al contador. Si el contador es igual a cero tras haber examinado todos los bits de comprobación (es

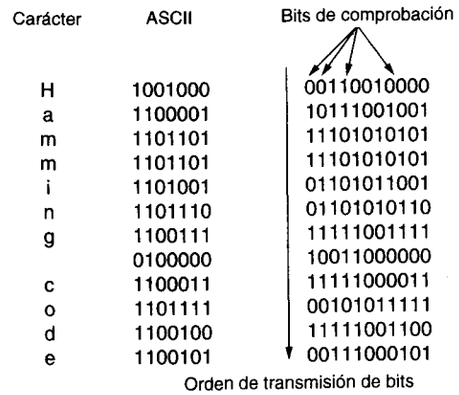


Figura 3-6. Uso de un código de Hamming para corregir errores en ráfaga.

decir, si todos fueron correctos), la palabra código se acepta como válida. Si el contador es diferente de cero, contiene el número del bit incorrecto. Por ejemplo, si los bits de comprobación 1, 2 y 8 tienen errores, el bit invertido es el 11, pues es el único comprobado por los bits 1, 2 y 8. En la figura 3-6 se muestran algunos caracteres ASCII de 7 bits codificados como palabras código de 11 bits usando el código de Hamming. Recuerde que los datos se encuentran en las posiciones de bit 3, 5, 6, 7, 9, 10 y 11.

Los códigos de Hamming sólo pueden corregir errores individuales. Sin embargo, hay un truco que puede servir para que los códigos de Hamming corrijan errores en ráfaga. Se dispone como matriz una secuencia de k palabras código consecutivas, con una palabra código por fila. Normalmente se transmitiría una palabra código a la vez, de izquierda a derecha. Para corregir los errores en ráfaga, los datos deben transmitirse una columna a la vez, comenzando por la columna de la extrema izquierda. Cuando todos los bits k han sido enviados, se envían la segunda columna, y así sucesivamente. Cuando el marco llega al receptor, la matriz se reconstruye, una columna a la vez. Si ocurre un error de longitud k, cuando mucho se habrá afectado 1 bit de cada una de las k palabras código; sin embargo, el código de Hamming puede corregir un error por palabra código, así que puede restaurarse la totalidad del bloque. Este método usa kr bits de comprobación para inmunizar bloques de km bits de datos contra una sola ráfaga de errores de longitud k o menos.

3.2.2. Códigos de detección de errores

Los códigos de corrección de errores a veces se utilizan para la transmisión de datos; por ejemplo, cuando el canal es símplex, por lo que no pueden solicitarse retransmisiones. Sin embargo, con mayor frecuencia se prefiere la detección de errores seguida de la retransmisión porque es más eficiente. Como ejemplo simple, considere un canal en el que los errores son aislados y la tasa de errores es de 10^{-6} por bit. Sea el tamaño de bloque 1000 bits. Para proporcionar corrección

de errores en bloques de 1000 bits se requieren 10 bits de comprobación; un megabit de datos requerirá 10,000 bits de comprobación. Para detectar un solo bloque con 1 bit de error, basta con un bit de paridad por bloque. Por cada 1000 bloques se tendrá que transmitir un bloque extra (1001 bits). El gasto extra del método de detección de errores + retransmisión es de sólo 2001 bits por megabit de datos, contra 10,000 bits con un código de Hamming.

Si se agrega un solo bit de paridad a un bloque y el bloque viene muy alterado por una ráfaga de errores prolongada, la probabilidad de que se detecte el error es de 0.5, lo que difícilmente es aceptable. Puede mejorarse mucho la probabilidad considerando a cada bloque por enviar como una matriz rectangular de n bits de ancho y k bits de alto. Se calcula por separado un bit de paridad para cada columna y se agrega a la matriz como última fila. La matriz se transmite entonces fila por fila. Al llegar el bloque, el receptor comprueba todos los bits de paridad. Si cualquiera de ellos está mal, solicita la retransmisión del bloque.

Este método puede detectar una sola ráfaga de duración n, pues sólo se cambiará un bit por columna. Sin embargo, una ráfaga de duración $n + 1$ pasará sin ser detectada si se invierte el primer bit, el último bit y todos los demás bits están correctos. (Una ráfaga de errores no implica que todos los bits están mal; sólo implica que cuando menos el primero y el último están mal.) Si el bloque está muy alterado por una ráfaga continua o por múltiples ráfagas más cortas, la probabilidad de que cualquiera de las n columnas tenga, por accidente, la paridad correcta es de 0.5, por lo que la probabilidad de aceptar un bloque alterado cuando no se debe es de 2^{-n}.

Aunque el esquema anterior puede ser adecuado en algunos casos, en la práctica se usa otro método muy difundido: el **código polinómico** (también conocido como **código de redundancia cíclica** o código CRC). Los códigos polinómicos se basan en el tratamiento de cadenas de bits como representaciones de polinomios con coeficientes de 0 y 1 solamente. Un marco de k bits se considera como la lista de coeficientes de un polinomio con k que van de x^{k-1} a x^0. Se dice que tal polinomio es de grado $k - 1$. El bit mayor (más izquierdo) es el coeficiente de x^{k-1}, el siguiente bit es el coeficiente de x^{k-2} y así sucesivamente. Por ejemplo, 110001 tiene 6 bits y por tanto representa un polinomio de seis términos con coeficientes 1, 1, 0, 0, 0 y 1: $x^5 + x^4 + x^0$.

La aritmética polinómica se hace módulo 2, de acuerdo con las reglas de la teoría de campos algebraicos. No hay acarreos para la suma, ni préstamos para la resta. Tanto la suma como la resta son idénticas a un OR EXCLUSIVO. Por ejemplo:

$$\begin{array}{rrrr} 10011011 & 00110011 & 11110000 & 01010101 \\ +\ 11001010 & +\ 11001101 & -\ 10100110 & -\ 10101111 \\ \hline 01010001 & 11111110 & 01010110 & 11111010 \end{array}$$

La división se lleva a cabo de la misma manera que en binario, excepto que la resta se hace módulo 2, igual que antes. Se dice que un divisor "cabe" en un dividendo si el dividendo tiene tantos bits como el divisor.

Cuando se emplea el método de código polinómico, el transmisor y el receptor deben acordar un **polinomio generador**, $G(x)$, por adelantado. Tanto los bits mayor como menor del generador deben ser 1. Para calcular la **suma de comprobación** para un marco con m bits,

correspondiente al polinomio $M(x)$, el marco debe ser más largo que el polinomio generador. La idea es anexar una suma de comprobación al final del marco de tal manera que el polinomio representado por el marco más la suma de comprobación sea divisible entre $G(x)$. Cuando el receptor recibe el marco con suma de comprobación, intenta dividirlo entre $G(x)$. Si hay un residuo, ha habido un error de transmisión.

El algoritmo para calcular la suma de comprobación es el siguiente:

1. Sea r el grado de $G(x)$. Anexe r bits cero al final del marco, para que ahora contenga $m + r$ bits y corresponda al polinomio $x^rM(x)$.
2. Divida la cadena de bits correspondiente a $x^rM(x)$ entre la correspondiente a $G(x)$ usando división módulo 2.
3. Reste el residuo (que siempre es de r o menos bits) a la cadena de bits correspondiente a $x^rM(x)$ usando resta módulo 2. El resultado es el marco con suma de comprobación a transmitirse. Llame a su polinomio $T(x)$.

En la figura 3-7 se ilustra el cálculo para un marco 1101011011 y $G(x) = x^4 + x + 1$.

Debe quedar claro que $T(x)$ es divisible (módulo 2) entre $G(x)$. En cualquier problema de división, si se resta el residuo del dividendo lo que queda es divisible entre el divisor. Por ejemplo, en base 10, si se divide 210,278 entre 10,941, el residuo es 2399. Si restamos 2399 a 210,278, lo que queda (207,879) es divisible entre 10,941.

Ahora analizaremos el alcance de este método. ¿Qué tipos de error se detectarán? Imagine que ocurre un error de transmisión tal que, en lugar de que llegue la cadena de bits para $T(x)$, llega $T(x) + E(x)$. Cada bit 1 en $E(x)$ corresponde a un bit que ha sido invertido. Si hay k bits 1 en $E(x)$, han ocurrido k errores de bit individuales. Una ráfaga de errores individual se caracteriza por un 1 inicial, una mezcla de ceros y unos, y un 1 final, siendo los demás bits 0.

Al recibir el marco con suma de comprobación, el receptor lo divide entre $G(x)$; es decir, calcula $[T(x) + E(x)]/G(x)$. $T(x)/G(x)$ es 0, por lo que el resultado del cálculo es simplemente $E(x)/G(x)$. Aquellos errores que casualmente correspondan a polinomios que contengan $G(x)$ como factor no serán detectados; todos los demás errores serán atrapados.

Si ha habido un error de un solo bit, $E(x) = x^i$, donde i determina qué bit está en error. Si $G(x)$ contenía dos o más términos, nunca será divisor exacto de $E(x)$, por lo que se detectarán los errores de un solo bit.

Si han ocurrido dos errores de un solo bit aislados, $E(x) = x^i + x^j$, donde $i > j$. Esto también se puede escribir como $E(x) = x^j(x^{i-j} + 1)$. Si suponemos que $G(x)$ no es divisible entre x, una condición suficiente para detectar todos los errores dobles es que $G(x)$ no sea divisor exacto de $x^k + 1$ para ninguna k hasta el valor máximo de $i - j$ (es decir, hasta la longitud máxima del marco). Se conocen polinomios sencillos de bajo grado que dan protección a marcos largos. Por ejemplo, $x^{15} + x^{14} + 1$ no dividirá exactamente a $x^k + 1$ para ningún valor de k menor que 32,768.

Si hay una cantidad impar de bits con error, $E(X)$ contiene un número impar de términos (por ejemplo, $x^5 + x^2 + 1$, pero no $x^2 + 1$). Curiosamente, no hay un polinomio con un número impar

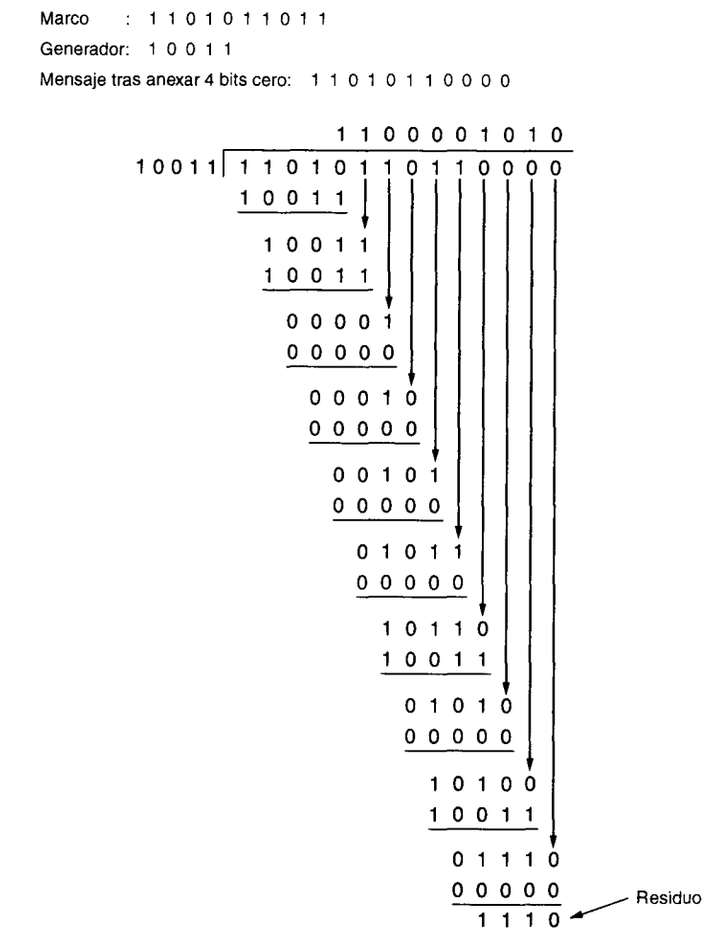


Figura 3-7. Cálculo de la suma de comprobación de código polinómico.

de términos que tenga $x + 1$ como factor en el sistema módulo 2. Haciendo $x + 1$ un factor de $G(x)$, podemos atrapar todos los errores consistentes en un número impar de bits invertidos.

Para comprobar que ningún polinomio con una cantidad impar de términos es divisible entre $x + 1$, suponga que $E(x)$ tiene un número impar de términos y que es divisible entre $x + 1$. Factorice $E(x)$ en $(x + 1)\,Q(x)$. Ahora evalúe $E(1) = (1 + 1)Q(1)$. Dado que $1 + 1 = 0$ (módulo 2), $E(1)$ debe ser cero. Si $E(x)$ tiene un número impar de términos, la sustitución de 1 por x en cualquier lugar siempre arrojará como resultado un 1. Por tanto, ningún polinomio con un número impar de términos es divisible entre $x + 1$.

Por último, y lo que es más importante, un código polinómico con r bits de comprobación detectará todos los errores en ráfaga de longitud $\leq r$. Un error en ráfaga de longitud k puede representarse mediante $x^i(x^{k-1} + \ldots + 1)$, donde i determina la distancia a la que se encuentra la ráfaga desde el extremo derecho del marco recibido. Si $G(x)$ contiene un término x^0, no tendrá x^i como factor, por lo que, si el grado de la expresión entre paréntesis es menor que el grado de $G(x)$, el residuo nunca puede ser cero.

Si la longitud de la ráfaga es de $r + 1$, el residuo de la división entre $G(x)$ será cero si, y sólo si, la ráfaga es idéntica a $G(x)$. Por la definición de ráfaga, el primer y el último bits deben ser 1, así que el que sean iguales o no depende de los $r - 1$ bits intermedios. Si se consideran igualmente probables todas las combinaciones, la probabilidad de que se acepte como válido tal marco incorrecto es de $1/2^{r-1}$.

También puede demostrarse que, cuando ocurre una ráfaga de errores mayor que $r + 1$, o cuando ocurren varias ráfagas más cortas, la probabilidad de que un marco incorrecto no sea detectado es de $1/2^r$, suponiendo que todos los patrones de bits sean igualmente probables.

Tres polinomios se han vuelto estándares internacionales:

CRC-12 $= x^{12} + x^{11} + x^3 + x^2 + x^1 + 1$
CRC-16 $= x^{16} + x^{15} + x^2 + 1$
CRC-CCITT $= x^{16} + x^{12} + x^5 + 1$

Los tres contienen $x + 1$ como factor primo. CRC-12 se usa cuando la longitud de un carácter es de 6 bits. Los otros dos se usan con caracteres de 8 bits. Una suma de comprobación de 16 bits, como CRC-16 o CRC-CCITT, detecta todos los errores sencillos y dobles, todos los errores con un número impar de bits, todos los errores en ráfaga de longitud 16 o menos, 99.997% de las ráfagas de errores de 17 bits y 99.998% de las ráfagas de 18 bits o mayores.

Aunque el cálculo requerido para obtener la suma de comprobación puede parecer complicado, Peterson y Brown (1961) han demostrado que puede construirse un circuito sencillo de registro de desplazamiento para calcular y verificar las sumas de comprobación por *hardware*. En la práctica, casi siempre se usa este *hardware*.

Durante décadas se ha supuesto que los marcos para los que se generan sumas de comprobación contienen bits aleatorios. Todos los análisis de algoritmos de suma de comprobación se han hecho bajo este supuesto. En fechas más recientes, la inspección de datos reales ha mostrado que este supuesto es equivocado. Como consecuencia, en algunas circunstancias los errores no detectados son mucho más comunes de lo que se pensaba anteriormente (Partridge *et al.*, 1995).

3.3. PROTOCOLOS ELEMENTALES DE ENLACE DE DATOS

Como introducción al tema de los protocolos, comenzaremos por estudiar tres protocolos de complejidad creciente. Para los lectores interesados, se puede conseguir un simulador de estos protocolos y otros subsecuentes a través de la WWW (véase el prefacio). Antes de estudiar los

protocolos, es útil hacer explícitos algunos de los supuestos implícitos del modelo de comunicaciones. Para comenzar, estamos suponiendo que en las capas física, de enlace de datos y de red hay procesos independientes que se comunican pasando mensajes de un lado a otro. En algunos casos, los procesos de la capa física y de la de enlace de datos se ejecutan en un procesador dentro de un chip especial de E/S y los de la capa de red lo hacen en la CPU principal, pero también puede haber otras implementaciones (por ejemplo, tres procesos en un solo chip de E/S; las capas física y de enlace de datos como procedimientos invocados por el proceso de la capa de red, y otros). En cualquier caso, el tratar las tres capas como procesos independientes hace más nítido conceptualmente el análisis y también sirve para subrayar la independencia de las capas.

Otro supuesto clave es que la máquina A quiere mandar una cadena larga de datos a la máquina B usando un servicio confiable orientado a conexiones. Después consideraremos el caso en que B también quiere mandar datos a A simultáneamente. Se ha supuesto que A tiene un suministro infinito de datos listos para ser enviados y nunca tiene que esperar a que se produzcan datos. Cuando la capa de enlace de datos de A solicita datos, la capa de red siempre es capaz de proporcionarlos de inmediato. (Esta restricción también será desechada posteriormente.)

En lo que concierne a la capa de enlace de datos, el paquete pasado a ella a través de la interfaz desde la capa de red es de puros datos, que deben ser entregados bit por bit a la capa de red del destino. El hecho de que la capa de red del destino pueda interpretar parte del paquete como un encabezado no es de importancia para la capa de enlace de datos.

Cuando la capa de enlace de datos acepta un paquete, lo encapsula en un marco agregando un encabezado y una cola de enlace de datos (véase la figura 1-11). Por tanto, un marco consiste en un paquete incorporado más cierta información de control (encabezado). Luego el marco se transmite a la otra capa de enlace de datos. Supondremos que existen procedimientos de biblioteca *to_physical_layer* (a la capa física) para enviar un marco y *from_physical_layer* (de la capa física) para recibir un marco. El *hardware* transmisor calcula y agrega la suma de comprobación, por lo que el *software* de la capa de enlace de datos no necesita preocuparse por ella. Podría usarse, por ejemplo, el algoritmo polinómico analizado antes en este capítulo.

Inicialmente el receptor no tiene nada que hacer. Sólo está esperando que ocurra algo. En los protocolos ejemplo de este capítulo indicamos que la capa de enlace de datos está esperando que algo ocurra con la llamada de procedimiento *wait_for_event(&event)* (esperar evento). Este procedimiento sólo regresa cuando algo ha ocurrido (por ejemplo, ha llegado un marco). Al regresar, la variable *event* indica lo que ha ocurrido. El grupo de eventos posibles difiere para cada uno de los diferentes protocolos que describiremos, y se definirán por separado para cada protocolo. Note que, en una situación más realista, la capa de enlace de datos no se quedará en un ciclo cerrado esperando un evento, como hemos sugerido, sino que recibirá una interrupción, la que hará que suspenda lo que estaba haciendo y proceda a manejar el marco entrante. Sin embargo, por sencillez ignoraremos todos los detalles de la actividad paralela en la capa de enlace de datos y supondremos que la capa está dedicada de tiempo completo a manejar nuestro canal.

Cuando llega un marco al receptor, el *hardware* calcula la suma de comprobación. Si la suma de comprobación es incorrecta (es decir, si hubo un error de transmisión), la capa de enlace de

datos es informada (*event* = *cksum_err*). Si el marco entrante llega sin daño, la capa de enlace de datos también es informada (*event* = *frame_arrival*) para que pueda adquirir el marco e inspeccionarlo usando *from_physical_layer*. Tan pronto como la capa de enlace de datos receptora adquiere un marco sin daños, revisa la información de control del encabezado y, si todo está bien, pasa la parte que corresponde al paquete a la capa de red. En ninguna circunstancia se entrega un encabezado de marco a la capa de red.

Hay una buena razón por la que la capa de red nunca debe recibir ninguna parte de el encabezado del marco: mantener completamente separados el protocolo de red y el de enlace de datos. Mientras la capa de red no sepa nada en absoluto sobre el protocolo de enlace de datos ni el formato del marco, éstos podrán cambiarse sin requerir cambios en el *software* de la capa de red. Al proporcionarse una interfaz rígida entre la capa de red y la de enlace de datos se simplifica en gran medida el diseño del *software*, pues los protocolos de comunicación de las diferentes capas pueden evolucionar independientemente.

En la figura 3-8 se muestran algunas declaraciones comunes (en C) para muchos de los protocolos que se analizarán después. Allí se definen cinco estructuras de datos: *boolean, seq_nr, packet, frame_kind* y *frame*. Un *boolean* (boleano) es un tipo enumerado que puede tener los valores *true* y *false* (verdadero y falso). Un *seq_nr* (número de secuencia) es un entero pequeño que sirve para numerar los marcos, a fin de distinguirlos. Estos números de secuencia van de 0 hasta *MAX_SEQ* (inclusive), que se define en cada protocolo que lo necesita. Un *packet* (paquete) es la unidad de información intercambiada entre la capa de red y la de enlace de datos en la misma máquina, o entre capas de red pares. En nuestro modelo siempre contiene *MAX_PKT* bytes, pero en la práctica sería de longitud variable.

Un *frame* (marco) está compuesto de cuatro campos: *kind, seq, ack* e *info*. Los primeros tres contienen información de control y el último puede contener los datos por transferir. Estos campos de control constituyen en conjunto el **encabezado del marco**. El campo *kind* (tipo) indica si hay datos en el marco, porque algunos de los protocolos distinguen entre los marcos que contienen exclusivamente información de control y los que contienen también datos. Los campos *seq* y *ack* se emplean para números de secuencia y acuses de recibo, respectivamente; su uso se describirá posteriormente con mayor detalle. El campo *info* de un marco de datos contiene un solo paquete; el campo *info* de un marco de control no se usa. En una implementación más realista se usaría un campo de *info* de longitud variable, omitiéndolo por completo en los marcos de control.

Es importante entender la relación entre paquete y marco. La capa de red construye un paquete tomando un mensaje de la capa de transporte y agregándole el encabezado de la capa de red. Este paquete se pasa a la capa de enlace de datos para su inclusión en el campo *info* de un marco de salida. Cuando el marco llega a su destino, la capa de enlace de datos extrae el paquete del marco y lo pasa a la capa de red. De esta manera, la capa de red puede actuar como si las máquinas pudieran intercambiar paquetes directamente.

En la figura 3-8 también se listan varios procedimientos que son rutinas de biblioteca cuyos detalles dependen de la implementación, por lo que no nos ocuparemos de su funcionamiento interno aquí. El procedimiento *wait_for_event* se queda en un ciclo cerrado esperando que algo

ocurra, como se mencionó antes. Con los procedimientos *to_network_layer* y *from_network_layer* la capa de enlace de datos pasa paquetes a la capa de red y acepta paquetes de ella, respectivamente. Note que *from_physical_layer* y *to_physical_layer* sirven para pasar marcos entre la capa de enlace de datos y la capa física, y que los procedimientos *to_network_layer* y *from_network_layer* se usan para pasar paquetes entre la capa de enlace de datos y la capa de red. En otras palabras, *to_network_layer* y *from_network_layer* tienen que ver con la interfaz entre las capas 2 y 3, y *from_physical_layer* y *to_physical_layer*, con la interfaz entre las capas 1 y 2.

En la mayoría de los protocolos suponemos un canal inestable que pierde marcos completos ocasionalmente. Para poder recuperarse de tales calamidades, la capa de enlace de datos transmisora debe arrancar un temporizador o reloj interno cada vez que envía un marco. Si no obtiene respuesta tras transcurrir cierto intervalo de tiempo predeterminado, el temporizador termina y la capa de enlace de datos recibe una señal de interrupción.

En nuestros protocolos, esto se maneja permitiendo que el procedimiento *wait_for_event* devuelva *event* = *timeout*. Los procedimientos *start_timer* y *stop_timer* sirven para arrancar y parar, respectivamente, el temporizador. Las terminaciones por temporizador sólo son posibles cuando el temporizador está funcionando. Se permite explícitamente llamar a *start_timer* cuando el temporizador está funcionando; tal llamada simplemente restablece el reloj para hacer que el temporizador termine después de haber transcurrido un intervalo completo de temporización (a menos que se restablezca o apague antes).

Los procedimientos *start_ack_timer* y *stop_ack_timer* sirven para controlar un temporizador auxiliar usado para generar acuses en ciertas condiciones.

Los procedimientos *enable_network_layer* (habilitar capa de red) y *disable_network_layer* (inhabilitar capa de red) se usan en los protocolos más complicados, en los que ya no suponemos que la capa de red siempre tiene paquetes que enviar. Cuando la capa de enlace de datos habilita a la capa de red, se permite que éste interrumpa cuando tenga que enviar un paquete. Esto lo indicamos con *event* = *network_layer_ready*. Cuando una capa de red está inhabilitada, no puede causar tales eventos. Teniendo cuidado respecto a cuándo habilita y deshabilita su capa de red, la capa de enlace de datos puede evitar que la capa de red la sature con paquetes para los que no tiene espacio de *buffer* (memoria intermedia).

Los números de secuencia de los marcos siempre están en el intervalo de 0 a *MAX_SEQ* (inclusive), donde *MAX_SEQ* es diferente para los distintos protocolos. Con frecuencia es necesario avanzar un número de secuencia en 1 circularmente (por ejemplo, *MAX_SEQ* va seguido de 0). La macro *inc* lleva a cabo este incremento. Esta función ha sido definida como una macro porque se usa en línea dentro de la trayectoria crítica. Como veremos después en este libro, el factor que limita el desempeño de una red con frecuencia es el procesamiento del protocolo, por lo que la definición como macros de operaciones sencillas como ésta no afecta la claridad de lectura del código y sí mejora el desempeño. También, ya que *MAX_SEQ* tendrá diferentes valores en diferentes protocolos, al hacer que *inc* sea una macro cabe la posibilidad de incluir todos los protocolos en el mismo binario sin conflictos. Esta capacidad es útil para el simulador.

Las declaraciones de la figura 3-8 son parte de todos los protocolos que siguen. Para ahorrar espacio y proporcionar una referencia práctica, se han extraído y listado juntas, pero conceptualmente

```
#define MAX_PKT 1024                    /* determina el tamaño del paquete en bytes */

typedef enum {false, true} boolean;     /* tipo boleano */
typedef unsigned int seq_nr;            /* números de secuencia o reconocimiento */
typedef struct {unsigned char data[MAX_PKT];} paquete;   /* definición de paquete */
typedef enum {data, ack, nak} frame_kind;                /* definición frame_kind */

typedef struct {                        /* los marcos se transportan en esta capa */
  frame_kind kind;                      /* ¿qué clase de marco es? */
  seq_nr seq;                           /* número de secuencia */
  seq_nr ack;                           /* número de acuse */
  packet info;                          /* paquete de la capa de red */
} frame;

/* Espera que ocurra un evento; devuelve el tipo de evento. */
void wait_for_event(event_type *event);

/* Obtiene un paquete de la capa de red para transmitirlo por el canal. */
void from_network_layer(packet *p);

/* Entrega información de un marco de entrada a la capa de red. */
void to_network_layer(packet *p);

/* Obtiene un marco de entrada de la capa física y lo copia en r. */
void from_physical_layer(frame *r);

/* Pasa el marco a la capa física para su transmisión. */
void to_physical_layer(frame *s);

/* Arranca el reloj y habilita el evento de terminación de temporización. */
void start_timer(seq_nr k);

/* Detiene el reloj e inhabilita el evento de terminación de la temporización. */
void stop_timer(seq_nr k);

/* Comienza el temporizador auxiliar y habilita el evento ack_timeout. */
void start_ack_timer(void);

/* Detiene el temporizador auxiliar y deshabilita el evento ack_timeout. */
void stop_ack_timer(void);

/* Permite que la capa de red cause un evento network_layer_ready. */
void enable_network_layer(void);

/* Evita que la capa de red cause un evento network_layer_ready. */
void disable_network_layer(void);

/* La macro inc se expande en línea: incrementa circularmente k. */
#define inc(k) if (k < MAX_SEQ) k = k + 1; else k = 0
```

Figura 3-8. Algunas definiciones necesarias en los protocolos que siguen. Estas definiciones se encuentran en el archivo *protocol.h*.

deberían estar integradas con los protocolos mismos. En C, esta integración se efectúa poniendo las definiciones en un archivo especial de encabezado, en este caso *protocol.h*, y usando la directiva #include del preprocesador de C para incluirlas en los archivos de protocolos.

3.3.1. Un protocolo símplex sin restricciones

Como ejemplo inicial consideraremos un protocolo que es lo más sencillo posible. Los datos se transmiten sólo en una dirección; las capas de red tanto del transmisor como del receptor siempre están listas, el tiempo de procesamiento puede ignorarse, hay un espacio infinito de *buffer* y, lo mejor de todo, el canal de comunicación entre las capas de enlace de datos nunca tiene problemas ni pierde marcos. Este protocolo completamente irreal, al que apodaremos "utopía", se muestra en la figura 3-9.

El protocolo consiste en dos procedimientos diferentes, uno transmisor y uno receptor. El transmisor se ejecuta en la capa de enlace de datos de la máquina de origen y el receptor se ejecuta en la capa de enlace de datos de la máquina de destino. No se usan números de secuencia ni acuses, por lo que no se necesita *MAX_SEQ*. El único tipo de evento posible es *frame_arrival*, la llegada de un marco (es decir, la llegada de un marco sin daños).

El transmisor está en un ciclo while infinito que sólo envía datos a la línea tan rápidamente como puede. El cuerpo del ciclo consiste en tres acciones: obtener un paquete de la (siempre dispuesta) capa de red, construir un marco de salida usando la variable *s* y enviar el marco a su destino. Este protocolo sólo utiliza el campo *info* del marco, pues los demás campos tienen que ver con los errores y el control de flujo, y aquí no hay errores ni restricciones de control de flujo.

El receptor es igualmente sencillo. De inicio, espera que algo ocurra, siendo la única posibilidad la llegada de un marco sin daños. En algún momento, el marco llega y el procedimiento *wait_for_event* regresa, conteniendo *event* el valor *frame_arrival* (que de todos modos es ignorado). La llamada a *from_physical_layer* remueve el marco recién llegado del *buffer* de *hardware* y lo pone en la variable *r*. Por último, la parte de datos se pasa a la capa de red y la capa de enlace de datos se retira para esperar el siguiente marco, suspendiéndose efectivamente hasta que llega el marco.

3.3.2. Protocolo símplex de parada y espera

Ahora nos desharemos del supuesto más irreal hecho en el protocolo 1: la capacidad de la capa de red receptora de procesar datos de entrada con una rapidez infinita (o, lo que es equivalente, la presencia en la capa de enlace de datos receptora de una cantidad infinita de espacio de *buffer* en el cual almacenar todos los marcos de entrada mientras esperan su respectivo turno). Todavía se supone que el canal de comunicaciones está libre de errores y que el tráfico de datos es símplex.

El problema principal que debemos resolver aquí es cómo evitar que el transmisor sature al receptor enviando datos a mayor velocidad de la que este último puede procesarlos. En esencia, si el receptor requiere un tiempo Δt para ejecutar *from_physical_layer* más *to_network_layer*, el

```
/* El protocolo 1 (utopía) considera la transmisión de datos en una sola dirección, del
   transmisor al receptor. Se supone que el canal de comunicación está libre de errores,
   y que el receptor es capaz de procesar toda la entrada a una rapidez infinita. En
   consecuencia, el transmisor se mantiene en un ciclo, enviando datos a la línea tan
   rápidamente como puede. */

typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender1(void)
{
  frame s;                          /* buffer para un marco de salida */
  packet buffer;                    /* buffer para un paquete de salida */

  while (true) {
      from_network_layer(&buffer);  /* consigue algo qué enviar */
      s.info = buffer;              /* lo copia en s para transmisión */
      to_physical_layer(&s);        /* lo envía */
  }                                 /* Mañana, y mañana, y mañana,
                                       Se arrastra a este mísero paso de día a día
                                       Hasta la última sílaba del tiempo recordado
                                         - Macbeth, V, v */
}

void receiver1(void)
{
  frame r;
  event_type event;                 /* ocupado por wait, pero no se usa aquí */

  while (true) {
      wait_for_event(&event);       /* la única posibilidad es frame_arrival */
      from_physical_layer(&r);      /* obtiene el marco de entrada */
      to_network_layer(&r.info);    /* pasa los datos a la capa de red */
  }
}
```

Figura 3-9. Protocolo símplex sin restricciones.

transmisor debe transmitir a una tasa media menor que un marco por tiempo Δt. Es más, si suponemos que no hay colocación automática en el *buffer* y encolamiento en el *hardware* del receptor, el transmisor nunca debe transmitir un marco nuevo hasta que el viejo haya sido obtenido por *from_physical_layer*, para que lo nuevo no sobreescriba lo viejo.

En ciertas circunstancias restringidas (por ejemplo, transmisión síncrona y una capa de enlace de datos receptora dedicada por completo a procesar la línea de entrada única), el trans-

misor podría introducir simplemente un retardo en el protocolo 1 y así reducir su velocidad lo suficiente para evitar que se sature el receptor. Sin embargo, es más común que la capa de enlace de datos tenga varias líneas a las cuales atender, y el intervalo de tiempo entre la llegada de un marco y su procesamiento puede variar considerablemente. Si los diseñadores de la red pueden calcular el comportamiento de peor caso del receptor, podrán programar el transmisor para que transmita con tanta lentitud que, aun si cada marco sufre el retardo máximo, no haya rebases. El problema con este enfoque es que es demasiado conservativo. Conduce a un aprovechamiento del ancho de banda muy por debajo del óptimo, a menos que el mejor y el peor caso sean casi iguales (es decir, la variación en el tiempo de reacción de la capa de enlace de datos sea pequeña).

Una solución más general para este dilema es hacer que el receptor proporcione realimentación al transmisor. Tras haber pasado un paquete a su capa de red, el receptor envía un pequeño marco ficticio de regreso al transmisor que, de hecho, autoriza al transmisor para transmitir el siguiente marco. Tras haber enviado un marco, el protocolo exige que el transmisor espere hasta que llegue el pequeño marco ficticio (es decir, el acuse).

Los protocolos en los que el transmisor envía un marco y luego espera un acuse antes de continuar se denominan de **parada y espera**. En la figura 3-10 se da un ejemplo de protocolo símplex de parada y espera.

Como en el protocolo 1, el transmisor comienza obteniendo un paquete de la capa de red, usándolo para construir un marco y enviarlo a su destino. Sólo que ahora, a diferencia del protocolo 1, el transmisor debe esperar hasta que llegue un marco de acuse antes de reiniciar el ciclo y obtener el siguiente paquete de la capa de red. La capa de enlace de datos transmisora no necesita inspeccionar el marco de entrada: sólo hay una posibilidad.

La única diferencia entre *receptor1* y *receptor2* es que, tras entregar un paquete a la capa de red, el *receptor2* envía un marco de acuse de regreso al transmisor antes de entrar nuevamente en el ciclo de espera. Ya que sólo es importante la llegada del marco en el transmisor, no su contenido, el receptor no necesita poner ninguna información específica en él.

Aunque el tráfico de datos en este ejemplo es símplex, yendo sólo del transmisor al receptor, los marcos sí viajan en ambos sentidos. En consecuencia, el canal de comunicación entre las dos capas de enlace de datos necesita ser capaz de transferencia bidireccional de información. Sin embargo, este protocolo implica una alternancia estricta del flujo: primero el transmisor envía un marco, después el receptor envía un marco, luego el transmisor envía otro marco, el receptor envía otro marco, y así sucesivamente. Basta con un canal físico semidúplex.

3.3.3. Protocolo símplex para un canal ruidoso

Consideremos ahora la situación normal de un canal de comunicación que comete errores. Los marcos pueden llegar dañados o perderse por completo. Sin embargo, suponemos que, si un marco se daña en tránsito, el *hardware* del receptor detectará esto cuando calcule la suma de comprobación. Si el marco está dañado de tal manera que pese a ello la suma de comprobación

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA

```
/* El protocolo 2 (parada y espera) también contempla un flujo unidireccional de datos del
   transmisor al receptor. Se supone nuevamente que el canal de comunicación está libre
   de errores, como en el protocolo 1. Sin embargo, esta vez el receptor tiene capacidad
   finita de buffer y capacidad finita de proceso, por lo que el protocolo debe evitar
   explícitamente que el transmisor sature al receptor con datos a mayor velocidad de la
   que puede manejar. */

typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender2(void)
{
  frame s;                          /* buffer para un marco de salida */
  packet buffer;                    /* buffer para un paquete de salida */
  event_type event;                 /* frame_arrival es la única posibilidad */

  while (true) {
      from_network_layer(&buffer);  /* consigue algo qué enviar */
      s.info = buffer;              /* lo copia en s para transmisión */
      to_physical_layer(&s);        /* adiós al pequeño marco */
      wait_for_event(&event);       /* no proceder hasta recibir la señal de
                                       continuación */
  }
}

void receiver2(void)
{
  frame r, s;                       /* buffers para los marcos */
  event_type event;                 /* frame_arrival es la única posibilidad */
  while (true) {
      wait_for_event(&event);       /* la única posibilidad es frame_arrival */
      from_physical_layer(&r);      /* obtiene el marco de entrada */
      to_network_layer(&r.info);    /* pasa los datos a la capa de red */
      to_physical_layer(&s);        /* envía un marco ficticio para despertar al
                                       transmisor */
  }
}
```

Figura 3-10. Protocolo símplex de parada y espera.

sea correcta, un caso excesivamente improbable, este protocolo (y todos los demás protocolos) puede fallar (es decir, entregar un paquete incorrecto a la capa de red).

A primera vista puede parecer que funcionaría una variación del protocolo 2: agregar un temporizador. El transmisor podría enviar un marco, pero el receptor sólo enviaría un marco de acuse si los datos llegaran correctamente. Si llegara un marco dañado al receptor, se desecharía.

Un rato después, el temporizador del transmisor terminaría y se enviaría el marco de nuevo. Este proceso se repetiría hasta que el marco por fin llegara intacto.

El esquema anterior tiene un defecto mortal. Medite el problema e intente descubrir lo que podría fallar antes de continuar leyendo.

Para ver lo que puede resultar mal, recuerde que es tarea de la capa de enlace de datos proporcionar una comunicación libre de errores y transparente entre los procesos de las capas de red. La capa de red de la máquina *A* pasa una serie de paquetes a la capa de enlace de datos, que debe asegurar que se entregue una serie de paquetes idéntica a la capa de red de la máquina *B* a través de su capa de enlace de datos. En particular, la capa de red en *B* no tiene manera de saber si el paquete se ha perdido o se ha duplicado, por lo que la capa de enlace de datos debe garantizar que ninguna combinación de errores de transmisión, por improbables que sean, pueda causar la entrega de un paquete duplicado a la capa de red.

Considere la siguiente situación:

1. La capa de red de *A* entrega el paquete 1 a su capa de enlace de datos. El paquete se recibe correctamente en *B* y se pasa a la capa de red de *B*. *B* envía un marco de acuse de regreso a *A*.

2. El marco de acuse se pierde por completo. Nunca llega. La vida sería mucho más sencilla si el canal sólo alterara o perdiera marcos de datos y no marcos de control, pero desgraciadamente el canal no hace distinciones.

3. La capa de enlace de datos de *A* en algún momento termina su temporización. Al no haber recibido un acuse, supone (incorrectamente) que su marco de datos se ha perdido o dañado, y envía nuevamente el marco que contiene el paquete 1.

4. El marco duplicado también llega perfectamente a la capa de enlace de datos de *B* y de ahí se pasa inadvertidamente a la capa de red. Si *A* está enviando un archivo a *B*, parte del archivo se duplicará (es decir, la copia del archivo reconstruida por *B* será incorrecta y el error no se habrá detectado). En otras palabras, el protocolo fallará.

Es claro que lo que se necesita es alguna manera de que el receptor sea capaz de distinguir entre un marco que está viendo por primera vez y una retransmisión. La forma evidente de lograr esto es hacer que el transmisor ponga un número de secuencia en el encabezado de cada marco que envía; así, el receptor puede examinar el número de secuencia de cada marco que llega para ver si es un marco nuevo o un duplicado que debe descartarse.

Dado que es deseable que el encabezado de los marcos sea pequeño, surge la pregunta: ¿cuál es el mínimo de bits necesarios para el número de secuencia? La única ambigüedad en este protocolo es entre un marco, m, y su sucesor directo, $m + 1$. Si el marco m se pierde o se daña, el receptor no acusará su recibo y el transmisor seguirá tratando de enviarlo. Una vez que el marco se recibe correctamente, el receptor envía un acuse de regreso al transmisor. Es aquí donde surge

el problema potencial. Dependiendo de si el marco de acuse llega de regreso al transmisor correctamente o no, el transmisor puede tratar de enviar m o $m + 1$.

El evento que indica al transmisor que puede enviar $m + 2$ es la llegada de un acuse de $m + 1$. Pero esto implica que m se recibió correctamente, y además que su acuse fue recibido correctamente por el transmisor (de otra manera, el transmisor no habría comenzado con $m + 1$, ya no digamos $m + 2$). Como consecuencia, la única ambigüedad es entre un marco y su antecesor o su sucesor inmediatos, no entre el antecesor y el sucesor mismos.

Por tanto, basta con un número de secuencia de 1 bit (0 o 1). En cada instante, el receptor espera un número de secuencia en particular. Cualquier marco de entrada que contenga un número de secuencia equivocado se rechaza como duplicado.

Cuando llega un marco que contiene el número de secuencia correcto, se acepta y se pasa a la capa de red, y el número de secuencia esperado se incrementa módulo 2 (es decir, 0 se vuelve 1 y 1 se vuelve 0).

Un ejemplo de este tipo de protocolo se muestra en la figura 3-11. Los protocolos en los que el transmisor envía un acuse positivo antes de avanzar al siguiente elemento de datos suelen llamarse **PAR** (*positive acknowledgment with retransmission*, **acuse positivo con retransmisión**), o **ARQ** (*automatic tepeat reQuest*, **solicitud automática de repetición**). Al igual que el protocolo 2, éste también transmite datos en una sola dirección; aunque puede manejar marcos perdidos (por terminación del temporizador), requiere que el intervalo de terminación del temporizador sea lo bastante grande para evitar terminaciones prematuras. Si el transmisor termina su temporización demasiado pronto, mientras el acuse aún está en camino, enviará un duplicado.

Cuando finalmente llegue el acuse previo, el transmisor pensará erróneamente que el marco apenas enviado es el que se está reconociendo, y no se dará cuenta de que puede haber otro marco de acuse en alguna otra parte del "tubo". Si el siguiente marco enviado se pierde por completo pero el acuse extra llega correctamente, el transmisor no intentará retransmitir el marco perdido y el protocolo fallará. En los protocolos posteriores, los marcos de acuse contendrán información para evitar este tipo de problemas. Por ahora, los marcos de acuse sólo serán ficticios, y supondremos una alternación estricta entre el transmisor y el receptor.

El protocolo 3 difiere de sus antecesores en que tanto el transmisor como el receptor tienen una variable cuyo valor se recuerda mientras la capa de enlace de datos está en estado de espera. El transmisor recuerda el número de secuencia del siguiente marco a enviar en *next_frame_to_send*; el receptor recuerda el número de secuencia del siguiente marco esperado en *frame_expected*. Cada protocolo tiene una corta fase de inicialización antes de entrar en el ciclo infinito.

Tras transmitir un marco, el transmisor arranca el temporizador. Si éste ya estaba corriendo, se restablecerá para conceder otro intervalo completo de temporización. El intervalo debe escogerse de modo que haya suficiente tiempo para que el marco llegue al receptor, sea procesado por él en el peor caso, y el acuse se propague de regreso al transmisor. Sólo cuando ha transcurrido ese intervalo de tiempo se puede suponer con seguridad que se ha perdido el marco transmitido o su acuse, y que se debe enviar un duplicado.


```
/* El protocolo 3 (par) permite el flujo unidireccional de datos por un canal no confiable. */

#define MAX_SEQ 1                       /* debe ser 1 para el protocolo 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
  seq_nr next_frame_to_send;            /* número de secuencia del siguiente marco de salida */
  frame s;                              /* variable de trabajo */
  packet buffer;                        /* buffer para un paquete de salida */
  event_type event;

  next_frame_to_send = 0;               /* inicializa números de secuencia de salida */
  from_network_layer(&buffer);          /* obtiene el primer paquete */
  while (true) {
      s.info = buffer;                  /* construye un marco para transmisión */
      s.seq = next_frame_to_send;       /* inserta un número de secuencia en el marco */
      to_physical_layer(&s);            /* lo envía */
      start_timer(s.seq);               /* si (respuesta tarda mucho), termina espera */
      wait_for_event(&event);           /* frame_arrival, cksum_err, timeout */
      if (event == frame_arrival) {
          from_network_layer(&buffer)   /* obtiene siguiente a enviar */
          inc(next_frame_to_send);      /* invierte next_frame_to_send */
      }
  }
}

void receiver3(void)
{
  seq_nr frame_expected;
  frame r, s;
  event_type event;

  frame_expected = 0;
  while (true) {
      wait_for_event(&event);           /* posibilidades: frame_arrival, cksum_err */
      if (event == frame_arrival) {     /* ha llegado un marco válido. */
          from_physical_layer(&r);      /* obtiene el marco recién llegado */
          if (r.seq == frame_expected) {/* esto es lo que hemos estado esperando. */
              to_network_layer(&r.info);/* pasa los datos a la capa de red */
              inc(frame_expected);      /* para la próxima, espera el otro número de
                                           secuencia */
          }
          to_physical_layer(&s);        /* no se usa ninguno de los campos */
      }
  }
}
```

Figura 3-11. Protocolo de acuse positivo con retransmisión.

Tras transmitir un marco y arrancar el temporizador, el transmisor espera que ocurra algo interesante. Hay tres posibilidades: llega un marco de acuse sin daño, llega un marco de acuse dañado o termina el temporizador. Si recibe un acuse válido, el transmisor obtiene el siguiente paquete de la capa de red y lo pone en el *buffer*, sobreescribiendo el paquete previo; también avanza el número de secuencia. Si llega un marco dañado, o no llega ningún marco, no cambian el *buffer* ni el número de secuencia, por lo que se puede enviar un duplicado.

Cuando llega un marco válido al receptor, su número de secuencia se verifica para ver si es un duplicado. Si no lo es, se acepta y pasa a la capa de red y se genera un acuse. Los duplicados y los marcos dañados no se pasan a la capa de red.

3.4. PROTOCOLO DE VENTANA CORREDIZA

En los protocolos previos, los marcos de datos se transmiten en una sola dirección. En la mayoría de las situaciones prácticas hay necesidad de transmitir datos en ambas direcciones. Una manera de lograr la transmisión dúplex integral es tener dos canales de comunicación separados y usar cada uno para tráfico de datos símplex (en diferentes direcciones). Si se hace esto, tenemos dos circuitos físicos separados, cada uno con un canal "directo" (para datos) y un canal "en reversa" (para acuses). En ambos casos, el ancho de banda del canal de reversa se desperdicia casi por completo. En efecto, el usuario está pagando dos circuitos, pero sólo usa la capacidad de uno.

Una mejor idea es usar el mismo circuito para datos en ambas direcciones. Después de todo, en los protocolos 2 y 3 ya se usaba el circuito para transmitir marcos en ambos sentidos, y el canal en reversa tiene la misma capacidad que el canal directo. En este modelo, los marcos de datos de *A* a *B* se mezclan con los marcos de reconocimiento (acuse de recibo) de *A* a *B*. Analizando el campo de tipo (*kind*) en el encabezado de un marco de entrada, el receptor puede saber si el marco es de datos o de acuse.

Aunque la intercalación de datos y marcos de control en el mismo circuito es una mejora respecto al uso de dos circuitos físicos separados, se puede lograr otra mejora. Al llegar un marco de datos, en lugar de enviar inmediatamente un marco de control independiente, el receptor se aguanta y espera hasta que la capa de red le pasa el siguiente paquete. El acuse se anexa al marco de datos de salida (usando el campo *ack* del encabezado del marco). En efecto, el acuse viaja gratuitamente en el siguiente marco de datos de salida. La técnica de retardar temporalmente los acuses para que puedan colgarse del siguiente marco de datos de salida se conoce como **incorporación** (*piggybacking*).

La ventaja principal de usar la incorporación en lugar de tener marcos de acuse independientes es un mejor aprovechamiento del ancho de banda disponible del canal. El campo *ack* (acuse) de el encabezado del marco cuesta sólo unos cuantos bits, mientras que un marco aparte requeriría de un encabezado, el acuse y una suma de comprobación. Además, el envío de menos marcos implica menos interrupciones de "ha llegado marco" y tal vez menos segmentos de

buffer en el receptor, dependiendo de la manera en que esté organizado el *software* del receptor. En el siguiente protocolo que examinaremos el campo de incorporación cuesta sólo 1 bit en el encabezado del marco y pocas veces cuesta más de unos pocos bits.

Sin embargo, la incorporación introduce una complicación inexistente con los acuses independientes. ¿Cuánto tiempo debe esperar la capa de enlace de datos un paquete al cual incorporar el acuse? Si la capa de enlace de datos espera más tiempo del que tarda en terminar el temporizador del transmisor, el marco será retransmitido, frustrando el propósito de enviar acuses. Si la capa de enlace de datos fuera un oráculo y pudiera predecir el futuro, sabría cuándo recibiría el siguiente paquete de la capa de red y podría decidir esperarlo o enviar de inmediato un acuse independiente, dependiendo del tiempo de espera proyectado. Por supuesto, la capa de enlace de datos no puede predecir el futuro, por lo que debe recurrir a algún esquema particular para el caso, como esperar un número fijo de milisegundos. Si llega rápidamente un nuevo paquete, el acuse se incorpora a él; de otra manera, si no ha llegado ningún paquete nuevo al final de este periodo de tiempo, la capa de enlace de datos manda un marco de acuse solo.

Además de ser únicamente símplex, el protocolo 3 puede fallar en ciertas condiciones peculiares relacionadas con la terminación temprana de la temporización. Sería mejor tener un protocolo que permaneciera sincronizado ante cualquier combinación de marcos alterados, marcos perdidos, y terminaciones prematuras. Los tres protocolos siguientes son más robustos y continúan funcionando aun en condiciones patológicas. Los tres pertenecen a la clase de protocolos llamados protocolos de **ventana corrediza**. Los tres difieren en cuanto a eficiencia, complejidad y requerimientos de *buffer*, como se explicará después.

En todos los protocolos de ventana corrediza, cada marco de salida contiene un número de secuencia con un intervalo que va de 0 hasta algún máximo. El máximo es generalmente $2^n - 1$, por lo que el número de secuencia cabe bien en un campo de n bits. El protocolo de ventana corrediza de parada y espera usa $n = 1$, restringiendo los números de secuencia a 0 y 1, pero las versiones más complejas pueden usar un n arbitrario.

La esencia de todos los protocolos de ventana corrediza es que, en cualquier instante, el transmisor mantiene un grupo de números de secuencia que corresponde a los marcos que tiene permitido enviar. Se dice que estos marcos caen dentro de la **ventana transmisora**. De manera semejante, el receptor mantiene una **ventana receptora** correspondiente al grupo de marcos que tiene permitido aceptar. La ventana del transmisor y la del receptor no necesitan tener los mismos límites inferior y superior, ni siquiera el mismo tamaño. En algunos protocolos las ventanas son de tamaño fijo, pero en otros pueden crecer y disminuir a medida que se envían y se reciben los marcos.

Aunque estos protocolos dan a la capa de enlace de datos mayor libertad en cuanto al orden en que puede enviar y recibir marcos, hemos conservado decididamente el requisito de que el protocolo debe entregar los paquetes a la capa de red del destino en el mismo orden en que se pasaron a la capa de enlace de datos de la máquina transmisora. Tampoco hemos cambiado el requisito de que el canal físico de comunicación es "de tipo alambre", esto es, que debe entregar todos los marcos en el orden en que fueron enviados.

Los números de secuencia en la ventana del transmisor representan marcos enviados pero aún no reconocidos. Cuando llega un paquete nuevo de la capa de red, se le da el siguiente número secuencial mayor, y la orilla superior de la ventana avanza en uno. Al llegar un acuse, la orilla inferior avanza en uno. De esta manera la ventana mantiene continuamente una lista de marcos no reconocidos.

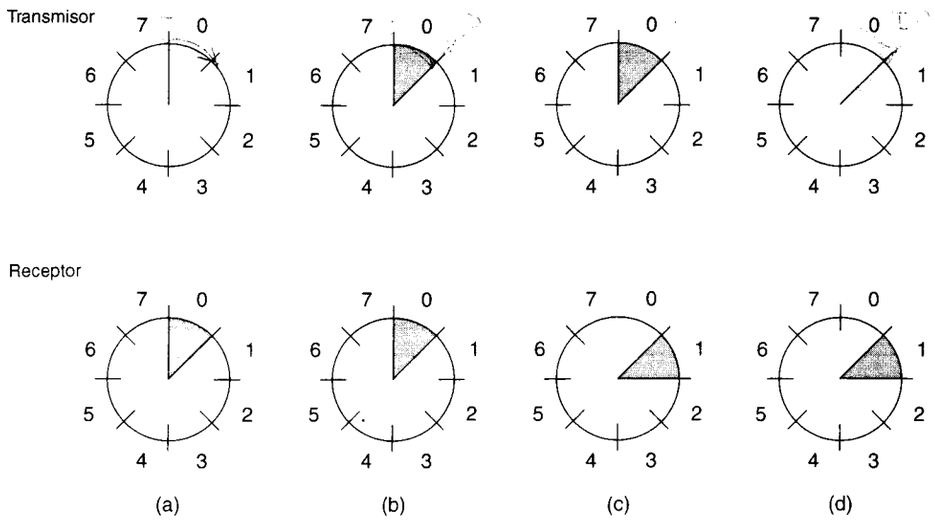


Figura 3-12. Ventana corrediza de tamaño 1, con un número de secuencia de 3 bits. (a) Al inicio. (b) Tras la transmisión del primer marco. (c) Tras la recepción del primer marco. (d) Tras la recepción del primer acuse.

Dado que los marcos que están en la ventana del transmisor pueden perderse o dañarse en tránsito, el transmisor debe mantener todos estos marcos en su memoria para su posible retransmisión. Por tanto, si el tamaño máximo de la ventana es *n*, el transmisor necesita *n buffers* para contener los marcos no reconocidos. Si la ventana llega a crecer a su tamaño máximo, la capa de enlace de datos transmisora deberá hacer que la capa de red se detenga hasta que se libere otro *buffer*.

La ventana de la capa de enlace de datos receptora corresponde a los marcos que puede aceptar. Todo marco que caiga fuera de la ventana se descartará sin comentarios. Cuando se recibe el marco cuyo número de secuencia es igual al extremo inferior de la ventana, se pasa a la capa de red, se genera un acuse y se avanza la ventana en uno. A diferencia de la ventana del transmisor, la ventana del receptor conserva siempre el mismo tamaño inicial. Note que un tamaño de ventana de 1 significa que la capa de enlace de datos sólo acepta marcos en orden, pero con ventanas más grandes esto no es así. La capa de red, en contraste, siempre


```
/* El protocolo 4 (de ventana corrediza) es bidireccional y más robusto que el protocolo 3. */

#define MAX_SEQ 1                                   /* debe ser 1 para el protocolo 4 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void protocol4 (void)
{
  seq_nr next_frame_to_send;            /* sólo 0 o 1 */
  seq_nr frame_expected;                /* sólo 0 o 1 */
  frame r, s;                           /* variables de trabajo */
  packet buffer;                        /* paquete actual que se envía */
  event_type event;

  next_frame_to_send = 0;               /* siguiente marco de la corriente de salida */
  frame_expected = 0;                   /* número de marco de llegada marco esperado */
  from_network_layer(&buffer);          /* obtiene un paquete de la capa de red */
  s.info = buffer;                      /* se prepara para enviar el marco inicial */
  s.seq = next_frame_to_send;           /* inserta el número de secuencia en el marco */
  s.ack = 1 – frame_expected;           /* acuse incorporado */
  to_physical_layer(&s);                /* transmite el marco */
  star_timer(s.seq);                    /* comienza a temporizar */
  while (true) {
     wait_for_event(&event);            /* frame_arrival, cksum_err o timeout */
     if (event == frame_arrival) { /* ha llegado un marco sin daño. */
          from_physical_layer(&r);      /* lo obtiene */
          if (r.seq == frame_expected)  {
               /* Maneja corriente de marcos de entrada. */
               to_network_layer(&r.info);   /* pasa el paquete a la capa de red */
               inc(frame_expected);         /* invierte el siguiente número de secuencia esperado */
          }
          if (r.ack == next_frame_to_send) { /* maneja corriente de marcos de salida. */
               from_network_layer(&buffer);       /* obtiene paquete nuevo de la capa de red */
               inc(next_frame_to_send);   /* invierte el número de secuencia del transmisor */
          }
     }
     s.info = buffer;                   /* construye marco de salida */
     s.seq = next_frame_to_send;        /* le introduce el número de secuencia */
     s.ack = 1 – frame_expected;        /* número de secuencia del último marco recibido */
     to_physical_layer(&s);             /* transmite un marco */
     start_timer(s.seq);                /* comienza a temporizar */
  }
}
```

Figura 3-13. Protocolo de ventana corrediza de 1 bit.

recibe los datos en el orden correcto, sin importar el tamaño de la ventana de la capa de enlace de datos.

En la figura 3-12 se muestra un ejemplo con un tamaño máximo de ventana de 1. Inicialmente no hay marcos pendientes, por lo que los extremos de la ventana del transmisor son iguales, pero a medida que pasa el tiempo la situación progresa como se muestra.

3.4.1. Un protocolo de ventana corrediza de un bit

Antes de lidiar con el caso general, examinemos un protocolo de ventana corrediza con un tamaño máximo de ventana de 1. Tal protocolo usa parada y espera, ya que el transmisor envía un marco y espera su acuse antes de transmitir el siguiente.

En la figura 3-13 se presenta tal protocolo. Como los demás, comienza por definir algunas variables. *Next_frame_to_send* indica qué marco está tratando de enviar el transmisor. De manera semejante, *frame_expected* indica qué marco espera el receptor. En ambos casos, 0 y 1 son las únicas posibilidades.

Normalmente, una de las dos capas de enlace de datos es la que comienza. En otras palabras, sólo uno de los programas de capa de enlace de datos debe contener las llamadas de procedimiento *to_physical_layer* y *start_timer* fuera del ciclo principal. Si ambas capas se iniciaran simultáneamente, surgiría una situación peculiar que se analizará después. La máquina que arranca obtiene el primer paquete de su capa de red, construye un marco a partir de él y lo envía. Al llegar este (o cualquier) marco, la capa de enlace de datos receptora lo revisa para ver si es un duplicado, igual que en el protocolo 3. Si el marco es el esperado, se pasa a la capa de red y la ventana del receptor se recorre hacia arriba.

El campo de acuse contiene el número del último marco recibido sin error. Si este número concuerda con el número de secuencia del marco que está tratando de enviar el transmisor, éste sabe que ha terminado con el marco almacenado en el *buffer* y que puede obtener el siguiente paquete de su capa de red. Si el número de secuencia no concuerda, debe continuar intentando enviar el mismo marco. Por cada marco que se recibe, se envía un marco de regreso.

Ahora examinemos el protocolo 4 para ver qué tan flexible es ante circunstancias patológicas. Suponga que *A* está tratando de enviar su marco 0 a *B* y que *B* está tratando de enviar su marco 0 a *A*. Suponga que *A* envía un marco a *B*, pero que el intervalo de temporización de *A* es un poco corto. En consecuencia, *A* podría terminar su temporización repetidamente, enviando una serie de marcos idénticos, todos con *seq* = 0 y *ack* = 1.

Al llegar el primer marco válido a *B*, es aceptado y *frame_expected* se pone en 1. Todos los marcos subsiguientes serán rechazados, pues *B* ahora espera marcos con el número de secuencia 1, no 0. Además, dado que los duplicados tienen *ack* = 1 y *B* aún está esperando un acuse de 0, *B* no pedirá un nuevo paquete a su capa de red.

Cada vez que llega un duplicado rechazado, *B* envía a *A* un marco que contiene *seq* = 0 y *ack* = 0. Tarde o temprano uno de éstos llegará correctamente a *A*, haciendo que *A* comience a enviar el siguiente paquete. Ninguna combinación de marcos perdidos o terminaciones prematuras puede hacer que el protocolo entregue paquetes duplicados a cualquiera de las capas de red, ni que omita un paquete, ni que entre en un interbloqueo.

Sin embargo, surge una situación peculiar si ambos lados envían simultáneamente un paquete inicial. Este problema de sincronización se ilustra en la figura 3-14. En la parte (a) se muestra la operación normal del protocolo. En (b) se ilustra la peculiaridad. Si *B* espera el primer marco de *A* entes de enviar el suyo, la secuencia es como se muestra en (a), y todos los marcos son aceptados. Sin embargo, si *A* y *B* inician la comunicación simultáneamente, se cruzan sus primeros marcos y las capas de enlace de datos entran en la situación (b). En (a) cada marco que llega trae un paquete nuevo para la capa de red; no hay duplicados. En (b) la mitad de los marcos contienen duplicados, aun cuando no hay errores de transmisión. Pueden ocurrir situaciones similares como resultado de terminaciones de temporizador prematuras, aun cuando un lado evidentemente comienza primero. De hecho, si ocurren varias terminaciones prematuras los marcos podrían enviarse tres o más veces.

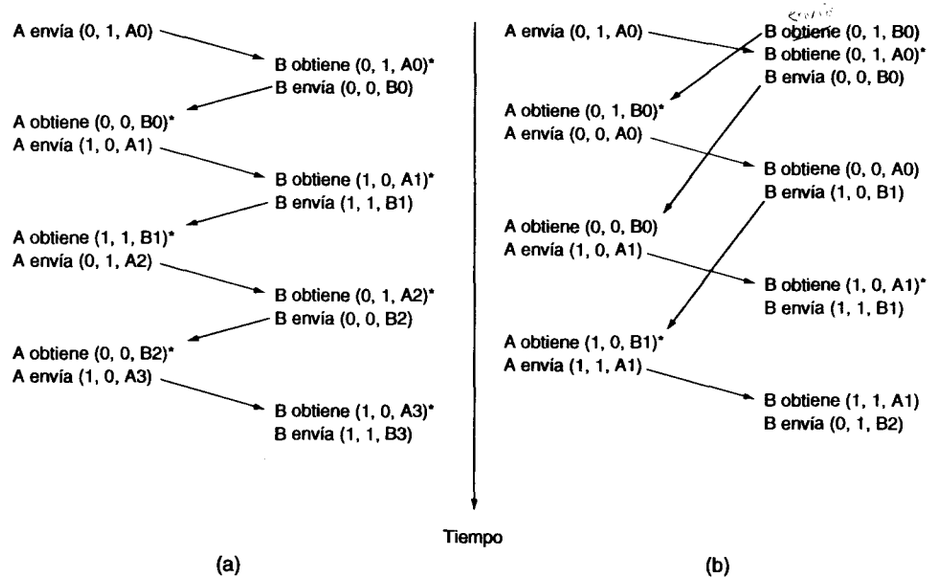


Figura 3-14. Dos situaciones para el protocolo 4. La notación es (secuencia, acuse, número de paquete). Un asterisco indica el lugar en que una capa de red acepta un paquete.

3.4.2. Un protocolo que usa regresar n

Hasta ahora hemos supuesto tácitamente que el tiempo de transmisión requerido para que un marco llegue al receptor más el necesario para que el acuse regrese es insignificante. A veces esta suposición es claramente falsa. En estas situaciones el tiempo de viaje redondo prolongado puede tener implicaciones importantes para la eficiencia del aprovechamiento del ancho de banda. Por ejemplo, considere un canal de satélite de 50 kbps con un retardo de propagación de ida

y vuelta de 500 mseg. Imagine que intentamos usar el protocolo 4 para enviar marcos de 1000 bits por medio del satélite. En $t = 0$, el transmisor intenta enviar el primer marco. En $t = 20$ mseg el marco ha sido enviado completamente. En las mejores circunstancias (sin esperas en el receptor y con un marco de acuse corto), no es sino hasta $t = 270$ mseg que el marco ha llegado por completo al receptor, y no es sino hasta $t = 520$ mseg que ha llegado el acuse de regreso al transmisor. Esto implica que el transmisor estuvo bloqueado durante el 500/520 = 96% del tiempo (es decir, sólo se usó el 4% del ancho de banda disponible). Queda claro que la combinación de un tiempo de tránsito grande, un ancho de banda grande y una longitud de marco corta es desastrosa en lo que toca a la eficiencia.

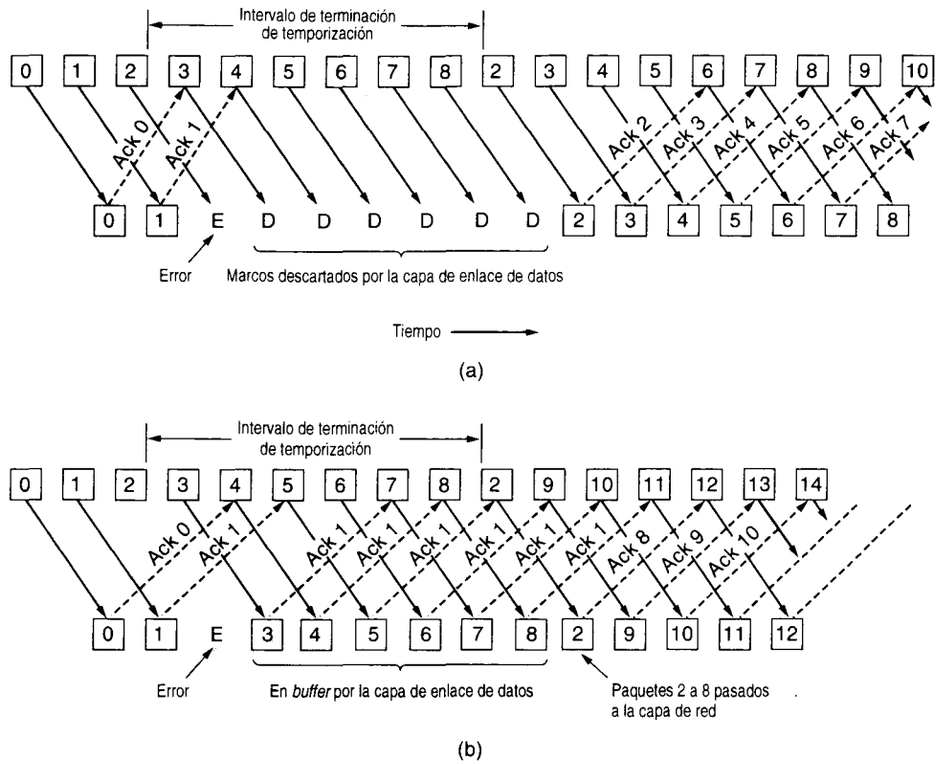


Figura 3-15. (a) Efecto de un error cuando el tamaño de la ventana del receptor es de 1. (b) Efecto de un error cuando el tamaño de la ventana del receptor es grande.

El problema antes descrito puede verse como una consecuencia de la regla que requiere que el transmisor espere un acuse antes de enviar otro marco. Si relajamos esa restricción, puede lograrse una eficiencia mucho mejor. Básicamente la solución está en permitir que el transmisor

envíe hasta w marcos antes de bloquearse, en lugar de sólo 1. Con una selección adecuada de w, el transmisor podrá transmitir marcos continuamente durante un tiempo igual al tiempo de tránsito de ida y vuelta sin llenar la ventana. En el ejemplo anterior, w debe ser de cuando menos 26. El transmisor comienza por enviar el marco 0, como antes. Para cuando ha terminado de enviar 26 marcos, en $t = 520$, llega el acuse del marco 0. A partir de entonces, los acuses llegarán cada 20 mseg, por lo que el transmisor siempre tendrá permiso de continuar justo cuando lo necesita. En todo momento hay pendientes 25 o 26 marcos no reconocidos. Dicho de otra manera, el tamaño máximo de la ventana del transmisor es de 26.

Esta técnica se conoce como **entubamiento**. Si la capacidad del canal es de b bits/seg, el tamaño de marco de l bits y el tiempo de propagación de ida y vuelta de R segundos, el tiempo requerido para transmitir un solo marco es de l/b segundos. Una vez que ha sido enviado el último bit de un marco de datos, hay un retardo de $R/2$ antes de que llegue ese bit al receptor y otro retardo de cuando menos $R/2$ para que el acuse llegue de regreso, lo que da un retardo total de R. En parada y espera la línea está ocupada durante l/b y ociosa durante R, dando una utilización de la línea de $l/(l + bR)$. Si $l < bR$, la eficiencia será menor que 50%. Ya que siempre hay un retardo diferente de cero para que el acuse se propague de regreso, en principio el entubamiento puede servir para mantener ocupada la línea durante este intervalo, pero si el intervalo es pequeño, la complejidad adicional no justifica el esfuerzo.

El envío de marcos en entubamiento por un canal de comunicación inestable presenta problemas serios. Primero, ¿qué ocurre si un marco a la mitad de una serie larga se daña o se pierde? Llegarán grandes cantidades de marcos sucesivos al receptor antes de que el transmisor se entere de que algo anda mal. Al llegar un marco dañado al receptor, obviamente debe descartarse, pero, ¿qué debe hacerse con los marcos correctos que le siguen? Recuerde que la capa de enlace de datos receptora está obligada a entregar paquetes a la capa de red en secuencia.

Hay dos enfoques básicos para manejar los errores durante el entubamiento. Una manera, llamada **regresa n**, es que el receptor simplemente descarte todos los marcos subsecuentes, sin enviar acuses para los marcos descartados. Esta estrategia corresponde a una ventana de recepción de tamaño 1. En otras palabras, la capa de enlace de datos se niega a aceptar cualquier marco excepto el siguiente que debe entregar a la capa de red. Si la ventana del transmisor se llena antes de terminar el temporizador, el entubamiento comenzará a vaciarse. En algún momento, el transmisor terminará de esperar y retransmitirá todos los marcos no reconocidos en orden, comenzando por el dañado o perdido. Este enfoque, que se muestra en la figura 3-15(a), puede desperdiciar bastante ancho de banda si la tasa de errores es alta.

La otra estrategia general para manejar los errores al transmitir marcos en cascada, llamada **repetición selectiva**, consiste en hacer que la capa de enlace de datos receptora almacene todos los marcos correctos a continuación del equivocado. Cuando el transmisor por fin se da cuenta de que algo está mal, sólo retransmite el marco malo, no todos sus sucesores, como se muestra en la figura 3-15(b). Si tiene éxito el segundo intento, la capa de enlace de datos receptora ahora tendrá muchos marcos correctos en secuencia, los cuales puede entregar rápidamente a la capa de red, reconociendo el número más alto.

Esta estrategia corresponde a una ventana del receptor mayor que 1. Cualquier marco dentro de la ventana puede ser aceptado y mantenido en el *buffer* hasta que todos los precedentes hayan

```
/* El protocolo 5 (transmisión en cascada) permite múltiples marcos pendientes. El transmisor puede
   enviar hasta MAX_SEQ marcos sin esperar un acuse. Además, a diferencia de los protocolos
   previos, no se supone que la capa de red tenga paquetes nuevos todo el tiempo. En cambio, la
   capa de red causa un evento network_layer_ready cuando hay un paquete por enviar. */

#define MAX_SEQ 7                       /* debe ser 2^n – 1 */
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready} event_type;
#include "protocol.h"

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
/* Devuelve true si (a <=b < c circularmente; false de otra manera. */
  if (((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a)))
       return(true);
    else
       return(false);
}

static void send_data(seq_nr frame_nr, seq_nr frame_expected, packet buffer[ ])
{
/* Construye y envía un marco de datos. */
  frame s;                              /* variable de trabajo */

  s.info = buffer[frame_nr];            /* inserta un paquete en el marco */
  s.seq = frame_nr;                     /* inserta número de secuencia en el marco */
  s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1); /* incorpora acuse */
  to_physical_layer(&s);                /* transmite el marco */
  start_timer(framer_nr);               /* comienza a temporizar */
}

void protocol5(void)
{
seq_nr next_frame_to_send;              /* MAX_SEQ > 1; se usa para corriente de salida */
  seq_nr ack_expected;                  /* marco más viejo aún no reconocido */
  seq_nr frame_expected;                /* siguiente marco esperado en corriente de entrada */
  frame r;                              /* variable de trabajo */
  packet buffer[MAX_SEQ];               /* buffers para corriente de salida */
  seq_nr nbuffered;                     /* # de buffers usados actualmente */
  seq_nr i;                             /* sirve para indizar el arreglo de buffers */
  event_type event;

  enable_network_layer();               /* permite eventos network_layer_ready */
  ack_expected = 0;                     /* siguiente acuse de entrada esperado */
  next_frame_to_send = 0;               /* siguiente marco de salida */
  frame_expected = 0;                   /* número de marco de entrada esperado */
  nbuffered = 0;                        /* inicialmente no hay paquetes en el buffer */
```



```
  while (true) {
    wait_for_event(&event);             /* cuatro posibilidades: ver event_type antes */

    switch(even) {
      case network_layer_ready:         /* la capa de red tiene un paquete que enviar */
           /* Acepta, guarda y transmite un marco nuevo. */
           from_network_layer(&buffer[next_frame_to_send]); /* obtiene paquete nuevo */
           nbuffered = nbuffered + 1;   /* expande la ventana del transmisor */
           send_data(next_frame_to_send, frame_expected, buffer); /* transmite el marco */
           inc(next_frame_to_send);     /* avanza límite superior de ventana del transmisor */

      case frame_arrival:               /* llegó un marco de datos o de control */
           from_physical_layer(&r);     /* obtiene el marco de entrada de la capa física */

           if (r.seq == frame_expected) {
                /* Sólo se aceptan marcos en orden. */
                to_network_layer(&r.info);  /* pasa paquete a la capa de red */
                inc(frame_expected);        /* avanza límite inferior de ventana del receptor */
           }

           /* Ack n implica n – 1, n – 2, etc. Verificar esto. */
           while (between(ack_expected, r.ack, next_frame_to_send)) {
                /* Maneja acuse incorporado. */
                nbuffered = nbuffered – 1;  /* un marco menos en el buffer */
                stop_timer(ack_expected);   /* marco llegó intacto; detiene temporizador */
                inc(ack_expected);          /* contrae la ventana del transmisor */
           }

      case cksum_err: ;                 /* ignora los marcos con error */

      case timeout:                     /* problemas; retransmite todos los marcos pendientes */
           next_frame_to_send = ack_expected;  /* comienza la retransmisión aquí */
           for (i = 1; i <= nbuffered; i++) {
                send_data(next_frame_to_send, frame_expected, buffer); /* reenvía 1 marco */
                inc(next_frame_to_send);    /* prepara el envío del siguiente */
           }
    }

    if (nbuffered < MAX_SEQ)
         enable_network_layer();
    else
         disable_network_layer();
  }
}
```

Figura 3-16. Protocolo de ventana corrediza usando regresa n.

UNIVERSIDAD DE LA REPUBLICA FACULTAD DE INGENIERIA BIBLIOTECA DOCUMENTACION MONTEVIDEO - URUGUAY

sido pasados a la capa de red. Este enfoque puede requerir cantidades grandes de memoria en capa de enlace de datos si la ventana es grande.

Estos dos enfoques alternativos son intercambios entre el ancho de banda y el espacio de *buffer* en la capa de enlace de datos. Dependiendo de qué recurso sea más valioso, se puede usar uno o el otro. En la figura 3-16 se muestra un protocolo de transmisión en entubamiento en el que la capa de enlace de datos sólo acepta marcos en orden; los marcos siguientes a un error son descartados. En este protocolo hemos omitido por primera vez el supuesto de que la capa de red siempre tiene un suministro infinito de paquetes que enviar. Cuando la capa de red tiene un paquete para enviar, puede causar la ocurrencia de un evento *network_layer_ready* (capa de red lista). Sin embargo, para poder cumplir la regla de control de flujo de que no debe haber más de *MAX_SEQ* marcos no reconocidos pendientes en cualquier momento, la capa de enlace de datos debe poder prohibir a la capa de red que la moleste con más trabajo. Los procedimientos de biblioteca *enable_network_layer* (habilitar capa de red) y *disable_network_layer* (inhabilitar capa de red) llevan a cabo esta función.

Note que pueden estar pendientes como máximo *MAX_SEQ* marcos, y no *MAX_SEQ* + 1 en cualquier momento, aun cuando haya *MAX_SEQ* + 1 números de secuencia diferentes: 0, 1, 2, . . . , *MAX_SEQ*. Para ver por qué es necesaria esta restricción, considere la siguiente situación con *MAX_SEQ* = 7.

1. El transmisor envía los marcos 0 a 7.
2. En algún momento llega al transmisor un acuse incorporado para el marco 7.
3. El transmisor envía otros ocho marcos, nuevamente con los números de secuencia 0 a 7.
4. Ahora llega otro acuse incorporado para el marco 7.

La pregunta es: ¿llegaron con éxito los ocho marcos que correspondían al segundo bloque o se perdieron los ocho (contando como pérdidas los rechazos siguientes a un error)? En ambos casos el receptor podría estar enviando el marco 7 como acuse. El transmisor no tiene manera de saberlo. Por esta razón, el número máximo de marcos pendientes debe restringirse a *MAX_SEQ*.

Aunque el protocolo 5 no pone en el *buffer* los marcos que llegan tras un error, no escapa del problema de los *buffers* por completo. Dado que un transmisor puede tener que retransmitir en un momento futuro todos los marcos no reconocidos, debe retener todos los marcos transmitidos hasta saber de seguro que han sido aceptados por el receptor. Al llegar un acuse para el marco n, los marcos $n-1$, $n-2$, y demás, son reconocidos automáticamente. Esta propiedad es especialmente importante cuando algunos marcos previos portadores de acuses se perdieron o se dañaron. Cuando llega un acuse, la capa de enlace de datos ve si se pueden liberar *buffers*. Si esto es posible (es decir, hay espacio disponible en la ventana), ya puede permitirse que una capa de red previamente bloqueada produzca más eventos *network_layer_ready*.

Debido a que este protocolo tiene múltiples marcos pendientes, necesita lógicamente múltiples temporizadores, uno por cada marco pendiente. Cada marco termina de temporizar independientemente de todos los demás. Todos estos temporizadores pueden simularse fácilmente en *software*, usando un único reloj de *hardware* que produzca interrupciones periódicas. Las terminaciones de temporización pendientes forman una lista ligada, indicando cada nodo de la lista la cantidad de pulsos de reloj que faltan para terminar la temporización, el número del marco temporizado y un apuntador al siguiente nodo.

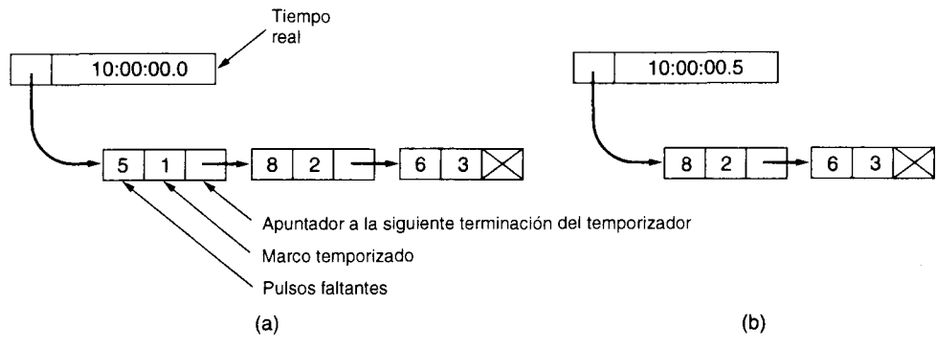


Figura 3-17. Simulación de múltiples temporizadores en *software*.

Como ilustración de una manera de implementar los temporizadores, considere el ejemplo de la figura 3-17. Suponga que el reloj pulsa una vez cada 100 mseg. Inicialmente la hora real es 10:00:00.0 y hay tres terminaciones pendientes, a las 10:00:00.5, 10:00:01.3 y 10:00:01.9. Cada vez que pulsa el reloj de *hardware* se actualiza el tiempo real y el contador de pulsos a la cabeza de la lista se decrementa. Al llegar a cero el contador de pulsos, se causa una terminación y se retira el nodo de la lista, como se muestra en la figura 3-17(b). Aunque esta organización requiere que la lista se examine al llamar a *start_timer* o a *stop_timer*, no implica de mucho trabajo por pulso. En el protocolo 5 estas dos rutinas tienen un parámetro que indica el marco a temporizar.

3.4.3. Protocolo usando repetición selectiva

El protocolo 5 funciona bien si los errores son poco frecuentes, pero si la línea es mala se desperdicia mucho ancho de banda en los marcos retransmitidos. Una estrategia alterna para el manejo de errores es permitir que el receptor acepte y coloque en *buffers* los marcos que siguen a un marco dañado o perdido. Tal protocolo no rechaza marcos simplemente porque se dañó o se perdió un marco anterior.

En este protocolo, tanto el transmisor como el receptor mantienen una ventana de números de secuencia aceptables. El tamaño de la ventana del transmisor comienza en 0 y crece hasta un máximo predefinido, *MAX_SEQ*. La ventana del receptor, en cambio, siempre es de tamaño fijo e igual a *MAX_SEQ*. El receptor tiene un *buffer* reservado para cada número de secuencia en su

ventana. Asociado a cada *buffer* hay un bit (*arrived*) que indica si el *buffer* está lleno o vacío. Al llegar un marco, su número de secuencia es revisado por la función *between* para ver si cae dentro de la ventana. De ser así, y si no ha sido recibido aún, se acepta y almacena. Esta acción se lleva a cabo sin importar si el marco contiene el siguiente paquete esperado por la capa de red. Por supuesto, debe mantenerse dentro de la capa de enlace de datos sin entregarse a la capa de red hasta que todos los marcos de número menor hayan sido entregados a la capa de red en el orden correcto. En la figura 3-18 se presenta un protocolo que usa este algoritmo.

La recepción no secuencial introduce ciertos problemas que no se presentan en los protocolos en los que los marcos sólo se aceptan en orden. Podemos ilustrar el problema fácilmente con un ejemplo. Suponga que tenemos un número de secuencia de tres bits, por lo que se permite al transmisor enviar hasta siete marcos antes de requerírsele esperar un acuse. Inicialmente las ventanas del transmisor y del receptor están como se muestra en la figura 3-19(a). El transmisor ahora transmite los marcos 0 a 6. La ventana del receptor le permite aceptar cualquier marco con un número de secuencia entre 0 y 6, inclusive. Los siete marcos llegan correctamente, por lo que el receptor los reconoce y avanza su ventana para permitir la recepción de 7, 0, 1, 2, 3, 4 o 5, como se muestra en la figura 3-19(b). Los siete *buffers* se marcan como vacíos.

Es en este punto en el que aparece un desastre en la forma de un rayo que cae en el poste telefónico, borrando todos los acuses. El transmisor en algún momento termina de temporizar y retransmite el marco 0. Al llegar este marco al receptor, se efectúa una comprobación para ver si está dentro de la ventana del receptor. Desafortunadamente, en la figura 3-19(b), el marco 0 está dentro de la nueva ventana, por lo que se acepta. El receptor envía un acuse incorporado para el marco 6, ya que se han recibido del 0 al 6.

El transmisor se entera con beneplácito que todos sus marcos transmitidos han llegado correctamente, por lo que avanza su ventana y envía de inmediato los marcos 7, 0, 1, 2, 3, 4 y 5. El marco 7 será aceptado por el receptor y su paquete se pasará directamente a la capa de red. Inmediatamente después, la capa de enlace de datos receptora revisa si ya tiene un marco 0 válido, descubre que sí, y pasa el paquete que contiene a la capa de red. En consecuencia, la capa de red obtiene un paquete incorrecto, y falla el protocolo.

La esencia del problema es que, una vez que el receptor ha avanzado su ventana, el nuevo intervalo de números de secuencia válidos se traslapa con el anterior. El siguiente grupo de marcos podría ser de duplicados (si se perdieron todos los acuses) o de nuevos (si se recibieron todos los acuses). El pobre receptor no tiene manera de distinguir entre estos dos casos.

La salida de este dilema está en asegurarse que, una vez que el transmisor haya avanzado su ventana, no haya traslape con la ventana original. Para asegurar que no haya traslape, el tamaño máximo de la ventana debe ser cuando menos de la mitad del intervalo de los números de secuencia, como en las figuras 3-19(c) y (d). Por ejemplo, si se usan 4 bits para los números de secuencia, éstos tendrán un intervalo de 0 a 15. Sólo ocho marcos no reconocidos deben estar pendientes en cualquier instante. De esa manera, si el receptor apenas ha aceptado

los marcos 0 a 7 y ha avanzado su ventana para permitir la aceptación de los marcos 8 a 15, puede distinguir sin ambigüedades si los marcos subsiguientes son retransmisiones (0 a 7) o nuevos (8 a 15). En general, el tamaño de la ventana para el protocolo 6 será (*MAX_SEQ* + 1)/2.

Una pregunta interesante es: ¿cuántos *buffers* deberá tener el receptor? En ninguna circunstancia puede aceptar marcos cuyos números de secuencia estén por debajo del borde inferior de la ventana o por encima del borde superior de la ventana. En consecuencia, el número de *buffers* necesarios es igual al tamaño de la ventana, no al intervalo de números de secuencia. En el ejemplo anterior de un número de secuencia de 4 bits, se requieren ocho *buffers*, numerados del 0 al 7. Al llegar el marco *i*, se coloca en el *buffer i* mod 8. Observe que, aun cuando *i* e $(i + 8)$ mod 8 están "compitiendo" por el mismo *buffer*, nunca están dentro de la ventana al mismo tiempo, pues ello implicaría un tamaño de ventana de cuando menos 9.

Por la misma razón, el número de temporizadores requeridos es igual al número de *buffers*, no al tamaño del espacio de secuencia. Efectivamente, hay un temporizador asociado a cada *buffer*. Cuando termina el temporizador, el contenido del *buffer* se retransmite.

En el protocolo 5 se supone implícitamente que el canal está fuertemente cargado. Al llegar un marco, no se envía de inmediato el acuse. En cambio, el acuse se incorpora en el siguiente marco de datos de salida. Si el tráfico en reversa es ligero, el acuse se detendrá durante un periodo largo. Si hay mucho tráfico en una dirección y no hay tráfico en la otra, sólo se envían *MAX_SEQ* paquetes y luego se bloquea el protocolo.

En el protocolo 6 se corrige este problema. Tras llegar un marco de datos en secuencia, se arranca un temporizador auxiliar mediante *start_ack_timer*. Si no se ha presentado tráfico en reversa antes de terminar este temporizador, se envía un marco de acuse independiente. Una interrupción debida al temporizador auxiliar es un evento *ack_timeout* (terminación de temporización de acuse). Con este arreglo, ahora es posible el flujo de tráfico unidireccional, pues la falta de marcos de datos en reversa a los que pueden incorporarse los acuses ya no es un obstáculo. Sólo existe un temporizador auxiliar, y si *start_ack_timer* se invoca mientras el temporizador está corriendo, se restablece a un intervalo completo de temporización del acuse.

Es indispensable que el lapso asociado al temporizador auxiliar sea notablemente más corto que el del temporizador usado para temporizar marcos de datos. Esta condición es necesaria para asegurarse que el acuse de un marco correctamente recibido llegue antes de que el transmisor termine su temporización y retransmita el marco.

El protocolo 6 utiliza una estrategia más eficiente que el 5 para manejar los errores. Cuando el receptor tiene razones para suponer que ha ocurrido un error, envía un marco de acuse negativo (NAK) al transmisor. Tal marco es una solicitud de retransmisión del marco especificado en él. Hay dos casos en los que el receptor debe sospechar: la llegada de un marco dañado, o la llegada de otro marco diferente del esperado (pérdida potencial del marco). Para evitar hacer múltiples solicitudes de retransmisión del mismo marco perdido, el receptor debe saber si ya se ha enviado un NAK para un marco dado. La variable *no_nak* del protocolo 6 es *true* si todavía no se ha enviado ningún NAK para *frame_expected*. Si el NAK se altera o se pierde no hay un daño real, pues de todos modos el transmisor terminará su temporización en

```
/* El protocolo 6 (recepción no secuencial) acepta marcos fuera de orden, pero pasa paquetes a la
   capa de red en orden. Hay un temporizador asociado a cada marco pendiente. Cuando termina el
   temporizador, sólo ese marco se retransmite, no todos los marcos pendientes, como en el
   protocolo 5. */

#define MAX_SEQ 7                                   /* debe ser 2^n – 1 */
#define NR_BUFS ((MAX_SEQ + 1)/2)
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready, ack_timeout} event_type;
#include "protocol.h"
boolean no_nak = true;                              /* no se ha enviado ningún nak todavía */

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
/* Igual que between en el protocolo5, pero más corto y menos claro. */
  return ((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a));
}

static void send_frame(frame_kind fk, seq_nr frame_nr, seq_nr frame_expected, packet buffer[ ])
{
/* Construye y envía un marco de datos, ack o nak. */
  frame s;                                          /* variable de trabajo */

  s.kind = fk;                                      /* kind == data, ack o nak */
  if (fk == data) s.info = buffer[frame_nr % NR_BUFS];
  s.seq = frame_nr;                                 /* sólo significativo para marcos de datos */
  s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);
  if (fk == nak) no_nak = false;                    /* por favor, un nak por marco */
  to_physical_layer(&s);                            /* transmite el marco */
  if (fk == data) star_time(frame_nr % NR_BUFS);
  stop_ack_timer();                                 /* no hay necesidad de un marco ack independiente */
}

void protocol6(void)
{
  seq_nr ack_expected;                              /* límite inferior de ventana del transmisor */
  seq_nr next_frame_to_send;                        /* límite superior de ventana del transmisor + 1 */
  seq_nr frame_expected;                            /* límite inferior de ventana del receptor */
  seq_nr too_far;                                   /* límite superior de ventana del receptor + 1 */
  seq_nr oldest_frame;                              /* ¿qué marco terminó su temporización? */
  int i;                                            /* indiza en el buffer */
  frame r;                                          /* variable de trabajo */
  packet out_buf[NR_BUFS];                          /* buffers para corriente de salida */
  packet in_buf[NR_BUFS];                           /* buffers para corriente de entrada */
  boolean arrived[NR_BUFS];                         /* mapa de bits de entrada */
  seq_nr nbuffered;                                 /* cantidad de buffers de salida usados actualmente */
  event_type event;

  enable_network_layer();                           /* inicializa */
  ack_expected = 0;                                 /* siguiente ack esperado en la corriente de entrada */
  next_frame_to_send = 0;                           /* número del siguiente marco de salida */
  frame_expected = 0;
  too_far = NR_BUFS;
  nbuffered = 0;                                    /* inicialmente no hay paquetes en el buffer */

  for (i = 0; i < NR_BUFS; i++) arrived[i] = false;
```



```
  wait_for_event(&event);                           /* cinco posibilidades: ver event_type antes */
  switch(event) {
    case network_layer_ready:                       /* acepta, guarda y transmite un marco nuevo */
         nbuffered = nbuffered + 1;                 /* expande la ventana */
         from_network_layer(&out_buf[next_frame_to_send % NR_BUFS]); /* obtiene paquete nuevo */
         send_frame(data, next_frame_to_send, frame_expected, out_buf); /* transmite el marco */
         inc(next_frame_to_send);                   /* avanza orilla superior de ventana */

    case frame_arrival:                             /* llegó un marco de datos o de control */
         from_physical_layer(&r);                   /* obtiene marco de entrada de la capa física */
         if (r.kind == data) {
              /* Llegó un marco sin daños. */
              if ((r.seq != frame_expected) && no_nak)
                send_frame(nak, 0, frame_expected, out_buf); else start_ack_timer();
              if (between(frame_expected, r.seq, too_far) && (arrived[r.seq%NR_BUFS] == false)) {
                   /* Los marcos pueden aceptarse en cualquier orden. */
                   arrived[r.seq % NR_BUFS] = true;   /* marca como lleno el buffer */
                   in_buf[r.seq % NR_BUFS] = r.info;  /* inserta datos en el buffer */
                   while (arrived[frame_expected % NR_BUFS]) {
                        /* Pasa marcos y avanza ventana. */
                        to_network_layer(&in_buf[frame_expected % NR_BUFS]);
                        no_nak = true;
                        arrived[frame_expected % NR_BUFS] = false;
                        inc(frame_expected);        /* avanza orilla inferior de ventana del receptor */
                        inc(too_far);               /* avanza orilla superior de ventana del receptor */
                        start_ack_timer();          /* para ver si se requiere un ack por separado */
                   }
              }
         }
         if((r.kind==nak) && between(ack_expected,(r.ack+1)%(MAX_SEQ+1), next_frame_to_send))
              send_frame(data, (r.ack+1) % (MAX_SEQ + 1), frame_expected, out_buf);

         while (between(ack_expected, r.ack, next_frame_to_send)) {
              nbuffered = nbuffered – 1;            /* maneja ack incorporado */
              stop_timer(ack_expected % NR_BUFS);   /* marco llegó intacto */
              inc(ack_expected);                    /* avanza orilla inferior de ventana del transmisor */
         }

    case cksum_err:
         if (no_nak) send_frame(nak, 0, frame_expected, out_buf); /* marco dañado */

    case timeout:
         send_frame(data, oldest_frame, frame_expected, out_buf); /* temporizador terminó */

    case ack_timeout:
         send_frame(ack,0,frame_expected, out_buf);     /* temporizador de ack terminó; envía ack */
    }

    if (nbuffered < NR_BUFS) enable_network_layer(); else disable_network_layer();
  }
}
```

Figura 3-18. Protocolo de ventana corrediza usando repetición selectiva.

algún momento y retransmitirá el marco perdido. Si llega el marco equivocado después de haberse enviado y de perderse un NAK, *no_nak* será *true* y el temporizador auxiliar arrancará. Cuando termine, se enviará un ACK para resincronizar el estado actual del transmisor con el del receptor.

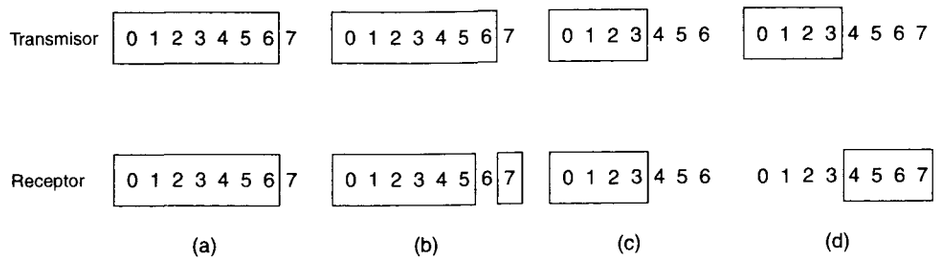


Figura 3-19. (a) Situación original con una ventana de tamaño 7. (b) Después de que se han enviado y recibido siete marcos, pero no se han reconocido. (c) Situación inicial con un tamaño de ventana de 4. (d) Después de que se han enviado y recibido cuatro marcos, pero no se han reconocido.

En algunas situaciones, el tiempo requerido para que un marco se propague a su destino, sea procesado ahí y regrese el acuse es (casi) constante. En estos casos, el transmisor puede ajustar su temporizador para que apenas sea mayor que el intervalo de tiempo esperado entre el envío de un marco y la recepción de su acuse. Sin embargo, si este tiempo es muy variable, el transmisor se enfrenta a la decisión de establecer el intervalo en un valor pequeño y arriesgarse a retransmisiones innecesarias, desperdiciando ancho de banda, o de establecerlo en un valor grande, quedándose inactivo por un periodo largo tras un error y desperdiciando también ancho de banda. Si el tráfico en reversa es esporádico, el tiempo antes del acuse será irregular, siendo más corto al haber tráfico en reversa y mayor al no haberlo. Las variaciones en el tiempo de proceso en el receptor también pueden ser un problema aquí. En general, cuando la desviación estándar del intervalo de acuse es pequeña en comparación con el intervalo mismo, el temporizador puede hacerse "justo" y los NAK no serán útiles. De otra manera, el temporizador deberá hacerse "holgado", y los NAK pueden acelerar apreciablemente la retransmisión de marcos perdidos o dañados.

Un aspecto muy relacionado con el asunto de la terminación de los temporizadores y los NAK es cómo determinar qué marco causó una terminación del temporizador. En el protocolo 5 siempre es *ack_expected* (acuse esperado), pues es el más viejo. En el protocolo 6 no hay ninguna manera sencilla de determinar quién ha terminado su temporización. Suponga que ya se transmitieron los marcos 0 a 4, de modo que la lista de marcos pendientes es 01234, en orden del más viejo al más nuevo. Ahora imagine que 0 termina su temporización, se transmite 5 (un

marco nuevo), 1 termina su temporización, 2 termina su temporización y se transmite 6 (otro marco nuevo). En este punto, la lista de marcos pendientes es 3405126, del más viejo al más nuevo. Si todo el tráfico de entrada se pierde durante un rato, los siete marcos pendientes terminarán su temporización en ese orden. Para evitar que el ejemplo se complique aún más, no hemos mostrado la administración de la temporización. En cambio, suponemos que la variable *oldest_frame* (marco más viejo) se establece en el momento de terminación de la temporización para indicar el marco cuya temporización ha terminado.

3.5. ESPECIFICACIÓN Y VERIFICACIÓN DE LOS PROTOCOLOS

Los protocolos realistas, y los programas que los implementan, con frecuencia son complicados. En consecuencia, se requiere mucha investigación para encontrar técnicas matemáticas formales con las cuales especificar y verificar los protocolos. En las secciones siguientes veremos algunos modelos y técnicas. Aunque estamos estudiándolos en el contexto de la capa de enlace de datos, también son aplicables a otras capas.

3.5.1. Modelos de máquina de estado finito

Un concepto clave empleado en muchos modelos de protocolos es el de la **máquina de estado finito**. Con esta técnica, cada **máquina de protocolo** (es decir, transmisor o receptor) siempre está en un estado específico en cualquier instante. Su estado consiste en todos los valores de sus variables, incluido el contador de programa.

En la mayoría de los casos puede agruparse un gran número de estados a fin de analizarlos. Por ejemplo, considerando el receptor del protocolo 3, podemos abstraer dos estados importantes de todos los posibles: la espera del marco 0 y la espera del marco 1. Todos los demás estados pueden considerarse como transitorios: pasos en el camino hacia uno de los estados principales. Típicamente, los estados se escogen como aquellos instantes en que la máquina de protocolo está esperando que ocurra el siguiente evento [es decir, la ejecución de la llamada de procedimiento *wait*(*event*) en nuestros ejemplos]. En este punto, el estado de la máquina de protocolo está determinado por completo por los estados de sus variables. El número de estados es entonces 2^n, donde n es el número de bits necesarios para representar todas las variables combinadas.

El estado del sistema completo es la combinación de todos los estados de las dos máquinas de protocolo y del canal. El estado del canal está determinado por su contenido. Usando nuevamente el protocolo 3 como ejemplo, el canal tiene cuatro posibles estados: un marco cero o un marco uno viajando del transmisor al receptor, un marco de acuse que va en el otro sentido o un canal vacío. Si modelamos el transmisor y el receptor como si ambos tuvieran dos estados, el sistema completo tiene 16 estados diferentes.

Vale la pena decir algo sobre el estado del canal. El concepto de un marco que está "en el canal" es, por supuesto, una abstracción. Lo que queremos decir en realidad es que un marco ha sido transmitido parcialmente, recibido parcialmente, pero aún no ha sido procesado en el

destino. Un marco permanece "en el canal" hasta que la máquina de protocolo ejecuta *FromPhysicalLayer* y lo procesa.

De cada estado hay cero o más **transiciones** posibles a otros estados. Las transiciones ocurren cuando sucede algún evento. Para una máquina de protocolo, podría ocurrir una transición al enviar un marco, al llegar un marco, al terminar un temporizador, al ocurrir una interrupción, etc. Para el canal, los eventos típicos son la inserción de un marco nuevo en el canal por una máquina de protocolo, la entrega de un marco a una máquina de protocolo o la pérdida de un marco debido a una ráfaga de ruido. Dada una descripción completa de las máquinas de protocolo y de las características del canal, es posible dibujar un grafo dirigido que muestre todos los estados como nodos y las transiciones como arcos dirigidos.

A un estado en particular se le designa **estado inicial**. Este estado corresponde a la descripción del sistema cuando comienza a funcionar, o en algún punto conveniente poco después. Desde el estado inicial pueden alcanzarse algunos, o quizá todos, los demás estados mediante una secuencia de transiciones. Usando técnicas bien conocidas de la teoría de grafos (por ejemplo, calculando el cierre transitorio de una gráfica), es posible determinar los estados que son alcanzables y los que no. Esta técnica es llamada **análisis de asequibilidad** (Lin *et al.*, 1987). Este análisis puede ser útil para determinar si un protocolo es correcto o no.

Formalmente, se puede considerar un modelo de máquina de estado finito como un cuádruple (*S, M, I, T*) donde:

S es el conjunto de estados en que pueden estar los procesos y el canal.

M es el conjunto de marcos que pueden intercambiarse a través del canal.

I es conjunto de estados iniciales de los procesos.

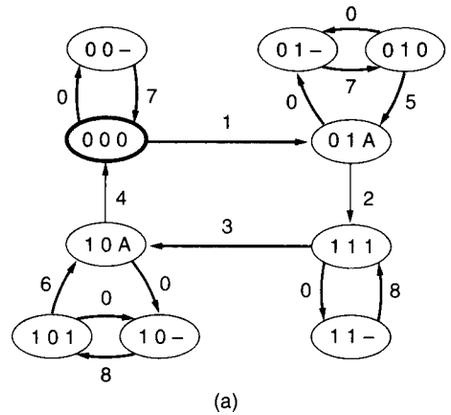
T es el conjunto de transiciones entre los estados.

Al principio todos los procesos están en sus estados iniciales. Entonces comienzan a ocurrir eventos, como marcos que se vuelven disponibles para transmisión o temporizadores que terminan. Cada evento puede causar que uno de los procesos o el canal emprenda una acción y cambie a un estado nuevo. Enumerando cuidadosamente cada sucesor posible para cada estado, podemos construir el grafo de asequibilidad y analizar el protocolo.

El análisis de asequibilidad puede servir para detectar una variedad de errores en la especificación del protocolo. Por ejemplo, si es posible que ocurra cierto marco en cierto estado y la máquina de estado finito no indica la acción a tomar, la especificación está en error (incompleta). Si existe un grupo de estados para los que no hay salida y de los que no se puede avanzar (recepción de marcos correctos), tenemos otro error (interbloqueo). Un error menos grave es una especificación de protocolo que indica la manera de manejar un evento en un estado en que el evento no puede ocurrir (transición ajena). También pueden detectarse otros errores.

Como ejemplo de modelo de máquina de estado finito, considere el lector la figura 3-20(a). Este grafo corresponde al protocolo 3, como se describió antes: cada máquina de protocolo tiene

dos estados y el canal tiene cuatro estados. Hay un total de 16 estados, no todos ellos alcanzables desde el inicial. Los inalcanzables no se muestran en la figura. Cada estado se etiqueta con tres caracteres, *XYZ*, donde *X* es 0 o 1, correspondiente al marco que el transmisor está tratando de enviar; *Y* también es 0 o 1, correspondiente al marco que espera el receptor, y *Z* es 0, 1, *A* o vacío (–), correspondiente al estado del canal. En este ejemplo, el estado inicial se ha escogido como (000). En otras palabras, el transmisor acaba de enviar el marco 0, el receptor espera el marco 0, y el marco 0 está actualmente en el canal.



(a)

Transición	¿Quién opera?	Marco aceptado	Marco emitido	A capa de red
		(marco perdido)		
0	—			—
1	R	0	A	Sí
2	T	A	1	—
3	R	1	A	Sí
4	T	A	0	—
5	R	0	A	No
6	R	1	A	No
7	T	(terminación)	0	—
8	T	(terminación)	1	—

(b)

Figura 3-20. (a) Diagrama de estados para el protocolo 3. (b) Transiciones.

Se muestran nueve tipos de transiciones en la figura 3-20. La transición 0 consiste en la pérdida del contenido del canal. La transición 1 consiste en la entrega correcta del paquete 0 al receptor, cambiando entonces el receptor su estado para esperar el marco 1 y emitir un acuse. La transición 1 también corresponde a que el receptor entrega el paquete 0 a la capa de red. Las otras transiciones se listan en la figura 3-20(b). La llegada de un marco con un error de suma de comprobación no se ha mostrado, pues no cambia el estado (en el protocolo 3).

Durante la operación normal, las transiciones 1, 2, 3 y 4 se repiten en orden una y otra vez. En cada ciclo se entregan dos paquetes, regresando el transmisor al estado original de tratar de enviar un marco nuevo con un número de secuencia 0. Si el canal pierde el marco 0, hace una transición del estado (000) al estado (00–). En algún momento, el transmisor termina de temporizar (transición 7) y el sistema regresa a (000). La pérdida de un acuse es más complicada, pues requiere dos transiciones, 7 y 5 u 8 y 6, para reparar el daño.

Una de las propiedades que debe tener un protocolo con un número de secuencia de 1 bit es que, sin importar la secuencia de eventos que ocurra, el receptor nunca debe entregar dos paquetes impares sin haber intervenido un paquete par, y al revés. Del grafo de la figura 3-20 podemos ver que este requisito puede establecerse más formalmente como "no debe existir

ninguna trayectoria desde el estado inicial en la que sucedan dos transiciones 1 sin que ocurra una transición 3 entre ellas, o al revés". En la figura se puede ver que el protocolo es correcto en este aspecto.

Otro requisito semejante es que no debe haber trayectorias en las que el transmisor pueda cambiar de estado dos veces (por ejemplo, de 0 a 1 y de regreso a 0) mientras el estado del receptor permanezca constante. De existir tal trayectoria, en la secuencia correspondiente de eventos se perderían dos marcos sin posibilidad de recuperación y sin que el receptor se diera cuenta. La secuencia de paquetes entregados tendrá un hueco no detectado de dos paquetes.

Otra propiedad importante de un protocolo es la ausencia de interbloqueos. Un **interbloqueo** es una situación en la que el protocolo no puede seguir avanzando (es decir, entregando paquetes a la capa de red), sea cual sea la secuencia de eventos que ocurra. En los términos del modelo gráfico, un interbloqueo se caracteriza por la existencia de un subconjunto de estados que es alcanzable desde el estado inicial y que tiene dos propiedades:

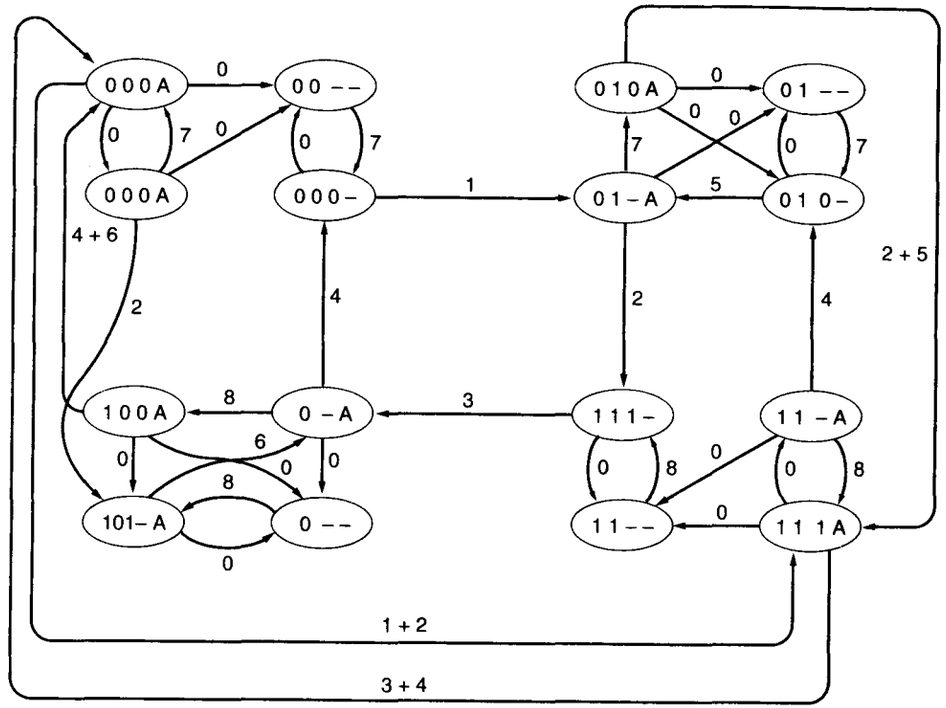
1. No hay transición hacia fuera del subconjunto.
2. No hay transiciones en el subconjunto que causen un avance.

Una vez en el estado de interbloqueo, el protocolo permanece ahí eternamente. De nuevo, es fácil ver en el grafo que el protocolo 3 no sufre de interbloqueos.

Ahora consideremos una variación del protocolo 3 en la que el canal semidúplex es reemplazado por un canal dúplex integral. En la figura 3-21 mostramos los estados como el producto de los estados de las dos máquinas de protocolo y los estados de los dos canales. Note que el canal directo ahora tiene tres estados: marco 0, marco 1 o vacío, y que el canal en reversa tiene dos estados, *A* o vacío. Las transiciones son las mismas que en la figura 3-20(b), excepto que cuando un marco de datos y un acuse están en el canal simultáneamente, hay una pequeña peculiaridad. El receptor no puede remover el marco de datos por sí mismo, pues eso implicaría tener dos acuses en el canal en el mismo momento, algo que no se permite en nuestro modelo (aunque es fácil diseñar un modelo que lo permita). De manera semejante, el transmisor no puede remover el acuse, porque eso implicaría emitir un segundo marco de datos antes de que el primero haya sido aceptado. En consecuencia, ambos eventos deben ocurrir simultáneamente, por ejemplo, la transición entre el estado (000A) y el estado (111A), etiquetado como 1 + 2 en la figura.

En la figura 3-21(a) hay trayectorias que hacen que el protocolo falle. En particular, hay trayectorias en las que el transmisor obtiene repetidamente paquetes nuevos, aun cuando los previos no hayan sido entregados correctamente. El problema surge porque ahora es posible que el transmisor termine de temporizar y envíe un marco nuevo sin alterar el acuse en el canal en reversa. Al llegar este acuse, se le considerará equivocadamente como aplicable a la transmisión actual, y no a la previa.

En la figura 3-21(b) se muestra una secuencia de estados que hace que el protocolo falle. En el cuarto y sexto estado de esta secuencia el transmisor cambia de estado, indicando que obtiene



(a)

(0 0 0 –), (0 1 – A), (0 1 0 A), (1 1 1 A), (1 1 – A), (0 1 0 –), (0 1 – A), (1 1 1 –)

(b)

Figura 3-21. (a) Grafo de estados para el protocolo 3 y un canal dúplex integral. (b) Secuencia de estados que causan la falla del protocolo.

un paquete nuevo de la capa de red, mientras que el receptor no cambia de estado, es decir, no entrega paquetes a la capa de red.

3.5.2. Modelos de red de Petri

La máquina de estado finito no es la única técnica para especificar protocolos formalmente. En esta sección describiremos otra técnica, la **red de Petri** (Danthine, 1980). Una red de Petri tiene cuatro elementos básicos: lugares, transiciones, arcos y señales. Un **lugar** representa un estado en el que puede estar (parte del) sistema. En la figura 3-22 se muestra una red de Petri con dos lugares, *A* y *B*, que aparecen como círculos. El sistema actualmente está en el estado *A*, indicado

por la **señal** (punto grueso) en el lugar *A*. Se indica una **transición** mediante una barra horizontal o vertical. Cada transición tiene cero o más **arcos de entrada**, que llegan de sus lugares de entrada, y cero o más **arcos de salida**, que van a sus lugares de salida.

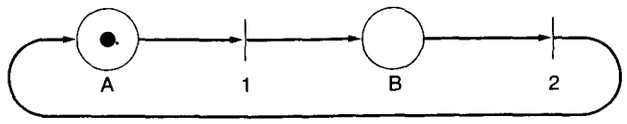


Figura 3-22. Red de Petri con dos lugares y dos transiciones.

Se **habilita** una transición si hay cuando menos una señal de entrada en cada uno de sus lugares de entrada. Cualquier transición habilitada puede **dispararse** a voluntad, quitando una señal de cada lugar de entrada y depositando una señal en cada lugar de salida. Si es diferente el número de arcos de entrada y de salida no se conservarán las señales. Si se habilitan dos o más transiciones, cualquiera de ellas puede dispararse. La decisión de disparo de una transición es indeterminada, por lo que las redes de Petri son útiles para modelar protocolos. La red de Petri de la figura 3-22 es determinista y puede servir para modelar cualquier proceso de dos fases (por ejemplo, el comportamiento de un bebé: come, duerme, come, duerme, y así sucesivamente). Como con todas las herramientas de modelado, se omiten los detalles innecesarios.

En la figura 3-23 se presenta el modelo de red de Petri de la figura 3-21. A diferencia del modelo de máquina de estado finito, aquí no hay estados compuestos; el estado del transmisor, el estado del canal y el estado del receptor se representan por separado. Las transiciones 1 y 2 corresponden al envío del marco 0 por el transmisor, normalmente y al momento de una terminación de temporización, respectivamente. Las transiciones 3 y 4 son análogas para el marco 1. Las transiciones 5, 6 y 7 corresponden a la pérdida del marco 0, de un acuse y del marco 1, respectivamente. Las transiciones 8 y 9 ocurren cuando un marco de datos con un número de secuencia equivocado llega al receptor. Las transiciones 10 y 11 representan la llegada al receptor del siguiente marco en la secuencia y su entrega a la capa de red.

Las redes de Petri pueden servir para detectar fallas de protocolo de una manera parecida a como se hace con máquinas de estado finito. Por ejemplo, si alguna secuencia de disparo incluyera la transición 10 dos veces sin interponerse la transición 11, el protocolo sería incorrecto. El concepto de interbloqueo en una red de Petri es semejante a su contraparte en una máquina de estado finito.

Las redes de Petri pueden representarse convenientemente en una forma algebraica semejante a una gramática. Cada transición contribuye una regla a la gramática. Cada regla especifica lugares de entrada y salida de la transición. Por ejemplo, la transición 1 de la figura 3-23 es $BD \rightarrow AC$. El estado actual de la red de Petri se representa como un conjunto de lugares sin orden, representándose cada lugar en el conjunto tantas veces como tenga señales. Cualquier regla que tenga presentes todos sus lugares del lado izquierdo puede dispararse, removiendo esos lugares del estado actual y agregando sus lugares de salida al estado actual. Las marcas de la figura 3-23 son *ACG*, por lo que puede aplicarse la regla 10 ($CG \rightarrow DF$), pero la regla 3 ($AD \rightarrow BE$) no puede aplicarse.

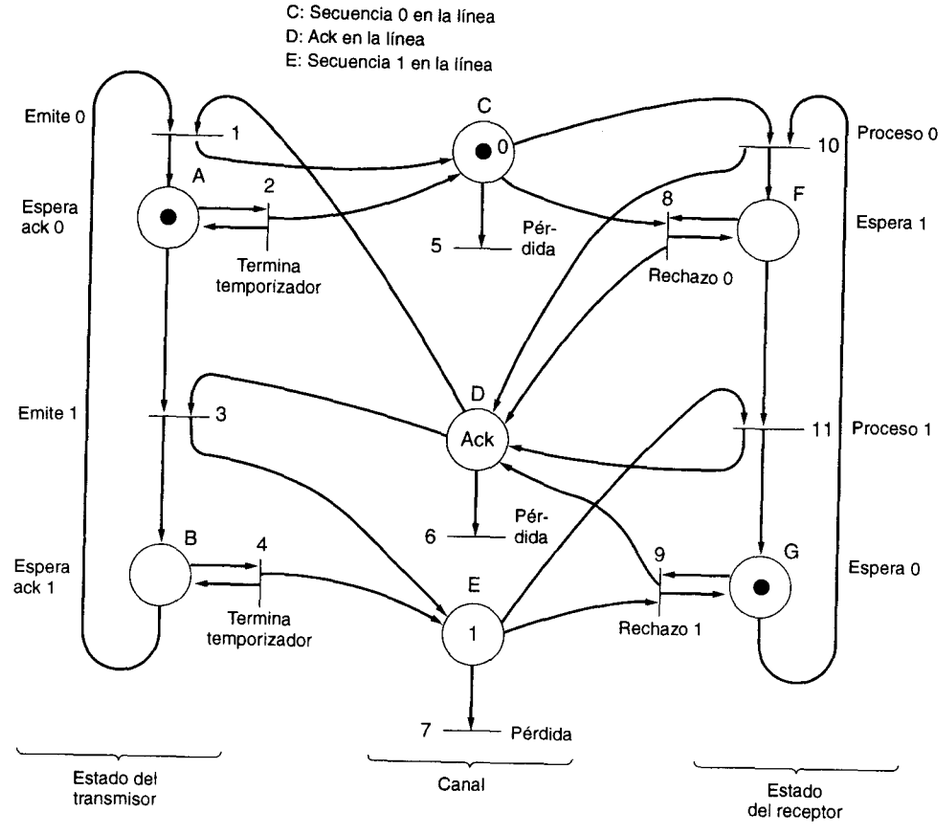


Figura 3-23. Modelo de red de Petri para el protocolo 3.

3.6. EJEMPLOS DE PROTOCOLOS DE ENLACE DE DATOS

En las siguientes secciones examinaremos varios protocolos de enlace de datos de amplio uso. El primero, HDLC, es común en X.25 y muchas otras redes. Después, examinaremos protocolos de enlace de datos usados en Internet y en las redes ATM, respectivamente. En capítulos subsecuentes también usaremos Internet y ATM como ejemplos reales.

3.6.1. HDLC — Control de enlace de datos de alto nivel

En esta sección examinaremos un grupo de protocolos íntimamente relacionados que, a pesar de ser un poco viejos, se siguen utilizando ampliamente en redes de todo el mundo. Todas se derivan del protocolo de enlace de datos usado en la SNA de IBM, llamado **SDLC** (*Synchronous Data Link Control protocol*, **protocolo de control sincrónico de enlace de datos**). Tras desarrollar SDLC, IBM lo sometió a ANSI y a ISO para su aceptación como estándar de Estados Unidos

e internacional, respectivamente. ANSI lo modificó convirtiéndolo en **ADCCP** (*Advanced Data Communication Control Procedure*, **procedimiento avanzado de control de comunicación de datos**), e ISO lo modificó para convertirlo en **HDLC** (*High-level Data Link Control*, **control de enlace de datos de alto nivel**). Luego, el CCITT adoptó y modificó HDLC para su **LAP** (*Link Access Procedure*, **procedimiento de acceso de enlace**) como parte del estándar de interfaz de red X.25, pero después lo modificó nuevamente a **LAPB** para hacerlo más compatible con una versión posterior de HDLC. Lo agradable de los estándares es que hay tantos de donde escoger. Es más, si no le gusta ninguno de ellos, simplemente puede sentarse a esperar el modelo del año próximo.

Todos estos protocolos se basan en el mismo principio. Todos están orientados a bits y usan el relleno de bits para lograr la transparencia de los datos; difieren sólo en aspectos menores, aunque irritantes. El análisis de protocolos orientados a bits que haremos a continuación pretende ser una introducción general. Si desea los detalles específicos de cualquier protocolo, consulte la definición adecuada.

Todos los protocolos orientados a bits usan la estructura de marco mostrada en la figura 3-24. El campo de *dirección* es de importancia primordial en las líneas con múltiples terminales, pues sirve para identificar una de las terminales. En líneas punto a punto a veces se usa para distinguir los comandos de las respuestas.

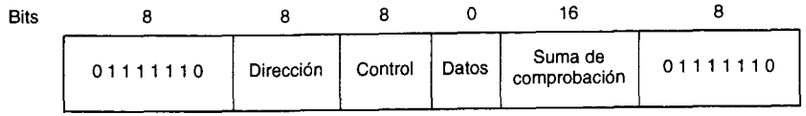


Figura 3-24. Formato de marco para protocolos orientados a bits.

El campo de *control* se usa para números de secuencia, acuses y otros propósitos, como se explicará posteriormente.

El campo de *datos* puede contener información arbitraria; puede ser de longitud arbitraria, aunque la eficiencia de la suma de comprobación decae al aumentar el tamaño del marco, debido a la mayor probabilidad de múltiples errores en ráfaga.

El campo de *suma de comprobación* es una variación menor del bien conocido código de redundancia cíclica, usando CRC-CCITT como generador de polinomios. La variación consiste en permitir la detección de bytes de indicación perdidos.

El marco está delimitado por otra secuencia de indicación (01111110). En líneas punto a punto inactivas se transmiten secuencias de indicación continuamente. El marco mínimo contiene tres campos y un total de 32 bits, excluyendo los indicadores a ambos lados.

Hay tres tipos de marcos: **de información**, **de supervisión** y **no numerados**. El contenido del campo de *control* para estos tres tipos se muestra en la figura 3-25. El protocolo emplea una ventana corrediza, con un número de secuencia de 3 bits. En cualquier momento pueden estar pendientes hasta siete marcos no reconocidos. El campo *secuencia* en la figura 3-25(a) es el número de secuencia del marco. El campo *siguiente* es un acuse incorporado. Sin embargo, todos

los protocolos se apegan a la convención de que, en lugar de incorporar el número del último marco recibido correctamente, usan el número del primer marco no recibido (es decir, el siguiente marco esperado). La decisión de usar el último marco recibido o el siguiente marco esperado es arbitraria; no importa la convención que se use, siempre y cuando se use con consistencia.

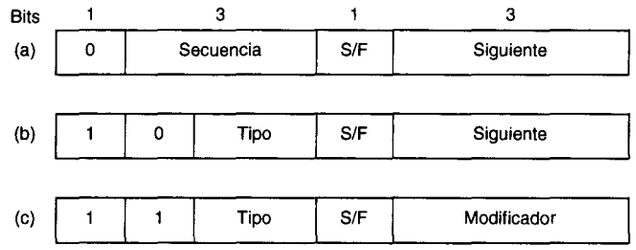


Figura 3-25. Campo de control de (a) un marco de información, (b) un marco de supervisión, (c) un marco sin número.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

El bit *S/F* significa *Sondeo/Final*. Se usa cuando una computadora (o concentrador) está sondeando un grupo de terminales. Cuando se usa como *S*, la computadora está invitando a la terminal a enviar datos. Todos los marcos enviados por la terminal, excepto el último, tienen el bit *S/F* puesto en *S*. El último se pone en *F*.

En algunos de los protocolos el bit *S/F* sirve para obligar a la otra máquina a enviar de inmediato un marco de supervisión, en lugar de esperar tráfico en reversa al cual incorporar la información de la ventana. El bit también tiene algunos usos menores en conexión con los marcos sin número.

Los diferentes tipos de marcos de supervisión se distinguen por el campo de *tipo*. El tipo 0 es un marco de acuse (oficialmente llamado RECEIVE READY, receptor listo) que sirve para indicar el siguiente marco esperado. Este marco se usa cuando no hay tráfico en reversa que se pueda aprovechar para incorporar acuses.

El tipo 1 es un marco de acuse negativo (oficialmente llamado REJECT, rechazo); sirve para indicar que se ha detectado un error de transmisión. El campo *siguiente* indica el primer marco en la secuencia que no se ha recibido correctamente (es decir, el marco a retransmitir). Se pide al transmisor retransmitir todos los marcos pendientes comenzando por *siguiente*. Esta estrategia es semejante a la de nuestro protocolo 5, más que a la de nuestro protocolo 6.

El tipo 2 es RECEIVE NOT READY (receptor no listo); reconoce todos los marcos hasta, pero sin incluir, *siguiente*, al igual que RECEIVE READY, pero le dice al transmisor que detenga el envío. El propósito de RECEIVE NOT READY es señalar ciertos problemas temporales en el receptor, como falta de *buffers*, y no servir como alternativa del control de flujo de ventana corrediza. Cuando el problema se resuelve, el receptor envía RECEIVE READY, REJECT o ciertos marcos de control.

El tipo 3 es SELECTIVE REJECT (rechazo selectivo); solicita la retransmisión de sólo el marco especificado. En este sentido es como nuestro protocolo 6, no como el 5, y por tanto es de mayor utilidad cuando el tamaño de la ventana del transmisor es la mitad del tamaño del espacio

secuencial, o menor. Por tanto, si un receptor desea colocar en *buffers* una secuencia de marcos para un potencial uso futuro, puede forzar la retransmisión de cualquier marco específico usando *Selective Reject*. HDLC y ADCCP permiten este tipo de marco, pero SDLC y LAPB no lo permiten (es decir, no hay rechazo selectivo), y los marcos de tipo 3 no están definidos.

La tercera clase de marco es el marco sin número que a veces se usa para propósitos de control, aunque también puede servir para llevar datos cuando se solicita un servicio no confiable sin conexión. Los diferentes protocolos orientados a bits tienen diferencias considerables aquí, en contraste con los otros dos tipos, donde son casi idénticos. Hay cinco bits disponibles para indicar el tipo de marco enviado, pero no se usan las 32 posibilidades.

Todos los protocolos proporcionan un comando, DISC (*DISConnect*, desconexión), que permite a una máquina anunciar que va a ser desactivada (por ejemplo, para mantenimiento preventivo). También cuentan con un comando que permite a una máquina que acaba de regresar y está en línea anunciar su presencia y obligar el regreso a cero de todos los números de secuencia. Este comando es SNRM (*Set Normal Response Mode*, establecer modo de respuesta normal). Desgraciadamente, el "modo de respuesta normal" es todo menos normal. Es un modo desbalanceado (es decir, asimétrico) en el que un extremo de la línea es el maestro y otro el subordinado. SNRM se remonta a una época en la que "comunicación de datos" significaba una terminal no inteligente que se comunicaba con una computadora, lo cual evidentemente es asimétrico. Para hacer más adecuado el protocolo cuando los dos interlocutores son iguales, HDLC y LAPB cuentan con un comando adicional, SABM (*Set Asynchronous Balanced Mode*, establecer modo asíncrono balanceado), que restablece la línea y declara que ambas partes son iguales. También cuentan con los comandos SABME y SNRME, que son iguales a SABM y a SNRM, respectivamente, excepto que habilitan un formato de marco extendido que maneja números de secuencia de 7 bits en lugar de 3 bits.

Un tercer comando proporcionado por todos los protocolos es FRMR (*FRaMe Reject*, rechazo de marco), que sirve para indicar que ha llegado un marco con suma de comprobación correcta pero semántica imposible). Ejemplos de semántica imposible son: un marco de supervisión tipo 3 en LAPB, un marco más corto de 32 bits, un marco de control ilegal, el acuse de un marco que estaba fuera de la ventana, etc. Los marcos FRMR contienen un campo de datos de 24 bits que indica por qué se rechazó el marco. Los datos incluyen el campo de control del marco rechazado, los parámetros de la ventana y un conjunto de bits que señalan errores específicos.

Los marcos de control pueden perderse o dañarse, igual que los de datos, por lo que deben ser reconocidos también. Se proporciona un marco de control especial para este propósito, llamado UA (*Unnumbered Acknowledgment*, acuse sin número). Dado que sólo puede estar pendiente un marco de control, nunca hay ambigüedades sobre el marco de control que está siendo reconocido.

Los marcos de control restantes tienen que ver con inicialización, sondeo e informe de estado. También hay un marco de control que puede contener información arbitraria, UI (*Unnumbered Information*, información sin número). Estos datos no se pasan a la capa de red, pues son para la capa de enlace de datos misma.

A pesar de su uso extendido, HDLC está lejos de ser perfecto. En (Fiorini *et al.*, 1995) puede encontrarse un análisis de los distintos problemas asociados a él.

3.6.2. La capa de enlace de datos en Internet

Internet consiste en máquinas individuales (*host* y enrutadores) y la infraestructura de comunicación que las conecta. Dentro de un solo edificio, las LAN se usan ampliamente para la interconexión, pero la mayor parte de la infraestructura de área extensa está construida a partir de líneas arrendadas punto a punto. En el capítulo 4 veremos las LAN; aquí examinaremos los protocolos de enlace de datos usados en las líneas punto a punto de Internet.

En la práctica, la comunicación punto a punto se utiliza principalmente en dos situaciones. Primero, miles de organizaciones tienen una o más LAN, cada una con cierta cantidad de *hosts* (computadoras personales, estaciones de trabajo, servidores y otros) junto con un enrútador o *router* (o un puente, que funcionalmente es parecido). Con frecuencia, los enrutadores se interconectan mediante una LAN principal, o *backbone*. Típicamente, todas las conexiones al mundo exterior pasan a través de uno o dos enrutadores que tienen líneas arrendadas punto a punto a enrutadores distantes. Son estos enrutadores y sus líneas arrendadas los que conforman las subredes de comunicación sobre las que está construida Internet.

La segunda situación en la que las líneas punto a punto desempeñan un papel principal en Internet son los millones de personas que tienen conexiones caseras a Internet a través de módems y líneas telefónicas de discado. Generalmente lo que ocurre es que la PC casera del usuario llama a un **proveedor de Internet**, que es una compañía comercial como America Online, CompuServe y Microsoft Network, o una universidad o compañía que suministra conectividad casera a Internet para sus estudiantes y empleados. A veces la PC casera sólo funciona como terminal orientada a caracteres conectada al sistema de tiempo compartido del proveedor del servicio Internet. En esta modalidad, el usuario puede ingresar comandos y ejecutar programas, pero los servicios gráficos de Internet, como la *World Wide Web*, no están disponibles. Esta manera de trabajar se llama **cuenta de armazón**.

Como alternativa, la PC casera puede llamar al enrutador del proveedor del servicio Internet y entonces actuar como *host* de Internet. Este método de operación no es distinto a tener una línea arrendada entre la PC y el enrutador, excepto que la conexión desaparece cuando el usuario termina la sesión. Con este enfoque están disponibles todos los servicios de Internet, incluidos los gráficos. En la figura 3-26 se ilustra una PC casera llamando al proveedor del servicio Internet.

Tanto para la conexión por línea arrendada de enrutador a enrutador como para la conexión conmutada de *host* a enrutador, se requiere de un protocolo punto a punto de enlace de datos en la línea para manejo de marcos, control de errores y las demás funciones de la capa de enlace de datos que hemos estudiado en este capítulo. Dos protocolos de este tipo se usan ampliamente en Internet, SLIP y PPP. Los estudiaremos ahora por turno.

SLIP — IP de línea en serie

SLIP es el más viejo de los protocolos; fue diseñado por Rick Adams en 1984 para conectar estaciones de trabajo Sun a Internet a través de una línea de discado usando un módem. El protocolo, descrito en RFC 1055, es muy sencillo. La estación sólo envía paquetes IP en bruto a

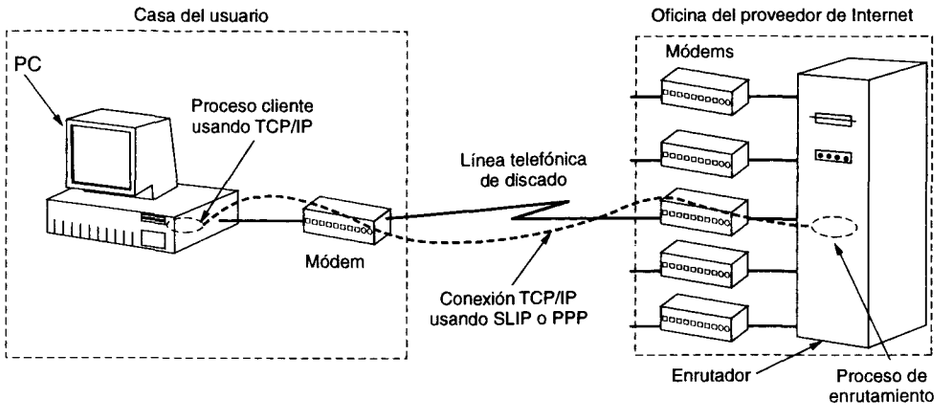


Figura 3-26. Computadora personal casera funcionando como *host* de Internet.

través de la línea, con un byte indicador especial (0xC0) al final para delimitar el marco. Si el byte indicador ocurre dentro del paquete IP, se usa una forma de relleno de caracteres, enviándose la secuencia de dos bytes (0xDB, 0xDC) en su lugar. Si ocurre 0xDB dentro del paquete IP, también se rellena. Algunas implementaciones de SLIP agregan un byte indicador tanto al principio como al final de cada paquete IP enviado.

La versiones más recientes de SLIP efectúan cierta compresión de encabezados TCP e IP. Lo que hacen es aprovechar el hecho de que los paquetes consecutivos con frecuencia tienen muchos campos de encabezado en común; se comprimen omitiendo aquellos campos que son iguales a los campos correspondientes del paquete IP previo. Es más, los campos que sí difieren no son enviados en su totalidad, sino como incrementos del valor previo. Estas optimizaciones se describen en RFC 1144.

Aunque aún se utiliza ampliamente, SLIP tiene algunos problemas serios. Primero, SLIP no efectúa detección o corrección de errores, por lo que es responsabilidad de las capas superiores detectar y recuperar marcos perdidos, dañados o fusionados.

En segundo lugar, SLIP sólo reconoce IP. Con el crecimiento de Internet para abarcar redes que no usan IP como su lenguaje nativo (por ejemplo, las LAN Novell), esta restricción se vuelve cada vez más grave.

Tercero, cada lado debe conocer por adelantado la dirección IP del otro; ninguna de las dos direcciones puede asignarse dinámicamente durante el establecimiento del enlace. Dada la escasez actual de direcciones IP, esta limitación es un problema importante, pues es imposible dar a cada usuario casero de Internet una dirección única de IP.

Cuarto, SLIP no proporciona ninguna forma de verificación de autenticidad, por lo que ninguna parte sabe realmente con quién está hablando. Con líneas arrendadas éste no es un problema, pero con líneas de discado sí lo es.

Quinto, SLIP no es un estándar aprobado de Internet, por lo que existen muchas versiones diferentes (e incompatibles). Esta situación no simplifica la interconexión.

PPP — Protocolo punto a punto

Para mejorar la situación, el IETF estableció un grupo dedicado a diseñar un protocolo de enlace de datos para líneas punto a punto que resolviera todos estos problemas y que pudiera volverse un estándar oficial de Internet. Este trabajo culminó con el **PPP** (*Point-to-Point Protocol*, **protocolo punto a punto**), que se define en RFC 1661 y que se ha desarrollado más en varios otros RFC (por ejemplo, los RFC 1662 y 1663). El PPP realiza detección de errores, reconoce múltiples protocolos, permite la negociación de direcciones de IP en el momento de la conexión, permite la verificación de autenticidad y tiene muchas mejoras respecto a SLIP. Aunque muchos proveedores de servicios de Internet aún reconocen tanto SLIP como PPP, el futuro claramente está en PPP, no sólo en las líneas por discado sino también en las líneas arrendadas de enrutador a enrutador.

PPP proporciona tres cosas:

1. Un método de enmarcado que delinea sin ambigüedades el final de un marco y el inicio del siguiente. El formato de marco también maneja la detección de errores.
2. Un protocolo de control de enlace para activar líneas, probarlas, negociar opciones y desactivarlas ordenadamente cuando ya no son necesarias. Este protocolo se llama **LCP** (*Link Control Protocol*, **protocolo de control de enlace**).
3. Un mecanismo para negociar opciones de capa de red con independencia del protocolo de red usado. El método escogido consiste en tener un **NCP** (*Network Control Protocol*, **protocolo de control de red**) distinto para cada capa de red reconocida.

Para ver la manera en que encajan estas piezas, consideremos la situación típica de un usuario casero llamando al proveedor de servicios de Internet para convertir una PC casera en un *host* temporal de Internet. La PC llama inicialmente al enrutador del proveedor a través de un módem. Una vez que el módem del enrutador ha contestado el teléfono y ha establecido una conexión física, la PC manda al enrutador una serie de paquetes LCP en el campo de carga útil de uno o más marcos PPP. Estos paquetes, y sus respuestas, seleccionan los parámetros PPP por usar.

Una vez que se han acordado estos parámetros, se envía una serie de paquetes NCP para configurar la capa de red. Típicamente, la PC quiere ejecutar una pila de protocolos TCP/IP, por lo que necesita una dirección de IP. No hay suficientes direcciones de IP para todos, por lo que normalmente cada proveedor de Internet tiene un bloque de ellas y asigna dinámicamente una a cada PC que se acaba de conectar para que la use durante su sesión. Si un proveedor posee n direcciones de IP, puede tener hasta n máquinas conectadas simultáneamente, pero su base total de clientes puede ser muchas veces mayor. Se utiliza el NCP para IP para asignar la dirección de IP.

En este momento, la PC es ya un *host* de Internet y puede enviar y recibir paquetes de IP, igual que los *host* permanentes. Cuando el usuario ha terminado, se usa NCP para desmantelar la conexión de la capa de red y liberar la dirección de IP. Luego se usa LCP para cancelar la cone-

xión de la capa de enlace de datos. Finalmente, la computadora indica al módem que cuelgue el teléfono, liberando la conexión de la capa física.

El formato de marco de PPP se escogió de modo que fuera muy parecido al formato de marco de HDLC, ya que no había razón para reinventar la rueda. La diferencia principal entre PPP y HDLC es que el primero está orientado a caracteres, no a bits. En particular, PPP, como SLIP, usa el relleno de caracteres en las líneas por discado con módem, por lo que todos los marcos tienen un número entero de bytes. No es posible enviar un marco que conste de 30.25 bytes, como con HDLC. No sólo pueden mandarse marcos PPP a través de líneas telefónicas de discado, sino que también pueden enviarse a través de SONET o de líneas HDLC auténticas orientadas a bits (por ejemplo, para conexiones enrutador-enrutador). El formato de marco PPP se muestra en la figura 3-27.

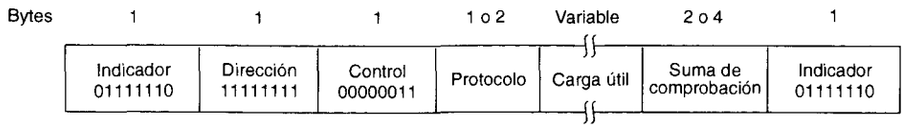


Figura 3-27. Formato de marco completo PPP para el modo operación sin numeración.

Todos los marcos PPP comienzan con el byte indicador estándar de HDLC (01111110), que se rellena si ocurre dentro del campo de carga útil. Luego viene el campo de *dirección*, al que siempre se asigna el valor binario 11111111 para indicar que todas las estaciones deben aceptar el marco. El empleo de este valor evita tener que asignar direcciones de enlace de datos.

El campo de *dirección* va seguido del campo de *control*, cuyo valor predeterminado es 00000011. Este valor indica un marco sin número. En otras palabras, PPP no proporciona por omisión transmisión confiable usando números de secuencia y acuses. En ambientes ruidosos, como los de las redes inalámbricas, se puede usar el modo numerado para transmisión confiable. Los detalles exactos se definen en RFC 1663.

Dado que los campos de *dirección* y de *control* son constantes en la configuración predeterminada, LCP proporciona los mecanismos necesarios para que las dos partes negocien una opción que simplemente los omita por completo y ahorre dos bytes por marco.

El cuarto campo PPP es el de *protocolo*. Su tarea es indicar la clase de paquete que está en el campo de *carga*. Se definen códigos para LCP, NCP, IP, IPX, AppleTalk y otros protocolos. Los protocolos que comienzan con un bit 0 son protocolos de capa de red como IP, IPX, OSI CLNP, XNS. Los que comienzan con un bit 1 se usan para negociar otros protocolos. Éstos incluyen LCP y un NCP diferente para cada protocolo de capa de red reconocido. El tamaño por omisión del campo de *protocolo* es de 2 bytes, pero puede negociarse su disminución a 1 byte usando LCP.

El campo de *carga útil* es de longitud variable, hasta algún máximo negociado. Si la longitud no se negocia con LCP durante el establecimiento de la línea, se usa una longitud predeterminada de 1500 bytes. De ser necesario se puede incluir un relleno después de la carga.

Después del campo de *carga útil* viene el campo de *suma de comprobación*, que normalmente es de 2 bytes, pero puede negociarse una suma de comprobación de 4 bytes.

En resumen, PPP es un mecanismo de enmarcado multiprotocolo adecuado para usarse a través de módems, líneas de serie de bits HDLC, SONET y otras capas físicas. Maneja detección de errores, negociación de opciones, compresión de encabezados y, opcionalmente, transmisión confiable con marcos HDLC.

Ahora pasemos del formato de los marcos PPP a la manera en que se activan y se desactivan las líneas. En el diagrama (simplificado) de la figura 3-28 se muestran las fases por las que pasa una línea cuando es activada, usada y desactivada. Esta secuencia se aplica tanto a las conexiones por módem como a las conexiones enrutador-enrutador.

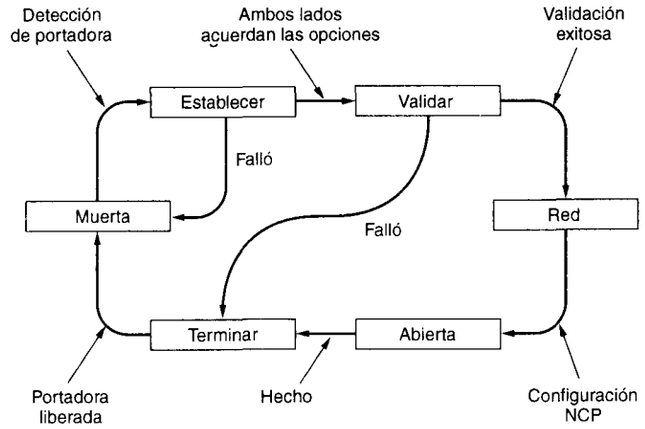


Figura 3-28. Diagrama de fase simplificado para activar y desactivar una línea.

Cuando la línea está *MUERTA*, no hay presencia de una portadora de capa física y no existe una conexión de capa física. Una vez establecida la conexión física, la línea pasa a *ESTABLECER*. En ese punto comienza la negociación de opciones LCP, que, de tener éxito, conduce a *VALIDAR*. Ahora las dos partes pueden verificar la identidad del otro, si lo desean. Al entrar en la fase de *RED* se invoca el protocolo NCP apropiado para configurar la capa de red. Si la configuración tiene éxito, se llega a *ABIERTA* y puede comenzar el transporte de datos. Al terminar el transporte, la línea pasa a la fase de *TERMINAR*, de donde regresa a *MUERTA* al desactivarse la portadora.

Se utiliza LCP para negociar las opciones del protocolo de enlace de datos durante la fase *ESTABLECER*. El protocolo LCP no se ocupa realmente de las opciones mismas, sino de los mecanismos de negociación; proporciona una vía para que el proceso iniciador haga una propuesta y para que el proceso que responde la acepte o la rechace, toda o en parte. También proporciona una vía para que los dos procesos prueben la calidad de la línea, y ver si consideran

que es lo suficientemente buena para establecer una conexión. Por último, el protocolo LCP también permite que las líneas se desactiven cuando ya no se necesitan.

Hay 11 tipos de paquetes LCP definidos en el RFC 1661; éstos se listan en la figura 3-29. Los cuatro tipos de *configuración* permiten que el iniciador (I) proponga valores de opción y que el contestador (R) las acepte o las rechace. En el último caso, el contestador puede hacer una propuesta alterna o anunciar que no está dispuesto a negociar las opciones en absoluto. Las opciones negociadas y sus valores propuestos son parte de los paquetes LCP.

Nombre	Dirección	Descripción
Solicitud-configuración	I → R	Lista de opciones y valores propuestos
Acuse-configuración	I ← R	Se aceptan todas las opciones
Acuse negativo-configuración	I ← R	No se aceptan algunas opciones
Rechazo-configuración	I ← R	Algunas opciones no son negociables
Solicitud-terminación	I → R	Solicitud de desactivación de la línea
Acuse-terminación	I ← R	Línea desactivada
Rechazo-código	I ← R	Se recibió una solicitud desconocida
Rechazo-protocolo	I ← R	Se solicitó un protocolo desconocido
Solicitud-eco	I → R	Favor de transmitir de regreso este marco
Respuesta-eco	I ← R	Aquí está el marco de regreso
Solicitud-descartar	I → R	Descartar este marco (para pruebas)

Figura 3-29. Los tipos de paquetes LCP.

Los códigos de *terminación* sirven para desactivar una línea cuando ya no se necesita. Los códigos de *rechazo-código* y *rechazo-protocolo* son usados por el contestador para indicar que recibió algo que no entiende. Esta situación puede significar que ha ocurrido un error de transmisión no detectado, pero más probablemente significa que el iniciador y el contestador están ejecutando versiones diferentes del protocolo LCP. Los códigos *eco* sirven para probar la calidad de la línea. Finalmente, *solicitud-descartar* se utiliza para depuración. Si cualquiera de los lados está teniendo problemas para poner bits en la línea, el programador puede usar este código para pruebas. Si logra pasar, el receptor simplemente lo descarta en lugar de realizar otra acción, lo que podría confundir a la persona que efectúa las pruebas.

Las opciones que pueden negociarse incluyen el establecimiento del tamaño máximo de la carga útil para los marcos de datos, la habilitación de la verificación de autenticidad y la selección del protocolo a usar, habilitando la supervisión de la calidad de la línea durante la operación normal y la selección de varias opciones de compresión de encabezados.

Hay poco que decir sobre los protocolos NCP en un sentido general. Cada uno es específico para algún protocolo de capa de red y permite que se hagan solicitudes de configuración específicas de ese protocolo. Para IP, por ejemplo, la posibilidad más importante es la asignación dinámica de direcciones.

3.6.3. La capa de enlace de datos en ATM

Ya es tiempo de comenzar nuestro viaje a través de las capas de protocolo ATM de la figura 1-30. La capa física de ATM cubre en términos generales las capas física y de enlace de datos del modelo OSI, siendo la subcapa dependiente del medio físico funcionalmente similar a la capa física de OSI y teniendo la subcapa de convergencia de transmisión (TC) la funcionalidad de enlace de datos. No hay características de capa física específicas de ATM. En su lugar, las células de ATM son transportadas por SONET, FDDI y otros sistemas de transmisión. Por tanto, nos concentraremos aquí en la funcionalidad de enlace de datos de la subcapa TC, pero estudiaremos posteriormente algunos aspectos de la interfaz con la subcapa inferior.

Cuando un programa de aplicación produce un mensaje que debe ser enviado, ese mensaje viaja hacia abajo a través de la pila de protocolos ATM, agregándosele encabezados y apéndices, y sufriendo segmentación en células. En algún momento, las células llegan a la subcapa TC para su transmisión. Veamos lo que pasa con ellas al salir por la puerta.

Transmisión de células

El primer paso es la suma de comprobación del encabezado. Cada célula contiene un encabezado de 5 bytes que consiste en 4 bytes de información del circuito virtual y de control seguidos de una suma de comprobación de 1 byte. Aunque el contenido del encabezado no es relevante para la subcapa TC, los lectores curiosos que deseen un avance deberán pasar a la figura 5-62. La suma de comprobación sólo cubre los primeros 4 bytes de encabezado, no el campo de carga útil; consiste en el residuo una vez que se han dividido los 32 bits de encabezado entre el polinomio $x^8 + x^2 + x + 1$. A esto se agrega la constante 01010101 para proporcionar robustez ante encabezados que contengan principalmente bits 0.

La decisión de obtener la suma de comprobación sólo del encabezado se tomó para reducir la probabilidad de entregar células incorrectamente debido a un error de encabezado, pero también para evitar pagar el precio de obtener la suma de comprobación del campo de carga útil, que es mucho más grande. Es asunto de las capas superiores el desempeñar esta función, si se desea. Para muchas aplicaciones de tiempo real, como voz y vídeo, es aceptable la pérdida de algunos bits de vez en cuando (aunque en algunos esquemas de compresión todos los marcos son iguales, si bien algunos son más iguales que otros). Debido a que cubre sólo el encabezado, el campo de suma de comprobación de 8 bits se denomina **HEC** (*Header Error Control*, **control de error de encabezado**).

Un factor que desempeñó un papel principal en este esquema de suma de comprobación es que el ATM fue diseñado para usarse en fibra, y la fibra es altamente confiable. Es más, un estudio extenso de la red telefónica de Estados Unidos ha demostrado que durante la operación normal el 99.64% de los errores en las líneas de fibra óptica son errores de un solo bit (AT&T y Bellcore, 1989). El esquema HEC corrige todos los errores de un bit y detecta también muchos

errores multibit. Si suponemos que la probabilidad de un error de 1 bit es de 10^{-8}, entonces la probabilidad de que una célula contenga un error multibit de encabezado es de alrededor de 10^{-13}. La probabilidad de que una célula pase con un error de encabezado no detectado es de aproximadamente 10^{-20}, lo que significa que, a velocidad OC-3, pasará una encabezado de célula errónea cada 90,000 años. Aunque esto puede parecer mucho tiempo, una vez que la Tierra tenga, digamos, 1000 millones de teléfonos ATM, utilizándose cada uno el 10% del tiempo, pasarán sin detección 1000 encabezados de célula errónea por año.

Para las aplicaciones que necesitan transmisión confiable en la capa de enlace de datos, Shacham y McKenney (1990) han desarrollado un esquema en el que a una secuencia de células consecutivas se aplica un OR EXCLUSIVO. El resultado, una célula completa, se agrega a la secuencia. Si se pierde o se daña gravemente una célula, puede reconstruirse a partir de la información disponible.

Una vez que se ha generado el HEC y se ha introducido en el encabezado de la célula, la célula está lista para transmitirse. Los medios de transmisión pertenecen a una de dos categorías: asíncronos y síncronos. Con un medio asíncrono se puede mandar una célula cuando esté lista para irse; no existen restricciones de tiempo.

En un medio síncrono las células deben transmitirse de acuerdo con un patrón de temporización predefinido. Si no hay una célula de datos disponible cuando se necesita, la subcapa TC debe inventar una. Éstas se llaman **células de relleno**.

Otro tipo de célula que no es de datos es la célula **OAM** (*Operation And Maintenance*, **operación y mantenimiento**). Las células OAM también son usadas por los conmutadores ATM para intercambiar información de control y otra información necesaria para mantener funcionando el sistema. Las células OAM también tienen algunas otras funciones especiales. Por ejemplo, la velocidad OC-3 de 155.52 Mbps equivale a la tasa de datos bruta de SONET, pero un marco STM-1 tiene 10 columnas de información de control de un total de 270, por lo que la carga útil de SONET es de sólo $260/270 \times 155.52$ Mbps = 149.76 Mbps. Para evitar inundar a SONET, una fuente ATM que use SONET normalmente enviará una célula OAM como cada 27a. célula, a fin de reducir la tasa de datos a 26/27 de 155.52 Mbps y, por tanto, igualar exactamente a SONET. La tarea de igualar la velocidad de salida del ATM a la velocidad del sistema de transmisión subyacente es una actividad importante de la subcapa TC.

Del lado del receptor, las células de relleno se procesan en la subcapa TC, pero las OAM se entregan a la capa ATM. Las células OAM se distinguen de las de datos por tener ceros en los tres bytes de encabezado, algo no permitido en las células de datos. El cuarto byte describe la naturaleza de la célula OAM.

Otra tarea importante de la subcapa TC es generar la información de enmarcado para el sistema de transmisión subyacente, si lo hay. Por ejemplo, una cámara de vídeo ATM podría producir una secuencia de células en la línea, pero también podría producir marcos SONET con células ATM intercaladas en la carga útil de SONET. En el último caso, la subcapa TC generaría los enmarcados SONET y empacaría las celdas ATM dentro de ellos, lo que no necesariamente es un asunto trivial pues una carga útil de SONET no contiene un número entero de células de 53 bytes.

Aunque las compañías telefónicas evidentemente piensan usar SONET como el sistema de transmisión subyacente para ATM, también se han definido correlaciones de ATM con los

campos de carga útil de otros sistemas, y se está trabajando en nuevas. En particular, existen correlaciones para T1, T3 y FDDI.

Recepción de células

En lo que respecta a la salida, la tarea de la subcapa TC es tomar una secuencia de células, agregarle un HEC a cada una, convertir el resultado en una corriente de bits, e igualar la corriente de bits con la velocidad de la transmisión física subyacente introduciendo células OAM como relleno. En la entrada, la subcapa TC hace exactamente lo inverso. Toma una corriente de bits de entrada, localiza los límites de las células, verifica los encabezados (descartando las células con encabezados no válidos), procesa las células OAM y pasa las células de datos a la capa ATM.

La parte más difícil es localizar los límites de las células en la corriente de bits de entrada. A nivel de bits, una célula es sólo una secuencia de $53 \times 8 = 424$ bits. No hay bytes 01111110 indicadores presentes para marcar el inicio y el final de una célula, como en HDLC. De hecho, no hay marcadores. ¿Cómo pueden reconocerse los límites de una célula en estas circunstancias?

En algunos casos, la capa física ayuda. En SONET, por ejemplo, se pueden alinear las células con la envoltura síncrona de carga útil, por lo que el apuntador SPE del encabezado de SONET apunta al comienzo de la primera célula completa. Sin embargo, a veces la capa física no proporciona ninguna ayuda para el enmarcado. ¿Entonces qué?

El truco es usar el HEC. A medida que entran los bits, la subcapa TC mantiene un registro de desplazamiento de 40 bits, entrando los bits por la izquierda y saliendo por la derecha. La subcapa TC inspecciona entonces los 40 bits para ver si son potencialmente una encabezado de célula válida. Si lo son, los 8 bits del extremo derecho serán un HEC válido para los 32 bits de la izquierda. Si no se cumple esta condición, el *buffer* no contiene una célula válida, en cuyo caso se recorren a la derecha un bit todos los bits del *buffer*, haciendo que un bit caiga por el extremo y se introduzca un nuevo bit de entrada al extremo izquierdo. Este proceso se repite hasta que se localiza un HEC válido. En este punto, el límite de la célula se conoce porque el registro de desplazamiento contiene un encabezado válido.

El problema con esta heurística es que el HEC sólo tiene 8 bits. Para cualquier registro de desplazamiento, aún uno que contenga bits aleatorios, la probabilidad de encontrar un HEC válido es de 1/256, un valor moderadamente grande. Usado por sí mismo, este procedimiento detectaría incorrectamente encabezados de célula con demasiada frecuencia.

Para mejorar la precisión del algoritmo de reconocimiento, se usa la máquina de estado finito de la figura 3-30. Se manejan tres estados: *HUNT, PRESYNCH* y *SYNCH*. En el estado *HUNT*, la subcapa TC recorre bits en los registros de desplazamiento uno a la vez buscando un HEC válido. Tan pronto encuentra uno, la máquina de estado finito se conmuta al estado *PRESYNCH*, lo que quiere decir que ha localizado tentativamente un límite de célula. Ahora recorre los siguientes 424 bits (53 bytes) sin examinarlos. Si su suposición respecto al límite de la célula fue correcta, el registro de desplazamiento ahora contendrá otro encabezado de célula válida, por lo que nuevamente ejecutará el algoritmo HEC. Si el HEC es incorrecto, la TC

regresará al estado *HUNT* y continuará buscando bit por bit un encabezado cuyo HEC sea correcto.

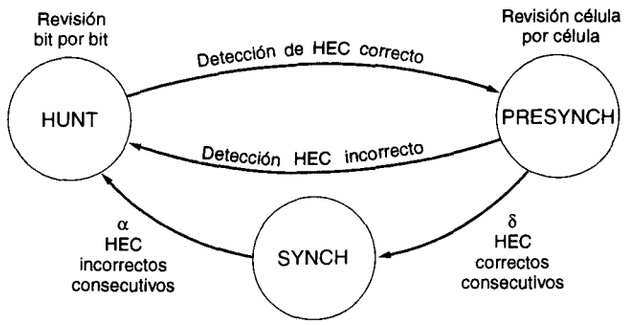


Figura 3-30. Heurística de delimitación de células.

Por otra parte, si el segundo HEC también es correcto, la TC podría haber encontrado algo, por lo que recorre otros 424 bits e intenta de nuevo. Continúa inspeccionando encabezados de esta manera hasta que ha encontrado δ encabezados correctos consecutivos, momento en el cual supone que está sincronizada y pasa al estado *SYNCH* para comenzar la operación normal. Note que la probabilidad de entrar accidentalmente en el estado *SYNCH* con una corriente de bits puramente aleatoria es de $2^{-8\delta}$, que puede hacerse arbitrariamente pequeña escogiendo un δ de suficiente tamaño. Sin embargo, el precio que se paga por un δ grande es un tiempo mayor para lograr la sincronización.

Además de la resincronización tras perder la sincronía (o al arranque), la subcapa TC necesita una heurística para determinar cuándo ha perdido la sincronía, por ejemplo tras haber sido introducido o borrado un bit en la corriente de bits. Sería imprudente darse por vencido si sólo un HEC fue incorrecto, ya que la mayoría de los errores son inversiones de bit, no inserciones ni eliminaciones. El camino más sensato aquí es descartar simplemente la célula con el encabezado equivocado y esperar que la siguiente sea correcta. Sin embargo, si α HEC seguidos están mal, la subcapa TC tiene que concluir que ha perdido la sincronía y debe regresar al estado *HUNT*.

Aunque es poco probable, es concebible que un usuario mal intencionado intente engañar a la subcapa TC introduciendo en el campo de carga útil de muchas células consecutivas un patrón de datos que imite el algoritmo HEC. Entonces, si se llegara a perder la sincronía, podría recuperarse en el lugar equivocado. Para hacer mucho más difícil este truco, los bits de carga útil se codifican al transmitirse y descodifican al recibirse.

Antes de dejar la subcapa TC, vale la pena un comentario. El mecanismo escogido para delimitar las células requiere que la subcapa TC entienda y use el encabezado de la capa ATM que está encima de ella. Hacer que una capa use el encabezado de una capa superior es una violación de las reglas básicas de la ingeniería de protocolos. El propósito de tener protocolos en capas es hacer que cada capa sea independiente de las que están por encima de él. Debería ser

posible, por ejemplo, cambiar el formato del encabezado de la capa ATM sin afectar la subcapa TC. Sin embargo, debido a la manera en que se logra la delimitación de células, no es posible hacer un cambio de este tipo.

3.7. RESUMEN

La tarea de la capa de enlace de datos es convertir la corriente de bits en bruto ofrecida por la capa física en una corriente de marcos a ser usados por la capa de red. Se emplean varios métodos de enmarcado, incluidos el conteo de caracteres, el relleno de caracteres y el relleno de bits. Los protocolos de enlace de datos pueden proporcionar control de errores para retransmitir marcos dañados o perdidos. Para evitar que un transmisor rápido sature a un receptor lento, el protocolo de enlace de datos también puede proporcionar control de flujo. El mecanismo de ventana corrediza se usa ampliamente para integrar el control de errores y el control de flujo de una manera conveniente.

Los protocolos de ventana corrediza pueden clasificarse por el tamaño de la ventana del transmisor y por el tamaño de la ventana del receptor. Cuando ambas son iguales a 1, el protocolo es de parada y espera. Cuando la ventana del transmisor es mayor que 1, por ejemplo para evitar el bloqueo del transmisor en un circuito con un retardo de propagación grande, el receptor puede programarse para descartar todos los marcos diferentes al siguiente de la secuencia (protocolo 5) o almacenar en *buffer* marcos fuera de orden hasta que se necesiten (protocolo 6).

Los protocolos puede modelarse usando varias técnicas para demostrar que son correctos (o no). Los modelos de máquina de estado finito y de red de Petri se usan frecuentemente para este propósito.

Muchas redes utilizan uno de los protocolos orientados a bits (SDLC, HDLC, ADCCP o LAPB) en la capa de enlace de datos. Todos estos protocolos usan bytes indicadores para delimitar marcos y relleno de bits para evitar que los bytes indicadores ocurran en los datos. Todos ellos también usan ventanas corredizas para el control de flujo. Internet emplea SLIP y PPP como protocolos de enlace de datos. Los sistemas ATM tienen su propio protocolo sencillo, que efectúa un mínimo de revisiones de error y no tiene control de flujo.

PROBLEMAS

1. Un mensaje de capa superior se divide en 10 marcos, cada uno de los cuales tiene 80% de probabilidad de llegar sin daño. Si el protocolo de enlace de datos no lleva a cabo control de errores, ¿cuántas veces debe reenviarse el mensaje en promedio para conseguir que pase todo?
2. El siguiente fragmento de datos ocurre a la mitad de una corriente de datos para la cual se ha usado el algoritmo de relleno de caracteres descrito en el texto: DLE, STX, A, DLE, B, DLE, ETX. ¿Cuál es la salida tras el relleno?
3. Si la cadena de bits 0111101111101111110 se rellena, ¿cuál es la cadena de salida?

4. Cuando se usa relleno de bits, ¿es posible que la pérdida, inserción o modificación de un solo bit cause un error no detectado por la suma de comprobación? ¿Si no, por qué no? ¿Si sí, cómo? ¿Desempeña aquí un papel la longitud de la suma de comprobación?
5. ¿Puede pensar en alguna circunstancia en la cual podría ser preferible un protocolo de ciclo abierto (por ejemplo, un código de Hamming) a los protocolos tipo realimentación analizados a lo largo de este capítulo?
6. Para proporcionar mayor confiabilidad de la que puede dar un solo bit de paridad, un esquema de codificación de detección de errores usa un bit de paridad para comprobar todos los bits de número impar y un segundo bit de paridad para todos los bits de número par. ¿Cuál es la distancia de Hamming de este código?
7. Una manera de detectar errores es transmitir los datos como un bloque de n filas de k bits por filas y agregar bits de paridad a cada fila y a cada columna. ¿Detectará este esquema todos los errores sencillos? ¿Los errores dobles? ¿Los errores triples?
8. Un bloque de bits con n filas y k columnas usa bits de paridad horizontales y verticales para la detección de errores. Suponga que se invierten exactamente 4 bits debido a errores de transmisión. Deduzca una expresión para la probabilidad de que el error no sea detectado.
9. ¿Qué residuo se obtiene al dividir $x^7 + x^5 + 1$ entre el polinomio generador $x^3 + 1$?
10. Los protocolos de enlace de datos casi siempre ponen el CRC en un apéndice, en lugar de un encabezado. ¿Por qué?
11. Un canal tiene una tasa de bits de 4 kbps y un retardo de propagación de 20 mseg. ¿Para qué intervalo de tamaños de marco da la parada y espera una eficiencia de cuando menos 50%?
12. Una red de líneas T1 de 3000 km de longitud se usa para transmitir marcos de 64 bytes con el protocolo 5. Si la velocidad de propagación es de 6 μseg/km, ¿de cuántos bits deben ser los números de secuencia?
13. Imagine un protocolo de ventana corrediza que usa tantos bits para los números de secuencia que nunca ocurre un reinicio. ¿Qué relaciones deben mantenerse entre los cuatro límites de la ventana y el tamaño de la ventana?
14. Si el procedimiento *between* del protocolo 5 revisara la condición $a \leq b \leq c$ en lugar de la condición $a \leq b < c$, ¿tendría esto algún efecto en la corrección o la eficiencia del protocolo? Explique su respuesta.
15. En el protocolo 6, cuando llega un marco de datos, se hace una revisión para ver si el número de secuencia es diferente del esperado y *NoNak* es verdadero. Si ambas condiciones se cumplen, se envía un NAK. De otra manera, se arranca el temporizador auxiliar. Suponga que se omite la cláusula *else*. ¿Afectaría este cambio la corrección del protocolo?
16. Suponga que el ciclo while de tres instrucciones cerca del final del protocolo 6 se elimina del código. ¿Afectará esto la corrección del protocolo o sólo su desempeño? Explique su respuesta.
17. Suponga que el caso para errores de suma de comprobación fuera eliminado del enunciado switch el protocolo 6. ¿Cómo afectaría este cambio la operación del protocolo?
18. En el protocolo 6 el código de *FrameArrival* tiene una sección que se usa para los NAK. Esta sección se invoca si el marco entrante es un NAK y se cumple otra condición. Describa una situación en la que la presencia de esta otra condición sea esencial.

19. Imagine que está escribiendo el *software* de la capa de enlace de datos para una línea que se usa para enviar datos a usted, pero no desde usted. El otro extremo usa HDLC, con un número de secuencia de tres bits y un tamaño de ventana de siete marcos. Usted podría querer manejar en *buffers* tantos marcos fuera de secuencia como fuera posible para aumentar la eficiencia, pero no se le permite modificar el *software* del lado transmisor. ¿Es posible tener una ventana de receptor mayor que uno y aun así garantizar que el protocolo nunca fallará? De ser así, ¿cuál es el tamaño de ventana más grande que se puede usar con seguridad?
20. Considere la operación del protocolo 6 en una línea de 1 Mbps libre de errores. El tamaño máximo de marco es de 1000 bits. Se generan nuevos paquetes a intervalos aproximados de 1 segundo. El intervalo de terminación del temporizador es de 10 mseg. Si se eliminara el temporizador especial de acuse ocurrirían terminaciones de temporizador innecesarias. ¿Cuántas veces se transmitiría el mensaje promedio?
21. En el protocolo 6, $MaxSeq = 2^n - 1$. Si bien esta condición es evidentemente deseable para hacer uso eficiente de los bits de encabezado, no hemos demostrado que sea esencial. ¿Funciona correctamente el protocolo con $MaxSeq = 4$, por ejemplo?
22. Se están enviando marcos de 1000 bits a través de un canal de satélite de 1 Mbps. Los acuses siempre se incorporan en los marcos de datos. Los encabezados son muy cortos. Se usan números de secuencia de tres bits. ¿Cuál es la utilización máxima de canal que se puede lograr para:
    (a) Parada y espera.
    (b) El protocolo 5.
    (c) El protocolo 6.
23. Calcule la fracción del ancho de banda que se desperdicia en información extra (encabezados y retransmisiones) para el protocolo 6 en un canal de satélite de 50 kbps con carga pesada, usando marcos de datos consistentes en 40 bits de encabezado y 3960 de datos. Nunca ocurren marcos ACK. Los marcos NAK son de 40 bits. La tasa de errores de los marcos de datos es de 1%, y la tasa de errores para los marcos NAK es despreciable. Los números de secuencia son de 8 bits.
24. Considere un canal de satélite de 64 kbps libre de errores que se usa para enviar marcos de datos de 512 bytes en una dirección y devolver acuses muy cortos en la otra. ¿Cuál es el rendimiento máximo con tamaños de ventana de 1, 7, 15 y 127?
25. Un cable de 100 km de longitud opera con una tasa de datos T1. La velocidad de propagación del cable es 2/3 de la velocidad de la luz. ¿Cuántos bits caben en el cable?
26. Redibuje la figura 3-21 para un canal dúplex integral que nunca pierde marcos. ¿Aún es posible la falla del protocolo?
27. Indique la secuencia de disparo de la red de Petri de la figura 3-23 correspondiente a la secuencia de estados (000), (01A), (01—), (010), (01A) de la figura 3-20. Explique con palabras lo que representa la secuencia.
28. Dadas las reglas de transición $AC \rightarrow B$, $B \rightarrow AC$, $CD \rightarrow E$ y $E \rightarrow CD$, dibuje la red de Petri descrita. A partir de la red de Petri, dibuje el grafo de estado finito alcanzable desde el estado inicial ACD. ¿Qué concepto conocido de la informática modela estas reglas de transición?
29. PPP se basa en gran medida en HDLC, que usa relleno de bits para evitar que los bytes indicadores accidentales en la carga útil produzcan confusión. Cite cuando menos una razón por la que PPP usa relleno de caracteres en su lugar.

30. ¿Cuál es la información extra mínima al enviar un paquete IP usando PPP? Cuente sólo la información extra introducida por PPP, no la de el encabezado de IP.

31. Considere la heurística ATM de delimitación de células con $\alpha = 5$, $\delta = 6$ y una tasa de errores por bit de 10^{-5}. Una vez que está en sincronía el sistema, ¿cuánto tiempo permanecerá así, a pesar de errores de bit ocasionales? Suponga que la línea está operando a OC-3.

32. Escriba un programa para simular estocásticamente el comportamiento de una red de Petri. El programa deberá leer las reglas de transición así como una lista de estados correspondientes a que la capa de enlace de la red emite un paquete nuevo o acepta un paquete nuevo. Del estado inicial, también leído, el programa debe escoger al azar transiciones habilitadas y dispararlas, verificando si un *host* llega a aceptar dos mensajes sin que el otro *host* emita uno nuevo entre ellos.

4

LA SUBCAPA DE ACCESO AL MEDIO

Como apuntamos en el capítulo 1, las redes pueden dividirse en dos categorías: las que usan conexiones punto a punto y las que usan canales de difusión. Este capítulo trata las redes de difusión y sus protocolos.

En cualquier red de difusión, el asunto clave es la manera de determinar quién puede usar el canal cuando hay competencia por él. Para aclarar el punto, considere una llamada en conferencia en la que seis personas, en seis teléfonos diferentes, están conectadas de modo que cada una puede oír y hablar con todas las demás. Es muy probable que cuando una de ellas deje de hablar, dos o más comenzarán a hacerlo a la misma vez, conduciendo al caos. En las reuniones cara a cara, se evita el caos por medios externos. Por ejemplo, en una reunión, la gente levanta la mano para solicitar permiso de hablar. Cuando únicamente hay un canal, la determinación de quién tiene el turno es mucho más complicada. Se conocen muchos protocolos para resolver el problema y forman el contenido de este capítulo. En la literatura, los canales de difusión a veces se denominan **canales multiacceso** o **canales de acceso aleatorio**.

Los protocolos usados para determinar quién sigue en un canal multiacceso pertenecen a una subcapa de la capa de enlace de datos llamada subcapa de **MAC** (*medium access control*, **control de acceso al medio**). La subcapa MAC tiene especial importancia en las LAN, casi todas las cuales usan un canal multiacceso como base de su comunicación. Las WAN, en contraste, usan enlaces punto a punto, excepto en las redes satelitales. Debido a que los canales multiacceso y las LAN están tan estrechamente relacionadas, en este capítulo analizaremos las LAN en general, al igual que las redes satelitales y otras redes de difusión.

Técnicamente, la subcapa MAC es la parte inferior de la capa de enlace de datos, por lo que lógicamente debimos haberla estudiado antes de examinar los protocolos punto a punto en el capítulo 3. No obstante, para la mayoría de la gente, la comprensión de los protocolos en los que intervienen muchas partes es más fácil una vez que se han entendido bien los protocolos de dos partes. Por esta razón nos hemos desviado de un orden de presentación estrictamente ascendente.

4.1. EL PROBLEMA DE REPARTO DEL CANAL

El tema central de este capítulo es la forma de repartir un solo canal de difusión entre usuarios competidores. Primero veremos los esquemas estático y dinámico en general. Luego examinaremos varios algoritmos específicos.

4.1.1. Reparto estático de canal en las LAN y las MAN

La manera tradicional de repartir un canal sencillo, como una troncal telefónica, entre varios usuarios competidores es la multiplexión por división en frecuencia (FDM, *Frequency Division Multiplexing*). Si sólo hay N usuarios, el ancho de banda se divide en N partes de igual tamaño (véase la figura 2-24), asignándosele a cada usuario una parte. Dado que cada usuario tiene una banda de frecuencia privada, no hay interferencia entre los usuarios. Cuando sólo hay una pequeña cantidad fija de usuarios, cada uno de los cuales tiene una carga de tráfico pesada (en *buffer*, memoria intermedia) (por ejemplo, en la oficina de conmutación de una portadora), la FDM es un mecanismo de reparto sencillo y eficiente.

Sin embargo, cuando el número de transmisores es grande y varía continuamente, o cuando el tráfico es en ráfagas, la FDM presenta algunos problemas. Si el espectro se divide en N regiones, y hay menos de N usuarios interesados en comunicarse actualmente, se desperdiciará una buena parte de espectro valioso. Si más de N usuarios quieren comunicarse, a algunos de ellos se les negará el permiso por falta de ancho de banda, aun si algunos de los usuarios que tienen asignada una banda de frecuencia apenas transmiten o reciben algo.

Aun suponiendo que el número de usuarios podría, de alguna manera, mantenerse constante en N, dividir el canal disponible en subcanales estáticos es inherentemente ineficiente. El problema básico es que, cuando algunos usuarios están inactivos, su ancho de banda simplemente se pierde. No lo están usando, y a nadie más se le permite usarlo. Es más, en la mayoría de los sistemas de cómputo el tráfico de datos es especialmente dado a las ráfagas (son comunes las relaciones de tráfico pico a tráfico medio de 1000:1). En consecuencia, la mayoría de los canales estarán inactivos la mayor parte del tiempo.

El desempeño pobre de la FDM estática puede verse fácilmente mediante un cálculo sencillo de la teoría de colas. Comencemos por el retardo medio, T, de un canal de C bps de capacidad, con una tasa de recepción de λ marcos/seg, teniendo cada marco una longitud que se obtiene de una función exponencial de densidad de probabilidad con una media de $1/\mu$ bits/marco:

$$T = \frac{1}{\mu C - \lambda}$$

Ahora dividamos el canal en N subcanales independientes, cada uno con capacidad de C/N bps. La tasa media de entrada de cada uno de los subcanales ahora será de λ/N. Recalculando T, obtenemos

$$T_{\text{FDM}} = \frac{1}{\mu(C/N) - (\lambda/N)} = \frac{N}{\mu C - \lambda} = NT \tag{4-1}$$

El retardo medio al usar FDM es N veces peor que si todos los marcos se acomodaran mágicamente de algún modo en una gran cola central.

Precisamente los mismos argumentos que se aplican a la FDM se aplican a la multiplexión por división en tiempo (TDM). A cada usuario se le asigna cada N-ésimo intervalo de tiempo. Si un usuario no usa la ranura asignada, simplemente se desperdicia. Ya que ninguno de los métodos tradicionales de reparto estático de canal funciona muy bien con tráfico en ráfagas, exploraremos ahora los métodos dinámicos.

4.1.2. Reparto dinámico de canales en las LAN y MAN

Antes de entrar en el primero de los muchos métodos de reparto de canales que se analizarán en este capítulo, vale la pena formular cuidadosamente el problema de reparto. Todo el trabajo hecho en esta área se basa en cinco supuestos clave, que se describen a continuación.

1. **Modelo de estación**. El modelo consiste en N **estaciones** independientes (computadoras, teléfonos, comunicadores personales, etc.), cada una con un programa o usuario que genera marcos para transmisión. La probabilidad de que un marco se genere en una ranura de longitud Δt es de $\lambda \Delta t$, donde λ es una constante (la tasa de llegada de marcos nuevos). Una vez que se ha generado un marco, la estación se bloquea y no hace nada hasta que el marco se ha transmitido con éxito.

2. **Supuesto de canal único**. Hay un solo canal disponible para todas las comunicaciones. Todas las estaciones pueden transmitir en él y pueden recibir de él. En lo referente al *hardware*, todas las estaciones son equivalentes, aunque el *software* del protocolo puede asignarles prioridades.

3. **Supuesto de colisión**. Si dos marcos se transmiten simultáneamente, se traslapan en el tiempo y la señal resultante se altera. Este evento se llama **colisión**. Todas las estaciones pueden detectar colisiones. Un marco en colisión debe transmitirse nuevamente después. No hay otros errores excepto aquellos generados por las colisiones.

4a. **Tiempo continuo**. La transmisión de un marco puede comenzar en cualquier momento. No hay reloj maestro que divida el tiempo en intervalos discretos.

4b. **Tiempo ranurado**. El tiempo se divide en intervalos discretos (ranuras). La transmisión de los marcos siempre comienza al inicio de una ranura. Una ranura puede contener 0, 1 o más marcos, correspondientes a una ranura inactiva, una transmisión con éxito, o una colisión, respectivamente.

5a. **Detección de portadora**. Las estaciones pueden saber si el canal está en uso antes de intentar usarlo. Si se detecta que el canal está en uso, ninguna estación intentará usarlo hasta que regrese a la inactividad.

5b. **Sin detección de portadora**. Las estaciones no pueden detectar el canal antes de intentar usarlo. Simplemente transmiten. Sólo después pueden determinar si la transmisión tuvo éxito.

Es importante un análisis de estos supuestos. El primero dice que las estaciones son independientes, y que se genera trabajo a velocidad constante. También supone implícitamente que cada estación sólo tiene un programa o usuario, así que mientras la estación esté bloqueada no se generará trabajo nuevo. Los modelos más complicados permiten estaciones multiprogramadas que pueden generar trabajo mientras la estación está bloqueada, pero el análisis de estas estaciones es mucho más complejo.

El supuesto de canal único es el meollo del asunto. No hay formas externas de comunicación. Las estaciones no pueden levantar la mano para solicitar que el maestro les ceda la palabra.

El supuesto de colisión también es básico, aunque en algunos sistemas (principalmente los de espectro amplio), este supuesto se relaja con resultados sorprendentes. También algunas LAN, como la *token ring*, usan un mecanismo de eliminación de contención que elimina las colisiones.

Hay dos supuestos alternativos sobre el tiempo. O es continuo o está ranurado. Algunos sistemas usan uno y otros el otro, por lo que estudiaremos y analizaremos ambos. Obviamente, para un sistema dado, sólo un supuesto es válido.

De manera semejante, una red puede tener detección de portadora o no. Las LAN generalmente tienen detección de portadora, pero las redes satelitales no (debido al gran retardo de propagación). Las estaciones en redes detectoras de portadora pueden terminar su transmisión prematuramente si descubren que está chocando con otra transmisión. Note que la palabra "portadora" en este sentido se refiere a una señal eléctrica en el cable y no tiene nada que ver con las portadoras comunes (por ejemplo, las compañías telefónicas), que datan de los tiempos del *Pony Express*.

4.2. PROTOCOLOS DE ACCESO MÚLTIPLE

Se conocen muchos algoritmos para repartir un canal de acceso múltiple. En las siguientes secciones estudiaremos una muestra representativa de los más interesantes y daremos algunos ejemplos de su uso.

4.2.1. ALOHA

En los años 70, Norman Abramson y sus colegas de la Universidad de Hawai inventaron un método novedoso y elegante para resolver el problema de reparto del canal. Desde entonces, su trabajo ha sido extendido por muchos investigadores (Abramson, 1985). Aunque el trabajo de

Abramson, llamado sistema ALOHA, usó la radiotransmisión basada en tierra, la idea básica es aplicable a cualquier sistema en el que usuarios no coordinados compiten por el uso de un solo canal compartido.

Analizaremos dos versiones de ALOHA: la pura y la ranura. Difieren en si se divide o no el tiempo en ranuras discretas en las que deben caber todos los marcos. El ALOHA puro no requiere sincronización de tiempo global; el ALOHA ranurado sí.

ALOHA puro

La idea básica de un sistema ALOHA es sencilla: permitir que los usuarios transmitan cuando tengan datos por enviar. Por supuesto, habrá colisiones, y los marcos en colisión se destruirán. Sin embargo, debido a la propiedad de retroalimentación de la difusión, un transmisor siempre puede saber si el marco fue destruido o no escuchando el canal, de la misma manera que los demás usuarios. Con una LAN, la retroalimentación es inmediata; vía satélite, hay un retardo de 270 mseg antes de que el transmisor sepa si la transmisión tuvo éxito. Si el marco fue destruido, el transmisor simplemente espera un tiempo aleatorio y lo envía de nuevo. El tiempo de espera debe ser aleatorio o los mismos marcos chocarán una y otra vez, en sincronía. Los sistemas en los que varios usuarios comparten un canal común de modo tal que puede dar pie a conflictos se conocen como sistemas de **contención**.

En la figura 4-1 se presenta un esbozo de la generación de marcos en un sistema ALOHA. Hemos hecho que los marcos sean todos de la misma longitud porque el rendimiento de los sistemas ALOHA es máximo con un tamaño uniforme de marco en lugar de marcos de longitud variable.

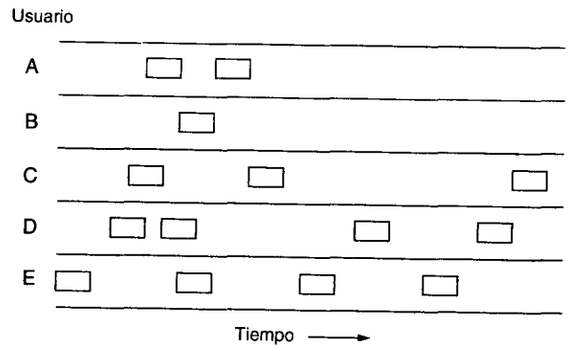


Figura 4-1. En ALOHA puro, los marcos se transmiten en momentos completamente arbitrarios.

Cada vez que dos marcos tratan de ocupar el canal al mismo tiempo, habrá una colisión y ambos se alterarán. Si el primer bit de un marco nuevo traslapa el último bit de un marco casi terminado, ambos marcos se destruirán por completo, y ambos tendrán que retransmitirse des-

pués. La suma de comprobación no puede (y no debe) distinguir entre una pérdida total y un encuentro cercano. Lo malo es malo.

Una pregunta por demás interesante es: ¿cuál es la eficiencia de un canal ALOHA? Es decir, ¿qué fracción de todos los marcos transmitidos escapa a las colisiones en estas caóticas circunstancias? Primero consideremos un conjunto infinito de usuarios interactivos sentados ante sus computadoras (estaciones). Un usuario siempre está en uno de dos estados, escribiendo o esperando. Inicialmente, todos los usuarios están en el estado de escritura. Al terminar una línea, el usuario deja de escribir, esperando una respuesta. La estación entonces transmite un marco que contiene la línea y verifica el canal para ver si llegó con éxito. De ser así, el usuario ve la respuesta y continúa escribiendo. Si no, el usuario continúa esperando y el marco se retransmite una y otra vez hasta que se envía con éxito.

Denotemos con "tiempo de marco" el tiempo necesario para transmitir un marco estándar de longitud fija (es decir, la longitud del marco dividida entre la tasa de bits). En este punto, suponemos que la población infinita de usuarios genera marcos nuevos según una distribución de Poisson con una media de S marcos por tiempo de marco. (La suposición de población infinita es necesaria para asegurar que S no disminuya a medida que se bloquean los usuarios.) Si $S > 1$, la comunidad de usuarios está generando marcos a una velocidad mayor que la que puede manejar el canal, y casi cada marco sufre una colisión. Para un rendimiento razonable esperaríamos que $0 < S < 1$.

Además de marcos nuevos, las estaciones también generan retransmisiones de marcos que previamente sufrieron colisiones. Supongamos además que la probabilidad de k intentos de transmisión por tiempo de marco, viejos y nuevos combinados, también es Poisson, con una media de G por tiempo de marco. Claramente, $G \geq S$. Con carga baja (es decir, $S \approx 0$), habrá pocas colisiones y, por tanto, pocas retransmisiones, por lo que $G \approx S$. Con carga alta habrá muchas colisiones, por lo que $G > S$. Con todas las cargas, el rendimiento es sólo la carga ofrecida, G, por la probabilidad de que una transmisión tenga éxito (es decir, $S = GP_0$, donde P_0 es la probabilidad de que un marco no sufra una colisión).

Un marco no sufrirá una colisión si no se envían otros marcos durante una ranura de un tiempo de marco desde su inicio, como se muestra en la figura 4-2. ¿En qué condiciones llegará sin daño el marco sombreado? Sea t el tiempo requerido para enviar un marco. Si cualquier otro usuario generó un marco entre el tiempo t_0 y $t_0 + t$, el final de ese marco chocará con el comienzo del marco sombreado. De hecho, el destino del marco sombreado se selló aun antes de enviar el primer bit pero, dado que en ALOHA puro una estación no escucha el canal antes de transmitir, no tiene manera de saber que otro marco ya está en camino. De manera parecida, cualquier otro marco que salga entre $t_0 + t$ y $t_0 + 2t$ chocará con el final del marco sombreado.

La probabilidad de que k marcos sean generados durante un tiempo de marco dado está dado por la distribución de Poisson:

$$\Pr[k] = \frac{G^k e^{-G}}{k!} \tag{4-2}$$

así que la probabilidad de cero marcos es simplemente e^{-G}. En una ranura de dos tiempos de marco de longitud, el número medio de marcos es de $2G$. La probabilidad de que no se inicie

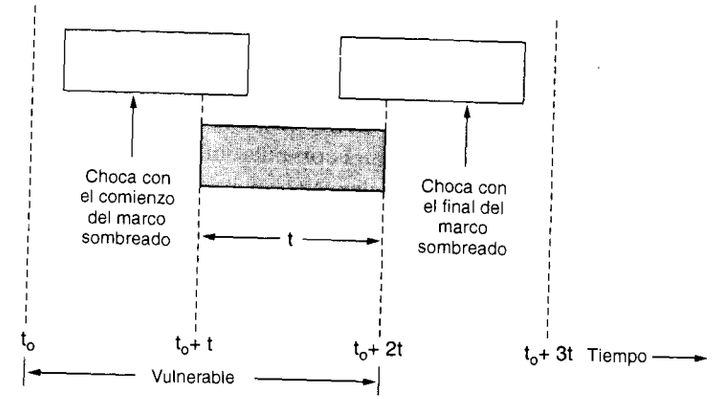


Figura 4-2. Periodo vulnerable para el marco sombreado.

otro tráfico durante todo el periodo vulnerable está dada entonces por $P_0 = e^{-2G}$. Si $S = GP_0$, obtenemos

$$S = Ge^{-2G}$$

La relación entre el tráfico ofrecido y el rendimiento se muestra en la figura 4-3. El rendimiento máximo ocurre a $G = 0.5$, con $S = 1/2e$, que es aproximadamente 0.184. En otras palabras, lo más que podemos esperar es un uso del canal del 18%. Este resultado no es muy alentador, pero con todo mundo transmitiendo al azar difícilmente podríamos esperar una tasa de éxito del 100%.

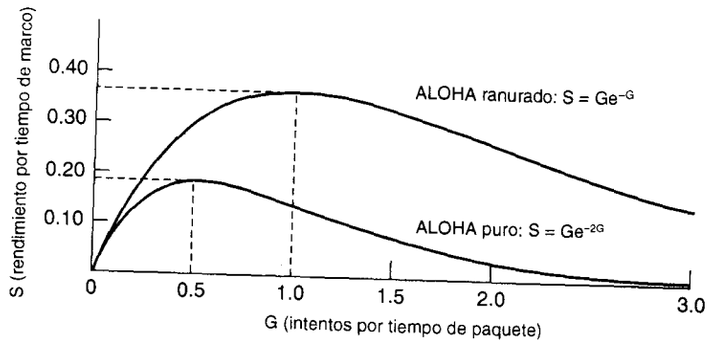


Figura 4-3. Rendimiento contra tráfico ofrecido de los sistemas ALOHA.

ALOHA ranurado

En 1972, Roberts publicó un método para duplicar la capacidad de un sistema ALOHA (Roberts, 1972). Su propuesta fue dividir el tiempo en intervalos discretos, correspondientes cada uno a un

marco. Este enfoque requiere que los usuarios acuerden límites de ranura. Una manera de lograr la sincronización sería tener una estación especial que emitiera una señal al comienzo de cada intervalo, como un reloj.

En el método de Roberts, que se conoce como **ALOHA ranurado**, en contraste con el **ALOHA puro** de Abramson, no se permite a una computadora enviar cada vez que se pulsa un retorno de carro. En cambio, se le obliga a esperar el comienzo de la siguiente ranura. Por tanto, el ALOHA puro continuo se convierte en uno discreto. Dado que el periodo vulnerable ahora es de la mitad, la probabilidad de que no haya más tráfico durante la misma ranura que nuestro marco de prueba es de e^{-G}, lo que conduce a

$$S = Ge^{-G} \tag{4-3}$$

Como se puede ver en la figura 4-3, el ALOHA ranurado tiene su pico en $G = 1$, con un rendimiento de $S = 1/e$, o aproximadamente 0.368, el doble que el ALOHA puro. Si el sistema está operando a $G = 1$, la probabilidad de una ranura vacía es de 0.368 (de la ecuación 4-2). Lo mejor que podemos esperar usando ALOHA ranurado es 37% de ranuras vacías, 37% de éxitos y 26% de colisiones. La operación con valores mayores de G reduce el número de ranuras vacías pero aumenta exponencialmente el número de colisiones. Para ver la manera en que se desarrolla este rápido crecimiento de colisiones con G, considere la transmisión de un marco de prueba. La probabilidad de que se evitará una colisión es de e^{-G}, que es la probabilidad de que los demás usuarios estén en silencio durante esa ranura. La probabilidad de una colisión es entonces de sólo $1 - e^{-G}$. La probabilidad de que una transmisión requiera exactamente k intentos (es decir, $k - 1$ colisiones seguidas de un éxito) es

$$P_k = e^{-G}(1 - e^{-G})^{k-1}$$

El número esperado de transmisiones, E, por retorno de carro introducido es

$$E = \sum_{k=1}^{\infty} kP_k = \sum_{k=1}^{\infty} ke^{-G}(1 - e^{-G})^{k-1} = e^{G}$$

Como resultado de la dependencia exponencial de E respecto a G, pequeños aumentos en la carga del canal pueden reducir drásticamente su desempeño.

4.2.2. Protocolos de acceso múltiple con detección de portadora

Con el ALOHA ranurado, el mejor aprovechamiento de canal que puede lograrse es $1/e$. Esto no es sorprendente pues, con estaciones que transmiten a voluntad propia, sin prestar atención a lo que están haciendo las demás estaciones, es inevitable que haya muchas colisiones. Sin embargo, en las redes de área local es posible que las estaciones detecten lo que están haciendo las demás estaciones y adapten su comportamiento con base en ello. Estas redes pueden lograr una utilización mucho mejor a $1/e$. En esta sección analizaremos algunos protocolos para mejorar el desempeño.

Los protocolos en los que las estaciones detectan una portadora (es decir, una transmisión) y actúan de acuerdo con ello se llaman **protocolos de detección de portadora**. Se ha propuesto una buena cantidad de ellos. Kleinrock y Tobagi (1975) han analizado detalladamente algunos

de esos protocolos. A continuación mencionaremos varias versiones de los protocolos de detección de portadora.

CSMA persistente y no persistente

El primer protocolo de detección de portadora que estudiaremos aquí se llama **CSMA persistente-1** (*Carrier Sense Multiple Access*, **acceso múltiple con detección de portadora**). Cuando una estación tiene datos por transmitir, primero escucha el canal para ver si otra está transmitiendo en ese momento. Si el canal está ocupado, la estación espera hasta que se desocupa. Cuando la estación detecta un canal en reposo, transmite un marco. Si ocurre una colisión, la estación espera una cantidad aleatoria de tiempo y comienza de nuevo. El protocolo se llama persistente-1 porque la estación transmite con una probabilidad de 1 cuando encuentra en reposo el canal.

El retardo de propagación tiene un efecto importante en el desempeño del protocolo. Hay una pequeña posibilidad de que, justo después de que una estación comienza a transmitir, otra estación está lista para enviar y detectar el canal. Si la señal de la primera estación no ha llegado aún a la segunda, esta última detectará un canal inactivo y comenzará a enviar también, resultando una colisión. Cuanto mayor sea el tiempo de propagación, más importante será este efecto, y peor el desempeño del protocolo.

Aun si el retardo de propagación es de cero, habrá colisiones. Si dos estaciones quedan listas a la mitad de la transmisión de una tercera, ambas esperarán respetuosamente hasta el fin de la transmisión y entonces comenzarán a transmitir simultáneamente, resultando una colisión. Si no fueran tan impacientes, habría menos colisiones. Aun así, este protocolo es mucho mejor que el ALOHA puro, ya que ambas estaciones tienen la decencia de desistir a interferir el marco de la tercera estación. Intuitivamente, esto conducirá a un mejor desempeño que el de ALOHA puro. Ocurre exactamente lo mismo con el ALOHA ranurado.

Un segundo protocolo de detección de portadora es el **CSMA no persistente**. En este protocolo se hace un intento consciente por ser menos egoísta que en el previo. Antes de enviar, una estación detecta el canal. Si nadie más está transmitiendo, la estación comienza a hacerlo. Sin embargo, si el canal ya está en uso, la estación no observa continuamente el canal a fin de tomarlo de inmediato al detectar el final de la transmisión previa. En cambio, espera un periodo de tiempo aleatorio y repite el algoritmo. Intuitivamente este algoritmo deberá conducir a una utilización mejor del canal y a mayores retardos que el CSMA persistente-1.

El último protocolo es el **CSMA persistente-p**, que se aplica a canales ranurados y funciona como sigue. Cuando una estación está lista para enviar, escucha el canal. Si el canal está en reposo, la estación transmite con una probabilidad p. Con una probabilidad $q = 1 - p$, se espera hasta la siguiente ranura. Si esa ranura también está en reposo, la estación transmite o espera nuevamente, con probabilidades p y q. Este proceso se repite hasta que el marco ha sido transmitido o hasta que otra estación ha comenzado a transmitir. En el segundo caso, la estación actúa como si hubiera habido una colisión (es decir, espera un tiempo aleatorio y comienza de nuevo). Si la estación detecta inicialmente que el canal está ocupado, espera hasta la siguiente ranura y aplica el algoritmo anterior. En la figura 4-4 se muestra el rendimiento contra tráfico ofrecido para los tres protocolos, así como para el ALOHA puro y el ranurado.

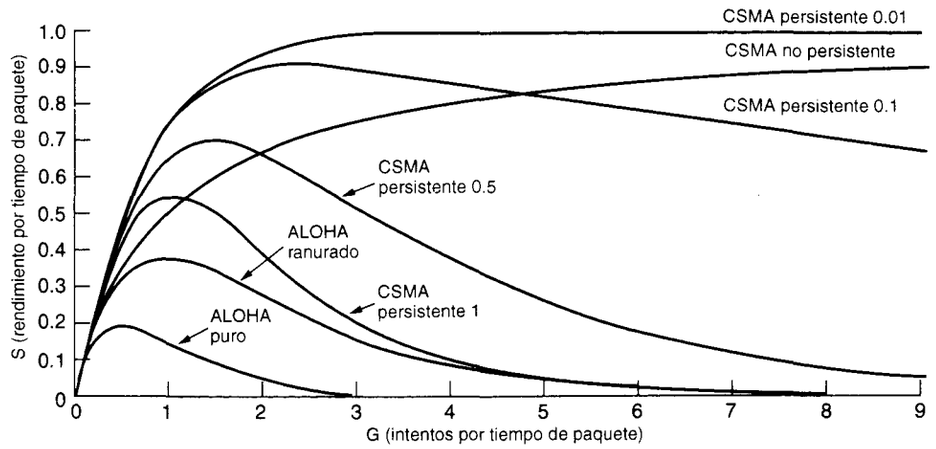


Figura 4-4. Comparación de la utilización del canal contra carga para varios protocolos de acceso aleatorio.

CSMA con detección de colisiones

Los protocolos CSMA persistentes y no persistentes ciertamente son una mejora respecto a ALOHA porque aseguran que ninguna estación comienza a transmitir cuando detecta que el canal está ocupado. Otra mejora es que las estaciones aborten sus transmisiones tan pronto como detecten una colisión. En otras palabras, si dos estaciones detectan que el canal está inactivo y comienzan a transmitir simultáneamente, ambas detectarán la colisión casi de inmediato. En lugar de terminar de transmitir sus marcos, que de todos modos están alterados irremediablemente, deben detener abruptamente la transmisión tan pronto como detectan la colisión. La terminación pronta de marcos dañados ahorra tiempo y ancho de banda. Este protocolo, conocido como **CSMA/CD** (*Carrier Sense Multiple Access with Collision Detection*, **acceso múltiple con detección de portadora y detección de colisiones**), se usa ampliamente en las LAN en la subcapa MAC.

CSMA/CD, al igual que muchos otros protocolos de LAN, utiliza el modelo conceptual de la figura 4-5. En el punto marcado t_0, una estación ha terminado de transmitir su marco. Cualquier otra estación que tenga un marco por enviar ahora puede intentar hacerlo. Si dos o más estaciones deciden transmitir simultáneamente, habrá una colisión. Las colisiones pueden detectarse observando la potencia o el ancho de pulso de la señal recibida y comparándola con la señal transmitida.

Una vez que una estación detecta una colisión, aborta la transmisión, espera un periodo de tiempo aleatorio e intenta de nuevo, suponiendo que ninguna otra estación ha comenzado a transmitir durante ese lapso. Por tanto, nuestro modelo de CSMA/CD consistirá en periodos alternantes de contención y transmisión, ocurriendo periodos muertos cuando todas las estaciones están calladas (por ejemplo, por falta de trabajo).

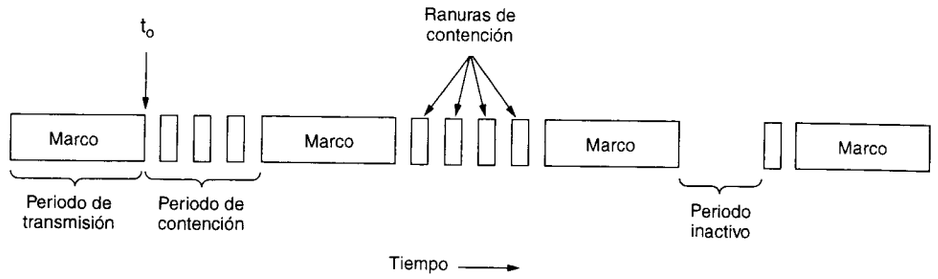


Figura 4-5. El CSMA/CD puede estar en uno de tres estados: contención, transmisión o inactivo.

Ahora observemos con cuidado los detalles del algoritmo de contención. Suponga que dos estaciones comienzan a transmitir exactamente en el momento t_0. ¿En cuánto tiempo se darán cuenta de que ha habido una colisión? La respuesta a esta pregunta es vital para determinar la longitud del periodo de contención, y por tanto el retardo y el rendimiento. El tiempo mínimo para detectar la colisión es sólo el tiempo que tarda la señal en propagarse de una estación a la otra.

Con base en este razonamiento, usted podría pensar que una estación que no detecta una colisión durante un tiempo igual al tiempo completo de propagación del cable tras comenzar su transmisión podrá estar segura de que ha tomado el cable. Por "tomar" queremos decir que todas las otras estaciones sabían que estaba transmitiendo y no interfirieron. Esta conclusión es equivocada. Considere la siguiente situación de peor caso. Sea τ el tiempo para que una señal se propague entre las dos estaciones más lejanas. En t_0, una estación comienza a transmitir. En $\tau - \varepsilon$, un instante antes de que la señal llegue a la estación más lejana, esa estación también comienza a transmitir. Por supuesto, detecta la colisión casi de inmediato y se detiene, pero la pequeña ráfaga de ruido causada por la colisión no regresa a la estación original hasta $2\tau - \varepsilon$. En otras palabras, en el peor caso una estación no puede estar segura de que ha tomado el canal hasta que ha transmitido durante 2τ sin detectar una colisión. Por esta razón, modelaremos la ranura de contención como un sistema ALOHA ranurado con un ancho de ranura de 2τ. En un cable coaxial de 1 km de longitud, $\tau \approx 5$ μseg. Por sencillez supondremos que cada ranura contiene sólo 1 bit. Por supuesto, una vez que ha tomado el canal, una estación puede transmitir con cualquier tasa que desee, no sólo a 1 bit cada 2τ seg.

Es importante darse cuenta de que la detección de colisiones es un proceso *analógico*. El *hardware* de la estación debe escuchar el cable mientras transmite. Si lo que lee es distinto de lo que puso en él, sabe que está ocurriendo una colisión. La implicación es que la codificación de la señal debe permitir que se detecten colisiones (por ejemplo, una colisión de dos señales de 0 volts bien podría ser imposible de detectar). Por esta razón, comúnmente se usa una codificación especial.

El CSMA/CD es un protocolo importante. Posteriormente en este capítulo estudiaremos una versión de él, IEEE 802.3 (Ethernet), que es un estándar internacional.

Para evitar cualquier mal entendido, es importante hacer notar que ningún protocolo de subcapa MAC garantiza la entrega confiable. Incluso en ausencia de colisiones, el receptor

podría no haber copiado correctamente el marco por varias razones (por ejemplo, falta de espacio de *buffer* o una interrupción no detectada).

4.2.3. Protocolos libres de colisiones

Aunque las colisiones no ocurren en CSMA/CD una vez que una estación ha tomado sin ambigüedades el canal, aún pueden ocurrir durante el periodo de contención. Estas colisiones afectan adversamente el desempeño del sistema, especialmente cuando el cable es largo (es decir, τ es grande) y los marcos cortos. A medida que las redes de fibra óptica muy largas y de gran ancho de banda se vuelven más comunes, la combinación de un τ grande y marcos cortos se volverá un problema cada vez más grave. En esta sección examinaremos algunos protocolos que resuelven la contención por el canal sin que haya colisiones, ni siquiera durante el periodo de contención.

En los protocolos que describiremos suponemos que hay N estaciones, cada una con una dirección única de 0 a $N - 1$ incorporada en *hardware*. El que algunas estaciones puedan estar inactivas parte del tiempo no importa. La pregunta básica persiste: ¿qué estación toma el canal tras una transmisión exitosa? Continuamos usando el modelo de la figura 4-5 con sus intervalos de contención discretos.

Un protocolo de mapa de bits

En nuestro primer protocolo libre de colisiones, el **método básico de mapa de bits**, cada periodo de contención consiste en exactamente N ranuras. Si la estación 0 tiene un marco por enviar, transmite un bit 1 durante la ranura 0. No está permitido a ninguna otra estación transmitir durante este intervalo. Sin importar lo que haga la estación 0, la estación 1 tiene la oportunidad de transmitir un 1 durante la ranura 1, pero sólo si tiene en cola un marco. En general, la estación j puede anunciar que tiene un marco por enviar introduciendo un bit 1 en la ranura j. Una vez que han pasado las N ranuras, cada estación tiene conocimiento completo de las estaciones que quieren transmitir. En ese punto, las estaciones comienzan a transmitir en orden numérico (véase la figura 4-6).

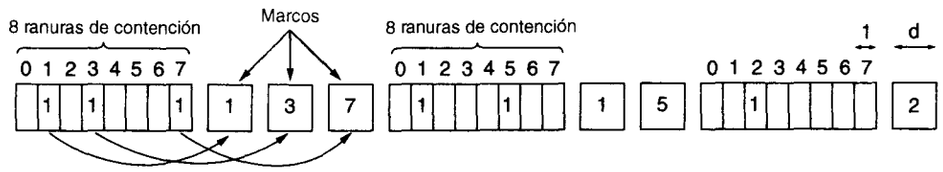


Figura 4-6. Protocolo básico de mapa de bits.

Dado que todos están de acuerdo en quién continúa, nunca habrá colisiones. Una vez que la última estación lista haya transmitido su marco, evento que pueden detectar fácilmente todas las estaciones, comienza otro periodo de contención de N bits. Si una estación queda lista justo

después de que ha pasado su ranura de bit, ha tenido mala suerte y deberá permanecer callada hasta que cada estación haya tenido su oportunidad y el mapa de bits haya comenzado de nuevo. Los protocolos como éste en los que el deseo de transmitir se difunde antes de la transmisión se llaman **protocolos de reservación**.

Analicemos brevemente el desempeño de este protocolo. Por conveniencia, mediremos el tiempo en unidades de la ranura de bit de contención, con marcos de datos consistentes en d unidades de tiempo. En condiciones de carga baja, el mapa de bits simplemente se repetirá una y otra vez, debido a la falta de marcos de datos.

Considere la situación desde el punto de vista de una estación de número bajo, como 0 o 1. Típicamente, cuando la estación queda lista para transmitir, la ranura "actual" estará en algún lugar a la mitad del mapa de bits. En promedio, la estación tendrá que esperar $N/2$ ranuras para que el barrido actual termine, y otras N ranuras para que el siguiente barrido se ejecute hasta su terminación, antes de poder comenzar a transmitir.

Las perspectivas para las estaciones de número alto son mejores. En general, éstas sólo tendrán que esperar la mitad de un barrido ($N/2$ ranuras de bit) antes de comenzar a transmitir. Las estaciones de número alto pocas veces tienen que esperar el siguiente barrido. Dado que las estaciones de número bajo deben esperar un promedio de $1.5N$ ranuras y las estaciones de número alto deben esperar en promedio $0.5N$ ranuras, la media de todas las estaciones es de N ranuras. La eficiencia del canal cuando la carga es baja es fácil de calcular. La información extra por marco es de N bits, y la cantidad de datos es de d bits, dando una eficiencia de $d/(N + d)$.

Si la carga es alta y todas las estaciones tienen algo que enviar todo el tiempo, el periodo de contención de N bits se prorratea entre N marcos, arrojando una información extra de sólo 1 bit por marco, o una eficiencia de $d/(d + 1)$. El retardo medio de un marco es igual a la suma del tiempo que está en cola en su estación más un $N(d + 1)/2$ adicional una vez que llega a la cabeza de su cola interna.

Conteo descendente binario

Un problema con el protocolo básico de mapa de bits es que la información extra es de 1 bit por estación. Podemos tener mejores resultados usando direcciones de estación binarias. Una estación que quiere usar el canal ahora difunde su dirección como una cadena binaria de bits, comenzando por el bit de orden mayor. Se supone que todas las direcciones tienen la misma longitud. A los bits en cada posición de dirección de las diferentes estaciones se aplica un OR BOOLEANO a todos juntos. Llamaremos a este protocolo **conteo descendente binario**; el cual se usa en Datakit (Fraser, 1987).

Para evitar conflictos, debe aplicarse una regla de arbitraje: tan pronto una estación ve que en una posición de bit de orden alto que en su dirección es 0 ha sido sobrescrita con un 1, se da por vencida. Por ejemplo, si las estaciones 0010, 0100, 1001 y 1010 están todas tratando de obtener el canal, en el primer tiempo de bit las estaciones transmiten 0, 0, 1 y 1 respectivamente. A éstos se les aplica el OR para formar un 1. Las estaciones 0010 y 0100 ven el 1 y saben que una estación de número mayor está compitiendo por el canal, por lo que se dan por vencidas durante esta ronda. Las estaciones 1001 y 1010 continúan.

UNIVERSIDAD DE
FACULTAD DE
DEPARTAM
DOCUMENTACION

El siguiente bit es 0, y ambas estaciones continúan. El siguiente bit es 1, por lo que la estación 1001 se da por vencida. La ganadora es la estación 1010, debido a que tiene la dirección mayor. Tras ganar el concurso, ahora puede transmitir un marco, después de lo cual comienza otro ciclo de concurso. El protocolo se ilustra en la figura 4-7.

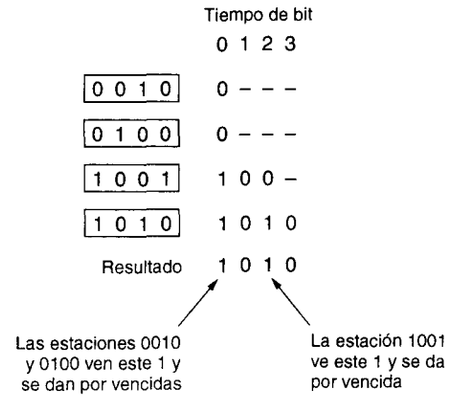


Figura 4-7. Protocolo de conteo descendente binario. Los guiones indican silencios.

La eficiencia de canal de este método es de $d/(d + \ln N)$. Sin embargo, si el formato de marco se escoge ingeniosamente de modo que la dirección del transmisor sea el primer campo del marco, ni siquiera estos ln N bits se desperdician, y la eficiencia es del 100%.

Mok y Ward (1979) han descrito una variación del conteo descendente binario usando una interfaz paralela en lugar de serial. También sugieren el uso de números virtuales de estación, permutándose circularmente tras cada transmisión los números virtuales de estación desde 0 hasta el de la estación ganadora, inclusive, a fin de darle mayor prioridad a las estaciones que han estado en silencio demasiado tiempo. Por ejemplo, si las estaciones *C, H, D, A, G, B, E, F* tienen prioridades 7, 6, 5, 4, 3, 2, 1 y 0 respectivamente, entonces una transmisión exitosa de *D* la pone al final de la lista, dando un orden de prioridad de *C, H, A, G, B, E, F, D*. Por tanto, *C* permanece como la estación virtual 7, pero *A* sube de 4 a 5 y *D* cae de 5 a 0. La estación *D* ahora sólo será capaz de adquirir el canal si ninguna otra estación lo quiere.

4.2.4. Protocolos de contención limitada

Hasta ahora hemos considerado dos estrategias básicas de adquisición del canal en una red de cable: los métodos por contención, como el CSMA, y los libres de colisión. Cada estrategia puede ser clasificada según lo bien que funciona en relación con las dos medidas de desempeño importantes, el retardo con carga baja y la eficiencia del canal con carga alta. En condiciones de carga baja, la contención (es decir, ALOHA puro o ranurado) es preferible debido a su bajo

retardo. A medida que aumenta la carga, la contención paulatinamente se vuelve menos atractiva, debido a que la información extra asociada al arbitraje del canal se vuelve mayor. Lo inverso se cumple para los protocolos libres de colisiones. Con carga baja, tienen un retardo alto, pero a medida que aumenta la carga mejora la eficiencia del canal en lugar de empeorar, como ocurre con los protocolos de contención.

Obviamente, sería agradable si pudiéramos combinar las mejores propiedades de los protocolos de contención y los libres de colisiones, desarrollando un protocolo nuevo que usara contención cuando la carga es baja para así tener un retardo bajo, y una técnica libre de colisiones cuando la carga es alta para lograr una buena eficiencia de canal. De hecho, existen tales protocolos, a los que llamaremos **protocolos de contención limitada**, y concluirán nuestro estudio de las redes de detección de portadora.

Hasta ahora, los únicos protocolos de contención que hemos estudiado han sido simétricos, es decir, cada estación intenta adquirir el canal con alguna probabilidad, p, usando todas las estaciones la misma p. Resulta interesante que el desempeño general del sistema a veces puede mejorarse usando un protocolo que asigne diferentes probabilidades a diferentes estaciones.

Antes de ver los protocolos asimétricos, repasemos rápidamente el desempeño en el caso simétrico. Suponga que k estaciones están contendiendo por el acceso al canal. Cada una tiene una probabilidad p de transmitir durante cada ranura. La probabilidad de que una estación adquiera con éxito el canal durante una ranura dada es entonces de $kp(1-p)^{k-1}$. Para encontrar el valor óptimo de p, diferenciamos con respecto de p, igualamos el resultado a cero y despejamos p. Al hacerlo, encontramos que el mejor valor de p es $1/k$. Sustituyendo $p = 1/k$, obtenemos

$$\Pr[\text{éxito con } p \text{ óptima}] = \left[\frac{k-1}{k}\right]^{k-1} \tag{4-4}$$

Esta probabilidad se presenta gráficamente en la figura 4-8. Para un número pequeño de estaciones, la probabilidad de éxito es buena, pero ya cuando la cantidad de estaciones alcanza cinco, la probabilidad ha caído a su valor asintótico de $1/e$.

Por la figura 4-8, es bastante evidente que la probabilidad de que una estación adquiera el canal sólo puede aumentar disminuyendo la cantidad de competencia. Los protocolos de contención limitada hacen precisamente eso. Primero dividen las estaciones en grupos (no necesariamente excluyentes). Sólo los miembros del grupo 0 pueden competir por la ranura 0. Si uno de ellos tiene éxito, adquiere el canal y transmite su marco. Si la ranura permanece desocupada o si hay una colisión, los miembros del grupo 1 contienden por la ranura 1, etc. Haciendo una división adecuada de las estaciones en grupos, puede reducirse el grado de contención para cada intervalo, operando así cada ranura cerca de la parte izquierda de la figura 4-8.

El truco está en cómo asignar las estaciones a las ranuras. Antes de ver el caso general, consideremos algunos casos especiales. En un extremo, cada grupo tiene sólo un miembro. Una asignación de este tipo garantiza que nunca habrá colisiones, pues cuando mucho una estación contiende por cada ranura. Hemos visto antes tales protocolos (por ejemplo, el conteo descendente binario). El siguiente caso especial es asignar dos estaciones por grupo. La probabilidad de que ambos intentarán transmitir durante una ranura es p^2, que para una p pequeña es despreciable. A medida que se asignan más estaciones a la misma ranura, aumenta la probabilidad de

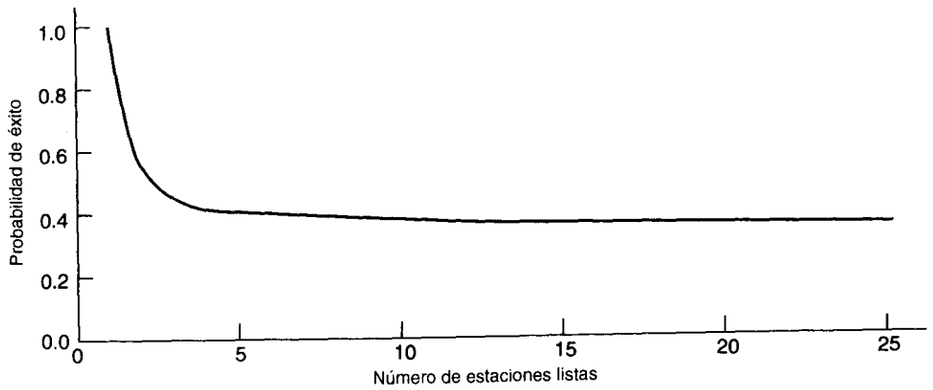


Figura 4-8. Probabilidad de adquisición de un canal de contención simétrica.

colisión, pero disminuye la longitud del barrido del mapa de bits necesario para dar a todos una oportunidad. El caso límite es un solo grupo que contiene todas las estaciones (es decir, ALOHA ranurado). Lo que necesitamos es una manera de asignar las estaciones a las ranuras de manera dinámica, con muchas estaciones por intervalo cuando la carga es baja y pocas estaciones (o incluso sólo una) por ranura cuando la carga es alta.

Protocolo de recorrido de árbol adaptable

Una manera particularmente sencilla de llevar a cabo la asignación necesaria es usar el algoritmo desarrollado por el ejército de Estados Unidos para hacer pruebas de sífilis a los soldados durante la Segunda Guerra Mundial (Dorfman, 1943). En pocas palabras, el ejército tomaba una muestra de sangre de N soldados. Se vaciaba una parte de cada muestra en un solo tubo de ensayo. Luego, esta muestra mezclada era examinada en busca de anticuerpos. Si no se encontraban, todos los soldados del grupo eran declarados sanos. Si se encontraban anticuerpos, se preparaban dos muestras mezcladas nuevas, una de los soldados 1 a $N/2$ y otra de los demás. El proceso se repetía recursivamente hasta que se determinaban todos los soldados infectados.

Para la versión de computadora de este algoritmo (Capetanakis, 1979) es conveniente pensar en las estaciones como las hojas de un árbol binario, como se ilustra en la figura 4-9. En la primera ranura de contención después de la transmisión satisfactoria de un marco, ranura 0, se permite que todas las estaciones intenten adquirir el canal. Si una de ellas lo logra, qué bueno. Si hay una colisión, entonces, durante la ranura 1, sólo aquellas estaciones que queden bajo el nodo 2 del árbol podrán competir. Si una de ellas adquiere el canal, la ranura que sigue al marco se reserva para las estaciones que están bajo el nodo 3. Si, por otra parte, dos o más estaciones bajo el nodo 2 quieren transmitir, habrá una colisión durante la ranura 1, en cuyo caso es el turno del nodo 4 durante la ranura 2.

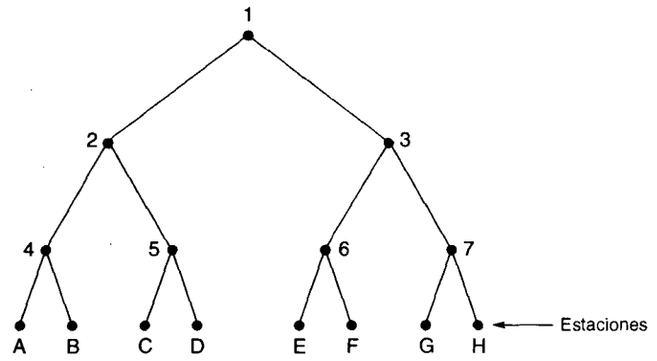


Figura 4-9. El árbol para ocho estaciones.

En esencia, si ocurre una colisión durante la ranura 0, se examina todo el árbol, primero en profundidad, para localizar todas las estaciones listas. Cada ranura de bit está asociada a un nodo en particular del árbol. Si ocurre una colisión, continúa la búsqueda recursivamente con los hijos izquierdo y derecho del nodo. Si una ranura de bit está inactiva, o si sólo hay una estación que transmite en ella, puede detenerse la búsqueda de su nodo, pues se han localizado todas las estaciones listas. (Si hubiera más de una, habría habido una colisión.)

Cuando la carga del sistema es pesada, apenas vale la pena dedicarle la ranura 0 al nodo 1, porque eso sólo tiene sentido en el caso poco probable de que precisamente una estación tenga un marco por enviar. De manera similar, se podría argumentar que los nodos 2 y 3 también podrían brincarse por la misma razón. En términos más generales, ¿en qué nivel del árbol debe comenzar la búsqueda? Es obvio que, a mayor carga, la búsqueda debe comenzar más abajo en el árbol. Supondremos que cada estación tiene una buena estimación del número de estaciones listas, q, por ejemplo, por la vigilancia del tráfico reciente.

Para proceder, numeremos los niveles del árbol desde arriba, con el nodo 1 de la figura 4-9 en el nivel 0, los nodos 2 y 3 en el nivel 1, etc. Nótese que cada nodo del nivel i tiene una fracción 2^{-i} de las estaciones por debajo de él. Si las q estaciones listas se distribuyen uniformemente, el número esperado de ellas por debajo de un nodo específico en el nivel i es sólo $2^{-i}q$. Intuitivamente, esperaríamos que el nivel óptimo para comenzar a examinar el árbol fuera el que tuviera un número medio de estaciones contendientes por ranura de 1, es decir, el nivel en el que $2^{-i}q = 1$. Resolviendo esta ecuación, encontramos que $i = \log_2 q$.

Se han descubierto numerosas mejoras al algoritmo básico, y han sido analizadas con cierto detalle por Bertsekas y Gallager (1992). Por ejemplo, considere el caso en el que las estaciones G y H son las únicas que quieren transmitir. En el nodo 1 ocurrirá una colisión, por lo que se intentará el 2 y se encontrará inactivo. No tiene caso probar el nodo 3, ya que está garantizado que tendrá una colisión (sabemos que dos o más estaciones bajo 1 están listas y que ninguna de ellas está bajo 2, por lo que todas deben estar bajo 3). La prueba de 3 puede omitirse para intentar 6. Al no arrojar nada esta prueba, también puede omitirse 7 para intentar G después.

4.2.5. Protocolos de acceso múltiple de división en longitud de onda

Un enfoque diferente para el reparto del canal es dividir el canal en subcanales usando FDM, TDM o ambas, y repartirlos dinámicamente según se necesite. Los esquemas como éste se usan comúnmente en las LAN de fibra óptica para permitir que diferentes conversaciones usen distintas longitudes de onda (es decir, frecuencias) al mismo tiempo. En esta sección examinaremos uno de tales protocolos (Humblet *et al.*, 1992).

Una manera sencilla de construir una LAN completamente óptica es usar un acoplador pasivo en estrella (véase la figura 2-10). En efecto, se fusionan dos fibras de cada estación a un cilindro de vidrio. Una fibra es para salidas al cilindro y la otra para entradas del cilindro. La salida de luz de cualquier estación ilumina el cilindro y puede ser detectada por todas las demás estaciones. Las estrellas pasivas pueden manejar cientos de estaciones.

Para permitir múltiples transmisiones al mismo tiempo, se divide el espectro en canales (bandas de longitud de onda), como se muestra en la figura 2-24. En este protocolo, **WDMA** (*Wavelength Division Multiple Access*, **acceso múltiple por división en longitud de onda**), se asignan dos canales a cada estación. Se proporciona un canal angosto como canal de control para enviar señales a la estación, y se proporciona un canal ancho para que la estación pueda enviar marcos de datos.

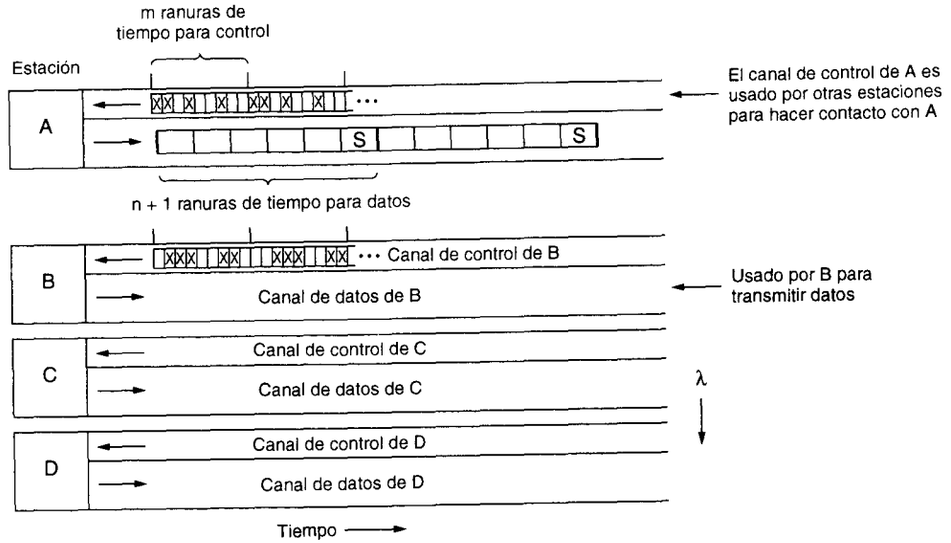


Figura 4-10. Acceso múltiple por división en longitud de onda.

Cada canal se divide en grupos de ranuras de tiempo, como se ilustra en la figura 4-10. Sea m el número de ranuras en el canal de control y $n + 1$ el número de ranuras en el canal de datos, donde n de éstos son para datos y el último es usado por la estación para informar su estado

(específicamente, qué ranuras de ambos canales están libres). En ambos canales, la secuencia de ranuras se repite indefinidamente, marcándose la ranura 0 de una manera especial para que los que llegan tarde la puedan detectar. Todos los canales se sincronizan con un solo reloj global.

El protocolo reconoce tres clases de tráfico: (1) tráfico orientado a conexión con tasa de datos constante, como vídeo sin comprimir, (2) tráfico orientado a conexión con tasa de datos variable, como transferencia de archivos, y (3) tráfico de datagramas, como paquetes UDP. En los dos protocolos orientados a conexión lo que se pretende es que, para que *A* se comunique con *B*, primero debe introducir un marco de SOLICITUD DE CONEXIÓN en una ranura libre del canal de control de *B*. Si *B* lo acepta, la comunicación puede llevarse a cabo por el canal de datos de *A*.

Cada estación tiene dos transmisores y dos receptores, como sigue:

1. Un receptor de longitud de onda fija para escuchar su propio canal de control.
2. Un transmisor sintonizable para enviar por el canal de control de otra estación.
3. Un transmisor de longitud de onda fija para la salida de marcos de datos.
4. Un receptor sintonizable para seleccionar el transmisor de datos a escuchar.

En otras palabras, cada estación escucha en su propio canal de control las solicitudes que llegan, pero tiene que sintonizarse a la longitud de onda del transmisor para obtener los datos. La sintonización de la longitud de onda se realiza con un interferómetro Fabry-Perot o Mach-Zehnder que filtra todas las longitudes de onda excepto la banda de longitud de onda deseada.

Consideremos ahora la manera en que la estación *A* establece un canal de comunicación clase 2 con la estación *B* para, digamos, transferencia de archivos. Primero, *A* sintoniza su receptor de datos con el canal de datos de *B* y espera la ranura de estado. Esta ranura indica cuáles ranuras de control están actualmente asignadas y cuáles están libres. En la figura 4-10, por ejemplo, vemos que, de las ocho ranuras de control de *B*, la 0, la 4 y la 5 están libres. El resto están ocupadas (indicado por la "x").

A escoge una de las ranuras de control libres, digamos la 4, e introduce ahí su mensaje de SOLICITUD DE CONEXIÓN. Ya que *B* revisa constantemente su canal de control, ve la solicitud y la acepta asignando la ranura 4 a *A*. Esta asignación se anuncia en la ranura de estado del canal de control. Cuando *A* ve la solicitud, sabe que tiene una conexión unidireccional. Si *A* solicitó una conexión bidireccional, *B* repite ahora el mismo algoritmo con *A*.

Es posible que, al mismo tiempo que *A* intenta tomar la ranura de control 4 de *B*, *C* hace lo mismo. Ninguno lo conseguirá, y ambos se darán cuenta del fracaso revisando la ranura de estado del canal de control de *B*. Ahora ambos esperan una cantidad de tiempo aleatoria y lo reintentan después.

En este punto, cada parte tiene un mecanismo libre de conflictos para enviar mensajes de control cortos entre ellas. Para llevar a cabo la transferencia de archivos, *A* ahora envía a *B* un mensaje de control que indica, por ejemplo, "observa por favor mi siguiente salida de datos en la ranura 3. Hay un marco de datos ahí para ti". Cuando *B* recibe el mensaje de control, sintoniza su receptor al canal de salida de *A* para leer el marco de datos. Dependiendo del protocolo de nivel superior, *B* puede usar el mismo mecanismo para enviar de regreso un reconocimiento, si lo desea.

Note que surge un problema si tanto *A* como *C* tienen conexiones con *B* y cada uno de ellos le indica a *B* que busque en la ranura 3. *B* escogerá uno al azar y se perderá la otra transmisión.

Para tráfico de tasa constante se usa una variación de este protocolo. Cuando *A* solicita una conexión, simultáneamente dice algo como: ¿está bien si te envío un marco cada vez que ocurra la ranura 3? Si *B* puede aceptar (es decir, no tiene compromisos previos para la ranura 3), se establece una conexión de ancho de banda garantizado. Si no, *A* puede intentarlo después con una propuesta distinta, dependiendo de las ranuras que tenga libres.

El tráfico clase 3 (datagramas) usa otra variación. En lugar de escribir un mensaje de SOLICITUD DE CONEXIÓN en la ranura de control que acaba de encontrar (4), escribe un mensaje HAY DATOS PARA TI EN LA RANURA 3. Si *B* está libre durante la siguiente ranura de datos 3, la transmisión tendrá éxito. De otra manera, se perderá el marco de datos. Así, nunca se requieren conexiones.

Son posibles diversas variantes del protocolo completo. Por ejemplo, en lugar de dar a cada estación su propio canal de control, se puede compartir un solo canal de control entre todas las estaciones. Se asigna a cada estación un bloque de ranuras de cada grupo, multiplexando efectivamente varios canales virtuales en uno solo físico.

También es posible arreglárselas con un solo transmisor sintonizable y un solo receptor sintonizable por estación haciendo que el canal de cada estación se divida en m ranuras de control seguidas de $n + 1$ ranuras de datos. La desventaja aquí es que los transmisores tienen que esperar más tiempo para capturar una ranura de control, y los marcos de datos consecutivos están más distantes porque se interpone cierta información de control.

Se han propuesto muchos otros protocolos de control WDMA, difiriendo en los detalles. Algunos tienen un canal de control, otros tienen varios. Algunos toman en cuenta el retardo de propagación, otros no; algunos hacen del tiempo de sintonización una parte explícita del modelo, otros lo ignoran. Los protocolos también difieren en términos de complejidad de procesamiento, rendimiento y capacidad de crecimiento. Para mayor información, véase (Bogineni *et al.*, 1993; Chen, 1994; Chen y Yum, 1991; Jia y Mukherjee, 1993; Levine y Akyildiz, 1995, y Williams *et al.*, 1993).

4.2.6. Protocolos de LAN inalámbricas

A medida que crece la cantidad de dispositivos de cómputo y comunicación portátiles, también crece la demanda de conexión de ellos con el mundo externo. Los primeros teléfonos portátiles ya tenían la capacidad de conectarse con otros teléfonos. Las primeras computadoras portátiles no tenían esta capacidad, pero poco después los módems se volvieron comunes. Para entrar en línea, estas computadoras tenían que conectarse a un receptáculo telefónico. El requisito de una conexión alambrada a la red fija significó que las computadoras eran portátiles, mas no móviles.

Para lograr una auténtica movilidad, las computadoras portátiles necesitan usar señales de radio (o infrarrojas) para comunicarse. De esta manera, los usuarios dedicados pueden leer y enviar correo electrónico mientras conducen o navegan. Un sistema de computadoras portátiles que se comunican por radio puede considerarse una LAN inalámbrica. Estas LAN tienen propiedades un tanto diferentes que las LAN convencionales y requieren protocolos de subcapa MAC especiales. En esta sección estudiaremos algunos de estos protocolos. Puede encontrarse más información sobre las LAN inalámbricas en (Davis y McGuffin, 1995, y Nemzow, 1995).

Una configuración común para una LAN inalámbrica es un edificio de oficinas con estaciones base ubicadas estratégicamente en distintas partes del edificio. Todas las estaciones base están alambradas entre ellas mediante cobre o fibra. Si la potencia de transmisión de las estaciones base y portátiles se ajusta a un alcance de 3 o 4 metros, entonces cada cuarto se vuelve una celda única, y el edificio entero se vuelve un sistema celular grande, como los sistemas de telefonía celular tradicionales que estudiamos en el capítulo 2. A diferencia de los sistemas telefónicos celulares, cada celda sólo tiene un canal, que cubre todo el ancho de banda disponible. Típicamente, el ancho de banda es de 1 a 2 Mbps.

En nuestros análisis siguientes haremos la suposición de simplificación de que todos los transmisores de radio tienen algún alcance fijo. Cuando un receptor está dentro del alcance de dos transmisores activos, la señal resultante generalmente se altera y resulta inútil (pero con ciertas excepciones que se analizarán después). Es importante darse cuenta de que en algunas LAN inalámbricas no todas las estaciones están dentro del alcance de todas las demás, lo que conduce a una variedad de complicaciones. Es más, para las LAN inalámbricas interiores, la presencia de paredes entre las estaciones puede tener un impacto importante sobre el alcance efectivo de cada estación.

Un enfoque inocente para usar una LAN inalámbrica podría ser intentar el CSMA; escuchar si hay otras transmisiones y sólo transmitir si nadie más lo está haciendo. El problema es que este protocolo no es realmente adecuado porque lo que importa es la interferencia en el receptor, no en el transmisor. Para ver la naturaleza de este problema, considere la figura 4-11, en la que se ilustran cuatro estaciones inalámbricas. Para nuestros fines, no importa cuáles son estaciones base ni cuáles son portátiles. El alcance de radio es tal que *A* y *B* están en el mismo alcance y potencialmente pueden interferirse. *C* también podría interferir tanto *B* como *D*, pero no *A*.

Figura 4-11. LAN inalámbrica. (a) *A* transmitiendo. (b) *B* transmitiendo.

Primero considere lo que ocurre cuando *A* está transmitiendo a *B*, como se muestra en la figura 4-11(a). Si *C* detecta el medio, no podrá escuchar a *A* porque *A* está fuera de su alcance, y por tanto deducirá falsamente que puede transmitir. Si *C* comienza a transmitir, interferirá en *B*, eliminando el marco de *A*. El problema de que una estación no puede detectar a un competidor

potencial por el medio, puesto que el competidor está demasiado lejos, se denomina **problema de estación oculta**.

Ahora consideremos la situación inversa: *B* transmitiendo a *A*, como se muestra en la figura 4-11(b). Si *C* detecta el medio, escuchará una transmisión y concluirá equivocadamente que no puede enviar a *D*, cuando de hecho tal transmisión causaría una mala recepción sólo en la zona entre *B* y *C*, en la que no está localizado ninguno de los receptores pretendidos. Esta situación se denomina **problema de estación expuesta**.

El problema es que, antes de comenzar una transmisión, una estación realmente quiere saber si hay actividad o no alrededor del receptor. El CSMA simplemente le indica si hay o no actividad alrededor de la estación que detecta la portadora. Con un alambre, todas las señales se propagan a todas las estaciones, de manera que sólo puede llevarse a cabo una transmisión en un momento dado en cualquier lugar del sistema. En un sistema basado en ondas de radio de corto alcance, pueden ocurrir transmisiones simultáneas si tienen diferentes destinos y estos destinos están fuera de alcance entre sí.

Otra forma de visualizar este problema es imaginar un edificio de oficinas en el que cada empleado tiene una computadora portátil inalámbrica. Suponga que Linda quiere enviar un mensaje a Melitón. La computadora de Linda detecta el ambiente local y, al no percibir actividad, procede a transmitir. Sin embargo, aún puede haber colisión en la oficina de Melitón, pues un tercero podría estar transmitiéndole actualmente desde una localidad tan alejada de la de Linda que su computadora no podría detectarlo.

MACA y MACAW

MACA (*Multiple Access with Collision Avoidance*, **acceso múltiple con prevención de colisiones**) (Karn, 1990) es uno de los primeros protocolos diseñados para LAN inalámbricas. Se usó como base del estándar IEEE 802.11 de LAN inalámbricas. El concepto en que se basa es que el transmisor estimule al receptor a enviar un marco corto, de manera que las estaciones cercanas puedan detectar esta transmisión y eviten ellas mismas hacerlo durante el siguiente marco de datos (grande). El MACA se ilustra en la figura 4-12.

Consideremos ahora la manera en que *A* envía un marco a *B*. *A* comienza por enviar un marco RTS (*Request To Send*, solicitud de envío) a *B*, como se muestra en la figura 4-12(a). Este marco corto (30 bytes) contiene la longitud del marco de datos que seguirá posteriormente. Entonces *B* contesta con un marco CTS (*Clear To Send*, libre para envío), como se muestra en la figura 4-12(b). El marco CTS contiene la longitud de los datos (copiada del marco RTS). A la recepción del marco CTS, *A* comienza la transmisión.

Ahora veamos cómo reaccionan las estaciones que escuchan cualquiera de estos marcos. Cualquier estación que escuche el RTS evidentemente está bastante cerca de *A* y debe permanecer en silencio durante el tiempo suficiente para que el CTS se transmita de regreso a *A* sin conflicto. Cualquier estación que escuche el CTS evidentemente está bastante cerca de *B* y debe permanecer en silencio durante el siguiente tiempo de transmisión de datos, cuya longitud puede determinar examinando el marco CTS.

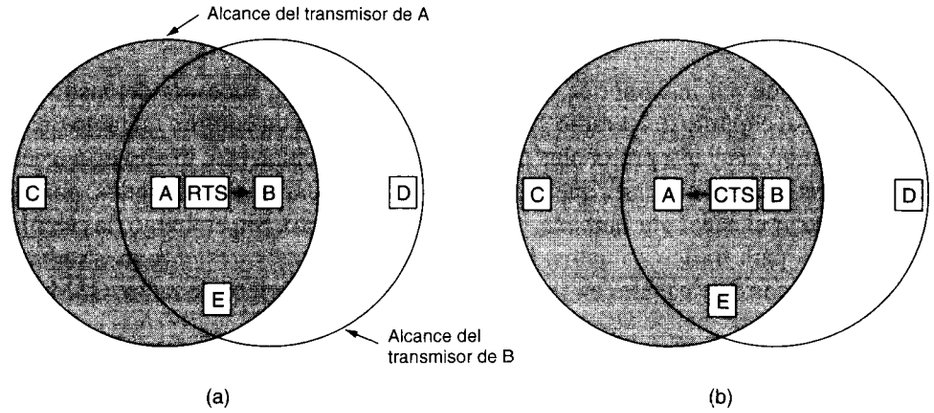


Figura 4-12. El protocolo MACA. (a) *A* enviando a *B* un RTS. (b) *B* respondiendo a *A* con un CTS.

En la figura 4-12, *C* está en el alcance de *A*, pero no en el de *B*. Por tanto, escucha el RTS de *A* pero no el CTS de *B*. Mientras no interfiera con el CTS, está libre para transmitir mientras se está enviando el marco de datos. En contraste, *D* está en el alcance de *B* pero no de *A*. No escucha el RTS pero sí el CTS. El escuchar el CTS le indica que está cerca de una estación que está a punto de recibir un marco, por lo que difiere el envío de cualquier cosa hasta el momento en que se espera la terminación de ese marco. La estación *E* escucha ambos mensajes de control y, como *D*, debe permanecer en silencio hasta que se haya completado el marco de datos.

A pesar de estas precauciones, aún pueden ocurrir colisiones. Por ejemplo, *B* y *C* pueden enviar marcos RTS a *A* al mismo tiempo. Estos chocarán y se perderán. En el caso de una colisión, un transmisor sin éxito (es decir, uno que no escucha un CTS en la ranura de tiempo esperado) espera un tiempo aleatorio y reintenta. El algoritmo empleado es el de retroceso exponencial binario, que estudiaremos cuando lleguemos a las LAN IEEE 802.3.

Con base en estudios de simulación del MACA, Bharghavan *et al.* (1994) afinaron el MACA para mejorar su desempeño y llamaron **MACAW** a su nuevo protocolo. Para comenzar notaron que, sin acuse de recibo de la capa de enlace de datos, los marcos no eran retransmitidos sino hasta que la capa de transporte notaba su ausencia, mucho después. Resolvieron este problema introduciendo un marco ACK tras cada marco de datos exitoso. También observaron que el CSMA puede servir para evitar que una estación transmita un RTS al mismo tiempo que otra estación cercana está haciendo lo mismo al mismo destino, por lo que se agregó la detección de portadora. Además, decidieron ejecutar el algoritmo de retroceso por separado para cada corriente de datos (par origen-destino), en lugar de para cada estación. Este cambio mejora la equidad del protocolo. Finalmente, agregaron un mecanismo para que las estaciones intercambiaran información sobre congestionamientos, y una manera de hacer que el algoritmo de retroceso reaccionara menos violentamente a problemas pasajeros, mejorando así el desempeño del sistema.

4.2.7. Radio celular digital

Una segunda forma de red inalámbrica es la radio celular digital, sucesora del sistema AMPS que estudiamos en el capítulo 2. La radio celular digital presenta un entorno un tanto distinto del de las LAN inalámbricas y usa protocolos diferentes. En particular, este medio está orientado a la telefonía, que requiere conexiones que duran minutos, en lugar de milisegundos, por lo que es más eficiente repartir el canal por llamada, en lugar de por marco. Sin embargo, las técnicas son igualmente válidas para tráfico de datos. En esta sección veremos tres enfoques radicalmente diferentes para el reparto de canales en sistemas digitales inalámbricos de radio, GSM, CDPD y CDMA.

GSM — Sistema global para comunicaciones móviles

La primera generación de teléfonos celulares era analógica, como se describió en el capítulo 2, pero la generación actual es digital y utiliza radio por paquetes. La transmisión digital tiene varias ventajas sobre la analógica para la comunicación móvil. Primero, pueden integrarse en un solo sistema voz, datos y fax. Segundo, a medida que se descubran mejores algoritmos de compresión de voz, se requerirá menos ancho de banda por canal. Tercero, se pueden usar códigos de corrección de errores para mejorar la calidad de la transmisión. Por último, las señales digitales pueden cifrarse para mejorar la seguridad.

Aunque podría haber sido agradable que todo el mundo hubiera adoptado el mismo estándar digital, no sucedió así. El sistema de Estados Unidos, IS-54, y el sistema japonés, JDC, fueron diseñados para ser compatibles con el sistema analógico de cada país, para que cada canal AMPS pudiera usarse tanto con comunicación analógica como con digital.

En contraste, el sistema digital europeo, **GSM** (*Global System for Mobile communications*, **sistema global para comunicaciones móviles**), se diseñó a partir de cero como un sistema completamente digital, sin concesiones para tener compatibilidad hacia atrás (por ejemplo, tener que usar las ranuras de frecuencia existentes). Dado que GSM también está más adelantado que el sistema de Estados Unidos y actualmente se usa en más de 50 países, dentro y fuera de Europa, lo tomaremos como ejemplo de radio celular digital.

GSM se diseñó originalmente para usarse en la banda de 900 MHz. Después, se repartieron frecuencias a 1800 MHz y se estableció ahí un segundo sistema, diseñado de manera muy semejante a GSM. Este sistema se llama **DCS 1800**, pero esencialmente es GSM.

El estándar GSM tiene más de 5000 [sic] páginas. Una parte considerable de este material se relaciona con los aspectos de ingeniería del sistema, especialmente el diseño de receptores para manejar la propagación multitrayectoria de señales y la sincronización de transmisores y receptores.

Un sistema GSM tiene un máximo de 200 canales dúplex por celda. Cada canal consiste en una frecuencia de enlace descendente (de la estación base a las estaciones móviles) y una frecuencia de enlace ascendente (de las estaciones móviles a la estación base). Cada banda de frecuencia tiene un ancho de 200 kHz, como se muestra en la figura 4-13.

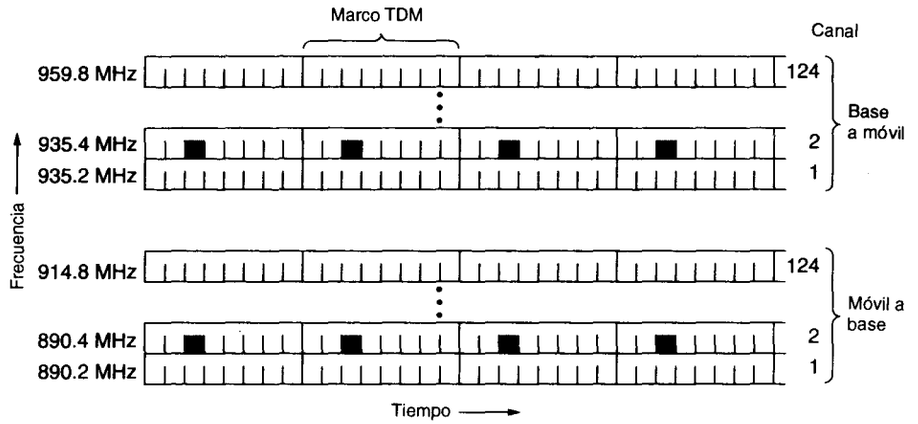


Figura 4-13. El GSM usa 124 canales de frecuencia, cada uno de los cuales emplea un sistema TDM de ocho ranuras.

Cada uno de los 124 canales de frecuencia maneja ocho conexiones separadas que usan multiplexión por división en el tiempo. Se asigna una ranura de tiempo en un canal a cada estación activa. En teoría, pueden reconocerse 992 canales por celda, pero la mayoría de ellos no están disponibles, a fin de evitar conflictos de frecuencia con las celdas vecinas. En la figura 4-13 las ocho ranuras de tiempo sombreadas pertenecen al mismo canal, cuatro de ellas para cada dirección. Si la estación móvil estuviera asignada a 890.4/935.4 MHz y la ranura 2 quisiera transmitir a la estación base, usaría las cuatro ranuras sombreadas inferiores (y las que le siguieran en el tiempo), poniendo datos en cada ranura hasta haber enviado todos los datos.

Las ranuras TDM mostradas en la figura 4-13 son parte de una compleja jerarquía de enmarcado. Cada ranura TDM tiene una estructura específica, y los grupos de ranuras TDM forman multimarcos, también con una estructura específica. En la figura 4-14 se muestra una versión simplificada de esta jerarquía. Aquí podemos ver que cada ranura TDM consiste en un marco de datos de 148 bits. Cada marco de datos comienza y termina con tres bits 0, para propósitos de delimitación del marco. También contiene dos campos de *información* de 57 bits, teniendo cada uno un bit de control que indica si el siguiente campo de *información* es para voz o datos. Entre los campos de *información* está un campo *sync* (instrucción) de 26 bits usado por el receptor para sincronizarse con los límites de marco del transmisor. Se transmite un marco de datos en 547 μseg, pero sólo se permite que un transmisor envíe un marco de datos cada 4.615 mseg, dado que está compartiendo el canal con otras siete estaciones. La tasa bruta de cada canal es de 270,833 bps, dividida entre ocho usuarios. Descontando la información extra, cada conexión puede enviar una señal comprimida de voz o 9600 bps de datos.

Como puede verse en la figura 4-14, ocho marcos de datos constituyen un marco TDM, y 26 marcos TDM constituyen un multimarco de 120 mseg. De los 26 marcos TDM en un multimarco,

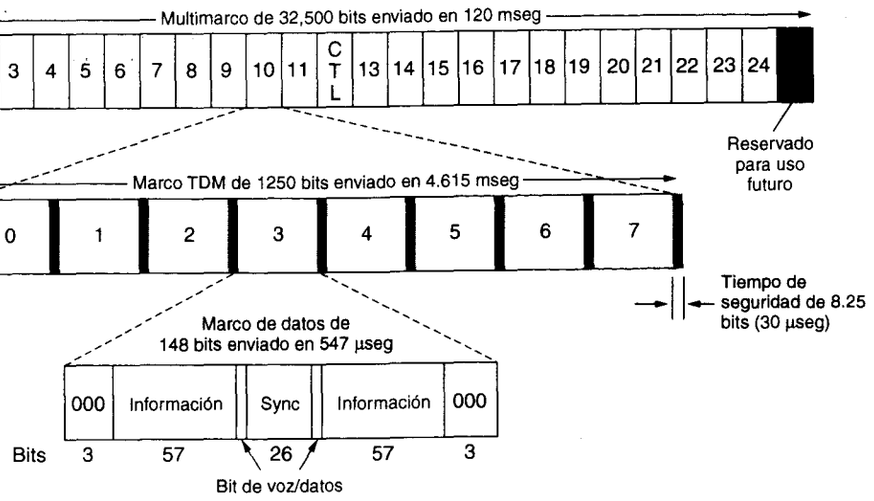


Figura 4-14. Parte de la estructura de enmarcado GSM.

la ranura 12 se usa para control y la ranura 25 se reserva para uso futuro, por lo que sólo hay disponibles 24 para el tráfico de usuario.

Sin embargo, además del multimarco de 26 ranuras mostrado en la figura 4-14, se usa también un multimarco de 51 ranuras (que no se muestra). Algunas de estas ranuras se usan para contener varios canales de control con los que se maneja el sistema. El **canal de control de difusión** es una corriente continua de salida de la estación base que contiene su identidad y el estado del canal. Todas las estaciones móviles observan la intensidad de su señal para ver cuándo han pasado a una celda nueva.

El **canal de control dedicado** se usa para la actualización de la ubicación, el registro y la preparación de llamadas. En particular, cada estación base mantiene una base de datos de las estaciones móviles que están actualmente bajo su jurisdicción. La información necesaria para mantener esta base de datos se envía por el canal de control dedicado.

Por último, está el **canal de control común**, que se divide en tres subcanales lógicos. El primero de estos subcanales es el **canal de anunciación**, que usa la estación base para anunciar las llamadas entrantes. Cada estación móvil lo vigila continuamente para detectar llamadas que debe contestar. El segundo es el **canal de acceso aleatorio**, que ejecuta un sistema ALOHA ranurado para que una estación móvil pueda solicitar una ranura en el canal dedicado de control. Usando esta ranura, la estación puede establecer una llamada. La ranura asignada se anuncia por el tercer subcanal, el **canal de otorgamiento de acceso**.

En conjunto, GSM es un sistema bastante complejo. Maneja el acceso a canales mediante una combinación de ALOHA ranurado, FDM y TDM. Para mayor información sobre el GSM, incluidos aspectos del sistema que no hemos estudiado (por ejemplo, la arquitectura de capas del protocolo), véase (Rahnema, 1993).

UNIVERSIDAD DE LA
FACULTAD DE IN
DEPARTAMEN
DOCUMENTACION
MONTEVIDEO - U

CDPD — Paquete celular digital de datos

Básicamente, GSM es de conmutación de circuitos. Una computadora móvil con un módem especial puede hacer una llamada usando un teléfono GSM de la misma manera que haría una en un teléfono alámbrico. Sin embargo, el empleo de esta estrategia no está exento de problemas. Por una parte, los pases entre las estaciones base son frecuentes, a veces incluso con usuarios estacionarios (las estaciones base pueden repartir a los usuarios para balancear la carga), y cada pase resulta en la pérdida de cerca de 300 mseg de datos. Por otra, GSM puede tener una tasa de errores elevada. La escritura de una "a" y la aparición de su eco como una "m" pronto se vuelve molesto. Por último, las llamadas inalámbricas son caras, y los costos se acumulan con rapidez porque el cargo es por minuto de conexión, no por byte enviado.

Un enfoque para resolver estos problemas es un servicio de datagrama digital conmutado por paquete llamado **CDPD** (*Cellular Digital Packet Data*, **paquete celular digital de datos**). CDPD está construido encima de AMPS (vea el capítulo 2) y es completamente compatible con AMPS. Básicamente, cualquier canal de 30 kHz inactivo se puede tomar temporalmente para enviar marcos de datos a una tasa bruta de 19.2 kbps. Como CDPD implica bastante información extra, la tasa neta de datos es más cercana a 9600 bps. Aun así, un sistema de datagramas inalámbrico para enviar, digamos, paquetes IP aprovechando el sistema existente de telefonía celular es una propuesta interesante para muchos usuarios, por lo que está creciendo con rapidez.

CDPD sigue estrechamente el modelo OSI. La capa física maneja los detalles de la modulación y la radiotransmisión, lo que no nos interesa aquí. También existen protocolos de enlace de datos, de red y de transporte, pero tampoco son especialmente interesantes para nosotros. Más bien, daremos una descripción general del sistema y luego describiremos el protocolo de acceso al medio. Para mayor información sobre el sistema CDPD completo, véase (Quick y Balachandran, 1993).

Un sistema CDPD consiste en tres tipos de estaciones: *hosts* móviles, estaciones base y estaciones base de interfaz (en la jerga de CDPD: sistemas móviles finales, sistemas móviles de base de datos y sistemas móviles intermedios de datos, respectivamente). Estas estaciones interactúan con *hosts* estacionarios y enrutadores estándar, del tipo encontrado en cualquier WAN. Los *hosts* móviles son las computadoras portátiles de los usuarios. Las estaciones base son los transmisores que hablan con los *hosts* móviles. Las estaciones base de interfaz son nodos especiales que establecen la interfaz entre todas las estaciones base del área de un proveedor de CDPD y un enrutador estándar (fijo) para transmisión posterior a través de Internet u otra WAN. Este arreglo se muestra en la figura 4-15.

Se definen tres tipos de interfaz en CDPD. La **interfaz E** (externa al proveedor de CDPD) conecta un área de CDPD a una red fija. Esta interfaz debe estar bien definida para permitir que CDPD se conecte a una variedad de redes. La **interfaz I** (interna al proveedor de CDPD) conecta dos áreas de CDPD entre ellas; debe estar estandarizada para permitir que los usuarios viajen entre áreas. La tercera es la **interfaz A** (interfaz de aire) entre la estación base y los *hosts* móviles. Ésta es la más interesante, por lo que ahora la examinaremos con mayor detalle.

Los datos a través de la interfaz de aire se envían usando compresión, cifrado y corrección de errores. Unidades de 274 bits de datos comprimidos y cifrados se envuelven en bloques de 378 bits usando un código de corrección de errores Reed-Solomon. A cada bloque RS se agregan siete

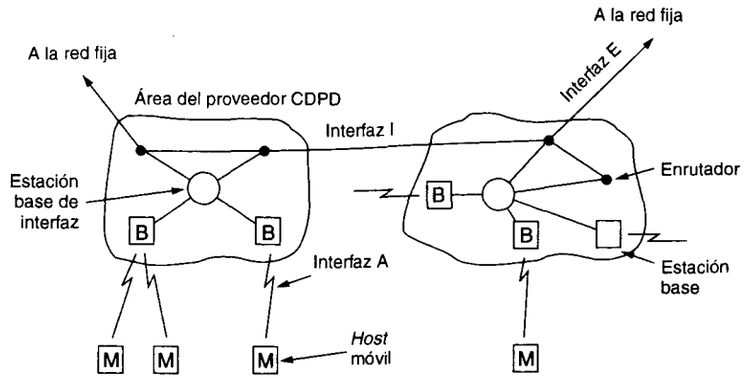


Figura 4-15. Ejemplo de sistema CDPD.

palabras indicadoras de 6 bits para formar bloques de un total de 420 bits. Cada bloque de 420 bits se divide en siete microbloques de 60 bits, que se envían consecutivamente. Cada microbloque tiene su propia palabra indicadora de 6 bits que sirve para indicar el estado del canal. Los microbloques pasan a través de un canal de enlace descendente de 19.2 kbps (de la base) o a través de un segundo canal de enlace ascendente de 19.2 kbps (a la base), en modo dúplex integral. En la práctica, tanto el canal de enlace descendente como el de enlace ascendente están ranurados en el tiempo, como una secuencia de microbloques de 60 bits. Cada microbloque dura 3.125 mseg.

Cada celda CDPD tiene disponible sólo un par de enlace descendente y enlace ascendente para datos. El canal de enlace descendente es fácil de manejar, ya que sólo hay un transmisor por celda: la estación base. Todos los marcos enviados se difunden, seleccionando cada *host* móvil aquellos destinados a él o a todos.

La parte complicada es el canal de enlace ascendente, por el que deben contender todos los *hosts* móviles que desean enviar. Cuando un *host* móvil tiene un marco por enviar, busca en el canal de enlace descendente un bit indicador que le indique si la ranura de enlace ascendente actual está ocupada o libre. Si está ocupada, en lugar de esperar la siguiente ranura de tiempo, el *host* salta un número aleatorio de ranuras y reintenta; si nuevamente ve que el canal de enlace ascendente está ocupado, espera un tiempo aleatorio mayor y repite el procedimiento. El tiempo estadístico medio de espera se duplica con cada intento fracasado. Cuando el *host* finalmente encuentra el c nal supuestamente inactivo, comienza a transmitir su microbloque.

El objeto de este algoritmo, llamado **DSMA** (*Digital Sense Multiple Access*, **acceso múltiple de detección digital**), es evitar que todas las estaciones móviles salten al canal de enlace ascendente tan pronto como se desocupe. Este mecanismo semeja un tanto el protocolo CSMA persistente-p que mencionamos antes, ya que también usa ranuras de tiempo discretas en ambos canales.

El problema es que, a pesar de DSMA, aún es posible una colisión con otro *host* móvil, ya que dos o más de ellos pueden escoger la misma ranura de tiempo para comenzar a transmitir. A fin de que los *hosts* móviles puedan descubrir si han sufrido una colisión, un bit indicador de cada microbloque indica si fue recibido correctamente un microbloque previo por el canal de en-

lace ascendente. Por desgracia, la estación base no puede tomar la determinación instantáneamente tras la terminación de un microbloque, por lo que la recepción correcta/incorrecta del microbloque n se retarda hasta el microbloque $n + 2$.

Puesto que un transmisor no puede determinar si una transmisión tuvo éxito, si el transmisor tiene más microbloques por enviar simplemente lo hace, sin tener que readquirir el canal. Si en la *siguiente* ranura de tiempo ve que falló su transmisión *previa*, se detiene. De otra manera, continúa transmitiendo hasta cierto número máximo de bloques Reed-Solomon o hasta que la estación base establece un bit indicador en el canal de enlace descendente para indicar que ha escuchado suficiente de este transmisor en particular por el momento.

Una propiedad adicional de CDPD es que los usuarios de datos son ciudadanos de segunda clase. Cuando una llamada de voz está a punto de ser asignada a un canal actualmente en uso por CDPD, la estación base envía una señal especial por el enlace descendente, cerrando el canal. Si la estación base ya sabe el número del nuevo canal CDPD, lo anuncia. De otra manera, los *hosts* móviles tienen que andar buscando entre un grupo de canales CDPD para encontrarlo. De esta manera, CDPD puede absorber cualquier capacidad inactiva que pudiera tener una celda, sin interferir con la gallina de los huevos de oro, la voz.

Por esta descripción debe quedar claro que CDPD se agregó al sistema de voz después de que éste ya estaba en operación, y que su diseño estuvo sujeto a la restricción de que no podían hacerse cambios al sistema de voz existente. En consecuencia, cuando ocurre una selección de canal para voz, el algoritmo no está consciente de la existencia de CDPD. Ésta es la razón por la que el canal CDPD a veces es tomado a la fuerza. Sin embargo, nada del diseño evita tener canales CDPD dedicados. A medida que CDPD crece en popularidad, con seguridad los proveedores reservarán canales exclusivamente para él.

CDMA — Acceso múltiple por división de código

Podría describirse GSM como una solución de fuerza bruta al reparto de canales; usa una combinación de prácticamente todas las técnicas conocidas (ALOHA, TDM, FDM) entrelazadas de manera compleja. El CDPD para transmisiones de marcos individuales es fundamentalmente un CSMA no persistente. Ahora examinaremos otro método para repartir un canal inalámbrico, **CDMA** (*Code Division Multiple Access*, **acceso múltiple por división de código**).

CDMA es completamente diferente de las demás técnicas de reparto que hemos estudiado hasta ahora. Algunas de éstas se han basado en la división del canal en bandas de frecuencia y la asignación de ella estáticamente (FDM) o por demanda (multiplexión por división en longitud de onda), con el propietario usando indefinidamente la banda. Otras reparten el canal en ráfagas, dando estáticamente a las estaciones el canal completo (TDM con ranuras de tiempo fijas) o dinámicamente (ALOHA). El CDMA permite que cada estación transmita en el espectro completo de frecuencia todo el tiempo. Las transmisiones múltiples simultáneas se separan usando la teoría de codificación. El CDMA también relaja el supuesto de que los marcos en colisión se alteran completamente. En cambio, supone que las señales múltiples se suman linealmente.

Antes de entrar en el algoritmo, consideremos la teoría de la fiesta de coctel de acceso a un canal. En un cuarto grande, muchas parejas de personas están conversando. TDM es cuando toda

la gente está en el centro del cuarto, pero se turnan para hablar, primero uno, luego otro. FDM es cuando la gente se junta en grupos ampliamente dispersos, llevando cada grupo su propia conversación al mismo tiempo que los demás, pero independientemente. CDMA es cuando todos están en el centro del salón hablando al mismo tiempo, pero hablando cada pareja un lenguaje distinto. La pareja de francoparlantes sólo se concentra en el francés, rechazando todo lo demás como ruido. Por tanto, la clave de CDMA es ser capaz de extraer la señal deseada mientras se rechaza todo lo demás como ruido aleatorio.

En CDMA, cada tiempo de bit se subdivide en m intervalos cortos llamados ***chips***. Comúnmente hay 64 o 128 *chips* por bit, pero en el ejemplo dado a continuación usaremos por sencillez 8 *chips*/bit.

A cada estación se asigna un código único de m bits, o **secuencia de *chips***. Para transmitir un bit 1, una estación envía su secuencia de *chips*; para transmitir un bit 0, envía el complemento a 1 de su secuencia de *chips*. No se permiten otros patrones. Por tanto, para $m = 8$, si la estación *A* tiene asignada la secuencia de *chips* 00011011, envía un bit 1 transmitiendo 00011011, y un bit 0 enviando 11100100.

El aumento en la cantidad de información a ser enviada de b bits/seg a mb *chips*/seg sólo puede lograrse si el ancho de banda disponible se incrementa por un factor de m, haciendo de CDMA una forma de comunicación de espectro amplio (suponiendo que no hay cambios en la técnica de modulación ni de codificación). Si tenemos una banda de 1 MHz disponible para 100 estaciones, con FDM cada una tendría 10 kHz y podría enviar a 10 kbps (suponiendo 1 bit por Hz). Con CDMA, cada estación usa la totalidad del 1 MHz, por lo que la razón de *chips* es de 1 *megachip* por segundo. Con menos de 100 *chips* por bit, el ancho de banda efectivo por estación es mayor para CDMA que para FDM y se resuelve también el problema de reparto del canal, como veremos en breve.

Para fines pedagógicos es más conveniente usar una notación bipolar, siendo el 0 binario −1 y el 1 binario +1. Mostraremos las secuencias de *chips* entre paréntesis, por lo que un bit para la estación *A* ahora se convierte en (−1 −1 −1 +1 +1 −1 +1 +1). En la figura 4-16(a) mostramos las secuencias de *chips* binarios asignadas a cuatro estaciones de ejemplo. En la figura 4-16(b) las mostramos en nuestra notación bipolar.

Cada estación tiene su propia secuencia de *chips* única. Utilicemos el símbolo **S** para indicar el vector m de los *chips* para la estación S y **S** para su negación. Todas las secuencias de *chips* son **ortogonales** en pares, con lo que queremos decir que el producto escalar normalizado de dos secuencias de *chips* distintas, **S** y **T** (que se escriben como **S•T**) es 0. En términos matemáticos,

$$\mathbf{S} \bullet \mathbf{T} \equiv \frac{1}{m} \sum_{i=1}^{m} S_i T_i = 0 \qquad (4\text{-}5)$$

En español llano, hay tantos pares iguales como diferentes. Esta propiedad de ortogonalidad demostrará ser crucial más adelante. Nótese que, si **S•T** = 0, entonces **S•$\overline{\mathbf{T}}$** también es 0. El producto escalar normalizado de cualquier secuencia de *chips* con ella misma es 1:

$$\mathbf{S} \bullet \mathbf{S} = \frac{1}{m} \sum_{i=1}^{m} S_i S_i = \frac{1}{m} \sum_{i=1}^{m} S_i^2 = \frac{1}{m} \sum_{i=1}^{m} (\pm 1)^2 = 1$$

A: 0 0 0 1 1 0 1 1	A: (−1 −1 −1 +1 +1 −1 +1 +1)
B: 0 0 1 0 1 1 1 0	B: (−1 −1 +1 −1 +1 +1 +1 −1)
C: 0 1 0 1 1 1 0 0	C: (−1 +1 −1 +1 +1 +1 −1 −1)
D: 0 1 0 0 0 0 1 0	D: (−1 +1 −1 −1 −1 −1 +1 −1)
(a)	(b)

Seis ejemplos:

− − 1 −	**C**	S_1 = (−1 +1 −1 +1 +1 +1 −1 −1)
− 1 1 −	**B** + **C**	S_2 = (−2 0 0 0 +2 +2 0 −2)
1 0 − −	**A** + **B**	S_3 = (0 0 −2 +2 0 −2 0 +2)
1 0 1 −	**A** + **B** + **C**	S_4 = (−1 +1 −3 +3 −1 −1 −1 +1)
1 1 1 1	**A** + **B** + **C** + **D**	S_5 = (−4 0 −2 0 +2 0 +2 −2)
1 1 0 1	**A** + **B** + $\overline{\mathbf{C}}$ + **D**	S_6 = (−2 −2 0 −2 0 −2 +4 0)

(c)

$S_1 \bullet C = (1 +1 +1 +1 +1 +1 +1 +1)/8 = 1$
$S_2 \bullet C = (2 +0 +0 +0 +2 +2 +0 +2)/8 = 1$
$S_3 \bullet C = (0 +0 +2 +2 +0 −2 +0 −2)/8 = 0$
$S_4 \bullet C = (1 +1 +3 +3 +1 −1 +1 −1)/8 = 1$
$S_5 \bullet C = (4 +0 +2 +0 +2 +0 −2 +2)/8 = 1$
$S_6 \bullet C = (2 −2 +0 −2 +0 −2 −4 +0)/8 = −1$

(d)

Figura 4-16. (a) Secuencias de chips binarios para cuatro estaciones. (b) Secuencias de chips bipolares. (c) Seis ejemplos de transmisión. (d) Recuperación de la señal de la estación C.

Esto ocurre porque cada uno de los m términos del producto escalar es 1, por lo que la suma es m. También nótese que **S•$\overline{\mathbf{S}}$** = −1.

Durante cada tiempo de bit, una estación puede transmitir un 1 enviando su secuencia de *chips*, puede transmitir un 0 enviando el negativo de su secuencia de *chips* o puede permanecer en silencio y no transmitir nada. Por el momento, supondremos que todas las estaciones están sincronizadas en el tiempo, por lo que todas las secuencias de *chips* comienzan en el mismo momento.

Cuando dos o más estaciones transmiten simultáneamente, sus señales bipolares se suman linealmente. Por ejemplo, si durante un periodo de *chip* la salida de tres estaciones es +1 y la de una estación es −1, el resultado es +2. Se puede pensar en esto como la suma de voltajes: tres estaciones con una salida de +1 volts y una estación con una salida de −1 volts da 2 volts.

En la figura 4-16(c) vemos seis ejemplos de una o más estaciones transmitiendo al mismo tiempo. En el primer ejemplo, *C* transmite un bit 1, por lo que sólo obtenemos la secuencia de *chips* de *C*. En el segundo ejemplo, tanto *B* como *C* transmiten bits 1, por lo que obtenemos la suma de sus secuencias bipolares de *chips*, es decir:

(−1 −1 +1 −1 +1 +1 +1 −1) + (−1 +1 −1 +1 +1 +1 −1 −1) = (−2 0 0 0 +2 +2 0 −2)

En el tercer ejemplo, la estación *A* envía un 1 y la estación *B* envía un 0. Las otras están en silencio. En el cuarto ejemplo, *A* y *C* envían un bit 1 mientras *B* envía un bit 0. En el quinto

ejemplo, las cuatro estaciones envían un bit 1. Finalmente, en el último ejemplo, *A*, *B* y *D* envían un bit 1, mientras que *C* envía un bit 0. Nótese que cada una de las seis secuencias S_1 a S_6 dadas en la figura 4-16(c) representa un solo tiempo de bit.

Para recuperar la cadena de bits de una estación individual, el receptor debe conocer las secuencias de *chips* de esa estación por adelantado. El receptor hace la recuperación calculando el producto escalar normalizado de la secuencia de *chips* recibida (la suma lineal de todas las estaciones que transmitieron) y la secuencia de *chips* de la estación cuya corriente de bits está tratando de recuperar. Si la secuencia de *chips* recibida es **S** y el receptor está tratando de escuchar una estación cuya secuencia de *chips* es **C**, simplemente calcula el producto escalar normalizado, **S•C**.

Para ver por qué esto funciona, imagine que dos estaciones, *A* y *C*, transmiten un bit 1 al mismo tiempo que *B* transmite un bit 0. El receptor ve la suma: $\mathbf{S} = \mathbf{A} + \overline{\mathbf{B}} + \mathbf{C}$ y calcula

$$\mathbf{S}\bullet\mathbf{C} = (\mathbf{A} + \overline{\mathbf{B}} + \mathbf{C})\bullet\mathbf{C} = \mathbf{A}\bullet\mathbf{C} + \overline{\mathbf{B}}\bullet\mathbf{C} + \mathbf{C}\bullet\mathbf{C} = 0 + 0 + 1 = 1$$

Los primeros dos términos desaparecen porque todos los pares de secuencias de *chips* se han escogido cuidadosamente para que sean ortogonales, como se muestra en la ecuación (4-5). Ahora debe quedar claro por qué debe imponerse esta propiedad a las secuencias de *chips*.

Una manera alterna de conceptuar esta situación es imaginar que las tres secuencias de *chips* entraron todas por separado, en lugar de sumadas. Entonces el receptor calculará por separado el producto escalar de cada una y sumará los resultados. Gracias a la propiedad de ortogonalidad, todos los productos escalares excepto **C•C** serán 0. Sumarlos y obtener después el producto escalar es, de hecho, lo mismo que obtener los productos escalares y luego sumarlos.

Para hacer más concreto el proceso de decodificación, consideremos nuevamente los seis ejemplos de la figura 4-16(d). Suponga que el receptor está interesado en extraer el bit enviado por la estación *C* de cada una de las seis sumas S_1 a S_6. El receptor calcula el bit sumando los productos en pares del **S** recibido y el vector **C** de la figura 4-16(b), y dividiendo luego el resultado entre 8 (dado que $m = 8$ aquí). Como se muestra, siempre se decodifica el bit correcto. Es igual que hablar francés.

En un sistema CDMA ideal sin ruido, la capacidad (es decir, el número de estaciones) puede hacerse arbitrariamente grande, de la misma manera que puede hacerse arbitrariamente grande la capacidad de un canal Nyquist sin ruido usando más y más bits por muestra. En la práctica, las limitaciones físicas reducen considerablemente la capacidad. Primero, hemos supuesto que todos los *chips* están sincronizados en el tiempo. En la realidad, esto es imposible. Lo que puede hacerse es que el transmisor y el receptor se sincronicen pidiendo al transmisor que envíe una secuencia de *chips* conocida, de longitud suficiente, a la que se pueda enganchar el receptor. Todas las demás transmisiones (no sincronizadas) son vistas entonces como ruido aleatorio. Sin embargo, si no hay demasiadas transmisiones, el algoritmo básico de decodificación aún funciona bastante bien. Se ha realizado gran cantidad de trabajo teórico que relaciona la superposición de las secuencias de *chips* con el nivel de ruido (Pickholtz *et al.*, 1982). Como podría esperarse, cuanto mayor es la secuencia de *chips*, mayor es la probabilidad de detectarla correctamente en presencia de ruido. Si se desea seguridad extra, la secuencia de bits puede usar un código de corrección de errores. Las secuencias de *chips* nunca utilizan códigos de corrección de errores.

Un supuesto implícito en el análisis anterior es que los niveles de potencia de todas las estaciones son iguales, según lo percibe el receptor. CDMA se usa comúnmente para sistemas inalámbricos con una estación base fija y muchas estaciones móviles a distancias variables de ella. Los niveles de potencia recibidos en la estación base dependen de la distancia a la que se encuentran los transmisores. Aquí, una buena heurística es que cada estación móvil transmita a la estación base en un nivel de potencia inverso al que recibe de la estación base, por lo que una estación móvil que reciba una señal débil de la base usará más potencia que una que recibe una señal fuerte. La estación base también puede dar órdenes explícitas a las estaciones móviles para que aumenten o disminuyan su potencia de transmisión.

También hemos supuesto que el receptor sabe quién es el transmisor. En principio, dada una capacidad de cómputo suficiente, el receptor puede escuchar todos los transmisores a la vez ejecutando en paralelo el algoritmo de decodificación para cada uno de ellos. En la realidad, baste con mencionar que es más fácil decirlo que hacerlo. CDMA también tiene muchos otros factores de complicación que han sido pasados por alto en esta breve introducción. No obstante, CDMA es un esquema ingenioso que está siendo aceptado rápidamente para comunicaciones inalámbricas móviles.

Los lectores con antecedentes sólidos en ingeniería eléctrica que quieran entender con mayor profundidad el CDMA deben leer (Viterbi, 1995). Un esquema alterno de dispersión, en el que la dispersión se hace en el tiempo en lugar de la frecuencia, se describe en (Crespo *et al.*, 1995).

4.3. ESTÁNDAR IEEE 802 PARA LAN Y MAN

Ya hemos terminado nuestro estudio general de los protocolos abstractos de reparto de canal, por lo que es tiempo de ver la manera en que se aplican estos principios a los sistemas reales, en particular a las LAN. Como se explicó en la sección 1.7.2, el IEEE ha producido varios estándares para las LAN. Estos estándares, conocidos en conjunto como **IEEE 802**, incluyen CSMA/CD, *token bus* y *token ring*. Los diferentes estándares difieren en la capa física y en la subcapa MAC, pero son compatibles en la capa de enlace de datos. Los estándares IEEE 802 han sido adoptados por el ANSI como estándares nacionales de Estados Unidos, por el NIST como estándares del gobierno de Estados Unidos y por la ISO como estándares internacionales (conocidos como ISO 8802). Estos documentos son sorprendentemente fáciles de entender (para ser estándares).

Los estándares se dividen en dos partes, cada una publicada como libro independiente. El estándar 802.1 es una introducción al grupo de estándares y define las primitivas de la interfaz. El estándar 802.2 describe la parte superior de la capa de enlace de datos, que usa el protocolo **LLC** (*Logical Link Control*, **control de enlace lógico**). Las partes 802.3 a 802.5 describen los tres estándares para LAN, CSMA/CD, *token bus* y *token ring*, respectivamente. Cada estándar cubre la capa física y el protocolo de la subcapa MAC. Las tres secciones siguientes cubren estos tres sistemas. Se puede encontrar información adicional en (Stallings, 1993b).

4.3.1. Estándar IEEE 802.3 y Ethernet

El estándar IEEE 802.3 es para una LAN CSMA/CD persistente-1. Para repasar el concepto, cuando una estación quiere transmitir, escucha el cable. Si el cable está ocupado, la estación espera hasta que se desocupa; de otra manera, transmite de inmediato. Si dos o más estaciones comienzan a transmitir simultáneamente por un cable inactivo, habrá una colisión. Todas las estaciones en colisión terminan entonces su transmisión, esperan un tiempo aleatorio y repiten de nuevo todo el proceso.

El estándar 802.3 tiene una historia interesante. El comienzo real fue el sistema ALOHA construido para permitir la comunicación por radio entre máquinas diseminadas por las islas hawaianas. Posteriormente, se agregó detección de portadora, y Xerox PARC construyó un sistema CSMA/CD de 2.94 Mbps para conectar más de 100 estaciones de trabajo personales a un cable de 1 km (Metcalfe y Boggs, 1976). Este sistema fue bautizado **Ethernet** por el *éter luminífero*, a través del cual alguna vez se pensó que se propagaba la radiación electromagnética. (Cuando el físico británico del siglo XIX James Clerk Maxwell descubrió que la radiación electromagnética podía describirse mediante una ecuación de onda, los científicos supusieron que el espacio debía estar lleno de algún medio etéreo en el que se propagaba la radiación. Sólo tras el famoso experimento de Michelson y Morley, en 1887, descubrieron los físicos que la radiación electromagnética podía propagarse en el vacío.)

La Ethernet de Xerox tuvo tanto éxito que Xerox, DEC e Intel diseñaron un estándar para una Ethernet de 10 Mbps. Este estándar formó la base del 802.3. El estándar 802.3 publicado difiere de la especificación Ethernet en cuanto a que describe una familia completa de sistemas CSMA/CD persistente-1, operando a velocidades de 1 a 10 Mbps en varios medios. También, el único campo de cabecera difiere entre los dos (el campo de longitud del 802.3 se usa para el tipo de paquete en Ethernet). El estándar inicial también da los parámetros para un sistema de banda base de 10 Mbps usando cable coaxial de 50 ohms. Los grupos de parámetros para los demás medios y velocidades vinieron después.

Mucha gente usa (incorrectamente) el nombre "Ethernet" en un sentido genérico para referirse a todos los protocolos CSMA/CD, aun cuando en realidad se refiere a un producto específico que casi implementa el 802.3. En los siguientes párrafos usaremos los términos "802.3" y "CSMA/CD", excepto cuando nos refiramos específicamente al producto Ethernet.

Cableado del 802.3

Dado que el nombre "Ethernet" se refiere al cable (el éter), comencemos nuestro estudio por ahí. Comúnmente se usan cinco tipos de cableado, como se muestra en la figura 4-17. Históricamente, llegó primero el cable **10Base5**, llamado popularmente **Ethernet grueso**; semeja una manguera de jardín amarilla, con marcas cada 2.5 metros para indicar los puntos de las derivaciones. (El estándar 802.3 no *requiere* realmente que el cable sea amarillo, pero sí lo *sugiere*.) Las conexiones a él se hacen generalmente usando **derivaciones vampiro**, en las que se introduce cuidadosamente una punta hasta la mitad del núcleo del cable coaxial. La notación 10Base5

Nombre	Cable	Segmento máximo	Nodos/seg	Ventajas
10Base5	Coaxial grueso	500 m	100	Bueno para *backbone*
10Base2	Coaxial delgado	200 m	30	Sistema más barato
10Base-T	Par trenzado	100 m	1024	Fácil mantenimiento
10Base-F	Fibra óptica	2000 m	1024	Mejor entre edificios

Figura 4-17. Los tipos más comunes de LAN 802.3 de banda base.

significa que opera a 10 Mbps, usa señalización de banda base y puede manejar segmentos de hasta 500 metros.

Históricamente, el segundo tipo de cable fue **10Base2** o **Ethernet delgado** que, a diferencia con el Ethernet grueso parecido a una manguera de jardín, se dobla con facilidad. Las conexiones se hacen usando conectores BNC estándar de la industria para formar uniones T, en lugar de usar derivaciones vampiro. Éstas son más fáciles de usar y más confiables. El Ethernet delgado es mucho más barato y más fácil de instalar, pero sólo puede extenderse 200 metros y puede manejar sólo 30 máquinas por segmento de cable.

La detección de rupturas de cable, derivaciones malas y conectores sueltos puede ser un problema importante en ambos medios. Por esta razón se han desarrollado técnicas para rastrear estos problemas. Básicamente, se inyecta un pulso de forma conocida en el cable. Si el pulso incide en un obstáculo o en el final del cable, se generará un eco que viajará de regreso. Si se cronometra cuidadosamente el intervalo entre el envío del pulso y la recepción del eco, es posible ubicar el origen del eco. Esta técnica se llama **reflectometría en el dominio del tiempo**.

Los problemas asociados con la localización de rupturas de cable han empujado a los sistemas a un tipo de patrón de alambrado diferente, en el que todas las estaciones tienen alambres que conducen a un **concertador** (*hub*) central. Generalmente, estos alambres son pares trenzados telefónicos, ya que la mayoría de los edificios de oficinas ya están alambrados de esta manera y normalmente hay bastantes pares extra disponibles. Este esquema se llama **10Base-T**.

Estos tres esquemas de alambrado se ilustran en la figura 4-18. Para 10Base5, se sujeta firmemente un **transceptor** (*transceiver*) alrededor del cable, de modo que su derivación haga contacto con el núcleo interno. El transceptor contiene la electrónica que maneja detección de portadora y detección de colisiones. Al detectarse una colisión, el transceptor también pone una señal no válida especial en el cable para asegurar que todos los demás transceptores también se den cuenta de que ha ocurrido una colisión.

Con 10Base5, un **cable de transceptor** (*transceiver cable*) conecta el transceptor a una tarjeta de interfaz en la computadora. El cable de transceptor puede tener hasta 50 metros de longitud y contiene cinco pares trenzados aislados individualmente. Dos de los pares son para entrada y salida de datos, respectivamente; dos más son para entrada y salida de señales de control. El quinto par, que no siempre se usa, permite que la computadora energice la electrónica del transceptor. Algunos transceptores permiten la conexión de hasta ocho computadoras cercanas a ellos, a fin de reducir la cantidad de transceptores necesarios.

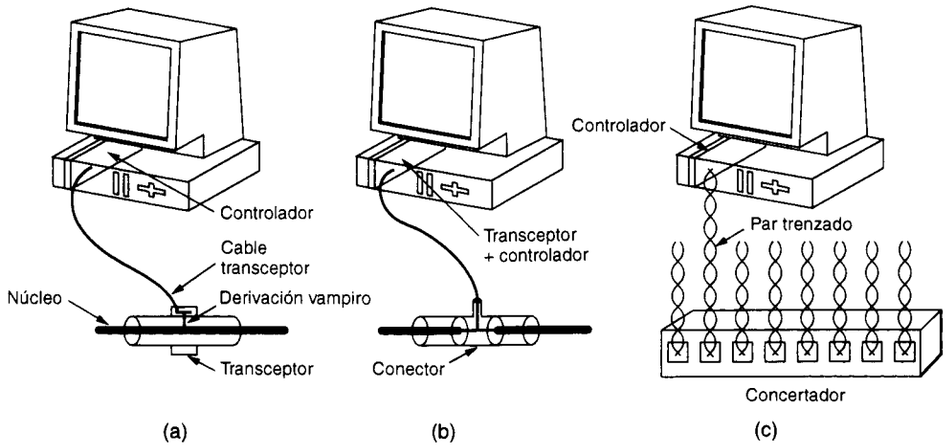


Figura 4-18. Tres tipos de cableado 802.3. (a) 10Base5. (b) 10Base2. (c) 10Base-T.

El cable del transceptor termina en una tarjeta de interfaz en la computadora. La tarjeta de interfaz contiene un *chip* controlador que transmite marcos al transceptor y recibe marcos de él. El controlador se encarga de ensamblar los datos en el formato de marco adecuado, así como de calcular las sumas de comprobación de los marcos de salida y de verificarlas en los marcos de entrada. Algunos *chips* controladores también administran una reserva de *buffers* para los marcos de entrada, una cola para la transmisión de los *buffers*, transferencias DMA con las computadoras anfitrionas y otros aspectos de gestión de la red.

Con 10Base2, la conexión al cable es sólo un conector BNC pasivo de unión T. La electrónica del transceptor está en la tarjeta controladora, y cada estación siempre tiene su propio transceptor.

Con 10Base-T no hay cable en absoluto, sólo el concertador (*hub*) (una caja llena de circuitos electrónicos). Agregar o remover estaciones es más sencillo con esta configuración, y las rupturas de cable pueden detectarse con facilidad. La desventaja de 10Base-T es que la longitud máxima del cable es de sólo 100 metros, tal vez 150 metros si se usa par trenzado de alta calidad (categoría 5). También, un concertador (*hub*) grande cuesta miles de dólares. Aun así, 10Base-T se está volviendo cada vez más común debido a la facilidad de mantenimiento que ofrece. Se analizará una versión más rápida de 10Base-T (100Base-T) posteriormente en este capítulo.

Una cuarta opción de cableado para 802.3 es **10Base-F**, que usa fibra óptica. Esta alternativa es cara debido al costo de los conectores y los terminadores, pero tiene excelente inmunidad contra el ruido y es el método a usar para conexiones entre edificios o entre concentradores muy separados.

En la figura 4-19 se muestran diferentes maneras de alambrar un edificio. En la figura 4-19(a), un solo cable se pasa entre cuarto y cuarto, conectándose cada estación a él en el punto más cercano. En la figura 4-19(b) una columna vertical corre del sótano al techo, conectándose a

él cables horizontales en cada piso mediante amplificadores especiales (repetidores). En algunos edificios los cables horizontales son delgados, y el *backbone* (columna vertebral) es grueso. La topología más general es en árbol, como en la figura 4-19(c), porque una red con dos trayectorias entre algunos pares de estaciones sufrirá interferencia entre las dos señales.

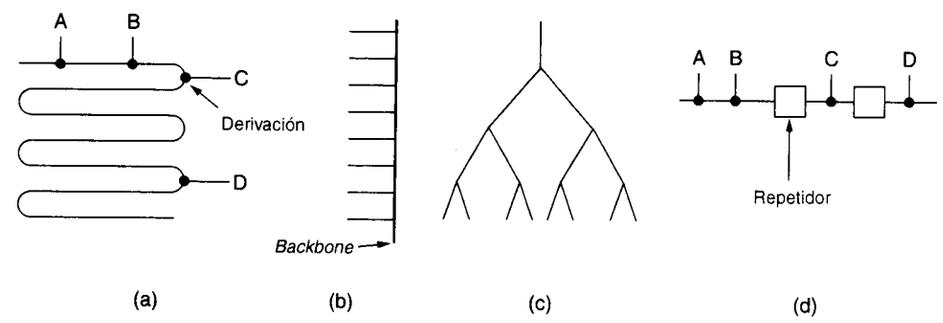


Figura 4-19. Topologías de cableado. (a) Lineal. (b) Columnar. (c) Árbol. (d) Segmentado.

Cada versión de 802.3 tiene una longitud máxima de cable por segmento. Para permitir redes mayores, se pueden conectar cables múltiples mediante **repetidores**, como se muestra en la figura 4-19(d). Un repetidor es un dispositivo de capa física que recibe, amplifica y retransmite señales en ambas direcciones. En lo que concierne al *software*, una serie de segmentos de cable conectados mediante repetidores no es diferente de un solo cable (excepto por el retardo introducido por los repetidores). Un sistema puede contener múltiples segmentos de cable y múltiples repetidores, pero ningún par de repetidores puede estar separado por más de 2.5 km y ninguna trayectoria entre dos transceptores puede atravesar más de cuatro repetidores.

Codificación Manchester

Ninguna de las versiones del 802.3 usa codificación binaria directa con 0 volts para un bit 0 y 5 volts para un bit 1, pues conduce a ambigüedades. Si una estación envía la cadena de bits 0001000, otros podrían interpretarla falsamente como 10000000 o 01000000, pues no pueden distinguir entre un transmisor inactivo (0 volts) y un bit 0 (0 volts).

Lo que se necesita es un mecanismo para que los receptores determinen sin ambigüedades el comienzo, el final o la mitad de cada bit sin referencia a un reloj externo. Dos de tales enfoques se llaman **codificación Manchester** y **codificación Manchester diferencial**. En la codificación Manchester, cada periodo de bit se divide en dos intervalos iguales. Un bit binario 1 se envía teniendo el voltaje alto durante el primer intervalo y bajo durante el segundo. Un 0 binario es justo lo inverso: primero bajo y después alto. Este esquema asegura que cada periodo de bit tiene una transición a la mitad, facilitando que el receptor se sincronice con el transmisor. Una desventaja de la codificación Manchester es que requiere el doble del ancho de banda que la

codificación binaria directa, pues los pulsos son de la mitad de ancho. La codificación Manchester se muestra en la figura 4-20(b).

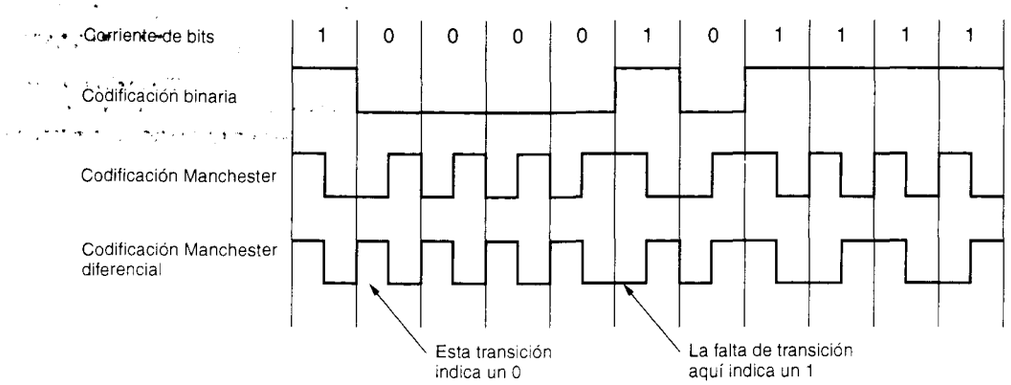


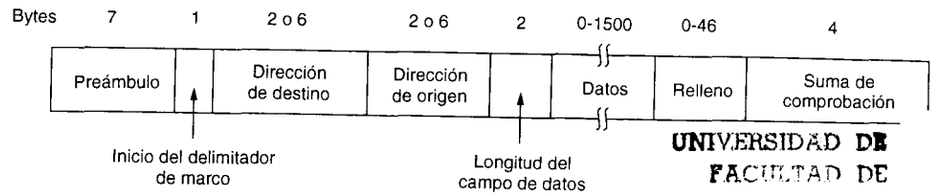
Figura 4-20. (a) Codificación binaria. (b) Codificación Manchester. (c) Codificación Manchester diferencial.

La codificación Manchester diferencial, que se muestra en la figura 4-20(c), es una variación de la codificación Manchester básica. En ella, un bit 1 se indica mediante la ausencia de una transición al comienzo del intervalo. Un bit 0 se indica mediante la presencia de un transición al inicio del intervalo. En ambos casos, también hay una transición a la mitad. El esquema diferencial requiere equipo más complejo, pero ofrece mejor inmunidad al ruido. Todos los sistemas 802.3 de banda base usan codificación Manchester debido a su sencillez. La señal alta es de +0.85 volts, y la señal baja es de −0.85 volts, dando un valor de cc de 0 volts.

Protocolo de subcapa MAC del 802.3

La estructura de marco del 802.3 (IEEE, 1985a) se muestra en la figura 4.21. Cada marco comienza con un *preámbulo* de 7 bytes, cada uno de los cuales contiene el patrón de bits 10101010. La codificación Manchester de este patrón produce una onda cuadrada de 10 MHz durante 5.6 μseg para permitir que el reloj del receptor se sincronice con el del transmisor. A continuación viene un byte de *inicio de marco* que contiene 10101011 para indicar el inicio del marco mismo.

El marco contiene dos direcciones, una para el destino y una para el origen. El estándar permite direcciones de 2 bytes y de 6 bytes, pero los parámetros definidos para el estándar de banda base de 10 Mbps usan sólo direcciones de 6 bytes. El bit de orden mayor de la dirección de destino es 0 para direcciones ordinarias y 1 para direcciones de grupo. Las direcciones de grupo permiten que varias estaciones escuchen en una sola dirección. Cuando el marco se envía a una dirección de grupo, todas las estaciones del grupo lo reciben. El envío a un grupo de estaciones se llama **multidifusión**. La dirección que consiste únicamente en bits 1 está reservada



UNIVERSIDAD DE
FACULTAD DE
DEPARTAMI
DOCUMENTACION
MONTEVIDEO

Figura 4-21. Formato de marco 802.3.

para **difusión**. Un marco que contiene sólo bits 1 en el campo de destino se entrega a todas las estaciones de la red.

Otra característica interesante del direccionamiento es el empleo del bit 46 (adyacente al bit de orden mayor) para distinguir las direcciones locales de las globales. Las direcciones locales son asignadas por cada administrador de la red y no tienen significado fuera de la red local. Las direcciones globales, en contraste, son asignadas por el IEEE para asegurar que no haya dos estaciones en ningún lugar del mundo que tengan la misma dirección global. Con $48 - 2 = 46$ bits disponibles, hay unas 7×10^{13} direcciones globales. La idea es que cualquier estación pueda direccionar de manera exclusiva cualquier otra estación con sólo dar el número correcto de 48 bits. Es tarea de la capa de red encontrar la manera de localizar el destino.

El campo de *longitud* indica cuántos bytes están presentes en el campo de datos, de un mínimo de 0 a un máximo de 1500. Aunque un campo de datos de 0 bytes es legal, causa un problema. Cuando un transceptor detecta una colisión, trunca el marco actual, lo que significa que aparecen bits perdidos y pedazos de marcos en el cable todo el tiempo. Para simplificar la diferenciación de los marcos válidos de la basura, el 802.3 establece que los marcos válidos deben tener cuando menos 64 bytes de longitud, desde la dirección de destino hasta la suma de comprobación. Si la parte de datos de un marco es de menos de 46 bytes, se usa el campo de relleno para rellenar el marco al tamaño mínimo.

Otra razón (más importante) para tener un marco de longitud mínima es evitar que una estación complete la transmisión de un marco corto antes de que el primer bit llegue al extremo más alejado del cable, donde podría tener una colisión con otro marco. Este problema se ilustra en la figura 4-22. En el momento 0, la estación *A*, en un extremo de la red, envía un marco. Llamemos τ al tiempo de propagación de este marco para llegar al otro extremo. Justo antes de que el marco llegue al otro extremo (es decir, en el momento $\tau - \varepsilon$) la estación más distante, *B*, comienza a transmitir. Cuando *B* detecta que está recibiendo más potencia de la que está enviando, sabe que ha ocurrido una colisión, por lo que aborta su transmisión y genera una ráfaga de ruido de 48 bits para avisar a las demás estaciones. En el momento 2τ, aproximadamente, el transmisor ve la ráfaga de ruido y aborta también su transmisión; luego espera un tiempo aleatorio antes de reintentar.

Si una estación intenta transmitir un marco muy corto, es concebible que ocurra una colisión, pero que la transmisión se complete antes de que la ráfaga de ruido llegue de regreso, en el momento 2τ. El transmisor entonces supondrá incorrectamente que el marco se envió con éxito. Para evitar que ocurra esta situación, todos los marcos deben tardar más de 2τ en enviarse. Para

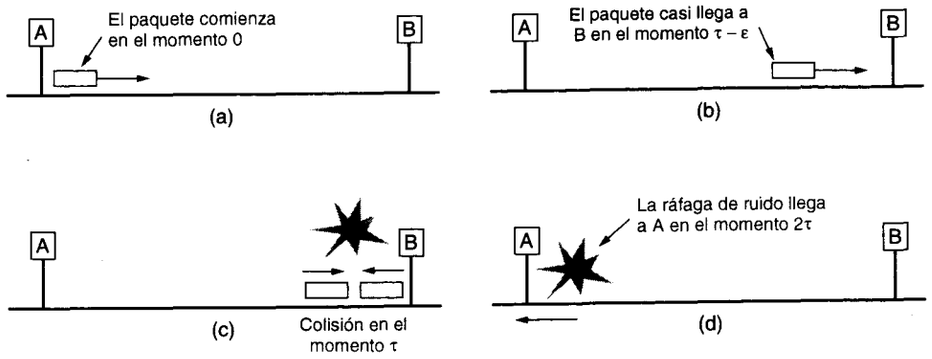


Figura 4-22. La detección de una colisión puede tardar hasta 2τ.

una LAN de 10 Mbps con una longitud máxima de 2500 metros y cuatro repetidores (a partir de la especificación 802.3), el marco mínimo permitido debe tardar 51.2 μseg. Este tiempo corresponde a 64 bytes. Los marcos con menos bytes se rellenan a 64 bytes.

A medida que aumenta la velocidad de la red, la longitud mínima del marco debe aumentar, o la longitud máxima del cable debe disminuir, de manera proporcional. Para una LAN de 2500 metros operando a 1 Gbps, el tamaño mínimo de marco tendría que ser de 6400 bytes. Como alternativa, el tamaño mínimo de marco podría ser de 640 bytes y la distancia máxima entre dos estaciones de 250 metros. Estas restricciones se vuelven cada vez más dolorosas a medida que progresamos hacia las redes de gigabits.

El campo final del 802.3 es la *suma de comprobación*. De hecho, ésta es un código de dispersión de 32 bits de los datos. Si algunos bits de datos se reciben erróneamente (debido a ruido en el cable), la suma de comprobación casi con certeza estará mal, y se detectará el error. El algoritmo de suma de comprobación es una verificación de redundancia cíclica del tipo analizado en el capítulo 3.

Algoritmo de retroceso exponencial binario

Ahora veamos cómo se efectúa el proceso de aleatorización cuando ocurre una colisión. El modelo es el de la figura 4-5. Tras una colisión, el tiempo se divide en ranuras discretas cuya longitud es igual al tiempo de propagación de ida y vuelta de peor caso en el éter (2τ). Contemplando la trayectoria más larga permitida por el 802.3 (2.5 km y cuatro repetidores), el tiempo de ranura se estableció en 512 tiempos de bit, o 51.2 μseg.

Tras la primera colisión, cada estación espera 0 o 1 tiempos de ranura antes de intentarlo de nuevo. Si dos estaciones entran en colisión y ambas escogen el mismo número aleatorio, habrá una nueva colisión. Tras la segunda colisión, cada una escoge 0, 1, 2 o 3 al azar y espera ese número de tiempos de ranura. Si ocurre una tercera colisión (la probabilidad de que esto ocurra es de 0.25), entonces la siguiente vez el número de ranuras a esperar se escogerá al azar del intervalo 0 a $2^3 - 1$.

En general, tras i colisiones, se escoge un número aleatorio entre 0 y $2^i - 1$, y se salta ese número de ranuras. Sin embargo, tras haberse alcanzado 10 colisiones, el intervalo de aleatorización se congela en un máximo de 1023 ranuras. Tras 16 colisiones, el controlador avienta la toalla e informa de un fracaso a la computadora. La recuperación posterior es responsabilidad de las capas superiores.

Este algoritmo, llamado **retroceso exponencial binario**, se escogió para adaptar dinámicamente el número de estaciones que intentan transmitir. Si el intervalo de aleatorización para todas las colisiones fuera de 1023, la posibilidad de que chocaran dos estaciones una segunda vez será insignificante, pero la espera promedio tras una colisión será de cientos de tiempos de ranura, introduciendo un retardo significativo. Por la otra parte, si cada estación siempre se retrasa 0 o 1 ranuras, entonces, al tratar de transmitir 100 estaciones al mismo tiempo, habría colisiones una y otra vez, hasta que 99 de ellas escogieran 0 y la estación restante escogiera 1, o viceversa. Esto podría tomar años. Haciendo que el intervalo de aleatorización crezca exponencialmente a medida que ocurren más y más colisiones, el algoritmo asegura un retardo pequeño cuando sólo unas cuantas estaciones entran en colisión, pero también asegura que la colisión se resuelva en un intervalo razonable cuando hay colisiones entre muchas estaciones.

Como se ha descrito hasta ahora, CSMA/CD no proporciona acuse de recibo. Ya que la simple ausencia de colisiones no garantiza que los bits no fueron alterados por picos de ruido en el cable, para una comunicación confiable el destino debe verificar la suma de comprobación y, de ser correcta, enviar un marco de acuse de recibo de regreso al origen. Normalmente, este acuse sería simplemente otro marco, en lo que concierne al protocolo, y tendría que pelear por tiempo de canal de la misma manera que un marco de datos. Sin embargo, una modificación sencilla del algoritmo de contención permite la confirmación rápida de la recepción de un marco (Tokoro y Tamaru, 1977). Todo lo que se necesitará es reservar para la estación de destino la primera ranura de contención siguiente a una transmisión exitosa.

802.3 Rendimiento

Examinemos ahora brevemente el desempeño del 802.3 en condiciones de carga pesada y constante, es decir, k estaciones siempre listas para transmitir. Es complicado un análisis riguroso del algoritmo de retroceso exponencial binario. En cambio, seguiremos a Metcalfe y Boggs (1976) y supondremos una probabilidad constante de retransmisión en cada ranura. Si cada estación transmite durante una ranura de contención con una probabilidad p, la probabilidad A de que una estación adquiera el canal durante esa ranura es de

$$A = kp(1 - p)^{k-1} \tag{4-6}$$

A es máxima cuando $p = 1/k$, con $A \rightarrow 1/e$ conforme $k \rightarrow \infty$. La probabilidad de que la ranura de contención tenga exactamente j ranuras de alojamiento en él es de $A(1 - A)^{j-1}$, por lo que el número medio de ranuras por contención está dado por

$$\sum_{j=0}^{\infty} jA(1 - A)^{j-1} = \frac{1}{A}$$

Ya que cada ranura tiene una duración de 2τ, la ranura medio de contención, w, es $2\tau/A$. Suponiendo una p óptima, el número medio de ranuras de contención nunca es mayor que e, por lo que w es, cuando mucho, $2\tau e \approx 5.4\tau$.

Si el marco medio tarda P segundos en transmitirse, cuando muchas estaciones tienen marcos por enviar,

$$\text{Eficiencia del canal} = \frac{P}{P + 2\tau/A} \tag{4-7}$$

Aquí vemos dónde entra la distancia máxima de cable entre dos estaciones en las cifras de rendimiento, dando lugar a topologías distintas de las de la figura 4-19(a). Cuanto mayor sea la longitud del cable, mayor será la ranura de contención. Al no permitir más de 2.5 km de cable y cuatro repetidores entre dos transceptores, el tiempo de ida y vuelta puede fijarse en 51.2 μseg, que a 10 Mbps corresponde a 512 bits o 64 bytes, el tamaño mínimo de marco.

Es instructivo formular la ecuación (4-7) en términos de la longitud de marco, F, el ancho de banda de la red, B, la longitud del cable, L, y la velocidad de propagación de la señal, c, para el caso óptimo de e ranuras de contención por marco. Con $P = F/B$, la ecuación (4-7) se convierte en

$$\text{Eficiencia del canal} = \frac{1}{1 + 2BLe/cF} \tag{4-8}$$

Cuando el segundo término del denominador es grande, la eficiencia de la red es baja. Más específicamente, un aumento en el ancho de banda o la distancia de la red (el producto BL) reduce la eficiencia de un marco de tamaño dado. Desafortunadamente, mucha investigación sobre *hardware* de redes está enfocada precisamente a aumentar este producto. La gente quiere un gran ancho de banda a través de distancias grandes (por ejemplo, en las MAN de fibra óptica), lo que sugiere que 802.3 tal vez no sea el mejor sistema para estas aplicaciones.

En la figura 4-23 se presenta gráficamente la eficiencia del canal contra el número de estaciones listas para $2\tau = 51.2$ μseg y una tasa de datos de 10 Mbps usando la ecuación (4-12). Con un tiempo de ranura de 64 bytes, no es sorprendente que los marcos de 64 bytes no sean eficientes. Por otra parte, con marcos de 1024 bytes y un valor asintótico de e ranuras de 64 bytes por intervalo de contención, el periodo de contención tiene 174 bytes de longitud y la eficiencia es de 0.85.

Para determinar el número medio de estaciones listas para transmitir en condiciones de carga alta, podemos usar la siguiente (y burda) observación. Cada marco atrapa el canal durante un periodo de contención más un tiempo de transmisión de marco, para un total de $P + w$ seg. El número de marcos por segundo, por tanto, es $1/(P + w)$. Si cada estación genera marcos a una velocidad media de λ marcos/seg, cuando el sistema está en el estado k, la tasa de entrada total de todas las estaciones no bloqueadas combinadas es de $k\lambda$ marcos/seg. Ya que en equilibrio las tasas de entrada y de salida deben ser idénticas, podemos igualar esas dos expresiones y despejar k. (Nótese que w es una función de k.) En (Bertsekas y Gallager, 1992) se da un análisis más elaborado.

Probablemente vale la pena mencionar que se ha realizado una gran cantidad de análisis teóricos de desempeño del 802.3 (y otras redes). Prácticamente todos estos trabajos han supuesto que el tráfico es Poisson. A medida que los investigadores han comenzado a examinar datos reales, se ha hecho evidente que el tráfico en redes pocas veces es Poisson, sino autosimilar (Paxson y Floyd, 1994, y Willinger *et al.*, 1995). Lo que esto significa es que la promediación durante periodos grandes de tiempo no hace más uniforme el tráfico. El número medio de paquetes en cada minuto de una hora tiene tanta variación como el número medio de paquetes en cada segundo de un minuto. La consecuencia de este descubrimiento es que la mayoría de los modelos de tráfico de red no se aplican al mundo real y deben tomarse con escepticismo.

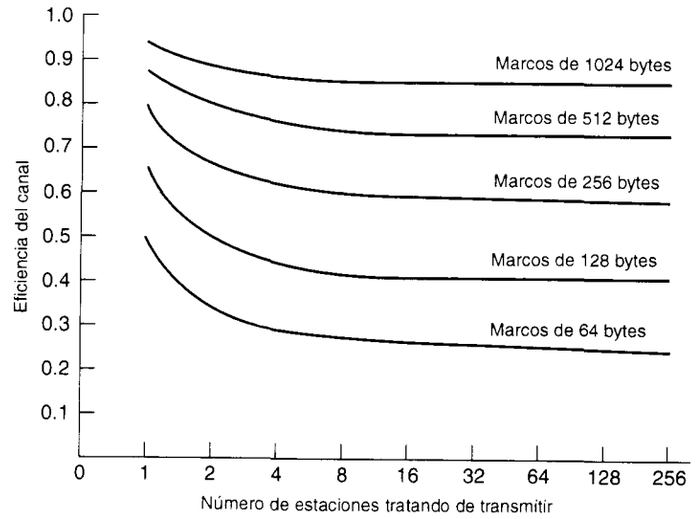


Figura 4-23. Eficiencia del 802.3 a 10 Mbps con tiempos de ranura de 512 bits.

LAN 802.3 conmutadas

A medida que se agregan más y más estaciones a una LAN 802.3, aumenta el tráfico. En algún momento, la LAN se saturará. Una solución al problema es utilizar una velocidad mayor, digamos 100 Mbps en lugar de 10 Mbps. Esta solución obliga a desechar todas las tarjetas adaptadoras de 10 Mbps y comprar nuevas, lo cual implica un gasto considerable. Si los *chips* de 802.3 están en las tarjetas principales de la computadora, tal vez ni siquiera sea posible cambiarlos.

Afortunadamente existe una solución diferente, menos drástica: una LAN 802.3 conmutada, como la que se muestra en la figura 4-24. El corazón de este sistema es un conmutador que contiene un canal en segundo plano de alta velocidad y espacio (típicamente) para 4 a 32 tarjetas de línea, conteniendo cada una de uno a ocho conectores. Lo más común es que cada conector tenga una conexión de par trenzado 10Base-T a una sola computadora anfitriona.

Cuando una estación quiere transmitir un marco 802.3, envía un marco estándar al conmutador. La tarjeta que recibe el marco lo revisa para ver si está destinado a una de las otras estaciones conectadas a la misma tarjeta. De ser así, el marco se copia ahí; si no, el marco se envía a

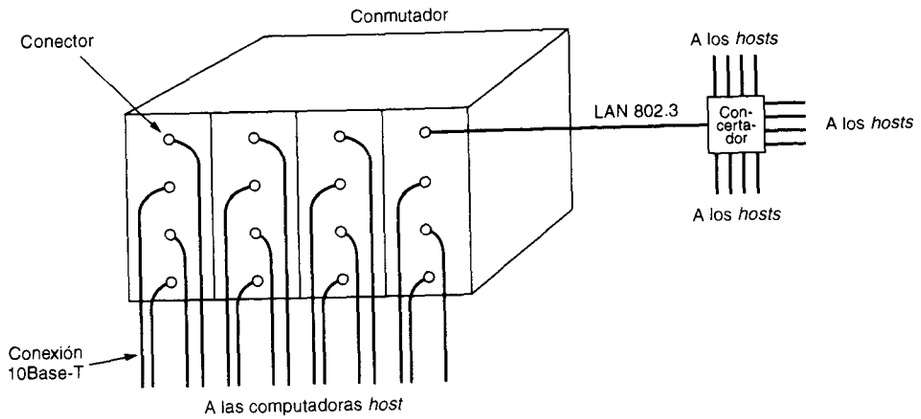


Figura 4-24. LAN 802.3 conmutada.

través del canal en segundo plano de alta velocidad a la tarjeta de la estación de destino. El canal de alta velocidad funciona a más de 1 Gbps usando un protocolo patentado.

¿Qué ocurre si dos máquinas conectadas a la misma tarjeta transmiten marcos al mismo tiempo? Depende de la manera en que haya sido construida la tarjeta. Una posibilidad es que todos los puertos de la tarjeta estén alambrados entre sí para formar una LAN local en la tarjeta. Las colisiones en esta tarjeta de LAN se detectan y manejan igual que cualquier otra colisión en una red CSMA/DC, usando las retransmisiones el algoritmo de retroceso exponencial binario. Con este tipo de tarjeta sólo es posible una transmisión por tarjeta en un momento dado, pero todas las tarjetas pueden estar transmitiendo en paralelo. Con este diseño, cada tarjeta forma su propio **dominio de colisión**, independiente de los demás.

Con el otro tipo de tarjeta, cada puerto de entrada se maneja con un *buffer*, por lo que los marcos de entrada se almacenan en la RAM de la tarjeta según llegan. Este diseño permite que todos los puertos de entrada reciban (y transmitan) marcos al mismo tiempo, para operación dúplex integral paralela. Una vez que se ha recibido por completo el marco, la tarjeta puede determinar si el marco está destinado a otro puerto de la misma tarjeta o a un puerto distante. En el primer caso, puede transmitirse directamente al destino. En el segundo, debe transmitirse a través del canal de alta velocidad a la tarjeta apropiada. Con este diseño, cada puerto es un dominio de colisión independiente, por lo que no ocurren colisiones. En muchos casos, el rendimiento total del sistema puede aumentarse en un orden de magnitud respecto al 10Base-5, que tiene un solo dominio de colisión para todo el sistema.

Dado que el conmutador sólo espera marcos 802.3 en cada puerto de entrada, es posible usar algunos de los puertos como concentradores. En la figura 4-24, el puerto de la esquina superior derecha no está conectado a una sola estación, sino a un concertador de 12 puertos. A medida que llegan los marcos al concertador, contienden por la LAN 802.3 de la manera usual, con colisiones y retroceso binario. Los marcos que tienen éxito llegan al conmutador y ahí se tratan como cualquier otro marco de entrada: se conmutan a la línea de salida correcta a través del canal de alta velocidad. Si todos los puertos de entrada están conectados a concertadores, en lugar de a estaciones individuales, el conmutador simplemente se vuelve un puente de 802.3 a 802.3. Estudiaremos los puentes posteriormente en este capítulo.

4.3.2. Estándar IEEE 802.4: *token bus*

Aunque 802.3 se usa ampliamente en entornos de oficinas, durante el desarrollo del estándar 802 la gente de General Motors y otras compañías interesadas en la automatización de la manufactura tuvieron dudas serias sobre él. Por una parte, debido al protocolo de MAC probabilístico, con un poco de mala suerte una estación podría tener que esperar un tiempo arbitrariamente largo para enviar un marco (es decir, el peor caso no está determinado). Por otra parte, los marcos de 802.3 no tienen prioridades, lo que los hace inadecuados para los sistemas en tiempo real en los que los marcos importantes no deben ser detenidos por esperar marcos sin importancia.

Un sistema sencillo con un peor caso conocido es un anillo en el que las estaciones toman turnos para enviar marcos. Si hay n estaciones y se quiere un tiempo T seg para enviar un marco, ningún marco tendrá que esperar más de nT seg para ser enviado. A la gente de automatización de manufactura del comité 802 le gustó la idea conceptual de un anillo, pero no les gustó la implementación física porque una ruptura en el cable del anillo inhabilitaría la red completa. Además, notaron que un anillo es poco adecuado para la topología lineal de la mayoría de las líneas de ensamble. Como resultado, se desarrolló un estándar nuevo, con la robustez del cable de difusión 802.3 pero el comportamiento conocido de peor caso de un anillo.

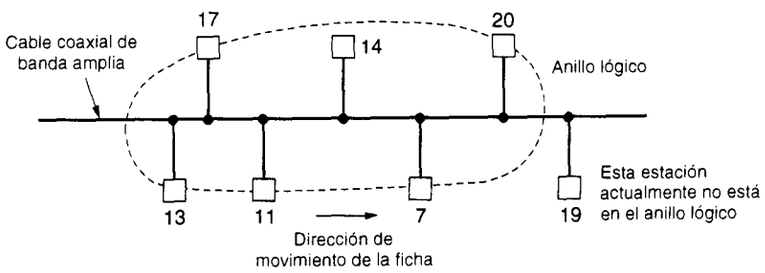


Figura 4-25. *Token bus.*

Este estándar, el 802.4 (Dirvin y Miller, 1986, e IEEE, 1985b), describe una LAN llamada ***token bus*** (*bus* de ficha). Físicamente, el *token bus* es un cable lineal o en forma de árbol al que se conectan las estaciones. Las estaciones están organizadas lógicamente en forma de anillo (ver figura 4-25), donde cada estación conoce la dirección de la estación a su "izquierda" y a su "derecha". Cuando se inicializa el anillo lógico, la estación de número más alto puede enviar el primer marco. Hecho esto, pasa el permiso a su vecino inmediato enviándole un marco de

control especial llamado **ficha** (*token*). La ficha se propaga alrededor del anillo lógico, teniendo permiso de transmitir marcos sólo quien tiene la ficha. Dado que sólo una estación a la vez tiene la ficha, no ocurren colisiones.

Un punto importante a notar es que no es importante el orden físico en el que están conectadas las estaciones al cable. Dado que el cable inherentemente es un medio de difusión, todas las estaciones reciben todos los marcos, descartando los que no estén dirigidos a ellas. Cuando una estación pasa la ficha, envía un marco de ficha dirigido específicamente a su vecino lógico en el anillo, sin importar la ubicación física de esa estación en el cable. También vale la pena notar que, cuando inicialmente se prenden las estaciones, no están en el anillo (por ejemplo, las estaciones 14 y 19 de la figura 4-25), por lo que el protocolo de MAC contempla la adición y eliminación de estaciones del anillo.

El protocolo MAC 802.4 es muy complicado, pues cada estación tiene que mantener 10 temporizadores distintos y más de dos docenas de variables de estado internas. El estándar 802.4 es mucho más largo que el 802.3; ocupa más de 200 páginas. Los dos estándares también tienen estilos bastante diferentes, pues el 802.3 presenta los protocolos como procedimientos en Pascal, y el 802.4, como máquinas de estado finito, con las acciones escritas en Ada®.

Para la capa física, el *token bus* usa el cable coaxial de 75 ohms de banda ancha empleado para la televisión por cable. Se permiten tanto sistemas de cable sencillo como doble, con o sin terminadores. Se permiten tres esquemas analógicos de modulación: codificación por desplazamiento de frecuencia de fase continua, codificación por desplazamiento de frecuencia de fase coherente y codificación por desplazamiento de fase de amplitud modulada multinivel duobinaria. Son posibles velocidades de 1, 5 y 10 Mbps. Es más, los esquemas de modulación no sólo proporcionan modos de representar 0, 1 e inactivo en el cable, sino también otros tres símbolos que sirven para el control de la red. En conjunto, la capa física es totalmente incompatible con el 802.3, y mucho más complicada.

Protocolo de subcapa MAC de *token bus*

Cuando se inicializa el anillo, las estaciones se insertan en orden por dirección de estación, de mayor a menor. La entrega de la ficha también se hace por dirección, de mayor a menor. Cada vez que una estación adquiere la ficha, puede transmitir marcos durante cierta cantidad de tiempo; después debe pasar la ficha a otra estación. Si los marcos son lo bastante cortos, pueden enviarse varios marcos consecutivos. Si una estación no tiene datos, pasa la ficha inmediatamente después de recibirla.

El *token bus* define cuatro clases de prioridad, 0, 2, 4 y 6 para tráfico, siendo la 0 la menor y la 6 la mayor. Lo más sencillo es visualizar cada estación como dividida internamente en cuatro subestaciones, cada una con un nivel de prioridad. A medida que llegan las entradas desde arriba a la subcapa MAC, los datos se examinan para ver su prioridad y se enrutan a una de las cuatro subestaciones. Por tanto, cada subestación mantiene su propia cola de marcos a transmitir.

Cuando la ficha llega a la estación a través del cable, se pasa internamente a la subestación de prioridad 6, que puede empezar a transmitir marcos, si tiene. Hecho esto (o al expirar el temporizador), la ficha se pasa internamente a la subestación de prioridad 4, que entonces puede

transmitir marcos hasta que expire el temporizador, en cuyo momento la ficha se pasa internamente a la subestación de prioridad 2. Este proceso se repite hasta que la subestación 0 ha enviado todos sus marcos o hasta que expira el temporizador. En ambos casos, en este punto la ficha se pasa a la siguiente estación del anillo.

Sin entrar en todos los detalles del manejo de los diferentes temporizadores, debe quedar claro que, ajustando éstos correctamente, podemos asegurar que una fracción garantizada del tiempo total de retención de la ficha pueda destinarse a tráfico de prioridad 6. Las prioridades menores tendrán que vivir con lo que sobre. Si las subestaciones de mayor prioridad no necesitan todo su tiempo asignado, las subestaciones de prioridad menor pueden hacer uso de la parte no utilizada, a fin de que no se desperdicie.

Este esquema de prioridades, que garantiza al tráfico de prioridad 6 una fracción conocida del ancho de banda de la red, puede servir para implementar tráfico en tiempo real. Por ejemplo, supóngase que los parámetros de una red de 50 estaciones operando a 10 Mbps se han ajustado para dar 1/3 del ancho de banda al tráfico de prioridad 6. Entonces cada estación tiene garantizados 67 kbps para el tráfico de prioridad 6. Este ancho de banda puede usarse para sincronizar los robots de una línea de ensamble o llevar un canal digital de voz por estación, sobrando un poco para información de control.

En la figura 4-26 se muestra el formato de marco de *token bus*. Desafortunadamente es distinto del formato de marco 802.3. El preámbulo sirve para sincronizar el reloj del receptor, como en el 802.3, excepto que aquí puede ser muy corto, hasta de 1 byte. Los campos de *delimitador inicial* y *delimitador final* se usan para marcar los límites de los marcos. Estos dos campos contienen codificación analógica de símbolos distintos de ceros y unos, por lo que no pueden ocurrir accidentalmente en los datos de usuario. Como resultado, no se necesita un campo de longitud.

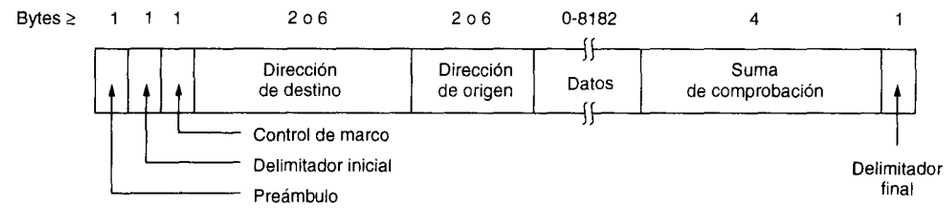


Figura 4-26. Formato de marco 802.4.

El campo de *control de marco* se usa para distinguir los marcos de datos de los de control. En los marcos de datos este campo lleva la prioridad del marco; también puede llevar un indicador que pida a la estación destino reconocer la recepción correcta o incorrecta del marco. Sin este indicador, no se permitiría al destino enviar nada, porque no tiene la señal. Este indicador convierte el *token bus* en algo parecido al esquema de reconocimiento de Tokoro y Tamaru.

En los marcos de control el campo de *control de marco* se usa para especificar el tipo de marco. Los tipos permitidos incluyen pase de ficha y varios marcos de mantenimiento del anillo,

incluidos los mecanismos para permitir la entrada de estaciones nuevas en el anillo y la salida de estaciones del anillo, y otros. Nótese que el protocolo 802.3 no tiene marcos de control. Todo lo que hace ahí la capa MAC es proporcionar una manera de poner marcos en el cable, sin importar su contenido.

Los campos de *dirección de destino* y *dirección de origen* son los mismos que en el 802.3 (sí, los dos grupos se comunicaban; no, no lograron ponerse de acuerdo en gran cosa). Como en el 802.3, una red dada debe usar sólo direcciones de 2 bytes o de 6 bytes, no una mezcla, en el mismo cable. El estándar inicial 802.4 permite ambos tamaños. El direccionamiento individual y de grupo, así como las asignaciones de direcciones locales y globales son idénticos a los de 802.3.

El campo de *datos* puede tener hasta 8182 bytes de longitud cuando se usan direcciones de 2 bytes, y de hasta 8174 bytes cuando se usan direcciones de 6 bytes. Esto es más de cinco veces la longitud máxima del marco 802.3, en el que se limitó la longitud para evitar que una estación se apropie del canal durante demasiado tiempo. En el *token bus* los temporizadores pueden usarse como medida antiacaparamiento, pero es bueno poder enviar marcos grandes cuando el tráfico en tiempo real no es un punto aplicable. El campo de *suma de comprobación* sirve para detectar errores de transmisión. Se usa el mismo algoritmo y el mismo polinomio que en el 802.3.

Los marcos de control de *token bus* se muestran en la figura 4-27 y se analizarán a continuación. El único que hemos visto hasta ahora es el marco de *ficha*, que sirve para pasar la ficha de estación en estación. Casi todos los demás se relacionan con agregar estaciones al anillo lógico y eliminarlas de él.

Campo de control de marco	Nombre	Significado
00000000	Claim_token	Reclamar la ficha durante la inicialización del anillo
00000001	Solicit_successor_1	Permitir que entren estaciones en el anillo
00000010	Solicit_successor_2	Permitir que entren estaciones en el anillo
00000011	Who_follows	Recuperación de una ficha perdida
00000100	Resolve_contention	Se usa cuando quieren entrar varias estaciones
00001000	Token	Pasar la ficha
00001100	Set_successor	Permitir que la estación se desconecte del anillo

Figura 4-27. Marcos de control del *token bus*.

Mantenimiento del anillo lógico

De tiempo en tiempo, las estaciones se encienden y quieren unirse al anillo. Otras se apagan y quieren salirse. El protocolo de la subcapa MAC proporciona una especificación detallada de la manera exacta en que se hace esto mientras se mantiene el límite de peor caso conocido sobre la rotación de la ficha. A continuación esbozaremos brevemente los mecanismos que se utilizan.

Una vez que se ha establecido el anillo, la interfaz de cada estación mantiene internamente las direcciones del antecesor y del sucesor. Periódicamente, el poseedor de la ficha envía uno de los marcos SOLICIT_SUCCESSOR mostrados en la figura 4-27 para solicitar peticiones de estaciones que desean unirse al anillo. El marco da la dirección del transmisor y de su sucesor. Las estaciones que estén dentro de ese intervalo pueden solicitar entrar (para mantener ordenado el anillo en orden descendente por dirección de estación).

Si ninguna estación solicita entrar durante un tiempo de ranura (2τ, como en el 802.3), la **ventana de respuesta** se cierra y el poseedor de la ficha continúa con sus asuntos normales. Si exactamente una estación solicita entrar, se le introduce al anillo, convirtiéndose en el sucesor del poseedor de la ficha.

Si dos o más estaciones solicitan entrar, habrá una colisión entre sus marcos y se alterarán, como en el 802.3. El poseedor de la ficha ejecuta entonces un algoritmo de arbitraje, comenzando por la difusión de un marco RESOLVE_CONTENTION. El algoritmo es una variación del conteo descendente binario, usando dos bits a la vez.

Además, todas las interfaces de estación mantienen internamente dos bits aleatorios. Estos bits se usan para retardar todas las solicitudes en 0, 1, 2 o 3 tiempos de ranura y así reducir aún más la contención. En otras palabras, dos estaciones sólo pueden entrar en colisión durante una solicitud si los dos bits de dirección en uso son iguales y tienen los mismos bits aleatorios. Para evitar que las estaciones que tienen que esperar 3 tiempos de ranura estén en desventaja permanente, los bits aleatorios se regeneran cada vez que se usan, o periódicamente cada 50 mseg.

La solicitud de estaciones nuevas no deberá interferir con el peor caso garantizado de rotación de la ficha. Cada estación tiene un temporizador que se restablece cada vez que adquiere la ficha. Cuando llega la ficha, el valor anterior de este temporizador (es decir, el tiempo de rotación de la ficha anterior) se inspecciona justo antes del restablecimiento del temporizador. Si excede cierto valor límite, es porque ha habido demasiado tráfico últimamente, por lo que no podrán efectuarse solicitudes en esta vuelta. En cualquier caso, sólo puede entrar una estación en cada solicitud, a fin de limitar el tiempo que puede consumirse en el mantenimiento del anillo. No se ofrece ninguna garantía respecto al tiempo que una estación tendrá que esperar para entrar en el anillo cuando el tráfico es pesado, pero en la práctica no debe ser de más de unos cuantos segundos. Esta incertidumbre es desafortunada, y hace que el 802.4 sea menos adecuado para sistemas de tiempo real de lo que aseguran sus seguidores.

La salida del anillo es fácil. Una estación, X, con su sucesor S y su antecesor P, deja el anillo, enviando a P un marco SET_SUCCESSOR que indica que a partir de ahora el sucesor es S, en lugar de X. Después, X simplemente deja de transmitir.

La inicialización del anillo es un caso especial de adición de estaciones nuevas. Considere un sistema inactivo con todas sus estaciones apagadas. Cuando entra en línea la primera estación, nota que no hay tráfico durante cierto periodo; entonces, envía un marco CLAIM_TOKEN. Al no escuchar competidores por la ficha, la estación crea una ficha y establece un anillo que sólo la contiene a ella. Periódicamente, la estación emite solicitudes de anexión de estaciones nuevas. A medida que se enciendan estaciones nuevas, responderán a estas solicitudes y se unirán al anillo usando el algoritmo de contención antes descrito. Tarde o temprano, cada estación que quiera

unirse al anillo podrá hacerlo. Si las primeras dos estaciones se encienden simultáneamente, el protocolo maneja esto permitiendo que soliciten la ficha usando el algoritmo estándar de conteo descendente binario modificado y los dos bits aleatorios.

Debido a los errores de transmisión y a las fallas de *hardware*, pueden surgir problemas con el anillo lógico o la señal. Por ejemplo, si una estación trata de pasar la ficha a una estación que se ha desactivado, ¿qué ocurre? La solución es directa. Tras pasar la ficha, una estación escucha si su sucesor transmite un marco o pasa la ficha; si no lo hace, se le pasa la ficha una segunda vez.

Si también eso falla, la estación transmite un marco WHO_FOLLOWS especificando la dirección de su sucesor. Cuando el sucesor de la estación fallida ve un marco nombrando a su antecesor, responde con un marco SET_SUCCESSOR a la estación a la que le falló el sucesor, nombrándose como nuevo sucesor. De esta manera, la estación fallida se retira de la red.

Ahora supongamos que una estación no pasa la ficha a su sucesor y tampoco logra localizar al sucesor del sucesor, que también podría haberse caído. La estación adopta una estrategia nueva enviando un marco SOLICIT_SUCCESSOR_2 para ver si queda *alguien* vivo. Nuevamente se ejecuta el protocolo estándar de contención, solicitando ahora un lugar todas las estaciones que quieren estar en el anillo. Finalmente, se restablece el anillo.

Ocurre otro tipo de problema si el poseedor de la ficha se desactiva y se lleva con él la ficha. Este problema se resuelve usando el algoritmo de inicialización del anillo. Cada estación tiene un temporizador que se restablece cada vez que aparece un marco en la red. Cuando este temporizador alcanza un valor límite, la estación envía un marco CLAIM_TOKEN, y el algoritmo de conteo binario modificado con bits aleatorios determina quién obtiene la ficha.

Un problema más es el de las fichas múltiples. Si una estación poseedora de la ficha nota una transmisión desde otra estación, se deshace de su ficha. Si había dos, ahora habrá una. Si había más de dos, este proceso se repetirá tarde o temprano hasta que todas las fichas, menos una, se desechen. Si por accidente se desecharan todas las fichas, la falta de actividad causaría que una o más estaciones trataran de reclamar la ficha.

4.3.3. Estándar IEEE 802.5: *token ring*

Las redes en anillo no son nada nuevo, pues (Pierce, 1972) se han utilizado desde hace mucho para redes tanto locales como de área amplia. Entre sus muchas características atractivas está el que un anillo no es realmente un medio de difusión, sino un conjunto de enlaces punto a punto individuales que, coincidentemente, forman un círculo. Los enlaces punto a punto implican una tecnología bien entendida y probada en el campo que puede operar en par trenzado, cable coaxial y fibra óptica. La ingeniería de anillos es casi completamente digital. En contraste, por ejemplo, el 802.3 tiene una componente analógica considerable para la detección de colisiones. Un anillo también es equitativo y tiene un límite superior conocido de acceso a canal. Por estas razones, IBM escogió el anillo como su LAN y el IEEE ha incluido el estándar ***token ring*** como el 802.5 (IEEE, 1985c; Latif *et al.*, 1992).

Un asunto fundamental en el diseño y análisis de cualquier red en anillo es la "longitud física" de un bit. Si la tasa de datos del anillo es de R Mbps, se emite un bit cada $1/R$ µseg. Con

una velocidad de propagación de señal típica de unos 200 m/µseg, cada bit ocupa $200/R$ metros del anillo. Esto significa, por ejemplo, que un anillo de 1 Mbps cuya circunferencia es de 1000 metros puede contener sólo 5 bits a la vez. Las implicaciones del número de bits en el anillo se aclararán después.

Como se mencionó antes, un anillo en realidad consiste en un conjunto de interfaces de anillo conectadas por líneas punto a punto. Cada bit que llega a una interfaz se copia en un *buffer* de 1 bit y luego se copia en el anillo nuevamente. Mientras está en el *buffer*, el bit se puede inspeccionar y posiblemente modificar antes de enviarse. Este paso de copiado introduce un retardo de 1 bit en cada interfaz. En la figura 4-28 se muestra un anillo y sus interfaces.

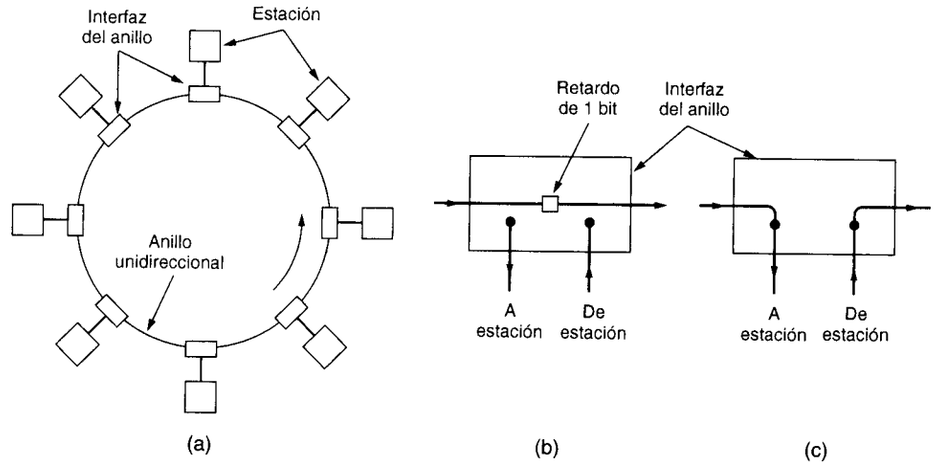


Figura 4-28. (a) Red en anillo. (b) Modo de escuchar. (c) Modo de transmitir.

En un *token ring* (anillo con ficha) circula un patrón de bit especial, llamado **ficha** (*token*) alrededor del anillo cuando todas las estaciones están inactivas. Cuando una estación quiere transmitir un marco, debe tomar la ficha y retirarla del anillo antes de transmitir. Esta acción se lleva a cabo invirtiendo un solo bit de la ficha de 3 bytes, lo que instantáneamente la convierte en los tres primeros bytes de un marco de datos normal. Debido a que sólo hay una ficha, sólo una estación puede transmitir en un instante dado, resolviendo por tanto el problema de acceso al canal de la misma manera en que lo resuelve el *token bus*.

Una implicación del diseño *token ring* es que el anillo mismo debe tener un retardo (*delay*) suficiente para contener una ficha completa que circule cuando todas las estaciones están inactivas. El retardo tiene dos componentes: el retardo de 1 bit introducido por cada estación y el retardo de propagación de señal. En la mayoría de los anillos, los diseñadores deben suponer que pueden apagarse estaciones en diferentes momentos, especialmente durante la noche. Si las interfaces se alimentan de energía o energizan desde el anillo, el apagado de la estación no

tiene ningún efecto sobre la interfaz, pero si las interfaces se energizan externamente deben estar diseñadas para conectar la entrada a la salida al interrumpirse la energía, con lo que se elimina el retardo de 1 bit. La cuestión aquí es que, en un anillo corto, podría ser necesario introducir un retardo artificial en el anillo por las noches para asegurar que pueda contener una ficha.

Las interfaces del anillo tienen dos modos operativos, escuchar y transmitir. En el modo de escuchar, los bits de entrada simplemente se copian en la salida, con un retardo de un tiempo de bit, como se muestra en la figura 4-28(b). En el modo de transmitir, al que se entra una vez que se tiene la ficha, la interfaz rompe la conexión entre la entrada y la salida, introduciendo sus propios datos en el anillo. Para poder conmutar del modo de escuchar al de transmitir en un tiempo de 1 bit, la interfaz generalmente necesita almacenar en su *buffer* uno o más marcos, en lugar de obtenerlos de la estación con tan poca anticipación.

A medida que los bits propagados alrededor del anillo regresan, son retirados por el transmisor. La estación transmisora puede guardarlos, para compararlos con los datos originales y verificar la confiabilidad del anillo, o descartarlos. Como el marco completo nunca aparece en el anillo en un mismo instante, esta arquitectura de anillo no pone límite al tamaño de los marcos. Una vez que una estación ha terminado de transmitir el último bit de su último marco, debe regenerar la ficha. Cuando ha regresado el último bit del marco, debe retirarse, y de inmediato la interfaz debe conmutarse nuevamente al modo de escuchar para evitar retirar la ficha que podría seguir si ninguna otra estación la ha retirado.

El manejo de acuses de recibo es directo en un *token ring*. El formato de marco sólo necesita incluir un campo de 1 bit para acuse de recibo, inicialmente cero. Cuando la estación de destino ha recibido un marco, establece el bit. Por supuesto, si el acuse significa que la suma de comprobación ha sido verificada, el bit debe seguir a la suma de comprobación y la interfaz con el anillo debe ser capaz de verificar la suma de comprobación tan pronto haya recibido el último bit. Cuando un marco se difunde a múltiples estaciones, debe usarse un mecanismo de acuse más complicado (si es que se usa uno).

Cuando el tráfico es ligero, la ficha pasará la mayor parte del tiempo circulando inactivamente alrededor del anillo. Ocasionalmente, una estación la tomará, transmitirá un marco, y entonces emitirá una ficha nueva. Sin embargo, cuando el tráfico es pesado, de modo que hay una cola en cada estación, tan pronto una estación termina su transmisión y regenera la ficha la siguiente estación verá y retirará la ficha. De esta manera, el permiso para enviar gira continuamente, por turno circular. La eficiencia de la red puede acercarse al 100% en condiciones de carga pesada.

Ahora pasemos de los *token rings* en general al estándar 802.5 en particular. En la capa física, el 802.5 especifica par trenzado operando a 1 o 4 Mbps, aunque IBM posteriormente introdujo una versión de 16 Mbps. Las señales se codifican usando codificación Manchester diferencial [véase la figura 4-20(c)], siendo alto (HI) y bajo (LO) señales positivas y negativas de magnitudes absolutas de 3.0 a 4.5 volts. Normalmente, la codificación Manchester diferencial usa HI-LO o LO-HI para cada bit, pero el 802.5 también usa HI-HI y LO-LO en ciertos bytes de control (por ejemplo, para marcar el inicio y el final de un marco). Estas señales que no son de datos ocurren en pares consecutivos y no introducen una componente de cc (*DC component*) en el voltaje del anillo.

Un problema con las redes de anillo es que, si se rompe el cable en alguna parte, el anillo se inhabilita. Este problema puede resolverse de manera muy elegante mediante el uso de un **centro de alambrado**, como se muestra en la figura 4-29. Si bien la red sigue siendo lógicamente un anillo, físicamente cada estación está conectada al centro de alambrado que contiene (cuando menos) dos pares trenzados, uno de datos a la estación y otro de datos de la estación.

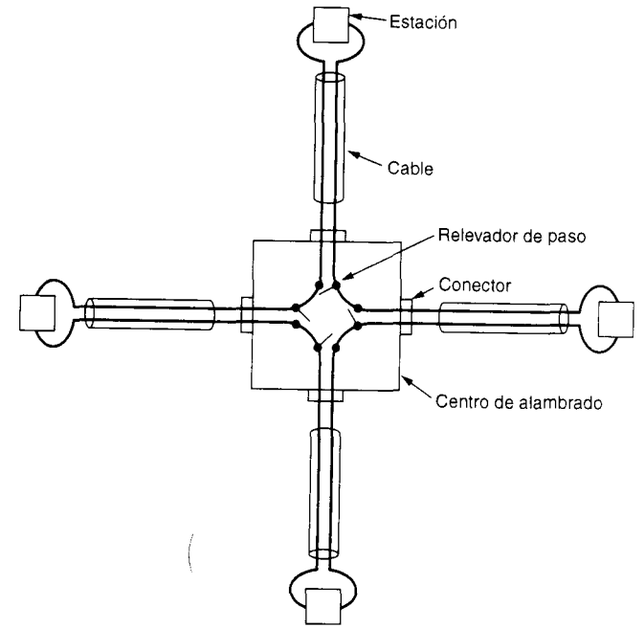


Figura 4-29. Cuatro estaciones conectadas mediante un centro de alambrado.

Dentro del centro de alambrado hay relevadores de paso (*bypass relays*) que se energizan mediante corriente de las estaciones. Si se rompe el anillo o se inactiva una estación, la pérdida de la corriente de operación desactivará el relevador, poniendo en puente la estación. Los relevadores también pueden operarse mediante *software* que permita a los programas de diagnóstico remover estaciones una a la vez para encontrar estaciones y segmentos de anillo con fallas. El anillo entonces puede continuar su operación teniendo en puente la sección dañada. Aunque el estándar 802.5 no requiere formalmente este tipo de anillo, llamado con frecuencia **anillo en estrella** (Saltzer *et al.*, 1983) de hecho la mayoría de las LAN 802.5 usan centros de alambrado para mejorar su confiabilidad y mantenimiento.

Cuando una red consiste en muchos grupos de estaciones muy separados, puede usarse una topología con muchos centros de alambrado. Sólo imagine que el cable a una de las estaciones de la figura 4-29 fuera reemplazado por un cable a un centro de alambrado distante. Aunque lógicamente todas las estaciones están en el mismo anillo, los requisitos de alambrado se

reducen significativamente. Un anillo 802.5 que utiliza un centro de alambrado tiene una topología semejante a una red 10Base-T basada en concertadores (*hubs*), pero los formatos y protocolos son diferentes.

Protocolo de subcapa MAC del *token ring*

La operación básica del protocolo de MAC es sencilla. Cuando no hay tráfico en el anillo, circula continuamente una ficha de 3 bytes, esperando que una estación la tome cambiando un bit 0 específico a bit 1, lo que convierte la ficha en la secuencia de inicio de marco. La estación entonces envía el resto de un marco de datos normal, como se muestra en la figura 4-30.

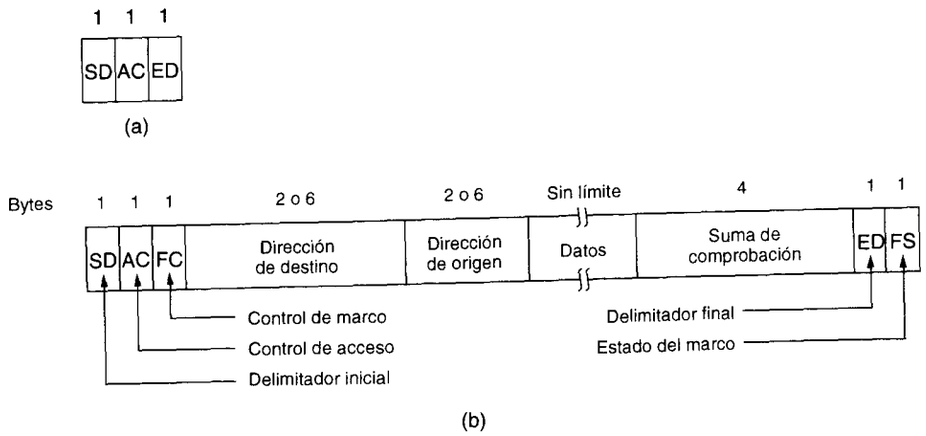


Figura 4-30. (a) Formato de ficha. (b) Formato de marco de datos.

En condiciones normales, el primer bit del marco recorrerá el anillo y regresará al transmisor antes de que el marco completo haya sido transmitido. Sólo un anillo muy grande será capaz de contener incluso un marco muy corto. En consecuencia, la estación transmisora debe drenar el anillo mientras continúa transmitiendo. Como se muestra en la figura 4-28(c), esto significa que los bits que han completado el viaje alrededor del anillo regresan al transmisor y ahí son removidos.

Una estación puede retener la ficha durante el **tiempo de retención de la ficha**, que es de 10 mseg a menos que una instalación establezca un valor distinto. Si queda suficiente tiempo tras la transmisión del primer marco para enviar más marcos, éstos pueden enviarse también. Una vez que han sido transmitidos todos los marcos pendientes, o si la transmisión de otro marco excedería el tiempo de retención de la ficha, la estación regenera el marco de ficha de 3 bits y lo pone en el anillo.

Los campos de *delimitador inicial* y *delimitador final* de la figura 4-30(b) marcan el inicio y el fin del marco. Cada uno contiene patrones Manchester diferenciales no válidos (HH y LL) para

distinguirlos de los bytes de datos. El byte de *control de acceso* contiene el bit de ficha y también el *bit de revisión*, los *bits de prioridad* y los *bits de reservación* (descritos más adelante). El byte de *control de marco* distingue los marcos de datos de los diferentes marcos de control posibles.

A continuación vienen los campos de *dirección destino* y *dirección origen*, que son iguales que en el 802.3 y el 802.4. A éstos siguen los datos, que pueden ser tan grandes como sea necesario, siempre y cuando el marco aún pueda transmitirse dentro del tiempo de retención de la ficha. El campo de *suma de comprobación*, como las direcciones de destino y origen, también es igual que en 802.3 y 802.4.

Un byte interesante que no está presente en los otros dos protocolos es el byte de *estado de marco*, que contiene los bits *A* y *C*. Cuando llega un marco a la interfaz de una estación con la dirección de destino, la interfaz enciende el bit *A* y lo pasa adelante. Si la interfaz copia el marco en la estación, también enciende el bit *C*. Una estación podría fallar al copiar un marco debido a falta de espacio de *buffer* u otras razones.

Cuando la estación transmisora drena el marco del anillo, examina los bits *A* y *C*. Son posibles tres combinaciones:

1. $A = 0$ y $C = 0$: destino no presente o no encendido.
2. $A = 1$ y $C = 0$: destino presente pero marco no aceptado.
3. $A = 1$ y $C = 1$: destino presente y marco copiado.

Este arreglo proporciona un acuse automático para cada marco. Si un marco es rechazado pero la estación aún está presente, el transmisor tiene la opción de intentar de nuevo posteriormente. Los bits *A* y *C* están presentes dos veces en el *estado de marco* para aumentar la confiabilidad, puesto que no están cubiertos por la suma de verificación.

El *delimitador final* contiene un bit *E* que se establece cuando cualquier interfaz detecta un error (por ejemplo, un patrón no Manchester donde no está permitido). También contiene un bit que puede servir para marcar el último marco de una secuencia lógica, algo parecido a un bit de fin de archivo.

El protocolo 802.5 tiene un elaborado esquema para manejar marcos con distintas prioridades. El marco de ficha de 3 bytes contiene un campo en el byte de enmedio que indica la prioridad de la ficha. Cuando una estación quiere transmitir un marco de prioridad *n*, debe esperar hasta que pueda capturar una ficha cuya prioridad sea menor o igual que *n*. Además, al pasar un marco de datos, una estación puede tratar de reservar la siguiente ficha escribiendo la prioridad del marco que quiere enviar en los *bits de reservación* del marco. Sin embargo, si se ha reservado ahí una prioridad mayor, la estación podría no hacer la reservación. Cuando el marco actual ha terminado, se genera la siguiente ficha con la prioridad reservada.

Un breve análisis demostrará que este mecanismo actúa como un trinquete, siempre aumentando la prioridad de la reservación. Para eliminar este problema, el protocolo incluye algunas reglas complicadas. La esencia del concepto es que la estación que eleva la prioridad tiene la obligación de disminuirla nuevamente cuando haya terminado.

Nótese que este esquema de prioridad es sustancialmente diferente del esquema del *token bus*, en el que cada estación siempre obtiene una parte equitativa del ancho de banda, sin importar

lo que hagan otras estaciones. En el *token ring*, una estación que sólo tiene marcos de prioridad baja puede morir de inanición esperando la aparición de una ficha de prioridad baja. Ciertamente ambos comités tenían gustos distintos en cuanto a sacrificar la justicia para todas las estaciones en aras de un buen servicio para el tráfico de prioridad alta.

Mantenimiento del anillo

El protocolo de *token bus* se esfuerza mucho por efectuar el mantenimiento del anillo de una manera completamente descentralizada. El protocolo de *token ring* maneja el mantenimiento de manera bastante diferente. Cada *token ring* tiene una **estación monitor** que supervisa el anillo. Si se inactiva el monitor, un protocolo de contención asegura que otra estación será electa rápidamente como monitor. (Cada estación tiene la capacidad de convertirse en monitor.) Cuando el monitor funciona adecuadamente, sólo él es responsable de ver que el anillo opere de manera correcta.

Cuando el anillo se activa o alguna estación nota que no hay monitor, puede transmitir un marco de control SOLICITAR FICHA. Si este marco recorre todo el anillo antes del envío de otros marcos SOLICITAR FICHA, el transmisor se convierte en el nuevo monitor (cada estación tiene capacidad de monitor integrada). Los marcos de control de *token ring* se muestran en la figura 4-31.

Campo de control	Nombre	Significado
00000000	Prueba dirección duplicada	Prueba si dos estaciones tienen la misma dirección
00000010	Indicación o faro	Usado para localizar rupturas en el anillo
00000011	Solicitar ficha	Intenta convertirse en monitor
00000100	Purga	Reinicializa el anillo
00000101	Monitor activo presente	Emitido periódicamente por el monitor
00000110	Monitor en espera presente	Anuncia la presencia de monitores potenciales

Figura 4-31. Marcos de control del *token ring*.

Entre las funciones del monitor están ver que no se pierda la ficha, emprender acciones cuando se rompe el anillo, limpiar el anillo después de la aparición de marcos alterados y buscar marcos huérfanos. Aparece un marco huérfano cuando una estación transmite un marco corto completo por un anillo grande y luego se inactiva o apaga antes de que el marco pueda ser drenado. Si no se hace nada, el marco circulará eternamente.

Para buscar fichas perdidas, el monitor tiene un temporizador que se establece al mayor intervalo posible sin ficha: cuando cada estación transmite durante el tiempo completo de retención de la ficha. Si se vence este temporizador, el monitor drena el anillo y envía una ficha nueva.

Al aparecer un marco alterado, el monitor puede detectarlo por su formato no válido o su suma de comprobación, abrir el anillo para drenarlo, y enviar una ficha nueva cuando el anillo se haya limpiado. Por último, el monitor detecta marcos huérfanos estableciendo el bit de *monitor* del byte de *control de acceso* cada vez que éste pasa por él. Si un marco que llega tiene este bit establecido, algo anda mal, pues significa que este mismo marco ha pasado dos veces por el monitor sin ser drenado, así que el monitor lo drena.

Una última función del monitor tiene que ver con la longitud del anillo. La ficha tiene 24 bits de longitud, lo que significa que el anillo debe tener el tamaño suficiente para contener 24 bits. Si los retardos de 1 bit en las estaciones más la longitud del cable suman menos de 24 bits, el monitor inserta bits de retardo adicionales para que pueda circular una ficha.

Una función de mantenimiento que no puede manejar el monitor es la localización de rupturas en el anillo. Cuando una estación nota que cualquiera de sus vecinas parece estar muerta, transmite un marco INDICACIÓN O FARO (*Beacon*) dando la dirección de la estación que tal vez está muerta. Cuando la señal se ha propagado tan lejos como puede, es posible ver cuántas estaciones están desactivadas y eliminarlas del anillo usando los relevadores de paso del centro de alambrado, todo sin intervención humana.

Es instructivo comparar los enfoques adoptados para controlar el *token bus* y el *token ring*. El comité del 802.4 le tenía pavor a los componentes centralizados que pudieran fallar de alguna manera inesperada y tirar el sistema al caerse ellos. Por tanto, diseñaron un sistema en el que el poseedor de la ficha actual tiene poderes especiales (por ejemplo, pedir solicitudes de ingreso al anillo), pero ninguna estación se distinguía de las demás en otros aspectos (por ejemplo, las responsabilidades administrativas de mantenimiento asignadas en el momento).

El comité del 802.5, por otro lado, pensó que tener un monitor centralizado haría mucho más sencillo el manejo de las fichas perdidas, los marcos huérfanos, etc. Además, en un sistema normal, las estaciones pocas veces se inactivan involuntariamente, por lo que tener que manejar ocasionalmente una contienda por un monitor nuevo no representa gran sufrimiento. El precio que se paga es que, si el monitor alguna vez realmente enloquece pero continúa enviando periódicamente marcos de control de MONITOR ACTIVO PRESENTE, ninguna estación se lo disputará. Los monitores no pueden ser impugnados.

Esta diferencia de enfoque surge de las distintas áreas de aplicación que tenían en mente los dos comités. El comité del 802.4 estaba pensando en términos de fábricas con grandes masas de metal en movimiento controladas por computadora. Las fallas de red podrían resultar en daños severos y tenían que evitarse a toda costa. El comité del 802.5 estaba interesado en la automatización de oficinas, donde podía tolerarse una falla de vez en cuando como el precio de tener un sistema más sencillo. Que el 802.4 sea, de hecho, más confiable que el 802.5 es tema de controversia.

4.3.4. Comparación del 802.3, el 802.4 y el 802.5

Al estar disponibles tres LAN diferentes e incompatibles, cada una con distintas propiedades, muchas organizaciones enfrentan la pregunta: ¿cuál debemos instalar? En esta sección veremos los tres estándares de LAN 802, señalando sus virtudes y sus defectos, comparándolos y poniéndolos en contraste.

Para comenzar, vale la pena hacer notar que los tres estándares de LAN usan una tecnología básicamente similar y obtienen un desempeño también similar. Si bien los científicos e ingenieros en computación pueden discutir sin parar los méritos del coaxial contra el par trenzado durante horas si se les permite, a la gente de los departamentos de mercadeo, personal o contabilidad probablemente no le importe mucho una cosa o la otra.

Comencemos por las ventajas del 802.3, el cual es, por mucho, el tipo más usado actualmente, con una base instalada enorme y con una considerable experiencia operativa. El protocolo es sencillo. Pueden instalarse estaciones al vuelo, sin tirar la red. Se usa un cable pasivo y no se requieren módems. Es más, el retardo con carga baja es prácticamente cero (las estaciones no tienen que esperar una ficha; simplemente transmiten de inmediato).

Por otra parte, el 802.3 tiene una componente analógica sustancial. Cada estación tiene que ser capaz de detectar la señal de la estación más débil, aun cuando esté transmitiendo ella misma, y todos los circuitos de detección de colisiones del transceptor son analógicos. Dada la posibilidad de que aborten marcos por colisión, el marco mínimo es de 64 bytes, lo que representa una carga extra sustancial cuando los datos consisten en un solo carácter enviado desde una terminal.

Además, el 802.3 es no determinístico, lo que con frecuencia es inapropiado para el trabajo en tiempo real [aunque es posible cierto trabajo de tiempo real simulando un *token ring* en *software* (Venkatramani y Chiueh, 1995)]. Esta LAN tampoco tiene prioridades. La longitud del cable está limitada a 2.5 km (a 10 Mbps) porque la longitud de ida y vuelta del cable determina el tiempo de ranura, y por tanto el desempeño. A medida que aumenta la velocidad, la eficiencia decae porque los tiempos de transmisión se reducen pero la ranura de contención no (el ancho de ranura es 2τ sin importar la tasa de datos). Alternativamente, el cable tiene que hacerse más corto. También, con carga alta, la presencia de colisiones se vuelve un problema mayor y puede afectar el rendimiento de manera seria.

Consideremos ahora el 802.4, el *token bus*. Esta LAN emplea equipo de televisión por cable altamente confiable, que se puede obtener sin necesidad de un pedido especial de numerosos proveedores. Es más determinístico que el 802.3, aunque las pérdidas repetidas de la ficha en momentos críticos pueden introducir más incertidumbre de la que están dispuestos a admitir sus partidarios. Esta LAN puede manejar marcos mínimos cortos.

El *token bus* también reconoce prioridades y puede configurarse para proporcionar una fracción garantizada del ancho de banda a tráfico con prioridad alta, como la voz digitalizada. Esta LAN también tiene rendimiento y eficiencia excelentes con carga alta, volviéndose efectivamente TDM. Por último, el cable de banda ancha puede manejar múltiples canales, no sólo de datos, sino también de voz y televisión.

Por el lado negativo, los sistemas de banda ancha emplean mucha ingeniería analógica e incluyen módems y amplificadores de banda amplia. El protocolo es extremadamente complicado y tiene un retardo sustancial cuando la carga es baja (las estaciones siempre deben esperar la ficha, aun en un sistema sin tráfico). Por otra parte, el *token bus* no es idóneo para las implementaciones de fibra óptica y tiene una base instalada de usuarios pequeña.

Consideremos ahora el *token ring*. Esta LAN maneja conexiones punto a punto, lo que significa que la ingeniería es más sencilla y puede ser completamente digital. Los anillos pueden construirse usando prácticamente cualquier medio de transmisión desde paloma mensajera hasta

fibra óptica. El par trenzado estándar es barato y sencillo de instalar. El uso de centros de alambrado hace al *token ring* la única LAN que puede detectar y eliminar automáticamente fallas de cableado.

Al igual que en el *token bus*, puede haber prioridades, aunque el esquema no es equitativo. También, como el *token bus*, son posibles los marcos cortos pero, a diferencia suya, también son posibles los arbitrariamente grandes, limitados sólo por el tiempo de retención de la ficha. Por último, el rendimiento y la eficiencia cuando la carga es alta son excelentes, como en el *token bus* y a diferencia del 802.3.

El punto negativo principal es la presencia de una función de supervisión centralizada, que introduce un componente crítico. Aun cuando un monitor muerto puede reemplazarse, uno enfermo puede causar dolores de cabeza. Además, como en todos los esquemas de paso de ficha, siempre hay un retardo a muy baja carga, porque el transmisor debe esperar la ficha.

También vale la pena señalar que ha habido numerosos estudios de las tres LAN. La conclusión principal que podemos sacar de estos estudios es que no podemos sacar conclusiones de ellos. Siempre se puede encontrar un grupo de parámetros que hacen que una de las LAN se vea mejor que las demás. En la mayoría de las circunstancias, las tres se desempeñan bien, por lo que probablemente son más importantes los factores distintos del desempeño al hacer una elección.

4.3.5. Estándar IEEE 802.6: *bus* doble de colas distribuidas

Ninguna de las LAN 802 que hemos estudiado hasta ahora es adecuada para usarse como MAN. Las limitaciones de longitud del cable y los problemas de desempeño cuando se conectan miles de estaciones las limitan a áreas de extensión pequeña. Para redes que cubren una ciudad completa, IEEE definió una MAN, llamada **DQDB** (*Distributed Queue Dual Bus*, **bus doble de colas distribuidas**), como el estándar 802.6. En esta sección estudiaremos su funcionamiento. Para información adicional, véase (Kessler y Train, 1992). En (Sadiku y Arvind, 1992) se da una bibliografía que lista 171 artículos sobre el DQDB.

En la figura 1-4 se ilustra la geometría básica del 802.6. Dos *buses* unidireccionales paralelos serpentean a través de la ciudad, con estaciones conectadas a ambos *buses* en paralelo. Cada *bus* tiene un *head-end*, que genera una cadena constante de células de 53 bytes. Cada célula viaja corriente abajo del *head-end*. Cuando la célula llega al final, sale del *bus*.

Cada célula lleva un campo de carga de 44 bytes, haciéndola compatible con algunos modos AAL. Cada célula contiene también dos bits de protocolo, *ocupado*, que se establece para indicar que la célula está ocupada, y *solicitud*, que puede establecerse cuando una estación quiere hacer una solicitud.

Para transmitir una célula, una estación necesita saber si el destino está a la izquierda o a la derecha de ella. Si el destino está a la derecha, el transmisor usa el *bus A*; de otra manera, usa el *bus B*. Los datos se introducen en cualquiera de los dos *buses* usando un circuito OR alambrado, de manera que la falla de una estación no desactive la red.

A diferencia de los otros protocolos de LAN 802, el 802.6 no es ávido. En todos los demás, si una estación tiene la oportunidad de enviar, lo hará. Aquí las estaciones se ponen en cola en el orden en el que van quedando listas para enviar y transmitir, en orden FIFO (*first in, first out*,

el que entra primero sale primero). La parte interesante del protocolo es la manera en la que logra el orden FIFO sin tener una cola central.

La regla básica es que las estaciones son amables: ceden el paso a estaciones más abajo de ellas. Esta amabilidad es necesaria para evitar una situación en la que la estación más cercana al *head-end* simplemente toma todas las células vacías a medida que pasan y las llena, haciendo que las otras más abajo mueran de hambre. Por sencillez, sólo examinaremos la transmisión en el *bus A*, pero la misma historia se aplica también al *bus B*.

Para simular la cola FIFO, cada estación mantiene dos contadores, *RC* y *CD*. *RC* (*Request Counter*, contador de solicitudes) cuenta el número de solicitudes pendientes corriente abajo hasta que la estación misma tiene un marco por enviar. En ese punto, *RC* se copia en *CD*, *RC* se restablece a 0 y ahora cuenta el número de solicitudes hechas después de que la estación quedó lista. Por ejemplo, si $CD = 3$ y $RC = 2$ para la estación k, las siguientes tres células vacías que pasen por la estación k se reservarán para estaciones corriente abajo, después la estación k podrá enviar, luego dos células más se reservarán para estaciones corriente abajo. Por sencillez, supondremos que una estación puede tener lista a la vez sólo una célula para transmisión.

Para enviar una célula, una estación primero debe hacer una reservación estableciendo el bit de *solicitud* en alguna célula en el *bus* de reversa (es decir, en el *bus B* para una transmisión que luego ocurrirá en el *bus A*). A medida que se propaga esta célula a través del *bus* en reversa, cada estación en el camino lo nota y aumenta su *RC*. Para ilustrar este concepto, usaremos un ejemplo. Inicialmente, todos los contadores *RC* están en 0, y no hay células en cola, como se muestra en la figura 4-32(a). Entonces, la estación *D* hace una solicitud, que causa que las estaciones *C*, *B* y *A* incrementen sus contadores *RC*, como se muestra en la figura 4-32(b). Tras eso, *B* hace una solicitud, copiando su valor *RC* actual en *CD* y conduciendo a la situación de la figura 4-32(c).

En este punto, el *head-end* del *bus A* genera una célula vacía. Cuando la célula pasa por *B*, esa estación ve que su $CD > 0$, por lo que no puede usar la célula vacía. (Cuando una estación tiene una célula en cola, *CD* representa su posición en la cola, siendo 0 el inicio de la cola.) En cambio, la estación disminuye *CD*. Cuando la célula aún vacía llega a *B*, esa estación ve que $CD = 0$, lo que significa que no hay nadie adelante de ella en la cola, por lo que hace un OR de sus datos en la célula y establece el bit de *ocupado*. Una vez que se ha efectuado la transmisión, tenemos la situación de la figura 4-32(d).

Cuando se genera la siguiente célula vacía, la estación *D* ve que ahora está al frente de la cola, y toma la célula (estableciendo un bit), como se ilustra en la figura 4-32(e). De esta manera, las estaciones se ponen en cola para tomar turnos, sin un administrador centralizado de colas.

Los sistemas DQDB ya están siendo instalados por muchas portadoras a través de ciudades enteras; típicamente operan hasta a 160 km a velocidades de 44.736 Mbps (T3).

4.3.6. Estándar IEEE 802.2: control lógico de enlace

Tal vez ahora sea el momento de dar un paso atrás y comparar lo que hemos aprendido en este capítulo con lo que estudiamos en el anterior. En el capítulo 3 vimos la manera en que dos máquinas se podían comunicar de manera confiable a través de una línea inestable usando varios

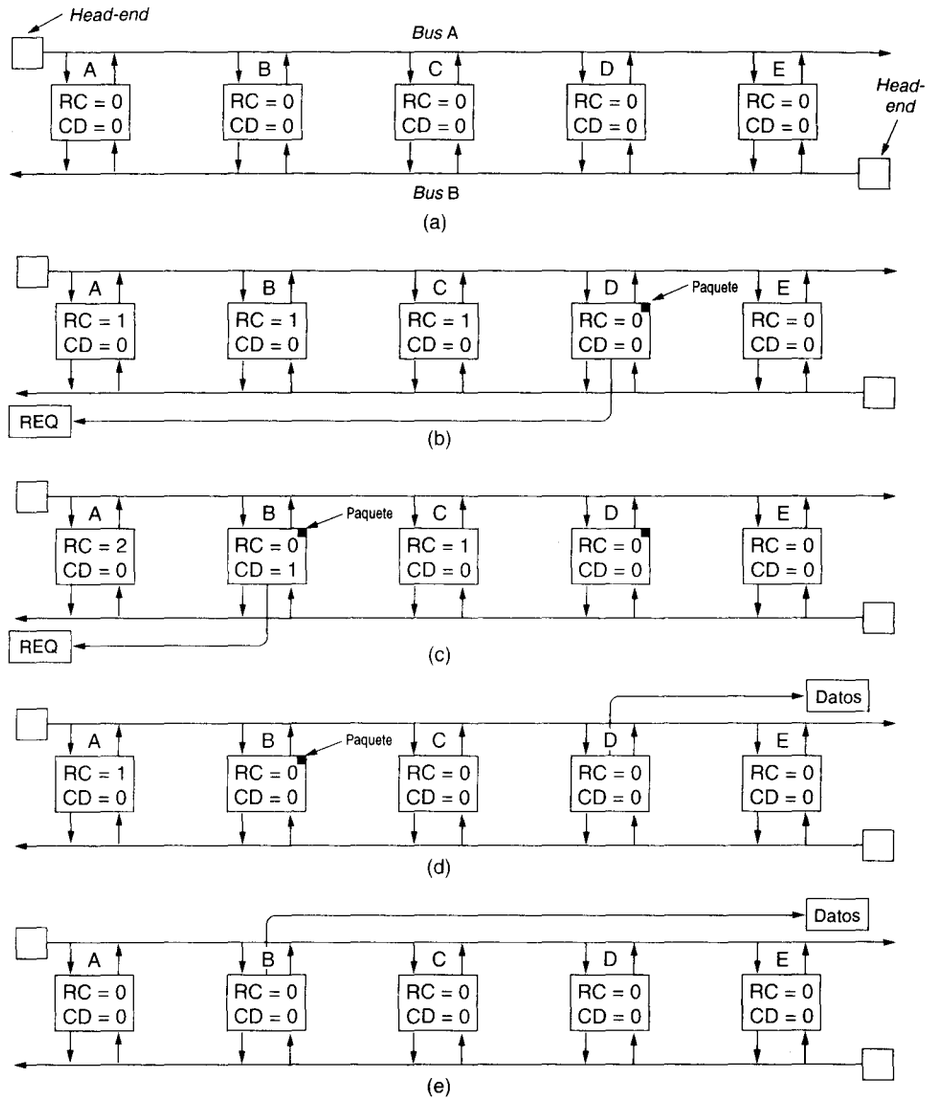


Figura 4-32. (a) Inicialmente, la MAN está inactiva. (b) Después de que D hace una solicitud. (c) Después de que B hace una solicitud. (d) Después de que D transmite. (e) Después de que B transmite.

protocolos de enlace de datos. Estos protocolos proporcionaban control de errores (mediante acuses) y control de flujo (usando una ventana corrediza).

En contraste, en este capítulo no hemos mencionado las comunicaciones confiables. Todo lo que ofrecen las LAN y las MAN 802 es un servicio de datagramas que hace su mejor esfuerzo.

A veces, este servicio es adecuado. Por ejemplo, para transportar paquetes IP no se requieren ni se esperan garantías. Un paquete IP simplemente puede introducirse en un campo de carga 802 y enviarse a su destino; si se pierde, que así sea.

Sin embargo, también hay sistemas en los que se desea un protocolo de enlace de datos con control de errores y control de flujo. El IEEE ha definido uno que puede operar encima de todos los protocolos LAN y MAN 802. Además, este protocolo, llamado **LLC** (*Logical Link Control*, **control lógico de enlace**), esconde las diferencias entre los diferentes tipos de redes 802, proporcionando un formato único y una interfaz con la capa de red. Este formato, interfaz y protocolo están basados estrechamente en OSI. El LLC forma la mitad superior de la capa de enlace de datos, con la subcapa de MAC por debajo de él, como se muestra en la figura 4-33.

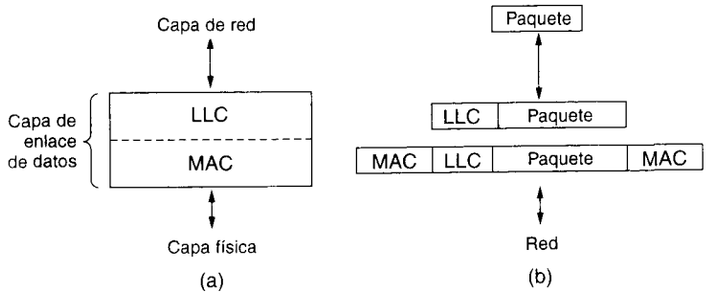


Figura 4-33. (a) Posición del LLC. (b) Formatos de protocolo.

El uso típico del LLC es el siguiente. La capa de red de la máquina transmisora pasa un paquete al LLC usando las primitivas de acceso del LLC. La subcapa LLC entonces agrega una cabecera LLC que contiene los números de secuencia y acuse. La estructura resultante se introduce entonces en el campo de carga útil de un marco 802.x y se transmite. En el receptor ocurre el proceso inverso.

El LLC proporciona tres opciones de servicio: servicio no confiable de datagramas, servicio reconocido de datagramas y servicio confiable orientado a conexión. La cabecera del LLC está basada en el antiguo protocolo HDLC. Se usa una variedad de formatos diferentes para datos y control. En el caso de datagramas reconocidos y servicio orientado a conexión, los marcos de datos contienen una dirección de origen, una dirección de destino, un número de secuencia, un número de acuse y unos cuantos bits misceláneos. En el servicio de datagramas no confiable, se omiten el número de secuencia y el número de acuse.

4.4. PUENTES

Muchas organizaciones tienen varias LAN y desean conectarlas. Las LAN pueden conectarse mediante dispositivos llamados **puentes** (*bridges*), que operan en la capa de enlace de datos. Este postulado implica que los puentes no examinan la cabecera de la capa de red y que pueden,

por tanto, copiar igualmente bien paquetes IP, IPX y OSI. En contraste, un enrutador IP, IPX u OSI puro puede manejar sólo sus propios paquetes nativos.

En las siguientes secciones veremos el diseño de los puentes, especialmente para conectar LAN 802.3, 802.4 y 802.5. Para un tratamiento detallado de los puentes y temas relacionados, véase (Perlman, 1992). Antes de entrar en la tecnología de los puentes, es importante examinar algunas situaciones comunes en las que se usan los puentes. Mencionaremos seis razones por las que una sola organización puede llegar a tener múltiples LAN. Primero, muchas universidades y departamentos corporativos tienen sus propias LAN, principalmente para interconectar sus propias computadoras personales, estaciones de trabajo y servidores. Dado que las metas de los distintos departamentos difieren, los departamentos escogen LAN diferentes, sin importarles lo que hagan los demás departamentos. Tarde o temprano surge la necesidad de interacción, para lo que se requieren puentes. En este ejemplo, surgieron múltiples LAN debido a la autonomía de sus dueños.

Segunda, la organización puede estar distribuida geográficamente en varios edificios, separados por distancias considerables. Puede ser más económico tener LAN independientes en cada edificio y conectarlas mediante puentes y enlaces infrarrojos que tender un solo cable coaxial a través de toda la zona.

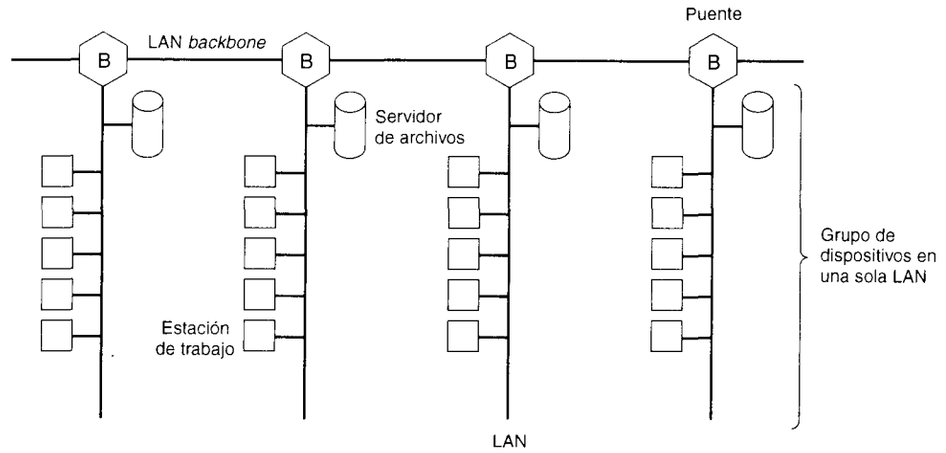


Figura 4-34. Varias LAN conectadas mediante un *backbone* para manejar una carga total mayor que la capacidad de una sola LAN.

Tercera, puede ser necesario dividir lo que lógicamente es una sola LAN en LAN individuales para manejar la carga. Por ejemplo, en muchas universidades miles de estaciones de trabajo están disponibles para los estudiantes y el cuerpo docente. Los archivos normalmente se guardan en máquinas servidoras de archivos, y son descargados a las máquinas de los usuarios a solicitud. La enorme escala de este sistema hace imposible poner todas las estaciones de trabajo en una sola LAN, pues el ancho de banda necesario es demasiado grande. En cambio, se usan varias

LAN conectadas mediante puentes, como se muestra en la figura 4-34. Cada LAN contiene un grupo de estaciones de trabajo con su propio servidor de archivos, por lo que la mayor parte del tráfico está restringida a una sola LAN y no agrega carga al *backbone*.

Cuarta, en algunas situaciones una sola LAN sería adecuada en términos de la carga, pero la distancia física entre las máquinas más distantes es demasiado grande (por ejemplo, mayor que 2.5 km para el 802.3). Aun si fuera fácil tender el cable, la red no funcionaría debido al retardo excesivamente grande del viaje de ida y vuelta. La única solución es segmentar la LAN e instalar puentes entre los segmentos. Usando puentes, puede aumentarse la distancia física total cubierta.

Quinta, está la cuestión de la confiabilidad. En una sola LAN, un nodo defectuoso que envíe constantemente una cadena de basura echará a perder la LAN. Pueden introducirse puentes en lugares críticos, como puertas para bloquear el fuego en un edificio, y así evitar que un solo nodo enloquecido tire el sistema completo. A diferencia de un repetidor, que sólo copia lo que ve, un puente puede programarse para ejercer algún arbitrio respecto a lo que envía y lo que no.

Sexta y última, los puentes pueden contribuir a la seguridad de la organización. La mayor parte de las interfaces de LAN tienen un **modo promiscuo**, en el que *todos* los marcos se entregan a la computadora, no sólo aquellos dirigidos a ella. Los espías y los intrusos aman esta característica. Al introducir puentes en varios lugares y tener cuidado de no reenviar tráfico delicado, es posible aislar partes de la red de manera que su tráfico no pueda escapar y caer en las manos equivocadas.

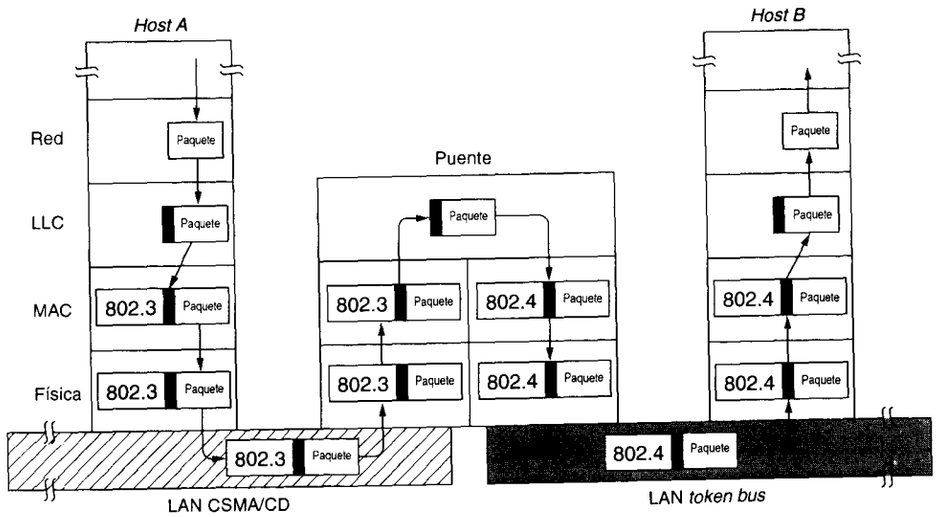


Figura 4-35. Operación de un puente LAN de 802.3 a 802.4.

Habiendo constatado la necesidad de los puentes, pasemos a la cuestión de su funcionamiento. En la figura 4-35 se ilustra la operación de un puente sencillo de dos puertos. El *host A* tiene un paquete por enviar. El paquete desciende a la subcapa LLC y adquiere una cabecera LLC; luego pasa a la subcapa MAC y se le añade una cabecera 802.3 (también un apéndice, que no se muestra en la figura). Esta unidad sale al cable y en algún momento se pasa hacia arriba a la subcapa MAC del puente, donde se le quita la cabecera 802.3. El paquete solo (con la cabecera LLC) se entrega entonces a la subcapa LLC del puente. En este ejemplo, el paquete está destinado a una subred 802.4 conectada al puente, por lo que pasa al lado 802.4 del puente y sale por ahí. Nótese que un puente que conecta k LAN diferentes tendrá k subcapas MAC y k capas físicas, una para cada tipo.

4.4.1. Puentes de 802.x a 802.y

Usted podría pensar ingenuamente que un puente entre una LAN 802 y otra sería completamente trivial. Éste no es el caso. En el resto de esta sección indicaremos algunas de las dificultades que se presentan al tratar de construir un puente entre las distintas LAN 802.

Cada una de las nueve combinaciones de 802.x a 802.y tiene su propio grupo de problemas. Sin embargo, antes de tratar éstos uno por uno, veamos algunos problemas generales comunes a todos los puentes. Para comenzar, cada una de las LAN usa un formato de marco diferente (véase la figura 4-36). No hay razón técnica válida para esta incompatibilidad. Simplemente es que ninguna de las corporaciones que promovieron los tres estándares (Xerox, GM e IBM) quería cambiar *el suyo*. Como resultado, cualquier copiado entre las diferentes LAN requiere reformateo, lo que gasta tiempo de CPU, obliga a un nuevo cálculo de suma de comprobación e introduce la posibilidad de errores no detectados debido a bits erróneos en la memoria del puente. Nada de esto habría sido necesario si los tres comités hubieran sido capaces de acordar un solo formato.

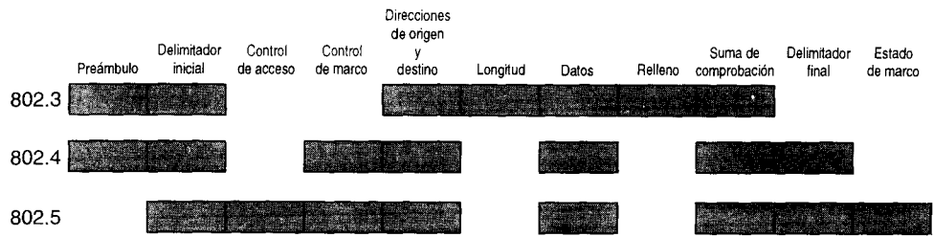


Figura 4-36. Formatos de marco de los IEEE 802.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

Un segundo problema es que las LAN interconectadas no necesariamente operan con la misma tasa de datos. Al enviar un grupo grande de marcos uno tras otro de una LAN rápida a una lenta, el puente no será capaz de deshacerse de los marcos a la misma velocidad a la que llegan; tendrá que ponerlos en *buffers*, esperando que no se le acabe la memoria. El problema también existe en alguna medida del 802.4 a 802.3 a 10 Mbps, pues se pierde un poco del ancho

de banda del 802.3 debido a colisiones. En realidad el 802.3 no tiene 10 Mbps, en tanto que el 802.4 sí lo tiene (bueno, casi). Los puentes que conectan a tres o más LAN tienen un problema parecido cuando varias LAN intentan alimentar simultáneamente a la misma LAN de salida.

Un problema sutil, pero importante, relacionado con el problema del puente como cuello de botella es el valor de los temporizadores (*timers*) en las capas superiores. Supóngase que la capa de red de la LAN 802.4 está tratando de enviar un mensaje muy largo como secuencia de marcos. Tras enviar el último, esta capa inicia un temporizador para esperar un acuse. Si el mensaje tiene que transitar por un puente a una LAN 802.5 más lenta, hay el peligro de que el temporizador termine antes de que el último marco haya sido reenviado a la LAN más lenta. La capa de red supondrá que el problema se debe a un marco perdido y simplemente retransmitirá de nuevo la secuencia completa. Tras n intentos fallidos, podrá darse por vencida e indicar a la capa de transporte que el destino está muerto.

Un tercer problema, que puede ser el más grave de todos, es que las tres LAN 802 tienen una longitud máxima de marco diferente. En el 802.3 esta longitud depende de los parámetros de la configuración pero, para el sistema estándar de 10 Mbps, la carga útil máxima es de 1500 bytes. En el 802.4 la longitud está fija en 8191 bytes. Para el 802.5 no hay límite superior, excepto que una estación no puede transmitir durante un tiempo mayor que el de retención de la ficha. Con el valor predeterminado de 10 mseg, la longitud máxima de marco es de 5000 bytes.

Surge un problema obvio cuando debe reenviarse un marco grande a una LAN que no puede aceptarlo. En esta capa es imposible la división del marco en partes. Todos los protocolos suponen que los marcos llegan o no. No se contempla el reensamblado de los marcos a partir de unidades más pequeñas. Esto no quiere decir que no se podrían desarrollar tales protocolos; se puede, y se ha hecho. La cuestión es que el 802 no tiene esta característica. Básicamente, no hay solución. Los marcos que son demasiado grandes para reenviarse deben descartarse. Así que habrá que olvidarse de la transparencia.

Consideremos ahora brevemente cada uno de los nueve casos de puentes 802.x a 802.y para ver qué otros problemas acechan entre las sombras. Del 802.3 al 802.3 es muy sencillo. Lo único que puede fallar es que la LAN de destino esté tan cargada que los marcos sigan entrando en el puente, pero el puente no pueda deshacerse de ellos. Si persiste esta situación durante un tiempo suficiente, se puede agotar el espacio de *buffer* del puente, y éste comenzará a tirar marcos. Dado que este problema siempre está potencialmente presente al reenviar a una LAN 802.3, no lo mencionaremos más. Con las otras dos LAN, cada estación, incluido el puente, tiene la garantía de adquirir la ficha periódicamente y no puede quedar excluido durante intervalos grandes.

Del 802.4 al 802.3 hay dos problemas. Primero, los marcos 802.4 llevan bits de prioridad que los marcos 802.3 no tienen. Como resultado, si dos LAN 802.4 se comunican mediante una LAN 802.3, la prioridad se perderá en la LAN intermedia.

El segundo problema es causado por una característica específica del 802.4: la entrega temporal de la ficha. Es posible que un marco 802.4 tenga un bit de cabecera puesto en 1 para pasar temporalmente la ficha al destino, a fin de permitirle enviar un marco de acuse. Sin

embargo, si se reenvía uno de tales marcos desde un puente, ¿qué debe hacer el puente? Si envía él mismo un marco de acuse, estará mintiendo, pues en realidad aún no ha sido entregado el marco. De hecho, el destino puede estar muerto.

Por otra parte, si el puente no genera el acuse, el transmisor casi con seguridad concluirá que el destino está muerto e informará de la falla a sus superiores. No parece haber ninguna manera de resolver este problema.

Del 802.5 al 802.3 tenemos un problema parecido. El formato de marco 802.5 tiene los bits *A* y *C* en el byte de estado del marco. Estos bits los establece el destino para indicar al transmisor si la estación direccionada vio el marco, y si lo copió. Nuevamente, el puente puede mentir y decir que el marco ha sido copiado pero, si luego resulta que el destino está inactivo, pueden ocurrir problemas serios. En esencia, la inserción de un puente en la red ha cambiado la semántica de los bits. Es difícil imaginar una solución adecuada a este problema.

Del 802.3 al 802.4 tenemos el problema de qué poner en los bits de prioridad. Parece bastante sensato hacer que el puente retransmita todos los marcos a la prioridad máxima, dado que probablemente ya han sufrido suficiente retardo.

Del 802.4 al 802.4 el único problema es qué hacer con la entrega temporal de la ficha. Cuando menos, aquí tenemos la posibilidad de que el puente logre reenviar el marco a una velocidad suficiente para obtener la respuesta antes de que termine el temporizador. Aun así, es un riesgo. Al reenviar el marco a la prioridad más alta, el puente miente inocentemente, pero de esa manera aumenta la probabilidad de obtener la respuesta a tiempo.

Del 802.5 al 802.4 tenemos el mismo problema que antes con los bits *A* y *C*. También la definición de los bits de prioridad es distinta para las dos LAN, pero los mendigos no pueden escoger. Al menos las dos LAN tienen el mismo número de bits de prioridad. Todo lo que puede hacer el puente es copiar y pasar los bits de prioridad, y esperar lo mejor.

Del 802.3 al 802.5, el puente debe generar bits de prioridad, pero no hay otros problemas especiales. Del 802.4 al 802.5 hay un problema potencial con marcos que son demasiado grandes, y se presenta nuevamente el problema de la entrega de la ficha. Finalmente, del 802.5 al 802.5 el problema es qué hacer nuevamente con los bits *A* y *C*. En la figura 4-37 se resumen los distintos problemas que hemos analizado.

Cuando el comité del IEEE 802 se dio a la tarea de desarrollar un estándar de LAN, fue incapaz de acordar un estándar único, por lo que produjo *tres* estándares incompatibles, como acabamos de ver con algún detalle. Por este fracaso el comité ha sido unánimemente criticado. Cuando posteriormente se asignó al mismo comité la tarea de diseñar un estándar de puentes para interconectar sus tres LAN incompatibles, decidió hacerlo mejor y lo hizo: desarrolló *dos* diseños de puente incompatibles. Hasta ahora nadie le ha pedido diseñar un estándar de pasarela entre sus dos puentes incompatibles, pero cuando menos la tendencia es en la dirección correcta.

Esta sección ha tratado los problemas que surgen al conectar dos LAN IEEE 802 mediante un solo puente. Las siguientes dos secciones tratan los problemas de la conexión de interredes grandes que contienen muchas LAN y muchos puentes, y los dos enfoques del IEEE para diseñar estos puentes.

LAN de origen \ LAN de destino	802.3 (CSMA/CD)	802.4 (*Token bus*)	802.4 (*Token ring*)
802.3		1, 4	1, 2, 4, 8
802.4	1, 5, 8, 9, 10	9	1, 2, 3, 8, 9, 10
802.5	1, 2, 5, 6, 7, 10	1, 2, 3, 6, 7	6, 7

Acciones:
1. Reformatea el marco y calcula suma de comprobación nueva.
2. Invierte el orden de bits.
3. Copia la prioridad, sea significativa o no.
4. Genera una prioridad ficticia.
5. Desecha la prioridad.
6. Drena el anillo (de alguna manera).
7. Establece los bits A y C (mintiendo).
8. Preocupación por los congestionamientos (LAN rápida a LAN lenta).
9. Preocupación por el retardo o la imposibilidad del reconocimiento de la entrega de la ficha.
10. Pánico si el marco es demasiado grande para la LAN de destino.

Parámetros supuestos:
802.3: marcos de 1500 bytes 10 Mbps (menos colisiones)
802.4: marcos de 8191 bytes 10 Mbps
802.5: marcos de 5000 bytes 4 Mbps

Figura 4-37. Problemas encontrados en la construcción de puentes entre 802.x y 802.y.

4.4.2. Puentes transparentes

El primer puente 802 es un **puente transparente** o **puente de árbol de extensión** (Perlman, 1992). La preocupación principal de la gente que apoyó este diseño fue la transparencia completa. Desde su punto de vista, una instalación con varias LAN debería ser capaz de salir y comprar puentes diseñados para el estándar IEEE, enchufar los conectores a los puentes, y todo debería funcionar perfectamente al instante. No deberían requerirse cambios de *hardware*, cambios de *software*, ajustes de conmutadores de direccionamiento, carga de tablas o parámetros de enrutamiento, nada. Simplemente conectar los cables y dedicarse a otro asunto. Además, la operación de las LAN existentes no debería ser afectada por los puentes en lo absoluto. Sorprendente, pero lo lograron.

Un puente transparente trabaja en modo promiscuo, aceptando todo marco transmitido por cualquier LAN a la que está conectado. Por ejemplo, considere la configuración de la figura 4-38. El puente B1 está conectado a las LAN 1 y 2, y el puente B2 está conectado a las LAN 2, 3 y 4. Un marco que llega al puente B1 de la LAN 1, destinado a *A*, puede descartarse de inmediato, puesto que ya está en la LAN adecuada, pero un marco que llegue para *C* o *F* por la LAN 1 se debe reenviar.

Al llegar un marco, un puente debe decidir si lo descarta o lo reenvía y, de ser lo segundo, en qué LAN debe poner el marco. Esta decisión se toma buscando la dirección de destino en una

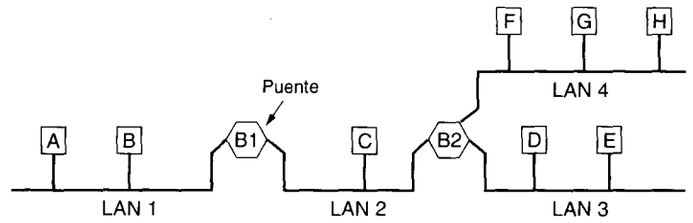


Figura 4-38. Configuración con cuatro LAN y dos puentes.

gran tabla (de dispersión) contenida en el puente. La tabla puede listar cada posible destino e indicar a qué línea de salida (LAN) pertenece. Por ejemplo, la tabla de B2 indicaría que *A* pertenece a la LAN 2, ya que todo lo que B2 tiene que saber es en qué LAN tiene que poner los marcos para *A*. El que se reenvíe posteriormente no le interesa.

Al conectarse inicialmente los puentes, todas las tablas de dispersión están vacías. Ninguno de los puentes sabe la ubicación de ninguno de los destinos, por lo que usa el algoritmo de inundación: cada marco de entrada para un destino desconocido se envía a todas las LAN a las que está conectado el puente, excepto a aquella por la que llegó. A medida que pasa el tiempo, los puentes aprenden la ubicación de los destinos, como se describe más adelante. Una vez que se conoce un destino, los marcos destinados a él se colocan en la LAN apropiada y ya no se efectúa el proceso de inundación.

El algoritmo empleado por los puentes transparentes es de **aprendizaje de lo sucedido**. Como se mencionó antes, los puentes operan en modo promiscuo, por lo que ven cada marco enviado por cualquiera de sus LAN. Examinando la dirección de origen, pueden saber qué máquina está accesible desde qué LAN. Por ejemplo, si el puente B1 de la figura 4-38 ve un marco en la LAN 2 que viene de *C*, sabe que se puede llegar a *C* a través de la LAN 2, por lo que hace una entrada en su tabla de dispersión anotando que los marcos que van a *C* deben usar la LAN 2. Cualquier marco subsiguiente dirigido a *C* que entre por la LAN 1 será reenviado, pero un marco para *C* que llegue por la LAN 2 será descartado.

La topología puede cambiar a medida que las máquinas y los puentes se encienden, se apagan y se mueven. Para manejar topologías dinámicas, cada vez que se hace una entrada en la tabla de dispersión se anota la hora de llegada del marco en la entrada. Cada vez que llega un marco cuyo destino ya está en la tabla, se actualiza su entrada con la hora actual. Por tanto, el tiempo asociado a cada entrada indica la última vez que se vio un marco de esa máquina.

Periódicamente, un proceso del puente recorre la tabla de dispersión y elimina las entradas que tienen más de unos cuantos minutos. De esta manera, si una computadora se desconecta de su LAN, se mueve a otra parte del edificio y se reconecta en otro lado, en pocos minutos entrará de nuevo en operación normal, sin intervención manual. Este algoritmo también significa que, si una máquina está quieta durante algunos minutos, cualquier tráfico enviado a ella tendrá que pasar por el proceso de inundación, hasta que ella misma envíe un marco.

El proceso de enrutamiento para un marco de entrada depende de la LAN por la que llega (la LAN de origen) y de la LAN en la que está su destino (la LAN de destino), como sigue:

1. Si la LAN de destino y la LAN de origen son la misma, se descarta el marco.
2. Si la LAN de destino y la de origen son distintas, se reenvía el marco.
3. Si la LAN de destino es desconocida, se usa el proceso de inundación.

A medida que llega cada marco, debe aplicarse este algoritmo. Existen *chips* VLSI especiales para realizar la búsqueda y actualización de la entrada de tabla, todo en unos cuantos microsegundos.

A fin de aumentar la confiabilidad, algunas instalaciones tienen dos o más puentes en paralelo entre pares de LAN, como se muestra en la figura 4-39. Este arreglo, sin embargo, también agrega algunos problemas adicionales, pues crea ciclos en la topología.

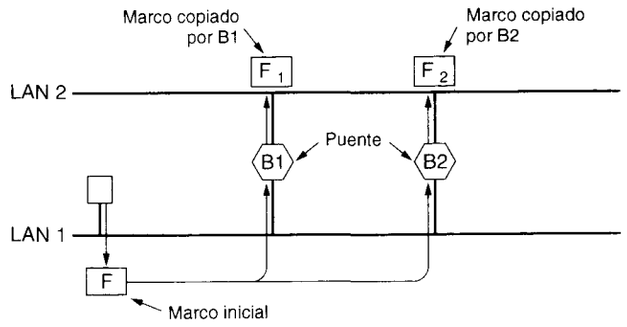


Figura 4-39. Dos puentes transparentes paralelos.

Un ejemplo sencillo de estos problemas es evidente al observar la manera en que se maneja un marco F con un destino desconocido en la figura 4-39. Cada puente, siguiendo las reglas normales de manejo de destinos desconocidos, usa inundación, que en este ejemplo significa simplemente copiar el marco en la LAN 2. Poco después, el puente 1 ve a F_2, un marco con un destino desconocido, el cual copia en la LAN 1, generando F_3 (que no se muestra). Del mismo modo, el puente 2 copia F_1 en la LAN 1, generando F_4 (que tampoco se muestra). El puente 1 ahora reenvía F_4 y el puente 2 copia F_3. Este ciclo continúa indefinidamente.

Puentes de árbol de extensión

La solución para este problema es comunicar los puentes entre sí y extender sobre la topología actual un árbol de extensión que alcance todas las LAN. En efecto, se ignoran algunas conexiones potenciales entre las LAN por el interés de construir una topología ficticia libre de ciclos. Por ejemplo, en la figura 4-40(a) vemos nueve LAN interconectadas por 10 puentes. Esta configuración puede abstraerse en un grafo con las LAN como nodos. Un arco conecta cada par

de LAN que están conectadas por un puente. El grafo puede reducirse a árbol de extensión eliminando los arcos que aparecen como líneas punteadas en la figura 4-40(b). En este árbol de extensión hay exactamente una trayectoria de cada LAN a cualquier otra LAN. Una vez que los puentes han acordado el árbol de extensión, todo el reenvío entre las LAN sigue al árbol de extensión. Dado que hay una trayectoria única de cada origen a cada destino, es imposible que ocurran ciclos.

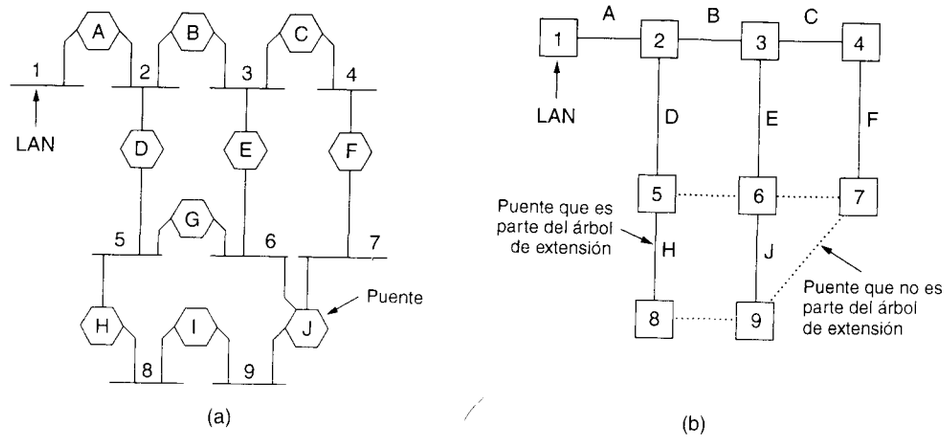


Figura 4-40. (a) LAN interconectadas. (b) Árbol de extensión que cubre las LAN. Las líneas punteadas no son parte del árbol.

Para construir el árbol de extensión, los puentes primero tienen que escoger un puente que sea la raíz del árbol; deben tomar esta decisión haciendo que cada uno difunda su número de serie, instalado por el fabricante y con garantía de ser único en el mundo. El puente con el número de serie menor se vuelve la raíz. A continuación, se construye un árbol de trayectorias mínimas de la raíz a cada puente y LAN. Este árbol es el árbol de extensión. Si falla un puente o una LAN, se calcula un árbol nuevo.

El resultado de este algoritmo es que se establece una trayectoria única de cada LAN hasta la raíz, y por tanto a todas las demás LAN. Aunque el árbol abarca todas las LAN, no necesariamente están presentes todos los puentes en el árbol (para evitar ciclos). Aun después de que se ha establecido el árbol de extensión, el algoritmo continúa operando a fin de detectar automáticamente cambios de topología y actualizar el árbol. El algoritmo distribuido que se usa para construir el árbol de extensión fue inventado por Perlman y se describe en detalle en (Perlman, 1992).

También pueden construirse puentes para conectar las LAN que están muy separadas. En este modelo, cada instalación consiste en un conjunto de LAN y puentes, de los cuales uno tiene conexión con una WAN. Los marcos para las LAN remotas viajan a través de la WAN. Puede usarse el algoritmo básico de árbol de extensión, de preferencia con ciertas optimizaciones para seleccionar un árbol que minimice la cantidad de tráfico en la WAN.

4.4.3. Puentes de enrutamiento desde el origen

Los puentes transparentes tienen la ventaja de ser fáciles de instalar. Simplemente se enchufan y a otra cosa. Por otra parte, no hacen uso óptimo del ancho de banda, ya que sólo utilizan un subconjunto de la topología (el árbol de extensión). La importancia relativa de estos dos (y otros) factores condujo a una división en el comité 802 (Pitt, 1988). La gente de CSMA/CD y *token bus* escogió el puente transparente. La gente de anillos (con apoyo de IBM) prefirió un esquema llamado **enrutamiento desde el origen**, que ahora describiremos. Para detalles adicionales, véase (Dixon, 1987).

En esencia, el enrutamiento desde el origen supone que el transmisor de cada marco sabe si el destino está en su propia LAN. Cuando la máquina de origen envía un marco a una LAN diferente, establece en 1 el bit de orden mayor de la dirección de origen, para marcarlo. Además, incluye en la cabecera del marco la trayectoria exacta que seguirá el marco.

Esta trayectoria se construye como sigue. Cada LAN tiene un número único de 12 bits, y cada puente tiene un número de 4 bits que lo identifica de manera única en el contexto de su LAN. Por tanto, dos puentes distanciados pueden tener el número 3, pero dos puentes entre las mismas dos LAN deben tener números de puente distintos. Entonces, una ruta es una secuencia de números de puente, LAN, puente, LAN, ... Haciendo referencia a la figura 4-38, la ruta de *A* a *D* sería (L1, B1, L2, B2, L3).

Un puente con enrutamiento de origen sólo está interesado en aquellos marcos que tienen el bit de orden mayor del destino puesto en 1. Para cada uno de tales marcos que ve, examina la ruta buscando el número de la LAN por la que llegó el marco. Si este número de LAN va seguido de su propio número de puente, el puente reenvía el marco a la LAN cuyo número sigue a su número de puente en la ruta. Si el número de la LAN entrante va seguido del número de algún otro puente, no reenvía el marco.

Este algoritmo se presta para tres posibles implementaciones:

1. *Software*: el puente opera en modo promiscuo, copiando todos los marcos en su memoria para ver si tienen establecido en 1 el bit de orden mayor del destino. De ser así, se sigue examinando el marco, de otro modo, no se continúa la inspección.
2. Híbrida: la interfaz de LAN del puente inspecciona el bit de orden mayor del destino y sólo acepta los marcos que tienen el bit establecido. Esta interfaz es fácil de incorporar en el *hardware* y reduce de manera importante la cantidad de marcos que debe inspeccionar el puente.
3. *Hardware*: la interfaz de la LAN no sólo revisa el bit de orden mayor del destino, sino que también examina la ruta para ver si este puente debe reenviar. Sólo los marcos que sí deben reenviarse son entregados al puente. Esta implementación requiere del *hardware* más complejo, pero no desperdicia ciclos de CPU del puente, porque se filtran todos los marcos que no son pertinentes.

Estas tres implementaciones varían considerablemente en costo y desempeño. La primera no tiene un costo adicional de *hardware* por la interfaz, pero puede requerir una CPU muy rápida

para manejar todos los marcos. La última no requiere un *chip* VLSI, pues descarga gran parte del procesamiento del puente al *chip*, por lo que puede usarse una CPU más lenta o, alternativamente, el puente puede manejar más LAN.

Implícito en el diseño del enrutamiento desde el origen está el hecho de que cada máquina de la interred conoce o puede encontrar la mejor trayectoria a todas las demás máquinas. La manera de descubrir estas rutas es una parte importante del algoritmo de enrutamiento desde el origen. El concepto básico es que, si el origen desconoce un destino, difunde un marco preguntando dónde está. Este **marco de descubrimiento** es reenviado por cada puente de modo que llegue a todas las LAN de la interred. Cuando regresa la respuesta, los puentes registran en ella su identidad, por lo que el transmisor original puede ver la ruta exacta que siguió y escoger la mejor.

Aunque este algoritmo ciertamente encuentra la mejor ruta (encuentra *todas* las rutas), sufre de una explosión de marcos. Considere la configuración de la figura 4-41, con *N* LAN conectadas linealmente mediante puentes triples. Cada marco de descubrimiento enviado por la estación 1 es copiado por cada uno de los tres puentes de la LAN 1, produciendo tres marcos de descubrimiento en la LAN 2. Todos estos son copiados por cada uno de los puentes, resultando en nueve marcos en la LAN 3. Para el momento en que alcanzamos la LAN *N*, están circulando 3^{N-1} marcos. Si se interpone una docena de grupos de puentes, se tendrán que inyectar más de medio millón de marcos de descubrimiento en la última LAN, causando un congestionamiento severo.

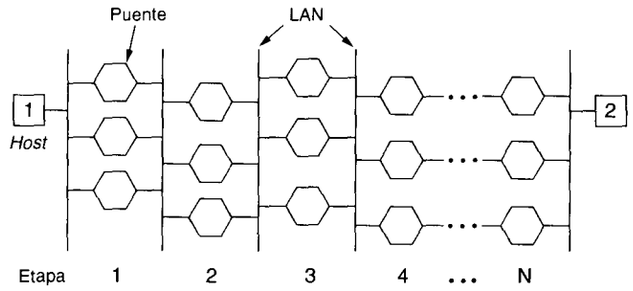


Figura 4-41. Serie de LAN conectadas mediante puentes triples.

Ocurre un proceso análogo con el puente transparente, sólo que no es tan grave. Cuando llega un marco desconocido, pasa por el proceso de inundación, pero sólo a través del árbol de extensión, por lo que el volumen total de marcos enviados es lineal respecto al tamaño de la red, no exponencial.

Una vez que el *host* ha descubierto una ruta a cierto destino, almacena la ruta en una memoria caché, para que el proceso de descubrimiento no tenga que ejecutarse la siguiente vez. Aunque este enfoque limita en gran medida el impacto de la explosión de marcos, impone un poco de carga administrativa a todos los *hosts*, y definitivamente el algoritmo completo no es transparente, lo que fue una de las metas originales, como se mencionó antes.

4.4.4. Comparación de los puentes 802

El puente transparente y el puente de enrutamiento desde el origen tienen sus ventajas y desventajas. En esta sección estudiaremos algunas de las principales que se presentan en resumen en la figura 4-42 y se cubren con mayor detalle en (Soha y Perlman, 1988, y en Zhang, 1988). Sin embargo, cabe señalar que cada uno de los puntos es materia de amplia discusión.

Aspecto	Puente transparente	Puente de enrutamiento desde el origen
Orientación	Sin conexiones	Orientado a conexión
Transparencia	Completamente transparente	No transparente
Configuración	Automático	Manual
Enrutamiento	Subóptimo	Óptimo
Localización	Aprendizaje en reversa	Marcos de descubrimiento
Fallas	Manejado por los puentes	Manejado por los *hosts*
Complejidad	En los puentes	En los *hosts*

Figura 4-42. Comparación del puente transparente y el puente de enrutamiento desde el origen.

En el corazón de las diferencias entre los dos tipos de puente está la distinción entre trabajo en red sin conexiones y orientado a conexiones. Los puentes transparentes no tienen el concepto de circuito virtual y enrutan cada marco de manera independiente de los demás. Los puentes de enrutamiento desde el origen, en cambio, determinan una ruta mediante marcos de descubrimiento y usan después esa ruta.

Los puentes transparentes son completamente invisibles para los *hosts* y son totalmente compatibles con todos los productos 802. Los puentes de enrutamiento desde el origen no son transparentes ni compatibles. Para usar enrutamiento desde el origen, los *hosts* deben estar perfectamente conscientes del esquema de puentes y deben participar activamente en él. La división de una LAN existente en dos LAN conectadas mediante un puente de enrutamiento desde el origen requiere cambios en el *software* del *host*.

Al usar puentes transparentes, no hay necesidad de administración de la red. Los puentes mismos se configuran para la topología de manera automática. Con los puentes de enrutamiento desde el origen, el administrador de la red debe instalar manualmente y los números de LAN de puente. Los errores, como la duplicación de un número de LAN o de puente, pueden ser muy difíciles de detectar y pueden causar que algunos marcos entren en un ciclo, pero no otros en rutas distintas. Además, al conectar mediante puentes transparentes dos interredes que antes no tenían conexión, no hay nada que hacer excepto conectarlas. En cambio, con el enrutamiento desde el origen, puede ser necesario cambiar manualmente muchos números de LAN para hacerlos únicos en la interred combinada.

Una de las pocas ventajas del enrutamiento desde el origen es que, en teoría, puede usar un enrutamiento óptimo, mientras que los puentes transparentes están restringidos al árbol de extensión. Además, el enrutamiento desde el origen también puede aprovechar los puentes paralelos entre dos LAN para dividir la carga. Que los puentes reales sean lo bastante listos como para hacer uso de estas ventajas teóricas es cuestionable.

En el puente transparente, la localización de destinos se hace mediante aprendizaje de lo sucedido, y en los puentes de enrutamiento desde el origen, mediante marcos de descubrimiento. La desventaja del aprendizaje de lo sucedido es que los puentes tienen que esperar hasta que llega un marco de una máquina en particular para saber dónde está esa máquina. La desventaja de los marcos de descubrimiento es la explosión exponencial en interredes de moderadas a grandes con puentes paralelos.

El manejo de fallas es bastante diferente en ambos esquemas. Los puentes transparentes se enteran con rapidez y automáticamente de las fallas de puentes y de LAN, así como de otros cambios de topología, con sólo escuchar los marcos de control de los demás. Los *hosts* no notan estos cambios en lo absoluto.

Con el enrutamiento desde el origen, la situación es bastante distinta. Al fallar un puente, las máquinas que enrutan a través de él notan inicialmente que sus marcos ya no están siendo reconocidos, por lo que terminan su temporización e intentan una y otra vez. Por fin concluyen que algo está mal, pero aún no saben si el problema es con el destino mismo o con la ruta actual. Sólo enviando otro marco de descubrimiento pueden saber si el destino está disponible. Desafortunadamente, al fallar un puente principal, muchos *hosts* experimentarán terminaciones de temporización (*time outs*) y enviarán nuevos marcos de descubrimiento antes de que se resuelva el problema, aun si hay una ruta alterna disponible. Esta mayor vulnerabilidad a las fallas es una de las debilidades principales de todos los sistemas orientados a conexiones.

Por último, llegamos a la complejidad y el costo, temas que provocan gran controversia. Si los puentes de enrutamiento desde el origen tienen un *chip* VLSI que lee sólo aquellos marcos que deben reenviarse, estos puentes experimentarán una carga de procesamiento de marcos menor y tendrán un desempeño mejor por una inversión dada en *hardware*. Sin este *chip*, el desempeño será peor, pues la cantidad de procesamiento por marco (la búsqueda de la ruta en la cabecera del marco) es sustancialmente mayor.

Además, el enrutamiento desde el origen agrega complejidad a los *hosts*: éstos deben almacenar las rutas, enviar marcos de descubrimiento y copiar información de ruta en cada marco. Todo esto requiere memoria y ciclos de CPU. Dado que típicamente los *hosts* superan en número a los puentes en uno a dos órdenes de magnitud, puede ser mejor agregar el costo y la complejidad extra a unos cuantos puentes, en lugar de a todos los *hosts*.

4.4.5. Puentes remotos

Un uso común de los puentes es la conexión de dos (o más) LAN distantes. Por ejemplo, una compañía puede tener plantas en varias ciudades, cada una con su propia LAN. Idealmente, todas las LAN deben estar interconectadas, por lo que el sistema completo actúa como una LAN grande.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA

Esta meta puede lograrse poniendo un puente en cada LAN y conectando los puentes en pares mediante líneas punto a punto (por ejemplo, líneas arrendadas a una compañía telefónica). En la figura 4-43 se ilustra un sistema sencillo con tres LAN. Aquí se aplican los algoritmos usuales de enrutamiento. La manera más sencilla de ver esto es considerar las tres líneas punto a punto como LAN sin *hosts*. Entonces tenemos un sistema normal de seis LAN interconectadas mediante cuatro puentes. Nada de lo que hemos estudiado hasta ahora indica que una LAN debe tener *hosts*.

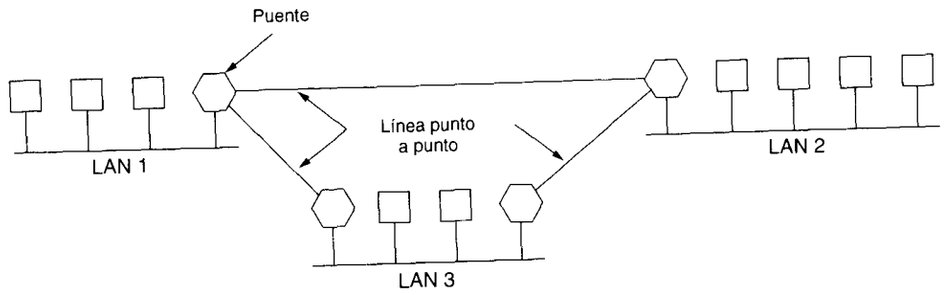


Figura 4-43. Se pueden usar puentes remotos para interconectar LAN distantes.

Se pueden usar varios protocolos en las líneas punto a punto. Una posibilidad es escoger algún protocolo de enlace de datos punto a punto estándar, poniendo marcos de MAC completos en el campo de carga útil. Esta estrategia funciona mejor si todas las LAN son idénticas, y el único problema es lograr que lleguen los marcos a la LAN correcta. Otra opción es eliminar la cabecera y el apéndice de MAC en el puente de origen y poner lo que queda en el campo de carga útil del protocolo punto a punto. En el puente de destino se pueden generar entonces una nueva cabecera y un nuevo apéndice de MAC. Una desventaja de este enfoque es que la suma de comprobación que llega al anfitrión de destino no es la calculada por el *host* de origen, por lo que podrían no detectarse los errores causados por bits erróneos en la memoria de un puente.

4.5. LAN DE ALTA VELOCIDAD

Todas las LAN y MAN 802 que acabamos de estudiar se basan en un alambre de cobre (dos alambres para el 802.6). Para velocidades bajas y distancias cortas esto es suficiente, pero para altas velocidades y mayores distancias las LAN deben basarse en fibra óptica o en redes de cobre altamente paralelas. La fibra tiene un gran ancho de banda, es delgada y ligera, no es afectada por la interferencia electromagnética de maquinaria pesada (lo que es importante cuando el cableado corre por cubos de elevador), descargas eléctricas ni relámpagos, y tiene una seguridad excelente porque es casi imposible hacer una derivación indetectable. En consecuencia, las LAN rápidas por lo regular usan fibra. En las siguientes secciones veremos algunas redes de área local que usan fibra óptica, así como una LAN de velocidad extremadamente alta que usa el tradicional alambre de cobre (aunque en grandes cantidades).

4.5.1. FDDI

La **FDDI** (*Fiber Distributed Data Interface*, **interfaz de datos distribuidos por fibra**) es una LAN *token ring* de fibra óptica de alto desempeño que opera a 100 Mbps y distancias de hasta 200 km con hasta 1000 estaciones conectadas (Black, 1994; Jain, 1994; Mirchandani y Khanna, 1993; Ross y Hamstra, 1993; Shah y Ramakrishnan, 1994, y Wolter, 1990). Esta red puede usarse de la misma manera que cualquiera de las LAN 802 pero, con su gran ancho de banda, otro uso común es como *backbone* para conectar varias LAN de cobre, como se muestra en la figura 4-44. La FDDI-II es la sucesora de la FDDI, modificada para manejar datos PCM sincrónicos de circuitos conmutados para voz o tráfico ISDN, además de datos normales. Nos referiremos a ambas simplemente como FDDI. Esta sección abarca tanto la capa física como la subcapa MAC de FDDI.

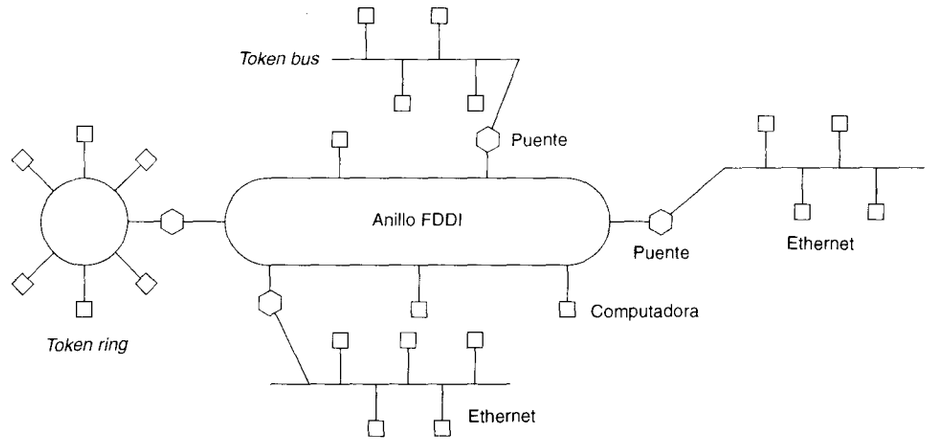


Figura 4-44. Anillo FDDI utilizado como *backbone* para conectar LAN y computadoras.

La FDDI usa fibras multimodo, pues el gasto adicional de las fibras monomodo no es necesario para redes que operan sólo a 100 Mbps; también usa LED (*Light-Emitting Diodes*, **diodos emisores de luz**), en lugar de láser, no sólo debido a su menor costo, sino también porque la FDDI a veces puede conectarse directamente a estaciones de trabajo de usuario. Hay el peligro de que usuarios curiosos ocasionalmente desenchufen el conector de la fibra y miren dentro de ella para ver el paso de los bits a 100 Mbps. Con un láser, el usuario curioso podría terminar con un agujero en su retina. Los LED son demasiado débiles para producir daño ocular, pero lo suficientemente intensos para transferir datos correctamente a 100 Mbps. La especificación de diseño de la FDDI pide no más de un error por cada 2.5×10^{10} bits. Muchas implementaciones se comportan aún mejor.

El cableado de la FDDI consiste en dos anillos de fibra, uno que transmite en dirección de las manecillas del reloj y otro en dirección contraria, como se muestra en la figura 4-45(a). Si se rompe cualquiera de ellos, se puede usar el otro como respaldo. Si ambos se rompen en el mismo

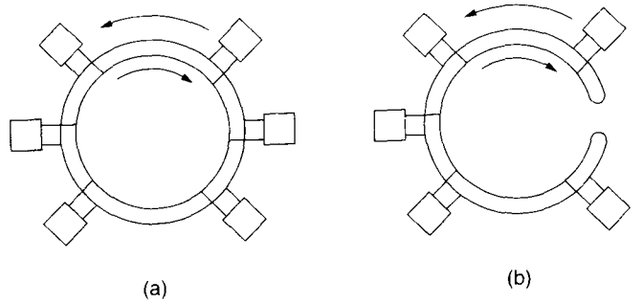


Figura 4-45. (a) La FDDI consiste en dos anillos que giran en direcciones opuestas. (b) En caso de fallar ambos anillos en un punto, pueden unirse los dos anillos para formar un solo anillo grande.

punto, por ejemplo debido a un incendio u otro accidente en el conducto del cable, pueden unirse los dos anillos en uno solo de aproximadamente el doble de longitud, como se muestra en la figura 4-45(b). Cada estación contiene relevadores (*relays*) que pueden servir para unir los dos anillos o para saltar (*bypass*) la estación en caso de problemas con ella. También pueden usarse centros de alambrado, como en el 802.5.

La FDDI define dos clases de estaciones, *A* y *B*. Las estaciones clase *A* se conectan a ambos anillos. Las estaciones clase *B*, más económicas, sólo se conectan a uno de los anillos. Dependiendo de la importancia que tenga la tolerancia de fallas, una instalación puede escoger estaciones clase *A* o clase *B*, o ambas.

La capa física no usa codificación Manchester porque la codificación Manchester a 100 Mbps requiere 200 megabaud, lo que se consideró demasiado costoso. En cambio, se usa un esquema llamado **4 de 5**. Cada grupo de 4 símbolos MAC (0, 1 y ciertos símbolos que no son de datos, como un inicio de marco) se codifica en el medio como grupo de 5 bits. Dieciséis de las 32 combinaciones son para datos, 3 son para delimitadores, 2 son de control, 3 para señalamiento al *hardware* y 8 no se usan (es decir, se reservan para versiones futuras del protocolo).

La ventaja de este esquema es que ahorra ancho de banda, pero la desventaja es la pérdida de la propiedad de autosincronización de la codificación Manchester. Para compensar esta pérdida, se usa un preámbulo grande para sincronizar el reloj del receptor con el reloj del transmisor. Además, se requiere que todos los relojes tengan una estabilidad de cuando menos 0.005%. Con esta estabilidad, se pueden enviar marcos de hasta 4500 bytes sin peligro de que el reloj del receptor se desincronice demasiado respecto a la corriente de datos.

Los protocolos FDDI básicos se basan en gran medida en los protocolos 802.5. Para transmitir datos, una estación debe primero capturar la ficha; luego transmite un marco y lo retira cuando regresa nuevamente. Una diferencia entre la FDDI y el 802.5 es que, en el 802.5, una estación no puede generar una ficha nueva hasta que su marco ha dado la vuelta completa y ha regresado. En la FDDI, con 1000 estaciones potenciales y 200 km de fibra, puede ser sustancial el tiempo perdido esperando que el marco recorra todo el anillo. Por esta razón, se decidió permitir que una estación ponga nuevamente una ficha en el anillo tan pronto ha terminado de transmitir sus marcos. En un anillo grande puede haber varios marcos a la vez.

Los marcos de datos FDDI son parecidos a los 802.5. El formato FDDI se muestra en la figura 4-46. Los campos de *delimitador inicial* y *delimitador final* marcan los límites del marco. El campo de *control de marco* indica el tipo de marco de que se trata (datos, control, etc.). El byte de *estado de marco* contiene bits de reconocimiento, parecidos a los del 802.5. Los otros campos son análogos al 802.5.

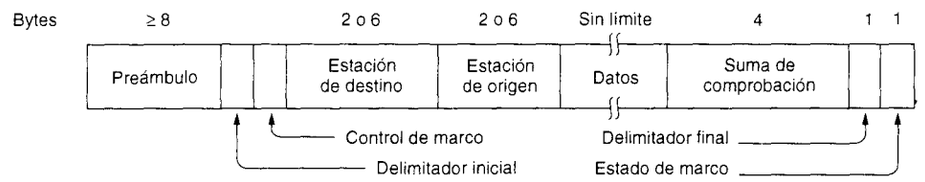


Figura 4-46. Formato de marco FDDI.

Además de los marcos normales (asincrónicos), la FDDI también permite marcos sincrónicos especiales para datos PCM de circuitos conmutados o ISDN. Los marcos sincrónicos se generan cada 125 μseg en una estación maestra para proporcionar las 8000 muestras/seg requeridas por los sistemas PCM. Cada uno de estos marcos tiene una cabecera, 16 bytes de datos conmutados fuera del circuito, y hasta 96 bytes de datos no de circuitos conmutados (es decir, hasta 96 canales PCM por marco).

Se escogió el número 96 porque permite que cuatro canales T1 (4 × 24) a 1.544 Mbps o tres canales CCITT E1 (3 × 32) a 2.048 Mbps quepan en un marco, haciéndolo adecuado para usarse en cualquier parte del mundo. Un marco sincrónico cada 125 μseg consume 6.144 Mbps de ancho de banda para los 96 canales de circuitos conmutados. Un máximo de 16 marcos sincrónicos cada 125 μseg permite hasta 1536 canales PCM y consume 98.3 Mbps.

Una vez que una estación ha adquirido una o más ranuras de tiempo en un marco sincrónico, se reservan esas ranuras para ella hasta que se liberan explícitamente. El ancho de banda total no usado por los marcos sincrónicos se reparte por demanda. En cada uno de estos marcos hay una máscara de bits para indicar qué ranuras están disponibles para la asignación por demanda. El tráfico asincrónico se divide en clases de prioridades, teniendo preferencia las prioridades mayores para recibir el ancho de banda sobrante.

El protocolo MAC de la FDDI usa tres temporizadores. El **temporizador de retención de la ficha** determina el tiempo que una estación puede continuar transmitiendo una vez que ha adquirido la ficha. Este temporizador evita que una estación acapare permanentemente el anillo. El **temporizador de rotación de la ficha** se reinicia cada vez que se ve la ficha. Si termina el temporizador, significa que no se ha visto la ficha durante una ranura demasiado grande; probablemente se ha perdido, por lo que inicia el procedimiento de recuperación de la ficha. Por último, el **temporizador de transmisión válida** se usa para terminar la temporización y recuperarse de ciertos errores transitorios del anillo.

La FDDI también tiene un algoritmo de prioridad semejante al de 802.4: determina las clases de prioridad que pueden transmitir durante un paso dado de la ficha. Si la ficha se adelanta

a lo programado, todas las prioridades pueden transmitir, pero si está retrasada, sólo pueden transmitir las mayores.

4.5.2. Ethernet rápido

Se suponía que la FDDI debía ser la LAN de la siguiente generación, pero realmente nunca se extendió su uso más allá del mercado de *backbone* (donde sigue desempeñándose bien). La administración de estaciones era demasiado complicada, lo que condujo a *chips* complejos y altos precios. El costo considerable de los *chips* FDDI evitó que los fabricantes de estaciones de trabajo se prestaran a hacer de la FDDI la red estándar, por lo que la producción en volumen jamás se dio y la FDDI nunca entró al mercado de masas. La lección que se debió aprender aquí fue la de mantener simples las cosas.

En cualquier caso, la incapacidad para despegar de la FDDI dejó un espacio para una LAN ordinaria con velocidades por encima de 10 Mbps. Muchas instalaciones necesitaban más ancho de banda y por tanto tenían muchas LAN de 10 Mbps conectadas mediante una maraña de repetidores, puentes, enrutadores y pasarelas, aunque para los administradores de red a veces parecía que se mantenían unidos con goma de mascar y alambre de gallinero.

Fue en este ambiente que el IEEE volvió a convocar en 1992 al comité del 802.3, con instrucciones de inventar una LAN más rápida. Una propuesta fue mantener el 802.3 exactamente como estaba, pero hacerlo más rápido. Otra propuesta fue rehacerlo por completo para darle muchas características nuevas, como tráfico en tiempo real y voz digitalizada, pero manteniendo su viejo nombre (por cuestiones de marketing). Tras algún forcejeo, el comité decidió mantener el 802.3 como estaba, pero hacerlo más veloz. La gente que apoyaba la propuesta perdedora hizo lo que haría cualquier persona en la industria de la computación en estas circunstancias: formó su propio comité y desarrolló el estándar de su propia LAN (que es el 802.12).

Las tres razones principales por las que el comité del 802.3 decidió diseñar una LAN 802.3 mejorada fueron:

1. La necesidad de compatibilidad hacia atrás con miles de LAN existentes.
2. El temor a que un protocolo nuevo tuviera problemas imprevistos.
3. El deseo de terminar el trabajo antes de que cambiara la tecnología.

El trabajo se hizo con rapidez (según las normas de los comités de estándares), y el resultado, el **802.3u**, fue aprobado oficialmente por el IEEE en junio de 1995. Técnicamente, el 802.3u no es un estándar nuevo, sino una adición al estándar 802.3 existente (para subrayar su compatibilidad hacia atrás). Ya que todo mundo lo llama **Ethernet rápido**, en lugar de 802.3u, nosotros también lo haremos.

El concepto principal en que se basa el Ethernet rápido es sencillo: mantener todos los formatos de paquete, interfaces y reglas de procedimientos anteriores, y simplemente reducir el tiempo de bit de 100 nseg a 10 nseg. Técnicamente, habría sido posible copiar el 10Base-5 o el 10Base-2 y aún detectar colisiones a tiempo con sólo reducir la longitud máxima del cable por un factor de 10. Sin embargo, las ventajas del alambrado 10Base-T fueron tan contundentes que

el Ethernet rápido se basa completamente en este diseño. Por tanto, todos los sistemas Ethernet rápido usan concertadores; no se permiten cables de derivación múltiple con derivaciones vampiro ni conectores BNC.

Sin embargo, aún tenían que tomarse algunas decisiones, siendo la más importante el tipo de cables a manejar. Un contendiente fue el par trenzado categoría 3. El argumento a su favor fue que prácticamente todas las oficinas del mundo occidental tienen cuando menos cuatro pares trenzados categoría 3 (o mejor) tendidos a un gabinete de alambrado telefónico en un alcance de 100 metros. A veces existen dos de tales cables. Por tanto, el uso de par trenzado categoría 3 posibilitaría el alambrado de computadoras de escritorio usando Ethernet rápido sin tener que realambrar completamente el edificio, una ventaja enorme para muchas organizaciones.

La desventaja principal del par trenzado categoría 3 es la incapacidad de conducir señales de 200 megabaud (100 Mbps con codificación Manchester) a 100 metros, la distancia máxima de computadora a concertador especificada para 10Base-T (véase la figura 4-17). En contraste, el par trenzado categoría 5 puede manejar fácilmente los 100 metros, y la fibra puede llegar mucho más lejos. La media escogida fue permitir las tres posibilidades, como se muestra en la figura 4-47, pero mejorar la solución de categoría 3 para darle la capacidad de conducción adicional necesaria.

Nombre	Cable	Segmento máximo	Ventajas
100Base-T4	Par trenzado	100 m	Usa UTP categoría 3
100Base-TX	Par trenzado	100 m	Dúplex integral a 100 Mbps
100Base-F	Fibra óptica	2000 m	Dúplex integral a 100 Mbps; tramos grandes

Figura 4-47. Cableado de Ethernet rápido.

El esquema UTP categoría 3, llamado **100Base-T4**, usa una velocidad de señalización de 25 MHz, sólo 25% más rápida que los 20 MHz del 802.3 estándar (recuerde que la codificación Manchester, como se indica en la figura 4-20, requiere dos periodos de reloj por cada uno de los 10 millones de bits por segundo). Para lograr el ancho de banda necesario, el 100Base-T4 requiere cuatro pares trenzados. Dado que el alambrado telefónico estándar ha tenido durante décadas cuatro pares trenzados por cable, la mayoría de las oficinas pueden manejar esto. Por supuesto, significa renunciar al teléfono de la oficina, pero eso seguramente es un precio bajo a pagar por un correo electrónico más rápido.

De los cuatro pares trenzados, uno siempre va al concertador, uno más siempre viene del concertador y los otros dos son conmutables a la dirección actual de transmisión. Para conseguir el ancho de banda necesario, no se usa codificación Manchester, pues con relojes modernos y distancias tan cortas ya no es necesario. Además, se envían señales ternarias, por lo que durante un solo periodo de reloj el alambre puede contener un 0, un 1 o un 2. Con tres pares trenzados en la dirección de transmisión y señalización ternaria, puede transmitirse cualquiera de 27 símbolos, posibilitando el envío de 4 bits con un poco de redundancia. La transmisión de 4 bits en cada

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE DOCUMENTACIÓN Y BIBLIOTECA
MONTEVIDEO - URUGUAY

uno de los 25 millones de ciclos de reloj por segundo da los 100 Mbps necesarios. Además, siempre hay un canal en reversa de 33.3 Mbps que utiliza el par trenzado restante. Este esquema, conocido como **8B6T** (mapa de 8 bits a 6 trits), seguramente no ganará premios por su elegancia, pero funciona con la planta de alambrado existente.

Para el alambrado de categoría 5, el diseño, **100Base-TX**, es más sencillo, pues los alambres pueden manejar tasas de reloj de hasta 125 MHz o más. Sólo se usan dos pares trenzados por estación, uno al concertador y uno de él. En lugar de sólo usar codificación binaria directa, se usa un esquema llamado **4B5B** a 125 MHz. Cada grupo de cinco periodos de reloj se usa para enviar 4 bits a fin de tener cierta redundancia, proporcionar suficientes transiciones para permitir una fácil sincronización de los relojes, crear patrones únicos para delimitar marcos, y ser compatible con la FDDI en la capa física. En consecuencia, el 100Base-TX es un sistema dúplex integral; las estaciones pueden transmitir a 100 Mbps y recibir a 100 Mbps al mismo tiempo. Además, se pueden tener dos líneas telefónicas para comunicación real en caso de que la computadora esté completamente ocupada navegando la *Web*.

La última opción, **100Base-FX**, usa dos hilos de fibra multimodo, uno para cada dirección, por lo que también es dúplex integral con 100 Mbps en cada dirección. Además, la distancia entre una estación y el concertador puede ser de hasta 2 km.

Son posibles dos tipos de concertadores con 100Base-T4 y 100Base-TX, conocidos en conjunto como **100Base-T**. En un concertador compartido, todas las líneas de entrada (o cuando menos todas las líneas que llegan a una tarjeta) están conectadas lógicamente, formando un solo dominio de colisión. Se aplican todas las reglas estándar, incluido el algoritmo de retroceso binario, por lo que el sistema funciona igual que el 802.3 tradicional. En particular, sólo una estación puede transmitir a la vez.

En un concertador conmutado, cada marco de entrada se coloca en el *buffer* de una tarjeta de línea (*plug-in line card*). Aunque esta característica hace que el concertador y las tarjetas sean más caros, también significa que todas las estaciones pueden transmitir (y recibir) al mismo tiempo, mejorando de manera importante el ancho de banda total del sistema, con frecuencia en un orden de magnitud o más. Los marcos en *buffer* se pasan a través de un canal de alta velocidad de la tarjeta de origen a la tarjeta de destino. El canal de alta velocidad en plano posterior no está estandarizado, ni necesita estarlo, ya que está completamente escondido dentro del conmutador. Si la experiencia pasada sirve de guía, los fabricantes de conmutadores competirán fuertemente para producir canales de alta velocidad a plano posterior cada vez más rápidos a fin de mejorar el rendimiento del sistema. Dado que los cables 100Base-FX son demasiado largos para el algoritmo normal de colisiones de Ethernet, deben conectarse a concertadores conmutados con *buffers*, de modo que cada uno sea un dominio de colisión por sí mismo.

Como nota final, prácticamente todos los conmutadores pueden manejar una mezcla de estaciones de 10 Mbps y 100 Mbps, para hacer más fácil la modernización. A medida que una instalación consigue más estaciones de 100 Mbps, todo lo que tiene que hacer es comprar la cantidad necesaria de tarjetas de línea nuevas e introducirlas en el conmutador.

En (Johnson, 1996) puede encontrarse mayor información sobre el Ethernet rápido. Para una comparación de las redes de área local de alta velocidad, en particular FDDI, Ethernet rápido, ATM y VG-AnyLAN, véase (Cronin *et al.*, 1994).

4.5.3. HIPPI — Interfaz paralela de alto desempeño

Durante la guerra fría, el Laboratorio Nacional de Los Álamos, el centro de diseño de armas nucleares del gobierno de Estados Unidos, compró rutinariamente cada uno de los modelos de supercomputadora en venta. Los Álamos también acumuló periféricos complicados, como dispositivos de almacenamiento masivo y estaciones de trabajo gráficas especiales para visualización científica. En esos años, cada fabricante tenía una interfaz distinta para conectar periféricos a su supercomputadora, por lo que no era posible compartir periféricos entre máquinas o interconectar dos supercomputadoras.

En 1987, los investigadores de Los Álamos comenzaron a trabajar con una interfaz éstándar de supercomputadora, con la intención de conseguir su estandarización y luego convencer a todos los fabricantes de que la usaran. (Dado el tamaño del presupuesto de computación de Los Álamos, cuando ellos hablaban, los fabricantes escuchaban.) La meta era una interfaz que cualquiera pudiera implementar rápida y eficientemente. El principio rector fue la sencillez. La interfaz no debía tener opciones ni requerir el diseño de *chips* nuevos, y debía tener el desempeño de una manguera contra incendios.

La especificación inicial indicaba una tasa de datos de 800 Mbps, porque las películas de bombas estallando requerían marcos de 1024 × 1024 pixeles, con 24 bits por pixel y 30 marcos/seg, para una tasa de datos total de 750 Mbps. Después, se coló una opción: una segunda tasa de datos de 1600 Mbps. Cuando esta propuesta, llamada **HIPPI** (*HIgh Performance Parallel Interface*, **interfaz paralela de alto desempeño**) se ofreció al ANSI para su estandarización, los proponentes fueron considerados lunáticos, ya que las LAN en los años 80 implicaban Ethernets de 10 Mbps.

La HIPPI se diseñó originalmente como canal de datos, no como LAN. Los canales de datos operan punto a punto, de un maestro (una computadora) a un subordinado (un periférico), con alambres dedicados y sin conmutación. No hay contención y el entorno es completamente predecible. Después, se hizo evidente la necesidad de conmutar un periférico de una supercomputadora a otra, y se añadió un conmutador de barras cruzadas al diseño de la HIPPI, como se ilustra en la figura 4-48.

A fin de lograr un desempeño de tal nivel usando sólo *chips* comunes, la interfaz básica se hizo de 50 bits de ancho, 32 bits de datos y 18 bits de control, por lo que el cable HIPPI contiene 50 pares trenzados. Cada 40 nseg se transfiere una palabra en paralelo a través de la interfaz. Para lograr 1600 Mbps, se usan dos cables y se transfieren dos palabras por ciclo. Todas las transferencias son símplex. Para lograr comunicación de dos vías, se requieren dos (o cuatro) cables. A estas velocidades, la longitud máxima del cable es de 25 metros.

Tras recuperarse de su pasmo inicial, el comité ANSI X3T9.3 generó un estándar HIPPI basado en la información de Los Álamos. El estándar cubre las capas física y de enlace de datos. Todo lo que esté por arriba de ellas es responsabilidad de los usuarios. El protocolo básico es que, para comunicarse, un *host* primero pide al conmutador de barras cruzadas que establezca una conexión. Entonces (generalmente) envía un solo mensaje y libera la conexión.

Los mensajes se estructuran con una palabra de control, una cabecera de hasta 1016 bytes y una parte de datos de hasta $2^{32} - 2$ bytes. Por razones de control de flujo, los mensajes se dividen

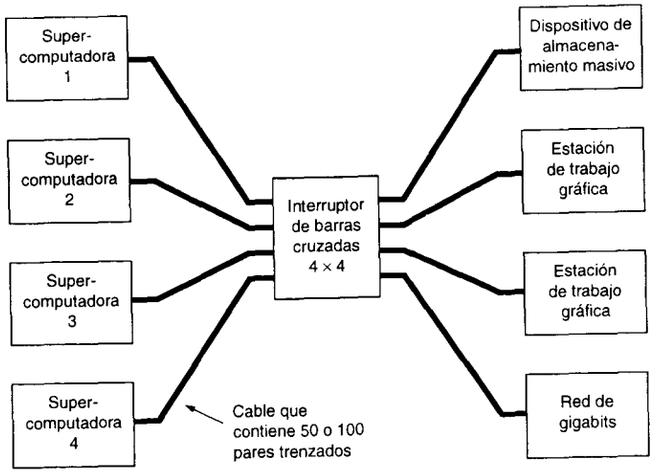


Figura 4-48. HIPPI que usa un interruptor de barras cruzadas.

en marcos de 256 palabras. Cuando el receptor es capaz de aceptar un marco, envía una señal al transmisor, quien entonces envía el marco. Los receptores también pueden pedir varios marcos a la vez. El control de errores consiste en un bit de paridad horizontal por palabra y una palabra de paridad vertical al final de cada marco. Las sumas de comprobación tradicionales se consideraron innecesarias y demasiado lentas.

La HIPPI fue implementada rápidamente por docenas de fabricantes y ha sido el estándar de interconexión de supercomputadoras durante años. Para mayor información sobre ella, véase (Hughes y Franta, 1994; Tolmie, 1992, y Tolmie y Renwick, 1993).

4.5.4. Canal de fibra

En la época en que se diseñó la HIPPI, la fibra óptica era demasiado cara y no se consideraba lo suficientemente confiable, por lo que la LAN más rápida construida hasta el momento se hizo con alambre telefónico de baja clasificación. Con el paso del tiempo, la fibra se volvió más barata y confiable, por lo que fue natural que tarde o temprano hubiera un intento por rehacer la HIPPI usando una sola fibra en lugar de 50 o 100 pares trenzados. Desafortunadamente, la disciplina que había establecido Los Álamos de parar las propuestas de características nuevas cada vez que aparecía una, se perdió en el camino. El sucesor de la HIPPI, llamado **canal de fibra** (*fibre channel*), es mucho más complicado y de implementación más cara. Queda por ver si disfrutará del éxito comercial de la HIPPI.

El canal de fibra maneja tanto conexiones de canal de datos como de red. En particular, puede usarse para conducir canales de datos que incluyen HIPPI, SCSI y el canal multiplexor empleado en las *mainframe* IBM. También puede conducir paquetes de red, incluidos IEEE 802, IP y ATM. Como en HIPPI, la estructura básica de los canales de fibra es un interruptor de barras

cruzadas que conecta las entradas con las salidas. Las conexiones pueden establecerse durante un solo paquete o por una ranura mucho mayor.

El canal de fibra apoya tres clases de servicio. La primera es la conmutación de circuitos pura, con entrega en orden garantizada. Los modos del canal de datos usan esta clase de servicio. La segunda clase es la conmutación de paquetes con entrega garantizada. La tercera clase es la conmutación de paquetes sin entrega garantizada.

El canal de fibra tiene una estructura de protocolos complicada, como se muestra en la figura 4-49. Aquí vemos cinco capas, que en conjunto cubren las capas física y de enlace de datos. La capa inferior tiene que ver con el medio físico. Hasta ahora, maneja tasas de datos de 100, 200, 400 y 800 Mbps. La segunda capa maneja la codificación de bits. El sistema empleado se parece un poco al de la FDDI, pero en lugar de usar 5 bits para codificar 16 símbolos válidos usa 10 bits para codificar 256 símbolos válidos, proporcionando algo de redundancia. Además, estas dos capas son equivalentes funcionalmente a la capa física de la OSI.

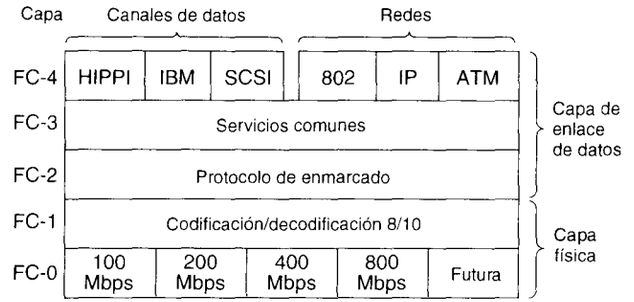


Figura 4-49. Capas de protocolo del canal de fibra.

La capa media define el formato de la distribución del marco y de la cabecera. Los datos se transmiten en marcos con cargas útiles de hasta 2048 bytes. La siguiente capa permitirá proporcionar servicios comunes a la capa superior posteriormente, según se requiera. Por último, la capa superior proporciona las interfaces con los distintos tipos de computadoras y periféricos reconocidos.

Como comentario al margen, aunque el canal de fibra fue diseñado en Estados Unidos, se escogió la pronunciación inglesa (*fibre channel*) en lugar de la estadounidense (*fiber channel*), porque el editor del estándar era británico. Se puede encontrar información adicional sobre el canal de fibra en (Tolmie, 1992). En (Tolmie, 1995) hay una comparación con la HIPPI y ATM.

4.6. REDES SATELITALES

Aunque la mayoría de los canales de acceso múltiple se encuentran en las LAN, hay un tipo de WAN que usa canales de acceso múltiple: las WAN basadas en comunicación satelital. En las siguientes secciones estudiaremos brevemente algunos de los problemas que se presentan en las

redes de área amplia basadas en satélite. También veremos algunos de los protocolos que se han diseñado para manejarlas.

Los satélites de comunicación por lo general tienen hasta una docena o más de transpondedores. Cada transpondedor tiene un haz que cubre una parte de la Tierra debajo de él, variando de un haz amplio de 10,000 km de diámetro hasta un haz localizado de sólo 250 km de diámetro. Las estaciones dentro del área del haz pueden enviar marcos al satélite en la frecuencia de enlace ascendente. El satélite entonces vuelve a difundirlos por la frecuencia de enlace descendente. Se usan diferentes frecuencias para el enlace ascendente y el descendente a fin de evitar que el transpondedor entre en oscilación. Los satélites sin procesamiento "a bordo", sino que simplemente repiten lo que escuchan (la mayoría de ellos), con frecuencia se llaman satélites de **codo**.

Cada antena puede enfocarse en un área, transmitir algunos marcos, y luego enfocarse en un área nueva. El enfoque es electrónico, pero aun así tarda algunos microsegundos. El tiempo durante el cual se apunta un haz sobre un área dada se llama el **tiempo de morada o permanencia** (*dwell time*). Para una máxima eficiencia, este tiempo no debe ser muy corto, porque se desperdiciará demasiado tiempo moviendo el haz. .

Al igual que en las LAN, uno de los puntos clave del diseño es la manera de repartir los canales del transpondedor. Sin embargo, a diferencia de las LAN, es imposible la detección de portadora, debido al retardo de propagación de 270 mseg. Cuando una estación detecta el estado de un canal de enlace descendente, escucha lo que ocurrió hace 270 mseg. La detección del canal de enlace ascendente generalmente es imposible. Como resultado, los protocolos CSMA/CD (que suponen que una estación transmisora puede detectar colisiones en los primeros tiempos de bit, y retraerse si está ocurriendo una) no pueden usarse con los satélites. De ahí la necesidad de otros protocolos.

Se emplean cinco clases de protocolos en el canal de acceso múltiple (de enlace ascendente): sondeo, ALOHA, FDM, TDM y CDMA. Aunque los hemos estudiado a todos, la operación satelital a veces agrega nuevas complicaciones. El problema principal es con el canal de enlace ascendente, ya que el de enlace descendente sólo tiene un transmisor (el satélite) y por tanto no tiene el problema de reparto del canal.

4.6.1. Sondeo

La forma tradicional de repartir un solo canal entre usuarios competidores es que alguien los sondee. Hacer que el satélite sondee por turno cada estación para ver si tiene un marco es prohibitivamente caro, dado el tiempo de 270 mseg requerido para cada secuencia de sondeo/respuesta.

Sin embargo, si todas las estaciones de tierra también están conectadas a una red de conmutación de paquetes (típicamente de poco ancho de banda), es concebible una variación menor de este concepto. La idea es disponer todas las estaciones en un anillo lógico, de modo que cada estación conozca su sucesor. Por este anillo terrestre circula una ficha. El satélite nunca ve la ficha. Sólo se permite a una estación transmitir por el enlace ascendente cuando ha capturado la ficha. Si el número de estaciones es pequeño y constante, el tiempo de transmisión de la ficha es corto y las ráfagas enviadas por el canal de enlace ascendente son mucho más grandes que el tiempo de rotación de la ficha, el esquema es moderadamente eficiente.

4.6.2. ALOHA

El ALOHA puro es fácil de implementar: cada estación simplemente envía cuando quiere. El problema es que la eficiencia del canal es de sólo 18%. En general, un factor de uso tan bajo es inaceptable para satélites que cuestan decenas de millones de dólares cada uno.

El uso del ALOHA ranurado duplica la eficiencia, pero agrega el problema de cómo sincronizar todas las estaciones para que todas sepan en qué momento comienza cada intervalo. Afortunadamente el satélite mismo tiene la respuesta, ya que inherentemente es un medio de difusión. Una estación de tierra, la **estación de referencia**, transmite periódicamente una señal especial cuya retransmisión es usada por todas las estaciones de tierra como origen del tiempo. Si todas las ranuras de tiempo tienen una longitud de ΔT, cada estación sabe ahora que la ranura de tiempo k comienza en el momento $k\Delta T$ después del origen del tiempo. Puesto que los relojes operan con tasas ligeramente diferentes, es necesaria una resincronización periódica para mantener a todos en fase. Una complicación adicional es que el tiempo de propagación desde el satélite es diferente para cada estación de tierra, pero este efecto puede compensarse.

Para aumentar el aprovechamiento del canal de enlace ascendente por encima de $1/e$, podemos pasar del canal de enlace ascendente individual de la figura 4-50(a) al esquema de enlace ascendente doble de la figura 4-50(b). Una estación con un marco por transmitir escoge uno de los canales de enlace ascendente al azar y envía el marco en la siguiente ranura. Cada enlace ascendente opera entonces un canal ALOHA ranurado independiente.

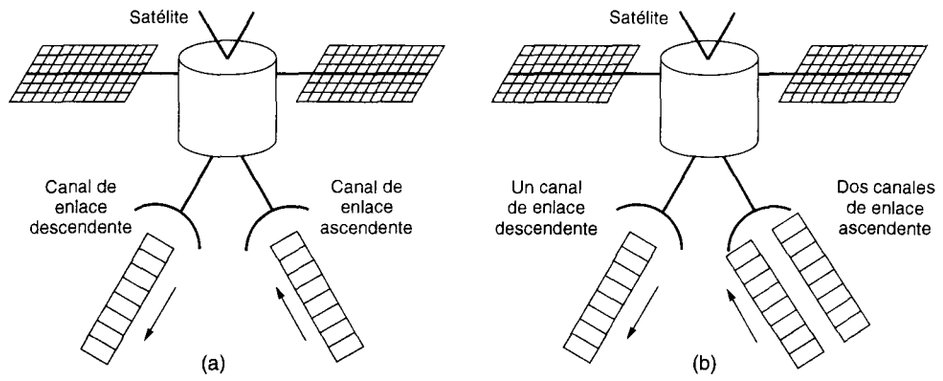


Figura 4-50. (a) Sistema ALOHA estándar. (b) Adición de un segundo canal de enlace ascendente.

Si uno de los canales de enlace ascendente contiene un solo marco, simplemente se transmite después en la ranura de enlace descendente. Si ambos canales tienen éxito, el satélite puede poner en *buffer* uno de los marcos y transmitirlo durante una ranura inactiva posterior. Calculando las probabilidades, puede demostrarse que, dada una cantidad infinita de espacio de *buffer*, puede elevarse la utilización del canal de enlace descendente hasta 0.736, al costo de aumentar los requisitos de ancho de banda en 50%.

4.6.3. FDM

La multiplexión por división en frecuencia es el esquema de reparto de canal más viejo y probablemente el más utilizado aún. Un transpondedor típico de 36 Mbps puede dividirse estáticamente en unos 500 canales PCM de 64,000 bps, operando cada uno en su propia frecuencia para evitar la interferencia con los demás.

Aunque es simple, la FDM también tiene algunos inconvenientes. Primero, se requieren bandas de seguridad (laterales) entre los canales para mantener separadas las estaciones. Este requisito existe porque no es posible construir transmisores que envíen toda su energía por la banda principal y nada por las bandas laterales. La cantidad de ancho de banda desperdiciado en las bandas laterales puede ser una parte sustancial del total.

Segundo, debe controlarse cuidadosamente la potencia de las estaciones. Si una estación pone demasiada potencia en la banda principal, también pondrá automáticamente demasiada potencia en las bandas laterales, ocurriendo un derramamiento a los canales adyacentes que causa interferencia. Por último, la FDM es una técnica completamente analógica que no se presta bien a una implementación en *software*.

Si el número de estaciones es pequeño y fijo, pueden repartirse por adelantado los canales de frecuencia. Sin embargo, si el número de estaciones, o la carga de cada una de ellas, puede fluctuar con rapidez, se necesita algún tipo de reparto dinámico de las bandas de frecuencia. Uno de tales mecanismos es el sistema **SPADE** empleado en algunos de los primeros satélites Intelsat. Cada repetidor SPADE se dividió en 794 canales de voz PCM símplex (de 64 kbps), junto con un canal de señalización común de 128 kbps. Los canales PCM se usaban en pares para proporcionar servicio dúplex integral. El ancho de banda de transpondedor total usado fue de 50 Mbps para la parte de enlace ascendente y otros 50 Mbps para el enlace descendente.

El canal de señalización común se dividió en unidades de 50 mseg. Una unidad contenía 50 ranuras de 1 mseg (128 bits). Cada ranura "pertenecía" a una de 50 estaciones de tierra (no más). Cuando una estación de tierra tenía datos por enviar, tomaba al azar un canal desocupado en ese momento y escribía el número del canal en su siguiente ranura de 128 bits. Si el canal seleccionado aún estaba desocupado cuando se veía la solicitud en el enlace descendente, el canal se consideraba como repartido y las demás estaciones se abstenían de intentar adquirirlo. Si dos o más estaciones trataban de reservar el mismo canal durante el mismo marco, ocurría una colisión y tenían que intentarlo de nuevo posteriormente. Al terminar de usar su canal, la estación enviaba un mensaje de liberación en su ranura del canal común.

4.6.4. TDM

Al igual que la FDM, la TDM es bien entendida y se usa ampliamente en la práctica; requiere sincronización de tiempo para las ranuras, pero esto puede proporcionarse mediante una estación de referencia, como se describió antes para el ALOHA ranurado. De modo parecido a la FDM, para un número pequeño e invariable de estaciones, la asignación de ranuras puede establecerse por adelantado y no cambiarse nunca, pero con un número variable de estaciones, o

un número fijo de estaciones con cargas variables en el tiempo, las ranuras de tiempo deben asignarse dinámicamente.

La asignación de ranuras puede hacerse de manera centralizada o descentralizada. Como ejemplo de asignación de ranuras centralizada, consideremos el **ACTS** (*Advanced Communication Technology Satellite*, **satélite de tecnología de comunicaciones avanzada**), que se diseñó para unas pocas docenas de estaciones (Palmer y White, 1990). El ACTS se lanzó en 1992 y tiene cuatro canales TDM independientes de 110 Mbps, dos de enlace ascendente y dos de enlace descendente. Cada canal está organizado como secuencia de marcos de 1 mseg, conteniendo cada marco 1728 ranuras de tiempo. Cada ranura tiene una carga útil de 64 bits, permitiendo a cada una contener un canal de voz de 64 kbps.

Los haces pueden conmutarse de un área geográfica a otra pero, dado que el movimiento del haz se lleva varios tiempos de ranura, los canales que se originan o terminan en la misma zona geográfica normalmente son asignados a ranuras contiguas para aumentar el tiempo de morada o permanencia y reducir al mínimo el tiempo perdido en el movimiento del haz. Por tanto, la administración de las ranuras de tiempo requiere un conocimiento profundo de la geografía de las estaciones para minimizar la cantidad de ranuras de tiempo desperdiciadas. Por ésta y otras razones, la administración de ranuras de tiempo la efectúa una de las estaciones de tierra, la **MCS** (*Master Control Station*, **estación de control maestro**).

La operación básica del ACTS es un proceso continuo de tres pasos, tardándose cada paso 1 mseg. En el paso 1, el satélite recibe un marco y lo almacena en una RAM interna de 1728 entradas. En el paso 2, una computadora a bordo copia cada entrada en la salida correspondiente (posiblemente para la otra antena). En el paso 3, el marco de salida se transmite por el enlace descendente.

Inicialmente, a cada estación se asigna cuando menos una ranura de tiempo. Si desea adquirir canales adicionales (para nuevas llamadas de voz), una estación envía una solicitud corta a la MCS. De la misma manera, la estación puede liberar un canal existente con un mensaje a la MCS. Estos mensajes ocupan una cantidad pequeña de bits de carga extra y proporcionan un canal de control especial a la MCS con una capacidad de unos 13 mensajes/seg por estación. Los canales son dedicados; no hay contención por ellos.

También es posible el reparto dinámico de ranuras TDM. A continuación analizaremos tres esquemas. En cada uno de ellos se dividen los marcos TDM en ranuras de tiempo, teniendo cada ranura un dueño (temporal). Sólo el dueño puede usar una ranura de tiempo.

El primer esquema supone que hay más ranuras que estaciones, por lo que puede asignarse una ranura inicial a cada estación (Binder, 1975). Si hay más ranuras que estaciones, las ranuras extra no se asignan a nadie. Si el dueño de una ranura no la quiere durante el grupo actual, se desperdicia. Una ranura vacía es una señal para todos los demás de que el dueño no tiene tráfico. Durante el siguiente marco, la ranura queda disponible para quien la quiera, con base en contención (ALOHA).

Si el dueño quiere recuperar "su" ranura inicial, transmite un marco, obligando a una colisión (si hay otro tráfico). Tras la colisión, todos, excepto el dueño, deben desistir de usar la ranura durante el siguiente marco. Por tanto, el dueño siempre puede comenzar a transmitir en un lapso de dos tiempos de marco, en el peor caso. Si la utilización del canal es baja, el sistema no se desempeña tan bien como el ALOHA ranurado normal ya que, después de una colisión, los

implicados deben abstenerse durante un marco para ver si el dueño quiere recuperar la ranura. En la figura 4-51(a) se muestra un marco con ocho ranuras, de las cuales siete pertenecen a *G, A, F, E, B, C* y *D*, respectivamente. La octava ranura no pertenece a nadie y cualquiera puede contender por ella.

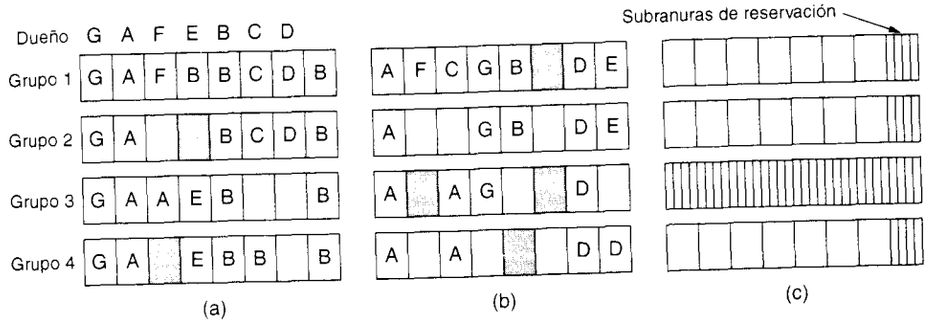


Figura 4-51. Esquemas de reservación. (a) Binder. (b) Crowther. (c) Roberts. Los marcos sombreados indican colisiones. Se muestran cuatro grupos consecutivos de ranuras para cada uno de los tres esquemas.

Es aplicable un segundo esquema aun cuando el número de estaciones es desconocido y variable (Crowther *et al.*, 1973). En este método, las ranuras no tienen dueños permanentes, como en el de Binder. En cambio, las estaciones compiten por ranuras usando ALOHA ranurado. Cuando tiene éxito una transmisión, la estación que hace la transmisión también tiene derecho a esa ranura durante el siguiente marco. Así, mientras una estación tenga datos por enviar, podrá continuar haciéndolo indefinidamente (sujeta a algunas reglas para evitar acaparamientos). En esencia, la propuesta permite una mezcla dinámica de ALOHA ranurado y TDM, variando según la demanda el número de ranuras destinadas a cada esquema. En la figura 4-51(b) también se muestra un marco con ocho ranuras. Inicialmente, *E* usa la última ranura, pero tras dos marcos ya no la necesita. La ranura permanece inactiva durante un marco y entonces *D* la toma y la conserva hasta haber terminado.

Un tercer esquema, debido a Roberts (1973), requiere que las estaciones hagan solicitudes por adelantado antes de transmitir. Cada marco contiene, digamos, una ranura especial [la última de la figura 4-51(c)], que se divide en *V* subranuras menores empleadas para hacer reservaciones. Cuando una estación quiere enviar datos, difunde un marco corto de solicitud en una subranura de reservación escogida al azar. Si la reservación tiene éxito (es decir, no hay una colisión), entonces se reserva la siguiente ranura normal (o más). Todos deben conocer la longitud de la cola (cantidad de ranuras reservadas) en todo momento, para que, al lograr una estación una reservación exitosa, sepa cuántas ranuras de datos deben pasar antes de transmitir. Las estaciones no necesitan saber *quién* está en la cola; simplemente necesitan conocer el tamaño de la cola. Cuando el tamaño de la cola cae a cero, todas las ranuras se revierten a subranuras de reservación a fin de acelerar el proceso de reservación.

Aunque la TDM se usa ampliamente, tanto con esquemas de reservación como sin ellos, tiene algunos inconvenientes. Por una parte, requiere que todas las estaciones se resincronicen en el tiempo, lo que en la práctica no es un asunto trivial, puesto que los satélites tienden a una deriva en su órbita, lo que cambia el tiempo de propagación a cada estación de tierra. También requiere que cada estación de tierra pueda alcanzar velocidades de ráfaga extremadamente altas. Por ejemplo, aun cuando una estación ACTS puede tener sólo un canal de 64 kbps, debe ser capaz de enviar una ráfaga de 64 bits en una ranura de tiempo de 578 nseg. En otras palabras, debe operar en realidad a 110 Mbps. A diferencia, una estación FDM de 64 kbps en realidad opera a 64 kbps.

4.6.5. CDMA

El esquema final es el CDMA. El CDMA evita el problema de sincronización de tiempo y también el problema de reparto del canal; es completamente descentralizado y totalmente dinámico.

Sin embargo, el CDMA tiene tres desventajas principales. Primero, la capacidad de un canal CDMA ante la presencia de ruido y de estaciones descoordinadas es típicamente menor de lo que se puede lograr con TDM. Segundo, con 128 *chips*/bit (que es un valor común), aunque la tasa de bits no sea alta, la tasa de *chips* sí lo es, necesitándose un transmisor rápido (léase caro). Tercero, pocos ingenieros entienden realmente el CDMA, lo que por lo general no aumenta la probabilidad de que lo usen, aun si es el mejor método para una aplicación en particular. No obstante, el CDMA ha sido usado por los cuerpos armados durante décadas, y ahora se está volviendo más común en aplicaciones comerciales.

4.7. RESUMEN

Algunas redes tienen un solo canal que se usa para todas las comunicaciones. En estas redes, el aspecto clave del diseño es el reparto del canal entre las estaciones competidoras que desean usarlo. Se han desarrollado muchos algoritmos de reparto de canal. En la figura 4-52 se presenta un resumen de algunos de los métodos de reparto de canal más importantes.

Los métodos de reparto más sencillos son la FDM y la TDM; son eficientes con un número de estaciones pequeño y tráfico continuo. Ambos esquemas se usan ampliamente en estas circunstancias; por ejemplo, para dividir el ancho de banda de los enlaces satelitales usados como troncales telefónicas.

Con un número grande y variable de estaciones, o con un tráfico en ráfagas, la FDM y la TDM son soluciones pobres. Se ha propuesto como alternativa el protocolo ALOHA, con y sin ranuras y control. El ALOHA y sus muchas variantes y derivaciones han sido ampliamente estudiados, analizados y usados en sistemas reales.

Cuando puede detectarse el estado del canal, las estaciones pueden evitar el comienzo de una transmisión mientras otra estación está transmitiendo. Esta técnica, la detección de portadora, ha producido varios protocolos que pueden usarse en redes LAN y MAN.

Se conoce una clase de protocolos que eliminan por completo la contención, o cuando menos la reducen considerablemente. El conteo descendente binario elimina por completo la

Método	Descripción
FDM	Dedica una banda de frecuencia a cada estación
TDM	Dedica una ranura de tiempo a cada estación
ALOHA puro	Transmisión no sincronizada en cualquier momento
ALOHA ranurado	Transmisión aleatoria en ranuras de tiempo bien definidas
CSMA persistente-1	Acceso múltiple con detección de portadora estándar
CSMA no persistente	Retardo aleatorio cuando se detecta ocupado el canal
CSMA persistente-p	CSMA, pero con una probabilidad p de persistir
CSMA/CD	CSMA, pero con aborto al detectar una colisión
Mapa de bits	Programación por turno circular usando un mapa de bits
Conteo descendente binario	La estación lista con el número más alto es la siguiente
Recorrido de árbol	Contención reducida mediante habilitación selectiva
División en longitud de onda	Esquema FDM dinámico para fibra
MACA, MACAW	Protocolos LAN inalámbricos
GSM	FDM más TDM para radio celular
CDPD	Radio en paquetes con un canal AMPS
CDMA	Todos hablan a la vez, pero en idiomas diferentes
Ethernet	CSMA/CD con retroceso binario exponencial
Token bus	Anillo lógico en un canal físico
Token ring	Captura la ficha para enviar un marco
DQDB	Encolamiento distribuido en una MAN de dos canales
FDDI	*Token ring* de fibra óptica
HIPPI	Barras cruzadas usando 50 o 100 pares trenzados
Canal de fibra	Barras cruzadas usando fibra óptica
SPADE	FDM con reparto dinámico del canal
ACTS	TDM con reparto centralizado de ranuras
Binder	TDM con ALOHA cuando el dueño de la ranura no está interesado
Crowther	ALOHA en el que el dueño de la ranura la conserva
Roberts	Tiempo de canal reservado por adelantado con ALOHA

Figura 4-52. Métodos de reparto de canal y sistemas para un canal común.

contención. El protocolo de recorrido de árbol la reduce dividiendo dinámicamente las estaciones en dos grupos separados, uno que puede transmitir y otro que no. Se intenta hacer la división de tal manera que sólo una estación lista para transmitir pueda hacerlo.

Las LAN inalámbricas tienen sus propios problemas y soluciones. El problema principal lo causan las estaciones ocultas, por lo que el CSMA no funciona. Una clase de soluciones, tipificadas por el MACA, intenta estimular las transmisiones en las cercanías del destino, para hacer que el CSMA funcione mejor.

En lo que toca a las computadoras móviles y los teléfonos, la tecnología en ascenso es la radio celular. El GSM, el CDPD y el CDMA se usan ampliamente.

Las LAN IEEE 802 son: CSMA/CD, *token bus* y *token ring*. Cada una tiene sus propias ventajas y desventajas, y cada una ha encontrado su propia comunidad de usuarios, así que probablemente continuará sirviendo a esa comunidad durante varios años más. La convergencia a un solo estándar LAN es poco probable. Una nueva adición a esta familia es DQDB, que se vende como MAN en muchas ciudades.

Una organización con varias LAN con frecuencia las conecta mediante puentes. Cuando un puente conecta dos o más tipos distintos de LAN, surgen nuevos problemas, algunos de ellos sin solución.

Aunque las LAN 802 son los caballos de batalla del día, los caballos de carrera son las FDDI, el Ethernet rápido, HIPPI y el canal de fibra. Todos estos ofrecen anchos de banda del orden de 100 Mbps o más.

Por último, las redes satelitales también emplean canales de acceso múltiple (para el enlace ascendente). Aquí se usan varios métodos de reparto de canal, que incluyen ALOHA, FDM, TDM y CDMA.

PROBLEMAS

1. Un grupo de N estaciones comparte un canal ALOHA puro de 56 kbps. La salida de cada estación es un marco de 1000 bits en promedio cada 100 seg, aun si el anterior no ha sido enviado (por ejemplo, las estaciones tienen *buffers*). ¿Cuál es el valor máximo de N?

2. Considere el retardo del ALOHA puro comparándolo con el ALOHA ranurado cuando la carga es baja. ¿Cuál es menor? Explique su respuesta.

3. Diez mil estaciones de reservaciones de una aerolínea compiten por un solo canal ALOHA ranurado. La estación promedio hace 18 solicitudes/hora. Una ranura dura 125 µseg. ¿Cuál es la carga aproximada total del canal?

4. Una gran población de usuarios de ALOHA genera 50 solicitudes/seg, incluidas tanto originales como retransmisiones. El tiempo se divide en ranuras de 40 mseg.
   (a) ¿Cuál es la posibilidad de éxito en el primer intento?
   (b) ¿Cuál es la probabilidad de exactamente k colisiones y luego una transmisión exitosa?
   (c) ¿Cuál es el número esperado de intentos de transmisión necesarios?

5. Mediciones en un canal ALOHA ranurado con una cantidad infinita de usuarios muestra que 10% de las ranuras están inactivas.
   (a) ¿Qué carga, *G*, tiene el canal?
   (b) ¿Cuál es el rendimiento?
   (c) ¿El canal está subcargado o sobrecargado?
6. En un sistema ALOHA ranurado de población infinita, la cantidad media de ranuras que espera una estación entre una colisión y su retransmisión es de 4. Grafique la curva de retardo contra rendimiento de este sistema.
7. Una LAN usa la versión de Mok y Ward del conteo descendente binario. En cierto momento, las 10 estaciones tienen los números de estación virtual 8, 2, 4, 5, 1, 7, 3, 6, 9 y 0. Las tres estaciones siguientes que van a enviar son: 4, 3 y 9, en ese orden. ¿Cuáles son los nuevos números de estación virtual una vez que las tres han terminado sus transmisiones?
8. Dieciséis estaciones contienden por un canal compartido que usa el protocolo de recorrido de árbol. Si todas las estaciones cuyas direcciones son números primos de pronto quedaran listas al mismo tiempo, ¿cuántas ranuras de bit se necesitan para resolver la contención?
9. Un conjunto de 2^n estaciones usa el protocolo de recorrido de árbol para arbitrar el acceso a un cable compartido. En cierto momento, dos de ellas quedan listas. ¿Cuál es el número de ranuras mínimo, máximo y medio para recorrer el árbol si $2^n \gg 1$?
10. Las LAN inalámbricas que estudiamos usaban protocolos como MACA en lugar de CSMA/CD. ¿En qué condiciones sería posible usar CSMA/CD?
11. ¿Qué propiedades tienen en común los protocolos de acceso a canal WDMA y GSM?
12. Usando la estructura de enmarcado del GSM de la figura 4-14, determine la frecuencia con que puede enviar marcos de datos cualquier usuario dado.
13. Suponga que *A*, *B* y *C* transmiten simultáneamente bits 0 usando un sistema CDMA con las secuencias de *chips* de la figura 4-16(b). ¿Cuál es la secuencia de *chips* resultante?
14. En el análisis sobre la ortogonalidad de las secuencias de *chips* del CDMA se indicó que, si $\mathbf{S} \bullet \mathbf{T} = 0$, entonces $\mathbf{S} \bullet \overline{\mathbf{T}}$ también es 0. Compruebe esto.
15. Considere una manera diferente de ver la propiedad de ortogonalidad de las secuencias de *chips* del CDMA. Cada bit de un par de secuencia puede ser igual o diferente. Exprese la propiedad de ortogonalidad en términos de igualdades y diferencias.
16. Un receptor CDMA recibe los siguientes *chips*: (−1 +1 −3 +1 −1 −3 +1 +1). Suponiendo las secuencias de *chips* definidas en la figura 4-16(b), ¿cuáles estaciones transmitieron y qué bits envió cada una?
17. Un edificio de oficinas de siete pisos tiene 15 oficinas adyacentes por piso. Cada oficina contiene un enchufe de pared para una terminal en la pared frontal, por lo que los enchufes forman una retícula triangular en el plano vertical. Suponiendo que es factible tender un cable recto entre cualquier par de enchufes, horizontal, vertical o diagonalmente, ¿cuántos metros de cable se necesitan para conectar todos los enchufes usando
   (a) una configuración en estrella con un solo enrutador en medio?
   (b) una LAN 802.3?
   (c) una red en anillo (sin centro de alambrado)?

18. ¿Cuál es la tasa de baud de la LAN 802.3 de 10 Mbps estándar?
19. Una LAN CSMA/CD (no la 802.3) de 10 Mbps y 1 km de largo tiene una velocidad de propagación de 200 m/μseg. Los marcos de datos tienen 256 bits de longitud, incluidos 32 bits de cabecera, suma de comprobación y un poco más de información extra. La primera ranura de bits tras una transmisión exitosa se reserva para que el receptor capture el canal y envíe un marco de acuse de 32 bits. ¿Cuál es la tasa de datos efectiva, excluyendo la carga extra, suponiendo que no hay colisiones?
20. Dos estaciones CSMA/CD intentan transmitir archivos grandes (multimarco). Tras el envío de cada marco, contienden por el canal usando el algoritmo de retroceso exponencial binario. ¿Cuál es la probabilidad de que la contención termine en la ronda *k*, y cuál es la cantidad media de rondas por periodo de contención?
21. Considere la construcción de una red CSMA/CD que opera a 1 Gbps a través de un cable de 1 km de longitud sin repetidores. La velocidad de la señal en el cable es de 200,000 km/seg. ¿Cuál es el tamaño mínimo de marco?
22. Dibuje la codificación Manchester para la cadena de bits 0001110101.
23. Dibuje la codificación Manchester diferencial para la cadena de bits del problema anterior. Suponga que la línea inicialmente está en el estado LO.
24. Un sistema *token bus* funciona como sigue. Cuando llega la ficha a una estación, se restablece un temporizador en 0. La estación entonces comienza a transmitir marcos de prioridad 6 hasta que el temporizador alcanza *T6*; luego se conmuta a los marcos de prioridad 4 hasta que el temporizador alcanza *T4*. Este algoritmo se repite entonces con prioridad 2 y prioridad 0. Si todas las estaciones tienen valores de temporización de 40, 80, 90 y 100 mseg para *T6* a *T0*, respectivamente, ¿qué fracción del ancho de banda total se reserva para cada clase de prioridad?
25. ¿Qué ocurre en un *token bus* si una estación acepta la ficha y se cae de inmediato? ¿Cómo maneja este caso el protocolo descrito en el texto?
26. Con una tasa de transmisión de 5 Mbps y una velocidad de propagación de 200 m/μseg, ¿a cuántos metros de cable equivale el retardo de 1 bit de una interfaz *token ring*?
27. El retardo total de un *token ring* debe ser suficiente para contener la ficha completa. Si el alambre no tiene la longitud suficiente, debe introducirse un retardo artificial. Explique por qué es necesario este retardo extra en el contexto de una ficha de 24 bits y un anillo con sólo 16 bits de retardo.
28. Un *token ring* de 1 km de longitud y 10 Mbps con carga muy alta tiene una velocidad de propagación de 200 m/μseg. Hay 50 estaciones a intervalos uniformes alrededor del anillo. Los marcos de datos son de 256 bits, incluidos 32 bits de carga extra. Se incorporan acuses a los marcos de datos, y por tanto se incluyen como bits de repuesto en los marcos de datos, por lo que efectivamente están libres. La ficha es de 8 bits. ¿La tasa de datos efectiva de este anillo es mayor o menor que la de una red CSMA/CD de 10 Mbps?
29. En un *token ring*, el transmisor retira el marco. ¿Qué modificaciones al sistema serían necesarias para hacer que el receptor retire el marco, y cuáles serían las consecuencias?
30. Un *token ring* de 4 Mbps tiene un valor de temporización de 10 mseg para la retención de la ficha. ¿De qué tamaño es el marco más grande que puede enviarse en este anillo?
31. ¿Tiene influencia el uso de un centro de alambrado en el desempeño del *token ring*?

32. Un *token ring* de fibra óptica empleado como MAN tiene 200 km de longitud y opera a 100 Mbps. Tras enviar un marco, la estación drena el marco del anillo antes de regenerar la ficha. La velocidad de propagación de la señal a través de la fibra es de 200,000 km/seg y el tamaño máximo de marco es de 1K bytes. ¿Cuál es la eficiencia máxima del anillo (ignorando todas las demás fuentes de carga extra)?

33. En la figura 4-32, la estación *D* quiere enviar una céiula. ¿A qué estación quiere enviarla?

34. El sistema de la figura 4-32 está inactivo. Un poco después, las estaciones *C, A* y *B* quedan listas para enviar, en ese orden y en secuencia rápida. Suponiendo que no se transmiten marcos de datos hasta que las tres han enviado una solicitud corriente arriba, muestre los valores de *RC* y de *CD* tras cada solicitud y tras los tres marcos de datos.

35. A veces se dice que el Ethernet es inadecuado para la computación en tiempo real, pues la ranura de retransmisión de peor caso no está limitado. ¿En qué circunstancias puede aplicarse el mismo argumento al *token ring*? ¿En qué circunstancias tiene un peor caso conocido el *token ring*? Suponga que el número de estaciones del *token ring* es fijo y se conoce.

36. Los marcos de Ethernet deben tener cuando menos 64 bytes de longitud para asegurar que el transmisor aún esté transmitiendo en caso de haber una colisión en el otro extremo del cable. El Ethernet rápido tiene el mismo tamaño mínimo de marco de 64 bytes, pero puede enviar los bits a una velocidad 10 veces mayor. ¿Cómo es posible mantener el mismo tamaño mínimo de marco?

37. Imagine dos puentes LAN que conectan, ambos, un par de redes 802.4. El primer puente confronta 1000 marcos por segundo de 512 bytes que deben reenviarse. El segundo confronta 200 marcos por segundo de 4096 bytes. ¿Qué puente cree usted que necesitará una CPU más rápida? Explique.

38. Suponga que los dos puentes del problema anterior conectan, ambos, una LAN 802.4 con una LAN 802.5. ¿Afectaría ese cambio la respuesta anterior?

39. Un puente entre una LAN 802.3 y una 802.4 tiene problemas de errores de memoria intermitentes. ¿Puede causar esto errores no detectados en los marcos transmitidos, o serán atrapados los errores gracias a las sumas de comprobación de los marcos?

40. El departamento de ciencias de la computación de una universidad tiene tres segmentos Ethernet conectados mediante dos puentes transparentes a una red lineal. Un día, el administrador renuncia y lo reemplaza apresuradamente alguien del centro de cómputo, que es una instalación *token ring* de IBM. El nuevo administrador, notando que los extremos de la red no están conectados, de inmediato ordena un puente transparente nuevo y conecta ambos extremos sueltos a él, formando un anillo cerrado. ¿Qué ocurre después?

41. Un anillo FDDI grande tiene 100 estaciones y un tiempo de rotación de ficha de 40 mseg. El tiempo de retención de la ficha es de 10 mseg. ¿Cuál es la eficiencia máxima que puede tener el anillo?

42. Considere la construcción de una interconexión de supercomputadoras usando el enfoque HIPPI, pero tecnología moderna. La trayectoria de datos ahora es de 64 bits de ancho y puede enviarse una palabra cada 10 nseg. ¿Cuál es el ancho de banda del canal?

43. En el texto se indicó que un satélite con dos canales de ALOHA ranurado de enlace ascendente y uno de enlace descendente puede lograr una utilización del enlace descendente de 0.736, dada una cantidad infinita de espacio de *buffer*. Muestre la manera en que puede lograrse este resultado.

5

LA CAPA DE RED

La capa de red se encarga de llevar los paquetes desde el origen hasta el destino. Llegar al destino puede requerir muchos saltos por enrutadores intermedios. Esta función ciertamente contrasta con la de la capa de enlace de datos, que sólo tiene la meta modesta de mover marcos de un extremo del alambre al otro. Por tanto, la capa de red es la capa más baja que maneja la transmisión de punta a punta. Para mayor información sobre ella, véase (Huitema, 1995, y Perlman, 1992).

Para lograr su cometido, la capa de red debe conocer la topología de la subred de comunicación (es decir, el grupo de enrutadores) y escoger las trayectorias adecuadas a través de ella; también debe tener cuidado de escoger las rutas a modo de evitar la carga extra de algunas de las líneas de comunicación y de los enrutadores mientras deja a otros sin trabajo. Por último, cuando el origen y el destino están en redes diferentes, es responsabilidad de la capa de red el manejo de estas diferencias y la resolución de los problemas que causan. En este capítulo estudiaremos todos estos temas y los ilustraremos con nuestros dos ejemplos, Internet y ATM.

5.1. TEMAS DE DISEÑO DE LA CAPA DE RED

En las siguientes secciones presentaremos una introducción a algunos de los problemas que deben enfrentar los diseñadores de la capa de red. Estos temas incluyen el servicio proporcionado a la capa de transporte y el diseño interno de la subred.

5.1.1. Servicios proporcionados a la capa de transporte

La capa de red proporciona servicios a la capa de transporte en la interfaz capa de red/capa de transporte. Esta interfaz muchas veces tiene especial importancia por otra razón: con frecuencia es la interfaz entre la portadora y el cliente, es decir, el límite de la subred. La portadora suele controlar los protocolos y las interfaces hasta la capa de red, inclusive; su trabajo es entregar los paquetes entregados a ella por sus clientes. Por esta razón, esta interfaz debe estar especialmente bien definida.

Los servicios de la capa de red se diseñaron con las siguientes metas en mente.

1. Los servicios deben ser independientes de la tecnología de subred.
2. La capa de transporte debe estar aislada de la cantidad, tipo y topología de las subredes presentes.
3. Las direcciones de red disponibles para la capa de transporte deben seguir un plan de numeración uniforme, aun a través de varias LAN y WAN.

Dadas estas metas, los diseñadores de la capa de red tienen mucha libertad para escribir especificaciones detalladas de los servicios que serán ofrecidos a la capa de transporte. Esta libertad con frecuencia degenera en una batalla campal entre dos facciones en conflicto. La discusión se centra en la interrogante de si la capa de red debe proporcionar servicio orientado a conexión o servicio sin conexiones.

Un bando (representado por la comunidad Internet) alega que la tarea de la subred es mover bits de un lado a otro, y nada más. Desde su punto de vista (basado en casi 30 años de experiencia con una red de computadoras real y operativa), la subred es inherentemente inestable, sin importar su diseño. Por tanto, los *hosts* deben aceptar el hecho de que es inestable y efectuar ellos mismos el control de errores (es decir, detección y corrección de errores) y el control de flujo.

Este punto de vista conduce directamente a la conclusión de que el servicio de red debe ser sin conexiones, tener las primitivas ENVIAR PAQUETE (*SEND PACKET*) y RECIBIR PAQUETE (*RECEIVE PACKET*), y casi nada más. En particular, no debe efectuarse ningún ordenamiento de paquetes ni control de flujo, pues de todos modos los *hosts* lo van a efectuar y probablemente se ganaría poco haciéndolo dos veces. Además, cada paquete debe llevar la dirección de destino completa, porque cada paquete enviado se transporta independientemente de sus antecesores, si los hay.

El otro bando (representado por las compañías telefónicas) argumenta que la subred debe proporcionar un servicio (razonablemente) confiable, orientado a conexión. Indican que una buena guía son 100 años de experiencia exitosa del sistema telefónico mundial. Desde este punto de vista, las conexiones deben tener las siguientes propiedades:

1. Antes de enviar datos, un proceso de capa de red en el lado transmisor debe establecer una conexión con su igual en el lado receptor. Se usa entonces esta conexión, que recibe un identificador especial, hasta que se han enviado todos los datos, en cuyo momento se libera explícitamente.
2. Al establecerse una conexión, los dos procesos pueden comenzar a negociar los parámetros, calidad y costo del servicio a proporcionar.
3. La comunicación es en ambas direcciones y los paquetes se entregan en secuencia.
4. Se proporciona automáticamente control de flujo para evitar que un transmisor rápido descargue paquetes en el conducto con mayor rapidez que aquella con que puede retirarlos el receptor, lo que conduce a un desbordamiento.

Son opcionales otras propiedades, como la entrega garantizada, la confirmación explícita de la entrega y los paquetes de alta prioridad. Como indicamos en el capítulo 1, el servicio sin conexiones es como el sistema postal y el servicio orientado a conexión es como el sistema telefónico.

La controversia entre el servicio orientado a conexión y el servicio sin conexiones en realidad tiene que ver con dónde está la complejidad. En el servicio orientado a conexión, está en la capa de red (subred); en el servicio sin conexiones, está en la capa de transporte (*hosts*). Los defensores del servicio sin conexiones dicen que la capacidad de cómputo de usuario se ha vuelto barata, por lo que no hay razón para no poner la complejidad en los *hosts*. Además, argumentan que la subred es una inversión (inter)nacional importante que durará décadas, por lo que no debe sobrecargarse con características que podrían volverse obsoletas rápidamente, pero que tendrán que calcularse como parte de la estructura de precios durante muchos años. Además, algunas aplicaciones, como la voz digitalizada y la recolección de datos en tiempo real, pueden considerar mucho más importante la entrega *rápida* que la entrega *exacta*.

Por otra parte, los defensores del servicio orientado a conexión dicen que la mayoría de los usuarios no tienen interés en ejecutar protocolos complejos de capa de transporte en sus máquinas. Lo que quieren es servicio confiable y libre de problemas, y este servicio puede proporcionarse mejor con conexiones de capa de red. Además, es más fácil proporcionar algunos servicios, como el audio y el vídeo de tiempo real, a través de una capa de red orientada a conexiones que a través de una capa de red sin conexiones.

Aunque pocas veces la discusión es en estos términos, aquí están implicados dos temas diferentes. Primero, si la red está orientada a conexión (se requiere preparación) o es sin conexiones (no se requiere preparación). Segundo, si la red es confiable (sin paquetes perdidos, duplicados ni alterados) o no confiable (los paquetes pueden perderse, duplicarse o alterarse). En teoría, existen las cuatro combinaciones, pero las combinaciones dominantes son confiable orientada a conexión y no confiable sin conexiones, por lo que las otras dos tienden a perderse en el alboroto.

Estos bandos están representados por nuestros dos ejemplos operativos. El Internet tiene una capa de red sin conexiones, y las redes ATM tienen una capa de red orientada a conexión. Surge una pregunta obvia sobre la manera en que trabaja Internet cuando atraviesa una subred con portadora basada en ATM. La respuesta es que el *host* de origen establece primero una conexión

ATM de capa de red con el *host* de destino, y luego envía por ella paquetes independientes (IP), como se muestra en la figura 5-1. Aunque este enfoque funciona, es ineficiente debido a que cierta funcionalidad está en ambas capas. Por ejemplo, la capa de red ATM garantiza que los paquetes siempre se entreguen en orden, pero el código TCP de todos modos contiene el mecanismo completo de administración y reordenamiento de paquetes desordenados. Para mayor información sobre la manera de ejecutar IP sobre ATM, véase el RFC 1577 y (Armitage y Adams, 1995).

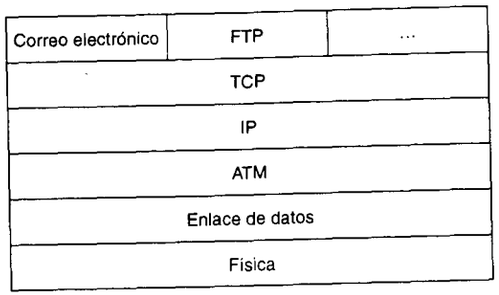


Figura 5-1. Ejecución de TCP/IP en una subred ATM.

5.1.2. Organización interna de la capa de red

Ahora que hemos visto las dos clases de servicio que puede proporcionar la capa de red a sus usuarios, es tiempo de ver su funcionamiento interno. Básicamente hay dos filosofías diferentes de organización de la subred, una que usa conexiones y otra que funciona sin conexiones. En el contexto de la operación *interna* de la subred, generalmente se llama **circuito virtual** a una conexión, por analogía con los circuitos físicos instalados por el sistema telefónico. Los paquetes independientes de la organización de tipo sin conexiones se llaman **datagramas**, por analogía con los telegramas.

Los circuitos virtuales generalmente se usan en subredes cuyo servicio primario está orientado a conexión, por lo que los describiremos en ese contexto. La idea en que se basan los circuitos virtuales es evitar tener que escoger una ruta nueva para cada paquete o célula enviada. En cambio, cuando se establece una conexión, se escoge y se recuerda una ruta de la máquina de origen a la de destino como parte del establecimiento de la conexión. Esa ruta se usa para todo el tráfico que fluye por la conexión, de la misma manera en que funciona el sistema telefónico. Al liberarse la conexión, también deja de existir el circuito virtual.

En contraste, en una subred de datagramas no se determinan rutas por adelantado, aun si el servicio está orientado a conexión. Cada paquete enviado se enruta de manera independiente de sus antecesores. Dos paquetes sucesivos pueden seguir rutas distintas. Si bien las subredes de datagramas tienen más trabajo que hacer, generalmente también son más robustas y se adaptan a

fallas y congestionamientos con mayor facilidad que las subredes de circuito virtual. Después estudiaremos las ventajas y desventajas de los dos enfoques.

Si los paquetes que fluyen a través de un circuito virtual dado siempre siguen la misma ruta a través de la subred, cada enrutador debe recordar a dónde enviar los paquetes para cada uno de los circuitos virtuales abiertos que pasan por él. Cada enrutador debe mantener una tabla con una entrada por circuito virtual abierto que pasa a través suyo. Cada paquete que viaja a través de la red debe contener un campo de número de circuito virtual en su cabecera, además de los números de secuencia, sumas de comprobación y demás. Al llegar un paquete al enrutador, el enrutador sabe la línea por la que llegó y el número del circuito virtual. Utilizando sólo esa información, debe enviar el paquete por la línea de salida correcta.

Al establecerse una conexión de red, se escoge un número de circuito virtual actualmente desocupado en esa máquina como identificador de la conexión. Dado que cada máquina escoge independientemente los números de circuito virtual, estos números tienen sólo significado local. Si tuvieran un significado global en toda la red, es posible que dos circuitos virtuales que tuvieran el mismo número de circuito virtual global pasaran a través de un mismo enrutador intermedio, provocando ambigüedades.

Dado que los circuitos virtuales pueden iniciarse desde cualquiera de los dos lados, ocurre un problema cuando se propagan establecimientos de llamada en ambas direcciones al mismo tiempo a lo largo de una cadena de enrutadores. En algún punto, habrán llegado a enrutadores adyacentes. Cada enrutador debe escoger ahora un número de circuito virtual para el circuito (dúplex integral) que está tratando de establecer. Si los enrutadores están programados para escoger el número menor no utilizado en el enlace, escogerán el mismo número, haciendo que dos circuitos virtuales no relacionados en la misma línea física tengan el mismo número. Cuando llega posteriormente un paquete de datos, el enrutador receptor no tiene manera de saber si se trata de un paquete de ida por un circuito o un paquete de regreso por el otro. Si los circuitos son símplex, no hay ambigüedad.

Nótese que se requiere que cada proceso indique cuándo ha terminado de usar un circuito virtual, de modo que el circuito virtual pueda purgarse de las tablas del enrutador y así recuperar el espacio. En las redes públicas, la motivación es el "palo", en lugar de la "zanahoria": a los usuarios siempre se les cobra por el tiempo de conexión, así como por los datos transportados. Además, debe tomarse alguna medida para manejar las máquinas que terminan sus circuitos virtuales con una caída abrupta, en lugar de liberarlos cortésmente.

Hasta aquí el uso de circuitos virtuales internos de la subred. La otra posibilidad es usar datagramas internamente, en cuyo caso los enrutadores no tienen una tabla con una entrada por cada circuito virtual abierto; en cambio, tienen un tabla que indica la línea de salida a usar para cada enrutador de destino posible. Estas tablas también son necesarias al usar internamente circuitos virtuales, a fin de determinar la ruta de un paquete de establecimiento.

Cada datagrama debe contener la dirección de destino completa. En una red grande, estas direcciones pueden ser bastante largas (por ejemplo, una docena de bytes o más). Al llegar un paquete, el enrutador busca la línea de salida a usar y envía el paquete. El establecimiento y liberación de conexiones de capa de red o de transporte no requiere ningún trabajo especial por parte de los enrutadores.

5.1.3. Comparación de las subredes de circuitos virtuales y de datagramas

Tanto los circuitos virtuales como los datagramas tienen sus seguidores y sus detractores. Ahora intentaremos resumir los argumentos de ambos bandos. Los aspectos principales se listan en la figura 5-2, aunque los puristas probablemente podrán encontrar ejemplos contrarios para todo lo indicado en la figura.

Asunto	Subred de datagramas	Subred de CV
Establecimiento del circuito	No necesaria	Requerida
Direccionamiento	Cada paquete contiene completas la dirección de origen y de destino	Cada paquete contiene un número de CV corto
Información de estado	La subred no contiene información de estado	Cada CV requiere espacio de tabla de la subred
Enrutamiento	Cada paquete se enruta independientemente	Ruta escogida cuando se establece el CV; todos los paquetes siguen esta ruta
Efecto de fallas del enrutador	Ninguno, exceptuando paquetes perdidos durante una caída	Terminan todos los CV que pasan a través del enrutador con falla
Control de congestionamientos	Difícil	Fácil si pueden asignarse por adelantado *buffers* suficientes a cada CV

Figura 5-2. Comparación de las subredes de datagramas y de circuitos virtuales.

Dentro de la subred, hay varias diferencias entre lo que sacrifican los circuitos virtuales y los datagramas. Una de ellas concierne al espacio de memoria del enrutador y el ancho de banda. Los circuitos virtuales permiten que los paquetes contengan números de circuito en lugar de direcciones de destino completas. Si los paquetes suelen ser bastante cortos, una dirección de destino completa en cada paquete puede representar una carga extra significativa y, por tanto, ancho de banda desperdiciado. El precio que paga por el uso interno de circuitos virtuales es el espacio de tabla en los enrutadores. Dependiendo del costo relativo de los circuitos de comunicación y de la memoria de enrutador, alguna de las dos cosas podría ser más barata.

Otro balance es el del tiempo de establecimiento contra el tiempo de análisis de la dirección. El uso de circuitos virtuales requiere una fase de establecimiento, que consume tiempo y recursos. Sin embargo, la determinación de lo que hay que hacer con un paquete de datos en una subred de circuitos virtuales es fácil: el enrutador simplemente usa el número de circuito para buscar en una tabla indizada y encontrar el lugar al que va el paquete. En una subred de datagramas se requiere un procedimiento más complicado para determinar a dónde va el paquete.

Los circuitos virtuales tienen algunas ventajas en cuanto a que evitan congestionamientos en la subred, pues los recursos pueden reservarse por adelantado al establecerse la conexión. Una vez que comienzan a llegar los paquetes, estarán ahí el ancho de banda y la capacidad de enrutamiento necesarios. Con una subred de datagramas, es más difícil evitar los congestionamientos.

Para los sistemas de procesamiento de transacciones (por ejemplo, las tiendas que llaman para verificar pagos con tarjeta de crédito), la carga extra requerida para establecer y terminar un circuito virtual puede ocupar mucho más tiempo que el uso real del circuito. Si la mayor parte del tráfico esperado es de este tipo, el uso de circuitos virtuales conmutados en la subred tiene poco sentido. Por otra parte, pueden ser de utilidad aquí los circuitos virtuales permanentes, establecidos manualmente y con duración de meses o años.

Los circuitos virtuales también tienen un problema de vulnerabilidad. Si se cae un enrutador, perdiéndose su memoria, tendrán que abortarse todos los circuitos virtuales que pasan por él, aun si se recupera un segundo después. Por el contrario, al caerse un enrutador de datagramas sólo sufrirán aquellos usuarios cuyos paquetes estaban encolados en el enrutador en el momento y, dependiendo de si ya habían sido reconocidos o no, tal vez ni siquiera todos. La pérdida de una línea de comunicación es fatal para los circuitos virtuales que la usan, pero puede compensarse fácilmente cuando se usan datagramas. Los datagramas también permiten que los enrutadores equilibren el tráfico a través de la subred, ya que las rutas pueden cambiarse a la mitad de una conexión.

Es importante indicar explícitamente que el servicio ofrecido (orientado a conexión o sin conexiones) es un asunto independiente de la estructura de la subred (circuitos virtuales o datagramas). En teoría, son posibles las cuatro combinaciones. Obviamente, son razonables la implementación con circuitos virtuales de un servicio orientado a conexión y la implementación con datagramas de un servicio sin conexiones. También tiene sentido la implementación de conexiones usando datagramas cuando la subred está tratando de proporcionar un servicio muy robusto.

La cuarta posibilidad, el servicio sin conexiones por una subred de circuitos virtuales, parece extraña, pero ciertamente ocurre. El ejemplo obvio es ejecutar IP en una subred ATM. Aquí se desea operar un protocolo sin conexiones existente sobre una nueva capa de red orientada a conexiones. Como se mencionó antes, ésta es más bien una solución *ad hoc* para un problema, no un buen diseño. En un sistema nuevo diseñado para operar en una subred ATM, normalmente no se pondría un protocolo sin conexiones, como IP, sobre una capa de red orientada a conexión, como ATM, agregando después un protocolo de transporte orientado a conexión encima del protocolo sin conexiones. En la figura 5-3 se muestran ejemplos de los cuatro casos.

UNIVERSIDAD DE
FACULTAD DE
DEPARTA
DOCUMENTACIO
MONTEVIDEO

5.2. ALGORITMOS DE ENRUTAMIENTO

La función principal de la capa de red es enrutar paquetes de la máquina de origen a la de destino. En la mayoría de las subredes, los paquetes requerirán varias escalas para completar el viaje. La única excepción importante son las redes de difusión, pero aun aquí es importante

Capa superior	Tipo de subred: Datagramas	Tipo de subred: Circuitos virtuales
Sin conexiones	UDP sobre IP	UDP sobre IP sobre ATM
Orientada a conexiones	TCP sobre IP	ATM AAL1 sobre ATM

Figura 5-3. Ejemplos de diferentes combinaciones de estructuras de servicios y subredes.

el enrutamiento si el origen y el destino no están en la misma red. Los algoritmos que escogen las rutas y las estructuras de datos que éstos usan son un área principal del diseño de la capa de red.

El **algoritmo de enrutamiento** es aquella parte del *software* de la capa de red encargada de decidir la línea de salida por la que se transmitirá un paquete de entrada. Si la subred usa datagramas internamente, esta decisión debe hacerse cada vez que llega un paquete de datos de entrada, dado que la mejor ruta podría haber cambiado desde la última vez. Si la subred usa circuitos virtuales internamente, las decisiones de enrutamiento se toman sólo al establecerse un circuito virtual nuevo. Por tanto, los paquetes de datos simplemente siguen la ruta previamente establecida. Este último caso a veces se llama **enrutamiento de sesión**, dado que una ruta permanece vigente durante la sesión de usuario completa (por ejemplo, durante una sesión interactiva desde una terminal, o durante una transferencia de archivo).

Sea que se escojan independientemente las rutas para cada paquete o sólo al establecerse nuevas conexiones, hay ciertas propiedades que son deseables en un algoritmo de enrutamiento: corrección, sencillez, robustez, estabilidad, equitatividad y optimalidad. La corrección y la sencillez apenas requieren comentarios, pero la necesidad de robustez puede ser menos obvia a primera vista. Una vez que entra en operación una red principal, puede esperarse que opere continuamente durante años sin fallas generales. Durante ese periodo habrá fallas de *hardware* y de *software* de todos tipos. Los *hosts*, enrutadores y líneas se activarán y desactivarán repetidamente y cambiará muchas veces la topología. El algoritmo de enrutamiento debe ser capaz de manejar los cambios de topología y tráfico sin requerir el aborto de todas las actividades en todos los *hosts* y el rearranque de la red con cada caída de un enrutador.

La estabilidad también es una meta importante del algoritmo de enrutamiento. Existen algoritmos de enrutamiento que nunca convergen en el equilibrio, sin importar el tiempo que permanezcan operativos. La equitatividad y la optimalidad pueden parecer algo obvio (ciertamente nadie se opondrá a ellas), pero resulta que con frecuencia son metas contradictorias. Para un ejemplo sencillo de este conflicto, véase la figura 5-4. Supóngase que hay suficiente tráfico entre A y A', entre B y B' y entre C y C' para saturar los enlaces horizontales. A fin de aumentar al máximo el flujo total, el tráfico de X a X' debe suspenderse por completo. Por desgracia, X y

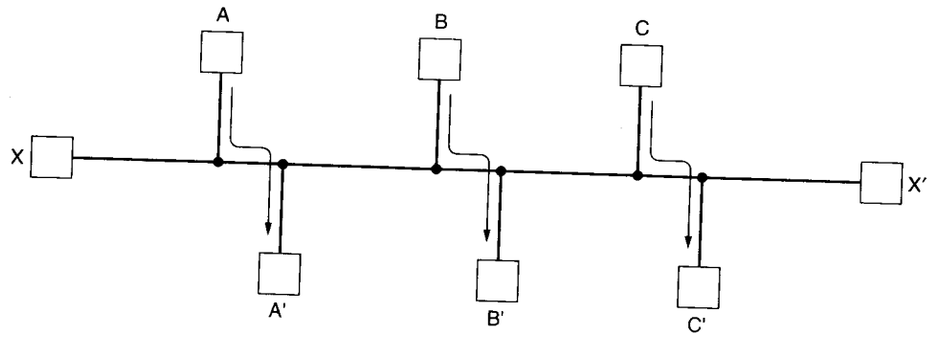


Figura 5-4. El conflicto entre equitatividad y optimización.

X' podrían no verlo de esta manera. Evidentemente se requiere una concesión entre la eficiencia global y la equitatividad hacia las conexiones individuales.

Antes de que podamos siquiera intentar encontrar un justo medio entre la equitatividad y la optimalidad, debemos decidir qué es lo que buscamos optimar. Un candidato obvio es la minimización del retardo medio de los paquetes, pero también lo es el aumento al máximo del rendimiento total de la red. Además, estas dos metas también están en conflicto, ya que la operación de cualquier sistema de colas cerca de su capacidad máxima implica un retardo de encolamiento grande. Como término medio, muchas redes intentan minimizar el número de escalas que tiene que hacer un paquete, puesto que la reducción de la cantidad de escalas tiende a reducir el retardo y también el consumo de ancho de banda, lo que tiende a mejorar el rendimiento.

Los algoritmos de enrutamiento pueden agruparse en dos clases principales: no adaptables y adaptables. Los **algoritmos no adaptables** no basan sus decisiones de enrutamiento en mediciones o estimaciones del tráfico y la topología actuales. En cambio, la decisión de qué ruta se usará para llegar de I a J (para todas las I y J) se calcula por adelantado, fuera de línea, y se carga en los enrutadores al iniciar la red. Este procedimiento se llama **enrutamiento estático**.

Los **algoritmos adaptables**, en contraste, cambian sus decisiones de enrutamiento para reflejar los cambios de topología, y generalmente también el tráfico. Los algoritmos adaptables difieren en el lugar de obtención de su información (por ejemplo, localmente, de los enrutadores adyacentes o de todos los enrutadores), el momento de cambio de sus rutas (por ejemplo, cada ΔT segundos, cuando cambia la carga o cuando cambia la topología), y la métrica usada para la optimalidad (por ejemplo, distancia, número de escalas o tiempo estimado de tránsito). En las siguientes secciones estudiaremos una variedad de algoritmos de enrutamiento, tanto estáticos como dinámicos.

5.2.1. Principio de optimación

Antes de entrar en algoritmos específicos, puede ser útil señalar que es posible hacer un postulado general sobre las rutas óptimas sin importar la topología o el tráfico de la red. Este postulado se conoce como **principio de optimación**, y establece que, si el enrutador J está en la trayectoria

óptima del enrutador *I* al enrutador *K*, entonces la trayectoria óptima de *J* a *K* también está en la misma ruta. Para ver esto, llamemos r_1 a la parte de la ruta de *I* a *J*, y r_2 al resto de la ruta. Si existiera una ruta mejor que r_2 entre *J* y *K*, podría concatenarse con r_1 para mejorar la ruta entre *I* y *K*, contradiciendo nuestra aseveración de que r_1r_2 es óptima.

Como consecuencia directa del principio de optimalidad, podemos ver que el grupo de trayectorias óptimas de todas las de orígenes a un destino dado forma un árbol con raíz en el destino. Tal árbol se llama **árbol de descenso** y se ilustra en la figura 5-5, donde la métrica de distancia es el número de escalas. Nótese que un árbol de descenso no necesariamente es único; pueden existir otros árboles con las mismas longitudes de trayectoria. La meta de todos los algoritmos de enrutamiento es descubrir y usar los árboles de descenso para todos los enrutadores.

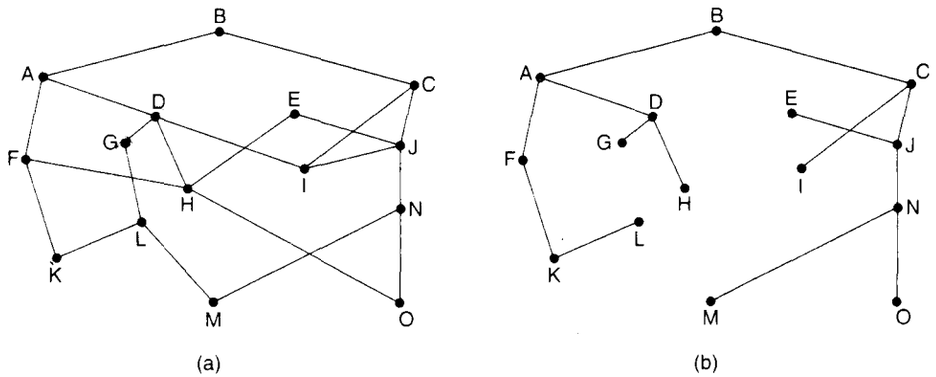


Figura 5-5. (a) Una subred. (b) Árbol descendente para el enrutador B.

Dado que un árbol de descenso ciertamente es un árbol, no contiene ciclos, por lo que cada paquete será entregado con un número de escalas finito y limitado. En la práctica, la vida no es tan fácil. Los enlaces y los enrutadores pueden caerse y reactivarse durante la operación, por lo que diferentes enrutadores pueden tener ideas distintas sobre la topología actual. También hemos evadido calladamente la cuestión de si cada enrutador tiene que adquirir individualmente la información en la cual basa su cálculo del árbol de descenso, o si esta información se obtiene por otros medios. Regresaremos a estos asuntos pronto. Con todo, el principio de optimalidad y el árbol de descenso proporcionan parámetros contra los que se pueden medir otros algoritmos de enrutamiento.

En las tres secciones siguientes veremos tres algoritmos estáticos de enrutamiento. Después, pasaremos a los adaptables.

5.2.2. Enrutamiento por la trayectoria más corta

Comencemos nuestro estudio de los algoritmos de enrutamiento con una técnica de amplio uso en muchas formas, porque es sencilla y fácil de entender. La idea es armar un grafo de la subred,

en el que cada nodo representa un enrutador y cada arco del grafo una línea de comunicación (llamada con frecuencia enlace). Para escoger una ruta entre un par dado de enrutadores, el algoritmo simplemente encuentra en el grafo la trayectoria más corta entre ellos.

El concepto de **trayectoria más corta** merece una explicación. Una manera de medir la longitud de una trayectoria es por la cantidad de escalas. Usando esta métrica, las trayectorias *ABC* y *ABE* de la figura 5-6 tienen la misma longitud. Otra métrica es la distancia geográfica en kilómetros, en cuyo caso *ABC* claramente es mucho mayor que *ABE* (suponiendo que la figura está dibujada a escala).

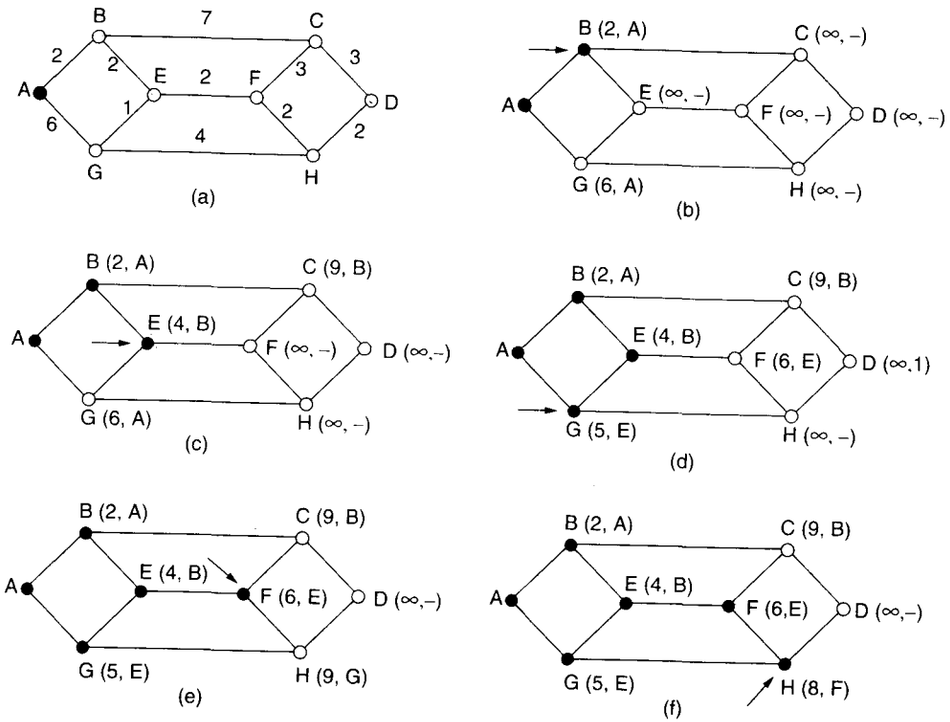


Figura 5-6. Los primeros cinco pasos del cálculo de la trayectoria más corta de *A* a *D*. Las flechas indican el nodo de trabajo.

Sin embargo, también son posibles muchas otras métricas además de las escalas y la distancia física. Por ejemplo, cada arco podría etiquetarse con el retardo medio de encolamiento y transmisión de un paquete de prueba estándar, determinado por series de prueba cada hora. Con estas etiquetas en el grafo, la trayectoria más corta es la trayectoria más rápida, en lugar de la trayectoria con menos arcos o kilómetros.

En el caso más general, las etiquetas de los arcos podrían calcularse como una función de la distancia, ancho de banda, tráfico medio, costo de comunicación, longitud media de las colas,

retardo medido y otros factores. Cambiando la función de ponderación, el algoritmo calcularía la trayectoria medida "más corta" de acuerdo con cualquiera de varios criterios, o una combinación de criterios.

Se conocen varios algoritmos de cálculo de la trayectoria más corta entre dos nodos de un grafo. Éste se debe a Dijkstra (1959). Cada nodo se etiqueta (entre paréntesis) con su distancia al nodo de origen a través de la mejor trayectoria conocida. Inicialmente no se conocen trayectorias, por lo que todos los nodos tienen la etiqueta infinito. A medida que avanza el algoritmo y se encuentran trayectorias, pueden cambiar las etiquetas, reflejando mejores trayectorias. Una etiqueta puede ser tentativa o permanente. Inicialmente todas las etiquetas son tentativas. Al descubrirse que una etiqueta representa la trayectoria más corta posible del origen a ese nodo, se vuelve permanente y no cambiar más.

Para ilustrar el funcionamiento del algoritmo de etiquetado, obsérvese el grafo ponderado no dirigido de la figura 5-6(a), donde las ponderaciones representan, por ejemplo, distancias. Queremos encontrar la trayectoria más corta posible de *A* a *D*. Comenzamos por marcar como permanente el nodo *A*, indicado por un círculo rellenado. Examinamos entonces, por turno, cada uno de los nodos adyacentes a *A* (el nodo de trabajo), reetiquetando cada uno con la distancia desde *A*. Cada vez que reetiquetamos un nodo, también lo etiquetamos con el nodo desde el que se hizo la prueba, para poder reconstruir después la trayectoria final. Habiendo examinado cada uno de los nodos adyacentes a *A*, examinamos todos los nodos etiquetados tentativamente en el grafo completo y hacemos permanente el de la etiqueta más pequeña, como se muestra en la figura 5-6(b). Éste se convierte en el nuevo nodo de trabajo.

Ahora comenzamos por *B*, y examinamos todos los nodos adyacentes a él. Si la suma de la etiqueta de *B* y la distancia desde *B* al nodo en consideración es menor que la etiqueta de ese nodo, tenemos una trayectoria más corta, por lo que reetiquetamos ese nodo.

Tras inspeccionar todos los nodos adyacentes al nodo de trabajo y cambiar las etiquetas tentativas (de ser posible) se busca en el grafo completo el nodo etiquetado tentativamente con el menor valor. Este nodo se hace permanente y se convierte en el nodo de trabajo para la siguiente ronda. En la figura 5-6 se muestran los primeros cinco pasos del algoritmo.

Para ver por qué funciona el algoritmo, véase la figura 5-6(c). En ese punto acabamos de hacer permanente a *E*. Supóngase que hubiera una trayectoria más corta que *ABE*, digamos *AXYZE*. Hay dos posibilidades: el nodo *Z* ya se hizo permanente, o no se ha hecho permanente. Si ya es permanente, entonces *E* ya se probó (en la ronda que siguió a aquella en la que se hizo permanente *Z*), por lo que la trayectoria *AXYZE* no ha escapado a nuestra atención.

Ahora considere el caso en el que *Z* aún está etiquetado tentativamente. O bien la etiqueta de *Z* es mayor o igual que la de *E*, en cuyo caso *AXYZE* no puede ser una trayectoria más corta que *ABE*, o es menor que la de *E*, en cuyo caso *Z*, y no *E*, se volverá permanente primero, permitiendo probar *E* desde *Z*.

Este algoritmo se da en la figura 5-7. La única diferencia entre el programa y el algoritmo antes descrito es que, en la figura 5-7, calculamos la trayectoria más corta posible comenzando por el nodo terminal, *t*, en lugar del nodo de origen, *s*. Dado que la trayectoria más corta posible de *t* a *s* en un grafo no dirigido es igual a la trayectoria más corta de *s* a *t*, no importa el extremo

por el que comencemos (a menos que haya varias trayectorias más cortas posibles, en cuyo caso la inversión de la búsqueda podría descubrir una distinta). La razón de una búsqueda en reversa es que cada nodo está etiquetado con su antecesor, en lugar de su sucesor. Al copiar la trayectoria final en la variable de salida, *path*, la trayectoria de salida se invierte. Al invertir la búsqueda, ambos efectos se cancelan, y la respuesta se produce en el orden correcto.

5.2.3. Inundación

Otro algoritmo estático es la **inundación**, en la que cada paquete de entrada se envía por cada una de las líneas de salida, excepto aquella por la que llegó. La inundación evidentemente genera grandes cantidades de paquetes duplicados; de hecho, una cantidad infinita a menos que se tomen algunas medidas para limitar el proceso. Una de tales medidas es un contador de escalas contenido en la cabecera de cada paquete, el cual disminuye en cada escala, descartándose el paquete al llegar el contador a cero. Idealmente, el contador de escalas debe inicializarse a la longitud de la trayectoria entre el origen y el destino. Si el transmisor no conoce el tamaño de la trayectoria, puede inicializar el contador al peor caso, es decir, el diámetro total de la subred.

Una técnica alterna para ponerle diques a la inundación es llevar un registro de los paquetes diseminados, para evitar enviarlos una segunda vez. Una manera de lograr esta meta es hacer que el enrutador de origen ponga un número de secuencia en cada paquete que recibe de sus *hosts*. Cada enrutador necesita entonces una lista por cada enrutador de origen que indique los números de secuencia originados en ese enrutador que ya ha visto. Si un paquete de entrada está en la lista, no se disemina.

Para evitar que la lista crezca sin límites, cada lista debe incluir un contador, *k*, que indique que todos los números de secuencia hasta *k* ya han sido vistos. Al llegar un paquete, es fácil comprobar si el paquete es un duplicado; de ser así, se descarta. Es más, no se necesita la lista completa por debajo de *k*, pues *k* la resume efectivamente.

Una variación de la inundación, un poco más práctica, es la **inundación selectiva**. En este algoritmo, los enrutadores no envían cada paquete de entrada por todas las líneas, sino sólo por aquellas que van aproximadamente en la dirección correcta. Generalmente no tiene mucho caso enviar un paquete dirigido al oeste a través de una línea dirigida al este, a menos que la topología sea extremadamente peculiar.

La inundación no es práctica en la mayoría de las aplicaciones, pero tiene algunos usos. Por ejemplo, en aplicaciones militares, donde pueden volar en pedazos grandes cantidades de enrutadores en cualquier momento, es altamente deseable la tremenda robustez de la inundación. En las aplicaciones de bases de datos distribuidas a veces es necesario actualizar concurrentemente todas las bases de datos, en cuyo caso puede ser útil la inundación. Un tercer posible uso de la inundación es como métrica contra la que pueden compararse otros algoritmos de enrutamiento. La inundación siempre escoge la trayectoria más corta posible, porque escoge en paralelo todas las trayectorias posibles. En consecuencia, ningún otro algoritmo puede producir un retardo más corto (si ignoramos la carga extra generada por el proceso de inundación mismo).

```
#define MAX_NODES 1024                  /* número máximo de nodos */
#define INFINITY 1000000000             /* un número mayor que cualquier trayectoria máxima */
int n, dist[MAX_NODES][MAX_NODES];      /* dist[i][j] es la distancia entre i y j */

void shortest_path(int s, int t, int path[ ])
{ struct state {                        /* la trayectoria con la que se está trabajando */
    int predecessor;                    /* nodo previo */
    int length;                         /* longitud del origen a este nodo */
    enum {permanent, tentative} label;  /* estado de la etiqueta */
  } state[MAX_NODES];

  int i, k, min;
  struct state *
          p;
  for (p = &state[0]; p < &state[n]; p++) {     /* estado de inicialización */
    p->predecessor = -1;
    p->length = INFINITY;
    p->label = tentative;
  }
  state[t].length = 0; state[t].label = permanent;
  k = t;                                /* k es el nodo de trabajo inicial */
  do {                                  /* ¿hay una trayectoria mejor desde k? */
    for (i = 0; i < n; i++)             /* este grafo tiene n nodos */
        if (dist[k][i] != 0 && state[i].label == tentative) {
            if (state[k].length + dist[k][i] < state[i].length) {
                state[i].predecessor = k;
                state[i].length = state[k].length + dist[k][i];
            }
        }

    /* Encuentra el nodo etiquetado tentativamente con la etiqueta menor. */
    k = 0; min = INFINITY;
    for (i = 0; i < n; i++)
        if (state[i].label == tentative && state[i].length < min) {
            min = state[i].length;
            k = i;
        }
    state[k].label = permanent;
  } while (k != s);

  /* Copia la trayectoria en el arreglo de salida. */
  i = 0; k = s;
  do {path[i++] = k; k = state[k].predecessor; } while (k >= 0);
}
```

Figura 5-7. Algoritmo de Dijkstra para calcular la trayectoria más corta a través de un grafo.

5.2.4. Enrutamiento basado en flujo

Los algoritmos estudiados hasta ahora sólo toman en cuenta la topología; no consideran la carga. Si, por ejemplo, siempre hay una gran cantidad de tráfico entre *A* y *B* en la figura 5-6, entonces podría ser mejor enrutar el tráfico de *A* a *C* a través de *AGEFC*, aun cuando esta trayectoria es mucho más larga que *ABC*. En esta sección veremos un algoritmo estático que usa tanto la topología como la carga para el enrutamiento. Su nombre es **enrutamiento basado en flujo**.

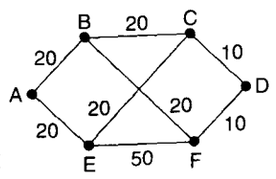
En algunas redes, la tasa media de flujo de datos entre cada par de nodos es relativamente estable y predecible. Por ejemplo, en una red corporativa de una cadena de tiendas de menudeo, cada tienda puede enviar pedidos, informes de ventas, actualizaciones de inventario y otros tipos de mensajes bien definidos a sitios conocidos siguiendo un patrón predefinido, por lo que el volumen total de tráfico varía poco de día a día. En las condiciones en que se conoce por adelantado el tráfico promedio de i a j y es razonablemente constante en el tiempo, es posible analizar matemáticamente los flujos para optimar el enrutamiento.

La idea en que se basa el análisis es que, para una línea dada, si se conocen la capacidad y el flujo promedio es posible calcular el retardo promedio de los paquetes en esa línea a partir de la teoría de colas. De los retardos promedio de todas las líneas, es directo el cálculo de un promedio ponderado por el flujo para obtener el retardo de paquete medio de la subred completa. El problema de enrutamiento se reduce entonces a encontrar el algoritmo de enrutamiento que produzca el retardo promedio mínimo para la subred. Véase la figura 5-8.

Para usar esta técnica, debe conocerse por adelantado cierta información. Primero, debe conocerse la topología de la subred. Segundo, debe estar dada la matriz de tráfico, F_{ij}. Tercero, debe estar disponible la matriz de capacidad, C_{ij}, donde se especifica la capacidad de cada línea en bps. Por último, debe escogerse un algoritmo tentativo de enrutamiento.

Como ejemplo de este método, considere la subred dúplex integral de la figura 5-8(a). Los pesos de los arcos dan las capacidades, C_{ij}, en cada dirección, medidas en kbps. La matriz de la figura 5-8(b) tiene una entrada por cada par origen-destino. La entrada par al origen i al destino j muestra la ruta a usar para el tráfico i-j y también la cantidad de paquetes/seg a enviar del origen i al destino j. Por ejemplo, de *B* a *D* van 3 paquetes/seg, y usan la ruta *BFD* para llegar ahí. Nótese que ya se ha aplicado un algoritmo de enrutamiento para obtener las rutas mostradas en la matriz.

Dada esta información, es directo el cálculo del total de la línea i, λ_i. Por ejemplo, el tráfico *B-D* contribuye 3 paquetes/seg a la línea *BF*, y también 3 paquetes/seg a la línea *FD*. De la misma manera, el tráfico *A-D* contribuye 1 paquete/seg a cada una de las tres líneas. En la columna λ_i de la figura 5-9 se muestra el tráfico total de cada línea dirigida al este. En este ejemplo, todo el tráfico es simétrico, es decir, el tráfico *XY* es idéntico al tráfico *YX*, para todas las *X* y *Y*. En las redes reales, esta condición no siempre se mantiene. En la figura también se muestra el número medio de paquetes/seg de cada línea, μC_i, suponiendo un tamaño medio de paquete de $1/\mu = 800$ bits.



(a)

Destino

Origen	A	B	C	D	E	F
A		9 AB	4 ABC	1 ABFD	7 AE	4 AEF
B	9 BA		8 BC	3 BFD	2 BFE	4 BF
C	4 CBA	8 CB		3 CD	3 CE	2 CEF
D	1 DFBA	3 DFB	3 DC		3 DCE	4 DF
E	7 EA	2 EFB	3 EC	3 ECD		5 EF
F	4 FEA	4 FB	2 FEC	4 FD	5 FE	

(b)

Figura 5-8. (a) Subred con capacidades de línea mostradas en kbps. (b) El tráfico en paquetes/seg y la matriz de enrutamiento.

La penúltima columna de la figura 5-9 indica el retardo medio por línea, derivado de la fórmula de la teoría de colas

$$T = \frac{1}{\mu C - \lambda}$$

donde $1/\mu$ es el tamaño medio del paquete en bits, C es la capacidad en bps y λ es el flujo medio en paquetes/seg. Por ejemplo, con una capacidad $\mu C = 25$ paquetes/seg y un flujo real de $\lambda = 14$ paquetes/seg, el retardo medio es de 91 mseg. Nótese que, con $\lambda = 0$, el retardo medio aún es de 40 mseg, porque la capacidad es de 25 paquetes/seg. En otras palabras, el "retardo" incluye tanto encolamiento como tiempo de servicio.

Para calcular el tiempo de retardo medio de la subred completa, tomamos la suma ponderada de cada una de las ocho líneas, siendo el peso la fracción del total de tráfico que usa esa línea. En este ejemplo, la media resulta ser de 86 mseg.

Para evaluar un algoritmo de enrutamiento diferente, podemos repetir el proceso completo, sólo que con diferentes flujos, para obtener un retardo promedio nuevo. Si nos restringimos a algoritmos de enrutamiento de una trayectoria, como hemos hecho hasta ahora, hay un número finito de maneras de enrutar paquetes de cada de origen a cada destino. Siempre es posible escribir un programa que sencillamente las pruebe todas, una tras otra, y encontrar la que tiene el retardo medio menor. Dado que este cálculo puede hacerse por adelantado fuera de línea, el que pueda consumir tiempo no es necesariamente un problema serio. Éste es entonces el mejor algoritmo de enrutamiento. Bertsekas y Gallager (1992) explican detalladamente el enrutamiento basado en flujo.

i	Línea	λ_i (paq./seg)	C_i (kbps)	μC_i (paq./seg)	T_i (mseg)	Peso
1	AB	14	20	25	91	0.171
2	BC	12	20	25	77	0.146
3	CD	6	10	12.5	154	0.073
4	AE	11	20	25	71	0.134
5	EF	13	50	62.5	20	0.159
6	FD	8	10	12.5	222	0.098
7	BF	10	20	25	67	0.122
8	EC	8	20	25	59	0.098

Figura 5-9. Análisis de la subred de la figura 5-8 usando un tamaño medio de paquete de 800 bits. El tráfico en reversa (*BA*, *CB*, etc.) es igual al tráfico directo.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE

5.2.5. Enrutamiento por vector de distancia

Las computadoras modernas generalmente usan algoritmos de enrutamiento dinámico en lugar de los estáticos antes descritos. En particular, dos algoritmos dinámicos, el enrutamiento por vector de distancia y el enrutamiento por estado de enlace, son los más comunes. En esta sección veremos el primer algoritmo. En la siguiente estudiaremos el segundo.

Los algoritmos de **enrutamiento por vector de distancia** operan haciendo que cada enrutador mantenga una tabla (por ejemplo, un vector) que da la mejor distancia conocida a cada destino y la línea a usar para llegar ahí. Estas tablas se actualizan intercambiando información con los vecinos.

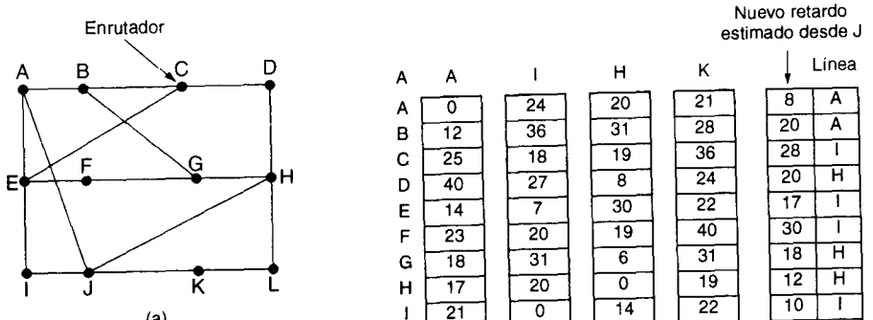
El algoritmo de enrutamiento por vector de distancia a veces recibe otros nombres, incluido el de algoritmo de enrutamiento **Bellman-Ford** distribuido y el de algoritmo **Ford-Fulkerson**, por los investigadores que lo desarrollaron (Bellman, 1957, y Ford y Fulkerson, 1962). Éste fue el algoritmo original de ARPANET y también se usó en Internet con el nombre RIP y en las versiones tempranas de DECnet y el IPX de Novell. Los enrutadores AppleTalk y Cisco usan protocolos por vector de distancia mejorados.

En el enrutamiento por vector de distancia, cada enrutador mantiene una tabla de enrutamiento indizada por, y conteniendo un registro de, cada enrutador de la subred. Esta entrada comprende dos partes: la línea preferida de salida hacia ese destino y una estimación del tiempo o distancia a ese destino. La métrica usada podría ser la cantidad de escalas, el retardo de tiempo en milisegundos, el número total de paquetes encolados por la trayectoria, o algo parecido.

Se supone que el enrutador conoce la "distancia" a cada uno de sus vecinos. Si la métrica es de escalas, la distancia simplemente es una escala. Si la métrica es la longitud de la cola, el enrutador simplemente examina cada cola. Si la métrica es el retardo, el enrutador puede medirlo directamente con paquetes especiales de ECO que el receptor simplemente marca con la hora y envía de regreso tan rápido como puede.

Por ejemplo, supóngase que se usa como métrica el retardo y que el enrutador conoce el retardo a cada uno de sus vecinos. Una vez cada T mseg, cada enrutador envía a todos sus vecinos una lista de sus retardos estimados a cada destino. También recibe una lista parecida de cada vecino. Imagine que una de estas tablas acaba de llegar del vecino X, siendo X_i la estimación de X respecto al tiempo que le toma llegar al enrutador i. Si el enrutador sabe que el retardo a X es de m mseg, también sabe que puede alcanzar el enrutador i a través de X en $X_i + m$ mseg vía X. Efectuando este cálculo para cada vecino, un enrutador puede encontrar la estimación que parezca ser la mejor y usar esa estimación y la línea correspondiente en su nueva tabla de enrutamiento. Nótese que la vieja tabla de enrutamiento no se usa en este cálculo.

Este proceso de actualización se ilustra en la figura 5-10. En la parte (a) se muestra una subred. En las primeras cuatro columnas de la parte (b) aparecen los vectores de retardo recibidos de los vecinos del enrutador J. A indica tener un retardo de 12 mseg a B, un retardo de 25 mseg a C, un retardo de 40 mseg a D, etc. Supóngase que J ha medido o estimado el retardo a sus miembros, A, I, H y K en 8, 10, 12 y 6 mseg, respectivamente.



(a)

	A	I	H	K	Nuevo retardo estimado desde J	Línea
A	0	24	20	21	8	A
B	12	36	31	28	20	A
C	25	18	19	36	28	I
D	40	27	8	24	20	H
E	14	7	30	22	17	I
F	23	20	19	40	30	I
G	18	31	6	31	18	H
H	17	20	0	19	12	H
I	21	0	14	22	10	I
J	9	11	7	10	0	–
K	24	22	22	0	6	K
L	29	33	9	9	15	K

Retardo JA es de 8 | Retardo JI es de 10 | Retardo JH es de 12 | Retardo JK es de 6 | Nueva tabla de enrutamiento para J

Vectores recibidos de los cuatro vecinos de J

(b)

Figura 5-10. (a) Subred. (b) Entrada de A, I, H, K, y la nueva tabla de enrutamiento de J.

Considere la manera en que J calcula su nueva ruta al enrutador G. Sabe que puede llegar a A en 8 mseg, y A indica ser capaz de llegar a G en 18 mseg, por lo que J sabe que puede contar con un retardo de 26 mseg a G si reenvía a través de A los paquetes destinados a G. Del mismo modo, J calcula el retardo a G a través de I, H y K en 41 (31 + 10), 18 (6 + 12) y 37 (31 + 6) mseg, respectivamente. El mejor de estos valores es 18, por lo que escribe una entrada en su tabla de enrutamiento indicando que el retardo a G es de 18 mseg, y que la ruta a usar es vía H. Se lleva

a cabo el mismo cálculo para los demás destinos, y la nueva tabla de enrutamiento se muestra en la última columna de la figura.

El problema del conteo a infinito

El enrutamiento por vector de distancia funciona en teoría, pero tiene un problema serio en la práctica: aunque converge en la respuesta correcta, puede hacerlo lentamente. En particular, reacciona con rapidez a las buenas noticias, pero con lentitud ante las malas. Considere un enrutador cuya mejor ruta al destino X es larga. Si en el siguiente intercambio el vecino A informa repentinamente un retardo corto a X, el enrutador simplemente se conmuta a modo de usar la línea a A para enviar tráfico hasta X. En un intercambio de vectores, se procesan las buenas noticias.

Para ver la rapidez de propagación de las buenas noticias, considere la subred de cinco nodos (lineal) de la figura 5-11, en la que la métrica de retardo es el número de escalas. Supóngase que A está desactivado inicialmente y que los otros enrutadores lo saben. En otras palabras, habrán registrado como infinito el retardo a A.

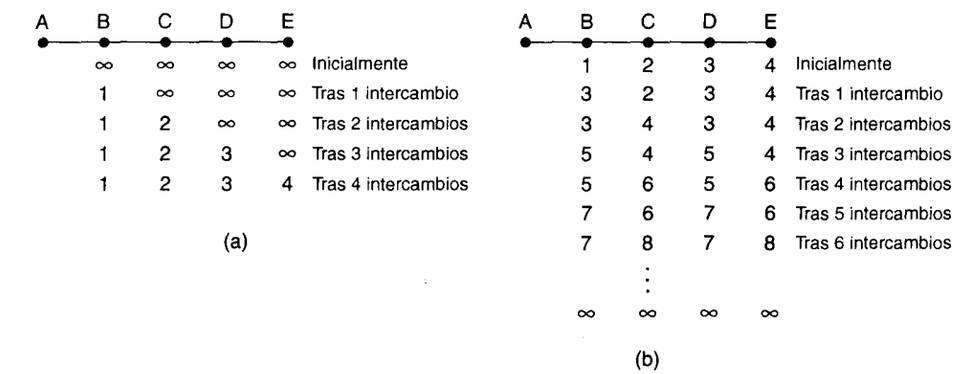


Figura 5-11. El problema de conteo a infinito.

Al activarse A, los demás enrutadores saben de él gracias a los intercambios de vectores. Por sencillez, supondremos que hay un gong gigantesco en algún lado, golpeado periódicamente para iniciar simultáneamente un intercambio de vectores entre todos los enrutadores. En el momento del primer intercambio, B se entera de que su vecino de la izquierda tiene un retardo de 0 hacia A. B crea entonces una entrada en su tabla de enrutamiento, indicando que A está a una escala de distancia hacia la izquierda. Los demás enrutadores aún piensan que A está desactivado. Las entradas de la tabla de enrutamiento de A en este punto se muestran en la segunda fila de la figura 5-11(a). Durante el siguiente intercambio, C se entera de que B tiene una trayectoria a A de longitud 1, por lo que actualiza su tabla de enrutamiento para indicar una trayectoria de longitud 2, pero D y E no se enteran de las buenas nuevas sino hasta después. Como es evidente, las buenas noticias se difunden a razón de una escala por intercambio. En una subred cuya

trayectoria mayor tiene una longitud de *N* escalas, en un lapso de *N* intercambios todo mundo sabrá sobre las líneas y enrutadores recientemente revividos.

Ahora consideremos la situación de la figura 5-11(b), en la que todas las líneas y enrutadores están activos inicialmente. Los enrutadores *B, C, D* y *E* tienen distancias a *A* de 1, 2, 3 y 4, respectivamente. De pronto, *A* se desactiva, o bien se corta la línea entre *A* y *B*, que de hecho es la misma cosa desde el punto de vista de *B*.

En el primer intercambio de paquetes, *B* no escucha nada de *A*. Afortunadamente, *C* dice: "No te preocupes. Tengo una trayectoria a *A* de longitud 2". *B* no sabe que la trayectoria de *C* pasa a través de *B* mismo. Hasta donde *B* sabe, *C* puede tener 10 líneas de salida, todas con trayectorias independientes a *A* de longitud 2. Como resultado, *B* ahora piensa que puede llegar a *A* por medio de *C*, con una longitud de trayectoria de 3. *D* y *E* no actualizan sus entradas para *A* en el primer intercambio.

En el segundo intercambio, *C* nota que cada uno de sus vecinos indica tener una trayectoria a *A* de longitud 3. *C* escoge una de ellas al azar y hace que su nueva distancia a *A* sea de 4, como se muestra en el tercer renglón de la figura 5-11(b). Los intercambios subsecuentes producen la historia mostrada en el resto de la figura 5-11(b).

A partir de esta figura, debe quedar clara la razón por la que las malas noticias viajan con lentitud: ningún enrutador tiene jamás un valor mayor en más de una unidad que el mínimo de todos sus vecinos. Gradualmente, todos los enrutadores elevan sus cuentas hacia el infinito, pero el número de intercambios requerido depende del valor numérico usado para el infinito. Por esta razón, es prudente hacer infinito igual a la trayectoria más larga, más 1. Si la métrica es el retardo de tiempo, no hay un límite superior bien definido, por lo que se necesita un valor alto para evitar tratar a una trayectoria con un retardo grande como si estuviera desactivada. Este problema se conoce como el problema de **conteo a infinito**, lo cual no es del todo sorprendente.

Recorte por horizonte dividido

En la literatura se han propuesto muchas soluciones específicas al problema del conteo a infinito, cada una más complicada y menos útil que la anterior. Describiremos una de ellas e indicaremos también por qué falla. El algoritmo de **horizonte dividido** funciona de la misma manera que el enrutamiento por vector de distancia, excepto que la distancia a *X* no se informa en la línea por la que se envían paquetes para *X* (en realidad, se informa como infinita). En el estado inicial de la figura 5-11(b), por ejemplo, *C* le dice a *D* la verdad sobre la distancia a *A*, pero *C* le dice a *B* que la distancia a *A* es infinita. De la misma manera, *D* le dice la verdad a *E*, pero le miente a *C*.

Veamos ahora lo que pasa cuando *A* se desactiva. En el primer intercambio, *B* descubre que la línea directa desapareció, y *C* está informando también una distancia infinita a *A*. Dado que ninguno de sus vecinos puede llegar a *A*, *B* establece también su distancia como infinita. En el siguiente intercambio, *C* escucha que *A* es inalcanzable desde sus dos vecinos, por lo que también marca a *A* como inalcanzable. Usando el horizonte dividido, las malas noticias se propagan a razón de una escala por intercambio. Esta velocidad es mucho mejor que sin el horizonte dividido.

La verdadera mala noticia es que el horizonte dividido, aunque se utiliza ampliamente, a veces falla. Considere, por ejemplo, la subred de cuatro nodos de la figura 5-12. Inicialmente, tanto *A* como *B* tienen una distancia a *D* de 2, y *C* tiene una distancia de 1 al mismo lugar.

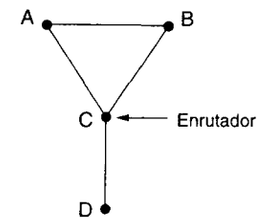


Figura 5-12. Ejemplo de dónde falla el horizonte dividido.

Ahora suponga que se desactiva la línea *CD*. Usando el horizonte dividido, tanto *A* como *B* le indican a *C* que no pueden llegar a *D*. Por tanto, *C* concluye de inmediato que *D* es inalcanzable y lo informa tanto a *A* como a *B*. Desafortunadamente *A* escucha que *B* tiene una trayectoria de longitud 2 a *D*, por lo que supone que puede llegar a *D* a través de *B* en tres escalas. De manera parecida, *B* concluye que puede llegar a *D* en tres escalas a través de *A*. En el siguiente intercambio, cada uno establece en 4 su distancia a *D*. Ambos cuentan gradualmente a infinito, que es precisamente el comportamiento que estamos tratando de evitar.

5.2.6. Enrutamiento por estado de enlace

El enrutamiento por vector de distancia se usó en ARPANET hasta 1979, cuando fue reemplazado por el enrutamiento por estado de enlace. Dos problemas principales causaron su defunción. Primero, dado que la métrica de retardo era la longitud de la cola, no tomaba en cuenta el ancho de banda al escoger rutas. Inicialmente, todas las líneas eran de 56 kbps, por lo que el ancho de banda no era importante, pero una vez que se modernizaron algunas líneas a 230 kbps y otras a 1.544 Mbps, el no tomar en cuenta el ancho de banda se volvió un problema importante. Por supuesto, habría sido posible cambiar la métrica de retardo para considerar también el ancho de banda, pero existía un segundo problema, que el algoritmo con frecuencia tardaba demasiado en convergir, aun con trucos como el horizonte dividido. Por estas razones, el algoritmo fue reemplazado por uno completamente nuevo, llamado **enrutamiento por estado de enlace**. Hoy en día se usan ampliamente variantes del enrutamiento por estado de enlace.

El concepto en que se basa el enrutamiento por estado de enlace es sencillo y puede postularse en cinco partes. Cada enrutador debe

1. Descubrir a sus vecinos y conocer sus direcciones de red.
2. Medir el retardo o costo para cada uno de sus vecinos.
3. Construir un paquete que indique todo lo que acaba de aprender.
4. Enviar este paquete a todos los demás enrutadores.
5. Calcular la trayectoria más corta a todos los demás enrutadores.

De hecho, la topología total y todos los retardos se miden experimentalmente y se distribuyen a cada enrutador. Entonces puede usarse el algoritmo de Dijkstra para encontrar la trayectoria más corta a todos los demás enrutadores. A continuación veremos con mayor detalle estos cinco pasos.

Conocimiento de los vecinos

Al ponerse en operación un enrutador, su primera tarea es averiguar quiénes son sus vecinos; esto lo logra enviando un paquete especial de HOLA (*HELLO*) por cada línea punto a punto. Se espera que el enrutador del otro extremo envíe de regreso una respuesta indicando quién es. Estos nombres deben ser globalmente únicos puesto que, cuando un enrutador distante escucha después que tres enrutadores están conectados a *F*, es indispensable que pueda determinar si los tres se refieren al mismo *F*.

Al conectarse dos o más enrutadores mediante una LAN, la situación es ligeramente más complicada. En la figura 5-13(a) se ilustra una LAN a la que están conectados directamente tres enrutadores, *A*, *C* y *F*. Cada uno de estos enrutadores está conectado a uno o más enrutadores adicionales, como se muestra.

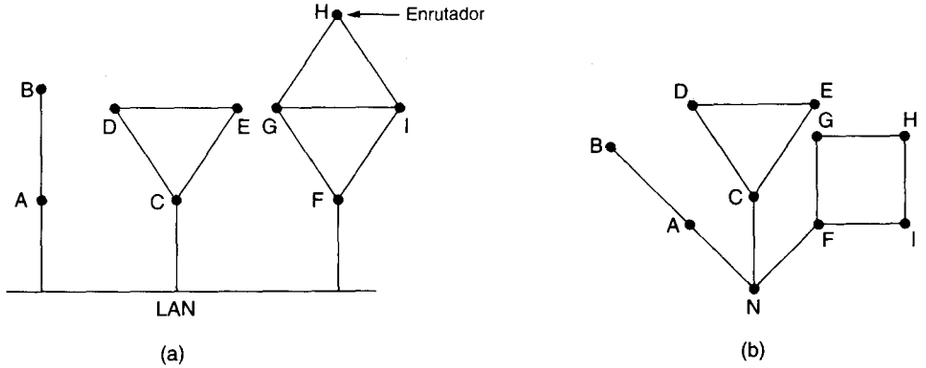


Figura 5-13. (a) Nueve enrutadores y una LAN. (b) Modelo de grafo de (a).

Una manera de modelar la LAN es considerarla como otro nodo, como se muestra en la figura 5-13(b). Aquí hemos introducido un nodo artificial nuevo, *N*, al que están conectados *A*, *C* y *F*. El hecho de que sea posible ir de *A* a *C* a través de la LAN se representa aquí mediante la trayectoria *ANC*.

Medición del costo de la línea

El algoritmo de enrutamiento por estado de enlace requiere que cada enrutador sepa, o cuando menos tenga una idea razonable, del retardo a cada uno de sus vecinos. La manera más directa de

determinar este retardo es enviar un paquete especial ECO (*ECHO*) a través de la línea, el cual debe enviar de regreso inmediatamente el otro lado. Si mide el tiempo de ida y vuelta y lo divide entre dos, el enrutador transmisor puede tener una idea razonable del retardo. Para obtener mejores resultados aún, la prueba puede llevarse a cabo varias veces y usarse el promedio.

Un asunto interesante es si se debe tomar en cuenta la carga al medir el retardo. Para considerar la carga, el temporizador de viaje redondo debe iniciarse al encolar el paquete de ECO. Si se desea ignorar la carga, el temporizador debe iniciarse cuando el paquete de ECO llega al frente de la cola.

Pueden citarse argumentos a favor en ambos sentidos. La inclusión de los retardos inducidos por el tráfico en las mediciones implica que, cuando un enrutador puede escoger entre dos líneas con el mismo ancho de banda, una con carga alta continua y otra sin ella, considerará como trayectoria más corta la ruta a través de la línea sin carga. Esta selección resultará en un mejor desempeño.

Desafortunadamente, también hay un argumento en contra de la inclusión de la carga en el cálculo del retardo. Considere la subred de la figura 5-14, dividida en dos partes, este y oeste, conectadas por dos líneas, *CF* y *EI*. Suponga que la mayor parte del tráfico entre el este y el oeste usa la línea *CF* y, como resultado, esta línea tiene tráfico alto con retardos grandes. La inclusión del retardo por encolamiento en el cálculo de la trayectoria más corta hará más atractiva a *EI*. Una vez instaladas las nuevas tablas de enrutamiento, la mayor parte del tráfico este-oeste pasará ahora por *EI*, carga extrando esta línea. En consecuencia, en la siguiente actualización, *CF* aparecerá como la trayectoria más corta. Como resultado, las tablas de enrutamiento pueden oscilar sin control, conduciendo a un enrutamiento errático y muchos problemas potenciales. Si se ignora la carga y sólo se considera el ancho de banda, no ocurre este problema. De manera alterna, puede distribuirse la carga entre ambas líneas, pero esta solución no aprovecha al máximo la mejor trayectoria.

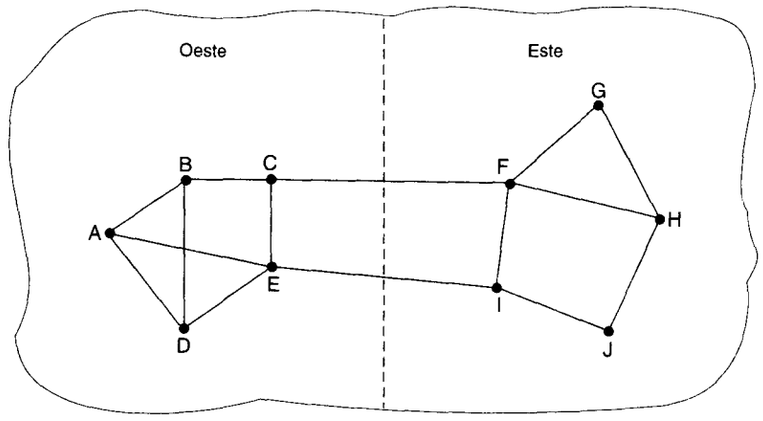


Figura 5-14. Subred en la que las partes este y oeste están conectadas por dos líneas.

Construcción de los paquetes de estado de enlace

Una vez que se ha recabado la información necesaria para el intercambio, el siguiente paso es que cada enrutador construya un paquete con todos los datos. El paquete comienza con la identidad del transmisor, seguida de un número de secuencia, una edad (que se describirá después) y una lista de vecinos. Para cada vecino, se cita el retardo a ese vecino. En la figura 5-15(a) se da un ejemplo de subred, mostrándose el retardo en las líneas. Los paquetes de estado de enlace de los seis enrutadores se muestran en la figura 5-15(b).

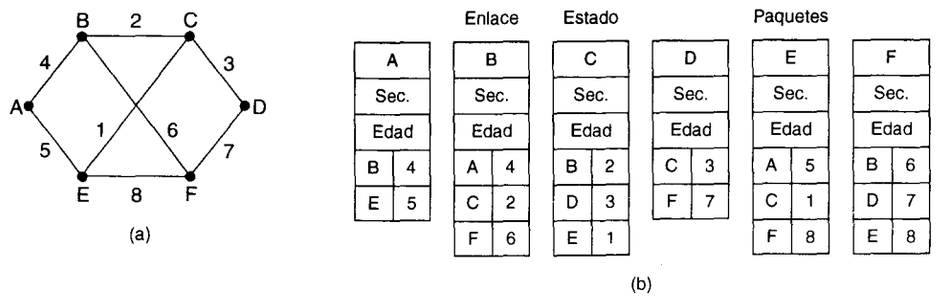


Figura 5-15. (a) Subred. (b) Paquetes de estado de enlace para esta subred.

Es fácil construir los paquetes de estado de enlace. La parte difícil es determinar cuándo construirlos. Una posibilidad es construirlos periódicamente, es decir, a intervalos regulares. Otra posibilidad es al ocurrir un evento significativo, como la caída o reactivación de una línea o de un vecino, o el cambio apreciable de sus propiedades.

Distribución de los paquetes de estado de enlace

La parte más complicada del algoritmo es la distribución confiable de los paquetes de estado de enlace. A medida que se distribuyen e instalan los paquetes, los enrutadores que reciban los primeros cambiarán sus rutas. En consecuencia, los distintos enrutadores podrían estar usando versiones diferentes de la topología, lo que puede conducir a inconsistencias, ciclos, máquinas inalcanzables y otros problemas.

Primero describiremos el algoritmo básico de distribución y luego lo refinaremos. La idea fundamental es usar inundación para distribuir los paquetes de estado de enlace. A fin de mantener controlada la inundación, cada paquete contiene un número de secuencia que se incrementa con cada paquete nuevo enviado. Los enrutadores llevan el registro de todos los pares (enrutador de origen, secuencia) que ven. Al llegar un paquete de estado de enlace, se revisa contra la lista de paquetes ya vistos. Si es nuevo, se reenvía a través de todas las líneas, excepto aquella por la que llegó. Si es un duplicado, se descarta. Si llega un paquete con

número de secuencia menor que el mayor visto hasta el momento, se rechaza como obsoleto.

Este algoritmo tiene algunos problemas, pero son manejables. Primero, si los números de secuencia vuelven a comenzar, reinará la confusión. La solución aquí es usar un número de secuencia de 32 bits. Con un paquete de estado de enlace por segundo, el tiempo para volver a empezar será de 137 años, por lo que puede ignorarse esta posibilidad.

Segundo, si llega a caerse un enrutador, perderá el registro de su número de secuencia. Si comienza nuevamente en 0, se rechazará como duplicado el siguiente paquete.

Tercero, si llega a corromperse un número de secuencia y se recibe 65,540 en lugar de 4 (un error de 1 bit), los paquetes 5 a 65,540 serán rechazados como obsoletos, dado que se piensa que el número de secuencia actual es 65,540.

La solución a todos estos problemas es incluir la edad de cada paquete después del número de secuencia y disminuirla una vez cada segundo. Al llegar la edad a cero, se descarta la información de ese enrutador. Normalmente, entra un paquete nuevo cada 10 minutos, digamos, por lo que la información de los enrutadores sólo expira cuando está caído el enrutador (o se pierden seis paquetes consecutivos, evento poco probable). El campo de edad también es disminuido por cada enrutador durante el proceso inicial de inundación para asegurar que no pueda perderse ningún paquete y sobrevivir durante un periodo de tiempo indefinido (se descarta un paquete cuya edad es cero).

Algunos refinamientos de este algoritmo lo hacen más robusto. Al llegar un paquete de estado de enlace a un enrutador para ser inundado, no se encola para transmisión inmediata. En cambio, entra en un área de retención donde espera un tiempo corto. Si antes de transmitirlo entra otro paquete de estado de enlace de la misma fuente, se comparan sus números de secuencia. Si son iguales, se descarta el duplicado. Si son diferentes, se deshecha el más viejo. Como protección contra los errores en las líneas enrutador-enrutador, todos los paquetes de estado de enlace requieren reconocimiento. Al desactivarse una línea, se examina el área de retención en orden por turno circular para seleccionar un paquete o reconocimiento a enviar.

La estructura de datos usada por el enrutador *B* para la subred de la figura 5-15(a) se describe en la figura 5-16. Cada renglón aquí corresponde a un paquete de estado de enlace recién llegado, pero aún no procesado por completo. La tabla registra dónde se originó el paquete, su número de secuencia y edad, y los datos. Además, hay indicadores de transmisión y reconocimiento para cada una de las tres líneas de *B* (a *A*, *C* y *F*, respectivamente). Los indicadores de envío significan que el paquete debe enviarse a través de la línea indicada. Los indicadores de reconocimiento significan que deben reconocerse ahí.

En la figura 5-16, el paquete de estado de enlace de *A* llegó directamente, por lo que debe enviarse a *C* y *F*, y reconocerse ante *A*, como lo muestran los bits de indicación. De la misma manera, el paquete de *F* tiene que reenviarse a *A* y a *C*, y reconocerse ante *F*.

Sin embargo, la situación del tercer paquete, de *E*, es diferente; llegó dos veces, la primera a través de *EAB* y la segunda por medio de *EFB*. En consecuencia, este paquete tiene que enviarse sólo a *C*, pero reconocerse tanto ante *A* como ante *F*, como lo indican los bits.

Si llega un duplicado mientras el original aún está en el *buffer*, los bits tienen que cambiar. Por ejemplo, si llega una copia del estado de *C* desde *F* antes de reenviarse la cuarta entrada de

Origen	Sec.	Edad	Indicadores envío A	C	F	Indicadores ACK A	C	F	Datos
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

Figura 5-16. *Buffer* de paquetes para el enrutador *B* de la figura 5-15.

la tabla, cambiarán los seis bits a 100011 para indicar que el paquete debe ser reconocido ante *F*, pero no enviarse ahí.

Cálculo de las nuevas rutas

Una vez que un enrutador ha acumulado un grupo completo de paquetes de estado de enlace, puede construir el grafo de la subred completa porque todos los enlaces están representados. De hecho, cada enlace se representa dos veces, una para cada dirección. Los dos valores pueden promediarse o usarse por separado.

Ahora puede ejecutarse localmente el algoritmo de Dijkstra para construir la trayectoria más corta posible a todos los destinos. Los resultados de este algoritmo pueden instalarse en las tablas de enrutamiento, y reiniciarse la operación normal.

Para una subred con n enrutadores, cada uno de los cuales tiene k vecinos, la memoria requerida para almacenar los datos de entrada es proporcional a kn. En las subredes grandes éste puede ser un problema. También puede serlo el tiempo de cómputo. Sin embargo, en muchas situaciones prácticas, el enrutamiento por estado de enlace funciona bien.

Sin embargo, problemas con el *hardware* o el *software* pueden causar estragos con este algoritmo (lo mismo que con otros). Por ejemplo, si un enrutador indica tener una línea que no tiene, u olvida una línea que sí tiene, el grafo de la subred será incorrecta. Si un enrutador deja de reenviar paquetes, o los corrompe al hacerlo, surgirán problemas. Por último, si al enrutador se le acaba la memoria o se ejecuta mal el algoritmo de cálculo de enrutamiento, ocurrirán cosas malas. A medida que la subred crece al orden de decenas o cientos de miles de nodos, la probabilidad de falla ocasional de un enrutador deja de ser insignificante. Lo importante es tratar de limitar el daño cuando ocurra lo inevitable. Perlman (1988) estudia detalladamente estos problemas y sus soluciones.

El enrutamiento por estado de enlace se usa ampliamente en las redes actuales, por lo que son pertinentes unas pocas palabras sobre algunos protocolos que lo usan. El protocolo OSPF,

que se emplea cada vez con mayor frecuencia en Internet, usa un algoritmo de estado de enlace. Describiremos el OSPF en la sección 5.5.5.

Otro protocolo de estado de enlace importante es el **IS-IS** (*Intermediate System-Intermediate System*, **sistema intermedio-sistema intermedio**), diseñado por DECnet y adoptado por la ISO para uso con su protocolo de capa de red sin conexiones, CLNP. Desde entonces se ha modificado para manejar también otros protocolos, siendo el más notable el IP. El IS-IS se usa en varios *backbone* de Internet (incluido el viejo *backbone* NSFNET) y en muchos sistemas celulares digitales, como CDPD. El NetWare de Novell usa una variante menor del IS-IS (NLSP) para el enrutamiento de paquetes IPX.

Básicamente, IS-IS distribuye una imagen de la topología de enrutadores, a partir de la cual se calculan las trayectorias más cortas. Cada enrutador anuncia, en su información de estado de enlace, las direcciones de capa de red que puede alcanzar directamente. Estas direcciones pueden ser IP, IPX, AppleTalk o cualquier otra dirección. El IS-IS inclusive puede manejar varios protocolos de red al mismo tiempo.

Muchas de las innovaciones diseñadas para IS-IS fueron adoptadas por el OSPF (OSPF se diseñó varios años después que IS-IS). Éstas incluyen un método de autoestabilización de las actualizaciones de estado de enlace por inundación, el concepto de un enrutador designado en una LAN y el método de cálculo y manejo de la división de trayectorias y métricas múltiples. En consecuencia, hay muy poca diferencia entre el IS-IS y el OSPF. La diferencia más importante es que el IS-IS está codificado de tal manera que es fácil y natural llevar simultáneamente información sobre varios protocolos de capa de red, característica que no tiene el OSPF. Esta ventaja es especialmente valiosa en ambientes multiprotocolo grandes.

5.2.7. Enrutamiento jerárquico

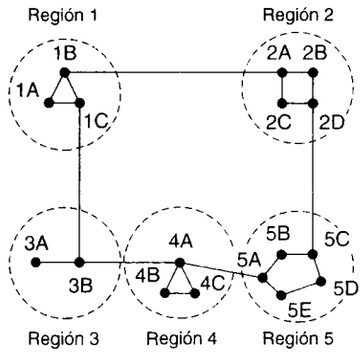
A medida que crecen en tamaño las redes, crecen proporcionalmente las tablas de enrutamiento del enrutador. Las tablas que siempre crecen no sólo consumen memoria del enrutador, sino que también se necesita más tiempo CPU para examinarlas y más ancho de banda para enviar informes de estado entre enrutadores. En cierto momento, la red puede crecer hasta el punto en que ya no es factible que cada enrutador tenga una entrada para cada uno de los demás enrutadores, por lo que el enrutamiento tendrá que hacerse jerárquicamente, como ocurre en la red telefónica.

Al usarse el enrutamiento jerárquico, los enrutadores se dividen en lo que llamaremos **regiones**, donde cada enrutador conoce todos los detalles de la manera de enrutar paquetes a destinos dentro de su propia región, pero no sabe nada de la estructura interna de las otras regiones. Al interconectar diferentes redes, es natural considerar cada una como región independiente, a fin de liberar a los enrutadores de una red de la necesidad de conocer la estructura topológica de las demás.

En las redes enormes puede ser insuficiente una jerarquía de dos niveles; puede ser necesario agrupar las regiones en cúmulos, los cúmulos en zonas, las zonas en grupos, etc., hasta que se nos agoten los nombres para los agregados. Como ejemplo de jerarquía multinivel, considere una posible forma de enrutar un paquete de Berkeley, California a Malindi, Kenya. El enrutador

de Berkeley conocería la topología detallada de California, pero podría enviar todo el tráfico exterior al enrutador de Los Ángeles. El enrutador de Los Ángeles podría enrutar el tráfico a otros enrutadores del país, pero enviaría el tráfico internacional a Nueva York. El enrutador de Nueva York tendría la programación para dirigir todo el tráfico al enrutador del país de destino encargado del manejo de tráfico internacional, digamos en Nairobi. Por último, el paquete encontraría su camino por el árbol de Kenya hasta llegar a Malindi.

En la figura 5-17 se da un ejemplo cuantitativo de enrutamiento en una jerarquía de dos niveles con cinco regiones. La tabla de enrutamiento completa para el enrutador 1*A* tiene 17 entradas, como se muestra en la figura 5-17(b). Si el enrutamiento es jerárquico, como en la figura 5-17(c), hay entradas para todos los enrutadores locales, igual que antes, pero las demás regiones se han condensado en un solo enrutador, por lo que todo el tráfico para la región 2 va a través de la línea 1*B*-2*A*, pero el resto del tráfico remoto viaja por la línea 1*C*-3*B*. El enrutamiento jerárquico redujo la tabla de 17 entradas a 7. A medida que crece la razón entre la cantidad de regiones y el número de enrutadores por región, aumentan los ahorros de espacio de tabla.



(a)

Tabla completa para 1A

Dest.	Línea	Escalas
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

Tabla jerárquica para 1A

Dest.	Línea	Escalas
1A	–	–
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

(c)

Figura 5-17. Enrutamiento jerárquico.

Desafortunadamente, estas ganancias de espacio no son gratuitas. Se paga un precio, y este precio adopta la forma de una longitud de trayectoria mayor. Por ejemplo, la mejor ruta de 1*A* a 5*C* es a través de la región 2 pero, con el enrutamiento jerárquico, todo el tráfico a la región 5 pasa por la región 3, porque eso es mejor para la mayoría de los destinos de la región 5.

Al volverse muy grande una sola red, surge una pregunta interesante: ¿cuántos niveles debe tener la jerarquía? Por ejemplo, considere una subred con 720 enrutadores. Si no hay jerarquía, cada enrutador necesita 720 tablas de enrutamiento. Si partimos la subred en 24 regiones de 30 enrutadores cada una, cada enrutador necesitará 30 entradas locales más 23 entradas remotas, para un total de 53 entradas. Si escogemos una jerarquía de tres niveles, con ocho cúmulos, cada uno de los cuales contiene 9 regiones de 10 enrutadores, cada enrutador necesita 10 entradas para los enrutadores locales, 8 entradas para el enrutamiento a otras regiones dentro de su propio cúmulo y 7 entradas para cúmulos distantes, para un total de 25 entradas. Kamoun y Kleinrock (1979) descubrieron que el número óptimo de niveles para una subred de enrutadores es de ln *N*, requiriéndose un total de *e* ln *N* entradas por enrutador. También han demostrado que el aumento en la longitud media efectiva de trayectoria causada por el enrutamiento jerárquico es lo bastante pequeña como para ser generalmente aceptable.

5.2.8. Enrutamiento para *hosts* móviles

Hoy día, millones de personas tienen computadoras portátiles, y generalmente quieren leer su correo electrónico y acceder a sus sistemas de archivos normales desde cualquier lugar del mundo. Estos *hosts* móviles generan una nueva complicación: para enrutar un paquete a un *host* móvil, la red primero tiene que encontrarlo. El tema de la incorporación de *hosts* móviles en una red es muy joven, pero en esta sección plantearemos algunos de los problemas relacionados y sugeriremos una posible solución.

El modelo del mundo que usan generalmente los diseñadores de red se muestra en la figura 5-18. Aquí tenemos una WAN que consiste en enrutadores y *hosts*. Conectadas a la WAN hay varias LAN, MAN y células inalámbricas del tipo estudiado en el capítulo 2.

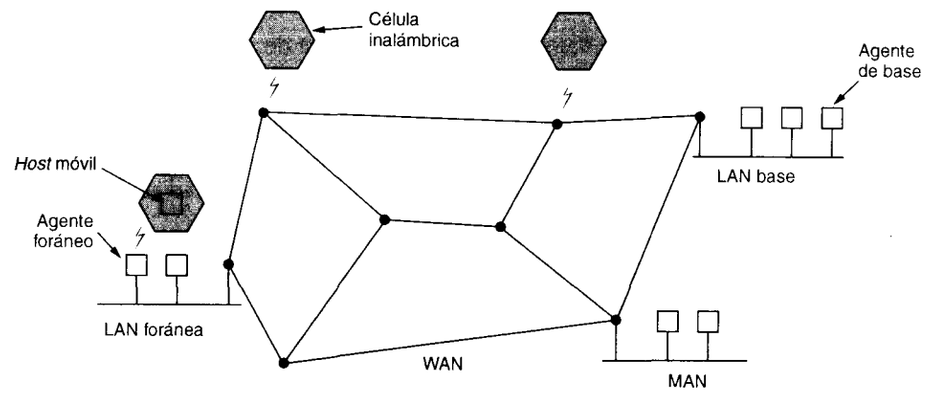


Figura 5-18. WAN a la que están conectadas LAN, MAN y células inalámbricas.

Se dice que son estacionarios los usuarios que nunca se mueven; se conectan a la red mediante alambres de cobre o fibra óptica. En contraste podemos distinguir otros dos tipos de

usuarios. Los usuarios migratorios básicamente son usuarios estacionarios que se mueven de un lugar fijo a otro de tiempo en tiempo, pero que usan la red sólo cuando están conectados físicamente a ella. Los usuarios errantes hacen su cómputo en movimiento, y quieren mantener sus conexiones mientras se mueven. Usaremos el término **usuarios móviles** para indicar cualquiera de las dos últimas categorías, es decir, todos los usuarios que están lejos de casa.

Se supone que todos los usuarios tienen una **localidad base** que nunca cambia. Los usuarios también tienen una dirección de base permanente, que puede servir para determinar su localidad base, de manera análoga a como el número telefónico 1-212-5551212 indica Estados Unidos (código de país 1) y Manhattan (212). La meta de enrutamiento en los sistemas con usuarios móviles es posibilitar el envío de paquetes a usuarios móviles usando su dirección base, y hacer que los paquetes lleguen eficientemente a ellos en cualquier lugar en el que puedan estar. Lo difícil, por supuesto, es encontrarlos.

En el modelo de la figura 5-18, el mundo se divide (geográficamente) en unidades pequeñas. Llamémoslas áreas, siendo un área típicamente una LAN o una célula inalámbrica. Cada área tiene uno o más **agentes foráneos**, que llevan el registro de todos los usuarios que visitan el área. Además, cada área tiene un **agente de base**, que lleva el registro de todos los usuarios móviles cuya base está en el área, pero que actualmente están visitando otra área.

Al entrar un usuario nuevo en un área, ya sea al conectarse a ella (por ejemplo, conectándose a la LAN), o simplemente al entrar en la célula, su computadora debe registrarse con el agente foráneo de ese lugar. El procedimiento de registro funciona típicamente de esta manera:

1. Periódicamente, cada agente foráneo difunde un paquete que anuncia su existencia y dirección. Un *host* móvil recién llegado puede esperar uno de estos mensajes, pero si no llega ninguno con suficiente rapidez, el *host* móvil puede difundir un paquete que diga: "¿hay agentes foráneos por ahí?"
2. El *host* móvil se registra con el agente foráneo, dando su dirección base, su dirección actual de capa de enlace de datos y cierta información de seguridad.
3. El agente foráneo se pone en contacto con el agente de base del *host* móvil y le dice: "uno de tus *hosts* está por aquí". El mensaje del agente foráneo al agente de base contiene la dirección de red del agente foráneo, así como la información de seguridad, para convencer al agente de base de que el *host* móvil en realidad está ahí.
4. El agente de base examina la información de seguridad, que contiene una marca de tiempo, para comprobar que fue generada en los últimos segundos. Si está conforme, indica al agente foráneo que proceda.
5. Cuando el agente foráneo recibe el reconocimiento del agente de base, hace una entrada en sus tablas e informa al *host* móvil que ahora está registrado.

Idealmente, al salir un usuario de un área, también eso debe anunciarse para permitir que se borre el registro pero, cuando terminan, muchos usuarios apagan abruptamente sus computadoras.

Al enviarse un paquete a un usuario móvil, se enruta a la LAN base del usuario, ya que eso es lo indicado en la dirección, como se ilustra en el paso 1 de la figura 5-19. Los paquetes

enviados al usuario móvil por su LAN base son interceptados por el agente de base, el cual entonces busca la localidad nueva (temporal) del usuario móvil y encuentra la dirección del agente foráneo que maneja al usuario móvil. El agente de base entonces hace dos cosas. Primero, encapsula el paquete en el campo de carga útil de un paquete exterior y envía este último al agente foráneo (paso 2 de la figura 5-19). Este mecanismo se llama proceso de túnel y lo veremos con mayor detalle después. Tras obtener el paquete encapsulado, el agente foráneo remueve el paquete original del campo de carga útil y lo envía al usuario móvil como marco de enlace de datos.

Segundo, el agente de base indica al transmisor que en lo futuro envíe paquetes al *host* móvil encapsulándolos en la carga útil de paquetes explícitamente dirigidos al agente foráneo, en lugar de simplemente enviarlos a la dirección base del usuario móvil (paso 3). Los paquetes subsiguientes ahora pueden enrutarse directamente al usuario por medio del agente foráneo (paso 4), omitiendo la localidad base por completo.

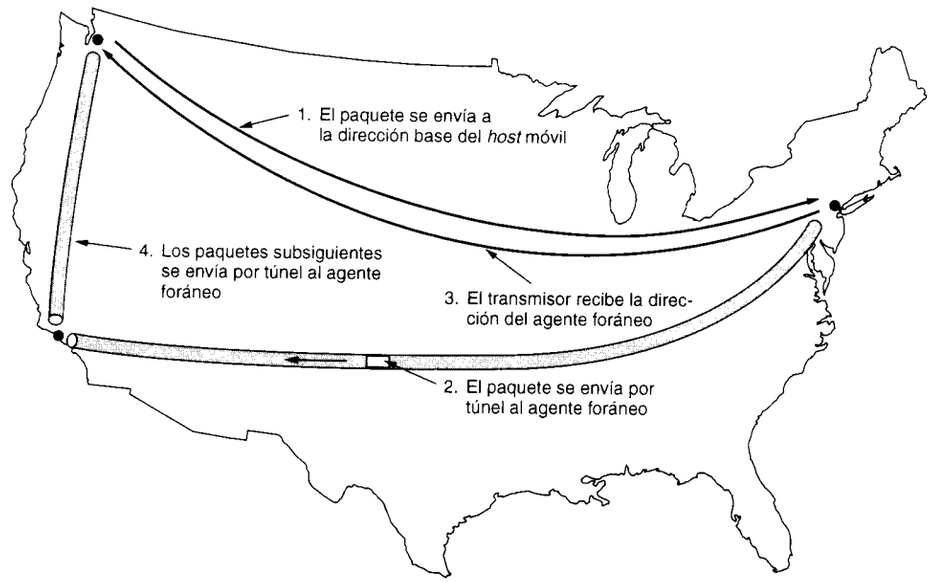


Figura 5-19. Enrutamiento de paquetes para usuarios móviles.

Los diferentes esquemas propuestos difieren en varios sentidos. Primero está el asunto de la parte del protocolo llevada a cabo por los enrutadores y la llevada a cabo por los *hosts* y, en el último caso, la capa que se encarga de ella en los *hosts*. Segundo, en unos cuantos esquemas, los enrutadores a lo largo del camino registran las direcciones en mapas, para poder interceptarlas y redirigir el tráfico aún antes de que llegue a la dirección base. Tercero, en algunos esquemas, cada visitante recibe una dirección temporal única; en otros, la dirección temporal se refiere a un agente que maneja el tráfico de todos los visitantes.

Cuarto, los esquemas difieren en la manera en que logran realmente que los paquetes dirigidos a un destino lleguen a uno diferente. Una posibilidad es cambiar la dirección de destino y simplemente retransmitir el paquete modificado. Como alternativa, el paquete completo, con dirección base y todo, puede encapsularse en la carga útil de otro paquete enviado a la dirección temporal. Por último, los esquemas difieren en sus aspectos de seguridad. Por lo general, cuando un *host* o enrutador recibe un mensaje del tipo "a partir de este momento, envíame por favor todo el correo de Genoveva", puede tener dudas acerca de con quién está hablando y de si se trata de una buena idea. Se han analizado y comparado varios protocolos de *host* móvil en (Ioannidis y Maguire, 1993; Myles y Skellern, 1993; Perkins, 1993; Teraoka *et al.*, 1993, y Wada *et al.*, 1993).

5.2.9. Enrutamiento por difusión

En algunas aplicaciones, los *hosts* necesitan enviar mensajes a varios otros *hosts* o a todos los demás. Por ejemplo, el servicio de distribución de informes ambientales, la actualización de los precios de la bolsa o los programas de radio en vivo podrían funcionar mejor difundiéndolos a todas las máquinas y dejando que aquellas interesadas lean los datos. El envío simultáneo de un paquete a todos los destinos se llama **difusión**; se han propuesto varios métodos para llevarla a cabo.

Un método de difusión que no requiere características especiales de la subred es que el origen simplemente envíe copias del paquete a todos los destinos. El método no sólo desperdicia ancho de banda, sino que también requiere que el origen tenga una lista completa de todos los destinos. En la práctica, ésta puede ser la única posibilidad, pero es el método menos deseable.

La inundación es otro candidato obvio. Aunque la inundación es poco adecuada para la comunicación punto a punto ordinaria, para difusión puede merecer consideración seria, especialmente si no es aplicable ninguno de los métodos descritos a continuación. El problema de la inundación como técnica de difusión es el mismo que tiene como algoritmo de enrutamiento punto a punto: genera demasiados paquetes y consume demasiado ancho de banda.

Un tercer algoritmo es el **enrutamiento multidestino**. Con este método, cada paquete contiene una lista de destinos o un mapa de bits que indica los destinos deseados. Al llegar un paquete al enrutador, el enrutador revisa todos los destinos para determinar el grupo de líneas de salida que necesitará. (Se necesita una línea de salida si es la mejor ruta a cuando menos uno de los destinos.) El enrutador genera una copia nueva del paquete para cada línea de salida a usar, e incluye en cada paquete sólo aquellos destinos que usan la línea. En efecto, el grupo de destinos se divide entre las líneas de salida. Tras una cantidad suficiente de escalas, cada paquete llevará sólo un destino, así que puede tratarse como un paquete normal. El enrutamiento multidestino es como los paquetes con direccionamiento individual, excepto que, cuando varios paquetes deben seguir la misma ruta, uno de ellos paga la tarifa completa y los demás viajan gratis.

Un cuarto algoritmo de difusión usa explícitamente el árbol de descenso para el enrutador que inicia la difusión, o cualquier otro árbol de extensión adecuado. El **árbol de extensión** es un subgrupo de la subred que incluye todos los enrutadores pero no contiene ciclos. Si cada enrutador sabe cuáles de sus líneas pertenecen al árbol de extensión, puede copiar un paquete de

entrada difundido en todas las líneas del árbol de extensión, excepto en aquella por la que llegó. Este método hace un uso excelente del ancho de banda, generando la cantidad mínima de paquetes necesarios para llevar a cabo el trabajo. El único problema es que cada enrutador debe tener conocimiento de algún árbol de extensión para que pueda funcionar. Algunas veces está disponible esta información (por ejemplo, con el enrutamiento por estado de enlace), pero a veces no (por ejemplo, con el enrutamiento por vector de distancia).

Nuestro último algoritmo de difusión es un intento de aproximar el comportamiento del anterior, aun cuando los enrutadores no saben nada en lo absoluto sobre árboles de extensión. La idea es excepcionalmente sencilla una vez planteada. Cuando llega un paquete difundido a un enrutador, éste lo revisa para ver si llegó por la línea normalmente usada para enviar paquetes *al* origen de la difusión. De ser así, hay excelentes posibilidades de que el paquete difundido haya seguido la mejor ruta desde el enrutador y, por tanto, sea la primera copia en llegar al enrutador. Siendo éste el caso, el enrutador reenvía copias del paquete por todas las líneas, excepto aquella por la que llegó. Sin embargo, si el paquete difundido llegó por una línea diferente de la preferida para llegar al origen, se descarta el paquete como probable duplicado.

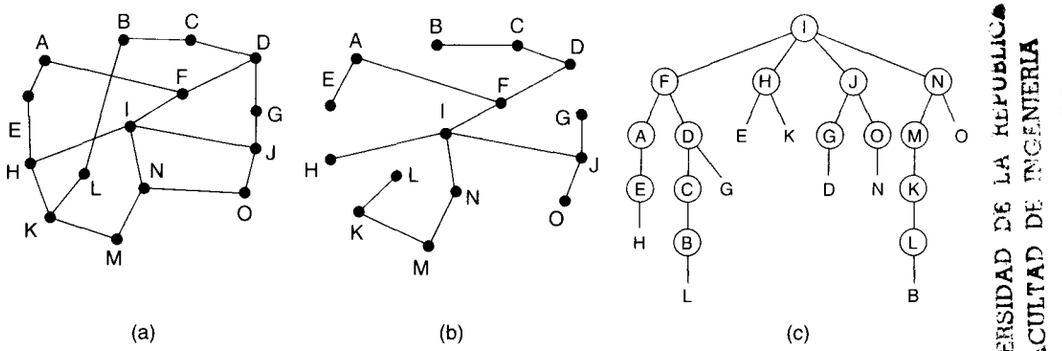


Figura 5-20. Reenvío por trayectoria invertida. (a) Subred. (b) Árbol de extensión. (c) Árbol construido mediante reenvío por trayectoria invertida.

En la figura 5-20 se muestra un ejemplo del algoritmo, llamado **reenvío por trayectoria invertida**. En la parte (a) se muestra una subred, en la parte (b) se muestra un árbol de descenso para el enrutador *I* de esa subred, y en la parte (c) se muestra el funcionamiento del algoritmo de trayectoria invertida. En la primera escala, *I* envía paquetes a *F, H, J* y *N*, como lo indica el segundo renglón del árbol. Cada uno de estos paquetes llega a *I* por la trayectoria preferida (suponiendo que la trayectoria preferida pasa a través del árbol de descenso), como lo indica el círculo alrededor de la letra. En la segunda escala, se generan ocho paquetes, dos por cada uno de los enrutadores que recibieron un paquete en la primera escala. Como resultado, los ocho llegan a enrutadores no visitados previamente, y todos menos uno llegan a través de la línea preferida. De los nueve paquetes generados en la tercera escala, sólo dos llegan por la trayectoria preferida (a *C* y *L*), así que sólo estos generan paquetes posteriores. Tras cinco escalas y 24

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE

paquetes, termina la difusión, en comparación con cuatro escalas y 14 paquetes si se hubiera seguido exactamente el árbol de descenso.

La ventaja principal del reenvío por trayectoria invertida es que es razonablemente eficiente y fácil de poner en práctica. No requiere que los enrutadores conozcan los árboles de extensión ni tiene la carga extra de una lista de destinos o de un mapa de bits en cada paquete de difusión, como los tiene el direccionamiento multidestino. Tampoco requiere mecanismos especiales para detener el proceso, como en el caso de la inundación (ya sea un contador de escalas en cada paquete y un conocimiento previo del diámetro de la subred, o una lista de paquetes ya vistos por origen).

5.2.10. Enrutamiento por multitransmisión

En algunas aplicaciones, procesos muy separados trabajan juntos en grupos; por ejemplo, un grupo de procesos que implementan una base de datos distribuida. Con frecuencia es necesario que un proceso envíe un mensaje a todos los demás miembros del grupo. Si el grupo es pequeño, se puede simplemente transmitir a todos los demás miembros un mensaje punto a punto. Si el grupo es grande, esta estrategia es cara. A veces puede usarse la difusión, pero su uso para informar a 1000 máquinas de una red que abarca un millón de nodos es ineficiente porque la mayoría de los receptores no están interesados en el mensaje (o, peor aún, definitivamente están interesados pero no deben verlo). Por tanto, necesitamos una manera de enviar mensajes a grupos bien definidos de tamaño numéricamente grande, pero pequeños en comparación con la totalidad de la red.

El envío de un mensaje a uno de tales grupos se llama **multitransmisión**, y su algoritmo de enrutamiento es el **enrutamiento por multitransmisión**. En esta sección describiremos una manera de lograr el enrutamiento por multitransmisión. Para información adicional, véase (Deering y Cheriton, 1990; Deering *et al.*, 1994, y Rajagopalan, 1992).

Para la multitransmisión se requiere administración de grupos. Se necesita alguna manera de crear y destruir grupos, y un mecanismo para que los procesos se unan a los grupos y salgan de ellos. La forma de realizar estas tareas no le concierne al algoritmo de enrutamiento. Lo que sí le concierne es que, cuando un proceso se una a un grupo, informe a su *host* del hecho. Es importante que los enrutadores sepan cuáles de sus *hosts* pertenecen a qué grupos. Los *hosts* deben informar a sus enrutadores de los cambios en los miembros del grupo, o los enrutadores deben enviar periódicamente la lista de sus *hosts*. De cualquier manera, los enrutadores aprenden qué *hosts* pertenecen a cuáles grupos. Los enrutadores les dicen a sus vecinos, de manera que la información se propaga a través de la subred.

Para hacer el enrutamiento de multitransmisión, cada enrutador calcula un árbol de extensión que cubre a todos los demás enrutadores de la subred. Por ejemplo, en la figura 5-21(a) tenemos una subred con dos grupos, 1 y 2. Algunos enrutadores están conectados a *hosts* que pertenecen a uno o ambos grupos, como se indica en la figura. En la figura 5-21(b) se muestra un árbol de extensión para el enrutador de la izquierda.

Al enviar un proceso un paquete multitransmisión a un grupo, el primer enrutador examina su árbol de extensión y lo recorta, removiendo todas las líneas que no conducen a *hosts* que son

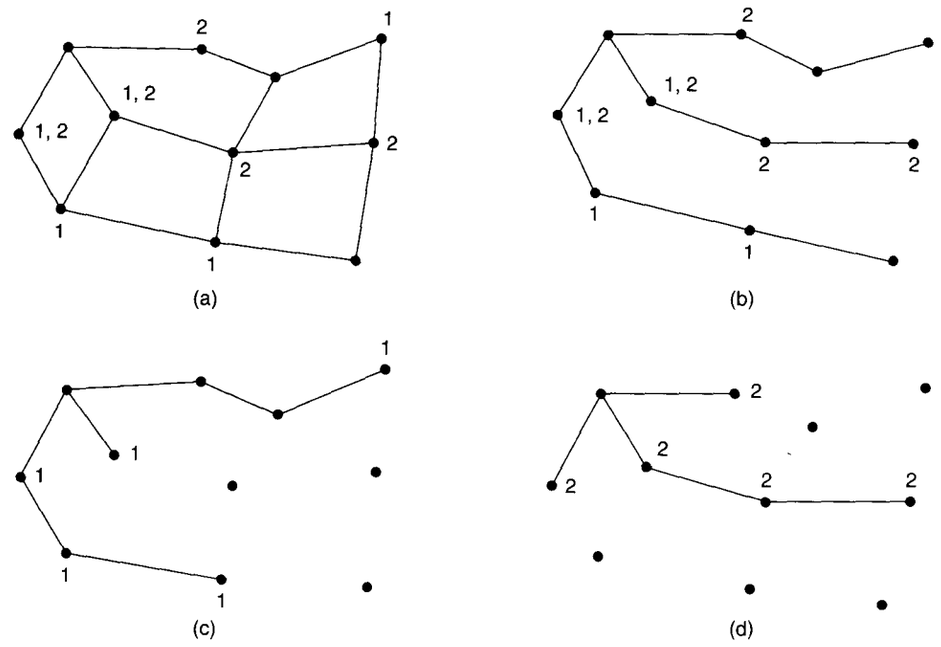


Figura 5-21. (a) Subred. (b) Árbol de extensión del enrutador del extremo izquierdo. (c) Árbol multitransmisión del grupo 1. (d) Árbol multitransmisión para el grupo 2.

miembros del grupo. En nuestro ejemplo de la figura 5-21, en la parte (c) se muestra el árbol de extensión recortado del grupo 1. De la misma manera, en la figura 5-21(d) se presenta el árbol de extensión recortado del grupo 2. Los paquetes multitransmisión se reenvían sólo a través del árbol de extensión apropiado.

Hay varias maneras de recortar el árbol de extensión. La más sencilla puede usarse si se maneja enrutamiento por estado de enlace, y cada enrutador está consciente de la topología completa de la subred, incluyendo qué *hosts* pertenecen a cuáles grupos. Entonces puede recortarse el árbol comenzando por el final de cada trayectoria y trabajando hacia la raíz, removiendo todos los enrutadores que no pertenecen al grupo en cuestión.

Con el enrutador por vector de distancia puede seguirse una estrategia de recorte diferente. El algoritmo básico es el reenvío por trayectoria invertida. Sin embargo, cuando un enrutador sin *hosts* interesados en un grupo en particular y sin conexiones con otros enrutadores recibe un mensaje de multitransmisión para ese grupo, responde con un mensaje de RECORTE (*PRUNE*), indicando al transmisor que no envíe más multitransmisiones para ese grupo. Cuando un enrutador que no tiene miembros del grupo entre sus propios *hosts* recibe uno de tales mensajes por todas sus líneas, también puede responder con un mensaje de RECORTE. De esta manera, la subred se recorta recursivamente.

Una desventaja potencial de este algoritmo es que no funciona bien en redes grandes. Supóngase que una red tiene n grupos, cada uno con un promedio de m miembros. Por cada

grupo deben almacenarse *m* árboles de extensión recortados, para un total de *mn* árboles. Si hay muchos grupos grandes, se necesitará bastante espacio para almacenar todos estos árboles.

Un diseño alterno usa **árboles de base de núcleo** (Ballardie *et al.*, 1993). Aquí se calcula un solo árbol de extensión por grupo, con la raíz (el núcleo) cerca de la mitad del grupo. Para enviar un mensaje multitransmisión, un *host* lo envía al núcleo, que entonces hace la multitransmisión a través del árbol de extensión. Aunque este árbol no será óptimo para todos los orígenes, la reducción de costos de almacenamiento de *m* árboles a un árbol por grupo representa un ahorro significativo.

5.3. ALGORITMOS DE CONTROL DE CONGESTIONAMIENTOS

Cuando hay demasiados paquetes presentes en la subred (o en una parte de ella), hay una degradación del desempeño. Esta situación se llama **congestionamiento**. En la figura 5-22 se muestra este síntoma. Cuando la cantidad de paquetes descargados en la subred por los *hosts* está dentro de su capacidad de conducción, todos se entregan (excepto unos pocos afligidos por errores de transmisión) y la cantidad entregada es proporcional al número enviado. Sin embargo, a medida que aumenta el tráfico, los enrutadores ya no pueden manejarlo y comienzan a perder paquetes. Esto tiende a empeorar las cosas. A muy alto tráfico, el desempeño se desploma por completo, y casi no hay entrega de paquetes.

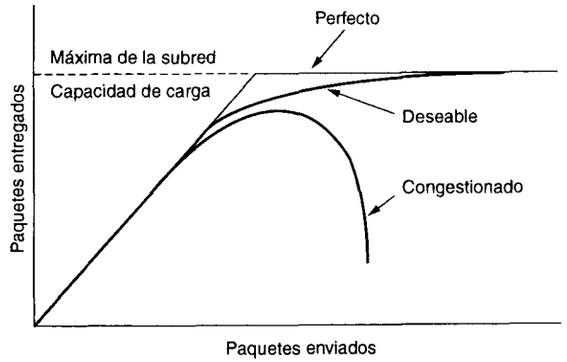


Figura 5-22. Cuando se genera demasiado tráfico, ocurren congestionamientos y se degrada marcadamente el desempeño.

El congestionamiento puede ocurrir por varias razones. Si repentinamente comienzan a llegar cadenas de paquetes por tres o cuatro líneas de entrada y todos necesitan la misma línea de salida, se generará una cola. Si no hay suficiente memoria para contenerlos a todos, se perderán paquetes. La adición de memoria puede ayudar hasta cierto punto, pero Nagle (1987) descubrió que si los enrutadores tienen una cantidad infinita de memoria, el congestionamiento empeora

en lugar de mejorar, ya que para cuando los paquetes llegan al principio de la cola su temporización ha terminado (repetidamente) y se han enviado duplicados. Todos estos paquetes serán debidamente reenviados al siguiente enrutador, aumentando la carga en todo el camino hasta el destino.

Los procesadores lentos también pueden causar congestionamientos. Si las CPU de los enrutadores son lentas para llevar a cabo las tareas de administración requeridas (*buffers* de encolamiento, actualización de tablas, etc.), pueden alargarse las colas, aun cuando haya un exceso de capacidad de línea. De la misma manera, las líneas de poco ancho de banda también pueden causar congestionamientos. La modernización de las líneas sin cambiar los procesadores, o viceversa, generalmente ayuda un poco, pero con frecuencia simplemente desplaza el cuello de botella. También la modernización de una parte del sistema, pero no de todo, simplemente mueve el cuello de botella a otra parte. El problema real con frecuencia es un mal apareamiento entre las partes del sistema. Este problema persistirá hasta que todos los componentes estén en equilibrio.

El congestionamiento tiende a alimentarse de sí mismo y empeorar. Si un enrutador no tiene *buffers* libres, debe ignorar los paquetes de entrada nuevos. Al descartarse un paquete, el enrutador transmisor (un vecino) puede terminar su temporización y retransmitirlo, tal vez muchas veces. Dado que no puede descartar el paquete mientras no sea reconocido, el congestionamiento en el lado receptor obliga al transmisor a abstenerse de liberar un *buffer* que normalmente habría desocupado. De este modo, el congestionamiento se acumula, como autos que se acercan a una caseta de cobro.

Vale la pena indicar explícitamente la diferencia entre el control de congestionamiento y el control de flujo, pues la relación es sutil. El control de congestionamiento tiene que ver con el aseguramiento de que la subred sea capaz de transportar el tráfico ofrecido. Es un asunto global, en el que interviene el comportamiento de todos los *hosts*, todos los enrutadores, el proceso de almacenamiento y reenvío dentro de los enrutadores y todos los demás factores que tienden a disminuir la capacidad de transporte de la subred.

El control de flujo, en contraste, se relaciona con el tráfico punto a punto entre un transmisor dado y un receptor dado; su tarea es asegurar que un transmisor rápido no pueda transmitir datos continuamente a una velocidad mayor que la que puede absorber el receptor. El control de flujo casi siempre comprende una retroalimentación directa del receptor al transmisor, para indicar al transmisor cómo van las cosas en el otro lado.

Para captar la diferencia entre estos dos conceptos, considere una red de fibra óptica con una capacidad de 1000 gigabits/seg en la que una supercomputadora está tratando de transferir un archivo a una computadora personal que opera a 1 Gbps. Aunque no hay congestionamiento (la red misma no está en problemas), se requiere control de flujo para obligar a la supercomputadora a detenerse con frecuencia para darle a la computadora personal un momento de respiro.

En el otro extremo, considere una red de almacenamiento y reenvío con líneas de 1 Mbps y 1000 computadoras grandes, la mitad de las cuales tratan de transferir archivos a 100 kbps a la otra mitad. Aquí el problema no es que los transmisores rápidos superen en poder a los receptores lentos, sino simplemente que el tráfico ofrecido total excede lo que la red puede manejar.

La razón por la que se confunden con frecuencia el control de congestionamiento y el control de flujo es que algunos algoritmos de control de congestionamiento operan enviando mensajes de regreso a varios orígenes, indicándoles que reduzcan su velocidad al meterse en

problemas la red. Por tanto, un *host* puede recibir un mensaje de "reducción de velocidad" tanto porque el receptor no puede manejar la carga como porque la red no la puede manejar. Regresaremos a este punto después.

Comenzaremos nuestro estudio del control de congestionamiento examinando un modelo general para manejarlo. Luego veremos enfoques generales para prevenirlo. Tras eso, veremos varios algoritmos dinámicos para manejarlo una vez que se ha establecido.

5.3.1. Principios generales del control de congestionamiento

Muchos problemas de los sistemas complejos, como las redes de computadoras, pueden verse desde el punto de vista de la teoría de control. Este enfoque conduce a la división de todas las soluciones en dos grupos: de ciclo abierto y de ciclo cerrado. Esencialmente, las soluciones de ciclo abierto intentan resolver el problema mediante un buen diseño, para asegurarse de que no ocurra desde un principio. Una vez que el sistema está operando, no se hacen correcciones a medio camino.

Las herramientas para llevar a cabo el control de ciclo abierto incluyen decidir cuándo aceptar tráfico nuevo, decidir cuándo descartar paquetes, y cuáles, y tomar decisiones de calendarización en varios puntos de la red. Todas tienen en común el hecho de que toman decisiones independientemente del estado actual de la red.

En contraste, las soluciones de ciclo cerrado se basan en el concepto de un ciclo de realimentación. Este enfoque tiene tres partes cuando se aplica al control de congestionamiento:

1. Supervisar el sistema para detectar cuándo y dónde ocurren congestionamientos.
2. Pasar esta información a lugares en los que puedan llevarse a cabo acciones.
3. Ajustar la operación del sistema para corregir el problema.

Pueden usarse varias métricas para supervisar la subred en búsqueda de congestionamientos. Las principales entre éstas son el porcentaje de paquetes descartados debido a falta de espacio de *buffer*, la longitud promedio de las colas, la cantidad de paquetes para los cuales termina la temporización, por lo que se transmiten de nuevo, el retardo promedio de los paquetes y la desviación estándar del retardo de paquete. En todos los casos, un aumento en las cifras indica un aumento en el congestionamiento.

El segundo paso del ciclo de realimentación es la transferencia de información relativa al congestionamiento desde el punto en que se detecta hasta el punto en que puede hacerse algo al respecto. La manera obvia es que el enrutador que detecta el congestionamiento envíe un paquete al origen (u orígenes) del tráfico, anunciando el problema. Por supuesto, estos paquetes extra aumentan la carga precisamente en el momento en que no se necesita más carga, es decir, cuando la subred está congestionada.

Sin embargo, existen otras posibilidades. Por ejemplo, puede reservarse un bit o campo en cada paquete para que los enrutadores lo llenen cuando el congestionamiento rebasa algún umbral. Al detectar un enrutador este estado congestionado, llena el campo de todos los paquetes de salida, para avisar a los vecinos.

Otro enfoque más es hacer que los *hosts* o enrutadores envíen periódicamente paquetes de prueba para preguntar explícitamente sobre el congestionamiento. Esta información puede usarse entonces para enrutar el tráfico alrededor de las áreas con problemas. Algunas estaciones de radio tienen helicópteros que vuelan sobre la ciudad para informar del congestionamiento en las calles, con la esperanza de que los escuchas enrutarán sus paquetes (autos) fuera de las zonas conflictivas.

En todos los esquemas de realimentación, la esperanza es que el conocimiento sobre los congestionamientos hará que los *hosts* emprendan acciones adecuadas con miras a reducir el congestionamiento. Para operar correctamente, debe ajustarse con cuidado la escala de tiempo. Si cada vez que llegan dos paquetes seguidos el enrutador grita ALTO, y cada vez que el enrutador está inactivo durante 20 μseg grita SIGA, el sistema oscilará sin control y nunca convergirá. Por otra parte, si un enrutador espera 30 minutos para asegurarse antes de decir cualquier cosa, el mecanismo de control de congestionamientos reaccionará tan lentamente que no será de utilidad. Para funcionar bien se requiere un justo medio, pero encontrar la constante de tiempo correcta no es un asunto trivial.

Se conocen muchos algoritmos de control de congestionamiento. Con objeto de organizarlos lógicamente, Yang y Reddy (1995) ha desarrollado una taxonomía de los algoritmos de control de congestionamiento; comienzan por dividir los algoritmos de ciclo abierto en algoritmos que actúan en el origen y que actúan en el destino. Los algoritmos de ciclo cerrado también se dividen en dos subcategorías: realimentación explícita contra realimentación implícita. En los algoritmos de realimentación explícita, se envían paquetes de regreso desde el punto de congestionamiento para avisar al origen. En los algoritmos implícitos, el origen deduce la existencia de un congestionamiento haciendo observaciones locales, como el tiempo necesario para el regreso de los reconocimientos.

La presencia de congestionamientos significa que la carga es (temporalmente) mayor (en una parte del sistema) que la que pueden manejar los recursos. Vienen a la mente dos soluciones: aumentar los recursos o disminuir la carga. Por ejemplo, la subred puede comenzar a usar líneas telefónicas de discado para aumentar temporalmente el ancho de banda entre ciertos puntos. En los sistemas como SMDS (véase el capítulo 1), se puede pedir a la portadora ancho de banda adicional por un rato. En los sistemas satelitales, el aumento en la potencia de transmisión frecuentemente da un mayor ancho de banda. La división del tráfico entre varias rutas en lugar de usar siempre la mejor también puede aumentar efectivamente el ancho de banda. Por último, pueden ponerse en línea enrutadores de repuesto que normalmente sirven sólo como respaldos (para hacer al sistema tolerante a fallas), a fin de contar con mayor capacidad cuando aparece un congestionamiento severo.

Sin embargo, a veces no es posible aumentar la capacidad, o ya ha sido aumentada al máximo. Entonces, la única forma de reducir el congestionamiento es disminuir la carga. Existen varias maneras de reducir la carga, que incluye negar el servicio a algunos usuarios, degradar el servicio para algunos o todos los usuarios y obligar a los usuarios a programar sus solicitudes de una manera más predecible.

Algunos de estos métodos, que estudiaremos en breve, se aplican mejor a los circuitos virtuales. En las subredes que usan circuitos virtuales internamente, estos métodos pueden usarse en la capa de red. En las subredes de datagramas algunas veces pueden utilizarse en las conexiones

de capa de transporte. En este capítulo nos enfocaremos en su uso en la capa de red. En el siguiente, veremos lo que puede hacerse en la capa de transporte para manejar los congestionamientos.

5.3.2. Políticas de prevención de congestionamientos

Comencemos nuestro estudio de los métodos de control de congestionamientos estudiando los sistemas de ciclo abierto. Estos sistemas están diseñados para reducir al mínimo el congestionamiento desde el inicio, en lugar de permitir que ocurra y reaccionar después del hecho. Los métodos tratan de lograr su meta usando políticas adecuadas en varios niveles. En la figura 5-23 vemos diferentes políticas de enlace de datos, red y transporte que pueden afectar el congestionamiento (Jain, 1990).

Capa	Políticas
Transporte	• Política de retransmisión • Política de caché de paquetes fuera de orden • Política de acuses de recibo • Política de control de flujo • Determinación de terminaciones de temporización
Red	• Circuitos virtuales *vs.* datagramas en la subred • Política de encolamiento y servicio de paquetes • Política de descartado de paquetes • Política de enrutamiento • Gestión de tiempo de vida de paquetes
Enlace de datos	• Política de retransmisiones • Política de caché de paquetes fuera de orden • Política de acuses de recibo • Política de control de flujo

Figura 5-23. Políticas que afectan el congestionamiento.

Comencemos por la capa de enlace de datos y avancemos hacia arriba. La política de retransmisiones tiene que ver con la rapidez con que termina de temporizar un transmisor y con lo que transmite al ocurrir una terminación de temporizador. Un transmisor nervioso que a veces termina de temporizar demasiado pronto y retransmite todos los paquetes pendientes usando el protocolo de regresar *n* impondrá una carga más pesada al sistema que un transmisor calmado que usa repetición selectiva. Muy relacionada con esto está la política de *caché*. Si los receptores rutinariamente descartan todos los paquetes que llegan fuera de orden, estos paquetes tendrán que transmitirse nuevamente después, creando una carga extra.

La política de acuse de recibos también afecta el congestionamiento. Si cada paquete es reconocido de inmediato, los paquetes de acuse de recibo generan tráfico extra. Sin embargo, si se guardan los acuses de recibo para incorporarlos en el tráfico en reversa, pueden resultar terminaciones de temporización y retransmisiones extra. Un esquema de control de flujo estricto (por ejemplo, una ventana pequeña) reduce la tasa de datos y permite, por tanto, atacar los congestionamientos.

En la capa de red, la decisión entre circuitos virtuales y datagramas afecta el congestionamiento, ya que muchos algoritmos de control de congestionamiento sólo funcionan con subredes de circuitos virtuales. La política de encolamiento y servicio de paquetes se refiere a que los enrutadores tengan una cola por línea de entrada, una cola por línea de salida o ambas cosas. También se relaciona con el orden de proceso de los paquetes (por ejemplo, por turno circular o con base en prioridades). La política de descartado es la regla que indica qué paquete descartar cuando no hay espacio. Una buena política puede ayudar a aliviar el congestionamiento y una mala puede hacerlo peor.

El algoritmo de enrutamiento puede evitar el congestionamiento distribuyendo el tráfico entre todas las líneas, pero uno malo puede enviar demasiado tráfico por líneas ya congestionadas. Por último, la administración del tiempo de vida de los paquetes se encarga del tiempo que puede vivir un paquete antes de ser descartado. Si este tiempo es demasiado grande, los paquetes perdidos pueden bloquear la operación durante un buen rato, pero si es demasiado corto, los paquetes pueden expirar antes de alcanzar su destino, induciendo retransmisiones.

En la capa de transporte surgen los mismos problemas que en la capa de enlace de datos, pero además es más difícil la determinación del intervalo de expiración, porque el tiempo de tránsito a través de la red es menos predecible que el tiempo de tránsito por un alambre entre dos enrutadores. Si el intervalo es demasiado corto, se enviarán innecesariamente paquetes extra; si es demasiado largo, se reducirá el congestionamiento, pero el tiempo de respuesta sufrirá cada vez que se pierda un paquete.

5.3.3. Conformación de tráfico

Una de las causas principales del congestionamiento es que el tráfico con frecuencia es por ráfagas. Si pudiera hacerse que los *hosts* transmitieran a una tasa uniforme, serían menos comunes los congestionamientos. Otro método de ciclo abierto para ayudar a manejar el congestionamiento es obligar a la transmisión de los paquetes a una tasa más predecible. Este enfoque de administración de congestionamiento se usa ampliamente en las redes ATM y se llama **conformación de tráfico**.

La conformación de tráfico se ocupa de la regulación de la *tasa* promedio (y tendencia a ráfagas) de la transmisión de datos. En contraste, los protocolos de ventana corrediza que estudiamos antes limitan la cantidad de datos en tránsito al mismo tiempo, no la rapidez a la que se transmiten. Al establecerse un circuito virtual, el usuario y la subred (es decir, el cliente y la portadora) acuerdan cierto patrón de tráfico (es decir, forma) para ese circuito. Mientras el cliente cumpla con su parte del trato y sólo envíe paquetes según el contrato acordado, la portadora promete entregarlos todos a tiempo. La conformación de tráfico reduce el congestionamiento y ayuda por tanto a la portadora a cumplir sus promesas. Tales arreglos no son tan importantes para las transferencias de archivos, pero son de gran importancia para los datos en tiempo real, como en las conexiones de audio y vídeo, que no toleran bien los congestionamientos.

En efecto, con la conformación de tráfico, el cliente le dice a la portadora: "mi patrón de transmisiones se parecerá a esto. ¿Puedes manejarlo?" Si la portadora está de acuerdo, surge el asunto de que la portadora sepa si el cliente está ajustándose al acuerdo, y de lo que debe hacer

si no es así. La supervisión de un flujo de tráfico se llama **vigilancia de tráfico**. El acuerdo de una conformación de tráfico y su vigilancia posterior son más sencillos en las subredes de circuitos virtuales que en las subredes de datagramas. Sin embargo, aun en las subredes de datagramas pueden aplicarse los mismos conceptos a las conexiones de capa de transporte.

Algoritmo de cubeta con goteo

Imagínese una cubeta con un pequeño agujero en el fondo, como se ilustra en la figura 5-24(a). Sin importar la rapidez con que entra agua en la cubeta, el flujo de salida tiene una tasa constante, ρ, cuando hay agua en la cubeta, y una tasa de cero cuando la cubeta está vacía. También, una vez que se llena la cubeta, cualquier agua adicional que entra se derrama por los costados y se pierde (es decir, no aparece en el flujo por debajo del agujero).

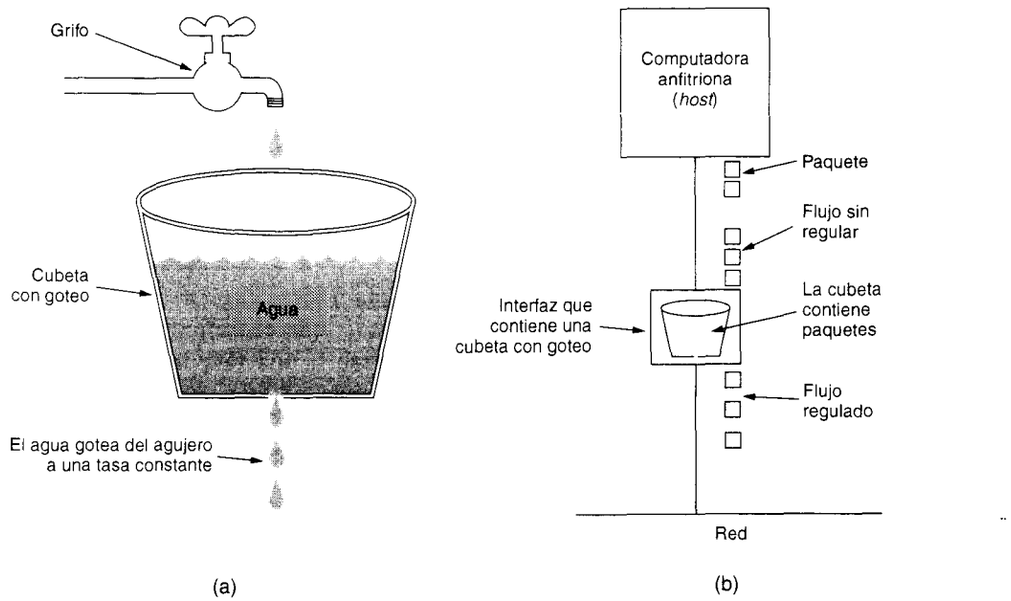


Figura 5-24. (a) Una cubeta con goteo, llena de agua. (b) Cubeta con goteo, llena de paquetes.

Puede aplicarse el mismo concepto a los paquetes, como se muestra en la figura 5-24(b). Conceptualmente, cada *host* está conectado a la red mediante una interfaz que contiene una cubeta con goteo, es decir, una cola interna finita. Si llega un paquete a la cola cuando está llena, se descarta el paquete. En otras palabras, si uno o más procesos del *host* tratan de enviar paquetes cuando ya está en cola la cantidad máxima, se descarta el paquete sin más. Este arreglo puede incorporarse en el *hardware* de la interfaz, o simularse a través del sistema operativo del *host*. El esquema fue propuesto inicialmente por Turner (1986), y se llama **algoritmo de cubeta con goteo** (*leaky bucket algorithm*). De hecho, no es otra cosa que un sistema de encolamiento de un solo servidor con un tiempo de servicio constante.

El *host* puede poner en la red un paquete por pulso de reloj. Nuevamente, esto puede obligarse desde la tarjeta de interfaz o desde el sistema operativo. Este mecanismo convierte un flujo desigual de paquetes de los procesos de usuario dentro del *host* en un flujo continuo de paquetes hacia la red, moderando las ráfagas y reduciendo en buena medida las posibilidades de congestionamiento.

Cuando los paquetes son todos del mismo tamaño (por ejemplo, células ATM), este algoritmo puede usarse como se describe. Sin embargo, al usarse paquetes de tamaño variable, con frecuencia es mejor permitir un número fijo de bytes por pulso, en lugar de un solo paquete. Por tanto, si la regla es de 1024 bytes por pulso, pueden recibirse por pulso un solo paquete de 1024 bytes, dos paquetes de 512 bytes, cuatro paquetes de 256 bytes, etc. Si el conteo de bytes residuales es demasiado bajo, el siguiente paquete debe esperar hasta el siguiente pulso.

La implementación del algoritmo de cubeta con goteo es fácil. La cubeta con goteo consiste en una cola finita. Al llegar un paquete, si hay espacio en la cola, se agrega a ella; de otro modo se descarta. En cada pulso de reloj se transmite un paquete (a menos que la cola esté vacía).

La cubeta con goteo que usa conteo de bits se implementa casi de la misma manera. En cada pulso se inicia un contador en n. Si el primer paquete de la cola tiene menos bytes que el valor actual del contador, se transmite y se disminuye el contador en esa cantidad de bytes. Pueden enviarse paquetes adicionales en tanto el contador sea lo suficientemente grande. Al caer el contador por debajo de la longitud del siguiente paquete de la cola, se detiene la transmisión hasta el siguiente pulso, en cuyo momento se sobrescribe el conteo de bytes residuales, perdiéndose.

Como ejemplo de cubeta con goteo, imagine que una computadora puede producir datos a razón de 25 millones de bytes/seg (200 Mbps) y que la red también opera a esta velocidad. Sin embargo, los enrutadores pueden manejar esta tasa de datos sólo durante intervalos cortos. Durante intervalos grandes funcionan mejor con tasas que no exceden 2 millones de bytes/seg. Ahora suponga que los datos llegan en ráfagas de un millón de bytes, con una ráfaga de 40 mseg cada segundo. Para reducir la tasa promedio a 2 MB/seg, podemos usar una cubeta con goteo de $\rho = 2$ MB/seg y capacidad C de 1 MB. Esto significa que las ráfagas de hasta 1 MB pueden manejarse sin pérdidas de datos, ya que se distribuyen a través de 500 mseg, sin importar la velocidad a la que lleguen.

En la figura 5-25(a) vemos la entrada de la cubeta con goteo operando a 25 MB/seg durante 40 mseg. En la figura 5-25(b) vemos la salida drenándose a una velocidad uniforme de 2 MB/seg durante 500 mseg.

Algoritmo de cubeta con ficha

El algoritmo de cubeta con goteo impone un patrón de salida rígido a la tasa promedio, sin importar la cantidad de ráfagas que tenga el tráfico. En muchas aplicaciones es mejor permitir que la salida se acelere un poco al llegar ráfagas grandes, por lo que se necesita un algoritmo más flexible, de preferencia uno que nunca pierda datos. El algoritmo de cubeta con ficha (**token bucket algorithm**) es uno de tales algoritmos. En este algoritmo, la cubeta con goteo contiene

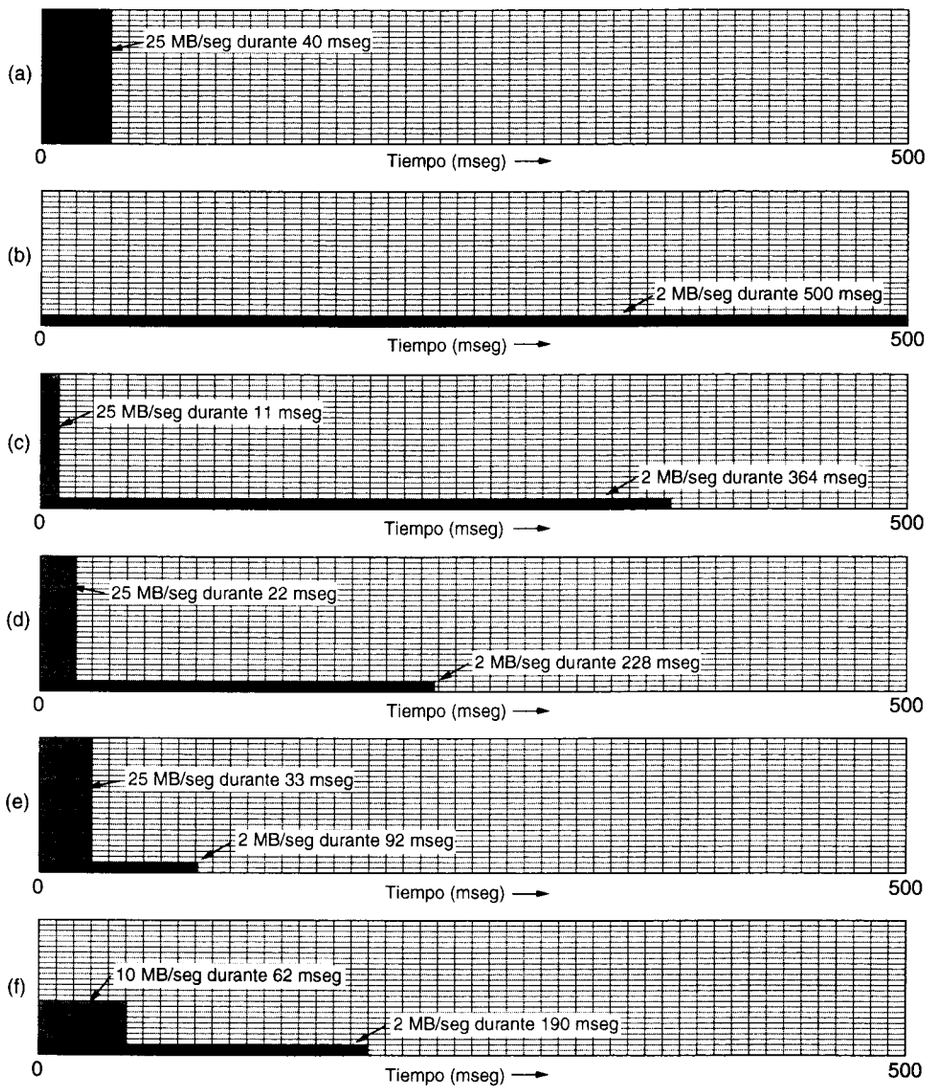


Figura 5-25. (a) Entrada a una cubeta con goteo. (b) Salida de una cubeta con goteo. (c) - (e) Salida de una cubeta con fichas con capacidades de 250kB, 500kB y 750kB. (f) Salida de una cubeta con fichas de 500kB que alimenta a una cubeta con goteo de 10 MB/seg.

fichas, generadas por un reloj a razón de una ficha cada ΔT seg. En la figura 5-26(a) vemos una cubeta que contiene tres fichas, con cinco paquetes esperando ser transmitidos. Para transmitir un paquete, el transmisor debe capturar y destruir una ficha. En la figura 5-26(b) vemos que han pasado tres de los cinco paquetes, pero los otros dos están atorados, esperando la generación de dos o más fichas.

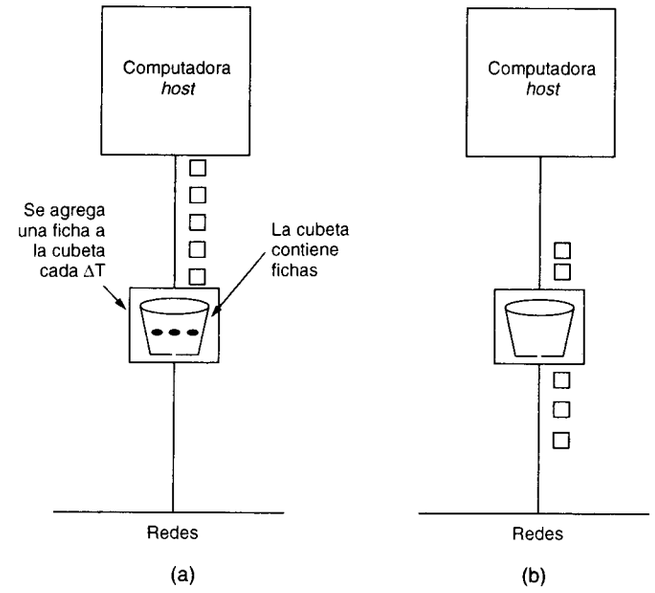


Figura 5-26. Algoritmo de cubeta con fichas. (a) Antes. (b) Después.

El algoritmo de cubeta con ficha ofrece una forma diferente de conformación de tráfico que el algoritmo de cubeta con goteo. Este último no permite que los *hosts* inactivos acumulen permisos para enviar posteriormente ráfagas grandes. El algoritmo de cubeta con ficha sí permite el ahorro, hasta el tamaño máximo de la cubeta, n. Esta propiedad significa que pueden enviarse a la vez ráfagas de hasta n paquetes, permitiendo cierta irregularidad en la corriente de salida y dando una respuesta más rápida a las ráfagas de entrada repentinas.

Otra diferencia entre los dos algoritmos es que el algoritmo de cubeta con ficha descarta las fichas cuando la cubeta se llena, pero nunca descarta los paquetes. En contraste, el algoritmo de cubeta con goteo descarta los paquetes cuando se llena la cubeta.

Aquí también es posible una variación menor, en la que cada ficha representa el derecho de transmisión, no de un paquete, sino de k bytes. Sólo puede transmitirse un paquete si hay suficientes fichas disponibles para cubrir su longitud en bytes. Las fichas fraccionarias se guardan para uso futuro.

Los algoritmos de cubeta con goteo y cubeta con ficha pueden servir también para regular el tráfico entre los enrutadores, de la misma manera en que se usan para regular la salida de un *host*, como en nuestros ejemplos. Sin embargo, una diferencia clara es que una cubeta con ficha que regula a un *host* puede hacer que el *host* detenga el envío cuando las reglas dicen que debe hacerlo. Indicar a un enrutador que detenga la transmisión mientras sigue recibiendo entradas puede resultar en la pérdida de datos.

La implementación del algoritmo básico de cubeta con ficha simplemente es una variable que cuenta fichas. El contador se incrementa en uno cada ΔT y disminuye en uno cada vez que se

envía un paquete. Al llegar a cero el contador, ya no pueden enviarse paquetes. En la variante de conteo de bits, se incrementa el contador en k bytes cada ΔT y disminuye en la longitud de cada paquete enviado.

En esencia, lo que hace la cubeta con ficha es permitir ráfagas, pero limitadas a una longitud máxima regulada. Véase un ejemplo en la figura 5-25(c). Aquí tenemos una cubeta de fichas con 250 KB de capacidad. Las fichas llegan con una tasa tal que permite la salida a 2 MB/seg. Suponiendo que la cubeta con ficha está llena cuando llega la ráfaga de 1 MB, la cubeta puede drenarse a la velocidad máxima de 25 MB/seg durante unos 11 mseg. Entonces tiene que desacelerarse hasta 2 MB/seg hasta que se ha enviado la ráfaga de entrada completa.

El cálculo de la longitud de ráfaga con tasa máxima es un tanto complicado. No es sólo la división de 1 MB entre 25 MB/seg, ya que, mientras se está enviando la ráfaga, llegan más fichas. Si S seg es la longitud de la ráfaga, C bytes es la capacidad de la cubeta con ficha, ρ bytes/seg es la tasa de llegada de fichas, y M bytes/seg es la tasa máxima de salida, podemos ver que una ráfaga de salida contiene un máximo de $C + \rho S$ bytes. También sabemos que la cantidad de bytes en una ráfaga a velocidad máxima con longitud de S segundos es MS. Por tanto, tenemos

$$C + \rho S = MS$$

Podemos resolver esta ecuación para obtener $S = C/(M - \rho)$. Para nuestros parámetros de C = 250 KB, M = 25 MB/seg y ρ = 2 MB/seg, tenemos un tiempo de ráfaga de unos 11 mseg. En la figura 5-25(d) y en la figura 5-25(e) se muestra la cubeta con ficha para capacidades de 500 KB y 750 KB, respectivamente.

Un problema potencial con el algoritmo de cubeta con ficha es que permite ráfagas largas, aunque puede regularse el intervalo máximo de ráfaga mediante una selección cuidadosa de ρ y M. Con frecuencia es deseable reducir la tasa pico, pero sin regresar al bajo valor de la cubeta con goteo original.

Una manera de lograr tráfico más uniforme es poner una cubeta con goteo después de la cubeta con ficha. La tasa de la cubeta con goteo deberá ser mayor que la ρ de la cubeta con ficha, pero menor que la tasa máxima de la red. En la figura 5-25(f) se muestra la salida de una cubeta con ficha de 500 KB seguida de una cubeta con goteo de 10 MB/seg.

La vigilancia de estos esquemas puede ser un tanto complicada. En esencia, la red tiene que simular el algoritmo y asegurarse que no se envíen más paquetes o bytes de lo permitido. Los paquetes de más se descartan o se degradan, como estudiaremos después.

5.3.4. Especificaciones de flujo

La conformación de tráfico es más efectiva cuando el transmisor, el receptor y la subred están de acuerdo. Para ponerse de acuerdo, es necesario especificar el patrón de tráfico de una manera precisa. Tal arreglo se llama **especificación de flujo**, y consiste en una estructura de datos que describe tanto el patrón de tráfico interpolado como la calidad del servicio deseado por las aplicaciones. Una especificación se puede aplicar tanto a los paquetes enviados por un circuito virtual como a una secuencia de datagramas enviados entre un origen y un destino (o aun a varios destinos).

En esta sección describiremos una especificación de flujo diseñada por Partridge (1992), la cual se presenta en la figura 5-27. La idea es que, antes de establecer una conexión, o antes del envío de una secuencia de datagramas, el origen presente la especificación de flujo a la subred, para su aprobación. La subred puede aceptarla, rechazarla, o presentar una contrapropuesta ("no puedo darte un retardo promedio de 100 mseg; ¿te sirve uno de 150 mseg?"). Una vez que el transmisor y la subred han llegado a un acuerdo, el transmisor puede preguntar al receptor si también está de acuerdo.

Características de la entrada	Servicio deseado
Tamaño máximo de paquetes (bytes)	Sensibilidad a pérdidas (bytes)
Tasa de cubeta con fichas (bytes/seg)	Intervalo de pérdidas (μseg)
Tamaño de la cubeta con fichas (bytes)	Sensibilidad a pérdidas por ráfaga (paquetes)
Tasa de transmisión máxima (bytes/seg)	Retardo mínimo notado (μseg)
	Variación máxima del retardo (μseg)
	Calidad de la garantía

Figura 5-27. Ejemplo de especificación de flujo.

Examinemos ahora los parámetros de nuestra especificación de flujo de ejemplo, comenzando por la especificación de tráfico. El *tamaño máximo de paquete* indica el tamaño que pueden tener los paquetes. Los siguientes dos parámetros suponen implícitamente que el tráfico se conformará mediante el algoritmo de cubeta con fichas trabajando con bytes; indican la cantidad de bytes que se colocan en la cubeta con fichas por segundo, así como el tamaño de la cubeta. Si la tasa es de r bytes/seg y el tamaño de la cubeta es de b bytes, entonces, en un intervalo de tiempo arbitrario Δt, la cantidad máxima de bytes que pueden enviarse es de $b + r\Delta t$. Aquí, el primer término representa el contenido máximo de la cubeta al inicio del intervalo y el segundo representa los créditos nuevos que han entrado durante el intervalo. La *tasa de transmisión máxima* es la tasa más alta que puede producir el *host* en cualquier condición, y especifica implícitamente el intervalo de tiempo en el que podría vaciarse la cubeta con fichas.

En la segunda columna se especifica lo que la aplicación pide a la subred. El primer y segundo parámetros representan el numerador y el denominador de una fracción que da la relación de pérdidas máxima aceptable (por ejemplo, 1 byte por hora). Alternamente, estos parámetros pueden indicar que el flujo es insensible a la pérdida de paquetes. La *sensibilidad a pérdidas en ráfaga* indica la cantidad de paquetes perdidos consecutivos que puede tolerarse.

Los siguientes dos parámetros de servicio tienen que ver con el retardo. El *retardo mínimo notado* indica el tiempo que puede retardarse un paquete sin que la aplicación se dé cuenta. En una transferencia de archivo, podría ser de un segundo, pero en un flujo de audio el límite podría ser de 3 mseg. La *variación máxima del retardo* intenta cuantificar el hecho de que algunas aplicaciones son insensibles al retardo mismo, pero son altamente sensibles a la **fluctuación**, es decir, a la cantidad de variación en el tiempo de tránsito de los paquetes de lado a lado. Este

parámetro es el doble de microsegundos que el retardo de un paquete puede variar respecto del promedio. Por tanto, un valor de 2000 significa que el paquete podrá llegar hasta 1 mseg antes o después, pero no más.

Por último, la *calidad de la garantía* indica si la aplicación realmente necesita lo que está pidiendo o no. Por un lado, las características de pérdida y retardo podrían ser metas ideales, pero no hay problema si no se logran; por otro, podrían ser tan importantes que, de no cumplirse, la aplicación simplemente termina. También son posibles posiciones intermedias.

Aunque hemos visto a la especificación de flujo como una solicitud de la aplicación a la subred, también puede ser un valor devuelto que indica lo que puede hacer la subred. Por tanto, puede usarse para una negociación prolongada del nivel de servicio.

Un problema inherente a cualquier especificación de flujo es que la aplicación podría no saber realmente lo que quiere. Por ejemplo, un programa de aplicación que se ejecuta en Nueva York podría conformarse con un retardo de 200 mseg a Sydney, pero estar muy descontenta con el mismo retardo de 200 mseg a Boston. Aquí, el "servicio mínimo" claramente es una función de lo que se cree que es posible.

5.3.5. Control de congestionamientos en las subredes de circuitos virtuales

Los métodos de control del congestionamiento antes descritos son básicamente de ciclo abierto: tratan de evitar que ocurran los congestionamientos, en lugar de manejarlos una vez ocurridos. En esta sección describiremos algunos enfoques para el control dinámico de congestionamientos en las redes de circuitos virtuales. En las siguientes dos veremos técnicas que pueden usarse en cualquier subred.

Una de las técnicas que se usa ampliamente para evitar que empeoren de los congestionamientos que ya han comenzado es el **control de admisión**. La idea es sencilla: una vez que se ha detectado el congestionamiento, no se establecen circuitos virtuales nuevos hasta que ha desaparecido el problema. Por tanto, fallan los intentos por establecer conexiones nuevas de capa de transporte. Permitir el acceso a más gente simplemente empeoraría las cosas. Aunque este enfoque es burdo, su implementación es sencilla. En el sistema telefónico, al carga extrarse un conmutador también se pone en práctica el control de admisión, al no darse tonos de marca.

Un enfoque alterno es permitir el establecimiento de nuevos circuitos virtuales, pero enrutando cuidadosamente los circuitos nuevos por otras rutas que no tienen problemas. Por ejemplo, considere la subred de la figura 5-28(a), en la cual están congestionados dos enrutadores, según se indica.

Supóngase que un *host* conectado al enrutador *A* quiere establecer una conexión con un *host* conectado al enrutador *B*. Normalmente, esta conexión pasaría a través de uno de los enrutadores congestionados. Para evitar esta situación, podemos redibujar la subred como se muestra en la figura 5-28(b), omitiendo los enrutadores congestionados y todas sus líneas. La línea de trazo interrumpido muestra una ruta posible para el circuito virtual que evita los enrutadores congestionados.

Otra estrategia que tiene que ver con los circuitos virtuales es negociar un acuerdo entre el *host* y la subred al establecerse un circuito virtual. Este arreglo normalmente especifica el volumen y la forma del tráfico, la calidad de servicio requerida y otros parámetros. Para cumplir

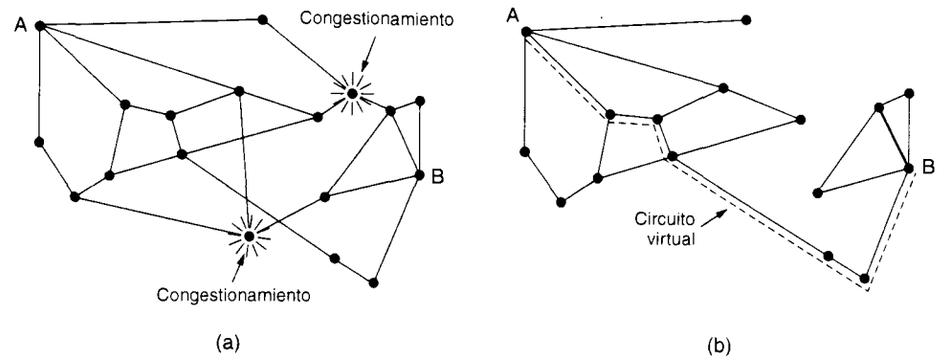


Figura 5-28. (a) Subred congestionada. (b) Subred redibujada que elimina el congestionamiento, y circuito virtual de *A* a *B*.

con su parte del acuerdo, la subred típicamente reservará servicios a lo largo de la trayectoria al establecerse el circuito. Estos recursos pueden incluir espacio en tablas y en *buffer* en los enrutadores y ancho de banda en las líneas. De esta manera, es poco probable que ocurran congestionamientos en los circuitos virtuales nuevos, porque está garantizada la disponibilidad de todos los recursos necesarios.

Este tipo de reservaciones pueden llevarse a cabo todo el tiempo como procedimiento operativo estándar, o sólo cuando está congestionada la subred. Una desventaja de hacerlo todo el tiempo es que se tiende a desperdiciar recursos. Si seis circuitos virtuales que podrían usar 1 Mbps pasan por la misma línea física de 6 Mbps, la línea tiene que marcarse como llena, aunque pocas veces ocurra que los seis circuitos virtuales transmitan al mismo tiempo. En consecuencia, el precio del control de congestionamiento es un ancho de banda no usado.

5.3.6. Paquetes de estrangulamiento

Veamos ahora un enfoque que puede usarse tanto en circuitos virtuales como en subredes de datagramas. Cada enrutador puede supervisar fácilmente el uso de sus líneas de salida y de otros recursos. Por ejemplo, puede asociar cada línea a una variable real, u, cuyo valor, entre 0.0 y 1.0, refleja el uso reciente de esa línea. Para tener una buena estimación de u, puede tomarse periódicamente una muestra del uso instantáneo de la línea, f (0 o 1), y actualizar u periódicamente de acuerdo con

$$u_{\text{nvo}} = au_{\text{ant}} + (1 - a)f$$

donde la constante a determina la rapidez con que el enrutador olvida la historia reciente.

Siempre que u rebasa el umbral, la línea de salida entra en un estado de "advertencia". Cada paquete nuevo que llega se revisa para ver si su línea de salida está en el estado de advertencia. De ser así, el enrutador envía un **paquete de estrangulamiento** de regreso al *host* de origen, dándole el destino encontrado en el paquete. El paquete original se marca (se enciende un bit de

cabecera) para que no pueda generar más paquetes de estrangulamiento más adelante en la trayectoria, y se reenvía de la manera normal.

Al recibir el paquete de estrangulamiento, el *host* de origen, está obligado a reducir el tráfico que envía al destino especificado en *X%*. Dado que otros paquetes destinados al mismo lugar probablemente ya están en camino y generarán nuevos paquetes de estrangulamiento, el *host* deberá ignorar los paquetes que se refieran a ese destino durante un intervalo de tiempo fijo. Tras la terminación de ese periodo, el *host* espera más paquetes de estrangulamiento durante otro intervalo. Si llega uno, es que la línea aún está congestionada, por lo que el *host* reduce el flujo aún más y comienza a ignorar nuevamente los paquetes de estrangulamiento. Si no llegan paquetes de estrangulamiento durante el periodo de escucha, el *host* puede aumentar el flujo nuevamente. La realimentación implícita en este protocolo puede ayudar a evitar el congestionamiento sin reducir ningún flujo a menos que ocurran problemas.

El *host* puede reducir el tráfico ajustando sus parámetros de política; por ejemplo, el tamaño de la ventana o la tasa de salida de la cubeta con goteo. Por lo común, el primer paquete de estrangulamiento hace que la tasa de datos se reduzca al 0.50 de su valor previo, el siguiente causa una reducción a 0.25, etc. Los aumentos se llevan a cabo en incrementos menores, para evitar que ocurra nuevamente el congestionamiento.

Se han propuesto diversas variaciones de este algoritmo de control de congestionamiento. Por una parte, los enrutadores pueden mantener varios umbrales. Dependiendo del umbral cruzado, el paquete de estrangulamiento puede contener una advertencia leve, una advertencia severa o un ultimátum.

Otra variación es el manejo de longitudes de cola o de uso de los *buffers*, en lugar del uso de línea, como señal de alarma. Por supuesto, puede usarse el mismo ponderado exponencial con esta métrica que con *u*.

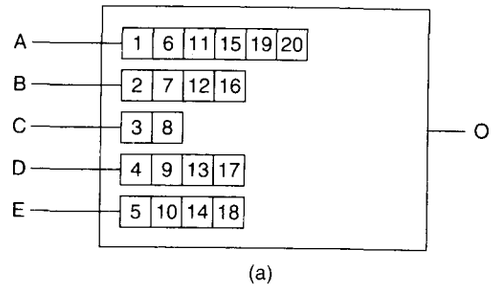
Encolamiento justo ponderado

Un problema con el uso de paquetes de estrangulamiento es que la acción a tomar por los *hosts* de origen es voluntaria. Supóngase que un enrutador está siendo abrumado por paquetes de cuatro orígenes, y que envía paquetes de estrangulamiento a todos ellos. Uno de ellos se refrena, como se espera, pero los otros tres siguen enviando a toda velocidad. El resultado es que el *host* honesto recibe una porción aún menor del ancho de banda del que tenía antes.

Para superar este problema, y por tanto hacer más atractivo el acatamiento, Nagle (1987) propuso un algoritmo de **encolamiento justo**. La esencia del algoritmo es que los enrutadores tienen varias colas para cada línea de salida, una para cada origen. Cuando una línea está ociosa, el enrutador examina las colas por turno, tomando el primer paquete de la siguiente cola. De esta manera, con *n hosts* compitiendo por una línea de salida dada, cada *host* envía uno de cada *n* paquetes. El envío de más paquetes no mejorará esta fracción. Algunos conmutadores ATM usan este algoritmo.

Aunque ayuda, el algoritmo tiene un problema: da más ancho de banda a los *hosts* que usan paquetes grandes que a los que usan paquetes pequeños. Demers *et al.* (1990) han sugerido una mejora en la que la búsqueda por turno se hace de manera que simula una ronda byte por byte, en

lugar de una ronda paquete por paquete. En efecto, se examinan repetidamente las colas, byte por byte, hasta que se encuentra el pulso de reloj en el que termina cada paquete. Se ordenan los paquetes en orden de terminación y se envían en ese orden. El algoritmo se ilustra en la figura 5-29.



Paquete	Tiempo de terminación
C	8
B	16
D	17
E	18
A	20

(b)

Figura 5-29. (a) Un enrutador con cinco paquetes encolados para la línea *O*. (b) Tiempos de terminación de los cinco paquetes.

En la figura 5-29(a) vemos paquetes de longitudes de 2 a 6 bytes. En el pulso (virtual) de reloj 1 se envía el primer byte del paquete por la línea *A*; luego sale el primer byte del paquete por la línea *B*, etc. El primer paquete que termina es el *C*, tras 8 pulsos. El orden se da en la figura 5-29(b). En ausencia de paquetes nuevos, los paquetes se enviarán en el orden listado, de *C* a *A*.

Uno de los problemas de este algoritmo es que da a todos los *hosts* la misma prioridad. En muchas situaciones, es deseable dar al servidor de archivos y a otros servidores más ancho de banda que a los clientes, de modo que puedan transmitir dos o más bytes por pulso. Este algoritmo modificado se llama **encolamiento justo ponderado**, y es de amplio uso. A veces el peso es igual a la cantidad de circuitos virtuales o de flujos que salen de una máquina, por lo que cada proceso recibe el mismo ancho de banda. En (Shreedhar y Varghese, 1995) se estudia una puesta en práctica eficiente del algoritmo.

Paquetes de estrangulamiento de escala por escala

A altas velocidades y distancias grandes, el envío de un paquete de estrangulamiento a los *hosts* de origen no funciona bien, porque la reacción es muy lenta. Considere por ejemplo a un *host* en San Francisco (enrutador *A* de la figura 5-30) que está enviando tráfico a un *host* en Nueva York (enrutador *D* de la figura 5-30) a 155 Mbps. Si al *host* de Nueva York se le comienza a terminar el espacio de *buffer*, le llevará unos 30 mseg a un paquete de estrangulamiento regresar a San Francisco para indicarle que disminuya su velocidad. La propagación de paquetes de estrangulamiento se muestra en el segundo, tercer y cuarto pasos de la figura 5-30(a). En esos 30 mseg se habrán enviado otros 4.6 megabits (por ejemplo, en 10,000 células ATM). Aun si el *host* de San Francisco se desactiva de inmediato, los 4.6 megabits en el conducto continuarán fluyendo, y habrá que encargarse de ellos. Sólo hasta el séptimo diagrama de la figura 5-30(a) el enrutador de Nueva York notará un flujo más lento.

Un enfoque alterno es hacer que el paquete de estrangulamiento tenga efecto en cada escala que haga, como se muestra en la secuencia de la figura 5-30(b). Aquí, tan pronto como el paquete de estrangulamiento llega a *F*, se obliga a *F* a reducir el flujo a *D*. Hacerlo requerirá que *F* destine más *buffers* al flujo, ya que el origen aún está transmitiendo a toda máquina, pero da a *D* un alivio inmediato, como en un mensaje comercial de un remedio contra el dolor de cabeza. En el siguiente paso, el paquete de estrangulamiento llega a *E*, e indica a éste que reduzca el flujo a *F*. Esta acción le impone una mayor carga a los *buffers* de *E*, pero le da un alivio inmediato a *F*. Por último, el paquete de estrangulamiento llega a *A* y efectivamente se reduce el flujo.

El efecto neto de este esquema de escala por escala es proporcionar un alivio rápido al punto de congestionamiento, a expensas de usar más *buffers* corriente arriba. De esta manera puede cortarse el congestionamiento en la raíz, pero sin perder paquetes. La idea se estudia con mayor detalle en (Mishra y Kanakia, 1992) donde también se dan resultados de simulaciones.

5.3.7. Desprendimiento de carga

Cuando ninguno de los métodos anteriores elimina el congestionamiento, los enrutadores pueden sacar la artillería pesada: el desprendimiento de carga. El **desprendimiento de carga** es una manera rebuscada de decir que, al inundarse los enrutadores de paquetes que no pueden manejar, simplemente los tiran. El término viene del mundo de la generación de energía eléctrica, donde se refiere a la práctica de instalaciones que intencionalmente producen apagones en ciertas áreas para salvar a la red completa de venirse abajo en días de verano calurosos en los que la demanda de energía eléctrica excede por mucho el suministro.

Un enrutador abrumado por paquetes puede simplemente escoger paquetes al azar para desprenderse de ellos, pero normalmente puede hacer algo mejor. El paquete a descartar puede depender de las aplicaciones que se están ejecutando. En la transferencia de archivos vale más un paquete viejo que uno nuevo, pues el deshacerse del paquete 6 y mantener los paquetes 7 a 10 causará un hueco en el receptor que podría obligar a la retransmisión de los paquetes 6 a 10 (si el receptor descarta rutinariamente los paquetes fuera de orden). En un archivo de 12 paquetes, deshacerse del paquete 6 podría requerir la retransmisión de los paquetes 7 a 12, y deshacerse del 10 podría requerir la retransmisión sólo del 10 al 12. En contraste, en multimedia es más importante un paquete nuevo que uno viejo. La política anterior (más viejo es mejor que más nuevo, se llama con frecuencia **vino**, y la última (más nuevo es mejor que más viejo) se llama con frecuencia **leche**.

Un paso por encima de esto en cuanto a inteligencia requiere la cooperación de los transmisores. En muchas aplicaciones, algunos paquetes son más importantes que otros. Por ejemplo, ciertos algoritmos de compresión de vídeo transmiten periódicamente un marco entero y sus marcos subsiguientes como diferencias respecto al último marco completo. En este caso, es preferible desprenderse de un paquete que es parte de una diferencia desprenderse de uno que es parte de un marco completo. Como otro ejemplo, considere la transmisión de un documento que contiene texto ASCII e imágenes. La pérdida de una línea de píxeles de una imagen es mucho menos dañina que la pérdida de una línea de texto legible.

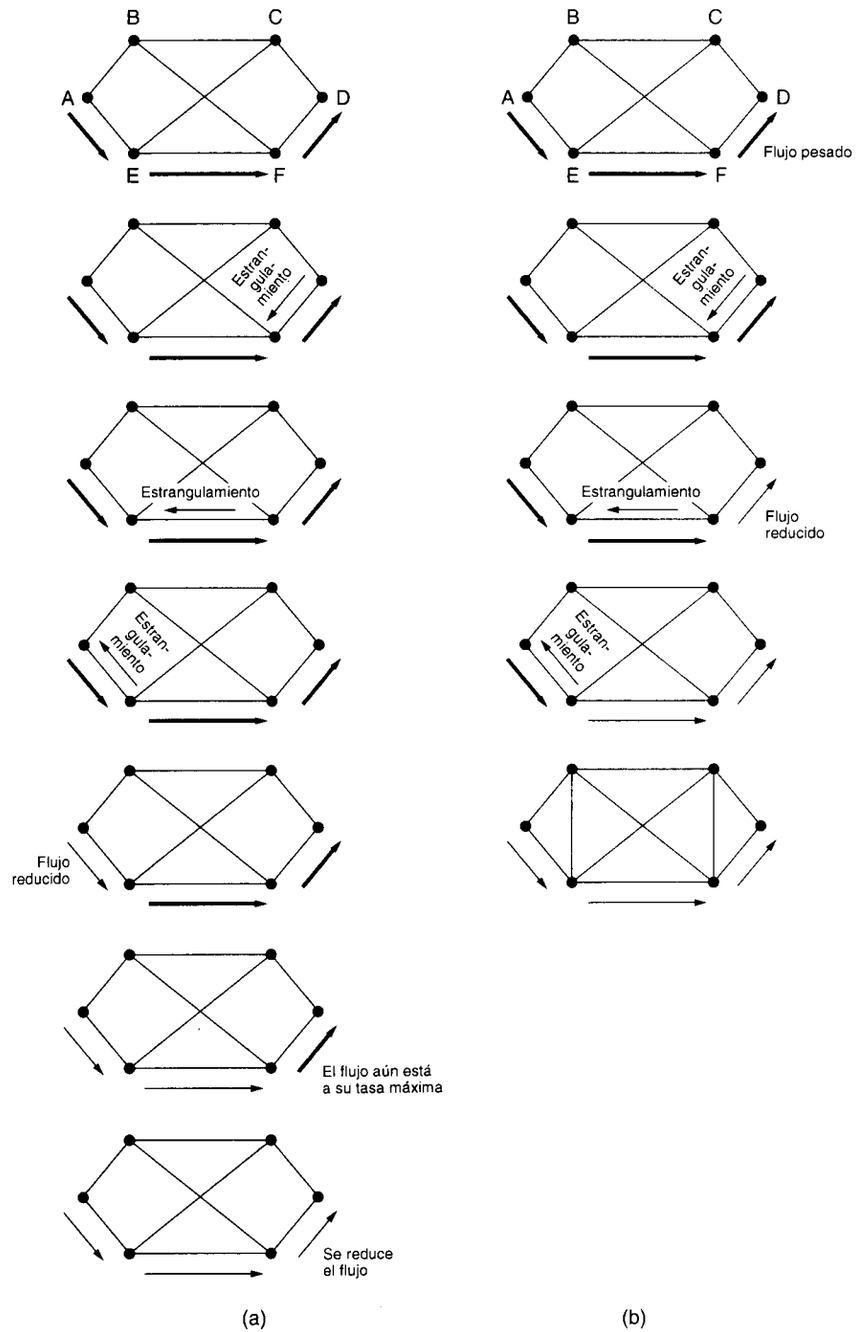


Figura 5-30. (a) Paquete de estrangulamiento que afecta sólo al origen. (b) Paquete de estrangulamiento que afecta cada escala por la que pasa.

Para poner en práctica una política inteligente de desprendimiento, las aplicaciones deben marcar sus paquetes con clases de prioridades para indicar su importancia. Si lo hacen, al tener que descartar paquetes, los enrutadores pueden desprenderse de los paquetes de clase más baja, luego de la siguiente clase más baja, etc. Por supuesto, a menos que haya un incentivo importante para marcar los paquetes con cualquier cosa que no sea MUY IMPORTANTE—NUNCA DESCARTAR, nadie lo hará.

El incentivo podría tener la forma de dinero, siendo más barato el envío de paquetes de baja prioridad que de los de alta prioridad. Como alternativa, las clases de prioridad podrían acoplarse a la conformación del tráfico. Por ejemplo, podría haber una regla que indicara que, al usarse el algoritmo de cubeta con fichas y llegar un paquete en un momento en que no hay ficha disponible, aún podrá enviarse, siempre y cuando esté marcado con la prioridad más baja posible, y por tanto sea candidato a descartarse en el momento en que ocurra algún problema. En condiciones de carga baja, los usuarios podrían estar contentos de operar de esta manera pero, a medida que aumenta la carga y los paquetes enviados comienzan a descartarse, podrían contenerse y sólo enviar paquetes al estar disponibles fichas.

Otra opción es permitir que los *hosts* excedan los límites especificados en el acuerdo negociado al establecer el circuito virtual (por ejemplo, usar un ancho de banda mayor que el permitido), pero sujetos a la condición de que el exceso de tráfico se marque con prioridad baja. Tal estrategia de hecho no es mala idea, porque hace un uso más eficiente de los recursos ociosos, permitiendo a los *hosts* su uso siempre y cuando nadie más esté interesado, pero sin establecer un derecho sobre ellos cuando los tiempos se vuelven difíciles.

La marca de paquetes por clase requiere uno o más bits de cabecera en los cuales poner la prioridad. Las células ATM tienen un bit reservado en la cabecera para este propósito, por lo que se etiqueta a cada célula ATM como de baja o alta prioridad. Los conmutadores ATM usan este bit al tomar decisiones de desprendimiento.

En algunas redes, los paquetes se agrupan en unidades mayores para fines de retransmisión. Por ejemplo, en las redes ATM, lo que hemos estado llamando "paquetes" son células de longitud fija. Estas células sólo son fragmentos de "mensajes". Al descartarse una célula, se retransmitirá el mensaje completo, no sólo la célula faltante. En estas condiciones, un enrutador que se desprende de una célula podría desechar las demás células de ese mensaje, ya que su transmisión cuesta ancho de banda y no sirve de nada: aun si llegaran a pasar, se retransmitirían después.

Los resultados de las simulaciones muestran que, al detectar un enrutador problemas en el horizonte, es mejor comenzar a descartar los paquetes pronto, en lugar de esperar a que se congestione por completo (Floyd y Jacobson, 1993; Romanow y Floyd, 1994). Hacerlo podría evitar que el congestionamiento siente sus reales.

5.3.8. Control de fluctuación

En aplicaciones como la transmisión de audio y vídeo no importa gran cosa si los paquetes tardan 20 mseg o 30 mseg en ser entregados, siempre y cuando el tiempo de tránsito sea constante. Si unos paquetes se tardan 20 mseg y otros 30 mseg, se obtendrá una calidad desigual del sonido o la imagen. Por tanto, el arreglo podría ser que el 99% de los paquetes se entreguen con un retardo

del orden de 24.5 mseg a 25.5 mseg. Por supuesto, el valor medio escogido debe ser factible. En otras palabras, debe calcularse una cantidad media de congestionamiento.

La fluctuación puede limitarse calculando el tiempo de tránsito esperado para cada escala de la trayectoria. Al llegar un paquete a un enrutador, el enrutador lo examina para saber qué tan adelantado o retrasado está respecto lo programado. Esta información se almacena en el paquete y se actualiza en cada escala. Si el paquete está adelantado, se retiene durante el tiempo suficiente para regresarlo a lo programado; si está retrasado, el enrutador trata de sacarlo rápidamente. De hecho, el algoritmo para determinar cuál de varios paquetes que compiten por una línea de salida debe seguir siempre puede escoger el paquete más retrasado. De esta manera, los paquetes adelantados se frenan y los paquetes retrasados se aceleran, reduciendo en ambos casos la cantidad de fluctuación.

5.3.9. Control de congestionamiento para multitransmisión

Todos los algoritmos de control de congestionamiento estudiados hasta el momento se aplican a mensajes de un solo origen a un solo destino. En esta sección describiremos una manera de manejar flujos de multitransmisión de varios orígenes a varios destinos. Por ejemplo, imagine varias estaciones de televisión de circuito cerrado que transmiten corrientes de audio y vídeo a un grupo de receptores, cada uno de los cuales puede ver una o más estaciones a la vez y es libre de conmutarse de estación a estación a voluntad. Una aplicación con esta tecnología puede ser una vídeoconferencia, en la cual cada participante puede enfocarse en el conferencista actual o en la expresión del jefe, según se desee.

En muchas aplicaciones multitransmisión, los grupos pueden cambiar de miembros dinámicamente, por ejemplo, cuando la gente entra en una vídeoconferencia o se aburre y cambia a un melodrama. En estas condiciones, el enfoque de hacer que los transmisores reserven ancho de banda por adelantado no funciona bien, ya que requeriría que cada transmisor llevara el registro de todas las entradas y salidas de su auditorio y regenerara el árbol de extensión con cada cambio. Para un sistema diseñado para transmitir televisión por cable, con millones de suscriptores, simplemente no funcionaría.

RSVP — Protocolo de reservación de recursos

Una solución interesante que puede controlar un entorno semejante es el protocolo **RSVP** (*Resource reSerVation Protocol*, **protocolo de reservación de recursos**) (Zhang *et al.*, 1993), el cual permite que varios transmisores transmitan a grupos múltiples de receptores, permite que receptores individuales conmuten canales libremente y perfecciona el uso del ancho de banda, eliminando al mismo tiempo el congestionamiento.

En su forma más sencilla, el protocolo usa enrutamiento de multitransmisión con árboles de extensión, como se vio antes. A cada grupo se le asigna un grupo de direcciones. Para enviar a un grupo, un transmisor pone la dirección del grupo en sus paquetes. El algoritmo estándar de multitransmisión construye entonces un árbol de extensión que cubre a todos los miembros del grupo. El algoritmo de enrutamiento no es parte del RSVP. La única diferencia con la multitransmisión

normal es un poco de información extra multitransmitida al grupo periódicamente para indicarle a los enrutadores a lo largo del árbol que mantengan ciertas estructuras de datos en sus memorias.

Como ejemplo, considere la red de la figura 5-31(a). Los *hosts* 1 y 2 son transmisores multitransmisión, y los *hosts* 3, 4 y 5 son receptores multitransmisión. En este ejemplo, los transmisores y los receptores son distintos pero, en general, los dos grupos pueden traslaparse. Los árboles de multitransmisión de los *hosts* 1 y 2 se muestran en la figura 5-31(b) y 5-31(c), respectivamente.

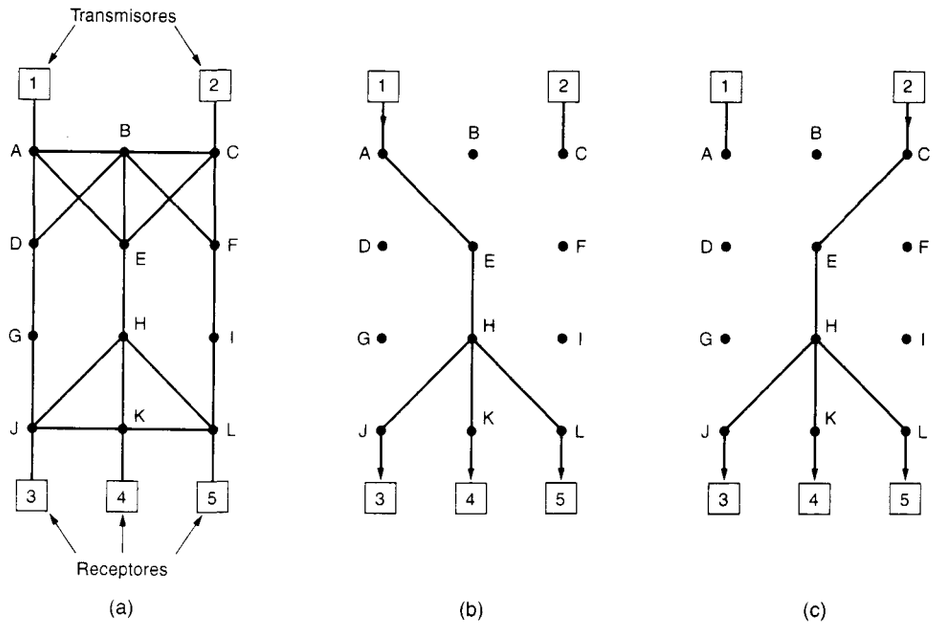


Figura 5-31. (a) Red. (b) Árbol de extensión de multitransmisión para el *host* 1. (c) Árbol de extensión de multitransmisión para el *host* 2.

Para obtener mejor recepción y eliminar el congestionamiento, cualquiera de los receptores de un grupo puede enviar un mensaje de reservación por el árbol al transmisor. El mensaje se propaga usando el algoritmo de reenvío por trayectoria invertida estudiado antes. En cada escala, el enrutador nota la reservación y reserva el ancho de banda necesario; si no hay suficiente ancho de banda disponible, informa de una falla. En el momento que el mensaje llega de regreso al origen, se ha reservado el ancho de banda desde el transmisor hasta el receptor que hace la solicitud de reservación a lo largo del árbol de extensión.

Un ejemplo de tales reservaciones se muestra en la figura 5-32(a). Aquí el *host* 3 ha solicitado un canal al *host* 1. Una vez establecido el canal, los paquetes pueden fluir de 1 a 3 sin congestionamientos. Ahora considere lo que sucede si el *host* 3 a continuación reserva un canal

al otro transmisor, el *host* 2, para que el usuario pueda ver dos programas de televisión a la vez. Se reserva una segunda trayectoria, como se muestra en la figura 5-32(b). Nótese que se requieren dos canales individuales del *host* 3 al enrutador *E*, porque se están transmitiendo dos corrientes independientes.

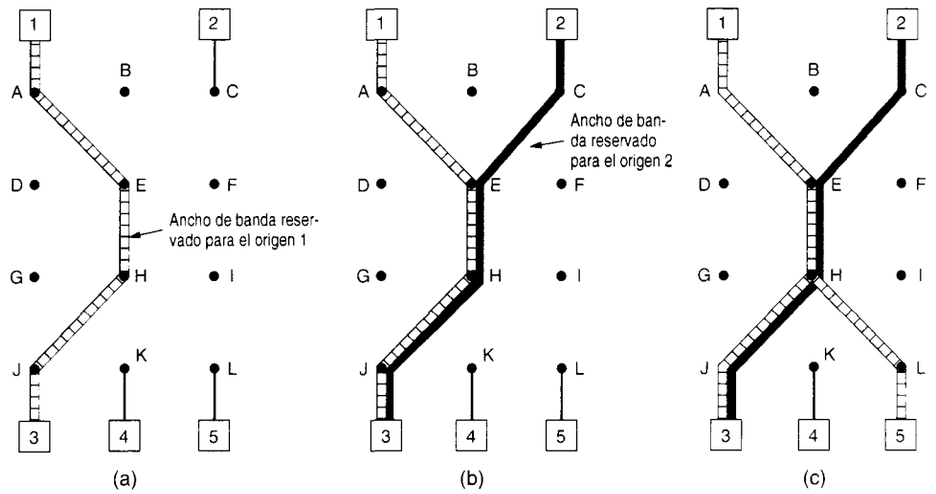


Figura 5-32. (a) El *host* 3 solicita un canal al *host* 1. (b) El *host* 3 solicita entonces un segundo canal al *host* 2. (c) El *host* 5 solicita un canal al *host* 1.

Por último, en la figura 5-32(c), el *host* 5 decide observar el programa transmitido por el *host* 1 y también hace una reservación. Primero se reserva ancho de banda dedicado hasta el enrutador *H*. Sin embargo, este enrutador ve que ya tiene una alimentación del *host* 1 por lo que, si ya se ha reservado el ancho de banda necesario, no necesita reservar nuevamente. Nótese que los *hosts* 3 y 5 podrían haber solicitado diferentes cantidades de ancho de banda (por ejemplo, el 3 tiene una televisión de blanco y negro, por lo que no quiere la información de color), así que la capacidad reservada debe ser lo bastante grande para satisfacer al receptor más voraz.

Al hacer una reservación, un receptor puede especificar (opcionalmente) uno o más orígenes de los que quiere recibir. También puede especificar si estas selecciones quedarán fijas durante toda la reservación, o si el receptor quiere mantener abierta la opción de cambiar los orígenes después. Los enrutadores usan esta información para optimar la planeación del ancho de banda. En particular, sólo se establece que dos receptores van a compartir una trayectoria si ambos están de acuerdo en no cambiar los orígenes posteriormente.

La razón de esta estrategia en el caso totalmente dinámico es que el ancho de banda reservado está desacoplado de la selección del origen. Una vez que un receptor ha reservado ancho de banda, puede conmutarse a otro origen y conservar la parte de la trayectoria existente que es válida para el nuevo origen. Si, por ejemplo, el *host* 2 está transmitiendo varias corrientes de

vídeo, el *host* 3 puede conmutarse entre ellas a voluntad sin cambiar su reservación: a los enrutadores no les importa el programa que está viendo el receptor.

5.4. INTERREDES

Hasta ahora hemos supuesto implícitamente que hay una sola red homogénea y que cada máquina usa el mismo protocolo en cada capa. Por desgracia, este supuesto es demasiado optimista. Existen muchas redes diferentes, que incluyen las LAN, MAN y WAN. En cada capa hay numerosos protocolos de uso muy difundido. En las siguientes secciones estudiaremos cuidadosamente los problemas que surgen cuando dos o más redes se juntan, formando una **interred**.

Existe considerable controversia sobre si la abundancia actual de tipos de red es una condición temporal que desaparecerá tan pronto como todo mundo se dé cuenta de lo maravillosa que es [indique aquí su red favorita], o si es una característica inevitable pero permanente del mundo, que está aquí para quedarse. Tener diferentes redes invariablemente implica tener diferentes protocolos.

Creemos que siempre habrá una variedad de redes (y por tanto, de protocolos) diferentes, por las siguientes razones. Antes que nada, la base instalada de redes diferentes es grande y está creciendo. Casi todas las instalaciones UNIX ejecutan TCP/IP. Muchos negocios grandes aún tienen *mainframes* que ejecutan SNA. DEC aún desarrolla DECnet. Las LAN de computadoras personales usan Novell NCP/IPX o AppleTalk. Los sistemas ATM comienzan a ser comunes. Por último, en las redes satelitales, celulares e infrarrojas se usan con frecuencia protocolos especializados. Esta tendencia continuará por años, debido a la gran cantidad de redes existentes y porque no todos los fabricantes perciben que es de interés suyo el que sus clientes puedan migrar fácilmente al sistema de otro fabricante.

Segundo, a medida que las computadoras y las redes se vuelven más baratas, el lugar de la toma de decisiones se desplaza hacia abajo. Muchas compañías tienen políticas en el sentido de que las compras de más de un millón de dólares tienen que ser aprobadas por la gerencia general, las compras de más de 100,000 dólares tienen que ser aprobadas por la gerencia media, pero las compras por debajo de 100,000 dólares pueden ser hechas por los jefes de los departamentos sin aprobación de los superiores. Esto puede conducir fácilmente a la instalación de Ethernet en el departamento de contabilidad, *token bus* en el departamento de ingeniería y *token ring* en el departamento de personal.

Tercero, las distintas redes (por ejemplo, ATM e inalámbricas) tienen tecnologías radicalmente diferentes, por lo que no debe sorprendernos que, a medida que hay avances nuevos en *hardware*, también se cree *software* nuevo adaptado al nuevo *hardware*. Por ejemplo, la casa típica ahora es como la oficina típica de hace 10 años: está llena de computadoras que no se hablan entre ellas. En el futuro, podría ser común que el teléfono, la televisión y otros aparatos estuvieran todos en red, para su control remoto. Esta nueva tecnología sin duda generará nuevos protocolos.

Como ejemplo de la interacción entre redes diferentes, considere el siguiente caso. En la mayoría de las universidades, los departamentos de informática e ingeniería electrónica tienen sus propias LAN, con frecuencia diferentes. Además, el centro de cómputo de la universidad

muchas veces tiene una *mainframe* y una supercomputadora, la primera para el cuerpo docente de humanidades que no desea meterse en el asunto del mantenimiento de computadoras, y la segunda para los físicos que quieren triturar números. Como consecuencia de estas diferentes redes e instalaciones, son fáciles de imaginar las siguientes situaciones:

1. LAN-LAN: Un informático descargando archivos para ingeniería.
2. LAN-WAN: Un informático enviando correo a un físico lejano.
3. WAN-WAN: Dos poetas intercambiando sonetos.
4. LAN-WAN-LAN: Ingenieros de diferentes universidades comunicándose.

En la figura 5-33 se ilustran estos cuatro tipos de conexiones como líneas punteadas. En cada caso, es necesario introducir una "caja negra" en las uniones entre dos redes, para manejar las conversiones necesarias a medida que los paquetes se mueven de una red a otra.

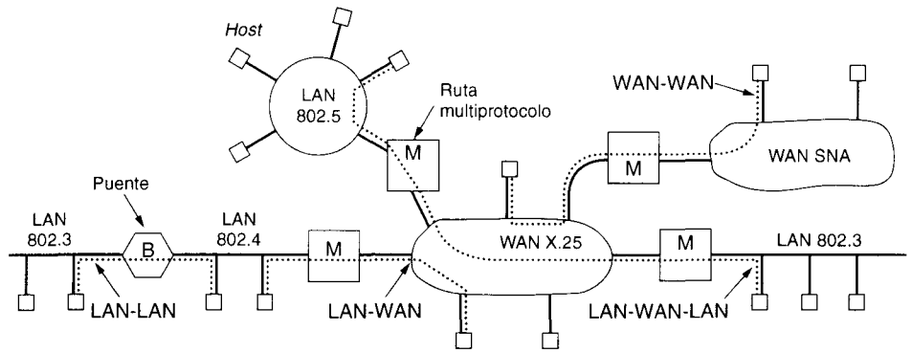


Figura 5-33. Interconexión de redes.

El nombre usado para la caja negra que conecta las dos redes depende de la capa que se encarga del trabajo. A continuación se indican algunos nombres comunes (aunque no hay mucho acuerdo en cuanto a la terminología de esta área).

Capa 1: Los repetidores copian bits individuales entre segmentos de cable.

Capa 2: Los puentes almacenan y reenvían marcos de enlace de datos entre las LAN.

Capa 3: Los enrutadores multiprotocolo reenvían paquetes entre redes disímiles.

Capa 4: Las pasarelas de transporte conectan corrientes de bytes en la capa de transporte.

Arriba de 4: Las pasarelas de aplicación permiten el trabajo conjunto por encima de la capa 4.

Por conveniencia, a veces usaremos el término "pasarela" (*gateway*) para indicar cualquier dispositivo que conecta dos o más redes disímiles.

Los **repetidores** son dispositivos de bajo nivel que simplemente amplifican o regeneran señales débiles. Se necesitan para proporcionar corriente que impulse la señal en cables largos. En el 802.3, por ejemplo, las propiedades de temporización del protocolo MAC (el valor de τ escogido) permiten cables de hasta 2.5 km, pero los *chips* del transceptor sólo pueden proporcionar potencia suficiente para llegar a 500 metros. La solución es usar repetidores para extender la longitud del cable donde se desea.

A diferencia de los repetidores, que copian los bits a medida que llegan, los **puentes** son dispositivos que almacenan y reenvían. Los puentes aceptan un marco entero y lo pasan a la capa de enlace de datos donde se verifica la suma de comprobación. Entonces el marco se envía a la capa física para reenviarlo por otra red. Los puentes pueden hacer cambios menores al marco antes de reenviarlo, como agregar o quitar algunos campos de la cabecera del marco. Puesto que los puentes son dispositivos de enlace de datos, no tienen tratos con las cabeceras de la capa 3 y superiores, y no pueden hacer cambios ni tomar decisiones que dependan de ellas.

Conceptualmente, los **enrutadores multiprotocolo** son parecidos a los puentes, excepto que se encuentran en la capa de red; simplemente toman paquetes de entrada de una línea y los reenvían por otra, al igual que todos los enrutadores, pero las líneas pueden pertenecer a redes diferentes y usar protocolos diferentes (por ejemplo, IP, IPX y el protocolo de paquetes sin conexiones de la OSI, CLNP). Como todos los enrutadores, los enrutadores multiprotocolo operan en el nivel de la capa de red.

Las **pasarelas de transporte** establecen una conexión entre dos redes en la capa de transporte. Estudiaremos esta posibilidad después cuando lleguemos a los circuitos virtuales concatenados.

Por último, las **pasarelas de aplicación** conectan dos partes de una aplicación en la capa de aplicación. Por ejemplo, para enviar correo de una máquina Internet usando el formato de correo de Internet a un buzón de correo MOTIS ISO, podría enviarse el mensaje a una pasarela de correo. Ésta desempacaría el mensaje, lo convertiría a formato MOTIS y lo reenviaría a la segunda red usando los protocolos de red y de transporte ahí empleados.

Cuando una pasarela está entre dos WAN operadas por organizaciones diferentes, posiblemente en países diferentes, la operación conjunta con una máquina clase estación de trabajo puede conducir a muchos desacuerdos y discusiones. Para eliminar estos problemas, puede usarse un enfoque un tanto diferente. La pasarela se divide por la mitad y se conectan ambas partes mediante un alambre. Cada una de las mitades se llama **media pasarela**, y cada una pertenece a, y es operada por, uno de los operadores de la red. El problema general del uso de pasarelas se reduce entonces a lograr un acuerdo sobre un protocolo común a usar en el alambre, uno que sea neutral y que no favorezca a ninguna parte. En la figura 5-34 se muestra tanto una pasarela completa como una media pasarela. Cualquiera de los dos tipos puede usarse en cualquiera de las capas (por ejemplo, también existen los medios puentes).

Dicho todo esto, la situación es más turbia en la práctica de lo que es en teoría. Muchos dispositivos comerciales combinan la funcionalidad de puente y de enrutador. La propiedad clave de un puente puro es que examina las cabeceras de los marcos de capa de enlace de datos y no inspecciona ni modifica los paquetes de capa de red que forman parte de los marcos. Un

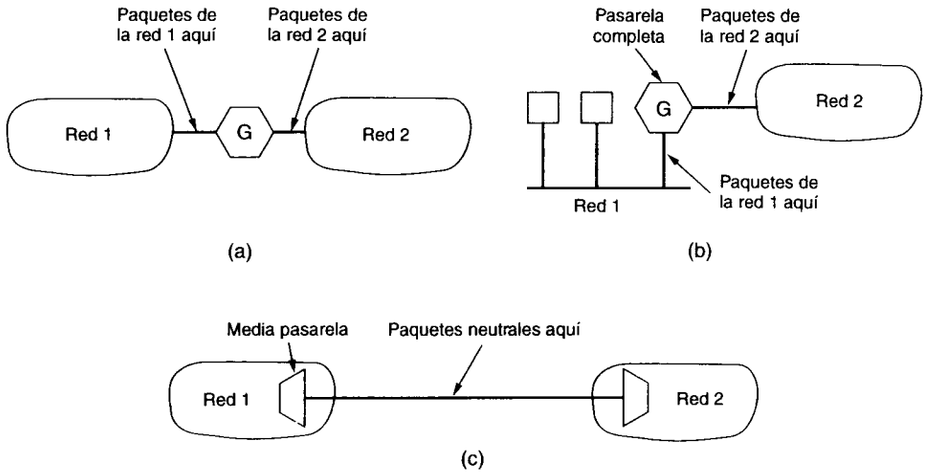


Figura 5-34. (a) Pasarela completa entre dos WAN. (b) Compasarela completa entre una LAN y una WAN. (c) Dos medias pasarelas.

puente no distingue, ni tampoco le importa, si el marco que está reenviado de una LAN 802.x a una 802.y contiene un paquete IP, IPX o CLNP en el campo de carga útil.

Un enrutador, en contraste, sabe muy bien si es un enrutador IP, IPX, CLNP o los tres combinados; examina estas cabeceras y toma decisiones con base en la dirección encontrada ahí. Por otra parte, cuando un enrutador puro entrega un paquete a la capa de enlace de datos, no sabe ni le importa si será llevado por un marco Ethernet o un marco *token ring*. Ésa es la función de la capa de enlace de datos.

La confusión en la industria viene de dos orígenes. Primero, desde el punto de vista funcional, los puentes y los enrutadores no son tan diferentes: aceptan las PDU (*Protocol Data Units*, unidades de datos de protocolo) de entrada, examinan algunos campos de cabecera, y toman decisiones sobre a dónde deben enviar las PDU, con base en información de las cabeceras y las tablas internas.

Segundo, muchos productos comerciales se venden con la etiqueta equivocada, o combinan la funcionalidad tanto de puentes como de enrutadores. Por ejemplo, los puentes de enrutamiento desde el origen no son realmente puentes, ya que requieren una capa de protocolo por encima de la capa de enlace de datos para hacer su trabajo. Si desea un análisis muy ilustrativo de los puentes y los enrutadores, vea el capítulo 12 de (Perlman, 1992).

5.4.1. Diferencias entre las redes

Las redes pueden diferir de muchas maneras. En la figura 5-35 listamos algunas de las diferencias que ocurren en la capa de red. La conciliación de estas diferencias es lo que hace más difícil el proceso de interconexión de redes que la operación con una sola red.

Aspecto	Algunas posibilidades
Servicio ofrecido	Sin conexiones, orientado a conexiones
Protocolos	IP, IPX, CLNP, AppleTalk, DECnet, etc.
Direccionamiento	Plano (802) o jerárquico (IP)
Multitransmisión	Presente o ausente (también multitransmisión)
Tamaño de paquete	Cada red tiene su propio máximo
Calidad del servicio	Puede estar presente o ausente; muchos tipos diferentes
Manejo de errores	Entrega confiable, ordenada y desordenada
Control de flujo	Ventana corrediza, control de tasa, otros o ninguno
Control de congestionamientos	Cubeta con goteo, paquetes de estrangulamiento, etc.
Seguridad	Reglas de confidencialidad, cifrado, etc.
Parámetros	Diferentes terminaciones de temporización, especificaciones de flujo, etc.
Contabilidad	Por tiempo de conexión, por paquete, por byte, o sin ella

Figura 5-35. Algunas de las muchas maneras en que pueden diferir las redes.

Cuando los paquetes enviados por un origen en una red deben transitar a través de una o más redes foráneas antes de llegar a la red de destino (que también puede ser diferente de la red de origen), pueden ocurrir muchos problemas en las interfaces entre las redes. Para comenzar, cuando deben transitar paquetes de una red orientada a conexión a una sin conexiones, deben reordenarse, algo que el transmisor no espera y que el receptor no está preparado para manejar. Con frecuencia se necesitarán conversiones de protocolo, que pueden ser difíciles si la funcionalidad requerida no puede expresarse. También se necesitarán conversiones de direcciones, lo que podría requerir algún tipo de sistema de directorio. El paso de paquetes multitransmisión a través de una red que no reconoce la multitransmisión requiere la generación de paquetes individuales para cada destino.

Los diferentes tamaños máximos de paquete usados por las diferentes redes son un dolor de cabeza importante. ¿Cómo se pasa un paquete de 8000 bytes a través de una red cuyo tamaño máximo de paquete es de 1500 bytes? Es importante la diferencia en la calidad de servicio cuando un paquete que tiene restricciones de tiempo real pasa a través de una red que no ofrece garantías de tiempo real.

El control de errores, de flujo y de congestionamientos suelen ser diferentes entre las diferentes redes. Si tanto el origen como el destino esperan la entrega de paquetes en secuencia y sin errores, pero una red intermedia simplemente descarta paquetes cuando huele un congestionamiento en el horizonte, o los paquetes pueden vagar sin sentido durante un rato y emerger repentinamente para ser entregados, muchas aplicaciones fallarán. Existen diferentes mecanismos de seguridad, ajustes de parámetros y reglas de contabilidad, e incluso leyes de confidencialidad internacionales, que pueden causar problemas.

5.4.2. Circuitos virtuales concatenados

Son comunes dos estilos de interred: la concatenación orientada a conexión de subredes de circuitos virtuales, y los datagramas estilo Internet. Ahora estudiaremos ambos. En el modelo de circuito virtual concatenado, que se muestra en la figura 5-36, se establece una conexión con un *host* de una red distante de un modo parecido a la manera en que se establecen normalmente las conexiones. La subred ve que el destino es remoto y construye un circuito virtual al enrutador más cercano a la red de destino; luego construye un circuito virtual de ese enrutador a una "pasarela" externa (enrutador multiprotocolo). Esta pasarela registra la existencia del circuito virtual en sus tablas y procede a construir otro circuito virtual a un enrutador de la siguiente subred. Este proceso continúa hasta llegar al *host* de destino.

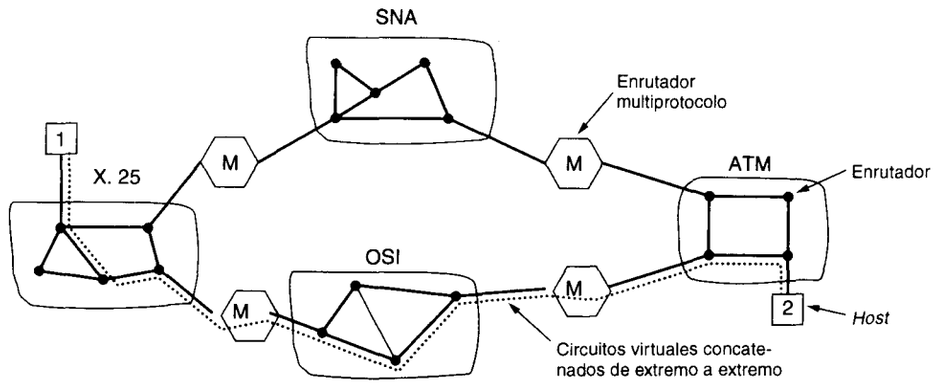


Figura 5-36. Interredes que usan circuitos virtuales concatenados.

Una vez que comienzan a fluir paquetes de datos por la trayectoria, cada pasarela releva los paquetes de entrada, haciendo las conversiones de formatos de paquete y números de circuito virtual según sea necesario. Obviamente, todos los paquetes de datos deben atravesar la misma secuencia de pasarelas y, por tanto, llegar en orden.

La característica esencial de este enfoque es que se establece una secuencia de circuitos virtuales desde el origen, a través de una o más pasarelas, hasta el destino. Cada pasarela mantiene tablas que indican los circuitos virtuales que pasan a través suyo, a dónde se deben enrutar y el número de circuito virtual nuevo.

Aunque en la figura 5-36 se muestra la conexión hecha con una pasarela completa, también podría hacerse con medias pasarelas.

Este esquema funciona mejor cuando todas las redes tienen aproximadamente las mismas propiedades. Por ejemplo, si todas ellas garantizan la entrega confiable de paquetes de capa de red, entonces, salvo una caída a lo largo de la ruta, el flujo del origen al destino también será confiable. De igual modo, si ninguna de ellas garantiza la entrega confiable, entonces la con-

catenación de los circuitos virtuales tampoco será confiable. Por otra parte, si la máquina de origen está en una red que garantiza la entrega confiable, pero una de las redes intermedias puede perder paquetes, la concatenación habrá cambiado fundamentalmente la naturaleza del servicio.

Los circuitos virtuales concatenados también son comunes en la capa de transporte. En particular, es posible construir un conducto de bits usando, digamos, OSI, que termine en una pasarela, y luego tener una conexión TCP entre esa pasarela y la siguiente. De este modo, puede construirse un circuito virtual de extremo a extremo abarcando diferentes redes y protocolos.

5.4.3. Interredes sin conexiones

El modelo alterno de interred es el modelo de datagramas, mostrado en la figura 5-37. En este modelo, el único servicio que ofrece la capa de red a la capa de transporte es la capacidad de inyectar datagramas en la subred y esperar que todo funcione bien. No hay noción en lo absoluto de un circuito virtual en la capa de red, ya no se diga de una concatenación de ellos. Este modelo no requiere que todos los paquetes que pertenecen a una conexión atraviesen la misma secuencia de pasarelas. En la figura 5-37 los datagramas del *host* 1 al *host* 2 toman diferentes rutas a través de la interred. Se toma una decisión de enrutamiento independiente para cada paquete, posiblemente dependiendo del tráfico en el momento de envío del paquete. Esta estrategia puede usar rutas múltiples y así lograr un ancho de banda mayor que el modelo de circuito virtual concatenado. Por otra parte, no hay garantía de que los paquetes llegarán al destino en orden, suponiendo que lleguen.

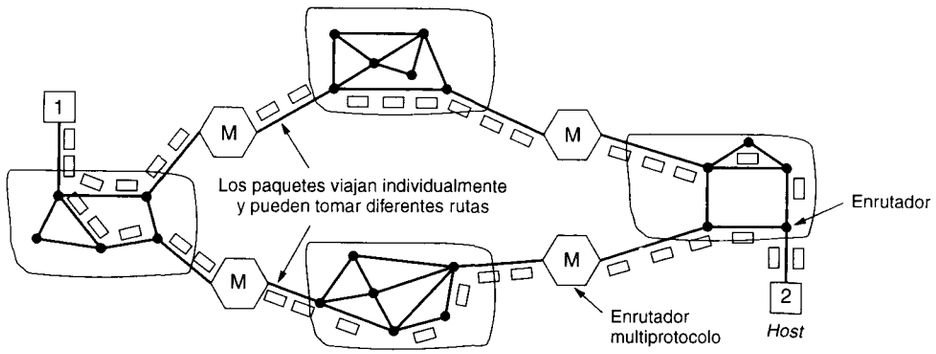


Figura 5-37. Interred sin conexiones.

El modelo de la figura 5-37 no es tan sencillo como parece. Por una parte, si cada red tiene su propio protocolo de capa de red, no es posible que un paquete de una red transite por otra. Podríamos imaginar a los enrutadores tratando de traducir de un formato a otro, pero a menos que los dos formatos sean parientes cercanos con los mismos campos de información, tales

conversiones siempre serán incompletas, y frecuentemente destinadas al fracaso. Por esta razón, pocas veces se intentan las conversiones.

Un segundo problema, más serio, es el direccionamiento. Imagínese un caso sencillo: un *host* de Internet está tratando de enviar un paquete IP a un *host* en un *host* OSI adyacente. El protocolo de datagramas de OSI, CLNP, se basó en el IP y es lo bastante parecido como para que una conversión pudiera funcionar bien. El problema es que todos los paquetes IP llevan la dirección Internet de 32 bits del *host* de destino en un campo de cabecera. Los *hosts* OSI no tienen direcciones de Internet de 32 bits; usan direcciones decimales parecidas a los números telefónicos.

Para hacer posible la conversión entre formatos en el enrutador multiprotocolo, alguien tendría que asignar una dirección de Internet de 32 bits a cada *host* OSI. Llevado al límite, este enfoque significaría la asignación de una dirección de Internet a cada máquina del mundo a la que pudiera querer hablar un *host* de Internet. También significaría asignar una dirección OSI a cada máquina del mundo a la que pudiera querer hablar una *host* OSI. Ocurre el mismo problema con todos los demás espacios de direcciones (SNA, AppleTalk, etc.). Los problemas aquí son insuperables. Además, alguien tendría que mantener una base de datos ligando todo con todo.

Otra idea es diseñar un paquete universal de "interred" y hacer que todos los enrutadores lo reconozcan. Este enfoque es, precisamente, lo que es el IP: un paquete diseñado para llevarse por muchas redes. El único problema es que existen IPX, CLNP y otros paquetes "universales", lo que hace que todos sean menos universales. Simplemente no es posible lograr que todos se pongan de acuerdo en un solo formato.

Recapitulemos ahora brevemente las dos maneras en que puede atacarse la interacción de redes. El modelo de circuito virtual concatenado tiene en esencia las mismas ventajas que el uso de circuitos virtuales en una sola subred: pueden reservarse *buffers* por adelantado, puede garantizarse la secuencia, pueden usarse cabeceras cortas y pueden evitarse los problemas causados por paquetes duplicados retrasados.

El modelo también tiene las mismas desventajas: el espacio de tablas requerido en los enrutadores para cada conexión abierta, la falta de enrutamiento alterno para evitar áreas congestionadas y la vulnerabilidad a fallas de los enrutadores en la trayectoria; también tiene la desventaja de ser difícil, si no imposible, su implementación si una de las redes que intervienen es una red no confiable de datagramas.

Las propiedades del enfoque por datagramas para interconectar redes son las mismas que las de las subredes de datagramas: un mayor potencial de congestionamiento, pero también mayor potencial de adaptarse a él, la robustez ante fallas de los enrutadores y la necesidad de cabeceras más grandes. En una interred son posibles varios algoritmos de enrutamiento adaptable, igual que en una sola red de datagramas.

Una ventaja principal del enfoque por datagramas para la interconexión de redes es que puede usarse en subredes que no usan circuitos virtuales. Muchas LAN, redes móviles (por ejemplo, flotas aéreas y navales) e incluso algunas WAN caen en esta categoría. Cuando una interred incluye una de éstas, surgen serios problemas si la estrategia de interredes se basa en circuitos virtuales.

5.4.4. Proceso de túnel

El manejo del caso general de lograr la interacción de dos redes diferentes es extremadamente difícil. Sin embargo, hay un caso especial común que puede manejarse. Este caso es cuando el *host* de origen y el de destino están en la misma clase de red, pero hay una red diferente en medio. Como ejemplo, piense en un banco internacional con una Ethernet basada en TCP/IP en París, una Ethernet basada en TCP/IP en Londres y una WAN PTT en medio, como se muestra en la figura 5-38.

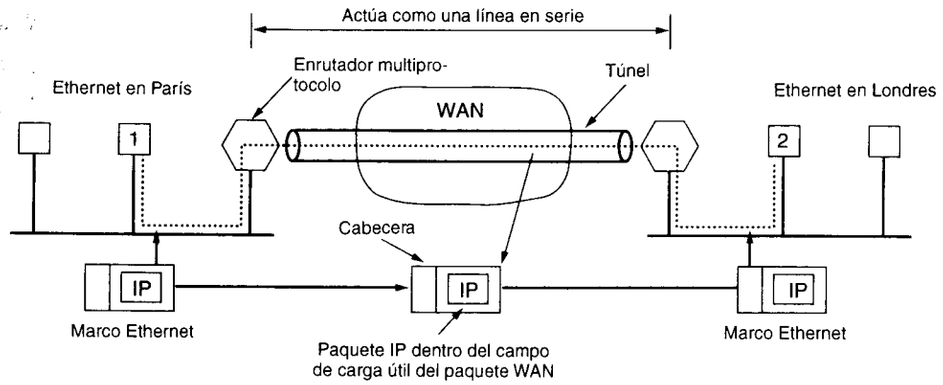


Figura 5-38. Proceso en túnel de un paquete de París a Londres.

La solución a este problema es una técnica llamada **proceso de túnel**. Para enviar un paquete IP al *host* 2, el *host* 1 construye el paquete que contiene la dirección IP del *host* 2, lo inserta en un marco Ethernet dirigido al enrutador multiprotocolo de París, y lo pone en el Ethernet. Cuando el enrutador multiprotocolo recibe el marco, retira el paquete IP, lo inserta en el campo de carga útil del paquete de capa de red de la WAN, y dirige este último a la dirección de la WAN del enrutador multiprotocolo de Londres. Al llegar ahí, el enrutador de Londres retira el paquete IP y lo envía al *host* 2 en un marco Ethernet.

La WAN pude visualizarse como un gran túnel que se extiende de un enrutador multiprotocolo al otro. El paquete IP simplemente viaja de un extremo del túnel al otro, bien acomodado en una caja bonita. No tiene que preocuparse por lidiar con la WAN. Tampoco tienen que hacerlo los *hosts* de cualquiera de los Ethernet. Sólo el enrutador multiprotocolo tiene que entender los paquetes IP y WAN. De hecho, la distancia completa entre la mitad de un enrutador multiprotocolo y la mitad del otro actúa como una línea en serie.

El proceso de túnel puede aclararse mediante una analogía. Considere una persona que maneja su auto de París a Londres. En Francia, el auto se mueve con su propia energía, pero al llegar al Canal de la Mancha, se carga en un tren de alta velocidad y se transporta a Inglaterra a través del Chunnel (los autos no pueden conducirse a través del Chunnel). En efecto, el auto se

transporta como carga, como se muestra en la figura 5-39. En el otro extremo, se libera el auto en las carreteras inglesas y nuevamente continúa moviéndose con sus propios medios. El proceso de túnel de paquetes a través de una red foránea funciona de la misma manera.

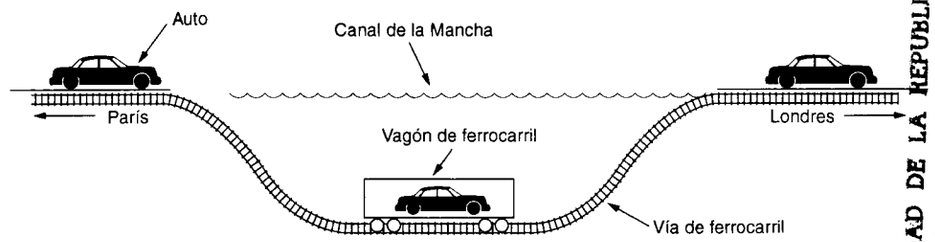


Figura 5-39. Paso por túnel de un auto de Francia a Inglaterra.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

5.4.5. Enrutamiento por interred

El enrutamiento a través de una interred es parecido al enrutamiento en una sola subred, pero con algunas complicaciones adicionales. Considere, por ejemplo, la interred de la figura 5-40(a) en la que cinco redes están conectadas mediante seis enrutadores multiprotocolo. Un modelo de grafo de esta situación se complica por el hecho de que cada enrutador multiprotocolo puede acceder (es decir, enviar paquetes) directamente a todos los demás enrutadores conectados a cualquier red a la que esté conectado. Por ejemplo, *B* en la figura 5-40(a) puede acceder directamente a *A* y a *C* a través de la red 2, y también a *D* a través de la red 3. Esto conduce al grafo de la figura 5-40(b).

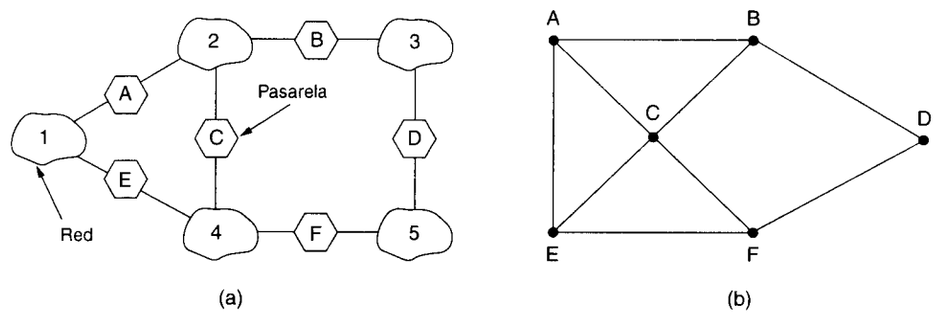


Figura 5-40. (a) Interred. (b) Grafo de la interred.

Una vez construido el grafo, pueden aplicarse algoritmos de enrutamiento conocidos, como el algoritmo de vector de distancia y el de estado de enlace, al grupo de enrutadores multiprotocolo. Esto da un algoritmo de enrutamiento de dos niveles: en cada red se usa un **protocolo de pasarela interior**, pero entre las redes se usa un **protocolo de pasarela exterior** ("pasarela" es

un término antiguo para "enrutador"). De hecho, ya que cada red es independiente, pueden usar algoritmos diferentes. Dado que cada red de una interred es independiente de las demás, con frecuencia se denomina **sistema autónomo** (**AS**, *Autonomous System*).

Un paquete de interred típico parte de su LAN dirigido al enrutador multiprotocolo local (en la cabecera de la capa de MAC). Al llegar ahí, el código de la capa de red decide por cuál enrutador multiprotocolo reenviará el paquete, usando sus propias tablas de enrutamiento. Si puede alcanzarse ese enrutador usando el protocolo de la red nativa del paquete, se reenvía directamente ahí. De otra manera, se envía por túnel, encapsulado en el protocolo requerido por la red que interviene. Este proceso se repite hasta que el paquete llega a la red de destino.

Una de las diferencias entre el enrutamiento interredes y el enrutamiento intrared es que el enrutamiento interredes con frecuencia requiere el cruce de fronteras internacionales. De pronto, entran en escena varias leyes, como las estrictas leyes suecas de confidencialidad sobre la exportación de datos personales de ciudadanos suecos. Otro ejemplo es la ley canadiense que indica que el tráfico de datos que se origina en Canadá y llega a un destino en Canadá no puede dejar el país. Esto significa que el tráfico de Windsor, Ontario a Vancouver no puede enrutarse a través de Detroit, Estados Unidos, que está cerca.

Otra diferencia entre el enrutamiento interior y el exterior es el costo. Dentro de una sola red, normalmente se aplica un solo algoritmo de cargo. Sin embargo, redes diferentes pueden estar bajo administraciones diferentes, y una ruta puede ser menos cara que otra. Del mismo modo, la calidad de servicio ofrecida por diferentes redes puede ser distinta, y ésta puede ser una razón para escoger una ruta y no otra.

En una interred grande, la selección de la mejor ruta puede ser una operación muy tardada. Estrin *et al.* (1992) han propuesto manejar este problema precalculando rutas para pares (origen, destino) comunes y almacenándolas en una base de datos que se consulta en el momento de seleccionarse la ruta.

5.4.6. Fragmentación

Cada red impone un tamaño máximo a sus paquetes. Estos límites tienen varias razones, entre ellas:

1. El *hardware* (por ejemplo, el ancho de una ranura de transmisión TDM).
2. El sistema operativo (por ejemplo, todos los *buffers* son de 512 bytes).
3. Los protocolos (por ejemplo, la cantidad de bits en el campo de longitud de paquete).
4. El cumplimiento de algún estándar (inter)nacional.
5. El deseo de reducir hasta cierto nivel las retransmisiones inducidas por errores.
6. El deseo de evitar que un paquete ocupe el canal demasiado tiempo.

El resultado de estos factores es que los diseñadores de redes no están en libertad de escoger cualquier tamaño máximo de paquete que deseen. Las cargas útiles máximas van desde 48 bytes

(células ATM) hasta 65,515 bytes (paquetes IP), aunque el tamaño de la carga útil en las capas superiores con frecuencia es más grande.

Surge un problema obvio cuando un paquete grande quiere viajar a través de una red cuyo tamaño máximo de paquete es demasiado pequeño. Una solución es asegurar que no ocurra el problema. En otras palabras, la interred debe usar un algoritmo de enrutamiento que evite el envío de paquetes a través de redes que no pueden manejarlos. Sin embargo, esta solución en realidad no es una solución. ¿Qué ocurre si el paquete original es demasiado grande para ser manejado por la red de destino? El algoritmo de enrutador no puede pasar por alto el destino.

Básicamente, la única solución al problema es permitir que las pasarelas dividan los paquetes en **fragmentos**, enviando cada paquete como paquete de interred individual. Sin embargo, como lo sabe cualquier padre de un niño pequeño, la conversión de un objeto grande en fragmentos pequeños es significativamente más fácil que el proceso inverso. (Los físicos inclusive le han dado un nombre a este efecto: segunda ley de la termodinámica.) Las redes de conmutación de paquetes también tienen problemas al unir nuevamente los fragmentos.

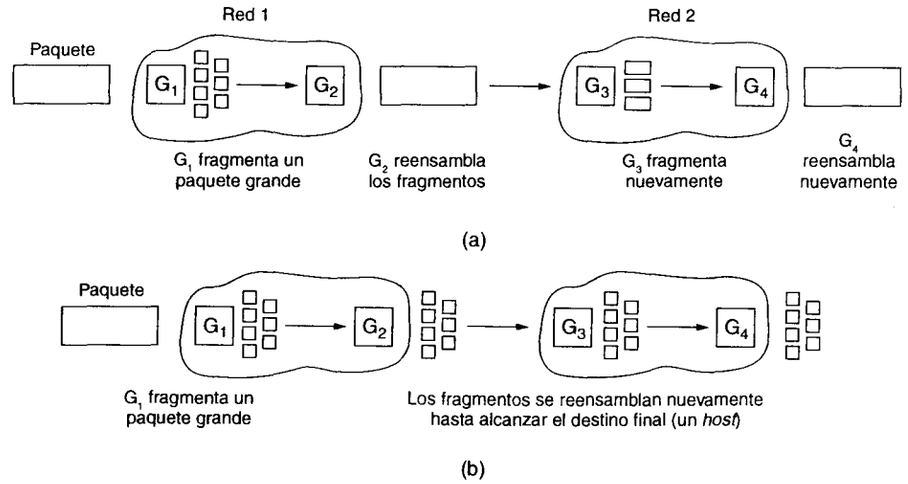


Figura 5-41. (a) Fragmentación transparente. (b) Fragmentación no transparente.

Existen dos estrategias opuestas para recombinar los fragmentos y recuperar el paquete original. La primera es hacer transparente la fragmentación causada por una red de "paquete pequeño" a las demás redes subsiguientes por las que debe pasar el paquete en el camino a su destino final. Esta opción se muestra en la figura 5-41(a). Al llegar un paquete de tamaño excesivo a la pasarela, ésta lo divide en fragmentos. Todos los fragmentos se dirigen a la misma pasarela de salida, donde se recombinan las piezas. De esta manera se ha hecho transparente el paso a través de la red de paquete pequeño. Las redes subsiguientes ni siquiera se enteran de que ha ocurrido una fragmentación. Las redes ATM, por ejemplo, tienen *hardware* especial para

proporcionar fragmentación transparente de paquetes en células y luego reensamblar las células en paquetes. En el mundo ATM, a la fragmentación se le llama segmentación; el concepto es el mismo, pero algunos de los detalles son diferentes.

La fragmentación transparente es sencilla, pero tiene algunos problemas. Por una parte, la pasarela de salida debe saber cuándo ha recibido todas las piezas, por lo que debe incluirse un campo de conteo o un bit de "fin de paquete" en cada paquete. Por otra parte, todos los paquetes deben salir por la misma pasarela. Al no permitir que algunos fragmentos sigan una ruta al destino final, y otros fragmentos una ruta distinta, puede bajar un poco el desempeño. Un último problema es la carga extra requerida para reensamblar y volver a fragmentar repetidamente un paquete grande que pasa a través de una serie de redes de paquete pequeño.

La otra estrategia de fragmentación es abstenerse de recombinar los fragmentos en las pasarelas intermedias. Una vez que se ha fragmentado un paquete, cada fragmento se trata como si fuera un paquete original. Todos los fragmentos pasan a través de la pasarela (o pasarelas) de salida, como se muestra en la figura 5-41(b). La recombinación ocurre sólo en el *host* de destino.

La fragmentación no transparente también tiene algunos problemas. Por ejemplo, requiere que "todos" los *hosts* sean capaces de hacer el reensamble. Otro problema es que, al fragmentarse un paquete grande, aumenta la carga extra total, pues cada fragmento debe tener una cabecera. En el primer método esta carga extra desaparece tan pronto se sale de la red de paquete pequeño, pero en este método la carga extra permanece durante el resto de la travesía. Sin embargo, una ventaja de este método es que ahora pueden usarse varias pasarelas de salida, lográndose un mejor desempeño. Por supuesto, si se está usando el modelo de circuito virtual concatenado, esta ventaja no es de ninguna utilidad.

Al fragmentarse un paquete, los fragmentos deben numerarse de tal manera que la corriente de datos original pueda reconstruirse. Una manera de numerar los fragmentos es usar un árbol. Si el paquete 0 debe dividirse, se llama a las partes 0.0, 0.1, 0.2, etc. Si estos fragmentos mismos deben fragmentarse después, se numera a las piezas 0.0.0, 0.0.1, 0.0.2, . . . , 0.1.0, 0.1.1, 0.1.2, etc. Si se reservan suficientes campos en la cabecera para el peor caso y no se generan duplicados en ningún lado, este esquema es suficiente para asegurar que todas las partes puedan reensamblarse correctamente en el destino, sin importar en qué orden lleguen.

Sin embargo, si cualquier red pierde o descarta paquetes, hay la necesidad de retransmisiones de extremo a extremo, con efectos poco afortunados para el sistema de numeración. Supóngase que se fragmenta inicialmente un paquete de 1024 bits en cuatro fragmentos del mismo tamaño, 0.0, 0.1, 0.2 y 0.3. Se pierde el fragmento 0.1, pero las otras partes llegan al destino. En algún momento termina la temporización del origen y se retransmite el paquete original, pero esta vez la ruta tomada pasa a través de una red con un límite de 512 bits, por lo que se generan dos fragmentos. Al llegar el nuevo fragmento 0.1 al destino, el receptor pensará que ya se han recibido las cuatro partes y reconstruirá incorrectamente el paquete.

Un sistema de numeración completamente diferente, y mejor, es que el protocolo de interred defina un tamaño de fragmento elemental lo bastante pequeño como para que el fragmento elemental pueda pasar a través de todas las redes. Al fragmentarse un paquete, todas las partes

son iguales al tamaño de fragmento elemental, excepto la última, que puede ser más corta. Un paquete de interred puede contener varios fragmentos, por razones de eficiencia. La cabecera de interred debe proporcionar el número de paquete original y el número del (primer) fragmento elemental contenido en el paquete. Como siempre, también debe haber un bit que indique que el último fragmento elemental contenido en el paquete de interred es el último del paquete original.

Este enfoque requiere dos campos de secuencia en la cabecera de interred: el número original de paquete y el número de fragmento. Evidentemente hay una concesión entre el tamaño del fragmento elemental y la cantidad de bits en el número de fragmento. Dado que se supone que el tamaño del fragmento elemental es aceptable para todas las redes, la fragmentación subsiguiente de un paquete de interred que contiene varios fragmentos no produce problemas. El límite último aquí es hacer que el fragmento elemental sea un solo bit o byte, siendo entonces el número de fragmento el desplazamiento del bit o byte en el paquete original, como se muestra en la figura 5-42.

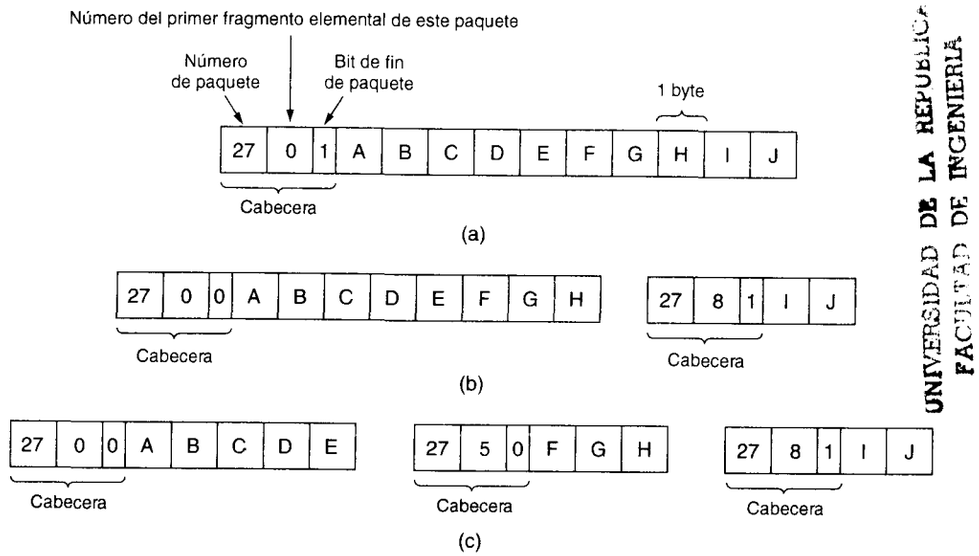


Figura 5-42. Fragmentación cuando el tamaño de datos elemental es de 1 byte. (a) Paquete original, que contiene 10 bytes de datos. (b) Fragmentos tras pasar a través de una red con un tamaño máximo de paquete de 8 bytes. (c) Fragmentos tras pasar a través de una pasarela de tamaño 5.

Algunos protocolos de interred llevan este método aún más lejos y consideran a toda la transmisión a través de un circuito virtual como un paquete gigantesco, por lo que cada fragmento contiene el número absoluto de byte del primer byte del fragmento. En (Kent y Mogul, 1987) se estudian algunos otros temas relacionados con la fragmentación.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

5.4.7. Muros de seguridad

La capacidad de conectar una computadora, donde sea, a otra computadora, también donde sea, es una bendición y una maldición. Para los individuos en casa, vagar por Internet es muy divertido. Para los administradores de seguridad de las corporaciones, es una pesadilla. La mayor parte de las compañías tiene grandes cantidades de información confidencial en línea (secretos industriales, planes de desarrollo de productos, estrategias de mercadeo, análisis financieros, etc.). La divulgación de esta información a un competidor podría tener consecuencias desastrosas.

Además del peligro de la fuga de información, también hay el peligro de la entrada de información no deseada. En particular, los virus, gusanos y otras plagas digitales (Kaufman *et al.*, 1995) pueden romper la seguridad, destruir información valiosa y desperdiciar grandes cantidades de tiempo del administrador tratando de poner en orden el desastre que dejan. Con frecuencia estas plagas son importadas por empleados descuidados que quieren jugar un nuevo y apasionante juego.

En consecuencia, se requieren mecanismos para mantener adentro los bits "buenos" y fuera los bits "malos". Un método es el cifrado. Este enfoque protege los datos en tránsito entre instalaciones seguras; lo estudiaremos en el capítulo 7. Sin embargo, el cifrado no hace nada para evitar el acceso de las plagas digitales y de los *hackers*. Para lograr esta meta, necesitamos ver los muros de seguridad (Chapman y Zwicky, 1995, y Cheswick y Bellovin, 1994).

Los **muros de seguridad** son simplemente una adaptación moderna del viejo sistema de seguridad medieval: un foso profundo alrededor del castillo. Este diseño obligaba a cualquiera que entrara o saliera del castillo a pasar por un solo puente levadizo, donde podía ser inspeccionado por la policía de E/S. En las redes, es posible el mismo truco: una compañía puede tener muchas LAN conectadas de manera arbitraria, pero todo el tráfico de o a la compañía es obligado a pasar a través de un puente levadizo electrónico (muro de seguridad), como se muestra en la figura 5-43.

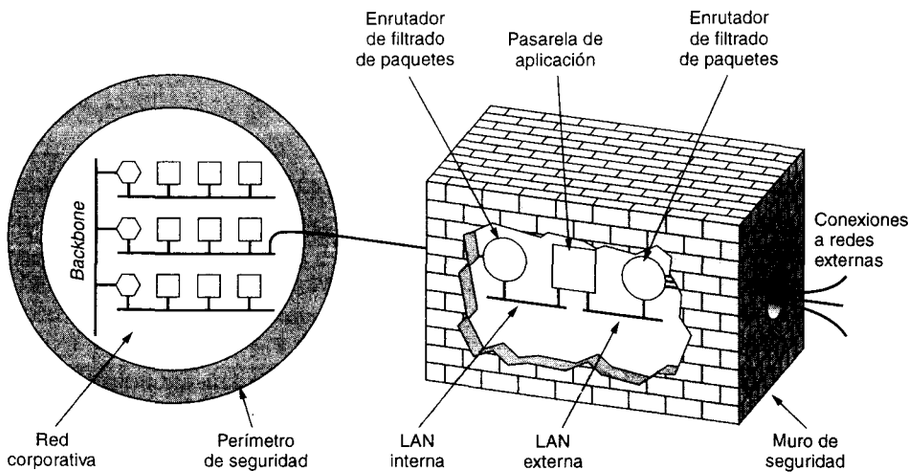


Figura 5-43. Muro de seguridad que consiste en dos filtros de paquetes y una pasarela de aplicación.

El muro de seguridad en esta configuración tiene dos componentes: dos enrutadores que filtran paquetes y una pasarela de aplicación. Existen configuraciones más sencillas, pero la ventaja de este diseño es que cada paquete debe transitar por dos filtros y una pasarela de aplicación para entrar o salir. No existe otra ruta. Los lectores que piensen que un solo punto de revisión es suficiente con toda seguridad no han tomado recientemente un vuelo internacional en una línea aérea comercial.

Cada **filtro de paquetes** es un enrutador estándar equipado con cierta funcionalidad extra, la cual permite la inspección de cada paquete de entrada o salida. Los paquetes que satisfacen algún criterio se reenvían normalmente. Los que fallan la prueba se descartan.

En la figura 5-43, lo más común es que el filtro de paquetes de la LAN interior revise los paquetes de salida, y el de la LAN exterior revise los paquetes de entrada. Los paquetes que saltan la primera valla pasan a la pasarela de aplicación para otra revisión. El objetivo de poner los dos filtros de paquetes en LAN diferentes es asegurar que ningún paquete entre o salga sin tener que pasar a través de la pasarela de aplicación; no hay ningún camino alrededor de ella.

Los filtros de paquetes comúnmente son manejados por tablas configuradas por el administrador del sistema. En estas tablas se listan los orígenes y los destinos aceptables, los orígenes y los destinos bloqueados, y las reglas predeterminadas sobre las acciones a tomar con paquetes que llegan de, o salen a, otras máquinas.

En el caso común de una instalación UNIX, un origen o destino consiste en una dirección IP y un puerto. Los puertos indican el servicio deseado. Por ejemplo, el puerto 23 es para Telnet, el puerto 79 es para Finger y el puerto 119 es para noticias USENET. Una compañía podría bloquear los paquetes de entrada de todas las direcciones de IP combinadas con uno de estos puertos. De esta manera, nadie fuera de la compañía podría establecer una sesión a través de Telnet, o buscar personas usando el *daemon* Finger. Es más, la compañía evitaría que sus usuarios pasaran todo el día leyendo las noticias de USENET.

El bloqueo de paquetes de salida es más difícil porque, aunque la mayoría de las instalaciones se apegan a las convenciones estándar de nombres de puertos, no están obligadas a hacerlo. Además, en algunos servicios importantes, como FTP (*File Transfer Protocol*, protocolo de transferencia de archivos), los números de puerto se asignan dinámicamente. Además, aunque el bloqueo de conexiones TCP es difícil, el bloqueo de paquetes UDP es aún más difícil porque se sabe muy poco de antemano sobre lo que harán. Muchos filtros de paquetes simplemente excluyen por completo el tráfico UDP.

La segunda mitad del mecanismo de muro de seguridad es la **pasarela de aplicación**. En lugar de simplemente ver paquetes en bruto, la pasarela opera en el nivel de aplicación. Por ejemplo, puede establecerse una pasarela de correo para examinar cada mensaje que entra o sale. Para cada uno, la pasarela decide si debe transmitirlo o descartarlo con base en los campos de cabecera, tamaño del mensaje e inclusive el contenido (por ejemplo, en una instalación militar, la presencia de palabras como "nuclear" o "bomba" podrían causar una acción especial).

Las instalaciones están en libertad de establecer una o más pasarelas de aplicación para aplicaciones específicas, pero no es poco común que las organizaciones suspicaces permitan el correo electrónico de entrada y de salida, y tal vez el uso de la *World Wide Web*, pero prohíban cualquier

otra cosa como demasiado peligrosa. En combinación con el cifrado y el filtrado de paquetes, este arreglo ofrece una cantidad limitada de seguridad al precio de algunos inconvenientes.

Una nota final se refiere a la comunicación inalámbrica y a los muros de seguridad. Es fácil diseñar un sistema que, desde el punto de vista lógico, sea completamente seguro pero que, en la práctica, tenga tantas fugas como una coladera. Esta situación puede ocurrir si algunas de las máquinas son inalámbricas y usan radiocomunicación, que pasa por encima del muro de seguridad en ambas direcciones.

5.5. LA CAPA DE RED EN INTERNET

En la capa de red, la Internet puede verse como un conjunto de subredes, o **sistemas autónomos** (**AS**, *Autonomous System*) interconectados. No hay una estructura real, pero existen varios *backbone* principales. Éstos se construyen a partir de líneas de alto ancho de banda y enrutadores rápidos. Conectadas a los *backbone* hay redes regionales (de nivel medio), y conectadas a estas redes regionales están las LAN de muchas universidades, compañías y proveedores de servicio Internet. En la figura 5-44 se presenta un dibujo de esta organización cuasijerárquica.

El pegamento que mantiene unida la Internet es el protocolo de capa de red, **IP** (*Internet Protocol*, **protocolo Internet**). A diferencia de la mayoría de los protocolos de capa de red anteriores, éste se diseñó desde el principio con la interconexión de redes en mente. Una buena manera de visualizar la capa de red es la siguiente. Su trabajo es proporcionar un medio de mejor

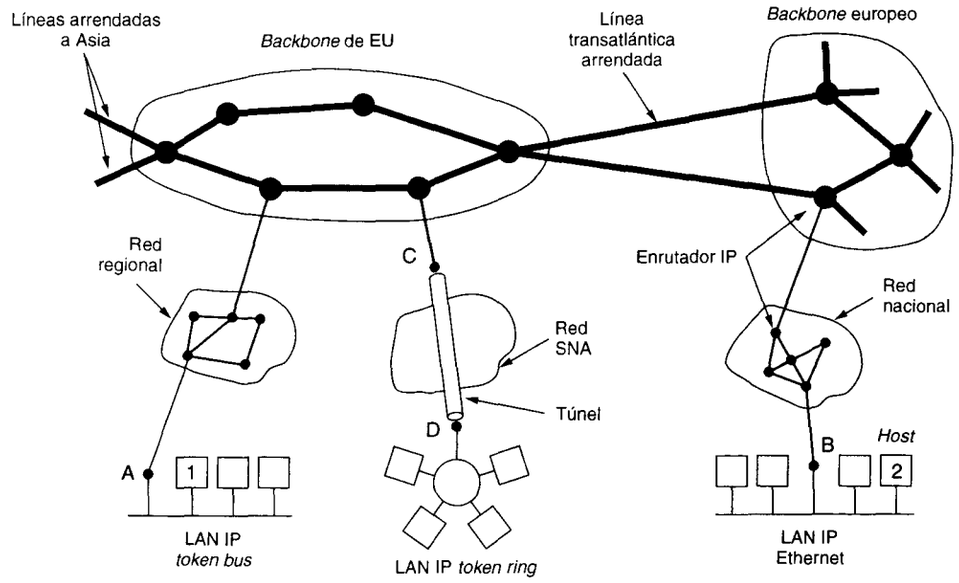


Figura 5-44. La Internet es un conjunto interconectado de muchas redes.

esfuerzo para el transporte de datagramas del origen al destino, sin importar si estas máquinas están en la misma red, o si hay otras redes entre ellas.

La comunicación en Internet funciona como sigue. La capa de transporte toma corrientes de datos y las divide en datagramas. En teoría, los datagramas pueden ser de hasta 64 Kbytes cada uno, pero en la práctica por lo general son de unos 1500 bytes. Cada datagrama se transmite a través de Internet, posiblemente fragmentándose en unidades más pequeñas en el camino. Cuando todas las piezas llegan finalmente a la máquina de destino, son reensambladas por la capa de red, dejando el datagrama original. Este datagrama entonces es entregado a la capa de transporte, que lo introduce en la corriente de entrada del proceso receptor.

5.5.1. El protocolo IP

Un lugar adecuado para comenzar nuestro estudio de la capa de red de Internet es el formato de los datagramas de IP mismos. Un datagrama IP consiste en una parte de cabecera y una parte de texto. La cabecera tiene una parte fija de 20 bytes y una parte opcional de longitud variable. El formato de la cabecera se muestra en la figura 5-45. Se transmite en orden *big endian*:* de izquierda a derecha, comenzando por el bit de orden mayor del campo de *versión*. (SPARC es *big endian*; Pentium es *little endian*.) En las máquinas *little endian*, se requiere conversión por *software* tanto para la transmisión como para la recepción.

El campo de *versión* lleva el registro de la versión del protocolo al que pertenece el datagrama. Al incluir la versión en cada datagrama es posible hacer que la transición entre versiones se lleve

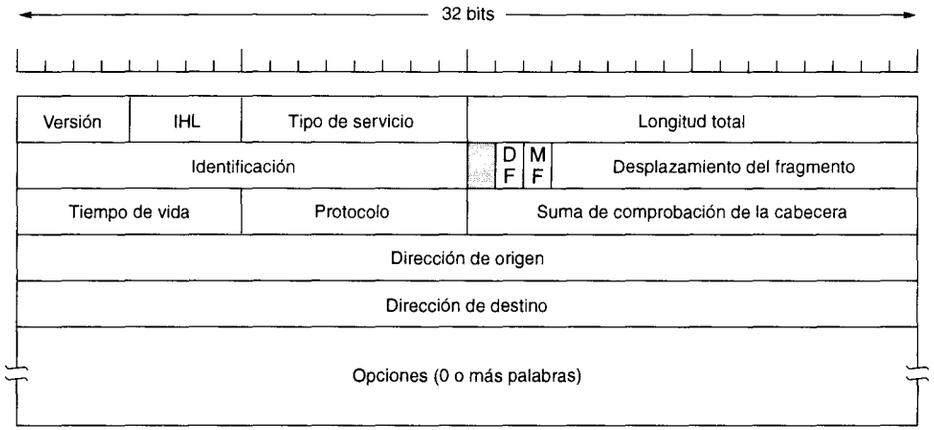


Figura 5-45. La cabecera de IP (protocolo de Internet).

* En *Los viajes de Gulliver*, de Jonathan Swift, las naciones de Lilliput y Blefescu estaban en guerra a causa de sus diferencias en cuanto a la forma correcta de comer huevos: rompiéndolos por el extremo angosto (*little end*) o por el más ancho (*big end*). En computación se aplican los términos *little endian* y *big endian* a tecnologías similares pero incompatibles que, a pesar de que no existe una razón de peso para preferir una a la otra, son defendidas con vehemencia por sus respectivos partidarios. (*N. del supervisor.*)

meses, o inclusive años, ejecutando algunas máquinas la versión vieja y otras la versión nueva.

Dado que la longitud de la cabecera no es constante, se incluye un campo en la cabecera, *IHL*, para indicar la longitud en palabras de 32 bits. El valor mínimo es de 5, cifra que aplica cuando no hay opciones. El valor máximo de este campo de 4 bits es de 15, lo que limita la cabecera a 60 bytes y, por tanto, el campo de opciones a 40 bytes. Para algunas opciones, por ejemplo para una que registre la ruta que ha seguido un paquete, 40 bytes es muy poco, lo que hace inútil esta opción.

El campo de *tipo de servicio* permite al *host* indicar a la subred el tipo de servicio que quiere. Son posibles varias combinaciones de confiabilidad y velocidad. Para voz digitalizada, la entrega rápida le gana a la entrega precisa. Para la transferencia de archivos, es más importante la transmisión libre de errores que la transmisión rápida.

El campo mismo contiene (de izquierda a derecha) un campo de *precedencia*; tres indicadores, *D, T* y *R*; y dos bits no usados. El campo de *precedencia* es una prioridad, de 0 (normal) a 7 (paquete de control de red). Los tres bits indicadores permiten al *host* especificar lo que le interesa más del grupo {retardo (*delay*), rendimiento (*throughput*), confiabilidad (*reliability*)}. En teoría, estos campos permiten a los enrutadores tomar decisiones entre, por ejemplo, un enlace satelital de alto rendimiento y alto retardo o una línea arrendada con bajo rendimiento y poco retardo. En la práctica, los enrutadores actuales ignoran por completo el campo de *tipo de servicio*.

La *longitud total* incluye todo el datagrama: tanto la cabecera como los datos. La longitud máxima es de 65,535 bytes. Actualmente este límite es tolerable, pero con las redes futuras de gigabits se requerirán datagramas más grandes.

El campo de *identificación* es necesario para que el *host* de destino determine a qué datagrama pertenece un fragmento recién llegado. Todos los fragmentos de un datagrama contienen el mismo valor de *identificación*.

A continuación viene un bit sin uso y luego dos campos de 1 bit. *DF* significa no fragmentar (*Don't Fragment*); es una orden para los enrutadores de que no fragmenten el datagrama, porque el destino es incapaz de juntar las piezas de nuevo. Por ejemplo, al arrancar una computadora, su ROM podría pedir el envío de una imagen de memoria a ella como un solo datagrama. Al marcar el datagrama con el bit *DF*, el transmisor sabe que llegará en una pieza, aún si significa que el datagrama debe evitar una red de paquete pequeño en la mejor trayectoria y tomar una ruta subóptima. Se requiere que todas las máquinas acepten fragmentos de 576 bytes o menos.

MF significa más fragmentos. Todos los fragmentos excepto el último tienen establecido este bit, que es necesario para saber cuándo han llegado todos los fragmentos de un datagrama.

El *desplazamiento del fragmento* indica en qué parte del datagrama actual va este fragmento. Todos los fragmentos excepto el último del datagrama deben tener un múltiplo de 8 bytes, que es la unidad de fragmento elemental. Dado que se proporcionan 13 bits, puede haber un máximo de 8192 fragmentos por datagrama, dando una longitud máxima de datagrama de 65,536 bytes, uno más que el campo de *longitud total*.

El campo de *tiempo de vida* es un contador que sirve para limitar la vida de un paquete. Se supone que este contador cuenta el tiempo en segundos, permitiendo una vida máxima de 255 seg; debe disminuirse en cada salto y se supone que disminuye muchas veces al encolarse durante un tiempo grande en un enrutador. En la práctica, simplemente cuenta los saltos. Cuando

el contador llega a cero, el paquete se descarta y se envía de regreso un paquete de aviso al *host* de origen. Esta característica evita que los datagramas vaguen eternamente, algo que de otra manera podría ocurrir si se llegan a corromper las tablas de enrutamiento.

Una vez que la capa de red ha ensamblado un datagrama completo, necesita saber qué hacer con él. El campo de *protocolo* indica la capa de transporte a la que debe entregarse. TCP es una posibilidad, pero también está UDP y algunos más. La numeración de los protocolos es global en toda la Internet, y se define en el RFC 1700.

La *suma de comprobación de la cabecera* verifica solamente la cabecera. Tal suma de comprobación es útil para la detección de errores generados por palabras de memoria erróneas en un enrutador. El algoritmo es sumar todas las medias palabras de 16 bits a medida que llegan, usando aritmética de complemento a uno, y luego obtener el complemento a uno del resultado. Para los fines de este algoritmo, se supone que la *suma de comprobación de la cabecera* es cero cuando llega. Este algoritmo es más robusto que una suma normal. Nótese que la *suma de comprobación de la cabecera* debe recalcularse en cada salto, pues cuando menos uno de los campos siempre cambia (el campo de *tiempo de vida*), pero pueden usarse trucos para acelerar el cálculo.

La *dirección de origen* y la *dirección de destino* indican el número de red y el número de *host*. Estudiaremos las direcciones de Internet en la siguiente sección. El campo de *opciones* se diseñó para proporcionar un recurso que permitiera que las versiones subsiguientes del protocolo incluyeran información no presente en el diseño original, para permitir a los experimentadores probar ideas nuevas y para evitar la asignación de bits de cabecera a información pocas veces necesaria. Las opciones son de longitud variable. Cada una empieza con un código de 1 byte que identifica la opción. Algunas opciones vienen seguidas de un campo de longitud de la opción de 1 byte, y luego de uno o más bytes de datos. El campo de *opciones* se rellena para completar múltiplos de cuatro bytes. Actualmente hay cinco opciones definidas, las que se listan en la figura 5-46, pero no todos los enrutadores reconocen a todas.

La opción de *seguridad* indica qué tan secreta es la información. En teoría, un enrutador militar puede usar este campo para especificar que no se enrute a través de ciertos países que los militares consideren "malos". En la práctica, todos los enrutadores lo ignoran, por lo que su única función real es la de ayudar a los espías a encontrar la información importante con mayor facilidad.

La opción de *enrutamiento estricto desde el origen* da la trayectoria completa desde el origen hasta el destino como secuencia de direcciones IP. Se requiere que el datagrama siga esa

Opción	Descripción
Seguridad	Especifica qué tan secreto es el datagrama
Enrutamiento estricto desde el origen	Indica la trayectoria completa a seguir
Enrutamiento libre desde el origen	Da una lista de los enrutadores que no deben evitarse
Registrar ruta	Hace que cada enrutador agregue su dirección de IP
Marca de tiempo	Hace que cada enrutador agregue su dirección y su marca de tiempo

Figura 5-46. Opciones del IP.

ruta exacta. Esta opción se usa sobre todo cuando los administradores de sistemas envíen paquetes de emergencia porque las tablas de enrutamiento se han corrompido, o para hacer mediciones de tiempo.

La opción de *enrutamiento libre desde el origen* requiere que el paquete pase por los enrutadores indicados en la lista, y en el orden especificado, pero se le permite pasar a través de otros enrutadores en el camino. Normalmente, esta opción sólo indicará algunos enrutadores, para obligar a una trayectoria en particular. Por ejemplo, si se desea obligar a un paquete de Londres a Sydney a ir hacia el oeste en lugar de hacia el este, esta opción podría especificar enrutadores en Nueva York, Los Ángeles y Honolulú. Esta opción es de mucha utilidad cuando las consideraciones políticas o económicas dictan pasar a través de, o evitar, ciertos países.

La opción de *registrar ruta* indica a los enrutadores a lo largo de la trayectoria que agreguen su dirección de IP al campo de opción. Esto permite a los administradores del sistema buscar fallas en los algoritmos de enrutamiento ("¿por qué todos los paquetes de Houston a Dallas pasan por Tokio primero?"). Al establecerse inicialmente ARPANET, ningún paquete pasaba nunca por más de nueve enrutadores, por lo que 40 bytes de opciones eran más que suficientes. Como se mencionó antes, ahora esto es demasiado poco.

Por último, la opción de *marca de tiempo* es como la opción de *registrar ruta*, excepto que además de registrar su dirección IP de 32 bits, cada enrutador también registra una marca de tiempo de 32 bits. Esta opción también es principalmente para búsqueda de fallas en los algoritmos de enrutamiento.

5.5.2. Direcciones IP

Cada *host* y enrutador de Internet tiene una dirección de IP, que codifica su número de red y su número de *host*. La combinación es única: no hay dos máquinas que tengan la misma dirección de IP. Todas las direcciones de IP son de 32 bits de longitud y se usan en los campos de *dirección de origen* y *dirección de destino* de los paquetes IP. Los formatos usados para las direcciones IP se muestran en la figura 5-47. Aquellas máquinas conectadas a varias redes tienen direcciones de IP diferentes en cada red.

Los formatos de clase A, B, C y D permiten hasta 126 redes con 16 millones de *hosts* cada una, 16,382 redes con hasta 64K *hosts*, 2 millones de redes (por ejemplo, LAN) de hasta 254 *hosts* cada una, y multitransmisión, en la cual se dirige un datagrama a múltiples *hosts*. Las direcciones que comienzan con 11110 se reservan para uso futuro. Hay decenas de miles de redes conectadas ahora a Internet, y la cifra se duplica cada año. Los números de red los asigna el **NIC** (*Network Information Center*, **centro de información de redes**) para evitar conflictos.

Las direcciones de red, que son números de 32 bits, generalmente se escriben en **notación decimal con puntos**. En este formato, cada uno de los 4 bytes se escribe en decimal, de 0 a 255. Por ejemplo, la dirección hexadecimal C0290614 se escribe como 192.41.6.20. La dirección de IP menor es 0.0.0.0 y la mayor 255.255.255.255.

Los valores 0 y −1 tienen significado especial, como se muestra en la figura 5-48. El valor 0 significa esta red o este *host*. El valor −1 se usa como dirección de difusión para indicar todos los *hosts* de la red indicada.

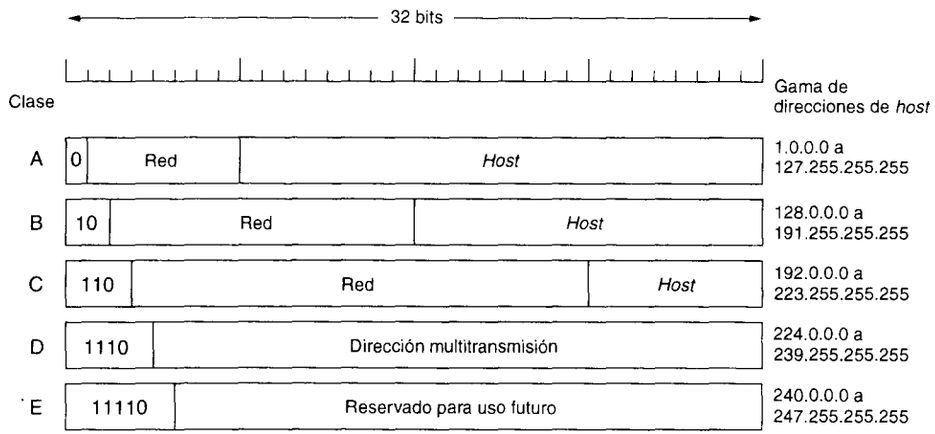


Figura 5-47. Formatos de dirección IP.

La dirección de IP 0.0.0.0 es usada por los *hosts* cuando están siendo arrancados, pero no se usa después. Las direcciones de IP con 0 como número de red se refieren a la red actual. Estas direcciones permiten que las máquinas se refieran a su propia red sin saber su número (pero tienen que saber su clase para saber cuántos 0 hay que incluir). La dirección que consiste solamente en unos permite la difusión en la red local, por lo común una LAN. Las direcciones con un número de red propio y solamente unos en el campo de *host* permite que las máquinas envíen paquetes de difusión a LAN distantes desde cualquier parte de Internet. Por último, todas las direcciones de la forma 127.xx.yy.zz se reservan para pruebas de realimentación. Los paquetes enviados a esa dirección no se colocan en el alambre; se procesan localmente y se tratan como paquetes de entrada. Esto permite que los paquetes se envíen a la red local sin que el transmisor conozca su número. Esta característica también se usa para la detección de fallas en el *software* de la red.

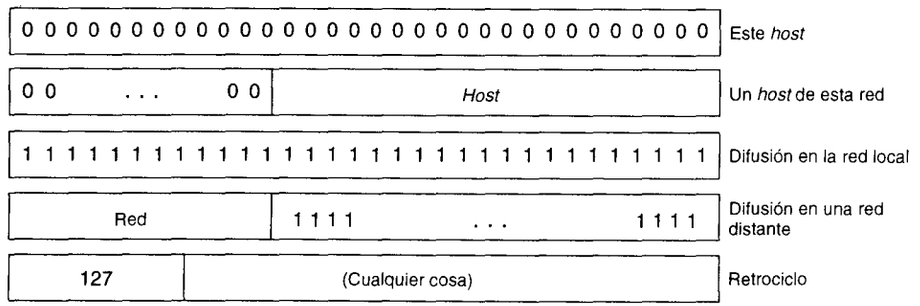


Figura 5-48. Direcciones especiales de IP.

5.5.3. Subredes

Como hemos visto, todos los *host* de una red deben tener el mismo número de red. Esta propiedad del direccionamiento IP puede causar problemas a medida que crecen las redes. Por ejemplo, considere una compañía que comienza con una LAN clase C en Internet. A medida que pasa el tiempo, podría adquirir más de 254 máquinas, y por tanto necesitar una segunda dirección clase C. De la misma manera, podría adquirir una segunda LAN de un tipo diferente y querer una dirección de IP diferente para ella (las LAN podrían estar unidas mediante un puente para formar una sola red IP, pero los puentes tienen sus propios problemas). Finalmente, podría terminar con muchas LAN, cada una con su propio enrutador y su propio número de red clase C.

Al crecer la cantidad de redes locales distintas, su administración puede volverse un dolor de cabeza grave. Cada vez que se instala una red nueva, el administrador del sistema tiene que comunicarse con el NIC para obtener un número de red nuevo. Después, este número debe anunciarse mundialmente. Además, mover una máquina de una LAN a otra requiere el cambio de su dirección de IP, lo que a su vez puede significar la modificación de sus archivos de configuración y también el anuncio de la nueva dirección de IP a todo el mundo. Si a otra máquina se le da la dirección de IP recién liberada, esa máquina recibirá el correo electrónico y otros datos dirigidos a la máquina original hasta que la dirección se haya propagado por todo el mundo.

La solución a este problema es permitir la división de una red en varias partes para uso interno, pero aún actuar como una sola red ante el mundo exterior. En la literatura sobre Internet, a estas partes se les llama **subredes**. Como mencionamos en el capítulo 1, este uso entra en conflicto con la "subred" cuyo significado es el grupo de todos los enrutadores y líneas de comunicación de una red. Esperamos que el contexto deje en claro el significado de que se trata. En esta sección usaremos la nueva definición. Si nuestra compañía en crecimiento tenía inicialmente una dirección clase B en lugar de clase C, puede comenzar simplemente por numerar los *hosts* de 1 a 254. Al llegar la segunda LAN podría decidir, por ejemplo, dividir el número de *host* de 16 bits en un número de subred de 6 bits y un número de *host* de 10 bits, como se muestra en la figura 5-49. Esta división permite 62 LAN (se reservan el 0 y el –1) cada una con hasta 1022 *hosts*.

Fuera de la red, la subred no es visible, por lo que la asignación de una subred nueva no requiere comunicación con el NIC ni la modificación de bases de datos externas. En este ejemplo, la primera subred podría usar direcciones de IP a partir de 130.50.4.1, la segunda subred podría empezar en 130.50.8.1, etcétera.

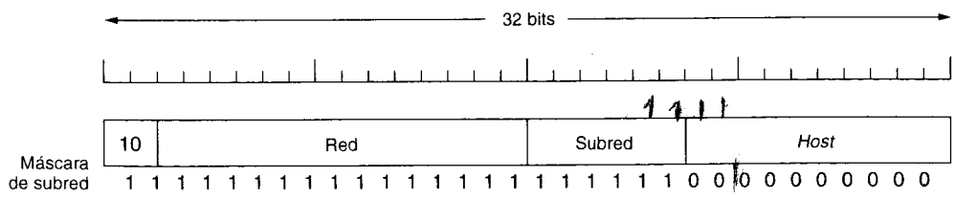


Figura 5-49. Una de las formas de generar una subred clase B.

Para ver el funcionamiento de las subredes, es necesario explicar la manera en que se procesan los paquetes IP en un enrutador. Cada enrutador tiene una tabla en la que se lista cierto número de direcciones IP (red, 0) y cierto número de direcciones IP (esta red, *host*). El primer tipo indica cómo llegar a redes distantes. El segundo tipo indica cómo llegar a redes locales. Asociada a cada tabla está la interfaz de red a usar para llegar al destino y cierta información adicional.

Al llegar un paquete de IP, se busca su dirección de destino en la tabla de enrutamiento. Si el paquete es para una red distante, se reenvía al siguiente enrutador de la interfaz dada en la tabla; si es para un *host* local (por ejemplo, en la LAN del enrutador), se envía directamente al destino. Si la red no está en la tabla, el paquete se reenvía a un enrutador predeterminado con tablas más extensas. Este algoritmo significa que cada enrutador sólo tiene que llevar el registro de otras redes y *hosts* locales, no de pares red-*host*, reduciendo en gran medida el tamaño de la tabla de enrutamiento.

Al introducirse subredes, se cambian las tablas de enrutamiento, agregando entradas con forma de (esta red, subred, 0) y (esta red, esta subred, *host*). Por tanto, un enrutador de la subred *k* sabe cómo llegar a todas las demás subredes y también cómo llegar a todos los *hosts* de la subred *k*; no tiene que saber los detalles sobre los *hosts* de otras subredes. De hecho, todo lo que se necesita es hacer que cada enrutador haga un AND booleano con la **máscara de subred** (véase la figura 5-49) para deshacerse del número de *host* y buscar la dirección resultante en sus tablas (tras determinar la clase de red de la que se trata). Por ejemplo, a un paquete dirigido a 130.50.15.6 que llega a un enrutador de la subred 5 se le hace un AND con la máscara de subred de la figura 5-49 para dar la dirección 130.50.12.0. Esta dirección se busca en las tablas de enrutamiento para averiguar la manera de llegar a los *hosts* de la subred 3. Así, el enrutador de la subred 5 se ahorra el trabajo de mantener un registro de las direcciones de enlace de datos de *hosts* que no pertenecen a la subred 5. El proceso de subred reduce así el espacio de tabla de enrutamiento creando una jerarquía de tres niveles.

5.5.4. Protocolos de control de Internet

Además del IP, que se usa para la transferencia de datos, Internet tiene varios protocolos de control que se usan en la capa de red, incluidos ICMP, ARP, RARP y BOOTP. En esta sección veremos cada uno de ellos.

Protocolo de control de mensajes de Internet

La operación de Internet es supervisada cuidadosamente por los enrutadores. Al ocurrir algo inesperado, el **ICMP** (*Internet Control Message Protocol*, **protocolo de control de mensajes de Internet**), que también se usa para probar Internet, informa del suceso. Se ha definido una docena de tipos de mensajes de ICMP; los más importantes se listan en la figura 5-50. Cada tipo de mensaje de ICMP se encapsula en un paquete IP.

El mensaje DESTINO INALCANZABLE (*DESTINATION UNREACHABLE*) se usa cuando la subred o un enrutador no pueden ubicar el destino, o un paquete con el bit *DF* no puede entregarse porque está en el camino una red de "paquete pequeño".

El mensaje de *tiempo excedido* (*TIME EXCEEDED*) se envía cuando un paquete se descarta debido a que su contador llega a cero. Este suceso es un síntoma de que los paquetes están en

Tipo de mensaje	Descripción
Destino inalcanzable	No pudo entregarse el paquete
Tiempo excedido	Campo de tiempo de vida llegó a cero
Problema de parámetro	Campo de cabecera no válido
Supresión de origen	Paquete de estrangulamiento
Reenvío	Enseña geografía a un enrutador
Solicitud de eco	Pregunta a una máquina si está viva
Respuesta de eco	Sí, estoy viva
Solicitud de marca de tiempo	Igual que la solicitud de eco, pero con marca de tiempo
Respuesta de marca de tiempo	Igual que la respuesta de eco, pero con marca de tiempo

Figura 5-50. Los principales tipos de mensajes ICMP.

ciclo, de que hay un congestionamiento enorme, o de que los valores de temporización son demasiado bajos.

El mensaje de PROBLEMA DE PARÁMETRO (*PARAMETER PROBLEM*) indica que se ha detectado un valor ilegal en un campo de cabecera. Este problema indica una falla en el *software* de IP del *host*, o posiblemente en el *software* de un enrutador transitado.

El mensaje SUPRESIÓN DE ORIGEN (*SOURCE QUENCH*) se usaba antes para controlar a los *hosts* que enviaban demasiados paquetes. Al recibir un *host* este mensaje, se esperaba que se refrenara. Este mensaje se usa poco en la actualidad porque, al ocurrir congestionamientos, estos paquetes tienden a echarle más leña al fuego. El control de congestionamiento de Internet se hace ahora en gran medida en la capa de transporte, y se estudiará en el capítulo 6.

El mensaje de REDIRECCIONAMIENTO (*REDIRECT*) se usa cuando un enrutador se da cuenta de que un paquete parece estar mal enrutado el enrutador lo usa para indicar al *host* transmisor el posible error.

Los mensajes de SOLICITUD DE ECO (*ECHO REQUEST*) y RESPUESTA DE ECHO (*ECHO REPLAY*) sirven para ver si un destino dado es alcanzable y está vivo. Al recibir el mensaje de ECO, se espera que el destino devuelva un mensaje de RESPUESTA DE ECO. Los mensajes SOLICITUD DE MARCA DE TIEMPO (*TIMESTAMP REQUEST*) y RESPUESTA DE MARCA DE TIEMPO (*TIMESTAMP REPLY*) son parecidos, excepto que el tiempo de llegada del mensaje y el tiempo de partida de la respuesta se registran en la respuesta. Este recurso se emplea para medir el desempeño de la red.

Además de estos mensajes, hay cuatro más que tienen que ver con el direccionamiento de Internet, para permitir que los *hosts* descubran sus números de red y para manejar el caso de varias LAN que comparten una sola dirección IP. El ICMP se define en el RFC 792.

Protocolo de resolución de direcciones

Aunque cada máquina de Internet tiene una (o más) direcciones de IP, éstas no pueden usarse para enviar paquetes porque el *hardware* de la capa de enlace de datos no entiende las direcciones de Internet. Hoy día, la mayoría de los *hosts* están conectados a alguna LAN mediante una tarjeta de interfaz que sólo entiende direcciones de LAN. Por ejemplo, todas las tarjetas Ethernet que se han fabricado vienen equipadas con una dirección de Ethernet de 48 bits. Los fabricantes de tarjetas Ethernet solicitan un bloque de direcciones de una autoridad central para asegurarse de que dos tarjetas no puedan tener la misma dirección (para evitar conflictos en caso de que las dos tarjetas aparezcan en la misma LAN). Las tarjetas envían y reciben marcos con base en direcciones de Ethernet de 48 bits. No saben nada sobre las direcciones de IP de 32 bits.

Ahora surge la pregunta: ¿cómo se proyectan las direcciones de IP en las direcciones de capa de enlace de datos, como las de Ethernet? Para explicar el funcionamiento de esto, usemos el ejemplo de la figura 5-51, en la que se ilustra una universidad pequeña con varias redes clase C. Aquí tenemos dos Ethernet, uno en el departamento de informática, con una dirección IP de 192.31.65.0, y otra en el de ingeniería electrónica, con dirección de IP de 192.31.63.0. Éstas se conectan mediante un anillo FDDI del campus con una dirección IP de 192.31.60.0. Cada máquina de un Ethernet tiene una dirección de Ethernet única, etiquetada *E1* a *E6*, y cada máquina del anillo FDDI tiene una dirección FDDI, etiquetada *F1* a *F3*.

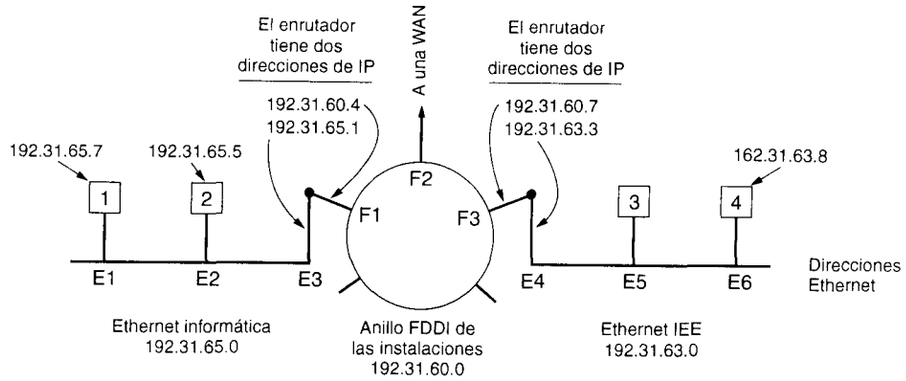


Figura 5-51. Tres redes clase C interconectadas; dos Ethernets y un anillo FDDI.

Comencemos por ver la manera en que un usuario del *host* 1 envía un paquete a un usuario del *host* 2. Supongamos que el transmisor sabe el nombre del receptor pretendido, posiblemente algo como *maria@eagle.cs.uni.edu*. El primer paso es encontrar la dirección IP del *host* 2, conocido como *eagle.cs.uni.edu*. Esta búsqueda la lleva a cabo el sistema de nombres de dominio (DNS, *Domain Name System*) que estudiaremos en el capítulo 7. Por lo pronto, simplemente supondremos que el DNS devuelve la dirección IP del *host* 2 (192.31.65.5).

El *software* de capa superior del *host* 1 construye ahora un paquete con 192.31.65.5 en el campo de *dirección de destino* y lo entrega al *software* de IP para su transmisión. El *software* de IP puede ver la dirección y saber si el destino está en su propia red, pero necesita una manera de encontrar la dirección Ethernet del destino. Una solución es tener un archivo de configuración en algún lugar del sistema que proyecte las direcciones IP en las direcciones Ethernet. Esta

solución ciertamente es posible pero, en organizaciones con miles de máquinas, el mantenimiento de estos archivos es una tarea susceptible a errores y con alto consumo de tiempo.

Una mejor solución es que el *host* 1 envíe un paquete de difusión por el Ethernet preguntando: "¿quién es el dueño de la dirección IP 192.31.65.5?" La difusión llegará a cada máquina del Ethernet 192.31.65.0, y cada una revisará su propia dirección IP. Sólo el *host* 2 responderá con su dirección Ethernet (*E2*). De esta manera, el *host* 1 aprende que la dirección de IP 192.31.65.5 está en el *host* con la dirección Ethernet *E2*. El protocolo para preguntar esto y obtener la respuesta se llama **ARP** (*Address Resolution Protocol*, **protocolo de resolución de direcciones**). Casi todas las máquinas de Internet lo ejecutan. Este protocolo se define en el RFC 826.

La ventaja del ARP sobre los archivos de configuración es la sencillez. El administrador del sistema no necesita hacer mucho, excepto asignar a cada máquina una dirección IP y decidir sobre las máscaras de subred. El ARP hace lo demás.

En este punto, el *software* de IP del *host* 1 construye un marco Ethernet dirigido a *E2*, pone el paquete IP (dirigido a 192.31.65.5) en el campo de carga útil, y lo arroja al Ethernet. La tarjeta de Ethernet del *host* 2 detecta este marco, lo reconoce como un marco para él mismo, lo recoge y causa una interrupción. El operador de Ethernet extrae el paquete IP de la carga útil y lo pasa al *software* de IP, que nota que está correctamente dirigido, y lo procesa.

Son posibles varias optimizaciones para hacer más eficiente el ARP. Por principio de cuentas, una vez que una máquina ha ejecutado el ARP, pone en caché el resultado por si necesita establecer contacto con la misma máquina pronto. La siguiente vez encontrará la proyección en su propio caché, eliminando por tanto la necesidad de una segunda difusión. En muchos casos, el *host* 2 necesitará devolver una respuesta, lo que lo obliga también a ejecutar el ARP para determinar la dirección Ethernet del transmisor. Esta transmisión ARP puede evitarse haciendo que el *host* 1 incluya su proyección de IP a Ethernet en el paquete ARP. Al llegar la difusión del ARP al *host* 2, se ingresa el par (192.31.65.7, E1) en el caché de ARP del *host* 2 para su uso futuro. De hecho, todas las máquinas que operan con Ethernet pueden ingresar esta proyección en sus cachés de ARP.

Otra optimización es hacer que cada máquina difunda su proyección al arranque. Esta difusión generalmente se hace en la forma de un ARP que busca su propia dirección de IP. No debe haber una respuesta, pero un efecto secundario de la difusión es crear una entrada en el caché de ARP de todos. Si llega una respuesta, es que dos máquinas tienen asignada la misma dirección de IP. La nueva debe informar de ello al administrador del sistema y no debe arrancar.

Para permitir el cambio de las proyecciones, por ejemplo, cuando se descompone una tarjeta Ethernet y se reemplaza por una nueva (y por tanto con una dirección Ethernet nueva), las entradas en el caché de ARP deben terminar su temporización la transcurrir algunos minutos.

Ahora veamos la figura 5-51 nuevamente, sólo que esta vez el *host* 1 quiere enviar un paquete al *host* 6 (192.31.63.8). El ARP fallará, porque el *host* 4 no verá la difusión (los enrutadores no reenvían las difusiones de nivel Ethernet). Hay dos soluciones. Primero, el enrutador CS podría configurarse para responder a las solicitudes de ARP para la red 192.31.63.0 (y posiblemente para otras redes locales). En este caso, el *host* 1 creará en el caché de ARP la entrada (192.31.63.8, E3) y felizmente enviará todo el tráfico para el *host* 4 al enrutador local. Esta

solución se llama **ARP apoderado** (*proxy* ARP). La segunda solución es hacer que el *host* 1 vea de inmediato que el destino está en una red remota y simplemente envíe todo ese tráfico a la dirección Ethernet predeterminada que maneja todo el tráfico remoto, en este caso *E3*. Esta solución no requiere hacer que el enrutador CS sepa a cuáles redes remotas sirve.

De cualquier manera, lo que ocurre es que el *host* 1 empaca el paquete IP en el campo de carga útil de un marco Ethernet dirigido a *E3*. Cuando el enrutador CS recibe el marco Ethernet, retira el paquete IP del campo de carga y busca la dirección IP en sus tablas de enrutamiento, descubriendo que los paquetes para la red 192.31.63.0 deben ir al enrutador 192.31.60.7. Si el enrutador no sabe ya la dirección FDDI de 192.31.60.7, difunde un paquete ARP por el anillo y se entera de que la dirección de su anillo es *F3*. El enrutador inserta entonces el paquete en el campo de carga útil de un marco FDDI dirigido a *F3* y lo pone en el anillo.

En el enrutador EE, el operador de FDDI retira el paquete del campo de carga útil y lo entrega al *software* de IP, el cual descubre que necesita enviar el paquete al 192.31.63.8. Si esta dirección no está en su caché de ARP, difunde una solicitud ARP por el Ethernet EE y se entera de que la dirección de destino es *E6*, por lo que construye un marco Ethernet dirigido a *E6*, pone el paquete en el campo de carga útil, y lo envía a través del Ethernet. Al llegar el marco Ethernet al *host* 4, el paquete se extrae del marco y se entrega al *software* de IP para su proceso.

Una transmisión del *host* 1 a una red distante a través de una WAN funciona en esencia de la misma manera, excepto que esta vez las tablas del enrutador CS le indican que use el enrutador WAN cuya dirección FDDI es *F2*.

Protocolo de resolución de direcciones en reversa

El ARP resuelve el problema de encontrar la dirección de Ethernet correspondiente a una dirección IP dada. A veces tiene que resolverse el problema inverso: dada una dirección Ethernet, ¿cuál es la dirección IP correspondiente? En particular, este problema ocurre al iniciarse una estación de trabajo sin disco. Tal máquina normalmente recibirá la imagen binaria de su sistema operativo de un servidor de archivos remoto, pero ¿cómo conoce su dirección de IP?

La solución es usar el **RARP** (*Reverse Address Resolution Protocol,* **protocolo de resolución de direcciones en reversa,** definido en el RFC 903). Este protocolo permite que una estación de trabajo recién iniciada difunda su dirección Ethernet y diga, "mi dirección Ethernet de 48 bits es 14.04.05.18.01.25. ¿Sabe alguien por ahí mi dirección de IP?" El servidor RARP ve esta solicitud, busca la dirección de Ethernet en sus archivos de configuración y envía de regreso la dirección de IP correspondiente.

El uso del RARP es preferible a integrar una dirección de IP en la imagen de memoria, pues permite usar la misma imagen para todas las máquinas. Si la dirección de IP estuviera incluido en la imagen, cada estación de trabajo necesitaría su propia imagen.

Una desventaja del RARP es que usa una dirección de destino que contiene únicamente unos (difusión limitada) para llegar al servidor RARP. Sin embargo, tales difusiones no son reenviadas por los enrutadores, por lo que se requiere un servidor RARP en cada red. Para superar este problema, se ha inventado un protocolo alterno de arranque llamado **BOOTP** (véanse los RFC

951, 1048 y 1084). A diferencia del RARP, BOOTP usa mensajes UDP, los cuales se reenvían a través de los enrutadores. Este protocolo también proporciona información adicional a una estación de trabajo sin disco, incluida la dirección IP del servidor de archivos que contiene la imagen de memoria, la dirección IP del enrutador predeterminado y la máscara de subred a usar. El BOOTP se describe en el RFC 951.

5.5.5. Protocolo de enrutamiento de pasarela interior: OSPF

Como mencionamos antes, la Internet se compone de una gran cantidad de sistemas autónomos (AS). Cada AS es operado por una organización diferente y puede usar internamente su propio algoritmo de enrutamiento. Por ejemplo, las redes internas de las compañías *X, Y* y *Z* generalmente se verían como tres AS si las tres estuvieran en Internet. Las tres pueden usar algoritmos de enrutamiento diferentes internamente. No obstante, la existencia de estándares, aun para enrutadores internos, simplifica la implementación en las líneas divisorias entre los AS y permite la reutilización de código. En esta sección estudiaremos el enrutamiento dentro de un AS; en el siguiente, veremos el enrutamiento entre varias AS. El algoritmo de enrutamiento interno de un AS se llama **protocolo de pasarela interior**; al algoritmo de enrutamiento entre varias AS se le llama **protocolo de pasarela exterior**.

El protocolo de pasarela interior original de Internet fue un protocolo de vector de distancia (RIP) basado en el algoritmo Bellman-Ford. Este protocolo funcionó bien en sistemas pequeños, pero menos bien a medida que los AS se volvieron más grandes; también padecía el problema de conteo a infinito y generalmente de una convergencia lenta, por lo que se reemplazó en mayo de 1979 por un protocolo de estado de enlace. En 1988, la *Internet Engineering Task Force* (grupo de trabajo de ingeniería de Internet) comenzó a trabajar en un sucesor. Ese sucesor, llamado **OSPF** (*Open Shortest Path First*, **abrir primero la trayectoria más corta**), se convirtió en estándar en 1990. Muchos fabricantes de enrutadores ya lo manejan, y se volverá el principal protocolo de pasarela interior en el futuro cercano. A continuación daremos una introducción al funcionamiento del OSPF. Para conocer la historia completa, véase el RFC 1247.

Dada la amplia experiencia con otros protocolos de enrutamiento, el grupo que diseñó el nuevo protocolo tenía una larga lista de requisitos a cumplir. Primero, el algoritmo tenía que publicarse como literatura abierta, de ahí la "O" (de *Open*) en OSPF. No serviría una solución patentada, que fuera propiedad de una compañía. Segundo, el nuevo protocolo tenía que reconocer una variedad de métricas de distancia, incluidas distancia física, retardo y otras. Tercero, tenía que ser un algoritmo dinámico, uno que se adaptara a los cambios de topología rápida y automáticamente.

Cuarto, algo nuevo para el OSPF, tenía que reconocer el enrutamiento basado en el tipo de servicio. El nuevo protocolo tenía que ser capaz de enrutar el tráfico de tiempo real de una manera y otros tipos de tráfico de otra manera. El protocolo IP tiene el campo de *tipo de servicio*, pero ningún protocolo de enrutamiento existente lo usaba.

Quinto, y relacionado con lo anterior, el nuevo protocolo tenía que efectuar equilibrio de cargas, dividiendo la carga entre varias líneas. La mayoría de los protocolos previos enviaban

todos los paquetes a través de la mejor ruta. La segunda mejor ruta no se usaba en absoluto. En muchos casos, la división de la carga a través de varias líneas produce un mejor desempeño.

Sexto, se requería el reconocimiento de sistemas jerárquicos. En 1988, la Internet se había hecho tan grande que no se podía esperar que un solo enrutador conociera la topología completa. El nuevo protocolo de enrutamiento tenía que diseñarse de modo que ningún enrutador tuviera que conocerla.

Séptimo, se requería un mínimo de seguridad para evitar que los estudiantes ociosos burlaran a los enrutadores enviándoles información de enrutamiento falsa. Por último, se requería un mecanismo para manejar los enrutadores que se conectaran a Internet a través de un túnel. Los protocolos previos no manejaban bien esto.

El OSPF reconoce tres tipos de conexiones y redes:

1. Líneas punto a punto entre dos enrutadores (exactamente).
2. Redes multiacceso con difusión (por ejemplo, la mayoría de las LAN).
3. Redes multiacceso sin difusión (por ejemplo, la mayoría de las WAN de conmutación de paquetes).

Una red **multiacceso** es una que puede tener varios enrutadores, cada uno de los cuales puede comunicarse directamente con todos los demás. Todas las LAN y WAN tienen esta propiedad. En la figura 5-52(a) se muestra un AS que contiene los tres tipos de redes. Nótese que los *hosts* generalmente no desempeñan ningún papel en el OSPF.

El OSPF funciona haciendo una abstracción del conjunto de redes, enrutadores y líneas en un grafo dirigido en el que a cada arco se le asigna un costo (distancia, retardo, etc.). Entonces se calcula la trayectoria más corta con base en los pesos de los arcos. Una conexión en serie entre dos enrutadores se representa mediante un par de arcos, uno en cada dirección. Sus pesos pueden ser diferentes. Una red multiacceso se representa mediante un nodo para la red misma más un nodo para cada enrutador. Los arcos del nodo de red a los enrutadores tienen un peso de 0 y se omiten del grafo.

En la figura 5-52(b) se muestra la representación gráfica de la red de la figura 5-52(a). Lo que hace fundamentalmente el OSPF es representar la red como un grafo de este tipo y luego calcular la trayectoria más corta de un enrutador a todos los demás.

Muchas de las AS de Internet son grandes y nada fáciles de manejar. El OSPF permite su división en **áreas** numeradas, donde un área es una red o un grupo de redes contiguas. Las áreas no se traslapan, pero no necesitan ser exhaustivas, es decir, algunos enrutadores podrían no pertenecer a ningún área. Un área es una generalización de una subred. Fuera de un área, su topología y detalles no son visibles.

Cada AS tiene un área de ***backbone***, llamada área 0. Todas las áreas se conectan al *backbone*, posiblemente mediante túneles, por lo que hay la posibilidad de ir de cualquier área del AS a cualquier otra a través del *backbone*. Un túnel se representa en el grafo como un arco y tiene un costo. Cada enrutador conectado a dos o más áreas es parte del *backbone*. Al igual que en otras áreas, la topología del *backbone* no es visible desde fuera del *backbone*.

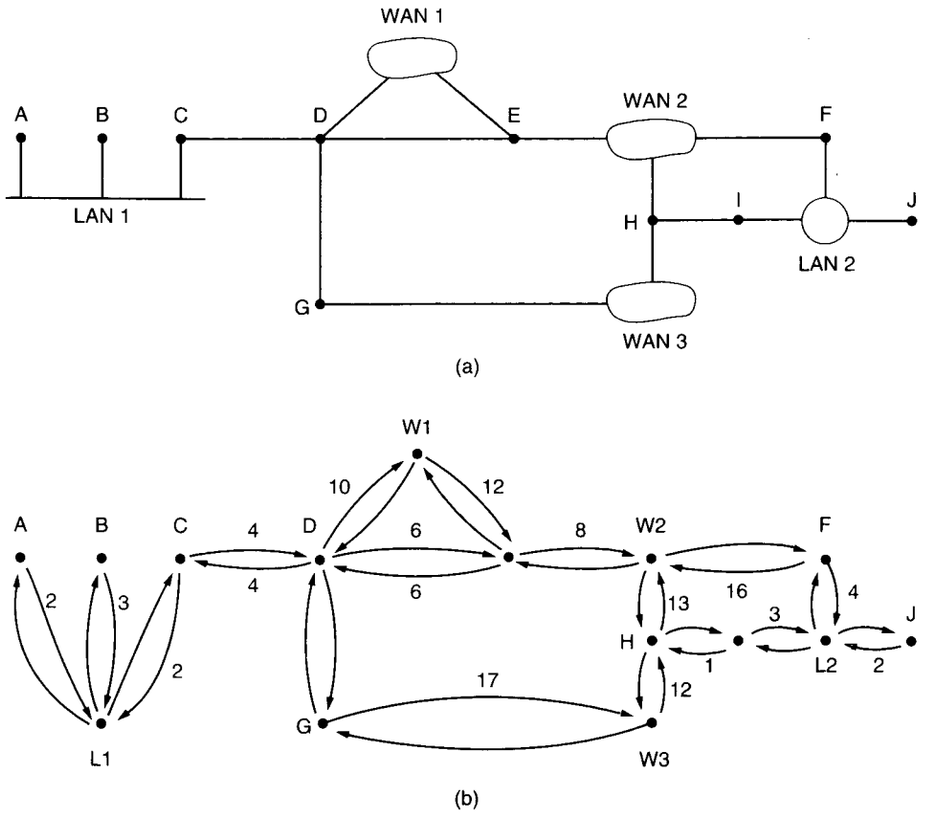


Figura 5-52. (a) Sistema autónomo. (b) Representación con grafos de (a).

Dentro de un área, cada enrutador tiene la misma base de datos de estado de enlace y ejecuta el mismo algoritmo de trayectoria más corta; su tarea principal es calcular la trayectoria más corta de sí mismo a todos los demás enrutadores del área, incluido el enrutador que está conectado al *backbone*, de los cuales debe haber cuando menos uno. Un enrutador que se conecta a dos áreas necesita las bases de datos de ambas áreas y debe ejecutar para cada una por separado el algoritmo de trayectoria más corta.

La manera en que el OSPF maneja el enrutamiento de tipo de servicio es teniendo varios grafos, uno etiquetado con los costos cuando la métrica es el retardo, otro etiquetado con los costos cuando la métrica es el rendimiento, y uno más etiquetado con los costos cuando la métrica es la confiabilidad. Aunque esto triplica el cálculo necesario, permite rutas separadas para optimar el retardo, el rendimiento y la confiabilidad.

Durante la operación normal, pueden necesitarse tres tipos de rutas: intraárea, interárea e interAS. Las rutas intraárea son la más fáciles, dado que el enrutador de origen ya conoce la trayectoria más corta al enrutador de destino. El enrutamiento interárea siempre procede en tres

pasos: va del origen al *backbone*, pasa a través del *backbone* al área de destino y va al destino. Este algoritmo obliga a una configuración en estrella en el OSPF, siendo el *backbone* el centro y las demás áreas los rayos. Los paquetes se enrutan del origen al destino "como vienen". No se encapsulan ni se envían por túnel, a menos que vayan a un área cuya única conexión al *backbone* sea un túnel. En la figura 5-53 se muestra parte de la Internet con AS y áreas.

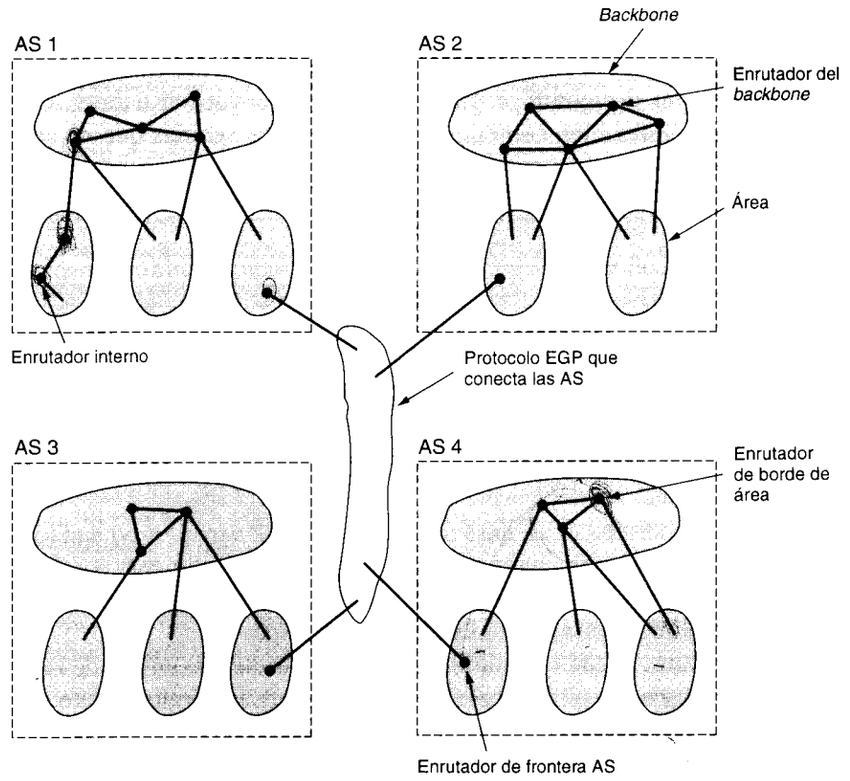


Figura 5-53. Relación entre los AS, los *backbone* y las áreas en el OSPF.

El OSPF distingue cuatro clases de enrutadores:

1. Enrutadores internos que están contenidos en una sola área.
2. Enrutadores de borde de área que conectan dos o más áreas.
3. Enrutadores de *backbone* que están en el *backbone*.
4. Enrutadores de frontera de AS que hablan con los enrutadores de otras AS.

Se permite que estas clases se traslapen. Por ejemplo, todos los enrutadores de borde son automáticamente parte del *backbone*. Además, un enrutador que está en el *backbone* pero no es

parte de ninguna otra área también es un enrutador interno. En la figura 5-53 se ilustran ejemplos de las cuatro clases de enrutadores.

Al arrancar un enrutador, envía mensajes de HOLA (*HELLO*) por todas sus líneas punto a punto y los multitransmite por las LAN al grupo que consiste en todos los demás enrutadores. En las WAN, el enrutador requiere cierta información de configuración para saber con quién tiene que hacer contacto. A partir de las respuestas, cada enrutador aprende quiénes son sus vecinos.

El OSPF funciona intercambiando información entre enrutadores **adyacentes**, que no es lo mismo que entre enrutadores vecinos. En particular, es ineficiente hacer que todos los enrutadores de una LAN hablen con todos los enrutadores de otra LAN. Para evitar esta situación, se elige un enrutador como **enrutador designado**, el cual se dice que es adyacente a todos los demás enrutadores, e intercambia información con ellos. Los enrutadores vecinos que no son adyacentes no intercambian información entre ellos. Siempre se mantiene actualizado un enrutador designado de respaldo para facilitar la transición en caso de que el enrutador designado primario se caiga.

Durante la operación normal, cada enrutador inunda periódicamente con mensajes de ACTUALIZACIÓN DE ESTADO DE ENLACE (*LINK STATE UPDATE*) a todos sus enrutadores adyacentes. Este mensaje indica el estado del enrutador y proporciona los costos usados en la base de datos topológica. Los mensajes de inundación se reconocen a fin de hacerlos confiables. Cada mensaje tiene un número de secuencia, por lo que un enrutador puede ver si un mensaje de ACTUALIZACIÓN DE ESTADO DE ENLACE de entrada es más viejo o más nuevo que el que tiene actualmente. Los enrutadores también pueden enviar estos mensajes cuando se activa o se desactiva una línea y cuando cambian sus costos.

Los mensajes de DESCRIPCIÓN DE LA BASE DE DATOS (*DATABASE DESCRIPTION*) dan los números de secuencia de todas las entidades de estado de enlace guardadas actualmente por el transmisor. Al comparar sus propios valores con los del transmisor, el receptor puede determinar quién tiene los valores más recientes. Estos mensajes se usan al activar la línea.

Cualquiera de las partes puede solicitar información de estado de enlace de la otra mediante mensajes de SOLICITUD DE ESTADO DE ENLACE (*LINK STATE REQUEST*). El resultado neto de este algoritmo es que cada par de enrutadores adyacentes verifica quién tiene los datos más recientes, y de esta manera se distribuye información nueva a través del área. Todos estos mensajes se envían como paquetes IP en bruto. Los cinco tipos de mensaje se resumen en la figura 5-54.

Tipo de mensaje	Descripción
Hola	Sirve para descubrir quiénes son los vecinos
Actualización de estado de enlace	Proporciona los costos del transmisor a sus vecinos
Reconocimiento de estado de enlace	Reconocimiento de la actualización de estado de enlace
Descripción de la base de datos	Anuncia las actualizaciones que tiene el transmisor
Solicitud de estado de enlace	Solicita información del compañero

Figura 5-54. Los cinco tipos de mensajes OSPF.

Por último, podemos juntar todas las piezas. Mediante inundación, cada enrutador informa a los demás enrutadores de su área acerca de sus vecinos y sus costos. Esta información permite a cada enrutador construir el grafo de su(s) área(s) y calcular la trayectoria más corta. El área de *backbone* también hace esto. Además, los enrutadores de *backbone* aceptan información de los enrutadores de borde de área a fin de calcular la mejor ruta de cada enrutador de *backbone* a los demás enrutadores. Esta información se propaga de regreso a los enrutadores de borde de área, quienes la divulgan en sus áreas. Usando esta información, un enrutador a punto de enviar un paquete interárea puede seleccionar el mejor enrutador de salida al *backbone*.

5.5.6. Protocolo de enrutamiento de pasarela exterior: BGP

Dentro de un solo AS, el protocolo de enrutamiento recomendado en Internet es el OSPF (aunque ciertamente no es el único en uso). Entre los AS, se usa un protocolo diferente, el **BGP** (*Border Gateway Protocol*, **protocolo de pasarela exterior**). Se requiere un protocolo diferente entre las AS porque las metas de un protocolo de pasarela interior y un protocolo de pasarela exterior no son iguales. Todo lo que tiene que hacer un protocolo de pasarela interior es mover paquetes con la mayor eficiencia posible del origen al destino; no necesita preocuparse por la política.

Los enrutadores de protocolo de pasarela exterior tienen que preocuparse por los asuntos políticos, y mucho. Por ejemplo, un AS corporativo podría querer ser capaz de enviar paquetes a cualquier instalación Internet y recibir paquetes de cualquier instalación Internet; sin embargo, tal vez no quiera transportar paquetes en tránsito que se originan en un AS foráneo y van a otro AS foráneo, aun si su propio AS está en la trayectoria más corta entre los dos AS foráneos ("ése es su problema, no el nuestro"). Por otra parte, el AS podría estar dispuesto a transportar tráfico en tránsito de sus vecinos, o inclusive de otros AS que pagaran por su servicio. Por ejemplo, las compañías telefónicas podrían estar felices de funcionar como portadoras para sus clientes, pero no para otros. Los protocolos de pasarela exterior en general, y en particular el BGP, se han diseñado para permitir muchos tipos de políticas de enrutamiento aplicables al tráfico interAS.

Las políticas típicas comprenden consideraciones políticas, de seguridad o económicas. Algunos ejemplos de restricciones de enrutamiento son:

1. Prohibición del tráfico en tránsito a través de ciertas AS.
2. Nunca poner a Irak en una ruta que comience en el Pentágono.
3. No utilizar a Estados Unidos para llegar de la Columbia Británica a Ontario.
4. Sólo transitar por Albania si no hay una alternativa hacia el destino.
5. El tráfico que comience o termine en IBM® no debe transitar por Microsoft®.

Las políticas se configuran manualmente en cada enrutador BGP. No son parte del protocolo mismo.

Desde el punto de vista de un enrutador BGP, el mundo consiste en otros enrutadores BGP y en las líneas que los conectan. Se consideran conectados dos enrutadores BGP si comparten una red común. Dado el interés especial del BGP en el tráfico en tránsito, las redes se agrupan en

una de tres categorías. La primera categoría es la de las **redes de punta**, que sólo tienen una conexión al grafo BGP; no se pueden usar para tráfico en tránsito porque no hay nadie del otro lado. Después vienen las **redes multiconectadas**. Éstas podrían usarse para el tráfico en tránsito, excepto que se niegan. Por último, están las **redes de tránsito**, como los *backbones*, que están dispuestas a manejar los paquetes de terceros, posiblemente con algunas restricciones.

Los pares de enrutadores BGP se comunican entre ellos estableciendo conexiones TCP. Este tipo de operación proporciona comunicación confiable y esconde todos los detalles de la red por la que se pasa.

El BGP fundamentalmente es un protocolo de vector de distancia, pero muy diferente de casi todos los demás, como el RIP. En lugar de mantener sólo el costo a cada destino, cada enrutador BGP lleva el registro de la trayectoria seguida. Del mismo modo, en lugar de dar periódicamente a cada vecino sus costos estimados a todos los destinos posibles, cada enrutador BGP le dice a sus vecinos la trayectoria exacta que está usando.

Como ejemplo, considere los enrutadores BGP mostrados en la figura 5-55(a). En particular, considere la tabla de enrutamiento de *F*. Suponga que se usa la trayectoria *FGCD* para llegar a *D*. Cuando los vecinos dan su información de rutas, proporcionan sus trayectorias completas, como se muestra en la figura 5-55(b) (por sencillez, sólo se muestra el destino *D*).

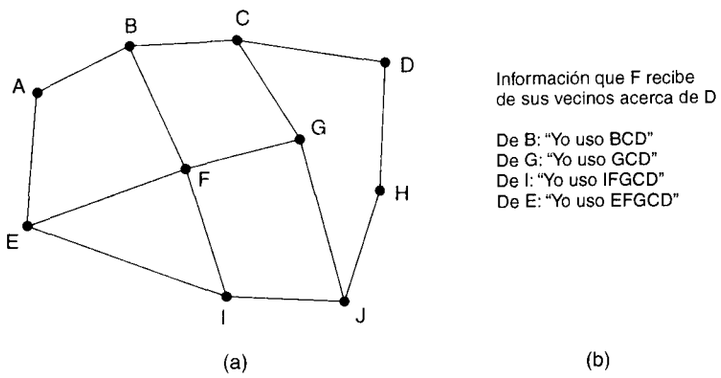


Figura 5-55. (a) Grupo de enrutadores BGP. (b) Información enviada a *F*.

Una vez que llegan todas las trayectorias de los vecinos, *F* las examina para ver cuál es la mejor. Pronto descarta las trayectorias de *I* y *E*, pues éstas pasan a través de *F* mismo. La decisión entonces está entre usar *B* o *G*. Cada enrutador BGP contiene un módulo que examina las rutas a un destino dado y las pondera, devolviendo un número para la "distancia" a ese destino por cada ruta. Cualquier ruta que viole una restricción por política automáticamente recibe una ponderación infinita. El enrutador entonces toma la ruta con la distancia más corta. La función de ponderación no es parte del protocolo BGP y puede ser cualquier función que quieran los administradores del sistema.

El BGP soluciona fácilmente el problema de conteo a infinito que es la plaga de otros algoritmos de enrutamiento por vector de distancia. Por ejemplo, supóngase que se cae *G* o se desactiva la línea *FG*. Entonces *F* recibe rutas de sus tres vecinos restantes. Estas rutas son *BCD*, *IFGCD* y *EFGCD*. Puede verse de inmediato que las dos últimas rutas no tienen sentido, pues pasan a través del mismo *F*, por lo que se escoge *FBCD* como ruta nueva. Otros algoritmos de vector de distancia con frecuencia toman las decisiones equivocadas porque no pueden saber cuáles de sus vecinos tienen rutas independientes a los destinos, y cuáles no. La definición actual del BGP está en el RFC 1654. Puede encontrarse información adicional útil en el RFC 1268.

5.5.7. Multidifusión Internet

La comunicación IP normal es entre un transmisor y un receptor. Sin embargo, en algunas aplicaciones es útil que un proceso pueda transmitir a una cantidad grande de receptores de manera simultánea. Como ejemplos están la actualización con réplica, las bases de datos distribuidas, la transmisión de cotizaciones de acciones a varias casas de bolsa y el manejo de conferencias telefónicas digitales (es decir, multiparte).

El IP maneja la multitransmisión, usando direcciones clase D. Cada dirección clase D identifica a un grupo de *hosts*. Hay 28 bits disponibles para la identificación de grupos, por lo que pueden existir 250 millones de grupos al mismo tiempo. Cuando un proceso envía un paquete a una dirección clase D, se hace un mejor esfuerzo para entregarlo a todos los miembros del grupo dirigido, pero no se dan garantías. Algunos miembros podrían no recibir el paquete.

Se reconocen dos tipos de dirección de grupo: las direcciones permanentes y las temporales. Un grupo permanente siempre está ahí y no tiene que configurarse. Cada grupo permanente tiene una dirección permanente de grupo. Algunos ejemplos de direcciones permanentes de grupo son:

224.0.0.1	Todos los sistemas de una LAN
224.0.0.2	Todos los enrutadores de una LAN
224.0.0.5	Todos los enrutadores OSPF de una LAN
224.0.0.6	Todos los enrutadores OSPF designados de una LAN

Deben crearse grupos temporales antes de poder usar estas direcciones. Un proceso puede pedir a su *host* que se una a un grupo específico; también puede pedirle que se retire del grupo. Cuando el último proceso de un *host* deja un grupo, ese grupo ya no está presente en el *host*. Cada *host* lleva el registro de los grupos a los que pertenecen sus procesos.

La multitransmisión se pone en práctica mediante enrutadores multitransmisión especiales, que pueden o no estar en los mismos puntos que los enrutadores estándar. Aproximadamente una vez cada minuto, todos los enrutadores multitransmisión envían una multitransmisión de *hardware* (por ejemplo, de capa de enlace de datos) a los *hosts* en su LAN (dirección 224.0.0.1) solicitándoles que informen los grupos a los que pertenecen sus procesos. Cada *host* envía de regreso respuestas a todas las direcciones clase D en las que está interesado.

Estos paquetes de solicitud y respuesta usan un protocolo llamado **IGMP** (*Internet Group Management Protocol*, **protocolo de administración de grupos de Internet**), que es vagamente

análogo al ICMP. Tiene sólo dos tipos de paquetes, solicitud y respuesta, teniendo cada uno un formato fijo y sencillo que contiene cierta información de control en la primera palabra del campo de carga útil y una dirección clase D en la segunda palabra; se describe en el RFC 1112.

El enrutamiento multitransmisión se hace usando árboles de extensión. Cada enrutador multitransmisión intercambia información con sus vecinos usando un protocolo de vector de distancia modificado a fin de que cada uno construya un árbol de extensión por grupo, cubriendo a todos los miembros del grupo. Se usan varias optimizaciones para podar el árbol y eliminar enrutadores y redes no interesadas en grupos en particular. El protocolo hace uso intensivo del proceso en túnel para evitar dar molestias a nodos que no están en un árbol de extensión.

5.5.8. IP móvil

Muchos usuarios de Internet tienen computadoras portátiles y quieren mantenerse conectados a Internet al visitar una instalación Internet lejana, y aun en el camino intermedio. Desafortunadamente, el sistema de direccionamiento IP hace que el trabajo lejos de casa sea más fácil de plantear que de hacer. En esta sección examinaremos el problema y la solución. En (Johnson, 1995) se da una descripción más detallada.

El verdadero villano es el esquema de direccionamiento mismo. Cada dirección IP contiene tres campos: la clase, el número de red y el número de *host*. Por ejemplo, considere la máquina con dirección IP 160.80.40.20. El 160.80 da la clase (B) y el número de red (8272); el 40.20 es el número de *host* (10260). Los enrutadores de todo el mundo tienen tablas de enrutamiento que indican la línea a usar para llegar a la red 160.80. Cuando llega un paquete con una dirección IP de destino de la forma 160.80.xxx.yyy, sale por esa línea.

Si de pronto, la máquina con esa dirección se lleva a algún lugar lejano, los paquetes para ella se seguirán enviando a su LAN (o enrutador) base. El dueño ya no podrá recibir correo electrónico, etc. Darle a la máquina una nueva dirección IP correspondiente a su nueva ubicación no es muy atractivo, pues habría que informar a una gran cantidad de gente, programas y bases de datos sobre el cambio.

Otro enfoque es hacer que los enrutadores usen direcciones IP completas para el enrutamiento, en lugar de sólo la clase y la red. Sin embargo, esta estrategia requeriría que cada enrutador tuviera tablas de millones de entradas, con un costo astronómico para Internet.

Cuando la gente comenzó a exigir la posibilidad de tener *hosts* móviles, el IETF estableció un grupo de trabajo para encontrar una solución. El grupo de trabajo pronto formuló varias metas consideradas deseables en cualquier solución. Las principales fueron:

1. Todo *host* móvil debe ser capaz de usar su dirección IP base en cualquier lugar.
2. No se permiten cambios al *software* de los *hosts* fijos.
3. No se permiten cambios al *software* del enrutador ni a sus tablas.
4. La mayoría de los paquetes para los *hosts* móviles no deben desviarse en el camino.
5. No debe incurrirse en carga extra cuando un *host* móvil está en su base.

La solución escogida fue la que se describe en la sección 5.2.8. En síntesis, cada instalación que quiera permitir que sus usuarios vaguen debe crear un agente de base. Cada instalación que permita visitantes tiene que crear un agente foráneo. Al aparecer un *host* móvil en una instalación foránea, se pone en contacto con el *host* foráneo y se registra. El *host* foráneo entonces se comunica con el agente de base del usuario y le da una **dirección de encargado** (*care-off address*), normalmente la misma dirección IP del agente foráneo.

Al llegar un paquete a la LAN base del usuario, llega por un enrutador conectado a la LAN. El enrutador entonces trata de localizar al *host* de la manera normal, difundiendo un paquete ARP que pregunte, por ejemplo: "¿cuál es la dirección Ethernet de 160.80.40.20?" El agente de base responde a esta solicitud dando su propia dirección Ethernet. El enrutador entonces envía los paquetes para 160.80.40.20 al agente de base. Éste a su vez, los envía en túnel a la dirección de encargado, encapsulándolos en el campo de carga útil de un paquete IP dirigido al agente foráneo. El agente foráneo entonces los desencapsula y los entrega a la dirección de enlace de datos del *host* móvil. Además, el agente de base entrega la dirección de encargado al transmisor, para que los paquetes futuros puedan enviarse en túnel directamente al agente foráneo. Esta solución cumple con todos los requisitos indicados antes.

Probablemente vale la pena mencionar un pequeño detalle. En el momento de moverse el *host* móvil, el enrutador probablemente tiene en caché sus direcciones Ethernet (que pronto dejarán de ser válidas). Para reemplazar esa dirección de Ethernet por la del agente de base, se usa un truco llamado **ARP gratuito** (*gratuitious ARP*). Éste es un mensaje especial al enrutador, no solicitado, que causa que reemplace una entrada específica de caché, en este caso la del *host* móvil a punto de irse. Al regresar el *host* móvil después, se usa el mismo truco para actualizar el caché del enrutador.

Nada del diseño impide que un *host* móvil sea su propio agente foráneo, pero este enfoque sólo funciona si el *host* móvil (en su capacidad como agente foráneo) está conectado lógicamente a Internet en su instalación actual. También, debe poder adquirir una dirección IP de encargado (temporal) para usarla. Esa dirección IP debe pertenecer a la LAN a la que está conectado actualmente.

La solución IETF para *hosts* móviles resuelve otros problemas no mencionados hasta ahora. Por ejemplo, ¿cómo localizar a los agentes? La solución es que cada agente difunda periódicamente sus direcciones y el tipo de servicios que está dispuesto a proporcionar (por ejemplo, base, foráneo o ambos). Al llegar un *host* móvil a algún lado, simplemente puede esperar estas difusiones, llamadas **anuncios** (*advertisements*). Como alternativa, puede difundir un paquete anunciando su llegada y esperar que el agente foráneo local responda.

Otro problema que tenía que resolverse es lo que había que hacer con los *hosts* móviles mal educados que salen sin decir adiós. La solución es hacer que el registro sea válido sólo durante un intervalo de tiempo fijo; si no se renueva periódicamente, termina su temporización, y así el agente foráneo puede limpiar sus tablas.

Un tema más es el de la seguridad. Cuando un agente de base recibe un mensaje solicitándole que redirija todos los paquetes de Nora a alguna dirección IP, no debe acceder a menos que esté convencido de que Nora es el origen de la solicitud, y no alguien que está tratando de hacerse pasar por ella. Se usan protocolos cifrados de verificación de autenticidad para este fin. Estudiaremos tales protocolos en el capítulo 7.

Un punto final cubierto por el grupo de trabajo se relaciona con los niveles de movilidad. Imagine un avión con un Ethernet a bordo usado por las computadoras de navegación y aeronáutica. En este Ethernet hay un enrutador estándar que habla con la Internet alambrada de tierra mediante un enlace de radio. Un buen día, a algún astuto ejecutivo de mercadeo se le ocurre instalar conectores Ethernet en los descansabrazos para que los pasajeros con computadoras móviles también se puedan conectar.

Ahora tenemos dos niveles de movilidad: las propias computadoras del avión, que son estacionarias respecto del Ethernet, y las computadoras de los pasajeros, que son móviles respecto de él. Además, el enrutador de a bordo es móvil respecto de los enrutadores de tierra. La movilidad respecto de un sistema que en sí mismo es móvil puede manejarse usando túneles recursivos.

5.5.9. CIDR — Enrutamiento interdominios sin clases

El IP ya ha estado en uso intensivo por más de una década; ha funcionado extremadamente bien, como lo demuestra el crecimiento exponencial de la Internet. Desgraciadamente, el IP se está convirtiendo con rapidez en una víctima de su propia popularidad: se le están acabando las direcciones. Este desastre inminente ha propiciado una gran cantidad de controversias y debates en la comunidad de Internet sobre lo que debe hacerse al respecto. En esta sección describiremos tanto el problema como varias soluciones propuestas. En (Huitema, 1996) puede encontrarse una descripción más completa.

En 1987 unos pocos visionarios predijeron que algún día la Internet podría crecer hasta 100,000 redes. La mayoría de los expertos minimizaron el problema aduciendo que ocurriría en muchas décadas, si es que llegaba a ocurrir. En 1996 se conectó la red 100,000. El problema, en pocas palabras, es que la Internet se está quedando rápidamente sin direcciones de IP. En teoría, existen cerca de dos mil millones de direcciones, pero la práctica de organizar el espacio de direcciones por clases (véase la figura 5-47) desperdicia millones de ellas. En particular, el verdadero villano es la red clase B. Para la mayoría de las organizaciones, una red clase A, con 16 millones de direcciones, es demasiado grande, y una red clase C, de 256 direcciones, es demasiado pequeña. Una red clase B, con 65,536, es la adecuada. En el folclor de Internet, esta situación se conoce como el **problema de los tres osos** (como en *Ricitos de Oro y los tres osos*).

En realidad, una dirección clase B es demasiado grande para la mayor parte de las organizaciones. Hay estudios que demuestran que más de la mitad de todas las redes clase B tienen menos de 50 *hosts*. Una red clase C habría bastado, pero sin duda todas las organizaciones que solicitaron una dirección clase B pensaron que un día el campo de *hosts* de 8 bits les quedaría pequeño. En retrospectiva, podría haber sido mejor que las redes clase C usaran 10 bits en lugar de 8 para el número de *host*, permitiendo 1022 *hosts* por red. De haber sido éste el caso, la mayor parte de las organizaciones probablemente se habría conformado con una red clase C, y habría habido medio millón de ellas (en vez de sólo 16,384 redes clase B).

Sin embargo, entonces habría surgido más rápidamente otro problema: la explosión de tablas de enrutamiento. Desde el punto de vista de los enrutadores, el espacio de direcciones IP es una jerarquía de dos niveles, con números de red y números de *host*. Los enrutadores no

tienen que saber de todos los *hosts*, pero sí tienen que saber de todas las redes. Si se usaran medio millón de redes clase C, todos los enrutadores de Internet necesitarían una tabla con medio millón de entradas, una por red, indicando la línea a usar para llegar a esa red, así como otra información.

Actualmente, el almacenamiento de medio millón de entradas tal vez es posible, aunque caro en los enrutadores críticos que guardan las tablas en la RAM estática de las tarjetas de E/S. Un problema más serio es que la complejidad de diferentes algoritmos relacionados con el mantenimiento de tablas crece con una tasa mayor que la lineal. Peor aún, mucho del *software* y *firmware* existente de los enrutadores se diseñó en un momento en que la Internet tenía 1000 redes conectadas, y tener 10,000 redes parecía estar a décadas de distancia. Las decisiones de diseño tomadas entonces ya no tienen nada de óptimas.

Además, diversos algoritmos de enrutamiento requieren que cada enrutador transmita sus tablas periódicamente. Cuanto más grandes sean las tablas, más probabilidad habrá de que algunas partes se pierdan en el camino, dando pie a datos incompletos en el otro lado y posiblemente a inestabilidades de enrutamiento.

El problema de las tablas de enrutamiento podría haberse resuelto usando una jerarquía más. Por ejemplo, podría haber servido hacer que cada dirección de IP tuviera un campo de país, estado, ciudad, red y *host*. Entonces cada enrutador sólo necesitaría saber cómo llegar a los países, a los estados o provincias de su propio país, a las ciudades de su estado o provincia y a las redes de su ciudad. Por desgracia, esta solución requeriría bastante más de 32 bits para las direcciones de IP y usaría ineficientemente las direcciones (Liechtenstein tendría tantos bits como Estados Unidos).

En resumen, la mayoría de las soluciones resuelven un problema pero crean uno nuevo. Una solución que ahora se está poniendo en práctica y que dará a Internet un respiro es el **CIDR** (*Classless InterDomain Routing*, **enrutamiento interdominios sin clases**). El concepto básico del CIDR, que se describe en el RFC 1519, es repartir las redes clase C restantes, de las que hay casi dos millones, en bloques de tamaño variable. Si una instalación necesita, digamos, 2000 direcciones, se le da un bloque de 2048 direcciones (ocho redes clase C contiguas), y no una dirección clase B completa. Del mismo modo, una instalación que necesita 8000 direcciones recibe 8192 direcciones (32 redes contiguas clase C).

Además de usar como unidades bloques de redes contiguas clase C, las reglas de asignación para las direcciones clase C también se cambiaron en el RFC 1519. El mundo se dividió en cuatro zonas, y a cada una se le dio una parte del espacio de direcciones clase C. La asignación fue la siguiente:

Las direcciones 194.0.0.0 a 195.255.255.255 son para Europa
Las direcciones 198.0.0.0 a 199.255.255.255 son para Norteamérica
Las direcciones 200.0.0.0 a 201.255.255.255 son para Centro y Sudamérica
Las direcciones 202.0.0.0 a 203.255.255.255 son para Asia y el Pacífico

De este modo, cada región recibió unos 32 millones de direcciones para repartir, con otros 320 millones de direcciones clase C, de la 204.0.0.0 a 223.255.255.255, guardadas como reserva

para el futuro. La ventaja de esta asignación es que ahora cualquier enrutador fuera de Europa que recibe un paquete dirigido a 194.xx.yy.zz o a 195.xx.yy.zz simplemente puede enviarlo a su pasarela europea estándar. En efecto, ahora se han comprimido 32 millones de direcciones en una sola entrada de tabla de enrutamiento. Lo mismo ocurre con las demás regiones.

Por supuesto, una vez que llega un paquete 194.xx.yy.zz a Europa, se requieren tablas de enrutamiento más detalladas. Una posibilidad es tener 131,072 entradas para las redes 194.0.0.xx a 195.255.255.xx, pero ésta es precisamente la explosión de tablas de enrutamiento que estamos tratando de evitar. En vez de ello, cada entrada de tabla de enrutamiento se extiende con una máscara de 32 bits. Al llegar un paquete, se extrae primero su dirección de destino. Luego (conceptualmente) se examina la tabla de enrutamiento entrada por entrada, enmascarando la dirección de destino y comparándola con las entradas de la tabla en busca de una igual.

Para hacer más claro este proceso de comparación, consideremos un ejemplo. Supóngase que la Universidad de Cambridge necesita 2048 direcciones y se le asignan las direcciones 194.24.0.0 a 194.24.7.255, junto con la máscara 255.255.248.0. En seguida, la Universidad de Oxford solicita 4096 direcciones. Puesto que un bloque de 4096 direcciones debe caer en un límite de 4096 bytes, no pueden asignarse las direcciones que comienzan en 194.8.0.0. En cambio, Oxford recibe 194.24.16.0 a 194.24.31.255, junto con la máscara 255.255.240.0. Ahora la Universidad de Edimburgo solicita 1024 direcciones, y se le asignan las direcciones 194.24.8.0 a 194.24.11.255 y la máscara 255.255.252.0.

Las tablas de enrutamiento de toda Europa se actualizan con tres entradas, que contienen una dirección base y una máscara. Estas entradas (en binario) son:

Dirección	Máscara
11000010 00011000 00000000 00000000	11111111 11111111 11111000 00000000
11000010 00011000 00010000 00000000	11111111 11111111 11110000 00000000
11000010 00011000 00001000 00000000	11111111 11111111 11111100 00000000

Ahora considere lo que ocurre cuando llega un paquete dirigido a 194.24.17.4, que en binario es

11000010 00011000 00010001 00000100

Primero se le hace un AND booleano con la máscara de Cambridge para obtener

11000010 00011000 00010000 00000000

Este valor no es igual a la dirección base de Cambridge, por lo que vuelve a hacerse un AND con la máscara de Oxford para obtener

11000010 00011000 00010000 00000000

Este valor sí es igual a la máscara de Oxford, por lo que el paquete se envía al enrutador de Oxford. En la práctica, las entradas de enrutador no se prueban secuencialmente; se usan trucos de indización para acelerar la búsqueda. También, es posible que dos entradas coincidan después del AND, en cuyo caso gana la que tenga una máscara con más bits 1. Por último, la misma idea puede aplicarse a todas las direcciones, no sólo a las nuevas direcciones clase C, por lo que, con el CIDR, las viejas redes clase A, B y C no se usan más para enrutamiento. Ésta es la razón por la que al CIDR se le llama enrutamiento sin clases. El CIDR se describe con mayor detalle en (Ford *et al.*, 1993, y Huitema, 1995).

5.5.10. IPv6

Si bien el CIDR puede alargar unos pocos años más de tiempo, todo mundo se da cuenta que los días del IP en su forma actual (IPv4) están contados. Además de estos problemas técnicos, hay otro asunto acechando. Hasta hace poco, la Internet ha sido utilizada en gran medida por universidades, industrias de alta tecnología y el gobierno (especialmente el Departamento de Defensa de Estados Unidos). Con la explosión del interés por la Internet que comenzó a mediados de los años 90, es muy posible que en el próximo milenio será usada por un grupo mucho más grande de gente, especialmente gente con necesidades diferentes. Por una parte, millones de personas con computadoras portátiles inalámbricas podrían usarla para mantenerse en contacto con sus bases. Por otra, con la inminente convergencia de las industrias de la computación, las comunicaciones y el entretenimiento, tal vez no pase mucho tiempo antes de que todos los televisores del mundo estén en un nodo Internet, produciendo mil millones de máquinas utilizadas para recibir vídeo a solicitud. En estas circunstancias, se hizo evidente que el IP tenía que evolucionar y volverse más flexible.

Al ver en el horizonte estos problemas, el IETF comenzó a trabajar en 1990 en una versión nueva del IP, una que nunca se quedaría sin direcciones, resolvería varios otros problemas y sería más flexible y eficiente también. Sus metas principales eran:

1. Manejar miles de millones de *hosts*, aún con asignación de espacio de direcciones ineficiente.
2. Reducir el tamaño de las tablas de enrutamiento.
3. Simplificar el protocolo, para permitir a los enrutadores el procesamiento más rápido de los paquetes.
4. Proporcionar mayor seguridad (verificación de autenticidad y confidencialidad) que el IP actual.
5. Prestar mayor atención al tipo de servicio, especialmente con datos en tiempo real.
6. Ayudar a la multitransmisión permitiendo la especificación de alcances.
7. Posibilitar que un *host* sea móvil sin cambiar su dirección.
8. Permitir que el protocolo evolucione.
9. Permitir que el protocolo viejo y el nuevo coexistan por años.

Para encontrar un protocolo que cumpliera con todos estos requisitos, la IETF hizo una convocatoria solicitando propuestas y estudios en el RFC 1550. Se recibieron 21 respuestas, no todas propuestas completas. En diciembre de 1992 había siete propuestas serias en la mesa; iban desde hacer cambios menores al IP hasta desecharlo y reemplazarlo por un protocolo completamente diferente.

Una propuesta fue ejecutar TCP sobre CLNP, que, con sus direcciones de 160 bits habría proporcionado suficiente espacio de direcciones para siempre y habría unificado dos protocolos de capa de red principales. Sin embargo, muchos pensaron que esto habría sido una aceptación de que algunas cosas en el mundo OSI en realidad estaban bien hechas, algo considerado políticamente incorrecto en los círculos de Internet. El CLNP se creó tomando como modelo al IP, por lo que los dos no son realmente tan diferentes. De hecho, el protocolo escogido es más diferente del IP que CLNP. Otra cosa en contra el CLNP fue su pobre manejo de tipos de servicio, algo requerido para transmitir multimedia eficientemente.

Tres de las mejores propuestas se publicaron en *IEEE Network* (Deering, 1993; Francis, 1993, y Katz y Ford, 1993). Tras muchos análisis, revisiones e intrigas, se seleccionó una versión modificada de la combinación de las propuestas de Deering y Francis, llamada ahora **SIPP** (*Simple Internet Protocol Plus*, **protocolo simple de Internet mejorado**), y se le dio la designación **IPv6** (el IPv5 ya estaba en uso con un protocolo experimental de flujos en tiempo real).

El IPv6 cumple los objetivos bastante bien: mantiene las buenas características del IP, descarta y reduce las malas, y agrega nuevas donde se necesitan. En general, IPv6 no es compatible con IPv4, pero es compatible con todos los demás protocolos Internet, incluidos TCP, UDP, ICMP, IGMP, OSPF, BGP y DNS, a veces requiriendo pequeñas modificaciones (principalmente para manejar direcciones más grandes). Las características principales del IPv6 se analizan a continuación. Puede encontrarse mayor información en los RFC 1883 a 1887.

Por principio, y lo más importante, el IPv6 tiene direcciones más grandes que el Ipv4; son de 16 bytes de longitud, lo que resuelve el problema que se buscaba resolver: proporcionar una cantidad prácticamente ilimitada de direcciones Internet. Hablaremos más sobre las direcciones un poco más adelante.

La segunda mejora principal del IPv6 es la simplificación de la cabecera, que contiene sólo 7 campos (contra 13 en el IPv4). Este cambio permite a los enrutadores procesar con mayor rapidez los paquetes y mejorar, por tanto, el rendimiento. También estudiaremos pronto la cabecera.

La tercera mejora importante fue el mejor apoyo de las opciones. Este cambio fue esencial con la nueva cabecera, pues campos que antes eran obligatorios ahora son opcionales. Además, es diferente la manera de representar las opciones, haciendo más sencillo que los enrutadores hagan caso omiso de opciones no dirigidas a ellos. Esta característica mejora el tiempo de procesamiento de los paquetes.

Una cuarta área en la que el IPv6 representa un avance importante es la seguridad. La IETF tenía infinidad de historias sobre preadolescentes precoces que usaban sus computadoras personales para meterse en bancos e instalaciones militares por todas partes de Internet. Se tenía la fuerte sensación de que había que hacerse algo para mejorar la seguridad. Las verificaciones de autenticidad y la confidencialidad son características clave del IP nuevo.

Por último, se tenía que prestar mayor atención al tipo de servicio que en el pasado. El IPv4 de hecho tiene un campo de 8 bits dedicado a este asunto, pero con el crecimiento esperado del tráfico multimedia en el futuro, se requiere mucho más.

La cabecera principal del IPv6

La cabecera del IPv6 se muestra en la figura 5-56. El campo de *versión* siempre es de 6 para el IPv6 (y de 4 para el IPv4). Durante el periodo de transición del IPv4 al IPv6, que probablemente se llevará una década, los enrutadores podrán examinar este campo para saber el tipo de paquete que tienen. Como nota al margen, esta prueba ocupa algunas instrucciones en la trayectoria crítica, por lo que muchas implementaciones probablemente la evitarán usando algún campo de la cabecera de enlace de datos para distinguir los paquetes IPv4 de los IPv6. De esta manera, los paquetes pueden pasarse directamente al manejador correcto de la capa de red. Sin embargo, hacer que la capa de enlace de datos esté consciente de los tipos de los paquetes de red viola por completo el principio de diseño de que ninguna capa debe estar enterada del significado de los bits entregados por la capa superior a ella. Los debates entre los bandos de "hacerlo bien" y "hacerlo rápido" sin duda serán largos y acalorados.

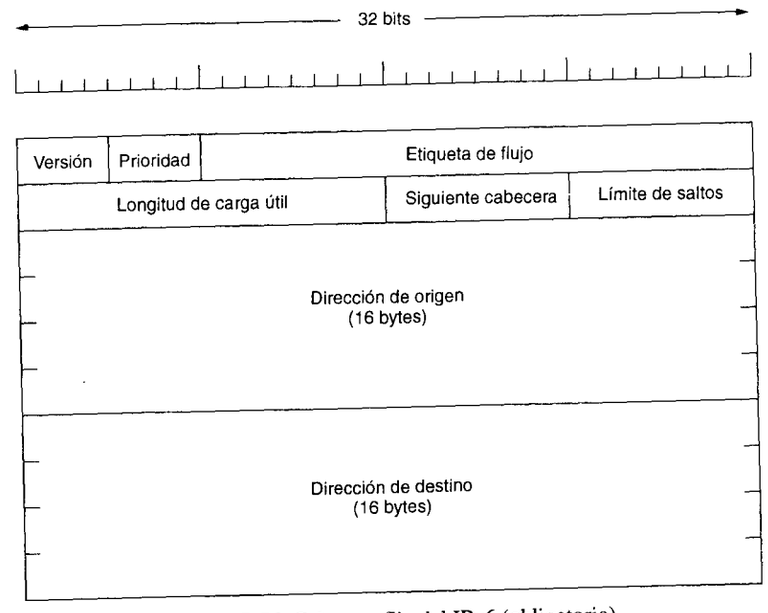


Figura 5-56. Cabecera fija del IPv6 (obligatoria).

El campo de *prioridad* se usa para distinguir entre paquetes a cuyas de orígenes se les puede controlar el flujo y aquellos a los que no. Los valores 0 a 7 son para transmisiones capaces de reducir su velocidad en caso de un congestionamiento. Los valores 8 a 15 son para tráfico de tiempo real cuya tasa de envío es constante, aun si todos los paquetes se están perdiendo. El audio y el vídeo caen en esta última categoría. Esta distinción permite a los enrutadores manejar mejor los paquetes en caso de congestionamientos. Dentro de cada grupo, los paquetes de

número más bajo son menos importantes que los paquetes de número alto. El estándar IPv6 sugiere, por ejemplo, usar 1 para noticias, 4 para FTP y 6 para conexiones Telnet, pues el retardo de un paquete de noticias durante algunos segundos no es perceptible, pero el retardo de un paquete Telnet ciertamente lo es.

El campo de *etiqueta de flujo* aún es experimental, pero se usará para permitir a un origen y a un destino establecer una seudoconexión con propiedades y requisitos particulares. Por ejemplo, una cadena de paquetes de un proceso en cierto *host* de origen dirigido a cierto proceso en cierto *host* de destino puede tener requisitos de retardo muy estrictos, y por tanto necesitar un ancho de banda reservado. El flujo puede establecerse por adelantado, dándole un identificador. Cuando aparece un paquete con una *etiqueta de flujo* diferente de cero, todos los enrutadores pueden buscarla en sus tablas internas para ver el tipo de tratamiento especial que requiere. En efecto, los flujos son un intento de tener lo mejor de ambos mundos: la flexibilidad de una subred de datagramas y las garantías de una subred de circuitos virtuales.

Cada flujo está designado por la dirección de origen, la dirección de destino y el número de flujo, por lo que pueden estar activos muchos flujos al mismo tiempo entre un par dado de direcciones IP. También, de esta manera, aun si dos flujos provenientes de *hosts* diferentes pero con el mismo número de flujo pasan por el mismo enrutador, el enrutador será capaz de distinguirlos usando las direcciones de origen y destino. Se espera que se escojan los números de flujo al azar, en lugar de asignarlos secuencialmente comenzando por el 1, para simplificar el proceso de dispersión en los enrutadores.

El campo de *longitud de carga útil* indica cuántos bytes siguen a la cabecera de 40 bytes de la figura 5-56. El nombre de campo se cambió de *longitud total* en el IPv4 porque el significado cambió ligeramente: los 40 bytes de cabecera ya no se cuentan como parte de la longitud, como antes.

El campo de *siguiente cabecera* revela el secreto. La razón por la que pudo simplificarse la cabecera es que puede haber cabeceras adicionales (opcionales) de extensión. Este campo indica cuál de las seis cabeceras de extensión (actualmente), de haberlas, sigue a ésta. Si esta cabecera es la última cabecera de IP, el campo de *siguiente cabecera* indica el manejador de protocolo de transporte (por ejemplo TCP, UDP) al que se entregará el paquete.

El campo de *límite de saltos* se usa para evitar que los paquetes vivan eternamente. En la práctica es igual al campo de *tiempo de vida* del IPv4, es decir, un campo que se disminuye en cada salto. En teoría, en el IPv4 era un tiempo en segundos, pero ningún enrutador lo usaba de esa manera, por lo que se cambió el nombre para reflejar la manera en que se usa realmente.

Luego vienen los campos de *dirección de origen* y *dirección de destino*. La propuesta original de Deering, el SIP, usaba direcciones de 8 bytes, pero durante el periodo de revisión muchas personas sintieron que, con direcciones de 8 bytes, en algunas décadas el IPv6 se quedaría sin direcciones, y que con direcciones de 16 bytes nunca se acabarían. Otros argumentaban que 16 bytes era demasiado, mientras que unos más estaban a favor de usar direcciones de 20 bytes para hacerlas compatibles con el protocolo de datagramas de OSI. Otra facción quería direcciones de tamaño variable. Tras mucho debate, se decidió que la mejor media sería una dirección de 16 bytes de longitud fija.

El espacio de direcciones del IPv6 se divide como se muestra en la figura 5-57. Las direcciones que comienzan con 80 ceros se reservan para direcciones IPv4. Se reconocen dos variantes,

Prefijo (binario)	Uso	Fracción
0000 0000	Reservado (incluye IPv4)	1/256
0000 0001	No asignado	1/256
0000 001	Direcciones OSI NSAP	1/128
0000 010	Direcciones IPX de Novell NetWare	1/128
0000 011	No asignado	1/128
0000 1	No asignado	1/32
0001	No asignado	1/16
001	No asignado	1/8
010	Direcciones basadas en proveedor	1/8
011	No asignado	1/8
100	Direcciones basadas en geografía	1/8
101	No asignado	1/8
110	No asignado	1/8
1110	No asignado	1/16
1111 0	No asignado	1/32
1111 10	No asignado	1/64
1111 110	No asignado	1/128
1111 1110 0	No asignado	1/512
1111 1110 10	Direcciones de enlace de uso local	1/1024
1111 1110 11	Direcciones de instalación de uso local	1/1024
1111 1111	Multitransmisión	1/256

Figura 5-57. Direcciones IPv6.

distinguidas por los siguientes 16 bits. Estas variantes se relacionan con la manera en que se enviarán en túnel los paquetes IPv6 a través de la infraestructura IPv4 existente.

El uso de prefijos distintos para las direcciones basadas en el proveedor y las basadas en la geografía es una media entre dos diferentes visiones del futuro de Internet. Las direcciones basadas en el proveedor tienen sentido si se piensa que en el futuro habrá algunas compañías que proporcionarán servicio Internet a sus clientes, lo que es parecido al servicio telefónico que prestan actualmente AT&T, MCI, Sprint, British Telecom y otras. Cada una de estas compañías tendrá una fracción del espacio de direcciones. Los primeros 5 bits que siguen al prefijo 010 sirven para indicar el registro donde se debe buscar al proveedor. Actualmente operan tres registros, para Norteamérica, Europa y Asia. Pueden agregarse hasta 29 registros nuevos posteriormente.

Cada registro está en libertad de dividir los 15 bytes restantes como crea conveniente. Se espera que la mayoría de ellos usarán un número de proveedor de 3 bytes, dando unos 16

millones de proveedores, a fin de permitir que una compañía grande actúe como su propio proveedor. Otra posibilidad es el uso de 1 byte para indicar proveedores nacionales y dejarlos efectuar asignaciones posteriores. De esta manera, pueden introducirse niveles adicionales de jerarquía, según sea necesario.

El modelo geográfico es el mismo que el actual de Internet, en el que los proveedores no desempeñan un papel importante. De esta manera, el IPv6 puede manejar ambos tipos de direcciones.

Las direcciones locales de enlace y de instalación sólo tienen significado local; pueden reutilizarse sin conflictos en cada organización, pero no pueden propagarse fuera de los límites de la organización, lo que las hace útiles para las organizaciones que actualmente usan muros de seguridad para protegerse del resto de Internet.

Las direcciones multitransmisión tienen un campo indicador de 4 bits y un campo de alcance de 4 bits a continuación del prefijo, y luego un identificador de grupo de 112 bits. Uno de los bits indicadores hace la distinción entre grupos permanentes y transitorios. El campo de alcance permite que una multitransmisión se limite al enlace, instalación u organización actuales, o al planeta. Estos cuatro alcances se distribuyen entre los 16 valores para permitir la adición posterior de nuevos alcances. Por ejemplo, el alcance planetario es 14, por lo que el código 15 está disponible para permitir la expansión futura de Internet a otros planetas, sistemas solares y galaxias.

Además de manejar direcciones unitransmisión estándar (punto a punto) y multitransmisión, el IPv6 también maneja un nuevo tipo de direccionamiento: transmisión a cualquiera. La **transmisión a cualquiera** (*anycasting*) es como la multitransmisión, en el sentido de que el destino es un grupo de direcciones, pero en lugar de tratar de entregar el paquete a todos, intenta entregarlo a uno solo, generalmente el más cercano. Por ejemplo, al ponerse en contacto con un grupo de servidores de archivos cooperativos, un cliente puede usar la transmisión a cualquiera para llegar al más cercano, sin tener que saber quién es. La transmisión a cualquiera usa direcciones normales de unitransmisión. Es responsabilidad del sistema de enrutamiento escoger al afortunado *host* que recibirá el paquete.

Se ha desarrollado una nueva notación para escribir direcciones de 16 bytes: se escriben como ocho grupos de cuatro dígitos hexadecimales, separados los grupos por dos puntos, como sigue:

8000:0000:0000:0000:0123:4567:89AB:CDEF

Ya que muchas direcciones tendrán muchos ceros en ellas, se han autorizado tres optimizaciones. Primero, los ceros a la izquierda de un grupo pueden omitirse, por lo que 0123 puede escribirse como 123. Segundo, pueden reemplazarse uno o más grupos de 16 ceros por un par de signos de dos puntos. Por tanto, la dirección anterior se vuelve ahora

8000::0123:4567:89AB:CDEF

Por último, las direcciones IPv4 pueden escribirse como un par de signos de dos puntos y un número decimal anterior separado por puntos, como por ejemplo

::192.31.20.46

Tal vez no sea necesario ser tan explícitos al respecto, pero hay muchas direcciones de 16 bytes. Específicamente, hay 2^{128} de ellas, lo que aproximadamente es 3×10^{38}. Si la Tierra completa, incluidos los océanos, estuviera cubierta de computadoras, el IPv6 permitiría 7×10^{23} direcciones de IP por metro cuadrado. Los estudiantes de química notarán que este número es mayor que el número de Avogadro. Aunque no fue la intención darle a cada molécula de la superficie terrestre su propia dirección de IP, no estamos lejos de ello.

En la práctica, el espacio de direcciones no se usará eficientemente, igual que el espacio de números telefónicos (el código de área de Manhattan, 212, está prácticamente lleno, pero el de Wyoming, 307, está casi vacío). En el RFC 1715, Huitema calculó que, usando la asignación de números telefónicos como guía, hasta en la situación más pesimista habrá más de 1000 direcciones IP por metro cuadrado de la superficie terrestre (tierra y agua). En cualquier situación probable, habrá billones de ellas por metro cuadrado. En pocas palabras, parece poco probable que se acabarán en el futuro previsible. También vale la pena indicar que hasta ahora sólo se ha asignado el 28% del espacio de direcciones. El otro 72% está disponible para propósitos futuros que aún no se han pensado.

Es instructivo comparar la cabecera de IPv4 (figura 5-45) con la de IPv6 (figura 5-56) para ver lo que se ha dejado fuera del IPv6. El campo *IHL* se fue porque la cabecera de IPv6 tiene una longitud fija. El campo de *protocolo* (*Protocol*) se retiró porque el campo de *siguiente cabecera* (*next header*) indica lo que sigue a la última cabecera de IP (por ejemplo, un segmento UDP o TCP).

Se retiraron todos los campos relacionados con la fragmentación, puesto que el IPv6 tiene un enfoque distinto hacia la fragmentación. Para empezar, todos los *hosts* y enrutadores que se ajustan al IPv6 deben reconocer paquetes de 576 bytes. Esta regla hace que sea menos posible que ocurra la fragmentación. Además, cuando un *host* envía un paquete de IPv6 demasiado grande, en lugar de fragmentarlo, el enrutador que es incapaz de reenviarlo devuelve un mensaje de error. Este mensaje indica al *host* que divida todos los paquetes futuros a ese destino. Es mucho más eficiente hacer que el *host* envíe paquetes del tamaño correcto desde el principio que hacer que los enrutadores los fragmenten sobre la marcha.

Por último, el campo de suma de comprobación (*checksum*) desaparece, porque su cálculo reduce en gran medida el desempeño. Con las redes confiables de hoy, además del hecho de que la capa de enlace de datos y las capas de transporte normalmente tienen sus propias sumas de comprobación, el provecho de otra suma de comprobación no valía el costo de desempeño que generaba. Al removerse estas características ha quedado un protocolo de capa de red compacto y sólido. Por tanto, la meta del IPv6 (un protocolo rápido y flexible con bastante espacio de direcciones) se ha cumplido con este diseño.

Cabeceras de extensión

Con todo, algunos de los campos faltantes en ocasiones son necesarios, por lo que el IPv6 introdujo el concepto de **cabecera de extensión** opcional. Estas cabeceras pueden usarse para proporcionar información extra, pero codificada de una manera eficiente. Hay seis tipos de cabeceras de extensión definidas actualmente, que se listan en la figura 5-58. Todas son opciona-

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE

les, pero si hay más de una, deben aparecer justo después de la cabecera fija, y de preferencia en el orden listado.

Cabecera de extensión	Descripción
Opciones salto por salto	Información diversa para los enrutadores
Enrutamiento	Ruta total o parcial a seguir
Fragmentación	Manejo de fragmentos de datagramas
Verificación de autenticidad	Comprobación de la identidad del transmisor
Carga útil cifrada de seguridad	Información sobre el contenido cifrado
Opciones de destino	Información adicional para el destino

Figura 5-58. Cabeceras de extensión del IPv6.

Algunas de las cabeceras tienen un formato fijo; otras contienen un número variable de campos de longitud variable. En éstos, cada elemento está codificado como una tupla (tipo, longitud, valor). El *tipo* es un campo de 1 byte que indica la opción de la que se trata. Los valores de *tipo* se han escogido de modo que los dos primeros bits indican a los enrutadores que no saben cómo procesar la opción lo que tienen que hacer. Las posibilidades son: saltar la opción, descartar el paquete, descartar el paquete y enviar de regreso un paquete ICMP, y lo mismo que lo anterior, pero no enviar paquetes ICMP a direcciones de multitransmisión (para evitar que un paquete de multitransmisión malo genere millones de informes ICMP).

La *longitud* también es un campo de 1 byte, e indica la longitud del valor (0 a 255 bytes). El *valor* es cualquier información requerida, de hasta 255 bytes.

La cabecera de salto por salto se usa para información que deben examinar todos los enrutadores a lo largo de la trayectoria. Hasta ahora, se ha definido una opción: manejo de datagramas de más de 64K. El formato de esta cabecera se muestra en la figura 5-59.

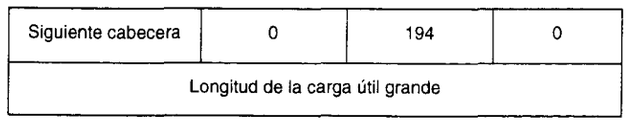


Figura 5-59. La cabecera de extensión salto por salto para datagramas grandes (jumbogramas).

Como con todas las cabeceras de extensión, ésta comienza con 1 byte que indica el tipo de cabecera que sigue. A este byte le sigue uno que indica la longitud de la cabecera salto por salto en bytes, excluyendo los primeros 8 bytes, que son obligatorios. Los 2 bytes siguientes indican que esta opción define el tamaño del datagrama (código 194) como número de 4 bytes. Los últimos 4 bytes indican el tamaño del datagrama. No se permiten los tamaños menores que 65,536, que darán como resultado el que el primer enrutador descarte el paquete y devuelva un mensaje

ICMP de error. Los datagramas que usan esta cabecera de extensión se llaman **jumbogramas**. El uso de jumbogramas es importante para las aplicaciones de supercomputadoras que deben transferir con eficiencia gigabytes de datos a través de Internet.

La cabecera de enrutamiento lista uno o más enrutadores que deben visitarse en el camino al destino. Hay la posibilidad tanto de enrutamiento estricto (se suministra la trayectoria completa) como libre (sólo se proporcionan enrutadores seleccionados), pero están combinados. El formato de la cabecera de enrutamiento se muestra en la figura 5-60.

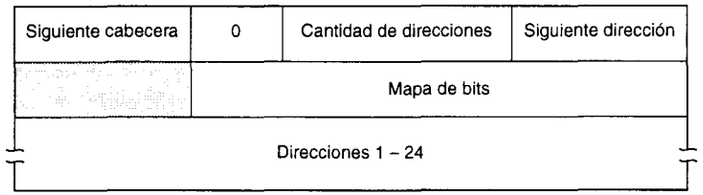


Figura 5-60. La cabecera de extensión para enrutamiento.

Los primeros 4 bytes de la cabecera de extensión de enrutamiento contienen cuatro enteros de 1 byte: el tipo de la siguiente cabecera, el tipo de enrutamiento (actualmente 0), la cantidad de direcciones presentes en esta cabecera (1 a 24) y el índice de la siguiente dirección a visitar. Este último campo comienza en 0 y aumenta con cada dirección que se visita.

Después viene un byte reservado seguido de un mapa de bits con bits para cada una de las 24 direcciones IPv6 potenciales que le siguen. Estos bits indican si debe visitarse cada dirección directamente después de la anterior (enrutamiento estricto desde el origen) o si pueden intervenir otros enrutadores intermedios (enrutamiento libre desde de origen).

La cabecera de fragmentación maneja la fragmentación de una manera parecida a la del IPv4. La cabecera contiene el identificador del datagrama, el número de fragmento y un bit que indica si seguirán más fragmentos. En el IPv6, a diferencia del IPv4, sólo el *host* de origen puede fragmentar un paquete. Los enrutadores a lo largo del camino no pueden hacerlo. Aunque este cambio es un rompimiento filosófico con el pasado, simplifica el trabajo del enrutador y acelera el enrutamiento. Como se mencionó antes, si un enrutador confronta un paquete demasiado grande, lo descarta y devuelve un paquete ICMP al origen. Esta información permite que el *host* de origen fragmente el paquete en pedazos más pequeños usando esta cabecera y lo intente de nuevo.

La cabecera de verificación de autenticidad proporciona un mecanismo mediante el cual el receptor de un paquete puede estar seguro de quién lo envió. Con el IPv4 no existe tal garantía. La carga cifrada de seguridad posibilita el cifrado del contenido de un paquete de modo que sólo el receptor pretendido pueda leerlo. Estas cabeceras usan técnicas de cifrado para lograr su cometido. Daremos una descripción breve a continuación, pero los lectores no familiarizados con el cifrado moderno podrían no entender la descripción ahora, y deberán regresar tras leer el capítulo 7 (en particular, la sección 7.1), que trata los protocolos de cifrado.

Cuando un transmisor y un receptor desean comunicarse con seguridad, primero deben ponerse de acuerdo en una o más claves secretas que sólo ellos conocen. La manera en que lo

hacen no es asunto del IPv6. A cada una de estas claves se le asigna un número de clave único de 32 bits. Los números de clave son globales, por lo que si Alicia está usando la clave 4 para hablar con Roberto, no puede usar una clave 4 para hablar con Carolina. Asociados a cada número de clave hay otros parámetros, como tiempo de vida de la clave y otros.

Para enviar un mensaje con verificación de autenticidad, el transmisor primero construye un paquete que consiste en todas las cabeceras IP y la carga útil, y luego reemplaza los campos que cambian en el camino (por ejemplo, el *límite de saltos*) por ceros. Se rellena el paquete con ceros hasta un múltiplo de 16 bytes. Del mismo modo, la clave secreta a usar se rellena con ceros hasta un múltiplo de 16 bytes. Ahora se calcula una suma de comprobación cifrada con base en la concatenación de la clave secreta rellenada, el paquete rellenado y nuevamente la clave secreta rellenada. Los usuarios pueden definir su propio algoritmo de suma de comprobación cifrada, pero se recomienda a los usuarios con poca habilidad en el cifrado usar el algoritmo predeterminado, el MD5.

Ahora llegamos al papel de la cabecera de verificación de autenticidad. Básicamente, ésta contiene tres partes. La primera consiste en 4 bytes que contienen el siguiente número de cabecera, la longitud de la cabecera de verificación de autenticidad, y 16 bits cero. Después viene el número de clave de 32 bits. Por último, se incluye la suma de comprobación MD5 (u otra).

El receptor entonces usa el número de clave para encontrar la clave secreta. La versión rellenada de ella se agrega al principio y al final de la carga útil rellenada, los campos de cabecera variables se llenan con ceros, y se calcula la suma de comprobación; si está de acuerdo con la suma incluida en la cabecera de verificación de autenticidad, el receptor puede estar seguro de que el paquete llegó del transmisor con quien comparte la clave secreta, y también puede estar seguro de que el paquete no fue alterado en el camino. Las propiedades del MD5 hacen imposible, desde el punto de vista de cómputo, que un intruso falsifique la identidad del transmisor o modifique el paquete de alguna manera que pueda escapar a la detección.

Es importante apuntar que la carga útil de un paquete con verificación de autenticidad se envía sin cifrado. Cualquier enrutador a lo largo del camino puede leer lo que dice. En muchas aplicaciones el secreto no es realmente importante, sólo la relación de autenticidad. Por ejemplo, si un usuario indica a su banco que pague su cuenta telefónica, probablemente no habrá necesidad real del secreto, pero hay una necesidad muy real de que el banco tenga la seguridad completa de quién envió el paquete que contiene la orden de pago.

Para paquetes que deben enviarse secretamente, se usa la cabecera de extensión de carga útil cifrada de seguridad. Ésta comienza con un número de clave de 32 bits, seguido de la carga cifrada. El algoritmo de cifrado es responsabilidad del transmisor y del receptor, pero por omisión es DES en modo de encadenamiento de bloque cifrado (CBC). Al usarse DES-CBC, el campo de carga útil comienza por el vector de inicialización (un múltiplo de 4 bytes), luego la carga útil, y luego el relleno a un múltiplo de 8 bytes. Si se desean tanto cifrado como verificación de autenticidad, se requieren ambas cabeceras.

La intención de la cabecera de opciones de destino es incluir campos que sólo deben interpretarse en el *host* de destino. En la versión inicial de IPv6, las únicas opciones definidas son las opciones nulas para rellenar esta cabecera a un múltiplo de 8 bytes, por lo que no se usará inicialmente; se incluyó para asegurar que el nuevo *software* de enrutamiento y de los *hosts* puedan manejarla, en caso de que a alguien se le ocurra una opción de destino algún día.

Controversias

Dados el proceso de diseño abierto y las fuertes opiniones de muchas de las personas participantes, no debería sorprender que muchas decisiones tomadas para el IPv6 fueran tema de fuertes controversias. Resumiremos a continuación algunas de ellas. Véanse en (Huitema, 1996) todos los detalles escabrosos.

Ya hemos mencionado el argumento sobre la longitud de las direcciones. El resultado fue una concesión: las direcciones de 16 bytes de longitud fija.

Surgió otra pelea sobre la longitud del campo de *límite de saltos*. Una parte sentía que la limitación de la cantidad máxima de saltos a 255 (implícita al usar un campo de 8 bits) fue un error grave. A fin de cuentas, hoy son comunes las trayectorias de 32 saltos, y en 10 años podrán ser comunes trayectorias mucho más grandes. Esta gente argumentaba que el uso de una dirección enorme era ir demasiado lejos, pero que el uso de una cuenta de saltos pequeña era tener una visión miope. Desde su punto de vista, el peor pecado que puede cometer un informático es proporcionar insuficientes bits en algún lugar.

La respuesta fue que se pueden hacer argumentos para aumentar todos los campos, lo que llevaría a una cabecera inflada. También, la función del campo de límite de saltos es evitar que los paquetes vaguen durante demasiado tiempo, y 65,535 saltos son demasiados. Por último, a medida que crezca Internet, se construirán más y más enlaces de larga distancia, posibilitando la ida de un país a otro en media docena de saltos, cuando mucho. Si se requieren más de 125 saltos para llegar del origen y el destino a sus pasarelas internacionales, algo está mal con las columnas vertebrales nacionales. Los de 8 bits ganaron esta partida.

Otra papa caliente fue el tamaño máximo de paquete. La comunidad de las supercomputadoras quería paquetes de más de 64 kB. Cuando una supercomputadora comienza a transferir, el asunto realmente va en serio, y no quiere que se le interrumpa cada 64 kB. El argumento en contra de los paquetes grandes es que, si un paquete de 1 MB llega a una línea T1 de 1.5 Mbps, el paquete bloqueará la línea durante más de 5 segundos, produciendo un retardo muy notorio para los usuarios interactivos que comparten la línea. Se llegó a una concesión: los paquetes normales se limitan a 64 kB, pero puede usarse la cabecera de extensión de salto por salto para permitir los jumbogramas.

Un tercer tema candente fue la desaparición de la suma de comprobación del IPv5. Para algunas personas esto representa algo parecido a quitarle los frenos a un automóvil. Hacerlo ciertamente aligera al automóvil, y por tanto puede ir más rápido pero, de ocurrir un evento inesperado, tendremos problemas.

El argumento en contra de las sumas de comprobación fue que cualquier aplicación a la que de verdad le importa la integridad de sus datos de todos modos tiene que tener una suma de comprobación en la capa de transporte, por lo que tener otra en el IP (además de la suma de comprobación de la capa de enlace de datos) es un exceso. Además, la experiencia mostraba que el cálculo de una suma de comprobación en el IP era un gasto importante en el IPv4. El bando en contra de la suma de comprobación ganó ésta, y el IPv6 no tiene una suma de comprobación.

Los *hosts* móviles también fueron tema de contienda. Si una computadora portátil vuela al otro lado del mundo, ¿puede continuar operando en el destino con la misma dirección IPv6, o

tiene que usar un esquema con agentes foráneos y agentes de base? Los *hosts* móviles también generan asimetrías en el sistema de enrutamiento. Es posible el caso en que una computadora móvil pequeña pueda escuchar fácilmente la señal emitida por un enrutador estacionario grande, pero que el enrutador estacionario no pueda escuchar la débil señal emitida por el *host* móvil. En consecuencia, algunas personas querían incluir reconocimiento explícito de *hosts* móviles en el IPv6. Este esfuerzo falló cuando no se pudo generar consenso para ninguna propuesta específica.

Probablemente la batalla principal fue sobre la seguridad. Todos estaban de acuerdo en que se necesitaba. La guerra fue sobre dónde y cuándo. Primero dónde. El argumento en favor de ponerla en la capa de red es que entonces se vuelve un servicio estándar que todas las aplicaciones pueden usar sin ninguna planeación adelantada. El argumento en contra es que las aplicaciones realmente seguras por lo general no quieren nada menos que el cifrado de terminal a terminal, donde la aplicación de origen hace el cifrado y la aplicación de destino lo deshace. Con cualquier cosa menos, el usuario está a merced de implementaciones de capa de red con fallas potenciales, sobre las que no tiene control. La respuesta a este argumento es que tales aplicaciones simplemente pueden abstenerse de usar las características de seguridad del IP y encargarse ellas mismas del asunto. La réplica a esto es que la gente que no confía en que la red lo haga bien no quiere pagar el precio de implementaciones de IP lentas y estorbosas que tengan esta capacidad, aun si está inhabilitada.

Otro aspecto sobre dónde poner la seguridad se relaciona con que muchos países (pero no todos) tienen leyes de exportación estrictas en lo referente a cifrado. Algunos, particularmente Francia e Irak, también restringen mucho su uso doméstico, de manera que la gente no pueda ocultar secretos de la policía. Como resultado, cualquier puesta en práctica de IP que use un sistema de cifrado lo bastante robusto como para tener algún valor no podría exportarse de los Estados Unidos (y de muchos otros países) a clientes mundiales. La mayoría de los proveedores de computadoras se oponen enérgicamente a tener que mantener dos juegos de *software*, uno para uso doméstico y otro para exportación.

Una solución potencial es que todos los proveedores muden sus talleres de cifrado a un país que no regule el cifrado, como Finlandia o Suiza. Se podría diseñar y fabricar *software* de cifrado sólido ahí y embarcarlo legalmente a todos los países, excepto Francia e Irak. El problema con este enfoque es que el diseño de parte del *software* de enrutamiento en un país y parte en otro puede llevar a problemas de integración.

La última controversia relacionada con la seguridad tiene que ver con la selección de los algoritmos predeterminados que todas las implementaciones deben reconocer. Si bien que se pensaba que el MD5 era relativamente seguro, los avances recientes en cifrado pueden debilitarlo. Ningún criptógrafo serio cree que el DES es seguro ante ataques de grandes gobiernos, pero probablemente es lo bastante bueno como para frustrar incluso al preadolescente más precoz, cuando menos por ahora. El arreglo fue entonces integrar seguridad en el IPv6, usar un algoritmo de suma de comprobación de primer nivel para una buena verificación de autenticidad y un algoritmo debilucho para el secreto, pero dar al usuario la opción de reemplazar estos algoritmos por los suyos propios.

Un punto donde no hubo controversia es que nadie espera que la Internet IPv4 se apague un domingo por la mañana y reinicie como Internet IPv6 la mañana del lunes. En cambio, se

convertirán "islas" aisladas a IPv6, comunicándose inicialmente a través de túneles. A medida que crezcan las islas IPv6, se integrarán a islas más grandes. Tarde o temprano todas las islas se integrarán, y la Internet habrá sido convertida por completo. Dada la inversión masiva en los enrutadores IPv4 actualmente instalados, el proceso de conversión probablemente tardará una década. Por esta razón, se ha puesto un enorme esfuerzo en asegurar que esta transición sea lo menos dolorosa posible.

5.6. LA CAPA DE RED EN LAS REDES ATM

Las capas del modelo ATM (véase la figura 1-30) no se proyectan en las capas OSI muy bien que digamos, lo que conduce a ambigüedades. La capa de enlace de datos OSI se encarga del protocolo de enmarcado y del de transferencia entre dos máquinas que están en el mismo alambre (o fibra) físico. Los protocolos de capa de enlace de datos son protocolos de un solo salto. No se ocupan de conexiones terminal a terminal, puesto que la conmutación y el enrutamiento no ocurren en la capa de enlace de datos. Sobre esto no hay duda.

La capa más baja que va del origen al destino, y por tanto comprende enrutamiento y conmutación (es decir, es multisalto), es la capa de red. La capa ATM se encarga de mover células del origen al destino, y en definitiva se relaciona con algoritmos y protocolos de enrutamiento en los conmutadores ATM. Esta capa también maneja las direcciones globales. Con tal funcionalidad, la capa ATM desempeña el trabajo esperado de la capa de red. La capa ATM no garantiza una confiabilidad del 100%, pero no se requiere un protocolo de capa de red.

También, la capa ATM se asemeja a la capa 3 del X.25, que nadie discute que sea un protocolo de capa de red. Dependiendo de estados de bits, el protocolo de capa de red X.25 puede o no ser confiable, pero la mayoría de las implementaciones lo tratan como no confiable. Dado que la capa ATM tiene la funcionalidad esperada de la capa de red, no tiene la funcionalidad esperada de la capa de enlace de datos, y es bastante parecida a otros protocolos existentes de capa de red, en este capítulo estudiaremos la capa ATM.

La confusión surge porque mucha gente de la comunidad ATM considera que la capa ATM es una capa de enlace de datos, e inclusive, al hacer la emulación de una LAN, una capa física. Mucha gente de la comunidad Internet también la considera como una capa de enlace de datos, porque quieren poner IP encima de ella, y el hacer de la capa ATM una capa de enlace de datos se adecua bien a este concepto. (Aunque, de acuerdo con este razonamiento, para la comunidad Internet *todas* las redes operan en la capa de enlace de datos, sin importar sus características físicas.)

El único problema es que la capa ATM no tiene las características de un protocolo de capa de enlace de datos: un protocolo de un solo salto usado por máquinas en las terminales opuestas de un alambre, como los protocolos 1 a 6 del capítulo 3. La capa ATM tiene las características de un protocolo de capa de red: circuitos virtuales terminal a terminal, conmutación y enrutamiento.

Esto recuerda al autor una vieja adivinanza:

Pregunta: ¿Cuántas piernas tendría una mula si llamáramos pierna a la cola?
Respuesta: Cuatro. El *llamar* pierna a la cola no la *convierte* en una pierna.

Dicho esto, al lector se le previene de la controversia que hay aquí, combinada con una buena dosis de puro sentimiento.

La capa ATM se orienta a conexiones, tanto en términos del servicio que ofrece como de la manera en que opera internamente. El elemento básico de la capa ATM es el circuito virtual (llamado con frecuencia **canal virtual**). Un circuito virtual normalmente es una conexión de un origen a un destino, aunque también se permiten conexiones multitransmisión. Los circuitos virtuales son unidireccionales, pero puede crearse un par de circuitos al mismo tiempo. Ambas partes del par se direccionan al mismo identificador, de modo que un circuito virtual de hecho es dúplex integral. Sin embargo, la capacidad del canal y otras propiedades pueden ser diferentes en las dos direcciones, e inclusive pueden ser de cero en una de ellas.

La capa ATM es inusual para un protocolo orientado a conexiones en el sentido de que no proporciona acuses de recibo. La razón de este diseño es que la ATM se diseñó para usarse en redes de fibra óptica, que son altamente confiables. Se consideró adecuado dejar el control de errores a las capas superiores. A fin de cuentas, el envío de acuses de recibo en la capa de enlace de datos o en la de red sólo es una optimización. Siempre es suficiente que la capa de transporte envíe un mensaje y luego envíe nuevamente el mensaje completo si no se tiene un acuse de recibo a tiempo.

Es más, las redes ATM se usan con frecuencia para tráfico en tiempo real, como audio y vídeo. Para este tipo de tráfico, la retransmisión ocasional de una célula mala es peor que simplemente ignorarla.

A pesar de su falta de acuses de recibo, la capa ATM sí proporciona una garantía firme: las células que se envíen por un circuito virtual nunca llegarán fuera de orden. Se permite a la subred ATM descartar células si ocurren congestionamientos, pero en ninguna circunstancia puede reordenar las células enviadas por un solo circuito virtual. Sin embargo, no se dan garantías sobre el orden si un *host* envía células por *diferentes* circuitos virtuales. Por ejemplo, si un *host* envía una célula por el circuito virtual 10 y luego envía otra por el circuito virtual 20 al mismo destino, la segunda célula podría llegar primero. Si las dos células se hubieran enviado por el mismo circuito virtual, la primera enviada siempre sería la primera en llegar.

La capa ATM reconoce una jerarquía de conexión de dos niveles que es visible a la capa de transporte. A lo largo de cualquier trayectoria de transmisión de un origen dado a un destino dado, un grupo de circuitos virtuales puede agruparse en lo que se llama una **trayectoria virtual**, como se presenta en la figura 5-61. Conceptualmente, una trayectoria virtual es como un haz de pares trenzados de cobre: al reenrutarlo, todos los pares (circuitos virtuales) se reenrutan juntos. Consideraremos en detalle las implicaciones de este diseño de dos niveles más adelante.

5.6.1. Formatos de célula

En la capa ATM se distinguen dos interfaces: la **UNI** (*User-Network Interface*, **interfaz usuario-red**) y la **NNI** (*Network-Network Interface*, **interfaz red-red**). La primera define el límite entre un *host* y una red ATM (en muchos casos, entre el cliente y la portadora). La última se aplica a la línea entre dos conmutadores ATM (el término ATM para los enrutadores).

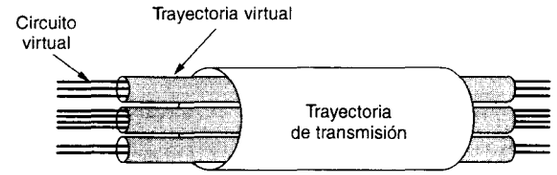


Figura 5-61. Una trayectoria de transmisión puede contener varias trayectorias virtuales, cada una de las cuales puede contener varios circuitos virtuales.

En ambos casos, las células consisten en una cabecera de 5 bytes seguida de una carga útil de 48 bytes, pero las dos cabeceras son ligeramente diferentes. Las cabeceras, como las definieron en el foro ATM, se ilustran en la figura 5-62. Las células se transmiten comenzando por el byte más a la izquierda, y por el bit más a la izquierda de cada byte.

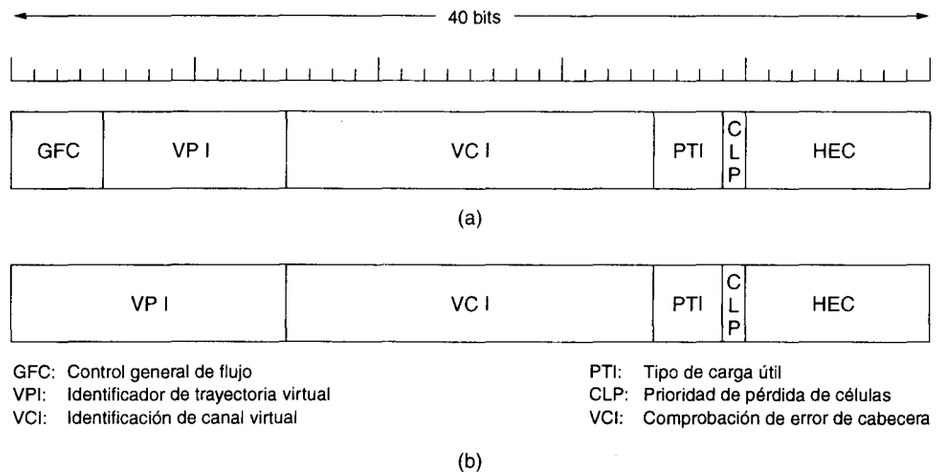


Figura 5-62. (a) La cabecera de capa ATM en la UNI. (b) La cabecera de capa ATM en la NNI.

El campo *GFC* está presente sólo en las células entre un *host* y la red; es sobrescrito por el primer conmutador al que llega, por lo que no tiene un significado de terminal a terminal, y no se entrega al destino. Originalmente se pensó que este campo tendría alguna utilidad para el control de flujo o de prioridad entre los *hosts* y las redes, pero no hay valores definidos para él, y la red lo ignora. La mejor manera de pensar en él es como una falla del estándar.

El campo *VPI* es un entero pequeño que selecciona una trayectoria virtual en particular (véase la figura 5-61). De la misma manera, el campo *VCI* selecciona un circuito virtual en particular en la trayectoria virtual seleccionada. Dado que el campo *VPI* tiene 8 bits (en la UNI) y el campo *VCI* tiene 16 bits, en teoría un *host* puede tener hasta 256 haces de VC, conteniendo cada uno hasta 65,536 circuitos virtuales. En realidad, la cantidad disponible de ambos es un

poco menor, pues algunos *VCI* se reservan para funciones de control, como el establecimiento de circuitos virtuales.

El campo *PTI* define el tipo de carga útil que contiene la célula, de acuerdo con los valores dados en la figura 5-63. Aquí los tipos de célula son proporcionados por el usuario, pero la información de congestionamientos es proporcionada por la red. En otras palabras, una célula enviada con *PTI* 000 podría llegar con 010 para avisar al destino que hay problemas en el camino.

Tipo de carga	Significado
000	Célula de datos de usuario, sin congestionamientos, célula tipo 0
001	Célula de datos de usuario, sin congestionamientos, célula tipo 1
010	Célula de datos de usuario, hubo congestionamiento, célula tipo 0
011	Célula de datos de usuario, hubo congestionamiento, célula tipo 1
100	Información de mantenimiento entre conmutadores adyacentes
101	Información de mantenimiento entre conmutadores de origen y destino
110	Célula de administración de recursos (usada para el control de congestionamientos ABR)
111	Reservado para función futura

Figura 5-63. Valores del campo *PTI*.

El bit *CLP* puede ser establecido por un *host* para distinguir entre el tráfico de alta prioridad y el de baja prioridad. Si ocurre un congestionamiento y deben descartarse células, los conmutadores primero intentan descartar las que tienen el *CLP* establecido en 1 antes de descartar cualquiera que lo tenga establecido en 0.

Por último, el campo *HEC* es una suma de comprobación de la cabecera; no verifica la carga útil. Un código Hamming de un número de 40 bits sólo requiere 5 bits, por lo que, con 8 bits, puede usarse un código más refinado. El escogido puede corregir todos los errores de un solo bit y puede detectar aproximadamente el 90% de los errores multibit. Numerosos estudios han mostrado que la gran mayoría de los errores de los enlaces ópticos son errores de un solo bit.

A continuación de la cabecera vienen 48 bytes de carga útil. Sin embargo, no todos los 48 bytes están disponibles para el usuario, pues algunos de los protocolos AAL ponen sus cabeceras y sus terminaciones dentro de la carga.

El formato NNI es igual al formato UNI, excepto que el campo *GFC* no está presente y esos 4 bits se usan para hacer que el campo *VPI* sea de 12 bits en lugar de 8.

5.6.2. Establecimiento de la conexión

ATM maneja circuitos virtuales tanto permanentes como conmutados. Los primeros siempre están presentes y pueden usarse a voluntad, como en las líneas arrendadas. Los segundos tienen que establecerse cada vez que se usan, como al hacer una llamada telefónica. En esta sección describiremos la manera en que se establecen los circuitos virtuales conmutados.

Técnicamente, el establecimiento de la conexión no es parte de la capa ATM, sino que es manejado por el plano de control (véase la figura 1-30) usando un protocolo ITU muy complicado llamado **Q.2931** (Stiller, 1995). No obstante, el lugar lógico para manejar el establecimiento de una conexión de capa de red es en la capa de red, y otros protocolos de capa de red parecidos se encargan del establecimiento de la conexión aquí, por lo que ahora lo estudiaremos.

Hay varias maneras de establecer una conexión. La normal es adquirir primero un circuito virtual para señalización, y usarlo. Para establecer tal circuito, células que contienen una solicitud se envían por la trayectoria virtual 0, circuito virtual 5. Si hay éxito, se abre un circuito virtual nuevo por el que pueden enviarse y recibirse solicitudes y respuestas de establecimiento de conexión.

La razón de este procedimiento de establecimiento de dos pasos es que de esta manera el ancho de banda reservado para el circuito virtual 5 (que apenas se usa) puede mantenerse extremadamente bajo. También, se proporciona una manera alterna de establecer circuitos virtuales. Algunas portadoras pueden permitir que los usuarios tengan trayectorias virtuales permanentes entre destinos predefinidos, o permitirles establecerlos dinámicamente. Una vez que un *host* tiene una trayectoria virtual a algún otro *host*, puede asignar él mismo circuitos virtuales, sin implicar a los conmutadores.

El establecimiento de un circuito virtual usa los seis tipos de mensajes listados en la figura 5-64. Cada mensaje ocupa una o más células y contiene el tipo de mensaje, la longitud y algunos parámetros. Los mensajes pueden ser enviados por un *host* a la red o por la red (generalmente en respuesta a un mensaje de otro *host*) a un *host*. También existen varios otros mensajes de informe de estado y de errores, pero no se muestran aquí.

Mensaje	Significado al ser enviado por un *host*	Significado al ser enviado por una red
ESTABLECER	Favor de establecer un circuito	Llamada entrante
LLAMADA EN PROCESO	Vi la llamada entrante	Se intentará su solicitud de llamada
CONEXIÓN	Acepto la llamada entrante	Se aceptó su solicitud de llamada
CONEXIÓN RECONOCIDA	Gracias por aceptar	Gracias por hacer la llamada
LIBERACIÓN	Favor de terminar la llamada	La otra parte ya no quiere hacer más
LIBERACIÓN COMPLETA	Reconocimiento de LIBERACIÓN	Reconocimiento de LIBERACIÓN

Figura 5-64. Mensajes usados para establecer y liberar conexiones.

El procedimiento normal para establecer una llamada es que un *host* envíe un mensaje de ESTABLECER (*SETUP*) por un circuito virtual especial. La red entonces responde con LLAMADA EN PROCESO (*CALL PROCEEDING*)para reconocer la recepción de la solicitud. A medida que el mensaje de ESTABLECER se propaga hacia el destino, es reconocido en cada salto por un mensaje de LLAMADA EN PROCESO.

Cuando el mensaje de ESTABLECER finalmente llega a su destino, el *host* de destino puede responder con CONEXIÓN (*CONNECT*) para aceptar la llamada. La red envía entonces un mensaje de CONEXIÓN RECONOCIDA (*CONNECT ACK*) para indicar que se ha recibido el mensaje de CONEXIÓN. A

medida que el mensaje de conexión se propaga de regreso al originador, cada conmutador que lo recibe lo reconoce con un mensaje de CONEXIÓN RECONOCIDA. Esta secuencia de eventos se presenta en la figura 5-65(a).

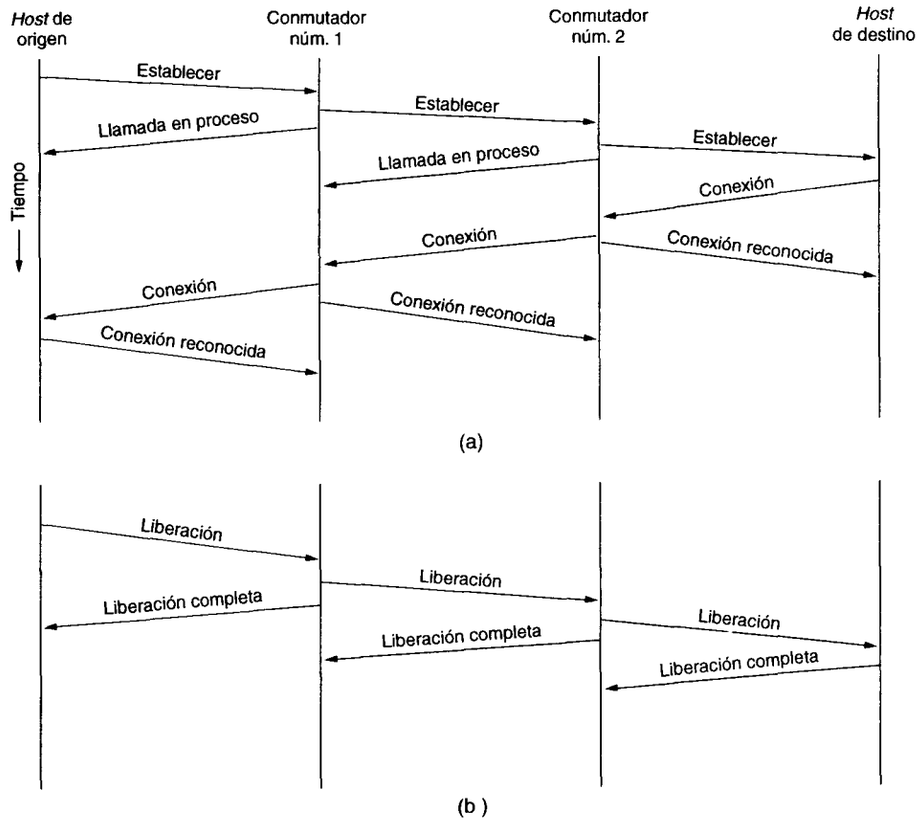


Figura 5-65. (a) Establecimiento de una conexión en una red ATM. (b) Liberación de la conexión.

La secuencia para liberar un circuito virtual es sencilla. El *host* que desea colgar simplemente envía un mensaje de LIBERACIÓN (*RELEASE*), que se propaga al otro extremo y causa que el circuito se libere. En cada salto a lo largo del camino se reconoce el mensaje, como se muestra en la figura 5-65(b).

Las redes ATM permiten el establecimiento de canales multitransmisión. Un canal multitransmisión tiene un transmisor y más de un receptor; se construye estableciendo una conexión con uno de los destinos de la manera normal. Entonces se envía el mensaje AGREGAR PARTE (*ADD PARTY*) para sumar un segundo destino al circuito virtual devuelto por la llamada previa. Pueden enviarse posteriormente más mensajes AGREGAR PARTE para aumentar el tamaño del grupo de multitransmisión.

A fin de establecer una conexión con un destino, es necesario especificar el destino, incluyendo su dirección en el mensaje de ESTABLECER. Las direcciones ATM tienen tres formas. La primera es de 20 bytes de longitud y se basa en las direcciones OSI. El primer byte indica en cuál de tres formatos está la dirección. En el primer formato, los bytes 2 y 3 especifican un país, y el byte 4 da el formato del resto de la dirección, que contiene una autoridad de 3 bytes, un dominio de 2 bytes, un área de 2 bytes y una dirección de 6 bytes, más algunos otros elementos. En el segundo formato, los bytes 2 y 3 designan a una organización internacional en lugar de un país. El resto de la dirección es igual que en el formato 1. De manera alterna, también se permite una forma más vieja de direccionamiento (CCITT E.164) que usa números telefónicos ISDN decimales de 15 dígitos.

5.6.3. Enrutamiento y conmutación

Al establecerse un circuito virtual, el mensaje de ESTABLECER sigue su camino a través de la red del origen al destino. El algoritmo de enrutamiento determina la trayectoria tomada por este mensaje y, por tanto, el circuito virtual. El estándar ATM no especifica ningún algoritmo de enrutamiento en particular, por lo que la portadora está en libertad de escoger entre los algoritmos estudiados antes en este capítulo, o usar otro diferente.

La experiencia con las redes previas orientadas a conexiones, como X.25, ha demostrado que puede gastarse una cantidad considerable de capacidad de cómputo de los conmutadores en la determinación de la manera de convertir la información del circuito virtual de cada célula en la selección de la línea de salida. Los diseñadores de ATM quisieron evitar esto, por lo que se ha diseñado la capa ATM para hacer posible un enrutamiento eficiente. En particular, la idea fue enrutar en el campo *VPI*, pero no en el campo *VCI*, excepto en el salto final en cada dirección, cuando las células se envían entre un conmutador y un *host*. Entre dos conmutadores sólo debía usarse la trayectoria virtual.

El uso de sólo los *VPI* entre los conmutadores del interior tiene varias ventajas. Para comenzar, una vez que se ha establecido una trayectoria virtual entre un origen y un destino, cualquier circuito virtual adicional a lo largo de esa trayectoria simplemente puede seguir la trayectoria existente. No se tienen que tomar nuevas decisiones de enrutamiento. Es como si ya se hubiera tendido un haz de pares trenzados entre el origen y el destino. El establecimiento de una conexión nueva simplemente requiere la asignación de uno de los pares sin uso.

Segundo, el enrutamiento de células individuales es más fácil cuando todos los circuitos virtuales de una trayectoria dada están siempre en el mismo haz. La decisión de enrutamiento sólo implica examinar un número de 12 bits, no un número de 12 bits y otro de 16 bits. Describiremos más adelante la manera en que ocurre la conmutación de células pero, aun sin entrar en detalles, debe ser claro que es factible la indización en una tabla de 2^{12} entradas, pero que en una tabla de 2^{28} no lo es.

Tercero, basar todo el enrutamiento en trayectorias virtuales facilita la conmutación de un grupo completo de circuitos virtuales. Considere por ejemplo la hipotética columna vertebral ATM de Estados Unidos ilustrada en la figura 5-66. Normalmente los circuitos virtuales de Nueva York a San Francisco pasan a través de Omaha y Denver. Sin embargo, suponga que

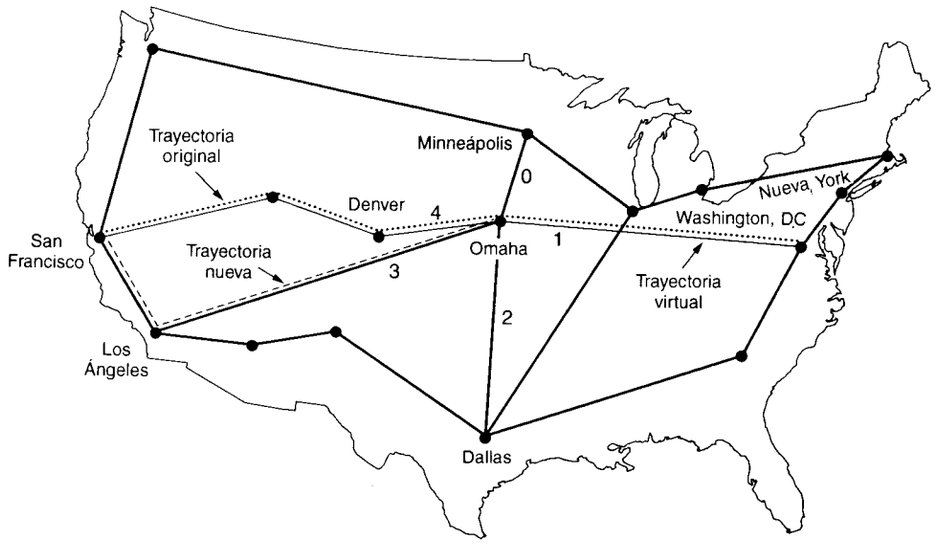


Figura 5-66. El reenrutamiento de una trayectoria virtual también reenruta todos sus circuitos virtuales.

ocurre una perturbación en la línea Omaha-Denver. Al reenrutar la trayectoria virtual Omaha-Denver a Los Ángeles y luego a San Francisco, pueden conmutarse todos los circuitos virtuales (potencialmente 65,535) en una sola operación, en lugar de miles de operaciones potenciales.

Por último, las trayectorias virtuales facilitan a las portadoras ofrecer grupos cerrados de usuarios (redes privadas) a los clientes corporativos. Una compañía puede establecer una red de trayectorias virtuales permanentes entre sus diferentes oficinas, y luego asignar circuitos virtuales entre estas trayectorias a solicitud. No pueden entrar llamadas en la red privada desde fuera, y ninguna llamada puede salir de la red privada, excepto a través de pasarelas especiales. A muchas compañías les gusta este tipo de seguridad.

Aún está por verse si los conmutadores realmente usarán el campo *VPI* para enrutamiento, como se planeó, o si usarán la combinación de los campos *VPI* y *VCI* (negando entonces todas las ventajas que acabamos de estudiar). La experiencia inicial en el campo no es alentadora.

Veamos ahora cómo se pueden enrutar células en un conmutador interno (uno que está conectado solamente a otros conmutadores, y no a ningún *host*). Para hacer concreto el planteamiento, considérese el conmutador de Omaha de la figura 5-66. Para cada una de sus cinco líneas de entrada tiene una tabla, *vpi_table*, indizada por los *VPI* de entrada, que indica cuál de las cinco líneas de salida debe usar y qué *VPI* debe poner en las células de salida. Supongamos que las cinco líneas tienen la numeración de 0 a 4, en dirección de las manecillas del reloj, comenzando por Minneápolis. Para cada línea de salida, el conmutador mantiene un mapa de bits que indica los *VPI* usados actualmente en esa línea.

Al activarse el conmutador, todas las entradas de todas las estructuras *vpi_table* se marcan como desocupadas. De la misma manera, todos los mapas de bits se marcan para indicar que

Origen	Línea de entrada	VPI de entrada	Destino	Línea de salida	VPI de salida	Trayectoria
Nueva York	1	1	San Francisco	4	1	Nueva
Nueva York	1	2	Denver	4	2	Nueva
Los Ángeles	3	1	Minneápolis	0	1	Nueva
Washington, DC	1	3	Los Ángeles	3	2	Nueva
Nueva York	1	1	San Francisco	4	1	Anterior
San Francisco	4	3	Washington, DC	1	4	Nueva
Washington, DC	1	5	San Francisco	4	4	Nueva
Nueva York	1	2	Denver	4	2	Anterior
San Francisco	4	5	Minneápolis	0	2	Nueva
Nueva York	1	1	San Francisco	4	1	Anterior

Figura 5-67. Algunas rutas a través del conmutador de Omaha de la figura 5-66.

están disponibles todos los *VPI* (excepto los reservados). Ahora supóngase que llegan llamadas, como se muestra en la figura 5-67.

A medida que se establecen las trayectorias virtuales (y circuitos virtuales), se hacen entradas en las tablas. Supondremos que los circuitos virtuales son dúplex integral, de modo que cada uno que se establece resulta en dos entradas, una para el tráfico directo del origen y otra para el tráfico en reversa del destino.

Las tablas que corresponden a las rutas de la figura 5-67 se muestran en la figura 5-68. Por ejemplo, la primera llamada genera la entrada (4, 1) para *VPI* 1 de la tabla de Washington, DC, porque se refiere a células que entran por la línea 1, con *VPI* 1 y con destino a San Francisco. Sin embargo, también se hace una entrada en la tabla de Denver para *VPI* 1 mostrando que las células que vienen de Denver con *VPI* 1 deben salir por la línea 1 con *VPI* 1. Estas son células que viajan en el otro sentido (de San Francisco a Nueva York) por esta trayectoria virtual. Nótese que en algunos casos dos o tres circuitos virtuales comparten una trayectoria común. No se necesitan entradas nuevas en las tablas para los circuitos virtuales adicionales que conecten un origen y un destino que ya tienen una trayectoria asignada.

Ahora podemos explicar la manera en que las células se procesan dentro de un conmutador. Supóngase que llega una célula por la línea 1 (Washington, DC) con *VPI* 3. El *hardware* o el *software* del conmutador usa el 3 como índice en la tabla de la línea 1 y ve que la célula debe salir por la línea 3 (Los Ángeles) con *VPI* 2; sobrescribe el campo *VPI* con un 2 y pone el número de la línea de salida, 3, en algún lugar de la célula, por ejemplo, en el campo *HEC* dado que, de todas maneras, éste tendrá que recalcularse más adelante.

Ahora la pregunta es cómo llevar la célula de su *buffer* de entrada actual a la línea 3. Sin embargo, este asunto (el enrutamiento dentro de un conmutador) se estudió detalladamente en el capítulo 2, y vimos la manera de hacerlo con conmutadores por *knockout* y Batcher-banianos.

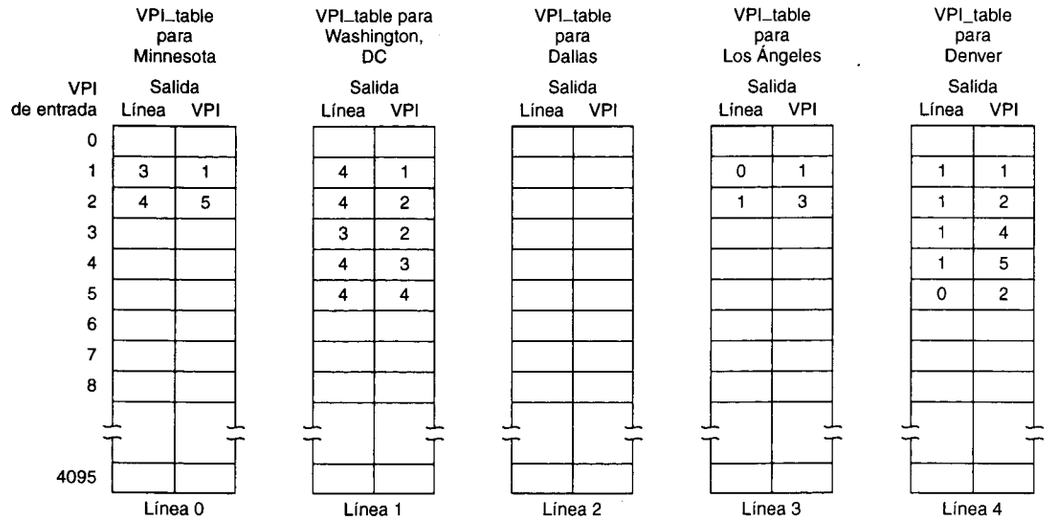


Figura 5-68. Entradas en las tablas para las rutas de la figura 5-67.

En este punto es fácil ver la manera en que puede reenrutarse un haz entero de circuitos virtuales, como se hace en la figura 5-66. Al cambiar la entrada de *VPI* 1 de la tabla de Washington, DC de (4, 1) a (3, 3), las células de Nueva York para San Francisco se desviarán a Los Ángeles. Por supuesto, hay que notificar este evento al conmutador de Los Ángeles, por lo que el conmutador tiene que generar y enviar un mensaje de ESTABLECER a Los Ángeles para establecer la trayectoria nueva con *VPI* 3. Una vez establecida esta trayectoria, todos los circuitos virtuales de Nueva York a San Francisco se redirigen a través de Los Ángeles, aun si hay miles de ellos. Si no existieran las trayectorias virtuales, cada circuito virtual tendría su propia entrada en la tabla y se reenrutaría por separado.

Vale la pena apuntar explícitamente que el análisis anterior se refiere al ATM en las WAN. En una LAN el asunto es mucho más sencillo. Por ejemplo, puede usarse una sola trayectoria virtual para todos los circuitos virtuales.

5.6.4. Categorías de servicios

Después de un buen tiempo de prueba y error, ya para la especificación 4.0 del ATM estaban claros los tipos de tráfico que llevaban las redes ATM y los tipos de servicios requeridos por el cliente. En consecuencia, el estándar se modificó para listar explícitamente las categorías de servicio comúnmente usadas, a fin de permitir a los proveedores de equipo optimizar sus tarjetas adaptadoras y los conmutadores para algunas o todas estas categorías. Las categorías de servicios seleccionadas como importantes se listan en la figura 5-69.

La clase **CBR** (*Constant Bit Rate*, **tasa de bits constante**) pretende simular un alambre de cobre o una fibra óptica (sólo que a un costo mucho mayor). Los bits se ponen en un extremo y

Clase	Descripción	Ejemplo
CBR	Tasa de bits constante	Circuito T1
RT-VBR	Tasa de bits variable: tiempo real	Vídeoconferencias en tiempo real
NRT-VBR	Tasa de bits variable: tiempo no real	Correo electrónico multimedia
ABR	Tasa de bits disponible	Consultas al *Web*
UBR	Tasa de bits no especificada	Transferencia de archivos en segundo plano

Figura 5-69. Categorías de servicios ATM.

salen por el otro. No hay comprobación de errores, control de flujo ni ningún otro proceso. No obstante, esta clase es esencial para hacer una transición suave entre el sistema telefónico actual y los sistemas B-ISDN del futuro, ya que los canales PCM de grado de voz, los circuitos T1 y la mayor parte del resto del sistema telefónico usa la transmisión de bits síncrona de tasa constante. Con la clase CBR puede transportarse directamente todo este tráfico a través de un sistema ATM. La CBR también es adecuada para todas las demás cadenas interactivas (es decir, en tiempo real) de audio y vídeo.

La siguiente clase, la **VBR** (*Variable Bit Rate*, **tasa variable de bits**), se divide en dos subclases, la de tiempo real (RT-VBR) y la de tiempo no real (NRT-VBR), respectivamente. La RT-VBR es para servicios que tienen tasas de bits variables en combinación con requisitos muy estrictos de tiempo real, como el vídeo comprimido interactivo (por ejemplo, videoconferencias). Debido a la manera en que funcionan el MPEG y otros esquemas de compresión, con un marco base completo seguido de una serie de diferencias entre el marco actual y el marco base, la tasa de transmisión varía intensamente en el tiempo (Pancha y El Zarki, 1994). A pesar de esta variación, es esencial que la red ATM no genere ninguna fluctuación en el patrón de llegada de las células, ya que esto causaría que la imagen apareciese inestable. En otras palabras, tanto el retardo medio de las células como la variación del retardo de las células deben estar cuidadosamente controlados. Por otra parte, aquí es tolerable un bit o célula perdido de vez en cuando y lo mejor es ignorarlo.

La otra subclase VBR es para tráfico en el que la entrega a tiempo es importante pero la aplicación puede tolerar una cierta cantidad de fluctuación. Por ejemplo, el correo electrónico multimedia normalmente se almacena temporalmente en el disco local del receptor antes de presentarse en pantalla, por lo que cualquier variación en los tiempos de entrega de las células se eliminaría antes de visualizarse el correo electrónico.

La categoría de servicio **ABR** (*Available Bit Rate*, **tasa de datos disponible**) se diseñó para trabajo en ráfagas cuya gama de ancho de banda se conoce aproximadamente. Un ejemplo típico podría ser su uso en una compañía que conecta sus oficinas mediante un conjunto de líneas arrendadas. Normalmente, la compañía puede seleccionar entre instalar suficiente capacidad para manejar la carga pico, lo que significa que algunas líneas están ociosas parte del día, o instalar la suficiente capacidad para la carga mínima, lo que conduce a congestionamientos durante la parte más atareada del día.

El uso del servicio ABR evita tener que comprometerse con un ancho de banda fijo. Con el ABR es posible decir, por ejemplo, que la capacidad entre dos puntos siempre será de 5 Mbps, pero puede tener picos de hasta 10 Mbps. El sistema garantizará entonces 5 Mbps todo el tiempo, y hará lo posible para proporcionar 10 Mbps cuando sea necesario, pero sin prometer nada.

La ABR es la única categoría de servicio en la que la red proporciona tasa de retroalimentación al transmisor, solicitándole que disminuya la velocidad al ocurrir congestionamientos. Suponiendo que el transmisor respeta tales solicitudes, se espera que la pérdida de células del tráfico ABR sea baja. Viajar por ABR es un poco como volar por lista de espera: si hay asientos sobrantes (sobrecapacidad), los pasajeros en lista de espera son transportados sin demora. Si hay capacidad insuficiente, tienen que esperar (a menos que esté disponible un poco del ancho de banda mínimo).

Por último llegamos a la **UBR** (*Unspecified Bit Rate*, **tasa de datos no especificada**), que no hace promesas y no realimenta información sobre los congestionamientos. Esta categoría se adapta bien al envío de paquetes IP, puesto que el IP tampoco hace promesas respecto a la entrega. Se aceptan todas las células UBR y, si sobra capacidad, también se entregan. Si ocurren congestionamientos, se descartan las células UBR sin realimentación al transmisor y sin esperar que el transmisor reduzca su velocidad.

Continuando con nuestra analogía de la lista de espera, con el UBR todos los pasajeros en lista de espera llegan a abordar, pero si a medio camino el piloto ve que se agota el combustible, los pasajeros en lista de espera son arrojados sin miramientos por la salida de emergencia. Para hacer atractiva la UBR, las portadoras tienden a hacerla más barata que las otras clases. Para aplicaciones que no tienen restricciones de entrega y quieren llevar ellas mismas su propio control de errores y de flujo, la UBR es una elección razonable. La transferencia de archivos, el correo electrónico y las noticias del USENET son candidatos potenciales para el servicio UBR, puesto que ninguna de estas aplicaciones tiene características de tiempo real.

Las propiedades de las diferentes categorías de servicio se resumen en la figura 5-70.

Característica de servicio	CBR	RT-VBR	NRT-VBR	ABR	UBR
Garantía de ancho de banda	Sí	Sí	Sí	Opcional	No
Adecuado para tráfico en tiempo real	Sí	Sí	No	No	No
Adecuado para tráfico en ráfagas	No	No	Sí	Sí	Sí
Realimentación sobre congestionamientos	No	No	No	Sí	No

Figura 5-70. Características de las categorías de servicio ATM.

5.6.5. Calidad del servicio

La calidad del servicio es un asunto importante para las redes ATM, en parte porque se usan para tráfico en tiempo real, como audio y vídeo. Al establecerse un circuito virtual, tanto la capa de transporte (por lo común un proceso de la máquina *host*, el "cliente") como la capa de red

ATM (por ejemplo, un operador de red, la "portadora") deben ponerse de acuerdo en un contrato que defina el servicio. En el caso de una red pública, este contrato debe tener implicaciones legales. Por ejemplo, si la portadora está de acuerdo en no perder más de una célula por cada mil millones y pierde dos células por cada mil millones, los abogados del cliente pueden alterarse y comenzar a gritar "¡incumplimiento de contrato!"

El contrato entre el cliente y la red tiene tres partes:

1. El tráfico que se generará.
2. El servicio acordado.
3. Los requisitos de cumplimiento.

Vale la pena apuntar que el contrato puede ser diferente para cada sentido. En una aplicación de vídeo a solicitud, el ancho de banda necesario para el control remoto del usuario al servidor de vídeo podría ser de 1200 bps. En el otro sentido podría ser de 5 Mbps. Cabe señalar que, si el cliente y la portadora no pueden acordar los términos, o si la portadora es incapaz de proporcionar el servicio deseado, no se establecerá el circuito virtual.

La primera parte del contrato es el **descriptor de tráfico** que caracteriza la carga que se generará. La segunda parte del contrato especifica la calidad del servicio deseada por el cliente y aceptada por la portadora. Tanto la carga como el servicio deben formularse en términos de cantidades medibles, para que el cumplimiento pueda determinarse objetivamente. Decir simplemente "carga moderada" o "buen servicio" no es suficiente.

Para hacer posible tener contratos de tráfico concretos, el estándar ATM define una serie de parámetros de **QoS** (*Quality of Service*, **calidad de servicio**) cuyos valores pueden negociar el cliente y la portadora. Para cada parámetro de calidad de servicio se especifica el desempeño de peor caso y se requiere que la portadora lo cumpla o lo mejore. En algunos casos, el parámetro es un mínimo; en otros es un máximo. Nuevamente, la calidad del servicio se especifica por separado para cada sentido; algunas de las más importantes se listan en la figura 5-71, pero no todas aplican a todas las categorías de servicio.

Los primeros tres parámetros especifican la rapidez a la que quiere transmitir el usuario. La **PCR** (*Peak Cell Rate*, **tasa de células pico**) es la rapidez máxima con que el transmisor planea enviar células. Este parámetro puede ser menor que lo permitido por el ancho de banda de la línea. Si el transmisor planea sacar células cada 4 μseg, su *PCR* es de 250,000 células/seg, aun si el tiempo real de transmisión de células puede ser de 2.7 μseg.

La **SCR** (*Sustained Cell Rate*, **tasa de células sostenida**) es la tasa esperada o requerida de células promediada en un intervalo de tiempo grande. Para el tráfico CBR, la *SCR* será igual a la *PCR*, pero para las demás categorías de servicio será sustancialmente menor. La razón *PCR/SCR* es una medida de las ráfagas del tráfico.

La **MCR** (*Minimum Cell Rate*, **tasa de células mínima**) es la cantidad mínima de células/seg que el cliente considera aceptable. Si la portadora es incapaz de garantizar esta cantidad de ancho de banda, debe rechazar la conexión. Cuando se solicita el servicio ABR, el ancho de banda real debe estar entre la *MCR* y la *PCR*, pero puede variar dinámicamente durante el

Parámetro	Siglas	Significado
Tasa de células pico	PCR	Tasa máxima a la que se enviarán células
Tasa de células sostenida	SCR	Tasa de células promedio a largo plazo
Tasa de células mínima	MCR	La tasa de células mínima aceptable
Tolerancia de variación de retardo de células	CDVT	La fluctuación de células máxima aceptable
Tasa de pérdidas de células	CLR	Fracción de las células perdidas o entregadas demasiado tarde
Retardo de transferencia de células	CTD	El tiempo que se lleva la entrega (medio y máximo)
Variación de retardo de células	CDV	La variación en los tiempos de entrega de células
Tasa de errores de células	CER	Fracción de las células entregadas sin error
Proporción de bloques de células con errores severos	SECBR	Fracción de los bloques alterados
Tasa de mala inserción de células	CMR	Fracción de las células entregadas a un destino equivocado

Figura 5-71. Algunos de los parámetros de calidad de servicio.

tiempo de vida de la conexión. Si el cliente y la portadora acuerdan establecer la *MCR* en 0, entonces el servicio ABR se vuelve similar al servicio UBR.

La **CVDT** (*Cell Variation Delay Tolerance*, **tolerancia de variación de retardo de células**) indica la cantidad de variación que habrá en los tiempos de transmisión de las células; se especifica independientemente para la *PCR* y la *SCR*. Con un origen perfecto que opera a la *PCR*, cada célula aparecerá *exactamente* 1/*PCR* después de la previa. Ninguna célula llegará temprano y ninguna llegará tarde, ni siquiera un picosegundo. Para un origen real que opera a la *PCR*, habrá un poco de variación en los tiempos de transmisión de las células. La pregunta es: ¿cuánta variación es aceptable? ¿Puede llegar una célula 1 nseg antes? ¿Qué tal 30 segundos? La *CDVT* controla la cantidad de variabilidad aceptable usando un algoritmo de cubeta con goteo que describiremos pronto.

Los siguientes tres parámetros describen características de la red y se miden en el receptor. Los tres son negociables. La **CLR** (*Cell Loss Ratio*, **tasa de pérdida de células**) es autoexplicativa. Mide la fracción de las células transmitidas que no se entregan en absoluto o que se entregan demasiado tarde, siendo inservibles por lo mismo (por ejemplo, en el tráfico en tiempo real). El **CTD** (*Cell Transfer Delay*, **retardo de transferencia de células**) es el tiempo promedio de tránsito del origen al destino. La **CDV** (*Cell Delay Variation*, **variación de retardo de células**) mide la uniformidad con que se entregan las células.

El modelo de *CTD* y *CDV* se muestra en la figura 5-72. Aquí vemos la probabilidad de que una célula tarde un tiempo t en llegar, en función de t. Para un origen, destino y ruta a través de los conmutadores intermedios dados, siempre hay un retardo mínimo debido al tiempo de propagación y de conmutación. Sin embargo, no todas las células llegan en el tiempo mínimo; la función de densidad de probabilidad por lo general tiene una cola grande. Escogiendo un valor de *CTD*, el cliente y la portadora de hecho acuerdan la tardanza con la que puede entregarse una célula y aún considerarla como correctamente entregada. Normalmente la *CDV* se escogerá de manera que α, la fracción de células rechazadas por llegar demasiado tarde, sea del orden

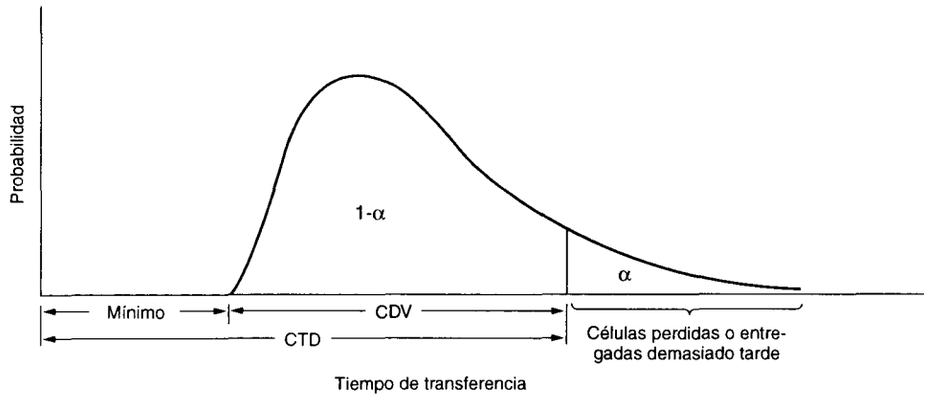


Figura 5-72. La función de densidad de probabilidad para los tiempos de llegada de las células.

de 10^{-10} o menos. La *CDV* mide la distribución de tiempos de entrega. Para el tráfico en tiempo real, este parámetro con frecuencia es más importante que el *CTD*.

Los últimos tres parámetros QoS especifican características de la red, y generalmente no son negociables. La **CER** (*Cell Error Ratio*, **razón de errores de células**) es la fracción de las células que se entregan con uno o más bits equivocados. La **SECBR** (*Severely-Errored Cell Block Ratio*, **tasa de bloques de células con errores severos**) es la fracción de bloques de N células en los que M o más células contienen errores. Por último, la **CMR** (*Cell Misinsertion Rate*, **tasa de mala inserción de células**) es la cantidad de células/seg que se entregan al destino equivocado debido a un error no detectado en la cabecera.

La tercera parte del contrato de tráfico indica lo que constituye el cumplimiento de las reglas. Si el cliente envía una célula demasiado pronto, ¿invalida esto el contrato? Si la portadora no alcanza a cumplir uno de los objetivos de calidad durante un periodo de 1 mseg, ¿puede interponer una demanda el cliente? De hecho, esta parte del contrato se negocia entre las partes y dice qué tan estrictamente se harán cumplir las primeras dos partes.

Los modelos de calidad de servicio del ATM y de Internet son algo diferentes, lo que tiene un impacto en sus respectivas puestas en práctica. El modelo ATM se basa estrictamente en las conexiones, mientras que el modelo de Internet usa datagramas más flujos (por ejemplo, RSVP). En (Crowcroft *et al.*, 1995) se hace una comparación entre estos dos modelos.

5.6.6. Conformación y vigilancia de tráfico

El mecanismo para usar y hacer cumplir los parámetros de calidad de servicio se basa (en parte) en un algoritmo específico, el algoritmo genérico de tasa de células (**GCRA, Generic Cell Rate Algorithm**), y se ilustra en la figura 5-73. Este algoritmo funciona revisando cada célula para ver si se ajusta a los parámetros de su circuito virtual.

El GCRA tiene dos parámetros. Estos especifican la tasa de llegada máxima permitida (*PCR*) y la cantidad de variación de ésta que puede tolerarse (*CDVT*). El recíproco de la *PCR*,

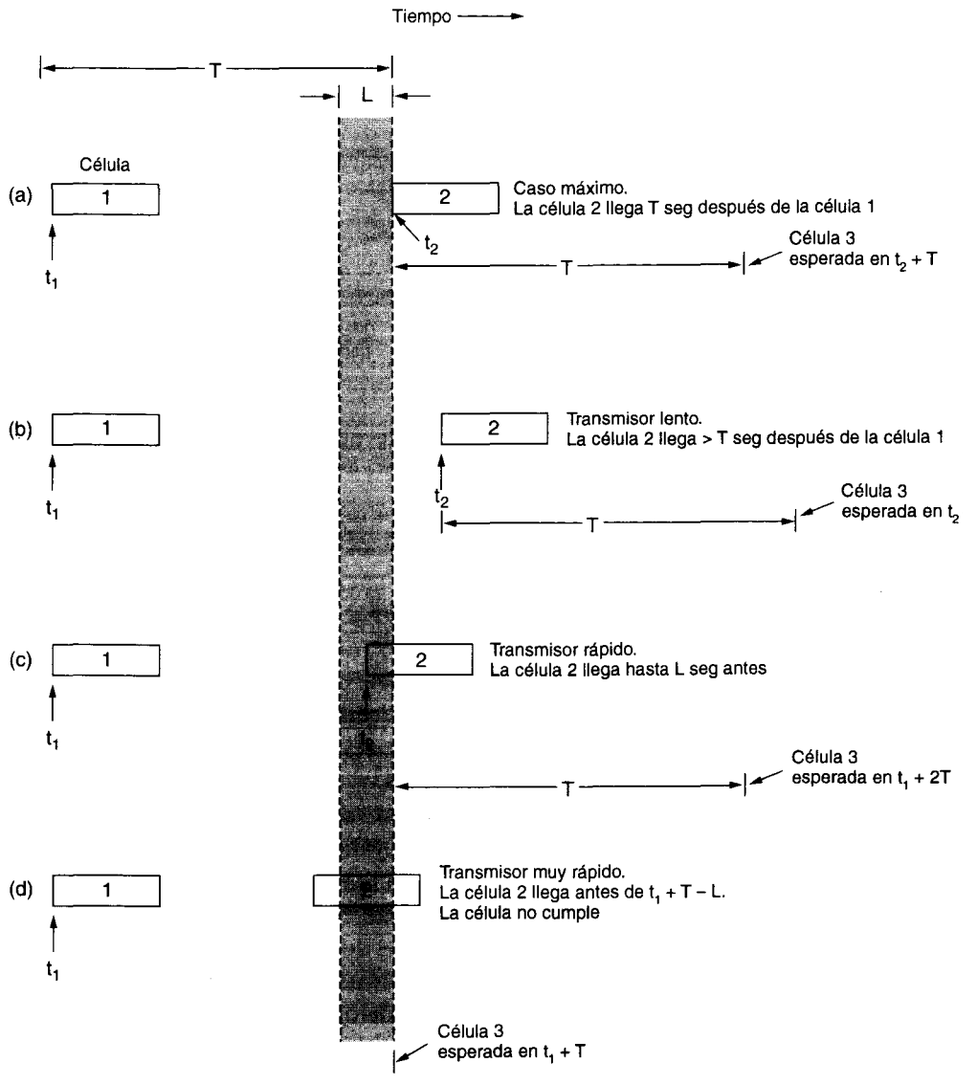


Figura 5-73. El algoritmo genérico de tasa de células.

$T = 1/PCR$, es el tiempo mínimo entre arribo de células, como se muestra en la figura 5-37(a). Si el cliente está de acuerdo en no enviar más de 100,000 células/seg, entonces $T = 10$ μseg. En el caso máximo, llega una célula puntualmente cada 10 μseg. Para evitar los números muy pequeños, trabajaremos con microsegundos, pero dado que todos los parámetros son números reales, la unidad de tiempo no importa.

Siempre se permite a un transmisor espaciar células consecutivas a intervalos mayores que T, como se muestra en la figura 5-73(b). Cualquier célula que llegue más de T μseg después de la previa cumple con lo establecido.

El problema surge cuando los transmisores que tienden a salir en falso, como en la figura 5-73(c) y (d). Si una célula llega un poco antes (a $t_1 + T - L$, o después), cumple, pero se espera la siguiente célula a $t_1 + 2T$, (no a $t_2 + T$), para evitar que el transmisor envíe cada célula L μseg antes, y por tanto aumente la tasa de células pico.

Si llega una célula más de L mseg antes, se declara como incumplidora. El tratamiento de las células incumplidoras es responsabilidad de la portadora. Algunas portadoras simplemente las descartan; otras las guardan, pero establecen el bit *CLP* para marcarlas como de baja prioridad, a fin de permitir a los conmutadores que descarten las células incumplidoras antes en el caso de un congestionamiento. El uso del bit *CLP* también puede ser diferente según las diferentes categorías de servicio de la figura 5-69.

Consideremos ahora lo que ocurre si un transmisor trata de hacer un poco de trampa, como se muestra en la figura 5-74(a). En lugar de esperar hasta el momento T para enviar la célula 2, el

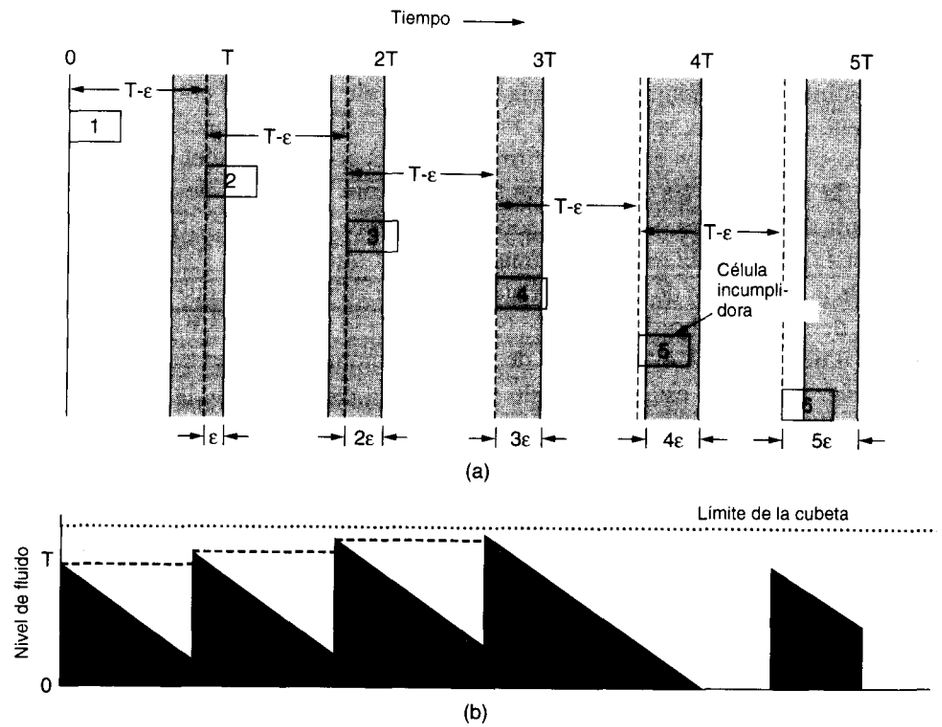


Figura 5-74. (a) Transmisor tratando de hacer trampa. (b) Mismo patrón de llegada de células, pero visto ahora como una cubeta con goteo.

transmisor la envía un poco antes, a $T - \varepsilon$, donde, digamos, $\varepsilon = 0.3L$. Esta célula se acepta sin problemas.

Ahora el transmisor envía la célula 3, nuevamente a $T - \varepsilon$ después de la célula previa, es decir, a $T - 2\varepsilon$. Nuevamente se acepta la célula. Sin embargo, cada célula posterior se acerca más y más al límite fatal $T - L$. En este caso, la célula 5 llega a $T - 4\varepsilon$ ($T - 1.2L$), que es demasiado pronto, por lo que se declara incumplidora y es descartada por la interfaz de la red.

Visto en estos términos, el algoritmo GCRA es un **algoritmo de programación virtual**; sin embargo, visto de otra manera, equivale a un algoritmo de cubeta con goteo, como se observa en la figura 5-74(b). Imagine que cada célula cumplidora que llega vierte T unidades de fluido en una cubeta con goteo. La cubeta gotea el fluido a razón de 1 unidad/mseg, por lo que tras T µseg ha salido todo. Si las células llegan precisamente cada T µseg, cada una que llegue encontrará recién vaciada la cubeta y la rellenará con T unidades de fluido. Por tanto, el nivel de fluido aumenta a T cuando llega una célula, y se reduce linealmente hasta llegar a cero. Esta situación se ilustra en la figura 5-74(b) entre 0 y T.

Dado que el fluido se drena linealmente con el tiempo, en un tiempo t tras la llegada de una célula, la cantidad que queda de su fluido es de $T - t$. En el momento que llega la célula 2, en $T - \varepsilon$, aún hay e unidades de fluido en la cubeta. La llegada de la célula nueva eleva este valor a $T + \varepsilon$. Del mismo modo, en el momento que llega la célula 3, quedan 2ε unidades en la cubeta, por lo que la célula nueva eleva el nivel de fluido a $T + 2\varepsilon$. Al llegar la célula 4, el nivel se eleva a $T + 3\varepsilon$.

Si esto continúa indefinidamente, alguna célula va a elevar el nivel por encima de la capacidad de la cubeta y, por tanto, se rechazará. Para ver de cuál se trata, calculemos ahora la capacidad de la cubeta. Queremos que el algoritmo de cubeta con goteo dé el mismo resultado que se observa en la figura 5-74(a), por lo que queremos que ocurra un desbordamiento cuando llegue una célula L µseg antes. Si el fluido sobrante requiere L µseg para drenarse, la cantidad de fluido debe ser L, dado que la rapidez de drenado es de 1 unidad/µseg. Por tanto, queremos que la capacidad de la cubeta sea de $L + T$ para que cualquier célula que llegue más de L mseg antes sea rechazada debido al desbordamiento de la cubeta. En la figura 5-74(b), al llegar la célula 5, la suma de T unidades a las 4ε unidades de fluido actualmente presentes eleva el nivel de la cubeta a $T + 4\varepsilon$. Dado que estamos usando $\varepsilon = 0.3L$ en este ejemplo, la cubeta se elevaría a $T + 1.2L$ por la llegada de la célula 5, por lo que se rechaza la célula, no se agrega fluido nuevo, y la cubeta en algún momento se vacía.

Para un T dado, si establecemos un L muy pequeño, la capacidad de la cubeta apenas será mayor de T, por lo que todas las células tendrán que enviarse con un espaciado muy uniforme. Sin embargo, si elevamos ahora L a un valor mucho mayor que T, la cubeta puede contener varias células, porque $T + L \gg T$. Esto significa que el transmisor puede transmitir una ráfaga de células una tras otra a la tasa pico y aun lograr su aceptación.

Podemos calcular fácilmente la cantidad de células cumplidoras, N, que pueden transmitirse una tras otra a la tasa de células pico ($PCR = 1/T$). Durante una ráfaga de N células, la cantidad total de fluido agregado a la cubeta es de NT, puesto que cada célula agrega T. Sin embargo, durante la ráfaga máxima, el fluido se drena de la cubeta a una razón de 1 unidad por intervalo de tiempo. Digamos que el tiempo de transmisión de célula es de δ unidades de tiempo. Nótese que

$\delta \leq T$ porque es perfectamente posible que un transmisor en una línea de 155.52 Mbps acuerde enviar no más de 100,000 células/seg, en cuyo caso $\delta = 2.73$ µseg y $T = 10$ µseg. Durante la ráfaga de N células, la cantidad de drenado es de $(N - 1)\delta$ porque el drenado no comienza hasta haberse completado la transmisión de la primera célula.

A partir de estas observaciones, establecemos que el incremento neto del fluido de la cubeta durante la ráfaga máxima es de $NT - (N - 1)\delta$. La capacidad de la cubeta es de $T + L$. Igualando estas dos cantidades obtenemos

$$NT - (N - 1)\delta = T + L$$

Si despejamos N de esta ecuación obtenemos

$$N = 1 + \frac{L}{T - \delta}$$

Sin embargo, si este número no es un entero, debe redondearse al entero menor más cercano para evitar el desbordamiento de la cubeta. Por ejemplo, con $PCR = 100{,}000$ células/seg, $\delta = 2.73$ µseg y $L = 50$ µseg, pueden enviarse siete células una tras otra a razón de 155.52 Mbps sin llenar la cubeta. Una octava célula sería incumplidora.

El GCRA normalmente se especifica dando los parámetros T y L. T simplemente es el recíproco de *PCR*; L es *CDVT*. El GCRA también se usa para asegurar que la tasa media de células no exceda la *SCR* durante ningún periodo sustancial de tiempo.

En este ejemplo, supusimos que las células llegan uniformemente. En la realidad no es así. Sin embargo, también puede usarse aquí el algoritmo de cubeta con goteo. Con cada llegada de célula, se hace una comprobación para ver si hay lugar en la cubeta para T unidades adicionales de fluido. Si lo hay, la célula es cumplidora; de otra manera no.

Además de proporcionar una regla respecto a las células cumplidoras y las que no lo son, el GCRA también estructura el tráfico a modo de eliminar una parte de las ráfagas. A menor *CDVT*, mayor efecto de emparejamiento, pero también mayor oportunidad de que las células se descarten como no cumplidoras. Algunas implementaciones combinan la cubeta con goteo GCRA con una cubeta con fichas para proporcionar mayor emparejamiento.

5.6.7. Control de congestionamientos

Aun con la conformación de tráfico, las redes ATM no cumplen automáticamente los requisitos de desempeño establecidos en el contrato de tráfico. Por ejemplo, el congestionamiento en los conmutadores intermedios siempre es un problema potencial, especialmente cuando circulan más de 350,000 células/seg por cada línea, y un conmutador puede tener 100 líneas. En consecuencia, se ha invertido una buena cantidad de esfuerzo para estudiar el asunto del desempeño y el congestionamiento de las redes ATM. En esta sección veremos algunos de los enfoques usados. Para información adicional, véase (Eckberg, 1992; Eckberg *et al.*, 1991; Hong y Suda, 1991; Jain, 1995, y Newman, 1994).

Las redes ATM deben manejar tanto el congestionamiento a largo plazo, causado por la generación de más tráfico del que puede manejar el sistema, como el congestionamiento a corto

plazo, causado por las ráfagas del tráfico. Por ello, se usan varias estrategias diferentes al mismo tiempo. Las más importantes de éstas pertenecen a tres categorías:

1. Control de admisión.
2. Reservación de recursos.
3. Control de congestionamiento basado en la tasa.

Ahora estudiaremos cada una de estas estrategias.

Control de admisión

En las redes de baja velocidad, generalmente es adecuado esperar a que ocurra un congestionamiento y reaccionar a él indicando al origen de los paquetes que reduzca su velocidad. En las redes de alta velocidad, este enfoque con frecuencia funciona mal, pues en el intervalo entre el envío de la notificación y la llegada de la notificación al origen pueden llegar miles de paquetes adicionales.

Además, muchas redes ATM tienen orígenes de tráfico en tiempo real que producen datos con una tasa intrínseca. Indicar a una de origen que reduzca su velocidad tal vez no funcione (imagine un teléfono digital nuevo con una luz roja: al señalarse un congestionamiento, la luz roja se enciende y se requiere que el que habla lo haga 25% más despacio).

En consecuencia, las redes ATM hacen hincapié en evitar que ocurra el congestionamiento. Sin embargo, para el tráfico CBR, VBR y UBR no hay ningún control de congestionamiento en absoluto, por lo que un poco de prevención vale más que una buena dosis de medicina. Una herramienta principal para evitar el congestionamiento es el control de admisión. Cuando un *host* quiere un circuito virtual nuevo, debe describir el tráfico que va a generar y el servicio esperado. La red puede entonces verificar si es posible manejar esta conexión sin afectar adversamente las conexiones existentes. Tendrán que examinarse varias rutas potenciales para encontrar una que pueda llevar a cabo la tarea. Si no puede localizarse ninguna ruta, se rechaza la llamada.

El rechazo de admisión debe hacerse de manera justa. ¿Es justo que un zángano que cambia continuamente el canal de su televisor pueda dejar fuera a 100 individuos ocupados que tratan de leer su correo electrónico? Si no se aplican controles, una cantidad pequeña de usuarios con gran ancho de banda puede afectar severamente a muchos usuarios con bajo ancho de banda. Para evitar esto, los usuarios deben dividirse en clases con base en el uso. La probabilidad de negación de servicio debe ser aproximadamente igual para todas las clases (posiblemente dando a cada clase su propio grupo de recursos).

Reservación de recursos

Estrechamente relacionada con el control de admisión está la técnica de reservación de recursos por adelantado, generalmente en el momento de establecimiento de la llamada. Dado que el descriptor de tráfico indica la tasa de células pico, la red tiene la posibilidad de reservar suficiente ancho de banda a lo largo de la trayectoria para manejar esa tasa. El ancho de banda puede reservarse haciendo que el mensaje de ESTABLECER reserve ancho de banda a lo largo de cada

línea que atraviesa, asegurándose por supuesto de que el ancho de banda total reservado a través de una línea sea menor que la capacidad de esa línea. Si el mensaje de ESTABLECER llega a una línea llena, debe dar marcha atrás y buscar una trayectoria alterna.

El descriptor de tráfico puede contener no sólo el ancho de banda pico, sino también el ancho de banda medio. Si, por ejemplo, un *host* quiere un ancho de banda pico de 100,000 células/seg, pero un ancho de banda medio de sólo 20,000 células/seg, en principio pueden multiplexarse cinco de tales circuitos en la misma troncal física. El problema es que las cinco conexiones podrían estar ociosas durante media hora, y luego comenzar a trabajar a la tasa pico, causando pérdidas masivas de células. Dado que el tráfico VBR puede multiplexarse estadísticamente, pueden ocurrir problemas con esta categoría de servicio. Se están estudiando posibles soluciones.

Control de congestionamiento basado en la tasa

Con el tráfico CBR y VBR, generalmente no es posible que el transmisor disminuya su velocidad, aun ante un congestionamiento, debido a la naturaleza de tiempo real o tiempo semirreal inherente del origen de información. Con la UBR a nadie le importa; si hay demasiadas células, las sobrantes se descartan.

Sin embargo, con el tráfico ABR es posible y razonable que la red señale a uno o más transmisores y les solicite disminuir su velocidad temporalmente hasta que la red se pueda recuperar. Es del interés de los transmisores el cumplimiento, ya que la red siempre puede castigarlos deshaciéndose de su exceso de células.

La manera de detectar, señalar y controlar el congestionamiento del tráfico ABR fue un tema importante durante el desarrollo del estándar ATM, con argumentos intensos a favor de varias soluciones propuestas. Ahora veremos brevemente algunas de las soluciones rechazadas, antes de examinar el ganador.

En una propuesta, cuando un transmisor deseaba enviar una ráfaga de datos, primero tenía que enviar una célula especial para reservar el ancho de banda necesario. Tras el regreso del acuse de recibo, podía comenzar la ráfaga. Aquí la ventaja es que el congestionamiento nunca ocurre, pues el ancho de banda requerido siempre está ahí cuando se necesita. El foro ATM rechazó esta solución debido al gran retardo potencial antes de poder transmitir.

Una segunda propuesta tenía conmutadores que enviaban de regreso células de estrangulamiento cada vez que comenzaba un congestionamiento. Al recibirse una de tales células, se esperaba que el transmisor redujera a la mitad su tasa de transmisión de células. Se propusieron varios esquemas para volver a elevar la tasa después, al despejarse el congestionamiento. Este esquema se rechazó porque las células de estrangulamiento podrían perderse en el congestionamiento, y porque el esquema parecía injusto para los usuarios pequeños. Por ejemplo, considérese un conmutador que recibe corrientes de 100 Mbps de cada uno de cinco usuarios, y una corriente de 100 kbps de otro usuario. Muchos miembros del comité sintieron que era inadecuado indicar al usuario de 100 kbps que cediera 50 kbps porque él causaba demasiado congestionamiento.

Una tercera propuesta aprovechaba el hecho de que los límites de los paquetes se marcan mediante un bit en la última célula. La idea era descartar células para aliviar el congestionamiento,

pero hacer esto de manera altamente selectiva. El conmutador debía barrer la corriente de células de entrada para encontrar el final de un paquete, y luego descartar todas las células del siguiente paquete. Por supuesto, este paquete podría transmitirse después, pero descartar todas las células *k* de un paquete a fin de cuentas conduce a la retransmisión de un solo paquete, lo que es mucho mejor que descartar *k* células al azar, asunto que puede dar pie a *k* retransmisiones de paquetes. Este esquema se rechazó por consideraciones de justicia, puesto que la siguiente marca de fin de paquete encontrada podría no pertenecer al transmisor causante de la carga extra del conmutador. Además el esquema no necesitaba estandarizarse. Cualquier proveedor de conmutadores estaría en libertad de escoger las células a descartar al ocurrir un congestionamiento.

Tras mucho debate, la batalla se enfocó en dos contendientes, una solución basada en crédito (Kung y Morris, 1995) y una basada en tasa (Bonomi y Fendick, 1995). La solución basada en crédito esencialmente era un protocolo dinámico de ventana corrediza. Requería que cada conmutador mantuviera, por cada circuito virtual, un crédito (de hecho, la cantidad de *buffers* reservados para ese circuito). Mientras cada célula transmitida tuviera un *buffer* esperándola, nunca podría surgir un congestionamiento.

El argumento en contra vino de los proveedores de conmutadores. No querían encargarse de la contabilidad para llevar el registro de los créditos y no querían reservar demasiados *buffers* por adelantado. La cantidad de carga extra y desperdicio requeridos se consideró muy grande, por lo que se adoptó el esquema de control de congestionamiento basado en tasa, que funciona como sigue.

El modelo básico es que, tras cada *k* células de datos, cada transmisor envía una célula especial **RM** (*Resource Management*, **administración de recursos**). Esta célula viaja por la misma trayectoria que las células de datos, pero recibe un tratamiento especial de los conmutadores a lo largo del camino. Al llegar al destino, la célula se examina, actualiza y envía de regreso al transmisor. La trayectoria completa de las células RM se muestra en la figura 5-75.

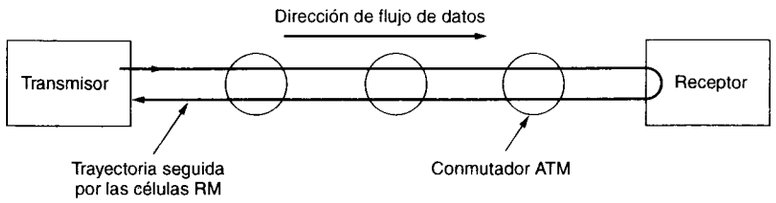


Figura 5-75. Trayectoria seguida por las células RM.

Además, se proporcionan otros dos mecanismos de control de congestionamientos. Primero, los conmutadores carga extrados pueden generar espontáneamente células RM y enviarlas de regreso al transmisor. Segundo, los conmutadores carga extrados pueden establecer el bit *PTI* central de las células de datos que viajan del transmisor al receptor. Sin embargo, ninguno de estos métodos es completamente confiable, pues estas células pueden perderse en el congestionamiento sin que nadie se dé cuenta. En cambio, una célula RM perdida será notada

por el transmisor cuando no regrese en el intervalo de tiempo esperado. Como nota al margen, el bit *CLP* no se usa para el control de congestionamientos.

El control de congestionamientos ABR se basa en la idea de que cada transmisor tiene una tasa actual, la **ACR** (*Actual Cell Rate*, **tasa de células real**), que está entre la *MCR* y la *PCR*. Al ocurrir un congestionamiento, la *ACR* se reduce (pero no por debajo de la *MCR*). Si no hay congestionamientos, la *ACR* se aumenta (pero no por encima de la *PCR*). Cada célula RM enviada contiene la tasa a la cual le gustaría transmitir al transmisor, posiblemente a la *PCR*, posiblemente a una menor. Este valor se llama **ER** (*Explicit Rate*, **tasa explícita**). A medida que la célula RM pasa a través de varios conmutadores en su camino al receptor, aquellos que están congestionados pueden reducir la *ER*. Ningún conmutador la puede aumentar. La reducción puede ocurrir tanto en sentido directo como en el inverso. Cuando el transmisor recibe la célula RM, puede ver la tasa mínima aceptable según todos los conmutadores a lo largo de la trayectoria. Entonces puede ajustar la *ACR*, de ser necesario, de acuerdo con lo que el conmutador más lento puede manejar.

El mecanismo de congestionamiento que usa el bit *PTI* central se integra en las células RM al hacer que el receptor incluya este bit (tomado de la última célula de datos) en cada célula RM enviada de regreso. El bit no puede tomarse de la célula RM misma porque todas las células RM tienen establecido este bit todo el tiempo, como se muestra en la figura 5-63.

La capa ATM es bastante complicada. En este capítulo sólo hemos destacado una parte de los temas. Para información adicional, véase (De Prycker, 1993; McDysan y Spohn, 1995; Minoli y Vitella, 1994, y La Porta *et al.*, 1994). Sin embargo, se advierte al lector que todas estas referencias estudian el estándar ATM 3, no el ATM 4, que no se terminó sino hasta 1996.

5.6.8. LAN ATM

A medida que se hace más obvio que la meta original del ITU de reemplazar la red telefónica pública conmutada por una red ATM se llevará mucho tiempo, la atención se está centrando en el uso de la tecnología ATM para conectar entre sí las LAN existentes. En este enfoque, una red ATM puede funcionar como LAN, conectando *hosts* individuales, o como puente, conectando varias LAN. Aunque ambos conceptos son interesantes, generan algunos problemas complicados que estudiaremos a continuación. Puede encontrarse información adicional acerca de las LAN ATM en (Chao *et al.*, 1994; Newman, 1994, y Truong *et al.*, 1995).

El problema principal que debe resolverse es cómo proporcionar un servicio de LAN sin conexiones a través de una red ATM orientada a conexiones. Una posible solución es introducir un servidor sin conexiones en la red. Cada *host* establece inicialmente una conexión con este servidor, y envía a él todos los paquetes para que los reenvíe. Aunque es sencilla, esta solución no aprovecha todo el ancho de banda de la red ATM, y el servidor sin conexiones fácilmente puede volverse un cuello de botella.

Un enfoque alterno, propuesto por el foro ATM, se muestra en la figura 5-76. Aquí cada *host* tiene circuitos virtuales ATM (potenciales) a todos los demás *hosts*. Estos circuitos virtuales pueden establecerse y liberarse dinámicamente según se necesite, o pueden ser circuitos virtuales permanentes. Para enviar un marco, el *host* de origen encapsula primero el paquete en el campo

de carga útil de un mensaje AAL ATM, y lo envía al destino, de la misma manera en que se transmiten marcos a través de Ethernet, *token ring* y otras LAN.

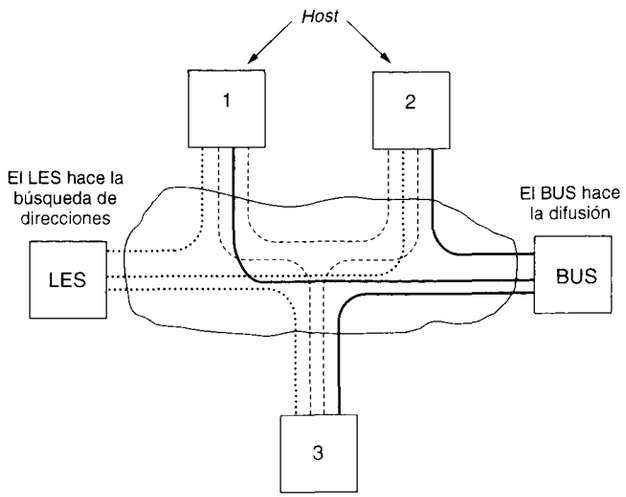


Figura 5-76. Simulación ATM de una LAN.

El problema principal generado por este esquema es cómo determinar los circuitos virtuales a los que pertenecen las direcciones de IP (u otras direcciones de capa de red). En una LAN 802 este problema se resuelve mediante el protocolo ARP, en el que un *host* puede difundir una solicitud como: "¿quién tiene la dirección de IP 192.31.20.47?" El *host* que usa esa dirección devuelve entonces una respuesta punto a punto, que se almacena en caché para su uso futuro.

Con una LAN ATM esta solución no funciona, pues las LAN ATM no manejan la difusión. Este problema se resuelve introduciendo un servidor nuevo, el **LES** (*LAN Emulation Server*, **servidor de simulación de LAN**). Para buscar una dirección de capa de red (por ejemplo, una dirección IP), un *host* envía un paquete (por ejemplo, una solicitud ARP) al LES, que entonces busca la dirección ATM correspondiente y la devuelve a la máquina que la solicita. Esta dirección entonces puede usarse para enviar paquetes encapsulados al destino.

Sin embargo, esta solución sólo resuelve el problema de localización del *host*. Algunos programas usan la difusión o la multitransmisión como parte esencial de la aplicación. Para estas aplicaciones, se introduce el **BUS** (*Broadcast/Unknown Server*, **servidor de difusión/desconocido**), que tiene conexiones a todos los *hosts* y puede simular la difusión enviando un paquete a todos ellos, uno por uno. Los *hosts* también pueden acelerar la entrega de un paquete a un *host* desconocido enviando el paquete al BUS para difundirlo y luego (en paralelo) buscar la dirección (para uso futuro) usando el LES.

Un modelo semejante a éste ha sido adoptado por el IETF como la manera oficial en que Internet usa una red ATM para transportar paquetes IP. En este modelo, el servidor LES se llama

servidor **ATMARP**, pero la funcionalidad es esencialmente la misma. La difusión y multitransmisión no se contemplan en la propuesta IETF. El modelo se describe en el RFC 1483 y el RFC 1577. Otra buena de origen de información es (Comer, 1995).

En el método del IETF puede agruparse un conjunto de *hosts* ATM para formar una **subred IP lógica** (LIS, *logical IP subnet*). Cada LIS tiene su propio servidor ATMARP. En efecto, una LIS actúa como una subred LAN virtual. Los *hosts* de la misma LIS pueden intercambiar directamente paquetes IP, pero los *hosts* de LIS diferentes deben pasar a través de un enrutador. La razón de tener las LIS es que cada *host* de una LIS debe (potencialmente) tener circuitos virtuales abiertos hacia todos los demás *hosts* de su LIS. Al restringir la cantidad de *hosts* por LIS, se puede reducir la cantidad de circuitos virtuales abiertos a un número manejable.

Otro uso de las redes ATM es como puentes para conectar varias LAN existentes. En esta configuración, sólo una máquina de cada LAN necesita una conexión ATM. Al igual que todos los puentes transparentes, el puente ATM debe escuchar en modo promiscuo a todas las LAN a las que está conectado, reenviando los marcos a donde se necesiten. Dado que los puentes sólo usan direcciones de MAC (no de IP), los puentes ATM deben construir un árbol de extensión, al igual que los puentes 802.

En resumen, si bien la simulación LAN del ATM es un concepto interesante, hay cuestionamientos serios sobre su desempeño y precio, y ciertamente enfrentan una fuerte competencia de las LAN y los puentes existentes, bien establecidos y optimizados. El que las LAN ATM reemplacen en algún momento a las LAN y puentes 802 está por verse.

5.7. RESUMEN

La capa de red proporciona servicios a la capa de transporte; puede basarse tanto en circuitos virtuales como en datagramas. En ambos casos, la tarea principal de esta capa es enrutar paquetes del origen al destino. En las subredes de circuitos virtuales se toma una decisión de enrutamiento al establecerse el circuito virtual; en las subredes de datagramas, se hace con cada paquete.

Se usan muchos algoritmos de enrutamiento en las redes de computadoras. Los algoritmos estáticos incluyen el enrutamiento por trayectoria más corta, la inundación y el enrutamiento basado en flujo. Los algoritmos dinámicos incluyen el enrutamiento por vector de distancia y el enrutamiento por estado de enlace. La mayoría de las redes usan alguno de éstos. Otros temas importantes relativos al enrutamiento son el enrutamiento jerárquico, el enrutamiento para *hosts* móviles, el enrutamiento por difusión y el enrutamiento por multitransmisión.

Las subredes pueden congestionarse, aumentando el retardo y reduciendo el rendimiento de los paquetes. Los diseñadores de redes intentan evitar los congestionamientos mediante un diseño adecuado. Las técnicas incluyen conformación de tráfico, especificaciones de flujo y reservación de ancho de banda. Si ocurre un congestionamiento, habrá que encargarse de él. Pueden enviarse paquetes de estrangulamiento de regreso, deshecharse parte de la carga, y aplicarse otros métodos.

Las redes difieren de varias maneras, por lo que cuando se conectan redes múltiples pueden ocurrir problemas. A veces los problemas pueden superarse enviando los paquetes en túnel a

través de una red hostil, pero si las redes de origen y destino son diferentes, este enfoque falla. Cuando las diferentes redes tienen diferentes tamaños máximos de paquete, puede requerirse una fragmentación.

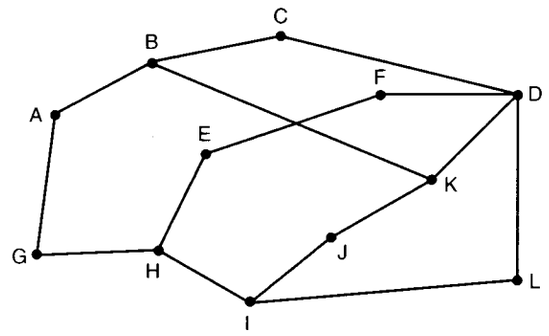
La Internet posee una copiosa variedad de protocolos relacionados con la capa de red. Éstos incluyen el protocolo de capa de transporte de datos, IP, pero también los protocolos de control ICMP, ARP y RARP, y los protocolos de enrutamiento OSPF y BGP. La Internet se está quedando sin direcciones de IP, por lo que se ha desarrollado una versión nueva del IP, el IPv6.

A diferencia de la Internet basada en datagramas, las redes ATM usan circuitos virtuales. Éstos deben establecerse antes de poder transmitir, y liberarse después de la transmisión. La calidad del servicio y el control de congestionamientos son asuntos de gran importancia en las redes ATM.

PROBLEMAS

1. Indique dos aplicaciones de ejemplo para las cuales es adecuado un servicio orientado a conexiones. Luego dé dos ejemplos en los que el servicio sin conexiones es lo mejor.
2. ¿Hay circunstancias en las cuales un servicio de circuito virtual entregará (o cuando menos debería entregar) paquetes en desorden? Explique.
3. Las subredes de datagramas enrutan cada paquete como unidad separada, independiente de las demás. Las subredes de circuitos virtuales no tienen que hacer esto, ya que cada paquete de datos sigue una ruta predeterminada. ¿Significa esto que las subredes de circuitos virtuales no necesitan la capacidad de enrutar paquetes aislados de un origen arbitrario a un destino arbitrario? Explique su respuesta.
4. Dé tres ejemplos de parámetros de protocolo que podrían negociarse al establecer una conexión.
5. Considere el siguiente problema de diseño que concierne la puesta en práctica del servicio de circuitos virtuales. Si los circuitos virtuales son internos a la subred, cada paquete de datos debe tener una cabecera de 3 bytes, y cada enrutador debe destinar hasta 8 bytes de almacenamiento para la identificación de circuitos. Si se usan internamente datagramas, se requieren cabeceras de 15 bytes, pero no se requiere espacio de tabla en los enrutadores. La capacidad de transmisión cuesta 1 centavo por cada 10^6 bytes, por salto. Puede comprarse memoria de enrutamiento por 1 centavo por byte y se deprecia en dos años (sólo horas de trabajo). Estadísticamente, la sesión promedio dura 1000 seg, tiempo durante el cual se transmiten 200 paquetes. El paquete medio requiere cuatro saltos. ¿Qué implementación es más económica, y por cuánto?
6. Suponiendo que todos los enrutadores y *hosts* están trabajando de manera adecuada y que el *software* de ambos está libre de errores, ¿hay alguna posibilidad, por pequeña que sea, de que un paquete sea entregado al destino equivocado?
7. Dé una heurística sencilla para encontrar dos trayectorias a través de una red de un origen dado a un destino dado que pueda sobrevivir a la pérdida de cualquier línea de comunicación (suponiendo que existen dos de tales trayectorias). Los enrutadores se consideran lo bastante confiables, por lo que no es necesario preocuparse por la posibilidad de caída de los enrutadores.

8. Considere la subred de la figura 5-15(a). Se usa enrutamiento por vector de distancia y acaban de llegar los siguientes vectores al enrutador *C*: de *B*: (5, 0, 8, 12, 6, 2); de *D*: (16, 12, 6, 0, 9, 10), y de *E*: (7, 6, 3, 9, 0, 4). Los retardos medidos a *B*, *D* y *E* son 6, 3 y 5, respectivamente. ¿Cuál es la nueva tabla de enrutamiento de *C*? Indique tanto la línea de salida a usar como el retardo esperado.
9. Si se registran los retardos como números de 8 bits en una red de 50 enrutadores, y se intercambian vectores de retardo dos veces por segundo, ¿qué ancho de banda por línea dúplex integral es consumido por el algoritmo de enrutamiento distribuido? Suponga que cada enrutador tiene tres líneas a los demás enrutadores.
10. En la figura 5-16 el OR booleano de los dos grupos de bits ACF es de 111 en cada columna. ¿Es éste un mero accidente, o es cierto en todas las circunstancias?
11. Con enrutamiento jerárquico de 4800 enrutadores, ¿qué tamaños de región y de *cluster* deben escogerse para reducir al mínimo el tamaño de la tabla de enrutamiento de una jerarquía de tres capas?
12. En el texto se indicó que, cuando un *host* móvil no está en casa, los paquetes enviados a su LAN base son interceptados por su agente de base. En una red IP de una LAN 802.3, ¿cómo logra esta intercepción el agente de base?
13. Viendo la subred de la figura 5-5, ¿cuántos paquetes se generan por una difusión de *B*, usando
    (a) reenvío por trayectoria invertida?
    (b) árbol de descenso?
14. Calcule un árbol de extensión multitransmisión para el enrutador *C* de la siguiente subred para un grupo con miembros en los enrutadores *A*, *B*, *C*, *D*, *E*, *F*, *I* y *K*.



15. Como posible mecanismo de control de congestionamientos en una subred que usa circuitos virtuales internamente, un enrutador podría abstenerse de reconocer un paquete recibido hasta que (1) sabe que su última transmisión por el circuito virtual se recibió con éxito y que (2) tiene un *buffer* libre. Por sencillez, supóngase que los enrutadores usan un protocolo de parada y espera y que cada circuito virtual tiene un *buffer* dedicado a él para cada sentido del tráfico. Si se requieren *T* seg para transmitir un paquete (de datos o de acuse de recibo) y hay *n* enrutadores en la trayectoria, ¿cuál es la velocidad con que se entregan paquetes al *host* de destino? Supóngase que los errores de transmisión son poco frecuentes y que la conexión *host*-enrutador es infinitamente rápida.

16. Una subred de datagramas permite que los enrutadores puedan deshacerse de paquetes cuando lo necesiten. La probabilidad de que un enrutador descarte un paquete es de p. Considere el caso de un *host* de origen conectado al enrutador de origen, que está conectado al enrutador de destino, y por él al *host* de destino. Si cualquiera de los enrutadores descarta un paquete, el *host* de origen tarde o temprano termina la temporización e intenta de nuevo. Si tanto las líneas *host*-enrutador como enrutador-enrutador se cuentan como saltos, ¿cuál es la media de

(a) saltos que hace un paquete por transmisión?
(b) transmisiones que hace un paquete?
(c) saltos requeridos por paquete recibido?

17. Cite una razón por la que el algoritmo de cubeta con goteo debe tener sólo un paquete por intervalo, independientemente del tamaño del paquete.

18. La variante de conteo de bytes del algoritmo de cubeta con goteo se usa en cierto sistema. La regla es que pueden enviarse por intervalo un paquete de 1024 bytes, dos paquetes de 512 bytes, etc. Indique una restricción seria de este sistema que no se menciona en el texto.

19. Una red ATM usa un esquema de cubeta con fichas para la conformación de tráfico. Se pone una ficha nueva en la cubeta cada 5 µseg. ¿Cuál es la tasa de datos neta máxima sustentable (excluyendo los bits de cabecera)?

20. Una computadora de una red de 6 Mbps se regula mediante una cubeta con fichas. La cubeta con fichas se llena a razón de 1 Mbps. Inicialmente está llena a su capacidad máxima de 8 megabits. ¿Durante cuánto tiempo puede la computadora transmitir a 6 Mbps?

21. En la figura 5-27 se muestran cuatro características de entrada para una especificación de flujo propuesta. Imagine que el tamaño máximo de paquete es de 1000 bytes, la velocidad de la cubeta con fichas es de 10 millones de bytes/seg, el tamaño de la cubeta con fichas es de 1 millón de bytes y la tasa máxima de transmisión es de 50 millones de bytes/seg. ¿Cuánto tiempo puede durar una ráfaga de máxima velocidad?

22. Un dispositivo acepta marcos del Ethernet al que está conectado; remueve el paquete que hay en cada marco, le agrega información sobre el enmarcado y lo transmite por una línea telefónica arrendada (su única conexión con el mundo exterior) a un dispositivo idéntico del otro lado. Este dispositivo retira el enmarcado, inserta el paquete en un marco *token ring*, y lo transmite a un *host* local a través de una LAN *token ring*. ¿Cómo llamaría usted a este dispositivo?

23. ¿Se requiere la fragmentación en interredes de circuitos virtuales concatenados, o sólo en los sistemas de datagramas?

24. El proceso en túnel a través de una red de circuitos virtuales concatenados es directo: el enrutador multiprotocolo de un extremo simplemente establece un circuito virtual con el otro extremo y pasa paquetes a través de él. ¿Puede usarse también el proceso en túnel en las subredes de datagramas? De ser así, ¿cómo?

25. Un datagrama IP que usa la opción de *enrutamiento estricto desde el origen* tiene que fragmentarse. ¿Piensa que la opción se copia en cada fragmento, o basta con simplemente ponerlo en el primer fragmento? Explique su respuesta.

26. Supóngase que, en lugar de usar 16 bits para la parte de red de una dirección clase B, se hubieran usado 20 bits. ¿Cuántas redes clase B habría?

27. Convierta la dirección de IP cuya representación hexadecimal es C22F1582 a notación decimal con puntos.

28. Una red clase B de Internet tiene una máscara de subred de 255.255.240.0. ¿Cuál es la cantidad máxima de *hosts* por subred?

29. Usted acaba de explicarle el protocolo ARP a un amigo. Al terminar, él dice: "Ya entiendo. El ARP proporciona un servicio a la capa de red, por lo que es parte de la capa de enlace de datos". ¿Qué le contesta usted?

30. El ARP y el RARP proyectan direcciones de un espacio al otro; en este sentido, son parecidos. Sin embargo, las implementaciones de estos protocolos son fundamentalmente diferentes. ¿En qué difieren principalmente?

31. Describa una manera de reensamblar fragmentos de IP en el destino.

32. La mayoría de los algoritmos de reensamble de datagramas IP tiene un temporizador para evitar que un fragmento perdido bloquee los *buffers* de reensamble por siempre. Suponga que un datagrama se divide en cuatro fragmentos. Llegan los primeros tres fragmentos, pero el último se retarda. Tarde o temprano termina el temporizador y los tres fragmentos en la memoria del receptor se descartan. Un poco después llega el último fragmento. ¿Qué debe hacerse con él?

33. La mayoría de los protocolos usa el número de saltos como la métrica que debe reducirse al mínimo al hacer cálculos de enrutamiento. En las redes ATM, la cantidad de saltos no es demasiado importante. ¿Por qué no? *Sugerencia:* repase el capítulo 2, para ver la manera en que funcionan los conmutadores ATM. ¿Usan almacenamiento y reenvío?

34. Tanto en IP como en ATM, la suma de comprobación cubre sólo la cabecera y no los datos. ¿Por qué supone que se escogió este diseño?

35. Una persona que vive en Boston viaja a Minneápolis, llevando consigo su computadora portátil. Para su sorpresa, la LAN de su destino en Minneápolis es una LAN inalámbrica IP, por lo que no tiene que conectarse. ¿Aún es necesario pasar por el sistema completo de agentes de base y agentes foráneos para que lleguen correctamente el correo electrónico y otros tipos de tráfico?

36. El IPv6 usa direcciones de 16 bytes. Si se asigna un bloque de 1 millón de direcciones cada picosegundo, ¿cuánto durarán las direcciones?

37. El campo de *protocolo* usado en la cabecera IPv4 no existe en la cabecera fija IPv6. ¿Por qué no?

38. Al introducirse el protocolo IPv6, ¿tiene que cambiarse el protocolo ARP? De ser así, ¿cuáles son los cambios conceptuales o técnicos?

39. En el capítulo 1 clasificamos las interacciones entre la red y los *hosts* usando cuatro clases de primitivas: *solicitud, indicación, respuesta* y *confirmación*. Clasifique los mensajes ESTABLECER y CONEXIÓN de la figura 5-65 según estas categorías.

40. Se está estableciendo un circuito virtual en una red ATM. Entre los *hosts* de origen y destino hay tres conmutadores ATM. ¿Cuántos mensajes (incluidos acuses de recibo) se enviarán para establecer el circuito?

41. La lógica usada para construir la tabla de la figura 5-67 es sencilla: siempre se asigna el *VPI* más bajo no utilizado a una conexión. Si se solicita un circuito virtual nuevo entre Nueva York y Denver, ¿qué *VPI* se le asignará?

42. En la figura 5-73(c), si una célula llega demasiado pronto, la siguiente aún debe llegar en $t_1 + 2T$. Supóngase que la regla fuera diferente; es decir, que la siguiente célula se espera en $t_2 + T$ y que el transmisor aprovechó al máximo esta regla. ¿Qué tasa máxima de células podría lograrse entonces? Para $T = 10$ µseg y $L = 2$ µseg, indique la tasa máxima de células original y nueva, respectivamente.

43. ¿Cuál es la longitud máxima de ráfaga en una conexión ATM ABR de 155.52 Mbps cuyo valor *PCR* es de 200,000 y cuyo valor L es de 25 µseg?

44. Escriba un programa para simular enrutamiento usando inundación. Cada paquete debe contener un contador que disminuye con cada salto. Al llegar el contador a cero, el paquete se descarta. El tiempo es discreto, manejando cada línea un paquete por intervalo de tiempo. Haga tres versiones del programa: todas las líneas inundadas, todas las líneas inundadas, excepto la línea de entrada, y sólo las mejores k líneas (seleccionadas estadísticamente) inundadas. Compare la inundación con el enrutamiento determinista ($k = 1$) en términos de retardo y ancho de banda usado.

45. Escriba un programa que simule una red de cómputo que usa tiempo discreto. El primer paquete de cada cola de enrutador hace un salto por intervalo de tiempo. Cada enrutador tiene una cantidad finita de *buffers*. Si llega un paquete y no hay lugar para él, se descarta y no se retransmite. En cambio, hay un protocolo terminal a terminal, completo con temporizadores y paquetes de reconocimiento, que tarde o temprano regenera el paquete del enrutador de origen. Dibuje en un grafo el rendimiento de la red en función del intervalo de terminación de temporización, parametrizado por tasa de errores.

6 LA CAPA DE TRANSPORTE

UNIVERSIDAD DE L
FACULTAD DE I
DEPARTAMEN
DOCUMENTACION Y
MONTEVIDEO - U

La capa de transporte no es sólo otra capa. Es el corazón de la jerarquía completa de protocolos. La tarea de esta capa es proporcionar un transporte de datos confiable y económico de la máquina de origen a la máquina de destino, independientemente de la red o redes físicas en uso. Sin la capa de transporte, el concepto total de los protocolos en capas tendría poco sentido. En este capítulo estudiaremos en detalle la capa de transporte, incluidos sus servicios, diseño, protocolos y desempeño.

6.1. EL SERVICIO DE TRANSPORTE

En las siguientes secciones haremos una introducción al servicio de transporte. Veremos el tipo de servicio proporcionado a la capa de aplicación (o de sesión, si existe) y especialmente la manera en que puede caracterizarse la calidad del servicio. Después veremos cómo las aplicaciones acceden al servicio de transporte, es decir, cómo es la interfaz.

6.1.1. Servicios proporcionados a las capas superiores

La meta fundamental de la capa de transporte es proporcionar un servicio eficiente, confiable y económico a sus usuarios, que normalmente son procesos de la capa de aplicación. Para lograr este objetivo, la capa de transporte hace uso de los servicios proporcionados por la capa de red.

El *hardware* o el *software* de la capa de transporte que se encarga del trabajo se llama **entidad de transporte**. La entidad de transporte puede estar en el núcleo (*kernel*) del sistema operativo, en un proceso de usuario independiente, en un paquete de biblioteca que forma parte de las aplicaciones de la red o en la tarjeta de interfaz de la red. En algunos casos, la portadora puede proporcionar servicio de transporte confiable, en cuyo caso la entidad de transporte reside en máquinas especiales de interfaz en la orilla de la subred a la que se conectan los *hosts*. La relación (lógica) entre las capas de red, transporte y aplicación se ilustra en la figura 6-1.

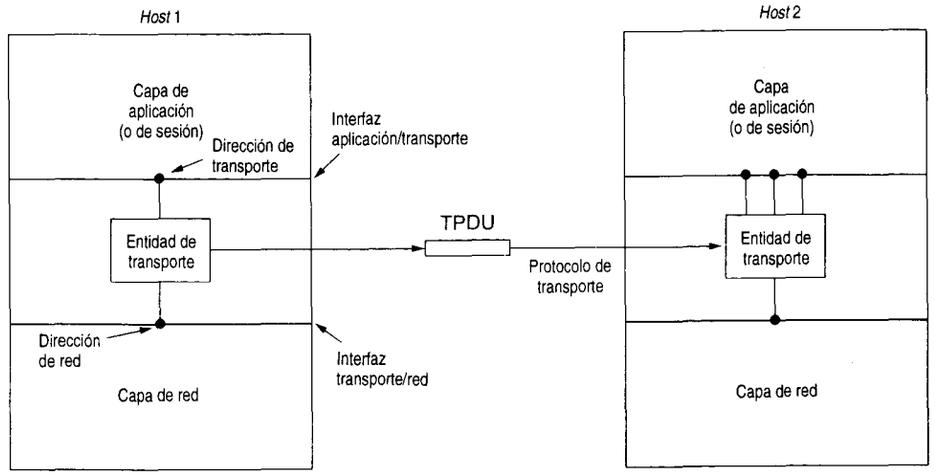


Figura 6-1. Las capas de red, transporte y aplicación.

Así como hay dos tipos de servicio de red, orientado a conexiones y sin conexiones, también hay dos tipos de servicio de transporte. El servicio de transporte orientado a conexiones es parecido al servicio de red orientado a conexiones en muchos sentidos. En ambos casos, las conexiones tienen tres fases: establecimiento, transferencia de datos y liberación. El direccionamiento y el control de flujo también son semejantes en ambas capas. Además, el servicio de transporte sin conexiones es muy parecido al servicio de red sin conexiones.

La pregunta obvia es entonces: ¿si el servicio de la capa de transporte es tan parecido al de la capa de red, por qué hay dos capas diferentes? ¿Por qué no es suficiente una sola capa? La respuesta es sutil, pero crucial, y nos remite a la figura 1-16. En esta figura podemos ver que la capa de red es parte de la subred de comunicación y es operada por la portadora (cuando menos en las WAN). ¿Qué sucede si la capa de red ofrece un servicio orientado a conexiones pero no es confiable? ¿Qué tal si esa capa pierde paquetes con frecuencia? ¿Qué ocurre si los enrutadores se vienen abajo de cuando en cuando?

Problemas, eso es lo que ocurre. Los usuarios no tienen control sobre la subred, por lo que no pueden resolver los problemas de un mal servicio usando mejores enrutadores o incluyendo

una porción mayor del manejo de errores en la capa de enlace de datos. La única posibilidad es poner otra capa encima de la capa de red que mejore la calidad del servicio. Si a la mitad de una transmisión larga se informa a una entidad de transporte que su conexión de red ha sido terminada abruptamente, sin indicación de lo sucedido a los datos actualmente en tránsito, la entidad puede establecer una nueva conexión de red con la entidad de transporte remota. Usando esta nueva conexión de red, la entidad puede enviar una solicitud a su igual preguntando cuáles datos llegaron y cuáles no, y reiniciar desde la interrupción.

Esencialmente, la existencia de la capa de transporte hace posible que el servicio de transporte sea más confiable que el servicio de red subyacente. La capa de transporte puede detectar y compensar paquetes perdidos y datos alterados. Es más, las primitivas del servicio de transporte pueden diseñarse de modo que sean independientes de las primitivas del servicio de red, que pueden variar considerablemente entre las redes (por ejemplo, el servicio LAN sin conexiones puede ser bastante diferente del servicio WAN orientado a conexiones).

Gracias a la capa de transporte, es posible escribir programas de aplicación usando un grupo estándar de primitivas, y hacer que estos programas trabajen en una variedad amplia de redes, sin tener que preocuparse de lidiar con interfaces de subred diferentes y transmisiones no confiables. Si ninguna red real tuviera fallas, y si todas tuvieran las mismas primitivas de servicio, la capa de transporte probablemente sería innecesaria. Sin embargo, en el mundo real esta capa cumple la función clave de aislar las capas superiores respecto de la tecnología, el diseño y las imperfecciones de la subred.

Por esta razón, mucha gente distingue entre las capas 1 a 4, por una parte, y la(s) capa(s) por encima de la 4, por la otra. Las cuatro capas inferiores pueden verse como el **proveedor del servicio de transporte**, y la(s) capa(s) superiores son el **usuario del servicio de transporte**. Esta distinción entre proveedor y usuario tiene un impacto considerable en el diseño de las capas y pone a la capa de transporte en una posición clave, ya que forma el límite principal entre el proveedor y el usuario del servicio confiable de transmisión de datos.

6.1.2. Calidad del servicio

Otra manera de ver la capa de transporte es considerar que su función primaria es mejorar la **QoS** (*Quality of Service*, **calidad del servicio**) proporcionada por la capa de red. Si el servicio de red es impecable, la capa de transporte tiene una tarea fácil. Si, por otra parte, el servicio de red es malo, la capa de transporte tiene que construir un puente sobre el abismo que separa lo que quieren los usuarios de transporte y lo que la capa de red proporciona.

Aunque a primera vista la calidad del servicio podría parecer un concepto vago (lograr que todos se pongan de acuerdo en lo que constituye un "buen" servicio es un ejercicio nada trivial), la QoS puede caracterizarse por varios parámetros específicos, como vimos en el capítulo 5. El servicio de transporte puede permitir que el usuario especifique valores preferidos, aceptables y mínimos para varios parámetros de servicio en el momento de establecerse una conexión. Algunos de los parámetros también se aplican al transporte sin conexiones. Es responsabilidad de la capa de transporte examinar estos parámetros y, dependiendo de los tipos de servicio de red disponibles, determinar si puede proporcionar el servicio requerido. En el resto de esta sección

estudiaremos algunos parámetros QoS, que se presentan resumidos en la figura 6-2. Nótese que pocas redes y protocolos proporcionan todos estos parámetros. Muchos simplemente hacen su mejor esfuerzo para reducir la tasa de errores residual, y hasta ahí. Otros tienen arquitecturas de QoS complejas (Campbell *et al.*, 1994).

Retardo de establecimiento de conexión
Probabilidad de falla de establecimiento de conexión
Rendimiento
Retardo de tránsito
Tasa de errores residual
Protección
Prioridad
Tenacidad

Figura 6-2. Parámetros de calidad de servicio típicos de la capa de transporte.

El *retardo de establecimiento de la conexión* es el tiempo que transcurre entre la solicitud de una conexión de transporte y la confirmación del usuario del servicio de transporte; incluye el retardo de proceso de la entidad de transporte remota. Al igual que con todos los parámetros que miden un retardo, a menor retardo, mejor servicio.

La *probabilidad de falla de establecimiento de la conexión* es la posibilidad de que una conexión no se establezca en el lapso de máximo retardo de establecimiento; por ejemplo, debido a congestionamiento de la red, falta de espacio de tablas en algún lado u otros problemas internos.

El parámetro de *rendimiento* mide la cantidad de bytes de datos de usuario transferidos por segundo, medido durante algún intervalo de tiempo. El rendimiento se mide por separado en cada sentido.

El *retardo de tránsito* mide el tiempo entre el envío de un mensaje por el usuario de transporte de la máquina de origen y su recepción por el usuario de transporte de la máquina de destino. Al igual que con el rendimiento, cada sentido se maneja por separado.

La *tasa de error residual* mide la cantidad de mensajes perdidos o alterados como una fracción del total enviado. En teoría, la tasa de error residual debe ser cero, ya que es responsabilidad de la capa de transporte ocultar todos los errores de la capa de red. En la práctica, este parámetro puede tener algún valor finito pequeño.

El parámetro de *protección* provee un mecanismo para que el usuario de transporte especifique su interés en que la capa de transporte proporcione protección de lectura y modificación de los datos transmitidos contra terceros no autorizados (interceptación).

El parámetro de *prioridad* proporciona un mecanismo para que el usuario indique que algunas de sus conexiones son más importantes que otras, y que, en el caso de un congestionamiento, se asegure de que las conexiones de alta prioridad sean atendidas antes que las de baja prioridad.

Por último, el parámetro de *tenacidad* da la probabilidad de que la capa de transporte por sí misma termine instantáneamente una conexión debido a problemas internos o congestionamiento.

Los parámetros de QoS los especifica el usuario de transporte cuando se solicita una conexión. Pueden darse tanto el valor deseado como el mínimo. En algunos casos, al ver los parámetros de QoS, la capa de transporte puede darse cuenta de inmediato de que algunos de ellos son inalcanzables, en cuyo caso indica al solicitante que falló el intento de conexión, sin molestarse en comunicarse con el destino. El informe de fallas especifica la razón de la falla.

En otros casos, la capa de transporte sabe que no puede lograr la meta deseada (por ejemplo, rendimiento de 600 Mbps), pero puede lograr una tasa menor, aunque aún aceptable (por ejemplo, 150 Mbps). Entonces envía la tasa menor y la tasa mínima aceptable a la máquina remota, solicitando establecer una conexión. Si la máquina remota no puede manejar el valor propuesto, pero puede manejar un valor por encima del mínimo, puede hacer una contraoferta. Si no puede manejar ningún valor superior al mínimo, rechaza el intento de conexión. Por último, se informa al usuario de transporte originador si se estableció o se rechazó la conexión y, si se estableció, los valores de los parámetros acordados.

Este proceso se llama **negociación abierta**. Una vez que se han negociado las opciones, permanecen así durante el resto de la vida de la conexión. Para evitar que los clientes se vuelvan demasiado codiciosos, muchas portadoras suelen cobrar más por una mejor calidad de servicio.

6.1.3. Primitivas de servicio de transporte

Las primitivas del servicio de transporte permiten a los usuarios de transporte (por ejemplo, programas de aplicación) el acceso al servicio de transporte. Cada servicio tiene sus propias primitivas de acceso. En esta sección examinaremos primero un servicio de transporte sencillo e hipotético y luego veremos un ejemplo real.

El servicio de transporte es parecido al servicio de red, pero hay algunas diferencias importantes. La principal es que la intención del servicio de red es modelar el servicio ofrecido por las redes reales, con todo y verrugas. Las redes reales pueden perder paquetes, por lo que el servicio de red generalmente no es confiable.

El servicio de transporte orientado a conexiones, en cambio, sí es confiable. Claro que las redes reales no están libres de errores, pero ése es precisamente el propósito de la capa de transporte: proporcionar un servicio confiable por una red no confiable.

Como ejemplo, considere dos procesos conectados mediante tubos en UNIX. Ambos consideran que la conexión entre ellas es perfecta. No quieren saber de acuses de recibo, paquetes perdidos, congestionamientos ni nada por el estilo. Lo que quieren es una conexión 100% confiable. El proceso *A* pone datos en un extremo del tubo y el proceso *B* lo saca por el otro. De esto se trata el servicio de transporte orientado a conexiones: de esconder las imperfecciones del

Primitiva	TPDU enviada	Significado
LISTEN (ESCUCHAR)	(ninguna)	Se bloquea hasta que algún proceso intenta el contacto
CONNECT (CONECTAR)	SOLICITUD DE CONEXIÓN	Intenta activamente establecer una conexión
SEND (ENVIAR)	DATOS	Envía información
RECEIVE (RECIBIR)	(ninguna)	Se bloquea hasta que llega una TPDU de DATOS
DISCONNECT (DESCONECTAR)	SOLICITUD DE DESCONEXIÓN	Este lado quiere liberar la conexión

Figura 6-3. Primitivas de un servicio de transporte sencillo.

servicio de red para que los procesos del usuario simplemente puedan suponer la existencia de una corriente de bits libre de errores.

Como nota al margen, la capa de transporte puede proporcionar también un servicio no confiable (de datagramas), pero hay relativamente poco que decir al respecto, por lo que en este capítulo nos concentraremos en el servicio de transporte orientado a conexiones.

Una segunda diferencia entre el servicio de red y el de transporte es a quién están dirigidos los servicios. El servicio de red lo usan sólo las entidades de transporte. Pocos usuarios escriben sus propias entidades de transporte y, por tanto, pocos usuarios o programas llegan a ver internamente el servicio de red. En contraste, muchos programas (y por tanto programadores) ven las primitivas de transporte. En consecuencia, el servicio de transporte debe ser cómodo y fácil de usar.

Para tener una idea de lo que podría ser el servicio de transporte, considere las cinco primitivas listadas en la figura 6-3. Esta interfaz de transporte ciertamente es sencilla, pero da la esencia de lo que debe hacer una interfaz de transporte orientada a conexiones: permite que los programas de aplicación establezcan, usen y liberen conexiones, lo que basta para muchas aplicaciones.

Para ver cómo podrían usarse estas primitivas, considere una aplicación con un servidor y cierta cantidad de clientes remotos. Para comenzar, el servidor ejecuta una primitiva LISTEN (escuchar), normalmente llamando a un procedimiento de biblioteca que hace una llamada de sistema para bloquear al servidor hasta la aparición de un cliente. Cuando un cliente quiere hablar con el servidor, ejecuta una primitiva CONNECT (conectarse). La entidad de transporte ejecuta esta primitiva bloqueando al que llama y enviando un paquete al servidor. Encapsulado en la carga útil de este paquete hay un mensaje de capa de transporte para la entidad de transporte del servidor.

Es pertinente aquí una nota rápida sobre la terminología. A falta de un mejor término, usaremos bajo protesta las siglas poco elegantes de **TPDU** (*Transport Protocol Data Unit*, **unidad de datos del protocolo de transporte**) para referirnos a los mensajes enviados de una entidad de transporte a otra. Por tanto, las TPDU (intercambiadas por la capa de transporte) están contenidas en paquetes (intercambiados por la capa de red). A su vez, los paquetes están contenidos en marcos (intercambiados por la capa de enlace de datos). Al llegar un marco, la capa de enlace de datos procesa la cabecera del marco y pasa el contenido del campo de carga útil del

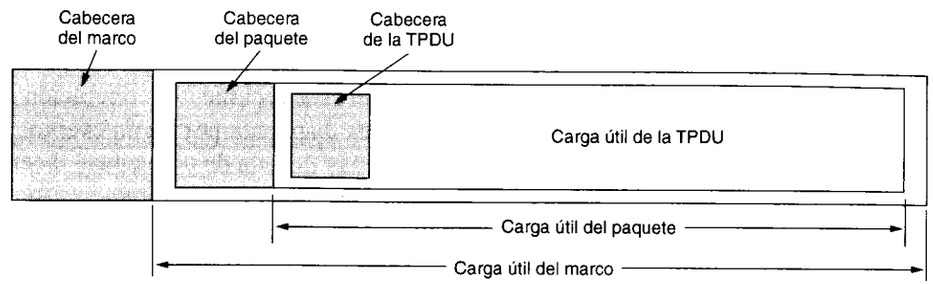


Figura 6-4. Anidamiento de las TPDU, los paquetes y los marcos.

marco a la entidad de red. La entidad de red procesa la cabecera del paquete y pasa el contenido de la carga útil del paquete a la entidad de transporte. Este anidamiento se ilustra en la figura 6-4.

Regresando a nuestro ejemplo de cliente-servidor, la llamada CONNECT del cliente causa el envío de una TPDU CONNECTION REQUEST (solicitud de conexión) al servidor. Al llegar ésta, la entidad de transporte la revisa para comprobar que el servidor está bloqueado en LISTEN (es decir, está interesado en manejar solicitudes). Entonces desbloquea el servidor y envía una TPDU CONNECTION ACCEPTED (conexión aceptada) de regreso al cliente. Al llegar esta TPDU, el cliente se desbloquea y se establece la conexión.

Ahora pueden intercambiarse datos usando las primitivas SEND (enviar) y RECEIVE (recibir). En la forma más simple, cualquiera de las dos partes puede emitir una RECEIVE (bloqueadora) para esperar que la otra parte emita una SEND. Al llegar la TPDU, el receptor se desbloquea y ya puede procesar la TPDU y enviar una respuesta. Mientras ambos lados puedan llevar el control de quién tiene el turno para transmitir, este esquema funcionará bien.

Nótese que, en la capa de red, incluso un intercambio de datos unidireccional es más complicado que en la capa de transporte. Cada paquete de datos enviado también será reconocido (tarde o temprano). Los paquetes que llevan una TPDU de control también son reconocidos, implícita o explícitamente. Estos reconocimientos son manejados por las entidades de transporte usando el protocolo de capa de red, y no son visibles para los usuarios de transporte. De la misma manera, las entidades de transporte necesitarán preocuparse por los temporizadores y las retransmisiones. Nada de esta maquinaria es vista por los usuarios de transporte. Para ellos, una conexión es un tubo de bits confiable: un usuario mete bits en él y mágicamente aparecen en el otro lado. Esta capacidad de esconder la complejidad es la razón por la cual los protocolos en capas son herramientas tan poderosas.

Cuando ya no se necesita una conexión, debe liberarse para desocupar espacio de tablas en las dos entidades de transporte. La desconexión tiene dos variantes: asimétrica y simétrica. En la variante asimétrica, cualquiera de los dos usuarios de transporte puede emitir una primitiva DISCONNECT (desconectar), que resulta en el envío de una TPDU DISCONNECT a la entidad de transporte remota. A su llegada, se libera la conexión.

En la variante simétrica, cada sentido se cierra por separado, independientemente del otro. Al emitir una DISCONNECT un lado, quiere decir que ya no tiene más datos por enviar, pero aún

está dispuesto a aceptar datos de su compañero. En este modelo, una conexión se libera cuando ambos lados han emitido una DISCONNECT.

En la figura 6-5 se presenta un diagrama de estados del establecimiento y liberación de una conexión con estas primitivas sencillas. Cada transición es disparada por algún evento, ya sea una primitiva ejecutada por el usuario de transporte local o la llegada de un paquete de entrada. Por sencillez, aquí suponemos que cada TPDU se reconoce por separado. También suponemos que se usa un modelo de desconexión simétrica, yéndose primero el cliente. Cabe señalar que este modelo es muy poco refinado. Veremos un modelo más realista después.

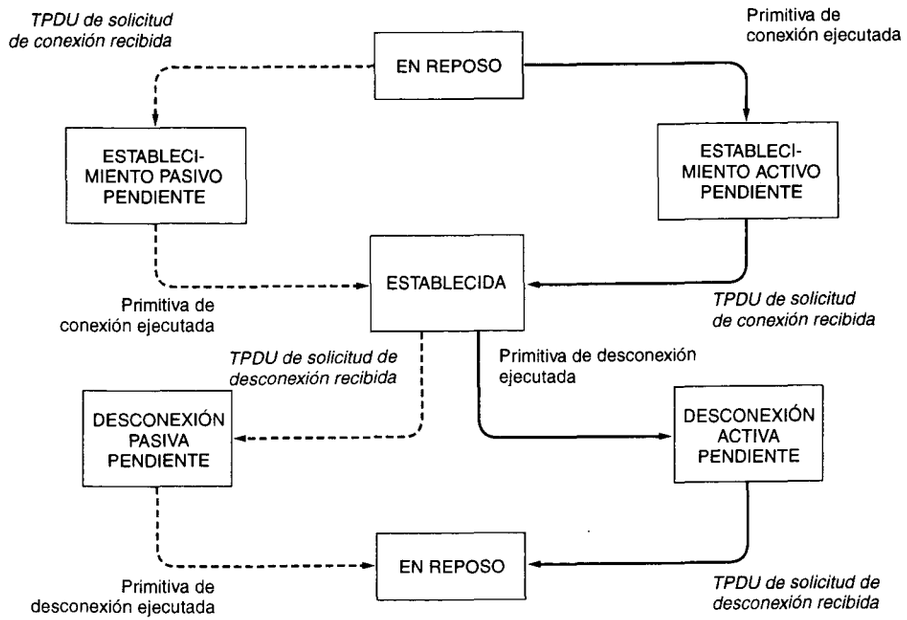


Figura 6-5. Diagrama de estados de un esquema sencillo de manejo de conexiones. Las transiciones rotuladas con cursivas son causadas por llegadas de paquetes. Las líneas continuas muestran la secuencia de estados del cliente. Las líneas punteadas muestran la secuencia de estados del servidor.

Sockets de Berkeley

Inspeccionemos brevemente otro grupo de primitivas de transporte, las primitivas de socket usadas en el UNIX de Berkeley para el TCP. Estas primitivas se listan en la figura 6-6. En términos generales, las primitivas siguen el modelo de nuestro primer ejemplo, pero ofrecen más características y flexibilidad. No veremos aquí las TPDU correspondientes. Ese análisis tendrá que esperar hasta que estudiemos el TCP posteriormente en este capítulo.

Las primeras cuatro primitivas de la lista son ejecutadas en ese orden por los servidores. La primitiva SOCKET crea un nuevo punto final y le asigna espacio de tablas dentro de la entidad de

Primitiva	Significado
SOCKET (ENCHUFAR)	Crea un nuevo punto terminal de comunicación
BIND (LIGAR)	Conecta una dirección local a un socket
LISTEN (ESCUCHAR)	Anuncia la disposición a aceptar conexiones; indica tamaño de cola
ACCEPT (ACEPTAR)	Bloquea al invocador hasta la llegada de un intento de conexión
CONNECT (CONECTAR)	Intenta activamente establecer una conexión
SEND (ENVIAR)	Envía datos a través de la conexión
RECEIVE (RECIBIR)	Recibe datos de la conexión
CLOSE (CERRAR)	Libera la conexión

Figura 6-6. Primitivas de socket para TCP.

transporte. Los parámetros de la llamada especifican el formato de direccionamiento a usar, el tipo de servicio deseado (por ejemplo, corriente de bytes confiable), y el protocolo. Una llamada SOCKET con éxito devuelve un descriptor de archivo ordinario para usarse con las siguientes llamadas, de la misma manera que lo hace una llamada OPEN.

Los sockets recién creados no tienen direcciones. Éstas se asignan usando la primitiva BIND (ligar). Una vez que un servidor ha ligado una dirección a un socket, los clientes remotos pueden conectarse a él. La razón para no hacer que la llamada SOCKET cree directamente una dirección es que a algunos procesos les importan sus direcciones (por ejemplo, han estado usando su misma dirección durante años y todos la conocen), mientras que a otros no les importa.

A continuación viene la llamada LISTEN (escuchar), que asigna espacio para poner en cola las llamadas entrantes por si varios clientes intentan conectarse al mismo tiempo. En contraste con LISTEN en nuestro primer ejemplo, en el modelo de sockets LISTEN no es una llamada bloqueadora.

Para bloquearse en espera de una conexión entrante, el servidor ejecuta una primitiva ACCEPT (aceptar). Cuando llega una TPDU solicitando una conexión, la entidad de transporte crea un socket nuevo con las mismas propiedades que el original y devuelve un descriptor de archivo para él. El servidor entonces puede ramificar (*fork*) un proceso o hilo para manejar la conexión en el socket nuevo y regresar a esperar la siguiente conexión en el socket original.

Ahora veamos el lado del cliente. Aquí también debe crearse un socket primero usando la primitiva SOCKET, pero no se requiere BIND, puesto que la dirección usada no le importa al servidor. La primitiva CONNECT (conectar) bloquea al invocador y comienza activamente el proceso de conexión. Al completarse éste (es decir, cuando se recibe la TPDU adecuada del servidor) el proceso cliente se desbloquea y se establece la conexión. Ambos lados pueden usar ahora SEND y RECEIVE para transmitir y recibir datos a través de la conexión dúplex integral.

La liberación de las conexiones a los sockets es simétrica. Cuando ambos lados han ejecutado una primitiva CLOSE, se libera la conexión.

6.2. ELEMENTOS DE LOS PROTOCOLOS DE TRANSPORTE

El servicio de transporte se implementa mediante un **protocolo de transporte** entre las dos entidades de transporte. En ciertos aspectos, los protocolos de transporte se parecen a los protocolos de enlace de datos que estudiamos detalladamente en el capítulo 3. Ambos se encargan del control de errores, la secuencia y el control de flujo, entre otros asuntos.

Sin embargo, existen diferencias significativas entre los dos. Estas diferencias se deben a diferencias importantes entre los entornos en que operan ambos protocolos, como se muestra en la figura 6-7. En la capa de enlace de datos, dos enrutadores se comunican directamente mediante un canal físico mientras que, en la capa de transporte, este canal físico es reemplazado por la subred completa. Esta diferencia tiene muchas implicaciones importantes para los protocolos.

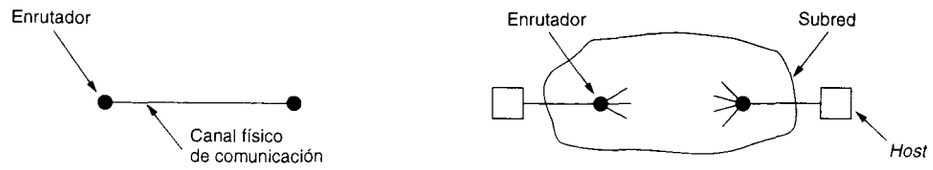


Figura 6-7. (a) Entorno de la capa de enlace de datos. (b) Entorno de la capa de transporte.

Por una parte, en la capa de enlace de datos no es necesario que un enrutador especifique el enrutador con el que quiere hablar; cada línea de salida especifica de manera única un enrutador en particular. En la capa de transporte, se requiere el direccionamiento explícito de los destinos.

Por otro lado, el proceso de establecimiento de una conexión a través del alambre de la figura 6-7(a) es sencillo: la otra terminal siempre está ahí (a menos que se haya caído, en cuyo caso no está ahí). Sea como sea, no hay demasiado que hacer. En la capa de transporte, el establecimiento inicial de la conexión es más complicado, como veremos.

Otra diferencia, muy irritante, entre la capa de enlace de datos y la capa de transporte es la existencia potencial de capacidad de almacenamiento en la subred. Al enviar un enrutador un marco, éste puede llegar o perderse, pero no puede rebotar de un lado a otro durante un rato, esconderse en un rincón alejado del mundo y emerger repentinamente en algún momento inoportuno 30 segundos después. Si la subred usa datagramas y enrutamiento adaptable internamente, hay una probabilidad no despreciable de que un paquete pueda almacenarse durante varios segundos y entregarse después. Las consecuencias de esta capacidad de almacenamiento de paquetes en la subred pueden ser desastrosas y requerir el uso de protocolos especiales.

Una última diferencia entre las capas de enlace de datos y de transporte es una de cantidad, más bien que de tipo. Se requieren *buffers* y control de flujo en ambas capas, pero la presencia de una cantidad de conexiones grande y dinámicamente variable en la capa de transporte puede

requerir un enfoque distinto del que se usa en la capa de enlace de datos. En el capítulo 3 vimos que algunos de los protocolos asignan una cantidad fija de *buffers* a cada línea de modo que, al llegar un marco, siempre haya un *buffer* disponible. En la capa de transporte, la gran cantidad de conexiones que deben manejarse hace menos atractiva la idea de dedicar muchos *buffers* a cada una. En las siguientes secciones examinaremos todos estos importantes temas, además de otros.

6.2.1. Direccionamiento

Cuando un proceso de aplicación desea establecer una conexión con un proceso de aplicación remoto, debe especificar a cuál debe conectarse. (El transporte sin conexiones tiene el mismo problema: ¿a quién debe enviarse cada mensaje?) El método que normalmente se emplea es definir direcciones de transporte en las que los procesos pueden estar a la escucha de solicitudes de conexión. En Internet, estos puntos terminales son pares (dirección IP, puerto local). En las redes ATM, son pares AAL-SAP. Usaremos el término neutral **TSAP** (*Transport Service Access Point*, **punto de acceso al servicio de transporte**). Los puntos terminales análogos de la capa de red (por ejemplo, direcciones de capa de red) se llaman entonces **NSAP** (*Network Service Access Point*, **puntos de acceso al servicio de red**). Las direcciones IP son ejemplos de NSAP.

En la figura 6-8 se ilustra la relación entre el NSAP, el TSAP, la conexión de red y la conexión de transporte de una subred orientada a conexiones (por ejemplo, ATM). Nótese que una entidad de transporte normalmente reconoce varios TSAP. En algunas redes también existen múltiples NSAP, pero en otras cada máquina sólo tiene un NSAP (por ejemplo, una dirección IP). Una posible situación de conexión para una conexión de transporte a través de una capa de red orientada a conexiones es la siguiente.

1. Un proceso servidor de hora del día del *host* 2 se conecta al TSAP 122 para esperar una llamada entrante. La manera en que un proceso se conecta a un TSAP está fuera del modelo de red y depende por entero del sistema operativo local. Podría, por ejemplo, usarse una llamada como nuestra LISTEN.
2. Un proceso de aplicación del *host* 1 quiere averiguar la hora del día, por lo que emite una solicitud CONNECT especificando el TSAP 6 como el origen y el TSAP 122 como destino.
3. La entidad de transporte del *host* 1 selecciona una dirección de red en su máquina (si tiene más de una) y establece una conexión de red entre ellas. (En una subred sin conexiones, no se establecería esta conexión de capa de red.) Usando esta conexión de capa de red, la entidad de transporte del *host* 1 puede hablar con la entidad de transporte del *host* 2.
4. Lo primero que dice la entidad de transporte de 1 a su igual de 2 es: "Buenos días. Me gustaría establecer una conexión de transporte entre mi TSAP 6 y tu TSAP 122. ¿Qué dices?"

5. La entidad de transporte de 2 pregunta entonces al servidor de hora del día en el TSAP 122 si está dispuesto a aceptar una conexión nueva. Si está de acuerdo, se establece la conexión de transporte.

Nótese que la conexión de transporte va de TSAP a TSAP, mientras que la conexión de red sólo cubre una parte del camino, de NSAP a NSAP.

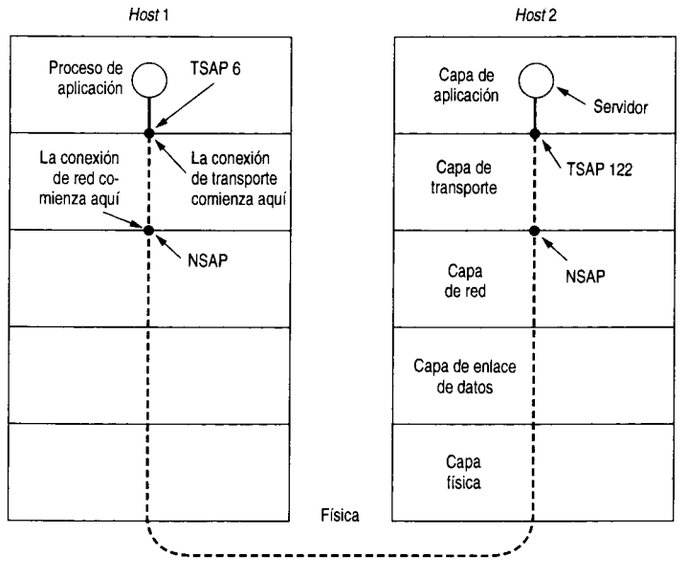


Figura 6-8. Los TSAP, NSAP y conexiones.

El panorama que hemos bosquejado está muy bien, excepto que hemos escondido un pequeño problema: ¿cómo sabe el proceso de usuario del *host* 1 que el servidor de hora del día (*time-of-day server*) está conectado al TSAP 122? Una posibilidad es que el servidor de hora del día se ha estado conectando al TSAP 122 durante años, y gradualmente todos los usuarios de la red han aprendido esto. En este modelo, los servicios tienen direcciones TSAP estables, que pueden imprimirse en papel y darse a los nuevos usuarios al unirse a la red.

Aunque las direcciones TSAP estables podrían funcionar bien con una cantidad pequeña de servicios clave que nunca cambian, en general los procesos de usuario frecuentemente quieren hablar con otros procesos de usuario que sólo existen durante un tiempo corto y no tienen una dirección TSAP conocida por adelantado. Es más, si puede haber muchos procesos de servidor, la mayoría de los cuales se usan pocas veces, es un desperdicio tenerlos activos a todos, escuchando en una dirección TSAP estable todo el día. En pocas palabras, se requiere un esquema mejor.

Uno de tales esquemas, empleado por los *hosts* UNIX de Internet, se muestra en la figura 6-9 de una manera simplificada; se conoce como **protocolo inicial de conexión**. En lugar de que cada servidor concebible escuche en un TSAP bien conocido, cada máquina que desea ofrecer

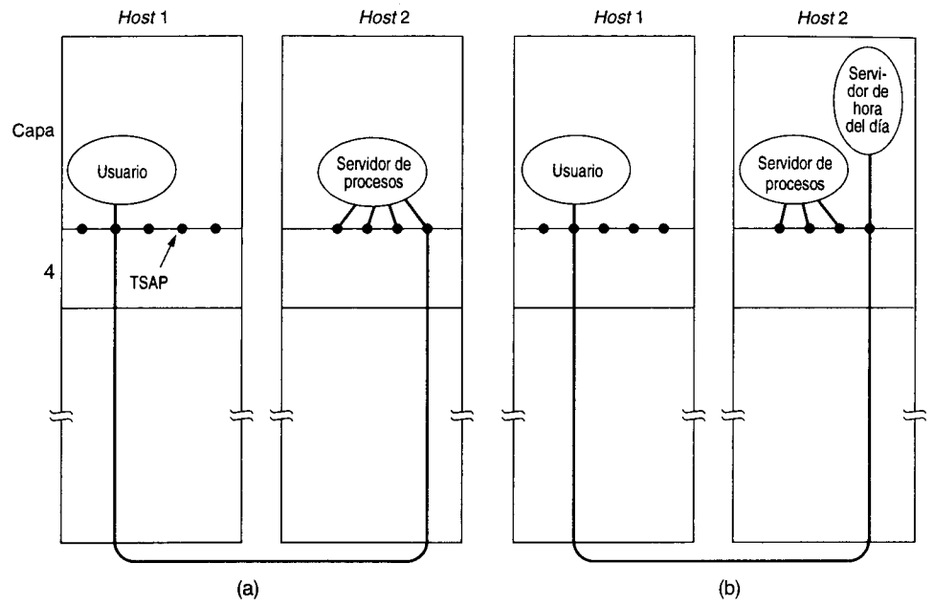


Figura 6-9. Manera en que un proceso de usuario del *host* 1 establece una conexión con un servidor de hora del día del *host* 2.

servicio a usuarios remotos tiene un **servidor de procesos** especial que actúa como apoderado (*proxy*) de los servidores de menor uso y escucha en un grupo de puertos al mismo tiempo, esperando una solicitud de conexión TCP. Los usuarios potenciales de un servicio comienzan por emitir una solicitud CONNECT, especificando la dirección TSAP (puerto TCP) del servicio que desean. Si no hay ningún servidor esperándolos, consiguen una conexión al servidor de procesos, como se muestra en la figura 6-9(a).

Tras obtener la solicitud entrante, el servidor de procesos genera el servidor solicitado, permitiéndole heredar la conexión con el usuario existente. El nuevo servidor entonces hace el trabajo requerido, mientras que el servidor de procesos retorna a escuchar solicitudes nuevas, como se muestra en la figura 6-9(b).

Aunque el protocolo de conexión inicial funciona bien para aquellos servidores que pueden crearse cuando se necesitan, hay muchas situaciones en las que los servicios existen independientemente del servidor de procesos. Por ejemplo, un servidor de archivos necesita operar un *hardware* especial (una máquina con un disco) y no puede simplemente crearse sobre la marcha cuando alguien quiere hablar con él.

Para manejar esta situación, se usa con frecuencia un esquema alterno. En este modelo, existe un proceso especial llamado **servidor de nombres**, o a veces **servidor de directorio**. Para encontrar la dirección TSAP correspondiente a un nombre de servicio dado, como "hora del día", el usuario establece una conexión con el servidor de nombres (que escucha en un TSAP bien conocido). Entonces el usuario envía un mensaje especificando el nombre del servicio, y

el servidor de nombres devuelve la dirección TSAP. Luego el usuario libera la conexión con el servidor de nombres y establece una nueva con el servicio deseado.

En este modelo, al crearse un servicio nuevo, debe registrarse en el servidor de nombres, dando tanto su nombre de servicio (generalmente una cadena ASCII) como la dirección de su TSAP. El servidor de nombres registra esta información en su base de datos interna por lo que, cuando llegan solicitudes posteriores, sabe las respuestas.

La función del servidor de nombres es análoga al operador de asistencia de directorio del sistema telefónico: proporciona una correspondencia de nombres a números. Al igual que en el sistema telefónico, es esencial que la dirección bien conocida del TSAP usada por el servidor de nombres (o el servidor de procesos del protocolo de conexión inicial) en realidad sea bien conocida. Si usted no conoce el número del operador de información, no puede llamar al operador de información para averiguarlo. Si usted cree que el número que se marca para obtener información es obvio, inténtelo alguna vez en otro país.

Supongamos ahora que el usuario ha localizado con éxito la dirección del TSAP al que se debe conectar. Otra pregunta interesante es ¿cómo sabe la entidad local de transporte en qué máquina está localizado ese TSAP? Más específicamente, ¿cómo sabe la entidad de transporte qué dirección de capa de red debe usar para establecer una conexión de red con la entidad de transporte remota que maneja el TSAP requerido?

La respuesta depende de la estructura de las direcciones TSAP. Una estructura posible es que las direcciones TSAP sean **direcciones jerárquicas**. Con direcciones jerárquicas, la dirección consiste en una secuencia de campos usados para dividir en porciones disconexas el espacio de direcciones. Por ejemplo, una dirección TSAP verdaderamente universal podría tener la siguiente estructura:

dirección = <galaxia> <estrella> <planeta> <país> <red> <*host*> <puerto>

Con este esquema es directa la localización de un TSAP en cualquier parte del universo conocido. De manera equivalente, si una dirección TSAP es una concatenación de una dirección NSAP y un puerto (un identificador local que especifica uno de los TSAP locales), entonces, cuando se le da una dirección TSAP a una entidad de transporte para conectarse a ella, usa la dirección NSAP contenida en la dirección TSAP para alcanzar la entidad de transporte remota adecuada.

Como ejemplo sencillo de dirección jerárquica, considere el número telefónico 19076543210. Este número puede analizarse como 1-907-654-3210, donde 1 es un código de país (Estados Unidos + Canadá), 907 es un código de área (Alaska), 654 es una oficina terminal de Alaska, y 3210 es uno de los "puertos" (líneas de suscriptor) de esa oficina terminal.

La alternativa al espacio jerárquico de direcciones es el **espacio plano de direcciones**. Si las direcciones TSAP no son jerárquicas, se requiere un segundo nivel de transformación para localizar la máquina adecuada. Tendría que haber un servidor de nombres que tomara direcciones de puerto como entrada y devolviera direcciones de red como salida. De manera alterna, en algunas situaciones (por ejemplo, en una LAN), es posible difundir una solicitud pidiendo a la máquina de destino que se identifique enviando un paquete.

6.2.2. Establecimiento de una conexión

El establecimiento de una conexión suena fácil, pero en realidad es sorprendentemente complicado. A primera vista, parecería suficiente con que una entidad de transporte enviara una TPDU CONNECTION REQUEST al destino y esperara una respuesta CONNECTION ACCEPTED. El problema ocurre cuando la red puede perder, almacenar o duplicar paquetes.

Imagine una subred que está tan congestionada que los reconocimientos casi nunca regresan a tiempo, y cada paquete termina de temporizar y se retransmite dos o tres veces. Suponga que la subred usa datagramas internamente, y que cada paquete sigue una ruta diferente. Algunos de los paquetes podrían atorarse en un congestionamiento de tráfico dentro de la subred y tardar mucho tiempo en llegar, es decir, se almacenarían en la subred y reaparecerían mucho después.

La peor pesadilla posible es la que sigue. Un usuario establece una conexión con un banco, envía mensajes indicando al banco que transfiera una gran cantidad de dinero a la cuenta de una persona no del todo confiable, y entonces libera la conexión. Por mala fortuna, cada paquete de la transacción se duplica y almacena en la subred. Tras liberar la conexión, todos los paquetes salen de la subred y llegan al destino en orden, solicitando al banco que establezca una conexión nueva, transfiera el dinero (nuevamente) y libere la conexión. El banco no tiene manera de saber que son duplicados; debe suponer que ésta es una segunda transacción independiente, y transfiere nuevamente el dinero. Durante el resto de esta sección estudiaremos el problema de los duplicados retardados, haciendo hincapié en los algoritmos para establecer conexiones de una manera confiable, de modo que pesadillas como la anterior no puedan ocurrir.

El meollo del problema es la existencia de duplicados retrasados. Esto puede atacarse de varias maneras, ninguna de las cuales es muy satisfactoria. Una es usar direcciones de transporte desechables. En este enfoque, cada vez que se requiere una dirección de transporte, se genera una nueva. Al liberarse la conexión, se descarta la dirección. Esta estrategia imposibilita el modelo de servidor de procesos de la figura 6-9.

Otra posibilidad es dar a cada conexión un identificador de conexión (es decir, un número de secuencia que se incremente con cada conexión establecida), seleccionado por la parte iniciadora, y ponerlo en cada TPDU, incluida la que solicita la conexión.

Tras la liberación de una conexión, cada entidad de transporte podría actualizar una tabla que liste conexiones obsoletas como pares (entidad de transporte igual, identificador de conexión). Cada vez que entrara una solicitud de conexión, podría cotejarse con la tabla para ver si pertenece a una conexión previamente liberada.

Por desgracia, este esquema tiene una falla básica: requiere que cada entidad de transporte mantenga una cierta cantidad de información histórica durante un tiempo indefinido. Si se cae una máquina, perdiéndose su memoria, ya no sabrá qué identificadores de conexión usó.

Más bien, necesitamos un enfoque diferente. En lugar de permitir que los paquetes vivan eternamente en la subred, debemos diseñar un mecanismo para matar a los paquetes viejos que

aún andan vagando por ahí. Si podemos asegurar que ningún paquete viva más allá de cierto tiempo conocido, el problema se vuelve algo más manejable.

El tiempo de vida de un paquete puede restringirse a un máximo conocido usando una de las siguientes técnicas:

1. Diseño de subred restringido.
2. Contador de saltos en cada paquete.
3. Marca de tiempo en cada paquete.

El primer método incluye cualquier método que evite que los paquetes hagan ciclos, combinado con una manera de limitar el retardo por congestionamientos a través de la trayectoria más larga posible (ahora conocida). El segundo método consiste en incrementar el conteo de saltos cada vez que se reenvía el paquete. El protocolo de enlace de datos simplemente descarta cualquier paquete cuyo contador de saltos ha excedido cierto valor. El tercer método requiere que cada paquete lleve la hora en la que fue creado, acordando los enrutadores descartar cualquier paquete que haya rebasado cierto tiempo predeterminado. Este último método requiere que los relojes de los enrutadores estén sincronizados, lo que no es una tarea fácil a menos que se logre la sincronización externamente a la red, por ejemplo escuchando la WWV o alguna otra estación de radio que difunda la hora exacta periódicamente.

En la práctica, necesitaremos garantizar no sólo que el paquete está muerto, sino también que todos los acuses de recibo están muertos, por lo que ahora introduciremos T, que es un múltiplo pequeño del tiempo de vida de paquete máximo verdadero. El múltiplo depende del protocolo, y simplemente tiene el efecto de hacer más grande a T. Si esperamos un tiempo T tras el envío de un paquete, podemos estar seguros de que todos los rastros suyos ya han desaparecido, y que ni él ni sus acuses de recibo aparecerán repentinamente de la nada para complicar el asunto.

Teniendo limitados los tiempos de vida de los paquetes, es posible proponer una manera a prueba de errores de establecer conexiones seguras. El método descrito a continuación se debe a Tomlinson (1975); resuelve el problema pero presenta algunas peculiaridades propias. El método fue refinado por Sunshine y Dalal (1978). En la práctica se usan ampliamente variantes suyas.

Para resolver el problema de una máquina que pierde toda la memoria acerca de su estado tras una caída, Tomlinson propuso equipar cada *host* con un reloj de hora del día. Los relojes de los diferentes *hosts* no necesitan estar sincronizados. Se supone que cada reloj tiene la forma de un contador binario que se incrementa a intervalos uniformes. Además, la cantidad de bits del contador debe ser igual o mayor que la cantidad de bits en los números de secuencia. Por último, y lo más importante, se supone que el reloj continúa operando aun ante la caída de un *host*.

La idea básica es asegurar que nunca estén pendientes dos TPDU de número idéntico al mismo tiempo. Cuando se establece una conexión, los k bits de orden menor del reloj se usan como número inicial de secuencia (también k bits). Por tanto, y a diferencia de los protocolos del capítulo 3, cada conexión comienza a numerar sus TPDU con un número de secuencia diferente. El espacio de secuencia también debe ser lo bastante grande para que, al dar la vuelta los

números de secuencia, las TPDU viejas con el mismo número de secuencia hayan desaparecido hace mucho tiempo. Esta relación lineal entre tiempo y números secuenciales iniciales se muestra en la figura 6-10.

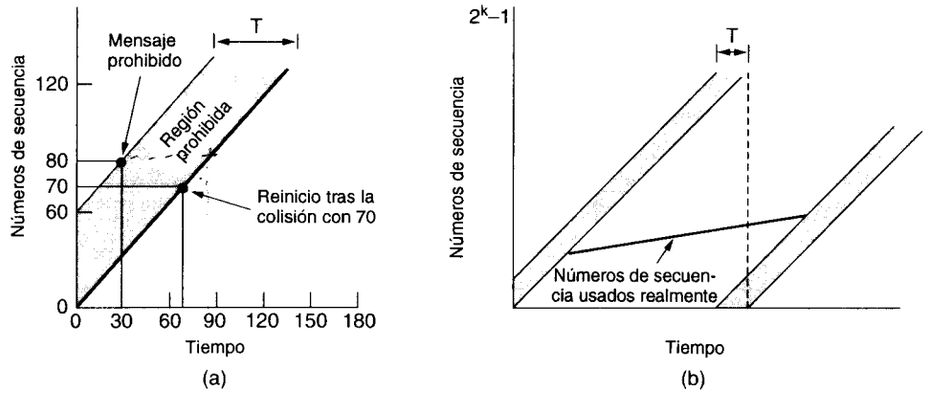


Figura 6-10. (a) Las TPDU no pueden entrar en la zona prohibida. (b) El problema de la resincronización.

Una vez que ambas entidades de transporte han acordado el número de secuencia inicial, puede usarse cualquier protocolo de ventana corrediza para el control de flujo de datos. En realidad, la curva inicial de números de secuencia (indicada por la línea gruesa) no es realmente lineal, sino una escalera, ya que el reloj avanza en pasos discretos. Por sencillez, ignoraremos este detalle.

Ocurre un problema cuando se cae un *host*. Al regresar nuevamente, sus entidades de transporte no saben dónde estaban en el espacio de secuencias. Una solución es requerir que las entidades de transporte estén inactivas durante T segundos tras una recuperación para permitir que todas las TPDU viejas mueran. Sin embargo, en una interred compleja, T puede ser bastante grande, por lo que no es atractiva esta estrategia.

Para evitar requerir T seg de tiempo muerto tras una caída, es necesario introducir una nueva restricción en el uso de números de secuencia. Podemos ver claramente la necesidad de esta restricción mediante un ejemplo. Sea T, el tiempo máximo de vida de un paquete, 60 seg. El pulso del reloj es de uno por segundo. Como se muestra en la figura 6-10, el número de secuencia inicial de una conexión abierta en el momento x será x. Imagine que, en $t = 30$ seg, una TPDU de datos ordinaria enviada a través de la conexión 5, previamente abierta, recibe el número de secuencia 80. Llamemos a esta TPDU X. De inmediato tras el envío de la TPDU X, el *host* se cae y reinicia pronto. En $t = 60$, el *host* comienza a reabrir las conexiones 0 a 4. En $t = 70$, reabre la conexión 5, usando el número de secuencia inicial 70, como se requiere. Durante los siguientes 15 segundos envía las TPDU de datos 70 a 80. Por tanto, en $t = 85$ se ha inyectado una TPDU nueva con número de secuencia 80 y conexión 5 en la subred. Por desgracia, la TPDU X aún

existe. Si ésta llegara al receptor antes de la nueva TPDU 80, la TPDU *X* sería aceptada y se rechazaría la TPDU 80 correcta como duplicado.

Para evitar tales problemas, debemos evitar la asignación de números de secuencia nuevos (es decir, asignados a TPDU nuevas) durante un tiempo *T* antes de su uso potencial como números iniciales de secuencia. Las combinaciones ilegales de tiempo y número de secuencia se muestran como la **región prohibida** en la figura 6-10(a). Antes de enviar cualquier TPDU por alguna conexión, la entidad de transporte debe leer el reloj y comprobar que no está en la región prohibida.

El protocolo puede meterse en problemas de dos maneras. Si un *host* envía demasiados datos con demasiada rapidez a través de una conexión recién abierta, la curva de número de secuencia real contra tiempo puede subir con mayor rapidez que la curva de número de secuencia inicial contra tiempo. Esto significa que la tasa de datos máxima en cualquier conexión es de una TPDU por pulso de reloj, y también significa que la entidad de transporte debe esperar hasta que el reloj pulse antes de abrir una nueva conexión tras un reinicio por caída, no sea que se use dos veces el mismo número de secuencia. Ambos puntos son argumentos a favor de un pulso de reloj corto (unos cuantos milisegundos).

Desafortunadamente, el ingreso en la región prohibida desde abajo al enviar con demasiada rapidez no es la única manera de meterse en problemas. Por la figura 6-10(b) debe quedar claro que con cualquier tasa de datos menor que la tasa del reloj, la curva de números de secuencia reales usados contra tiempo tarde o temprano entrará en la región prohibida por la izquierda. Cuanto mayor sea la pendiente de la curva de números de secuencia reales, mayor será el retardo de este evento. Como indicamos antes, justo antes de enviar cada TPDU, la entidad de transporte debe comprobar que no está a punto de entrar en la región prohibida; de ser así, debe retardar la TPDU durante *T* seg o resincronizar los números de secuencia.

El método basado en reloj resuelve el problema del duplicado retrasado de las TPDU de datos, pero para que este método resulte de utilidad, debe establecerse primero una conexión. Dado que las TPDU de control también pueden retrasarse, hay el problema potencial de lograr que ambos lados acuerden en número de secuencia inicial. Supóngase, por ejemplo, que se establecen conexiones haciendo que el *host* 1 envíe una TPDU CONNECTION REQUEST con el número de secuencia inicial y el número de puerto de destino a un igual remoto, el *host* 2. El receptor, el *host* 2, reconoce entonces esta solicitud enviando de regreso una TPDU CONNECTION ACCEPTED. Si la TPDU CONNECTION REQUEST se pierde, pero aparece con retardo una CONNECTION REQUEST duplicada en el *host* 2, se establecerá incorrectamente la conexión.

Para resolver este problema, Tomlinson (1975) desarrolló el **protocolo de acuerdo de tres vías** (*three-way handshake*). Este protocolo de establecimiento no requiere que ambos lados comiencen a transmitir con el mismo número de secuencia, por lo que puede usarse con otros métodos de sincronización distintos del método de reloj global. El procedimiento normal de establecimiento al iniciar el *host* 1 se muestra en la figura 6-11(a). El *host* 1 escoge un número de secuencia, *x*, y envía al *host* 2 una TPDU CONNECTION REQUEST que lo contiene. El *host* 2 responde con una TPDU CONNECTION ACCEPTED reconociendo *x* y anunciando su propio número de secuencia inicial, *y*. Por último, el *host* 1 reconoce la selección de un número de secuencia inicial del *host* 2 en la primera TPDU de datos que envía.

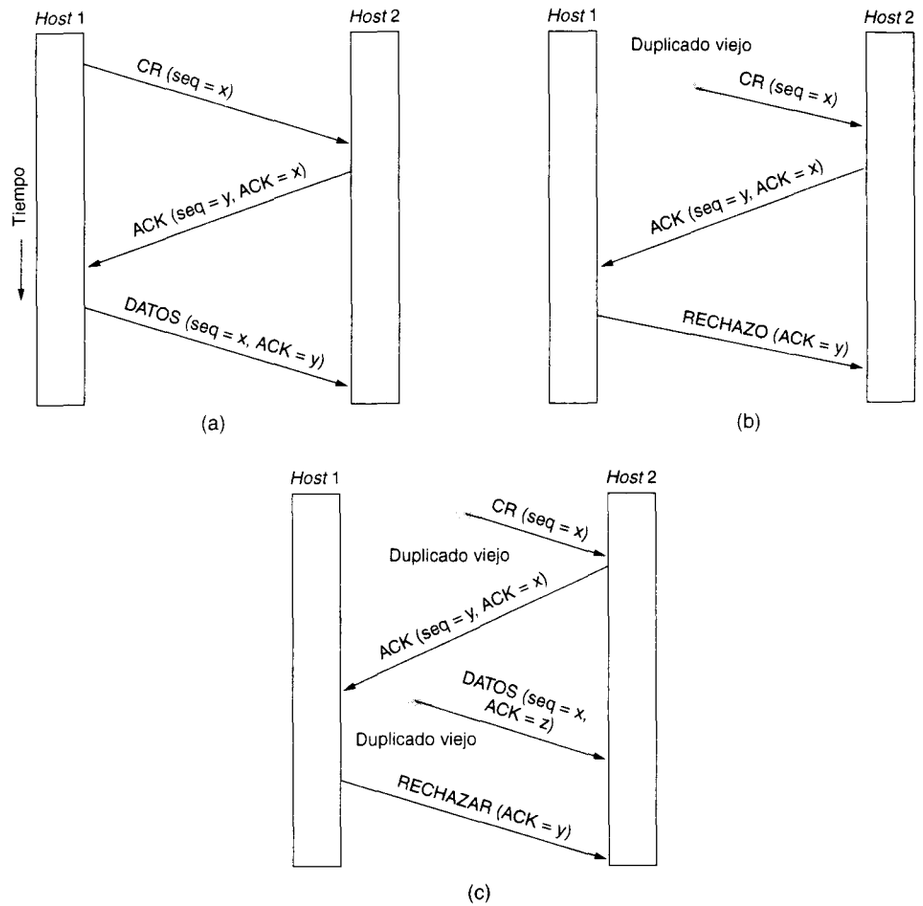


Figura 6-11. Tres situaciones de protocolo para establecer una conexión usando un protocolo de acuerdo de tres vías (*three-way handshake*). CR y ACK significan SOLICITUD DE CONEXIÓN y CONEXIÓN ACEPTADA, respectivamente. (a) Operación normal. (b) SOLICITUD DE CONEXION duplicada vieja que aparece de la nada. (c) SOLICITUD DE CONEXIÓN duplicada y ACK duplicada.

Ahora veamos la manera en que funciona el protocolo de acuerdo de tres vías en presencia de una TPDU de control duplicada con retraso. En la figura 6-12(b), la primera TPDU es una CONNECTION REQUEST duplicada con retraso de una conexión vieja. Esta TPDU llega al *host* 2 sin el conocimiento del *host* 1. El *host* 2 reacciona a esta TPDU enviando al *host* 1 una TPDU CONECTION ACCEPTED, solicitando de hecho la comprobación de que el *host* 1 en verdad trató de establecer una nueva conexión. Al rechazar el *host* 1 el intento de establecimiento del *host* 2, éste se da cuenta de que fue engañado por un duplicado con retardo y abandona la conexión. De esta manera, un duplicado con retardo no causa daño.

El peor caso ocurre cuando están flotando por ahí en la subred tanto una CONNECTION REQUEST retardada como un acuse de recibo de una CONNECTION ACCEPTED. Este caso se muestra en la figura 6-11(c). Como en el ejemplo previo, el *host* 2 recibe una CONNECTION REQUEST retrasada y la contesta. En este momento es crucial notar que el *host* 2 ha propuesto usar *y* como número de secuencia inicial para el tráfico del *host* 2 al *host* 1, sabiendo bien que no existen todavía TPDU que contengan el número de secuencia *y* ni acuses de recibo de *y*. Cuando llega el segundo TPDU retrasado al *host* 2, el hecho de que se reconoció *z* en lugar de *y* indica al *host* 2 que éste también es un duplicado viejo. Lo importante a notar aquí es que no haya combinación de CONNECTION REQUEST, CONNECTION ACCEPTED u otras TPDU que puedan causar la falla del protocolo, permitiendo el establecimiento de una conexión accidentalmente cuando nadie la quiere.

Un esquema alterno para el establecimiento confiable de conexiones ante duplicados retardados se describe en (Watson, 1981). En él, se usan temporizadores múltiples para excluir eventos indeseados.

6.2.3. Liberación de una conexión

La liberación de una conexión es más fácil que su establecimiento. No obstante, hay más escollos de los que uno podría imaginar. Como mencionamos antes, hay dos estilos de terminación de una conexión: liberación asimétrica y liberación simétrica. La liberación asimétrica es la manera en que funciona el sistema telefónico: cuando una parte cuelga, se rompe la conexión. La liberación simétrica trata la conexión como dos conexiones unidireccionales distintas, y requiere que cada una se libere por separado.

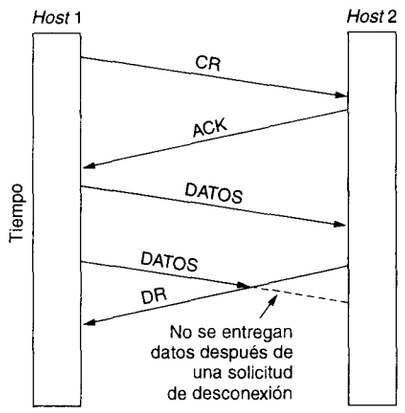


Figura 6-12. Desconexión abrupta con pérdida de datos.

La liberación asimétrica es abrupta y puede resultar en la pérdida de datos. Considere la situación de la figura 6-12. Tras establecerse la conexión, el *host* 1 envía una TPDU que llega

adecuadamente al *host* 2. Entonces el *host* 1 envía otra TPDU. Desgraciadamente, el *host* 2 emite una DISCONNECT antes de llegar la segunda TPDU. El resultado es que se libera la conexión y se pierden datos.

Es obvio que se requiere un protocolo de liberación más refinado para evitar la pérdida de datos. Una posibilidad es usar la liberación simétrica, en la que cada dirección se libera independientemente de la otra. Aquí, un *host* puede continuar enviando datos aun tras haber enviado una TPDU DISCONNECT.

La liberación simétrica es ideal cuando cada proceso tiene una cantidad fija de datos por enviar y sabe con certidumbre cuándo los ha enviado. En otras situaciones, la determinación de si se ha efectuado o no todo el trabajo y si debe terminarse o no la conexión no es tan obvia. Podríamos pensar en un protocolo en el que el *host* 1 diga: "Ya terminé. ¿Terminaste también?" Si el *host* 2 responde: "Ya terminé también. Adiós", la conexión puede liberarse con seguridad.

Por desgracia, este protocolo no siempre funciona. Hay un problema famoso que tiene que ver con este asunto; se llama **problema de los dos ejércitos**. Imagine que un ejército blanco está acampado en un valle, como se muestra en la figura 6-13. En los dos cerros que rodean al valle hay ejércitos azules. El ejército blanco es más grande que cualquiera de los dos ejércitos azules por separado, pero juntos éstos son más grandes que el ejército blanco. Si cualquiera de los dos ejércitos azules ataca por su cuenta, será derrotado, pero si los dos atacan simultáneamente obtendrán la victoria.

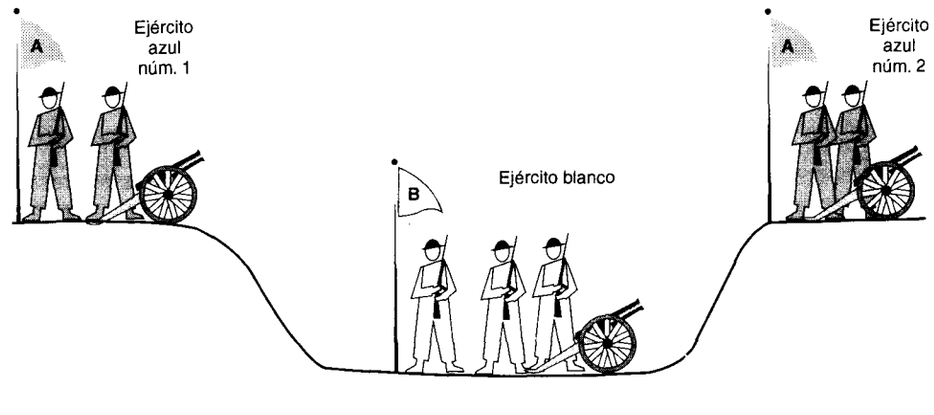


Figura 6-13. Problema de los dos ejércitos.

Los ejércitos azules quieren sincronizar sus ataques. Sin embargo, su único medio de comunicación es el envío de mensajeros a pie a través del valle, donde podrían ser capturados, perdiéndose el mensaje (es decir, tienen que usar un canal de comunicación no confiable). La pregunta es: ¿Existe un protocolo que permita que los ejércitos azules ganen?

Supóngase que el comandante del ejército azul núm. 1 envía un mensaje que dice: "Propongo que ataquemos al amanecer del 29 de marzo. ¿Qué les parece?" Ahora supongamos que llega

el mensaje y que el comandante del ejército azul núm. 2 está de acuerdo, y que su respuesta llega con seguridad al ejército azul núm. 1. ¿Ocurrirá el ataque? Probablemente no, porque el comandante núm. 2 no sabe si su respuesta llegó. Si no llegó, el ejército azul núm. 1 no atacará, y sería tonto de su parte emprender el ataque.

Mejoremos ahora el protocolo haciéndolo un protocolo de acuerdo de tres vías (*three-way handshake*). El iniciador de la propuesta original debe dar un acuse de recibo de la respuesta. Suponiendo que no se pierden mensajes, el ejército azul núm. 2 recibirá el acuse de recibo, pero ahora el que dudará será el comandante del ejército azul núm. 1. A fin de cuentas, no sabe si ha llegado su acuse de recibo y, si no llegó, sabe que el ejército azul núm. 2 no atacará. Podríamos probar ahora un protocolo de acuerdo de cuatro vías, pero tampoco ayudaría.

De hecho, puede demostrarse que no existe un protocolo que funcione. Supóngase que existiera algún protocolo. O el último mensaje del protocolo es esencial, o no lo es. Si no lo es, hay que eliminarlo (así como los demás mensajes no esenciales) hasta que quede un protocolo en el que todos los mensajes sean esenciales. ¿Qué ocurre si el mensaje final no pasa? Acabamos de decir que es esencial, por lo que, si se pierde, el ataque no ocurrirá. Dado que el transmisor del mensaje final nunca puede estar seguro de su llegada, no se arriesgará a atacar. Peor aún, el otro ejército azul sabe esto, por lo que no atacará tampoco.

Para ver la aplicación del problema de los dos ejércitos a la liberación de conexiones, simplemente sustituya "atacar" por "desconectar". Si ninguna de las partes está preparada para desconectarse hasta estar convencida de que la otra está preparada para desconectarse también, nunca ocurrirá la desconexión.

En la práctica, generalmente estamos más dispuestos a correr riesgos al liberar conexiones que al atacar ejércitos blancos, por lo que la situación no es del todo desesperada. En la figura 6-14 se ilustran cuatro situaciones de liberación usando un protocolo de acuerdo de tres vías. Aunque este protocolo no es infalible, generalmente es adecuado.

En la figura 6-14(a) vemos el caso normal en el que uno de los usuarios envía una TPDU DR (solicitud de desconexión, DISCONNECTION REQUEST) a fin de iniciar la liberación de una conexión. Al llegar, el receptor devuelve también una TPDU de DR e inicia un temporizador, para el caso de que se pierda su DR.

Al llegar esta DR, el transmisor original envía de regreso una TPDU de ACK y libera la conexión. La liberación de una conexión significa que la entidad de transporte remueve la información sobre la conexión de su tabla de conexiones abiertas y avisa de alguna manera al dueño de la conexión (el usuario de transporte). Esta acción es diferente de la emisión de una primitiva DISCONNECT por parte de un usuario de transporte.

Si se pierde la TPDU ACK, como se muestra en la figura 6-14(b), la situación se salva por el temporizador. Si expira el temporizador, se libera de todos modos la conexión.

Ahora consideremos el caso de la pérdida del segundo DR. El usuario que inicia la desconexión no recibirá la respuesta esperada, terminará de temporizar y comenzará de nuevo. En la figura 6-14(c) vemos la manera en que funciona esto, suponiendo que la segunda vez no se pierden algunas TPDU y que todas las TPDU se entregan correctamente y a tiempo.

Nuestra última situación, la figura 6-14(d), es la misma que la figura 6-14(c), excepto que ahora suponemos que todos los intentos repetidos de retransmitir la DR también fallan debido a la

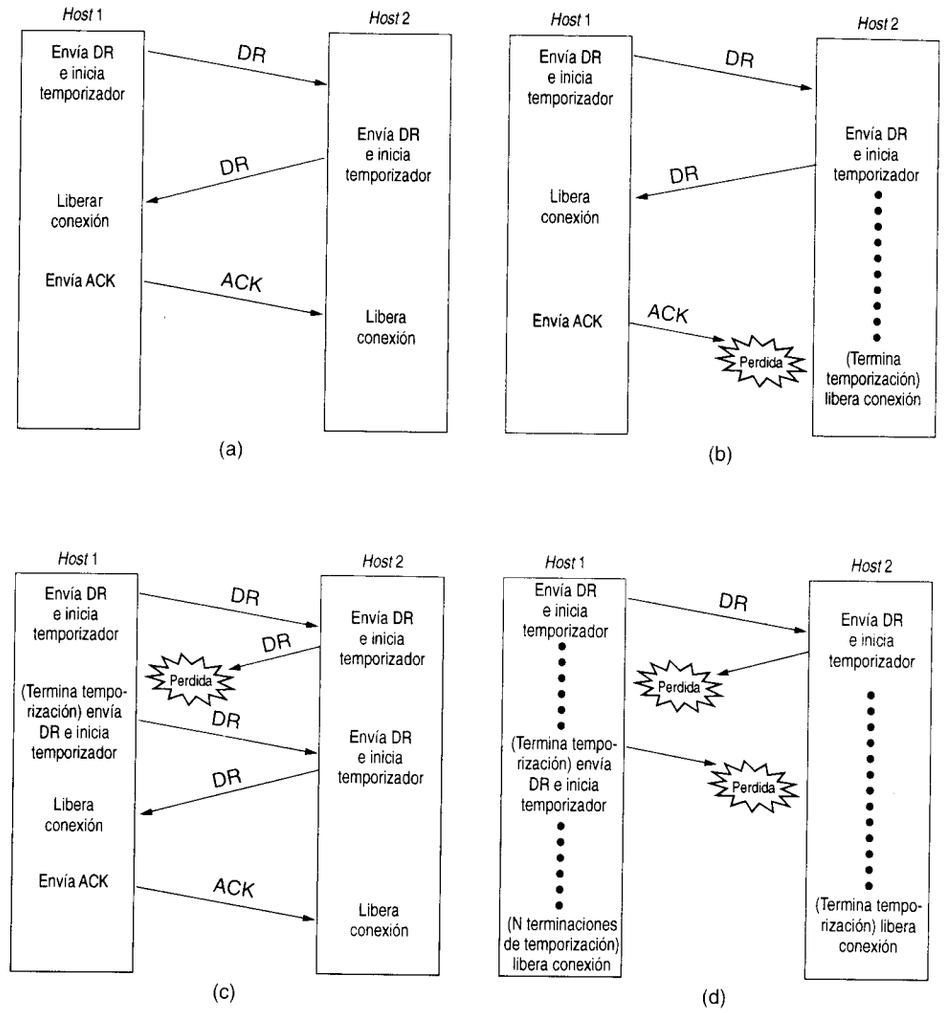


Figura 6-14. Cuatro situaciones de protocolo de liberación de una conexión. (a) Caso normal del protocolo de acuerdo de tres vías (*three-way handshake*). (b) Pérdida del último ACK. (c) Respuesta perdida. (d) Respuesta perdida y pérdida de las DR subsecuentes.

pérdida de algunas TPDU. Tras *N* reintentos, el transmisor simplemente se da por vencido y libera la conexión. Mientras tanto, el receptor termina de temporizar y también se sale.

Aunque este protocolo generalmente es suficiente, en teoría puede fallar si se pierden el DR inicial y *N* retransmisiones. El transmisor se dará por vencido y liberará la conexión, pero el otro lado no sabrá nada sobre los intentos de desconexión y seguirá plenamente activo. Esta situación resulta en una conexión medio abierta.

Pudimos haber evitado este problema no permitiendo que el transmisor se diera por vencido tras *N* reintentos, sino obligándolo a seguir insistiendo hasta recibir una respuesta. Sin embargo, si se permite que el otro lado termine de temporizar, entonces el transmisor continuará eternamente, pues nunca hará su aparición una respuesta. Si no permitimos que el lado receptor termine la temporización, entonces el protocolo se atora en la figura 6-14(b).

Otra manera de cortar la conexión medio abierta es tener una regla que diga que, si no ha llegado ninguna TPDU durante cierta cantidad de segundos, se libera automáticamente la conexión. De esta manera, si un lado llega a desconectarse, el otro lado detectará la falta de actividad y también se desconectará. Por supuesto que si se pone en práctica esta regla, es necesario que cada entidad de transporte tenga un temporizador que se detenga y se reinicie con cada envío de una TPDU. Si expira este temporizador, se transmite una TPDU ficticia, simplemente para evitar que el otro lado se desconecte. Por otra parte, si se usa la regla de desconexión automática y se pierden demasiadas TPDU ficticias una tras otra en una conexión en reposo, primero un lado y luego el otro se desconectarán automáticamente.

No insistiremos más en este punto, pero ya debe quedar claro que la liberación de una conexión no es ni remotamente tan sencilla como parece inicialmente.

6.2.4. Control de flujo y *buffers*

Habiendo examinado el establecimiento y la liberación de conexiones con algún detalle, veamos ahora la manera en que se manejan las conexiones mientras están en uso. Uno de los asuntos clave nos es conocido: el control de flujo. En muchos aspectos el problema del control de flujo en la capa de transporte es igual que en la capa de enlace de datos, pero en otros es diferente. La similitud básica es que en ambas capas se requiere una ventana corrediza u otro esquema en cada conexión para evitar que un transmisor rápido abrume a un receptor lento. La diferencia principal es que un enrutador por lo regular tiene relativamente pocas líneas, y que un *host* puede tener numerosas conexiones. Esta diferencia hace que sea impráctico implementar la estrategia de *buffers* de enlace de datos en la capa de transporte.

En los protocolos de enlace de datos del capítulo 3, los marcos se manejaban en *buffers* tanto en el enrutador transmisor como en el receptor. Por ejemplo, en el protocolo 6 se requiere que tanto el transmisor como el receptor dediquen *MaxSeq* + 1 *buffers* a cada línea, la mitad para entradas y la mitad para salidas. En un *host* con un máximo de, digamos, 64 conexiones y un número de secuencia de 4 bits, este protocolo requerirá 1024 *buffers*.

En la capa de enlace de datos, el lado transmisor debe poner en *buffer* los marcos de salida porque cabe la posibilidad de que tengan que retransmitirse. Si la subred provee un servicio de datagramas, la entidad de transporte transmisora también debe manejar *buffers*, por la misma razón. Si el receptor sabe que el transmisor pone en *buffer* todas las TPDU hasta que se reconocen, el receptor podría o no dedicar *buffers* específicos a conexiones específicas, según considere necesario. Por ejemplo, el receptor podría mantener un solo grupo de *buffers* compartido por todas las conexiones. Cuando entra una TPDU, se hace un intento por adquirir dinámicamente un *buffer* nuevo. Si hay uno disponible, se acepta la TPDU; de otro modo, se descarta. Dado que el transmisor está preparado para retransmitir las TPDU perdidas por la subred, no hay nada de

malo en hacer que el receptor se deshaga de las TPDU, aunque se desperdicien algunos recursos. El transmisor simplemente sigue intentando hasta que recibe un reconocimiento.

En resumen, si el servicio de red no es confiable, el transmisor debe poner en *buffer* todas las TPDU enviadas, igual que la capa de enlace de datos. Sin embargo, con un servicio de red confiable son posibles otras componendas. En particular, si el transmisor sabe que el receptor siempre tiene espacio de *buffer*, no necesita retener copias de las TPDU que envía. Sin embargo, si el receptor no puede garantizar que se aceptará cada TPDU de entrada, el transmisor tendrá que usar *buffers* de todas maneras. En el último caso, el transmisor no puede confiar en el acuse de recibo de la capa de red porque éste sólo significa que ha llegado la TPDU, no que ha sido aceptada. Regresaremos después a este importante punto.

Aun si el receptor ha acordado manejar los *buffers*, todavía queda la cuestión del tamaño de los *buffers*. Si la mayoría de las TPDU tiene aproximadamente el mismo tamaño, es natural organizar los *buffers* como una reserva de *buffers* de tamaño idéntico, con una TPDU por *buffer*, como en la figura 6-15(a). Sin embargo, si hay una variación grande en el tamaño de las TPDU, de unos cuantos caracteres ingresados en una terminal hasta miles de caracteres provenientes de transferencias de archivos, el fondo de *buffers* de tamaño fijo presenta problemas. Si el tamaño de los *buffers* se escoge igual a la TPDU más grande, se desperdiciará espacio cada vez que llegue una TPDU corta. Si el tamaño de *buffer* se escoge menor que el tamaño máximo de TPDU, se requerirán varios *buffers* para las TPDU grandes, con la complejidad asociada.

Otra forma de enfrentar el problema del tamaño de los *buffers* es el uso de *buffers* de tamaño variable, como en la figura 6-15(b). La ventaja aquí es un mejor uso de la memoria, al costo de

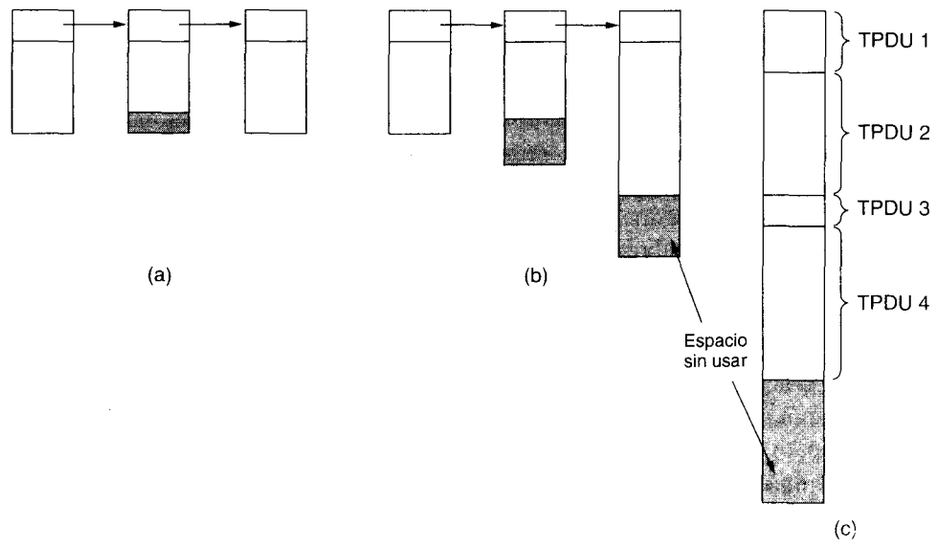


Figura 6-15. (a) *Buffers* encadenados de tamaño fijo. (b) *Buffers* encadenados de tamaño variable. (c) Un gran *buffer* circular por conexión.

una gestión de *buffers* más complicada. Una tercera posibilidad es dedicar un solo *buffer* circular grande por conexión, como en la figura 6-15(c). Este sistema también hace buen uso de la memoria cuando todas las conexiones tienen una carga alta, pero es deficiente si algunas conexiones son de poca carga.

La media óptima entre los *buffers* en el origen y los *buffers* en el destino depende del tipo de tráfico transportado por la conexión. Con un tráfico de bajo ancho de banda con ráfagas, como el producido por una terminal interactiva, es mejor no dedicarle *buffers*, sino adquirirlos de manera dinámica en ambos extremos. Dado que el transmisor no puede estar seguro de que el receptor será capaz de adquirir un *buffer*, el transmisor debe retener una copia de la TPDU hasta recibir su acuse de recibo. Por otra parte, con la transferencia de archivos y otro tráfico de alto ancho de banda, es mejor si el receptor dedica una ventana completa de *buffers*, para permitir el flujo de datos a máxima velocidad. Por tanto, para un tráfico en ráfagas de bajo ancho de banda, es mejor mantener *buffers* en el transmisor; para alto ancho de banda, es mejor hacerlo en el receptor.

A medida que se abren y cierran conexiones, y a medida que cambia el patrón de tráfico, el transmisor y el receptor necesitan ajustar dinámicamente sus asignaciones de *buffers*. En consecuencia, el protocolo de transporte debe permitir a un *host* transmisor solicitar espacio de *buffer* en el otro extremo. Los *buffers* podrían repartirse por conexión o, en conjunto, para todas las conexiones en operación entre los dos *hosts*. Como alternativa, el receptor, sabiendo su situación de *buffers* (pero sin saber el tráfico generado) podría indicar al transmisor "te he reservado *X* *buffers*". Si aumentase la cantidad de conexiones abiertas, podría ser necesario reducir una asignación, por lo que el protocolo debe contemplar esta posibilidad.

Una manera razonablemente general de manejar la asignación dinámica de *buffers* es desacoplarlos de los acuses de recibo, en contraste con los protocolos de ventana corrediza del capítulo 3. La gestión dinámica de *buffers* implica, de hecho, una ventana de tamaño variable. Inicialmente, el transmisor solicita una cierta cantidad de *buffers*, con base en su necesidades percibidas. El receptor entonces otorga tantos *buffers* como puede. Cada vez que el transmisor envía una TPDU, debe disminuir su asignación, deteniéndose por completo al llegar la asignación a cero. El receptor entonces incorpora tanto los reconocimientos como las asignaciones de *buffer* al tráfico en reversa.

En la figura 6-16 se muestra un ejemplo de la manera en que podría trabajar la gestión dinámica de ventanas en una subred de datagramas con números de secuencia de 4 bits. Supóngase que la información de asignación de *buffers* viaja en TPDU distintas, como se muestra, y no se incorpora en el tráfico en reversa. Inicialmente, *A* quiere ocho *buffers*, pero se le otorgan solamente cuatro. Entonces envía tres TPDU, de las cuales se pierde la tercera. La TPDU 6 reconoce la recepción de todas las TPDU hasta el número de secuencia 1, inclusive, permitiendo por tanto que *A* libere esos *buffers*, y además informa a *A* que tiene permiso de enviar tres TPDU más comenzando después de 1 (es decir, las TPDU 2, 3 y 4). *A* sabe que ya ha enviado el número 2, por lo que piensa que puede enviar las TPDU 3 y 4, lo que procede a hacer. En este punto se bloquea y debe esperar una nueva asignación de *buffers*. Por otro lado, las retransmisiones inducidas por terminaciones de temporización (línea 9) sí pueden ocurrir durante el bloqueo, pues usan *buffers* ya asignados. En la línea 10, *B* reconoce la recepción de todas la TPDU hasta la 4, inclusive, pero se niega a permitir que *A* continúe. Tales situaciones son imposibles con los

protocolos de ventana fija del capítulo 3. La siguiente TPDU de *B* a *A* asigna otro *buffer* y permite a *A* continuar.

	A	Mensaje	B	Comentarios
1	→	<solicito 8 *buffers*>	→	A quiere 8 *buffers*
2	←	<ack = 15, buf = 4>	←	B sólo otorga los mensajes 0 a 3
3	→	<seq = 0, data = m0>	→	A tiene 3 *buffers* libres
4	→	<seq = 1, data = m1>	→	A tiene 2 *buffers* libres
5	→	<seq = 2, data = m2>	···	Mensaje perdido, pero A piensa que le queda 1
6	←	<ack = 1, buf = 3>	←	B reconoce 0 y 1, permite 2-4
7	→	<seq = 3, data = m3>	→	A tiene un *buffer* libre
8	→	<seq = 4, data = m4>	→	A tiene 0 *buffers* libres, y debe detenerse
9	→	<seq = 2, data = m2>	→	A termina de temporizar y retransmite
10	←	<ack = 4, buf = 0>	←	Todo reconocido, pero A bloqueado aún
11	←	<ack = 4, buf = 1>	←	A puede enviar 5 ahora
12	←	<ack = 4, buf = 2>	←	B encontró un *buffer* nuevo en algún lado
13	→	<seq = 5, data = m5>	→	A tiene 1 *buffer* libre
14	→	<seq = 6, data = m6>	→	A está bloqueado nuevamente
15	←	<ack = 6, buf = 0>	←	A aún está bloqueado
16	···	<ack = 6, buf = 4>	←	Interbloqueo potencial

Figura 6-16. Asignación dinámica de buffers. Las flechas muestran la dirección de la transmisión. Los puntos suspensivos (...) indican una TPDU perdida.

Pueden surgir problemas potenciales con los esquemas de asignación de *buffers* de este tipo en las redes de datagramas si hay pérdidas de TPDU. Observe la línea 16. *B* ha asignado ahora más *buffers* a *A*, pero la TPDU de asignación se perdió. Dado que las TPDU de control no están en secuencia, *A* se encuentra estancado. Para evitar esta situación, cada *host* debe enviar periódicamente una TPDU de control con el acuse de recibo y estado de *buffers* de cada conexión. De esta manera, el estancamiento se romperá tarde o temprano.

Hasta ahora hemos supuesto tácitamente que el único límite impuesto a la tasa de datos del transmisor es la cantidad de espacio de *buffer* disponible en el receptor. A medida que siga cayendo significativamente el precio de la memoria, podría llegar a ser factible equipar a los *hosts* con tanta memoria que la falta de *buffers* dejaría de ser un problema.

Si el espacio de *buffer* ya no limita el flujo máximo, aparecerá otro cuello de botella: la capacidad de carga de la subred. Si enrutadores adyacentes pueden intercambiar cuando mucho *x* marcos/seg y hay *k* trayectorias sin traslapo entre un par de *hosts*, no hay manera de que esos *hosts* puedan intercambiar más de *kx* TPDU/seg, sin importar la cantidad de espacio de *buffer* disponible en cada terminal. Si el transmisor presiona demasiado (es decir, envía más de *kx* TPDU/seg), la subred se congestionará, pues será incapaz de entregar las TPDU a la velocidad con que llegan.

Lo que se necesita es un mecanismo basado en la capacidad de carga de la subred en lugar de la capacidad de *buffer* del receptor. Es claro que el mecanismo de control de flujo debe aplicarse al transmisor para evitar que estén pendientes demasiadas TPDU sin acuse de recibo al mismo tiempo. Belsnes (1975) propuso el uso de un esquema de flujo de ventana corrediza en el que el transmisor ajusta dinámicamente el tamaño de la ventana para igualarla a la capacidad de carga de la red. Si la red puede manejar c TPDU/seg y el tiempo de ciclo (incluidos transmisión, propagación, encolamiento, proceso en el receptor y devolución del acuse de recibo) es de r, entonces la ventana del transmisor debe ser cr. Con una ventana de este tamaño, el transmisor normalmente opera con la tubería llena. Cualquier pequeña disminución en el desempeño de la red causará que se bloquee.

A fin de ajustar periódicamente el tamaño de la ventana, el transmisor podría vigilar ambos parámetros y calcular después el tamaño de ventana deseado. La capacidad de carga puede determinarse con sólo contar la cantidad de TPDU con acuse de recibo durante algún periodo de tiempo y dividirla entre el periodo de tiempo. Durante la medición, el transmisor debe enviar a la mayor velocidad posible, para asegurarse que sea la capacidad de carga de la red, y no la baja tasa de entrada, el factor limitante de la tasa de acuses de recibo. El tiempo requerido para el acuse de recibo de una TPDU transmitida puede medirse exactamente y mantenerse una media de operación. Dado que la capacidad de la red depende de la cantidad de tráfico en ella, debe ajustarse el tamaño de la ventana con frecuencia para responder a los cambios en la capacidad de carga. Como veremos después, la Internet usa un esquema parecido.

6.2.5. Multiplexión

La multiplexión de varias conversaciones en conexiones, circuitos virtuales y enlaces físicos desempeña un papel importante en diferentes capas de la arquitectura de la red. En la capa de transporte puede surgir la necesidad de multiplexión por varias razones. Por ejemplo, en las redes que usan circuitos virtuales en la subred, cada conexión abierta consume espacio de tabla en los enrutadores durante la duración completa de la conexión. Si también se dedican *buffers* al circuito virtual en cada enrutador, un usuario que haya dejado establecida una sesión entre una terminal y una máquina remota durante el descanso para el café estará consumiendo recursos caros. Aunque esta implementación de la conmutación de paquetes echa por tierra una de las razones principales para tener conmutación de paquetes (facturar al usuario con base en la cantidad de datos enviados, no el tiempo de conexión), muchas portadoras han escogido este enfoque porque se parece mucho al modelo de conmutación de circuitos al que se han acostumbrado por décadas.

La consecuencia de una estructura de precios que penaliza fuertemente a las instalaciones que tienen muchos circuitos virtuales abiertos durante largos periodos de tiempo es que se vuelve atractiva la multiplexión de diferentes conexiones de transporte en la misma conexión de red. Esta forma de multiplexión, llamada **multiplexión ascendente**, se muestra en la figura 6-17(a). En esta figura, cuatro conexiones de transporte diferentes usan la misma conexión de red (por ejemplo, un circuito virtual ATM) al *host* remoto. Cuando el tiempo de conexión es un

componente importante de la facturación de la portadora, es responsabilidad de la capa de transporte agrupar las conexiones de transporte de acuerdo con su destino y hacer corresponder cada grupo con la mínima cantidad de conexiones de red. Si se hacen corresponder demasiadas conexiones de transporte con una sola conexión de red, el rendimiento será bajo, porque la ventana generalmente estará llena y los usuarios tendrán que esperar su turno para enviar un mensaje. Si se hacen corresponder muy pocas conexiones con una conexión de red, el servicio será caro. Al usarse multiplexión ascendente con ATM, tenemos la situación irónica (¿trágica?) de tener que identificar la conexión usando un campo de la cabecera de transporte, aunque ATM proporciona expresamente para ese propósito más de 4000 números de circuito virtual por trayectoria virtual.

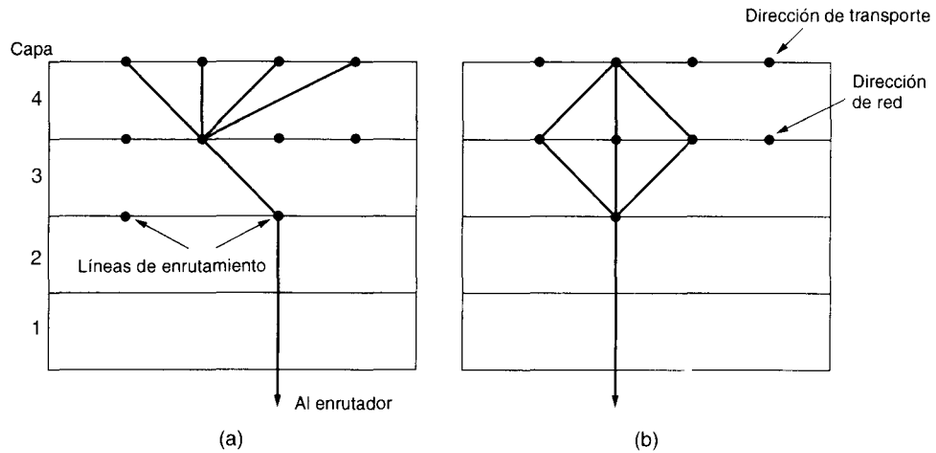


Figura 6-17. (a) Multiplexión ascendente. (b) Multiplexión descendente.

La multiplexión también puede ser útil en la capa de transporte por otra razón, relacionada con las decisiones técnicas de la portadora, en lugar de sus decisiones de precio. Supóngase, por ejemplo, que cierto usuario importante necesita una conexión de alto ancho de banda de tarde en tarde. Si la subred obliga a un control de flujo de ventana corrediza con un número de secuencia de n bits, el usuario deberá detener el envío tan pronto estén pendientes $2^n - 1$ paquetes y deberá esperar que los paquetes se propaguen al *host* remoto y se reconozcan. Si la conexión física es vía satélite, el usuario está limitado de hecho a $2^n - 1$ paquetes cada 540 mseg. Con, por ejemplo, $n = 8$ y paquetes de 128 bytes, el ancho de banda útil es de 484 kbps, aunque el ancho de banda del canal físico sea más de 100 veces mayor.

Una posible solución es hacer que la capa de transporte abra varias conexiones de red y distribuya el tráfico entre ellas por turno circular, como se indica en la figura 6-17(b). Este *modus operandi* se llama **multiplexión descendente**. Con k conexiones de red abiertas, el ancho

de banda efectivo se aumenta en un factor de k. Con 4095 circuitos virtuales, paquetes de 128 bytes y un número de secuencia de 8 bits, en teoría es posible lograr tasas de datos de más de 1.6 Gbps. Por supuesto que este desempeño puede lograrse sólo si la línea de salida puede manejar 1.6 Gbps, puesto que los 4095 circuitos virtuales aún se envían a través de una línea física, cuando menos en la figura 6-17(b). Si hay disponibles varias líneas de salida, puede usarse también la multiplexión descendente para aumentar aún más el desempeño.

6.2.6. Recuperación de caídas

Si los *hosts* y los enrutadores están sujetos a caídas, la recuperación de éstas se vuelve un tema importante. Si la entidad de transporte está por entero dentro de los *hosts*, la recuperación de caídas de red y de enrutador es directa. Si la capa de red proporciona servicio de datagramas, las entidades de transporte esperan la pérdida de algunas TPDU todo el tiempo, y saben cómo manejarla. Si la capa de red proporciona servicio orientado a conexiones, entonces la pérdida de un circuito virtual se maneja estableciendo uno nuevo y sondeando la entidad de transporte remota para saber cuáles TPDU ha recibido y cuáles no. Estas últimas pueden retransmitirse.

Un problema más complicado es la manera de recuperarse de caídas del *host*. En particular, podría ser deseable que los clientes sean capaces de continuar trabajando cuando los servidores caen y se reinician rápidamente. Para ilustrar la dificultad, supongamos que un *host*, el cliente, envía un archivo grande a otro *host*, el servidor de archivos, usando un protocolo de parada y espera sencillo. La capa de transporte del servidor simplemente pasa las TPDU entrantes al usuario de transporte, una por una. A medio camino de la transmisión se cae el servidor. Al regresar, sus tablas se reinicializan, por lo que ya no sabe con precisión dónde estaba.

En un intento por recuperar su estado previo, el servidor podría enviar una TPDU de difusión a todos los demás *hosts*, anunciando que acaba de caerse y solicitando a sus clientes que informen sobre el estado de todas las conexiones abiertas. Cada cliente puede estar en uno de dos estados: una TPDU pendiente, *S1*, o ninguna TPDU pendiente, *S0*. Con base en esta información de estado, el cliente debe decidir si retransmitirá o no la TPDU más reciente.

A primera vista parecería obvio: el cliente debe retransmitir sólo si tiene una TPDU pendiente no reconocida (es decir, si está en el estado *S1*) cuando se entera de la caída. Sin embargo, una inspección más cercana revela dificultades con este enfoque ingenuo. Considere, por ejemplo, la situación en la que la entidad de transporte del servidor envía primero un reconocimiento y luego ejecuta la actualización del proceso de aplicación. La escritura de una TPDU en esta corriente de salida y el envío de un reconocimiento son dos eventos indivisibles diferentes que no pueden hacerse simultáneamente. Si ocurre una caída tras el envío del acuse de recibo pero antes de hacer la escritura, el cliente recibirá el acuse de recibo y estará, por tanto, en el estado *S0* al llegar el anuncio de recuperación de la caída. Entonces el cliente no retransmitirá, pensando (incorrectamente) que la TPDU llegó. Esta decisión del cliente conduce a una TPDU faltante.

En este punto el lector podría pensar: "Ese problema se resuelve fácilmente. Todo lo que hay que hacer es reprogramar la entidad de transporte para que primero haga la escritura y luego

envíe el acuse de recibo". Piénselo mejor. Imagine que se ha hecho la escritura pero que la caída ocurre antes de poder enviar el acuse de recibo. El cliente estará en el estado *S1*, y por tanto retransmitirá, conduciendo a una TPDU duplicada no detectada en la corriente de salida al proceso de aplicación del servidor.

Sin importar cómo se programen el transmisor y el receptor, siempre habrá situaciones en las que el protocolo no podrá recuperarse correctamente. El servidor puede programarse de una de dos maneras: mandar acuse de recibo primero o escribir primero. El cliente puede programarse de cuatro maneras: siempre retransmitir la última TPDU, nunca retransmitir la última TPDU, retransmitir sólo en el estado *S0* o retransmitir sólo en el estado *S1*. Esto da ocho combinaciones pero, como veremos, para cada combinación existe algún grupo de eventos que hacen que falle el protocolo.

Son posibles tres eventos en el servidor: el envío de un acuse de recibo (*A*), la escritura al proceso de salida (*W*) y una caída (*C*). Los tres eventos pueden ocurrir en seis órdenes diferentes: *AC(W)*, *AWC*, *C(AW)*, *C(WA)*, *WAC* y *WC(A)*, donde los paréntesis se usan para indicar que ni *A* ni *W* pueden seguir a *C* (es decir, una vez que el servidor se cae, se cayó). En la figura 6-18 se muestran las ocho combinaciones de la estrategia cliente-servidor y las secuencias de eventos válidas para cada una. Nótese que para cada estrategia hay alguna secuencia de eventos que causa la falla del protocolo. Por ejemplo, si el cliente siempre retransmite, el evento *AWC* generará un duplicado no detectado, aun si los otros dos eventos funcionan adecuadamente.

Estrategia usada por el *host* transmisor

Estrategia usada por el *host* transmisor	Primero ACK, luego escritura			Primero escribe, luego ACK		
	AC(W)	AWC	C(AW)	C(WA)	WAC	WC(A)
Siempre retransmitir	BIEN	DUP.	BIEN	BIEN	DUP.	DUP.
Nunca retransmitir	PERD.	BIEN	PERD.	PERD.	BIEN	BIEN
Retransmitir con S0	BIEN	DUP.	PERD.	PERD.	DUP.	BIEN
Retransmitir con S1	PERD.	BIEN	BIEN	BIEN	BIEN	DUP.

BIEN = El protocolo funciona correctamente
DUP. = El protocolo genera un mensaje duplicado
PERD. = El protocolo pierde un mensaje

Figura 6-18. Diferentes combinaciones de estrategias cliente y servidor.

Hacer más elaborado el protocolo no sirve de nada. Aunque el cliente y el servidor intercambien varias TPDU antes de que el servidor intente escribir, para que el cliente sepa exactamente lo que está a punto de ocurrir, el cliente no tiene manera de saber si ha ocurrido una caída justo antes o justo después de la escritura. La conclusión es inevitable: según nuestra regla básica de que no

debe haber eventos simultáneos, la caída de un *host* y su recuperación no pueden hacerse transparentes a las capas superiores.

En términos más generales, este resultado puede replantearse como que la recuperación de una caída de capa N sólo puede hacerla la capa $N + 1$, y sólo si la capa superior retiene suficiente información de estado. Como se mencionó antes, la capa de transporte puede recuperarse de fallas de la capa de red, siempre y cuando cada extremo de una conexión lleve el registro de dónde está.

Este problema nos lleva al tema de qué significa en realidad un acuse de recibo de extremo a extremo. En principio, el protocolo de transporte es de extremo a extremo, y no está encadenado como las capas inferiores. Ahora considere el caso de un usuario que introduce solicitudes de transacciones con una base de datos remota. Suponga que la entidad de transporte remota está programada para pasar primero las TPDU a la siguiente capa superior y luego emitir los acuses de recibo. Aun en este caso, la recepción de un acuse de recibo en la máquina del usuario no significa necesariamente que el *host* remoto se quedó encendido suficiente tiempo para actualizar la base de datos. Probablemente es imposible lograr un verdadero acuse de recibo de extremo a extremo, cuya recepción significa que el trabajo se ha hecho, y cuya falta significa que no. Este punto lo analizan con mayor detalle Saltzer *et al.* (1984).

6.3. UN PROTOCOLO DE TRANSPORTE SENCILLO

Para hacer más concretas las ideas estudiadas hasta ahora, en esta sección examinaremos detalladamente una capa de transporte de ejemplo. El ejemplo se ha seleccionado con cuidado para que sea razonablemente realista, pero lo bastante sencillo para que sea fácil de entender. Las primitivas de servicio abstractas que usaremos son las primitivas orientadas a conexiones de la figura 6-3.

6.3.1. Las primitivas de servicio de ejemplo

Nuestro primer problema es cómo expresar de manera concreta estas primitivas de transporte. CONNECT (conexión) es fácil: sencillamente tenemos un procedimiento de biblioteca *connect* que puede llamarse con los parámetros adecuados necesarios para establecer una conexión. Los parámetros son los TSAP locales y remotos. Durante la llamada, el indicador se bloquea (es decir, se suspende) mientras la entidad de transporte intenta establecer la conexión. Si la conexión tiene éxito, se desbloquea el solicitante y puede comenzar la transmisión de datos.

Cuando un proceso quiere poder aceptar llamadas entrantes, llama a *listen* (escuchar), especificando un TSAP en particular en el que quiere escuchar. El proceso entonces se bloquea hasta que algún proceso remoto intenta establecer una conexión con su TSAP.

Nótese que este modelo es altamente asimétrico. Un lado es pasivo, ejecutando un *listen* y esperando hasta que ocurre algo. El otro lado es activo e inicia la conexión. Surge una pregunta interesante sobre lo que debe hacerse si el lado activo comienza primero. Una estrategia es hacer que falle el intento de conexión si no hay un escucha en el TSAP remoto. Otra estrategia

es hacer que el iniciador se bloquee (posiblemente para siempre) hasta que aparezca un escucha.

Un término medio, usado en nuestro ejemplo, es mantener la solicitud de conexión del lado receptor durante cierto intervalo de tiempo. Si un proceso de ese *host* llama a *listen* antes de que termine la temporización, se establece la conexión; de otro modo se rechaza y se desbloquea al solicitante, devolviéndole un error.

Para liberar una conexión, usaremos un procedimiento *disconnect* (desconectar). Cuando ambos lados se han desconectado, se libera la conexión. En otras palabras, estamos usando un modelo de desconexión simétrico.

La transmisión de datos tiene precisamente el mismo problema que el establecimiento de una conexión: el envío es activo, pero la recepción es pasiva. Usaremos la misma solución para la transmisión de datos que para el establecimiento de la conexión, una llamada activa *send* (envío) que transmite datos, y una llamada pasiva de *receive* (recibir) que de bloquea hasta que llega una TPDU.

Nuestra definición concreta de servicio consiste por tanto en cinco primitivas: CONNECT, LISTEN, DISCONNECT, SEND y RECEIVE. Cada primitiva corresponde exactamente a un procedimiento de biblioteca que ejecuta la primitiva. Los parámetros de las primitivas de servicio y los procedimientos de biblioteca son los siguientes:

```
connum = LISTEN(local)
connum = CONNECT(local, remote)
status = SEND(connum, buffer, bytes)
status = RECEIVE(connum, buffer, bytes)
status = DISCONNECT(connum)
```

La primitiva LISTEN anuncia la disposición del que llama a aceptar solicitudes de conexión dirigidas a los TSAP indicados. El usuario de la primitiva se bloquea hasta que se hace un intento de conexión a él. No hay expiración de la temporización.

La primitiva CONNECT toma dos parámetros, un TSAP local (es decir, dirección de transporte), *local*, y un TSAP remoto, *remote*, e intenta establecer una conexión de transporte entre los dos; si tiene éxito, devuelve en *connum* un número no negativo que sirve para identificar la conexión en llamadas subsiguientes; si falla, la razón del fracaso se pone en *connum* como número negativo. En nuestro modelo sencillo, cada TSAP puede participar sólo en una conexión de transporte, por lo que una razón posible de falla es que una de las direcciones de transporte ya esté en uso. Algunas otras razones son: *host* remoto caído, dirección local ilegal y dirección remota ilegal.

La primitiva SEND transmite el contenido del *buffer* como mensaje por la conexión de transporte indicada, posiblemente en varias unidades si es demasiado grande. Los errores posibles, devueltos en *status*, son falta de conexión, dirección de *buffer* ilegal y cuenta negativa.

La primitiva RECEIVE indica el deseo del solicitante de aceptar datos. El tamaño del mensaje de entrada se pone en *bytes*. Si el proceso remoto ha liberado la conexión o la dirección de *buffer* es ilegal (por ejemplo, fuera del programa de usuario), se asigna a *status* a un código de error que indica la naturaleza del problema.

La primitiva DISCONNECT termina una conexión de transporte; el parámetro *connum* indica cuál. Los errores posibles son que *connum* pertenezca a otro proceso, o que *connum* no sea un identificador de conexión válido. El código de error (o 0 si todo funcionó bien) se devuelve en *status*.

6.3.2. La entidad de transporte de ejemplo

Antes de ver el código de la entidad de transporte de ejemplo, tenga bien presente que este ejemplo es análogo a los primeros ejemplos presentados en el capítulo 3: es para propósitos pedagógicos, no una propuesta seria. En aras de la sencillez, se han omitido aquí muchos de los detalles técnicos (como comprobación extensa de errores) necesarios para un sistema de producción.

La capa de transporte hace uso de las primitivas de servicio de red para enviar y recibir las TPDU. Para este ejemplo, necesitamos escoger las primitivas de servicio de red a usar. Una selección podría haber sido el servicio de datagramas no confiable. No lo hemos seleccionado para mantener sencillo el ejemplo. Con el servicio de datagramas no confiable, el código de transporte habría sido grande y complejo, encargándose principalmente de paquetes perdidos y retrasados. Es más, muchas de estas ideas ya se analizaron con detalle en el capítulo 3.

En cambio, hemos optado por usar un servicio de red confiable orientado a conexiones. De esta manera, podremos enfocarnos a los asuntos de transporte que no ocurren en las capas inferiores. Éstos incluyen el establecimiento de conexiones, la liberación de conexiones y el manejo de crédito, entre otros. Un servicio sencillo de transporte construido encima de una red ATM podría verse así.

En general, la entidad de transporte puede ser parte del sistema operativo del *host* o puede ser un paquete de rutinas de biblioteca operando en el espacio de direcciones del usuario; también puede estar contenida en un *chip* coprocesador o en una tarjeta de red conectada al *backplane* del *host*. Por sencillez, nuestro ejemplo se ha programado como si fuera un paquete de biblioteca, pero los cambios necesarios para hacerlo parte del sistema operativo son mínimos (principalmente la manera de acceder a los *buffers*).

Vale la pena señalar, sin embargo, que en este ejemplo la "entidad de transporte" no es en realidad una entidad independiente, sino parte del proceso de usuario. En particular, cuando el usuario ejecuta una primitiva que se bloquea, como LISTEN, toda la entidad de transporte se bloquea también. Aunque este diseño es adecuado para un *host* con un solo proceso de usuario, en un *host* con varios usuarios sería más natural hacer que la entidad de transporte sea un proceso aparte, diferente de todos los procesos de usuario.

La interfaz con la capa de red se establece mediante los procedimientos *to_net* y *from_net* (no se muestran). Cada uno tiene seis parámetros. Primero viene la identificación de la conexión, que se relaciona uno a uno con los circuitos virtuales de la red. A continuación vienen los bits *Q* y *M* que, al estar establecidos en 1, indican "mensaje de control" y "continúan más datos de este mensaje en el siguiente paquete", respectivamente. Tras esto tenemos el tipo de paquete, seleccionado entre los seis tipos de paquete listados en la figura 6-19. Por último, tenemos un apuntador a los datos mismos y un entero que indica el número de bytes de datos.

Paquete de red	Significado
CALL REQUEST (solicitud de llamada)	Se envía para establecer una conexión
CALL ACCEPTED (llamada aceptada)	Respuesta a CALL REQUEST
CLEAR REQUEST (solicitud de desconexión)	Se envía para liberar una conexión
CLEAR CONFIRMATION (confirmación de desconexión)	Respuesta a CLEAR REQUEST
DATA (datos)	Sirve para transportar datos
CREDIT (crédito)	Paquete de control para manejar la ventana

Figura 6-19. Paquetes de capa de red usados en nuestro ejemplo.

En las llamadas a *to_net*, la entidad de transporte llena todos los parámetros para que los lea la capa de red; en las llamadas de *from_net*, la capa de red desmembra un paquete de entrada antes de pasarlo a la entidad de transporte. Al pasar información como parámetros de procedimiento en lugar de pasar el paquete de salida o entrada mismo, la capa de transporte queda protegida de los detalles del protocolo de capa de red. Si la entidad de transporte intentara enviar un paquete estando llena la ventana corrediza del circuito virtual subyacente, se suspende en *to_net* hasta haber espacio en la ventana. Este mecanismo es transparente para la entidad de transporte y lo controla la capa de red usando comandos como *enable_transport_layer* y *disable_transport_layer* análogos a los descritos en los protocolos del capítulo 3. La gestión de la ventana de la capa de paquetes también la realiza la capa de red.

Además de este mecanismo de suspensión transparente, hay procedimientos *sleep* y *wakeup* (dormir y despertar), que no se muestran, invocados por la entidad de transporte. El procedimiento *sleep* se invoca cuando la entidad de transporte está bloqueada lógicamente en espera de un evento externo, generalmente la llegada de un paquete. Tras haber llamado a *sleep*, la entidad de transporte (y por supuesto el proceso de usuario) deja de ejecutarse.

El código de la entidad de transporte se muestra en la figura 6-20. Cada conexión está siempre en uno de siete estados, a saber:

1. IDLE (inactivo) — No se ha establecido todavía una conexión.
2. WAITING (en espera) — Se ha ejecutado CONNECT y enviado CALL REQUEST.
3. QUEUED (en cola) — Ha llegado una CALL REQUEST; aún no hay LISTEN.
4. ESTABLISHED (establecida) — Se ha establecido la conexión.
5. SENDING (enviando) — El usuario está esperando permiso de enviar un paquete.
6. RECEIVING (recibiendo) — Se ha hecho un RECEIVE.
7. DISCONNECTING (desconexión) — Se ha hecho localmente un DISCONNECT.

Las transiciones entre estados pueden ocurrir al suceder cualquiera de los siguientes eventos: se ejecuta una primitiva, llega un paquete o termina el temporizador.

```
#define MAX_CONN 32                     /* cantidad máxima de conexiones simultáneas */
#define MAX_MSG_SIZE 8192               /* tamaño máximo de mensaje, en bytes */
#define MAX_PKT_SIZE 512                /* paquete más grande en bytes */
#define TIMEOUT 20
#define CRED 1
#define OK 0

#define ERR_FULL -1
#define ERR_REJECT -2
#define ERR_CLOSED -3
#define LOW_ERR -3

typedef int transport_address;
typedef enum {CALL_REQ,CALL_ACC,CLEAR_REQ,CLEAR_CONF,DATA_PKT,CREDIT} pkt_type;
typedef enum {IDLE,WAITING,QUEUED,ESTABLISHED,SENDING,RECEIVING,DISCONN} cstate;

/* Variables globales. */
transport_address listen_address;       /* dirección local en la que se está escuchando */
int listen_conn;                        /* identificación de conexión para escucha */
unsigned char data[MAX_PKT_SIZE];       /* área de trabajo para paquetes de datos */

struct conn {
  transport_address local_address, remote_address;
  cstate state;                         /* estado de esta conexión */
  unsigned char *user_buf_addr;         /* apuntador al buffer de recepción */
  int byte_count;                       /* conteo de envío/recepción */
  int clr_req_received;                 /* establecido al recibir paquete CLEAR_REQ */
  int timer;                            /* sirve para terminar temporización de paquetes CALL_REQ */
  int credits;                          /* cantidad de mensajes que se puede enviar */
} conn[MAX_CONN];

void sleep(void);                       /* prototipos */
void wakeup(void);
void to_net(int cid, int q, int m, pkt_type pt, usigned char *p, int bytes);
void from_net(int *cid, int *q, int *m, pkt_type *pt, unsigned char *p, int *bytes);

int listen(transport_address t)
{ /* El usuario quiere escuchar si hay una conexión. Ve si CALL_REQ ya llegó. */
  int i = 1, found = 0;

  for (i = 1; i <= MAX_CONN; i++)       /* busca en la tabla CALL_REQ */
     if (conn[i].state == QUEUED && conn[i].local_address == t) {
          found = i;
          break;
     }

  if (found == 0) {
     /* No hay CALL_REQ esperando. Se duerme hasta que llega o termina la temporización. */
     listen_address = t; sleep(); i = listen_conn ;
  }
  conn[i].state = ESTABLISHED;          /* se establece la conexión */
  conn[i].timer = 0;                    /* no se usa temporizador */
```



```
  listen_conn = 0;                      /* se supone que 0 es una dirección no válida */
  to_net(i, 0, 0, CALL_ACC, data, 0);   /* indica a la red que acepte una conexión */
  return(i);                            /* devuelve identificador de conexión */
}

int connect(transport_address l, transport_address r)
{ /* El usuario quiere conectarse a un proceso remoto;  envía paquete CALL_REQ. */
  int i;
  struct conn *cptr;

  data[0] = r;  data[1] = l;            /* el paquete CALL_REQ necesita éstos */
  i = MAX_CONN;                         /* busca hacia atrás en la tabla */
  while (conn[i].state != IDLE && i > 1) i = i - 1;
  if (conn[i].state == IDLE) {
     /* Crea una entrada en la tabla indicando que se ha enviado CALL_REQ. */
     cptr = &conn[i];
     cptr->local_address = l; cptr->remote_address = r;
     cptr->state = WAITING; cptr->clr_req_received = 0;
     cptr->credits = 0; cptr->timer = 0;
     to_net(i, 0, 0, CALL_REQ, data, 2);
     sleep();                           /* espera CALL_ACC o CLEAR_REQ */
     if (cptr->state == ESTABLISHED) return(i);
     if (cptr->clr_req_received) {
          /* El otro lado rechazó la llamada. */
          cptr->state = IDLE;           /* Regresa al estado de reposo */
          to_net(i, 0, 0, CLEAR_CONF, data, 0);
          return(ERR_REJECT);
     }
  } else return(ERR_FULL);              /* rechaza conexión: no hay espacio de tabla */
}

int send(int cid, unsigned char bufptr[], int bytes)
{ /* El usuario quiere enviar un mensaje. */
  int i, count, m;
  struct conn *cptr = &conn[cid];

  /* Entra en el estado de transmisión. */
  cptr->state = SENDING;
  cptr->byte_count = 0;                 /* # bytes enviados hasta ahora en este mensaje */
  if (cptr->clr_req_received == 0 && cptr->credits == 0) sleep();
  if (cptr->clr_req_received == 0) {
     /* Crédito disponible; divide mensaje en paquetes de ser necesario. */
     do {
          if (bytes - cptr->byte_count > MAX_PKT_SIZE) {/* mensaje multipaquete */
               count = MAX_PKT_SIZE; m = 1;   /* más paquetes después */
          } else {                      /* mensaje de un solo paquete */
               count = bytes - cptr->byte_count; m = 0;   /* último paquete de este mensaje */
          }
          for (i = 0; i < count; i++) data [i] = bufptr[cptr->byte_count + i];
          to_net(cid, 0, m, DATA_PKT, data, count);   /* envía un paquete */
          cptr->byte_count = cptr->byte_count + count;   /* incrementa bytes enviados hasta ahora */
     } while (cptr->byte_count < bytes);      /* repite ciclo hasta enviar todo el mensaje */
```

```
      cptr->credits--;                    /* cada mensaje consume un crédito */
      cptr->state = ESTABLISHED;
      return(OK);
   } else {
      cptr->state = ESTABLISHED;
      return(ERR_CLOSED);                 /* falló transmisión: el igual quiere desconectarse */
   }
}

int receive(int cid, unsigned char bufptr[], int *bytes)
{ /* Usuario preparado para recibir un mensaje. */
  struct conn *cptr = &conn[cid];

  if (cptr->clr_req_received == 0) {
      /* Conexión aún establecida; trata de recibir. */
      cptr->state = RECEIVING;
      cptr->user_buf_addr = bufptr;
      cptr->byte_count = 0;
      data[0] = CRED;
      data[1] = 1;
      to_net(cid, 1, 0, CREDIT, data 2);  /* envía crédito */
      sleep( );                           /* se bloquea esperando datos */
      *bytes = cptr->byte_count;
  }
  cptr->state = ESTABLISHED;
  return(cptr->clr_req_received ? ERR_CLOSED : OK);
}

int disconnect(int cid)
{ /* El usuario quiere liberar una conexión. */
  struct conn  *cptr = &conn[cid];

  if (cptr->clr_req_received) {           /* el otro lado inició la terminación */
      cptr->state = IDLE;                 /* ahora se libera la conexión */
      to_net(cid, 0, 0, CLEAR_CONF, data, 0);
  } else {                                /* iniciamos la terminación */
      cptr-> state = DISCONN;             /* no liberada hasta que el otro lado esté de acuerdo */
      to_net(cid, 0, 0, CLEAR_REQ, data, 0);
  }
  return(OK);
}

void packet_arrival(void)
{ /* Ha llegado un paquete, obtenerlo y procesarlo. */
  int cid;                                /* conexión por la que llegó el paquete */
  int count, i, q, m;
  pkt_type ptype;   /* CALL_REQ, CALL_ACC, CLEAR_REQ, CLEAR_CONF, DATA_PKT, CREDIT */
  unsigned char data[MAX_PKT_SIZE];       /* parte de datos del paquete de entrada */
  struct conn *cptr;

  from_net(&cid, &q, &m, &ptype, data, &count);   /* obténlo */
  cptr = &conn[cid];
```



```
switch (ptype) {
  case CALL_REQ:                          /* usuario remoto quiere establecer una conexión */
      cptr->local_address = data[0];  cptr->remote_address = data[1];
      if (cptr->local_address == listen_address) {
          listen_conn = cid;  cptr->state = ESTABLISHED;  wakeup();
      } else {
          cptr->state = QUEUED;  cptr->timer = TIMEOUT;
      }
      cptr->clr_req_received = 0;  cptr->credits = 0;

  case CALL_ACC:                          /* usuario remoto ha aceptado nuestra CALL_REQ */
      cptr->state = ESTABLISHED;
      wakeup();

  case CLEAR_REQ:                         /* el usuario remoto quiere desconectarse o rechazar llamada */
      cptr->clr_req_received = 1;
      if (cptr->state == DISCONN) cptr->state = IDLE;   /* restaura colisión */
      if (cptr->state == WAITING || cptr->state == RECEIVING || cptr->state == SENDING) wakeup();

  case CLEAR_CONF:                        /* usuario remoto está de acuerdo en desconectar */
      cptr->state = IDLE;

  case CREDIT:                            /* usuario remoto espera datos */
      cptr->credits += data[1];
      if (cptr->state == SENDING) wakeup();

  case DATA_PKT:                          /* usuario remoto envió datos */
      for (i = 0; i < count; i++) cptr->user_buf_addr[cptr->byte_count + i] = data[i];
      cptr->byte_count += count;
      if (m == 0) wakeup();
  }
}

void clock(void)
{ /* Pulsó el reloj, ver si terminó la temporización de solicitudes de conexión en cola. */
  int i;
  struct conn *cptr;

  for (i = 1; i <= MAX_CONN; i++) {
      cptr = &conn[i];
      if (cptr->timer > 0) {              /* temporizador estaba operando */
          cptr->timer--;
          if (cptr->timer == 0) {         /* temporizador expiró */
              cptr->state = IDLE;
              to_net(i, 0, 0, CLEAR_REQ, data 0);
          }
      }
  }
}
```

Figura 6-20. Entidad de transporte de ejemplo.

Los procedimientos mostrados en la figura 6-20 son de dos tipos. Casi todos pueden invocarse directamente desde los programas de usuario. Sin embargo, *packet_arrival* y *clock* son diferentes; se disparan espontáneamente por eventos externos: la llegada de un paquete y los pulsos del reloj, respectivamente. De hecho, son rutinas de interrupción. Supondremos que no se invocan mientras está ejecutándose un procedimiento de entidad de transporte; sólo pueden invocarse cuando el proceso de usuario está dormido o ejecutándose fuera de la entidad de transporte. Esta propiedad es crucial para el funcionamiento correcto de la entidad de transporte.

La existencia del bit Q (calificador) en la cabecera del paquete nos permite evitar la carga extra de una cabecera de protocolo de transporte. Los mensajes de datos ordinarios se envían como paquetes de datos con $Q = 0$. Los mensajes de control de protocolo de transporte, de los cuales sólo hay uno (CREDIT) en nuestro ejemplo, se envían como paquetes de datos con $Q = 1$. Estos mensajes de control son detectados y procesados por la entidad de transporte receptora.

La principal estructura de datos usada por la entidad de transporte es el arreglo *conn*, que tiene un registro por cada conexión potencial. El registro mantiene el estado de la conexión, incluidas las direcciones de transporte en ambos extremos, la cantidad de mensajes enviados y recibidos por la conexión, el estado actual, el apuntador al *buffer* de usuario, la cantidad de bytes de los mensajes actuales enviados o recibidos hasta el momento, un bit que indica si el usuario remoto ha emitido o no un DISCONNECT, un temporizador y un contador de permisos que sirve para habilitar el envío de mensajes. No todos estos campos se usan en nuestro ejemplo sencillo, pero una entidad de transporte completa los necesitaría todos, y tal vez más. Se supone que cada entrada de *conn* está inicialmente en el estado inactivo (*IDLE*).

Cuando el usuario llama a CONNECT, la capa de red recibe instrucciones para enviar un paquete CALL REQUEST (solicitud de llamada) a la máquina remota, y el usuario se pone a dormir. Cuando llega el paquete CALL REQUEST al otro lado, la entidad de transporte se interrumpe para ejecutar *packet_arrival* a fin de comprobar que el usuario local está escuchando en la dirección especificada. De ser así, se devuelve un paquete CALL ACCEPTED (llamada aceptada) y se despierta al usuario remoto; de no ser así, se encola la CALL REQUEST durante *TIMEOUT* pulsos de reloj. Si se hace un LISTEN en este periodo, se establece la conexión; de otro modo, termina la temporización y se rechaza la solicitud con un paquete CLEAR REQUEST (borrar solicitud). Este mecanismo es necesario para evitar que el iniciador se bloquee eternamente en el caso de que un proceso remoto no quiera conectarse con él.

Aunque hemos eliminado la cabecera del protocolo de transporte, aún necesitamos una manera de llevar el control de cuál paquete pertenece a cuál conexión de transporte, ya que pueden existir varias conexiones al mismo tiempo. El enfoque más sencillo es usar el número de circuito virtual de la capa de red también como número de conexión de transporte. Es más, puede usarse también el número de circuito virtual como índice del arreglo *conn*. Al llegar un paquete por el circuito virtual k de la capa de red, pertenecerá a la conexión de transporte k, cuyo estado está en el registro *conn*[k]. En las conexiones iniciadas en un *host*, la entidad de transporte originadora selecciona el número de conexión. En las llamadas entrantes, la capa de red hace la selección, escogiendo cualquier número de circuito virtual no usado.

Para evitar tener que proporcionar y administrar *buffers* en la entidad de transporte, se usa aquí un mecanismo de control de flujo diferente de la ventana corrediza tradicional. Cuando un

usuario llama a RECEIVE, se envía un **mensaje de crédito** especial a la entidad de transporte de la máquina transmisora y se registra en el arreglo *conn*. Al llamar a SEND, la entidad de transporte revisa si ha llegado un crédito por la conexión especificada. De ser así, el mensaje se envía (en paquetes múltiples de ser necesario) y se disminuye el crédito; en caso contrario, la entidad de transporte se pone a dormir hasta la llegada de un crédito. Este mecanismo garantiza que no se enviará nunca un mensaje a menos que el otro lado ya haya hecho un RECEIVE. Como resultado, siempre que llega un mensaje hay la garantía de un *buffer* disponible en el cual puede ponerse. El esquema puede generalizarse fácilmente para permitir que los receptores proporcionen múltiples *buffers* y soliciten múltiples mensajes.

Debe tenerse presente la sencillez de la figura 6-20. Una entidad de transporte realista normalmente comprobaría la validez de todos los parámetros proporcionados por el usuario, manejaría la recuperación de caídas de la capa de red, se encargaría de colisiones de llamadas, y apoyaría un servicio de red más general, incluidas funciones como interrupciones, datagramas y versiones no bloqueadoras de las primitivas SEND y RECEIVE.

6.3.3. El ejemplo como máquina de estados finitos

Escribir una entidad de transporte es un trabajo difícil y exigente, especialmente para los protocolos más realistas. A fin de reducir la posibilidad de cometer errores, podemos representar el estado del protocolo como máquina de estados finitos.

Ya hemos visto que nuestro protocolo de ejemplo tiene siete estados por conexión. También es posible aislar 12 eventos que pueden ocurrir para pasar una conexión de un estado a otro. Cinco de estos eventos son las cinco primitivas de servicio. Otros seis son las llegadas de los seis tipos de paquete legales. El último es la expiración del temporizador. En la figura 6-21 se muestran las acciones principales del protocolo en forma de matriz. Las columnas son los estados y las filas son los 12 eventos.

Cada entrada de la matriz (es decir, la máquina de estados finitos) de la figura 6-21 tiene hasta tres campos: un predicado, una acción y un estado nuevo. El predicado indica en qué condiciones se emprende la acción. Por ejemplo, en la entrada de la esquina superior izquierda, si se ejecuta una LISTEN y no hay más espacio de tabla (predicado *P1*) falla la LISTEN y no cambia el estado. Por otra parte, si ya ha llegado un paquete CALL REQUEST para la dirección de transporte en la que se está escuchando (predicado *P2*), se establece de inmediato la conexión. Otra posibilidad es que *P2* sea falso, es decir, que no ha entrado ningún CALL REQUEST, en cuyo caso la conexión permanece en el estado *IDLE*, esperando un paquete CALL REQUEST.

Es importante indicar que la selección de estados a usar en la matriz no es fijada completamente por el protocolo mismo. En este ejemplo, no hay ningún estado *LISTENING* (escuchando), lo que sería algo razonable después de una LISTEN. No hay estado *LISTENING* porque un estado se asocia a un registro de conexión, y no se crea un registro de conexión mediante LISTEN. ¿Por qué no? Porque hemos decidido usar los números de circuito virtual de la capa de red como identificadores de la conexión, y para una LISTEN el número de circuito virtual es escogido por la capa de red al llegar el paquete CALL REQUEST.

Estado

		En reposo	Espera	Encolado	Establecido	Transmitiendo	Recibiendo	Desconexión
Primitivas	LISTEN	P1: ~/Reposo P2: A1/Estab. $\overline{P2}$: A2/Reposo		~/Estab.				
	CONNECT	P1: ~/Reposo $\overline{P1}$: A3/Espera						
	DISCONNECT				$\overline{P4}$: A5/Reposo $\overline{P4}$: A6/Desc.			
	SEND				P5: A7/Estab. $\overline{P5}$: A8/Envío			
	RECEIVE				A9/Recibe			
Paquetes de entrada	Call_req	P3: A1/Estab. $\overline{P3}$: A4/Encolado						
	Call_acc		~/Estab.					
	Clear_req		~/Reposo		A10/Estab.	A10/Estab.	A10/Estab.	~/Reposo
	Clear_conf							~/Reposo
	DataPkt						A12/Estab.	
	Credit				A11/Estab.	A7/Estab.		
Reloj	Timeout			~/Reposo				

Predicados	Acciones	
P1: Tabla de conexiones llena	A1: Envía Call_acc	A7: Envía mensaje
P2: Call_req pendiente	A2: Espera Call_req	A8: Espera crédito
P3: LISTEN pendiente	A3: Envía Call_req	A9: Envía crédito
P4: Clear_req pendiente	A4: Inicia temporizador	A10: Establece indicador Clr_req_received
P5: Crédito disponible	A5: Envía Clear_conf	A11: Registra crédito
	A6: Envía Clear_req	A12: Acepta mensaje

Figura 6-21. El protocolo de ejemplo como máquina de estados finitos. Cada entrada tiene un predicado opcional, una acción opcional y el estado nuevo. La tilde indica que no se realizó ninguna acción importante. Una barra sobre un predicado indica la negación del predicado. Las entradas en blanco corresponden a eventos imposibles o no válidos.

Las acciones *A1* a *A12* son las acciones principales, como el envío de paquetes y el inicio de temporizadores. No se listan todas las acciones menores, como la preparación inicial de los campos de un registro de conexión. Si una acción comprende despertar un proceso dormido, las

acciones que siguen a su despertar también cuentan. Por ejemplo, si entra un paquete CALL REQUEST y un proceso estaba dormido esperándolo, la transmisión del paquete CALL ACCEPT que sigue al despertar cuenta como parte de la acción de CALL REQUEST. Tras la ejecución de cada acción, la conexión puede pasar a un estado nuevo, como se muestra en la figura 6-21.

La ventaja de representar el protocolo como una matriz es triple. Primero, de esta manera es mucho más fácil que el programador revise sistemáticamente cada combinación de estado y evento para ver si se requiere una acción. En las implementaciones de producción se usarían algunas de las combinaciones para manejo de errores. En la figura 6-21 no se hace distinción entre situaciones imposibles e ilegales. Por ejemplo, si una conexión está en el estado *waiting*, el evento DISCONNECT es imposible porque el usuario está bloqueado y no puede ejecutar primitivas. Por otra parte, en el estado *sending*, no se esperan paquetes de datos porque no se ha emitido crédito. La llegada de un paquete de datos es un error del protocolo.

La segunda ventaja de la representación en matriz del protocolo es en su implementación. Podemos imaginar un arreglo de dos dimensiones en el que el elemento $a[i][j]$ es un apuntador o índice al procedimiento que maneja el suceso del evento i cuando se está en el estado j. Una posible puesta en práctica es escribir la entidad de transporte como ciclo corto, esperando un evento en la parte superior del ciclo. Al ocurrir un evento, se localiza la conexión pertinente y se extrae su estado. Sabiendo ahora el evento y el estado, la entidad de transporte simplemente los indiza en el arreglo a e invoca el procedimiento adecuado. Este enfoque produce un diseño mucho más regular y sistemático que nuestra entidad de transporte.

La tercera ventaja del enfoque de máquina de estados finitos se aprecia en la descripción del protocolo. En algunos documentos de estándares, se dan los protocolos como máquinas de estados finitos del tipo de la figura 6-21. Pasar de este tipo de descripción a una entidad de transporte operante es mucho más fácil si la entidad de transporte también es operada por una máquina de estados finitos basada en la del estándar.

La desventaja principal del enfoque de máquina de estados finitos es que puede ser más difícil de entender que el ejemplo de programación directa que usamos inicialmente. Sin embargo, puede resolverse parcialmente este problema dibujando la máquina de estados finitos en forma de grafo, como se hace en la figura 6-22.

6.4. LOS PROTOCOLOS DE TRANSPORTE DE INTERNET (TCP Y UDP)

La Internet tiene dos protocolos principales en la capa de transporte, un protocolo orientado a conexiones y uno sin conexiones. En las siguientes secciones estudiaremos ambos. El protocolo orientado a conexiones es el TCP; el protocolo sin conexiones es el UDP. Dado que el UDP es básicamente el IP con la adición de una cabecera corta, nos enfocaremos al TCP.

El **TCP** (*Transmission Control Protocol*, **protocolo de control de transmisión**) se diseñó específicamente para proporcionar una corriente de bytes confiable a través de una interred no confiable. Una interred es diferente de una sola red porque las distintas partes pueden tener

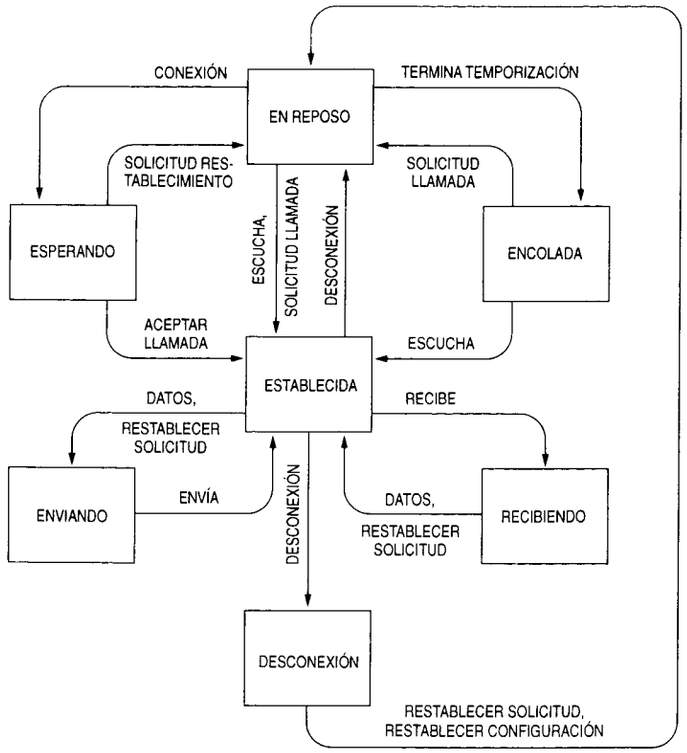


Figura 6-22. El protocolo de ejemplo en forma gráfica. Por sencillez, se han omitido las transiciones que dejan sin cambio el estado de las conexiones.

topologías, anchos de banda, retardos, tamaños de paquete y otros parámetros con grandes diferencias. Se diseñó el TCP para adaptarse dinámicamente a las propiedades de la interred y para ser robusto ante muchos tipos de fallas.

El TCP se definió formalmente en el RFC 793. A medida que pasó el tiempo, se detectaron varios errores e inconsistencias, y se cambiaron los requisitos de algunas áreas. Estas clarificaciones y algunas correcciones de fallas se detallan en el RFC 1122. En el RFC 1323 se presentan las extensiones.

Cada máquina que reconoce el TCP tiene una entidad de transporte TCP, ya sea un proceso de usuario o una parte del núcleo que maneja las corrientes TCP y tiene interfaz con la capa IP. Una entidad TCP acepta corrientes de datos de usuario de los procesos locales, los divide en partes que no exceden 64K bytes (en la práctica, generalmente de unos 1500 bytes), y envía cada parte como datagrama IP independiente. Cuando llegan a una máquina datagramas IP que contienen datos TCP, son entregados a la entidad TCP, que reconstruye las corrientes originales de bytes. Por sencillez, a veces sólo usaremos "TCP" para indicar la entidad de transporte TCP (una parte del *software*) o el protocolo TCP (un grupo de reglas). Por el contexto quedará claro a

qué nos estamos refiriendo. Por ejemplo, en "el usuario proporciona al TCP los datos", claramente nos estamos refiriendo a la entidad de transporte TCP.

La capa IP no ofrece ninguna garantía de que los datagramas se entregarán adecuadamente, por lo que es responsabilidad del TCP terminar de temporizar y retransmitirlos según se necesite. Los datagramas que sí llegan pueden hacerlo desordenadamente; también es responsabilidad del TCP reensamblarlos en mensajes con la secuencia adecuada. En pocas palabras, el TCP debe proveer la confiabilidad que la mayoría de los usuarios quiere y que el IP no proporciona.

6.4.1. Modelo de servicio TCP

El servicio TCP se obtiene haciendo que tanto el transmisor como el receptor creen puntos terminales, llamados sockets, como se explicó en la sección 6.1.3. Cada socket tiene un número (dirección) de socket que consiste en la dirección IP del *host* y en un número de 16 bits local a ese *host*, llamado **puerto**. Puerto es el nombre en TCP de un TSAP. Para obtener el servicio TCP, debe establecerse explícitamente una conexión entre un socket de la máquina transmisora y un socket de la máquina receptora. Las llamadas de socket se listan en la figura 6-6.

Puede usarse un socket para varias conexiones al mismo tiempo. En otras palabras, dos o más conexiones pueden terminar en el mismo socket. Las conexiones se identifican mediante los identificadores de ambas terminales, es decir (*socket1, socket2*). No se usan números de circuito virtual ni ningún otro identificador.

Los números de puerto por debajo del 256 se llaman **puertos bien conocidos** (*well-known ports*) y se reservan para servicios estándar. Por ejemplo, cualquier proceso que desee establecer una conexión a un *host* para transferir un archivo usando FTP puede conectarse al puerto 21 del *host* de destino para comunicarse con su *daemon* (proceso reentrante) de FTP. De la misma manera, para establecer una sesión interactiva remota usando TELNET, se usa el puerto 23. La lista de puertos bien conocidos se da en el RFC 1700.

Todas las conexiones TCP son dúplex integral y punto a punto. Dúplex integral significa que el tráfico puede ir en ambos sentidos al mismo tiempo. Punto a punto significa que cada conexión tiene exactamente dos puntos terminales. El TCP no reconoce la multitransmisión ni la difusión.

Una conexión TCP es una corriente de bytes, no una corriente de mensajes. Los límites de mensaje no se conservan de extremo a extremo. Por ejemplo, si el proceso transmisor hace cuatro escrituras de 512 bytes en una corriente TCP, estos datos pueden entregarse al proceso receptor como cuatro bloques de 512 bytes, dos bloques de 1024 bytes, un bloque de 2048 bytes (véase la figura 6-23), o de algún otro modo. No hay manera de que el receptor detecte las unidades en las que se escribieron los datos.

Los archivos de UNIX tienen también esta propiedad. El lector de un archivo no puede saber si el archivo se escribió bloque por bloque, byte por byte, o todo de golpe. Como sucede con los archivos UNIX, el *software* del TCP no tiene idea de lo que significan los bytes y no tiene interés en averiguarlo. Un byte simplemente es un byte.

Cuando una aplicación pasa datos al TCP, el TCP puede enviarlos de inmediato o guardarlos en un *buffer* (a fin de reunir una cantidad mayor de información para enviarla junta), a discreción

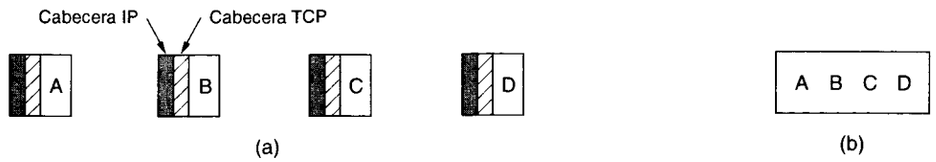


Figura 6-23. (a) Cuatro segmentos de 512 bytes enviados como datagramas IP independientes. (b) Los 2048 bytes de datos entregados a la aplicación con una sola llamada READ.

propia. Sin embargo, a veces la aplicación simplemente quiere que los datos se envíen de inmediato. Por ejemplo, supóngase que un usuario está en sesión con una máquina remota. Tras la terminación de una línea de comandos y el retorno de carro, es esencial que la línea se envíe a la máquina remota de inmediato y no se guarde en *buffer* hasta la entrada de la siguiente línea. Para obligar la salida de datos, las aplicaciones pueden usar la bandera *(flag)* PUSH que ordena al TCP no retrasar la transmisión.

Algunas de las primeras aplicaciones usaron la bandera PUSH como una especie de marcador para delinear los límites de los mensajes. Aunque este truco a veces funciona, en ocasiones falla porque no todas las implementaciones de TCP pasan la bandera PUSH a la aplicación del lado receptor. Es más, si llegan otros PUSH adicionales antes de la transmisión del primero (por ejemplo, por estar ocupada la línea de salida), el TCP está en libertad de juntar todos los datos que entran por PUSH en un solo datagrama IP, sin separación entre las diferentes partes.

Una última característica del servicio TCP que vale la pena mencionar son los **datos urgentes**. Cuando un usuario interactivo pulsa la tecla DEL o CTRL-C para interrupir un cómputo remoto ya iniciado, la aplicación transmisora pone cierta información de control en la corriente de datos y se la da al TCP junto con la bandera URGENT. Este evento hace que el TCP deje de acumular datos y transmita de inmediato todo lo que tiene para esa conexión.

Al recibirse los datos urgentes en el destino, se interrumpe la aplicación receptora (por ejemplo, recibe una señal en términos de UNIX), para que la aplicación pueda detener lo que estaba haciendo y leer la corriente de datos hasta encontrar los datos urgentes. Se marca el fin de los datos urgentes, por lo que la aplicación sabe cuándo han terminado. El comienzo de los datos urgentes no se marca; es responsabilidad de la aplicación averiguarlo. Este esquema básicamente proporciona un mecanismo burdo de señalamiento, y deja todo lo demás en manos de la aplicación.

6.4.2. El protocolo TCP

En esta sección daremos un repaso general del protocolo TCP; en la siguiente veremos la cabecera del protocolo, campo por campo. Cada byte de una conexión TCP tiene su propio número de secuencia de 32 bits. En un *host* que opera a toda velocidad en una LAN de 10 Mbps,

en teoría los números de secuencia podrían volver a comenzar en una hora, pero en la práctica tarda mucho más tiempo. Se usan los números de secuencia tanto para los acuses de recibo como para el mecanismo de ventana, que utilizan campos de cabecera de 32 bits distintos.

La entidad TCP transmisora y la receptora intercambian datos en forma de segmentos. Un **segmento** consiste en una cabecera TCP fija de 20 bytes (más una parte opcional) seguida de cero o más bytes de datos. El *software* de TCP decide el tamaño de los segmentos; puede acumular datos de varias escrituras para formar un segmento, o dividir los datos de una escritura en varios segmentos. Hay dos límites que restringen el tamaño de segmento. Primero, cada segmento, incluida la cabecera TCP, debe caber en la carga útil de 65,535 bytes del IP. Segundo, cada red tiene una **unidad máxima de transferencia**, o **MTU** (*maximum transfer unit*), y cada segmento debe caber en la MTU. En la práctica, las MTU generalmente son de unos cuantos miles de bytes y por tanto definen el límite superior del tamaño de segmento. Si un segmento pasa a través de una serie de redes sin fragmentarse y luego se topa con una cuya MTU es menor que el segmento, el enrutador de la frontera fragmenta el segmento en dos o más segmentos más pequeños.

Un segmento demasiado grande para transitar por una red puede dividirse en varios segmentos mediante un enrutador. Cada segmento nuevo recibe sus propias cabeceras TCP e IP, por lo que la fragmentación en los enrutadores aumenta la carga extra total (puesto que cada segmento adicional agrega 40 bytes de información de cabecera).

El protocolo básico usado por las entidades TCP es el protocolo de ventana corrediza. Cuando un transmisor envía un segmento, también inicia un temporizador. Cuando llega el segmento al destino, la entidad TCP receptora devuelve un segmento (con datos, si existen, de otro modo sin ellos) que contiene un número de acuse de recibo igual al siguiente número de secuencia que espera recibir. Si el temporizador del transmisor expira antes de la recepción del acuse de recibo, el transmisor envía de nuevo el segmento.

Aunque este protocolo suena sencillo, tiene muchos vericuetos que cubriremos a continuación. Por ejemplo, dado que los segmentos pueden fragmentarse, es posible que llegue una parte del segmento transmitido y que la entidad TCP receptora envíe un acuse de recibo, pero la otra parte se pierda. También pueden llegar segmentos fuera de orden, por lo que los bytes 3072-4095 podrían llegar pero no enviarse acuse de recibo porque los bytes 2048-3071 no han aparecido aún. También pueden retardarse segmentos en tránsito durante tanto tiempo que el transmisor termina de temporizar y retransmite nuevamente. Si un segmento retransmitido toma una ruta distinta a la del original y se fragmenta de manera diferente, pueden llegar esporádicamente partes tanto del original como del duplicado, requiriéndose una administración cuidadosa para lograr una corriente de bits confiable. Por último, siendo tantas las redes que constituyen la Internet, es posible que un segmento pueda toparse ocasionalmente con una red congestionada (o rota) en alguna parte de su trayectoria.

El TCP debe estar preparado para manejar y resolver estos problemas de una manera eficiente. Se ha invertido una cantidad considerable de esfuerzo en la optimización del desempeño de las corrientes TCP, incluso ante problemas de red. A continuación se estudiarán varios de los algoritmos usados por muchas implementaciones de TCP.

6.4.3. La cabecera de segmento TCP

En la figura 6-24 se muestra la distribución de un segmento TCP. Cada segmento comienza con una cabecera de formato fijo de 20 bytes. La cabecera fija puede ir seguida de opciones de cabecera. Tras las opciones, si las hay, pueden continuar hasta 65,535 – 20 – 20 = 65,515 bytes de datos, donde los primeros 20 se refieren a la cabecera IP y los segundos a la cabecera TCP. Los segmentos sin datos son legales y se usan por lo común para acuses de recibo y mensajes de control.

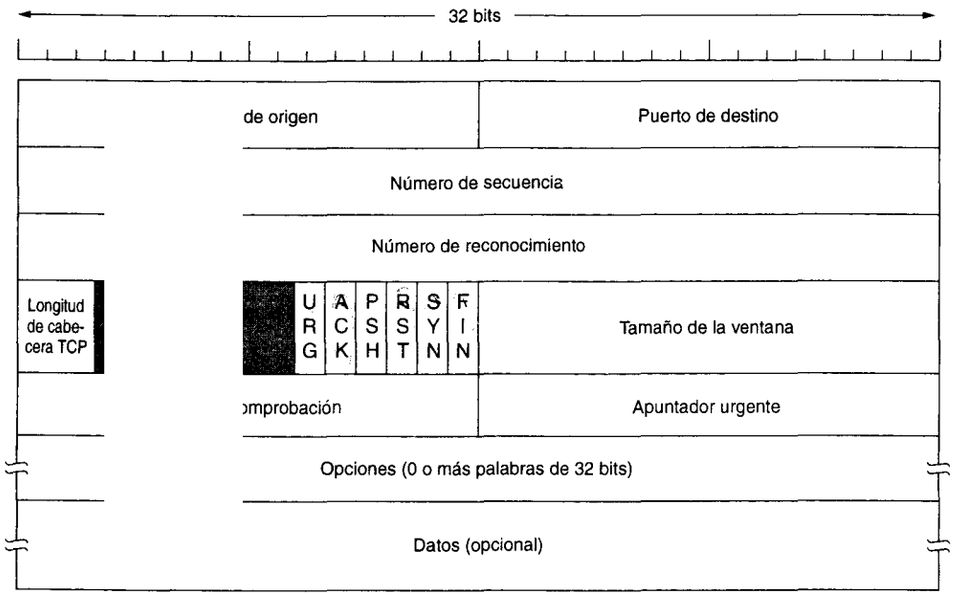


Figura 6-24. Cabecera TCP.

Realicemos la disección de la cabecera TCP campo por campo. Los campos de *puerto de origen* y *puerto de destino* identifican los puntos terminales locales de la conexión. Cada *host* puede decidir por sí mismo la manera de asignar sus propios puertos comenzando por el 256. La dirección de un puerto más la dirección IP de su *host* forman un TSAP único de 48 bits. Los números de socket de origen y de destino en conjunto identifican la conexión.

Los campos de *número de secuencia* y *número de acuse de recibo* desempeñan sus funciones normales. Nótese que el segundo especifica el siguiente byte esperado, no el último byte correctamente recibido. Ambos tienen 32 bits de longitud porque cada byte de datos está numerado en una corriente TCP.

La *longitud de cabecera TCP* indica la cantidad de palabras de 32 bits contenidas en la cabecera TCP. Esta información es necesaria porque el campo de *opciones* es de longitud

variable, por lo que la cabecera también. Técnicamente, este campo en realidad indica el comienzo de los datos en el segmento, medido en palabras de 32 bits, pero ese número es simplemente la longitud de la cabecera en palabras, por lo que el efecto es el mismo.

A continuación viene un campo de 6 bits que no se usan. El que este campo haya sobrevivido intacto durante más de una década es testimonio de lo bien pensado que está el TCP. Protocolos menos bien hechos lo habrían necesitado para corregir errores del diseño original.

Ahora vienen seis banderas de 1 bit. *URG* se establece en 1 si está en uso el *apuntador urgente*. El *apuntador urgente* sirve para indicar un desplazamiento en bytes a partir del número actual de secuencia en el que se encuentran datos urgentes. Este recurso sustituye los mensajes de interrupción. Como se mencionó antes, este recurso es un mecanismo rudimentario para permitir que el transmisor envíe una señal al receptor sin implicar al TCP en la razón de la interrupción.

El bit *ACK* se establece en 1 para indicar que el *número de acuse de recibo* es válido. Si el *ACK* es 0, el segmento no contiene un acuse de recibo, por lo que se ignora el campo de *número de acuse de recibo*.

El bit *PSH* indica datos empujados (con PUSH). Por este medio se solicita atentamente al receptor entregar los datos a la aplicación a su llegada y no ponerlos en *buffer* hasta la recepción de un *buffer* completo (lo que podría hacer en otras circunstancias por razones de eficiencia).

Se usa el bit *RST* para restablecer una conexión que se ha confundido debido a una caída de *host* u otra razón; también sirve para rechazar un segmento no válido o un intento de abrir una conexión. Por lo general, si usted recibe un segmento con el bit *RST* encendido, tiene un problema entre manos.

El bit *SYN* se usa para establecer conexiones. La solicitud de conexión tiene *SYN* = 1 y *ACK* = 0 para indicar que el campo de acuse de recibo incorporado no está en uso. La respuesta de conexión sí lleva un reconocimiento, por lo que tiene *SYN* = 1 y *ACK* = 1. En esencia, el bit *SYN* se usa para denotar CONNECTION REQUEST y CONNECTION ACCEPTED, usándose el bit *ACK* para distinguir entre ambas posibilidades.

El bit *FIN* se usa para liberar una conexión; especifica que el transmisor no tiene más datos que transmitir. Sin embargo, tras cerrar una conexión, un proceso puede continuar recibiendo datos indefinidamente. Ambos segmentos, *SYN* y *FIN*, tienen números de secuencia y por tanto tienen la garantía de procesarse en el orden correcto.

El control de flujo en el TCP se maneja usando una ventana corrediza de tamaño variable. El campo de *ventana* indica la cantidad de bytes que pueden enviarse comenzando por el byte que ya se ha enviado acuse de recibo. Es válido un campo de *ventana* de 0, e indica que se han recibido los bytes hasta *número de acuse de recibo* – 1, inclusive, pero que el receptor actualmente necesita un descanso y quisiera no recibir más datos por el momento, gracias. El permiso para enviar puede otorgarse después enviando un segmento con el mismo *número de acuse de recibo* y un campo de *ventana* distinto de cero.

También se proporciona una *suma de comprobación* para confiabilidad extrema. Es una suma de comprobación de la cabecera, los datos y la pseudocabecera conceptual mostrada en la figura 6-25. Al realizar este cálculo, se establece el campo de *suma de comprobación* del TCP en cero, y se rellena el campo de datos con un byte cero adicional si la longitud es un número non. El algoritmo de suma de comprobación simplemente suma todas las palabras de 16 bits en

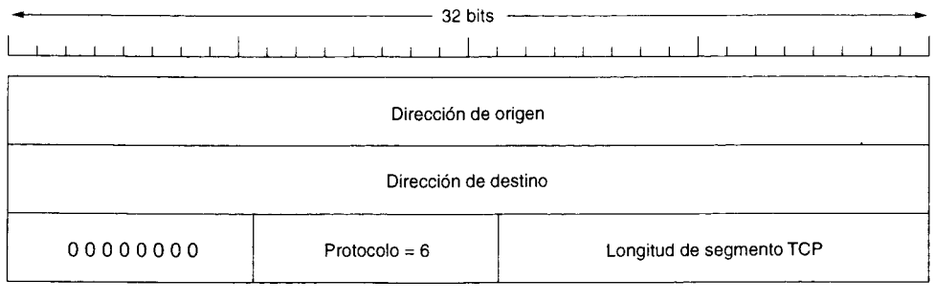


Figura 6-25. Pseudocabecera incluida en la suma de comprobación del TCP.

complemento a 1 y luego obtiene el complemento a 1 de la suma. Como consecuencia, al realizar el cálculo el receptor con el segmento completo, incluido el campo de *suma de comprobación*, el resultado debe ser 0.

La pseudocabecera contiene las direcciones IP de 32 bits de las máquinas de origen y de destino, el número de protocolo de TCP (6), y la cuenta de bytes del segmento TCP (incluida la cabecera). La inclusión de la pseudocabecera en el cálculo de la suma de comprobación TCP ayuda a detectar paquetes mal entregados, pero hacerlo viola la jerarquía de protocolos puesto que las direcciones de IP que contiene pertenecen a la capa IP, no a la capa TCP.

El campo de *opciones* se diseñó para contar con una manera de agregar características extra no cubiertas por la cabecera normal. La opción más importante es la que permite que cada *host* especifique la carga útil TCP máxima que está dispuesto a aceptar. El uso de segmentos grandes es más eficiente que el de segmentos pequeños, puesto que la cabecera de 20 bytes puede entonces amortizarse entre más datos, pero los *hosts* pequeños tal vez no puedan manejar segmentos muy grandes. Durante el establecimiento de la conexión, cada lado puede anunciar su máximo y ver el de su compañero. El más pequeño de los dos números es el ganador. Si un *host* no usa esta opción, predetermina una carga útil de 536 bytes. Se requiere que todos los *hosts* Internet acepten segmentos TCP de 536 + 20 = 556 bytes.

En las líneas con alto ancho de banda, alto retardo o ambas cosas, la ventana de 64 KB con frecuencia es un problema. En una línea T3 (44.736 Mbps) se requieren sólo 12 mseg para enviar una ventana completa de 64 KB. Si el retardo de propagación de ida y vuelta es de 50 mseg (típico de una fibra transcontinental), el transmisor estará inactivo 3/4 del tiempo en espera de acuses de recibo. En una conexión satelital la situación es peor aún. Un tamaño de ventana más grande permitirá al transmisor continuar enviando datos, pero como el campo de *tamaño de ventana* es de 16 bits, es imposible expresar tal tamaño. En el RFC 1323 se propuso una opción de *escala de ventana* que permite al transmisor y al receptor negociar un factor de escala de ventana. Este número permite que ambos lados desplacen el campo de tamaño de ventana hasta 16 bits a la izquierda, permitiendo por tanto un máximo de 2^{32} bytes. La mayoría de las implementaciones actuales de TCP manejan esta opción.

Otra opción propuesta en el RFC 1106 y ahora de uso difundido es el empleo de la repetición selectiva en lugar del protocolo de regresar *n* (*go back n*). Si el receptor recibe un segmento malo

y luego una gran cantidad de segmentos buenos, el protocolo TCP normal en algún momento terminará de temporizar y retransmitirá todos los segmentos sin acuse de recibo, incluidos los que se recibieron correctamente. El RFC 1106 introdujo los NAK, para permitir que el receptor solicite un segmento (o segmentos) específico. Tras recibirlo, puede enviar un acuse de recibo de todos los datos que tiene en *buffer*, reduciendo de esta manera la cantidad de datos retransmitidos.

6.4.4. Gestión de una conexión TCP

En el TCP se establecen las conexiones usando el protocolo de acuerdo de tres vías (*three-way handshake*) estudiado en la sección 6.2.2. Para establecer una conexión, un lado, digamos el servidor, espera pasivamente una conexión entrante ejecutando las primitivas LISTEN y ACCEPT y especificando cierto origen o bien nadie en particular.

El otro lado, digamos el cliente, ejecuta una primitiva CONNECT especificando la dirección y el puerto IP con el que se desea conectar, el tamaño máximo de segmento TCP que está dispuesto a aceptar y opcionalmente algunos datos de usuario (por ejemplo, una contraseña). La primitiva CONNECT envía un segmento TCP con el bit *SYN* encendido y el bit *ACK* apagado, y espera una respuesta.

Al llegar el segmento al destino, la entidad TCP ahí revisa si hay un proceso que haya ejecutado un LISTEN en el puerto indicado en el campo de *puerto de destino*. Si no lo hay, envía una contestación con el bit *RST* encendido para rechazar la conexión.

Si algún proceso está escuchando en el puerto, ese proceso recibe el segmento TCP entrante y puede entonces aceptar o rechazar la conexión; si la acepta, se devuelve un segmento de acuse de recibo. La secuencia de segmentos TCP enviados en el caso normal se muestra en la figura 6-26(a).

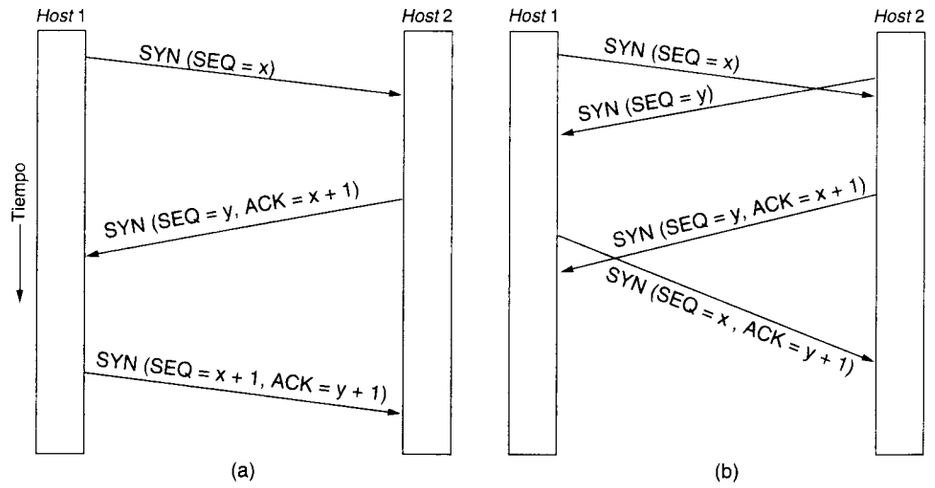


Figura 6-26. (a) Establecimiento de una conexión TCP en el caso normal. (b) Colisión de llamadas.

Nótese que un segmento *SYN* consume 1 byte de espacio de secuencia, por lo que puede reconocerse sin ambigüedades.

En el caso en que dos *hosts* intentan simultáneamente establecer una conexión entre los mismos dos sockets, la secuencia de eventos es la que se ilustra en la figura 6-26(b). El resultado de estos eventos es que sólo se establece una conexión, no dos, pues las conexiones se identifican por sus puntos terminales. Si el primer establecimiento resulta en una conexión identificada por (x, y), al igual que el segundo, sólo se hace una entrada de tabla, es decir, de (x, y).

El número de secuencia inicial de una conexión no es 0 por las razones que señalamos antes. Se usa un esquema basado en reloj, con un pulso de reloj cada 4 μseg. Por seguridad adicional, al caerse un *host*, no podrá reiniciarse durante el tiempo máximo de paquete (120 seg) para asegurar que no haya paquetes de conexiones previas vagando por Internet.

Aunque las conexiones TCP son dúplex integral, para entender la manera en que se liberan las conexiones es mejor visualizarlas como un par de conexiones símplex. Cada conexión símplex se libera independientemente de su igual. Para liberar una conexión, cualquiera de las partes puede enviar un segmento TCP con el bit *FIN* establecido, lo que significa que no tiene más datos por transmitir. Al reconocerse el *FIN*, ese sentido se apaga. Sin embargo, puede continuar un flujo de datos indefinido en el otro sentido. Cuando ambos sentidos se han apagado, se libera la conexión. Normalmente se requieren cuatro segmentos TCP para liberar una conexión, un *FIN* y un *ACK* para cada sentido. Sin embargo, es posible que el primer *ACK* y el segundo *FIN* estén contenidos en el mismo segmento, reduciendo la cuenta total a tres.

Al igual que con las llamadas telefónicas en las que ambas partes dicen adiós y cuelgan el teléfono simultáneamente, ambos extremos de una conexión TCP pueden enviar segmentos *FIN* al mismo tiempo. Ambos se reconocen de la manera normal, y se apaga la conexión. De hecho, en esencia no hay diferencia entre la liberación secuencial o simultánea por parte de los *hosts*.

Para evitar el problema de los dos ejércitos, se usan temporizadores. Si no llega una respuesta a un *FIN* en un máximo de dos tiempos de vida de paquete, el transmisor del *FIN* libera la conexión. Tarde o temprano el otro lado notará que, al parecer, ya nadie lo está escuchando, y también terminará su temporización. Aunque esta solución no es perfecta, dado el hecho de que teóricamente es imposible una solución perfecta tendremos que conformarnos con ella. En la práctica, pocas veces ocurren problemas.

Los pasos requeridos para establecer y liberar conexiones pueden representarse en una máquina de estados finitos con los 11 estados listados en la figura 6-27. En cada estado son legales ciertos eventos. Al ocurrir un evento legal, debe emprenderse alguna acción. Si ocurren otros eventos, se informa un error.

Cada conexión comienza en el estado *CLOSED* (cerrado) y deja ese estado cuando hace una apertura pasiva (LISTEN), o una apertura activa (CONNECT). Si el otro lado realiza la acción opuesta, se establece una conexión y el estado se vuelve *ESTABLISHED* (establecido). La liberación de la conexión puede iniciarse desde cualquiera de los dos lados. Al completarse, el estado regresa a *CLOSED*.

La máquina de estados finitos misma se muestra en la figura 6-28. El caso común de un cliente que se conecta activamente a un servidor pasivo se indica con líneas gruesas (continuas

Estado	Descripción
CLOSED	No hay conexión activa ni pendiente
LISTEN	El servidor espera una llamada
SYN RCVD	Llegó solicitud de conexión; espera ACK
SYN SENT	La aplicación comenzó a abrir una conexión
ESTABLISHED	Estado normal de transferencia de datos
FIN WAIT 1	La aplicación dijo que ya terminó
FIN WAIT 2	El otro lado acordó liberar
TIMED WAIT	Espera que todos los paquetes mueran
CLOSING	Ambos lados intentaron cerrar simultáneamente
CLOSE WAIT	El otro lado inició una liberación
LAST ACK	Espera que todos los paquetes mueran

Figura 6-27. Estados usados en la máquina de estados finitos de administración de conexiones TCP.

para el cliente, punteadas para el servidor). Las líneas delgadas son secuencias de eventos poco comunes. Cada línea de la figura 6-28 se marca mediante un par *evento/acción*. El evento puede ser una llamada de sistema iniciada por el usuario (CONNECT, LISTEN, SEND o CLOSE), la llegada de un segmento (*SYN, FIN, ACK* o *RST*) o, en un caso, una terminación de temporización del doble del tiempo de vida máximo del paquete. La acción es el envío de un segmento de control (*SYN, FIN* o *RST*), o nada, indicado por —. Los comentarios aparecen entre paréntesis.

El diagrama puede entenderse mejor siguiendo primero la trayectoria de un cliente (la línea continua gruesa) y luego la de un servidor (línea punteada gruesa). Al emitir una solicitud CONNECT una aplicación de la máquina cliente, la entidad TCP local crea un registro de conexión, lo marca para indicar que está en el estado *SYN SENT*, y envía un segmento *SYN*. Nótese que muchas conexiones pueden estar abiertas (o en proceso de apertura) al mismo tiempo como parte de varias aplicaciones, por lo que el estado es por conexión y se asienta en el registro de conexiones. Al llegar el *SYN+ACK*, el TCP envía el *ACK* final del protocolo de acuerdo de tres vías (*three-way handshake*) y se conmuta al estado *ESTABLISHED*. Ahora pueden enviarse y recibirse datos.

Al terminar una aplicación, ejecuta una primitiva CLOSE, que causa que la entidad TCP local envíe un segmento *FIN* y espere el *ACK* correspondiente (recuadro punteado rotulado "cierre activo"). Al llegar el *ACK*, se hace una transición al estado *FIN WAIT 2*, y ya está cerrado un sentido de la conexión. Cuando también cierra el otro lado, llega un *FIN*, para el cual se envía un acuse de recibo. Ahora ambos lados están cerrados, pero el TCP espera un tiempo igual al tiempo de vida máximo del paquete para garantizar que todos los paquetes de la conexión hayan muerto, como protección en caso de la pérdida de un acuse de recibo. Al expirar el temporizador, el TCP borra el registro de la conexión.

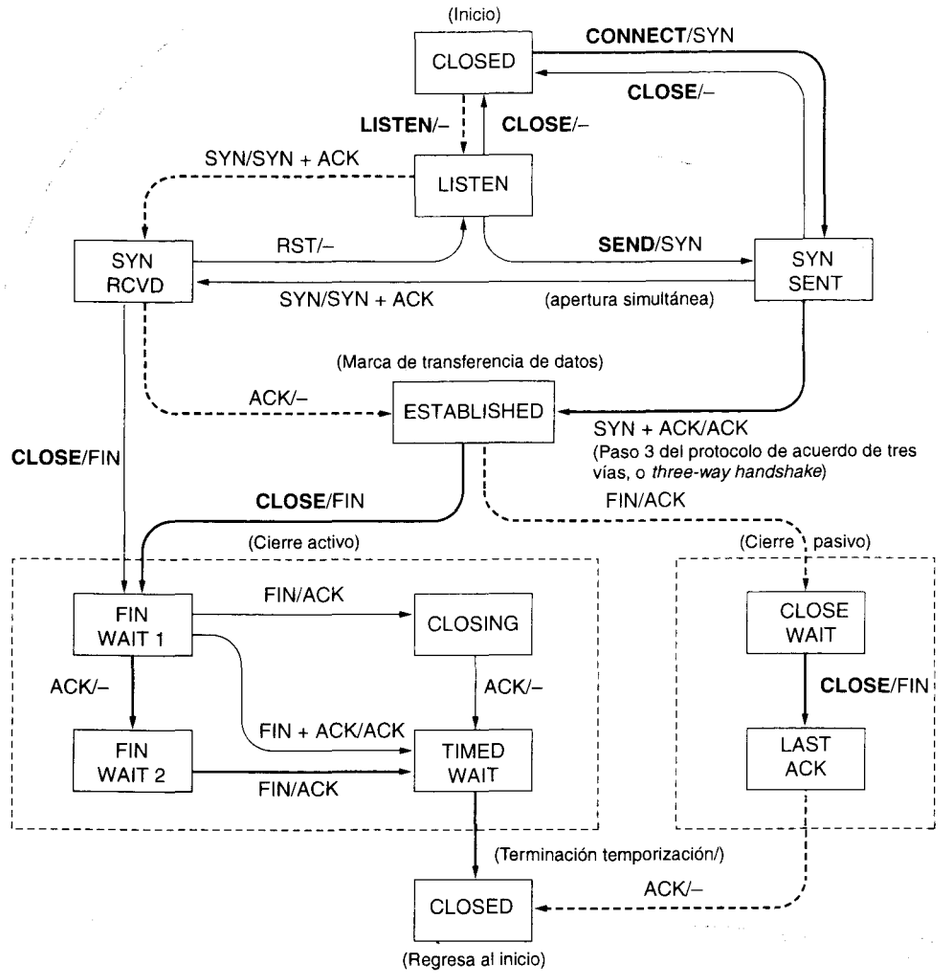


Figura 6-28. Máquina de estados finitos de administración de conexiones TCP. La línea continua gruesa es la trayectoria normal de un cliente. La línea punteada gruesa es la trayectoria normal de un servidor. Las líneas delgadas son eventos poco comunes.

Examinemos ahora la administración de la conexión desde el punto de vista del servidor. El servidor hace un LISTEN y se detiene a esperar la aparición de alguien. Al llegar un *SYN*, se envía un acuse de recibo y el servidor pasa al estado *SYN RCVD*. Cuando llega el acuse de recibo del *SYN* del servidor, el acuerdo de tres vías (*three-way handshake*) se ha completado y el servidor regresa al estado *ESTABLISHED*. Ahora puede ocurrir la transferencia de datos.

Cuando el cliente ha tenido suficiente, hace un CLOSE, que causa la llegada de un *FIN* al servidor (recuadro punteado rotulado "cierre pasivo"). Entonces se envía una señal al servidor.

Cuando éste también hace un CLOSE, se envía un *FIN* al cliente. Al llegar el acuse de recibo del cliente, el servidor libera la conexión y elimina el registro de conexión.

6.4.5. Política de transmisión del TCP

La administración de ventanas en el TCP no está atada directamente a los acuses de recibo como en la mayoría de los protocolos de enlace de datos. Por ejemplo, supóngase que el receptor tiene un *buffer* de 4096 bytes, como se muestra en la figura 6-29. Si el transmisor envía un segmento de 2048 bytes que se recibe correctamente, el receptor enviará el acuse de recibo del segmento. Sin embargo, dado que ahora sólo tiene 2048 de espacio de *buffer* (hasta que la aplicación retire algunos datos del *buffer*), anunciará una ventana de 2048 comenzando con el siguiente byte esperado.

Ahora el transmisor envía otros 2048 bytes, para los cuales el receptor envía el acuse de recibo, pero la ventana anunciada es de 0. El transmisor debe detenerse hasta que el proceso

Figura 6-29. Administración de ventanas en TCP.

de aplicación del *host* receptor retire algunos datos del *buffer*, en cuyo momento el TCP puede anunciar una ventana más grande.

Cuando la ventana es de 0, el transmisor normalmente no puede enviar segmentos, salvo dos excepciones. Primero, pueden enviarse datos urgentes (por ejemplo, para permitir que el usuario mate el proceso en ejecución en la máquina remota). Segundo, el transmisor puede enviar un segmento de 1 byte para hacer que el receptor reanuncie el siguiente byte esperado y el tamaño de la ventana. El estándar TCP proporciona explícitamente esta opción para evitar un interbloqueo si llega a perderse un anuncio de ventana.

No se requiere que los transmisores envíen datos tan pronto como llegan de la aplicación. Tampoco se requiere que los receptores envíen acuses de recibo tan pronto como sea posible. Por ejemplo, en la figura 6-29, cuando llegaron los primeros 2 KB de datos, el TCP, sabiendo que tenía disponible una ventana de 4 KB, habría actuado perfectamente bien si simplemente pone en *buffer* los datos hasta la llegada de otros 2 KB, para poder transmitir un segmento con una carga útil de 4 KB. Esta libertad puede explotarse para mejorar el desempeño.

Considere una conexión TELNET a un editor interactivo que reacciona con cada pulso de tecla. En el peor caso, al llegar un carácter a la entidad TCP transmisora, el TCP crea un segmento TCP de 21 bytes que entrega al IP para su envío como datagrama IP de 41 bytes. Del lado receptor, el TCP envía de inmediato un acuse de recibo de 40 bytes (20 bytes de cabecera TCP y 20 bytes de cabecera IP). Después, cuando el editor ha leído el byte, el TCP envía una actualización de ventana, recorriendo la ventana 1 byte hacia la derecha. Este paquete también es de 40 bytes. Por último, cuando el editor ha procesado el carácter, lo retransmite como paquete de 41 bytes. En conjunto, se usan 162 bytes de ancho de banda y se envían cuatro segmentos por cada carácter pulsado. Cuando es escaso el ancho de banda, no es deseable este método de operación.

Un enfoque que usan muchas implementaciones del TCP para mejorar esta situación es el retardo de los acuses de recibo y de las actualizaciones de ventana durante 500 mseg con la esperanza de que lleguen algunos datos con los cuales viajar gratuitamente. Suponiendo que el editor hace eco en un lapso de 500 mseg, sólo se necesita enviar ahora un paquete de 41 bytes de regreso al usuario remoto, recortando a la mitad la cuenta de paquetes y el uso de ancho de banda.

Aunque esta regla reduce la carga impuesta a la red por el receptor, el transmisor aún opera de manera ineficiente al enviar paquetes de 41 bytes que contienen 1 byte de datos. Una manera de reducir este uso es empleando el **algoritmo de Nagle** (Nagle, 1984). Lo que sugirió Nagle es sencillo: al llegar datos al transmisor un byte a la vez, simplemente se envía el primer byte y se pone en *buffer* los demás hasta el acuse de recibo del byte pendiente. Luego se transmiten todos los caracteres en *buffer* en un segmento TCP y se comienzan a poner los datos en *buffer* hasta haber sido reconocidos todos. Si el usuario escribe con rapidez y la red es lenta, puede entrar una cantidad importante de caracteres en cada segmento, reduciendo en gran medida el ancho de banda usado. Además, el algoritmo permite el envío de un nuevo paquete si han entrado suficientes datos para llenar la mitad de la ventana o la totalidad de un segmento.

El algoritmo de Nagle se usa ampliamente en las implementaciones de TCP, pero hay veces en que es mejor inhabilitarlo. En particular, al operar una aplicación X-Windows a través de Internet, los movimientos del ratón tienen que enviarse a la computadora remota. Su acumulación para enviarlos en ráfagas hace que el movimiento del cursor sea errático, lo que no complace mucho a los usuarios.

Otro problema que puede arruinar el desempeño del TCP es el **síndrome de ventana tonta** (*silly window syndrome*) (Clark, 1982). Este problema ocurre cuando se pasan datos a la entidad transmisora en bloques grandes, pero una aplicación interactiva del lado receptor lee datos a razón de 1 byte a la vez. Para ver el problema, observe la figura 6-30. Inicialmente, el *buffer* TCP del lado receptor está lleno y el transmisor lo sabe (es decir, tiene un tamaño de ventana de 0). Entonces la aplicación interactiva lee un carácter de la corriente TCP. Esta acción hace feliz al TCP receptor, por lo que envía una actualización de ventana al transmisor indicando que está bien que envíe 1 byte. El transmisor accede y envía 1 byte. El *buffer* ahora está lleno, por lo que el receptor reconoce el segmento de 1 byte pero establece la ventana en 0. Este comportamiento puede continuar indefinidamente.

La solución de Clark es evitar que el receptor envíe una actualización de ventana para 1 byte. En cambio, se le obliga a esperar hasta tener disponible una cantidad decente de espacio, y luego lo anuncia. Específicamente, el receptor no debe enviar una actualización de ventana hasta que pueda manejar el tamaño máximo de segmento que anunció al establecerse la conexión, o que su *buffer* esté vacío a la mitad, lo que sea más pequeño.

Además, el transmisor también puede ayudar al no enviar segmentos muy pequeños. En cambio, debe intentar esperar hasta haber acumulado suficiente espacio en la ventana para enviar un segmento completo, o cuando menos uno que contenga la mitad del tamaño de *buffer* del receptor (que debe estimar a partir del patrón de las actualizaciones de ventana que ha recibido anteriormente).

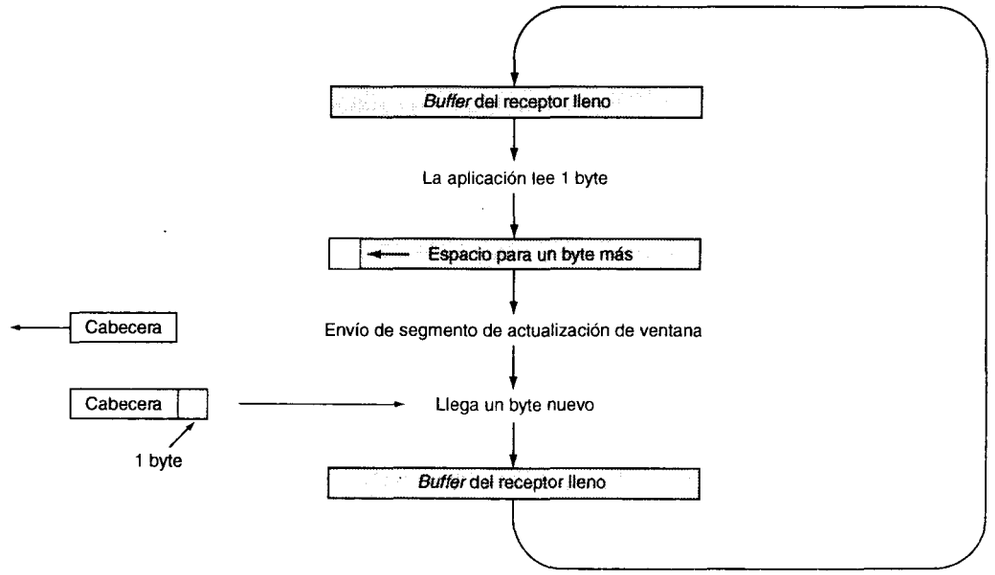


Figura 6-30. Síndrome de ventana tonta.

El algoritmo de Nagle y la solución de Clark al síndrome de ventana tonta son complementarios. Nagle trataba de resolver el problema causado por la entrega de datos al TCP desde la aplicación transmisora un byte a la vez. Clark trataba de resolver el problema de que la aplicación receptora toma los datos del TCP un byte a la vez. Ambas soluciones son válidas y pueden operar juntas. La meta es que el transmisor no envíe segmentos pequeños y que el receptor no los pida.

El TCP receptor también puede hacer más para mejorar el desempeño que simplemente actualizar ventanas en unidades grandes. Al igual que el TCP transmisor, tiene la capacidad de poner datos en *buffer*, por lo que puede bloquear una solicitud READ de la aplicación hasta poder proporcionar un bloque grande de datos. Hacer esto reduce la cantidad de llamadas al TCP y, por tanto, la carga extra. Por supuesto, también aumenta el tiempo de respuesta, pero en las aplicaciones no interactivas, como la transferencia de archivos, la eficiencia puede tener mayor peso que el tiempo de respuesta a las solicitudes individuales.

Otro problema del receptor es qué debe hacer con los segmentos fuera de orden. Pueden conservarse o descartarse, al albedrío del receptor. Por supuesto, los acuses de recibo pueden enviarse sólo después de haber recibido todos los datos hasta el byte reconocido. Si el receptor recibe los segmentos 0, 1, 2, 4, 5, 6 y 7, puede enviar un acuse de recibo de todos los bytes hasta el último byte del segmento 2, inclusive. Al terminar el transmisor la temporización, retransmitirá el segmento 3. Si el receptor tiene en *buffer* los segmentos 4 a 7, al recibir el segmento 3 puede enviar un acuse de recibo de todos los bytes hasta el final del segmento 7.

6.4.6. Control de congestionamientos con TCP

Cuando la carga ofrecida a cualquier red es mayor que la que puede manejar, se genera un congestionamiento. La Internet no es ninguna excepción. En esta sección estudiaremos los algoritmos que se han desarrollado durante la última década para manejar los congestionamientos. Aunque la capa de red también intenta manejarlos, gran parte del trabajo pesado recae sobre el TCP porque la solución real al congestionamiento es la disminución de la tasa de datos.

En teoría, puede manejarse el congestionamiento aprovechando un principio de física: la ley de conservación de los paquetes. La idea es no inyectar un paquete nuevo en la red hasta que salga uno viejo (es decir, se entregue). El TCP intenta lograr esta meta manipulando dinámicamente el tamaño de la ventana.

El primer paso del manejo de congestionamientos es su detección. Antaño la detección de congestionamientos era muy difícil. Una terminación de temporización causada por un paquete perdido pudo deberse a (1) ruido en la línea de transmisión o (2) el descartamiento de paquetes en el enrutador congestionado. La determinación de esta diferencia era difícil.

Hoy día, la pérdida de paquetes por errores de transmisión es relativamente rara debido a que las troncales de larga distancia son de fibra (aunque las redes inalámbricas son otra historia). En consecuencia, la mayoría de las terminaciones de temporización en Internet se deben a congestionamientos. Todos los algoritmos TCP de Internet suponen que las terminaciones de temporización son causadas por congestionamientos y las revisan en busca de problemas, de la misma manera que los mineros observan a sus canarios.

Antes de analizar la manera en que el TCP reacciona al congestionamiento, describiremos primero lo que hace para evitar que ocurra. Al establecerse una conexión, se tiene que seleccionar un tamaño de ventana adecuado. El receptor puede especificar una ventana con base en su tamaño de *buffer*. Si el transmisor se ajusta a su tamaño de ventana, no ocurrirán problemas por desbordamiento de *buffers* en la terminal receptora, pero aún pueden ocurrir debido a congestionamientos internos en la red.

En la figura 6-31 ilustramos este problema hidráulicamente. En la figura 6-31(a) vemos un tubo grueso que conduce a un receptor de poca capacidad. Mientras el transmisor no envíe más agua de la que puede contener la cubeta, no se perderá agua. En la figura 6-31(b), el factor limitante no es la capacidad de la cubeta, sino la capacidad de conducción interna de la red. Si entra demasiada agua a alta velocidad, esta retrocederá, perdiéndose algo (en este caso, por el desbordamiento del embudo).

La solución de Internet es aceptar que existen dos problemas potenciales (capacidad de la red y capacidad del receptor) y manejarlos por separado. Para ello, cada transmisor mantiene dos ventanas: la ventana que ha otorgado el receptor y una segunda ventana, la **ventana de congestionamiento**; cada una refleja la cantidad de bytes que puede enviar el transmisor. La cantidad de bytes que pueden enviarse es la cifra menor de las dos ventanas. Por tanto, la ventana efectiva es el mínimo de lo que el transmisor piensa que es correcto y lo que el receptor piensa

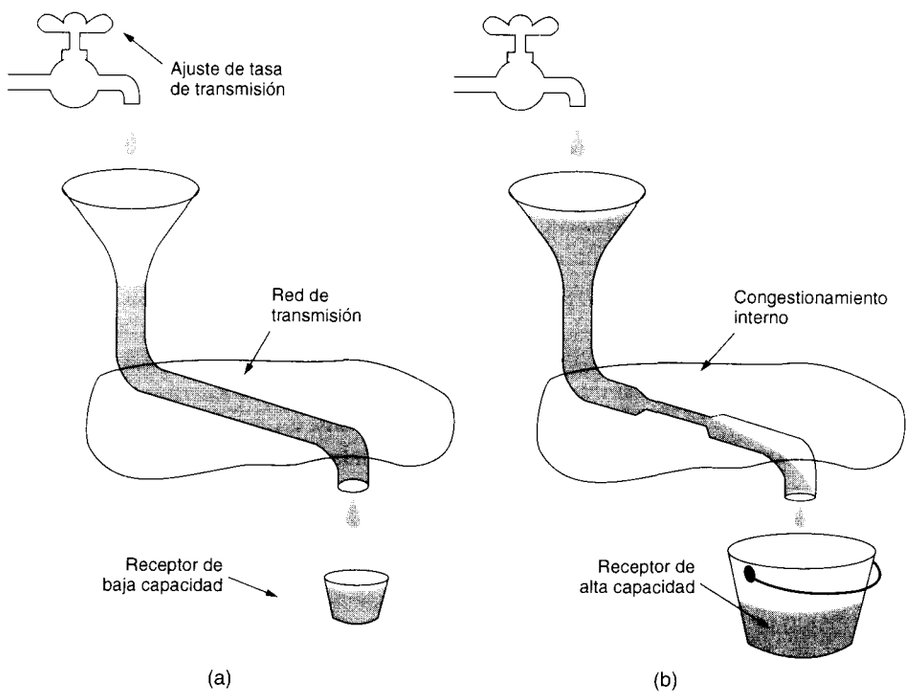


Figura 6-31. (a) Red rápida alimentando un receptor de baja capacidad. (b) Red lenta alimentando un receptor de alta capacidad.

que está bien. Si el receptor dice "envía 8K" pero el transmisor sabe que las ráfagas de más de 4K saturan la red, envía 4K. Por otra parte, si el receptor dice "envía 8K" y el transmisor sabe que las ráfagas de hasta 32K pueden llegar sin problemas, envía los 8K solicitados.

Al establecerse una conexión, el transmisor asigna a la ventana de congestionamiento el tamaño de segmento máximo usado por la conexión; entonces envía un segmento máximo. Si se recibe el acuse de recibo de este segmento antes de terminar el temporizador, el transmisor agrega el equivalente en bytes de un segmento a la ventana de congestionamiento para hacerla de dos segmentos de tamaño máximo, y envía dos segmentos. A medida que se reconoce cada uno de estos segmentos, se aumenta el tamaño de la ventana de congestionamiento en un segmento máximo. Cuando la ventana de congestionamiento es de n segmentos, si de todos los n se reciben acuses de recibo a tiempo, se aumenta el tamaño de la ventana de congestionamiento en la cuenta de bytes correspondiente a n segmentos. De hecho, cada ráfaga reconocida con éxito duplica la ventana de congestionamiento.

La ventana de congestionamiento sigue creciendo exponencialmente hasta ocurrir una terminación de temporización o alcanzar el tamaño de la ventana receptora. La idea es que, si las ráfagas de 1024, 2048 y 4096 bytes funcionan bien, pero una ráfaga de 8192 produce una terminación de temporización, la ventana de congestionamiento debe establecerse en 4096 para evitar el congestionamiento. Mientras el tamaño de la ventana de congestionamiento permanezca en 4096, no se enviará una ráfaga de mayor longitud, sin importar la cantidad de espacio de ventana otorgada por el receptor. Este algoritmo se llama **arranque lento**, pero no es lento en lo absoluto (Jacobson, 1988); es exponencial, y se requiere que todas las implementaciones de TCP lo manejen.

Veamos ahora el algoritmo de control de congestionamiento de Internet, el cual usa un tercer parámetro, el **umbral**, inicialmente de 64K, además de las ventanas de recepción y congestionamiento. Al ocurrir una terminación de temporización, se establece el umbral en la mitad de la ventana de congestionamiento actual, y la ventana de congestionamiento se restablece a un segmento máximo. Luego se usa el arranque lento para determinar lo que puede manejar la red, excepto que el crecimiento exponencial termina al alcanzar el umbral. A partir de ese punto, las transmisiones exitosas aumentan linealmente la ventana de congestionamiento (en un segmento máximo por ráfaga) en lugar de uno por segmento. En efecto, este algoritmo está suponiendo que probablemente es aceptable recortar la ventana de congestionamiento a la mitad, y luego aumentarla gradualmente a partir de ahí.

Como ilustración de la operación del algoritmo de congestionamiento, véase la figura 6-32. El tamaño máximo de segmento aquí es de 1024 bytes. Inicialmente, la ventana de congestionamiento era de 64K, pero ocurre una terminación de temporización, así que se establece el umbral en 32K y la ventana de congestionamiento en 1K, para la transmisión 0. La ventana de congestionamiento entonces crece exponencialmente hasta alcanzar el umbral (32K). A partir de entonces, crece linealmente.

La transmisión 13 tiene mala suerte (debería saberlo) y ocurre una terminación de temporización. Se establece el umbral en la mitad de la ventana actual (ahora de 40K, por lo que la mitad es de 20K), e inicia de nuevo el arranque lento. Al llegar los acuses de recibo de la transmisión 18, los primeros cuatro incrementan la ventana de congestionamiento en un segmento máximo, pero después de eso el crecimiento se vuelve lineal nuevamente.

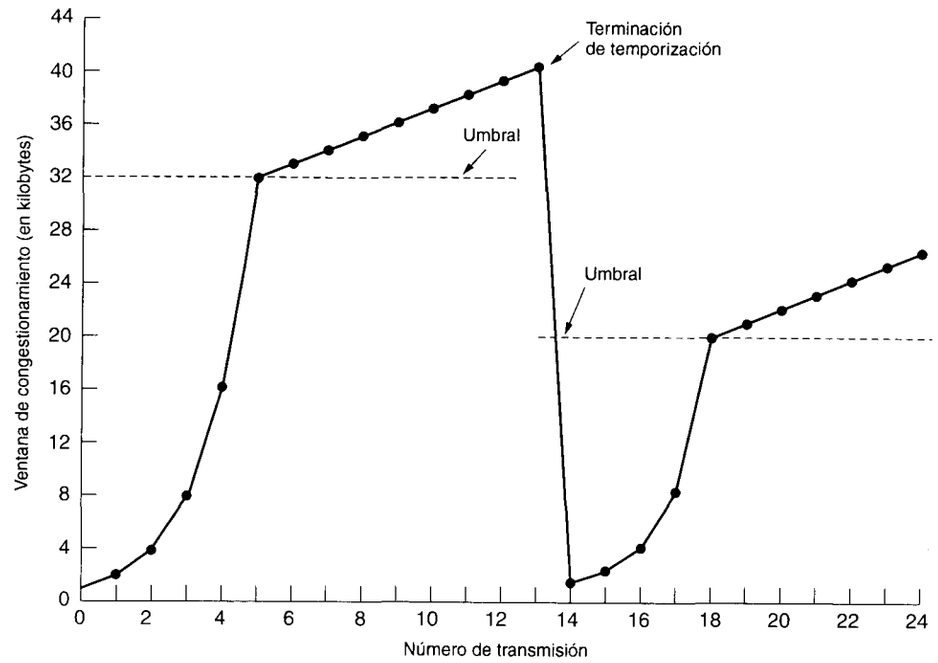


Figura 6-32. Ejemplo del algoritmo de congestionamiento de Internet.

Si no ocurren más terminaciones de temporización, la ventana de congestionamiento continuará creciendo hasta el tamaño de la ventana del receptor. En ese punto, dejará de crecer y permanecerá constante mientras no ocurran más terminaciones de temporización y la ventana del receptor no cambie de tamaño. Como nota al margen, si llega un paquete SOURCE QUENCH (suprimir origen) de ICMP y pasa al TCP, este evento será tratado de la misma manera que una terminación de temporización. Continúa el trabajo para mejorar los mecanismos de control de congestionamientos.

Por ejemplo, Brakmo *et al.* (1994) ha informado una mejora del rendimiento del TCP del 40 al 70% controlando con mayor precisión el reloj, prediciendo los congestionamientos antes de ocurrir terminaciones de temporización y usando este sistema de pronto aviso para mejorar el algoritmo de arranque lento.

6.4.7. Gestión de temporizadores del TCP

El TCP usa varios temporizadores (al menos conceptualmente) para hacer su trabajo. El más importante de éstos es el **temporizador de retransmisión**. Al enviarse un segmento, se inicia un temporizador de retransmisiones. Si el acuse de recibo del segmento llega antes de expirar el temporizador, éste se detiene. Si, por otra parte, el temporizador termina antes de llegar el acuse de recibo, se retransmite el segmento (y se inicia nuevamente el temporizador). Surge entonces la pregunta: ¿qué tan grande debe ser el intervalo de terminación del temporizador?

Este problema es mucho más difícil en la capa de transporte de Internet que en los protocolos de enlace de datos generales del capítulo 3. En este último caso, el retardo esperado es altamente predecible (es decir, tiene una variación baja), por lo que el temporizador puede ajustarse para terminar justo después del momento en que se espera el acuse de recibo, como se muestra en la figura 6-33(a). Dado que los acuses de recibo pocas veces se retardan en la capa de enlace de datos, la ausencia de un acuse de recibo en el momento esperado generalmente significa que el marco o el acuse de recibo se han perdido.

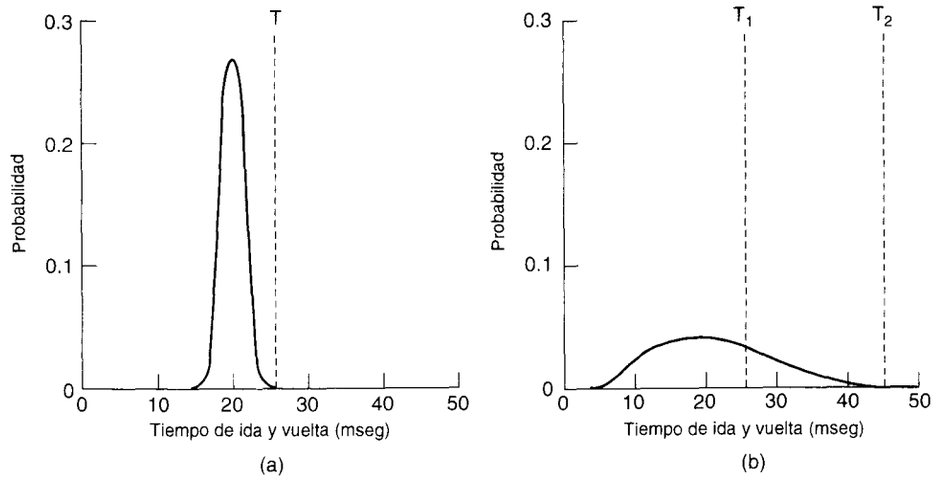


Figura 6-33. (a) Densidad de probabilidad de los tiempos de llegada de reconocimientos en la capa de enlace de datos. (b) Densidad de probabilidad de los tiempos de llegada de reconocimientos para el TCP.

El TCP enfrenta a un entorno radicalmente distinto. La función de densidad de probabilidad del tiempo que tarda en regresar un acuse de recibo TCP se parece más a la figura 6-33(b) que a la figura 6-33(a). Es complicada la determinación del tiempo de ida y vuelta al destino. Aun cuando se conoce, la selección del intervalo de terminación de temporización también es difícil. Si se hace demasiado corto, digamos T_1 en la figura 6-33(b), ocurrirán retransmisiones innecesarias, cargando la Internet con paquetes inútiles. Si se hace demasiado largo (T_2), el desempeño sufrirá debido al gran retardo de retransmisión de cada paquete perdido. Es más, la varianza y la media de la distribución de llegadas de acuses de recibo pueden variar con rapidez en unos cuantos segundos, a medida que se generan y se resuelven congestionamientos.

La solución es usar un algoritmo muy dinámico que ajuste constantemente el intervalo de terminación de temporización, con base en mediciones continuas del desempeño de la red. El algoritmo que generalmente usa el TCP lo debemos a Jacobson (1988) y funciona como sigue. Por cada conexión, el TCP mantiene una variable, *RTT* (*round-trip time*), que es la mejor estimación actual del tiempo de ida y vuelta al destino en cuestión. Al enviarse un segmento, se inicia un temporizador, tanto para ver el tiempo que tarda el acuse de recibo como para disparar

una retransmisión si se tarda demasiado. Si llega el acuse de recibo antes de expirar el temporizador, el TCP mide el tiempo que tardó el acuse de recibo, digamos *M*. Entonces actualiza *RTT* de acuerdo con la fórmula

$$RTT = \alpha RTT + (1 - \alpha)M$$

donde α es un factor de amortiguamiento que determina el peso que se le da al valor anterior. Por lo común, $\alpha = 7/8$.

Aun dado un buen valor de *RTT*, la selección de una terminación de temporización de retransmisión adecuada es un asunto no trivial. Normalmente el TCP usa βRTT, pero el truco es seleccionar β. En las implementaciones iniciales, β siempre era 2, pero la experiencia demostró que un valor constante era inflexible puesto que no respondía cuando subía la variación.

En 1988, Jacobson propuso hacer que β fuera aproximadamente proporcional a la desviación estándar de la función de densidad de probabilidad del tiempo de llegada de los acuses de recibo, por lo que una variación grande significa una β grande, y viceversa. En particular, sugirió el uso de la *desviación media* como una forma rápida de estimar la *desviación estándar*. Su algoritmo requiere mantener otra variable amortiguada, *D*, la desviación. Al llegar un acuse de recibo, se calcula la diferencia entre el valor esperado y el observado, $|RTT - M|$. Un valor amortiguado de esta cifra se mantiene en *D* mediante la fórmula

$$D = \alpha D + (1 - \alpha)\,|RTT - M|$$

donde α puede ser o no el mismo valor usado para amortiguar *RTT*. Si bien *D* no es exactamente igual a la desviación estándar, es bastante buena, y Jacobson demostró la manera de calcularla usando sólo sumas, restas y desplazamientos de enteros, lo que es una gran ventaja. La mayoría de las implementaciones TCP usan ahora este algoritmo y establecen el intervalo de terminación de temporización en

$$\text{Terminación de temporización} = RTT + 4*D$$

La selección del factor 4 es un tanto arbitraria, pero tiene dos ventajas. Primero, puede hacerse la multiplicación por 4 con un solo desplazamiento. Segundo, reduce al mínimo las terminaciones de temporización y las retransmisiones innecesarias porque menos del 1% de todos los paquetes llegan más de cuatro desviaciones estándar tarde. (En realidad, Jacobson sugirió inicialmente que se usara 2, pero el trabajo posterior ha demostrado que 4 da un mejor desempeño.)

Un problema que ocurre con la estimación dinámica de *RTT* es qué se debe hacer cuando un segmento termina de temporizar y se envía de nuevo. Cuando llega el acuse de recibo, no es claro si éste se refiere a la primera transmisión o a una posterior. Si se adivina mal se puede contaminar seriamente la estimación de *RTT*. Phil Karn descubrió este problema de la manera difícil. Él es un radioaficionado interesado en la transmisión de paquetes TCP/IP a través de la radio amateur, un medio notoriamente poco confiable (en un buen día, pasarán la mitad de los paquetes). Karn hizo una propuesta sencilla: no actualizar el *RTT* con ninguno de los segmentos retransmitidos. En cambio, se duplica la terminación de temporización con cada falla hasta que los segmentos pasan a la primera. Este sistema se llama **algoritmo de Karn** y lo usan la mayoría de las implementaciones TCP.

El temporizador de retransmisiones no es el único usado por el TCP. El segundo temporizador es el **temporizador de persistencia**, diseñado para evitar el siguiente interbloqueo. El receptor envía un acuse de recibo con un tamaño de ventana de 0, indicando al transmisor que espere. Después, el receptor actualiza la ventana, pero se pierde el paquete con la actualización. Ahora, tanto el transmisor como el receptor están esperando que el otro haga algo. Cuando termina el temporizador de persistencia, el transmisor envía una prueba al receptor. La respuesta de la prueba indica el tamaño de la ventana. Si aún es de cero, se inicia el temporizador de persistencia nuevamente, repitiéndose el ciclo. Si es diferente de cero, pueden enviarse datos.

Un tercer temporizador usado en algunas implementaciones es el **temporizador de seguir con vida** (*keepalive timer*). Cuando una conexión ha estado ociosa durante demasiado tiempo, el temporizador de seguir con vida puede terminar, haciendo que un lado compruebe que el otro aún está ahí. Si no se recibe respuesta, se termina la conexión. Esta característica es motivo de controversias porque agrega carga extra y puede terminar una conexión por lo demás saludable debido a una partición temporal de la red.

El último temporizador usado en cada conexión TCP es el que se usa en el estado *TIMED WAIT* durante el cierre; opera durante el doble del tiempo máximo de vida de paquete para asegurar que, al cerrarse una conexión, todos los paquetes creados por ella hayan muerto.

6.4.8. UDP

El grupo de protocolos de Internet también maneja un protocolo de transporte sin conexiones, el **UDP** (*User Data Protocol*, **protocolo de datos de usuario**). El UDP ofrece a las aplicaciones un mecanismo para enviar datagramas IP en bruto encapsulados sin tener que establecer una conexión. Muchas aplicaciones cliente-servidor que tienen una solicitud y una respuesta usan el UDP en lugar de tomarse la molestia de establecer y luego liberar una conexión. El UDP se describe en el RFC 768. Un segmento UDP consiste en una cabecera de 8 bytes seguida de los datos. La cabecera se muestra en la figura 6-34. Los dos puertos sirven para lo mismo que en el TCP: para identificar los puntos terminales de las máquinas de origen y destino. El campo de *longitud UDP* incluye la cabecera de 8 bytes y los datos. La *suma de comprobación UDP* incluye la misma pseudocabecera de formato mostrada en la figura 6-25, la cabecera UDP y los datos UDP, rellenados a una cantidad par de bytes de ser necesario. Esta suma es opcional y se almacena como 0 si no se calcula (un cálculo real de 0 se almacena como una sucesión de unos, que es lo mismo en complemento a 1). Inutilizarla resulta absurdo, a menos que la calidad de los datos no importe (por ejemplo, voz digitalizada).

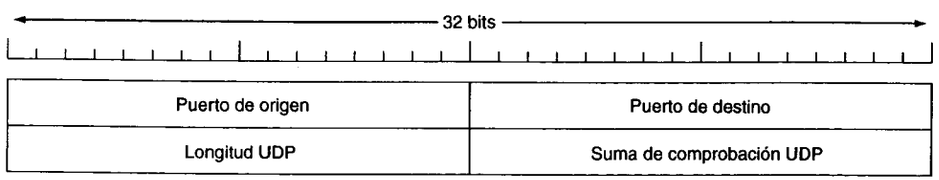


Figura 6-34. La cabecera UDP.

6.4.9. TCP y UDP inalámbricos

En teoría, los protocolos de transporte deben ser independientes de la tecnología de la capa de red subyacente. En particular, el TCP no debería preocuparse si el IP está operando por fibra o por radio. En la práctica sí importa, puesto que la mayoría de las implementaciones de TCP han sido optimizadas cuidadosamente con base en supuestos que se cumplen en las redes alámbricas, pero no en las inalámbricas. Ignorar las propiedades de la transmisión inalámbrica puede conducir a implementaciones del TCP correctas desde el punto de vista lógico pero con un desempeño horrendo.

El problema principal es el algoritmo de control de congestionamientos. Hoy día, casi todas las implementaciones de TCP suponen que las terminaciones de temporización ocurren por congestionamientos, no por paquetes perdidos. En consecuencia, al terminar un temporizador, el TCP disminuye su velocidad y envía con menor ímpetu (por ejemplo, el algoritmo de arranque lento de Jacobson). Lo que se pretende con este enfoque es reducir la carga de la red y aliviar así el congestionamiento.

Desafortunadamente, los enlaces de transmisión inalámbrica son muy poco confiables; pierden paquetes todo el tiempo. El enfoque adecuado para el manejo de paquetes perdidos es enviarlos nuevamente, tan pronto como sea posible. La reducción de la velocidad simplemente empeora las cosas. Si, digamos, se pierde el 20% de todos los paquetes, entonces cuando el transmisor envía 100 paquetes/seg, el rendimiento es de 80 paquetes/seg. Si el transmisor reduce su velocidad a 50 paquetes/seg, el rendimiento cae a 40 paquetes/seg.

En efecto, al perderse un paquete en una red alambrada, el transmisor debe reducir la velocidad. Cuando se pierde uno en una red inalámbrica, el transmisor debe acelerar. Cuando el transmisor no sabe de qué clase de red se trata, es difícil tomar la decisión correcta.

Con frecuencia, la trayectoria del transmisor al receptor no es homogénea. Los primeros 1000 km podrían ser a través de una red alambrada, pero el último km podría ser inalámbrico. Ahora es más difícil la decisión correcta en el caso de una terminación de temporización, ya que importa el lugar donde ocurrió el problema. Una solución propuesta por Bakne y Badrinath (1995), el **TCP indirecto**, es la división de la conexión TCP en dos conexiones distintas, como se muestra en la figura 6-35. La primera conexión va del transmisor a la estación base. La segunda va de la estación base al receptor. La estación base simplemente copia paquetes entre las conexiones en ambas direcciones.

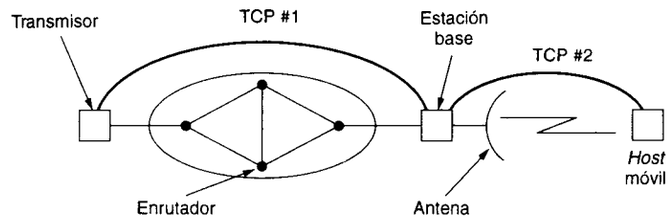


Figura 6-35. División de una conexión TCP en dos conexiones.

La ventaja de este esquema es que ahora ambas conexiones son homogéneas. Las terminaciones de temporización de la primera conexión pueden reducir la velocidad del transmisor, y las terminaciones de temporización de la segunda pueden acelerarla. La desventaja es que se viola por completo la semántica del TCP. Dado que cada parte de la conexión es una conexión TCP completa, la estación base reconoce cada segmento TCP de la manera normal, sólo que ahora la recepción de un reconocimiento en el transmisor no significa que el receptor recibió el segmento, sino únicamente que la estación base lo recibió.

Una solución diferente, debida a Balakrishnan *et al.* (1995), no divide la semántica del TCP; funciona haciendo varias modificaciones pequeñas al código de capa de red de la estación base. Uno de los cambios es la adición de un agente espía que observa y pone en caché los segmentos TCP que van al *host* móvil y los acuses de recibo que regresan de él. Cuando el agente espía ve un segmento TCP que sale al *host* móvil, pero no ve el regreso de un acuse de recibo antes de que su temporizador (relativamente corto) termine, simplemente retransmite ese segmento, sin indicar al origen que lo está haciendo.

El agente también genera una retransmisión cuando ve acuses de recibo duplicados del *host* móvil, lo que invariablemente significa que algo le ha fallado a este *host*. Los acuses de recibo duplicados se descartan en el lugar, para evitar que el origen los malinterprete como una señal de congestionamiento.

Sin embargo, una desventaja de esta transparencia es que, si el enlace inalámbrico tiene muchas pérdidas, el origen puede terminar de temporizar esperando un acuse de recibo e invocar el algoritmo de control de congestionamiento. Con el TCP indirecto, el algoritmo de control de congestionamiento nunca iniciará hasta que realmente haya un congestionamiento en la parte alambrada de la red.

El documento de Balakrishnan *et al.* también sugiere una solución al problema de segmentos perdidos que se originan en el *host* móvil. Al notar una estación base un hueco en los números de secuencia de entrada, genera una solicitud de repetición selectiva de los bytes faltantes usando una opción TCP. Gracias a estos dos mecanismos, el enlace inalámbrico se hace más confiable en ambas direcciones, sin que el origen lo sepa y sin cambiar la semántica del TCP.

Si bien el UDP no tiene los mismos problemas que el TCP, la comunicación inalámbrica también le produce dificultades. El problema principal es que los programas usan el UDP pensando que es altamente confiable; saben que no hay garantías, pero aun así esperan que sea casi perfecto. Un entorno inalámbrico estará muy lejos de serlo. En aquellos programas capaces de recuperarse de la pérdida de mensajes UDP, pasar repentinamente de un entorno en el que pueden perderse mensajes, pero rara vez ocurre, a uno en el que se pierden constantemente, puede dar pie a un desempeño desastroso.

La comunicación inalámbrica también afecta otras áreas, no sólo el desempeño. Por ejemplo, ¿cómo encuentra un *host* móvil una impresora local a la cual conectarse, en lugar de usar su impresora base? Algo parecido a esto es cómo acceder a la página del WWW de la célula local, aun si no se conoce su nombre. También, los diseñadores de páginas WWW tienden a suponer que hay mucho ancho de banda disponible. Un logotipo grande en cada página se vuelve contraproducente si su transmisión tarda 30 seg a 9600 bps cada vez que se hace referencia a la página, irritando sobremanera a los usuarios.

6.5. LOS PROTOCOLOS DE CAPA ATM AAL

En realidad no está claro si ATM tiene o no una capa de transporte. Por una parte, la capa ATM tiene la funcionalidad de una capa de red, y hay otra capa por encima de ella (AAL), lo que de alguna manera hace que AAL sea una capa de transporte. Algunos expertos están de acuerdo con este punto de vista (por ejemplo, De Prycker, 1993, página 112). Uno de los protocolos usados aquí (el AAL 5) es funcionalmente parecido al UDP, que sin duda es un protocolo de transporte.

Por otra parte, ninguno de los protocolos AAL proporciona una conexión confiable de extremo a extremo, como lo hace el TCP (aunque podrían con ciertos cambios menores). También, en la mayoría de las aplicaciones se usa otra capa de transporte encima de la AAL. En lugar de iniciar un debate, estudiaremos la capa AAL y sus protocolos en este capítulo sin sostener que es una verdadera capa de transporte.

La capa AAL de las redes ATM es radicalmente distinta del TCP, principalmente porque a los diseñadores les interesó principalmente transmitir corrientes de voz y vídeo, y aquí la entrega rápida es mucho más importante que la entrega correcta. Recuerde que la capa ATM simplemente envía células de 53 bytes en sucesión. No hay control de errores, no hay control de flujo y no hay ningún otro control. En consecuencia, no satisface los requisitos de la mayoría de las aplicaciones.

Para subsanar este problema, en la recomendación I.363 la ITU definió una capa de extremo a extremo por encima de la capa ATM. Esta capa, llamada **AAL** (*ATM Adaptation Layer*, **capa de adaptación de ATM**) tiene una historia tortuosa, llena de errores, correcciones y tareas inconclusas. En las siguientes secciones examinaremos esta capa y su diseño.

La meta de la AAL es proporcionar servicios útiles a programas de aplicación y protegerlos de la mecánica de dividir datos en células en el origen y reorganizarlos en el destino. Cuando la ITU comenzó a definir la AAL, se dio cuenta de que aplicaciones diferentes tienen requisitos distintos, por lo que organizó el espacio de servicio en tres ejes:

1. Servicio en tiempo real o servicio en tiempo no real.
2. Servicio de tasa de bits constante o servicio de tasa de bits variable.
3. Servicio orientado a conexiones o servicio sin conexiones.

En principio, con tres ejes y dos valores en cada eje, pueden definirse ocho servicios distintos, como se muestra en la figura 6-36. La ITU sintió que sólo cuatro de éstos eran de alguna utilidad, y los nombró clases A, B, C y D, como se indica. Los otros no se contemplaron. A partir del ATM 4.0, la figura 6-36 resulta un tanto obsoleta, por lo que se ha presentado aquí más bien como

antecedente para ayudar a entender la razón por la que los protocolos AAL se diseñaron de esta manera. En lugar de estas clases de servicio, la diferencia principal ahora es entre las clases de tráfico que estudiamos en el capítulo 5 (ABR, CBR, NRT-VBR, RT-VBR y UBR).

	A		B	C			D	
Sincronización	Tiempo real	Ninguna	Tiempo real	Ninguna	Tiempo real	Ninguna	Tiempo real	Ninguna
Tasa de bits	Constante		Variable		Constante		Variable	
Modo	Orientado a conexiones				Sin conexiones			

Figura 6-36. Clases originales de servicio reconocidas por la AAL (ahora obsoletas).

Para manejar estas cuatro clases de servicio, la ITU definió cuatro protocolos, AAL 1 a AAL 4. Sin embargo, luego se descubrió que los requisitos técnicos de las clases C y D eran tan parecidos que se combinaron el AAL 3 y el AAL 4 para formr el AAL 3/4. Entonces la industria de la computación, que estuvo dormida durante el cambio, se dio cuenta de que ninguna de esas clases era útil, y resolvió el problema definiendo otro protocolo, el AAL 5. Estudiaremos todos éstos un poco más adelante. También veremos un interesante protocolo de control usado en los sistemas ATM.

6.5.1. Estructura de la capa de adaptación de ATM

La capa de adaptación de ATM se divide en dos partes principales, una de las cuales se subdivide aún más, como se muestra en la figura 6-37.

La parte superior de la capa de adaptación de ATM se llama subcapa de convergencia. Su tarea es establecer la interfaz con la aplicación. Esta subcapa consiste en una subparte común a todas las aplicaciones (para un protocolo AAL dado) y una subparte específica para cada aplicación. Las funciones de cada una de estas partes dependen del protocolo pero pueden incluir enmarcado de mensajes y detección de errores.

Además, en el origen, la subcapa de convergencia se encarga de aceptar corrientes de bits o mensajes de longitud arbitraria de las aplicaciones y dividirlos en unidades de transmisión de 44 a 48 bytes. El tamaño exacto depende del protocolo, ya que algunos protocolos usan parte de la carga útil ATM de 48 bytes para sus propias cabeceras. En el destino, esta subcapa recompone los mensajes. En general, se conservan, cuando están presentes, los límites de los mensajes. En otras palabras, si el origen envía cuatro mensajes de 512 bytes, llegarán como cuatro mensajes de 512 bytes, no uno de 2048 bytes. En las corrientes de datos no existen límites, por lo que no se preservan.

La parte más baja de la AAL se llama subcapa **SAR** (*Segmentation And Reassembly*, **segmentación y recomposición**). Esta subcapa puede agregar cabeceras y apéndices a las unidades

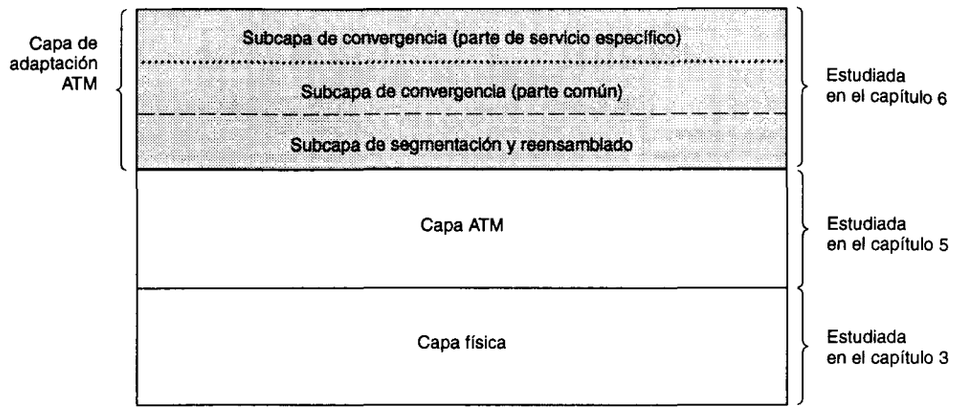


Figura 6-37. Modelo ATM que muestra la capa de adaptación ATM y sus subcapas.

de datos entregadas a ella por la subcapa de convergencia para formar cargas útiles de célula. Estas cargas se entregan a la capa ATM para su transmisión. En el destino, la subcapa SAR reensambla las células para obtener mensajes. La subcapa SAR básicamente se ocupa de las células y la subcapa de convergencia se ocupa de los mensajes.

El funcionamiento general de las subcapas de convergencia y SAR se muestra en la figura 6-38. Al llegar un mensaje al AAL desde la aplicación, la subcapa de convergencia puede darle una cabecera y apéndice a cada parte y pasarla a la capa ATM para su transmisión como célula independiente. Nótese que en la figura se muestra el caso más general porque algunos de los protocolos AAL tienen cabeceras y/o apéndice nulos.

La subcapa SAR también tiene algunas funciones adicionales para algunas clases de servicio (pero no todas). En particular, a veces maneja la detección de errores y la multiplexión. La subcapa SAR está presente en todas las clases de servicio, aunque el trabajo que realiza depende del protocolo específico.

La comunicación entre la aplicación y la subcapa AAL usa las primitivas *request* e *indication* OSI estándar que estudiamos en el capítulo 1. La comunicación entre las subcapas usa primitivas distintos.

6.5.2. AAL 1

AAL 1 es el protocolo usado para transmitir tráfico clase A, es decir, tráfico orientado a conexiones de tiempo real y con tasa de bits constante, como audio o vídeo sin compresión. Los bits son alimentados por la aplicación a una velocidad constante y deben entregarse en el otro lado a la misma velocidad constante, con retardo, fluctuación y carga extra mínimos. La entrada es una corriente de bits, sin límites de mensaje. Para este tráfico no se usan los protocolos de detección de errores como el de parada y espera porque los retardos que generan las terminaciones de temporización y las retransmisiones no son aceptables. Sin embargo, las células faltantes se

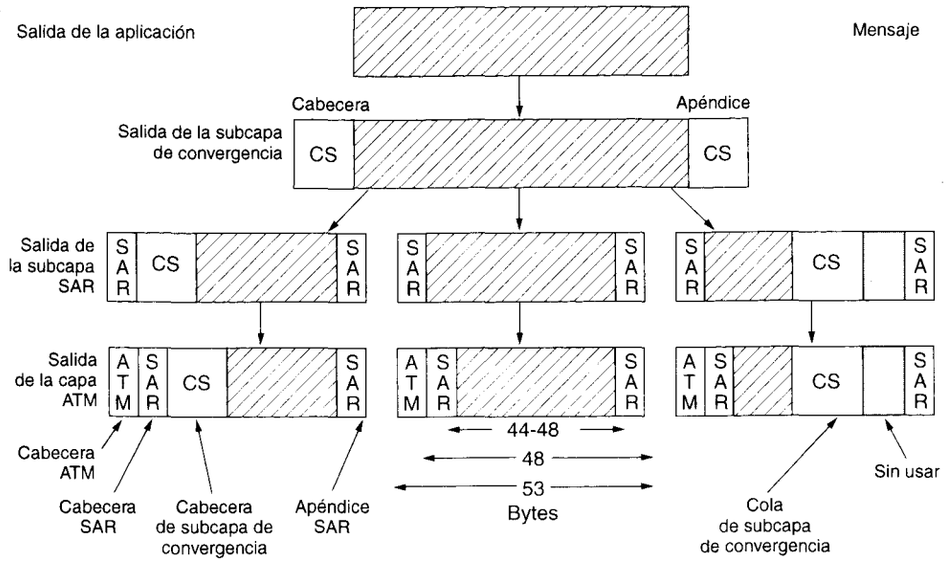


Figura 6-38. Las cabeceras y apéndices que pueden agregarse a un mensaje en una red ATM.

informan a la aplicación, que entonces, si lo desea, puede tomar sus propias medidas para recuperarlas.

AAL 1 tiene una subcapa de convergencia y una subcapa SAR. La primera detecta células perdidas y mal introducidas. (Una célula mal introducida es aquella que se entrega al destino equivocado como resultado de un error no detectado en el identificador de su circuito virtual o su trayectoria virtual.) Esta subcapa también amortigua el tráfico de entrada para proporcionar entrega de células a una tasa constante. Por último, la subcapa de convergencia divide los mensajes o la corriente de entrada en unidades de 46 o 47 bytes que se entregan a la subcapa SAR para su transmisión. En el otro extremo se extraen estas unidades y se reconstruye la entrada original. La subcapa de convergencia de AAL 1 no tiene ninguna cabecera de protocolo propia.

En contraste, la subcapa SAR de AAL 1 sí tiene un protocolo. Los formatos de sus células se muestran en la figura 6-39. Ambos formatos comienzan con una cabecera de 1 byte que contiene un número de secuencia de célula de 3 bits, *SN*, para detectar células perdidas o mal introducidas. A este campo le sigue un número de protección de secuencia (es decir, suma de comprobación) de 3 bits, el *SNP*, basado en el número de secuencia, para permitir la corrección de errores individuales y la detección de errores dobles en el campo de secuencia. Esta suma usa una comprobación de redundancia cíclica con el polinomio $x^3 + x + 1$. Un bit de paridad par que cubre el byte de cabecera reduce aún más la posibilidad de un número de secuencia equivocado. Las células AAL 1 no necesitan llevar 47 bytes. Por ejemplo, para transmitir voz digitalizada que llega a razón de 1 byte cada 125 μseg, el llenado de una célula con 47 bytes implicaría la recolección de muestras durante 5.875 mseg. Si este retardo antes de la transmisión es inacepta-

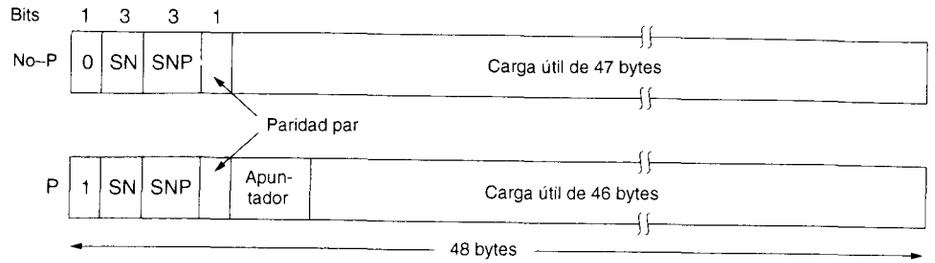


Figura 6-39. Formato de célula del AAL 1.

ble, pueden enviarse células parciales. En este caso, el número de bytes de datos reales por célula es igual para todas las células, y se acuerda por adelantado.

Las células *P* se usan cuando deben preservarse los límites de los mensajes. El campo de *apuntador* sirve para indicar el desfasamiento del comienzo del siguiente mensaje. Sólo las células con un número de secuencia par pueden ser células *P*, por lo que el apuntador está en el intervalo de 0 a 92, para que apunte dentro de la carga útil de su propia célula o de la que sigue. Nótese que este esquema permite que los mensajes tengan una cantidad arbitraria de bytes, por lo que pueden enviarse mensajes continuamente y no necesitan alinearse con los límites de la célula.

El bit de orden mayor del campo *apuntador* se reserva para uso futuro. El bit inicial de cabecera de todas las células de número impar forma una corriente de datos usada para la sincronización del reloj.

6.5.3. AAL 2

El AAL 1 se diseñó para corrientes de datos sencillas en tiempo real, orientadas a conexiones, y sin detección de errores, excepto por células faltantes o mal introducidas. Para el audio o vídeo puro sin compresión, o cualquier otra corriente de datos en la que unos cuantos bits alterados de vez en cuando no representan un problema, es adecuado el AAL 1.

En el audio o vídeo comprimido la tasa puede variar considerablemente con el tiempo. Por ejemplo, muchos esquemas de compresión transmiten un marco completo de vídeo y luego envían durante varios marcos sólo las diferencias entre los marcos subsiguientes y el último marco completo. Cuando la cámara es estacionaria y nada se mueve, los marcos de diferencia son pequeños, pero cuando la cámara está haciendo una panorámica rápida, son grandes. Además, los límites de los mensajes deben conservarse para que pueda reconocerse el comienzo del siguiente marco completo, aún en presencia de células perdidas o datos erróneos. Por estas razones se requiere un protocolo más complejo. Se diseñó para estos fines el AAL 2.

Como en AAL 1, la subcapa CS no tiene un protocolo, pero la subcapa SAR sí. El formato de célula SAR se muestra en la figura 6-40; tiene una cabecera de 1 byte y un apéndice de 2 bytes, dejando espacio para 45 bytes de datos por célula.

El campo *SN* (*Sequence Number*, número de secuencia) sirve para numerar las células en orden a fin de detectar células faltantes o mal introducidas. El campo *IT* (*Information Type*, tipo

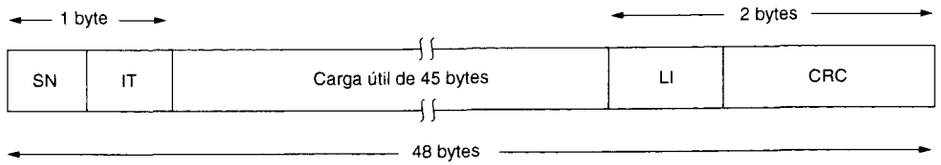


Figura 6-40. Formato de célula AAL 2.

de información) indica que la célula es el comienzo, la mitad o el fin de un mensaje. El campo *LI* (*Length Indicator,* indicador de longitud) indica el tamaño de la carga en bytes (puede ser menor que 45 bytes). Por último, el campo *CRC* es una suma de comprobación de la célula completa que permite detectar errores.

Aunque puede parecer extraño, los tamaños de campo no se incluyen en el estándar. De acuerdo con un enterado, ya al final del proceso de estandarización el comité se dio cuenta de que el AAL 2 tenía tantos problemas que no debería usarse. Desafortunadamente, era demasiado tarde para detener el proceso de estandarización, y había una fecha que cumplir. En un último esfuerzo, el comité quitó todos los tamaños de campo para que pudiera publicarse el estándar formal a tiempo, pero de manera tal que nadie pudiera usarlo. Así es la vida en el mundo de los estándares.

6.5.4. AAL 3/4

Originalmente, la ITU tenía protocolos diferentes para las clases C y D, servicio orientado a conexiones y servicio sin conexiones para transporte de datos sensibles a pérdidas y errores, pero no dependientes del tiempo. Luego la ITU descubrió que no había necesidad real de dos protocolos, por lo que los combinó en uno solo, el AAL 3/4.

El AAL 3/4 puede operar de dos maneras: corriente o mensajes. En el modo de mensajes, cada llamada de la aplicación al AAL 3/4 inyecta un mensaje en la red. El mensaje se entrega como tal, es decir, se conservan los límites del mensaje. En el modo de corrientes no se conservan los límites. El análisis siguiente se concentrará en el modo de mensajes. En cada modo hay disponibles transporte confiable y no confiable (es decir, sin garantía).

Una característica del AAL 3/4 no presente en los otros protocolos es la multiplexión. Este aspecto del AAL 3/4 permite que viajen por el mismo circuito virtual múltiples sesiones (por ejemplo, sesiones interactivas remotas) de un solo *host*, y que se separen en el destino, como se ilustra en la figura 6-41.

La razón por la que es deseable esta facultad es que las portadoras con frecuencia cobran por cada establecimiento de conexión y por cada segundo que permanece abierta la conexión. Si un par de *hosts* tiene abiertas varias sesiones simultáneas, dar a cada una su propio circuito virtual será más caro que la multiplexión de todas ellas en el mismo circuito virtual. Si un circuito virtual tiene suficiente ancho de banda para manejar todo, no hay necesidad de más de uno.

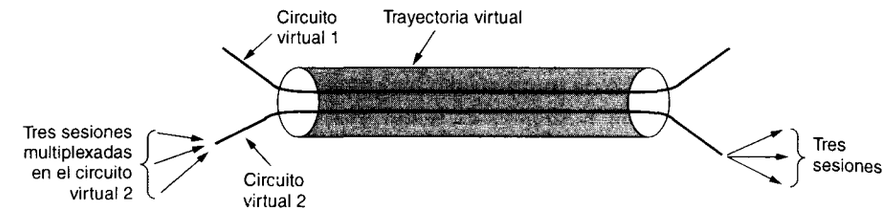


Figura 6-41. Multiplexión de varias sesiones en un circuito virtual.

Todas las sesiones que usan un solo circuito virtual reciben la misma calidad de servicio, ya que ésta se negocia para cada circuito virtual.

Esta cuestión es la verdadera razón por la que originalmente había formatos AAL 3 y AAL 4 separados: los estadounidenses querían multiplexión y los europeos no, así que cada grupo creó su propio estándar. Después, los europeos decidieron que ahorrar 10 bits de cabecera no valían el precio de que Estados Unidos y Europa no se pudieran comunicar. Por el mismo costo, se habrían podido empecinar en sus puntos de vista y tendríamos cuatro estándares AAL incompatibles (de los cuales uno está incompleto) en lugar de tres.

A diferencia del AAL 1 y el AAL 2, el AAL 3/4 tiene tanto un protocolo de subcapa de convergencia como uno de subcapa SAR. Los mensajes de hasta 65,535 bytes entran en la subcapa de convergencia desde la aplicación; primero se rellenan a un múltiplo de 4 bytes y luego se les agrega una cabecera y un apéndice, como se muestra en la figura 6-42.

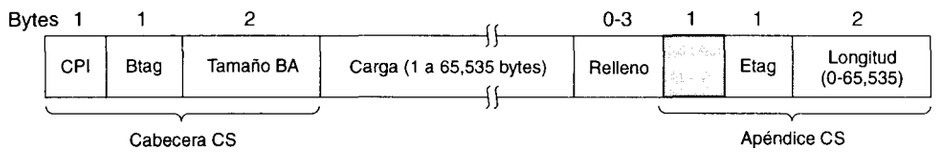


Figura 6-42. Formato de mensaje de subcapa de convergencia del AAL 3/4.

El campo *CPI* (*Common Part Indicator*, indicador de parte común) indica el tipo de mensaje y la unidad de conteo de los campos *tamaño BA* y *longitud*. Los campos *Btag* y *Etag* sirven para enmarcar los mensajes. Los dos bytes deben ser iguales y se incrementan en uno en cada mensaje nuevo enviado. Este mecanismo verifica si hay pérdida o mala introducción de células. El campo *tamaño BA* se usa para asignación de *buffers*; indica al receptor la cantidad de espacio de *buffer* a asignar para el mensaje antes de su llegada. El campo de *longitud* da nuevamente la longitud de la carga útil; en el modo de mensaje, debe ser igual al *tamaño BA*, pero en el modo de corriente puede ser diferente. El apéndice también contiene un byte no usado.

Una vez que la subcapa de convergencia ha construido el mensaje y le ha agregado una cabecera y una cola, como se muestra en la figura 6-42, pasa el mensaje a la subcapa SAR, que

lo divide en bloques de 44 bytes. Nótese que, para manejar multiplexión, la subcapa de convergencia puede construir varios mensajes internamente al mismo tiempo y pasar bloques de 44 bytes a la subcapa SAR, primero de un mensaje, luego de otro, en cualquier orden.

La subcapa SAR inserta cada bloque de 44 bytes en la carga útil de una célula cuyo formato se muestra en la figura 6-43. Estas células se transmiten entonces al destino para su reensamble, tras lo cual se lleva a cabo la verificación de la suma de comprobación y se realiza alguna acción, de ser necesario.

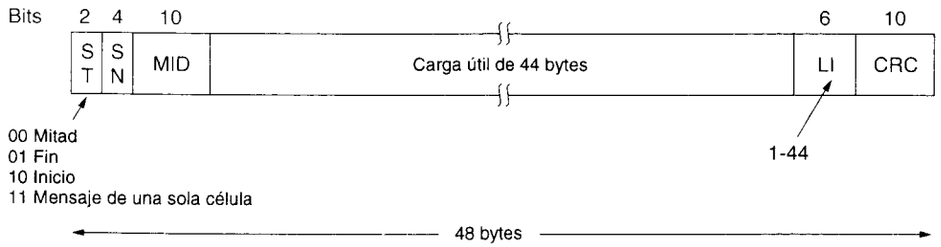


Figura 6-43. Formato de célula AAL 3/4.

Los campos de la célula AAL 3/4 son los siguientes. El campo *ST* (*Segment Type*, tipo de segmento) sirve para enmarcar los mensajes; indica si la célula inicia un mensaje, está a la mitad de él, es la última célula o si es un mensaje pequeño (es decir, de una sola célula). Sigue un número de secuencia de 4 bits, *SN*, para detectar células faltantes o mal introducidas. El campo *MID* (*Multiplexing ID*, identificador de multiplexión) sirve para llevar el registro de la sesión a la que pertenece cada célula. Recuerde que la subcapa de convergencia puede tener varios mensajes, pertenecientes a diferentes sesiones, en *buffer* al mismo tiempo, y que puede enviar partes de estos mensajes en cualquier orden que desee. Todas las partes de los mensajes que pertenecen a la sesión *i* llevan *i* en el campo *MID*, para poderlos reagrupar correctamente en el destino. El apéndice contiene la longitud de la carga útil y la suma de comprobación de la célula.

Nótese que el AAL 3/4 tiene carga extra de dos capas de protocolo: 8 bytes que se agregan a cada mensaje y 4 bytes que se agregan a cada célula. En suma, es un mecanismo de peso pesado, especialmente para los mensajes cortos.

6.5.5. AAL 5

Los protocolos AAL 1 a AAL 3/4 se diseñaron en gran medida para la industria de las telecomunicaciones y fueron estandarizados por el ITU sin mucha realimentación de la industria de la computación. Cuando la industria de la computación finalmente despertó y comenzó a entender las implicaciones de la figura 6-43, surgió una especie de pánico. La complejidad y la ineficiencia generadas por dos capas de protocolo, aunadas a la suma de comprobación extrañamente corta (sólo 10 bits), hizo que algunos investigadores inventaran un protocolo de adaptación nuevo. Lo

llamaron **SEAL** (*Simple Efficient Adaptation Layer*, **capa de adaptación sencilla y eficiente**), lo que sugiere qué pensaban estos diseñadores de los anteriores. Tras cierto debate, el foro ATM aceptó el SEAL y le asignaron el nombre AAL 5. Para mayor información sobre el AAL 5 y sus diferencias respecto al AAL 3/4, véase (Suzuki, 1994).

El AAL 5 ofrece varios tipos de servicio a sus aplicaciones. Una posibilidad es el servicio confiable (es decir, entrega garantizada con control de flujo para evitar desbordamientos). Otra posibilidad es el servicio no confiable (es decir, sin entrega garantizada), con opción de descartar o pasar a la aplicación (reportando como malas) las células con errores de suma de comprobación. Se apoyan tanto la unitransmisión como la multitransmisión, pero la multitransmisión no ofrece entrega garantizada.

Al igual que el AAL 3/4, el AAL 5 reconoce tanto el modo de mensajes como el modo de corriente. En el modo de mensajes, una aplicación puede pasar un datagrama de 1 a 65,535 bytes de longitud a la capa AAL para la entrega a su destino, ya sea con garantía o con el mejor esfuerzo. A su llegada a la subcapa de convergencia, se rellena un mensaje y se le agrega un apéndice, como se muestra en la figura 6-44. La cantidad de relleno (0 a 47 bytes) se selecciona para hacer que el mensaje completo, incluido el relleno y el apéndice, sea un múltiplo de 48 bytes. El AAL 5 no tiene una cabecera de subcapa de convergencia, sólo un apéndice de 8 bytes.

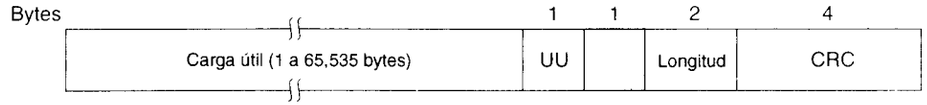


Figura 6-44. Formato de mensaje de subcapa de convergencia del AAL 5.

El campo *UU* (*User to User*, usuario a usuario) no lo usa la capa AAL misma, una capa superior puede usarlo para sus propios fines; por ejemplo, números de secuencia o multiplexión. La capa superior en cuestión puede ser la subparte específica para servicio de la subcapa de convergencia. El campo de *longitud* indica el tamaño de la carga útil real, en bytes, sin contar el relleno. Se usa un valor de 0 para abortar el mensaje actual a medio camino. El campo *CRC* es la suma de comprobación estándar de 32 bits del mensaje completo, incluido el relleno y el apéndice (con el campo *CRC* en 0). Se reserva un campo de 8 bits del apéndice para uso futuro.

El mensaje se transmite pasándolo a la subcapa SAR, que no agrega cabeceras ni apéndices; en cambio, divide el mensaje en unidades de 48 bytes y pasa cada una de ellas a la capa ATM para su transmisión. También le indica a la capa ATM que establezca un bit en el campo *PTI* de la última célula, de manera que se conserven los límites del mensaje. Puede argumentarse que ésta es una mezcla incorrecta de capas de protocolo, pues la capa AAL no debería estar usando bits de la cabecera de la capa ATM. Hacerlo viola el principio más básico de la ingeniería de protocolos y sugiere que las capas debieron diseñarse de manera diferente.

La ventaja principal del AAL 5 sobre el AAL 3/4 es una eficiencia mucho mayor. Si bien el AAL 3/4 agrega sólo 4 bytes por mensaje, también agrega 4 bytes por célula, reduciendo la

capacidad de carga a 44 bytes, una pérdida del 8% en mensajes grandes. El AAL 5 tiene un apéndice ligeramente más grande por mensaje (8 bytes), pero no tiene carga extra en cada célula. La falta de números de secuencia en las células se compensa con la suma de comprobación más grande, que puede detectar células perdidas, mal introducidas o faltantes sin necesidad de números de secuencia.

En la comunidad Internet, se espera que la manera normal de comunicarse con las redes ATM será el transporte de paquetes IP en el campo de carga útil del AAL 5. En el RFC 1483 y el RFC 1577 se estudian varios temas relacionados con este enfoque.

6.5.6. Comparación de los protocolos AAL

El lector tiene aquí derecho a pensar que los diferentes protocolos AAL parecen mal pensados e innecesariamente semejantes entre ellos. También es cuestionable el valor de tener subcapas de convergencia y SAR separadas, sobre todo porque el AAL 5 no tiene nada en la subcapa SAR. Una cabecera ATM ligeramente mejorada podría haber proporcionado secuencia, multiplexión y enmarcado más que adecuados.

Algunas de las diferencias entre los diferentes protocolos AAL se resumen en la figura 6-45. Éstas se relacionan con la eficiencia, el manejo de errores, la multiplexión y la relación entre las subcapas AAL.

Aspecto	AAL 1	AAL 2	AAL 3/4	AAL 5
Clase de servicio	A	B	C/D	C/D
Multiplexión	No	No	Sí	No
Delimitación de mensaje	Ninguna	Ninguna	Btag/Etag	Bit en PTI
Asignación de *buffers* por adelantado	No	No	Sí	No
Bytes de usuario disponibles	0	0	0	1
Relleno CS	0	0	Palabra de 32 bits	0-47 byts
Carga extra del protocolo CS (bytes)	0	0	8	8
Suma de comprobación CS	Ninguna	Ninguna	Ninguna	32 bits
Bytes de carga útil SAR	46-47	45	44	48
Carga extra de protocolo SAR (bytes)	1-2	3	4	0
Suma de comprobación SAR	Ninguna	Ninguna	10 bits	Ninguna

Figura 6-45. Algunas diferencias entre los diferentes protocolos AAL.

La impresión general que da el AAL es la de demasiadas variantes con demasiadas diferencias mínimas y un trabajo sin terminar. Las cuatro clases de servicio originales, A, B, C y D, de hecho han sido abandonadas. El AAL 1 probablemente no es necesario; el AAL 2 está incompleto; el AAL 3 y el AAL 4 nunca vieron la luz del día y el AAL 3/4 es ineficiente y tiene una suma de comprobación demasiado corta.

El futuro es del AAL 5, pero aun ahí hay lugar para mejoras. Los mensajes AAL 5 debieron tener un número de secuencia y un bit para distinguir los mensajes de datos y de control, para que pudiera usarse como protocolo de transporte confiable. Inclusive había espacio desocupado para ellos en el apéndice. Tal como está, para un transporte confiable se requiere además la carga adicional de una capa de transporte, cuando pudo haberse evitado. Si el comité AAL hubiera presentado su trabajo como proyecto escolar, el profesor probablemente se los habría devuelto con instrucciones de corregirlo y presentarlo nuevamente cuando lo concluyesen. Pueden encontrarse más críticas al ATM en (Sternbenz *et al.*, 1995).

6.5.7. SSCOP — protocolo de servicio específico orientado a conexiones

A pesar de todos estos diferentes protocolos AAL, ninguno de ellos proporciona conexiones de transporte confiables y sencillas punto a punto. En las aplicaciones en las que se requieren, existe otro protocolo AAL: el **SSCOP** (*Service Specific Connection Oriented Protocol*, **protocolo de servicio específico orientado a conexiones**). Sin embargo, el SSCOP sólo se usa para control, no para transmisión de datos.

Los usuarios del SSCOP envían mensajes, a cada uno de los cuales se asigna un número de secuencia de 24 bits. Los mensajes pueden ser de hasta 64K bytes y no se fragmentan; deben entregarse en orden. A diferencia de algunos otros protocolos de transporte confiable, los mensajes faltantes siempre se retransmiten usando repetición selectiva en lugar de regresar n.

El SSCOP es fundamentalmente un protocolo dinámico de ventana corrediza. Para cada conexión, el receptor mantiene una ventana de números de secuencia de mensaje que puede recibir, y un mapa de bits en el que marca los que ya tiene. Esta ventana puede cambiar de tamaño durante la operación del protocolo.

Lo que hace poco común al SSCOP es el manejo de los acuses de recibo: no hay incorporación. En su lugar, el transmisor sondea periódicamente al receptor para solicitarle que le envíe el mapa de bits que contiene el estado de la ventana. Con base en el resultado, el transmisor descarta los mensajes que han sido aceptados y actualiza la ventana. El SSCOP se describe con detalle en (Henderson, 1995).

6.6. DESEMPEÑO

Los asuntos relacionados con el desempeño son muy importantes en las redes de cómputo. Cuando hay cientos o miles de computadoras conectadas entre sí, son comunes las interacciones complejas, con consecuencias imprevisibles. Frecuentemente esta complejidad conduce a un desempeño pobre, sin que nadie sepa por qué. En las siguientes secciones examinaremos muchos temas relacionados con el desempeño de las redes para ver los tipos de problemas que existen y lo que se puede hacer para resolverlos.

Desafortunadamente, el entendimiento del desempeño de las redes es más un arte que una ciencia. Muy poca de la teoría tiene en realidad alguna utilidad en la práctica. Lo mejor que

podemos hacer es dar reglas empíricas derivadas de los tropiezos y ejemplos actuales tomados del mundo real. Intencionalmente hemos postergado este análisis hasta después de estudiar la capa de transporte en las redes TCP y ATM, a fin de poder señalar los lugares en los que se han hecho bien o mal las cosas.

La capa de transporte no es el único lugar en el que surgen asuntos relacionados con el desempeño. Vimos algunos de ellos en la capa de red, en el capítulo anterior. No obstante, la capa de red tiende a ocuparse principalmente del enrutamiento y el control de congestionamientos. Los puntos más amplios, orientados al sistema, tienden a relacionarse con el transporte, por lo que este capítulo es un lugar adecuado para examinarlos.

En las siguientes cinco secciones estudiaremos cinco aspectos del desempeño de las redes:

1. Problemas de desempeño.
2. Medición del desempeño de una red.
3. Diseño de sistemas con mejor desempeño.
4. Procesamiento de TPDU rápido.
5. Protocolos para redes futuras de alto desempeño.

Como comentario, necesitamos un nombre para las unidades intercambiadas por las unidades de transporte. El término de TCP, segmento, es confuso en el mejor de los casos, y nunca se usa fuera del mundo TCP en este contexto. Los términos correctos de ATM, CS-PDU, SAR-PDU y CPCS-PDU, son específicos de ATM. Los paquetes claramente se refieren a la capa de red y los mensajes pertenecen en la capa de aplicación. A falta de un término estándar, volveremos a llamar TPDU a las unidades intercambiadas por las entidades de transporte. Cuando queramos referirnos tanto a TPDU como a paquetes, usaremos paquete como término colectivo, como en "la CPU debe ser lo bastante rápida como para procesar los paquetes de entrada en tiempo real". Con esto nos referimos tanto al paquete de capa de red como a la TPDU encapsulada en él.

6.6.1. Problemas de desempeño en las redes de cómputo

Algunos problemas de desempeño, como el congestionamiento, son causados por sobrecargas temporales de los recursos. Si repentinamente llega más tráfico a un enrutador que el que puede manejar, se creará un congestionamiento y el desempeño bajará. Ya estudiamos el congestionamiento en detalle en el capítulo anterior.

El desempeño también se degrada cuando hay un desequilibrio estructural de los recursos. Por ejemplo, si una línea de comunicación de gigabits está conectada a una PC de bajo rendimiento, la pobre CPU no será capaz de procesar los paquetes de entrada a la velocidad suficiente, y se perderán algunos. Estos paquetes se retransmitirán tarde o temprano, agregando un retardo, desperdiciando ancho de banda y reduciendo en general el desempeño.

Las sobrecargas también pueden generarse sincrónicamente. Por ejemplo, si una TPDU contiene un parámetro erróneo (por ejemplo, el puerto o proceso al que está destinado), en

muchos casos el receptor cortésmente enviará una notificación de error. Ahora considere lo que podría ocurrir si se difundiera una TPDU errónea a 10,000 máquinas: cada una podría devolver un mensaje de error. La **tormenta de difusión** resultante podría paralizar la red. El UDP adoleció de este problema hasta que se cambió el protocolo para hacer que los *hosts* evitaran responder a errores en las TPDU de UDP enviadas a direcciones de difusión.

Un segundo ejemplo de carga extra síncrona es lo que ocurre tras una falla del suministro eléctrico. Al regresar la energía, todas las máquinas saltan simultáneamente a sus ROM para reiniciarse. Una secuencia de rearranque podría requerir acudir primero a algún servidor (RARP) para conocer la identidad verdadera de la máquina, y luego a un servidor de archivos para obtener una copia del sistema operativo. Si cientos de máquinas hacen todo esto al mismo tiempo, el servidor probablemente se vendría abajo por la carga.

Aun en ausencia de sobrecargas síncronas y con suficientes recursos disponibles, puede haber un bajo desempeño debido a la falta de afinación del sistema. Por ejemplo, si una máquina tiene bastante potencia y memoria de CPU, pero no se ha asignado suficiente memoria como espacio de *buffer*, ocurrirán desbordamientos y se perderán varias TPDU. De la misma manera, si el algoritmo de calendarización no tiene una prioridad lo bastante alta como para procesar las TPDU de entrada, podrán perderse algunas.

Otro asunto relativo a la afinación es el ajuste correcto de las temporizaciones. Cuando se envía una TPDU, normalmente se utiliza un temporizador para protegerse contra pérdidas. Si se hace muy corta la temporización, ocurrirán retransmisiones innecesarias, congestionando los alambres; si se hace demasiado larga, ocurrirán retardos innecesarios tras la pérdida de una TPDU. Otros parámetros afinables incluyen el tiempo de espera para incorporar datos a paquetes antes de enviar acuses de recibo por separado, y la cantidad de retransmisiones antes de darse por vencido.

Las redes de gigabits traen consigo nuevos problemas de desempeño. Considere, por ejemplo, el envío de datos de San Diego a Boston cuando el *buffer* del receptor es de 64K bytes. Supóngase que el enlace es de 1 Gbps y que el retardo de la luz a través de la fibra es de 20 mseg. Inicialmente, en $t = 0$, el tubo está vacío, como se muestra en la figura 6-46(a). Apenas 500 µseg después [figura 6-46(b)] todas las TPDU están en la fibra. La TPDU a la cabeza ahora estará en el vecindario de Brawley, todavía en las profundidades del sur de California. Sin embargo, el transmisor debe detenerse hasta recibir una actualización de ventana.

Después de 20 mseg, la TPDU puntera llega a Boston, como se muestra en la figura 6-46(c), y se envía acuse de recibo. Por último, 40 mseg después de comenzar, llega el primer acuse de recibo al transmisor y puede transmitirse la segunda ráfaga. Dado que la línea de transmisión se usó durante 0.5 mseg de un total de 40, la eficiencia es de aproximadamente 1.25%. Esta situación es típica de la operación de protocolos anteriores sobre líneas de gigabits.

Una cantidad que conviene recordar durante el análisis del desempeño de redes es el **producto ancho de banda-retardo** que se obtiene al multiplicar el ancho de banda (en bits/seg) por el tiempo de retardo de ida y vuelta (en seg). El producto es la capacidad del tubo desde el transmisor al receptor y de regreso (en bits).

Para el ejemplo de la figura 6-46, el producto ancho de banda-retardo es de 40 millones de bits. En otras palabras, el transmisor tendría que enviar una ráfaga de 40 millones de bits para

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

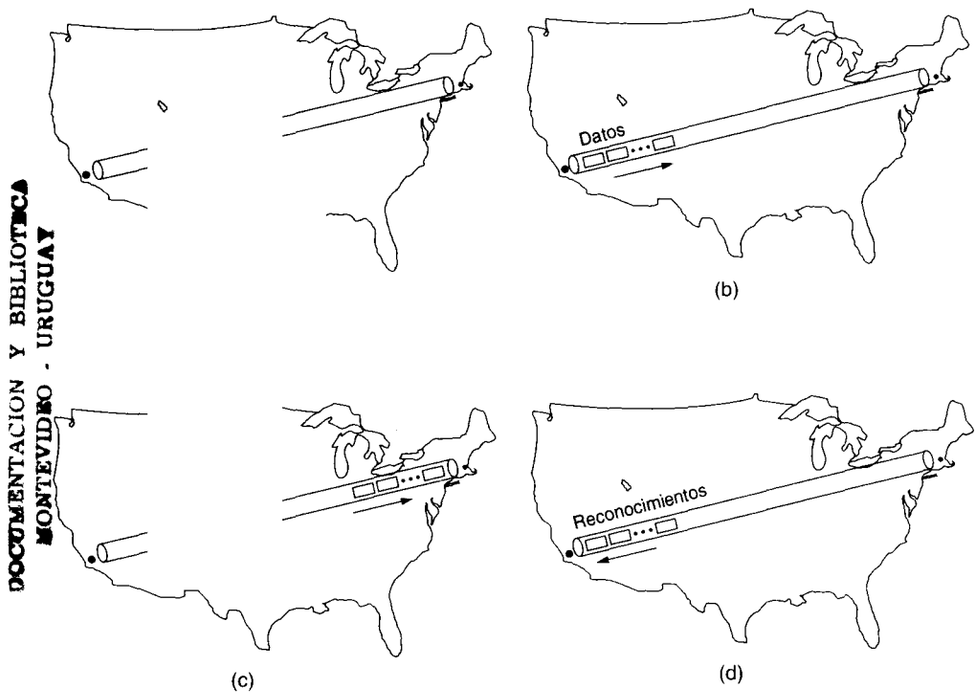


Figura 6-46. Estados de transmisión de un megabit de San Diego a Boston. (a) En $t = 0$. (b) Tras 500 µseg. (c) Tras 20 mseg. (d) Tras 40 mseg.

trabajar a toda velocidad hasta la llegada del primer acuse de recibo. Se requiere esta cantidad de bits para llenar el tubo (en ambas direcciones). Ésta es la razón por la que una ráfaga de medio millón de bits sólo logra una eficiencia del 1.25%: es sólo el 1.25% de la capacidad del tubo.

La conclusión a sacar aquí es que, para lograr un buen desempeño, la ventana del receptor debe tener cuando menos el tamaño del producto ancho de banda-retardo, y de preferencia ser un poco más grande, puesto que el receptor podría no responder instantáneamente. Para una línea transcontinental de gigabits se requieren cuando menos 5 megabytes para cada conexión.

Si la eficiencia es muy baja para el envío de un megabit, imagine lo que será al enviar unos cuantos cientos de bytes para una llamada a un procedimiento remoto. A menos que pueda encontrarse otro uso para la línea mientras el primer cliente espera su respuesta, una línea de gigabits no es mejor que una línea de megabits, sólo más cara.

Otro problema de desempeño que ocurre con las aplicaciones de tiempo crítico como audio y vídeo es la fluctuación. Tener un tiempo medio de transmisión corto no es suficiente. También se requiere una desviación estándar pequeña. El logro de un tiempo medio de transmisión corto con una desviación estándar pequeña requiere esfuerzos serios de ingeniería.

6.6.2. Medición del desempeño de las redes

Cuando una red tiene un desempeño pobre, sus usuarios frecuentemente se quejan con los operadores, exigiendo mejoras. Para mejorar el desempeño, los operadores deben primero determinar exactamente lo que ocurre. Para saberlo, los operadores deben efectuar mediciones. En esta sección veremos las mediciones de desempeño de las redes. El estudio siguiente se basa en el trabajo de Mogul (1993). Para un análisis más detallado del proceso de medición, véase (Jain, 1991, y Villamizan y Song, 1995).

El ciclo básico usado para mejorar el desempeño de las redes contiene los siguientes pasos:

1. Medir los parámetros pertinentes y el desempeño de la red.
2. Tratar de entender lo que ocurre.
3. Cambiar un parámetro.

Estos pasos se repiten hasta que el desempeño sea lo bastante bueno o que quede claro que se han hecho todas las mejoras posibles.

Las mediciones pueden hacerse de muchas maneras y en muchos lugares (tanto físicos como en la pila de protocolos). El tipo de medición más básico es arrancar un temporizador al iniciar una actividad y medir el tiempo que tarda la actividad. Por ejemplo, saber el tiempo que toma el acuse de recibo de una TPDU es una medición clave. Otras mediciones se hacen con contadores que registran la frecuencia con que ocurre un evento (por ejemplo, cantidad de TPDU perdidas). Por último, con frecuencia nos interesa saber la cantidad de algo, como el número de bytes procesados durante cierto intervalo de tiempo.

La medición del desempeño y los parámetros de una red tiene muchos escollos potenciales. A continuación describimos algunos de ellos. Cualquier intento sistemático de medir el desempeño de una red debe tener cuidado de evitarlos.

Asegúrese que el tamaño de la muestra es lo bastante grande

No mida el tiempo de envío de una TPDU, sino repita la medición, digamos, un millón de veces y obtenga el promedio. Una muestra grande reducirá la incertidumbre de la media y la desviación estándar medidas. Esta incertidumbre puede calcularse usando fórmulas estadísticas estándar.

Asegúrese de que las muestras son representativas

Idealmente, la secuencia total de un millón de mediciones debería repetirse a horas del día y de la semana diferentes para ver el efecto de diferentes cargas del sistema sobre la cantidad medida. Por ejemplo, las mediciones de congestionamientos sirven de poco si se toman en un momento en el que no hay congestionamientos. A veces los resultados pueden ser contraintuitivos inicial-

UNIVERSIDAD D
FACULTAD D
DEPARTA
DOCUMENTACIO

mente, como la presencia de congestionamiento intenso a las 10, 11, 13 y 14 horas, pero sin congestionamiento al mediodía (cuando todos los usuarios están en el refrigerio).

Tenga cuidado al usar relojes de intervalos grandes

Los relojes de computadora funcionan sumando uno a un contador a intervalos regulares. Por ejemplo, un temporizador de milisegundos suma uno al contador cada 1 mseg. El uso de tal temporizador para medir un evento que tarda menos de 1 mseg no es imposible, pero requiere cuidado.

Por ejemplo, para medir el tiempo de envío de una TPDU, el reloj del sistema (digamos, en milisegundos) debe leerse al entrar en el código de capa de transporte, y nuevamente al salir. Si el tiempo de envío real de la TPDU es de 300 μseg, la diferencia entre las dos lecturas será 0 o 1, ambas cifras equivocadas. Sin embargo, si la medición se repite un millón de veces y se suman todas las mediciones y se dividen entre un millón, el tiempo medio tendrá una exactitud del orden de menos de 1 μseg.

Asegúrese de que no ocurre nada inesperado durante sus pruebas

Realizar mediciones en un sistema universitario el día en que tiene que entregarse un importante proyecto de laboratorio puede dar resultados diferentes a los que se podrían obtener el día siguiente. Del mismo modo, si un investigador ha decidido difundir una vídeoconferencia por la red mientras usted hace sus pruebas, sus resultados podrían alterarse. Es mejor ejecutar las pruebas en un sistema inactivo y crear la carga completa usted mismo, pero aun este enfoque tiene algunos escollos. Usted podría pensar que nadie usará la red a las 3 A.M., pero esa podría ser precisamente la hora en la que el programa automático de respaldos comienza a copiar todos los discos a vídeocinta. Es más, puede haber un tráfico pesado hacia sus fantásticas páginas del *World Wide Web* desde husos horarios distantes.

El caché puede arruinar las mediciones

Si quiere medir los tiempos de transferencia de archivos, la manera obvia de hacerlo es abrir un archivo grande, leerlo todo, cerrarlo y ver el tiempo que tarda. Luego se repetiría la medición muchas veces más para obtener un buen promedio. El problema es que el sistema puede manejar el archivo en caché, por lo que sólo la primera medición realmente comprende tráfico de red. Las demás son sólo lecturas del caché local. Los resultados de tales mediciones son esencialmente inservibles (a menos que se quiera medir el desempeño del caché).

Con frecuencia puede evitarse el almacenamiento en caché simplemente desbordando el caché. Por ejemplo, si el caché es de 10 MB, el ciclo de prueba podría abrir, leer y cerrar dos archivos de 10 MB en cada vuelta, en un intento por obligar a que la tasa de encuentros del caché sea de 0. Aun así, se recomienda cuidado a menos que esté completamente seguro de que entiende el algoritmo de caché.

Los *buffers* pueden tener un efecto similar. Un programa de servicio de desempeño del TCP/IP bastante común ha llegado a informar que el UDP puede lograr un desempeño sustancialmente mayor al permitido por la línea física. ¿Por qué ocurre esto? Una llamada al UDP normalmente devuelve el control tan pronto como el núcleo ha aceptado el mensaje y lo ha agregado a la cola de transmisión. Si hay suficiente espacio de *buffer*, cronometrar 1000 llamadas UDP no implica que todos los datos se han enviado. La mayoría de ellos podría estar aún en el núcleo, pero el programa de servicio de desempeño piensa que se han transmitido todos.

Entienda lo que está midiendo

Al medir el tiempo de lectura de un archivo remoto, las mediciones dependen de la red, los sistemas operativos tanto en el cliente como en el servidor, las tarjetas de interfaz de *hardware* empleadas, sus controladores y otros factores. Si se tiene cuidado, finalmente se descubrirá el tiempo de transferencia de archivos para la configuración en uso. Si la meta es afinar esa configuración en particular, tales mediciones son adecuadas.

Sin embargo, si se están haciendo mediciones similares en tres sistemas diferentes a fin de seleccionar la tarjeta de interfaz de red a adquirir, sus resultados podrían desviarse por completo por el hecho de que uno de los controladores de la red realmente está mal y solamente está aprovechando el 10% del desempeño de la tarjeta.

Tenga cuidado con la extrapolación de los resultados

Suponga que hace mediciones con cargas de red simuladas que van desde 0 (en reposo) a 0.4 (40% de la capacidad), como lo indican los puntos de datos y la línea continua que los atraviesa en la figura 6-47. Puede ser tentador extrapolar linealmente, como lo indica la línea punteada. Sin embargo, muchos resultados de encolamiento comprenden un factor de $1/(1-\rho)$, donde ρ es la carga, por lo que los valores verdaderos pueden parecerse más a la línea punteada.

6.6.3. Diseño de sistemas para un mejor desempeño

La medición y los ajustes pueden con frecuencia mejorar considerablemente el desempeño, pero no pueden sustituir un buen diseño original. Una red mal diseñada puede mejorarse sólo hasta un límite. Más allá, tiene que rehacerse desde el principio.

En esta sección presentaremos algunas reglas empíricas basadas en la experiencia con muchas redes. Estas reglas se relacionan con el diseño del sistema, no sólo con el diseño de la red, ya que el *software* y el sistema operativo con frecuencia son más importantes que los enrutadores y las tarjetas de interfaz. La mayoría de estas ideas han sido del conocimiento común de los diseñadores de redes durante años y se han pasado verbalmente de generación en generación. Mogul fue el primero en postularlas explícitamente (1993); nuestro estudio sigue principalmente la secuencia del suyo. Otra fuente apropiada es (Metcalfe, 1993).

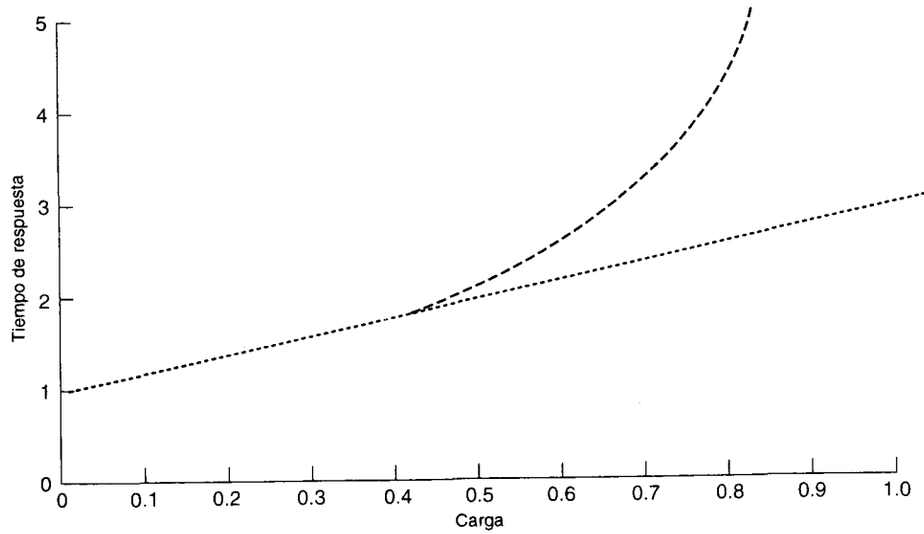


Figura 6-47. Respuesta en función de la carga.

Regla #1: La velocidad de la CPU es más importante que la velocidad de la red

Una amplia experiencia ha demostrado que en casi todas las redes la carga extra de los sistemas operativos y protocolos domina al tiempo real en el alambre. Por ejemplo, en teoría, el tiempo mínimo de RPC de un Ethernet es de 102 µseg, correspondiente a una solicitud mínima (64 bytes) seguida de una respuesta mínima (64 bytes). En la práctica, la reducción del tiempo RPC a 1500 µseg es un logro considerable (Van Renesse *et al.*, 1988). Nótese que 1500 µseg es 15 veces peor que el mínimo teórico. Casi toda la carga extra está en el *software*.

De la misma manera, el problema principal al operar a 1 Gbps es llevar los bits desde el *buffer* del usuario hasta la primera fibra a la velocidad suficiente y lograr que la CPU receptora los procese tan rápidamente como entran. En pocas palabras, si se duplica la velocidad de la CPU, con frecuencia casi se puede duplicar el rendimiento. La duplicación de la capacidad de la red en muchos casos no tiene efecto, ya que el cuello de botella generalmente está en los *hosts*.

Regla #2: Reducir el número de paquetes para reducir la carga extra de software

El procesamiento de una TPDU tiene cierta cantidad de carga extra por TPDU (por ejemplo, procesamiento de cabeceras) y cierta cantidad de procesamiento por byte (por ejemplo, procesar la suma de comprobación). Al enviar 1 millón de bytes, la carga extra por byte es la misma sin importar el tamaño de la TPDU. Sin embargo, el uso de TPDU de 128 bytes implica 32 veces más carga extra por TPDU que el uso de TPDU de 4K. Esta carga extra crece con rapidez.

Además de la carga extra de las TPDU, hay una carga extra en las capas inferiores que se debe considerar. Cada paquete que llega causa una interrupción. En un procesador RISC moderno, cada interrupción rompe la tubería de la CPU, interfiere con el caché, requiere un cambio del contexto en la administración de la memoria y obliga al almacenamiento de una cantidad de registros de CPU importante. Una reducción de n veces en las TPDU enviadas reduce la carga extra de la interrupción y de los paquetes en un factor de n.

Esta observación es un argumento a favor de la recolección de una cantidad importante de datos antes de su transmisión, a fin de reducir las interrupciones en el otro lado. El algoritmo de Nagle y la solución de Clark al síndrome de la ventana tonta (*silly window*) son intentos por lograr precisamente esto.

Regla #3: Reducir al mínimo las conmutaciones de contexto

Las conmutaciones de contexto (por ejemplo, del modo de núcleo al modo de usuario) son mortales; pueden tener los mismos inconvenientes que las interrupciones, siendo la peor una larga serie de fallas de caché iniciales. Las conmutaciones de contexto pueden reducirse haciendo que el procedimiento de biblioteca que envía los datos los guarde en un *buffer* interno hasta tener una buena cantidad de ellos. De la misma manera, en el lado receptor las TPDU de entrada pequeñas deben juntarse y pasarse al usuario como un bloque completo y no individualmente, para reducir al mínimo las conmutaciones de contexto.

En el mejor caso, un paquete de entrada causa una conmutación de contexto del usuario actual al núcleo, y luego una conmutación al proceso receptor para darle los nuevos datos. Desafortunadamente, en muchos sistemas operativos ocurren conmutaciones de contexto adicionales. Por ejemplo, si el administrador de la red ejecuta un proceso especial en el espacio de usuario, un paquete de llegada tenderá a causar una conmutación de contexto del usuario actual al núcleo, luego otra del núcleo al administrador de red, seguida de otra de regreso al núcleo y, por último, una del núcleo al proceso receptor. Esta secuencia se muestra en la figura 6-48. Todas estas conmutaciones de contexto en cada paquete desperdician mucho tiempo de CPU y tienen un efecto devastador sobre el desempeño de la red.

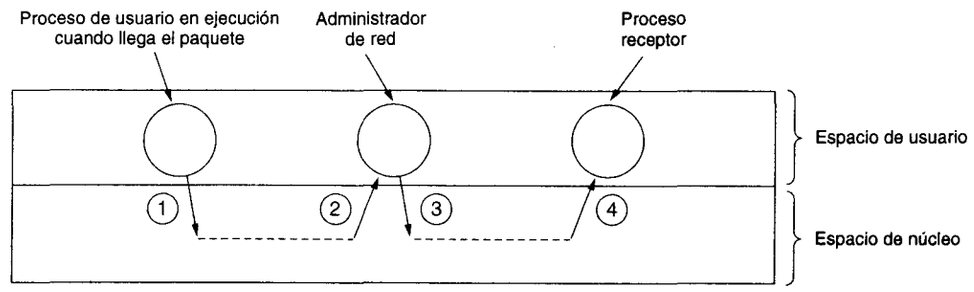


Figura 6-48. Cuatro conmutaciones de contexto para manejar un paquete con un administrador de red de espacio de usuario.

Regla #4: Reducir al mínimo las copias

Peores que las conmutaciones de contexto múltiples son las copias múltiples. No es inusitado que un paquete de entrada se copie tres o cuatro veces antes de entregarse la TPDU que contiene. Después de recibirse un paquete en la interfaz de la red en un *buffer* especial integrado a una tarjeta, es común que se copie en un *buffer* del núcleo. De ahí se copia en el *buffer* de capa de red, luego en el *buffer* de la capa de transporte y, por último, en el proceso de aplicación receptor.

Un sistema operativo ingenioso copiará una palabra a la vez, pero no es raro que se requieran unas cinco instrucciones por palabra (carga, almacenamiento, incremento de un registro de índice, prueba de fin de datos y ramificación condicional). En una máquina de 50 MIPS, tres copias de cada paquete a cinco instrucciones por palabra de 32 bits copiada requieren 75 nseg por byte de entrada. Tal máquina, por tanto, puede aceptar datos a una velocidad máxima de unos 107 Mbps. Si incluimos la carga extra del procesamiento de la cabecera, el manejo de interrupciones y las conmutaciones de contexto, podrían lograrse 50 Mbps, sin considerar el proceso real de los datos. Queda claro que el manejo de una línea de 1 Gbps está fuera de consideración.

De hecho, es probable que también una línea de 50 Mbps está fuera de consideración. En el cálculo anterior hemos supuesto que una máquina de 50 MIPS puede ejecutar 50 millones de instrucciones por segundo. En la realidad, las máquinas sólo pueden operar a tales velocidades si no hacen referencia a la memoria. Las operaciones de memoria con frecuencia son tres veces más lentas que las instrucciones registro a registro, por lo que la obtención de 16 Mbps de la línea de 1 Gbps podría considerarse muy buena. Nótese que la asistencia de *hardware* no ayuda aquí. El problema es que el sistema operativo está ejecutando demasiadas copias.

Regla #5: Puede comprarse más ancho de banda, pero no un retardo menor

Las tres reglas que siguen tienen que ver con la comunicación, más que con el proceso del protocolo. La primera regla indica que, si se desea más ancho de banda, se puede comprar. La instalación de una segunda fibra junto a la primera duplica el ancho de banda, pero no hace nada para reducir el retardo. Para que el retardo sea más corto es necesario mejorar el *software* del protocolo, el sistema operativo o la interfaz de red. Incluso si se mejoran las tres cosas, el retardo no se reducirá si el cuello de botella es el tiempo de transmisión.

Regla #6: Evitar el congestionamiento es mejor que recuperarse de él

La vieja máxima de que más vale prevenir que lamentar ciertamente se aplica al congestionamiento en redes. Al congestionarse una red, se pierden paquetes, se desperdicia ancho de banda, se introducen retardos inútiles, y otras cosas. La recuperación requiere tiempo y paciencia; es mejor que no sea necesaria. Evitar el congestionamiento es como recibir una vacuna: duele un poco en el momento, pero evita algo que sería mucho más doloroso.

Regla #7: Evitar terminaciones de temporización

Los temporizadores son necesarios en las redes, pero deben usarse con cuidado y deben reducirse al mínimo las terminaciones de temporización. Al expirar un temporizador, lo común es que se repita una acción. Si realmente es necesario repetir la acción, que así sea, pero su repetición innecesaria es un desperdicio.

La manera de evitar el trabajo extra es tener cuidado de que los intervalos de temporización sean más bien conservadores. Un temporizador que tarda demasiado en expirar agrega una pequeña cantidad de retardo extra a una conexión en el caso (improbable) de la pérdida de una TPDU. Un temporizador que termina cuando no debería consume tiempo de CPU valioso, desperdicia ancho de banda e impone una carga extra tal vez a docenas de enrutadores sin razón alguna.

6.6.4. Procesamiento rápido de las TPDU

La moraleja de la historia anterior es que el obstáculo principal en las redes rápidas es el *software* de los protocolos. En esta sección veremos algunas maneras de acelerar este *software*. Para mayor información, véase (Clark *et al.*, 1989; Edwards y Muir, 1995, y Chandranmenon y Varghese, 1995).

La carga extra de procesamiento de las TPDU tiene dos componentes: la carga extra por TPDU y la carga extra por byte. Ambas deben combatirse. La clave para el procesamiento rápido de las TPDU es separar el caso normal (transferencia normal de un solo sentido) y manejarlo como caso especial. Aunque se necesita una secuencia de TPDU especiales para entrar en el estado *ESTABLISHED*, una vez ahí el procesamiento de las TPDU es directo hasta que un lado cierra la conexión.

Comencemos por examinar el lado transmisor en el estado *ESTABLISHED* cuando hay datos por transmitir. Por claridad, supondremos que la entidad de transporte está en el núcleo, aunque los mismos conceptos se aplican si es un proceso de espacio de usuario o una biblioteca en el proceso transmisor. En la figura 6-49, el proceso transmisor está en el núcleo para ejecutar SEND. Lo primero que hace la entidad de transporte es probar si éste es el caso normal: el estado es *ESTABLISHED*, ningún lado está tratando de cerrar la conexión, se está enviando una TPDU normal (es decir, no fuera de banda) completa, y hay suficiente espacio de ventana disponible en el receptor. Si se cumplen todas la condiciones, no se requieren pruebas adicionales y puede seguirse la trayectoria rápida a través de la entidad de transporte transmisora.

En el caso normal, las cabeceras de las TPDU de datos consecutivas son casi iguales. Para aprovechar este hecho, se almacena una cabecera prototipo en la entidad de transporte. Al principio de la trayectoria rápida, la cabecera se copia lo más rápidamente posible en un *buffer* de trabajo, palabra por palabra. Los campos que cambian de una TPDU a otra se sobreescriben en el *buffer*. Con frecuencia, estos campos se deducen fácilmente de las variables de estado, como el siguiente número de secuencia. A continuación se pasan a la capa de red un apuntador a la cabecera completa de la TPDU más un apuntador a los datos de usuario. Aquí puede seguirse la misma estrategia (no se muestra en la figura 6-49). Por último, la capa de red entrega el paquete resultante a la capa de enlace de datos para su transmisión.

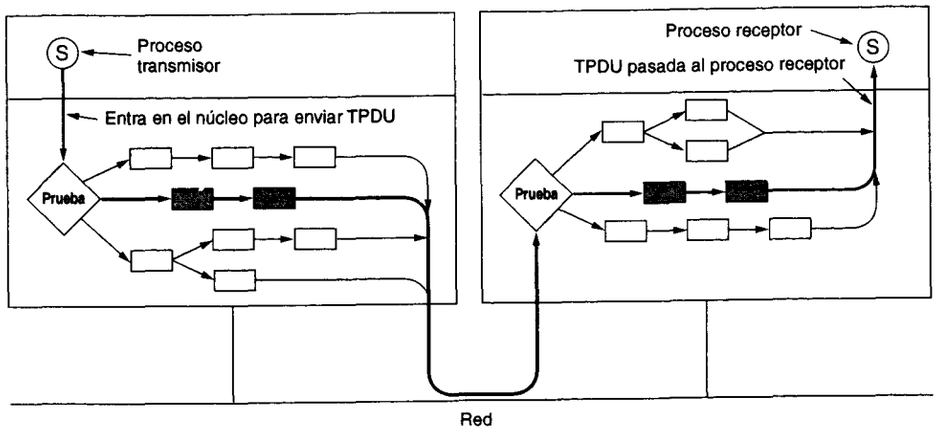


Figura 6-49. La trayectoria rápida del transmisor al receptor se indica con una línea gruesa. Los pasos de procesamiento de esta trayectoria se muestran sombreados.

Como ejemplo del funcionamiento de este principio en la práctica, consideremos el TCP/IP. En la figura 6-50(a) se muestra la cabecera TCP. Los campos que son iguales en varias TPDU consecutivas durante un flujo en un solo sentido aparecen sombreados. Todo lo que tiene que hacer la entidad de transporte transmisora es copiar las cinco palabras de la cabecera prototipo en el *buffer* de salida, actualizar el número de secuencia (copiándolo de una palabra en la memoria), calcular la suma de comprobación e incrementar el número de secuencia en la memoria. Entonces puede entregar la cabecera y los datos a un procedimiento IP especial para enviar una TPDU normal máxima. El IP entonces copia su cabecera prototipo de cinco palabras [véase la figura 6-50-(b)] en el *buffer*, llena el campo de *identificación* y calcula su suma de comprobación. El paquete ya está listo para transmitirse.

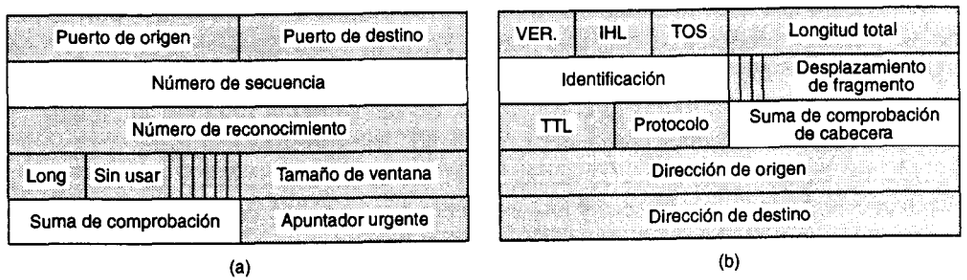


Figura 6-50. (a) Cabecera TCP. (b) Cabecera IP. En ambos casos, los campos sombreados se toman sin cambios del prototipo.

Veamos ahora al proceso de trayectoria rápida del lado receptor de la figura 6-49. El paso 1 es localizar el registro de conexión de la TPDU entrante. En el ATM, es fácil encontrar el registro

de conexión: el campo *VPI* puede usarse como índice de la tabla de trayectorias para encontrar la tabla de circuitos virtuales de esa trayectoria, y el *VCI* puede servir como índice para encontrar el registro de conexión. En el TCP, el registro de conexión puede almacenarse en una tabla de dispersión (*hash*) en la que alguna función sencilla de las dos direcciones IP y los dos puertos es la clave. Una vez localizado el registro de conexión, ambas direcciones y ambos puertos deben compararse para comprobar que se ha encontrado el registro correcto.

Una optimización que con frecuencia acelera la búsqueda del registro de conexión aún más es sencilla: mantener un apuntador al último registro usado e intentar ése primero. Clark *et al.* (1989) probó esto y observó una tasa de éxito mayor al 90%. Otras heurísticas de búsqueda se describen en (McKenney y Dove, 1992).

A continuación se revisa la TPDU para ver si es normal: el estado es *ESTABLISHED*, ninguno de los dos lados está tratando de cerrar la conexión, la TPDU es completa, no hay indicadores especiales encendidos y el número de secuencia es el esperado. Estas pruebas se llevan apenas unas cuantas instrucciones. Si se cumplen todas las condiciones, se invoca un procedimiento especial TCP de trayectoria rápida.

La trayectoria rápida actualiza el registro de la conexión y copia los datos en el espacio de usuario. Mientras copia, el procedimiento también calcula la suma de comprobación, eliminando una pasada extra por los datos. Si la suma de comprobación es correcta, se actualiza el registro de conexión y se devuelve un reconocimiento. El esquema general de hacer primero una comprobación rápida para ver si la cabecera es la esperada y tener un procedimiento especial para manejar ese caso se llama **predicción de cabecera**. Muchas implementaciones del TCP lo usan. Cuando esta optimización y todas las demás estudiadas en este capítulo se usan juntas, es posible conseguir que el TCP opere al 90% de la velocidad de una copia local de memoria a memoria, suponiendo que la red misma es lo bastante rápida.

Dos áreas en las que son posibles mejoras sustanciales del desempeño son el manejo de *buffers* y la administración de los temporizadores. El aspecto importante del manejo de *buffers* es evitar el copiado innecesario, como se explicó antes. La administración de los temporizadores es importante porque casi ninguno de los temporizadores expira; se ajustan para protegerse contra pérdidas de TPDU, pero la mayoría de las TPDU llegan correctamente, y sus reconocimientos también. Por tanto, es importante optimizar el manejo de los temporizadores para el caso en que los temporizadores casi nunca expiran.

Un esquema común consiste en usar una lista ligada de eventos de temporización ordenada por hora de expiración. La entrada inicial contiene un contador que indica la cantidad de pulsos de reloj que faltan para la expiración. Cada entrada subsecuente contiene un contador que indica el rezago en pulsos después de la entrada previa. Por tanto, si los temporizadores expiran a 3, 10 y 12 pulsos, respectivamente, los tres contadores son de 3, 7 y 2, respectivamente.

Después de cada pulso de reloj, se decrementa el contador de la cabecera inicial. Cuando llega a cero, su evento se procesa y el siguiente elemento de la lista es ahora el inicial; su contador no necesita cambiarse. En este esquema, la inserción y eliminación de temporizadores son operaciones caras, con tiempos de ejecución proporcionales a la longitud de la lista.

Puede usarse un enfoque más eficiente si el intervalo máximo del temporizador está limitado y se conoce por adelantado. Aquí puede utilizarse un arreglo llamado **rueda de temporización**,

como se muestra en la figura 6-51. Cada ranura corresponde a un pulso de reloj. El tiempo actual en la figura es $T = 4$. Los temporizadores se programan para expirar 3, 10 y 12 pulsos más adelante. Si se establece un temporizador nuevo para expirar en siete pulsos, simplemente se crea una entrada en la ranura 11. Del mismo modo, si el temporizador establecido para $T + 10$ tiene que cancelarse, debe examinarse la lista que comienza en la ranura 14 para eliminar la entrada pertinente. Nótese que el arreglo de la figura 6-51 no puede manejar temporizadores más allá de $T + 15$.

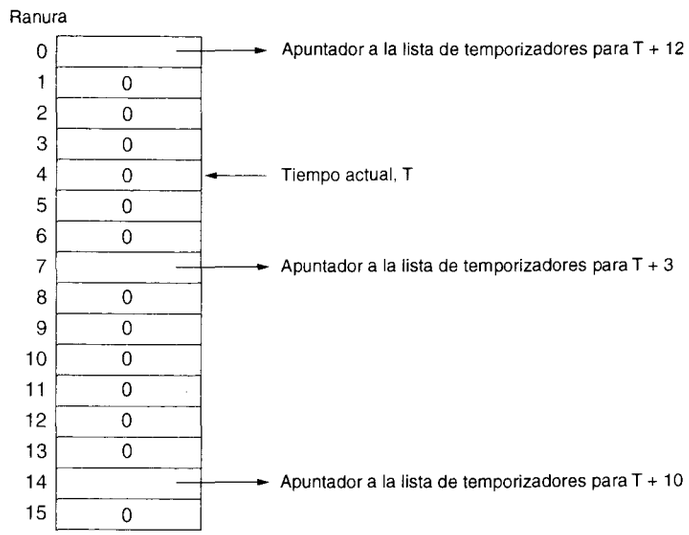


Figura 6-51. Rueda de temporización.

Cuando el reloj pulsa, el apuntador de tiempo actual avanza una ranura (circularmente). Si la entrada a la que ahora se apunta es diferente de cero, todos sus temporizadores se procesan. Se estudian muchas variaciones de la idea básica en (Varghese y Lauck, 1987).

6.6.5. Protocolos de redes de gigabits

Al inicio de la década de 1990 comenzaron a aparecer las redes de gigabits. La primera reacción de la gente fue usar en ellas los viejos protocolos, pero pronto surgieron varios problemas. En esta sección estudiaremos algunos de estos problemas y las medidas que están tomando los protocolos nuevos para resolverlos. Puede encontrarse más información en (Baransel *et al.*, 1995, y Partridge, 1994).

El primer problema es que muchos protocolos usan números de secuencia de 16 o 32 bits. Antaño, 2^{32} era una aproximación muy buena al infinito; hoy ya no lo es. Con una tasa de datos de 1 Gbps, se requieren unos 32 segundos para enviar 2^{32} bytes. Si los números de secuencia se refieren a bytes, como ocurre en el TCP, entonces un transmisor puede arrancar a toda velocidad trasmitiendo desde el byte 0 y, 32 segundos después, estar de nueva cuenta en el byte 0. Aun

suponiendo que se han recibido los acuses de recibo de todos los bytes, el transmisor no puede transmitir con seguridad datos nuevos desde 0 porque los paquetes viejos todavía podrían estar flotando en algún lado. En el Internet, por ejemplo, los paquetes pueden vivir 120 seg. Si se numeran los paquetes en lugar de los bytes, el problema es menos severo, a menos que los números de secuencia sean de 16 bits, en cuyo caso el problema es peor aún.

La causa de esta situación es que muchos diseñadores de protocolos simplemente supusieron, tácitamente, que el tiempo requerido para usar el espacio de secuencia completo excedería por mucho el tiempo de vida máximo de los paquetes. En consecuencia, no había necesidad de preocuparse por el problema de que duplicados viejos sobrevivieran aún al dar la vuelta los números de secuencia. A velocidades de gigabits, falla ese supuesto implícito.

Un segundo problema es que las velocidades de comunicación han mejorado con mucha mayor rapidez que las velocidades de cómputo. (Nota a los ingenieros en computación: ¡salgan a darles una paliza a los ingenieros en comunicaciones! Contamos con ustedes.) En los años 70, ARPANET operaba a 56 kbps y tenía computadoras que operaban a aproximadamente 1 MIPS. Los paquetes eran de 1008 bits, por lo que ARPANET era capaz de entregar unos 56 paquetes/seg. Con casi 18 mseg disponibles por paquete, un *host* podía darse el lujo de gastar 18,000 instrucciones en el procesamiento de un paquete. Claro que hacerlo requeriría todo el tiempo de CPU, pero podrían ejecutarse 9000 instrucciones por paquete y aun así tener la mitad del tiempo de CPU libre para llevar a cabo tareas reales.

Compare estas cantidades con las computadoras modernas de 100 MIPS que intercambian paquetes de 4 kB por una línea de gigabits. Los paquetes pueden entrar a una velocidad de más de 30,000 por segundo, por lo que el procesamiento de los paquetes debe completarse en 15 mseg si queremos reservar la mitad de la CPU para las aplicaciones. En 15 mseg, una computadora de 100 MIPS puede ejecutar 1500 instrucciones, apenas 1/6 de lo que tenían disponibles los *hosts* de ARPANET. Es más, las instrucciones RISC modernas hacen menos por instrucción de lo que hacían las viejas instrucciones CISC, por lo que el problema es aún peor que lo que parece. En conclusión, hay menos tiempo disponible para el procesamiento de los protocolos del que había antes, por lo que los protocolos deben volverse más sencillos.

Un tercer problema es que el protocolo de regresar n se desempeña mal en las líneas con un producto ancho de banda-retardo grande. Consideremos, por ejemplo, una línea de 4000 km que opera a 1 Gbps. El tiempo de transmisión de ida y vuelta es de 40 mseg, durante el cual un transmisor puede enviar 5 megabytes. Si se detecta un error, pasarán 40 mseg antes de que el transmisor se entere. Si se usa regresa n, el transmisor tendrá que retransmitir no sólo el paquete erróneo, sino también los 5 megabytes de paquetes que llegaron después. Evidentemente, éste es un desperdicio masivo de recursos.

Un cuarto problema es que las líneas de gigabits son fundamentalmente diferentes de las líneas de megabits en el sentido de que las líneas largas están limitadas por el retardo en lugar del ancho de banda. En la figura 6-52 mostramos el tiempo que tarda la transferencia de un archivo de 1 megabit a 4000 km con varias velocidades de transmisión. A velocidades de hasta 1 Mbps, el tiempo de transmisión está dominado por la velocidad con que pueden enviarse los bits. A 1 Gbps, el retardo de ida y vuelta de 40 mseg domina al 1 mseg que se tarda la inserción de los bits en la fibra. Aumentos mayores del ancho de banda apenas tienen algún efecto.

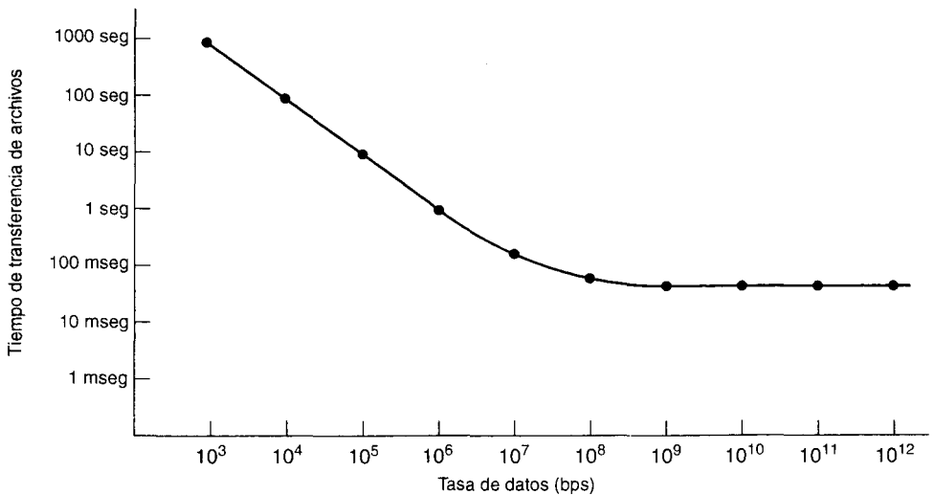


Figura 6-52. Tiempo de transferencia y reconocimiento de un archivo de 1 megabit a través de una línea de 4000 km.

La figura 6-52 tiene implicaciones desafortunadas para los protocolos de red; dice que los protocolos de parada y espera, como el RPC, tienen un límite superior de desempeño inherente. Este límite lo establece la velocidad de la luz. Ningún progreso tecnológico en la óptica mejorará la situación (aunque ayudarían nuevas leyes de la física).

Un quinto problema que vale la pena mencionar no es tecnológico ni de protocolo, como los otros, sino una consecuencia de las aplicaciones nuevas. En pocas palabras, en muchas aplicaciones de gigabits, como las de multimedia, la variación en los tiempos de llegada de los paquetes es tan importante como el retardo medio mismo. Una tasa de entrega lenta pero uniforme con frecuencia es preferible a una rápida pero variable.

Pasemos ahora de los problemas a las maneras de resolverlos. Primero haremos algunas observaciones generales, luego veremos los mecanismos de protocolo, la organización de los paquetes y el *software* de los protocolos.

El principio básico que deben aprender de memoria todos los diseñadores de redes de gigabits es:

Diseñar pensando en la velocidad, no en la optimización del ancho de banda.

Los protocolos viejos con frecuencia se diseñaban tratando de reducir al mínimo la cantidad de bits en el alambre, comúnmente usando campos pequeños y empacándolos en bytes y palabras. Hoy día hay más que suficiente ancho de banda. El procesamiento del protocolo es el problema, por lo que los protocolos deberían diseñarse para reducirlo al mínimo.

Una manera tentadora de acelerar el procedimiento es construir interfaces de red rápidas en *hardware*. Lo malo de esta estrategia es que, a menos que el protocolo sea excesivamente sencillo, "*hardware*" simplemente significa una tarjeta con una segunda CPU y su propio programa. Para evitar que el coprocesador de la red sea tan caro como la CPU principal, con frecuencia se usa un *chip* más lento. La consecuencia de este diseño es que una buena parte del tiempo la CPU principal (rápida) está esperando que una segunda CPU (lenta) haga el trabajo crítico. Es un mito pensar que la CPU principal tiene otras tareas que hacer mientras espera. Es más, cuando dos CPU de propósito general se comunican, pueden ocurrir condiciones de competencia, por lo que se requieren protocolos complejos entre los dos procesadores para sincronizarlos correctamente. Generalmente el mejor enfoque es hacer que los protocolos sean sencillos y dejar que la CPU principal haga el trabajo.

Veamos ahora el asunto de la realimentación en los protocolos de alta velocidad. Debido al ciclo de retardo grande (relativamente), debe evitarse la realimentación: la señalización del receptor al transmisor tarda demasiado. Un ejemplo de realimentación es el control de la tasa de transmisión mediante un protocolo de ventana corrediza. Para evitar los retardos (grandes) inherentes en el envío de actualizaciones de ventana del receptor al transmisor, es mejor usar un protocolo basado en la tasa. En tal protocolo, el transmisor puede enviar todo lo que quiera, siempre y cuando no envíe a mayor velocidad que cierta tasa acordada de antemano entre el transmisor y el receptor.

Un segundo ejemplo de realimentación es el algoritmo de arranque lento de Jacobson. Este algoritmo efectúa múltiples pruebas para saber qué tanto puede manejar la red. Con las redes de alta velocidad, efectuar media docena de pruebas pequeñas para ver la respuesta de la red es un desperdicio de ancho de banda enorme. Un esquema más eficiente es hacer que el transmisor, el receptor y la red reserven los recursos necesarios en el momento de establecer la conexión. La reservación de recursos por adelantado también tiene la ventaja de facilitar la reducción de la fluctuación. En pocas palabras, la elevación de las velocidades inevitablemente empuja el diseño hacia la operación orientada a conexiones, o a algo muy parecido.

La organización de los paquetes es una consideración importante en las redes de gigabits. La cabecera debería contener la menor cantidad de campos posible, a fin de reducir el tiempo de procesamiento. Estos campos deben ser lo bastante grandes como para cumplir su cometido, y estar alineados con los límites de palabra para facilitar su procesamiento. En este contexto, "bastante grande" significa que no ocurren problemas como números de secuencia que dan la vuelta mientras existen aún paquetes viejos, receptores incapaces de anunciar suficiente espacio de ventana porque el campo de ventana es demasiado pequeño, etcétera.

La cabecera y los datos deben tener sumas de comprobación aparte, por dos razones. Primera, hacer posible la obtención de la suma de comprobación de la cabecera, pero no de los datos. Segunda, comprobar que la cabecera está correcta antes de comenzar a copiar los datos en el espacio de usuario. Es deseable calcular la suma de comprobación de los datos al mismo tiempo que se copian los datos en el espacio de usuario, pero si el contenido de la cabecera está equivocado, el copiado podría hacerse a un proceso equivocado. Para evitar un copiado incorrecto pero permitir que se calcule la suma de comprobación de los datos durante el copiado, es esencial que las dos sumas de comprobación estén separadas.

El tamaño máximo de los datos debe ser lo bastante grande para permitir una operación eficiente inclusive ante retardos grandes. También, cuanto mayor es el tamaño del bloque de datos, menor es la fracción del ancho de banda total dedicada a las cabeceras.

Otra característica valiosa es la capacidad de enviar una cantidad normal de datos junto con la solicitud de conexión. Así, puede ahorrarse el tiempo de un viaje de ida y vuelta.

Por último, es importante decir algo sobre el *software* de los protocolos. Una idea clave es concentrarse en el caso exitoso. Muchos protocolos viejos tienden a hacer hincapié en lo que se debe hacer cuando algo falla (por ejemplo, cuando se pierde un paquete). Para hacer que los protocolos operen rápidamente, el diseñador debería enfocarse en reducir al mínimo el tiempo de procesamiento cuando todo funciona bien. Es secundaria la reducción al mínimo del tiempo de procesamiento cuando ocurren errores.

Un segundo asunto relacionado con el *software* es la minimización del tiempo de copiado. Como vimos antes, el copiado de datos con frecuencia es el origen principal de carga extra. Idealmente, el *hardware* debe poner en la memoria cada paquete de entrada como un bloque contiguo de datos. El *software* entonces debe copiar este paquete en el *buffer* del usuario con un solo copiado de bloque. Dependiendo del modo de funcionamiento del caché, inclusive puede ser deseable evitar un ciclo de copiado. En otras palabras, para copiar 1024 palabras, la manera más rápida podría ser la ejecución de 1024 instrucciones MOVE una tras otra (o 1024 pares cargar-almacenar). La rutina de copiado es tan crítica que debe desarrollares cuidadosamente en código de ensamblador, a menos que haya una manera de lograr que el compilador produzca el código óptimo.

A finales de los años 80, hubo un interés efímero en los protocolos rápidos de propósito especial como NETBLT (Clark *et al.*, 1987), VTMP (Cheriton y Williamson, 1989) y XTP (Chesson, 1989). En (Doeringer *et al.*, 1990) se hace un estudio. Sin embargo, la tendencia ahora es hacia la simplificación de protocolos de propósito general para hacerlos también rápidos. ATM presenta muchas de las características que analizamos antes, al igual que IPv6.

6.7. RESUMEN

La capa de transporte es la clave para entender los protocolos en capas. Esta capa proporciona varios servicios, siendo el más importante una corriente de bytes punto a punto, confiable y orientada o conexiones desde el transmisor hasta el receptor. Se accede a ella a través de primitivas de servicio que permiten establecer, usar y liberar conexiones.

Los protocolos de transporte deben ser capaces de administrar conexiones a través de redes no confiables. El establecimiento de conexiones se complica por la existencia de paquetes duplicados retardados que pueden reaparecer en momentos inoportunos. Para manejarlos, se requieren protocolos de acuerdo de tres vías (*three-way handshakes*) para establecer las conexiones. La liberación de una conexión es más sencilla que su establecimiento, pero aun así es un asunto poco trivial debido al problema de los dos ejércitos.

Aun si la capa de red es completamente confiable, la capa de transporte tiene bastante trabajo, como vimos en nuestro ejemplo: debe manejar todas las primitivas de servicio, administrar conexiones y temporizadores y asignar y usar créditos.

El protocolo de transporte principal de Internet es el TCP. Éste usa una cabecera de 20 bytes en todos los segmentos. Los segmentos pueden ser fragmentados por los enrutadores de Internet, por lo que los *hosts* deben estar preparados para efectuar el reensamblado. Se ha trabajado mucho para optimizar el desempeño del TCP, usando los algoritmos de Nagle, Clark, Jacobson, Karn y otros.

El ATM tiene cuatro protocolos en la capa AAL. Los cuatro dividen los mensajes en células en el origen y reconstruyen los mensajes en el destino. Las subcapas CS y SAR agregan sus propias cabeceras y apéndices de varias maneras, dejando de 44 a 48 bytes de carga útil por célula.

El desempeño de las redes generalmente es dominado por la carga extra de procesamiento de los protocolos y las TPDU, y esta situación empeora a mayores velocidades. Los protocolos deberían diseñarse para reducir al mínimo la cantidad de TPDU, de conmutaciones de contexto y de veces que se copia cada TPDU. En las redes de gigabits se requieren protocolos sencillos que usen control de flujo por tasa, no por crédito.

PROBLEMAS

1. En nuestras primitivas de ejemplo de la figura 6-3, LISTEN es una llamada bloqueadora. ¿Es estrictamente necesario esto? De no serlo, explique cómo debe usarse una primitiva no bloqueadora. ¿Qué ventaja tendría esto respecto al esquema descrito en el texto?
2. En el modelo de la figura 6-5, se supone que la capa de red puede perder paquetes, y por tanto deben reconocerse individualmente. Supóngase que la capa de red es 100% confiable y que nunca pierde paquetes. ¿Qué cambios, si acaso, se necesitarían en la figura 6-5?
3. Imagine un problema generalizado de n ejércitos, en el que el acuerdo entre dos de los ejércitos es suficiente para la victoria. ¿Existe un protocolo que permita que ganen los azules?
4. Supóngase que el esquema operado por reloj para generar números de secuencia iniciales se usa con un contador de reloj de 15 bits de ancho. El reloj pulsa cada 100 mseg, y el tiempo de vida máximo de un paquete es de 60 seg. ¿Con qué frecuencia ocurre la resincronización
   (a) en el peor caso?
   (b) cuando los datos consumen 240 números de secuencia/min?
5. ¿Por qué tiene que ser el tiempo de vida máximo de paquete, T, lo bastante grande para asegurar que han desaparecido no sólo el paquete, sino también sus acuses de recibo?
6. Imagine que se usa un acuerdo de dos vías (*two-way handshake*) en lugar de uno de tres vías para establecer las conexiones. En otras palabras, no se requiere el tercer mensaje. ¿Son posibles ahora los interbloqueos? Dé un ejemplo o demuestre que no pueden existir.
7. Considere el problema de la recuperación después de una caída del *host* (es decir, la figura 6-18). Si el intervalo entre la escritura y el envío de un reconocimiento, o viceversa, puede hacerse relativamente pequeño, ¿cuáles son las dos mejores estrategias transmisor-receptor para reducir al mínimo la posibilidad de una falla del protocolo?
8. ¿Son posibles los interbloqueos con la entidad de transporte descrita en el texto?
9. Por curiosidad, el implementador de la entidad de transporte de la figura 6-20 ha decidido incluir contadores en el procedimiento *sleep* para obtener estadísticas sobre el arreglo *conn*. Entre éstas están la cantidad de conexiones en cada uno de los siete estados posibles, n_i ($i = 1, \ldots, 7$). Tras escribir un programa masivo en FORTRAN para analizar los datos, nuestro implementador descubrió que la relación $\Sigma n_i = MAX_CONN$ parece ser verdadera siempre. ¿Hay otras invariantes en las que intervengan sólo estas siete variables?

10. ¿Qué ocurre cuando el usuario de la entidad de transporte dada en la figura 6-20 envía un mensaje de longitud cero? Explique el significado de su respuesta.

11. Por cada evento que puede ocurrir en la entidad de transporte de la figura 6-20, indique si es legal o no cuando el usuario está durmiendo en el estado *sending*.

12. Explique las ventajas y desventajas de los créditos sobre los protocolos de ventana corrediza.

13. La fragmentación de datagramas y su reensamble son manejados por el IP y son invisibles para el TCP. ¿Significa esto que el TCP no tiene que preocuparse por los datos que llegan en el orden equivocado?

14. A un proceso del *host* 1 se le asignó el puerto *p* y a un proceso del *host* 2 se le asignó el puerto *q*. ¿Es posible que haya más de dos conexiones TCP entre estos dos puertos al mismo tiempo?

15. La carga máxima de un segmento TCP es de 65,515 bytes. ¿Por qué se escogió un número tan extraño?

16. Describa dos formas de entrar en el estado *SYN RCVD* de la figura 6-28.

17. Indique una desventaja potencial del uso del algoritmo de Nagle en una red muy congestionada.

18. Considere el efecto de usar arranque lento en una línea con un tiempo de ida y vuelta de 10 mseg y sin congestionamientos. La ventana receptora es de 24 kB y el tamaño máximo de segmento es de 2 kB. ¿Cuánto tiempo pasará antes de poder enviar la primera ventana completa?

19. Supóngase que la ventana de congestionamiento del TCP está ajustada a 18K bytes y que ocurre una terminación de temporización. ¿Qué tan grande será la ventana si las siguientes cuatro ráfagas de transmisión tienen éxito? Supóngase que el tamaño máximo de segmento es de 1 kB.

20. Si el tiempo de ida y vuelta del TCP, *RTT*, actualmente es de 30 mseg y los siguientes reconocimientos llegan después de 26, 32 y 24 mseg, respectivamente, ¿cuál es la nueva estimación de *RTT*? Use $\alpha = 0.9$.

21. Una máquina TCP envía ventanas de 65,535 bytes por un canal de 1 Gbps que tiene un retardo de 10 mseg en un solo sentido. ¿Cuál es el rendimiento máximo que se puede lograr? ¿Es eficiente la línea?

22. En una red que tiene un tamaño máximo de TPDU de 128 bytes, un tiempo de vida de TPDU máximo de 30 seg, y un número de secuencia de 8 bits, ¿cuál es la tasa máxima de datos por conexión?

23. ¿Por qué existe el UDP? ¿No habría bastado con permitir que los procesos de usuario enviaran paquetes IP en bruto?

24. Un grupo de *N* usuarios ubicados en el mismo edificio usan la misma computadora remota a través de una red ATM. El usuario medio genera *L* líneas de tráfico (entrada + salida) por hora, en promedio, siendo la longitud media de línea de *P* bytes, excluyendo las cabeceras ATM. La portadora de paquetes cobra *C* centavos por byte de datos de usuario transportado, más *X* centavos por hora, por cada circuito virtual ATM abierto. ¿En qué condiciones es efectivo desde el punto de vista del costo multiplexar las *N* conexiones de transporte en el mismo circuito virtual ATM, si tal multiplexión agrega 2 bytes de datos a cada paquete? Supóngase que un solo circuito virtual ATM tiene suficiente ancho de banda para todos los usuarios.

25. ¿Puede manejar el AAL 1 mensajes más cortos que 40 bytes usando el esquema con el campo de *apuntador*? Explique su respuesta.

26. Adivine los tamaños que tenían los campos del AAL 2 antes de que los quitaran del estándar.

27. El AAL 3/4 permite la multiplexión de varias sesiones en un solo circuito virtual. Dé un ejemplo de situación en la que eso no tiene ningún valor. Suponga que un circuito virtual tiene suficiente ancho de banda para llevar todo el tráfico. *Sugerencia*: Piense sobre trayectorias virtuales.

28. ¿Cuál es el tamaño de la carga útil del mensaje de mayor longitud que cabe en una sola célula AAL 3/4?

29. Cuando se envía un mensaje de 1024 bytes con el AAL 3/4, ¿qué eficiencia se obtiene? En otras palabras, ¿qué fracción de los bits transmitidos comprende bits de datos útiles? Repita el problema para el AAL 5.

30. Un dispositivo ATM transmite mensajes de una sola célula a 600 Mbps. Una de cada 100 células se altera completamente debido al ruido aleatorio. ¿Cuántos errores no detectados pueden esperarse por semana con la suma de comprobación de 32 bits del AAL 5?

31. Un cliente envía una solicitud de 128 bytes a un servidor situado a 100 km a través de una fibra óptica de 1 gigabit. ¿Cuál es la eficiencia de la línea durante la llamada al procedimiento remoto?

32. Considere nuevamente la situación del problema anterior. Calcule el tiempo de respuesta mínimo posible tanto para la línea dada de 1 Gbps como para una línea de 1 Mbps. ¿Cuáles son sus conclusiones?

33. Suponga que está midiendo el tiempo de recepción de una TPDU. Al ocurrir una interrupción, usted lee el reloj del sistema en milisegundos; al terminar de procesarse la TPDU, lee el reloj de nuevo. Se mide 0 mseg 270,000 veces y 1 mseg 730,000 veces. ¿Cuál es el tiempo de recepción de una TPDU?

34. Una CPU ejecuta instrucciones a razón de 100 MIPS. Los datos pueden copiarse 64 bits a la vez, costando seis instrucciones cada palabra copiada. Si un paquete de entrada tiene que copiarse dos veces, ¿puede este sistema manejar una línea de 1 Gbps? Por sencillez, considere que todas las instrucciones, inclusive aquellas que leen o escriben en la memoria, operan a la velocidad máxima de 100 MIPS.

35. Para encontrar una solución al problema de los números de secuencia que vuelven a comenzar mientras aún existen paquetes viejos, podrían usarse números de secuencia de 64 bits. Sin embargo, en teoría, una fibra óptica puede operar a 75 Tbps. ¿Qué tiempo de vida máximo de paquete se requiere para asegurar que las redes futuras de 75 Tbps no tenga este problema aun con números de secuencia de 64 bits? Suponga que cada byte tiene su propio número de secuencia, como ocurre en el TCP.

36. En el texto calculamos que una línea de gigabits entrega 30,000 paquetes/seg al *host*, lo que deja sólo 1500 instrucciones para procesar los paquetes si la CPU dedica la mitad del tiempo a las aplicaciones. Este cálculo supuso un paquete de 4 kB. Repita el cálculo para un paquete de tamaño ARPANET (128 bytes).

37. En una red de 1 Gbps que opera sobre 4000 km, el factor limitante es el retardo, no el ancho de banda. Considere una MAN en la que el origen y el destino están a una distancia promedio de 20 km. ¿A qué tasa de datos el retardo de ida y vuelta debido a la velocidad de la luz equivale al retardo de transmisión de un paquete de 1 kB?

38. Modifique el programa de la figura 6-20 para efectuar recuperación de errores. Agregue un tipo de paquete nuevo, *reset*, que puede llegar tras haberse abierto una conexión en ambos lados, sin que ninguno la haya cerrado después. Este evento, que ocurre simultáneamente en ambos lados de la conexión, implica que todos los paquetes que estaban en tránsito han sido entregados o destruidos, pero en ninguno de los dos casos continúan en la subred.

39. Escriba un programa que simule la administración de *buffers* en una entidad de transporte que usa una ventana corrediza para control de flujo en lugar del sistema de créditos de la figura 6-20. Permita que los procesos de capa superior abran conexiones, envíen datos y cierren conexiones al azar. Para mantener la sencillez, haga que todos los datos viajen de la máquina *A* a la máquina *B*, y ninguno en el otro sentido. Experimente con diferentes estrategias de asignación de *buffers* en *B*, como dedicar *buffers* a conexiones específicas en lugar de tener un fondo de *buffers* común, y mida el rendimiento total logrado con cada una.

7

LA CAPA DE APLICACIÓN

Habiendo concluido todos los preliminares, ahora llegamos a la capa de aplicación, donde pueden encontrarse todas las aplicaciones interesantes. Las capas por debajo de la de aplicación están ahí para proporcionar transporte confiable, pero no hacen ningún trabajo verdadero para los usuarios. En este capítulo estudiaremos algunas aplicaciones reales.

Sin embargo, aun en la capa de aplicación se necesitan protocolos de apoyo que permitan el funcionamiento de las aplicaciones reales; veremos tres de ellos antes de comenzar con las aplicaciones mismas. La primera área es la seguridad, que no es un solo protocolo, sino una gran cantidad de conceptos y protocolos que pueden usarse para asegurar la confidencialidad donde sea necesaria. La segunda es el DNS, que maneja los nombres en Internet. El tercer protocolo de apoyo sirve para la administración de la red. Después de esto, estudiaremos cuatro aplicaciones reales: el correo electrónico, USENET (noticias de red), *World Wide Web* y, por último, multimedia.

7.1. SEGURIDAD DE LA RED

Durante las primeras décadas de su existencia, las redes de computadoras fueron usadas principalmente por investigadores universitarios para el envío de correo electrónico, y por empleados corporativos para compartir impresoras. En estas condiciones, la seguridad no recibió mucha atención. Pero ahora, cuando millones de ciudadanos comunes usan redes para sus transacciones bancarias, compras y declaraciones de impuestos, la seguridad de las redes aparece en el hori-

zonte como un problema potencial de grandes proporciones. En las siguientes secciones estudiaremos la seguridad de las redes desde varios ángulos, señalaremos muchos peligros y estudiaremos varios algoritmos y protoçolos para hacer más seguras las redes.

La seguridad es un tema amplio que cubre una multitud de pecados. En su forma más sencilla, la seguridad se ocupa de garantizar que los curiosos no puedan leer, o peor aún, modificar mensajes dirigidos a otros destinatarios; se preocupa por la gente que intenta acceder a servicios remotos no autorizados; también se ocupa de mecanismos para verificar que el mensaje supuestamente enviado por la autoridad fiscal que indica: "Pague el viernes o aténgase a las consecuencias" realmente viene de ella y no de la mafia. La seguridad también se ocupa del problema de la captura y reproducción de mensajes legítimos, y de la gente que intenta negar que envió ciertos mensajes.

La mayoría de los problemas de seguridad son causados intencionalmente por gente maliciosa que intenta ganar algo o hacerle daño a alguien. Algunos tipos de delincuentes más comunes se listan en la figura 7-1. Debe quedar claro por esta lista que hacer segura una red comprende mucho más que simplemente mantener los programas libres de errores; implica ser más listo que adversarios a menudo inteligentes, dedicados y a veces bien financiados. Debe quedar claro también que las medidas para detener a los adversarios casuales tendrán poco impacto sobre los serios.

Adversario	Meta
Estudiante	Divertirse husmeando el correo de la gente
Hacker	Probar el sistema de seguridad de alguien; robar datos
Representante de ventas	Indicar que representa a toda Europa, no sólo a Andorra
Hombre de negocios	Descubrir el plan estratégico de mercadeo de un competidor
Ex empleado	Vengar su despido
Contador	Estafar dinero de una compañía
Corredor de bolsa	Negar una promesa hecha a un cliente por correo electrónico
Timador	Robar números de tarjeta de crédito
Espía	Conocer la fuerza militar de un enemigo
Terrorista	Robar secretos de guerra bacteriológica

Figura 7-1. Algunas personas que causan problemas de seguridad, y por qué.

Los problemas de seguridad de las redes pueden dividirse en términos generales en cuatro áreas interrelacionadas: secreto, validación de identificación, no repudio y control de integridad. El secreto tiene que ver con mantener la información fuera de las manos de usuarios no autorizados. Esto es lo que normalmente viene a la mente cuando la gente piensa en la seguridad de las redes. La validación de identificación se encarga de determinar con quién se está hablando antes

de revelar información delicada o hacer un trato de negocios. El no repudio se encarga de las firmas: ¿cómo comprobar que su cliente realmente colocó una orden electrónica por 10 millones de marinolas con rosca a la izquierda a 89 centavos cada una cuando él luego alega que el precio era de 69 centavos? Por último, ¿cómo puede asegurarse de que un mensaje recibido realmente fue el enviado, y no algo que un adversario malicioso modificó en el camino o cocinó por su propia cuenta?

Todos estos temas (secreto, validación de identificación, no repudio y control de integridad) son pertinentes también en los sistemas tradicionales, pero con algunas diferencias importantes. El secreto y la integridad se logran usando correo registrado y poniendo bajo llave los documentos. El robo del tren del correo es más difícil de lo que era en los tiempos de Jesse James.

También, la gente puede por lo general distinguir entre un documento original en papel y una fotocopia, y con frecuencia esto es importante. Como prueba, haga una fotocopia de un cheque válido. Trate de cobrar el cheque original en su banco el lunes. Ahora trate de cobrar la fotocopia del cheque el martes. Observe la diferencia de comportamiento del banco. Con los cheques electrónicos, el original y la copia son idénticos. Es posible que los bancos tarden un poco en acostumbrarse a esto.

La gente valida la identificación de otra gente al reconocer sus caras, voces y letra. Las pruebas de firmas se manejan mediante firmas en papel, sellos, etc. Generalmente puede detectarse la alteración de documentos con el auxilio de expertos en escritura, papel y tinta. Ninguna de estas opciones está disponible electrónicamente. Es obvio que se requieren otras soluciones.

Antes de entrar en las soluciones mismas, vale la pena dedicar un instante a considerar el lugar que corresponde a la seguridad de las redes en la pila de protocolos. Probablemente no es un solo lugar. Cada capa tiene algo que contribuir. En la capa física podemos protegernos contra la intervención de las líneas encerrando las líneas de transmisión en tubos sellados que contienen gas argón a alta presión. Cualquier intento de hacer un agujero en el tubo liberará un poco de gas, reduciendo la presión y disparando una alarma. Algunos sistemas militares usan esta técnica.

En la capa de enlace de datos, los paquetes de una línea punto a punto pueden codificarse al dejar una máquina y descodificarse al llegar a otra. Los detalles pueden manejarse en la capa de enlace de datos, desconociendo las capas superiores lo que ocurre. Sin embargo, esta solución se viene abajo cuando los paquetes tienen que atravesar varios enrutadores, puesto que los paquetes tienen que descifrarse en cada enrutador, dejándolos vulnerables a ataques dentro del enrutador. Además, no se contempla que algunas sesiones estén protegidas (por ejemplo, aquellas que comprenden compras en línea mediante tarjeta de crédito) y otras no. No obstante, el **cifrado de enlace** (*link encryption*), como se llama a este método, puede agregarse fácilmente a cualquier red y con frecuencia es útil.

En la capa de red, pueden instalarse muros de seguridad para mantener adentro (o afuera) a los paquetes. Vimos los muros de seguridad en el capítulo 5. En la capa de transporte, pueden cifrarse conexiones enteras, terminal a terminal, es decir, proceso a proceso. Aunque estas soluciones ayudan en los asuntos relacionados con el secreto y mucha gente hace un gran esfuerzo para mejorarlas, ninguna de ellas resuelve los problemas de validación de identificación y no repudio de una manera suficientemente general. Para atacar estos problemas, las soluciones deben implementarse en la capa de aplicación, razón por la cual las estudiamos en este capítulo.

7.1.1. Cifrado tradicional

El cifrado tiene una larga y colorida historia. En esta sección simplemente delinearemos algunos de los puntos importantes, como antecedentes para lo que sigue. Si desea una historia completa, el libro de Kahn (1967) aún es lectura recomendada. Un estudio detallado de la tecnología de vanguardia actual puede encontrarse en (Kaufman *et al.*, 1995; Schneier, 1996, y Stinson, 1995).

Históricamente, cuatro grupos de gente han usado y contribuido al arte del cifrado: los militares, el cuerpo diplomático, los diaristas y los amantes. De éstos, los militares han tenido el papel más importante y han allanado el camino. Dentro de las organizaciones militares, los mensajes a cifrar tradicionalmente se han entregado a empleados mal pagados para su codificación y transmisión. El inmenso volumen de los mensajes ha impedido asignar esta labor a unos cuantos especialistas de élite.

Hasta la llegada de las computadoras, una de las restricciones principales del cifrado había sido la capacidad del empleado encargado de la codificación para realizar las transformaciones necesarias, frecuentemente en un campo de batalla con poco equipo. Una restricción adicional ha sido la dificultad de cambiar rápidamente de un método de cifrado a otro, puesto que esto significa el reentrenamiento de una gran cantidad de gente. Sin embargo, el peligro de que un empleado fuera capturado por el enemigo ha hecho indispensable la capacidad de cambiar el método de cifrado al instante, de ser necesario. De estos requisitos en conflicto se deriva el modelo de la figura 7-2.

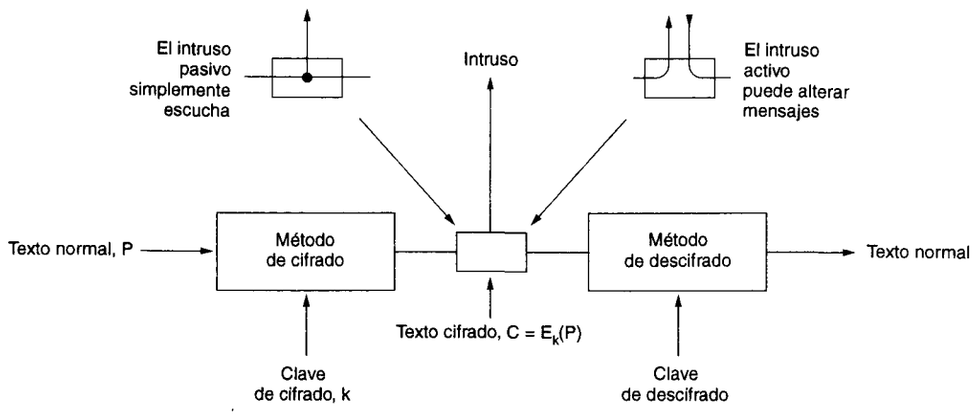


Figura 7-2. El modelo de cifrado.

Los mensajes a cifrar, conocidos como **texto normal** (*plain text*), se transforman mediante una función parametrizada por una **clave**. La salida del proceso de cifrado, conocida como **texto cifrado** (*ciphertext*), se transmite después, muchas veces mediante mensajero o radio. Suponemos que el enemigo, o **intruso**, escucha y copia con exactitud el texto cifrado completo. Sin

embargo, a diferencia del destinatario original, el intruso no conoce la clave de cifrado y no puede descifrar fácilmente el texto cifrado. A veces el intruso no sólo puede escuchar el canal de comunicación (intruso pasivo) sino que también puede registrar mensajes y reproducirlos después, inyectar sus propios mensajes y modificar los mensajes legítimos antes de que lleguen al destinatario (intruso activo). El arte de descifrar se llama **criptoanálisis**. El arte de diseñar cifradores (criptografía) y de descifrarlos (criptoanálisis) se conocen colectivamente como **criptología**.

A menudo resulta útil tener una notación para relacionar el texto normal, el texto cifrado y las claves. Usaremos $C = E_K(P)$ para indicar que el cifrado del texto normal P usando la clave K da el texto cifrado C. Del mismo modo, $P = D_K(C)$ representa el descifrado de C para obtener el texto normal nuevamente. Por tanto,

$$D_k(E_k(P)) = P$$

Esta notación sugiere que E y D son sólo funciones matemáticas, lo cual es cierto. El único truco es que ambas son funciones de dos parámetros, y hemos escrito uno de los parámetros (la clave) como subíndice, en lugar de como argumento, para distinguirlo del mensaje.

Una regla fundamental de la criptografía es que se debe suponer que el criptoanalista conoce el método general de cifrado usado. En otras palabras, el criptoanalista sabe cómo funciona el método de cifrado, E, de la figura 7-2. La cantidad de esfuerzo necesario para inventar, probar e instalar un método nuevo cada vez que el viejo está en peligro, o se piensa que lo está, siempre ha hecho impráctico mantenerlo en secreto, y el pensar que es secreto cuando no lo es hace más daño que bien.

Aquí es donde entra la clave. La clave consiste en una cadena corta (relativamente) que selecciona uno de muchos cifrados potenciales. En contraste con el método general, que tal vez se cambie cada cierto número de años, la clave puede cambiarse con la frecuencia requerida. Por tanto, nuestro modelo básico es un método general estable y conocido públicamente pero parametrizado por una clave secreta y fácilmente cambiable.

La naturaleza no secreta del algoritmo debe subrayarse. Al hacer público el algoritmo, el criptógrafo recibe asesoría gratuita de una gran cantidad de criptólogos académicos ansiosos por descifrar el código del sistema para que puedan publicar trabajos demostrando su inteligencia. Si muchos expertos han tratado de descifrar el algoritmo durante cinco años después de su publicación y nadie lo ha logrado, probablemente es bastante sólido.

El secreto real es la clave, y su longitud es un aspecto importante del diseño. Considere una cerradura de combinación. El principio general es que se introducen dígitos en secuencia. Todo el mundo lo sabe, pero la clave es secreta. Una longitud de clave de dos dígitos significa que hay 100 posibilidades. Una clave de tres dígitos de longitud significa 1000 posibilidades, y una clave de seis dígitos de longitud significa un millón. Cuanto más grande es la clave, mayor será el **factor de trabajo** que tendrá que enfrentar el criptoanalista. El factor de trabajo para descifrar el sistema mediante una búsqueda exhaustiva del espacio de claves crece exponencialmente con la longitud de la clave. El secreto radica en tener un algoritmo robusto (pero público) y una clave larga. Para evitar que su hermano menor lea su correo electrónico, las claves de 64 bits son suficientes. Para mantener a raya gobiernos poderosos, se requieren claves de hasta 256 bits.

Desde el punto de vista del criptoanalista, el problema del criptoanálisis tiene tres variaciones principales. Cuando él tiene cierta cantidad de texto cifrado pero no tiene texto común, enfrenta el problema de **sólo texto cifrado**. Los criptogramas que aparecen en la sección de acertijos de los diarios presentan este tipo de problema. Cuando se tiene un poco de texto cifrado y el texto normal correspondiente, el problema se conoce como **texto normal conocido** (*known plaintext*). Por último, cuando el criptoanalista tiene la capacidad de cifrar textos normales que él escoge, enfrenta el problema de **texto normal seleccionado** (*chosen plaintext*). Los criptogramas de los periódicos podrían descifrarse trivialmente si el criptoanalista pudiese hacer preguntas como: ¿cuál es el cifrado de ABCDE?

Los novatos en el negocio del cifrado con frecuencia suponen que, si un cifrado puede resistir un ataque de sólo texto cifrado, entonces es seguro. Este supuesto es muy ingenuo. En muchos casos, el criptoanalista puede hacer una buena estimación de partes del texto normal. Por ejemplo, lo primero que muchos sistemas de tiempo compartido dicen cuando se les llama es "INDIQUE SU CLAVE". Equipado con algunos pares concordantes de texto normal-texto cifrado, el trabajo del criptoanalista se vuelve mucho más fácil. Para lograr seguridad, el criptógrafo debe ser conservador y asegurarse que el sistema sea inviolable aun si su oponente puede cifrar cantidades arbitrarias de texto normal seleccionado.

Los modelos de cifrado históricamente se dividen en dos categorías: cifrados por sustitución y cifrados por transposición. Ahora reseñaremos cada uno de estos brevemente como antecedente para la criptografía moderna.

Cifrados por sustitución

En un **cifrado por sustitución**, cada letra o grupo de letras se reemplaza por otra letra o grupo de letras para disfrazarla. Uno de los cifrados más viejos conocidos es el **cifrado de César**, atribuido a Julio César. En este método, *a* se vuelve *D*, *b* se vuelve *E*, *c* se vuelve *F*, ... , y *z* se vuelve *C*. Por ejemplo, *ataque* se vuelve *DWDTXH*. En los ejemplos, el texto normal se presentará en minúsculas y el texto cifrado en mayúsculas.

Una pequeña generalización del cifrado de César permite que el alfabeto de texto cifrado se desplace *k* letras, en lugar de siempre 3. En este caso, *k* se convierte en una clave del método general de alfabetos desplazados circularmente. El cifrado de César posiblemente engañó a los cartagineses, pero no ha engañado a nadie desde entonces.

La siguiente mejora es hacer que cada uno de los símbolos del texto normal, digamos las 26 letras del abecedario (no incluimos la *ñ*) inglés, tengan una correspondencia con alguna otra letra. Por ejemplo,

```
texto normal:   a b c d e f g h i j k l m n o p q r s t u v w x y z
texto cifrado:  Q W E R T Y U I O P A S D F G H J K L Z X C V B N M
```

Este sistema general se llama **sustitución monoalfabética**, siendo la clave la cadena de 26 letras correspondiente al alfabeto completo. Para la clave anterior, el texto normal *ataque* se transformaría en el texto cifrado *QZQJXT*.

A primera vista, esto podría parecer un sistema seguro, porque, aunque el criptoanalista conoce el sistema general (sustitución letra por letra), no sabe cuál de las $26! \approx 4 \times 10^{26}$ claves posibles se está usando. En contraste con el cifrado de César, intentarlas todas no es un enfoque muy prometedor. Aun a 1 μseg por solución, una computadora tardaría 10^{13} años en intentar todas las claves.

No obstante, si se cuenta con una cantidad aún pequeña de texto cifrado, el cifrado puede descifrarse fácilmente. El ataque básico aprovecha las propiedades estadísticas de los lenguajes naturales. En inglés, por ejemplo (que es el idioma del autor de este libro), la *e* es la letra más común, seguida de *t, o, a, n, i,* etc. Las combinaciones de dos letras más comunes, o **digramas**, son *th, in, er, re* y *an*. Las combinaciones de tres letras más comunes, o **trigramas**, son *the, ing, and* e *ion*.

Un criptoanalista que intenta descifrar una codificación monoalfabética comenzaría por contar la frecuencia relativa de todas las letras del texto cifrado. Entonces podría asignar tentativamente la más común a la letra *e* y la siguiente más común a la letra *t*. Vería entonces los trigramas para encontrar uno común de la forma *tXe*, lo que sugerirá fuertemente que *X* es *h*. De la misma manera, si el patrón *thYt* ocurre con frecuencia, *Y* probablemente representa *a*. Con esta información se puede buscar un trigrama frecuente de la forma *aZW*, que con mucha probabilidad es *and*. Adivinando las palabras comunes, digramas y trigramas, y conociendo los patrones probables de las vocales y consonantes, el criptoanalista construye un texto normal tentativo, letra por letra.

Otro enfoque es adivinar una palabra o frase probable. Por ejemplo, considere el siguiente texto cifrado de una compañía contable (en bloques de cinco caracteres):

```
CTBMN BYCTC BTJDS QXBNS GSTJC BTSWX CTQTZ CQVUJ
QJSGS TJQZZ MNQJS VLNSX VSZJU JDSTS JQUUS JUBXJ
DSKSU JSNTK BGAQJ ZBGYQ TLCTZ BNYBN QJSW
```

Una palabra muy probable en un mensaje de una compañía contable de un país de habla inglesa es *financial*. Usando nuestro conocimiento de que *financial* tiene una letra repetida (*i*), con cuatro letras intermedias entre su aparición, buscamos letras repetidas en el texto cifrado con este espaciado. Encontramos 12 casos, en las posiciones 6, 15, 27, 31, 42, 48, 56, 66, 70, 71, 76 y 82. Sin embargo, sólo dos de éstos, el 31 y 42, tienen la siguiente letra (correspondiente a *n* en el texto normal) repetida en el lugar adecuado. De estos dos, sólo el 31 tiene también la *a* en la posición correcta, por lo que sabemos que *financial* comienza en la posición 30. A partir de aquí, la deducción de la clave es fácil usando las estadísticas de frecuencia del texto en inglés.

Cifrados por transposición

Los cifrados por sustitución conservan el orden de los símbolos de texto normal, pero los disfrazan. Los **cifrados por transposición**, en contraste, reordenan las letras pero no las disfrazan. En la figura 7-3 se presenta un cifrado de transposición común, la transposición columnar.

La clave del cifrado es una palabra o frase que no contiene letras repetidas. En este ejemplo, la clave es MEGABUCK. El propósito de la clave es numerar las columnas, estando la columna 1 bajo la letra clave más cercana al inicio del alfabeto, y así sucesivamente. El texto normal se escribe horizontalmente, en filas. El texto cifrado se lee por columnas, comenzando por la columna cuya letra clave es la más baja.

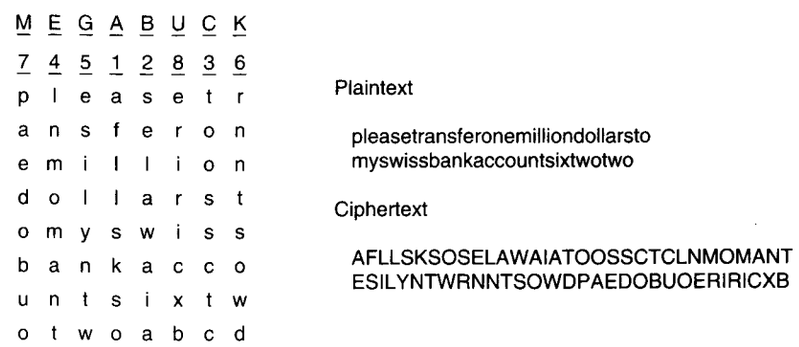


Figura 7-3. Cifrado por transposición.

Para descifrar un cifrado por transposición, el criptoanalista debe primero estar consciente de que está tratando con un cifrado de transposición. Observando la frecuencia de *E, T, A, O, I, N,* etc., es fácil ver si se ajustan al patrón usual del texto normal. De ser así, es evidente que se trata de un cifrado por transposición, pues en tal cifrado cada letra se representa a sí misma.

El siguiente paso es adivinar la cantidad de columnas. En muchos casos, puede adivinarse una palabra o frase probable por el contexto del mensaje. Por ejemplo, supóngase que nuestro criptoanalista sospecha que la frase de texto normal *milliondollars* aparece en algún lugar del mensaje. Observe que los digramas *MO, IL, LL, LA, IR* y *OS* ocurren en el texto cifrado como resultado de que esta frase da la vuelta. La letra de texto cifrado *O* sigue a la letra de texto cifrado *M* (es decir, son adyacentes verticalmente en la columna 4) puesto que están separados en la frase probable por una distancia igual a la longitud de la clave. Si se hubiera usado una clave de longitud siete, habrían ocurrido los digramas *MD, IO, LL, LL, IA, OR* y *NS*. De hecho, para cada longitud de clave, se produce un grupo diferente de digramas en el texto cifrado. Buscando las diferentes posibilidades, el criptoanalista con frecuencia puede determinar fácilmente la longitud de la clave.

El paso restante es ordenar las columnas. Cuando la cantidad de columnas, k, es pequeña, puede examinarse cada uno de los pares de columnas $k(k-1)$ para ver si la frecuencia de sus digramas es igual a la del texto normal. El par con la mejor concordancia se supone correctamente ubicado. Ahora cada columna restante se prueba tentativamente como el sucesor de este par. La columna cuyas frecuencias de digramas y trigramas produce la mejor concordancia se toma tentativamente como correcta. La columna antecesora se encuentra de la misma manera. El proceso completo se repita hasta encontrar un orden potencial. Es probable que el texto normal sea reconocible en este punto (por ejemplo, si aparece *milloin*, quedará claro en dónde está el error).

Algunos cifrados de transposición aceptan un bloque de longitud fija como entrada y producen un bloque de longitud fija como salida. Estos cifrados pueden describirse por completo con sólo dar una lista que indique el orden en el que deben salir los caracteres. Por ejemplo, el cifrado de la figura 7-3 puede verse como un cifrado de bloque de 64 caracteres. Su salida es 4, 12, 20, 28, 36, 44, 52, 60, 5, 13, . . . , 62. En otras palabras, el cuarto carácter de entrada, *a*, es el primero en salir, seguido del décimosegundo, *f*, etcétera.

Rellenos de una sola vez

La construcción de un cifrado inviolable en realidad es bastante sencilla; la técnica se conoce desde hace décadas. Primero se escoge una cadena de bits al azar como clave. Luego se convierte el texto normal en una cadena de bits, por ejemplo usando su representación ASCII. Por último, se calcula el OR EXCLUSIVO de estas dos cadenas, bit por bit. El texto cifrado resultante no puede descifrarse porque cada texto normal posible es un candidato igualmente probable. El texto cifrado no proporciona al criptoanalista ninguna información en absoluto. En una muestra suficientemente grande de texto cifrado, cada letra ocurrirá con la misma frecuencia, al igual que cada digrama y cada trigrama.

Desafortunadamente, este método, conocido como **relleno de una sola vez** (*one-time pad*), tiene varias desventajas prácticas. Para comenzar, la clave no puede memorizarse, por lo que tanto el transmisor como el receptor deben llevar una copia por escrito consigo. Si cualquiera de ellos puede ser capturado, las claves escritas evidentemente son indeseables. Además, la cantidad total de datos que puede transmitirse está limitada a la cantidad de clave disponible. Si el espía tiene suerte y descubre abundante información, podría ser incapaz de transmitirlos todos a su base porque la clave se agotó. Otro problema es la sensibilidad del método a la pérdida o inserción de caracteres. Si el transmisor y el receptor pierden la sincronía, todos los datos a partir de ahí aparecerán alterados.

Con la llegada de las computadoras, el relleno de una sola vez podría volverse práctico para algunas aplicaciones. La fuente de la clave podría ser un CD especial que contenga varios gigabits de información, y si se transporta en una caja de CD de música y va precedida por algunas canciones, ni siquiera resultaría sospechoso. Por supuesto, a velocidades de red de gigabits, tener que introducir un CD nuevo cada cinco segundos podría volverse tedioso. Por esta razón, comenzaremos ahora a ver los algoritmos modernos de cifrado que pueden procesar cantidades arbitrariamente grandes de texto normal.

7.1.2. Dos principios criptográficos fundamentales

Aunque estudiaremos muchos sistemas criptográficos diferentes en las siguientes páginas, hay dos principios que los sostienen a todos y que es importante entender. El primer principio es que todos los mensajes cifrados deben contener redundancia, es decir, información no necesaria para

entender el mensaje. Un ejemplo puede dejar en claro la necesidad de esto. Considere una compañía de compras por teléfono, El Perezoso (EP), que tiene 60,000 productos. Pensando que son muy eficientes, los programadores de EP deciden que los mensajes de pedidos deben consistir en un nombre de cliente de 16 bytes seguido de un campo de datos de 3 bytes (1 byte para la cantidad y 2 para el número del producto). Los últimos 3 bytes deben cifrarse usando una clave muy grande conocida sólo por el cliente y EP.

Inicialmente, esto podría parecer seguro, y en cierto sentido lo es, puesto que los intrusos pasivos no pueden descifrar los mensajes. Desafortunadamente, el sistema también tiene una falla mortal que lo vuelve inútil. Supóngase que un empleado recientemente despedido quiere vengarse de EP por cesarlo. Antes de irse, se lleva con él (parte de) la lista de clientes; entonces trabaja toda la noche escribiendo un programa para generar pedidos ficticios usando nombres reales de los clientes. Dado que no tiene la lista de claves, simplemente pone números aleatorios en los últimos 3 bytes y envía cientos de pedidos a EP.

Al llegar estos mensajes, la computadora de EP usa el nombre del cliente para localizar la clave y descifrar el mensaje. Para mala suerte de EP, casi cada mensaje de 3 bytes es válido, por lo que la computadora comienza a imprimir instrucciones de embarque. Aunque podría parecer extraño que un cliente ordene 137 juegos de columpios para niño, o 240 cajas de arena, en lo que a la computadora concierne el cliente bien podría estar planeando abrir una cadena de lotes con juegos infantiles. De esta manera, un intruso activo (el ex empleado) puede causar muchísimos problemas, aun cuando no pueda entender los mensajes que genera su computadora.

Este problema puede *resolverse* agregando redundancia a todos los mensajes. Por ejemplo, si se extienden los mensajes de pedido a 12 bytes, de los cuales los primeros 9 deben ser ceros, entonces este ataque ya no funciona, porque el ex empleado ya no puede generar una cadena grande de mensajes válidos. La moraleja de esta historia es que todos los mensajes deben contener una cantidad considerable de redundancia para que los intrusos activos no puedan enviar basura al azar y lograr que se interprete como mensajes válidos.

Sin embargo, la adición de redundancia también simplifica a los criptoanalistas el descifrado de los mensajes. Supóngase que el negocio de pedidos por correo es altamente competido, y que la competencia principal de El Perezoso, El Bolsón, estaría encantado de conocer la cantidad de cajas de arena que EP vende. Para ello, ha intervenido la línea telefónica de EP. En el esquema original con mensajes de 3 bytes, el criptoanálisis era prácticamente imposible puesto que, tras adivinar una clave, el criptoanalista no tenía manera de saber si había adivinado correctamente. A fin de cuentas, casi cualquier mensaje era legal. Con el nuevo esquema de 12 bytes, es fácil para el criptoanalista distinguir un mensaje válido de uno no válido.

Por tanto, el principio criptográfico número uno es que todos los mensajes deben contener redundancia para evitar que los intrusos activos engañen al receptor y lo hagan actuar ante un mensaje falso. Sin embargo, esta misma redundancia simplifica mucho la violación del sistema por parte de los intrusos pasivos, por lo que aquí tenemos un poco de tensión. Es más, la redundancia nunca debe tener la forma de n ceros al inicio o al final del mensaje, ya que el análisis de tales mensajes con algunos algoritmos criptográficos da resultados más predecibles, simplificando la tarea del criptoanalista. Una cadena aleatoria de palabras sería mejor para incluir la redundancia.

El segundo principio criptográfico es que deben tomarse algunas medidas para evitar que los intrusos activos reproduzcan mensajes viejos. Si no se toman tales medidas, nuestro ex empleado podría conectarse a la línea telefónica de EP y simplemente continuar repitiendo mensajes válidos enviados previamente. Una de tales medidas es la inclusión en cada mensaje de una marca de tiempo válida durante, digamos, 5 minutos. El receptor puede entonces guardar los mensajes unos 5 minutos, para compararlos con los mensajes nuevos que lleguen y filtrar los duplicados. Los mensajes con mayor antigüedad que 5 minutos pueden descartarse, dado que todas las repeticiones enviadas más de 5 minutos después también se rechazarán como demasiado viejas. Más adelante estudiaremos medidas diferentes de las marcas de tiempo.

7.1.3. Algoritmos de clave secreta

La criptografía moderna usa las mismas ideas básicas que la criptografía tradicional, la transposición y la sustitución, pero su orientación es distinta. Tradicionalmente, los criptógrafos han usado algoritmos sencillos y se han apoyado en claves muy largas para la seguridad. Hoy día es cierto lo inverso: el objetivo es hacer el algoritmo de cifrado tan complicado y rebuscado que inclusive si el criptoanalista obtiene cantidades enormes de texto cifrado a su gusto, no será capaz de entender nada.

Las transposiciones y las sustituciones pueden implantarse mediante circuitos sencillos. En la figura 7-4(a) se muestra un dispositivo, conocido como **caja P** (la P significa permutación), que se usa para efectuar una transposición de una entrada de 8 bits. Si se designan los 8 bits de arriba hacia abajo como 01234567, la salida de esta caja P en particular es 36071245. Mediante el alambrado interno adecuado, puede hacerse que una caja P efectúe cualquier transposición, y lo puede hacer prácticamente a la velocidad de la luz.

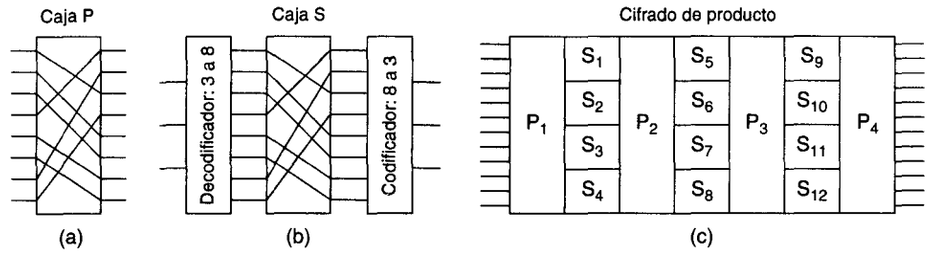


Figura 7-4. Elementos básicos del cifrado de producto. (a) Caja P. (b) Caja S. (c) Producto.

Las sustituciones se llevan a cabo mediante **cajas S**, como se muestra en la figura 7-4(b). En este ejemplo, se ingresa un texto normal de 3 bits y sale un texto cifrado de 3 bits. La entrada de 3 bits selecciona una de las ocho líneas de salida de la primera etapa y la establece en 1; las demás líneas son 0. La segunda etapa es una caja P. La tercera etapa codifica en binario nuevamente la línea de entrada seleccionada. Con el alambrado que se muestra, si los ocho números

octales 01234567 entraran uno tras otro, la secuencia de salida sería de 24506713. En otras palabras se ha reemplazado el 0 por el 2, el 1 por el 4, etc. Nuevamente, mediante el alambrado adecuado de la caja P dentro de la caja S, puede lograrse cualquier sustitución.

La potencia real de estos elementos básicos sólo se hace aparente cuando ponemos en cascada una serie completa de cajas para formar un **cifrado de producto**, como se muestra en la figura 7-4(c). En este ejemplo, se transponen 12 líneas de entrada en la primera etapa. En teoría, sería posible hacer que la segunda etapa sea una caja S que relacione un número de 12 bits con otro número de 12 bits. En vez de ello, tal dispositivo requeriría $2^{12} = 4096$ alambres cruzados en su etapa media. En cambio, la entrada se divide en cuatro grupos de 3 bits, cada uno de los cuales se sustituye independientemente de los demás. Aunque este método es menos general, aún es poderoso. Mediante la inclusión de una cantidad suficiente de etapas en el cifrado de producto, puede hacerse de la salida una función excesivamente complicada de la entrada.

DES

En enero de 1977, el gobierno de Estados Unidos adoptó un cifrado de producto desarrollado por IBM como su estándar oficial para información no secreta. Este cifrado, el **DES** (*Data Encryption Standard*, **estándar de cifrado de datos**), se adoptó ampliamente en la industria para usarse con productos de seguridad. Ya no es seguro en su forma original (Wayner, 1995), pero aún es útil en una forma modificada. Ahora explicaremos el funcionamiento del DES.

En la figura 7-5(a) se muestra un esbozo del DES. El texto normal se cifra en bloques de 64 bits, produciendo 64 bits de texto cifrado. El algoritmo, que se parametriza mediante una clave de 56 bits, tiene 19 etapas diferentes. La primera etapa es una transposición, independiente de la clave, del texto normal de 64 bits. La última etapa es el inverso exacto de esta transposición. La etapa previa a la última intercambia los 32 bits de la izquierda y los 32 bits de la derecha. Las 16 etapas restantes son funcionalmente idénticas, pero se parametrizan mediante diferentes funciones de la clave. El algoritmo se ha diseñado para permitir que el descifrado se haga con la misma clave que el cifrado. Los pasos simplemente se ejecutan en el orden inverso.

La operación de una de estas etapas intermedias se ilustra en la figura 7-5(b). Cada etapa toma dos entradas de 32 bits y produce dos salidas de 32 bits. La salida de la izquierda simplemente es una copia de la entrada de la derecha. La salida de la derecha es el OR EXCLUSIVO a nivel de bit de la entrada izquierda y una función de la entrada derecha y la clave de esta etapa, K_i. Toda la complejidad reside en esta función.

La función consiste en cuatro pasos, ejecutados en secuencia. Primero se construye un número de 48 bits, E, expandiendo el R_{i-1} de 32 bits según una regla fija de transposición y duplicación. Después, se aplica un OR EXCLUSIVO a E y K_i. Esta salida entonces se divide en ocho grupos de 6 bits, cada uno de los cuales se alimenta a una caja S distinta. Cada una de las 64 entradas posibles a la caja S se transforma en una salida de 4 bits. Por último, estos 8×4 bits se pasan a través de una caja P.

En cada una de las 16 iteraciones, se usa una clave diferente. Antes de iniciarse el algoritmo, se aplica una transposición de 56 bits a la clave. Justo antes de cada iteración, la clave se divide

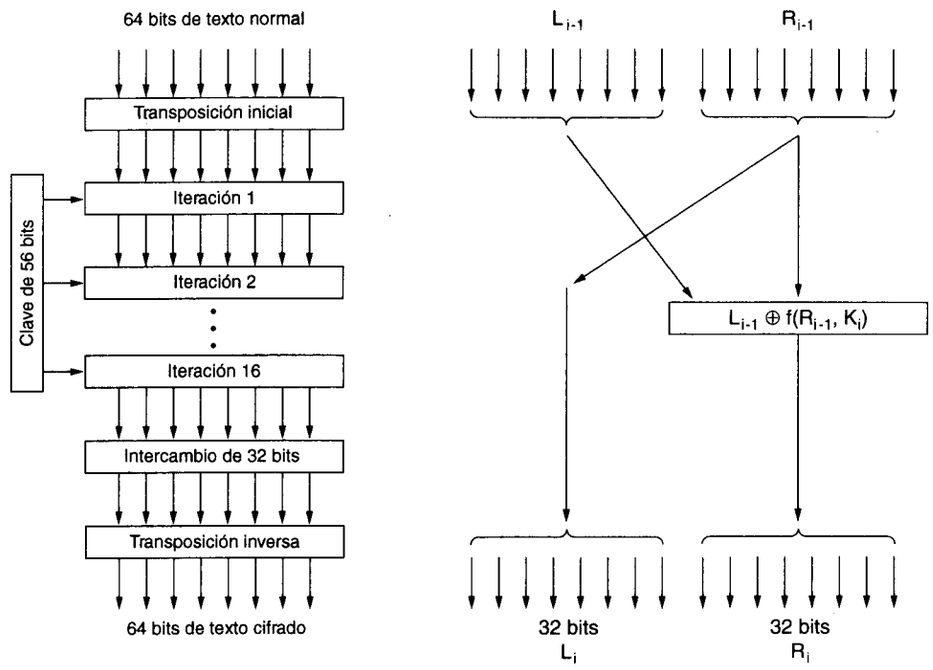


Figura 7-5. El estándar de cifrado de datos. (a) Esbozo general. (b) Detalle de una iteración.

en dos unidades de 28 bits, cada una de las cuales se gira hacia la izquierda una cantidad de bits dependiente del número de iteración. K_i se deriva de esta clave girada aplicándole otra transposición de 56 bits. En cada vuelta se extrae y permuta de los 56 bits un subgrupo de 48 bits diferente.

Encadenamiento DES

A pesar de toda esta complejidad, el DES básicamente es un cifrado por sustitución monoalfabética que usa un carácter de 64 bits. Cada vez que entra el mismo bloque de texto normal de 64 bits por el frente, sale el mismo bloque de texto cifrado de 64 bits por atrás. Un criptoanalista puede explotar esta propiedad como ayuda para violar el DES.

Para ver la manera en que esta propiedad de cifrado por sustitución monoalfabética puede usarse para subvertir el DES, consideremos el cifrado de un mensaje grande de la manera obvia: dividiéndolo en bloques consecutivos de 8 bytes (64 bits) y cifrándolos uno tras otro con la misma clave. El último bloque se rellena a 64 bits, de ser necesario. Esta técnica se conoce como **modo de libro de código electrónico**.

En la figura 7-6 tenemos el comienzo de un archivo de computadora que lista los bonos anuales que ha decidido otorgar una compañía a sus empleados. Este archivo consiste en registros consecutivos de 32 bytes, uno por empleado, en el formato que se muestra: 16 bytes para el nombre, 8 bytes para el puesto y 8 bytes para el bono. Cada uno de los 16 bloques de 8 bytes (numerados del 0 al 15) se cifra con el DES.

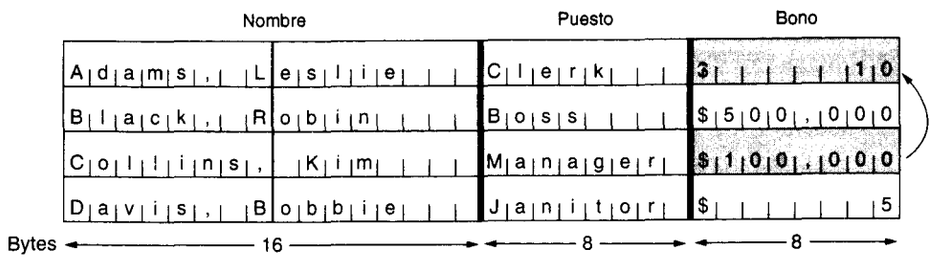


Figura 7-6. El texto normal de un archivo cifrado como 16 bloques DES.

Leslie acaba de pelearse con el jefe y no espera un bono muy grande. Kim, en cambio, es la favorita del jefe, y todo el mundo lo sabe. Leslie puede acceder al archivo después de ser cifrado pero antes de su envío al banco. ¿Puede Leslie rectificar esta situación injusta, dado sólo el archivo cifrado?

No hay problema. Todo lo que necesita Leslie es hacer una copia del bloque de texto cifrado 11 (que contiene el bono de Kim) y usarlo para reemplazar el bloque de texto cifrado 3 (que contiene el bono de Leslie). Aun sin saber lo que dice el bloque 11, Leslie puede esperar tener una Navidad bastante feliz este año. (Copiar el bloque de texto cifrado 7 es otra posibilidad, pero es más fácil su detección; además, Leslie no es una persona codiciosa.)

Para frustrar este tipo de ataque, el DES (y todos los cifrados en bloques) puede encadenarse de varias maneras para que el reemplazo de un bloque de la manera en que lo hizo Leslie haga que el texto normal descifrado comenzando por el bloque reemplazado sea basura. Una forma de encadenar es por **encadenamiento de bloque cifrado**. En este método, que se muestra en la figura 7-7, a cada bloque de texto normal se le hace un OR EXCLUSIVO (#) con el bloque de texto cifrado previo antes de cifrarse. En consecuencia, el mismo bloque de texto normal no corresponde con el mismo bloque de texto cifrado, y el cifrado ya no es un cifrado por sustitución monoalfabética grande. Al primer bloque se le hace un OR EXCLUSIVO con un **vector de inicialización, IV**, seleccionado al azar, que se transmite junto con el texto cifrado.

Podemos ver cómo funciona el encadenamiento de bloques cifrados examinando el ejemplo de la figura 7-7. Comenzamos por calcular $C_0 = E(P_0 \text{ XOR } IV)$. Después calculamos $C_1 = E(P_1 \text{ XOR } C_0)$, etc. El descifrado funciona de la manera opuesta, con $P_0 = IV \text{ XOR } D(C_0)$, etc. Nótese que el cifrado del bloque i es una función de todo el texto normal de los bloques 0 a $i - 1$, por lo que el texto normal genera un texto cifrado diferente dependiendo de dónde ocurre. Una

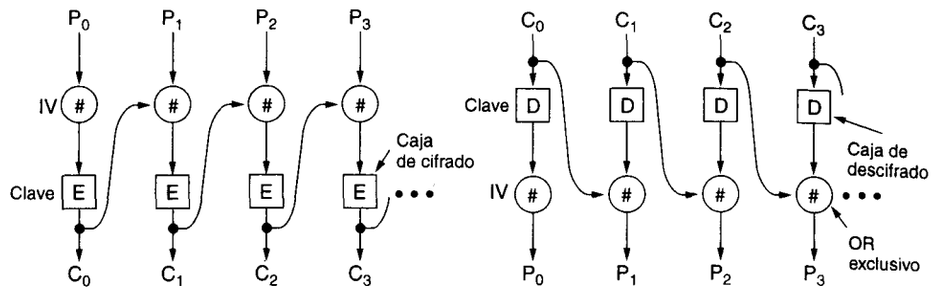


Figura 7-7. Encadenamiento de bloques cifrados.

transformación como la que hizo Leslie produciría basura durante dos bloques, comenzando por el campo de bono de Leslie. Para un oficial de seguridad astuto, esta peculiaridad sugeriría dónde se podría iniciar la investigación subsecuente.

El encadenamiento de bloques cifrados también tiene la ventaja de que el mismo bloque de texto normal no produce el mismo bloque de texto cifrado, dificultando el criptoanálisis. De hecho, ésta es la razón principal de su uso.

Sin embargo, el encadenamiento de bloques cifrados tiene la desventaja de requerir la llegada de un bloque completo de 64 bits antes de poder iniciar el descifrado. Para usarse con terminales interactivas, en las que la gente puede escribir líneas de menos de ocho caracteres y luego detenerse, esperando una respuesta, este modo no es apropiado. Para el cifrado byte por byte puede usarse el **modo de realimentación de cifrado**, que se ilustra en la figura 7-8. En esta figura, se muestra el estado de la máquina de cifrado después de cifrar y enviar los bytes 0 a 9. Al llegar el byte de texto normal 10, como se aprecia en la figura 7-8(a), el algoritmo DES opera con el registro de desplazamiento de 64 bits para generar un texto cifrado de 64 bits. Se extrae el byte de la izquierda de ese texto cifrado y se le aplica un OR EXCLUSIVO con P_{10}. Ese byte se envía por la línea de transmisión. Además, el registro de desplazamiento se desplaza a la izquierda 8 bits, causando la expulsión de C_2 por la izquierda y la introducción de C_{10} en la posición que C_9 dejó vacante a la derecha. Nótese que el contenido del registro de desplazamiento depende de la historia previa completa del texto normal, por lo que un patrón que se repite varias veces en el texto normal se cifrará de manera diferente en cada ocasión. Como ocurre con el encadenamiento de bloques cifrados, se requiere un vector de inicialización para echar a andar el mecanismo.

El descifrado con modo de realimentación de cifrado simplemente hace lo mismo que el cifrado. En particular, el contenido del registro de desplazamiento se *cifra*, no se *descifra*, por lo que el byte seleccionado al que se le hace el OR EXCLUSIVO con C_{10} para obtener P_{10} es el mismo que aquel al que se le hizo el OR EXCLUSIVO con P_{10} para generar C_{10} inicialmente. Mientras los dos registros de desplazamiento permanezcan idénticos, el descifrado funcionará correctamente.

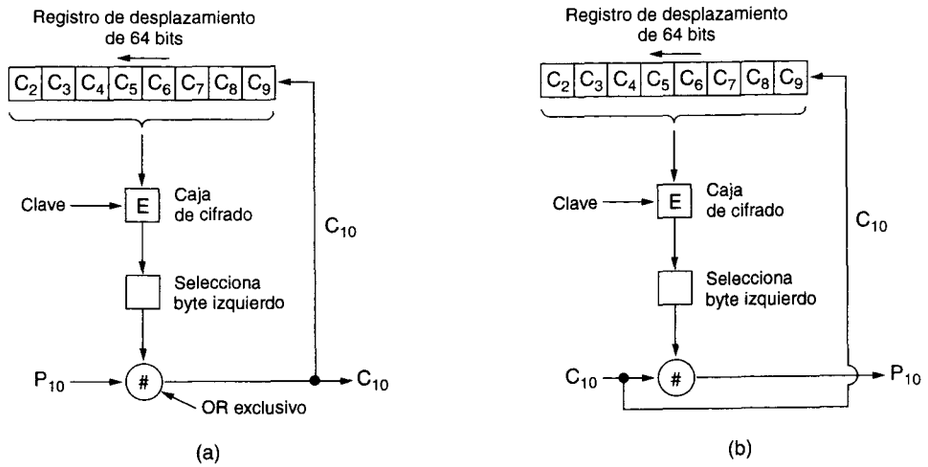


Figura 7-8. Modo de realimentación de cifrado.

Como nota al margen, debe indicarse que, si un bit del texto cifrado accidentalmente se invierte durante la transmisión, los 8 bytes descifrados cuando el byte malo esté en el registro de desplazamiento tendrán error. Una vez que el byte equivocado sea expulsado del registro de desplazamiento, se generará nuevamente texto normal correcto. Por tanto, el efecto de un solo bit invertido está bastante delimitado y no arruina el resto del mensaje.

Sin embargo, existen aplicaciones en las que un error de transmisión de 1 bit que arruina 64 bits de texto normal es un efecto demasiado grande. Para estas aplicaciones existe una cuarta opción, el **modo de realimentación de salida**, que es idéntico al modo de realimentación de cifrado, excepto que el byte realimentado por el lado derecho del registro de desplazamiento se toma justo antes de la caja de OR EXCLUSIVO, no justo después.

El modo de realimentación de salida tiene la propiedad de que un error de 1 bit en el texto cifrado causa un error de 1 solo bit en el texto normal resultante. Por otra parte, es menos seguro que los otros modos, y debe evitarse su uso de propósito general. El modo de libro de código electrónico también debe evitarse excepto en circunstancias especiales (por ejemplo, el cifrado de un solo número aleatorio, como una clave de sesión). Para la operación normal, debe usarse el encadenamiento de bloques cifrados cuando la entrada llega en unidades de 8 bytes (por ejemplo, para cifrar archivos de disco) y el modo de realimentación de cifrado debe usarse para cadenas de entrada irregulares, como la entrada de un teclado.

Descifrado del DES

El DES ha estado rodeado de controversias desde el día en que se lanzó; se basó en un cifrado desarrollado y patentado por IBM, llamado Lucifer, excepto que el cifrado de IBM usaba una clave de 128 bits en lugar de 56 bits. Cuando el gobierno federal de Estados Unidos quiso

estandarizar un método de cifrado para uso no confidencial, "invitó" a IBM a "debatir" el asunto con la NSA, la dependencia de descifrado de códigos del gobierno, que es el empleador de matemáticos y criptólogos más grande del mundo. La NSA es tan secreta que una broma de la industria reza:

Pregunta: ¿Qué significa NSA?
Respuesta: Ninguna Supuesta Agencia.

En realidad, NSA significa *National Security Agency*, Agencia Nacional de Seguridad.

Tras estos debates, la IBM redujo la clave de 128 a 56 bits y decidió mantener en secreto el proceso de diseño del DES. Mucha gente sospechó que la longitud de la clave se redujo para asegurar que la NSA pudiera descifrar el código, pero no ninguna organización de menor presupuesto. El objetivo del diseño secreto supuestamente era esconder una puerta secreta que pudiera facilitar aún más el descifrado del DES por la NSA. Cuando un empleado de la NSA discretamente pidió al IEEE que cancelara una conferencia planeada sobre criptografía, eso incomodó aún más a la gente.

En 1977, dos investigadores de criptografía de Stanford, Diffie y Hellman (1977) diseñaron una máquina para violar el DES y estimaron que podría construirse por unos 20 millones de dólares. Dado un trozo pequeño de texto normal y el texto cifrado correspondiente, esta máquina podría encontrar la clave mediante una búsqueda exhaustiva del espacio de claves de 2^{56} en menos de un día. Hoy día tal máquina costaría tal vez un millón de dólares. En (Wiener, 1994) se presenta un diseño detallado de una máquina que puede descifrar el DES mediante búsqueda exhaustiva en unas cuatro horas.

Aquí presentamos otra estrategia. Aunque el cifrado por *software* es 1000 veces más lento que el cifrado por *hardware*, una computadora casera de alto nivel aún podría efectuar unos 250,000 cifrados/seg en *software*, y es probable que esté ociosa unos 2 millones de segundos al mes. Este tiempo de inactividad podría usarse para descifrar el DES. Si alguien pusiera un mensaje en uno de los populares grupos de noticias de Internet, no debería ser difícil conseguir las 140,000 personas necesarias para revisar las 7×10^{16} claves en un mes.

Probablemente la idea más innovadora para descifrar el DES es la **lotería china** (Quisquater y Girault, 1991). En este diseño, cada radio y televisión tiene que equiparse con un *chip* DES barato capaz de realizar 1 millón de cifrados/seg en *hardware*. Suponiendo que cada una de las 1.2 mil millones de personas de China tiene un radio o televisión, cada vez que el gobierno quiera descifrar un mensaje codificado con DES, simplemente difunde el par texto normal/texto cifrado, y cada uno de los 1.2 mil millones de *chips* comienza a buscar su sección preasignada del espacio de claves. En 60 segundos se encontrarán una o más correspondencias. Para asegurar que se informen, los *chips* podrían programarse para desplegar o anunciar el mensaje:

¡FELICIDADES! ACABA DE GANAR LA LOTERÍA CHINA. PARA COBRAR SU PREMIO, POR FAVOR LLAME AL 1-800-GRAN-PREMIO.

La conclusión que se puede obtener de estos argumentos es que el DES ya no debería usarse para nada importante. Sin embargo, aunque 2^{56} es un miserable 7×10^{16}, 2^{112} es un magnífico

5×10^{33}. Aun con mil millones de *chips* DES efectuando mil millones de operaciones por segundo, se requerirían 100 millones de años para examinar detalladamente un espacio de claves de 112 bits. Por tanto, surge la idea de simplemente ejecutar el DES dos veces, con dos claves de 56 bits diferentes.

Desgraciadamente, Merkle y Hellman (1981) han desarrollado un método que hace sospechoso al doble cifrado. Se llama ataque de **encuentro a la mitad** (*meet-in-the-middle*) y funciona como sigue (Hellman, 1980). Supóngase que alguien ha cifrado doblemente una serie de bloques de texto normal usando el modo de libro de código electrónico. Para unos pocos valores de i, el criptoanalista tiene pares igualados (P_i, C_i) donde

$$Ci = E_{k2}(E_{k1}(P_i))$$

Si ahora aplicamos la función de descifrado, D_{K2}, a cada lado de esta ecuación, obtenemos

$$D_{k2}(C_i) = E_{k1}(P_i) \tag{7-1}$$

porque el cifrado de x y su descifrado posterior con la misma clave produce x.

El ataque de encuentro a la mitad usa esta ecuación para encontrar las claves DES, $K1$ y $K2$, como sigue:

1. Calcular $R_i = E_i(P_1)$ para los 2^{56} valores de i, donde E es la función de cifrado DES. Ordenar esta tabla en orden ascendente según R_i.
2. Calcular $S_j = D_j(C_1)$ para todos los 2^{56} valores de j, donde D es la función de descifrado DES. Ordenar esta tabla en orden ascendente según S_j.
3. Barrer la primera tabla en busca de un R_i igual a algún S_j de la segunda tabla. Al encontrar un par, tenemos un par de claves (i, j) tal que $D_j(C_1) = E_i(P_1)$. Potencialmente, i es K_1 y j es K_2.
4. Comprobar si $E_j(E_i(P_2))$ es igual a C_2. Si lo es, intentar todos los demás pares (texto normal, texto cifrado). De no serlo, continuar buscando pares en las dos tablas.

Ciertamente ocurrirán muchas falsas alarmas antes de encontrar las claves reales, pero tarde o temprano se encontrarán. Este ataque requiere sólo 2^{57} operaciones de cifrado o descifrado (para construir las dos tablas), mucho menos que 2^{112}; sin embargo, también requiere un total de 2^{60} bytes de almacenamiento para las dos tablas, por lo que actualmente no es factible en su forma básica, pero Merkle y Hellman han mostrado varias optimizaciones y concesiones que permiten menos almacenamiento a expensas de más cómputo. En conclusión, el cifrado doble usando el DES probablemente no es mucho más seguro que el cifrado sencillo.

El cifrado triple es otro asunto. Ya para 1979, IBM se dio cuenta de que la longitud de la clave DES era demasiado corta y diseñó una manera de aumentarla efectivamente usando codificación triple (Tuchman, 1979). El método seleccionado, que se ha incorporado al estándar internacional 8732, se ilustra en la figura 7-9. Aquí se usan dos claves y tres etapas. En la primera etapa, el texto normal se cifra con K_1. En la segunda etapa, el DES se ejecuta en modo de descifrado, usando K_2 como clave. Por último, se hace otro cifrado usando K_1.

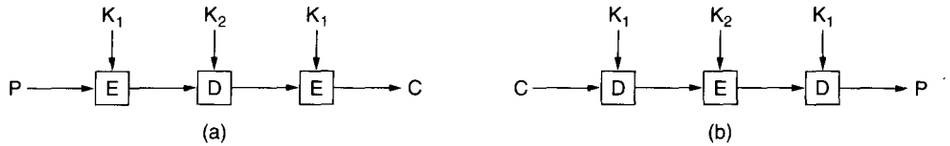


Figura 7-9. Cifrado triple usando el DES.

Este diseño inmediatamente da pie a dos preguntas. Primero, ¿por qué sólo se usan dos claves en lugar de tres? Segundo, ¿por qué se usa EDE (cifrado-descifrado-cifrado) en lugar de EEE (cifrado-cifrado-cifrado)? La razón de que se usen dos claves es que incluso los criptógrafos más paranoicos coinciden en que 112 bits son suficientes para las aplicaciones comerciales por ahora. Subir a 168 bits simplemente agregaría la carga extra innecesaria de administrar y transportar otra clave.

La razón para cifrar, descifrar y luego cifrar de nuevo es la compatibilidad en reversa con los sistemas DES de una sola clave. Tanto las funciones de cifrado como de descifrado son correspondencias entre grupos de números de 64 bits. Desde el punto de vista criptográfico, las dos correspondencias son igualmente robustas. Sin embargo, usando EDE en lugar de EEE, una computadora que usa cifrado triple puede hablar con otra que usa cifrado sencillo simplemente estableciendo $K_1 = K_2$. Esta propiedad permite la introducción gradual del cifrado triple, algo que no interesa a los criptógrafos académicos, pero de importancia considerable para IBM y sus clientes.

No se conoce ningún método para descifrar el DES triple en modo EDE. Van Oorschot y Wiener (1988) han presentado un método para acelerar la búsqueda de EDE en un factor de 16, pero aun con su ataque, el EDE es muy seguro. Para cualquiera que desea nada menos que lo mejor, se recomienda el EEE con tres diferentes claves de 56 bits (168 bits en total).

Antes de dejar el tema del DES, vale la pena cuando menos mencionar dos recientes avances del criptoanálisis. El primero es el **criptoanálisis diferencial** (Biham y Shamir, 1993). Esta técnica puede usarse para atacar cualquier cifrado en bloques; funciona comenzando por un par de bloques de texto normal que difieren sólo en una cantidad pequeña de bits y observando cuidadosamente lo que ocurre en cada iteración interna a medida que avanza el cifrado. En muchos casos, algunos patrones son mucho más comunes que otros, y esta observación conduce a un ataque probabilístico.

El otro avance que vale la pena mencionar es el **criptoanálisis lineal** (Matsui, 1994). Que puede descifrar el DES con sólo 2^{43} textos comunes conocidos. Funciona haciendo un OR EXCLUSIVO entre ciertos bits de texto normal y texto cifrado para generar 1 bit. Al hacerse repetidamente, la mitad de los bits deben ser de ceros y la otra deben ser unos. Sin embargo, con frecuencia los cifrados introducen un sesgo en una dirección o en la otra, y este sesgo, por pequeño que sea, puede explotarse para reducir el factor de trabajo. Para los detalles, véase el trabajo de Matsui.

IDEA

Tal vez toda esta insistencia en la razón por la que el DES es inseguro es algo parecida a querer azotar a un caballo muerto, pero la realidad es que el DES de cifrado sencillo aún es de amplio uso en aplicaciones seguras, como el proceso bancario mediante cajeros electrónicos automatizados. Si bien esta selección probablemente fue adecuada cuando se hizo, hace más de una década, ya no es adecuada.

En este punto, el lector probablemente se pregunta con toda razón: "si el DES es tan débil, ¿por qué no ha inventado nadie un mejor cifrado de bloques?" La realidad es que se han propuesto muchos cifrados de bloques, incluidos BLOWFISH (Schneier, 1994), Crab (Kaliski y Robshaw, 1994), FEAL (Shimizu y Miyaguchi, 1988), KHAFRE (Merkle, 1991), LOKI91 (Brown *et al.*, 1991), NEWDES (Scott, 1985), REDOCII (Cusick y Wood, 1991), y SAFER K64 (Massey, 1994). Schneier (1996) analiza todos estos e incontables más. Probablemente el más interesante e importante de los cifrados de bloques posteriores al DES es el **IDEA** (*International Data Encryption Algorithm*, **algoritmo internacional de cifrado de datos**) (Lai y Massey, 1990, y Lai, 1992). Estudiemos ahora el IDEA con mayor detalle.

El IDEA fue diseñado por dos investigadores en Suiza, por lo que probablemente está libre de cualquier "asesoría" de la NSA que podría haber introducido una puerta secreta; usa una clave de 128 bits, lo que lo hará inmune durante décadas a los ataques de la fuerza bruta, la lotería china y a los ataques de encuentro a la mitad. También se diseñó para resistir el criptoanálisis diferencial. No hay ninguna técnica o máquina conocida actualmente que se crea que puede violar el IDEA.

La estructura básica del algoritmo semeja al DES en cuanto a que se alteran bloques de entrada de texto normal de 64 bits en una secuencia de iteraciones parametrizadas para producir bloques de salida de texto cifrado de 64 bits, como se muestra en la figura 7-10(a). Dada la extensa alteración de bits (por cada iteración, cada uno de los bits de salida depende de cada uno de los bits de entrada), basta con ocho iteraciones. Como con todos los cifrados de bloque, el IDEA también puede usarse en el modo de realimentación de cifrado y en los demás modos del DES.

Los detalles de una iteración se presentan en la figura 7-10(b). Se usan tres operaciones, todas sobre números sin signo de 16 bits. Estas operaciones son OR EXCLUSIVO, suma módulo 2^{16} y multiplicación módulo $2^{16} + 1$. Las tres se pueden efectuar fácilmente en una microcomputadora de 16 bits ignorando las partes de orden mayor de los resultados. Las operaciones tienen la propiedad de que ningunos dos pares obedecen la ley asociativa ni la ley distributiva, dificultando el criptoanálisis. La clave de 128 bits se usa para generar 52 subclaves de 16 bits cada una, 6 por cada una de las ocho iteraciones y 4 para la transformación final. El descifrado usa el mismo algoritmo que el cifrado, sólo que con subclaves diferentes.

Se han construido implementaciones tanto en *software* como en *hardware* del IDEA. La primera implementación operaba en una 386 a 33 MHz y lograba una rapidez de cifrado de 0.88 Mbps. En una máquina moderna que opera a una velocidad 10 veces mayor, deben poderse lograr 9 Mbps en *software*. Se construyó en ETH Zurich un *chip* VLSI experimental de 25 MHz que cifra a razón de 177 Mbps.

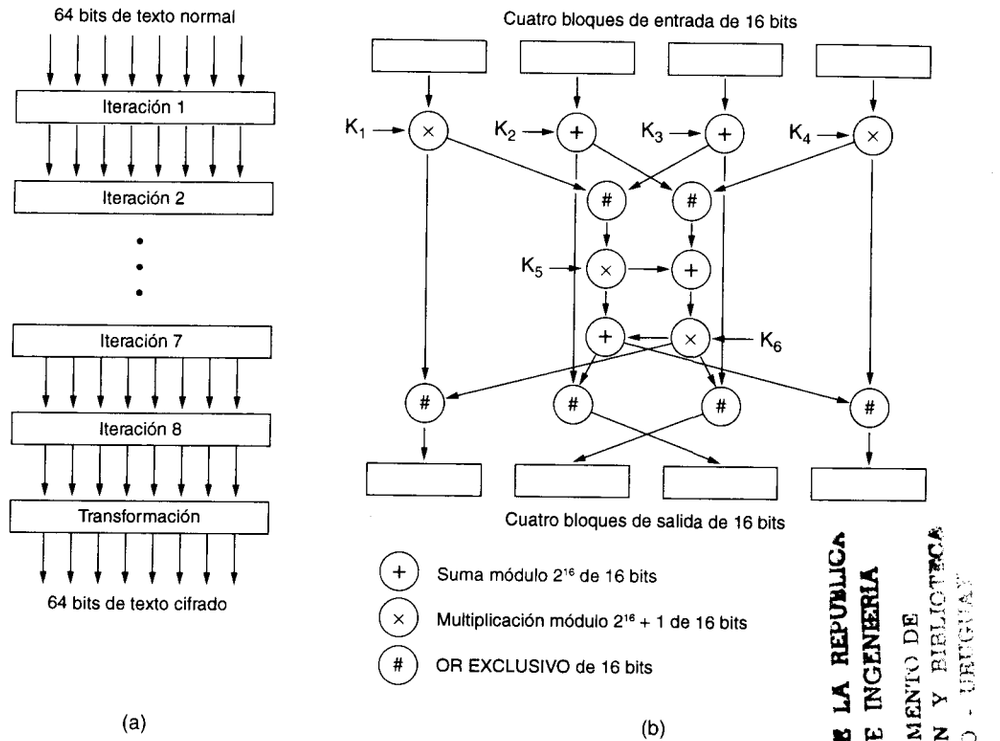


Figura 7-10. (a) IDEA. (b) Detalle de una iteración.

7.1.4. Algoritmos de clave pública

Históricamente el problema de distribución de claves siempre ha sido la parte débil de la mayoría de los criptosistemas. Sin importar lo robusto que sea un criptosistema, si un intruso puede robar la clave, el sistema no vale nada. Dado que todos los criptólogos siempre daban por hecho que la clave de cifrado y la clave de descifrado eran la misma (o que se podía derivar fácilmente una de la otra) y que la clave tenía que distribuirse a todos los usuarios del sistema, parecía haber un problema inherente: las claves se tenían que proteger contra robo, pero también se tenían que distribuir, por lo que no podían simplemente guardarse en una caja fuerte.

En 1976, dos investigadores de la Universidad de Stanford, Diffie y Hellman (1976), propusieron una clase nueva de criptosistema, en el que las claves de cifrado y descifrado eran diferentes y la clave de descifrado no podía derivarse de la clave de cifrado. En su propuesta, el algoritmo de cifrado (con clave), E, y el algoritmo de descifrado (con clave), D, tenían que

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

cumplir con los tres requisitos siguientes. Estos requisitos pueden expresarse sencillamente como sigue:

1. $D(E(P)) = P$.
2. Es excesivamente difícil deducir D de E.
3. E no puede descifrarse mediante un ataque de texto normal seleccionado.

El primer requisito dice que, si aplicamos D a un mensaje cifrado, $E(P)$, obtenemos nuevamente el mensaje de texto normal original, P. El segundo requisito no requiere explicación. El tercer requisito es necesario porque, como veremos en un momento, los intrusos pueden experimentar a placer con el algoritmo. En estas condiciones, no hay razón para que una clave de cifrado no pueda hacerse pública.

El método funciona como sigue. Una persona, llamémosla Alicia, que quiera recibir mensajes secretos, primero diseña dos algoritmos, E_A y D_A, que cumplan los requisitos anteriores. El algoritmo de cifrado y la clave, E_A, se hacen públicos, de ahí el nombre de **criptografía de clave pública** (para contrastar con la criptografía tradicional de clave secreta). Esto podría hacerse poniéndolos en un archivo accesible a cualquiera que quiera leerlo. Alicia publica el algoritmo de descifrado (para conseguir la asesoría gratuita), pero mantiene secreta la clave de descifrado. Por tanto, E_A es pública, pero D_A es secreta.

Ahora veamos si podemos resolver el problema de establecer un canal seguro entre Alicia y Benjamín, que nunca han tenido contacto previo. Se supone que tanto la clave de cifrado de Alicia, E_A, como la clave de cifrado de Benjamín, E_B, están en un archivo de lectura pública. (Básicamente, se espera que todos los usuarios de la red publiquen sus claves de cifrado tan pronto como se vuelven usuarios de la red.) Ahora, Alicia toma su primer mensaje, P, calcula $E_B(P)$ y lo envía a Benjamín. Benjamín entonces lo descifra aplicando su clave secreta D_B [es decir, calcula $D_B(E_B(P)) = P$]. Nadie más puede leer el mensaje cifrado, $E_B(P)$, porque se supone que el sistema de cifrado es robusto y porque es demasiado difícil derivar D_B de la E_B públicamente conocida. Alicia y Benjamín ahora se pueden comunicar con seguridad.

Es útil una nota sobre terminología. La criptografía de clave pública requiere que cada usuario tenga dos claves: una clave pública, usada por todo el mundo para cifrar mensajes a enviar a ese usuario, y una clave privada, que necesita el usuario para descifrar los mensajes. Consistentemente nos referiremos a estas claves como claves *públicas* y *privadas*, respectivamente, y las distinguiremos de las claves *secretas* usadas tanto para cifrado como descifrado en la criptografía convencional (también llamada **de clave simétrica**).

El algoritmo RSA

La única dificultad estriba en que necesitamos encontrar algoritmos que realmente satisfagan estos tres requisitos. Debido a las ventajas potenciales de la criptografía de clave pública, muchos investigadores están trabajando a todo vapor, y ya se han publicado algunos algoritmos. Un buen método fue descubierto por un grupo del M.I.T. (Rivest *et al.*, 1978). Es conocido por

las iniciales de sus tres descubridores (Rivest, Shamir, Adleman): **RSA**. Su método se basa en ciertos principios de la teoría de los números. Ahora resumiremos el uso del método; para los detalles, consulte el trabajo original.

1. Seleccionar dos números primos grandes, p y q (generalmente mayores que 10^{100}).
2. Calcular $n = p \times q$ y $z = (p - 1) \times (q - 1)$.
3. Seleccionar un número primo con respecto a z, llamándolo d.
4. Encontrar e tal que $e \times d = 1 \ mod \ z$.

Con estos parámetros calculados por adelantado, estamos listos para comenzar el cifrado. Dividimos el texto normal (considerado como una cadena de bits) en bloques, para que cada mensaje de texto normal, P, caiga en el intervalo $0 \leq P < n$. Esto puede hacerse agrupando el texto normal en bloques de k bits, donde k es el entero más grande para el que $2^k < n$ es verdad.

Para cifrar un mensaje, P, calculamos $C = P^e \pmod{n}$. Para descifrar C, calculamos $P = C^d \pmod{n}$. Puede demostrarse que, para todos los P del intervalo especificado, las funciones de cifrado y descifrado son inversas. Para ejecutar el cifrado, se necesitan e y n. Para llevar a cabo el descifrado, se requieren d y n. Por tanto, la clave pública consiste en el par (e, n), y la clave privada consiste en (d, n).

La seguridad del método se basa en la dificultad para factorizar números grandes. Si el criptoanalista pudiera factorizar n (conocido públicamente), podría encontrar p y q y, a partir de éstos, z. Equipado con el conocimiento de z y de e, puede encontrar d usando el algoritmo de Euclides. Afortunadamente, los matemáticos han estado tratando de factorizar números grandes durante los últimos 300 años, y las pruebas acumuladas sugieren que se trata de un problema excesivamente difícil.

De acuerdo con Rivest y colegas, la factorización de un número de 200 dígitos requiere 4 mil millones de años de tiempo de cómputo; la factorización de un número de 500 dígitos requiere 10^{25} años. En ambos casos, se supone el uso del mejor algoritmo conocido y de una computadora con un tiempo de instrucción de 1 μseg. Aun si las computadoras continúan aumentando su velocidad en un orden de magnitud cada década, pasarán siglos antes de que sea factible la factorización de un número de 500 dígitos, y para entonces nuestros descendientes simplemente pueden escoger un p y un q todavía más grandes.

Un ejemplo pedagógico trivial del algoritmo RSA se muestra en la figura 7-11. Para este ejemplo hemos seleccionado $p = 3$ y $q = 11$, dando $n = 33$ y $z = 20$. Un valor adecuado de d es $d = 7$, puesto que 7 y 20 no tienen factores comunes. Con estas selecciones, e puede encontrarse resolviendo la ecuación $7e = 1 \pmod{20}$, que produce $e = 3$. El texto cifrado, C, de un mensaje de texto normal, P, se da por la regla $C = P^3 \pmod{33}$. El texto cifrado lo descifra el receptor de acuerdo con la regla $P = C^7 \pmod{33}$. En la figura se muestra como ejemplo el cifrado del texto normal "SUZANNE".

Dado que los números primos escogidos para este ejemplo son tan pequeños, P debe ser menor que 33, por lo que cada bloque de texto normal puede contener sólo un carácter. El resultado es un cifrado por sustitución monoalfabética, no muy impresionante. En cambio, si

Texto normal (P)			Texto cifrado (C)		Después del descifrado	
Simbólico	Numérico	P^3	P^3 (mod 33)	C^7	C^7 (mod 33)	Simbólico
S	19	6859	28	13492928512	19	S
U	21	9261	21	1801088541	21	U
Z	26	17576	20	1280000000	26	Z
A	01	1	1	1	1	A
N	14	2744	5	78125	14	N
N	14	2744	5	78125	14	N
E	05	125	26	8031810176	5	E
		Cálculo del transmisor		Cálculo del receptor		

Figura 7-11. Ejemplo del algoritmo RSA.

hubiéramos seleccionado p y $q \approx 10^{100}$, podríamos tener $n \approx 10^{200}$, para que cada bloque pueda ser de hasta 664 bits ($2^{664} \approx 10^{200}$) u 83 caracteres de 8 bits, contra 8 caracteres para el DES.

Cabe señalar que el uso del RSA como lo hemos descrito es semejante a usar DES en modo ECB: el mismo bloque de entrada da el mismo bloque de salida. Por tanto, se requiere alguna forma de encadenamiento para el cifrado de datos. Sin embargo, en la práctica la mayoría de los sistemas basados en RSA usan criptografía de clave pública principalmente para distribuir claves de sesión de una sola vez para su uso con DES, IDEA u otros algoritmos semejantes. El RSA es demasiado lento para poder cifrar grandes volúmenes de datos.

Otros algoritmos de clave pública

Aunque el RSA se usa ampliamente, de ninguna manera es el único algoritmo de clave pública conocido. El primer algoritmo de clave pública fue el de la mochila (Merkle y Hellman, 1978). La idea aquí es que alguien es dueño de una gran cantidad de objetos, todos con pesos diferentes. El dueño cifra el mensaje seleccionando secretamente un subgrupo de los objetos y colocándolos en la mochila. El peso total de los objetos contenidos en la mochila se hace público, así como la lista de todos los objetos posibles. La lista de los objetos que se metieron en la mochila se mantiene en secreto. Con ciertas restricciones adicionales, el problema de determinar una lista posible de los objetos con el peso dado se consideró no computable, y formó la base del algoritmo de clave pública.

El inventor del algoritmo, Ralph Merkle, estaba bastante seguro de que este algoritmo no podía violarse, por lo que ofreció una recompensa de 100 dólares a cualquiera que pudiera descifrarlo. Adi Shamir (la "S" de RSA) pronto lo violó y cobró la recompensa. Sin desmotivarse, Merkle reforzó el algoritmo y ofreció una recompensa de 1000 dólares a quien pudiera violar el nuevo. Ron Rivest (la "R" de RSA) pronto lo descifró y cobró la recompensa. Merkle no se atrevió a ofrecer 10,000 dólares para la siguiente versión, por lo que "A" (Leonard Adleman) no tuvo suerte. Aunque se ha modificado nuevamente, el algoritmo de la mochila no se considera seguro y pocas veces se usa.

Otros esquemas de clave pública se basan en la dificultad para calcular logaritmos discretos (Rabin, 1979). El Gamal (1985) y Schnorr (1991) han inventado algoritmos basados en este principio.

Existen algunos otros esquemas, como los basados en curvas elípticas (Menezes y Vanstone, 1993), pero las tres categorías principales son las basadas en la dificultad para factorizar números grandes, calcular logaritmos discretos y determinar el contenido de una mochila a partir de su peso. Estos problemas se consideran verdaderamente difíciles de resolver porque los matemáticos han estado trabajando en ellos durante años sin adelantos importantes.

7.1.5. Protocolos de validación de identificación

La **validación de identificación** (*authentication*) es la técnica mediante la cual un proceso comprueba que su compañero de comunicación es quien se supone que es y no un impostor. La verificación de la identidad de un proceso remoto ante un intruso activo malicioso es sorprendentemente difícil y requiere protocolos complejos basados en criptografía. En esta sección estudiaremos algunos de los muchos procesos de validación de identificación que se usan en las redes de cómputo inseguras.

Como nota, alguna gente confunde la autorización con la validación de identificación. La validación de identificación se encarga del asunto de comprobar si realmente nos estamos comunicando con un proceso específico. La autorización se encarga de lo que puede hacer el proceso. Por ejemplo, un proceso cliente se comunica con un servidor de archivos y le dice: "soy el proceso de Sebastián y quiero borrar el archivo *recetas.old*". Desde el punto de vista del servidor de archivos, deben contestarse dos preguntas:

1. ¿Es éste realmente el proceso de Sebastián (validación de identificación)?
2. ¿Tiene permiso Sebastián de borrar el archivo *recetas.old* (autorización)?

Sólo tras contestar afirmativamente sin ambigüedades ambas preguntas puede llevarse a cabo la acción solicitada. La primera pregunta realmente es la clave. Una vez que el servidor de archivos sabe con quién está hablando, la comprobación de la autorización simplemente es cuestión de buscar entradas en las tablas locales. Por esta razón, nos concentraremos en la validación de identificación en esta sección.

El modelo general que usan todos los protocolos de validación de identificación es éste. Un usuario (en realidad, un proceso) iniciador, digamos Alicia, quiere establecer una conexión segura con un segundo usuario, Benjamín. Alicia y Benjamín a veces se llaman **principales**, los personajes centrales de nuestra historia. Benjamín es un banquero con el que Alicia quiere hacer negocios. Alicia comienza por enviar un mensaje a Benjamín o a un **centro de distribución de claves (KDC)** confiable, que siempre es honesto. Siguen varios otros intercambios de mensajes en diferentes direcciones. A medida que se envían estos mensajes, un intruso malicioso, Trudy, [†]

[†] Agradezco a Kaufman$_1$ *et al.*$_{23}$ (1995) el que me revelaran su nombre.

puede interceptarlos, modificarlos o reproducirlos para engañar a Alicia y a Benjamín, o simplemente para estropear sus actividades.

No obstante, una vez que se ha completado el protocolo, Alicia está segura de que está hablando con Benjamín, y Benjamín está seguro de que está hablando con Alicia. Es más, en la mayoría de los protocolos, los dos también habrán establecido una **clave de sesión** secreta para usarla durante la conversación subsiguiente. En la práctica, por razones de desempeño, todo el tráfico de datos se cifra usando criptografía de clave secreta, aunque la criptografía de clave pública se usa ampliamente con los protocolos de validación de identificación mismos y para establecer la clave de sesión.

El objetivo de usar una clave de sesión nueva, seleccionada aleatoriamente, para cada nueva conexión es reducir al mínimo la cantidad de tráfico que es enviado con las claves secretas o públicas del usuario, reducir la cantidad de texto cifrado que puede obtener un intruso, y reducir al mínimo el daño provocado por la caída de un proceso, si su vaciado de memoria cae en las manos equivocadas. Con suerte, la única clave presente en ese momento será la clave de la sesión. Todas las claves permanentes debieron dejarse en blanco después de establecerse la sesión.

Validación de identificación basada en clave secreta compartida

Para nuestro primer protocolo de validación de identificación, supondremos que Alicia y Benjamín ya comparten una clave secreta, K_{AB}. (En los protocolos formales, abreviaremos Alicia como A y Benjamín como B, respectivamente.) Esta clave compartida podría haberse acordado telefónicamente o en persona pero, en cualquier caso, no a través de la (insegura) red.

Este protocolo se basa en un principio encontrado en muchos protocolos de validación de identificación: una parte envía un número aleatorio a la otra, que entonces lo transforma de una manera especial y devuelve el resultado. Tales protocolos se llaman protocolos **reto-respuesta** (*challenge-response*). En éste, y en los subsiguientes protocolos de validación de identificación, se usará la siguiente notación:

A, B son las identidades de Alicia y Benjamín

Las R_i son los retos, donde el subíndice identifica al retador

Las K_i son claves, donde i indica el dueño; K_S es la clave de la sesión

La secuencia de mensajes de nuestro primer protocolo de validación de identificación de clave compartida se muestra en la figura 7-12. En el mensaje 1, Alicia envía su identidad, A, a Benjamín de manera que la entienda Benjamín. Benjamín, por supuesto, no tiene manera de saber si este mensaje vino de Alicia o de Trudy, por lo que escoge un reto, un número aleatorio grande, R_B, y lo envía a "Alicia" como mensaje 2, en texto normal. Alicia entonces cifra el mensaje con la clave que comparte con Benjamín y envía el texto cifrado, $K_{AB}(R_B)$, en el mensaje 3. Cuando Benjamín ve este mensaje, de inmediato sabe que vino de Alicia porque Trudy no conoce K_{AB} y, por tanto, no pudo haberlo generado. Es más, dado que se seleccionó R_B al azar de un espacio grande (digamos, números aleatorios de 128 bits), es muy poco probable que Trudy haya visto R_B y su respuesta en una sesión previa.

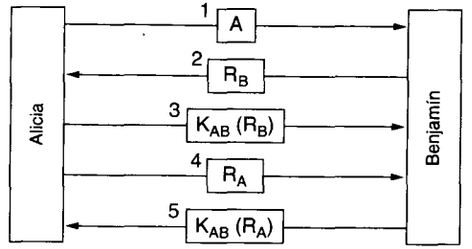


Figura 7-12. Validación de identificación de dos vías usando un protocolo de reto-respuesta.

En este punto, Benjamín está seguro de que está hablando con Alicia, pero Alicia no está segura de nada; hasta donde sabe, Trudy podría haber interceptado el mensaje 1 y enviado R_B como respuesta. Tal vez Benjamín murió anoche. Para saber a quién le está hablando, Alicia selecciona un número al azar, R_A, y lo envía a Benjamín como texto normal, en el mensaje 4. Cuando Benjamín responde con $K_{AB}(R_A)$, Alicia sabe que está hablando con Benjamín. Si desean establecer ahora una clave de sesión, Alicia puede seleccionar una, K_S, y enviársela a Benjamín cifrada con K_{AB}.

Aunque el protocolo de la figura 7-12 funciona, contiene mensajes de más. Estos pueden eliminarse combinando información, como se ilustra en la figura 7-13. Aquí, Alicia inicia el protocolo de reto-respuesta en lugar de esperar que Benjamín lo haga. De la misma manera, al tiempo que responde al reto de Alicia, Benjamín envía el suyo propio. El protocolo completo puede reducirse a tres mensajes en lugar de cinco.

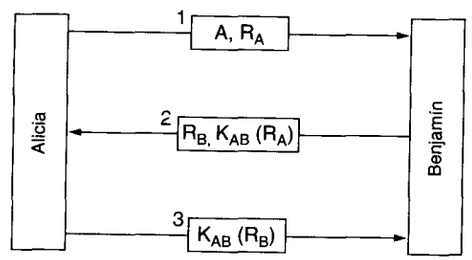


Figura 7-13. Protocolo de validación de identificación de dos vías más corto.

¿Es este nuevo protocolo una mejora sobre el original? En un sentido, sí: es más corto. Desgraciadamente, también está mal. En ciertas circunstancias, Trudy pude burlar este protocolo usando lo que se conoce como **ataque por reflexión**. En particular, Trudy puede inutilizarlo si es posible abrir al mismo tiempo varias sesiones con Benjamín. Esta situación sería cierta si, por ejemplo, Benjamín es un banco y está preparado para aceptar muchas conexiones simultáneas de los cajeros automáticos.

El ataque por reflexión de Trudy se muestra en la figura 7-14; comienza cuando Trudy indica que ella es Alicia y envía R_T. Benjamín responde, como de costumbre, con su propio reto, R_B. Ahora Trudy está atorada. ¿Qué puede hacer? No conoce $K_{AB}(R_B)$.

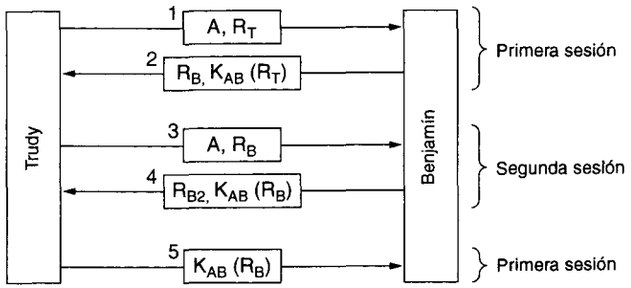


Figura 7-14. Ataque por reflexión.

Trudy puede abrir una segunda sesión con el mensaje 3, proporcionando como su reto el R_B tomado del mensaje 2. Benjamín lo cifra sin chistar y envía de regreso $K_{AB}(R_B)$ en el mensaje 4. Ahora Trudy tiene la información faltante, por lo que puede completar la primera sesión y abortar la segunda. Benjamín ahora está convencido de que Trudy es Alicia, por lo que, cuando ella solicita el saldo de su cuenta bancaria, él se lo da sin más. Después, cuando ella solicita la transferencia de todo a una cuenta secreta de un banco suizo, él lo hace sin dudarlo.

La moraleja de esta historia es:

El diseño de un protocolo de validación de identificación correcto es más difícil de lo que parece.

Tres reglas generales que frecuentemente son de ayuda son las siguientes:

1. Hacer que el iniciador demuestre quién es antes de que lo tenga que hacer el respondedor. En este caso, Benjamín regala información valiosa antes de que Trudy dé cualquier prueba de su identidad.
2. Hacer que el iniciador y el respondedor usen diferentes claves para comprobación, incluso si esto significa tener dos claves compartidas, K_{AB} y K'_{AB}.
3. Hacer que el iniciador y el respondedor tomen sus retos de diferentes conjuntos. Por ejemplo, el iniciador debe usar números pares y el respondedor números nones.

Las tres reglas se violaron aquí, con resultados desastrosos. Nótese que nuestro primer protocolo de validación de identificación (de cinco mensajes) requiere que Alicia compruebe primero su identidad, por lo que ese protocolo no está a merced del ataque por reflexión.

Establecimiento de una clave compartida: intercambio de claves Diffie-Hellman

Hasta ahora hemos supuesto que Alicia y Benjamín comparten una clave secreta. ¿Qué sucede si no es así? ¿Cómo pueden establecer una? Una manera sería hacer que Alicia llame a Benjamín y le dé su clave por teléfono, pero él podría comenzar por decir: "¿Cómo sé que tú eres Alicia y no Trudy?" Ellos podrían intentar concertar una cita, trayendo cada uno un pasaporte, su licencia de conducir y tres tarjetas de crédito bancarias, pero siendo gente ocupada, tal vez no encuentren una fecha aceptable sino hasta dentro de varios meses. Afortunadamente, aunque suene increíble, hay una manera de que completos desconocidos establezcan una clave secreta a plena luz del día, aun con Trudy registrando cuidadosamente cada mensaje.

El protocolo que permite que dos extraños establezcan una clave secreta compartida se llama **intercambio de claves Diffie-Hellman** (Diffie y Hellman, 1976), y funciona como sigue. Alicia y Benjamín tienen que acordar dos números primos grandes, n y g, donde $(n - 1)/2$ también es primo, y g debe cumplir ciertas condiciones. Estos números pueden ser públicos, por lo que cualquiera de ellos puede escoger n y g y decírselo al otro abiertamente. Ahora, Alicia escoge un número grande (digamos de 512 bits), x, y lo mantiene en secreto. De la misma manera, Benjamín escoge un número secreto grande, y.

Alicia inicia el protocolo de intercambio de claves enviando a Benjamín un mensaje que contiene $(n, g, g^x \bmod n)$, como se muestra en la figura 7-15. Benjamín responde enviando a Alicia un mensaje que contiene $g^y \bmod n$. Ahora Alicia toma el número que Benjamín le envió y lo eleva a la potencia x para obtener $(g^y \bmod n)^x$. Benjamín lleva a cabo una tarea parecida para obtener $(g^x \bmod n)^y$. Por las leyes de la aritmética modular, ambos cálculos arrojan $g^{xy} \bmod n$. Ahora, Alicia y Benjamín comparten una clave secreta, $g^{xy} \bmod n$.

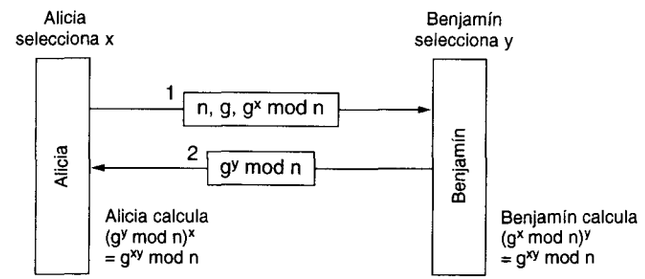


Figura 7-15. Intercambio de claves Diffie-Hellman.

Por supuesto, Trudy ha visto ambos mensajes, así que conoce g y n del mensaje 1. Si Trudy pudiera calcular x y y, podría averiguar la clave secreta. El problema es que, dado sólo $g^x \bmod n$, no es posible encontrar x. No se conoce un algoritmo práctico para calcular logaritmos discretos módulo un número primo muy grande.

Para hacer más concreto el ejemplo anterior, usaremos los valores (completamente irreales) de $n = 47$ y $g = 3$. Alicia selecciona $x = 8$ y Benjamín selecciona $y = 10$. Ambos se mantienen en

secreto. El mensaje de Alicia a Benjamín es (47, 3, 28) porque 3^8 mod 47 es 28. El mensaje de Benjamín a Alicia es (17). Alicia calcula 17^8 mod 47, que es 4. Benjamín calcula 28^{10} mod 47, que es 4. Alicia y Benjamín han determinado independientemente que la clave secreta ahora es 4. Trudy tiene que resolver la ecuación 3^x mod 47 = 28, lo que puede lograrse mediante una búsqueda exhaustiva en el caso de números pequeños como éste, pero no cuando todos los números tienen cientos de bits de longitud. Todos los algoritmos actualmente conocidos simplemente tardan demasiado, aun usando una supercomputadora masivamente paralela.

A pesar de la elegancia del algoritmo Diffie-Hellman, hay un problema: cuando Benjamín recibe la tripleta (47, 3, 28), ¿cómo sabe que es de Alicia y no de Trudy? No hay manera de saberlo. Desgraciadamente, Trudy puede explotar este hecho para engañar tanto a Alicia como a Benjamín, como se ilustra en la figura 7-16. Aquí, mientras Alicia y Benjamín seleccionan x y y respectivamente, Trudy selecciona su propio número aleatorio, z. Alicia envía el mensaje 1 destinado a Benjamín. Trudy lo intercepta y envía el mensaje 2 a Benjamín, usando el g y el n correctos (que de todas maneras son públicos) pero con su propia z en lugar de x. Trudy también envía el mensaje 3 de regreso a Alicia. Después, Benjamín envía el mensaje 4 a Alicia, que es interceptado y guardado tmbién por Trudy.

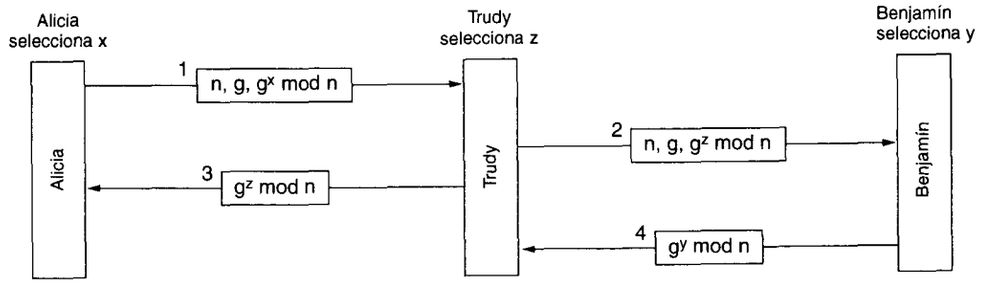


Figura 7-16. Ataque de brigada de cubetas.

Ahora todos hacen la aritmética modular. Alicia calcula la clave secreta como g^{xz} mod n, y también lo hace Trudy (para los mensajes a Alicia). Benjamín calcula g^{yz} mod n, al igual que Trudy (para los mensajes a Benjamín). Alicia piensa que está hablando con Benjamín, por lo que establece una clave de sesión (con Trudy). Lo mismo hace Benjamín. Cada mensaje que Alicia envía durante la sesión cifrada es capturado por Trudy, almacenado, modificado si ella así lo desea, y pasado (opcionalmente) a Benjamín. Lo mismo ocurre en la otra dirección. Trudy ve todo y puede modificar todos los mensajes si lo desea, mientras que tanto Alicia como Benjamín están pensando equivocadamente que tienen un canal seguro entre ambos. Este ataque se conoce como **ataque de brigada de cubetas**, porque se parece vagamente a las brigadas de bomberos voluntarios antiguas en las que se pasaban las cubetas de agua en fila desde el carro de bomberos hasta el incendio. También se llama **ataque del hombre (mujer) en medio**, lo que no debe confundirse con el encuentro a la mitad de los cifrados de bloque. Afortunadamente, con la ayuda de algoritmos más complicados se puede frustrar este ataque.

Validación de identificación usando un centro de distribución de claves

El establecimiento de un secreto compartido con un extraño casi funcionó, pero no por completo. Por otra parte, probablemente no valía la pena desde el principio (ataque de uvas agrias). Para hablarle a n personas de esta manera se requerirían n claves. Para la gente muy popular, la administración de claves se volvería una verdadera carga, especialmente si cada clave tuviera que almacenarse en el *chip* de una tarjeta de plástico individual.

Un enfoque diferente es introducir un centro de distribución de claves confiable (KDC) (*key distribution center*). En este modelo, cada usuario tiene una sola clave compartida con el KDC. La validación de identificación y la administración de claves de sesión ahora pasan a través del KDC. El protocolo de validación de identificación KDC más simple conocido es la **rana de boca amplia** (Burrows *et al.*, 1990) (llamado así por el apodo que se consiguió su inventor [Burrows] durante sus días en la universidad). Este protocolo se ilustra en la figura 7-17.

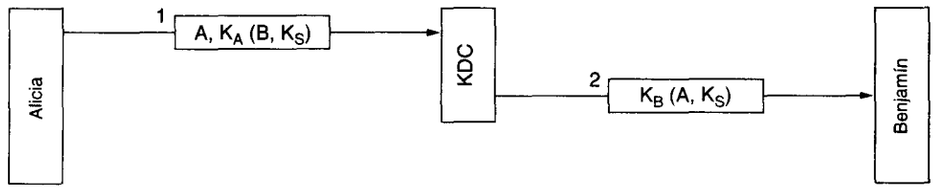


Figura 7-17. Protocolo de validación de identificación de rana de boca amplia.

El concepto en que se basa el protocolo de rana de boca amplia es sencillo: Alicia escoge una clave de sesión, K_S, e indica al KDC que desea hablar con Benjamín usando K_S. Este mensaje se cifra con la clave secreta que comparte Alicia (solamente) con el KDC, K_A. El KDC descifra este mensaje, extrayendo la identidad de Benjamín y la clave de sesión; luego construye un mensaje nuevo que contiene la identidad de Alicia y la clave de sesión y envía este mensaje a Benjamín. Este cifrado se hace con K_B, la clave secreta que Benjamín comparte con el KDC. Cuando Benjamín descifra el mensaje, sabe que Alicia quiere hablar con él, y también conoce la clave que ella desea usar.

La validación de identificación aquí es gratuita. El KDC sabe que el mensaje 1 debe haber venido de Alicia, puesto que nadie más habría sido capaz de cifrarlo con la clave secreta de Alicia. De la misma manera, Benjamín sabe que el mensaje 2 debe haber venido del KDC, en quien confía, puesto que nadie más sabe su clave secreta.

Desgraciadamente, este protocolo tiene una falla seria. Trudy necesita dinero, por lo que piensa en algún servicio legítimo que puede prestar a Alicia, hace una oferta atractiva, y consigue el trabajo. Después de hacerlo, Trudy solicita cortésmente a Alicia que le pague mediante una transferencia bancaria. Alicia entonces establece una clave de sesión con su banquero, Benjamín, y le envía un mensaje solicitando la transferencia del dinero a la cuenta de Trudy.

Mientras tanto, Trudy ha regresado a su viejo hábito, espiar la red; copia tanto el mensaje 2 de la figura 7-17 como la solicitud de transferencia de dinero que le sigue. Posteriormente, Trudy

reproduce ambos para Benjamín. Benjamín los recibe y piensa: "Alicia debió haber contratado nuevamente a Trudy. Aparentemente trabaja bien". Benjamín entonces transfiere una cantidad igual de dinero de la cuenta de Alicia a la de Trudy. Algún tiempo después del par de mensajes número 50, Benjamín sale a buscar a Trudy para ofrecerle un préstamo grande para que pueda expandir su negocio, que evidentemente tiene éxito. Este problema se llama **ataque por repetición** (*replay attack*).

Son posibles varias soluciones al ataque por repetición. La primera es incluir una marca de tiempo en cada mensaje. Entonces, si alguien recibe un mensaje obsoleto, puede descartarlo. El problema con este enfoque es que los relojes nunca están perfectamente sincronizados en toda una red, por lo que tiene que haber un intervalo durante el cual la marca de tiempo sea válida. Trudy puede repetir el mensaje durante este intervalo y salirse con la suya.

La segunda solución es poner un número de mensaje único, de una sola ocasión, a veces llamado **núnico**, en cada mensaje. Cada parte entonces tiene que recordar todos los núnicos y rechazar cualquier mensaje que contenga un núnico previamente usado. Pero los núnicos tienen que recordarse indefinidamente, no vaya a ser que Trudy esté tratando de reproducir un mensaje de cinco años de antigüedad. También, si una máquina se cae y pierde su lista de núnicos, nuevamente es vulnerable a un ataque por repetición. Las marcas de tiempo y los núnicos pueden combinarse para limitar el tiempo que pueden recordarse los núnicos, pero ciertamente el protocolo se volverá mucho más complicado.

Un enfoque más refinado para la validación de identificación es usar un protocolo multisentido de reto-respuesta. Un ejemplo bien conocido de tal protocolo es el de **validación de identificación Needham-Schroeder** (Needham y Schroeder, 1978), del cual se muestra una variante en la figura 7-18.

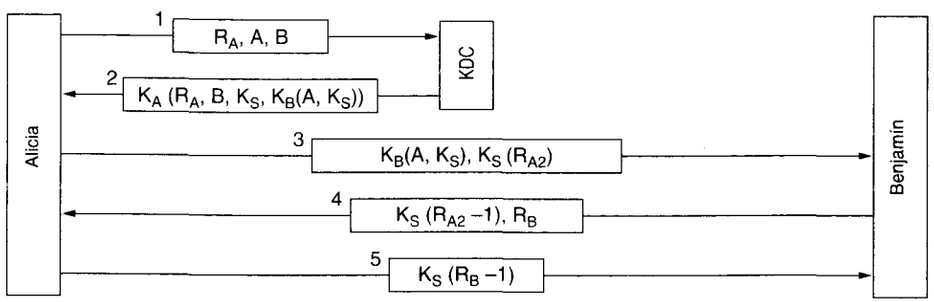


Figura 7-18. Protocolo de validación de identificación Needham-Schroeder.

El protocolo comienza cuando Alicia indica al KDC que quiere hablar con Benjamín. Este mensaje contiene un número aleatorio grande, R_A, como núnico. El KDC devuelve el mensaje 2 que contiene el número aleatorio de Alicia, una clave de sesión, y un billete que puede enviar a Benjamín. El objeto del número aleatorio, R_A, es asegurar a Alicia que el mensaje 2 es reciente, y no una repetición. La identidad de Benjamín también se incluye por si a Trudy se le ocurre

reemplazar B del mensaje 1 por su propia identidad, para que el KDC cifre el billete al final del mensaje 2 con K_T en lugar de K_B. El billete cifrado con K_B se incluye dentro del mensaje cifrado para evitar que Trudy lo reemplace por otra cosa en su camino hacia Alicia.

Alicia ahora envía el billete a Benjamín, junto con un nuevo número aleatorio, R_{A2}, cifrado con la clave de la sesión, K_S. En el mensaje 4, Benjamín devuelve $K_S(R_{A2} - 1)$ para demostrar a Alicia que está hablando con el verdadero Benjamín. El envío de regreso de $K_S(R_{A2})$ no habría funcionado, puesto que Trudy lo podría haber robado del mensaje 3.

Tras recibir el mensaje 4, Alicia está convencida de que está hablando con Benjamín, y de que no se pudieron haber usado repeticiones hasta el momento. A fin de cuentas, Alicia acaba de generar R_{A2} hace unos cuantos milisegundos. El propósito del mensaje 5 es convencer a Benjamín de que realmente está hablando con Alicia, y de que tampoco se han usado repeticiones aquí. Al hacer que cada parte genere un reto y responda a otro, se elimina la posibilidad de un ataque por repetición.

Aunque este protocolo parece bastante sólido, tiene una ligera debilidad. Si Trudy llega a obtener una clave de sesión vieja en texto normal, puede iniciar una nueva sesión con Benjamín repitiendo el mensaje 3 correspondiente a la clave obtenida y convencerlo de que ella es Alicia (Denning y Sacco, 1981). Esta vez Trudy puede desfalcar la cuenta de Alicia sin tener que llevar a cabo el servicio legítimo ni siquiera una sola vez.

Needham y Schroeder publicaron posteriormente un protocolo que corrige este problema (Needham y Schroeder, 1987). En el mismo número de la misma publicación, Otway y Rees (1987) publicaron un protocolo que también resuelve el problema pero de una manera más corta. En la figura 7-19 se muestra un protocolo Otway-Rees ligeramente modificado.

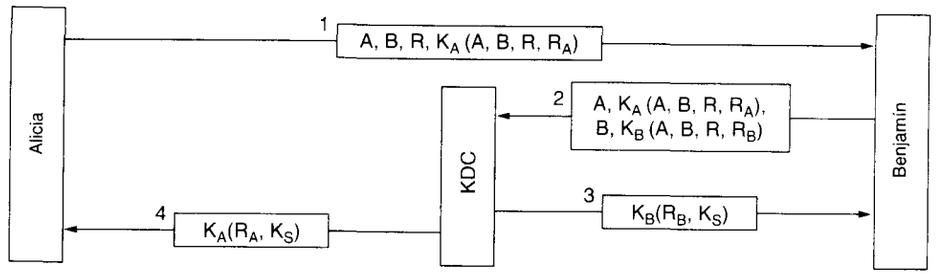


Figura 7-19. Protocolo de validación de identificación Otway-Rees (ligeramente simplificado).

En el protocolo Otway-Rees, Alicia comienza por generar un par de números aleatorios: R, que se usará como identificador común, y R_A, que Alicia usará para retar a Benjamín. Cuando Benjamín recibe este mensaje, construye un mensaje nuevo a partir de la parte cifrada del mensaje de Alicia, y uno análogo propio. Ambas partes cifradas con K_A y K_B identifican a Alicia y a Benjamín, contienen el identificador común y contienen un reto.

El KDC comprueba que el R de ambas partes es igual. Los R podrían no serlo porque Trudy alteró el R del mensaje 1 y reemplazó parte del mensaje 2. Si los dos R son iguales, el KDC se

convence de que el mensaje de solicitud de Benjamín es válido, y genera una clave de sesión y la cifra dos veces, una para Alicia y otra para Benjamín. Cada mensaje contiene el número aleatorio del receptor como prueba de que el KDC, y no Trudy, generó el mensaje. En este punto, tanto Alicia como Benjamín poseen la misma clave de sesión y pueden comenzar a comunicarse. La primera vez que intercambien mensajes de datos, ambos podrán ver que el otro tiene una copia idéntica de K_S, por lo que la validación de identificación está completa.

Validación de identificación usando Kerberos

Un protocolo de validación de identificación usado en muchos sistemas reales es **Kerberos**, basado en una variación del de Needham-Schroeder; se llama así por un perro de varias cabezas de la mitología griega que solía cuidar la entrada al averno (supuestamente para mantener fuera a los indeseables). Kerberos se diseñó en el M.I.T. para permitir a los usuarios de estaciones de trabajo el acceso a recursos de una manera segura; su principal diferencia respecto del protocolo de Needham-Schroeder es el supuesto de que todos los relojes están bastante bien sincronizados. El protocolo ha pasado por varias iteraciones. La V4 es la versión de uso más difundido en la industria, por lo que la describiremos. Después diremos algunas palabras sobre su sucesor, V5. Para mayor información, véase (Neuman y Ts'o, 1994, y Steiner *et al.*, 1988).

Kerberos comprende tres servidores además de Alicia (una estación de trabajo cliente):

Servidor de validación de identificación (AS): verifica los usuarios durante el establecimiento de la sesión
Servidor de otorgamiento de billetes (TGS): emite "billetes de identidad comprobada"
Benjamín el servidor: es quien en realidad hace el trabajo que Alicia quiere que se realice

El AS es semejante al KDC en el sentido de que comparte una contraseña secreta con cada usuario. El trabajo del TGS es emitir billetes que puedan convencer a los servidores de que el portador de un billete TGS en realidad es quien dice ser.

Para iniciar una sesión, Alicia se sienta frente a una estación de trabajo pública arbitraria y teclea su nombre. La estación de trabajo envía su nombre al AS como texto normal, como se muestra en la figura 7-20. Lo que regresa es una clave de sesión y un billete, $K_{TGS}(A, K_S)$, dirigido al TGS. Estos elementos se empacan juntos y se cifran usando la clave secreta de Alicia, de modo que sólo Alicia puede descifrarlos. No es sino hasta que llega el mensaje 2 que la estación de trabajo solicita la contraseña de Alicia. La contraseña entonces se usa para generar K_A, a fin de descifrar el mensaje 2 y obtener la clave de la sesión y el billete TGS contenido en ella. En este punto, la estación de trabajo sobrescribe la contraseña de Alicia para asegurarse que solamente esté en la estación de trabajo durante unos cuantos milisegundos a lo sumo. Si Trudy intenta establecer una sesión haciéndose pasar por Alicia, la contraseña que ella escriba estará equivocada y la estación de trabajo detectará esto porque la parte estándar del mensaje 2 será incorrecta.

Tras establecer la sesión, Alicia puede indicar a la estación de trabajo que quiere hacer contacto con Benjamín, el servidor de archivos. La estación de trabajo envía entonces el mensaje

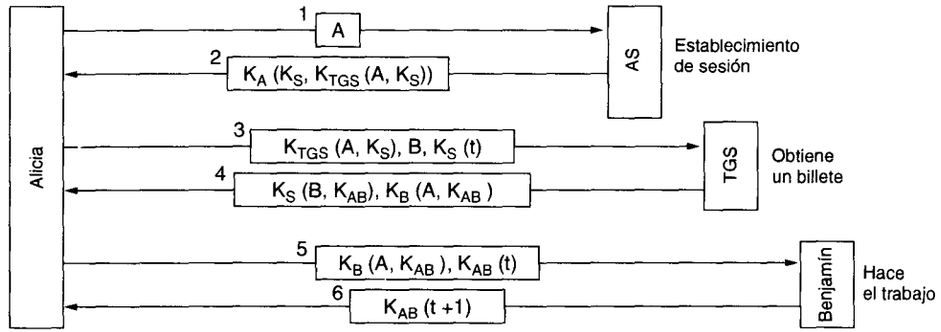


Figura 7-20. Funcionamiento del Kerberos V4.

3 al TGS solicitando un billete para usarlo con Benjamín. El elemento clave de esta solicitud es $K_{TGS}(A, K_S)$, que se cifra con la clave secreta del TGS y sirve como prueba de que el transmisor en realidad es Alicia. El TGS responde creando una clave de sesión, K_{AB}, para que Alicia la use con Benjamín. Se devuelven dos versiones de esta clave. La primera se cifra sólo con K_S, para que Alicia pueda leerla. La segunda se cifra con la clave de Benjamín, K_B, para que Benjamín pueda leerla.

Trudy puede copiar el mensaje 3 e intentar usarlo de nuevo, pero se verá frustrada por la marca de tiempo cifrada, t, que se envía junto con él. Trudy no puede reemplazar la marca de tiempo por una más reciente, pues no conoce K_S, la clave de sesión usada por Alicia para hablar con el TGS. Aun si Trudy repite el mensaje 3 rápidamente, todo lo que obtendrá es otra copia del mensaje 4, que no pudo descifrar la primera vez y no podrá descifrar la segunda vez tampoco.

Ahora Alicia puede enviar K_{AB} a Benjamín para establecer una sesión con él. Este intercambio también recibe una marca de tiempo. La respuesta es prueba para Alicia de que en realidad está hablando con Benjamín, no con Trudy.

Tras esta serie de intercambios, Alicia puede comunicarse con Benjamín con la protección de K_{AB}. Si luego ella decide que necesita hablar con otro servidor, Carolina, simplemente repetirá el mensaje 3 al TGS, sólo que ahora especificará C en lugar de B. El TGS responderá rápidamente con un billete cifrado con K_C que Alicia puede enviar a Carolina y que Carolina aceptará como prueba de que vino de Alicia.

El objeto de todo este trabajo es que ahora Alicia puede acceder a los servidores de toda la red de una manera segura, y que su contraseña nunca ha tenido que atravesar la red. De hecho, la contraseña sólo tuvo que estar en su propia estación de trabajo durante algunos milisegundos. Sin embargo, nótese que cada servidor efectúa su propia autorización. Cuando Alicia presenta su billete a Benjamín, éste simplemente indica a Benjamín quién lo envió. Lo que Alicia tenga permiso de hacer lo determinará Benjamín.

Dado que los diseñadores del Kerberos no esperaban que todo el mundo confiara en un solo servidor de validación de identificación, tomaron las providencias necesarias para tener varios **reinos**, cada uno con su propio AS y su propio TGS. Si quiere obtener un billete para un servidor de un reino distante, Alicia pedirá a su propio TGS un billete aceptado por el TGS del reino

distante. Si el TGS distante se ha registrado con el TGS local (de la misma manera en que lo hacen los servidores locales), el TGS local le dará a Alicia un billete válido para el TGS distante. Entonces ella podrá trabajar por allá, por ejemplo obteniendo billetes para los servidores remotos de ese reino. Nótese, sin embargo, que para que partes de dos reinos trabajen, cada una debe confiar en el TGS de la otra.

El Kerberos V5 es más refinado que el V4, y tiene más carga extra, además, usa la ASN.1 (*Abstract Syntax Notation 1*, notación de sintaxis abstracta 1) del OSI para describir los tipos de datos y tiene pequeños cambios en los protocolos. Esta versión tiene mayores tiempos de vida de los billetes, permite la renovación de éstos y puede emitir billetes posfechados. Además, cuando menos en teoría, no es dependiente del DES, como V4, y maneja reinos múltiples.

Validación de identificación usando criptografía de clave pública

La validación de identificación mutua también puede hacerse usando criptografía de clave pública. Para comenzar, supongamos que Alicia y Benjamín ya conocen las claves públicas del otro (un asunto nada trivial). Ellos quieren establecer una sesión y luego usar criptografía de clave secreta en esa sesión, ya que típicamente es de 100 a 1000 veces más rápida que la criptografía de clave pública. El propósito de este intercambio inicial entonces es validar la identificación de ambos y acordar una clave de sesión secreta compartida.

Este procedimiento puede hacerse de varias maneras. Una típica se muestra en la figura 7-21. Aquí, Alicia comienza por cifrar su identidad y un número al azar, R_A, usando la clave pública (o de cifrado) de Benjamín, E_B. Cuando Benjamín recibe este mensaje, no sabe si vino de Alicia o de Trudy, pero entra en el juego y devuelve a Alicia un mensaje que contiene el R_A de Alicia, su propio número aleatorio R_B, y la clave de sesión propuesta, K_S.

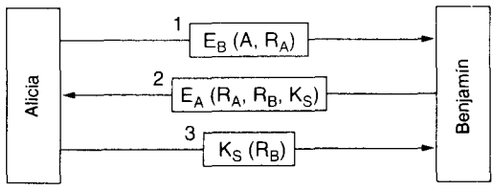


Figura 7-21. Validación de identificación mutua usando criptografía de clave pública.

Cuando Alicia recibe el mensaje 2, lo descifra usando su clave privada y encuentra R_A en él, lo que la hace sentirse cómoda. El mensaje debió haber venido de Benjamín, puesto que Trudy no tiene manera de determinar R_A. Es más, el mensaje debe ser reciente y no una repetición, puesto que ella acaba de enviar R_A a Benjamín. Alicia acepta la sesión devolviendo el mensaje 3. Cuando Benjamín ve R_B cifrado con la clave de sesión que él acaba de generar, sabe que Alicia recibió el mensaje 2 y comprobó R_A.

¿Qué puede hacer Trudy para sabotear este protocolo? Puede crear un mensaje 1 falso y engañar a Benjamín para que sondee a Alicia, pero Alicia verá un R_A que no envió y no continuará.

Trudy no pude falsificar el mensaje 3 de manera convincente, puesto que no conoce R_B ni K_S y no puede determinarlos sin la clave privada de Alicia. Mala suerte.

No obstante, el protocolo sí tiene una debilidad: supone que Alicia y Benjamín ya conocen la clave pública del otro. Supongamos que no es así. Alicia podría simplemente enviar a Benjamín su clave pública en el primer mensaje y pedir a Benjamín que devuelva la suya en el siguiente. El problema de este enfoque es que está sujeto a un ataque de brigada de cubetas. Trudy puede capturar el mensaje de Alicia a Benjamín y devolver su propia clave a Alicia. Alicia pensará que tiene una clave para hablar con Benjamín cuando, de hecho, tiene una clave para hablar con Trudy. Ahora Trudy puede leer todos los mensajes cifrados con lo que Alicia piensa que es la clave pública de Benjamín.

El intercambio inicial de claves públicas puede evitarse almacenando todas las claves públicas en una base de datos pública. Así, Alicia y Benjamín pueden obtener la clave pública del otro de la base de datos. Desgraciadamente, Trudy aún puede poner en práctica el ataque de brigada de cubetas interceptando las solicitudes a la base de datos y enviando respuestas simuladas que contengan su propia clave. A fin de cuentas, ¿cómo pueden saber Alicia y Benjamín que las respuestas vinieron de la base de datos, y no de Trudy?

Rivest y Shamir (1984) han diseñado un protocolo que frustra el ataque de brigada de cubetas de Trudy. En su **protocolo de interbloqueo**, tras el intercambio de claves públicas, Alicia envía sólo la mitad de su mensaje a Benjamín, digamos, sólo los bits pares (después del cifrado). Benjamín entonces responde con sus bits pares. Tras recibir los bits pares de Benjamín, Alicia envía sus bits nones, y luego lo hace Benjamín.

El truco aquí es que, cuando Trudy recibe los bits pares de Alicia, no puede descifrar el mensaje, aunque tiene la clave privada. En consecuencia, Trudy es incapaz de volver a cifrar los bits pares usando la clave pública de Benjamín. Si Trudy envía basura a Benjamín, el protocolo continuará, pero Benjamín pronto se dará cuenta de que el mensaje reensamblado no tiene sentido y de que ha sido engañado.

7.1.6. Firmas digitales

La validación de identificación de muchos documentos legales, financieros y de otros tipos se determina por la presencia o ausencia de una firma manuscrita autorizada. Las fotocopias no cuentan. Para que los sistemas computarizados de mensajes reemplacen el transporte físico de papel y tinta, debe encontrarse una solución a estos problemas.

El problema de inventar un reemplazo para las firmas manuscritas es difícil. Básicamente, lo que se requiere es un sistema mediante el cual una parte pueda enviar un mensaje "firmado" a otra parte de modo que

1. El receptor pueda verificar la identidad proclamada del transmisor.
2. El transmisor no pueda repudiar después el contenido del mensaje.
3. El receptor no haya podido confeccionar el mensaje él mismo.

El primer requisito es necesario, por ejemplo, en los sistemas financieros. Cuando la computadora de un cliente ordena a la computadora de un banco que compre una tonelada de oro, la

computadora del banco necesita asegurarse de que la computadora que da la orden realmente pertenece a la compañía a la que se le aplicará el débito.

El segundo requisito es necesario para proteger al banco contra fraudes. Supongamos que el banco compra la tonelada de oro, e inmediatamente después cae marcadamente el precio del oro. Un cliente deshonesto podría demandar al banco, alegando que nunca emitió una orden para comprar el oro. Cuando el banco presenta un mensaje en la corte, el cliente niega haberlo enviado.

El tercer requisito es necesario para proteger al cliente en el caso de que el precio del oro suba mucho y que el banco trate de falsificar un mensaje firmado en el que el cliente solicitó un lingote de oro en lugar de una tonelada.

Firmas de clave secreta

Un enfoque de las firmas digitales sería tener una autoridad central que sepa todo y en quien todos confíen, digamos el *Big Brother* (*BB*). Cada usuario escoge entonces una clave secreta y la lleva personalmente a las oficinas de *BB*. Por tanto, sólo Alicia y *BB* conocen la clave secreta de Alicia, K_A, etcétera.

Cuando Alicia quiere enviar un mensaje de texto normal firmado, P, a su banquero, Benjamín, genera $K_A(B, R_A, t, P)$ y lo envía como se muestra en la figura 7-22. *BB* ve que el mensaje es de Alicia, lo descifra, y envía un mensaje a Benjamín como se muestra. El mensaje a Benjamín contiene el texto normal del mensaje de Alicia y también el mensaje firmado $K_{BB}(A, t, P)$, donde t es una marca de tiempo. Ahora, Benjamín atiende la solicitud de Alicia.

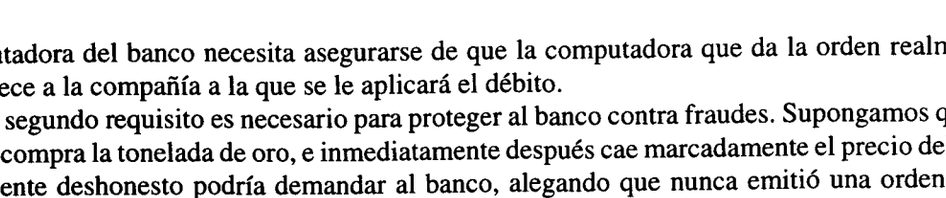


Figura 7-22. Firmas digitales con el *Big Brother*.

¿Qué ocurre si Alicia luego niega el envío del mensaje? El paso 1 es que todos demandan a todos (cuando menos en Estados Unidos). Por último, cuando el caso llega a la corte y Alicia niega haber enviado a Benjamín el mensaje en disputa, el juez pregunta a Benjamín por qué está tan seguro de que el mensaje en disputa vino de Alicia y no de Trudy. Benjamín primero indica que *BB* no aceptaría un mensaje de Alicia a menos que estuviera cifrado con K_A, por lo que no hay posibilidad de que Trudy enviara a *BB* un mensaje falso de Alicia.

Benjamín entonces presenta la prueba A, $K_{BB}(A, t, P)$. Benjamín dice que éste es un mensaje firmado por *BB* que comprueba que Alicia envió P a Benjamín. El juez entonces pide a *BB* (en quien todo el mundo confía) descifrar la prueba A. Cuando *BB* testifica que Benjamín dice la verdad, el juez se pronuncia a favor de Benjamín. Caso cerrado.

Un problema potencial del protocolo de firma de la figura 7-22 es que Trudy repita cualquiera de los dos mensajes. Para minimizar este problema, se usan en todos los intercambios marcas de tiempo. Es más, Benjamín puede revisar todos los mensajes recientes para ver si se usó R_A en cualquiera de ellos. De ser así, el mensaje se descarta como repetición. Nótese que Benjamín rechazará los mensajes muy viejos con base en la marca de tiempo. Para protegerse contra ataques de repetición instantánea, Benjamín simplemente examina el R_A de cada mensaje de entrada para ver si un mensaje igual se recibió de Alicia durante la hora pasada. Si no, Benjamín puede suponer con seguridad que ésta es una solicitud nueva.

Firmas de clave pública

Un problema estructural del uso de la criptografía de clave secreta para las firmas digitales es que todos tienen que confiar en el *Big Brother*. Es más, el *Big Brother* lee todos los mensajes firmados. Los candidatos más lógicos para operar el servidor del *Big Brother* son el gobierno, los bancos y los abogados. Estas organizaciones no inspiran confianza completa a todos los ciudadanos. Por tanto, sería bueno si la firma de documentos no requiriese una autoridad confiable.

Afortunadamente, la criptografía de clave pública puede hacer una contribución importante aquí. Supongamos que los algoritmos públicos de cifrado y descifrado tienen la propiedad de que $E(D(P)) = P$ además de la propiedad normal de $D(E(P)) = P$. (El RSA tiene esta propiedad, por lo que el supuesto es razonable.) Suponiendo que éste es el caso, Alicia puede enviar un mensaje de texto normal firmado, P, a Benjamín transmitiendo $E_B(D_A(P))$. Nótese que Alicia conoce su propia clave de descifrado (privada), D_A, así como la clave pública de Benjamín, E_B, por lo que la construcción de este mensaje es algo que Alicia puede hacer.

Cuando Benjamín recibe el mensaje, lo transforma usando su clave privada, como es normal, produciendo $D_A(P)$, como se muestra en la figura 7-23. Benjamín almacena este texto en un lugar seguro y lo descifra usando E_A para obtener el texto normal original.

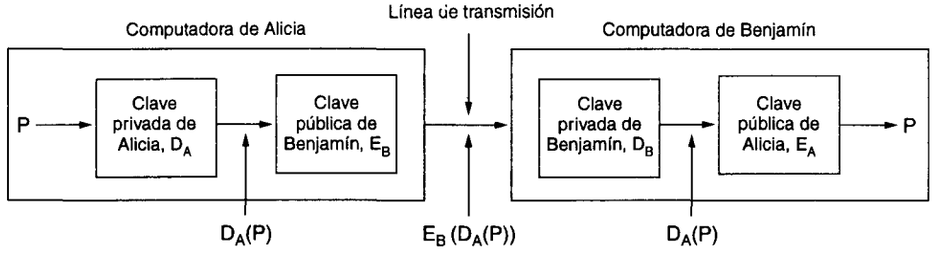


Figura 7-23. Firmas digitales usando criptografía de clave pública.

Para ver cómo funciona la propiedad de firma, supongamos que Alicia posteriormente niega haber enviado el mensaje P a Benjamín. Cuando se presenta el caso en la corte, Benjamín puede presentar tanto P como $D_A(P)$. El juez puede comprobar fácilmente que Benjamín tiene un mensaje válido cifrado por D_A con solo aplicarle E_A. Puesto que Benjamín no conoce la clave

privada de Alicia, la única forma en que Benjamín pudo haber adquirido un mensaje cifrado con ella sería que Alicia en efecto lo hubiera enviado. Mientras está en la cárcel por perjurio y fraude, Alicia tendrá tiempo suficiente para diseñar nuevos e interesantes algoritmos de clave pública.

Aunque el uso de la criptografía de clave pública para las firmas digitales es un esquema elegante, hay problemas relacionados con el entorno en el que opera más que con el algoritmo básico. Por una parte, Benjamín puede demostrar que un mensaje fue enviado por Alicia siempre y cuando D_A permanezca en secreto. Si Alicia divulga su clave secreta, el argumento ya no se mantiene, puesto que cualquiera pudo haber enviado el mensaje, incluido el mismo Benjamín.

El problema podría surgir, por ejemplo, si Benjamín es el corredor de bolsa de Alicia. Alicia le indica a Benjamín que compre ciertas acciones o bonos. Inmediatamente después, el precio cae en picada. Para repudiar su mensaje a Benjamín, Alicia corre a la policía para informarles que robaron su casa y que también sustrajeron su clave. Dependiendo de las leyes de su estado o país, puede o no ser responsable legalmente, especialmente si indica no haber descubierto el robo hasta después de llegar del trabajo, varias horas después.

Otro problema con el esquema de firmas es qué ocurre si Alicia decide cambiar su clave. Hacerlo ciertamente es legal, y probablemente es una buena idea cambiar la clave periódicamente. Si luego surge un caso en la corte, como se describió antes, el juez aplicará la E_A *actual* a $D_A(P)$ y descubrirá que no produce P. Benjamín quedará en ridículo en ese momento. En consecuencia, parece que sí se requiere alguna autoridad para registrar todos los cambios de clave y sus fechas.

En principio, cualquier algoritmo de clave pública puede usarse para firmas digitales. El estándar *de facto* de la industria es el algoritmo RSA, y muchos productos de seguridad lo usan. Sin embargo, en 1991, el NIST (*National Institute of Standards and Technology*) propuso el uso de una variación del algoritmo de clave pública de El Gamal para su nuevo **Estándar de firmas digitales** (**DSS**). La seguridad de El Gamal radica en la dificultad para calcular logaritmos discretos, en lugar de la dificultad para factorizar números grandes.

Como es normal cuando el gobierno intenta dictar estándares criptográficos, hubo una protesta general. El DSS se criticó por ser

1. Demasiado secreto (el NSA diseñó el protocolo para usar El Gamal).
2. Demasiado nuevo (El Gamal no ha sido analizado exhaustivamente).
3. Demasiado lento (10 a 40 veces más lento que el RSA para comprobar firmas).
4. Demasiado inseguro (clave fija de 512 bits).

En una modificación posterior, el cuarto punto se vino abajo cuando se permitieron claves de hasta 1024 bits. Aún no está claro si el DSS echará raíces. Para mayores detalles, véase (Kaufman *et al.*, 1995, Schneier, 1996, y Stinson, 1995).

Compendios de mensaje

Una crítica a los métodos de firma es que con frecuencia reúnen dos funciones: validación de identificación y secreto. En muchos casos se requiere la validación de identificación, pero no del

secreto. Puesto que la criptografía es lenta, a menudo es deseable poder mandar documentos de texto normal firmados. A continuación describiremos un esquema de validación de identificación que no requiere el cifrado del mensaje completo (De Jonge y Chaum, 1987).

Este esquema se basa en la idea de una función de dispersión unidireccional que toma una parte arbitrariamente grande de texto común y a partir de ella calcula una cadena de bits de longitud fija. Esta función de dispersión, llamada **compendio de mensaje** (*message digest*), tiene tres propiedades importantes:

1. Dado P, es fácil calcular $MD(P)$.
2. Dado $MD(P)$, es imposible encontrar P.
3. Nadie puede generar dos mensajes que tengan el mismo compendio de mensaje.

Para cumplir el criterio 3, la dispersión debe ser de cuando menos 128 bits de longitud, y de preferencia mayor.

El cálculo de un compendio de mensaje a partir de un trozo de texto normal es mucho más rápido que el cifrado de ese texto normal con un algoritmo de clave pública, por lo que los compendios de mensaje pueden usarse para acelerar los algoritmos de firma digital. Para ver su funcionamiento, considere nuevamente el protocolo de firma de la figura 7-22. En lugar de firmar P con $K_{BB}(A, t, P)$, BB ahora calcula el compendio de mensaje aplicando MD a P para producir $MD(P)$. BB entonces incluye $K_{BB}(A, t, MD(P))$ como quinto elemento de la lista cifrada con K_B que se envía a Benjamín, en lugar de $K_{BB}(A, t, P)$.

Si surge una disputa, Benjamín puede presentar tanto P como $K_{BB}(A, t, MD(P))$. Una vez que el *Big Brother* lo ha descifrado para el juez, Benjamín tiene $MD(P)$, que está garantizado que es genuino, y el P supuesto. Dado que es prácticamente imposible que Benjamín encuentre otro mensaje que dé esta parcialización, el juez se convencerá fácilmente de que Benjamín dice la verdad. Este uso de compendios de mensaje ahorra tanto tiempo de cifrado como costos de transporte y almacenamiento de mensajes.

Los compendios de mensaje funcionan también en los criptosistemas de clave pública, como se muestra en la figura 7-24. Aquí, Alicia primero calcula el compendio de mensaje de su texto normal; luego firma el compendio de mensaje y envía tanto el compendio firmado como el texto normal a Benjamín. Si Trudy reemplaza P en el camino, Benjamín verá esto cuando calcule $MD(P)$ él mismo.

Se ha propuesto una variedad de funciones de compendio de mensaje. Las de mayor uso son MD5 (Rivest, 1992) y SHA (NIST, 1993). **MD5** es la quinta de una serie de funciones de

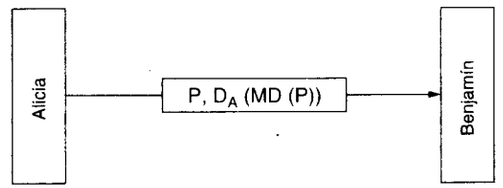


Figura 7-24. Firmas digitales usando compendios de mensaje.

dispersión diseñadas por Ron Rivest; opera alterando los bits de una manera tan complicada que cada bit de salida es afectado por cada bit de entrada. Muy brevemente, comienza por rellenar el mensaje a una longitud de 448 bits (módulo 512). Después se agrega la longitud original del mensaje como entero de 64 bits para dar una entrada total cuya longitud es un múltiplo de 512 bits. El último paso de precálculo es la inicialización de un *buffer* de 128 bits a un valor fijo.

Ahora comienza el cálculo. Cada ronda toma un bloque de 512 bits de entrada y lo mezcla por completo con el *buffer* de 128 bits. Por si fuera poco, se introduce también una tabla construida a partir de la función seno. El objetivo de usar una función conocida como el seno no es porque sea más aleatoria que un generador de números aleatorios, sino para evitar cualquier sospecha de que el diseñador construyó una puerta secreta ingeniosa por la que sólo él puede entrar. La negativa de IBM a hacer públicos los principios en que se basó el diseño de las cajas S del DES dio pie a una gran cantidad de especulación sobre las puertas secretas. Se hacen cuatro rondas por cada bloque de entrada. Este proceso continúa hasta que todos los bloques de entrada se han consumido. El contenido del *buffer* de 128 bits forma el compendio de mensaje. El algoritmo ha sido optimizado para su implementación en máquinas de 32 bits. Como consecuencia, podría no ser lo bastante rápido para redes futuras de alta velocidad (Touch, 1995).

La otra función importante de compendio de mensaje es el **SHA** (*Secure Hash Algorithm*, **algoritmo seguro de dispersión**), desarrollado por la NSA y bendecido por el NIST. Al igual que el MD5, SHA procesa datos de entrada en bloques de 512 bits pero, a diferencia del MD5, genera un compendio de mensaje de 160 bits. El SHA comienza por rellenar el mensaje, y luego agrega una cadena de 64 bits para obtener un múltiplo de 512 bits; por último, inicializa su *buffer* de salida de 160 bits.

Por cada bloque de entrada, el *buffer* de salida se actualiza usando al bloque de entrada de 512 bits. No se usa ninguna tabla de números aleatorios (ni de valores senoidales), pero por cada bloque se calculan 80 rondas, resultando en una mezcla exhaustiva. Cada grupo de 20 rondas usa diferentes funciones de mezcla. Puesto que el código de dispersión de SHA es 32 bits más largo que el de MD5, siendo todo lo demás igual, el primero es 2^{32} más seguro que el MD5; sin embargo, también es más lento que el MD5, y tener un código de dispersión que no es una potencia de dos a veces puede ser un inconveniente. Por lo demás, los dos algoritmos son técnicamente semejantes. Políticamente, el MD5 se define en un RFC y se usa de manera intensiva en Internet. El SHA es un estándar del gobierno, y lo usan compañías que tienen que usarlo porque el gobierno les dice que lo hagan, o aquellas que quieren seguridad extra. Una versión modificada, el SHA-1, ha sido aprobada como estándar por el NIST.

El ataque de cumpleaños

En el mundo de la criptografía, nada es lo que parece. Podríamos pensar que se requieren del orden de 2^m operaciones para subvertir un compendio de mensaje de m bits. De hecho, con frecuencia $2^{m/2}$ operaciones son suficientes si se usa el **ataque de cumpleaños**, un enfoque publicado por Yuval (1979) en su ahora clásico trabajo "*How to Swindle Rabin*" (Cómo estafar a Rabin).

La idea de este ataque proviene de una técnica que con frecuencia usan los profesores de matemáticas en sus cursos de probabilidad. La pregunta es: ¿Cuántos estudiantes se necesitan en un grupo antes de que la probabilidad de tener dos personas con el mismo cumpleaños exceda 1/2? Muchos estudiantes suponen que la respuesta debe ser mucho mayor que 100. De hecho, la teoría de la probabilidad indica que es de apenas 23. Sin hacer un análisis riguroso, intuitivamente, con 23 personas, podemos formar $(23 \times 22)/2 = 253$ pares diferentes, cada uno de los cuales tiene una probabilidad de 1/365 de cumplir el requisito. Bajo esta luz, la respuesta ya no es en realidad tan sorprendente.

En términos más generales, si hay alguna correspondencia entre las entradas y las salidas con n entradas (gente, mensajes, etc.) y k salidas posibles (cumpleaños, compendios de mensaje, etc.) hay $n(n-1)/2$ pares de entradas. Si $n(n-1)/2 > k$, la posibilidad de que cuando menos una coincida es bastante buena. Por lo tanto, en términos aproximados, es probable una igualación para $n > \sqrt{k}$. Este resultado significa que un compendio de mensaje de 64 bits probablemente puede violarse generando unos 2^{32} mensajes y buscando dos con el mismo compendio de mensaje.

Veamos ahora un ejemplo práctico. El Departamento de Informática de la Universidad Estatal tiene una cátedra para un miembro facultativo y tiene dos candidatos, Tomás y Daniel. Tomás fue contratado dos años antes que Daniel, por lo que su candidatura será considerada antes. Si Tomás obtiene el puesto, mala suerte para Daniel. Tomás sabe que la jefa del departamento, Marilyn, tiene buen concepto de su trabajo, por lo que le pide que escriba una carta de recomendación para el rector, quien decidirá el caso de Tomás. Una vez enviadas, todas las cartas se vuelven confidenciales.

Marilyn le dice a su secretaria, Elena, que escriba una carta al rector, delineando lo que quiere en ella. Cuando está lista, Marilyn la revisará, calculará y firmará el compendio de 64 bits y lo enviará al rector. Elena puede mandar la carta después por correo electrónico.

Desafortunadamente para Tomás, Elena está relacionada románticamente con Daniel y le gustaría dejar fuera a Tomás, por lo que escribe la carta siguiente con las 32 opciones entre corchetes.

Estimado rector Sánchez,

Esta [*carta* | *mensaje*] es para dar mi opinión [*franca* | *honesta*] sobre el profesor Tomás Guerrero, que [*ahora* | *este año*] [*es candidato a* | *está en espera de*] una cátedra. He [*conocido a* | *trabajado con*] el profesor Guerrero durante [*unos* | *casi*] seis años. Es un investigador [*sobresaliente* | *excelente*] de gran [*talento* | *habilidad*] conocido [*mundialmente* | *internacionalmente*] por sus [*brillantes* | *creativas*] investigaciones sobre [*muchos* | *una gran variedad de*] problemas [*difíciles* | *desafiantes*].

Él es también un [*profesor* | *educador*] [*altamente* | *grandemente*] [*respetado* | *admirado*]. Sus estudiantes han hecho evaluaciones [*sobresalientes* | *espectaculares*] de sus [*clases* | *cursos*]; es el [*profesor* | *instructor*] [*más admirado* | *más querido*] [*del departamento* | *por nosotros*].

[*Además* | *Por otra parte*], el profesor Guerrero es [*hábil* | *diligente*] para obtener financiamiento. Sus [*subvenciones* | *contratos*] han aportado una cantidad [*importante* | *sustancial*] de dinero [*al* | *a nuestro*] departamento. [*Este dinero ha* | *Estos fondos han*] [*posibilitado* | *permitido*] que [*emprendamos* | *pongamos en práctica*] muchos programas [*especiales* | *importantes*], [*como* | *por ejemplo*] su programa Estado 2000. Sin estos fondos [*seríamos incapaces de* | *no podríamos*] continuar este programa, que es tan [*importante* | *esencial*] para ambos. Recomiendo encarecidamente que se le otorgue la cátedra.

Desgraciadamente para Tomás, tan pronto como Elena termina de redactar y mecanografiar esta carta, también escribe una segunda:

Esta [*carta* | *mensaje*] es para dar mi opinión [*franca* | *honesta*] sobre el profesor Tomás Guerrero, que [*ahora* | *este año*] [*es candidato a* | *está en espera de*] una cátedra. He [*conocido a* | *trabajado con*] el profesor Guerrero durante [*unos* | *casi*] seis años. Es un investigador [*malo* | *mediocre*] poco conocido en su [*campo* | *área*]. Sus investigaciones [*casi nunca* | *pocas veces*] muestran [*entendimiento* | *comprensión*] de los problemas [*clave* | *principales*] [*del día* | *de nuestros tiempos*].

Es más, no es un [*profesor* | *educador*] [*respetado* | *admirado*]. Sus estudiantes han hecho evaluaciones [*pobres* | *malas*] de sus [*clases* | *cursos*]; es el [*maestro* | *instructor*] menos querido [*del departamento* | *por nosotros*], conocido [*principalmente* | *más*] en [*el* | *nuestro*] departamento por su [*tendencia* | *propensión*] a [*ridiculizar* | *avergonzar*] a los estudiantes lo bastante [*tontos* | *imprudentes*] como para hacer preguntas durante su clase.

[*Además* | *Por otra parte*], el profesor Guerrero no es [*hábil* | *diligente*] para obtener financiamiento. Sus [*subvenciones* | *contratos*] han aportado una cantidad [*insustancial* | *insignificante*] de dinero [*al* | *a nuestro*] departamento. A menos que [*se recolecte dinero nuevo* | *se consigan fondos nuevos*] pronto tendremos que cancelar algunos programas esenciales, como su programa Estado 2000. Desgraciadamente, en estas [*condiciones* | *circunstancias*] no puedo recomendarlo de buena [*conciencia* | *fe*] a usted para [*la cátedra* | *un puesto permanente*].

Ahora Elena prepara su computadora para calcular los 2^{32} compendios de mensaje para cada carta durante la noche. Con suerte, un compendio de la primera carta será igual a un compendio de la segunda. De no serlo puede agregar algunas opciones más e intentar de nuevo durante el fin de semana. Supongamos que encuentra una correspondencia. Llamemos a la carta "buena" *A* y a la "mala" *B*.

Ahora Elena envía la carta *A* a Marilyn para su aprobación. Marilyn, por supuesto, la aprueba, calcula su compendio de mensaje de 64 bits, firma el compendio y manda por correo electrónico el compendio firmado al rector Sánchez. Independientemente, Elena envía la carta *B* al rector.

Al recibir la carta y el compendio de mensaje firmado, el rector ejecuta el algoritmo de compendio de mensaje con la carta *B*, ve que coincide con lo que Marilyn le envió, y despide a Tomás. (Final opcional: Elena le dice a Daniel lo que hizo. Daniel se horroriza y rompe con ella. Elena se pone furiosa y confiesa su falta a Marilyn. Marilyn llama al rector. Tomás consigue la cátedra a fin de cuentas.) Con el MD5, el ataque de cumpleaños es infactible porque aun a una velocidad de mil millones de compendios por segundo, se requerirían más de 500 años para calcular los 2^{64} compendios de dos cartas con 64 variantes cada una, e incluso entonces no se garantiza una equivalencia.

7.1.7. Aspectos sociales

Las implicaciones de la seguridad de las redes para la confidencialidad individual y social en general son abrumadoras. A continuación mencionaremos unos cuantos de los temas sobresalientes.

A los gobiernos no les gusta que los ciudadanos guarden secretos. En algunos países (por ejemplo, Francia), toda la criptografía no gubernamental simplemente está prohibida a menos que el gobierno tenga todas las claves empleadas. Como apuntan Kahn (1980) y Selfridge y Schwartz (1980), la práctica gubernamental de escuchar secretamente se practica en una escala mucho más masiva de lo que podría soñar la mayoría de la gente, y el gobierno quiere más que simplemente una pila de bits indescifrables a cambio de sus esfuerzos.

El gobierno de Estados Unidos ha propuesto un esquema de cifrado para los teléfonos digitales futuros que incluye una característica especial para permitir a la policía la intervención y descifrado de todas las llamadas telefónicas hechas en ese país. El gobierno promete no usar esta característica sin una orden judicial, pero muchos aún se acuerdan de la manera en que el director de FBI, J. Edgar Hoover, intervino ilegalmente los teléfonos de Martin Luther King, Jr. y otros en un intento por neutralizarlos. La policía dice que necesita este poder para atrapar criminales. El debate entre ambos lados es airado, por llamarlo de alguna manera. En (Kaufman *et al.*, 1995) se presenta un estudio de la tecnología en cuestión (Clipper). Una manera de sortear esta tecnología y enviar mensajes que el gobierno no pueda leer se describe en (Blaze, 1994, y Schneier, 1996). En (Hoffman, 1995) se presentan las posturas de todas las partes del debate.

Estados Unidos tiene una ley (22 U.S.C. 2778) que prohíbe a los ciudadanos la exportación de material de guerra, como tanques y aviones de combate, sin autorización del Departamento de la Defensa. Para los fines de esta ley, el *software* criptográfico se clasifica material de guerra. Phil Zimmermann, quien escribió PGP (*Pretty Good Privacy*, confidencialidad bastante buena), un programa de protección de correo electrónico, ha sido acusado de violar esta ley, aunque el gobierno reconoce que Zimmermann no exportó el programa (pero lo dio a un amigo que lo puso en Internet, donde los extranjeros lo podían obtener). Muchos consideraron este incidente como una violación grave de los derechos de un ciudadano estadounidense que trabaja para proteger la confidencialidad de la gente.

No ser ciudadano estadounidense tampoco ayuda. El 9 de julio de 1986 tres investigadores del Instituto Weizmann de Israel solicitaron una patente en Estados Unidos para un nuevo esquema de firma digital de su invención. Durante los siguientes seis meses los investigadores presentaron su trabajo en conferencias por todo el mundo. El 6 de enero de 1987, la oficina de patentes les indicó que notificaran a todos los estadounidenses que conocían sus resultados que la divulgación de las investigaciones podría hacerlos acreedores a dos años de prisión, una multa de 10,000 dólares, o ambas cosas. La oficina de patentes también quería una lista de todos los extranjeros que sabían algo sobre las investigaciones. Para conocer el final de esta historia, véase (Landau, 1988).

Las patentes son otro tema candente. Casi todos los algoritmos de clave pública están patentados. La protección de patentes dura 17 años. La patente del RSA, por ejemplo, vence el 20 de septiembre del 2000.

La seguridad de las redes tiene más implicaciones políticas que casi ningún otro tema técnico, y con justa razón, puesto que se relaciona con la diferencia entre una democracia y un estado policiaco en la era digital. Las ediciones de marzo de 1993 y noviembre de 1994 de *Communications of the ACM* incluyen amplias secciones sobre la seguridad telefónica y de redes, respectivamente, con argumentos que explican y defienden muchos puntos de vista. El capítulo

25 del libro de seguridad de Schneier trata la política de la criptografía (Schneier, 1996) lo mismo que el capítulo 8 de su libro de correo electrónico (Schneier, 1995). La confidencialidad y las computadores también se estudian en (Adam, 1995). Estas referencias son muy recomendables para los lectores que deseen ampliar sus estudios sobre este tema.

7.2. DNS — SISTEMA DE NOMBRES DE DOMINIO

Los programas pocas veces hacen referencia a los *hosts*, buzones de correo y otros recursos por sus direcciones binarias de red. En lugar de números binarios, los programas usan cadenas ASCII, como *tana@art.ucsb.edu.* Sin embargo, la red misma sólo entiende direcciones binarias, por lo que se requiere algún mecanismo para convertir las cadenas ASCII en direcciones de red. En las siguientes secciones estudiaremos la manera en que se logra esta correspondencia en Internet.

Hace mucho, en los tiempos del ARPANET, simplemente había un archivo, *hosts.txt*, en el que se listaban todos los *hosts* y sus direcciones IP. Cada noche, todos los *hosts* obtenían este archivo de la instalación en la que se mantenía. En una red de unas cuantas máquinas grandes de tiempo compartido, este enfoque funcionaba razonablemente bien.

Sin embargo, cuando miles de estaciones de trabajo se conectaron a la red, todos se dieron cuenta de que este enfoque no podría continuar funcionando eternamente. Por una parte, el archivo se volvería demasiado grande. Un problema aún más importante era que ocurrirían conflictos constantes con los nombres de los *hosts* a menos de que los nombres se administraran centralmente, algo impensable en una red internacional enorme. Para resolver estos problemas, se inventó el **DNS** (*Domain Name System*, **sistema de nombres de dominio**).

La esencia del DNS es la invención de un esquema de nombres jerárquico basado en dominio y una base de datos distribuida para implementar este esquema de nombres. El DNS se usa principalmente para relacionar las direcciones de *host* y destinos de correo electrónico con las direcciones IP, pero también puede usarse con otros fines. El DNS se define en los RFC 1034 y 1035.

Muy brevemente, el modo de usar el DNS es el siguiente. Para relacionar un nombre con una dirección IP, un programa de aplicación llama a un procedimiento de biblioteca llamado **resolvedor**, pasándole el nombre como parámetro. El resolvedor envía un paquete UDP a un servidor DNS local, que entonces busca el nombre y devuelve la dirección IP al resolvedor, que entonces lo devuelve al solicitante. Con la dirección IP, el programa puede entonces establecer una conexión TCP con el destino, o enviarle paquetes UDP.

7.2.1. El espacio de nombres del DNS

La administración de un grupo grande y continuamente cambiante de nombres es un problema nada sencillo. En el sistema postal, la administración de nombres se hace requiriendo letras para especificar (implícita o explícitamente) el país, estado o provincia, ciudad y calle, y dirección del destinatario. Con este tipo de direccionamiento jerárquico, no hay confusión entre el Marvin Anderson de Main St., en White Plains, N.Y. y el Marvin Anderson de Main St., en Austin, Texas. El DNS funciona de la misma manera.

Conceptualmente, la Internet se divide en varios cientos de **dominios** de nivel superior, cada uno de los cuales abarca muchos *hosts*. Cada dominio se divide en subdominios, y éstos se dividen nuevamente, etc. Todos estos dominios pueden representarse mediante un árbol, como se muestra en la figura 7-25. Las hojas del árbol representan los dominios que no tienen subdominios (pero que, por supuesto, contienen máquinas). Un dominio de hoja puede contener un solo *host*, o puede representar a una compañía y contener miles de *hosts*.

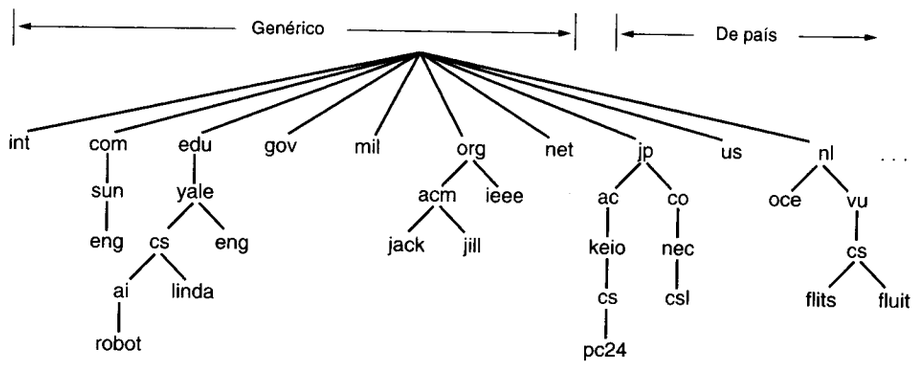


Figura 7-25. Parte del espacio de nombres de dominio de Internet.

Los dominios de nivel superior vienen en dos sabores: genéricos y de país. Los dominios genéricos son *com* (*comercial*), *edu* (instituciones educativas), *gov* (el gobierno federal de Estados Unidos), *int* (ciertas organizaciones internacionales), *mil* (las fuerzas armadas de Estados Unidos), *net* (proveedores de red) y *org* (organizaciones no lucrativas). Los dominios de país incluyen una entrada para cada país, como se define en el ISO 3166.

Cada dominio se nombra por la trayectoria hacia arriba desde él a la raíz (sin nombre). Los componentes se separan por puntos. Por tanto, el departamento de ingeniería de Sun Microsystems podría ser *eng.sun.com.*, a diferencia de un nombre tipo UNIX como */com/sun/eng*. Nótese que este nombramiento jerárquico significa que *eng.sun.com.* no entra en conflicto con un uso potencial de *eng* en *eng.yale.edu.*, que podría usarse en el departamento de inglés de Yale.

Los nombres de dominio pueden ser absolutos o relativos. Un nombre de dominio absoluto termina con un punto (por ejemplo, *eng.sun.com.*), y uno relativo no. Los nombres relativos tienen que interpretarse en algún contexto para determinar de manera única su significado verdadero. En ambos casos, un dominio nombrado se refiere a un nodo específico del árbol y a todos los nodos por debajo de él.

Los nombres de dominio no hacen distinción entre las mayúsculas y las minúsculas, por lo que *edu* y *EDU* significan la misma cosa. Los nombres de componentes pueden ser de hasta 63 caracteres de longitud, y los nombres de trayectoria completa no deben exceder los 255 caracteres.

En principio, los dominios pueden introducirse en el árbol de tres maneras diferentes. Por ejemplo, *cs.yale.edu* podría estar listado también bajo el dominio de país *us* como *cs.yale.ct.us*.

En la práctica, casi todas las organizaciones de Estados Unidos están bajo un dominio genérico, y casi todas las de fuera de Estados Unidos están bajo el dominio de su país. No hay ninguna regla contra el registro bajo dos dominios de nivel superior, pero hacerlo podría causar confusión, por lo que pocas organizaciones lo hacen.

Cada dominio controla el modo de asignación de los dominios que están debajo de él. Por ejemplo, Japón tiene los dominios *ac.jp* y *co.jp* que son espejos de *edu* y *com*. Los Países Bajos no hacen esta distinción y ponen a todas las organizaciones directamente bajo *nl*. Por tanto, los siguientes tres son departamentos universitarios de informática:

1. *cs.yale.edu* (Universidad de Yale, en Estados Unidos)
2. *cs.vu.nl* (Vrije Universiteit, en los Países Bajos)
3. *cs.keio.ac.jp* (Universidad Keio, en Japón)

Para crear un dominio nuevo, se requiere el permiso del dominio en el que se incluirá. Por ejemplo, si se inicia un grupo VLSI en Yale y quiere que se le conozca como *vlsi.cs.yale.edu*, requiere permiso de quien administra *cs.yale.edu*. De la misma manera, si se crea una universidad nueva, digamos la Universidad del Norte de Dakota del Sur, debe solicitar al administrador del dominio *edu* que le asigne *unsd.edu*. De esta manera se evitan los conflictos de nombres y cada dominio puede llevar el registro de todos sus subdominios, como *cs.unsd.edu*, sin obtener el permiso de nadie más arriba en el árbol.

Los nombres reflejan los límites organizacionales, no las redes físicas. Por ejemplo, si los departamentos de informática e ingeniería electrónica se ubican en el mismo edificio y comparten la misma LAN, de todas maneras pueden tener dominios diferentes. De la misma manera, si el departamento de informática está dividido entre el edificio Babbage y el edificio Turing, todos los *hosts* de ambos edificios pertenecerán, normalmente, al mismo dominio.

7.2.2. Registros de recursos

Cada dominio, sea un *host* individual o un dominio de nivel superior, puede tener un grupo de **registros de recursos** asociados a él. En un *host* individual, el registro de recursos más común es simplemente su dirección IP, pero también existen muchos otros tipos de registros de recursos. Cuando un resolvedor da un nombre de dominio al DNS, lo que recibe son los registros de recursos asociados a ese nombre. Por tanto, la función real del DNS es relacionar los dominios de nombres con los registros de recursos.

Un registro de recursos tiene cinco tuplas. Aunque éstas se codifican en binario por cuestión de eficiencia, en la mayoría de las presentaciones los registros de recursos se dan como texto ASCII, una línea por registro de recurso. El formato que usaremos es el siguiente:

```
Nombre_dominio  Tiempo_de_vida  Tipo  Clase  Valor
```

El *nombre_dominio* indica el dominio al que pertenece este registro. Normalmente existen muchos registros por dominio y cada copia de la base de datos contiene información de muchos

dominios. Por tanto, este campo es la llave primaria de búsqueda usada para atender las consultas. El orden de los registros en la base de datos no es significativo. Cuando se hace una consulta relativa a un dominio, se devuelven todos los registros coincidentes de la clase solicitada.

El campo de *tiempo_de_vida* es una indicación de la estabilidad del registro. La información altamente estable recibe un valor grande, como 86400 (la cantidad de segundos en un día). La información altamente volátil recibe un valor pequeño, como 60 (1 minuto). Regresaremos a este punto después de haber estudiado el proceso de caché.

El campo *tipo* indica el tipo de registro de que se trata. Los tipos más importantes se listan en la figura 7-26.

Tipo	Significado	Valor
SOA	Inicio de autoridad	Parámetros para esta zona
A	Dirección de IP de un *host*	Entero de 32 bits
MX	Intercambio de correo	Prioridad, dominio dispuesto a aceptar correo electrónico
NS	Servidor de nombres	Nombre de un servidor para este dominio
CNAME	Nombre canónico	Nombre de dominio
PTR	Apuntador	Alias de una dirección IP
HINFO	Descripción del *host*	CPU y SO en ASCII
TXT	Texto	Texto ASCII no interpretado

Figura 7-26. Tipos principales de registro de recurso DNS.

Un registro *SOA* proporciona el nombre de la fuente primaria de información sobre la zona del servidor de nombres (que se describe más adelante), la dirección de correo electrónico de su administrador, un número de serie único y varias banderas y temporizadores.

El tipo de registro más importante es el registro *A* (dirección) que contiene una dirección IP de 32 bits de algún *host*. Cada *host* de Internet debe tener cuando menos una dirección IP, para que otras máquinas puedan comunicarse con él. Algunos *hosts* tienen dos o más conexiones de red, en cuyo caso tendrán un registro de recurso tipo *A* por cada conexión de red (y, por tanto, por cada dirección IP).

El siguiente tipo de registro más importante es el registro *MX*, que especifica el nombre del dominio que está preparado para aceptar correo electrónico del dominio especificado. Un uso común de este registro es permitir que una máquina que no está en Internet reciba correo electrónico de las instalaciones Internet. La entrega se logra haciendo que la instalación no Internet establezca un arreglo con alguna instalación Internet para que acepte correo electrónico dirigido a ella y lo reenvíe usando cualquier protocolo acordado por las dos.

Por ejemplo, supóngase que Cathy se graduó en informática en la UCLA. Tras recibir su título en informática, ella inicia una compañía, la Electrobrain Corporation, para comercializar sus ideas. No puede aún darse el lujo de una conexión con Internet, por lo que llega a un arreglo

con UCLA para recibir su correo electrónico ahí. Varias veces al día puede llamar y consultar su correo.

Acto seguido, Cathy registra su compañía con el dominio *com* y se le asigna el dominio *electrobrain.com*. Luego ella podría pedir al administrador del dominio *com* que agregue un registro *MX* a la base de datos *com*, a saber:

```
electrobrain.com   86400   IN   MX   1   mailserver.cs.ucla.edu
```

De esta manera, el correo se reenviará a UCLA donde Cathy puede recogerlo estableciendo una conexión. Como alternativa, la UCLA podría llamarla y transferir el correo electrónico mediante cualquier protocolo acordado mutuamente.

Los registros *NS* especifican servidores de nombres. Por ejemplo, cada base de datos DNS normalmente tiene un registro *NS* por cada dominio de nivel superior, de modo que el correo electrónico pueda enviarse a partes alejadas del árbol de nombres. Regresaremos a este punto más adelante.

Los registros *CNAME* permiten la creación de alias. Por ejemplo, una persona familiarizada con los nombres de Internet en general que quiere enviar un mensaje a alguien cuya clave de acceso es *paul* y está en el departamento de informática del M.I.T. podría adivinar que *paul@cs.mit.edu* funcionará. De hecho, esta dirección no funciona, puesto que el dominio del departamento de informática del M.I.T. es *lcs.mit.edu*. Sin embargo, como servicio para la gente que no sabe esto, el M.I.T. podría crear una entrada *CNAME* para encaminar a la gente y a los programas en la dirección correcta. La entrada podría ser como ésta:

```
cs.mit.edu   86400   IN   CNAME   lcs.mit.edu
```

Al igual que *CNAME*, *PTR* apunta a otro nombre. Sin embargo, a diferencia de *CNAME*, que en realidad es sólo una definición de macro, *PTR* es un tipo de datos DNS normal, cuya interpretación depende del contexto en que se encontró. En la práctica, *PTR* casi siempre se usa para asociar un nombre a una dirección IP a fin de permitir búsquedas de la dirección IP y devolver el nombre de la máquina correspondiente.

Los registros *HINFO* permiten que la gente conozca el tipo de máquina y sistema operativo al que corresponde un dominio. Por último, los registros *TXT* permiten a los dominios identificarse de modos arbitrarios. Ambos tipos de registro son para el provecho de los usuarios. Ninguno es obligatorio, por lo que los programas no pueden contar con que los recibirán (y probablemente no puedan manejarlos si los obtienen).

Regresando a la estructura general de los registros de recurso, el cuarto campo de cada registro de recurso es la *clase*. Para la información de Internet, es siempre *IN*. Para la información que no pertenece a Internet, pueden utilizarse otros códigos.

Por último, llegamos al campo de *valor*. Este campo puede ser un número, un nombre de dominio o una cadena ASCII. La semántica depende del tipo de registro. En la figura 7-26 se presenta una descripción corta de los campos de *valor* para cada uno de los tipos principales de registro.

Como ejemplo del tipo de información que podría encontrarse en la base de datos de un dominio, véase la figura 7-27. En esta figura se ilustra una parte de una base de datos (semihipotética)


```
; Datos autorizados correspondientes a cs.vu.nl
cs.vu.nl.       86400   IN   SOA      star boss (952771,7200,7200,2419200,86400)
cs.vu.nl.       86400   IN   TXT      "Faculteit Wiskunde en Informatica."
cs.vu.nl.       86400   IN   TXT      "Vrije Universiteit Amsterdam."
cs.vu.nl.       86400   IN   MX       1 zephyr.cs.vu.nl.
cs.vu.nl.       86400   IN   MX       2 top.cs.vu.nl.

flits.cs.vu.nl. 86400   IN   HINFO    Sun Unix
flits.cs.vu.nl. 86400   IN   A        130.37.16.112
flits.cs.vu.nl. 86400   IN   A        192.31.231.165
flits.cs.vu.nl. 86400   IN   MX       1 flits.cs.vu.nl.
flits.cs.vu.nl. 86400   IN   MX       2 zephyr.cs.vu.nl.
flits.cs.vu.nl. 86400   IN   MX       3 top.cs.vu.nl.
www.cs.vu.nl.   86400   IN   CNAME    star.cs.vu.nl
ftp.cs.vu.nl.   86400   IN   CNAME    zephyr.cs.vu.nl

rowboat                 IN   A        130.37.56.201
                        IN   MX       1 rowboat
                        IN   MX       2 zephyr
                        IN   HINFO    Sun Unix

little-sister           IN   A        130.37.62.23
                        IN   HINFO    Mac MacOS

laserjet                IN   A        192.31.231.216
                        IN   HINFO    "HP Laserjet IIISi" Proprietary
```

Figura 7-27. Parte de una posible base de datos DNS para *cs.vu.nl*.

correspondiente al dominio *cs.vu.nl* mostrado en la figura 7-25. La base de datos contiene siete tipos de registros de recurso.

La primera línea no de comentario de la figura 7-27 da un poco de información básica sobre el dominio, de lo que ya no nos ocuparemos. Las dos siguientes líneas dan información textual sobre la localización del dominio. Luego vienen dos entradas que dan el primer y segundo lugar a donde se intentará entregar correo electrónico dirigido a *person@cs.vu.nl*. La *zephyr* (una máquina específica) debe intentarse primero. Si falla, debe intentarse la *top* después.

Tras la línea en blanco, que se agregó para hacer más clara la lectura, siguen líneas que indican que la *flits* es una estación de trabajo Sun que ejecuta UNIX, y dan sus dos direcciones IP. Después se indican tres posibilidades para manejar el correo electrónico enviado a *flits.cs.vu.nl*. La primera opción naturalmente es la *flits* misma, pero si está inactiva, la *zephyr* y la *top* son la segunda y tercera opciones. Luego viene un alias, *www.cs.vu.nl*, para que esta dirección pueda usarse sin designar una máquina específica. La creación de este alias permite a *cs.vu.nl* cambiar su servidor del *World Wide Web* sin invalidar la dirección que la gente usa para llegar a él. Un argumento parecido es válido para *ftp.cs.vu.nl*.

Las siguientes cuatro líneas contienen una entrada típica de una estación de trabajo, en este caso, *rowboat.cs.vu.nl.* La información proporcionada contiene la dirección IP, los destinos de correo primarios y secundarios, e información sobre la máquina. Luego viene una entrada para un sistema no UNIX incapaz de recibir correo por sí mismo, seguida de una entrada para la impresora láser.

Lo que no se muestra (y no está en este archivo) son las direcciones IP a usar para buscar los dominios de nivel superior. Éstas se requieren para buscar *hosts* distantes pero, dado que no son parte del dominio *cs.vu.nl*, no están en este archivo. Tales direcciones son suministradas por los servidores raíz, cuyas direcciones IP están presentes en un archivo de configuración del sistema y se cargan en el caché del DNS al iniciar el servidor DNS. Tienen temporizaciones muy grandes por lo que, una vez cargadas, nunca se purgan del caché.

7.2.3. Servidores de nombres

Cuando menos en teoría, un solo servidor de nombres podría contener la base de datos DNS completa y responder a todas las consultas sobre ella. En la práctica, este servidor estaría tan sobrecargado que sería inservible. Es más, si llegara a caerse, la Internet completa se vendría abajo.

Para evitar los problemas asociados a tener una sola fuente de información, el espacio de nombres DNS se divide en **zonas** no traslapantes. Una manera posible de dividir el espacio de nombres de la figura 7-25 se muestra en la figura 7-28. Cada zona contiene una parte del árbol y también contiene servidores de nombres que tienen la información de autorización correspondiente a esa zona. Normalmente, una zona tendrá un servidor de nombres primario, que obtiene su información de un archivo en su disco, y uno o más nombres de servidores secundarios, que obtienen su información del servidor de nombres primario. Para mejorar la confiabilidad, algunos servidores de cierta zona pueden situarse fuera de la zona.

El lugar donde se colocan los límites de las zonas dentro de una zona es responsabilidad del administrador de esa zona. Esta decisión se toma en gran medida basada en la cantidad de

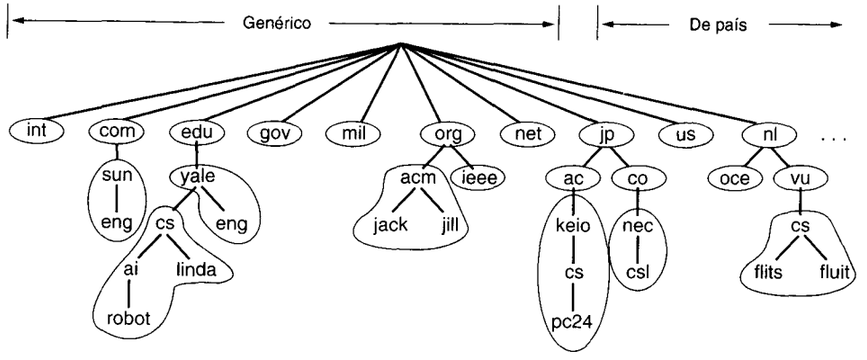


Figura 7-28. Parte del espacio de nombres DNS, donde se muestra la división en zonas.

servidores de nombres deseados y su ubicación. Por ejemplo, en la figura 7-28, Yale tiene un servidor para *yale.edu* que maneja *eng.yale.edu*, pero no *cs.yale.edu*, que es una zona aparte con sus propios servidores de nombres. Tal decisión podría tomarse cuando un departamento, como el de inglés, no desea operar su propio servidor de nombres, pero un departamento como el de informática sí. En consecuencia, *cs.yale.edu* es una zona aparte, pero *eng.yale.edu* no.

Cuando un resolvedor tiene una consulta referente a un nombre de dominio, la pasa a uno de los servidores de nombres locales. Si el dominio que se busca cae bajo la jurisdicción del servidor de nombres, como *ai.cs.yale.edu*, que cae bajo *cs.yale.edu*, devuelve los registros de recursos autorizados. Un **registro autorizado** es uno que viene de la autoridad que administra el registro, y por tanto siempre es correcto. Los registros autorizados contrastan con los registros en caché, que podrían no estar actualizados.

Por otro lado, si el dominio es remoto y no hay información disponible localmente sobre el dominio solicitado, el servidor de nombres envía un mensaje de consulta para el dominio solicitado al servidor de nombres de nivel superior. Para hacer más claro este proceso, considere el ejemplo de la figura 7-29. Aquí, un resolvedor de *flits.cs.vu.nl* quiere saber la dirección IP del *host linda.cs.yale.edu*. En el paso 1, envía una consulta al servidor de nombres local, *cs.vu.nl*. Esta consulta contiene el nombre de dominio buscado, el tipo (*A*) y la clase (*IN*).

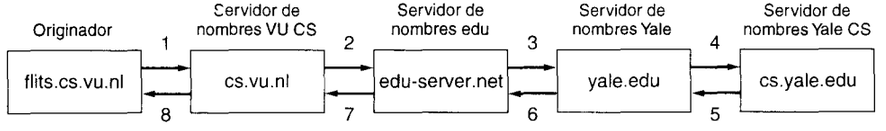


Figura 7-29. Manera en que un resolvedor busca un nombre remoto en ocho pasos.

Supongamos que el servidor de nombres local nunca ha tenido una consulta para este dominio antes, y no sabe nada sobre él; puede preguntar a algunos otros servidores de nombres cercanos, pero si ninguno de ellos sabe nada, enviará un paquete UDP al servidor de *edu* indicado en su base de datos (véase la figura 7-29), *edu-server.net*. Es improbable que este servidor sepa la dirección de *linda.cs.yale.edu*, y probablemente tampoco sabe la de *cs.yale.edu*, pero debe conocer a todos sus hijos, por lo que reenvía la solicitud al servidor de nombres de *yale.edu* (paso 3). A su vez, éste reenvía la solicitud a *cs.yale.edu* (paso 4), que debe tener los registros de recursos autorizados. Puesto que cada solicitud es de un cliente a un servidor, el registro de recursos solicitado regresa a través de los pasos 5 a 8.

Una vez que estos registros regresan al servidor de nombres *cs.vu.nl*, son ingresados en caché ahí, para el caso de que se necesiten después. Sin embargo, esta información no es autorizada, puesto que los cambios hechos en *cs.yale.edu* no se propagarán a todos los cachés del mundo que puedan saber sobre ella. Por esta razón, las entradas de caché no deben vivir demasiado tiempo. Ésta es la razón de que el campo *tiempo_de_vida* se incluya en cada registro de recurso; indica a los servidores de nombres remotos el tiempo durante el cual deben mantener en caché los registros. Si cierta máquina ha tenido la misma dirección IP durante años, puede ser

seguro poner en caché esa información durante un día. En el caso de información más volátil, podría ser más seguro purgar los registros tras unos cuantos segundos o un minuto.

Vale la pena mencionar que el método de consultas aquí descrito se conoce como **consulta recurrente**, puesto que cada servidor que no tiene toda la información solicitada sale a encontrarla en algún lado y luego la informa. Es posible un procedimiento alternativo. En él, cuando una consulta no puede satisfacerse localmente, falla la consulta, pero se devuelve el nombre del siguiente servidor a intentar a lo largo de la línea. Este procedimiento confiere al cliente mayor control sobre el proceso de búsqueda. Algunos servidores no implementan consultas recurrentes y siempre devuelven el nombre del siguiente servidor a intentar.

También vale la pena indicar que, cuando un cliente DNS no recibe una respuesta antes de terminar su temporizador, normalmente intentará otro servidor la siguiente vez. La suposición aquí es que el servidor probablemente está inactivo, y no que la solicitud o la respuesta se perdieron.

7.3. SNMP — PROTOCOLO SENCILLO DE ADMINISTRACIÓN DE REDES

En los primeros días de ARPANET, si el retardo a algún *host* se volvía inexplicablemente grande, la persona que detectaba el problema simplemente ejecutaba el programa Ping para rebotar un paquete en el destino. Observando las marcas de tiempo en la cabecera del paquete devuelto, podía ubicarse el problema y emprender alguna acción apropiada. Además, la cantidad de enrutadores era tan pequeña que era factible hacer *ping* en cada uno para ver si estaba enfermo.

Cuando ARPANET se convirtió en la Internet mundial, con múltiples columnas vertebrales (*backbones*) y operadores, esta solución dejó de ser adecuada, por lo que se requirieron mejores herramientas de administración de la red. En los RFC 1028 y 1067 se definieron dos intentos tempranos, que vivieron poco tiempo. En mayo de 1990, se publicó el RFC 1157, definiendo la versión 1 del **SNMP** (*Simple Network Management Protocol*, **protocolo sencillo de administración de redes**). Junto con un documento acompañante (el RFC 1155) sobre información de administración, el SNMP proporcionó una manera sistemática de supervisar y administrar una red de cómputo. Esta estructura y su protocolo se implementaron ampliamente en los productos comerciales y se volvieron los estándares de facto para la administración de redes.

A medida que se adquirió experiencia, se hicieron evidentes las limitaciones del SNMP, por lo que se definió (en los RFC 1441 a 1452) una versión mejorada del SNMP (SNMPv2), que se volvió un estándar Internet. En las secciones siguientes haremos un análisis breve del modelo y el protocolo SNMP (queriendo decir SNMPv2).

Aunque el SNMP se diseñó con la idea de que fuera sencillo, cuando menos un autor ha logrado producir un libro de 600 páginas sobre él (Stallings, 1993a). Para descripciones más compactas (450-550 páginas), véanse los libros de Rose (1994) y Rose y McCloghrie (1995), quienes fueron parte del grupo de diseñadores del SNMP. Otras referencias son (Feit, 1995, y Hein y Griffiths, 1995).

7.3.1. El modelo SNMP

El modelo SNMP de una red administrada consta de cuatro componentes:

1. Nodos administrados.
2. Estaciones administradas.
3. Información de administración.
4. Un protocolo de administración.

Estas partes se ilustran en la figura 7-30 y se analizan a continuación.

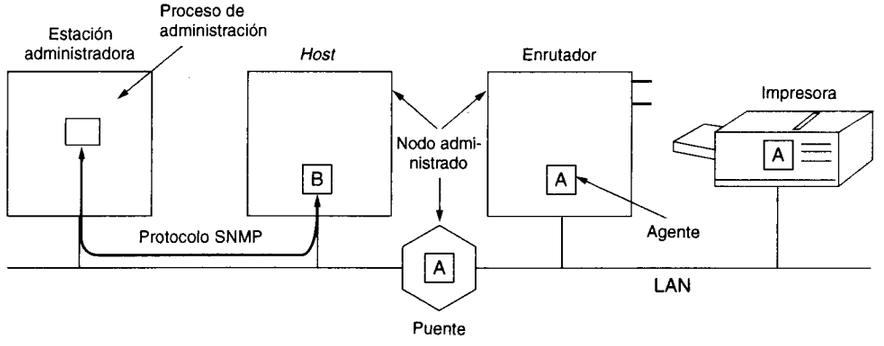


Figura 7-30. Componentes del modelo de administración SNMP.

Los nodos administrados pueden ser *hosts*, enrutadores, puentes, impresoras u otros dispositivos capaces de comunicar información de estado al mundo exterior. Para ser administrado directamente por el SNMP, un nodo debe ser capaz de ejecutar un proceso de administración SNMP, llamado **agente SNMP**. Todas las computadoras cumplen este requisito, al igual que una cantidad creciente de puentes, enrutadores y dispositivos periféricos diseñados para uso en redes. Cada agente mantiene una base de datos local de variables que describen su estado e historia y que afectan su operación.

La administración de la red se hace desde **estaciones administradoras**, que son, de hecho, computadoras de propósito general que ejecutan un *software* de administración especial. La estación administradora contiene uno o más procesos que se comunican con los agentes a través de la red, emitiendo comandos y recibiendo respuestas. En este diseño, toda la inteligencia está en las estaciones administradoras, a fin de mantener a los agentes tan sencillos como sea posible y minimizar su impacto sobre los dispositivos en los que se ejecutan. Muchas estaciones administradoras tienen una interfaz gráfica de usuario para que el administrador de la red pueda inspeccionar el estado de la red y emprenda acciones cuando se requieran.

Muchas redes reales son de varios proveedores, con *hosts* de uno o más fabricantes, puentes y enrutadores de otras compañías e impresoras de otros fabricantes. A fin de permitir que una estación administradora (potencialmente de otro proveedor más) hable con estos componentes diversos, la naturaleza de la información mantenida por todos los dispositivos debe especificarse rígidamente. Hacer que la estación administradora pregunte a un enrutador su tasa de pérdida de paquetes no es de utilidad si el enrutador no lleva el registro de su tasa de pérdidas. Por tanto, el SNMP describe (con riguroso detalle) la información exacta de cada tipo de agente que tiene que administrar y el formato con que éste tiene que proporcionarle los datos. La parte más grande del modelo SNMP es la definición de quién tiene que llevar el registro de qué y cómo se comunica esta información.

Muy brevemente, cada dispositivo mantiene una o más variables que describen su estado. En la documentación del SNMP, estas variables se llaman **objetos**, pero el término es engañoso porque no son objetos en el sentido de un sistema orientado a objetos, puesto que sólo tienen estados y no métodos (aparte de leer y escribir sus valores). Sin embargo, el término está tan difundido (por ejemplo, se usa en varias palabras reservadas del lenguaje de especificación) que lo usaremos aquí. El conjunto de todos los objetos posibles de una red se da en la estructura de datos llamada **MIB** (*Management Information Base*, **base de información de administración**).

La estación administradora interactúa con los agentes usando el protocolo SNMP. Este protocolo permite a la estación administradora consultar el estado de los objetos locales de un agente, y cambiarlo de ser necesario. La mayor parte del SNMP consiste en este tipo de comunicación consulta-respuesta.

Sin embargo, a veces ocurren sucesos no planeados. Los nodos administrados pueden caerse y volver a activarse, las líneas pueden desactivarse y levantarse de nuevo, pueden ocurrir congestionamientos, etc. Cada suceso significativo se define en un módulo MIB. Cuando un agente nota que ha ocurrido un suceso significativo, de inmediato lo informa a todas las estaciones administradoras de su lista de configuración. Este informe se llama **interrupción** SNMP (por razones históricas). El informe por lo general sencillamente indica que ha ocurrido un suceso; es responsabilidad de la estación administradora emitir consultas para averiguar los detalles. Dado que la comunicación entre los nodos administrados y la estación administradora no es confiable (es decir, no se valida), es deseable que la estación administradora de todas maneras sondee ocasionalmente cada nodo administrado, buscando sucesos inusuales, sólo por no dejar. El modelo de sondeo a intervalos grandes con aceleración al recibirse una interrupción se llama **sondeo dirigido a interrupción**.

Este modelo supone que cada nodo administrado es capaz de ejecutar un agente SNMP internamente. Los dispositivos más viejos y los dispositivos no contemplados para usarse en redes podrían no tener esta capacidad. Para manejarlos, el SNMP define un **agente apoderado**, es decir un agente que supervisa uno o más dispositivos no SNMP y se comunica con la estación administradora a nombre de ellos, y que posiblemente se comunique con los dispositivos mismos usando algún protocolo no estándar.

Por último, la seguridad y la validación de identificación desempeñan un papel preponderante en el SNMP. Una estación administradora también tiene la capacidad de aprender mucho sobre cada nodo que está bajo su control, y también puede apagarlos todos. Por tanto, es de gran

importancia que los agentes estén convencidos de que las consultas supuestamente originadas por la estación administradora realmente vienen de la estación administradora. En el SNMPv1, la estación administradora comprobaba su identidad poniendo una contraseña (de texto normal) en cada mensaje. En el SNMPv2, la seguridad se mejoró considerablemente usando técnicas criptográficas modernas del tipo que ya hemos estudiado. Sin embargo, esta adición hizo más voluminoso a un protocolo de por sí grande, y luego se desechó.

7.3.2. ASN.1 — Notación de sintaxis abstracta 1

El corazón del modelo SNMP es el grupo de objetos administrados por los agentes y leídos y escritos por la estación administradora. Para hacer posible la comunicación multiproveedor, es esencial que estos objetos se definan de una manera estándar y neutral desde el punto de vista de los proveedores. Es más, se requiere una forma estándar de codificarlos para su transferencia a través de una red. Si bien las definiciones en C satisfarían el primer requisito, tales definiciones no definen una codificación de bits en el alambre de modo tal que una estación administradora de 32 bits *little endian* de complemento a 2 pueda intercambiar información sin ambigüedades con un agente en una CPU de 16 bits *big endian* de complemento a uno.

Por esta razón, se requiere un lenguaje de definición de objetos estándar, así como reglas de codificación. El lenguaje usado por el SNMP se toma del OSI y se llama **ASN.1** (*Abstract Syntax Notation One*, **notación de sintaxis abstracta uno**). Al igual que una buena parte del OSI, es grande, complejo y no muy eficiente. (El autor está tentado a decir que, al llamarlo ASN.1 en lugar de simplemente ASN, los diseñadores implícitamente aceptaron que pronto sería reemplazado por el ASN.2, pero cortésmente se abstendrá de decirlo.) La supuesta ventaja del ASN.1 (la existencia de reglas de codificación de bits no ambiguas) ahora es en realidad una debilidad, puesto que las reglas de codificación se optimizan para minimizar la cantidad de bits en el alambre, al costo de desperdiciar tiempo de CPU en ambos extremos para su codificación y decodificación. Un esquema más sencillo, que usara enteros de 32 bits alineados según límites de 4 bytes probablemente habría sido mejor. Sin embargo, para bien o para mal, el SNMP está impregnado con ASN.1 (aunque se trata de un subgrupo simplificado), por lo que cualquiera que desee entender en verdad el SNMP debe conocer bien el ASN.1. De ahí, la siguiente explicación.

Comencemos por el lenguaje de descripción de datos, descrito en el Estándar internacional 8824. Después, estudiaremos las reglas de codificación, descritas en el Estándar internacional 8825. La sintaxis abstracta ASN.1 esencialmente es un lenguaje de declaración de datos primitivos. Permite al usuario definir objetos primitivos y luego combinarlos para formar otros más complejos. Una serie de declaraciones en ASN.1 es funcionalmente similar a las declaraciones encontradas en los archivos de cabecera asociados a muchos programas en C.

El SNMP tiene algunas convenciones lexicográficas que respetaremos. Sin embargo, éstas no son exactamente las mismas que usa ASN.1. Los tipos de datos interconstruidos se escriben en mayúsculas (por ejemplo, *INTEGER*). Los tipos definidos por el usuario comienzan con una letra mayúscula pero deben contener cuando menos un carácter distinto de una letra mayúscula. Los identificadores pueden contener letras mayúsculas y minúsculas, dígitos y guiones, pero deben comenzar con una letra minúscula (por ejemplo, *counter*). Los espacios blancos (tabuladores,

retornos de carro, etc.) no son relevantes. Por último, los comentarios comienzan con — y continúan hasta la siguiente aparición de —.

Los tipos de datos básicos del ASN.1 permitidos en el SNMP se muestran en la figura 7-31. (Generalmente ignoraremos las características del ASN.1, como los tipos *BOOLEAN* y *REAL*, que no están permitidos en el SNMP.) El uso de los códigos se describirá después.

Tipo primitivo	Significado	Código
INTEGER	Entero de longitud arbitraria	2
BIT STRING	Cadena de 0 o más bits	3
OCTET STRING	Cadena de 0 o más bytes sin signo	4
NULL	Marcador de lugar	5
OBJECT IDENTIFIER	Tipo de datos definido oficialmente	6

Figura 7-31. Tipos de datos primitivos ASN.1 permitidos en el SNMP.

Una variable de tipo *INTEGER* (entero) puede, en teoría, adoptar cualquier valor entero, pero otras reglas de SNMP limitan su alcance. Como ejemplo del uso de los tipos, considere la manera en que se declara y (opcionalmente) se establece inicialmente en 100 una variable, *count*, de tipo *INTEGER* en ASN.1:

```
count INTEGER ::= 100
```

Con frecuencia se requiere un subtipo cuyas variables se restringen a valores específicos o a un intervalo específico. Éstos se pueden declarar como sigue:

```
Status ::= INTERGER { up(1), down(2), unknown(3) }

PacketSize ::= INTERGER (0..1023)
```

Las variables del tipo *BIT STRING* (cadena de bits) y *OCTET STRING* (cadena de octetos) contienen cero o más bits y bytes, respectivamente. Un bit tiene 0 o 1. Un byte cae en el intervalo de 0 a 255, inclusive. En ambos tipos puede darse una longitud de cadena y un valor inicial.

Los *OBJECT IDENTIFIERS* (identificadores de objetos) ofrecen una manera de identificar objetos. En principio, cada objeto definido en cada estándar oficial puede identificarse de manera única. El mecanismo consiste en definir un árbol de estándares, y colocar cada objeto en cada estándar en una localidad única del árbol. La parte del árbol que incluye la MIB del SNMP se muestra en la figura 7-32.

El nivel superior del árbol lista todas las organizaciones de estándares importantes del mundo (desde el punto de vista de la ISO), es decir, el ISO y el CCITT (ahora ITU), más la combinación de los dos. Del nodo *iso* se definen cuatro arcos, uno de los cuales es para *identified-organization* (organización identificada), lo cual es una admisión por parte de la ISO de que tal vez algunas otras personas también intervienen vagamente en los estándares. El Departamento

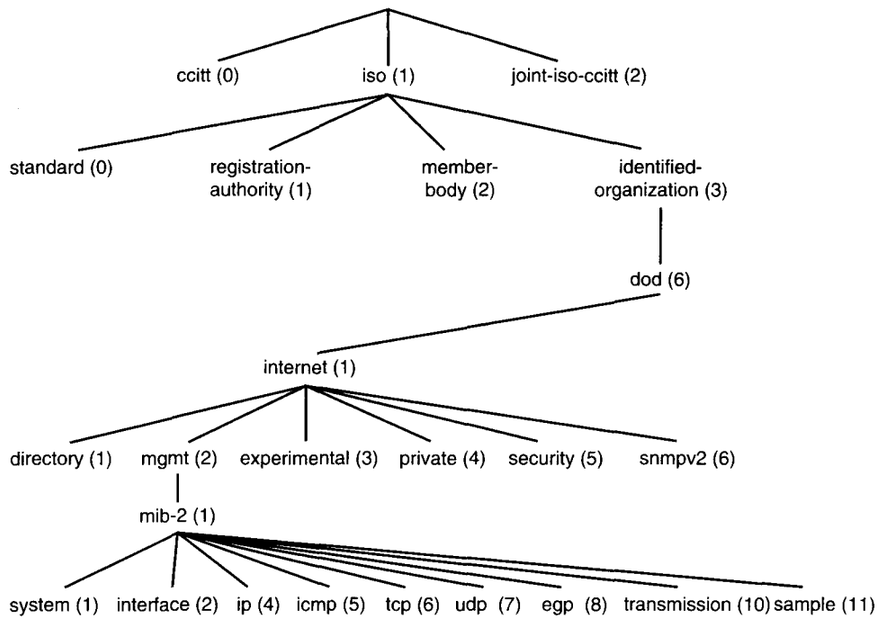


Figura 7-32. Parte del árbol de nombres de objetos del ASN.1.

de la Defensa de Estados Unidos tiene un lugar asignado en este subárbol, y ya asignó el número de Internet 1 en su jerarquía. En la jerarquía de Internet, la MIB del SNMP tiene el código 1.

Cada arco de la figura 7-32 tiene tanto una etiqueta como un número, de modo que los nodos pueden identificarse con una lista de arcos, usando etiqueta(número) o números. Por tanto, todos los objetos de la MIB de SNMP se identifican mediante una etiqueta de la forma

```
{iso identigfied-organization(3) dod(6) internet(1) mgmt(2) mib-2(1) ...}
```

o, de manera alternativa {1 3 6 1 2 1 ...}. También se permiten formas mixtas. Por ejemplo, la identificación anterior también puede escribirse como

```
{internet(1) 2 1 ...}
```

De esta manera, cada objeto de cada estándar puede representarse como un *OBJECT IDENTIFIER*.

El ASN.1 define cinco maneras de construir tipos nuevos a partir de los básicos. *SEQUENCE* es una lista ordenada de tipos, parecida a una estructura en C y a un registro en Pascal. *SEQUENCE OF* es un arreglo de una dimensión de un solo tipo. *SET* y *SET OF* son análogos, pero desordenados. *CHOICE* crea una unión a partir de una lista dada de tipos. Los dos constructores de conjuntos no se usan en ninguno de los documentos SNMP.

Otra manera de crear tipos nuevos es haciendo referencia a los viejos. La referencia de un tipo es algo parecido a la práctica de C de definir tipos nuevos, digamos *time_t* y *size_t*, ambos

de tipo *long*, pero que se usan en contextos diferentes. Las referencias vienen en cuatro categorías: universal, a nivel aplicación, específicas para el contexto, y privadas. Cada referencia consiste en una etiqueta y un entero que identifica la referencia. Por ejemplo,

```
Counter32  ::= [APLICATION 1] INTEGER (0..4294967295)

Gauge32    ::= [APLICATION 2] INTEGER (0..4294967295)
```

define dos tipos diferentes a nivel aplicación, los cuales se implementan mediante enteros sin signo de 32 bits, pero que son conceptualmente diferentes. El primero podría, por ejemplo, volver a comenzar en cero cuando llegue a su valor máximo, y el segundo podría simplemente seguir devolviendo el valor máximo hasta que se disminuya o se restablezca.

Un tipo referido puede tener la palabra clave *IMPLICIT* después del corchete que cierra cuando el tipo de lo que sigue es evidente a partir del contexto (lo que no es cierto en un *CHOICE*, por ejemplo). Esto permite una codificación de bits más eficiente, puesto que la referencia no tiene que transmitirse. En un tipo que comprende un *CHOICE* entre dos tipos diferentes, debe transmitirse una referencia para indicar al receptor el tipo que está presente.

El ASN.1 define un mecanismo de macros complejo, que se usa intensivamente en el SNMP. Puede usarse una macro como una especie de prototipo para generar un grupo de tipos y valores nuevos, cada uno con su propia sintaxis. Cada macro define algunas palabras clave (posiblemente opcionales) que se usan en la llamada para identificar a los parámetros (es decir, los parámetros de macro se identifican mediante una palabra clave, no por su posición). Los detalles del funcionamiento de las macros ASN.1 están más allá del alcance de este libro. Baste con decir que una macro se invoca dando su nombre seguido de (parte de) sus palabras clave y sus valores para esta llamada. Las macros se expanden en el momento de la compilación, no durante la ejecución. A continuación se citarán algunos ejemplos de macros.

Sintaxis de transferencia ASN.1

Una **sintaxis de transferencia** ASN.1 define la manera en que los valores de los tipos ASN.1 se convierten sin ambigüedad en una secuencia de bytes para su transmisión (y se decodifican sin ambigüedad en la otra terminal). La sintaxis de transferencia usada por el ASN.1 se llama **BER** (*Basic Encoding Rules*, **reglas básicas de codificación**). El ASN.1 tiene otras sintaxis de transferencia que el SNMP no usa. Las reglas son recurrentes, por lo que la codificación de un objeto estructurado es simplemente la concatenación de las codificaciones de los objetos componentes. De esta manera, todas las codificaciones de objetos pueden reducirse a una secuencia bien definida de objetos primitivos codificados. La codificación de estos objetos, a su vez, la definen las BER.

El principio que guía las reglas básicas de codificación es que cada valor transmitido, tanto primitivo como construido, consiste en cuatro campos:

1. Identificador (tipo o etiqueta).
2. La longitud del campo de datos, en bytes.

3. El campo de datos.
4. La bandera de fin de contenido, si se desconoce la longitud de los datos.

El último está permitido por el ASN.1, pero prohibido específicamente en el SNMP, por lo que supondremos que la longitud de datos siempre se conoce.

El primer campo identifica el elemento que le sigue, y tiene tres subcampos, como se muestra en la figura 7-33. Los dos bits de orden mayor identifican el tipo de etiqueta. El siguiente bit indica si el valor es primitivo (0) o no (1). Las banderas de etiqueta son 00, 01, 10 y 11, para *UNIVERSAL, APPLICATION*, específica para el contexto y *PRIVATE*, respectivamente. Los 5 bits restantes pueden usarse para codificar el valor de la etiqueta si ésta se encuentra en el intervalo de 0 a 30. Si la etiqueta es de 31 o más, los 5 bits de orden menor contienen 11111, con el valor verdadero en el siguiente byte o bytes.

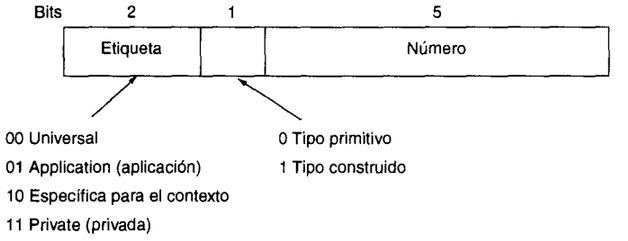


Figura 7-33. El primer byte de cada elemento de datos enviado en la sintaxis de transferencia ASN.1.

La regla empleada para codificar valores de las etiquetas mayores que 30 se ha diseñado para manejar números arbitrariamente grandes. Cada byte identificador después del primero contiene 7 bits de datos. El bit de orden mayor se establece en 0 en todos los bytes menos el último byte. Por tanto, pueden manejarse valores de etiqueta de hasta $2^7 - 1$ en 2 bytes y de hasta $2^{14} - 1$ en 3 bytes.

La codificación de los tipos *UNIVERSAL* es directa. Cada tipo primitivo tiene asignado un código, que se da en la tercera columna de la figura 7-31. *SEQUENCE* y *SEQUENCE OF* comparten el código 16. *CHOICE* no tiene un código, puesto que cualquier valor enviado siempre tiene un tipo específico. Los otros códigos son para tipos no usados en el SNMP.

A continuación del campo de identificador de 1 byte viene un campo que indica la cantidad de bytes que ocupan los datos. Las longitudes menores que 128 bytes se codifican directamente en 1 byte cuyo bit de la izquierda es 0; las mayores usan varios bytes, conteniendo el primer byte un 1 en el bit de orden mayor y el campo de longitud (de hasta 127 bytes) en los 7 bits de orden menor. Por ejemplo, si la longitud de los datos es de 1000 bytes, el primer byte contiene 130 para indicar que continúa un campo de longitud de 2 bytes. Entonces vienen 2 bytes cuyo valor es de 1000, con el byte de orden mayor primero.

La codificación del campo de datos depende del tipo de datos presente. Los enteros se codifican en complemento a dos. Un entero positivo menor que 128 requiere 1 byte, un entero positivo menor que 32,768 requiere 2 bytes, etc. El byte más significativo se transmite primero.

Las cadenas de bits se codifican como ellas mismas. El único problema es la manera de indicar la longitud. El campo de longitud indica la cantidad de *bytes* que tiene el valor, no la cantidad de *bits*. La solución escogida es transmitir un byte antes de la cadena de bits real que indique la cantidad de bits (0 a 7) del byte final que no se usa. Por tanto, la codificación de la cadena de 9 bits ´010011111´ sería 07, 4F, 80 (hexadecimal).

Las cadenas de octetos son fáciles. Los bytes de la cadena simplemente se transmiten en estilo *big endian* estándar, de izquierda a derecha.

El valor nulo se indica estableciendo la longitud del campo en 0. De hecho, no se transmite ningún valor numérico.

Un *OBJECT IDENTIFIER* se codifica como la secuencia de enteros que representa. Por ejemplo, la Internet es {1, 3, 6, 1}. Sin embargo, dado que el primer número siempre es 0, 1 o 2, y el segundo es menor que 40 (por definición: la ISO simplemente no reconocerá la 41ª categoría que aparezca en su puerta), los primeros dos números, a y b, se codifican como 1 byte que tiene el valor $40a + b$. Para la Internet, este número es 43. Como de costumbre, los números mayores que 127 se codifican en varios bytes, conteniendo el primero su bit de orden mayor establecido en 1 y una cuenta de bytes en los otros 7 bits.

Ambos tipos de secuencia se transmiten enviando primero el tipo o referencia, luego la longitud total de la codificación de todos los campos, seguidos de los campos mismos. Los campos se envían en orden.

La codificación de un valor *CHOICE* es igual a la codificación de la estructura de datos que se está transfiriendo.

En la figura 7-34 se muestra un ejemplo de la codificación de algunos valores. Los valores codificados son el *INTEGER* 49, la *OCTET STRING* ´110´, la cadena de octetos ˝xy˝, el único valor posible de *NULL*, el *OBJECT IDENTIFIER* de Internet {1, 3, 6, 1}, y un valor *Gauge32* de 14.

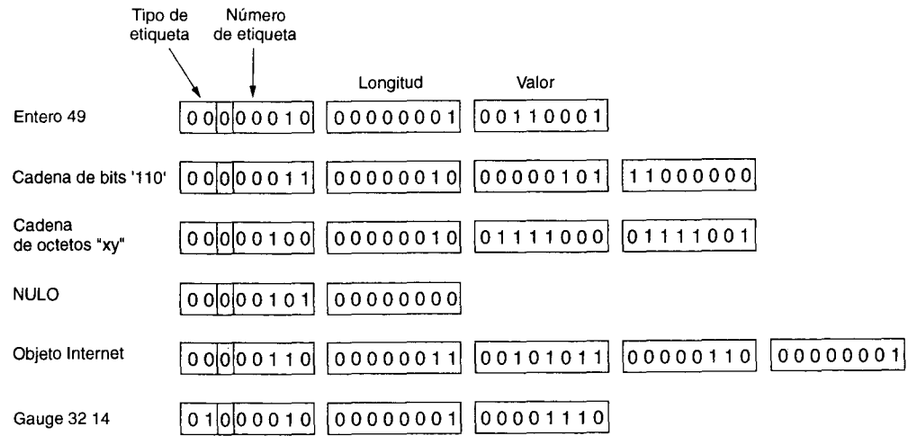


Figura 7-34. Codificación ASN.1 de algunos valores de ejemplo.

7.3.3. SMI — Estructura de la información de administración

En la sección precedente, estudiamos sólo aquellas partes del ASN.1 que se usan en el SNMP. En realidad, los documentos SNMP se organizan de manera diferente. El RFC 1442 primero dice que el ASN.1 se usará para describir las estructuras de datos SNMP, pero luego dedica 57 páginas a desechar partes del estándar ASN.1 que no quiere y agregar nuevas definiciones (en ASN.1) que se requieren. En particular, el RFC 1442 define cuatro macros clave y ocho nuevos tipos de datos que se usan intensivamente en todo el SNMP. Es este sub-super-conjunto del ASN.1, que recibe el poco elegante nombre de **SMI** (*Structure of Management Information*, **estructura de información de administración**), lo que se usa en realidad para definir las estructuras de datos del SNMP.

Aunque este enfoque es un tanto burocrático, son necesarias algunas reglas si los productos de cientos de proveedores han de hablar entre sí y realmente entender lo que dicen. Por tanto, es pertinente hablar un poco sobre el SMI.

En el nivel más bajo, las variables SNMP se definen como objetos individuales. Los objetos relacionados se reúnen en grupos, y los grupos se integran en módulos. Por ejemplo, existen grupos para los objetos IP y los objetos TCP. Un enrutador puede reconocer los grupos IP, dado que a su administrador le interesa la cantidad de paquetes perdidos. Por otra parte, un enrutador de bajo nivel podría no reconocer el grupo TCP, dado que no necesita usar TCP para desempeñar sus funciones de enrutamiento. Se pretende que los proveedores que reconocen un grupo reconozcan todos los objetos de ese grupo. Sin embargo, un proveedor que reconoce un módulo no tiene que reconocer todos sus grupos, pues podría ser que no todos se aplicaran al dispositivo.

Todos los módulos MIB comienzan con una invocación de la macro *MODULE-IDENTITY*. Sus parámetros proporcionan el nombre y la dirección del implementador, la historia de modificaciones y otra información administrativa. Típicamente, a esta llamada le sigue una invocación de la macro *OBJECT-IDENTITY*, que indica si el módulo cabe en el árbol de nombres de la figura 7-32.

Después vienen una o más invocaciones de la macro *OBJECT-TYPE*, que nombra las variables manejadas y especifica sus propiedades. El agrupamiento de variables en grupos se hace por convención; no hay enunciados *BEGIN-GROUP* ni *END-GROUP* en el ASN.1 o el SMI.

La macro *OBJECT-TYPE* tiene cuatro parámetros requeridos y cuatro (a veces) opcionales. El primer parámetro requerido es *SYNTAX* y define el tipo de datos de la variable como uno de los tipos listados en la figura 7-35. En su mayor parte, estos tipos deberían ser autoexplicativos, con los siguientes comentarios. El sufijo 32 se usa cuando el implementador en realidad quiere un número de 32 bits, incluso si las CPU de todas las máquinas a la vista son de 64 bits. Los *gauges* (medidores) difieren de los contadores en que no comienzan otra vez en cero cuando llegan a sus límites; se quedan ahí. Si un enrutador ha perdido exactamente 2^{32} paquetes, es mejor informar esto como $2^{32} - 1$ que como 0. El SMI también maneja arreglos, pero no los trataremos aquí. Para los detalles, véase (Rose, 1994).

Nombre	Tipo	Bytes	Significado
INTEGER	Numérica	4	Entero (32 bits en las implementaciones actuales)
Counter32	Numérica	4	Contador sin signo de 32 bits que da la vuelta
Gauge32	Numérica	4	Valor sin signo que no da la vuelta
Integer32	Numérica	4	Entero de 32 bits, incluso en una CPU de 64 bits
UInteger32	Numérica	4	Como Integer32, pero sin signo
Counter64	Numérica	8	Contador de 64 bits
TimeTicks	Numérica	4	Pulsos de tiempo en centésimas de segundo desde algún momento
BIT STRING	Cadena	4	Mapa de bits de 1 a 32 bits
OCTET STRING	Cadena	≥ 0	Cadena de bytes de longitud variable
Opaque	Cadena	≥ 0	Obsoleto; sólo para compatibilidad hacia atrás
OBJECT IDENTIFIER	Cadena	> 0	Lista de enteros de la figura 7-32
IpAddress	Cadena	4	Dirección Internet decimal con puntos
NsapAddress	Cadena	< 22	Dirección NSAP de OSI

Figura 7-35. Tipos de datos usados para las variables SNMP supervisadas.

Además de requerir una especificación del tipo de datos usados por la variable que se está declarando, la macro *OBJECT TYPE* requiere tres parámetros más. *MAX-ACCESS* contiene información sobre el acceso a la variable. Los valores más comunes son lectura-escritura y sólo lectura. Si una variable es lectura-escritura, la estación administradora puede establecerla. Si es sólo lectura, la estación administradora puede leerla pero no establecerla.

El *STATUS* tiene tres valores posibles. Una variable actual se ajusta a la especificación SNMP actual. Una variable obsoleta no se ajusta, pero se ajustaba a alguna versión más vieja. Una variable desaprobada está en medio; en realidad es obsoleta, pero el comité que escribió el estándar no se atrevió a decir esto en público por miedo a la reacción de los proveedores cuyos productos la usan. Sin embargo, sus días están contados.

El último parámetro requerido es *DESCRIPTION*: una cadena ASCII que indica lo que hace la variable. Si un administrador compra un bonito dispositivo nuevo, lo consulta desde la estación de administración y descubre que lleva el registro de *pktCnt*, la obtención del campo *DESCRIPTION* supuestamente dará una pista del tipo de paquetes que está contando. Este campo está dirigido exclusivamente al usuario humano (no le sirve a la computadora).

Un ejemplo sencillo de una declaración *OBJECT TYPE* se da en la figura 7-36. La variable se llama *lostPackets* y puede ser útil en un enrutador o en otro dispositivo que maneja paquetes. El valor después del signo ::= lo coloca en el árbol.


```
lostPackets OBJECT TYPE
    SYNTAX Counter32          -- usa un contador de 32 bits
    MAX-ACCESS read-only      -- la estación administradora no puede cambiarla
    STATUS current            -- esta variable (aún) no está obsoleta
    DESCRIPTION
        "La cantidad de paquetes perdidos desde la última activación"
    ::= {experimental 20}
```

Figura 7-36. Variable SNMP de ejemplo.

7.3.4. La MIB — Base de información de administración

El conjunto de objetos administrados por el SNMP se define en la MIB. Por conveniencia, estos objetos se agrupan (actualmente) en 10 categorías, que corresponden a los 10 nodos bajo *mib-2* en la figura 7-32. (Nótese que *mib-2* corresponde al SNMPv2 y que el objeto 9 ya no está presente.) La intención de las 10 categorías es proporcionar una base de lo que debe entender una estación administradora. Ciertamente se agregarán nuevas categorías y objetos en el futuro, y los proveedores están en libertad de definir objetos adicionales para sus productos. Las 10 categorías se definen en la figura 7-37.

Grupo	# objetos	Descripción
System	7	Nombre, ubicación y descripción del equipo
Interfaces	23	Interfaces de red y su tráfico medido
AT	3	Traducción de direcciones (desaprobada)
IP	42	Estadísticas de paquetes IP
ICMP	26	Estadísticas sobre los paquetes ICMP recibidos
TCP	19	Algoritmos, parámetros y estadísticas TCP
UDP	6	Estadísticas de tráfico UDP
EGP	20	Estadísticas de tráfico de protocolo de pasarela exterior
Transmission	0	Reservado para MIB de medio específico
SNMP	29	Estadísticas de tráfico SNMP

Figura 7-37. Grupos de objetos del MIB-II de Internet.

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE
DOCUMENTACIÓN Y BIBLIOTECA
MONTEVIDEO - URUGUAY

Aunque las limitaciones de espacio nos impiden entrar en los detalles de los 175 objetos definidos en la MIB-II, algunos comentarios pueden ser útiles. El grupo *system* permite al administrador encontrar el nombre del dispositivo, su constructor, el *hardware* y el *software* que contiene, su ubicación y lo que se supone que hace. También se proporcionan la hora del último

arranque y el nombre y dirección de la persona de contacto. Esta información implica que una compañía puede contratar administración de sistemas de otra compañía en una ciudad lejana y lograr que esta compañía sea capaz de determinar con facilidad la configuración actualmente administrada y la persona a contactar si hay problemas en diversos dispositivos.

El grupo *interfaces* tiene que ver con los adaptadores de red; lleva la pista de la cantidad de paquetes y bytes enviados y recibidos de la red, la cantidad de descartados, la cantidad de difusiones y el tamaño de la cola de salida.

El grupo AT estaba presente en la MIB-I y proporcionaba información sobre la correspondencia de direcciones (por ejemplo, direcciones Ethernet a IP). Esta información se movió a unas MIB específicas para protocolo en el SNMPv2.

El grupo IP se encarga del tráfico que entra y sale del nodo. Es especialmente pródigo en contadores que llevan el registro de la cantidad de paquetes descartados por diversas razones (por ejemplo, la inexistencia de una ruta conocida al destino o la falta de recursos). También están disponibles estadísticas sobre la fragmentación de datagramas y su reensamble. Todos estos elementos son particularmente importantes para la administración de las redes.

El grupo ICMP trata los mensajes de error de IP. Básicamente, este grupo tiene un contador por cada mensaje ICMP para registrar la cantidad que ha sido enviada de ese tipo.

El grupo TCP supervisa la cantidad actual y acumulada de conexiones abiertas, segmentos enviados y recibidos, así como varias estadísticas de error.

El grupo UDP lleva una bitácora de la cantidad de datagramas UDP enviados y recibidos, y la cantidad de estos últimos que no se pudieron entregar debido a un puerto desconocido o alguna otra razón.

El grupo EGP se usa para enrutadores que manejan el protocolo de pasarela exterior. El EGP lleva el registro de la cantidad de paquetes de cada tipo que salieron, entraron y se reenviaron correctamente, y que entraron y se descartaron.

El grupo de transmisión aparta un lugar para las MIB específicas para un medio. Por ejemplo, las estadísticas específicas de Internet pueden almacenarse aquí. El propósito de incluir un grupo vacío en la MIB-II es reservar el identificador {internet 2 1 9} para tales propósitos.

El último grupo es para recolectar estadísticas sobre la operación del SNMP mismo: la cantidad de mensajes enviados, el tipo de los mensajes, y demás.

La MIB-II se definió formalmente en el RFC 1213. El grueso del RFC 1213 consiste en 175 llamadas a macros semejantes a las de la figura 7-36, con comentarios que delinean los 10 grupos. Por cada uno de los 175 objetos definidos, se da el tipo de datos junto con una descripción en inglés del uso de la variable. Para mayor información sobre la MIB-II, el lector debe referirse a este RFC.

7.3.5. El protocolo SNMP

Ya hemos visto que el modelo subyacente del SNMP es una estación administradora que envía solicitudes a los agentes de nodos administrados, haciendo consultas sobre las 175 variables antes mencionadas, y sobre muchas otras variables específicas del proveedor. Nuestro último

tema es el protocolo mismo al que le hablan la estación administradora y los agentes. Este protocolo se define en el RFC 1448.

La forma normal de uso del SNMP es que la estación administradora envíe una solicitud a un agente pidiéndole información o mandándole actualizar su estado de cierta manera. Idealmente, la respuesta del agente simplemente es la información solicitada o la confirmación de que ha actualizado su estado según se solicitó. Los datos se envían empleando la sintaxis de transferencia del ASN.1. Sin embargo, también pueden informarse varios errores, como "no existe tal variable".

El SNMP define siete mensajes que pueden enviarse. Los seis mensajes de un iniciador se listan en la figura 7-38 (el séptimo mensaje es el de respuesta). Los primeros tres mensajes solicitan la devolución de valores de variables. El primer formato nombra explícitamente las variables que quiere. El segundo solicita la siguiente variable, permitiendo al administrador ir paso por paso a través de la MIB en orden alfabético (la predeterminada es la primera variable). El tercero es para transferencias grandes, como tablas.

Mensaje	Descripción
Get-request	Solicita el valor de una o más variables
Get-next-request	Solicita la variable que sigue a ésta
Get-bulk-request	Obtiene una tabla grande
Set-request	Actualiza una o más variables
Inform-request	Mensaje de administrador a administrador describiendo la MIB local
SnmpV2-trap	Informe de interrupciones de agente a administrador

Figura 7-38. Tipos de mensajes SNMP.

Después viene un mensaje que permite al administrador actualizar las variables de un agente, hasta el límite permitido por la especificación del objeto, claro está. Sigue una solicitud informal que permite a un administrador indicarle a otro las variables que está manejando. Por último, viene el mensaje enviado de una agente a un administrador cuando ha habido una interrupción.

7.4. CORREO ELECTRÓNICO

Habiendo terminado de ver algunos de los protocolos de apoyo usados en la capa de aplicación, por fin llegamos a las aplicaciones concretas. A la pregunta: "¿qué vas a hacer ahora?", poca gente contestará: "voy a buscar algunos nombres con el DNS". La gente dice que va a leer su correo electrónico o las noticias, a navegar por el Web, o a ver una película en la red. En lo que resta de este capítulo explicaremos con cierto detalle el funcionamiento de estas cuatro aplicaciones.

El **correo electrónico**, o *email*, como se le conoce en inglés, ha existido por más de dos décadas. Los primeros sistemas de correo electrónico simplemente consistían en protocolos de transferencia de archivos, con la convención de que la primera línea de cada mensaje (es decir, archivo) contenía la dirección del destinatario. A medida que pasó el tiempo, las limitaciones de este enfoque se hicieron obvias. Algunas de las quejas eran:

1. El envío de un mensaje a un grupo de gente era laborioso. Los administradores con frecuencia necesitaban esta facilidad para enviar memorandos a todos sus subordinados.
2. Los mensajes no tenían estructura interna, dificultando el proceso por computadora. Por ejemplo, si un mensaje reenviado se incluía en el cuerpo de otro mensaje, la extracción de la parte reenviada del mensaje recibido era difícil.
3. El originador (remitente) nunca sabía si el mensaje había llegado o no.
4. Si alguien planeaba ausentarse por un viaje de negocios durante varias semanas y quería que todo el correo entrante fuera manejado por su secretaria, esto no era fácil de arreglar.
5. La interfaz de usuario estaba mal integrada al sistema de transmisión, requiriendo que el usuario primero editara un archivo, luego saliera del editor e invocara el programa de transferencia.
6. No era posible crear y enviar mensajes que contuvieran una mezcla de texto, dibujos, facsímiles y voz.

A medida que se acumuló experiencia, se propusieron sistemas de correo electrónico más elaborados. En 1982, se publicaron las propuestas de correo electrónico del ARPANET como RFC 821 (protocolo de transmisión) y RFC 822 (formato de mensaje). Desde entonces se han convertido en los estándares de facto de Internet. Dos años después, el CCITT bosquejó su recomendación X.400, que luego se tomó como base para el MOTIS de OSI. En 1988, el CCITT modificó el X.400 para alinearlo con MOTIS. MOTIS debía ser la aplicación insignia del OSI, un sistema que era todo para todos.

Tras una década de competencia, los sistemas de correo electrónico basados en el RFC 822 se usan ampliamente, y los basados en el X.400 han desaparecido bajo el horizonte. La manera en que un sistema armado por un puñado de graduados en informática venció a un estándar internacional oficial con fuerte respaldo de todos los PTT del mundo, muchos gobiernos y una parte importante de la industria de la computación trae a la mente la historia bíblica de David y Goliat. La razón del éxito del RFC 822 no es que sea tan bueno, sino que el X.400 está tan mal diseñado y es tan complejo que nadie pudo implementarlo bien. Dada la opción entre un sistema de correo electrónico sencillo pero funcional basado en el RFC 822 y un sistema de correo electrónico X.400 supuestamente maravilloso pero disfuncional, muchas organizaciones escogieron lo primero. Si desea una diatriba extensa sobre los defectos del X.400, véase el apéndice C de (Rose, 1993). En consecuencia, nuestro análisis del correo electrónico se enfocará en el RFC 821 y el RFC 822 como se usa en Internet.

7.4.1. Arquitectura y servicios

En esta sección proporcionaremos una panorámica de lo que pueden hacer los sistemas de correo electrónico y la manera en que están organizados. Normalmente consisten en dos subsistemas: los **agentes de usuario**, que permiten a la gente leer y enviar correo electrónico, y los **agentes de transferencia de mensajes**, que mueven los mensajes del origen al destino. Los agentes de usuario son programas locales que proporcionan un método basado en comandos, basado en menús o de interacción gráfica con el sistema de correo electrónico. Los agentes de transferencia de mensajes son por lo común daemons del sistema que operan en segundo plano y mueven correo electrónico a través del sistema.

Por lo común, los sistemas de correo electrónico desempeñan cinco funciones básicas, como se describe a continuación. La **composición** se refiere al proceso de crear mensajes y respuestas. Aunque puede usarse cualquier editor de texto para el cuerpo del mensaje, el sistema mismo puede proporcionar asistencia con el direccionamiento y los numerosos campos de cabecera que forman parte de cada mensaje. Por ejemplo, al contestar un mensaje, el sistema de correo electrónico puede extraer la dirección del originador e insertarla automáticamente en el lugar adecuado de la respuesta.

La **transferencia** se refiere a mover mensajes del originador al destinatario. En gran medida, esto requiere establecer una conexión con el destino o alguna máquina intermedia, enviar el mensaje y liberar la conexión. El sistema de correo electrónico debe hacer esto automáticamente, sin molestar al usuario.

La **generación del informe** tiene que ver con indicar al remitente lo que ocurrió con el mensaje: ¿se entregó, se rechazó o se perdió? Existen muchas aplicaciones en las que la confirmación de la entrega es importante y puede incluso tener una implicación legal ("Bien, Su Señoría, mi sistema de correo electrónico no es muy confiable, por lo que creo que la citación electrónica se perdió en algún lado").

La **visualización** de los mensajes de entrada es necesaria para que la gente pueda leer su correo electrónico. A veces se requiere cierta conversión o debe invocarse un visor especial, por ejemplo si el mensaje es un archivo PostScript o voz digitalizada. También se intentan a veces conversiones y formateo sencillos.

La **disposición** es el paso final y se refiere a lo que el destinatario hace con el mensaje tras recibirlo. Las posibilidades incluyen tirarlo antes de leerlo, desecharlo tras leerlo, guardarlo, etc. También debe ser posible recuperar y releer mensajes guardados, reenviarlos o procesarlos de otras maneras.

Además de estos servicios básicos, la mayoría de los sistemas de correo electrónico proporcionan una gran variedad de características avanzadas. Brevemente mencionaremos algunas de éstas. Cuando la gente se muda, o cuando está lejos durante cierto tiempo, podría querer el reenvío de su correo electrónico, por lo que el sistema debe ser capaz de hacer esto de manera automática.

La mayoría de los sistemas permite a los usuarios crear **buzones de correo** (*mailboxes*) para almacenar el correo entrante. Se requieren comandos para crear y destruir buzones de correo, inspeccionar el contenido de los buzones, insertar y borrar mensajes de los buzones, etcétera.

Los gerentes corporativos con frecuencia necesitan enviar un mensaje a cada uno de sus subordinados, clientes o proveedores. Esto da pie al concepto de **lista de correo** (*mailing list*), que es una lista de direcciones de correo electrónico. Al enviarse un mensaje a la lista de correo, se entregan copias idénticas a todos los de la lista.

El correo registrado es otra idea importante, para permitir al originador saber que su mensaje ha llegado. Como alternativa, podría desearse la notificación automática de correo que no se puede entregar. De cualquier modo, el originador debe tener algún control sobre el informe de lo ocurrido.

Otras características avanzadas son copias al carbón, correo electrónico de alta prioridad, correo electrónico secreto (cifrado), destinatarios alternos si el primario no está disponible y la capacidad de que las secretarias manejen el correo de su jefe.

El correo electrónico se usa ampliamente hoy día en la industria para la comunicación intracompañía. Permite que los empleados remotos cooperen en proyectos complejos, incluso estando en otros husos horarios. Al eliminar la mayoría de las señales asociadas al rango, edad y género, los debates realizados por correo electrónico tienden a enfocarse principalmente en las ideas, no en el *status* corporativo. Con el correo electrónico, una idea brillante de un estudiante becario puede tener más impacto que una idea tonta de un vicepresidente ejecutivo. Algunas compañías han estimado que el correo electrónico ha mejorado su productividad hasta en 30% (Perry y Adam, 1992).

Una idea clave de todos los sistemas modernos de correo electrónico es la distinción entre la **envoltura** y su contenido. La envoltura encapsula el mensaje; contiene toda la información necesaria para transportar el mensaje, como dirección de destino, prioridad y nivel de seguridad, la cual es diferente del mensaje mismo. Los agentes de transporte del mensaje usan la envoltura para enrutar, al igual que la oficina postal.

El mensaje dentro de la envoltura contiene dos partes: la **cabecera** y el **cuerpo**. La cabecera contiene información de control para los agentes de usuario. El cuerpo es por completo para el destinatario humano. Las envolturas y los mensajes se ilustran en la figura 7-39.

7.4.2. El agente de usuario

Los sistemas de correo electrónico tienen dos partes básicas, como hemos visto: los agentes de usuario y los agentes de transferencia de mensajes. En esta sección veremos a los agentes de usuario. Un agente de usuario normalmente es un programa (a veces llamado lector de correo) que acepta una variedad de comandos para componer, recibir y contestar los mensajes, así como para manipular los buzones de correo. Algunos agentes de usuario tienen una interfaz elegante operada por menús o por iconos que requiere un ratón, mientras que otros esperan comandos de un carácter desde el teclado. Funcionalmente, ambos son iguales.

Envío de correo electrónico

Para enviar un mensaje de correo electrónico, el usuario debe proporcionar el mensaje, la dirección de destino y, posiblemente, algunos otros parámetros (por ejemplo, prioridad o nivel de seguridad). El mensaje puede producirse con un editor de textos independiente, un programa

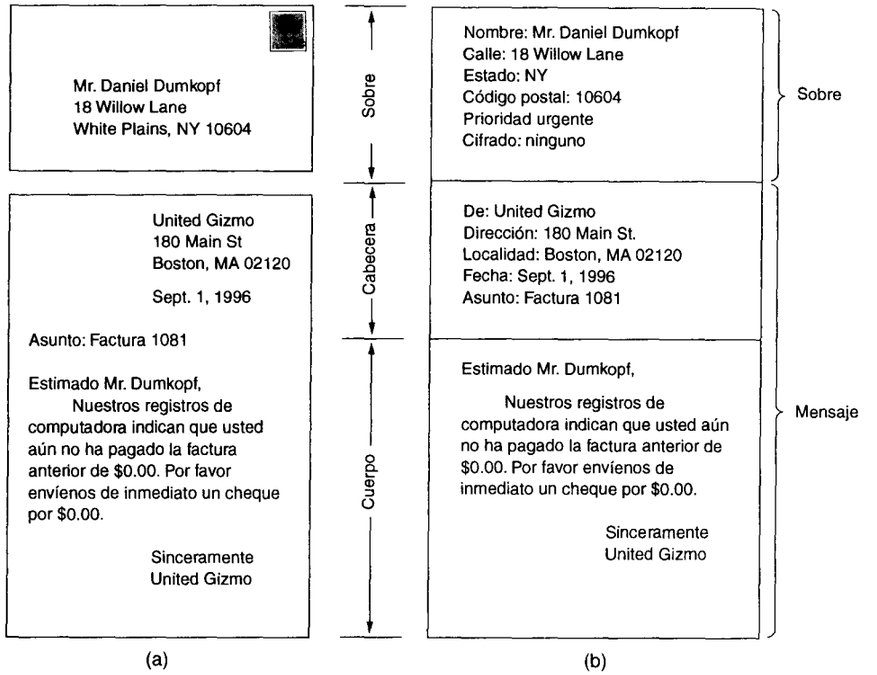


Figura 7-39. Sobres y mensajes. (a) Correo postal. (b) Correo electrónico.

de procesamiento de textos o, posiblemente, un editor de textos incorporado en el agente de usuario. La dirección de destino debe estar en un formato que el agente de usuario pueda manejar. Muchos agentes de usuario esperan direcciones DNS de la forma *buzón_de_correo@ubicación*. Dado que las hemos estudiado antes, no repetiremos ese material aquí.

Sin embargo, vale la pena indicar que existen otras formas de direccionamiento. En particular, las direcciones X.400 tienen un aspecto radicalmente diferente del de las direcciones DNS; se componen de pares *atributo* = *valor*, por ejemplo,

```
/C=US/SP=MASSACHUSETTS/L=CAMBRIDGE/PA=360 MEMORIAL DR./CN=KEN SMITH/
```

Esta dirección especifica un país, estado, localidad, dirección personal y nombre común (Ken Smith). Son posibles muchos otros atributos, de modo que puede enviarse correo electrónico a alguien cuyo nombre no se conoce, siempre y cuando se conozca un número suficiente de los otros atributos (por ejemplo, compañía y puesto). Mucha gente piensa que esta forma de nombrar es considerablemente menos funcional que los nombres DNS.

Sin embargo, con toda justicia, los diseñadores del X.400 supusieron que la gente usaría **alias** (cadenas cortas asignadas por el usuario) para identificar a los destinatarios, por lo que nunca verían siquiera las direcciones completas. Sin embargo, el *software* necesario nunca

estuvo ampliamente disponible, por lo que la gente que enviaba correo a los usuarios con direcciones X.400 con frecuencia tenía que escribir cadenas como la anterior. En contraste, la mayoría de los sistemas de correo electrónico de Internet siempre han permitido que los usuarios tengan archivos de alias.

La mayoría de los sistemas de correo electrónico manejan listas de correo, por lo que un usuario puede enviar el mismo mensaje a un grupo de personas con un solo comando. Si la lista de correo se mantiene localmente, el agente usuario puede sencillamente enviar un mensaje por separado a cada destinatario. Sin embargo, si la lista se mantiene remotamente, entonces los mensajes se multiplicarán ahí. Por ejemplo, si un grupo de observadores de pájaros tiene una lista de correo llamada *birders* instalada en *meadowlark.arizona.edu*, entonces cualquier mensaje enviado a *birders@meadowlark.arizona.edu* se enrutará a la Universidad de Arizona y se multiplicará ahí para producir mensajes individuales para todos los miembros de la lista de correo, sin importar el lugar del mundo en que estén. Los usuarios de esta lista de correo no pueden saber que es una lista de correo. Podría ser el buzón de correo personal del profesor Gabriel O. Birders.

Lectura del correo electrónico

Por lo común, al arrancar un agente de usuario, buscará en el buzón del usuario el correo electrónico recibido, antes de presentar otra cosa en la pantalla. Entonces puede anunciar la cantidad de mensajes en el buzón o presentar un resumen de una línea de cada uno y esperar un comando.

Como ejemplo del funcionamiento de un agente de usuario, veamos una situación típica de correo. Tras iniciar el agente de usuario, el usuario solicita un resumen de su correo electrónico. Aparece entonces una pantalla como la de la figura 7-40. Cada línea se refiere a un mensaje. En este ejemplo, el buzón contiene ocho mensajes.

Cada línea de la pantalla contiene varios campos extraídos del sobre o de la cabecera del mensaje correspondiente. En un sistema sencillo de correo electrónico, la selección de campos a presentar está incorporada en el programa. En uno más refinado, el usuario puede especificar los campos a presentar proporcionando un **perfil de usuario**, un archivo que describe el formato de

#	Indicadores	Bytes	Remitente	Asunto
1	K	1030	asw	Cambios a MINIX
2	KA	6348	radia	Comentarios sobre el material que me enviaste
3	K F	4519	Amy N. Wong	Solicitud de información
4		1236	bal	Fecha límite para propuesta de subvención
5		103620	kaashoek	Texto del documento DCS
6		1223	emily E.	Apuntador a página de WWW
7		3110	saniya	Informes de arbitraje sobre el documento
8		1204	dmr	Re: Visita de mis estudiantes

Figura 7-40. Ejemplo de exhibición del contenido de un buzón.

presentación. En este ejemplo, el primer campo es el número de mensaje. El segundo campo, *indicadores*, puede contener una *K*, lo que significa que el mensaje no es nuevo, sino que ha sido leído previamente y guardado en el buzón; una *A*, lo que quiere decir que el mensaje ya ha sido contestado; y/o una *F*, lo que indica que ese mensaje ha sido reenviado a alguien más. Son posibles también otros indicadores.

El tercer campo indica la longitud del mensaje y el cuarto indica quién envió el mensaje. Dado que este campo simplemente se extrae del mensaje, puede contener nombres de pila, nombres completos, iniciales, claves de acceso o cualquier otra cosa que el remitente decida poner ahí. Por último, el campo de *asunto* da un resumen breve del tema del mensaje. La gente que no incluye un campo de *asunto* con frecuencia descubre que las respuestas a su correo electrónico tienden a no recibir la prioridad más alta.

Una vez que se han visualizado las cabeceras, el usuario puede ejecutar cualquiera de los comandos disponibles. Un conjunto típico se lista en la figura 7-41. Algunos de los comandos requieren un parámetro. El signo # significa que se espera el número de un mensaje (o tal vez varios mensajes). Como alternativa, puede usarse la letra *a* para referirse a todos los mensajes.

Comando	Parámetro	Descripción
h	#	Presenta cabecera(s) en la pantalla
c		Presenta cabecera actual únicamente
t	#	Tipo de mensaje(s) en la pantalla
s	dirección	Envía mensaje
f	#	Reenvía mensaje(s)
a	#	Contesta mensaje(s)
d	#	Borra mensaje(s)
u	#	Recupera mensaje(s) previamente borrados
m	#	Mueve mensaje(s) a otro buzón
k	#	Guarda mensaje(s) tras salir
r	buzón	Lee un buzón nuevo
n		Va al siguiente mensaje y lo presenta
b		Regresa al mensaje previo y lo presenta
g	#	Va a un mensaje específico, pero no lo presenta
e		Sale del sistema de correo y actualiza el buzón

Figura 7-41. Comandos típicos de manejo de correo.

Existen incontables programas de correo electrónico. Nuestro programa de ejemplo está modelado según el usado por el sistema Mmdf de UNIX, y es bastante sencillo. El comando *h* presenta una o más cabeceras con el formato de la figura 7-40. El comando *c* imprime la cabecera del mensaje actual. El comando *t* presenta (visualiza) en la pantalla el mensaje o mensajes requeridos. Los comandos posibles son *t 3* para presentar el mensaje *3*, *t 4-6* para presentar los mensajes 4 a 6, y *t a* para presentarlos todos.

El siguiente grupo de tres comandos tiene que ver con el envío de mensajes en lugar de su recepción. El comando *s* envía un mensaje llamando a un editor adecuado (por ejemplo, el indicado en el perfil de usuario) para que el usuario pueda elaborar el mensaje. Mediante revisores ortográficos, de gramática y de dicción se puede comprobar que el mensaje sea sintácticamente correcto. Desafortunadamente, la generación actual de programas de correo electrónico no tiene revisores para ver si el remitente sabe de lo que está hablando. Cuando se ha terminado el mensaje, está listo para su transmisión al agente de transferencia de mensajes.

El comando *f* reenvía un mensaje desde el buzón, solicitando una dirección a la cual enviarlo. El comando *a* extrae la dirección de origen del mensaje a ser contestado y llama al editor para que el usuario pueda elaborar la contestación.

El siguiente grupo de comandos es para manipular los buzones de correo. Los usuarios por lo regular tienen un buzón por cada usuario con el que mantienen correspondencia, además del buzón para el correo electrónico de entrada que ya hemos visto. El comando *d* borra un mensaje del buzón, pero el comando *u* deshace el borrado. (El mensaje en realidad no se borra sino hasta que el usuario sale del programa de correo electrónico.) El comando *m* transfiere un mensaje a otro buzón. Ésta es la manera normal de guardar correo electrónico importante una vez leído. El comando *k* guarda el mensaje indicado en el buzón después de leerse. Si se lee un mensaje pero no se guarda explícitamente, se emprende alguna acción predeterminada al salir del programa de correo electrónico, como pasarlo a un buzón especial predeterminado. Por último, el comando *r* sirve para terminar con el buzón actual y pasar a otro.

Los comandos *n*, *b* y *g* son para moverse dentro del buzón actual. Es común que un usuario lea el mensaje 1, lo conteste, mueva o borre, y luego indique *n* para obtener el siguiente. La utilidad de este comando es que el usuario no tiene que llevar el registro de dónde está. Es posible ir hacia atrás usando *b* o a un mensaje específico mediante *g*.

Por último, el comando *e* termina el programa de correo electrónico y hace los cambios necesarios, como borrar algunos mensajes y marcar otros como guardados. Este comando sobreescribe el buzón, reemplazando su contenido.

En los sistemas de correo electrónico diseñados para principiantes, cada uno de estos comandos típicamente está asociado a un icono de la pantalla, por lo que el usuario no tiene que recordar que *a* significa *respuesta*. En cambio, tiene que recordar que la pequeña imagen de una persona con su boca abierta significa respuesta y no visualizar mensaje.

Debe quedar claro por este ejemplo que el correo electrónico ha recorrido un largo camino desde los días en que era simplemente una transferencia de archivos. Los agentes de usuario refinados hacen posible la administración de grandes volúmenes de correo electrónico. Para la gente como el autor, que (a su pesar) recibe y envía miles de mensajes al año, tales herramientas son invaluables.

7.4.3. Formatos de mensaje

Pasemos ahora de la interfaz de usuario al formato de los mensajes de correo electrónico mismos. Primero veremos el correo electrónico ASCII básico usando el RFC 822. Tras eso, veremos las extensiones multimedia del RFC 822.

RFC 822

Los mensajes consisten en una envoltura primitiva (descrita en el RFC 821), algunos campos de cabecera, una línea en blanco, y el cuerpo del mensaje. Cada campo de cabecera consiste (lógicamente) en una sola línea de texto ASCII que contiene el nombre del campo, dos puntos (:) y, para la mayoría de los campos, un valor. El RFC 822 es un estándar viejo y no distingue claramente la envoltura de los campos de cabecera, como lo haría un estándar nuevo. En el uso normal, el agente de usuario construye un mensaje y lo pasa al agente de transferencia de mensajes, quien entonces usa algunos campos de la cabecera para construir la envoltura, una mezcla un tanto anticuada de mensaje y envoltura.

Los principales campos de cabecera relacionados con el transporte del mensaje se listan en la figura 7-42. El campo *To:* indica la dirección DNS del destinatario primario. Está permitido tener varios destinatarios. El campo *Cc:* indica la dirección de los destinatarios secundarios. En términos de entrega, no hay diferencia entre los destinatarios primarios y los secundarios. Es una diferencia por entero sicológica que puede ser importante para los participantes, pero que no lo es para el sistema de correo. El término *Cc:* (copia al carbón) es un poco anticuado, puesto que las computadoras no usan papel carbón, pero está muy establecido. El campo *Bcc:* (copia al carbón ciega) es como el campo *Cc:*, excepto que esta línea se borra de todas las copias enviadas a los destinatarios primarios y secundarios. Esta característica permite a la gente mandar copias a terceros sin que los destinatarios primarios y secundarios lo sepan.

Cabecera	Significado
To:	Direcciones de *email* de los destinatarios primarios
Cc:	Direcciones de *email* de los destinatarios secundarios
Bcc:	Direcciones de *email* para las copias al carbón ciegas
From:	Persona o personas que crearon el mensaje
Sender:	Dirección de correo electrónico del remitente
Received:	Línea agregada por cada agente de transferencia en la ruta
Return-Path:	Puede usarse para identificar una trayectoria de regreso al remitente

Figura 7-42. Campos de cabecera RFC 822 relacionados con el transporte de mensajes.

Los siguientes dos campos, *From:* y *Sender:* indican quién escribió y envió el mensaje, respectivamente; pueden ser diferentes. Por ejemplo, un ejecutivo de negocios puede escribir un mensaje, pero su secretaria podría ser la que lo transmita. En este caso, el ejecutivo estaría listado en el campo *From:* y la secretaria en el campo *Sender:*. Se requiere el campo *From:*, pero el campo *Sender:* puede omitirse si es igual al campo *From:*. Estos campos se requieren en caso de que el mensaje no pueda entregarse y deba devolverse al remitente.

Se agrega una línea que contiene *Received:* por cada agente de transferencia de mensajes en el camino. La línea contiene la identidad del agente, la fecha y hora de recepción del mensaje, y otra información que puede servir para encontrar fallas en el sistema de enrutamiento.

El campo *Return-Path:* lo agrega el agente de transferencia de mensajes final y su intención era indicar la manera de regresar al remitente. En teoría, esta información puede tomarse de todas las cabeceras *Received:* (exceptuando el nombre del buzón del remitente), pero pocas veces se llena de esta manera y comúnmente contiene sólo la dirección del remitente.

Además de los campos de la figura 7-42, los mensajes RFC 822 también pueden contener una variedad de campos de cabecera usados por los agentes de usuario o los destinatarios. Los más comunes se listan en la figura 7-43. La mayoría de ellos son autoexplicativos, por lo que no los veremos en detalle.

Cabecera	Significado
Date:	Fecha y hora de envío del mensaje
Reply-To:	Dirección de *email* a la que deben enviarse las contestaciones
Message-Id:	Número único para referencia posterior a este mensaje
In-Reply-To:	Identificador del mensaje al que éste responde
References:	Otros identificadores de mensaje pertinentes
Keywords:	Claves seleccionadas por el usuario
Subject:	Resumen corto del mensaje para exhibir en una línea

Figura 7-43. Algunos campos usados en la cabecera de mensaje RFC 822.

El campo *Reply-To:* a veces se usa cuando ni la persona que redactó el mensaje ni la que lo envió quieren ver la respuesta. Por ejemplo, un administrador de mercadeo escribe un mensaje de correo electrónico para informar a los clientes sobre un producto nuevo. El mensaje lo envía una secretaria, pero el campo *Reply-To:* indica el jefe del departamento de ventas, quien puede contestar preguntas y surtir pedidos.

El documento RFC 822 explícitamente indica que los usuarios pueden inventar cabeceras nuevas para su uso privado, siempre y cuando comiencen con la cadena *X-*. Se garantiza que no habrá cabeceras futuras que usen nombres que comiencen con *X-*, a fin de evitar conflictos entre las cabeceras oficiales y las privadas. A veces, algunos estudiantes universitarios, pasándose de listos, incluyen campos como *X-Fruta-del-Día:* o *X-Enfermedad-de-la-Semana:*, que son legales, aunque no muy ilustrativos.

Tras las cabeceras viene el cuerpo del mensaje. Los usuarios pueden poner aquí lo que les venga en gana. Algunos terminan sus mensajes con firmas complicadas, incluidas caricaturas sencillas en ASCII, citas de autoridades mayores y menores, pronunciamientos políticos y renuncias de responsabilidades de todo tipo (por ejemplo, la corporación ABC no es responsable de mis opiniones; ni siquiera las puede entender).

MIME — Extensiones multipropósito de correo Internet

En los primeros días de ARPANET, el correo electrónico consistía exclusivamente en mensajes de texto escritos en inglés y expresados en ASCII. En tal entorno, el RFC 822 hacía todo el trabajo: especificaba las cabeceras, pero dejaba el contenido en manos del usuario. Hoy día, en la red mundial de Internet, este enfoque ya no es adecuado. Los problemas incluyen envío y recepción de:

1. Mensajes en idiomas con acentos (por ejemplo, español, francés y alemán).
2. Mensajes en alfabetos no latinos (por ejemplo, hebreo y ruso).
3. Mensajes en idiomas sin alfabetos (por ejemplo, chino y japonés).
4. Mensajes que no contienen texto (por ejemplo, audio y vídeo).

Se propuso una solución en el RFC 1341 y se actualizó en el RFC 1521. Esta solución, llamada **MIME** (*Multipurpose Internet Mail Extensions*, **extensiones multipropósito de correo Internet**), se usa ampliamente. Ahora la describiremos. Para información adicional sobre MIME, véase el RFC 1521 o (Rose, 1993).

La idea básica de MIME es continuar usando el formato RFC 822, pero agregar una estructura al cuerpo del mensaje y definir reglas de codificación para los mensajes no ASCII. Al no desviarse del 822, los mensajes MIME pueden enviarse usando los programas y protocolos de correo electrónico existentes. Todo lo que tiene que cambiarse son los programas transmisores y receptores, lo que pueden hacer los usuarios mismos.

El MIME define cinco nuevas cabeceras de mensaje, como se muestra en la figura 7-44. La primera de éstas simplemente indica al agente de usuario receptor del mensaje que está tratando con un mensaje MIME, así como la versión del MIME que usa. Se considera que cualquier mensaje que no contenga una cabecera *MIME-Version:* es un mensaje de texto normal en inglés, y se procesa como tal.

La cabecera *Content-Description:* es una cadena ASCII que dice lo que está en el mensaje. Esta cabecera es necesaria para que el destinatario sepa si vale la pena descodificar y leer el mensaje. Si la cadena dice: "Foto del jerbo de Bárbara" y la persona que recibe el mensaje no es un gran fanático de los jerbos, el mensaje probablemente será descartado en lugar de descodificado para dar una foto a color de alta definición.

Cabecera	Significado
MIME-Version:	Identifica la versión de MIME
Content-Description:	Cadena de texto que describe el contenido
Content-Id:	Identificador único
Content-Transfer-Encoding:	Cómo se envuelve el mensaje para su transmisión
Content-Type:	Naturaleza del mensaje

Figura 7-44. Cabeceras RFC 822 agregadas por MIME.

La cabecera *Content-Id:* identifica el contenido; usa el mismo formato que la cabecera estándar *Message-Id:*.

Content-Transfer-Encoding: indica la manera en que está envuelto el cuerpo para su transmisión a través de una red donde se podría tener problemas con la mayoría de los caracteres distintos de las letras, números y signos de puntuación. Se proporcionan cinco esquemas (más un escape hacia nuevos esquemas). El esquema más sencillo es simplemente texto ASCII. Los caracteres ASCII usan 7 bits y pueden transportarse directamente mediante el protocolo de correo electrónico siempre y cuando ninguna línea exceda 1000 caracteres.

El siguiente esquema más sencillo es lo mismo, pero usando caracteres de 8 bits, es decir, todos los valores de 0 a 255. Este esquema de codificación viola el protocolo (original) del correo electrónico de Internet, pero es usado por algunas partes de Internet que implementan ciertas extensiones al protocolo original. Mientras que la declaración de la codificación no la hace legal, hacerla explícita puede cuando menos explicar las cosas cuando algo sucede mal. Los mensajes que usan codificación de 8 bits deben aún adherirse a la longitud máxima de línea estándar.

Peor aún son los mensajes que usan codificación binaria. Éstos son archivos binarios arbitrarios que no sólo utilizan los 8 bits, sino que ni siquiera respetan el límite de 1000 caracteres por línea. Los programas ejecutables caen en esta categoría. No se da ninguna garantía de que los mensajes en binario llegarán correctamente, pero mucha gente los envía de todos modos.

La manera correcta de codificar mensajes binarios es usar **codificación base64**, a veces llamada **armadura ASCII**. En este esquema, se dividen grupos de 24 bits en unidades de 6 bits, enviándose cada unidad como carácter ASCII legal. La codificación es "A" para 0, "B" para 1, etc., seguidas por las 26 letras minúsculas, los 10 dígitos y, por último, + y / para el 62 y 63, respectivamente. Las secuencias == y = se usan para indicar que el último grupo contenía sólo 8 o 16 bits, respectivamente. Los retornos de carro y avances de línea se ignoran, por lo que pueden introducirse a voluntad para mantener la línea lo suficientemente corta. Puede enviarse un texto binario arbitrario usando este esquema.

En el caso de mensajes que son casi completamente ASCII, pero con algunos caracteres no ASCII, la codificación base 64 es algo ineficiente. En cambio, se usa una codificación conocida como **codificación entrecomillada-imprimible**. Simplemente es ASCII de 7 bits, con todos los caracteres por encima de 127 codificados como un signo de igual seguido del valor del carácter en dos dígitos hexadecimales.

En resumen, los datos binarios deben enviarse codificados en base 64 o en forma entrecomillada-imprimible. Cuando hay razones válidas para no usar uno de estos esquemas, es posible especificar una codificación definida por el usuario en la cabecera *Content-Transfer-Encoding:*.

La última cabecera mostrada en la figura 7-44 en realidad es la más interesante. Especifica la naturaleza del cuerpo del mensaje. Hay siete tipos definidos en el RFC 1521, cada uno de los cuales tiene uno o más subtipos. El tipo y el subtipo se separan mediante una diagonal, como en

```
Content-Type: video/mpeg
```

El subtipo debe indicarse explícitamente en la cabecera; no se proporcionan valores predeterminados. La lista inicial de tipos y subtipos especificada en el RFC 1521 se da en la figura 7-45. Se

han agregado muchos otros desde entonces, y se agregan entradas nuevas todo el tiempo, a medida que surge la necesidad.

Tipo	Subtipo	Descripción
Text	Pain	Texto sin formato
	Richtext	Texto con comandos de formato sencillos
Image	Gif	Imagen fija en formato GIF
	Jpeg	Imagen fija en formato JPEG
Audio	Basic	Sonido
Video	Mpeg	Película en formato MPEG
Application	Octet-stream	Secuencia de bytes no interpretada
	Postscript	Documento imprimible en PostScript
Message	Rfc822	Mensaje MIME RFC 822
	Partial	Mensaje dividido para su transmisión
	External-body	El mensaje mismo debe obtenerse de la red
Multipart	Mixed	Partes independientes en el orden especificado
	Alternative	Mismo mensaje en diferentes formatos
	Parallel	Las partes deben verse simultáneamente
	Digest	Cada parte es un mensaje RFC 822 completo

Figura 7-45. Tipos y subtipos MIME definidos en el RFC 1521.

Repasemos ahora la lista de tipos. El tipo *text* es para texto normal. La combinación *text/plain* es para mensajes ordinarios que pueden visualizarse como se reciben, sin codificación ni ningún procesamiento posterior. Esta opción permite el transporte de mensajes ordinarios en MIME con sólo unas pocas cabeceras extra.

El subtipo *text/richtext* permite la inclusión de un lenguaje de marcación sencillo en el texto. Este lenguaje proporciona una manera independiente del sistema para indicar negritas, cursivas, tamaños en puntos menores y mayores, sangrías, justificaciones, sub y superíndices, y formatos sencillos de página. El lenguaje de marcación se basa en el SGML, el *Standard Generalized Markup Language*, lenguaje generalizado estándar de marcación, también usado como la base del HTML del *World Wide Web*. Por ejemplo, el mensaje

Ha llegado el <bold> momento </bold>, dijo la <italic> morsa </italic>...

se presentaría como

Ha llegado el **momento**, dijo la *morsa*...

Es responsabilidad del sistema receptor seleccionar la presentación adecuada. Si hay negritas y cursivas disponibles, pueden usarse; de otra manera, pueden usarse colores, parpadeos, vídeo inverso, etc., como énfasis. Los diferentes sistemas pueden, y de hecho lo hacen, hacer selecciones distintas.

El siguiente tipo MIME es *image*, que se usa para transmitir imágenes fijas. Hoy día se usan muchos formatos para almacenar y transmitir imágenes, tanto con compresión como sin ella. Dos de estos, GIF y JPEG son subtipos oficiales, pero sin duda se agregarán otros después.

Los tipos *audio* y *vídeo* son para sonido e imágenes en movimiento, respectivamente. Nótese que *vídeo* incluye sólo la información visual, no la pista de sonido. Si se debe transmitir una película con sonido, tal vez sea necesario transmitir las partes de vídeo y de audio por separado, dependiendo del sistema de codificación usado. El único formato de vídeo definido hasta ahora es el diseñado por el grupo de modesto nombre *Moving Picture Experts Group* (grupo de expertos en imágenes en movimiento), el MPEG.

El tipo *application* es un tipo general para los formatos que requieren procesamiento externo no cubierto por ninguno de los otros tipos. Un *octet-stream* simplemente es una secuencia de bytes no interpretados. A la recepción de una de tales cadenas, un agente de usuario probablemente debería presentarla en pantalla sugiriendo al usuario que se copie en un archivo y solicitando un nombre de archivo. El procesamiento posterior es responsabilidad del usuario.

El otro subtipo definido es *postscript*, que se refiere al lenguaje PostScript producido por Adobe Systems y ampliamente usado para describir páginas impresas. Muchas impresoras tienen intérpretes PostScript integrados. Aunque un agente de usuario puede simplemente llamar a un intérprete PostScript externo para visualizar los archivos PostScript entrantes, hacerlo no está exento de riesgos. PostScript es un lenguaje de programación completo. Con el tiempo necesario, una persona lo bastante masoquista podría escribir un compilador de C o un sistema de manejo de base de datos en PostScript. La visualización de un mensaje PostScript entrante puede hacerse ejecutando el programa PostScript contenido en él. Además de exhibir algo de texto, este programa puede leer, modificar o borrar los archivos del usuario, y tener otros efectos secundarios desagradables.

El tipo *message* permite que un mensaje esté encapsulado por completo dentro de otro. Este esquema es útil para reenviar, por ejemplo, correo electrónico. Cuando se encapsula un mensaje RFC 822 completo en un mensaje exterior, debe usarse el subtipo *rfc822*.

El subtipo *partial* hace posible dividir un mensaje encapsulado en pedazos y enviarlos por separado (por ejemplo, si el mensaje encapsulado es demasiado grande). Los parámetros hacen posible reensamblar correctamente todas las partes en el destino en el orden correcto.

Por último, el subtipo *external-body* puede usarse para mensajes muy grandes (por ejemplo, películas en vídeo). En lugar de incluir el archivo MPEG en el mensaje, se da una dirección FTP y el agente de usuario del receptor puede obtenerlo a través de la red al momento que se requiera. Esta facilidad es especialmente útil al enviar una película a una lista de correo, cuando sólo se espera que unos cuantos la vean (imagine correo electrónico chatarra que contenga vídeos de propaganda).

El último tipo es *multipart*, que permite que un mensaje contenga más de una parte, con el comienzo y el fin de cada parte claramente delimitados. El subtipo *mixed* permite que cada parte sea diferente, sin ninguna estructura adicional impuesta. En contraste, con el subtipo *alternative*, cada parte debe contener el mismo mensaje, pero expresado en un medio o codificación diferente. Por ejemplo, un mensaje puede enviarse como ASCII común, como texto enriquecido (*richtext*) y como PostScript. Un agente de usuario adecuadamente diseñado que reciba uno de tales mensajes lo presentará en PostScript de ser posible. La segunda posibilidad sería como texto enriquecido. Si ninguna de las dos fuera posible, se presentaría el texto ASCII normal. Las partes deben ordenarse de la más sencilla a la más compleja para ayudar a que los receptores con agentes de usuario pre-MIME puedan encontrarle algún sentido al mensaje (por ejemplo, aun un usuario pre-MIME puede leer texto ASCII común).

El subtipo *alternative* puede servir también para varios lenguajes. En este contexto, puede pensarse en la piedra Rosetta como uno de los primeros mensajes *multipart/alternative*.

Un ejemplo multimedia se muestra en la figura 7-46. Aquí se transmiten una felicitación de cumpleaños como texto y como canción. Si el receptor tiene capacidad de audio, el agente de usuario traerá el archivo de sonido, *birthday.snd*, y lo ejecutará. Si no, se presenta la letra en la pantalla en absoluto silencio. Las partes están delimitadas por dos guiones seguidos de la cadena (definida por el usuario) especificada en el parámetro *boundary*.

Nótese que la cabecera *Content-Type* aparece en tres lugares en este ejemplo. En el nivel superior, la cabecera indica que el mensaje tiene varias partes. En cada parte, la cabecera indica el tipo y subtipo de la parte. Por último, en el texto de la segunda parte, la cabecera es necesaria para indicarle al agente de usuario la clase de archivo externo que debe conseguir. Para indicar esta pequeña diferencia de uso, hemos usado letras en minúscula, aunque las cabeceras no hacen distinciones entre mayúsculas y minúsculas. De la misma manera, se requiere la cabecera *content-transfer-encoding* para cualquier cuerpo externo que no esté codificado como ASCII de 7 bits.

Regresando a los subtipos para mensajes multiparte, existen dos posibilidades más. El subtipo *parallel* se usa cuando todas las partes deben "verse" simultáneamente. Por ejemplo, las películas con frecuencia tienen un canal de audio y un canal de vídeo. Las películas son más efectivas si estos dos canales se reproducen en paralelo, en lugar de consecutivamente.

Por último, el subtipo *digest* se usa cuando se empacan muchos mensajes juntos en un mensaje compuesto. Por ejemplo, algunos grupos de discusión de Internet recolectan mensajes de los subscriptores y luego los envían como un solo mensaje *multipart/digest*.

7.4.4. Transferencia de mensajes

El sistema de transferencia de mensajes se ocupa de transmitir los mensajes del remitente al destinatario. La manera más sencilla de hacer esto es establecer una conexión de transporte de la máquina de origen a la de destino y sencillamente transferir el mensaje. Tras examinar la manera en que se hace normalmente esto, estudiaremos algunas situaciones en las que no funciona esto, y lo que puede hacerse al respecto.

```
From: elinor@abc.com
To: carolyn@xyz.com
MIME-Version: 1.0
Message-Id: <0704760941.AA00747@abc.com>
Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
Subject: La Tierra orbita al Sol un número entero de veces

Éste es el preámbulo. El agente de usuario lo ignora. Tenga un bonito día.

--qwertyuiopasdfghjklzxcvbnm
Content-Type: text/richtext

Feliz cumpleaños a ti
Feliz cumpleaños a ti
Feliz cumpleaños, querida <bold> Carolyn </bold>
Feliz cumpleaños a ti

--qwertyuiopasdfghjklzxcvbnm
Content-Type: message/external-body;
     access-type="anon-ftp",
     site="bicycle.abc.com";
     directory="pub";
     name="birthday.snd"

content-type: audio/basic
content-transfer-encoding: base64
--qwertyuiopasdfghjklzxcvbnm--
```

Figura 7-46. Mensaje multiparte que contiene alternativas de texto enriquecido y audio.

SMTP — Protocolo sencillo de transferencia de correo

En el Internet, el correo electrónico se entrega al hacer que la máquina de origen establezca una conexión TCP con el puerto 25 de la máquina de destino. Escuchando en este puerto está un *daemon* de correo electrónico que habla con el **SMTP** (*Simple Mail Transfer Protocol*, **protocolo sencillo de transferencia de correo**). Este *daemon* acepta conexiones de entrada y copia mensajes de ellas a los buzones adecuados. Si no puede entregarse un mensaje, se devuelve al transmisor un informe de error que contiene la primera parte del mensaje que no pudo entregarse.

El SMTP es un protocolo ASCII sencillo. Tras establecer la conexión TCP con el puerto 25, la máquina transmisora, operando como cliente, espera que la máquina receptora, operando como servidor, hable primero. El servidor comienza por enviar una línea de texto que proporciona su identidad e indica si está preparado o no para recibir correo. Si no lo está, el cliente libera la conexión y lo intenta después.

Si el servidor está dispuesto a aceptar correo electrónico, el cliente anuncia de quién viene el mensaje, y a quién está dirigido. Si existe tal destinatario en el destino, el servidor da al cliente permiso para enviar el mensaje. Entonces el cliente envía el mensaje y el servidor acusa su recibo. Por lo general no se requieren sumas de comprobación porque el TCP proporciona una corriente de bits confiable. Si hay más correo electrónico, se envía ahora. Una vez que todo el correo electrónico ha sido intercambiado en ambas direcciones, se libera la conexión. Un ejemplo de diálogo para el envío del mensaje de la figura 7-46, incluidos los códigos numéricos usados por el SMTP, se muestra en la figura 7-47. Las líneas enviadas por el cliente se marcan con *C:*; aquellas enviadas por el servidor se marcan con *S:*.

Pueden ser útiles algunos comentarios sobre la figura 7-47. El primer comando del cliente ciertamente es *HELO*. De las dos abreviaturas de cuatro caracteres de *HELLO*, ésta tiene numerosas ventajas sobre su competidora.* La razón por la que los comandos tienen que ser de cuatro caracteres se ha perdido en las brumas del tiempo.

En la figura 7-47, el mensaje se envía a un solo receptor, por lo que se usa un solo comando *RCPT*. Se permiten muchos de tales comandos para enviar un solo mensaje a receptores múltiples; cada uno se acusa o rechaza individualmente. Incluso si se rechazan algunos destinatarios (porque no existen en el destino), el mensaje puede enviarse a los demás.

Por último, aunque la sintaxis de los comandos de cuatro caracteres del cliente se especifica con rigidez, la sintaxis de las respuestas es menos rígida. Sólo cuenta el código numérico. Cada implementación puede poner la cadena que desee después del código.

Aunque el protocolo SMTP está bien definido (por el RFC 821), pueden surgir algunos problemas. Uno se relaciona con la longitud del mensaje. Algunas implementaciones más viejas no pueden manejar mensajes mayores que 64 kB. Otro problema se relaciona con las terminaciones de temporización. Si el cliente y el servidor tienen temporizaciones diferentes, uno de ellos puede terminar mientras que el otro continúa trabajando, terminando inesperadamente la conexión. Por último, en contadas ocasiones, pueden dispararse tormentas de correo infinitas. Por ejemplo, si el *host* 1 contiene la lista de correo *A* y el *host* dos contiene la lista de correo *B* y cada lista contiene una entrada para la otra lista, entonces un mensaje enviado a cualquiera de las listas generará una cantidad sin fin de tráfico de correo electrónico.

Para superar algunos de estos problemas, se ha definido el SMTP extendido (**ESMTP**) en el RFC 1425. Los clientes que deseen usarlo deben enviar inicialmente un mensaje *EHLO*, en lugar de *HELO*. Si el saludo se rechaza, esto indica que el servidor es un servidor SMTP normal, y el cliente debe proceder de la manera normal. Si se acepta el *EHLO*, entonces se permiten los comandos y parámetros nuevos. La estandarización de estos comandos y parámetros es un proceso en curso.

Pasarelas de correo electrónico

El correo electrónico usando SMTP funciona mejor cuando tanto el transmisor como el receptor están en Internet y pueden manejar conexiones TCP entre el transmisor y el receptor. Sin

* En inglés, *hell* significa "infierno". (N. del supervisor.)

```
                    S: 220 servicio SMTP xyz.com listo
C: HELO abc.com
                    S: 250 xyz.com dice hola a abc.com
C: MAIL FROM: <elinor@abc.com>
                    S: 250 transmisor ok
C: RCPT TO: <carolyn@xyz.com>
                    S: 250 receptor ok
C: DATA
                    S: 354 envía correo; termina con una línea únicamente con "."
C: From: elinor@abc.com
C: To: carolyn@xyz.com
C: MIME-Version: 1.0
C: Message-Id: <0704760941.AA00747@abc.com>
C: Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
C: Subject: La Tierra orbita al Sol un número entero de veces
C:
C: Éste es el preámbulo. El agente de usuario lo ignora. Tenga un bonito día.
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: text/richtext
C:
C: Feliz cumpleaños a ti
C: Feliz cumpleaños a ti
C: Feliz cumpleaños, querida <bold> Carolyn </bold>
C: Feliz cumpleaños a ti
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: message/external-body;
C:      access-type="anon-ftp";
C:      site="bicycle.abc.com";
C:      directory="pub";
C:      name="birthday.snd"
C:
C: content-type: audio/basic
C: content-transfer-encoding: base64
C: --qwertyuiopasdfghjklzxcvbnm
C: .
                    S: 250 mensaje aceptado
C: QUIT
                    S: 221 xyz.com cerrando conexión
```

Figura 7-47. Transferencia de un mensaje de *elinor@abc.com* a *carolyn@xyz.com*.

embargo, muchas máquinas que no están en Internet también quieren transmitir y recibir correo electrónico de instalaciones de Internet. Por ejemplo, muchas compañías intencionalmente no quieren estar en Internet por razones de seguridad. Algunas de ellas incluso se aíslan de Internet levantando muros de seguridad entre ellas mismas y la Internet.

Ocurre otro problema cuando el transmisor habla sólo RFC 822 y el otro habla sólo un protocolo de correo X.400 o alguno patentado, específico del proveedor. Dado que estos dos mundos difieren en cuanto al formato de mensaje y los protocolos, es imposible la comunicación directa.

Ambos problemas se resuelven usando **pasarelas de correo electrónico**. En la figura 7-48, el *host* 1 sólo habla TCP/IP y RFC 822, mientras que el *host* 2 habla sólo TP4 y X.400 de OSI. No obstante, pueden intercambiar correo usando una pasarela de correo electrónico. El procedimiento es que el *host* 1 establece una conexión TCP con la pasarela y luego usa SMTP para transferir un mensaje (1) ahí. El *daemon* de la pasarela pone el mensaje en el *buffer* de mensajes destinados al *host* 2. Después, se establece una conexión TP4 (el equivalente OSI del TCP) con el *host* 2 y el mensaje (2) se transfiere usando el equivalente OSI del SMTP. Todo lo que tiene que hacer el proceso de la pasarela es extraer los mensajes entrantes de una cola y depositarlos en otra.

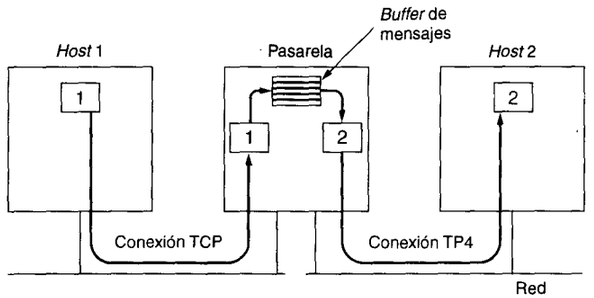


Figura 7-48. Transferencia de correo electrónico usando una pasarela de correo electrónico de capa de aplicación.

Parece fácil, pero no lo es. El primer problema es que las direcciones Internet y las direcciones X.400 son diferentes por completo. Se requiere un complejo mecanismo de tranformación entre ellas. El segundo problema es que los campos de envoltura y de cabecera presentes en un sistema podrían no estar presentes en el otro. Por ejemplo, si un sistema requiere clases de prioridad y el otro no tiene este concepto, en una dirección debe desecharse información valiosa y en la otra debe generarse de la nada.

Un problema más grave aún es qué debe hacerse si son incompatibles partes del cuerpo. ¿Qué debe hacer una pasarela con un mensaje de Internet cuyo cuerpo contiene una referencia a un archivo de audio que debe ser obtenido mediante FTP si el sistema de destino no maneja este concepto? ¿Qué debe hacer la pasarela cuando un sistema X.400 le dice que entregue un mensaje a cierta dirección pero, si falla, envíe el contenido por fax? El uso del fax no es parte del modelo RFC 822. Queda claro que aquí no hay soluciones sencillas. Para los mensajes de texto sencillos sin estructura en ASCII, las pasarelas son una solución razonable, pero para cualquier cosa más complicada la idea tiende a desmoronarse.

Entrega final

Hasta ahora, hemos supuesto que todos los usuarios trabajan en máquinas capaces de enviar y recibir correo electrónico. Con frecuencia esta situación es falsa. Por ejemplo, en muchas compañías los usuarios trabajan en PC de escritorio que no están en Internet y que son incapaces de enviar o recibir correo electrónico fuera de la compañía. En cambio, la compañía tiene uno o más servidores de correo electrónico que pueden enviar y recibir correo electrónico. Para enviar o recibir mensajes, una PC debe hablar con un servidor de correo electrónico usando alguna clase de protocolo de entrega.

Un protocolo sencillo usado para traer correo electrónico de un buzón remoto es el **POP3** (*Post Office Protocol*, **protocolo de oficina postal**), que se define en el RFC 1225. El POP3 tiene comandos para que un usuario establezca una sesión, la termine, obtenga mensajes y los borre. El protocolo mismo consiste en texto ASCII y se asemeja un poco al SMTP. El objetivo del POP3 es obtener correo electrónico del buzón remoto y almacenarlo en la máquina local del usuario para su lectura posterior.

Un protocolo más refinado es el **IMAP** (*Interactive Mail Access Protocol*, **protocolo interactivo de acceso a correo**), que se define en el RFC 1064. IMAP se diseñó para ayudar al usuario que tiene varias computadoras, tal vez una estación de trabajo en la oficina, una PC en casa y una computadora portátil para el camino. La idea en que se basa el IMAP es que el servidor de correo electrónico mantenga un depósito central al que pueda accederse desde cualquier máquina. Por tanto, a diferencia del POP3, el IMAP no copia el correo electrónico en la máquina personal del usuario porque el usuario puede tener varias.

El IMAP tiene muchas características, como la capacidad de direccionar el correo no por número de arribo, como se hace en la figura 7-40, sino usando atributos (por ejemplo, dame el primer mensaje de Sam). En este enfoque, un buzón se parece más a un sistema de base de datos relacional que a una secuencia lineal de mensajes.

Un tercer protocolo de entrega es el **DMSP** (*Distributed Mail System Protocol*, **protocolo de sistema de correo distribuido**), que es parte del sistema PCMAIL y se describe en el RFC 1056. Éste no supone que todo el correo electrónico está en un servidor, como el POP3 y el IMAP. En cambio, permite a los usuarios descargar correo del servidor a una estación de trabajo, PC o portátil y luego desconectarse. El correo electrónico puede leerse y contestarse estando desconectado. Al ocurrir una reconexión después, el correo electrónico se transferirá y el sistema se resincronizará.

Independientemente de si el correo electrónico se entrega directamente a la estación de trabajo del usuario o a un servidor remoto, muchos sistemas proporcionan ganchos para el procesamiento adicional del correo electrónico entrante. Una herramienta especialmente valiosa para muchos usuarios de correo electrónico es la capacidad de establecer **filtros**. Éstos son reglas que se consultan cuando llega el correo electrónico o cuando se inicia el agente de usuario. Cada regla especifica una condición y una acción. Por ejemplo, una regla podría decir que cualquier mensaje de Andrew S. Tanenbaum debe presentarse en una fuente negrita roja parpadeante de 24 puntos (o, de manera alternativa, descartarse automáticamente sin comentarios).

Otra característica de entrega con que a menudo se cuenta es la capacidad para reenviar (temporalmente) el correo electrónico entrante a una dirección diferente. Esta dirección incluso puede ser una computadora operada por un sistema localizador comercial, que entonces llama al usuario por radio o satélite, presentando la línea *Subject:* en su *beeper*.

Otra característica común de la entrega final es la capacidad de instalar un ***daemon* de vacaciones**. Éste es un programa que examina cada mensaje de entrada y envía al transmisor una respuesta insípida como

Hola. Estoy de vacaciones. Regresaré el 24 de agosto. Que tenga un bonito día.

Tales respuestas también pueden especificar la manera de manejar asuntos urgentes en ese ínterin, indican que otras personas podrían resolver problemas específicos, etc. La mayoría de los *daemon*s de vacaciones mantienen un registro de a quiénes se enviaron respuestas prefabricadas y se abstienen de enviar a la misma persona una segunda respuesta. Los mejores también verifican si el mensaje de entrada se envió a una lista de correo, y de ser así, no envían ninguna respuesta prefabricada. (La gente que envía mensajes a listas grandes de correo durante el verano probablemente no quiere recibir cientos de respuestas con los detalles de las vacaciones de todos.)

El autor recientemente se topó con una forma extrema de procesamiento de entregas cuando envió un mensaje de correo electrónico a una persona que asegura recibir 600 mensajes al día. Su identidad no se divulgará, no sea que los lectores de este libro también le envíen correo electrónico. Llamémoslo Juan.

Juan instaló un robot de correo electrónico que revisa cada mensaje entrante para ver si es de un corresponsal nuevo. De serlo, devuelve un mensaje prefabricado explicando que Juan ya no puede leer personalmente todo su correo electrónico. En cambio, ha producido un documento FAQ (de preguntas comunes) que contesta muchas preguntas que se le hacen con frecuencia. Por lo regular, los grupos de noticias tienen documentos FAQ, no gente.

El FAQ de Juan da su dirección, fax y números telefónicos e indica la manera de comunicarse con su compañía; explica la manera de contratarlo como expositor y describe dónde pueden conseguirse sus artículos y otros documentos. El FAQ también proporciona referencias al *software* que ha escrito, una conferencia que imparte, un estándar que editó, etc. Tal vez este enfoque sea necesario, pero probablemente un FAQ personal es el último símbolo de *estatus*.

7.4.5. Confidencialidad en el correo electrónico

Cuando un mensaje de correo electrónico se envía entre dos sitios distantes, generalmente transitará por docenas de máquinas en el camino. Cualquiera de éstas puede leer y registrar el mensaje para su uso posterior. La confidencialidad es inexistente, a pesar de lo que piensa mucha gente (Wisband y Reining,1995). No obstante, mucha gente quisiera poder enviar correo electrónico que el destinatario pueda leer, pero nadie más: ni su jefe, ni los *hackers*, ni siquiera el gobierno. Este deseo ha estimulado a varias personas y grupos a aplicar los principios criptográficos que estudiamos antes al correo electrónico para producir correo seguro. En las siguientes secciones estudiaremos dos sistemas de correo electrónico seguros de amplio uso, PGP y PEM. Para información adicional, véase (Kaufman *et al.*, 1995; Schneier, 1995; Stallings, 1995b, y Stallings, 1995c).

PGP — Bastante buena confidencialidad

Nuestro primer ejemplo, **PGP** (*Pretty Good Privacy*, **bastante buena confidencialidad**) es en esencia el vástago de una sola persona, Phil Zimmermann (Zimmermann, 1995a, 1995b). PGP es un paquete completo de seguridad de correo electrónico que proporciona confidencialidad, validación de identificación, firmas digitales y compresión, todo de una forma fácil de usar. Es más, el paquete completo, incluido todo el código fuente, se distribuye sin cargo por Internet, boletines electrónicos y redes comerciales. Debido a su calidad, precio (cero) y fácil disponibilidad en las plataformas MS-DOS/Windows, UNIX y Macintosh, es de amplio uso hoy día. También hay una versión comercial disponible para aquellas compañías que requieren apoyo.

PGP también se ha visto envuelto en varias controversias (Levy, 1993). Puesto que está disponible gratuitamente en Internet, el gobierno de Estados Unidos ha alegado que la posibilidad de que lo obtengan extranjeros constituye una violación de las leyes concernientes a la exportación de armamentos. Se produjeron versiones posteriores fuera de Estados Unidos para evitar esta restricción. Otro problema ha tenido que ver con una supuesta violación de la patente del RSA, pero ese problema ya fue resuelto en las versiones 2.6 en adelante. Sin embargo, no a todo el mundo le gusta la idea de que la gente guarde secretos, por lo que los enemigos del PGP siempre están al acecho en las sombras, esperando el momento de atacar. Acorde con la situación, el lema de Zimmermann es: "si la confidencialidad se vuelve ilegal, sólo los ilegales tendrán confidencialidad".

El PGP intencionalmente usa algoritmos criptográficos existentes en lugar de inventar nuevos; se basa en gran medida en el RSA, el IDEA y el MD5, todos algoritmos que han resistido análisis extensos de colegas y no fueron diseñados ni influidos por ninguna agencia gubernamental que estuviera tratando de debilitarlos. Para la gente que tiende a desconfiar del gobierno, esta propiedad es un gran punto a favor.

El PGP maneja compresión de textos, secreto y firmas digitales, y también proporciona amplios recursos de manejo de claves. Para ver cómo funciona el PGP, consideremos el ejemplo de la figura 7-49. Aquí, Alicia quiere enviar un mensaje de texto normal firmado, P, a Benjamín de una manera segura. Tanto Alicia como Benjamín tienen claves RSA privadas (D_X) y públicas (E_X). Supongamos que cada uno conoce la clave pública del otro; cubriremos después la administración de claves.

Alicia comienza por invocar el programa PGP en su computadora. El PGP primero dispersa su mensaje, P, usando MD5, y luego cifra la dispersión resultante usando su clave RSA privada, D_A. Cuando Benjamín recibe el mensaje, puede descifrar la dispersión con la clave pública de Alicia y comprobar que es correcta. Incluso si alguien más (por ejemplo, Trudy) pudiera adquirir la dispersión en esta etapa y descifrarla con la clave pública de Alicia, la robustez del MD5 asegura que sería infactible computacionalmente producir otro mensaje con la misma dispersión MD5.

La dispersión cifrada y el mensaje original ahora están concatenados en un solo mensaje, $P1$, y comprimidos mediante el programa ZIP, que usa el algoritmo Ziv-Lempel (Ziv y Lempel, 1977). Llamemos a la salida de este paso $P1.Z$.

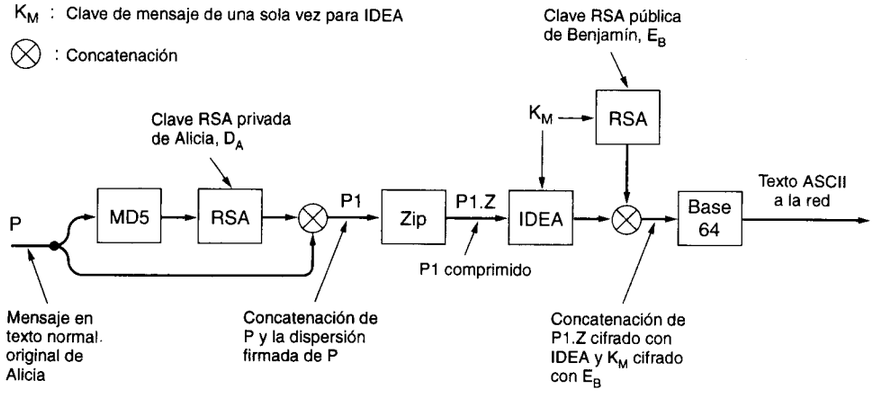


Figura 7-49. PGP en operación para el envío de un mensaje.

A continuación, el PGP solicita a Alicia una entrada al azar. Tanto el contenido como la velocidad de tecleo se usan para generar una clave de mensaje IDEA de 128 bits, K_M (llamada clave de sesión en la literatura PGP, pero que en realidad es un nombre inapropiado, pues no hay una sesión). K_M se usa ahora para cifrar $P1.Z$ con IDEA en modo de realimentación de cifrado. Además, K_M se cifra con la clave pública de Benjamín, E_B. Estos dos componentes se concatenan y convierten a base64, como estudiamos en la sección sobre MIME. El mensaje resultante contiene entonces sólo letras, dígitos y los símbolos +, / e =, lo que significa que puede ponerse en un cuerpo RFC 822 y esperar que llegue sin modificación.

Cuando Benjamín recibe el mensaje, invierte la codificación base64 y descifra la clave IDEA usando su clave RSA privada. Con la clave así obtenida, Benjamín descifra el mensaje para obtener $P1.Z$. Tras descomprimir esto, Benjamín separa el texto normal de la dispersión cifrada y descifra la dispersión usando la clave pública de Alicia. Si la dispersión del texto normal concuerda con su propio cálculo MD5, sabe que P es el mensaje correcto y que vino de Alicia.

Vale la pena señalar que el RSA sólo se usa aquí en dos lugares: para cifrar la parcialización MD5 de 128 bits y para cifrar la clave IDEA de 128 bits. Aunque el RSA es lento, sólo tiene que cifrar 256 bits, lo cual no es un gran volumen de datos. Es más, los 256 bits de texto normal son excesivamente aleatorios, por lo que se requerirá una cantidad considerable de trabajo por parte de Trudy para determinar si una clave adivinada es la correcta. El cifrado de trabajo pesado lo hace IDEA, que es más rápido que el RSA en varios órdenes de magnitud. Por tanto, el PGP proporciona seguridad, compresión y firma digital, y lo hace de una manera mucho más eficiente que el esquema ilustrado en la figura 7-23.

El PGP maneja tres longitudes de clave RSA. Es responsabilidad del usuario seleccionar la más adecuada. Las longitudes son:

1. Casual (384 bits): puede ser descifrada hoy día por gente con grandes presupuestos.
2. Comercial (512 bits): podría ser descifrable por organizaciones de tres iniciales.
3. Militar (1024): indescifrable.

Ha habido cierto debate sobre una cuarta categoría: extraterrestre (2048 bits), que no podría ser descifrada por nadie ni nada en el universo, pero aún no ha sido aceptada. Dado que el RSA se usa sólo para dos cálculos pequeños, probablemente todos deberían usar claves militares siempre, excepto tal vez en algunas PC-XT viejas.

En la figura 7-50 se muestra el formato de un mensaje PGP. El mensaje tiene tres partes, que contienen la clave IDEA, la firma y el mensaje, respectivamente. La parte de la clave no sólo contiene la clave, sino también un identificador de clave, puesto que se permite a los usuarios tener varias claves públicas.

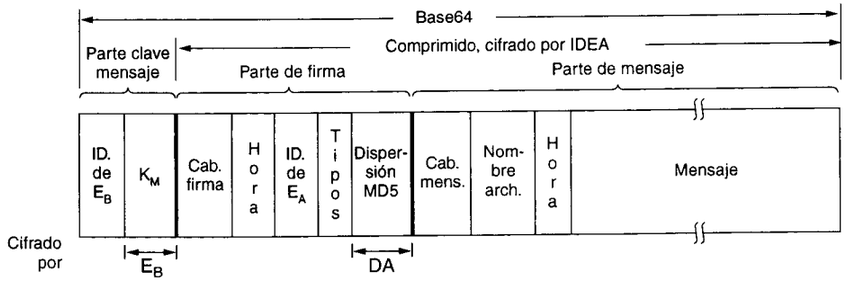


Figura 7-50. Mensaje PGP.

La parte de firma contiene una cabecera, de la que no nos ocuparemos aquí. A la cabecera le sigue una marca de tiempo, el identificador de la clave pública del transmisor, que puede servir para descifrar la dispersión de la firma, cierta información de tipo que identifica los algoritmos usados (para permitir el uso del MD6 y el RSA2 cuando se inventen) y la dispersión cifrada misma.

La parte del mensaje también contiene una clave, el nombre predeterminado del archivo a usar si el receptor escribe el archivo en disco, una marca de tiempo de creación del mensaje y, por último, el mensaje mismo.

La administración de claves ha recibido mucha atención en el PGP, puesto que es el talón de Aquiles de todos los sistemas de seguridad. Cada usuario mantiene localmente dos estructuras de datos: un anillo de claves privadas y un anillo de claves públicas. El **anillo de claves privadas** contiene uno o más pares de clave privada-clave pública personales. La razón para reconocer varios pares por usuario es permitir a los usuarios cambiar sus claves públicas periódicamente o cuando se piensa que una está comprometida, sin invalidar los mensajes que actualmente están en preparación o en tránsito. Cada par asociado tiene un identificador, para

que el remitente de un mensaje pueda indicar al destinatario la clave pública usada para cifrarlo. Los identificadores de mensaje consisten en los 64 bits de orden menor de la clave pública. Los usuarios son responsables de evitar conflictos con sus identificadores de clave pública. Las claves privadas en el disco se cifran usando una contraseña especial (arbitrariamente grande) para protegerlas contra ataques furtivos.

El **anillo de claves públicas** contiene claves públicas de los corresponsales del usuario; se requieren para cifrar las claves de mensaje asociadas a cada mensaje. Cada entrada del anillo de claves públicas contiene no sólo la clave pública, sino también su identificador de 64 bits y una indicación del nivel de confianza del usuario en la clave.

El problema que se ataca aquí es el siguiente. Supóngase que se mantienen claves públicas en boletines electrónicos. Una manera como Trudy pueda leer el correo electrónico secreto de Benjamín es atacar el tablero de avisos y reemplazar la clave pública de Benjamín por una de su propia selección. Cuando Alicia obtiene la clave que supuestamente es de Benjamín, Trudy puede montar un ataque de brigada de cubetas contra Benjamín.

Para evitar tales ataques, o cuando menos minimizar sus consecuencias, Alicia necesita saber qué tanto puede confiar en el elemento llamado "clave de Benjamín" de su anillo de claves públicas. Si sabe que Benjamín personalmente le entregó un disquete con la clave, puede establecer el valor de confianza en el nivel más alto.

Sin embargo, en la práctica, la gente con frecuencia recibe claves públicas haciendo una consulta a un servidor de claves confiable, de los cuales hay bastantes en operación en Internet. Al recibir un servidor de claves la solicitud de la clave pública de alguien, genera una respuesta que contiene la clave pública, una marca de tiempo, y la fecha de expiración de la clave; luego parcializa esta respuesta con MD5 y firma la respuesta con su propia clave privada de modo que la parte solicitante pueda verificar quién la envió. Es responsabilidad del usuario asignar un nivel de confianza a las claves mantenidas por el administrador del sistema local, la compañía telefónica, la ACM, la asociación de abogados, el gobierno, o cualquier otro que decida entrar en el negocio de administrar claves.

PEM — Correo con confidencialidad mejorada

En contraste con el PGP, que inicialmente fue asunto de una sola persona, nuestro segundo ejemplo, el **PEM** (*Privacy Enhanced Mail*, **correo con confidencialidad mejorada**), es un estándar oficial de Internet y se describe en cuatro RFC: el RFC 1421 al RFC 1424. De manera muy general, el PEM cubre el mismo territorio que el PGP: confidencialidad y validación de identificación para sistemas de correo electrónico basados en el RFC 822. Sin embargo, también tiene algunas diferencias respecto al PGP en cuanto a enfoque y tecnología. A continuación describiremos el PEM y luego lo compararemos con el PGP. Para mayor información sobre PEM, véase (Kent, 1993).

Los mensajes enviados usando PEM primero se convierten a una forma canónica para que puedan tener las mismas convenciones de espacios blancos (por ejemplo, tabuladores, espacios al final) y el uso de retornos de carro y avances de línea. Esta transformación se lleva a cabo para eliminar los efectos de los agentes de transferencia de mensajes que modifican los mensajes que

no son de su gusto. Sin canonización, tales modificaciones pueden afectar las dispersiones de los mensajes en sus destinos.

A continuación, se calcula una dispersión del mensaje usando MD2 o MD5. No es opcional, como en PGP. Después se cifra la concatenación de la dispersión y el mensaje usando DES. En vista de las debilidades conocidas de una clave de 56 bits, esta selección ciertamente es sospechosa. El mensaje cifrado puede entonces codificarse con codificación base64 y transmitirse al destinatario. Las listas de correo se reconocen explícitamente.

Como con el PGP, cada mensaje se cifra con una clave de una sola vez que se incorpora en el mensaje. La clave puede protegerse mediante RSA o con DES triple usando EDE. En la práctica, todos usan RSA, por lo que nos concentraremos en él. De hecho, tenemos que hacerlo: el PEM no indica cómo se debe hacer la administración de claves con DES.

La administración de claves es más estructurada que en el PGP. Las claves se certifican mediante **autoridades certificadoras** que producen certificados indicando el nombre del usuario, su clave pública y la fecha de expiración de la clave. Cada certificado tiene un número de serie único que lo identifica. Los certificados incluyen una dispersión MD5 firmada por la clave privada de la autoridad certificadora. Estos certificados se ajustan a la recomendación ITU X.509 para certificados de claves públicas y, como tales, usan nombres X.400 como el ejemplo de Tom Smith que presentamos antes.

El PGP tiene un esquema parecido (sin el uso del X.509), pero tiene un problema: ¿debe creer un usuario en la autoridad certificadora? El PEM resuelve este problema certificando las autoridades certificadoras, usando lo que se llama **PCA** (*Policy Certification Authorities*, **autoridades de certificación de políticas**). Éstas, por su parte, son certificadas por la **IPRA** (*Internet Policy Registration Authority*, **autoridad de registro de políticas de Internet**), que es el máximo árbitro sobre quién es bueno y quién es malo.

Cada PCA debe definir una política oficial en el momento de registrarse y archivarla con la IPRA. Estas declaraciones deben ser firmadas por el IPRA y hacerse públicas. Por ejemplo, una PCA puede insistir en que los usuarios bajo su jurisdicción se presenten personalmente con un certificado de nacimiento, licencia de conducir, pasaporte, dos tarjetas de crédito bancarias, un testigo vivo y una clave pública en disquete. Otra PCA podría aceptar registros por correo electrónico de completos extraños. Al hacer públicas las declaraciones de políticas, los usuarios tienen una base para decidir en qué autoridades pueden confiar. No se ha tomado ninguna medida para asegurar que las políticas realmente se pongan en práctica.

Se contemplan tres tipos diferentes de autoridades certificadoras. Una autoridad organizacional puede emitir certificados para sus empleados. La mayoría de las compañías operarán su propia autoridad. Una residencial operará en nombre de ciudadanos comunes, de manera parecida a los proveedores de servicios de Internet, que proporcionan servicio a cualquiera que esté dispuesto a pagarlo. Por último, hay un esquema planeado para el registro anónimo. Con todas estas autoridades certificadoras por ahí, debe quedar clara la necesidad de que las PCA las vigilen de cerca.

Aunque es rígidamente jerárquico y burocrático, este esquema tiene la ventaja respecto al PGP de hacer potencialmente práctica la revocación de certificados. Se requiere la revocación de certificados si un usuario quiere cambiar su clave pública, por ejemplo, porque está comprometida o su autoridad certificadora ha sido objeto de robo. La revocación se logra cuando un

usuario dice a la autoridad certificadora que su clave pública está comprometida (o posiblemente al revés). La autoridad certificadora entonces agrega el número de serie del certificado que ha dejado de ser válido a una lista de certificados revocados, lo firma y distribuye la lista a todo mundo.

Cualquiera que desee enviar un mensaje PEM a un usuario debe, por tanto, revisar la lista más reciente de revocaciones para ver si la clave pública en caché aún es válida. Este proceso es análogo a que un comerciante revise la lista de tarjetas de crédito robadas antes de aceptar una. Los críticos del PEM argumentan que estar revisando todo el tiempo es demasiado trabajo, por lo que nadie se tomará la molestia. Los que están a su favor argumentan que las computadoras no se aburren; si se programan para revisar todo el tiempo, lo harán.

Algunas de las semejanzas y diferencias entre el PGP y el PEM se listan en la figura 7-51. La mayoría de estos puntos ya han sido cubiertos, pero vale la pena comentar algunos. La validación de identificación parece más importante en el PEM que en el PGP, puesto que es obligatoria en el PEM y opcional en el PGP. También, el PEM lleva la información de validación de identificación fuera de la envoltura de cifrado, lo que significa que la red puede comprobar el origen de cada mensaje. Como consecuencia, los espías pueden registrar quién envía a quién, aunque no pueden leer los mensajes.

Dejando a un lado todas estas diferencias técnicas, hay también una diferencia cultural sorprendente. El PGP, que no es un estándar oficial de Internet, tiene la cultura Internet. El PEM, que es un estándar oficial de Internet, no la tiene. El PGP se basó en lo que Dave Clark llama "consenso aproximado y código operativo". Alguien (Zimmermann) pensó en una solución a un problema muy conocido, la implementó bien y liberó el código fuente para el uso de todos. El PEM comenzó como un estándar oficial de cuatro partes, usando ASN.1 para definir el formato, X.400 para definir los nombres y X.509 para definir los certificados; usa una jerarquía organizacional rígida de tres niveles con varios tipos de autoridades certificadoras, incluyendo declaraciones de políticas certificadas oficialmente y el requisito de que todo mundo confíe en la IPRA. Las implementaciones vinieron después y están muy por debajo del PGP en cuanto a calidad, cantidad y disponibilidad en muchas plataformas. En pocas palabras, el PGP parece un paquete Internet típico, mientras que el PEM presenta la mayoría de las características de un estándar OSI que la gente de Internet odia y los PTT aman. Trate de entender esto.

7.5. NOTICIAS USENET

Una de las aplicaciones más comunes de las redes de cómputo es el sistema mundial de grupos de noticias llamado **noticias de red** (*net news*). Con frecuencia, esto se conoce como **USENET**, que se remonta a una red física de UNIX a UNIX independiente que alguna vez condujo el tráfico usando un programa llamado **uucp**. Hoy día, una buena parte del tráfico se lleva por la Internet, pero USENET e Internet no son lo mismo. Algunas instalaciones Internet no reciben las noticias de red y otras reciben noticias sin estar en Internet.

En las siguientes secciones describiremos el USENET. Primero lo veremos desde el punto de vista del usuario. Luego describiremos su implementación.

Asunto	PGP	PEM
¿Maneja cifrado?	Sí	Sí
¿Maneja validación de identificación?	Sí	Sí
¿Maneja no repudio?	Sí	Sí
¿Maneja compresión?	Sí	No
¿Maneja canonización?	No	Sí
¿Maneja listas de correo?	No	Sí
¿Usa codificación base64?	Sí	Sí
Algoritmo actual de cifrado de datos	IDEA	DES
Longitud de clave para cifrado de datos (bits)	128	56
Algoritmo actual para administración de claves	RSA	RSA o DES
Longitud de clave para administración de claves	384/512/1024	Variable
Espacio de nombres de usuario	Definido por usuario	X.400
¿Se ajusta al X.509?	No	Sí
¿Hay que confiar en alguien?	No	Sí (IPRA)
Certificación de claves	Ad hoc	Jerarquía IPRA/PCA/CA
Revocación de claves	Fortuita	Mejor
¿Pueden los espías leer los mensajes?	No	No
¿Pueden los espías leer las firmas?	No	Sí
¿Estándar Internet?	No	Sí
Diseñado por	Equipo pequeño	Comité de estándares

Figura 7-51. Comparación del PGP y el PEM.

7.5.1. USENET desde el punto de vista del usuario

Un grupo de noticias es un foro de debate mundial sobre un tema específico. La gente interesada en el tema puede "suscribirse" al grupo de noticias. Los suscriptores pueden usar un tipo especial de agente de usuario, un lector de noticias, para leer todos los artículos (mensajes) publicados en el grupo de noticias. La gente también puede publicar artículos en el grupo de noticias. Cada artículo publicado en un grupo de noticias automáticamente se entrega a todos los suscriptores, en cualquier lugar del mundo donde puedan estar. La entrega típicamente tarda entre unos cuantos segundos y varias horas, dependiendo de lo alejados que estén el transmisor y el receptor de las rutas comunes. En efecto, un grupo de noticias es algo como una lista de correo, pero internamente se implementa de manera diferente. También puede visualizarse como un tipo de multitransmisión de alto nivel.

La cantidad de grupos de noticias es tan grande (probablemente más de 10,000) que se organizan jerárquicamente para poderlos manejar. En la figura 7-52 se muestran los niveles superiores de las jerarquías "oficiales". También existen otras jerarquías, pero por lo regular están dirigidas a consumo regional o están en idiomas diferentes del inglés. Una de las otras jerarquías, *alt*, es especial. *Alt* es para los grupos oficiales lo que un mercado callejero es para una tienda departamental; es una mescolanza caótica y desregulada de grupos de noticias sobre todos los temas, algunos de los cuales son muy populares, y la mayoría de los cuales son mundiales.

Nombre	Temas
Comp	Computadoras, informática y la industria de la computación
Sci	Ciencias físicas e ingeniería
Humanities	Literatura y humanidades
News	Debates sobre el USENET mismo
Rec	Actividades recreativas, incluye deportes y música
Mic	Todo lo que no cabe en otro lado
Soc	Relaciones y asuntos sociales
Talk	Diatribas, polémicas, debates y argumentos a granel
Alt	Árbol alterno que cubre prácticamente todo

Figura 7-52. Jerarquías USENET en orden descendente de relación señal a ruido.

Los grupos *comp* fueron los grupos USENET originales. Estos grupos están poblados de informáticos, profesionales de la computación y aficionados a las computadoras. Cada uno ofrece debates técnicos sobre un tema relacionado con el *hardware* o *software* de cómputo.

Los grupos *sci* y *humanities* están poblados de científicos, estudiantes y aficionados interesados en física, química, biología, Shakespeare, etc. No es sorprendente que la jerarquía *sci* es mucho mayor que *humanities*, pues el concepto mismo de la comunicación electrónica instantánea con colegas de todo el mundo es algo que a la mayoría de los científicos le gusta, pero sobre lo que la mayoría de los humanistas cuando menos tiene reservas. C.P. Snow tenía razón.

La jerarquía *news* se usa para debatir y administrar el sistema de noticias mismo. Los administradores de sistemas pueden encontrar ayuda aquí, y aquí tienen lugar los debates sobre la creación de grupos de noticias nuevos.

Las jerarquías cubiertas hasta el momento tienen un tono profesional, un tanto académico. Eso cambia con *rec*, que trata sobre actividades recreativas y aficiones. No obstante, las personas que publican aquí tienen conocimientos amplios sobre sus intereses respectivos.

A medida que seguimos avanzando, llegamos a *soc*, que tiene muchos grupos de noticias relacionados con política, sexo, religión, diferentes culturas nacionales y genealogía. *Talk* cubre

temas controvertidos y está poblado de gente con opiniones fuertes, pero débil en cuanto a información y cifras. *Alt* es un árbol alterno completo que opera bajo sus propias reglas.

Cada una de las categorías listadas en la figura 7-52 se divide en subcategorías, recursivamente. Por ejemplo, *rec.sport* se ocupa de deportes, *rec.sport.basketball* es sobre baloncesto y *rec.sport.basketball.women* trata el baloncesto femenil. En la figura 7-53 se presenta una muestra de algunos de los grupos de noticias de cada categoría. En muchos casos, la existencia de grupos adicionales puede inferirse cambiando los parámetros obvios. Por ejemplo, *comp.lang.c* es sobre el lenguaje de programación C, pero la *.c* puede cambiarse por prácticamente cualquier otro lenguaje de programación para generar el nombre del grupo de noticias correspondiente.

Existen muchos programas lectores de noticias. Al igual que los lectores de correo electrónico, algunos se basan en el teclado; otros se basan en el ratón. En casi todos los casos, cuando el lector de noticias inicia, revisa un archivo para ver los grupos de noticias a los que está suscrito el usuario; después, por lo regular, presenta un resumen de una línea de cada artículo no leído aún del primer grupo de noticias y espera que el usuario seleccione uno o más. Los artículos seleccionados se presentan en la pantalla, uno a la vez. Tras su lectura, los artículos pueden descartarse, guardarse, imprimirse, etcétera.

Los lectores de noticias también permiten a los usuarios suscribirse y cancelar las suscripciones a los grupos de noticias. El cambio de una suscripción simplemente significa la edición del archivo local que lista los grupos de noticias a los que está suscrito el usuario. Para hacer una analogía, la suscripción a un grupo de noticias es como ver un programa de televisión. Si usted quiere ver un programa cada semana, simplemente lo hace. No necesita registrarse antes con una autoridad central.

Los lectores de noticias también manejan la publicación. El usuario compone un artículo y luego emite un comando o hace clic en un icono para enviar el artículo. En un lapso de un día, llegará a casi todos los suscriptores mundiales del grupo de noticias en el que se publicó. Es posible hacer **publicaciones cruzadas** (*crossposts*) de un artículo, es decir, enviarlo a varios grupos de noticias con un solo comando. También es posible restringir la distribución geográfica de una publicación. Un anuncio del coloquio del martes en Stanford probablemente no será de mucho interés, digamos, en Hong Kong, por lo que la publicación puede restringirse a California.

La sociología de USENET es única, por decirlo de alguna manera. Nunca antes ha sido posible que miles de personas que no se conocen tengan debates mundiales sobre una enorme variedad de temas. Por ejemplo, es posible ahora que alguien con un problema lo publique en la red. Al día siguiente, el autor de la publicación puede tener 18 soluciones y, con un poco de suerte, sólo 17 de ellas equivocadas.

Desafortunadamente, algunas personas usan su nuevo poder de comunicación para comunicarse irresponsablemente con un grupo grande. Cuando alguien publica un mensaje que indica: "A la gente como ustedes deberían fusilarla", los ánimos se caldean y sigue un torrente de publicaciones injuriosas, llamado **guerra de flamas** (*flamewar*).

Esta situación puede atacarse de dos maneras, una individual y otra colectiva. Los usuarios individuales pueden instalar un **archivo de eliminación** (*killfile*), que especifica que los artículos con cierto tema o de cierta persona deben descartarse a su llegada, antes de exhibirse.

Nombre	Temas
Comp.ai	Inteligencia artificial
Comp.databases	Diseño e implementación de sistemas de bases de datos
Comp.lang.c	El lenguaje de programación C
Comp.os.minix	Sistema operativo educativo MINIX de Tanenbaum
Comp.os.ms-windows.video	*Hardware* y *software* de vídeo para Windows
Sci.bio.entomology.lepidoptera	Investigación sobre mariposas y polillas
Sci.geo.earthquakes	Geología, sismología y terremotos
Sci.med.orthopedics	Cirugía ortopédica
Humanities.lit.authors.shakespeare	Dramas y poesía de Shakespeare
News.groups	Posibles grupos de noticias nuevos
News.lists	Listas relacionadas con el USENET
Rec.arts.poems	Poesía gratuita
Rec.food.chocolate	Sabroso
Rec.humor.funny	¿Has oído el chiste sobre el granjero que...?
Rec.music.folk	Discusiones sobre música folclórica
Misc.jobs.offered	Anuncios de plazas disponibles
Misc.health.diabetes	La vida día a día con diabetes
Soc.culture.estonia	Vida y cultura de Estonia
Soc.singles	Solteros y sus intereses
Soc.couples	Parejas. Graduados de soc.singles
Talk.abortion	Aborto. Sin señal, puro ruido
Talk.rumors	De aquí salen los rumores
Alt.alien.visitors	Lugar para informar de viajes en OVNI
Alt.bermuda.triangle	Si lee esto, desaparece misteriosamente
Alt.sex.voyeurism	Asómese y véalo usted mismo
Alt.tv.simpsons	Bart y compañía

Figura 7-53. Pequeña selección de los grupos de noticias.

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

La mayoría de los lectores de noticias también permite matar el hilo de una discusión individual. Esta característica es útil cuando parece que una discusión comienza a entrar en un ciclo infinito.

Si se molestan suficientes suscriptores de un grupo con la contaminación del grupo de noticias, pueden proponer la moderación del grupo de noticias. Un **grupo de noticias moderado** es uno en el que sólo una persona, el moderador, puede publicar artículos en el grupo de noticias. Todas las publicaciones de un grupo moderado se envían automáticamente al moderador, quien publica las buenas y descarta las malas. Algunos temas tienen un grupo de noticias moderado y uno no moderado.

Dado que cada día se suscriben miles de personas a USENET por primera vez, tienden a hacerse las mismas preguntas de principiante una y otra vez. Para reducir este tráfico, muchos grupos de noticias han construido un documento **FAQ** (*Frequently Asked Questions*, **preguntas**

frecuentes) que intenta contestar todas las preguntas de los principiantes. Algunos de éstos son bastante exhaustivos y abarcan más de 100 páginas. El encargado del mantenimiento típicamente los publica una o dos veces al mes.

El USENET está lleno de jerga como BTW (*By The Way*, por cierto), ROFL (*Rolling On the Floor and Laughing*, tirado en el piso y muerto de risa) e IMHO (*In My Humble Opinion*, desde mi humilde punto de vista). Mucha gente también usa pequeños símbolos ASCII llamados **caritas** (*smileys*) o **iconos de emoción** (*emoticons*). Algunos de los más interesantes se reproducen en la figura 7-54. En casi todos los casos, el girar el libro 90 grados en dirección de las manecillas del reloj hará más claros los símbolos. (Sanderson y Dougherty, 1993) tienen un minilibro con más de 650 caritas.

Carita	Significado	Carita	Significado	Carita	Significado
:-)	Estoy contento	=I:-)	Abraham Lincoln	:+)	Nariz grande
:-(	Estoy triste/enojado	=):-)	Tío Sam	:-))	Papada
:-I	No me importa	*<:-)	Santa Claus	:-(){	Bigote
;-)	Estoy guiñando	<:-(	Tonto	#:-)	Cabello enmarañado
:-(0)	Estoy gritando	(-:	Australiano	8-)	Usa lentes
:-(*)	Estoy vomitando	:-)X	Hombre con corbata de moño	C:-)	Cerebro grande

Figura 7-54. Algunas caritas.

Aunque la mayoría de la gente usa su verdadero nombre en las publicaciones, hay quienes desean permanecer anónimos, especialmente cuando publican en grupos de noticias sujetos de controversia y cuando publican anuncios personales en los grupos de noticias relacionados con la búsqueda de un compañero(a). Este deseo ha conducido a la creación de **reenviadores anónimos**, que son servidores que aceptan mensajes de correo electrónico (incluidas publicaciones) y cambian los campos *From:, Sender:* y *Reply-To:* para hacer que apunten al reenviador en lugar de al remitente. Algunos de los reenviadores asignan números a los usuarios y reenvían el correo electrónico dirigiéndolo a estos números, para que la gente pueda enviar respuestas de correo electrónico a las publicaciones anónimas como "Mujer blanca soltera 25 busca hombre blanco soltero/divorciado 20-30..." Es dudoso que estos reenviadores puedan guardar sus secretos cuando la policía local sienta curiosidad por la identidad de algún usuario (Barlow, 1995).

A medida que más gente se suscribe al USENET, hay una demanda constante de grupos de noticias nuevos más especializados. En consecuencia, se ha establecido un procedimiento para crear grupos nuevos. Supongamos que a alguien le gustan las cucarachas y quiere hablar con otros fanáticos de las cucarachas; publica un mensaje en *news.groups* nombrando el grupo propuesto, digamos *rec.animals.wildlife.cockroaches* y describiendo la razón por la que son tan importantes (las cucarachas son fascinantes; hay 3500 especies; vienen en rojo, amarillo, verde, café y negro; aparecieron en la tierra mucho antes que los primeros dinosaurios; probablemente fueron los primeros animales voladores, etc.). También se indica si debe ser un grupo moderado o no.

Surge entonces el debate. Cuando termina, se hace una votación por correo electrónico. Los votos se publican, identificando quién voto de qué manera (para evitar fraudes). Si la relación entre los votos sí y los no es mayor que 2:1 y hay cuando menos 100 más a favor que en contra, el moderador de *news.groups* publica un mensaje aceptando el nuevo grupo de noticias. Este mensaje es el aviso a los administradores de sistemas de todo el mundo de que el nuevo grupo ha sido bendecido por los altos poderes y que ahora es oficial.

La creación de un grupo nuevo es menos formal en la jerarquía *alt*, y ésa es, de hecho, la razón de ser de *alt*. Algunos de los grupos de noticias que hay ahí están tan cercanos a los límites legales y morales de lo tolerable que nunca habrían sido aceptados mediante un voto público. En efecto, la gente que los apoyó simplemente pasó de largo el procedimiento normal y creó su propia jerarquía. No obstante, una buena parte de la jerarquía *alt* es bastante convencional.

7.5.2. Implementación de USENET

Algunos de los grupos de noticias más pequeños se implementan como listas de correo. Si alguien quiere publicar un artículo en una de tales listas, lo envía a la dirección de la lista de correo, lo que causa el envío de copias a todas las direcciones de la lista de correo.

Sin embargo, si la mitad de los estudiantes de una universidad grande se suscribiera a *alt.sex*, los servidores de ahí se vendrían abajo por el peso del correo entrante. En consecuencia, USENET generalmente no se implementa usando listas de correo. En cambio, cada instalación (universidad, compañía o proveedor de servicio Internet) almacena el correo entrante en un solo directorio, digamos *news*, con subdirectorios para *comp*, *sci*, etc. Éstos a su vez tienen subdirectorios como *news/comp/os/minix*. Todas las noticias entrantes se depositan en el directorio apropiado. Los lectores de noticias simplemente traen los artículos de ahí según los necesitan. Este arreglo significa que cada instalación sólo requiere una copia de cada artículo, sin importar la cantidad de gente subscrita a su grupo de noticias. Tras unos cuantos días, los artículos expiran y se remueven del disco.

Para entrar en USENET, una instalación debe tener una **alimentación de noticias** (*newsfeed*) de otra instalación que ya está en USENET. Puede pensarse en el grupo de todas las instalaciones que reciben noticias de red como los nodos de un grafo dirigido. Las líneas de transmisión que conectan pares de nodos forman los arcos del grafo. Este grafo es el USENET. Nótese que estar en Internet no es necesario ni suficiente para estar en USENET.

Periódicamente, cada instalación que desea noticias puede sondear sus alimentadores de noticias, solicitando las noticias nuevas respecto al contacto previo. De haberlas, se recolectan esas noticias y se almacenan en el subdirectorio adecuado de *news*. De esta manera, las noticias se difunden por la red. También es igualmente posible que la alimentación de noticias, en lugar del receptor, tome la iniciativa y haga contacto cuando hay suficientes noticias nuevas. Inicialmente, la mayoría de las instalaciones sondeaban sus alimentadores de noticias, pero ahora es principalmente al revés.

No todas las instalaciones reciben todos los grupos de noticias. Hay varias razones para esto. Primero, la alimentación total de noticias excede los 500 MB diarios y crece con rapidez. El almacenamiento de todo requeriría una gran cantidad de espacio en disco. Segundo, el tiempo y

el costo de transmisión son asuntos importantes. A 28.8 kbps, se requieren 39 horas y una línea telefónica dedicada para transmitir 24 horas de noticias. Incluso a 56 kbps, conseguir todo requiere una línea dedicada durante casi 20 horas al día. De hecho, el volumen total ahora es tan grande que se han creado alimentaciones de noticias vía satélite.

Tercero, no todas las instalaciones se interesan en todos los temas. Por ejemplo, es poco probable que mucha gente de compañías finlandesas quiera leer *rec.arts.manga* (sobre tiras cómicas japonesas). Por último, algunos grupos de noticias son demasiado subidos de color para los gustos de muchos administradores de sistemas, que los proscriben, a pesar de existir un interés local considerable. En diciembre de 1995, la red mundial CompuServe dejó de incluir (temporalmente) todos los grupos de noticias con "sex" en el nombre, porque algún burócrata alemán pensó que sería una buena manera de combatir la pornografía. La protesta resultante fue predecible, instantánea, mundial y muy ruidosa.

Los artículos de noticias tienen el mismo formato que los mensajes de correo electrónico RFC 822, pero con la adición de algunas cabeceras extra. Esta propiedad los hace fáciles de transportar y compatibles con casi todo el *software* de correo electrónico existente. Las cabeceras de noticias se definen en el RFC 1036. En la figura 7-55 se presenta un artículo de ejemplo.

```
From: Vogel@nyu.edu
Message-Id: <54731 @nyu.edu>
Subject: Avistamiento de pájaro
Path: cs.vu.nl!sun4nl!EU.net!news.sprintlink.net!in2.uu.net!pc144.nyu.edu!news
Newsgroups: rec.birds
Followup-To: rec.birds
Distribution: world
Nntp-Posting-host: nuthatch.bio.nyu.edu
References:
Organización: New York University
Lines: 4
Summary: Adivinen lo que vi

Acabo de ver un avestruz en la Calle 52 y la Quinta Avenida de Nueva York. ¿Es su época
de migración? ¿Lo vio alguien más?

Jay Vogel
```

Figura 7-55. Ejemplo de artículo noticioso.

Son pertinentes algunas palabras sobre las cabeceras de noticias. La cabecera *Path:* es la lista de nodos que atravesó el mensaje para llegar del remitente al destinatario. En cada salto, la máquina reenviadora puso su nombre al frente de la lista. Esta lista proporciona una trayectoria de regreso al remitente. El uso de signos de admiración (llamados *bang*) se remonta a las direcciones USENET, que antedata al DNS.

La cabecera *Newsgroups:* indica los grupos a los que pertenece el mensaje. Puede contener más de un nombre de grupo de noticias. Cualquier mensaje publicado en forma cruzada en

varios grupos de noticias contendrá todos sus nombres. Dado que se permiten nombres múltiples aquí, se requiere la cabecera *Followup-To:* para indicarle a la gente dónde puede publicar los comentarios y reacciones, a fin de poner la discusión subsecuente en un solo grupo de noticias.

La cabecera *Distribution:* indica la distancia a la que se distribuirá la publicación; puede contener uno o más códigos de estado o país, el nombre de una instalación o red específica, o "world" (todo el mundo).

La cabecera *Nntp-Posting-Host:* es análoga a la cabecera *Sender:* del RFC 822. Indica la máquina que publicó el artículo, aun si fue compuesto en otra máquina (NNTP es el protocolo de noticias, que se describe más adelante).

La cabecera *References:* indica que este artículo es una respuesta a un artículo anterior e identifica a ese artículo. Se requiere en todos los artículos que son una continuación y se prohíbe cuando se comienza una discusión nueva.

La cabecera *Organization:* puede usarse para indicar la compañía, universidad o agencia a la que está afiliada el remitente. Los artículos que llenan esta cabecera con frecuencia tienen una nota al final para indicar que, en caso de ser una tontería, no es culpa de la organización.

La cabecera *Lines:* especifica la longitud del cuerpo. Las líneas de cabecera y la línea en blanco que separa la cabecera del cuerpo no se toman en cuenta.

Las líneas *Subject:* atan los hilos de la discusión. Muchos lectores de noticias tienen un comando para permitir que el usuario vea el siguiente artículo sobre el tema actual, en lugar del siguiente artículo que llegó. También los archivos de eliminación y los comandos de eliminación usan esta cabecera para saber lo que deben rechazar.

Por último, el *Summary:* normalmente se usa para resumir un artículo de continuación. En los artículos de continuación, la cabecera *Subject:* contiene "Re: " seguido del tema original.

NNTP — Protocolo de transferencia de noticias de red

Veamos ahora la manera en que se difunden los artículos por la red. El algoritmo inicial simplemente inundaba los artículos por cada línea de USENET. Aunque esto funcionó durante un tiempo, tarde o temprano el volumen de tráfico hizo impráctico este esquema, por lo que tuvo que diseñarse algo mejor.

Su reemplazo fue un protocolo llamado **NNTP** (*Network News Transfer Protocol*, **protocolo de transferencia de noticias de red**), que se define en el RFC 977. El NNTP recuerda un poco al SMTP; un cliente emite comandos en ASCII y un servidor emite respuestas en forma de números decimales codificados en ASCII. La mayoría de las máquinas USENET usan ahora el NNTP.

El NNTP se diseñó para dos fines. La primera meta era permitir la propagación de artículos de una máquina a otra a través de una conexión confiable (por ejemplo, TCP). La segunda meta era permitir la lectura remota de noticias a los usuarios cuyas computadoras no pueden recibir noticias. El NNTP se usa ampliamente para ambas cosas, pero nos concentraremos en la manera en que los artículos de noticias se difunden por la red usando NNTP.

Como se mencionó antes, son posibles dos enfoques. En el primero, obtención de noticias, el cliente llama a uno de sus alimentadores de noticias y solicita noticias nuevas. En el segundo,

empuje de noticias, el alimentador de noticias llama al cliente y anuncia que tiene noticias. Los comandos NNTP reconocen ambos enfoques, al igual que la lectura remota de noticias.

Para obtener artículos recientes, un cliente debe primero establecer una conexión TCP con el puerto 119 de uno de sus alimentadores de noticias. Detrás de este puerto está el *daemon* NNTP, esperando clientes todo el tiempo, o bien se crea sobre la marcha según se necesita. Tras establecerse la conexión, el cliente y el servidor se comunican usando una secuencia de comandos y respuestas. Estos comandos y respuestas sirven para asegurar que el cliente reciba todos los artículos que necesita, pero no duplicados, sin importar la cantidad de alimentadores de noticias que use. Los principales comandos usados para mover artículos entre *daemons* de noticias se listan en la figura 7-56.

Comando	Significado
LIST	Dame una lista de todos los grupos de noticias y artículos que tienes
NEWGROUPS fecha hora	Dame una lista de grupos de noticias creados después de fecha/hora
GROUP grp	Dame una lista de todos los artículos en grp
NEWGROUPS grps fecha hora	Dame una lista de artículos nuevos de los grupos especificados
ARTICLE id	Dame un artículo específico
POST	Tengo un artículo para ti que se publicó aquí
IHAVE id	Tengo id de artículo. ¿Lo quieres?
QUIT	Termina sesión

Figura 7-56. Comandos NNTP principales para la difusión de noticias.

Los comandos *LIST* y *NEWGROUPS* permiten que el cliente averigüe qué grupos tiene el servidor. El primero produce la lista completa. El segundo lista sólo los grupos creados después de la fecha y hora especificadas. Si el cliente sabe que la lista es larga, es más eficiente que él lleve el control de lo que tiene cada uno de sus alimentadores de noticias y simplemente solicite actualizaciones. La respuesta de cada uno de estos comandos es una lista, en ASCII, con un grupo de noticias por línea, que proporciona el nombre del grupo de noticias, el número del último artículo que tiene el servidor, el número del primer artículo que tiene el servidor, y una bandera que indica si se permite la publicación en este grupo de noticias o no.

Una vez que el cliente sabe qué grupos de noticias tiene el servidor, puede comenzar a preguntar por los artículos que tiene el servidor (por ejemplo, para grupos viejos cuando se usa *NEWGROUPS*). Los comandos *GROUP* y *NEWNEWS* tienen este fin. De nuevo, el primero produce la lista completa y el segundo sólo lista las actualizaciones posteriores a la fecha y hora indicadas, normalmente la hora de la última conexión con este alimentador de noticias. El primer parámetro puede contener asteriscos, para indicar todos. Por ejemplo, *comp.os.** significa todos los grupos de noticias que comienzan con la cadena *comp.os*.

Una vez que el cliente tiene una lista completa de los artículos existentes en los grupos (e incluso antes de tener la lista completa), puede comenzar a preguntar por los artículos que

necesita usando el comando *ARTICLE*. Una vez que están dentro todos los artículos adquiridos, el cliente puede ofrecer artículos que ha adquirido de otros alimentadores de noticias usando el comando *IHAVE*, y artículos que fueron publicados localmente mediante el comando *POST*. El servidor puede aceptarlos o rechazarlos, según desee. Al concluir el cliente, puede terminar la sesión usando *QUIT*. De esta manera, cada máquina tiene control completo sobre los artículos que recibe de cada alimentador de noticias, eliminando todos los artículos duplicados.

Como ejemplo del funcionamiento del NNTP, considere un proveedor de información, *decente.net*, que quiere evitar controversias a toda costa, por lo que el único grupo de noticias que ofrece es *soc.parejas* y *misc.chicos*. Sin embargo, la gerencia no quiere encerrarse y está dispuesta a manejar otros grupos de noticias, siempre y cuando no contengan material potencialmente ofensivo para nadie; por tanto, quiere que se le informe de todos los grupos de nueva creación para poder tomar una decisión informada a nombre de sus clientes. Una posible situación entre *decente.com* actuando como cliente, y su alimentador de noticias, *feeder.com*, actuando como servidor, se muestra en la figura 7-57. Esta situación usa el enfoque de obtención de noticias (el cliente inicia la conexión para solicitar noticias). Las anotaciones entre paréntesis son comentarios y no forman parte del protocolo NNTP.

En esta sesión, *decente.com* pregunta primero si hay noticias para *soc.parejas*. Cuando se le indica que hay dos artículos, obtiene ambos y los almacena en *news/soc/parejas* como archivo aparte. Cada archivo se nombra por su número de artículo. Luego *decente.com* pregunta sobre *misc.chicos* y se le indica que hay un artículo; lo obtiene y lo pone en *news/misc/chicos*.

Habiendo obtenido todas las noticias sobre los grupos que lleva, ahora busca grupos nuevos y se le indica que han aparecido dos grupos nuevos desde la última sesión. Uno de ellos parece prometedor, por lo que se obtienen los artículos. El otro se ve peligroso, por lo que no se toma. (*decente.com* ha hecho una inversión importante en *software* de inteligencia artificial para poder ser capaz de determinar lo que va a llevar simplemente viendo los nombres.)

Tras haber adquirido todos los artículos que quiere, *decente.com* ofrece a *feeder.com* un artículo nuevo publicado por alguien de esta instalación. Se acepta la oferta y se transfiere el artículo. Ahora *decente.com* ofrece otro artículo, uno que vino de su otro alimentador de noticias. Dado que *feeder.com* ya tiene éste, lo rechaza. Por último, *decente.com* termina la sesión y libera la conexión TCP.

El enfoque de empuje de noticias es parecido. Comienza por la llamada del alimentador de noticias a la máquina que debe recibir las noticias. El alimentador de noticias normalmente lleva el registro de los grupos de noticias a los que están suscritos sus clientes, y comienza por anunciar su primer artículo en el primero de estos grupos de noticias usando el comando *IHAVE*. El receptor potencial entonces revisa sus tablas para ver si ya tiene el artículo, y puede aceptarlo o rechazarlo. Si el artículo es aceptado, se transmite, seguido de una línea que contiene un punto. Entonces el alimentador de noticias anuncia el segundo artículo, al igual que los demás, hasta que se han transferido todas las noticias.

Un problema con ambos, la obtención de noticias y el empuje de noticias, es que usan parada y espera. Por lo regular, se pierden 100 mseg en la espera de la respuesta a una pregunta. Con 100,000 o más artículos de noticias nuevos por día, este tiempo perdido se acumula para generar una carga extra sustancial.

```
      S: 200 Servidor NNTP feeder.com a su servicio (respuesta a una conección nueva)
C: NEWNEWS soc.parejas 960901 030000 (¿hay noticias nuevas en soc.parejas?)
      S: 230 Continúa lista de 2 artículos
      S: <13281 @psyc.berkeley.edu>      (el artículo 1 de 2 en soc.parejas es de Berkeley)
      S: <162721 @aol.com>               (el artículo 2 de 2 en soc.parejas es de AOL)
      S: .                               (fin de lista)
C: ARTICLE <13281 @psyc.berkeley.edu>    (por favor dame el artículo de Berkeley)
      S: 220 Sigue <13281@psyc.berkeley.edu>
      S: (artículo completo <13281@psyc.berkeley.edu> se envía aquí)
      S: .                               (fin de artículo)
C: ARTICLE <162721 @aol.com>             (por favor dame el artículo de AOL)
      S: 220 Sigue <162721@aol.com>
      S: (artículo completo <162721@aol.com> se envía aquí)
      S: .                               (fin de artículo)
C: NEWNEWS misc.chicos 960901 030000     (¿hay noticias nuevas en misc.chicos?)
      S: 230 Sigue lista de 1 artículo
      S: <43222@bio.rice.edu>            (1 artículo de Rice)
      S: .                               (fin de lista)
C: ARTICLE <43222@bio.rice.edu>          (por favor dame el artículo de Rice)
      S: 220 Sigue <43222@bio.rice.edu>
      S: (artículo completo <43222@bio.rice.edu> se envía aquí)
      S: .                               (fin de artículo)
C: NEWGROUPS 960901 030000
      S: 231 Siguen 2 grupos nuevos
      S: rec.mascotas
      S: rec.desnudo
      S: .
C: NEWNEWS rec.mascotas 0 0              (lista todo lo que tienes)
      S: 230 Sigue lista de 1 artículo
      S: <124@fido.net>                  (1 artículo de fido.net)
      S: .                               (fin de lista)
C: ARTICLE <124@fido.net>                (por favor dame el artículo fido.net)
      S: 220 sigue <124@fido.net>
      S: (se envía el artículo completo aquí)
      S: .
C: POST
      S: 340                             (por favor envía tus publicaciones)
C:    (artículo publicado en decente.com se envía aquí)
      S: 240                             (artículo recibido)
C: IHAVE <5321@foo.com>
      S: 435                             (Ya lo tengo, por favor no lo envíes)
C: QUIT
      S: 205                             (Que tengas un bonito día)
```

Figura 7-57. Manera en que *decente.com* podría adquirir artículos nuevos de su alimentador de noticias.

7.6. LA *WORLD WIDE WEB*

La *World Wide Web* es un armazón arquitectónico para acceder a documentos vinculados distribuidos en miles de máquinas de toda la Internet; en cinco años, pasó de ser una manera de distribuir datos sobre física de alta energía a la aplicación que millones de personas piensan que es "La Internet". Su enorme popularidad se deriva del hecho de que tiene una interfaz gráfica atractiva que es fácil de usar por los principiantes y proporciona un enorme cúmulo de información sobre casi cualquier tema concebible, desde aborígenes hasta zoología.

La *Web* (también conocida como **WWW**) comenzó en 1989 en el CERN, el Centro Europeo de Investigación Nuclear. El CERN tiene varios aceleradores en los que los científicos de los países europeos participantes llevan a cabo investigaciones sobre física de partículas. Estos equipos con frecuencia tienen miembros de media docena de países o más. La mayoría de los experimentos son altamente complejos, y requieren años de planeación adelantada y construcción de equipo. La *Web* surgió de la necesidad de lograr que estos grandes grupos de investigadores dispersos internacionalmente colaboraran usando un conjunto siempre cambiante de informes, planos, dibujos, fotos y otros documentos.

La propuesta inicial de una red (*web*) de documentos vinculados surgió del físico del CERN Tim Berners-Lee en marzo de 1989. El primer prototipo (basado en texto) estaba en operación 18 meses después. En diciembre de 1991 se hizo una demostración pública en la conferencia Hypertext '91 en San Antonio, Texas. El desarrollo continuó durante el siguiente año, culminando con la liberación de la primera interfaz gráfica, Mosaic, en febrero de 1993 (Vetter *et al.*, 1994).

Mosaic tuvo tanto éxito que, un año después, su autor, Marc Andreessen, dejó el Centro Nacional de Aplicaciones de Supercómputo, donde se desarrolló Mosaic, para formar una compañía, Netscape Communications Corp., cuya meta fue desarrollar clientes, servidores y otros tipos de *software* de la *Web*. Cuando Netscape se volvió pública en 1995, los inversionistas, aparentemente pensando que ésta era la siguiente Microsoft, pagaron 1,500 millones de dólares por las acciones. Esta marca fue aún más sorprendente porque la compañía sólo tenía un producto, operaba con grandes pérdidas y había anunciado en su prospecto que no esperaba tener ganancias en un futuro cercano.

En 1994, el CERN y el M.I.T. firmaron un acuerdo para establecer el World Wide Web Consortium, una organización dedicada al desarrollo de la *Web*, la estandarización de protocolos y el fomento de interoperabilidad entre las instalaciones. Berners-Lee se convirtió en el director. Desde entonces, cientos de universidades y compañías se han unido al consorcio. El M.I.T. opera la parte de Estados Unidos del consorcio, y el centro francés de investigación, INRIA, opera la parte europea. Aunque hay más libros sobre la *Web* de los que pueden contarse, el mejor lugar para recibir información actualizada sobre la *Web* es (naturalmente) la *Web* misma. La página base (*home page*) del consorcio puede encontrarse en *http://www.w3.org*. Los lectores interesados pueden encontrar ahí vínculos con páginas que cubren todos los documentos y actividades del consorcio.

En las siguientes secciones describiremos la *Web* desde el punto de vista del usuario y, especialmente, su funcionamiento interno. Puesto que la *Web* básicamente es un sistema cliente-

servidor, estudiaremos tanto el lado del cliente (es decir, el usuario) como el lado del servidor. Después analizaremos el lenguaje de escritura de las páginas de la Web (HTML y Java). Por último, examinaremos la manera de encontrar información en la *Web*.

7.6.1. El lado del cliente

Desde el punto de vista del usuario, la *Web* consiste en un enorme conjunto a nivel mundial de documentos, generalmente llamados **páginas**. Cada página puede contener vínculos (apuntadores) con otras páginas relacionadas en cualquier lugar del mundo. Los usuarios pueden seguir un vínculo (por ejemplo, haciendo clic en él), lo que los lleva a la página apuntada. Este proceso puede repetirse indefinidamente, posiblemente atravesando cientos de páginas vinculadas al hacerlo. Se dice que las páginas que apuntan a otras páginas usan **hipertexto**.

Las páginas se ven mediante un programa llamado **visor** (*browser*); Mosaic y Netscape son dos de los visores más populares. El visor obtiene la página solicitada, interpreta el texto y los comandos de formateo que contiene, y exhibe la página, adecuadamente formateada, en la pantalla. En la figura 7-58(a) se da un ejemplo. Al igual que muchas páginas de la *Web*, ésta comienza con un título, contiene cierta información y termina con la dirección de correo electrónico del encargado del mantenimiento de la página. Las cadenas de texto que son vínculos a otras páginas, llamadas **hipervínculos** (*hyperlinks*), se resaltan, ya sea mediante subrayado, presentación en un color especial, o ambas cosas. Para seguir un vínculo, el usuario coloca el cursor en el área resaltada (usando el ratón o las teclas de flechas) y la selecciona (haciendo clic con un botón del ratón o pulsando ENTER). Aunque existen algunos visores no gráficos, como Lynx, no son tan comunes como los visores gráficos, por lo que nos concentraremos en estos últimos. También se están desarrollando visores basados en voz.

Los usuarios con curiosidad respecto al Departamento de psicología animal pueden aprender más sobre él haciendo clic en su nombre (subrayado). El visor entonces trae la página a la que está vinculado el nombre y la presenta, como se muestra en la figura 7-58(b). También puede hacerse clic en los elementos subrayados aquí para traer otras páginas, y así se puede continuar. La página nueva puede estar en la misma máquina que la primera, o en otra máquina del otro lado del mundo. El usuario no puede saberlo. La obtención de páginas la hace el visor sin ayuda del usuario. Si el usuario llega a regresar a la página principal, los vínculos ya seguidos tal vez aparezcan con un subrayado punteado (y posiblemente otro color) para distinguirlos de los vínculos que no se han seguido. Nótese que hacer clic en la línea de *Información de la Universidad* de la página principal no tiene ningún efecto; no está subrayada, lo que significa que simplemente es texto y no está vinculada con otra página.

La mayoría de los visores tienen muchos botones y características que simplifican la navegación en la *Web*. Muchos tienen un botón para regresar a la página anterior, un botón para avanzar a la siguiente página (sólo operativo una vez que el usuario ha regresado de ella), y un botón para ir directamente a la propia página base del usuario. Muchos visores tienen un botón o elemento de menú para poner una marca en una página dada y otro para visualizar la lista de marcas, haciendo posible regresar a cualquiera de ellas mediante un solo clic del ratón. Las páginas también pueden almacenarse en disco o imprimirse. Generalmente hay numerosas

BIENVENIDOS A LA PÁGINA BASE DE WWW DE LA UNIVERSIDAD DE EAST PODUNK

- Información sobre la Universidad
  - Información sobre inscripciones
  - Mapa de las instalaciones
  - Indicaciones para llegar a las instalaciones
  - El cuerpo estudiantil de la UEP
- Departamentos académicos
  - Departamento de psicología animal
  - Departamento de estudios alternativos
  - Departamento de cocina microbiótica
  - Departamento de estudios no tradicionales
  - Departamento de estudios tradicionales

Webmaster@eastpodunk.edu

(a)

DEPARTAMENTO DE PSICOLOGÍA ANIMAL

- Información sobre posibles carreras
- Personal
  - Miembros del profesorado
  - Estudiantes de postgrado
  - Personal no académico
- Proyectos de investigación
- Puestos disponibles
- Nuestros cursos más populares
  - Manejo de herbívoros
  - Administración de caballos
  - Negociando con su mascota
  - Construcción de perreras amables con el usuario
- Lista completa de cursos

Webmaster@animalpsyc.eastpodunk.edu

(b)

Figura 7-58. (a) Página de Web. (b) Página accedida al hacer clic en *Departamento de psicología animal*.

opciones disponibles para controlar la distribución de la pantalla y ajustar varias preferencias del usuario. En (Berghel, 1996) se comparan nueve visores.

Además de tener texto normal (no subrayado) e hipertexto (subrayado), las páginas de la *Web* también pueden contener iconos, dibujos de líneas, mapas y fotografías. Cada uno de estos puede vincularse (opcionalmente) con otra página. Hacer clic en uno de estos elementos causa que el visor traiga la página vinculada y la exhiba, igual que ocurre con el texto. En el caso de imágenes como fotografías y mapas, la página que se trae a continuación puede depender de la parte de la imagen en la que se hizo clic.

No todas las páginas pueden visualizarse de la manera convencional. Por ejemplo, algunas páginas consisten en pistas de audio, fragmentos de vídeo, o ambas cosas. El resultado de mezclar páginas de hipertexto con otros medios se llama **hipermedia**. Algunos visores pueden presentar todos los tipos de hipermedia, pero otros no; en vez de ello, revisan un archivo de configuración para ver la manera de manejar los datos recibidos. Normalmente, el archivo de configuración indica el nombre de un programa, llamado **visor externo**, o **aplicación ayudante**, que se ejecutará con la página como entrada. Si no hay un visor configurado, el visor generalmente le pide al usuario que escoja uno. Si no existe ningún visor, el usuario puede indicar al visor que guarde la página entrante en un archivo en disco, o la descarte. Las aplicaciones ayudantes para producir voz hacen posible que incluso los usuarios ciegos accedan a la *Web*. Otras aplicaciones ayudantes contienen intérpretes de lenguajes de *Web* especiales, haciendo posible descargar y ejecutar programas desde las páginas de la *Web*. Este mecanismo hace posible extender la funcionalidad de la *Web* misma.

Muchas páginas de la *Web* contienen imágenes grandes, que tardan mucho tiempo en cargarse. Por ejemplo, traer una imagen no comprimida de 640 × 480 (VGA) con 24 bits por pixel (992 KB) requiere unos cuatro minutos a través de un módem de 28.8 kbps. Algunos visores manejan la carga lenta de imágenes obteniendo y presentando primero el texto y luego trayendo las imágenes mismas. Esta estrategia ofrece al usuario algo que leer mientras las imágenes llegan y permite que el usuario cancele el cargado si la página no es lo suficientemente interesante para justificar la espera. Una estrategia alterna es proporcionar una opción para inhabilitar la obtención y presentación automática de imágenes.

Algunos escritores de páginas intentan interesar a usuarios potencialmente aburridos presentando imágenes de una manera especial. Primero, la imagen aparece con baja definición. Luego se llenan gradualmente los detalles. Para el usuario, ver la imagen completa tras unos cuantos segundos, aunque sea con baja definición, con frecuencia es preferible a verla construirse lentamente desde arriba, línea de barrido por línea de barrido.

Algunas páginas de la *Web* contienen formas que solicitan información al usuario. Las aplicaciones típicas para estas formas son la búsqueda de elementos proporcionados por el usuario en una base de datos, la colocación de un pedido de un producto y la participación en una encuesta de opinión. Otras páginas de la *Web* contienen mapas que permiten a los usuarios hacer clic en ellos para exhibir acercamientos u obtener información sobre alguna zona geográfica. El manejo de formas y de mapas activos (con clic) requiere un proceso más complejo que simplemente traer una página conocida. Describiremos después la manera de implementar estas características.

Algunos visores usan el disco local para poner en caché las páginas que han traído. Antes de traer una página, se comprueba si ya está en el caché local. De ser así, sólo es necesario verificar si la página está actualizada. De ser así, la página no necesita cargarse nuevamente. Como resultado, normalmente es muy rápido hacer clic en el botón BACK para ver la página previa.

Para ser *host* de un visor de la *Web*, una máquina debe estar directamente en Internet o, cuando menos, tener una conexión SLIP o PPP con un enrutador u otra máquina conectada directamente a Internet. Este requisito existe porque la manera en que un visor trae una página es estableciendo una conexión TCP con la máquina en la que está la página, y luego enviando un mensaje a través de la conexión solicitando la página. Un visor no funcionará si no puede establecer una conexión TCP a una máquina arbitraria de Internet.

A veces son impresionantes los extremos a los que llega la gente para acceder a la *Web*. Cuando menos una compañía ofrece servicio de *Web* por fax. Un cliente sin acceso a Internet llama al servidor de *Web* por fax y se conecta usando el teclado del teléfono; luego teclea un código que identifica la página de *Web* deseada, la cual se envía al fax del cliente.

7.6.2. El lado del servidor

Cada instalación de la *Web* tiene un proceso servidor que escucha en el puerto TCP 80, esperando conexiones entrantes de los clientes (normalmente visores). Tras establecerse una conexión, el cliente envía una solicitud y el servidor envía una respuesta. Después se libera la conexión. El protocolo que define las solicitudes y respuestas legales se llama HTTP; lo estudiaremos con algún detalle a continuación, pero un ejemplo sencillo de su uso puede proporcionar una idea razonable sobre el funcionamiento de los servidores de la *Web*. En la figura 7-59 se muestra la manera en que encajan varias de las partes de la *Web*.

Para este ejemplo, podemos imaginar que el usuario acaba de hacer clic en alguna parte del texto o, tal vez, en un icono que apunta a la página cuyo nombre (URL — *Uniform Resource Locator*, localizador uniforme de recursos) es *http://www.w3.org/hypertext/WWW/TheProject.html*. También explicaremos los URL posteriormente en este capítulo. Por el momento, basta con saber que un URL tiene tres partes: el nombre del protocolo (*http*), el nombre de la máquina donde se encuentra la página (*www.w3.org*) y el nombre del archivo que contiene la página (*hypertext/WWW/TheProject.html*). Los pasos que se ejecutan entre el clic del usuario y la presentación de la página son los siguientes:

1. El visualizador determina el URL (viendo lo que se seleccionó).
2. El visualizador solicita al DNS la dirección IP de *www.w3.org*.
3. El DNS contesta con 18.23.0.23.
4. El visualizador establece una conexión TCP con el puerto 80 en 18.23.0.23.
5. A continuación, el visualizador emite un comando *GET /hypertext/WWW/TheProject.html*.
6. El servidor *www.w3.org* envía el archivo *TheProject.html*.

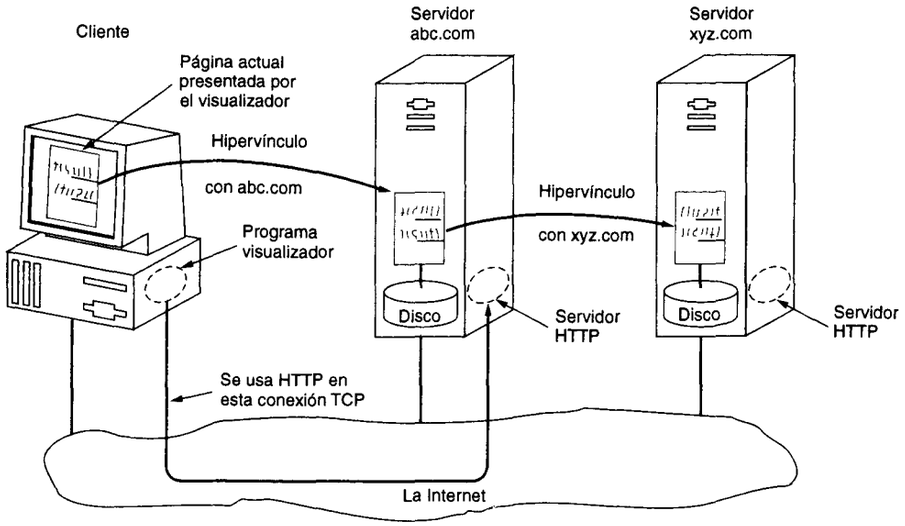


Figura 7-59. Partes del modelo de la *Web*.

7. Se libera la conexión TCP.
8. El visualizador presenta todo el texto de *TheProject.html*.
9. El visualizador trae y presenta todas las imágenes de *TheProject.html*.

Muchos visualizadores presentan el paso que están ejecutando en el momento en una línea de estado en la parte inferior de la pantalla. De esta manera, cuando el desempeño es malo, el usuario puede ver si se debe a que el DNS no responde, el servidor no responde o simplemente hay congestionamiento en la red durante la transmisión de la página.

Vale la pena indicar que, para cada imagen en línea (icono, dibujo, fotografía, etc.) de una página, el visualizador establece una conexión TCP nueva con el servidor pertinente para traer la imagen. Sobra decir que si una página contiene muchos iconos, todos en el mismo servidor, el establecer, usar y liberar una conexión nueva para cada uno no es muy eficiente, pero simplifica la implementación. Las modificaciones futuras del protocolo se encargarán del problema de la eficiencia. En (Mogul, 1995) se hace una propuesta.

Puesto que el HTTP es un protocolo ASCII como el SMTP, es bastante fácil que una persona en una terminal (sin usar un visualizador) hable directamente con servidores de la Web. Todo lo que se necesita es una conexión TCP al puerto 80 del servidor. La manera más sencilla de efectuar tal conexión es usar el programa Telnet. En la figura 7-60 se muestra un ejemplo de cómo se puede lograr esto. Aquí, las líneas marcadas con *C:* son tecleadas por el usuario (cliente), las líneas marcadas con *T:* son producidas por el programa Telnet, y las líneas marcadas con *S:* son producidas por el servidor de M.I.T.


```
C: telnet www.w3.org 80
T: Trying 18.23.0.23 ...
T: Connected to www.w3.org.
T: Escape character is '^]'.
C: GET /hypertext/WWW/TheProject.html HTTP/1.0
C:
    S: HTTP/1.0 200 Document follows
    S: MIME-Version: 1.0
    S: Server: CERN/3.0
    S: Content-Type: text/html
    S: Content-Length: 8247
    S:
    S: <HEAD> <TITLE> The World Wide Web Consortium (W3C) </TITLE> </HEAD>
    S: <BODY>
    S: <H1> <IMG ALIGN=MIDDLE ALT="W3C" SRC="Icons/WWW/w3c_96x67.gif">
    S: El World Wide Web Consortium </H1> <P>
    S:
    S: La World Wide Web es el universo de la información accesible por red.
    S: El <A HREF="Consortium/"> World Wide Web Consortium </A>
    S: existe para hacer realidad el potencial completo de la Web. <P>
    S:
    S: El W3C trabaja con la comunidad mundial para producir
    S:
    S: El W3C trabaja con la comunidad mundial para producir
    S: <A HREF="#Specifications"> especificaciones </A> y
    S: <A HREF="#Reference"> software de referencia </A>.
    S: El W3C es financiado por industriales
    S: <A HREF"Consortium/Member/List.html"> miembros </A>
    S: pero sus productos están disponibles gratuitamente para todos. <P>
    S:
    S: En este documento:
    S: <menu>
    S: <LI> <A HREF="#Specifications"> Especificaciones y áreas de desarrollo de la Web </A>
    S: <LI> <A HREF="#Reference"> Software de la Web </A>
    S: <LI> <A HREF="#Community"> La World Wide Web y la comunidad de la Web </A>
    S: <LI> <A HREF="#Joining"> Participación en el W3C </A>
    S: </menu>
    S: <P> <HR>
    S: Los anfitriones del W3C son
    S: el <A HREF="http://www.lcs.mit.edu/"> Laboratorio de informática </A> del
    S: <A HREF="http://web.mit.edu/"> MIT </A>, y
    S: y, en Europa, el <A HREF="http://www.inria.fr/"> INRIA </A>.
    S: </BODY>
```

Figura 7-60. Situación ejemplo para obtener una página de la *Web*.

Se invita al lector a intentar este diálogo personalmente (de preferencia desde un sistema UNIX, puesto que otros sistemas no devuelven el estado de la conexión). Asegúrese de incluir los espacios y la versión del protocolo en la línea *GET*, y la línea en blanco que sigue a la línea *GET*. Como nota, el texto que se recibirá será diferente del que se muestra en la figura 7-60 por cuatro razones. Primero, la salida de ejemplo se resumió y editó para que cupiera en una página. Segundo, se limpió un tanto la salida para evitar poner en ridículo al autor, quien sin duda esperaba que miles de personas examinaran la página formateada, pero que ninguna revisara el HTML que la produjo. Tercero, el contenido de la página cambia constantemente. Cuarto, aquí se presenta una traducción. No obstante, este ejemplo debe dar una idea razonable sobre el funcionamiento del HTTP.

Lo que muestra el ejemplo es lo siguiente. El cliente, en este caso una persona, pero normalmente un visualizador, primero se conecta con cierto *host* y envía un comando solicitando una página en particular y especificando el protocolo y la versión a usar (HTTP/1.0). En la línea 7, el servidor responde con una línea de estado que indica el protocolo usado (el mismo que el cliente) y el código 200, que significa OK. Esta línea va seguida de un mensaje MIME RFC 822, del que se muestran cinco de las líneas de cabecera en la figura (se han omitido varias más por cuestión de espacio). Luego viene una línea en blanco, seguida del cuerpo del mensaje. Para enviar una fotografía, el campo *Content-Type* podría ser

```
Content-Type: Image/GIF
```

De esta manera, los tipos MIME permiten el envío de objetos arbitrarios de una manera estándar. Como nota, la cabecera *Content-Transfer-Encoding* de MIME no se requiere, puesto que el TCP permite el envío de corrientes arbitrarias de bytes, incluso imágenes, sin modificación. El significado de los comandos encerrados entre corchetes angulares usados en la página de ejemplo se estudiará más adelante en este capítulo.

No todos los servidores hablan HTTP. En particular, muchos servidores viejos usan FTP, Gopher u otros protocolos. Dado que hay una gran cantidad de información útil disponible en los servidores FTP y Gopher, una de las metas de diseño de la *Web* fue poner esta información a la disposición de los usuarios de la *Web*. Una solución es hacer que el visualizador use estos protocolos al hablar con un servidor FTP o Gopher. De hecho, algunos visualizadores usan esta solución, pero hacer que los visualizadores entiendan todos los protocolos posibles aumenta innecesariamente su tamaño.

Con frecuencia se usa una solución diferente: los servidores apoderados (Luotonen y Altis, 1994). Un **servidor apoderado** es una clase de pasarela que habla HTTP con el visualizador pero FTP, Gopher o algún otro protocolo con el servidor; acepta solicitudes HTTP y las traduce a, digamos, solicitudes FTP, de modo que el visualizador no tenga que entender ningún otro protocolo más que HTTP. El servidor apoderado puede ser un programa que se ejecuta en la misma máquina que el visualizador, pero también puede estar en una máquina independiente en algún lugar de la red, sirviendo a muchos visualizadores. En la figura 7-61 se muestra la diferencia entre un visualizador que puede hablar FTP y otro que usa un apoderado.

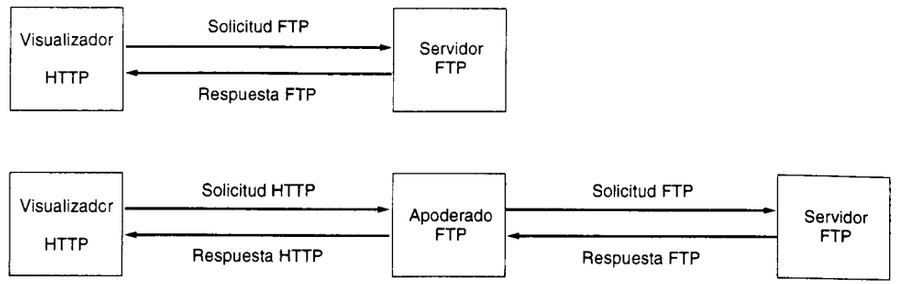


Figura 7-61. (a) Visualizador que habla FTP. (b) Visualizador que no.

Con frecuencia, los usuarios pueden configurar sus visualizadores con apoderados para protocolos que los visualizadores no hablan. De esta manera se aumenta la gama de fuentes de información a las que tiene acceso el visualizador.

Además de actuar como intermediario con protocolos desconocidos, los servidores apoderados tienen varias otras funciones importantes, como el manejo de caché. Un servidor apoderado con caché recolecta y guarda todas las páginas que pasan por él. Cuando un usuario solicita una página, el servidor apoderado verifica si tiene la página. De ser así, puede comprobar si la página aún está vigente. En caso de que la página siga estando vigente, se pasará al usuario; de otro modo, se traerá una copia nueva.

Por último, una organización puede poner un apoderado dentro de su muro de seguridad para que los usuarios puedan acceder a la *Web*, pero sin tener acceso completo a la Internet. En esta configuración, los usuarios pueden hablar con el servidor apoderado, pero es el servidor apoderado el que se comunica con las instalaciones remotas y trae las páginas a nombre de sus clientes. Este mecanismo puede usarse, por ejemplo, en las escuelas de enseñanza media superior para bloquear el acceso a sitios de la *Web* que el director considere inadecuadas para las tiernas mentes juveniles.

Para información sobre uno de los servidores de *Web* más comunes (el *daemon* HTTP de NCSA) y su desempeño, véase (Katz *et al.*, 1994, y Kwan *et al.*, 1995).

HTTP — Protocolo de transferencia de hipertexto

El protocolo estándar de transferencia de la *Web* es el **HTTP** (*HyperText Transfer Protocol*, **protocolo de transferencia de hipertexto**). Cada interacción consiste en una solicitud ASCII seguida de una respuesta tipo MIME RFC 822. Aunque es muy común el uso del TCP para la conexión de transporte, no es requerido formalmente por el estándar. Si las redes ATM se vuelven lo bastante confiables, las solicitudes y respuestas HTTP podrían transportarse igualmente en mensajes AAL 5.

El HTTP está evolucionando constantemente. Se usan varias versiones y se están desarrollando otras. El material presentado a continuación es relativamente básico y es poco probable que cambie su concepto, pero algunos detalles podrían ser un poco diferentes en las versiones futuras.

El protocolo HTTP consiste en dos elementos bastante diferentes: el grupo de solicitudes de los visualizadores a los servidores y el grupo de respuestas en el otro sentido. Ahora los veremos por partes.

Todas las versiones más recientes de HTTP reconocen dos tipos de solicitud: solicitudes sencillas y solicitudes completas. Una solicitud sencilla es sólo una línea *GET* que nombra la página deseada, sin la versión del protocolo. La respuesta es la página en bruto, sin cabeceras, sin MIME y sin codificación. Para ver su funcionamiento, intente establecer una conexión Telnet con el puerto 80 de *www.w3.org* (como se muestra en la primera línea de la figura 7-60) y luego teclee

```
GET /hypertext/WWW/TheProject.html
```

pero esta vez sin HTTP/1.0. Se devolverá la página sin indicación de su tipo de contenido. Este mecanismo es necesario para la compatibilidad hacia atrás; su uso se reducirá a medida que los visualizadores y los servidores basados en solicitudes completas se vuelvan la regla.

Las solicitudes completas se indican por la presencia de la versión del protocolo en la línea de solicitud *GET*, como en la figura 7-60. Las solicitudes pueden consistir en múltiples líneas, seguidas de una línea en blanco para indicar el final de la solicitud, razón por la que se requirió la línea en blanco en la figura 7-60. La primera línea de una solicitud completa contiene el comando (*GET* es sólo una posibilidad), la página deseada y el protocolo/versión. Las líneas subsiguientes contienen cabeceras RFC 822.

Aunque el HTTP se diseñó para usarse en la *Web*, ha sido intencionalmente más general de lo necesario con miras a aplicaciones futuras orientadas a objetos. Por esta razón, la primera palabra de la línea de solicitud completa es sencillamente el nombre del **método** (comando) a ejecutar con la página de la *Web* (u objeto general). Los métodos interconstruidos se listan en la figura 7-62. Después de acceder a objetos generales, también pueden estar disponibles métodos adicionales específicos para ese objeto. Los nombres son sensibles a mayúsculas y minúsculas, por lo que *GET* es un método legal, pero *get* no.

El método *GET* solicita al servidor que envíe la página (con lo que queremos decir objeto, en el caso más general), codificada adecuadamente en MIME. Sin embargo, si a la solicitud *GET* le

Método	Descripción
GET	Solicita leer una página de *Web*
HEAD	Solicita leer la cabecera de una página de *Web*
PUT	Solicita almacenar una página de *Web*
POST	Adiciona a un recurso nombrado (p. ej., página de *Web*)
DELETE	Elimina la página de *Web*
LINK	Conecta dos recursos existentes
UNLINK	Rompe una conexión existente entre dos recursos

Figura 7-62. Métodos de solicitud HTTP interconstruidos.

sigue una cabecera *If-Modified-Since*, el servidor sólo envía los datos si fueron modificados después de la fecha proporcionada. Usando este mecanismo, un visualizador al que se solicitó una página que está en caché puede solicitarla condicionalmente al servidor, dando la hora de modificación asociada a la página. Si la página en caché aún es válida, el servidor simplemente devuelve una línea de estado anunciando el hecho, eliminando por tanto la carga extra de transferir de nuevo la página.

El método *HEAD* simplemente pide la cabecera del mensaje, sin la página. Este método puede servir para obtener la hora de la última modificación, para recolectar información con fines de indización, o simplemente para probar la validez de un URL. No existen las solicitudes *HEAD* condicionales.

El método *PUT* es el inverso de *GET*: en lugar de leer una página, la escribe. Este método hace posible construir un conjunto de páginas de la *Web* en un servidor remoto. El cuerpo de la solicitud contiene la página y puede codificarse usando MIME, en cuyo caso las líneas que siguen a *PUT* podrían incluir cabeceras *Content-Type* y de validación de identificación, para demostrar que el solicitante efectivamente tiene permiso de ejecutar la operación solicitada.

Algo parecido a *PUT* es el método *POST*; también lleva un URL pero, en lugar de reemplazar los datos existentes, se "anexa" a ellos en algún sentido generalizado. La publicación de un mensaje en un grupo de noticias y la adición de un archivo a un sistema de boletines electrónicos son ejemplos de anexión en este contexto. Claramente, la intención aquí es hacer que la *Web* asuma la funcionalidad del sistema de noticias USENET.

DELETE hace lo que podría esperarse: elimina la página. Como con *PUT*, la validación de identificación y los permisos desempeñan un papel principal. No hay garantía de que *DELETE* tendrá éxito, puesto que, incluso si el servidor HTTP remoto está dispuesto a borrar la página, el archivo subyacente puede tener un modo que prohiba al servidor HTTP su modificación o eliminación.

Los métodos *LINK* y *UNLINK* permiten establecer conexiones entre páginas existentes u otros recursos.

Cada solicitud recibe una respuesta que consiste en la línea de estado y, posiblemente, información adicional (por ejemplo, toda o parte de una página de la *Web*). La línea de estado puede tener el código 200 (OK) o cualquiera de varios códigos de error, por ejemplo 304 (no modificado), 400 (lista errónea) o 403 (prohibido).

Los estándares HTTP describen las cabeceras y los cuerpos de mensaje con considerable detalle. Basta decir que son muy parecidos a los mensajes MIME RFC 822, por lo que no los veremos aquí.

7.6.3. Escritura de una página de *Web* en HTML

Las páginas de *Web* se escriben en un lenguaje llamado **HTML** (*HyperText Markup Language*, **lenguaje de marcación de hipertexto**). El HTML permite a los usuarios producir páginas de *Web* que incluyen texto, gráficos y apuntadores a otras páginas de *Web*. Comenzaremos nuestro estudio del HTML con estos apuntadores, puesto que son el pegamento que mantiene unida a la *Web*.

Los URL — Localizadores uniformes de recursos

Repetidamente hemos dicho que las páginas de *Web* pueden contener apuntadores a otras páginas de *Web*. Ahora es tiempo de ver la implementación de estos apuntadores. Cuando se creó la *Web*, de inmediato fue evidente para lograr que una página de *Web* apuntara a otra se requerían mecanismos para nombrar y localizar las páginas. En particular, había tres preguntas que debían contestarse antes de poder presentar visualmente una página:

1. ¿Cómo se llama la página?
2. ¿Dónde está la página?
3. ¿Cómo se puede acceder a la página?

Si cada página tuviera asignado de alguna manera un nombre único, no habría ambigüedad al identificar las páginas. No obstante, el problema no se resolvería. Considere un paralelismo entre gente y páginas. En Estados Unidos, casi todos tienen un número de seguro social, que es un identificador único, puesto que no hay dos personas que tengan el mismo. Sin embargo, armados sólo con el número de seguro social, no hay manera de encontrar la dirección del dueño y, ciertamente, no hay manera de saber si se debe escribir a la persona en inglés, español o chino. La *Web* tiene básicamente los mismos problemas.

La solución escogida identifica las páginas de una manera que resuelve los tres problemas a la vez. A cada página se le asigna un **URL** (*Uniform Resource Locator*, **localizador uniforme de recursos**) que sirve efectivamente como nombre mundial de la página. Los URL tienen tres partes: el protocolo (también llamado esquema), el nombre DNS de la máquina en la que se encuentra la página y un nombre local que indica de manera única la página específica (generalmente sólo un nombre de archivo de la máquina en la que reside). Por ejemplo, el URL del departamento del autor es

```
http://www.cs.vu.nl/welcome.html
```

Este URL consta de tres partes: el protocolo (*http*), el nombre DNS del *host* (*www.cs.vu.nl*) y el nombre del archivo (*welcome.html*), con cierta puntuación que separa las partes.

Muchas instalaciones cuentan con ciertos atajos para los nombres de archivos. Por ejemplo, *~user/* podría tener una correspondencia con el directorio WWW de *user*, con la convención de que una referencia al directorio mismo implica cierto archivo, digamos *index.html*. Por tanto, la página base del autor puede encontrarse en

```
http//www.cs.vu.nl/~ast/
```

aunque el nombre real del archivo es distinto. En muchas instalaciones, un nombre de archivo nulo se refiere por omisión a la página base de la organización.

Ahora debe quedar clara la manera en que funciona el hipertexto. Para que pueda hacerse clic en una parte del texto, el escritor de la página debe proporcionar dos elementos de información: el texto visualizable en el que se puede hacer clic y el URL de la página a la que se debe ir si se selecciona el texto. Al seleccionarse el texto, el visualizador busca el nombre del *host* usando DNS. Armado ahora con la dirección IP del *host*, el visualizador establece entonces una conexión TCP con el *host*, y envía por esa conexión el nombre de archivo usando el protocolo especificado. Eureka. Regresa la página. Esto es precisamente lo que vimos en la figura 7-60.

Este esquema URL es abierto en el sentido de que es fácil tener protocolos distintos de HTTP. De hecho, se han definido los URL de varios otros protocolos comunes, y muchos visualizadores los entienden. En la figura 7-63 se listan formas ligeramente simplificadas de los más comunes.

Nombre	Usado para	Ejemplo
http	Hipertexto (HTML)	http://www.cs.vu.nl/~ast/
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Archivo local	/usr/suzanne/prog.c
news	Grupo de noticias	news:comp.os.minix
news	Artículo	news:AA0134223112@cs.utah.edu
gopher	Gopher	gopher://gopher.tc.umn.edu/11/Libraries
mailto	Envío de correo electrónico	mailto:kim@acm.org
telnet	Ingreso remoto	telnet://www.w3.org:80

Figura 7-63. Algunos URL comunes.

Repasemos brevemente la lista. El protocolo *http* es el lenguaje nativo de la *Web*, el hablado por los servidores HTTP; maneja todos los métodos de la figura 7-62, al igual que cualquier método específico del objeto que se requiera.

El protocolo *ftp* se usa para acceder a archivos mediante FTP, el protocolo de transferencia de archivos de Internet. El FTP tiene más de dos décadas de existencia y está bien atrincherado. Numerosos servidores FTP de todo el mundo permiten que gente de cualquier lugar de Internet establezca una sesión y descargue los archivos colocados en el servidor FTP. La *Web* no cambia esto; simplemente hace más sencilla la obtención de archivos mediante FTP, puesto que FTP tiene una interfaz un tanto arcaica. Con el tiempo, el FTP probablemente desaparecerá, pues no hay ninguna ventaja particular en que una instalación opere un servidor FTP en lugar de un servidor HTTP, que puede hacer todo lo que un servidor FTP y más (aunque hay algunas discusiones sobre su eficiencia).

Es posible acceder a un archivo local como página de *Web*, ya sea usando el protocolo *file* o, más sencillamente, con sólo nombrarlo. Este enfoque es parecido a usar FTP pero no requiere un servidor. Por supuesto que sólo funciona con archivos locales.

El protocolo *news* permite a un usuario de la *Web* llamar un artículo como si fuera una página de *Web*. Esto significa que un visualizador de la *Web* simultáneamente es un lector de noticias. De hecho, muchos visualizadores tienen botones o elementos de menú para simplificar la lectura de las noticias USENET todavía más que usando los lectores de noticias estándar.

Se reconocen dos formatos para el protocolo *news*. El primero especifica un grupo de noticias y puede usarse para obtener una lista de artículos de una instalación de noticias preconfigurada. El segundo requiere que se indique el identificador de un artículo específico, en este caso *AA0134223112@cs.utah.edu*. El visualizador entonces trae el artículo dado de su instalación preconfigurada de noticias usando el protocolo NNTP.

El protocolo *gopher* corresponde al sistema Gopher, que se diseñó en la Universidad de Minnesota y se bautizó por los equipos atléticos de la escuela, los Golden Gophers (también es una expresión en jerga que significa "*go for*", es decir, ve y trae). Gopher es anterior a la *Web* por varios años. Se trata de un esquema de recuperación de información, conceptualmente parecido a la *Web* misma, pero que sólo maneja texto y no imágenes. Cuando un usuario se conecta con un servidor Gopher, se le presenta un menú de archivos y directorios, cualquiera de los cuales puede vincularse a otro menú Gopher de cualquier lugar del mundo.

La gran ventaja del Gopher sobre la *Web* es que funciona muy bien con las terminales ASCII de 25 × 80, de las que todavía quedan bastantes, y, dado que está basado en texto, es muy rápido. En consecuencia, hay miles de servidores Gopher en todo el mundo. Usando el protocolo *gopher*, los usuarios de la *Web* pueden acceder a Gopher y hacer que cada menú Gopher se presente como página de *Web* con capacidad de clic. Si no conoce Gopher, intente el ejemplo dado en la figura 7-63 o haga que su máquina de búsqueda de la *Web* busque "gopher".

Aunque el ejemplo dado no lo ilustra, también es posible enviar una consulta completa a un servidor Gopher usando el protocolo *gopher+*. Lo que se presenta es el resultado de hacer consultas en el servidor Gopher remoto.

Los dos últimos protocolos no tienen en realidad el sabor de la obtención de páginas de la *Web*, y no son reconocidos por todos los visualizadores, pero de todas maneras son útiles. El protocolo *mailto* permite a los usuarios enviar correo electrónico desde un visualizador de la *Web*. El procedimiento es hacer clic en el botón OPEN y especificar un URL que consista en *mailto:* seguido de la dirección de correo electrónico del destinatario. La mayoría de los visualizadores responderán presentando una forma que contiene espacios para el tema y otras líneas de cabecera, así como para el texto del mensaje.

El protocolo *telnet* sirve para establecer una conexión en línea con una máquina remota. Se usa de la misma manera que el programa Telnet, lo cual no es sorprendente, puesto que la mayoría de los visualizadores simplemente llaman al programa Telnet como aplicación ayudante. Como ejercicio, intente de nuevo el diálogo de la figura 7-60, pero usando ahora un visualizador de la *Web*.

En pocas palabras, los URL se han diseñado no sólo para permitir a los usuarios navegar por la *Web*, sino también para entenderse con FTP, noticias, Gopher, correo electrónico y Telnet, haciendo innecesarios los programas especializados de interfaz de usuario para esos otros servicios, e integrando por tanto casi todos los accesos a Internet en un solo programa, el visualizador de la *Web*. Si no fuera por el hecho de que este esquema fue diseñado por un investigador de física, podría fácilmente pensarse que lo creó el departamento de publicidad de una compañía de *software*.

A pesar de todas estas agradables propiedades, el uso creciente de la *Web* ha sacado a la luz una debilidad inherente del esquema URL. Un URL apunta a un *host* específico. En caso de

páginas a las que se hace referencia constante, sería deseable tener varias copias muy distantes, para reducir el tráfico de la red. El problema es que los URL no proporcionan ninguna manera de referirse a una página sin decir simultáneamente dónde está. No hay manera de decir: "quiero la página xyz, pero no me importa de dónde la traigas". Para resolver este problema y hacer posible la duplicación de páginas, el IETF está trabajando en un **sistema de identificadores universales** de recursos, o **URI** (*Universal Resource Identifiers*). Puede pensarse en un URI como un URL generalizado. Este tema es el objeto de numerosas investigaciones.

Aunque hemos estudiado sólo los URL absolutos, también existen los URL relativos. La diferencia es análoga a la diferencia entre el nombre de archivo absoluto */usr/ast/foobar* y simplemente *foobar* cuando el contexto está definido sin ambigüedades.

HTML — Lenguaje de marcaje de hipertexto

Ahora que tenemos una buena idea del funcionamiento de los URL, es tiempo de ver el HTML mismo. El HTML es una aplicación del estándar ISO 8879, **SGML** (*Standard Generalized Markup Language*, **lenguaje de marcación estándar generalizado**), pero especializado en hipertexto y adaptado a la *Web*.

Como se mencionó antes, el HTML es un lenguaje de marcación, un lenguaje para describir la manera en que debe formatearse un documento. El término "marcación" viene de los viejos días cuando los revisores de texto en realidad marcaban los documentos para indicarle al formador de impresión (en aquellos tiempos un ser humano) las fuentes a usar y otras cosas. Los lenguajes de marcación contienen, por tanto, comandos explícitos para el formateo. Por ejemplo, en HTML <B> significa **inicia modo de negritas** y </B> significa **termina modo de negritas**. La ventaja de un lenguaje de marcación sobre uno que no tiene marcación explícita es que la escritura de un visualizador para él es cosa directa: el visualizador simplemente tiene que entender los comandos de marcación. TeX y troff son otros ejemplos bien conocidos de lenguajes de marcación.

Los documentos escritos en un lenguaje de marcación se pueden contrastar con los documentos producidos con un procesador de textos WYSIWYG (*What You See Is What You Get*, lo que se ve es lo que se recibe), como MS-Word® o WordPerfect®. Estos sistemas pueden almacenar sus archivos con una marcación oculta integrada para poder reproducirlos después, pero no todos funcionan de esta manera. Los procesadores de textos para Macintosh, por ejemplo, mantienen la información de formateo en estructuras de datos aparte, no como comandos integrados en los archivos de usuario.

Al integrar los comandos de marcación dentro de cada archivo HTML y estandarizarlos, se hace posible que cualquier visualizador de la *Web* lea y reformatee cualquier página de *Web*. La capacidad de reformatear las páginas de *Web* tras su recepción es crucial porque una página puede haberse producido en una ventana de pantalla completa en una pantalla de 1024 × 768 con color de 24 bits, pero puede tener que presentarse en una ventana pequeña de una pantalla de 640 × 480 con color de 8 bits. Los procesadores de textos WYSIWYG patentados no pueden usarse en la *Web* porque sus lenguajes de marcación internos (si los tienen) no están estandarizados entre los proveedores, máquinas y sistemas operativos, y tampoco manejan reformateo para

ventanas de diferentes tamaños y pantallas de diferentes definiciones. Sin embargo, los programas de procesamiento de textos pueden ofrecer la opción de guardar documentos en HTML en lugar del formato propietario del proveedor, y algunos de hecho ya lo hacen.

Como el HTTP, el HTML está en constante estado de cambio. Cuando Mosaic era el único visualizador, el lenguaje que interpretaba, el HTML 1.0, era el estándar de facto. Cuando llegaron nuevos visualizadores, hubo la necesidad de un estándar Internet formal, por lo que se generó el estándar HTML 2.0. El HTML 3.0 se creó inicialmente como esfuerzo de investigación para agregar muchas características nuevas al HTML 2.0, incluidas tablas, barras de herramientas, fórmulas matemáticas, hojas de estilos avanzados (para definir la distribución de las páginas y el significado de los símbolos), y otras cosas.

La estandarización oficial del HTML está siendo administrada por el WWW Consortium, pero varios proveedores de visualizadores han agregado sus propias extensiones *ad hoc*. Estos proveedores esperan lograr que la gente escriba páginas de *Web* usando sus extensiones, a fin de que los lectores de estas páginas necesiten el visualizador del proveedor para interpretar adecuadamente las páginas. Esta tendencia no simplifica la estandarizacion del HTML.

A continuación haremos una introducción breve al HTML, sólo para dar una idea de lo que es. Aunque ciertamente es posible escribir documentos HTML con cualquier editor estándar (y mucha gente lo hace), también es posible usar editores HTML especiales que pueden hacer la mayoría del trabajo (pero que por lo mismo dan al usuario menos control sobre todos los detalles del resultado final).

Una página de *Web* decorosa consiste en una cabecera y un cuerpo encerrado entre **etiquetas** (comandos de formateo) <HTML> y </HTML>, aunque la mayoría de los visualizadores no se quejan si faltan estas etiquetas. Como puede verse en la figura 7-64(a), la cabecera está delimitada por las etiquetas <HEAD> y </HEAD>, y el cuerpo, por las etiquetas <BODY> y </BODY>. Los comandos dentro de las etiquetas se llaman **directivas**. La mayoría de las etiquetas HTML tienen este formato, es decir, <ALGO> para marcar el comienzo de algo, y </ALGO> para marcar su fin. Hay numerosos ejemplos más en HTML. La mayoría de los visualizadores tienen un elemento de menú VIEW SOURCE o algo parecido. Su selección presenta el código fuente HTML de las páginas actuales, en lugar de su salida formateada.

Las etiquetas pueden escribirse tanto en minúsculas como en mayúsculas. Por tanto, <HEAD> y <head> significan la misma cosa, pero el primero resalta mejor para la mayoría de los lectores. La distribución real del documento HTML no es importante. Los analizadores ignoran los espacios extra y los retornos de carro, puesto que tienen que reformatear el texto para acomodarlo en el área de presentación. En consecuencia, pueden agregarse espacios virtuales a voluntad para hacer más legibles los documentos HTML, algo que la mayoría necesita urgentemente. Como consecuencia adicional, no pueden usarse líneas en blanco para separar párrafos, puesto que simplemente se ignoran. Se requiere una etiqueta específica.

Algunas etiquetas tienen parámetros (nombrados). Por ejemplo

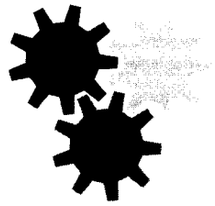
```
<IMG SRC"abc" ALT="foobar">
```

es una etiqueta, <IMG>, con el parámetro *SRC* puesto a *abc* y el parámetro *ALT* igual a *foobar*. Para cada etiqueta, el estándar HTML da una lista de los parámetros permitidos, si los hay, y lo


```
<HTML> <HEAD> <TITLE> ADMINÍCULOS AMALGAMADOS, S.A. </TITLE> </HEAD>
<BODY> <H1>  Bienvenidos a la página base de AASA </H1>
<IMG SRC="http://www.widget.com/images/logo.gif" ALT="AWI Logo"> <BR>
Estamos muy contentos de que haya solicitado visitar la página base de <B> Adminículos
Amalgamados </B>. Esperamos que <I> usted </I> encuentre aquí toda la información que necesita.
<P> A continuación presentamos vínculos con la información sobre nuestro surtido de productos finos.
Puede ordenar electrónicamente (por WWW), por teléfono o por fax. <HR>
<H2> Información sobre nuestros productos </H2>
<UL> <LI> <A HREF="http://widget.com/products/big">  Adminículos grandes  </A>
     <LI> <A HREF="http://widget.com/products/little">  Adminículos pequeños  </A>
</UL>
<H2> Números telefónicos </H2>
<UL> <LI> Teléfono: 1-800-ADMINIC
     <LI> Fax: 1-415-765-4321
</UL> </CUERPO> </HTML>
```

(a)

Bienvenidos a la página base de AASA



Estamos muy contentos de que haya solicitado visitar la página base de **Adminículos Amalgamados**. Esperamos que *usted* encuentre aquí toda la información que necesita.

A continuación presentamos vínculos con la información sobre nuestro surtido de productos finos. Puede ordenar electrónicamente (por WWW), por teléfono o por fax.

Información sobre nuestros productos

- Adminículos grandes
- Adminículos pequeños

Números telefónicos

- Teléfono: 1-800-ADMINIC
- Fax: 1-415-765-4321

(b)

Figura 7-64. (a) HTML para una página de *Web* de ejemplo. (b) Página con formato.

que significan. Puesto que cada parámetro se nombra, el orden en que se dan los parámetros no es de importancia.

Técnicamente, los documentos se escriben en el conjunto de caracteres Latin-1 del ISO 8859-1, pero para los usuarios cuyos teclados sólo reconocen ASCII, hay secuencias de escape para los caracteres especiales, como è. La lista de caracteres especiales se proporciona en el estándar. Todos esos caracteres comienzan con un signo & y terminan con un punto y coma. Por ejemplo, è produce è y é produce é. Puesto que <, >, y & tienen significados especiales, pueden expresarse sólo con sus secuencias de escape, < > y & respectivamente.

El elemento principal de la cabecera es el título, delimitado por <TITLE> y </TITLE>, pero pueden estar presentes también ciertos tipos de metainformación. El título mismo no se presenta en la página. Algunos visualizadores lo usan para rotular la ventana de la página.

Veamos ahora algunas de las otras características mostradas en la figura 7-64. Todas las etiquetas usadas en esa figura y algunas más se presentan en la figura 7-65. Las cabeceras se generan con una etiqueta <H*n*>, donde *n* es un dígito del intervalo 1 a 6. <H1> es la cabecera más importante; <H6> la menos importante. Es responsabilidad del visualizador presentarlas de manera adecuada en la pantalla. Comúnmente, las cabeceras de número menor se presentarán con una fuente más grande y gruesa. El visualizador puede también seleccionar colores distintos para cada nivel de cabecera. Típicamente, las cabeceras <H1> son grandes y en negritas, con cuando menos una línea en blanco arriba y abajo. En contraste, las cabeceras <H2> tienen una fuente más pequeña y con menos espacio arriba y abajo.

Las etiquetas <B> e <I> se usan para entrar en el modo de negritas y el de cursivas, respectivamente. Si el visualizador no es capaz de presentar negritas y cursivas, debe usar algún otro método para mostrarlas (por ejemplo, un color diferente para cada una o tal vez vídeo inverso). En lugar de especificar estilos físicos como negritas y cursivas, los autores pueden usar estilos lógicos como <DN> (definir), <EM> (énfasis débil), <STRONG> (énfasis fuerte) y <VAR> (variables de programa). Los estilos lógicos se definen en un documento llamado **hoja de estilos**. La ventaja de los estilos lógicos es que, cambiando una definición, pueden cambiarse todas las variables, por ejemplo, de cursivas a una fuente de paso fijo.

El HTML proporciona varios mecanismos para hacer las listas, incluidas listas anidadas. La etiqueta <UL> comienza una lista desordenada. Los elementos individuales, que se marcan con la etiqueta <LI> en el fuente, aparecen con viñetas (•) al principio. Una variante de este mecanismo es <OL>, que es para listas ordenadas. Cuando se usa esta etiqueta, los elementos <LI> son numerados por el visualizador. Una tercera opción es <MENU>, que generalmente produce una lista más compacta en la pantalla, sin viñetas y sin números. Aparte del uso de diferentes etiquetas de inicio y fin, <UL>, <OL> y <MENU> tienen la misma sintaxis y resultados parecidos.

Además de los mecanismos de listas mostrados en la figura 7-65, hay dos más que vale la pena mencionar. <DIR> puede usarse para hacer tablas cortas. También <DL> y </DL> pueden crear listas de definición (glosarios) con entradas de dos partes, que se identifican con <DT> y <DD> respectivamente. La primera es para el nombre, la segunda para su significado. Estas características son sobreseídas en buena medida por el mecanismo de tablas (más general y complejo), que se describe a continuación.

Etiqueta	Descripción
<HTML> ... </HTML>	Declara que la página de *Web* está escrita en HTML
<HEAD> ... </HEAD>	Delimita la cabecera de la página
<TITLE> ... </TITLE>	Delimita el título (no se presenta en la página)
<BODY> ... </BODY>	Delimita el cuerpo de la página
<H*n*> ... </H*n*>	Delimita una cabecera nivel *n*
<B> ... </B>	Pone ... en negritas
<I> ... </I>	Pone ... en cursivas
<UL> ... </UL>	Corchetes de una lista desordenada (con viñetas)
<OL> ... </OL>	Corchetes de una lista numerada
<MENU> ... </MENU>	Corchetes de un menú de <LI> elementos
<LI>	Comienzo de una lista (no hay </LI>)
 	Obliga salto de renglón aquí
<P>	Comienzo de párrafo
<HR>	Pleca horizontal
<PRE> ... </PRE>	Texto preformateado; no reformatear
<IMG SRC="...">	Carga una imagen aquí
<A HREF="...">...</A>	Define un hipervínculo

Figura 7-65. Selección de etiquetas HTML comunes. Algunas tienen parámetros adicionales.

Las etiquetas <BR>, <P> y <HR> indican límites entre secciones del texto. El formato preciso puede determinarse por la hoja de estilos asociada a la página. La etiqueta <BR> simplemente fuerza una línea nueva. Por lo común, los visualizadores no insertan una línea en blanco tras una <BR>. En contraste, <P> comienza un párrafo y podría, por ejemplo, insertar una línea en blanco y posiblemente una sangría. (En teoría, </P> existe para marcar el fin del párrafo, pero pocas veces se usa; la mayoría de los autores del HTML ni siquiera saben que existe.) Por último, <HR> fuerza una línea nueva y dibuja una línea horizontal a lo ancho de la pantalla.

El HTML 1.0 no tenía capacidad para presentar tablas u otra información formateada. Peor aún, si el escritor de HTML formateaba cuidadosamente una tabla mediante el uso juicioso de espacios y retornos de carro, los visualizadores ignoraban toda la distribución y presentaban la página como si todo el material formateado estuviera sin formatear. Para evitar que los visualizadores alteraran un texto cuidadosamente distribuido, se incluyeron las etiquetas <PRE> y </PRE>; son instrucciones al visualizador para que simplemente presente de manera literal todo lo que está en medio, carácter por carácter, sin cambiar nada. A medida que las tablas y otras características rebuscadas de distribución tengan mayor implementación, disminuirá la necesidad de <PRE>,

excepto para los listados de programa, en los que la mayoría de los programadores no tolerará ningún formateo más que el suyo.

El HTML permite la inclusión de imágenes en línea en una página de *Web*. La etiqueta <IMG> especifica que se cargará una imagen en la posición actual de la página; puede tener varios parámetros. El parámetro *SRC* indica el URL (o URI) de la imagen. El estándar HTML no especifica los formatos gráficos permitidos. En la práctica, todos los visualizadores reconocen archivos GIF y muchos reconocen también archivos JPEG. Los visualizadores pueden reconocer otros formatos, pero esta extensión es un arma de doble filo. Si un usuario está acostumbrado a un visualizador que reconoce, digamos, archivos BMP, puede incluir éstos en sus páginas de *Web* y luego sorprenderse de que otros visualizadores simplemente ignoren todo su maravilloso arte.

Otros parámetros de <IMG> son *ALIGN*, que controla la alineación de la imagen con respecto a la línea base del texto (*TOP, MIDDLE, BOTTOM*), *ALT*, que proporciona texto a usar en lugar de la imagen cuando el usuario ha inhabilitado las imágenes, e *ISMAP*, un indicador de que la imagen es un mapa activo.

Por último, llegamos a los hipervínculos, que usan las etiquetas <A> (ancla) y </A>. Al igual que <IMG>, <A> tiene varios parámetros, incluidos *HREF* (el URL), *NAME* (el nombre del hipervínculo) y *METHODS* (métodos de acceso), entre otros. Se presenta el texto entre <A> y </A>. Si se selecciona, se sigue el hipervínculo a una nueva página. También se permite poner una imagen <IMG> ahí, en cuyo caso hacer clic en la imagen también activa el hipervínculo.

Como ejemplo, considere el siguiente fragmento HTML:

```
<A HREF="http://www.nasa.gov"> Página base de la NASA </A>
```

Cuando se presenta una página con este fragmento, lo que aparece en la ventana es

Página base de la NASA

Si el usuario hace clic en este texto, el visualizador de inmediato trae la página cuya URL es *http://www.nasa.gov* y la presenta.

Como segundo ejemplo, considere ahora

```
<A HREF="http://www.nasa.gov"> <IMG SRC="shuttle.gif" ALT="NASA"> </A>
```

Al presentarse, esta página muestra una fotografía (por ejemplo, del transbordador espacial). Al hacer clic en la foto se pasa a la página base de la NASA, igual que al hacer clic en el texto subrayado en el ejemplo anterior. Si el usuario inhabilita la presentación automática de imágenes, el texto NASA se presentará donde iría la fotografía.

La etiqueta <A> puede aceptar un parámetro *NAME* para plantar un hipervínculo, de modo que pueda hacerse referencia a él desde la página. Por ejemplo, algunas páginas de *Web* comienzan con un índice de materias en el que se puede hacer clic. Al hacer clic en un elemento del índice de materias, el usuario salta a la sección correspondiente de la página.

Una característica que el HTML 2.0 no incluyó, y que extrañaron muchos autores de páginas, fue la capacidad de crear tablas a cuyas entradas pudiese hacerse clic para activar hipervínculos.

Como consecuencia, se trabajó mucho para agregar tablas al HTML 3.0. A continuación haremos una introducción muy breve a las tablas, sólo para captar su sabor básico.

Una tabla HTML consiste en una o más filas, teniendo cada una una o más **celdas**. Las celdas pueden contener una gran variedad de material, incluidos texto, figuras e inclusive otras tablas. Las celdas pueden unirse, de modo que, por ejemplo, una cabecera puede abarcar varias columnas. Los autores de páginas tienen control limitado sobre la distribución, lo que incluye alineación, estilos de bordes y márgenes de celdas, pero los visualizadores tienen la última palabra al presentar las tablas.

En la figura 7-66(a) se lista una definición de tabla HTML, y en la figura 7-66(b) se muestra una posible presentación. Este ejemplo sólo ilustra unas cuantas características básicas de las tablas HTML. Las tablas comienzan con la etiqueta <TABLE>. Puede proporcionarse información adicional para describir las propiedades generales de la tabla.

La etiqueta <CAPTION> puede usarse para proporcionar un pie de figura. Cada fila comienza con una etiqueta <TR> (fila de tabla). Las celdas individuales se marcan como <TH> (cabecera de tabla) o <TD> (datos de tabla). La distinción se hace para permitir que los visualizadores usen diferentes presentaciones para ellas, como hemos hecho en el ejemplo.

Se permiten muchas otras etiquetas en las tablas, que incluyen maneras de especificar alineaciones horizontales y verticales de las celdas, justificaciones en las celdas, bordes, agrupamiento de celdas, unidades y otras cosas.

Formas

El HTML 1.0 básicamente era de un solo sentido. Los usuarios podían llamar las páginas de los proveedores de información, pero era difícil enviar información en el otro sentido. A medida que más y más organizaciones comerciales comenzaron a usar la *Web*, creció la demanda del tráfico de dos vías. Por ejemplo, muchas compañías querían poder tomar pedidos de productos mediante sus páginas de *Web*, los proveedores de *software* querían distribuir *software* por medio de la *Web* y hacer que sus clientes llenaran sus tarjetas de registro electrónicamente, y las compañías que ofrecían búsquedas en la *Web* querían que sus clientes pudieran indicar claves de búsqueda.

Estas demandas condujeron a la inclusión de **formas** a partir del HTML 2.0. Las formas contienen marcos o botones que permiten a los usuarios proporcionar información o tomar decisiones, y luego devuelven la información al dueño de la página. Las formas usan la etiqueta <INPUT> para este propósito. Esta etiqueta tiene una variedad de parámetros para determinar el tamaño, naturaleza y uso del marco presentado. Las formas más comunes son campos en blanco para aceptar texto del usuario, marcos que pueden marcarse, mapas activos y botones SUBMIT. El ejemplo de la figura 7-67 ilustra algunas de estas opciones.

Comencemos nuestro estudio de las formas repasando este ejemplo. Como todas las formas, ésta se encierra entre las etiquetas <FORM> y </FORM>. El texto no delimitado por etiquetas simplemente se presenta. Se permiten todas las etiquetas normales (por ejemplo, <B>) en una forma. Se usan tres tipos de marcos de entrada en esta forma.

El primer tipo de marco de entrada sigue al texto "Nombre". El marco tiene 46 caracteres de longitud y espera que el usuario escriba una cadena, que luego se almacena en la variable

```
<HTML> <HEAD> <TITLE> Página de ejemplo con tabla </TITLE> </HEAD>
<BODY>
<TABLE BORDER=ALL RULES=ALL>
<CAPTION> Algunas diferencias entre versiones de HTML </CAPTION>
<COL ALIGN=LEFT>
<COL ALIGN=CENTER>
<COL ALIGN=CENTER>
<COL ALIGN=CENTER>
<TR> <TH> Elemento <TH>HTML 1.0 <TH>HTML 2.0 <TH>HTML 3.0
<TR> <TH> Mapas e imágenes activos <TD> <TD> x <TD> x
<TR> <TH> Ecuaciones <TD> <TD> <TD> x
<TR> <TH> Formas <TD> <TD> x <TD> x
<TR> <TH> Hipervínculos <TD> x <TD> x <TD> x
<TR> <TH> Imágenes <TD> x <TD> x <TD> x
<TR> <TH> Listas <TD> x <TD> x <TD> x
<TR> <TH> Barras de herramientas <TD> <TD> <TD> x
<TR> <TH> Tablas <TD> <TD> <TD> x
</TABLE> </BODY> </HTML>
```

(a)

Algunas diferencias entre versiones de HTML

Elemento	HTML 1.0	HTML 2.0	HTML 3.0
Mapas e imágenes activos		X	X
Ecuaciones			X
Formas		X	X
Hipervínculos	X	X	X
Imágenes	X	X	X
Listas	X	X	X
Barras de herramientas			X
Tablas			X

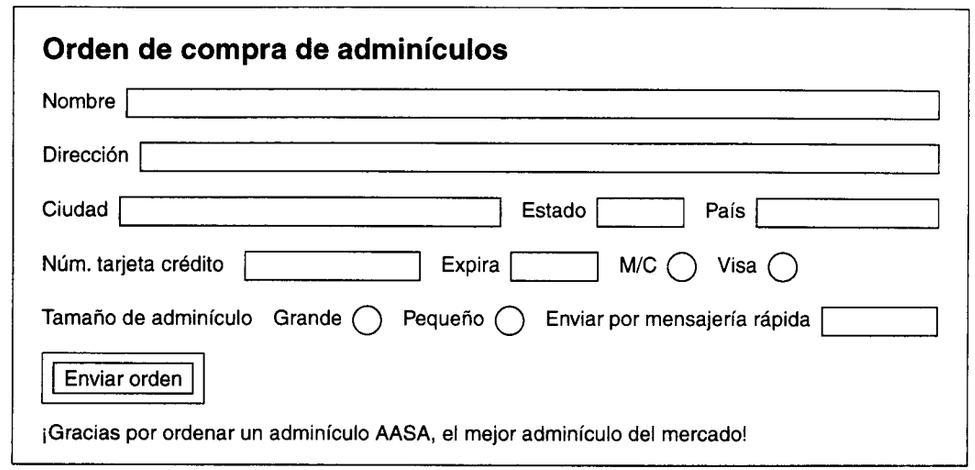
Figura 7-66. (a) Tabla HTML de ejemplo. (b) Posible presentación de esta tabla.

customer para procesarse posteriormente. La etiqueta <P> instruye al visualizador que presente el texto y marcos subsiguientes en la siguiente línea, aunque haya espacio en la línea actual. Usando <P> y otras etiquetas de distribución, el autor de la página puede controlar la manera en que se ve la forma en la pantalla.

La siguiente línea de la forma solicita la dirección del usuario, de 40 columnas de ancho, también en una línea individual. Luego viene una línea que solicita la ciudad, el estado y el país. Aquí no se usan etiquetas <P> entre los campos, por lo que el visualizador los presentará todos en la misma línea, si caben. En lo que concierne al visualizador, este párrafo simplemente contiene seis elementos: tres cadenas que alternan con tres marcos. Presenta todo esto de manera


```
<HTML> <HEAD> <TITLE> FORMA DE PEDIDO DE CLIENTES AASA </TITLE> </HEAD>
<BODY>
<H1> Orden de compra de adminículos </H1>
<FORM ACTION="http://widget.com/cgi-bin/widgetorder" METHOD=POST>
Nombre <INPUT NAME="customer" SIZE=46> <P>
Dirección <INPUT NAME="address" SIZE=40> <P>
Ciudad <INPUT NAME="city" SIZE=20> Estado <INPUT NAME="state" SIZE =4>
País <INPUT NAME="country" SIZE=10> <P>
Núm. tarjeta crédito <INPUT NAME="cardno" SIZE=10>
Expira <INPUT NAME="expires" SIZE=4>
M/C <INPUT NAME="cc" TYPE=RADIO VALUE="mastercard">
VISA <INPUT NAME="cc" TYPE=RADIO VALUE="visacard"> <P>
Tamaño de adminículo Grande <INPUT NAME="product" TYPE=RADIO VALUE="expensive">
Pequeño <INPUT NAME="product" TYPE=RADIO VALUE="cheap">
Enviar por mensajería rápida <INPUT NAME="express" TYPE=CHECKBOX> <P>
<INPUT TYPE=SUBMIT VALUE="Enviar orden"> <P>
¡Gracias por ordenar un adminículo AASA, el mejor adminículo del mercado!
</FORM> </BODY> </HTML>
```

(a)



(b)

Figura 7-67. (a) Código HTML para una forma de pedido. (b) Página formateada.

lineal de izquierda a derecha, pasando a una línea nueva cada vez que la línea actual ya no pueda contener el siguiente elemento. Por tanto, es posible que, en una pantalla de 1024×768, las tres cadenas y sus marcos correspondientes aparezcan en la misma línea, pero en una pantalla de 640×480 se dividan en dos líneas. En el peor caso, la palabra "País" está al final de una línea y su marco al principio de la siguiente. No hay manera de decirle al visualizador que obligue a que el marco esté adyacente al texto.

La siguiente línea solicita el número de tarjeta de crédito y su fecha de expiración. La transmisión de números de tarjeta de crédito por Internet sólo debe hacerse cuando se han tomado las medidas de seguridad adecuadas. Por ejemplo, algunos, pero no todos, los visualizadores cifran la información enviada por los usuarios. Aun entonces, la comunicación segura y la administración de claves son asuntos complicados y están sujetos a muchos tipos de ataques, como vimos antes.

A continuación de la fecha de expiración encontramos una característica nueva: botones de radio. Éstos se usan cuando debe tomarse una decisión entre dos o más alternativas. El modelo intelectual aquí es un radio de automóvil con media docena de botones para seleccionar estaciones. El visualizador presenta estas opciones de una forma que permite que el usuario las seleccione y las deseleccione haciendo clic en ellas (o usando el teclado). Al hacer clic en una de las opciones, se apagan todas los demás del mismo grupo. La presentación visual depende de la interfaz gráfica que se usa. Es responsabilidad del visualizador escoger una forma consistente con Windows, Motif, OS/2 Warp, o cualquier sistema de ventanas que se maneje. El tamaño del adminículo también usa dos botones de radio. Los dos grupos se distinguen por su campo *NAME*, no por alcance estático usando algo como <RADIOBUTTON> ... </RADIOBUTTON>.

Los parámetros de *VALUE* sirven para indicar el botón de radio que se ha presionado. Dependiendo de las opciones de tarjeta de crédito que haya seleccionado el usuario, a la variable *cc* se asignará la cadena "mastercard" o a la cadena "visacard".

Después de los dos grupos de botones de radio, llegamos a la opción de embarque, representada por un marco de tipo *CHECKBOX*, que puede estar encendido o apagado. A diferencia de los botones de radio, de los que sólo se puede seleccionar uno del grupo, cada marco de tipo *CHECKBOX* puede estar encendido o apagado, independientemente de los demás. Por ejemplo, al ordenar una pizza mediante la página de *Web* de Electropizza, el usuario puede seleccionar sardinas *y* cebolla *y* piña (si se atreve), pero no puede seleccionar pequeña *y* mediana *y* grande para la misma pizza. Los ingredientes de la pizza se representarían con tres marcos independientes del tipo *CHECKBOX*, mientras que el tamaño de la pizza sería un grupo de botones de radio.

Como nota, en las listas muy largas en las que debe hacerse una selección, los botones de radio son poco prácticos. Por tanto, se proporcionan las etiquetas <SELECT> y </SELECT> para delimitar una lista de alternativas, pero con la semántica de los botones de radio (a menos que se dé el parámetro *MULTIPLE*, en cuyo caso la semántica es la de los marcos marcables). Algunos visualizadores presentan los elementos entre <SELECT> y </SELECT> como menú desplegable.

Hemos visto dos de los tipos interconstruidos de la etiqueta <INPUT>: *RADIO* y *CHECKBOX*. De hecho, ya hemos visto también un tercero: *TEXT*. Como este tipo es el predeterminado, no nos molestamos en incluir el parámetro *TYPE = TEXT*, pero podríamos haberlo hecho. Otros dos tipos son *PASSWORD* y *TEXTAREA*. Un marco *PASSWORD* es igual a un marco *TEXT*, excepto que los caracteres no se presentan cuando se escriben. Un marco *TEXTAREA* es igual a un marco *TEXT*, excepto que puede contener varias líneas.

Regresando al ejemplo de la figura 7-67, ahora nos topamos con un ejemplo de botón *SUBMIT*. Al hacer clic en él, la información del usuario escrita en la forma se envía a la máquina

que proporcionó la forma. Como los demás tipos, *SUBMIT* es una palabra reservada que el visualizador entiende. La cadena *VALUE* aquí es la etiqueta del botón, y se presenta. Todos los marcos pueden tener valores; únicamente necesitamos esa característica aquí. En el caso de los marcos *TEXT*, el contenido del campo *VALUE* se presenta junto con la forma, pero el usuario puede modificarlo o borrarlo. Los marcos *CHECKBOX* y *RADIO* también pueden inicializarse, pero con un campo llamado *CHECKED* (puesto que *VALUE* simplemente da el texto, pero no indica una selección preferida).

El visualizador también entiende el botón *RESET*. Si se hace clic en él, la forma se restablece a su estado inicial.

Vale la pena mencionar dos tipos más. El primero es el tipo *HIDDEN*, que sólo es una salida; no puede hacerse clic en él ni modificarse. Por ejemplo, al trabajar con una serie de páginas en las cuales deben hacerse selecciones, las selecciones previas podrían ser del tipo *HIDDEN*, para evitar que se cambien.

Nuestro último tipo es *IMAGE*, que es para mapas activos (y otras imágenes en las que se puede hacer clic). Cuando el usuario hace clic en el mapa, las coordenadas (*x, y*) del píxel seleccionado (es decir, la posición actual del ratón) se almacenan en variables y la forma se devuelve automáticamente al dueño para su procesamiento posterior.

Las formas pueden devolverse de tres maneras: usando el botón *SUBMIT*, haciendo clic en un mapa activo, o tecleando ENTER en una forma *TEXT* de un solo elemento. Cuando se devuelve una forma, debe realizarse alguna acción. La acción se especifica mediante los parámetros de la etiqueta <FORM>. El parámetro *ACTION* especifica el URL (o URI) al que debe informarse de la devolución, y el parámetro *METHOD* indica el método a usar. El orden de estos (y todos los demás) parámetros no es significativo.

La manera en que las variables de la forma se devuelven al dueño de la página depende del valor del parámetro *METHOD*. Con *GET*, la única manera de devolver valores es haciendo trampa: se agregan al URL, separados por un signo de interrogación. Este enfoque puede hacer que una URL tenga miles de caracteres de longitud. No obstante, este método se usa con frecuencia porque es sencillo.

Si se usa el método *POST* (véase la figura 7-62), el cuerpo del mensaje contiene las variables de la forma y sus valores. Se usa & para separar los campos; el + representa el carácter de espacio. Por ejemplo, la respuesta a la forma de adminículos podría ser:

```
customer=John+Doe&address=100+Main+St.&city=White+Plains&
state=NY&country=USA&cardno=1234567890&expires=6/98&cc=mastercard&
product=cheap&express=on
```

La cadena regresaría al servidor como una línea, no tres. Si no se selecciona un *CHECKBOX*, se omite de la cadena. Es responsabilidad del servidor darle sentido a esta cadena.

Afortunadamente ya hay disponible un método estándar de manejo de los datos de una forma. Se llama **CGI** (*Common Gateway Interface*, **interfaz de pasarela común**). Consideremos una manera común de usarla. Supóngase que alguien tiene una base de datos interesante

(por ejemplo, un índice de las páginas de *Web* por clave y tema) y quiere ponerla a disposición de los usuarios de la *Web*. La manera CGI de hacer disponible la base de datos es escribir un guión (o programa) que hace las veces de interfaz (pasarela) entre la base de datos y la *Web*. A este guión se le asigna un URL, que por convención está en el directorio *cgi-bin*. Los servidores HTTP saben (o se les puede decir) que cuando tengan que invocar un método de una página localizada en *cgi-bin*, deberán interpretar el nombre de archivo como si fuera un guión o programa ejecutable, e iniciarlo.

Tarde o temprano, algún usuario abrirá la forma asociada a nuestro guión de adminículos y lo visualiza. Una vez llenada la forma, el usuario hará clic en el botón SUBMIT. Esta acción causa que el visualizador establezca una conexión TCP con el URL listado en el parámetro *ACTION* de la forma (el guión del directorio *cgi-bin*). Entonces el visualizador invocará la operación especificada por el *METHOD* de la forma, generalmente *POST*. El resultado de esta operación es que se inicia el guión y se presenta (por medio de la conexión TCP, en la entrada estándar) con la cadena grande dada antes. Además, se establecen varias variables de entorno. Por ejemplo, la variable de entorno *CONTENT_LENGTH* indica la longitud de la cadena de entrada.

En este punto, la mayoría de los guiones necesitan analizar su entrada para ponerla en un formato más conveniente. Esto puede lograrse llamando a una de las muchas bibliotecas o procedimientos de guión disponibles. El guión puede entonces interactuar con su base de datos de la manera que desee. Por ejemplo, los mapas activos normalmente usan guiones CGI para emprender diferentes acciones dependiendo del lugar al que apuntó el usuario.

Los guiones CGI también pueden producir salidas y hacer muchas otras cosas además de aceptar entradas de las formas. Si un hipervínculo apunta a un guión CGI, al invocarse ese vínculo se inicia el guión, con varias variables de entorno establecidas para proporcionar información referente al usuario. El guión escribe entonces un archivo (por ejemplo, una página HTML) en la salida estándar, quede donde se envía al visualizador, que lo interpreta. Este mecanismo hace posible que el guión genere páginas de *Web* a la medida en el momento.

Para bien o para mal, algunas instalaciones de la *Web* que contestan consultas tienen una base de datos de anuncios que pueden incluirse selectivamente en la página de *Web* que se construye, dependiendo de lo que busca el usuario. Si el usuario busca "automóvil", podría presentarse un anuncio de General Motors, y si se está buscando "vacaciones", podría aparecer un anuncio de United Airlines. Estos anuncios generalmente incluyen texto e imágenes en las que puede hacerse clic.

7.6.4. Java

El HTML permite describir la manera en que deben exhibirse las páginas estáticas de la *Web*, incluidas tablas y fotografías. Con el recurso *cgi-bin*, también es posible tener una cantidad limitada de interacción de dos sentidos (formas, etc.). Sin embargo, no es posible la interacción rápida con las páginas de la *Web* escritas en HTML. Para hacer posibles páginas de *Web* altamente interactivas, se requiere un mecanismo diferente. En esta sección describiremos uno de tales mecanismos, el lenguaje e intérprete Java[MR].

Java se originó cuando algunas personas de Sun Microsystems trataban de desarrollar un lenguaje nuevo adecuado para programar aparatos caseros orientados a información. Después se reorientó el lenguaje hacia la *World Wide Web*. Aunque Java toma prestadas muchas ideas y algo de la sintaxis de C y C++, es un lenguaje nuevo orientado a objetos, incompatible con ambos. A veces se dice que, en grande, Java es como Smalltalk pero, en pequeño, es como C o C++.

La idea principal de usar Java para páginas de *Web* interactivas es que una página de *Web* puede apuntar a un programa Java pequeño, llamado **applet** (que podría traducirse como "aplicacioncita"). Cuando el visualizador llega a ella, la *applet* se baja a la máquina cliente y se ejecuta ahí de una manera segura. Debe ser estructuralmente imposible que la *applet* lea o escriba archivos que no está autorizada a acceder. Debe también ser imposible que la *applet* introduzca virus o cause algún otro daño. Por estas razones, y para lograr transportabilidad entre máquinas, las *applets* se compilan para crear un código de bytes después de escribirse y depurarse. Son estos programas en código de bytes a los que apuntan las páginas de *Web*, de manera parecida a como se apunta a las imágenes. Al llegar una *applet*, se ejecuta interpretativamente en un entorno seguro.

Antes de entrar en los detalles del lenguaje Java, vale la pena decir algunas palabras sobre la utilidad del sistema Java y las razones por las que la gente quiere incluir *applets* Java en sus páginas de *Web*. Por una parte, las *applets* permiten que las páginas de *Web* se vuelvan interactivas. Por ejemplo, una página de *Web* puede contener un tablero para jugar gato, Otelo o ajedrez, y jugar un juego con el usuario. El programa de juego (escrito en Java) se baja junto con su página de *Web*. Como segundo ejemplo, pueden presentarse formas complejas (por ejemplo, hojas de cálculo), llenando los usuarios elementos y viendo instantáneamente los cálculos.

Es posible que, a la larga, el modelo de gente que compra programas, los instala y los ejecuta localmente será reemplazado por un modelo en el que la gente hace clic en las páginas de *Web* y baja *applets* que trabajan para ella, posiblemente en colaboración con un servidor o base de datos remoto. En lugar de llenar la declaración de impuestos a mano o usando un programa especial, la gente podrá hacer clic en la página base de la autoridad hacendaria para bajar una *applet* de impuestos. Esta *applet* podría hacer algunas preguntas, luego comunicarse con el patrón, banco y corredor de bolsa de la persona a fin de reunir la información necesaria sobre sueldos, intereses y dividendos, llenar la forma y presentarla para verificación y envío.

Otra razón para ejecutar *applets* en la máquina cliente es que hace posible la adición de animación y sonido a las páginas de *Web* sin tener que llamar a visualizadores externos. El sonido puede reproducirse cuando se carga la página, como música de fondo, o cuando ocurre algún suceso específico (por ejemplo, hacer clic en el gato hace que maúlle). Ocurre lo mismo con la animación. Como la *applet* se ejecuta localmente, aun si se está interpretando, puede escribir en cualquier parte de la pantalla de la manera que quiera y a muy alta velocidad (en comparación con un guión *cgi-bin* remoto).

El sistema Java tiene tres partes:

1. Un compilador de Java a código de bytes.
2. Un visualizador que entiende *applets*.
3. Un intérprete de código de bytes.

El programador escribe la *applet* en Java, luego la compila, obteniendo código de bytes. Para incluir esta *applet* compilada en una página de *Web*, se ha inventado una nueva etiqueta HTML, <APPLET>. Un uso típico es:

```
<APPLET CODE=game.class WIDTH=100 HEIGHT=200> </APPLET>
```

Cuando el visualizador ve la etiqueta <APPLET>, trae la *applet* compilada *game.class* de la instalación de la página actual de la *Web* (o, si está presente otro parámetro, *CODEBASE*, del URL que especifica). El visualizador entonces pasa la *applet* al intérprete local de código de bytes para su ejecución (o interpreta la *applet* él mismo, si tiene un intérprete interno). Los parámetros *WIDTH* y *HEIGHT* dan el tamaño de la ventana predeterminada de la *applet*, en píxeles.

De alguna manera, la etiqueta <APPLET> es análoga a la etiqueta <IMG>. En ambos casos, el visualizador trae un archivo y luego lo entrega a un intérprete (posiblemente interno) para presentarlo en un área limitada de la pantalla; luego continúa procesando la página de *Web*.

Para algunas aplicaciones que requieren muy alto desempeño, algunos intérpretes Java tienen la capacidad de compilar programas de código de bytes a lenguaje máquina sobre la marcha, según se requiera.

Como consecuencia de este modelo, los visualizadores basados en Java son extensibles de una manera que no lo son los visualizadores de primera generación. Éstos básicamente son intérpretes HTML que tienen módulos interconstruidos para manejar los diferentes protocolos necesarios, como HTTP 1.0, FTP, etc., al igual que decodificadores de varios formatos de imagen. Se muestra un ejemplo en la figura 7-68(a). Si alguien inventa o populariza un formato nuevo, como audio o MPEG-2, estos visualizadores viejos no serán capaces de leer las páginas que los contienen. Cuando mucho, el usuario tendrá que encontrar, bajar e instalar un visualizador externo adecuado.

Con un visualizador basado en Java, la situación es diferente. Al arranque, el visualizador de hecho es una máquina virtual Java vacía, como se muestra en la figura 7-68(b). Al cargar *applets* HTML y HTTP, se vuelve capaz de leer páginas de *Web* estándar. Sin embargo, a medida que se requieren protocolos y decodificadores nuevos, sus clases se cargan de manera dinámica, posiblemente a través de la red desde instalaciones especificadas en las páginas de *Web*. Poco tiempo después, el visualizador podría verse como la figura 7-68(c).

Por tanto, si alguien inventa un formato nuevo, todo lo que tiene que hacer la persona es incluir el URL de una *applet* para manejarla en una página de *Web*, y el visualizador automáticamente traerá y cargará la *applet*. Ningún visualizador de primera generación es capaz de traer e instalar automáticamente visualizadores externos nuevos sobre la marcha. La capacidad de cargar

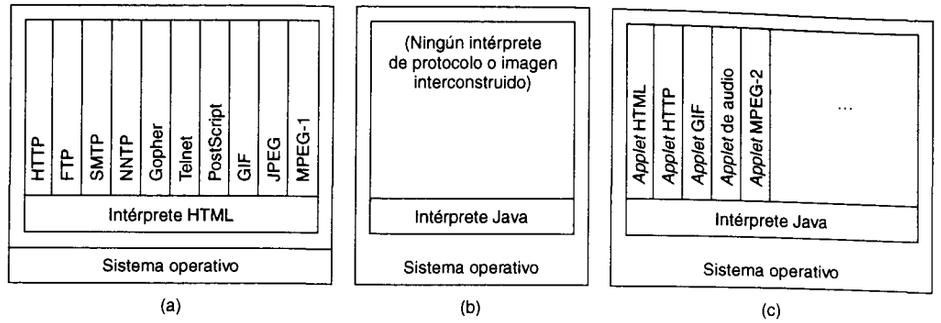


Figura 7-68. (a) Visualizador de primera generación. (b) Visualizador basado en Java al arranque. (c) El visualizador de (b) después de trabajar un rato.

visualizadores dinámicamente significa que la gente puede experimentar fácilmente con formatos nuevos sin primero tener que realizar interminables juntas de estandarización para llegar a un consenso.

Esta capacidad de extensión también se aplica a los protocolos. En algunas aplicaciones se requieren protocolos especiales, por ejemplo, protocolos seguros para aplicaciones bancarias y comerciales. Con Java, estos protocolos pueden cargarse dinámicamente según se requiera, y no hay necesidad de lograr una estandarización universal. Para comunicarse con la compañía X, simplemente bajamos su *applet* de protocolo. Para hablar con la compañía Y, traemos su *applet* de protocolo. No hay necesidad de que X y Y acuerden un protocolo estándar.

Introducción al lenguaje Java

Los objetivos antes listados han conducido a un lenguaje de tipo seguro, orientado a objetos, con capacidad multihilos interconstruida y sin características no definidas o dependientes del sistema. Lo que sigue es una descripción muy simplificada de Java, simplemente para dar una idea de su sabor. Se han omitido muchas características, detalles, opciones y casos especiales en aras de la brevedad. La especificación completa del lenguaje, y mucho más sobre Java, está disponible en la *Web* misma (por supuesto), en *http://java.sun.com*. En (Campione y Walrath, 1996, y Van der Linden, 1996) pueden encontrarse tutoriales sobre Java. La historia completa puede encontrarse en (Arnold y Gosling, 1996, y Gosling *et al.*, 1996). Para una comparación breve entre Java y la respuesta de Microsoft a él (Blackbird), véase (Singleton, 1996).

Como mencionamos antes, en pequeño, Java es parecido a C y C++. Las reglas léxicas, por ejemplo, son muy parecidas [por ejemplo, las fichas (*tokens*) se delimitan mediante espacio blanco, y pueden insertarse líneas nuevas entre cualquier par de fichas]. Pueden introducirse comentarios usando la sintaxis de C (/* ... */) o la sintaxis de C++ (// ...).

Java tiene ocho tipos de datos primitivos, que se listan en la figura 7-69. Cada tipo tiene un tamaño específico, independiente de la implementación local. Por tanto, a diferencia de C, donde un entero puede tener 16, 32 o 64 bits, dependiendo de la arquitectura de la máquina

subyacente, un int de Java siempre tiene 32 bits, ni uno más ni uno menos, sin importar el tipo de máquina en el que se ejecute el intérprete. Esta consistencia es esencial, dado que la misma *applet* debe ejecutarse en máquinas de 16 bits, 32 bits y 64 bits, y todas deben dar los mismos resultados.

Tipo	Tamaño	Descripción
Byte	1 byte	Entero con signo entre –128 y +127
Short	2 bytes	Entero con signo de 2 bytes
Int	4 bytes	Entero con signo de 4 bytes
Long	8 bytes	Entero con signo de 8 bytes
Float	4 bytes	Número de punto flotante IEEE de 4 bytes
Double	8 bytes	Número de punto flotante IEEE de 8 bytes
Boolean	1 bit	Los únicos valores son verdadero y falso
Char	2 bytes	Carácter en Unicode

Figura 7-69. Tipos de datos Java básicos.

Las variables aritméticas (los primeros 6 tipos) pueden combinarse usando los operadores aritméticos normales (incluyidos ++ y – –) y compararse usando los operadores relacionales normales (por ejemplo, <, <=, ==, !=). Se permiten las conversiones entre tipos si tienen sentido.

Java usa caracteres en Unicode de 16 bits en lugar de ASCII, por lo que las variables de caracteres tienen 2 bytes de longitud. Los primeros 127 caracteres Unicode son los mismos que en ASCII, para permitir compatibilidad hacia atrás. Arriba de éstos hay algunos símbolos gráficos y luego los caracteres necesarios para el ruso, el árabe, el hebreo, el japonés (kanji, katakana e hiragana) y prácticamente todos los demás idiomas. Los caracteres no presentes en ASCII pueden representarse mediante \u seguido de cuatro dígitos hexadecimales. Por ejemplo, \u0ae6 es el cero en Gujarati.

Java permite la declaración de arreglos de una dimensión. Por ejemplo,

```
int[] table;
```

declara un arreglo, *table*, pero no le asigna espacio. Eso puede hacerse después, como en el C++, mediante, por ejemplo

```
table = new int [1024];
```

para asignar un arreglo con 1024 entradas. No es necesario (ni siquiera posible) devolver arreglos que ya no se necesitan; el recolector de basura los toma. Por tanto, no se necesitan las rutinas de biblioteca *malloc* y *free*, altamente susceptibles a errores, para la administración del almacenamiento. Los arreglos pueden inicializarse, y pueden usarse arreglos de arreglos para obtener mayor dimensionalidad, como en C. Están disponibles cadenas, pero se definen por clase, en lugar de ser simplemente arreglos de caracteres que terminan con un byte nulo.

Los enunciados de control de Java se muestran en la figura 7-70. Los primeros nueve tienen esencialmente la misma sintaxis y semántica que en C, excepto que, donde se requiere una expresión booleana, el lenguaje realmente insiste en una expresión booleana. También los postulados break y continue ahora pueden llevar etiquetas que indican cuál de los ciclos etiquetados debe terminarse o repetirse.

Enunciado	Descripción	Ejemplo
Assignment	Asigna un valor	n = i + j;
If	Decisión booleana	if (k < 0) k = 0; else k = 2*k;
Switch	Selecciona un caso	switch (b) {case 1: n++; case 2: n– –;}
For	Iteración	for (i = 0; i < n; i++) a[i] = b[i];
While	Repetición	while (n < k) n += i;
Do	Repetición	do {n = n + n} while (n < m);
Break	Enunciado de salida	break etiqueta;
Return	Regreso	return n;
Continue	Siguiente iteración	continue etiqueta;
Throw	Causa excepción	throw new IllegalArgumentException();
Try	Alcance de excepción	try {...} catch (Exception e) {return –1};
Synchronized	Exclusión mutua	synchronized void update(int s) {...}

Figura 7-70. Enunciados Java. La notación {...} indica un bloque de código.

Los siguientes dos enunciados están en C++, pero no en C. Los enunciados throw y try se encargan del manejo de excepciones. Java define una variedad de excepciones estándar, como un intento de dividir entre cero, y permite a los programadores definir y generar sus propias excepciones. Los programadores pueden escribir manejadores para atrapar excepciones, haciendo innecesario probar constantemente si algo ha funcionado mal (por ejemplo, al leer de un archivo). El enunciado throw genera una excepción, y el enunciado try define un alcance para asociar los manejadores de excepciones a un bloque de código en el que podría ocurrir una excepción.

El enunciado synchronized es nuevo en Java y tiene que ver con el hecho de que los programas Java puedan tener múltiples hilos de control. Para evitar condiciones de competencia, este enunciado delimita un bloque de código (o un procedimiento completo) que no debe tener más de un hilo activo a la vez. Tales bloques de código generalmente se llaman **regiones críticas**. Cuando se ejecuta el enunciado synchronized, el hilo que lo ejecuta debe adquirir el candado asociado a la región crítica, ejecutar el código y luego liberar el candado. Si el candado no está disponible, el hilo espera hasta que queda libre. Al proteger procedimientos completos de esta manera y usar variables de condición, el programador dispone de toda la capacidad de los monitores (Hoare, 1974).

Los programas en Java pueden llamarse con argumentos. El procesamiento de líneas de comandos es parecido al de C, excepto que el arreglo de argumentos se llama *args* en lugar de *argv* y *args*[0] es el primer parámetro, no el nombre del programa. En la figura 7-71 se ilustra un pequeño programa Java que calcula una tabla de factoriales, simplemente para dar una idea de cómo se ve un programa Java pequeño.

```
class Factorial { /* Este programa consiste en una sola clase con dos métodos. */

  public static void main (int argc, String args[ ]) { // programa principal
  long i, f, lower = 1, upper = 20;        // declaraciones de cuatro longs

  for (i = lower; i <= upper; i++) {       // ciclo de menor a mayor
      f = factorial(i);                    // f = i!
      System.out.println(i + " " + f);     // imprime i y f
  }
}

  static long factorial (long k) {         // función factorial recursiva
     if (k == 0)
          return 1;                        // 0! = 1
     else
          return k * factorial(k-1);       // k! = k * (k-1)!
  }
}
```

Figura 7-71. Programa Java para calcular e imprimir de 0! a 20!

A pesar de que ambos son lenguajes orientados a objetos basados en C, Java y C++ difieren en algunos aspectos. Algunas características se quitaron de Java para hacerlo seguro respecto a los tipos y más fácil de leer. Éstas incluyen #define, typedef, enum, union, struct la sobrecarga de operadores, los apuntadores explícitos, las variables globales, las funciones aisladas y funciones amigas. Casi está de más decir que el enunciado goto se ha enviado a ese lugar especial reservado para las características obsoletas de los lenguajes de programación. Se agregaron otras características para dar al lenguaje mayor potencia. Las características agregadas incluyen recolección de basura, multihilos, interfaces de objetos y paquetes.

Orientación a objetos en Java

En los lenguajes de procedimientos como Pascal o C, un programa consiste en un conjunto de variables y procedimientos, sin ningún principio general de organización. En contraste, en los **lenguajes orientados a objetos** (casi) todo es un objeto. Un **objeto** normalmente contiene algunas variables de estado internas (es decir, escondidas), junto con algunos procedimientos públicos, llamados **métodos**, para acceder a ellos. Se espera (y puede obligarse a que) los programas que usan el objeto invoquen los métodos para manipular el estado del objeto. De esta manera, el escritor del objeto puede controlar la manera en que los programas usan la información del objeto. Este principio se llama **encapsulamiento**, y es la base de toda la programación orientada a objetos.

Java intenta tomar lo mejor de ambos mundos: puede usarse como lenguaje de procedimientos tradicional o como lenguaje orientado a objetos. El programa en Java de la figura 7-71, por ejemplo, podría haberse escrito igualmente bien en C, y esencialmente de la misma manera. Desde este punto de vista, un subgrupo de Java podría considerarse como una versión depurada de C. Sin embargo, para escribir páginas de *Web*, es mejor considerar a Java como un lenguaje orientado a objetos, por lo que estudiaremos su orientación a objetos en esta sección.

Un programa en Java consiste en uno o más **paquetes**, cada uno de los cuales contiene algunas definiciones de clases. Los paquetes pueden accederse remotamente a través de una red, por lo que aquellos destinados para ser usados por mucha gente deben tener nombres únicos. Normalmente se usan nombres jerárquicos, comenzando por el inverso del nombre DNS de su máquina, por ejemplo:

```
EDU.univ.cs.catie.games.chess
```

Una definición de **clase** es una plantilla para producir ejemplares de objetos, cada uno de los cuales contiene las mismas variables de estado y los mismos métodos que todos los otros ejemplares de objetos de su clase. Sin embargo, los valores de las variables de estado de los diferentes objetos son independientes. Las clases, por tanto, son como moldes para cortar galletas: no son las galletas mismas, pero sirven para producir galletas estructuralmente idénticas, produciendo cada molde de galletas una forma diferente de galleta. Una vez producidas, las galletas (objetos) son independientes entre sí.

Los objetos en Java pueden producirse dinámicamente durante la ejecución, por ejemplo mediante

```
object = new ClassName()
```

Estos objetos se almacenan en el montículo y los remueve el recolector de basura cuando ya no se necesitan. De esta manera, la administración de almacenamiento en Java es manejada por el sistema, sin necesidad de los temidos procedimientos *malloc* y *free*, ni tampoco de apuntadores específicos.

Cada clase se basa en otra clase. Se dice que una clase de nueva definición es una **subclase** de la clase en la que se basa, la **superclase**. Una (sub)clase siempre **hereda** los métodos de su superclase; puede o no tener acceso directo a las variables internas de la superclase, dependiendo de si la superclase lo quiere o no. Por ejemplo, si una superclase, *A*, tiene los métodos *M1, M2* y *M3*, y una subclase *B*, define un método nuevo, *M4*, entonces los objetos creados en *B* tendrán los métodos *M1, M2, M3* y *M4*. La propiedad por la que una clase automáticamente adquiere todos los métodos de su superclase se llama **herencia**, y es una propiedad importante de Java. La acción de agregar nuevos métodos a los métodos de la superclase se llama **extender** la superclase. Como nota, algunos lenguajes orientados a objetos permiten que las clases hereden los métodos de dos o más superclases (herencia múltiple), pero los diseñadores de Java pensaron que esta propiedad era demasiado desordenada e intencionalmente la dejaron fuera.

Dado que cada clase tiene una y sólo una superclase inmediata, el grupo de todas las clases de un programa en Java forma un árbol. La clase de hasta arriba del árbol se llama **Object**, y todas las demás clases heredan sus métodos. Cualquier clase cuya superclase no se menciona explícitamente en su definición es por omisión una subclase de la clase *Object*. La clase *Factorial* de la figura 7-71, por ejemplo, es una subclase de *Object*.

Veamos ahora un ejemplo de los conceptos orientados a objetos que hemos presentado hasta ahora. En la figura 7-72 tenemos un paquete que define dos subclases, *ComplexNumber*, para definir y usar números complejos (es decir, números con una parte real y otra imaginaria), y *test*, para ilustrar la manera en que puede usarse *ComplexNumber*.

```
class ComplexNumber {          // Define una subclase de Object llamada ComplexNumber
  // Hidden data.
  protected double re, im;     // partes real e imaginaria

  // Cinco métodos para administrar los datos ocultos.
  public void complex(double x, double y) {re = x; im = y;}
  public double Real() {return re;}
  public double Imaginary() {return im;}
  public double Magnitude() {return Math.sqrt(re*re + im*im);}
  public double Angle() {return Math.atan(im/re);}
}

class test {                   // Segunda clase, para probar ComplexNumber
  public static void main (String args[ ]) {
     ComplexNumber c;          // declara un objeto de la clase ComplexNumber

     c = new ComplexNumber();  // asigna realmente memoria para c
     c.Complex(3.0, 4.0);      // invoca el método Complex para inicializar c
     System.out.println("La magnitud de c es " + c.Magnitude() );
  }
}
```

Figura 7-72. Paquete que define dos clases.

Al igual que *Factorial*, la clase *ComplexNumber* se basa en *Object*, puesto que no se nombra ninguna otra superclase en su definición. Cada objeto de la clase *ComplexNumber* representa un número complejo. Cada objeto de esta clase contiene dos variables ocultas, *re* e *im*, ambas números de punto flotante de 64 bits, para representar las partes real e imaginaria, respectivamente. Estas variables no pueden accederse desde fuera de la definición de clase (y sus subclases), puesto que se han declarado como protected. Si se hubieran declarado como private, habrían sido visibles sólo para *ComplexNumber*, y no para ninguna subclase. Por ahora, private habría sido adecuado, pero pronto definiremos una subclase. Si las variables se hubieran declarado como public, habrían sido visibles en cualquier lugar en que fuera visible el paquete, destruyendo así gran parte del valor de la programación orientada a objetos. No obstante, existen situaciones en las que se necesita hacer que el estado interno de un objeto sea público.

Se definen cinco métodos para los objetos que pertenecen a la clase *ComplexNumber*. Los usuarios de la clase están restringidos por tanto a las operaciones proporcionadas por estos cinco métodos, y no pueden acceder al estado directamente. Un ejemplo de la manera en que se crean, inicializan y usan los objetos de clase *ComplexNumber* se da en *test*.

Cuando se compila este paquete, el compilador de Java produce dos archivos binarios (de código de bytes), uno para cada una de las clases, y se nombran según su clase. Al teclearse el comando

```
java test
```

da como resultado la invocación del intérprete Java con la clase *test* como parámetro. El intérprete entonces busca un método llamado *main* y, al encontrarlo, lo ejecuta. El resultado de la ejecución es que se imprime la línea

```
La magnitud de c es 5
```

Ahora definamos una subclase de *ComplexNumber*, simplemente para ver cómo se hace. Se comienza por importar la clase original, para averiguar qué hace y cuáles son sus métodos. Luego se define una extensión de *ComplexNumber*, que llamaremos *HairyNumber*. La nueva clase hereda automáticamente los cinco métodos presentes en la superclase. Para hacer más interesantes las cosas, definiremos un sexto método, *AddTo*, en la subclase, para sumar un número complejo al objeto, aumentando sus partes real e imaginaria.

La definición de la subclase se muestra en la figura 7-73, junto con otro programa de prueba que ilustra la manera en que puede usarse un objeto que pertenece a la clase *HairyNumber*. Cuando se ejecuta el nuevo programa de prueba, se imprime

```
h = (–0.5,6)
```

Recuerde que los seis métodos pueden usarse con los objetos *a* y *h*, sin importar dónde se definió cada método. Si definimos ahora otra subclase más basada en *HairyNumber* y le damos, por decir, tres métodos, los objetos producidos a partir de ella tendrán nueve métodos válidos.

Además de agregar métodos nuevos a su superclase, una subclase puede reemplazar los métodos existentes simplemente redefiniéndolos. Por tanto, es posible que una subclase redefina todos los métodos heredados de su superclase, con lo que los objetos que pertenecen a las dos clases no tendrán nada en común. Sin embargo, el hacerlo es de mal gusto, y debe evitarse.

Por último, una clase Java puede definir varios métodos con el mismo nombre pero diferentes parámetros y diferentes definiciones. Cuando el compilador ve una invocación de método que usa este nombre, tiene que basarse en los tipos de los parámetros para determinar el método a usar. Esta propiedad se llama **carga extra** o **polimorfismo**. A diferencia de C++, donde los operadores pueden sobrecargarse también, en Java sólo pueden sobrecargarse los métodos, no los operadores, para hacer más fáciles de entender los programas.

```
import ComplexNumber;              // importa el paquete ComplexNumber

class HairyNumber extends ComplexNumber {   // define una clase nueva
  public void AddTo (ComplexNumber z) {      // con un método
     re = re + z.Real();
     im = im + z.Imaginary();
  }
}

class test2 {                      // programa de prueba para HairyNumber
  public static void main(String args []) {
     HairyNumber a, h;             // declara dos HairyNumber

     a = new HairyNumber();        // asigna memoria para a
     h = new HairyNumber();        // asigna memoria para h
     a.Complex(1.0, 2.0);          // asigna un valor a a
     h.Complex(-1.5, 4.0);         // asigna un valor a h
     h.AddTo(a);                   // invoca el método AddTo en h
     System.out.println("h = (" + h.Real() * "," + h.Imaginary() + ")" );
  }
}
```

Figura 7-73. Subclase de ComplexNumber que define un método nuevo.

La interfaz de programadores de aplicaciones

Además del lenguaje básico, los diseñadores de Java han definido e implementado unas 200 clases con la versión inicial. Los métodos contenidos en estas clases forman una especie de entorno estándar para los creadores de programas Java. Las clases están escritas en Java, por lo que son transportables a todas las plataformas y sistemas operativos.

Aunque un estudio detallado de todas estas clases y métodos ciertamente está fuera del alcance de este libro, puede ser de interés una descripción breve. Las 200 clases se agrupan en siete paquetes de tamaño desigual, cada uno de los cuales se enfoca hacia algún tema central. Las *applets* que necesiten un paquete en particular pueden incluirlo usando el enunciado import de Java. Los métodos contenidos pueden usarse según se requiera. Este mecanismo reemplaza la necesidad de incluir archivos de cabecera en C; también reemplaza la necesidad de bibliotecas, puesto que los paquetes se cargan dinámicamente durante la ejecución al ser invocados.

Los siete paquetes se resumen en la figura 7-74. El paquete *java.lang* contiene clases que pueden verse como parte del lenguaje, pero no lo son técnicamente. Éstas incluyen clases para administrar las clases mismas, hilos y manejo de excepciones. También están aquí las bibliotecas estándar de matemáticas y de cadenas.

Al igual que C, el lenguaje Java no contiene primitivas de E/S. La E/S se hace cargando y usando el paquete *java.io*, análogo a la biblioteca de E/S estándar de C. Se proporcionan

Paquete	Funcionalidad de ejemplo
Java.lang	Clases, hilos, excepciones, matemáticas, cadenas
Java.io	E/S de corrientes y archivos de acceso aleatorio, impresión
Java.net	Sockets, direcciones IP, URL, datagramas
Java.util	Pilas, tablas de dispersión, vectores, hora, fecha
Java.applet	Traer y presentar páginas de *Web*, audio, clase Object
Java.awt	Eventos, diálogo, menús, fuentes, gráficos, manejo de ventanas
Java.awt.image	Colores, recorte, filtrado y conversión de imágenes
Java.awt.peer	Acceso al sistema de ventana subyacente

Figura 7-74. Los paquetes incluidos en el API estándar.

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE
DOCUMENTACIÓN Y BIBLIOTECA
MONTEVIDEO - URUGUAY

métodos para leer y escribir cadenas, archivos de acceso aleatorio y hacer el formateo necesario para imprimir. En la figura 7-71 vimos uno de estos métodos, *println*, que hace impresión con formato.

Muy relacionado con la E/S está el transporte por red. Los métodos que buscan y manejan las direcciones IP se localizan aquí. El acceso a los sockets también forma parte de este paquete, al igual que la preparación de datagramas. La transmisión misma se maneja con *java.io*.

La siguiente clase es *java.util*; contiene las clases y métodos para las estructuras de datos comunes, como pilas y tablas de dispersión, de modo que los programadores no tengan que reinventar constantemente la rueda. También se lleva a cabo aquí la administración de fecha y hora.

El paquete *java.applet* contiene parte de la maquinaria básica de las *applets*, incluidos métodos para traer páginas de *Web* comenzando por sus URL. También tiene métodos para presentar páginas de *Web* y ejecutar segmentos de audio (por ejemplo, música de fondo). El paquete *java.applet* también contiene la clase *Object*. Todos los objetos heredan sus métodos, a menos que se reemplacen. Estos métodos incluyen hacer clones de un objeto, comparar la igualdad de dos objetos, convertir un objeto en una cadena, y otros más.

Por último, llegamos a *java.awt* y sus dos subpaquetes. AWT significa **Abstract Window Toolkit**, herramientas de ventana abstracta, y está diseñado para hacer que las *applets* sean transportables entre los sistemas de ventanas. Por ejemplo, ¿cómo debe una *applet* dibujar un rectángulo en la ventana de tal manera que pueda ejecutarse la misma versión compilada (en código de bytes) en UNIX, Windows y Macintosh, aunque cada uno tenga su propio sistema de ventanas? Parte del paquete se encarga de dibujar en la pantalla, por lo que hay métodos para colocar líneas, figuras geométricas, texto, menús, botones, barras de desplazamiento y muchos otros elementos en ella. Los programadores de Java llaman a estos métodos para escribir en la pantalla. Es responsabilidad del paquete *java.awt* hacer las llamadas adecuadas al sistema operativo local para llevar a cabo el trabajo. Esta estrategia significa que *java.awt* tiene que reescribirse

para cada plataforma nueva, pero las *applets* entonces son independientes de la plataforma, lo que es mucho más importante.

Otra tarea importante de esta clase es la administración de eventos. La mayoría de los sistemas de ventanas fundamentalmente son controlados por eventos. Lo que quiere decir esto es que el sistema operativo detecta pulsaciones de teclas, movimientos del ratón, presión y liberación de botones, y otros eventos, y los convierte en llamadas a procedimientos de usuario. En el caso de Java, se proporciona una gran biblioteca de métodos para manejar estos eventos en *java.awt*. Su uso simplifica la escritura de programas que interactúen con el sistema local de ventanas y aun así sean 100% transportables a máquinas con diferentes sistemas operativos y diferentes sistemas de ventanas.

Una parte del trabajo de este paquete se hace en *java.awt.image*, por ejemplo el manejo de imágenes, y en *java.awt.peer*, que permite el acceso al sistema subyacente de ventanas.

Seguridad

Uno de los aspectos más importantes de Java es sus características de seguridad. Cuando se trae una página de *Web* que contiene una *applet*, ésta automáticamente se ejecuta en la máquina del cliente. Idealmente, no debe caerse ni tirar la máquina del cliente.

Es más, no se requiere mucha imaginación para visualizar a un estudiante emprendedor preparando una página de *Web* que contiene algún interesante juego nuevo, y luego haciendo publicidad mundialmente a su URL (por ejemplo, publicándola de manera cruzada en todos los grupos de noticias). Lo que no se menciona en la publicación es el pequeño detalle de que la página también contiene una *applet* que, al ejecutarse, de inmediato cifra todos los archivos del disco duro del usuario. Cuando ha terminado, la *applet* anuncia lo que ha hecho y menciona cortésmente que los usuarios que deseen comprar la clave de descifrado deben enviar 1000 dólares en billetes de baja denominación a cierto apartado postal de Panamá.

Además del esquema de millonario instantáneo anterior, hay otros peligros inherentes a permitir la ejecución de código extraño en una máquina. Una *applet* podría andar a la caza de información interesante (correo electrónico almacenado, el archivo de contraseñas, las cadenas del entorno local, etc.) y devolverlas o enviarlas por correo electrónico a través de la red. La *applet* podría también consumir recursos (por ejemplo, llenar discos), presentar fotografías obscenas o consignas políticas en la pantalla, o hacer ruidos irritantes usando la tarjeta de sonido.

Los diseñadores de Java estaban bien conscientes de estos problemas, y erigieron una serie de barreras contra ellos. La primera línea de defensa es un lenguaje seguro respecto a los tipos. Java tiene especificaciones detalladas de los tipos de datos, arreglos verdaderos con verificación de límites y no tiene apuntadores. Estas restricciones hacen imposible que un programador de Java construya un apuntador para leer y escribir localidades arbitrarias de la memoria.

Sin embargo, Trudy, que ha dejado de tratar de descifrar protocolos criptográficos y ha entrado en el negocio mucho más interesante de escribir *applets* maliciosas en Java, simplemente puede escribir o modificar un compilador de C para producir códigos de bytes de Java, pasando por alto todas las salvaguardas proporcionadas por el lenguaje y el compilador de Java.

La segunda línea de defensa es que, antes de ejecutarse una *applet* entrante, debe pasar por un verificador de código de bytes, el cual busca intentos de fabricar apuntadores, ejecutar instrucciones o llamar a métodos con parámetros no válidos, usar variables antes de inicializarlas, etc. Estas comprobaciones supuestamente garantizan que sólo se ejecutarán las *applets* legales, pero Trudy ciertamente trabajará duro para encontrar trucos que el verificador no detecte.

La tercera línea de defensa es el cargador de clases. Puesto que las clases pueden cargarse sobre la marcha, hay el peligro de que una *applet* pueda cargar una de sus propias clases para reemplazar una clase de sistema crítica, pasando por alto las verificaciones de seguridad de esa clase. Este ataque de caballo de Troya se ha hecho imposible dando a cada clase su propio espacio de nombres (una especie de directorio abstracto), y buscando cuidadosamente las clases de sistema antes de buscar clases de usuario. En otras palabras, si el usuario carga una versión maliciosa de *println*, nunca se usará porque siempre se encontrará antes el *println* oficial.

La cuarta línea de defensa es que algunas clases estándar tienen interconstruidas sus propias medidas de seguridad. Por ejemplo, la clase de acceso a archivos mantiene una lista de archivos a los que puede accederse mediante *applets*, y presenta un marco de diálogo cada vez que una *applet* intenta hacer algo que viola las reglas de protección.

A pesar de todas estas medidas, se esperan problemas de seguridad. Primero, puede haber errores en el *software* de Java que los programadores astutos pueden explotar para pasar por alto la seguridad. El infame gusano de 1988 de Internet usó un error del *daemon* Finger de UNIX para detener súbitamente miles de máquinas en toda la Internet (Hafner y Markoff, 1991, y Spafford, 1989).

Segundo, si bien puede ser posible evitar que una *applet* haga cualquier cosa excepto escribir en la pantalla, muchas *applets* necesitarán más poder, por lo que, cuando soliciten privilegios adicionales, los usuarios podrían otorgarlos de mala gana (o inocentemente). Por ejemplo, las *applets* podrían necesitar escribir archivos temporales, por lo que los usuarios podrían darles acceso al directorio */tmp*, pensando que no hay nada de importancia ahí. Desgraciadamente, la mayoría de los editores guardan ahí las versiones temporales de los documentos y correo electrónico que se están editando, de modo que las *applets* maliciosas pueden copiarlos y tratar de enviarlos a través de la red. Por supuesto, puede ser posible bloquear el acceso de las *applets* a la red, pero entonces muchas podrían no funcionar, por lo que tendría que otorgárseles también este poder.

Pero incluso en el poco probable caso de que se niegue el acceso a la red a las *applets*, éstas pueden ser capaces de transmitir información usando **canales disimulados** (Lampson, 1973). Por ejemplo, después de obtener alguna información, una *applet* podría formar una corriente de bits usando el reloj de tiempo real del sistema local. Para enviar un 1, calcula intensamente durante ΔT; para enviar un 0, simplemente espera ΔT.

Para adquirir esta información, el dueño de la *applet* puede establecer una conexión con la máquina del cliente a fin de leer algunas de sus páginas de *Web* públicas o procesar con FTP algunos de sus archivos públicos. Vigilando cuidadosamente la tasa de entrada de datos, el dueño de la *applet* puede ver si ésta está haciendo cálculos (y reduciendo, por tanto, la corriente de salida observada) o descansando. Por supuesto, este canal es ruidoso, pero puede manejarse mediante técnicas estándar. La corriente puede dividirse en marcos delimitados por bytes indica-

dores, los marcos individuales pueden usar un código de corrección de errores robusto, y todos los marcos pueden enviarse dos o tres veces. Existen muchos otros canales disimulados, y es extremadamente difícil descubrirlos y bloquearlos a todos. Para mayor información sobre los problemas de seguridad en Java, véase (Dean y Wallach, 1995).

En pocas palabras, Java genera muchas posibilidades y oportunidades nuevas en la *World Wide Web*; permite que las páginas de *Web* sean interactivas y contengan animación y sonido; también permite que los visualizadores sean infinitamente extensibles. Sin embargo, el modelo Java de bajar *applets* también genera algunos problemas de seguridad nuevos y graves que aún no se han resuelto por completo.

7.6.5. Localización de información en la *Web*

Aunque la *Web* contiene una vasta cantidad de información, no siempre es fácil encontrar el elemento adecuado. A fin de hacer más fácil que las personas encuentren las páginas útiles para ellas, varios investigadores han escrito programas que indizan la *Web* de varias maneras. Algunos de éstos ya son tan comunes que se han vuelto comerciales. Los programas que hacen búsquedas en la *Web* a veces se llaman **máquinas de búsqueda**, **arañas**, **orugas**, **gusanos**, o ***knowbots*** (robots de conocimientos). En esta sección haremos una breve introducción a este tema. Para mayor información, véase (Pinkerton, 1994, y McBryan, 1994).

Aunque la *Web* es enorme, reducida a su parte más esencial, la *Web* es un grafo enorme, siendo las páginas los nodos y los hipervínculos los arcos. Los algoritmos para visitar todos los nodos de un grafo son bien conocidos. Lo que dificulta la indización de la *Web* es la enorme cantidad de datos que deben manejarse y el hecho de que constantemente están cambiando.

Comencemos nuestro estudio por una meta sencilla: indizar todas las palabras clave de los títulos de las páginas de *Web*. Para nuestro algoritmo necesitaremos tres estructuras de datos. Primero, necesitamos un arreglo lineal grande, *url_table*, que contiene millones de entradas, enúltima instancia, una por cada página de *Web*. Este arreglo debe mantenerse en memoria virtual, de modo que las partes que no sean de mucho uso se paginen automáticamente a disco. Cada entrada contiene dos apuntadores, uno a la URL de la página y otro al título de la página. Estos dos elementos son cadenas de longitud variable y pueden mantenerse en un montículo (una porción grande no estructurada de memoria virtual a la que pueden agregarse cadenas). El montículo es nuestra segunda estructura de datos.

La tercera estructura de datos es una tabla de dispersión de n entradas de tamaño que se usa como sigue. Cualquier URL puede pasarse por una función de dispersión para producir un entero no negativo menor que n. Todos los URL que se dispersan al valor k se encadenan en una lista vinculada que comienza en la entrada k de la tabla de dispersión. El principal uso de la tabla de dispersión es comenzar por un URL y ser capaz de determinar rápidamente si ya está presente en la tabla *url_table*. Estas tres estructuras de datos se ilustran en la figura 7-75.

La construcción del índice requiere dos fases: búsqueda e indización. Comencemos con una máquina sencilla para hacer la búsqueda. El corazón de la máquina de búsqueda es un procedimiento recurrente, *process_url*, que toma una cadena URL como entrada y opera como sigue.

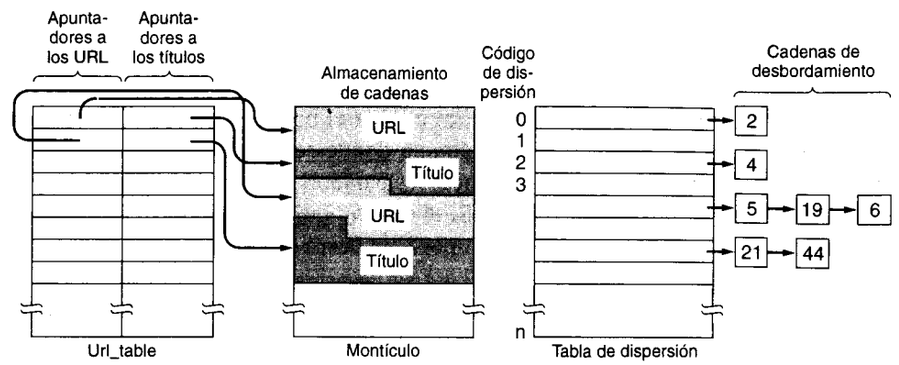


Figura 7-75. Estructuras de datos usadas en una máquina de búsqueda sencilla.

Primero, dispersa el URL para ver si ya está presente en *url_table*. De ser así, ha terminado y regresa de inmediato. Cada URL se procesa sólo una vez.

Si el URL no es conocido aún, se obtiene su página. El URL y el título se copian entonces en el montículo y se generan apuntadores a estas dos cadenas en *url_table*. El URL también se ingresa en la tabla de dispersión.

Por último, *process_url* extrae todos los hipervínculos de la página y llama a *process_url* una vez por hipervínculo, pasando el URL del hipervínculo como parámetro de entrada.

Para ejecutar la máquina de búsqueda, se llama a *process_url* con algún URL inicial. Al regresar, todas las páginas alcanzables desde ese URL han sido ingresadas en *url_table* y se ha completado la fase de búsqueda.

Aunque este diseño es sencillo y correcto en teoría, tiene un problema serio en un sistema tan grande como la *Web*. El problema es que este algoritmo hace una búsqueda primero en profundidad, y en algún momento entrará en recursión tantas veces como la trayectoria no cíclica más larga de la *Web*. Nadie sabe qué tan larga es esta trayectoria, pero probablemente tiene miles de hipervínculos de longitud. Como consecuencia, cualquier máquina de búsqueda que use esta búsqueda primero en profundidad probablemente tendrá un desbordamiento de su pila antes de terminar el trabajo.

En la práctica, las máquinas de búsqueda primero recolectan todos los hipervínculos de cada página que leen, retiran todos los que ya se han procesado, y guardan los demás. Entonces se hace una búsqueda en la *Web* primero en profundidad; es decir, cada vínculo de una página se sigue y se recolectan todos los hipervínculos de todas las páginas a las que apuntan, pero no se siguen en el orden obtenido.

La segunda fase lleva a cabo la indización de palabras clave. El proceso de indización recorre linealmente la *url_table*, procesando cada entrada por turno. Para cada entrada, se examina el título y se seleccionan todas las palabras no encontradas en la lista de parada. (La lista de parada evita la indización de preposiciones, conjunciones, artículos y otras palabras que se encuentran muchas veces pero que son de poco valor.) Por cada palabra seleccionada, se

escribe en un archivo una línea que consiste en la palabra seguida del número de entrada en la tabla *url_table*. Cuando se ha barrido la tabla completa, se ordena el archivo por palabra.

El índice tendrá que almacenarse en disco y puede usarse como sigue. El usuario llena una forma, listando una o más palabras clave y hace clic en el botón SUBMIT. Esta acción envía una solicitud *POST* a un guión CGI de la máquina donde se encuentra el índice. Este guión (o, más probablemente, programa) busca entonces las palabras clave en el índice hasta encontrar el grupo de índices *url_table* para cada una. Si el usuario quiere el AND BOOLEANO de las palabras clave, se calcula la intersección del grupo. Si se desea el OR BOOLEANO, se calcula la unión del grupo.

El guión ahora hace una búsqueda indizada en *url_table* para encontrar todos los títulos y URL. Éstos se combinan para formar una página de *Web* y se devuelven al usuario como la respuesta al *POST*. El visualizador presenta ahora la página, permitiendo al usuario hacer clic en los elementos que le parezcan interesantes.

¿Suena fácil? No lo es. Tienen que resolverse los siguientes problemas en cualquier sistema práctico:

1. Algunos URL son obsoletos (es decir, apuntan a páginas que ya no existen).
2. Algunas máquinas serán temporalmente inalcanzables.
3. No todas las páginas serán alcanzables desde el URL inicial.
4. Algunas páginas pueden ser alcanzables sólo desde mapas activos.
5. Algunos documentos no pueden indizarse (por ejemplo, fragmentos de audio).
6. No todos los documentos tienen título (útil).
7. Podría agotarse la memoria o el espacio de disco de la máquina de búsqueda.
8. El proceso completo podría tardar demasiado.

Los URL obsoletos desperdician tiempo, pero no causan demasiados problemas porque el servidor en el que se supone que están responde de inmediato con un código de error. En contraste, cuando el servidor está inactivo, todo lo que observa la máquina de búsqueda es un gran retardo al establecer la conexión TCP. Para evitar que se quede esperando indefinidamente, debe tener una temporización. Si la temporización es demasiado corta, se perderán varias URL válidas; si es demasiado grande, la búsqueda se hará muy lenta.

La selección del URL inicial ciertamente es un asunto importante. Si la máquina de búsqueda comienza por la página base de algún astrofísico, tarde o temprano podría encontrar todo sobre astronomía, física, química y ciencias espaciales, pero podría pasar de largo páginas sobre medicina veterinaria, inglés antiguo y *rock and roll*. Estos grupos podrían simplemente estar desunidos. Una solución es recolectar un grupo de varios URL tan grande como sea posible, y luego usar cada uno de ellos como página inicial. Los URL iniciales pueden recolectarse de los artículos de noticias de USENET y la versión de la semana pasada de la tabla *url_table*, puesto que algunas de estas páginas podrían haber cambiado recientemente (por ejemplo, uno de los

astrofísicos se casó con una veterinaria, y solemnemente actualizaron sus páginas base para que apuntaran una a la otra).

La indización funciona bien con texto, pero cada vez más páginas contienen elementos que no son de texto, como fotografías, audio y vídeo. Un enfoque es probar cada URL nuevo encontrado con el método *HEAD*, simplemente para devolver su cabecera MIME. Cualquier cosa que no sea tipo *texto* no se busca.

El 20% de las páginas de *Web* no tiene título, y muchas de aquellas que sí lo tienen, tienen uno casi inútil ("Página de Pepe"). Una gran mejora del índice básico es incluir no sólo títulos, sino también todo el hipertexto. En este enfoque, al barrerse una página, se registran también todos los hipervínculos, junto con la página de la que vinieron y la página a la que apuntan. Tras completarse la fase de búsqueda, pueden indizarse también todas las hiperpalabras.

Aún más ambicioso es indizar todas las palabras importantes de cada página. Para determinar las palabras importantes, puede calcularse la frecuencia de aparición de todas las palabras no encontradas en la lista de parada (por página de *Web*). Probablemente valga la pena indizar las 10 o 20 palabras más frecuentes. A fin de cuentas, si la palabra "hígado" es la más común de una página, hay la posibilidad de que la página será de interés para los cirujanos biliares (o para los cocineros). Algunas máquinas de búsqueda (por ejemplo, Lycos) usan esta estrategia.

Por último, la máquina de búsqueda puede quedarse sin memoria o terminársele el tiempo. Una estrategia es rediseñar los algoritmos más cuidadosamente. Un enfoque por entero distinto es hacer lo que hace Harvest y descargar el trabajo (Bowman *et al.*, 1994, 1996). En particular, Harvest proporciona un programa que se ejecuta en servidores cooperativos. Este programa hace toda la búsqueda localmente y transmite de regreso el índice local terminado. En la instalación central, todos los índices locales se integran en el índice maestro. Este enfoque reduce en varios órdenes de magnitud la cantidad de memoria, tiempo de CPU y ancho de banda de red necesarios, pero tiene la desventaja principal de requerir que todos los servidores de la *Web* cooperen ejecutando *software* ajeno. Dados los problemas potenciales con los virus y gusanos, al solicitarse a un administrador de sistema: "¿por favor puedes ejecutar este programa en tu máquina por mí?", no debe sorprender que muchos se nieguen.

Es pertinente una pequeña recomendación. Aunque la escritura de una máquina de búsqueda suena fácil, una con fallas puede provocar caos en la red al generar grandes cantidades de solicitudes espurias, no sólo desperdiciando ancho de banda, sino también abrumando a muchos servidores debido a la carga. Si usted no puede resistir la tentación de escribir su propia máquina de búsqueda, la ética de la red requiere que la restrinja al dominio de su propio DNS local hasta que esté completamente libre de errores.

7.7. MULTIMEDIA

Los multimedia son la vaca sagrada de las redes. Cuando se menciona la palabra, tanto los lunáticos como los de traje comienzan a salivar como si estuvieran ante un gran banquete. Los primeros ven retos técnicos inmensos para proporcionar vídeo (interactivo) a solicitud en cada casa. Los segundos ven enormes ganancias en el asunto. Ningún libro sobre redes estaría com-

pleto sin, al menos, una introducción al tema. Dada la extensión de éste, la nuestra será necesariamente breve. Para información adicional sobre este tema fascinante y potencialmente lucrativo, véase (Buford, 1994; Deloddere *et al.*, 1994; Dixit y Skelly, 1995; Fluckiger, 1995; Minoli, 1995, y Steinmetz y Nahrstedt, 1995).

Literalmente, los multimedia sencillamente son dos o más medios. Si el editor de este libro quisiera unirse a la euforia actual por los multimedia, podría anunciar que el libro usa tecnología multimedia. A fin de cuentas, contiene dos medios: texto y gráficos (las figuras). No obstante, cuando la mayoría de la gente se refiere a multimedia, generalmente quieren decir la combinación de dos o más **medios continuos**, es decir, medios que tienen que ejecutarse durante cierto intervalo de tiempo bien definido, generalmente con alguna interacción con el usuario. En la práctica, normalmente los dos medios son audio y vídeo, es decir, sonido más imágenes en movimiento. Por esta razón, comenzaremos nuestro estudio con una introducción a la tecnología del audio y el vídeo. Después los combinaremos y pasaremos a los sistemas multimedia reales, incluidos vídeo a solicitud y el sistema multimedia de Internet, MBone.

7.7.1. Audio

Una onda de audio (sonido) es una onda acústica (de presión) de una dimensión. Al entrar una onda acústica en el oído, el tímpano vibra, causando que los pequeños huesos del oído interno vibren con él, enviando pulsos nerviosos al cerebro. Estos pulsos los percibe el escucha como sonido. De manera parecida, cuando una onda acústica incide en un micrófono, el micrófono genera una señal eléctrica, que representa la amplitud del sonido como una función del tiempo. La representación, procesamiento, almacenamiento y transmisión de tales señales de audio es una parte principal del estudio de los sistemas multimedia.

La gama de frecuencias perceptibles por el oído humano va de 20 Hz a 20,000 Hz, aunque algunos animales, notablemente los perros, pueden escuchar frecuencias más altas. El oído escucha de manera logarítmica, por lo que la relación entre dos sonidos de amplitudes A y B se expresa convencionalmente en **dB** (**decibeles**) de acuerdo con la fórmula

$$\text{dB} = 20 \log_{10}(A / B)$$

Si definimos como 0 dB el límite inferior de la audibilidad (una presión de unas 0.0003 dinas/cm^2) para una onda senoidal de 1 kHz, una conversación ordinaria es de unos 50 dB y el umbral del dolor es de 120 dB, lo que representa una gama dinámica de un factor de un millón. Para evitar cualquier confusión, los A y B indicados antes son *amplitudes*. Si usáramos el nivel de potencia, que es proporcional al cuadrado de la amplitud, el coeficiente del logaritmo sería 10, no 20.

El oído es sorprendentemente sensible a variaciones de sonido que duran apenas unos milisegundos. El ojo, en cambio, no nota cambios en el nivel de luz que duran unos cuantos milisegundos. El resultado de esta observación es que fluctuaciones de apenas unos cuantos milisegundos durante una transmisión multimedia afectan la calidad del sonido percibido más de lo que afectan la calidad percibida de la imagen.

Las ondas de audio pueden convertirse a una forma digital mediante un **ADC** (*Analog Digital Converter*, **convertidor analógico a digital**). Un ADC toma un voltaje eléctrico como entrada y genera un número binario de salida. En la figura 7-76(a) vemos un ejemplo de onda senoidal. Para representar esta señal de manera digital, simplemente la muestreamos cada ΔT segundos, como lo muestra la altura de las barras de la figura 7-76(b). Si una onda de sonido no es una onda senoidal pura, sino una superposición de ondas senoidales en las que la componente de más alta frecuencia es f, entonces el teorema de Nyquist (véase el capítulo 2) establece que es suficiente tomar muestras a una frecuencia $2f$. Una frecuencia de muestreo mayor no tiene ningún valor, porque no están presentes las frecuencias mayores que detectaría este muestreo más frecuente.

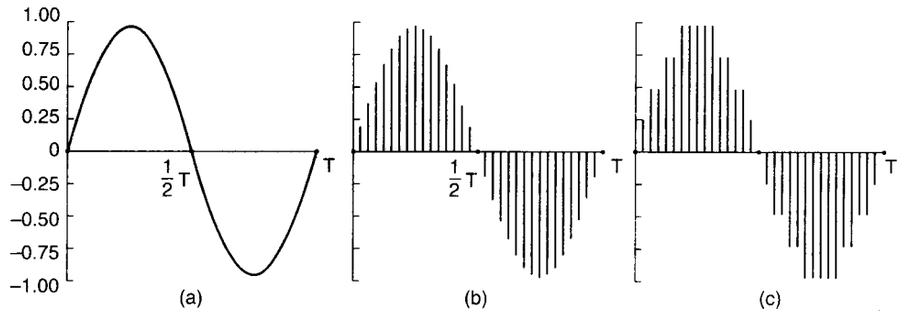


Figura 7-76. (a) Onda senoidal. (b) Muestreo de la onda senoidal. (c) Cuantización de las muestras a 3 bits.

Las muestras digitales nunca son exactas. Las muestras de 3 bits de la figura 7-76(c) permiten sólo ocho valores, de –1.00 a +1.00 en incrementos de 0.25. Una muestra de 8 bits permitirá 256 valores diferentes. Una muestra de 16 bits permitirá 65,536 valores diferentes. El error introducido por la cantidad finita de bits por muestra se llama **ruido de cuantización**. Si éste es demasiado grande, el oído lo detecta.

Dos ejemplos bien conocidos de sonido muestreado son el teléfono y los discos compactos de audio. La modulación de código de pulso, como la usada en el sistema telefónico, emplea muestras de 7 bits (Norteamérica y Japón) y 8 bits (Europa), 8000 veces por segundo. Este sistema da una tasa de datos de 56,000 bps o 64,000 bps. Con sólo 8000 muestras/seg, las frecuencias por arriba de 4 kHz se pierden.

Los CD de audio son digitales, con una tasa de muestreo de 44,100 muestras/seg, suficientes para capturar frecuencias de hasta 22,050 Hz, lo que es bueno para la gente, malo para los perros. Las muestras tienen 16 bits cada una, y son lineales dentro de la gama de amplitudes. Nótese que las muestras de 16 bits permiten sólo 65,536 valores diferentes, aunque la gama dinámica del oído es de aproximadamente 1 millón si se mide en pasos del tamaño del sonido audible más pequeño. Por tanto, el uso de sólo 16 bits por muestra genera algún ruido de cuantización (aunque no se cubre la gama dinámica completa; no se supone que los CD deban lastimar). Con

44,100 muestras/seg de 16 bits cada una, un CD de audio necesita un ancho de banda de 705.6 kbps para monofónico y 1.411 Mbps para estéreo. Si bien esto es menos de lo que necesita el vídeo (véase más adelante), aun así se requiere un canal T1 completo para transmitir sonido estéreo de calidad CD.

El sonido digitalizado puede procesarse fácilmente por *software* en las computadoras. Existen docenas de programas para computadoras personales que permiten a los usuarios grabar, visualizar, editar, mezclar y almacenar ondas de sonido de fuentes múltiples. Hoy día, casi todas las grabaciones y ediciones profesionales de sonido son digitales.

Muchos instrumentos musicales tienen incluso una interfaz digital. Cuando salieron los primeros instrumentos digitales, cada uno tenía su propia interfaz, pero al poco tiempo se desarrolló un estándar, el **MIDI** (*Music Instrument Digital Interface*, **interfaz digital de instrumentos musicales**), que fue adoptado por prácticamente toda la industria musical. Este estándar especifica el conector, cable y formato de mensaje. Cada mensaje MIDI consiste en un byte de estado seguido de cero o más bytes de datos. Un mensaje MIDI transporta un evento significativo musicalmente. Los eventos comunes son la pulsación de una tecla, el movimiento de un control deslizante y la liberación de un pedal. El byte de estado indica el evento, y los bytes de datos dan parámetros, como la tecla pulsada y la velocidad con la que se pulsó.

Cada instrumento tiene un código MIDI asignado a él. Por ejemplo, un piano de cola tiene el 0, una marimba tiene el 12 y un violín tiene el 40. Esto se requiere para evitar que un concierto para flauta se reproduzca como un concierto para tuba. La cantidad definida de "instrumentos" es de 127. Sin embargo, algunos de éstos no son instrumentos, sino efectos especiales como trinos de pájaro, helicópteros y el aplauso enlatado que acompaña a muchos programas de televisión.

El corazón de todo sistema MIDI es un sintetizador (con frecuencia una computadora) que acepta mensajes y genera música a partir de ellos. El sintetizador entiende los 127 instrumentos, por lo que genera un espectro de potencia diferente para el Do central de una trompeta que para el de un xilófono. La ventaja de transmitir música usando MIDI, en comparación con el envío de una onda digitalizada, es la enorme reducción de ancho de banda, frecuentemente en un factor de 1000. La desventaja del MIDI es que el receptor necesita un sintetizador MIDI para reconstruir la música nuevamente, y diferentes sintetizadores pueden dar reproducciones ligeramente diferentes.

La música, por supuesto, es sólo un caso especial del audio en general, pero es importante. Otro caso especial es la voz. La voz humana tiende a estar en la gama de 600 a 6000 Hz. La voz se compone de vocales y consonantes, que tienen diferentes propiedades. Las vocales se producen cuando la cavidad vocal no está obstruida, produciendo resonancias cuya frecuencia fundamental depende del tamaño y la forma del sistema vocal y la posición de la lengua y quijada del que habla. Estos sonidos son casi periódicos durante intervalos de 30 mseg. Las consonantes se producen cuando la cavidad vocal está parcialmente bloqueada. Estos sonidos son menos regulares que las vocales.

Algunos sistemas de generación y transmisión de voz hacen uso de modelos del sistema vocal para reducir la voz a unos cuantos parámetros (por ejemplo, tamaños y formas de varias cavidades), en lugar de simplemente muestrear la forma de onda de la voz.

7.7.2. Vídeo

El ojo humano tiene la propiedad de que, cuando una imagen incide en la retina, se retiene durante algunos milisegundos antes de decaer. Si una secuencia de imágenes incide a 50 o más imágenes/seg, el ojo no nota que está viendo imágenes discretas. Todos los sistemas de vídeo (es decir, televisión) aprovechan este principio para producir imágenes en movimiento.

Sistemas Analógicos

Para entender los sistemas de vídeo, es mejor comenzar por la anticuada y sencilla televisión en blanco y negro. Para representar la imagen bidimensional que está frente a ella como un voltaje unidimensional en función del tiempo, la cámara barre rápidamente un haz de electrones a lo ancho de la imagen y lentamente hacia abajo, registrando la intensidad de la luz a su paso. Al final del barrido, llamado **marco**, el haz hace un retrazado. Esta intensidad como función de tiempo se difunde, y los receptores repiten el proceso de barrido para reconstruir la imagen. El patrón de barrido que usan tanto la cámara como el receptor se muestra en la figura 7-77. (Como nota, las cámaras CCD integran en lugar de barrer, pero algunas cámaras y todos los monitores hacen un barrido.)

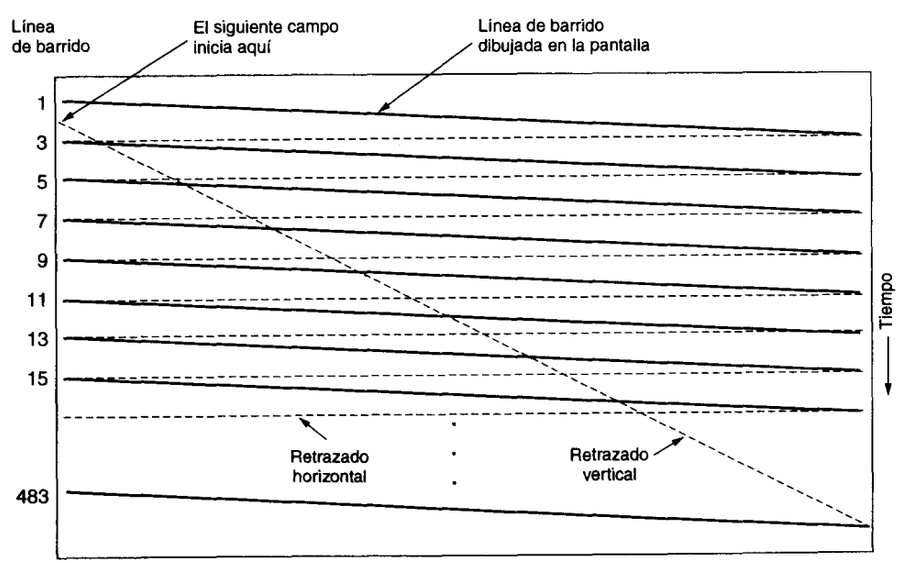


Figura 7-77. Patrón de barrido usado para el vídeo y la televisión NTSC.

Los parámetros de barrido exactos varían de país en país. El sistema usado en Norte y Sudamérica y en Japón tiene 525 líneas de barrido, una relación de aspecto horizontal a vertical

de 4:3, y 30 marcos/seg. El sistema europeo tiene 625 líneas de barrido, la misma relación 4:3, y 25 marcos/seg. En ambos sistemas, no se presentan unas cuantas líneas de arriba y abajo (para aproximar una imagen rectangular a los CRT originales, que eran redondos). Sólo se presentan 483 de las 525 líneas NTSC (y 576 de las 625 líneas de barrido PAL/SECAM). El haz se apaga durante el retrazado vertical, por lo que muchas estaciones (especialmente en Europa) usan este intervalo para difundir Teletexto (páginas de texto que contienen noticias, informes meteorológicos, deportes, precios de acciones, etcétera).

Aunque 25 marcos/seg es suficiente para capturar una imagen continua, con esa tasa de marcos mucha gente, especialmente las personas mayores, percibe un parpadeo de la imagen (porque las imágenes viejas se desvanecen de la retina antes de la aparición de la nueva). En lugar de aumentar la tasa de marcos, lo que requeriría usar más del escaso ancho de banda, se emplea un enfoque diferente. En lugar de presentar las líneas de barrido en orden, primero se presentan las líneas de barrido nones, y luego las líneas de barrido pares. Cada uno de estos medios marcos se llama **campo**. Hay experimentos que muestran que, aunque la gente nota el parpadeo a 25 marcos/seg, no lo nota a 50 campos/seg. Esta técnica se llama **entrelazado**. Se dice que la televisión (o vídeo) no entrelazada es **progresiva**.

El vídeo de color usa el mismo patrón de barrido que el monocromático (blanco y negro), excepto que, en lugar de presentar la imagen mediante un solo haz en movimiento, se usan tres haces que se mueven al unísono. Se usa un haz para cada uno de los colores primarios aditivos: rojo, verde y azul (RGB). Esta técnica funciona porque puede construirse cualquier color a partir de la superposición lineal de rojo, verde y azul con las intensidades apropiadas. Sin embargo, para la transmisión por un solo canal, las tres señales de color deben combinarse en una sola señal **compuesta**.

Cuando se inventó la televisión a color, eran técnicamente posibles varios métodos de presentación de los colores, así que varios países tomaron decisiones diferentes, lo que condujo a sistemas que aún son incompatibles. (Nótese que estas selecciones no tienen nada que ver con VHS contra Betamax contra P2000, que son métodos de grabación.) En todos los países, un requisito político fue que todos los programas que se transmitieran a color tenían que ser suceptibles de recibirse en los televisores existentes de blanco y negro. En consecuencia, no era aceptable el esquema más sencillo, codificar por separado las señales RGB. Tampoco es el RGB el esquema más eficiente.

El primer sistema de color fue estandarizado en Estados Unidos por el Comité Nacional de Estándares de Televisión, que prestó sus siglas al estándar: **NTSC**. La televisión a color se introdujo en Europa varios años después, y para entonces la tecnología había mejorado significativamente, conduciendo a sistemas con mejor inmunidad contra el ruido y mejores colores. Éstos se llaman **SECAM** (*SEquentiel Couleur Avec Memoire*, **color secuencial con memoria**), que se usa en Francia y Europa Oriental, y **PAL** (*Phase Alternating Line*, **línea de fases alternas**), usado en el resto de Europa. La diferencia en la calidad del color entre NTSC y PAL/SECAM ha producido una broma de la industria que sugiere que NTSC significa en realidad Nunca Tendrás Semejanza en los Colores.

Para que las transmisiones puedan verse en receptores de blanco y negro, los tres sistemas combinan linealmente las señales RGB en una señal de **luminancia** (brillo) y dos de **crominancia** (color), aunque usan diferentes coeficientes para construir estas señales a partir de las señales RGB. Resulta interesante que el ojo es mucho más sensible a la señal de luminancia que a las señales de crominancia, por lo que estas últimas no necesitan transmitirse con tanta precisión. En consecuencia, la señal de luminancia puede difundirse a la misma frecuencia que la vieja señal de blanco y negro, y puede recibirse en los televisores de blanco y negro. Las dos señales de crominancia se difunden en bandas angostas a frecuencias mayores. Algunos aparatos de televisión tienen controles etiquetados brillo, matiz y saturación (o brillo, tinte y color) para controlar estas tres señales por separado. Es necesario el entendimiento de la luminancia y la crominancia para comprender el funcionamiento de la compresión de vídeo.

En los últimos años ha habido un interés considerable en la **HDTV** (*High Definition Television*, **televisión de alta definición**), que produce imágenes más nítidas al duplicar (aproximadamente) la cantidad de líneas de barrido. Estados Unidos, Europa y Japón han desarrollado sistemas HDTV, todos diferentes y todos mutuamente incompatibles. Los principios básicos de la HDTV en términos de barrido luminancia, crominancia, etc., son semejantes a los sistemas actuales. Sin embargo, los tres formatos tienen una proporción dimensional común de 16:9 en lugar 4:3 para lograr una correspondencia mejor con el formato usado para el cine (que se graba en película de 35 mm).

Para una introducción a la tecnología de televisión, véase (Buford, 1994).

Sistemas digitales

La representación más sencilla del vídeo digital es una secuencia de marcos, consistiendo cada uno en una malla rectangular de elementos de imagen, o **pixeles**. Cada píxel puede ser un solo bit, para representar blanco o negro. La calidad de tal sistema es parecida a la que se obtiene al enviar una fotografía a color por fax: espantosa. (Inténtelo si puede; si no, fotocopie una fotografía a color en una máquina copiadora que no maneje tonos.)

El siguiente paso es usar 8 bits por píxel para representar 256 niveles de gris. Este esquema da vídeo en blanco y negro de alta calidad. Para vídeo a color, los sistemas buenos usan 8 bits por cada uno de los colores RGB, aunque casi todos los sistemas los mezclan en vídeo compuesto para su transmisión. Aunque el uso de 24 bits por píxel limita la cantidad de colores a unos 16 millones, el ojo humano no puede diferenciar tantos colores. Las imágenes digitales de color se producen usando tres haces de barrido, uno por color. La geometría es la misma que en el sistema analógico de la figura 7-77, excepto que las líneas continuas de barrido ahora se reemplazan por elegantes filas de pixeles discretos.

Para producir una imagen uniforme, el vídeo digital, al igual que el vídeo analógico, debe presentar cuando menos 25 marcos/seg. Sin embargo, dado que los monitores de computadora de alta calidad con frecuencia barren la pantalla a partir de las imágenes almacenadas en memoria a razón de 75 veces por segundo o más, no se requiere entrelazado y, en consecuencia, normalmente no se usa. Basta con repintar (es decir, redibujar) el mismo marco tres veces consecutivas para eliminar el parpadeo.

En otras palabras, la uniformidad de movimiento la determina la cantidad de imágenes *diferentes* por segundo, mientras que el parpadeo lo determina la cantidad de veces por

segundo que se pinta el marco. Estos dos parámetros son diferentes. Una imagen fija pintada a 20 marcos/seg no mostrará un movimiento inestable, pero parpadeará porque un marco se desvanecerá de la retina antes de que aparezca el siguiente. Una película con 20 marcos diferentes por segundo, pintados cada uno cuatro veces seguidas, no parpadeará, pero el movimiento parecerá no uniforme.

El significado de estos dos parámetros se aclara cuando consideramos el ancho de banda necesario para transmitir vídeo digital a través de una red. Todos los monitores de computadora actuales usan la relación de aspecto 4:3 para poder usar tubos de rayos catódicos económicos de producción en masa diseñados para el mercado de televisión de consumidor. Las configuraciones comunes son 640 × 480 (VGA), 800 × 600 (SVGA) y 1024 × 768 (XGA). Una pantalla XGA con 24 bits por píxel y 25 marcos/seg requiere una alimentación a 472 Mbps. Ni siquiera el OC-9 es lo bastante bueno para esto, y la conexión de una portadora SONET OC-9 a la casa de todos no está precisamente en la agenda. La duplicación de esta tasa para evitar el parpadeo es aún menos atractiva. Una mejor solución es transmitir 25 marcos/seg y hacer que la computadora almacene cada uno y lo pinte dos veces. La televisión difundida no usa esta estrategia porque las televisiones no tienen memoria y, en todo caso, las señales analógicas no pueden almacenarse en RAM sin primero convertirse a un formato digital, lo que requeriría *hardware* extra. Como consecuencia, se necesita el entrelazado para la televisión difundida, pero no para el vídeo digital.

7.7.3. Compresión de datos

Debe ser obvio a estas alturas que la transmisión de material multimedia en formato sin compresión es impensable. La única esperanza es que sea posible la compresión masiva. Afortunadamente, un gran número de investigaciones durante las últimas décadas ha conducido a muchas técnicas y algoritmos de compresión que hacen factible la transmisión multimedia. En esta sección estudiaremos algunos métodos para la compresión de datos multimedia, especialmente imágenes. Para mayores detalles, véase (Fluckiger, 1995, y Steinmetz y Nahrstedt, 1995).

Todos los sistemas de compresión requieren dos algoritmos: uno para la compresión de los datos en el origen y otro para la descompresión en el destino. En la literatura, estos algoritmos se conocen como algoritmos de **codificación** y **decodificación**, respectivamente. También usaremos esta terminología aquí.

Estos algoritmos tienen ciertas asimetrías cuya comprensión es importante. Primero, en muchas aplicaciones un documento multimedia, digamos una película, sólo se codificará una vez (al almacenarse en el servidor de multimedia), pero se decodificará miles de veces (al ser vista por los clientes). Esta asimetría significa que es aceptable que el algoritmo de codificación sea lento y requiera *hardware* caro, siempre y cuando el algoritmo de decodificación sea rápido y no requiera *hardware* costoso. A fin de cuentas, el operador de un servidor multimedia podría estar dispuesto a rentar una supercomputadora paralela durante algunas semanas para codificar su biblioteca de vídeo completa, pero requerir que los consumidores renten una supercomputadora durante dos horas para ver un vídeo seguramente no tendrá mucho éxito. Muchos sistemas de

compresión prácticos van a los extremos para lograr que la decodificación sea rápida y sencilla, aun al precio de hacer lenta y complicada la codificación.

Por otra parte, para los multimedia de tiempo real, como las videoconferencias, la codificación lenta es inaceptable. La codificación debe ocurrir al momento, en tiempo real. En consecuencia, los multimedia de tiempo real usan algoritmos o parámetros diferentes que el almacenamiento de vídeos en disco, con frecuencia resultando en una compresión significativamente menor.

Una segunda asimetría es que no es necesario que el proceso de codificación/decodificación se pueda invertir. Por ejemplo, al comprimir, transmitir y descomprimir un archivo de datos, el usuario espera recibir el original, correcto hasta el último bit. En multimedia este requisito no existe. Generalmente es aceptable que la señal de vídeo después de codificar y decodificar sea ligeramente diferente del original. Cuando la salida decodificada no es exactamente igual a la entrada original, se dice que el sistema es **con pérdidas** (*lossy*). Si la entrada y la salida son idénticas, el sistema es **sin pérdidas** (*lossless*). Los sistemas con pérdidas son importantes porque aceptar una pequeña pérdida de información pude ofrecer ventajas enormes en la relación de compresión posible.

Codificación entrópica

Los esquemas de compresión pueden dividirse en dos categorías generales: codificación entrópica y codificación por fuente. Los estudiaremos por turno.

La **codificación entrópica** simplemente manipula las corrientes de bits sin importar lo que significan; es una técnica general, sin pérdidas y completamente reversible que puede aplicarse a todos los tipos de datos. La ilustraremos mediante tres ejemplos.

Nuestro primer ejemplo de codificación entrópica es la **codificación por longitud de series**. En muchos tipos de datos son comunes las cadenas de símbolos repetidos (bits, números, etc.). Éstas pueden reemplazarse por un marcador especial no permitido en los datos en otras condiciones, seguido del símbolo que indica la serie, y luego la cantidad de veces que ocurre. Si el marcador especial mismo ocurre en los datos, se duplica (como en el relleno de caracteres). Por ejemplo, considere la siguiente cadena de dígitos decimales:

```
3150000000000008458711111111111116354674000000000000000000000065
```

Si ahora introducimos A como el marcador y usamos números de dos dígitos para la cuenta de repeticiones, podemos codificar la cadena de dígitos anterior como

```
315A01284587A11316354674A02265
```

Aquí la codificación por longitud de serie ha recortado a la mitad la cadena de datos.

Las series son comunes en multimedia. En audio, el silencio con frecuencia se representa mediante series de ceros. En vídeo, series del mismo color ocurren en las tomas del cielo, paredes y muchas superficies planas. Todas estas series pueden comprimirse considerablemente.

Nuestro segundo ejemplo de codificación entrópica es la **codificación estadística**. Con esto nos referimos al uso de un código corto para representar símbolos comunes y uno largo para

representar los poco frecuentes. El código Morse usa este principio, siendo E un • y Q -- • -, etc. La codificación Huffman y el algoritmo Ziv-Lempel usados por el programa Compress de UNIX también usan codificación estadística.

Nuestro tercer ejemplo de codificación entrópica es la codificación **CLUT** (*Color Look Up Table*, **tabla de búsqueda de colores**). Considere una imagen que usa codificación RGB con 3 bytes/píxel. En teoría, la imagen podría contener hasta 2^{24} valores de color diferentes. En la práctica, normalmente contendrá muchos menos valores, especialmente si la imagen es una caricatura o un dibujo generado por computadora, en lugar de una fotografía. Supóngase que sólo se usan en realidad 256 valores de colores. Puede lograrse una compresión de casi un factor de tres construyendo una tabla de 768 bytes que liste los valores RGB de los 256 colores usados, y representando luego cada píxel por el índice de su valor RGB en la tabla. Aquí vemos un ejemplo claro en el que la codificación es más lenta que la decodificación porque la codificación requiere una búsqueda en la tabla, mientras que la decodificación puede hacerse con una sola operación de indización.

Codificación por fuente

Ahora llegamos a la **codificación por fuente**, que aprovecha las propiedades de los datos para producir mayor compresión (generalmente con pérdidas). Aquí también ilustraremos el concepto con tres ejemplos. Nuestro primer ejemplo es la **codificación diferencial**, en la que una secuencia de valores (por ejemplo, muestras de audio) se codifican representando cada una como la diferencia respecto al valor previo. La modulación por código de pulso diferencial, que vimos en el capítulo 2, es un ejemplo de esta técnica. Es con pérdidas porque la señal puede saltar tanto entre dos valores consecutivos que la diferencia no quepa en el campo proporcionado para expresar las diferencias, por lo que, cuando menos, se registrará un valor incorrecto y se perderá un poco de información.

La codificación diferencial es un tipo de codificación por fuente porque aprovecha la propiedad de que son poco probables los saltos grandes entre puntos de datos consecutivos. No todas las secuencias de números tienen esta propiedad. Un ejemplo en el que falta esta propiedad es una lista generada por computadora de números telefónicos aleatorios que será usada por una empresa de ventas por teléfono para molestar a la gente durante la comida. Las diferencias entre números telefónicos consecutivos de la lista requerirán tantos bits para representarse como los números mismos.

Nuestro segundo ejemplo de codificación por fuente consiste en **transformaciones**. Gracias a la transformación de señales de un dominio a otro, la compresión puede volverse mucho más fácil. Considere, por ejemplo, la transformación de Fourier de la figura 2-1(e). Aquí una función de tiempo se representa como una lista de amplitudes. Dados los valores exactos de todas las amplitudes, la función original puede reconstruirse perfectamente. Sin embargo, dados sólo los valores de, digamos, las primeras ocho amplitudes redondeadas a dos lugares decimales, aún puede ser posible reconstruir la señal tan bien que el escucha no pueda saber que se ha perdido un poco de información. La ganancia es que la transmisión de ocho amplitudes requiere muchos menos bits que la transmisión de la forma de onda muestreada.

Las transformaciones también se aplican a datos de imágenes de dos dimensiones. Supóngase que la matriz de 4 × 4 de la figura 7-78(a) representa los valores de la escala de grises de una imagen monocromática. Podemos transformar estos datos restando el valor de la esquina superior izquierda de todos los elementos excepto él mismo, como se muestra en la figura 7-78(b). Esta transformación podría ser útil si se usa codificación de longitud variable. Por ejemplo, los valores entre −7 y +7 podrían codificarse con números de 4 bits, y los valores entre 0 y 255 podrían codificarse como un código especial de 4 bits (−8) seguido de un número de 8 bits.

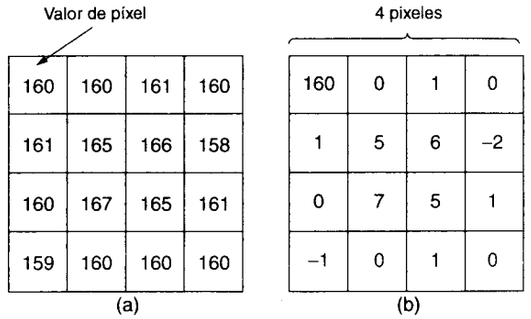


Figura 7-78. (a) Valores de píxel para parte de una imagen. (b) Transformación en la que el elemento de la esquina superior izquierda se resta de todos los elementos menos él mismo.

Aunque esta transformación sencilla es sin pérdidas, otras, más útiles, no lo son. Una transformación espacial de dos dimensiones muy importante es la **DCT** (*Discrete Cosine Transformation*, **transformación por coseno discreto**) (Feig y Winograd, 1992). Esta transformación tiene la propiedad de que, con imágenes sin discontinuidades pronunciadas, la mayor parte de la capacidad espectral está en los primeros términos, permitiendo que los últimos se ignoren sin mucha pérdida de información. Regresaremos a la DCT pronto.

Nuestro tercer ejemplo de codificación por fuente es la **cuantización vectorial**, que también puede aplicarse directamente a los datos de imagen. Aquí, la imagen se divide en rectángulos de tamaño fijo. Además de la imagen misma, necesitamos una tabla de rectángulos del mismo tamaño que los rectángulos de imagen (probablemente construidos a partir de la imagen). Esta tabla se llama **libro de códigos**. Cada rectángulo se transmite buscándolo en el libro de códigos y simplemente mandando el índice en lugar del rectángulo. Si el libro de códigos se crea dinámicamente (es decir, por imagen), debe transmitirse también. Ciertamente, si una pequeña cantidad de rectángulos domina la imagen, son posibles grandes ahorros en ancho de banda.

Un ejemplo de cuantización vectorial se muestra en la figura 7-79. En la figura 7-79(a) tenemos una malla de rectángulos de tamaño no especificado. En la figura 7-79(b) tenemos el libro de códigos. La corriente de salida es sólo la lista de enteros 001022032200400 mostrada en la figura 7-79(c). Cada entero representa una entrada del libro de códigos.

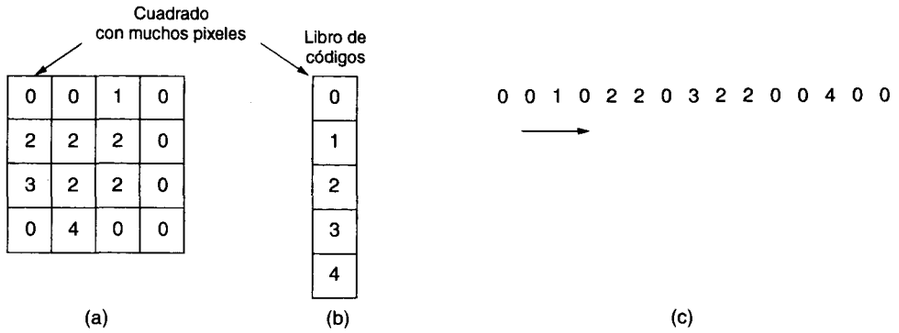


Figura 7-79. Ejemplo de cuantización vectorial. (a) Imagen dividida en marcos. (b) Libro de códigos de la imagen. (c) La imagen codificada.

En cierto sentido, la cuantización vectorial es sólo una generalización de dos dimensiones de la CLUT. Sin embargo, la verdadera diferencia es lo que ocurre si no se encuentra una equivalencia. Son posibles tres estrategias. La primera es simplemente usar la mejor equivalencia. La segunda es usar la mejor equivalencia y agregar alguna información sobre la manera de mejorar la equivalencia (por ejemplo, agregar el valor medio real). La tercera es usar la mejor equivalencia y agregar lo que sea necesario para permitir al decodificador la reconstrucción exacta de los datos. Las primeras dos estrategias tienen pérdidas, pero ofrecen alta compresión. La tercera no tiene pérdidas, pero es menos efectiva como algoritmo de compresión. De nuevo vemos que la codificación (equivalencia de patrones) consume mucho más tiempo que la decodificación (indización en una tabla).

Estándar JPEG

El estándar **JPEG** (*Joint Photographic Experts Group*, **grupo conjunto de expertos en fotografía**) para la compresión de imágenes fijas de tono continuo (por ejemplo, fotografías) fue desarrollado por expertos en fotografía trabajando con el auspicio conjunto del ITU, la ISO y el IEC, otro grupo de estándares. Es importante para multimedia porque, a primera vista, el estándar multimedia para imágenes en movimiento, MPEG, es simplemente la codificación JPEG de cada marco por separado, más algunas características extra para la compresión intermarcos y la detección de movimiento. El JPEG se define en el Estándar Internacional 10918.

El JPEG tiene cuatro modos y muchas opciones; se parece más a una lista de compras que a un solo algoritmo. No obstante, para nuestros fines sólo es relevante el modo secuencial con pérdidas, y éste se ilustra en la figura 7-80. Es más, nos concentraremos en la manera en que el JPEG se usa normalmente para codificar imágenes de vídeo RGB de 24 bits y dejaremos fuera algunos de los detalles menores por cuestiones de sencillez.

El paso 1 de la codificación de una imagen con el JPEG es la preparación del bloque. Para ser específicos, supongamos que la entrada JPEG es una imagen RGB de 640 × 480 con 24 bits/píxel, como se muestra en la figura 7-81(a). Puesto que el uso de luminancia y crominancia da

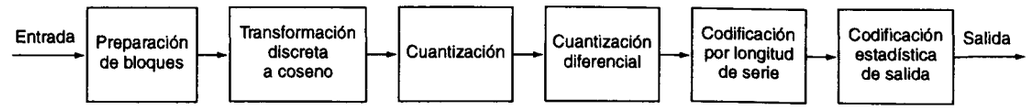


Figura 7-80. Operación del JPEG en el modo secuencial libre.

una mejor compresión, primero calcularemos la luminancia, Y, y las dos crominancias, I y Q (para NTSC), de acuerdo con las siguientes fórmulas:

$$Y = 0.30R + 0.59G + 0.11B$$
$$I = 0.60R - 0.28G - 0.32B$$
$$Q = 0.21R - 0.52G + 0.31B$$

En PAL, las crominancias se llaman U y V, y los coeficientes son diferentes, pero la idea es la misma. SECAM es diferente tanto de NTSC como de PAL.

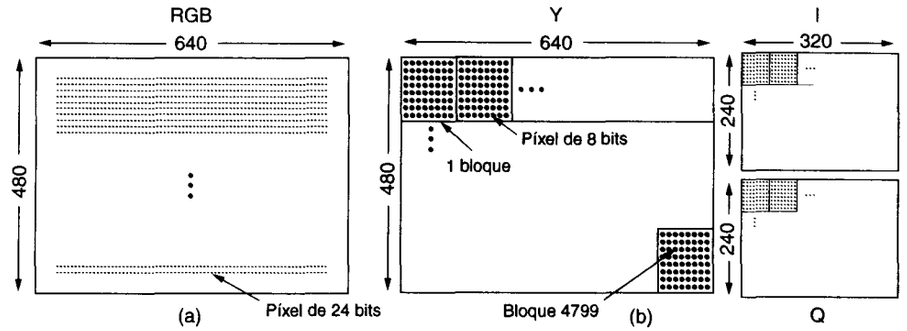


Figura 7-81. (a) Datos de entrada RGB. (b) Tras la preparación de bloques.

Se construyen matrices separadas para Y, I y Q, cada una con elementos en el intervalo de 0 a 255. A continuación se promedian marcos de cuatro pixeles en las matrices I y Q para reducirlos a 320 × 240. Esta reducción tiene pérdidas, pero el ojo apenas lo nota, ya que responde a la luminancia más que a la crominancia; no obstante, comprime los datos en un factor de dos. Ahora se resta 128 a cada elemento de las tres matrices para poner el 0 a la mitad de la gama. Por último, cada matriz se divide en bloques de 8 × 8. La matriz Y tiene 4800 bloques; las otras dos tienen 1200 bloques cada una, como se muestra en la figura 7-81(b).

El paso 2 del JPEG es aplicar individualmente una transformación coseno discreta (DCT) a cada uno de los 7200 bloques. La salida de cada DCT es una matriz 8 × 8 de coeficientes DCT. El elemento DCT (0, 0) es el valor medio del bloque. Los otros elementos indican la cantidad de potencia espectral que hay en cada frecuencia espacial. En teoría, una DCT no tiene pérdidas pero, en la práctica, el uso de números de punto flotante y funciones trascendentales siempre introducen

algún error de redondeo que resulta en una pequeña pérdida de información. Estos elementos normalmente decaen rápidamente al alejarse del origen (0, 0), como se sugiere en la figura 7-82.

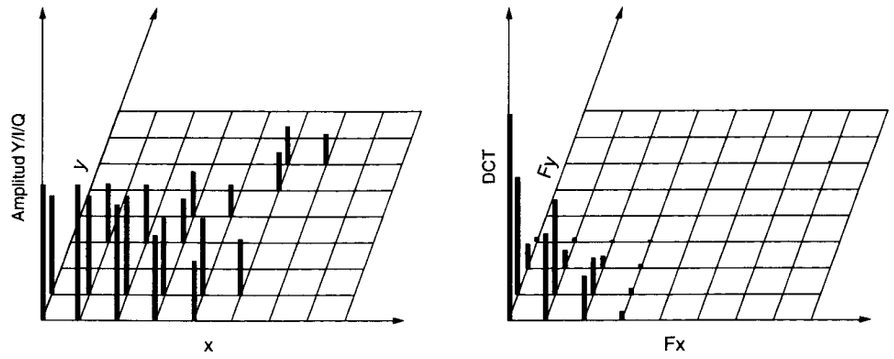


Figura 7-82. (a) Un bloque de la matriz *Y*. (b) Coeficientes DCT.

Una vez completa la DCT, el JPEG sigue con el paso 3, llamado **cuantización**, en el que se eliminan los coeficientes DCT menos importantes. Esta transformación (con pérdidas) se hace dividiendo cada uno de los coeficientes de la matriz DCT de 8 × 8 entre un peso tomado de una tabla. Si todos los pesos son 1, la transformación no hace nada. Sin embargo, si los pesos aumentan marcadamente desde el origen, las frecuencias espaciales más altas se descartarán rápidamente.

En la figura 7-83 se da un ejemplo de este paso. Aquí vemos la matriz DCT inicial, la tabla de cuantización y el resultado obtenido al dividir cada elemento DCT entre el elemento correspondiente de la tabla de cuantización. Los valores de la tabla de cuantización no son parte del estándar JPEG. Cada aplicación debe proporcionar sus propios valores, permitiéndole controlar el equilibrio pérdidas-compresión.

El paso cuatro reduce el valor (0, 0) de cada bloque (el de la esquina superior izquierda) reemplazándolo por el valor de su diferencia respecto al elemento correspondiente del bloque previo. Dado que estos elementos son las medias de sus respectivos bloques, deben cambiar lentamente, por lo que al tomarse sus valores diferenciales se debe reducir la mayoría de ellos a valores pequeños. No se calculan diferenciales de los otros valores. Se llama a los valores (0, 0) componentes de cc; los otros son las componentes de ca.

El paso 5 hace lineales los 64 elementos y aplica codificación por longitud de serie a la lista. El barrido del bloque de izquierda a derecha y luego de arriba abajo no concentra los ceros, por lo que se usa un patrón de barrido en zigzag, como se muestra en la figura 7-84. En este ejemplo, el patrón de zigzag produce 38 ceros consecutivos el final de la matriz. Esta cadena puede reducirse a una sola cuenta diciendo que hay 38 ceros.

Ahora tenemos una lista de números que representan la imagen (en espacio de transformación). El paso 6 aplica codificación de Huffman a los números para su almacenamiento o transmisión.

Coeficientes DCT

150	80	40	14	4	2	1	0
92	75	36	10	6	1	0	0
52	38	26	8	7	4	0	0
12	8	6	4	2	1	0	0
4	3	2	0	0	0	0	0
2	2	1	1	0	0	0	0
1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Coeficientes cuantizados

150	80	20	4	1	0	0	0
92	75	18	3	1	0	0	0
26	19	13	2	1	0	0	0
3	2	2	1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Tabla de cuantización

1	1	2	4	8	16	32	64
1	1	2	4	8	16	32	64
2	2	2	4	8	16	32	64
4	4	4	4	8	16	32	64
8	8	8	8	8	16	32	64
16	16	16	16	16	16	32	64
32	32	32	32	32	32	32	64
64	64	64	64	64	64	64	64

Figura 7-83. Cálculo de los coeficientes DCT cuantizados.

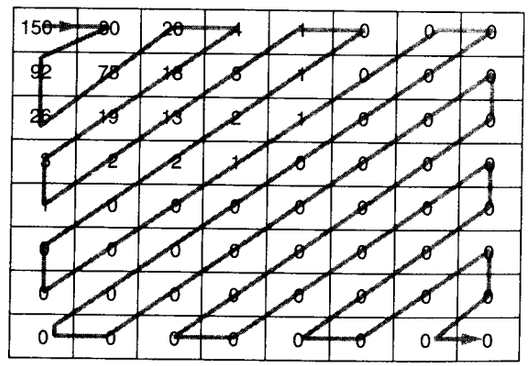


Figura 7-84. Orden en que se transmiten los valores cuantizados.

El JPEG puede parecer complicado, pero eso es porque *es* complicado. Aun así, dado que con frecuencia produce una compresión de 20:1 o mejor, se usa ampliamente. La decodificación de una imagen JPEG ejecuta hacia atrás el algoritmo. A diferencia de algunos de los algoritmos de compresión que hemos visto, el JPEG es más o menos simétrico: la decodificación tarda tanto como la codificación.

Como detalle interesante, debido a las propiedades matemáticas de la DCT, es posible ejecutar ciertas transformaciones (por ejemplo, rotación de imagen) sobre la matriz transformada sin regenerar la imagen original. Estas transformaciones se estudian en (Shen y Sethi, 1995). También se aplican propiedades semejantes al audio comprimido MPEG (Broadhead y Owen, 1995).

Estándar MPEG

Por último, llegamos al corazón del asunto: los estándares **MPEG** (*Motion Picture Experts Group*, **grupo de expertos en imágenes en movimiento**). Éstos son los algoritmos principales usados para comprimir vídeos y han sido estándares internacionales desde 1993. Puesto que las películas contienen tanto imágenes como sonido, el MPEG puede comprimir tanto audio como vídeo pero, dado que el vídeo usa más ancho de banda y también contiene mayor redundancia que el audio, nos enfocaremos principalmente en la compresión de vídeo MPEG en lo que sigue.

El primer estándar terminado fue el MPEG-1 (Estándar Internacional 11172); su meta fue producir salida con calidad de videograbadora (352 × 240 para NTSC) usando una tasa de bits de 1.2 Mbps. Puesto que, como vimos antes, el vídeo no comprimido por sí solo puede requerir 472 Mbps, reducirlo a 1.2 Mbps no es nada trivial, aun con esta menor definición. El MPEG-1 puede transmitirse por líneas de transmisión de par trenzado a distancias modestas. El MPEG-1 también se usa para almacenar películas en CD-ROM, en formatos CD-I y CD-Vídeo.

El siguiente estándar de la familia MPEG fue el MPEG-2 (estándar internacional 13818), que se diseñó originalmente para comprimir vídeo con calidad de difusión a 4-6 Mbps, de modo que pudiera caber en un canal de difusión NTSC o PAL. Después se expandió el MPEG-2 para manejar definiciones mayores, incluida HDTV. El MPEG-4 es para videoconferencias de mediana definición con tasas de marco bajas (10 marcos/seg) y a bajo ancho de banda (64 kbps). Esto permite sostener videoconferencias por un solo canal N-ISDN B. Dada la numeración, podría pensarse que el siguiente estándar será el MPEG-8. En realidad, la ISO está numerándolos linealmente, no exponencialmente. Originalmente existió el MPEG-3, que estaba dirigido a la HDTV, pero ese proyecto luego se canceló, y la HDTV se agregó al MPEG-2.

Los principios básicos del MPEG-1 y el MPEG-2 son parecidos, pero los detalles son diferentes. A primera vista, el MPEG-2 es un supergrupo del MPEG-1, con características, formatos de marco y opciones de codificación adicionales. Es probable que a la larga el MPEG-1 dominará en las películas en CD-ROM y el MPEG-2 dominará en la transmisión de vídeo a largas distancias. Estudiaremos el MPEG-1 primero y el MPEG-2 después.

El MPEG-1 tiene tres partes: audio, vídeo y sistema, que integra los otros dos, como se muestra en la figura 7-85. Los codificadores de audio y vídeo funcionan independientemente, lo

que hace surgir la cuestión de cómo se sincronizan las dos cadenas en el receptor. Este problema se resuelve teniendo un reloj de sistema de 90 kHz que suministra el valor de tiempo a ambos codificadores. Estos valores son de 33 bits, para permitir que las películas duren 24 horas sin dar la vuelta. Estas marcas de tiempo se incluyen en la salida codificada y se propagan hasta el receptor, que puede usarlas para sincronizar las corrientes de audio y vídeo.

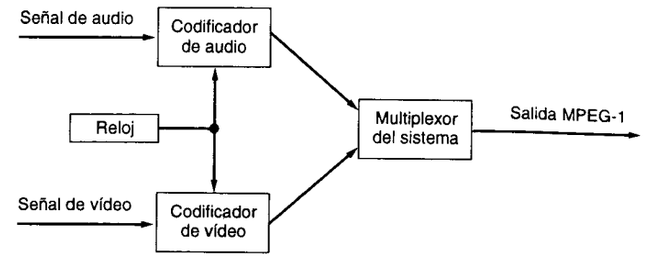


Figura 7-85. Sincronización de las corrientes de audio y vídeo en el MPEG-1.

La compresión de audio MPEG se hace muestreando la forma de onda a 32 kHz, 44.1 kHz o 48 kHz. Se puede manejar sonido monofónico, estéreo disjunto (cada canal comprimido por separado) o estéreo combinado (explotación de la redundancia intercanal). El algoritmo se organiza en tres capas, cada una de las cuales aplica optimizaciones adicionales para obtener mayor compresión (a mayor costo). La capa 1 es el esquema básico. Esta capa se usa, por ejemplo, en el sistema de cinta digital DCC. La capa 2 agrega asignación de bits avanzada al esquema básico. Se usa para audio de CD-ROM y pistas sonoras de cine. La capa 3 agrega filtros híbridos, cuantización no uniforme, codificación Huffman y otras técnicas avanzadas.

El MPEG puede comprimir un CD de *rock and roll* a 96 kbps sin pérdidas perceptibles en la calidad del audio, aun para fanáticos del *rock* que aún no han sufrido pérdidas auditivas. Para un concierto de piano se requieren cuando menos 128 kbps. Son diferentes porque la relación señal a ruido del *rock and roll* es mucho mayor que en un concierto de piano (desde el punto de vista de la ingenía, al menos).

La compresión de audio se logra ejecutando una transformación de Fourier rápida con la señal de audio para transformarla del dominio del tiempo al dominio de la frecuencia. El espectro resultante se divide entonces en 32 bandas de frecuencia, que se procesan por separado. Cuando hay presentes dos canales estéreo, también puede aprovecharse la redundancia inherente a tener dos fuentes de audio altamente traslapadas. La corriente de audio MPEG-1 resultante se puede ajustar desde 32 kbps hasta 448 kbps. En (Pan, 1995) se presenta una introducción al proceso.

Consideremos ahora la compresión de vídeo MPEG-1. Existen dos clases de redundancia en las películas: espacial y temporal. El MPEG-1 aprovecha ambas. La redundancia espacial puede utilizarse simplemente codificando por separado cada marco mediante JPEG. Este enfoque se ocasionalmente, sobre todo cuando se requiere acceso aleatorio a cada marco, como en la

edición de producciones de vídeo. En este modo, se puede lograr un ancho de banda comprimido del orden de 8 a 10 Mbps.

Puede lograrse compresión adicional aprovechando el hecho de que los marcos consecutivos a menudo son casi idénticos. Este efecto es menor que lo que podría pensarse, puesto que muchos cineastas hacen cortes entre escenas cada 3 o 4 segundos (tome el tiempo de una película y cuente las escenas). No obstante, incluso una serie de 75 marcos muy parecidos ofrece el potencial de una reducción importante respecto a la simple codificación de cada marco por separado mediante JPEG.

En las escenas en las que la cámara y el fondo son estacionarios y uno o dos actores se mueven con lentitud, casi todos los pixeles serán idénticos de un marco a otro. Aquí será suficiente la simple resta de cada marco del anterior y la aplicación de JPEG a la diferencia. Sin embargo, en las escenas en las que la cámara hace una panorámica o un acercamiento, esta técnica falla gravemente. Lo que se necesita es una manera de compensar este movimiento. Esto es precisamente lo que hace el MPEG, y es la diferencia principal entre el MPEG y el JPEG.

La salida del MPEG-1 consiste en cuatro tipos de marco:

1. Marcos I (intracodificados): imágenes fijas autocontenidas codificadas en JPEG.
2. Marcos P (predictivos): diferencia bloque por bloque con el marco anterior.
3. Marcos B (bidireccionales): diferencias con el marco anterior y el siguiente.
4. Marcos D (codificación CD): promedios de bloque usados para avance rápido.

Los marcos I son simplemente imágenes fijas codificadas con JPEG, usando también luminancia de definición completa y crominancia de definición media sobre cada eje. Es necesario hacer que los marcos I aparezcan periódicamente en la corriente de salida por tres razones. Primera, el MPEG-1 puede usarse para multitransmisión, sintonizándose los usuarios a voluntad. Si todos los marcos dependieran de sus antecesores remontándose al primer marco, cualquiera que no recibiera el primer marco no podría decodificar nunca los marcos subsiguientes. Segunda, si un marco se recibiera con error, no sería posible ninguna decodificación posterior. Tercera, sin marcos I, al hacer un avance o retroceso rápido, el decodificador tendría que calcular cada marco por el que pasa para conocer el valor completo de aquel en el que se detiene. Por estas tres razones, se insertan marcos I en la salida una o dos veces por segundo.

Los marcos P, en contraste, codifican las diferencias entre marcos; se basan en la idea de los **macrobloques**, que cubren 16×16 pixeles de espacio de luminancia y 8×8 pixeles de espacio de crominancia. Un macrobloque se codifica buscando en el marco previo algo igual a o ligeramente diferente de él.

Un ejemplo de caso en el que serían útiles los marcos P se ve en la figura 7-86. Aquí vemos tres marcos consecutivos que tienen el mismo fondo, pero en los que cambia la posición de una persona. Los macrobloques que contienen el fondo serán exactamente iguales, pero los macrobloques que contienen la persona estarán desfasados en alguna cantidad desconocida y tendrán que rastrearse.

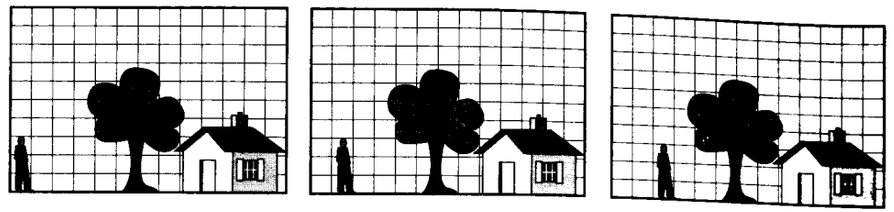


Figura 7-86. Tres marcos consecutivos.

El estándar MPEG-1 no especifica la manera de efectuar la búsqueda, ni qué tan profunda o buena debe ser para que sirva. Éstas son decisiones de cada implementación. Por ejemplo, una implementación podría buscar un macrobloque en la posición actual, pero del marco anterior, y con todas las demás posiciones desplazadas $\pm\Delta x$ en la dirección x y $\pm\Delta y$ en la dirección y. Para cada posición, podría calcularse la cantidad de equivalencias en la matriz de luminancia. La posición con el puntaje más alto se declararía ganadora, siempre que estuviera por encima de un umbral predefinido. En caso contrario, se diría que falta el macrobloque. Por supuesto, pueden usarse algoritmos mucho más refinados.

Si se encuentra un macrobloque, se codifica tomando la diferencia respecto a su valor en el marco previo (para la luminancia y ambas crominancias). Estas matrices de diferencias son el objeto de la transformación por coseno discreto, cuantización, codificación por longitud de serie y codificación Huffman, al igual que en el JPEG. El valor del macrobloque en la cadena de salida es entonces el vector de movimiento (la distancia que se movió el macrobloque de su posición previa en cada sentido), seguido de la lista de codificación Huffman de los números. Si el macrobloque no se encuentra en el marco previo, se codifica el valor actual con JPEG, igual que en un marco I.

Es claro que este algoritmo es altamente asimétrico. Las implementaciones están en libertad de intentar todas las posiciones del marco previo si lo desean, en un intento desesperado por localizar todos los macrobloques previos. Este enfoque reducirá al mínimo la cadena MPEG-1 codificada al costo de una codificación muy lenta. Este enfoque podría estar bien para la codificación de una sola vez de una cineteca, pero sería terrible para las videoconferencias en tiempo real.

De la misma manera, cada implementación está en libertad de decidir lo que constituye un macrobloque "encontrado". Esta libertad permite a los implementadores competir según la calidad y velocidad de sus algoritmos, pero siempre produce MPEG-1 conformante. Sea cual sea el algoritmo de búsqueda usado, la salida final es la codificación JPEG del macrobloque actual, o la codificación JPEG de la diferencia entre el macrobloque actual y el del marco previo, con un desplazamiento especificado respecto al marco actual.

Hasta ahora, la decodificación del MPEG-1 es directa. La decodificación de marcos I es igual a la decodificación de marcos JPEG. La decodificación de marcos P requiere que el decodificador maneje en *buffer* el marco previo, y luego construya el nuevo en un segundo *buffer*

con base en macrobloques completamente codificados y macrobloques que contienen diferencias respecto al marco previo. El nuevo marco se ensambla macrobloque por macrobloque.

Los marcos B son parecidos a los marcos P, excepto que permiten que el macrobloque de referencia esté en un marco previo o en un marco posterior. Esta libertad adicional permite mejorar la compensación del movimiento y es útil también cuando pasan objetos por delante o detrás de otros objetos. Para ejecutar la codificación de marcos B, el codificador necesita tener a la vez tres marcos decodificados en la memoria: el anterior, el actual y el siguiente. Aunque los marcos B producen la mejor compresión, no todas las implementaciones los reconocen.

Los marcos D sólo se usan para visualizar una imagen de baja definición al hacer un reembobinado o avance rápido. Hacer la decodificación MPEG-1 normal en tiempo real ya es bastante difícil. Esperar que el decodificador lo haga mientras trabaja a 10 veces su velocidad normal es pedir demasiado. Cada entrada de marco D simplemente es el valor promedio de un bloque, sin mayor codificación, simplificando la presentación en tiempo real. Este mecanismo es importante para permitir que la gente barra un vídeo a alta velocidad en busca de una escena en particular.

Habiendo terminado nuestro tratamiento del MPEG-1, pasemos al MPEG-2. La codificación MPEG-2 es parecida en lo fundamental a la codificación MPEG-1, con marcos I, marcos P y marcos B. Sin embargo, no se reconocen los marcos D. También, la transformación coseno es de 10×10 en lugar de 8×8, para dar 50% más de coeficientes y, por tanto, mayor calidad. Puesto que el MPEG-2 está dirigido a la televisión difundida al igual que a las aplicaciones en CD-ROM, reconoce imágenes tanto progresivas como entrelazadas, mientras que el MPEG-1 reconoce sólo las imágenes progresivas. También son diferentes otros detalles menores entre los dos estándares.

En lugar de reconocer un solo nivel de definición, el MPEG-2 reconoce cuatro: baja (352×240), principal (720×480), alta-1440 (1440×1152) y alta (1920×1080). La baja definición es para videograbadoras y compatibilidad hacia abajo con el MPEG-1. La principal es la normal para difusión NTSC. Las otras dos son para HDTV.

Además de tener cuatro niveles de definición, el MPEG-2 también maneja cinco **perfiles**. Cada perfil está pensado para algún área de aplicación. El perfil principal es para propósito general, y probablemente la mayoría de los *chips* estará optimizada para el perfil principal y el nivel de definición principal. El perfil sencillo es parecido al principal, excepto que excluye el uso de marcos B para simplificar el *software* de codificación y decodificación. Los otros perfiles tienen que ver con la escalabilidad y la HDTV. Los perfiles son diferentes en términos de la presencia o ausencia de marcos B, definición de crominancia y escalabilidad de la cadena de bits codificada a otros formatos.

La tasa de datos comprimidos de cada combinación de definición y perfil es diferente; va desde unos 3 Mbps hasta 100 Mbps para HDTV. El caso normal es de unos 3 a 4 Mbps. En (Pancha y El Zarki, 1994) se dan algunos datos del desempeño de MPEG.

El MPEG-2 tiene una manera más general de multiplexar audio y vídeo que el modelo MPEG-1 de la figura 7-85; define una cantidad ilimitada de corrientes elementales, incluidos vídeo y audio, pero también corrientes de datos que deben sincronizarse con el audio y el vídeo;

por ejemplo, subtítulos en varios idiomas. Cada una de las corrientes se empaca primero con marcas de tiempo. Se muestra un ejemplo sencillo de dos corrientes en la figura 7-87.

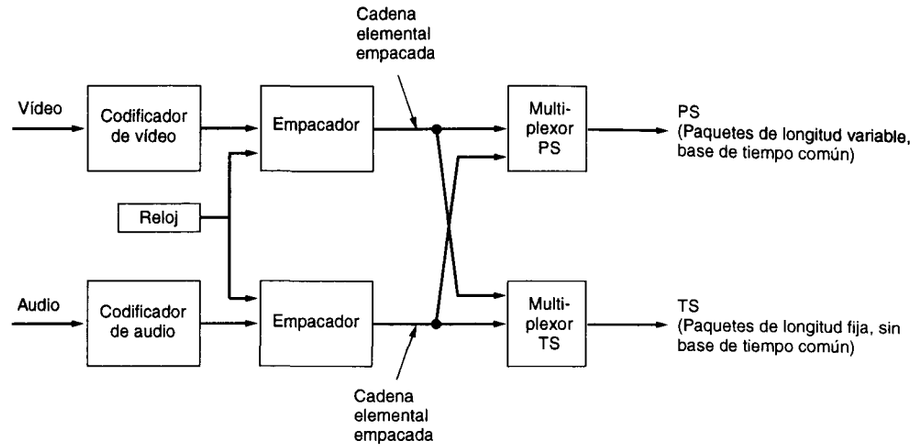


Figura 7-87. Multiplexión de dos corrientes en MPEG-2.

La salida de cada empacador es una **PES** (*Packetized Elementary Stream*, **corriente elemental empacada**). Cada PES tiene unos 30 campos de cabecera e indicadores, incluidas longitudes, identificadores de corriente, control de cifrado, estado de derechos de autor, marcas de tiempo y un CRC.

Las corrientes PES de audio, vídeo y posiblemente datos se multiplexan juntas en una sola corriente de salida para su transmisión. Se definen dos tipos de corriente. La **corriente de programa** MPEG-2 es parecida a la corriente de los sistemas MEPG-1 de la figura 7-85; se usa para multiplexar varias corrientes elementales que tienen una base común y tienen que presentarse de una manera sincronizada. La corriente de programa usa paquetes grandes de longitud variable

La otra corriente MPEG-2 es la **corriente de transporte**. Se usa para multiplexar corrientes (incluidas corrientes de programa) que no usan una base de tiempo común. Los paquetes de corriente de transporte son de longitud fija (188 bytes), para simplificar la limitación del efecto de los paquetes dañados o perdidos durante la transmisión.

Vale la pena indicar que todos los esquemas de codificación que hemos estudiado se basan en el modelo de codificación con pérdidas seguida de transmisión sin pérdidas. Ni el JPEG ni el MPEG pueden recuperarse de paquetes perdidos o dañados sin que se note el problema. Un enfoque diferente para la transmisión de imágenes es transformar éstas de manera que se separe la información importante de la menos importante (como la DCT, por ejemplo). Luego se agrega

una cantidad considerable de redundancia (incluso paquetes duplicados) a la información importante y ninguna a la menos importante. Si se pierden o se alteran algunos paquetes, aún puede ser posible presentar imágenes razonables sin retransmitir. Estas ideas se describen con mayor detalle en (Danskin *et al.*, 1995), y se aplican especialmente a la multitransmisión, en la que la realimentación de cada receptor de todos modos es imposible.

7.7.4. Vídeo a solicitud

El vídeo a solicitud a veces se compara con una tienda electrónica de renta de vídeos. El usuario (cliente) selecciona cualquiera de una gran cantidad de vídeos y comienza a verlo de inmediato. No se necesita ningún viaje a la tienda. Sobra decir que la implementación del vídeo a solicitud es un tanto más complicada que su descripción. En esta sección haremos un repaso general de los conceptos básicos y de su implementación. Puede encontrarse una descripción de una implementación real en (Nelson y Linton, 1995). Otras referencias relevantes son (Chang *et al.*, 1994; Hodge *et al.*, 1993, y Little y Venkatesh, 1994).

¿Es el vídeo a solicitud en realidad como rentar un vídeo, o se parece más a seleccionar una película de un sistema de cable de 500 o 5000 canales? La respuesta tiene implicaciones técnicas importantes. En particular, los usuarios que rentan vídeos están acostumbrados a la idea de poder detener el vídeo, hacer un viaje rápido a la cocina o al baño, y luego continuar a partir de donde detuvieron el vídeo. Los espectadores de televisión no esperan poner los programas en pausa.

Si el vídeo a solicitud va a competir con éxito contra las tiendas de renta de vídeo, puede ser necesario que los usuarios detengan, inicien y rebobinen los vídeos a voluntad. Para ello, el proveedor de vídeos tendrá que transmitir una copia individual a cada quien.

Por otra parte, si el vídeo a solicitud se considera más como una televisión avanzada, entonces puede ser suficiente hacer que el proveedor de vídeo inicie cada vídeo popular, digamos, cada 10 minutos, y luego lo exhiba sin parar. Un usuario que desee ver un vídeo popular podría tener que esperar hasta 10 minutos para que comience. Aunque no es posible la pausa/reinicio aquí, un espectador que regrese a la sala tras una interrupción corta puede pasar a otro canal que muestre el mismo vídeo, pero con 10 minutos de retraso. Una parte del material se repetirá, pero no se perderá nada. Este esquema se llama **casi vídeo a solicitud**, y ofrece la posibilidad de un costo mucho menor, puesto que la misma alimentación del servidor de vídeo puede llegar a muchos usuarios al mismo tiempo. La diferencia entre vídeo a solicitud y casi vídeo a solicitud es parecida a la diferencia entre manejar su propio auto y tomar el autobús.

Ver películas (casi) a solicitud es una de una gran gama de servicios nuevos que podrían hacerse realidad una vez que estén disponibles las redes de ancho de banda amplio. El modelo general que usa mucha gente se ilustra en la figura 7-88. Aquí vemos una red de *backbone* de área amplia (nacional o internacional) de alto ancho de banda como centro del sistema. Conectadas a ella hay miles de redes de distribución locales, como TV por cable o sistemas de distribución telefónica. Los sistemas de distribución locales llegan hasta las casas de la gente, donde terminan en **cajas de control**, que de hecho son potentes computadoras personales especializadas.

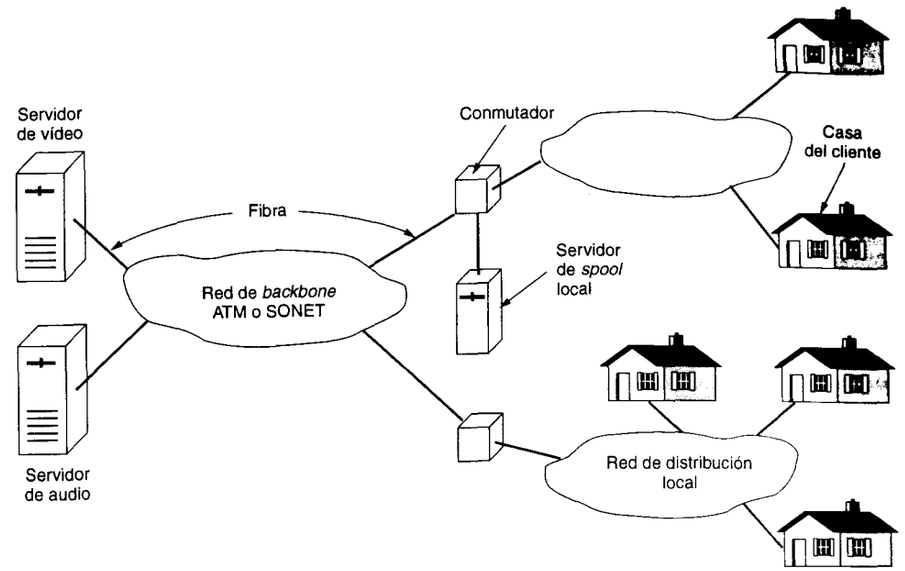


Figura 7-88. Esquema general de un sistema de vídeo a solicitud.

Conectados al *backbone* mediante fibras ópticas de alto ancho de banda hay miles de proveedores de información. Algunos de estos ofrecerán vídeo o audio de pago por evento. Otros ofrecerán servicios especializados, como compras desde casa (con la capacidad de girar una lata de sopa y ver la lista de ingredientes, o ver un fragmento de vídeo sobre el modo de operación de una podadora operada por gasolina). Si duda, pronto habrá la disponibilidad de deportes, noticias, capítulos viejos del "Chapulín Colorado", acceso a la WWW e innumerables otras posibilidades.

También incluidos en el sistema hay servidores se *spool* de E/S locales que permitirán acercar los vídeos al usuario, a fin de ahorrar ancho de banda durante las horas pico. La manera en que encajarán estas partes y quién será el dueño de qué son temas de intenso debate en la industria. A continuación examinaremos el diseño de las partes principales del sistema: los servidores de vídeo, la red de distribución y las cajas de control.

Servidores de vídeo

Para tener (casi) vídeo a solicitud, necesitamos **servidores de vídeo** capaces de almacenar y sacar una gran cantidad de películas simultáneamente. La cantidad total de películas que se han filmado se estima en 65,000 (Minoli, 1995). Al comprimirse con MPEG-2, una película normal ocupa unos 4 GB de almacenamiento, por lo que 65,000 de ellas requerirán unos 260 terabytes. Sume a esto todos los programas de televisión, filmaciones de deportes, noticieros, catálogos

parlantes de compras, etc., y queda claro que tenemos un problema de almacenamiento de proporciones mayúsculas en nuestras manos.

La manera más sencilla de almacenar volúmenes grandes de información es en cinta magnética. Siempre ha sido el caso y probablemente seguirá siéndolo. Una cinta DAT puede almacenar 8 GB (dos películas) a un costo de 5 dólares/gigabyte. Hay en el mercado servidores mecánicos grandes que almacenan miles de cintas y tienen un brazo de robot para traer cualquier cinta e introducirla en la unidad de cinta. El problema de estos sistemas es el tiempo de acceso (especialmente para la segunda película en la cinta), la tasa de transferencia y la cantidad limitada de unidades de cinta (para servir n películas a la vez, se requieren n unidades).

Afortunadamente, la experiencia de las tiendas de renta de vídeos, bibliotecas públicas y otras organizaciones similares demuestra que no todos los elementos son igualmente populares. Experimentalmente, cuando hay N películas disponibles, la fracción de todas las solicitudes que pide el elemento número k en popularidad es de aproximadamente C/k (Chervenak, 1994). Aquí, C se calcula para normalizar la suma a 1, es decir

$$C = 1/(1 + 1/2 + 1/3 + 1/4 + 1/5 + \cdots + 1/N)$$

Por tanto, la película más vista es siete veces más popular que la película número siete. Este resultado se conoce como **ley de Zipf** (Zipf, 1949).

El hecho de que algunas películas sean mucho más populares que otras sugiere una solución posible en forma de jerarquía de almacenamiento, como se muestra en la figura 7-89. Aquí, el desempeño aumenta a medida que se sube en la jerarquía.

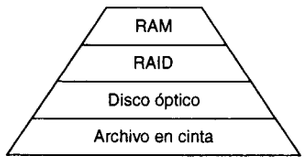


Figura 7-89. Jerarquía de almacenamiento de un servidor de vídeo.

Una alternativa a la cinta es el almacenamiento óptico. Los CD-ROM actuales contienen sólo 650 MB, pero la siguiente generación podrá contener unos 4 GB, lo que los hará adecuados para la distribución de películas MPEG-2. Aunque los tiempos de búsqueda son lentos en comparación con los discos magnéticos (100 mseg contra 10 mseg), su bajo costo y alta confiabilidad hace que los servidores ópticos con miles de CD-ROM sean una buena alternativa a la cinta para las películas de mayor uso.

Siguen los discos magnéticos. Éstos tienen tiempos de acceso cortos (10 mseg), altas tasas de transferencia (10 MB/seg) y capacidades sustanciales (10 GB), lo que los hace adecuados para guardar películas que en realidad se están transmitiendo (en oposición a simplemente estar almacenadas en caso de que alguien las quiera). Su desventaja principal es el alto costo de almacenar películas a las que pocas veces se accede.

Hasta arriba de la pirámide de la figura 7-89 está la RAM. La RAM es el medio de almacenamiento más rápido, pero también el más caro; es el más adecuado para películas de las que se están enviando partes distintas a destinos diferentes al mismo tiempo (por ejemplo, vídeo a solicitud a 100 usuarios que comenzaron en momentos diferentes). Cuando los precios de la RAM caigan a 10 dólares por megabyte, una película de 4 GB ocupará 40,000 dólares de RAM, por lo que tener 100 películas en RAM costará 4 millones de dólares por los 400 GB de memoria. Aun así, para un servidor de vídeo de 10 millones de dólares, este costo bien podría valer la pena si cada película tiene suficientes clientes simultáneos.

Dado que un servidor de vídeo en realidad es sólo un dispositivo masivo de E/S en tiempo real, necesita una arquitectura de *hardware* y *software* diferente de la de una PC o estación de trabajo UNIX. La arquitectura del *hardware* de un servidor de vídeo típico se ilustra en la figura 7-90. El servidor tiene una o más CPU RISC de alto desempeño, cada una con un poco de memoria local, una memoria principal compartida, un caché de RAM masivo para películas populares, una variedad de dispositivos de almacenamiento para guardar las películas, y algún *hardware* de red, normalmente una interfaz óptica con una red ATM (o SONET) a OC-3 o mayor. Estos subsistemas se conectan mediante un *bus* de velocidad extremadamente alta (cuando menos 1 GB/seg).

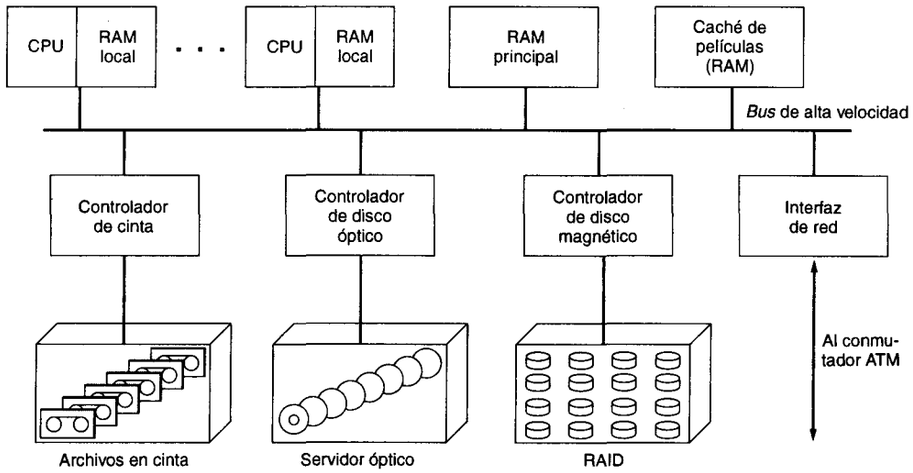


Figura 7-90. Arquitectura del *hardware* de un servidor de vídeo típico.

Ahora veamos brevemente el *software* de un servidor de vídeo. Las CPU se usan para aceptar solicitudes de los usuarios, localizar películas, mover datos entre dispositivos, facturar al cliente y muchas funciones más. Algunas de éstas no son de sincronización crítica, pero muchas otras sí, por lo que algunas de las CPU, si no todas, tendrán que ejecutar un sistema operativo de tiempo real, como un micronúcleo de tiempo real. Estos sistemas normalmente dividen el trabajo en tareas pequeñas, con un plazo de terminación conocido. El planificador puede ejecu-

tar entonces un algoritmo del tipo del siguiente plazo de terminación más cercano o el algoritmo de tasa monotónica (Liu y Layland, 1973).

El *software* de la CPU también define la naturaleza de la interfaz que el servidor presenta a los clientes (servidores de *spool* o cajas de control colocadas sobre el televisor). Son comunes dos diseños. El primero es un sistema tradicional de archivos, en el que los clientes pueden abrir, leer, escribir y cerrar archivos. Aparte de las complicaciones generadas por la jerarquía de almacenamiento y las consideraciones de tiempo real, uno de tales servidores puede tener un sistema de archivos modelado según el de UNIX.

El segundo tipo de interfaz se basa en el modelo de la videograbadora. Los comandos al servidor solicitan que abra, ejecute, haga pausa, avance rápido y rebobine archivos. La diferencia con el modelo UNIX es que, una vez que se ha emitido un comando PLAY, el servidor simplemente sigue sacando datos a velocidad constante, sin requerirse comandos nuevos. El corazón del *software* del servidor de vídeo es el *software* de administración de disco, que tiene dos tareas principales: colocar películas en el disco magnético cuando tienen que traerse de almacenamiento óptico o cinta, y manejar solicitudes de disco para las muchas corrientes de salida. La colocación de las películas es importante, puesto que puede afectar en gran medida el desempeño.

Dos posibles maneras de organizar el almacenamiento en disco son la granja de discos y el arreglo de discos. En la **granja de discos**, cada unidad contiene pocas películas completas. Por cuestiones de desempeño y confiabilidad, cada película debe estar presente en cuando menos dos unidades, tal vez más. La otra organización de almacenamiento es el **arreglo de discos** o **RAID** (*Redundant Array of Inexpensive Disks*, **arreglo redundante de discos baratos**), en el que cada película se divide entre varias unidades, por ejemplo, el bloque 0 en el disco 0, el bloque 1 en el disco 1, etc., con el bloque $n - 1$ la unidad $n - 1$. Tras esto, el ciclo se repite, con el bloque n en la unidad 0, etc. Esta organización se llama **división** (*striping*).

Un arreglo de discos con división tiene varias ventajas respecto a una granja de discos. Primero, las n unidades pueden operar en paralelo, aumentando el desempeño en un factor de n. Segundo, el arreglo puede hacerse redundante agregando una unidad extra a cada grupo de n, donde la unidad redundante contiene el OR EXCLUSIVO bloque por bloque de las otras unidades, a fin de permitir la recuperación completa de datos en caso de falla de una unidad. Por último, se resuelve el problema del equilibrio de la carga (no se requiere colocación manual para evitar que todas las películas populares se encuentren en la misma unidad). Por otra parte, la organización por arreglo de discos es más complicada que la granja de discos y es altamente sensible a diversas fallas; también es poco adecuada para operaciones de videograbadora como rebobinado y avance rápido de una película. Un estudio de simulación que compara las dos organizaciones se da en (Chervenak *et al.*, 1995).

Muy relacionada con la colocación de bloques está la localización de bloques de disco. El esquema UNIX de tener un árbol desbalanceado de bloques de disco a los que se apunta mediante ínodos generalmente es inaceptable, puesto que los archivos de vídeo son enormes, por lo que la mayoría de los bloques sólo puede localizarse a través de un bloque indirecto triple, lo que significa muchos accesos a disco extra (Tanenbaum, 1992). En vez de ello, es común vincular los bloques en una lista de vínculo sencillo o doble. A veces se usa un índice (ínodo) tipo UNIX para permitir acceso aleatorio.

La otra tarea del *software* del disco es dar servicio a todas las corrientes de salida en tiempo real y cumplir sus restricciones de temporización. Una corriente de vídeo MPEG-2 a 25 marcos/seg requiere obtener y transmitir unos 14 kB cada 40 mseg, pero la cantidad real varía considerablemente dado que los marcos I, P y B tienen diferentes relaciones de compresión. En consecuencia, para mantener una tasa de salida uniforme, se requieren *buffers* en ambas terminales de la corriente.

En la figura 7-91 vemos una escalera que muestra la cantidad total de datos traídos del disco para una corriente de vídeo dada (suponiendo que la película está en disco). Dicha cantidad aumenta en saltos discretos, un salto por cada bloque leído. Sin embargo, la transmisión debe ocurrir a una tasa más uniforme, de modo que el proceso de lectura debe mantenerse adelantado respecto al proceso de transmisión. El área sombrada de la figura indica datos que se han obtenido del disco pero aún no se transmiten.

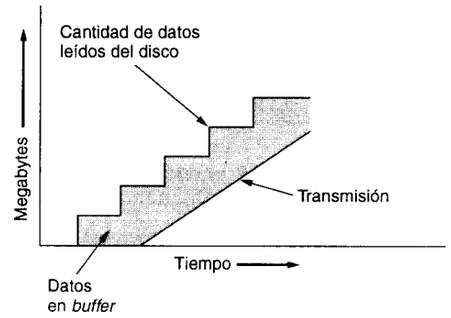


Figura 7-91. Proceso de *buffer* del disco en el servidor.

Los discos normalmente se programan usando el algoritmo del elevador, que comienza el movimiento del brazo hacia adentro y sigue avanzándolo hasta llegar al cilindro más interno, procesando todas las solicitudes que puede cumplir por orden de cilindro. Al llegar tan lejos como puede, el brazo invierte su sentido y comienza a moverse hacia afuera, procesando de nuevo en orden todas las solicitudes pendientes que encuentra en el camino. Aunque este algoritmo reduce el tiempo de búsqueda, no ofrece garantías en cuanto al desempeño en tiempo real, por lo que no es útil para un servidor de vídeo.

Un mejor algoritmo es llevar el registro de todas las corrientes de vídeo y hacer una lista del siguiente bloque que cada una necesita. Entonces, estos números de bloque se ordenan y se leen los bloques en orden por cilindro. Al leerse el último bloque, la siguiente ronda comienza con la obtención del número del bloque que ahora está a la cabeza de cada corriente. Estos números se ordenan y se leen en orden por cilindro, continuando así el proceso. Este algoritmo mantiene un desempeño de tiempo real para todos los cilindros, pero también reduce al mínimo el tiempo de búsqueda en comparación con un algoritmo puro de servicio según orden de aparición.

Otro problema de *software* es el control de admisión. Si entra una solicitud para una corriente nueva, ¿puede aceptarse sin arruinar el desempeño de tiempo real de las corrientes existentes? Un

algoritmo que puede servir para tomar una decisión examina el peor caso para ver si se garantiza que sea posible ir de k a $k + 1$ corrientes, con base en las propiedades conocidas de la CPU, la RAM y el disco. Otro algoritmo simplemente examina todas las propiedades estadísticas.

Otro asunto importante relacionado con el *software* del servidor es la manera de manejar la presentación en pantalla durante un avance o retroceso rápido (para que la gente pueda hacer búsquedas visuales). Los marcos D proporcionan la información necesaria para el MPEG-1 pero, a menos que se marquen y se almacenen de alguna manera especial, el servidor no podrá encontrarlos sin decodificar la corriente completa y, normalmente, los servidores no llevan a cabo decodificación MPEG durante la transmisión. En el MPEG-2 se requerirá algún otro mecanismo, cuando menos para facilitar el encuentro y decodificación de los marcos I.

Por último, el cifrado también es un asunto importante. Al multitransmitir películas (por ejemplo, si la red de distribución local es un sistema de TV por cable), se requiere cifrado para asegurar que sólo los clientes puedan ver las películas. Son posibles dos enfoques: el precifrado y el cifrado en el momento. Si las películas se almacenan cifradas, entonces cualquiera que logre averiguar la clave de una película podrá verla gratuitamente, puesto que siempre se usa la misma clave. El cifrado independiente de cada corriente es más seguro, pero también más costoso en términos de recursos de cómputo.

También es importante la administración de claves. El enfoque normal es cifrar al momento mediante un algoritmo sencillo, pero cambiar con frecuencia la clave, de modo que, si un intruso puede descifrar la clave en 10 minutos, para entonces será obsoleta.

La red de distribución

La red de distribución es el grupo de conmutadores y líneas entre el origen y el destino. Como vimos en la figura 7-88, esta red consiste en un *backbone* SONET o ATM (o ATM sobre SONET), conectado a la red local de distribución. Generalmente el *backbone* es conmutado y no la red de distribución local.

Los requisitos principales impuestos al *backbone* son alto ancho de banda y baja fluctuación. En un *backbone* SONET puro, es sencillo lograr esto: el ancho de banda está garantizado y la fluctuación es cero, puesto que la red es síncrona. En un *backbone* ATM, o ATM sobre SONET, la calidad del servicio es muy importante; se controla mediante el algoritmo de cubeta con fugas y todas las demás técnicas que estudiamos con detalle en el capítulo 5, por lo que no repetiremos ese análisis aquí. Para información adicional sobre el MPEG en tiempo real por *backbones* ATM, véase (Dixit y Skelly, 1995, y Morales *et al.*, 1995). A continuación nos enfocaremos en la red de distribución local, asunto que apenas hemos tocado hasta ahora.

La distribución local es caótica, pues las diferentes compañías usan redes distintas en diferentes regiones. Las compañías telefónicas, las de televisión por cable y los nuevos participantes están convencidos de que el que llegue primero será el gran ganador, por lo que ahora estamos viendo una proliferación en la instalación de nuevas tecnologías. Los cuatro esquemas principales de distribución local de vídeo a solicitud tienen las siglas ADSL, FTTC, FTTH y HFC. Ahora los explicaremos.

El **ADSL** (*Asymmetric Digital Subscriber Line*, **línea asimétrica digital de suscriptor**) fue el primer competidor de la industria telefónica por el premio de la distribución local (Chen y Waring, 1994). La idea es que a prácticamente cada casa de Estados Unidos, Europa y Japón ya entra un par trenzado de cobre (para el servicio telefónico analógico). Si estos alambres pudieran usarse para el vídeo a solicitud, las compañías telefónicas podrían ganar.

El problema, claro está, es que estos alambres no pueden manejar ni siquiera MPEG-1 a través de su recorrido normal de 10 km, ya no se diga MPEG-2. La solución ADSL aprovecha los avances en el procesamiento digital de señales para eliminar electrónicamente los ecos y otros ruidos de la línea. Como se muestra en la figura 7-92, cada suscriptor ADSL recibe una unidad ADSL casera que contiene un *chip* para el procesamiento de señales digitales. El teléfono y la caja de control se conectan a la unidad ADSL. En el otro extremo del lazo local, se conecta otra unidad ADSL. Ésta puede estar en la oficina terminal de la compañía telefónica o, si el lazo local es demasiado grande, en el extremo de una fibra óptica cercana a la casa.

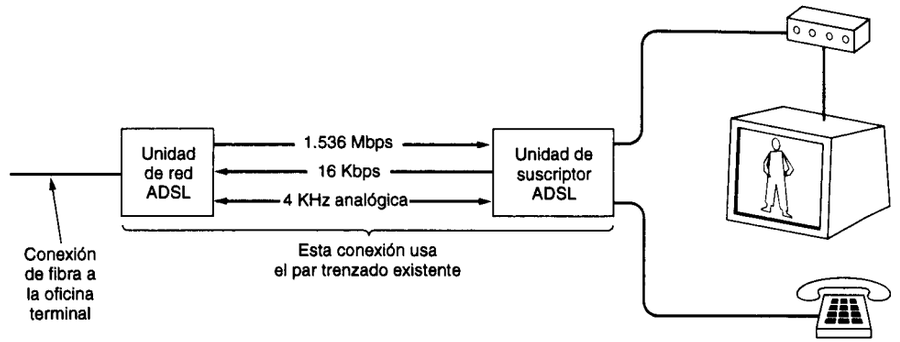


Figura 7-92. ADSL como red de distribución local.

El ADSL-1 ofrece un canal de enlace descendente de 1.536 Mbps (T1 menos el bit 193), pero sólo un canal de enlace ascendente de 16 kbps. Además está ahí el viejo canal analógico telefónico de 4 kHz (o, en algunos casos, dos canales digitales N-ISDN). La idea es que el enlace ascendente tenga suficiente ancho de banda para que el usuario solicite películas, y el enlace descendente tenga bastante ancho de banda para enviarlas codificadas en MPEG-1. El ADSL debe verse más como una solución rápida y sin complicaciones que una solución a largo plazo, pero ya se está instalando en varias ciudades. También se está trabajando en versiones mejoradas, llamadas ADSL-2 y ADSL-3. Esta última permite transportar MPEG-2 por lazos locales de hasta 2 km.

El segundo diseño de las compañías telefónicas es el **FTTC** (*Fiber To The Curb*, **fibra hasta la acera**). Vimos este diseño en la figura 2-23(a). En FTTC, la compañía telefónica tiende fibra óptica de la oficina terminal a cada barrio residencial, terminando en un dispositivo llamado

ONU (*Optical Network Unit*, **unidad de red óptica**). La ONU se llama "caja de unión" en la figura 2-23(a). En una ONU pueden terminar hasta 16 lazos locales de cobre. Estos lazos son lo bastante cortos para operar T1 o T2 dúplex integral en ellos, permitiendo películas MPEG-1 y MPEG-2, respectivamente. Además son posibles videoconferencias para trabajadores remotos y negocios pequeños, pues el FTTC es simétrico.

La tercera solución de las compañías telefónicas es tender fibra a todas las casas. Este sistema se llama **FTTH** (*Fiber To The Home*, **fibra hasta la casa**). En él, todos mundo puede tener una portadora OC-1, OC-3 o inclusive mayor, si se requiere. El FTTH es muy caro y tardará años en hacerse realidad, pero ciertamente abrirá una vasta gama de nuevas posibilidades cuando finalmente ocurra.

ADSL, FTTC y FTTH son redes de distribución local punto a punto, lo cual no es sorprendente dada la organización del sistema telefónico actual. Un enfoque completamente diferente es el **HFC** (*Hybrid Fiber Coax*, **híbrido fibra/coaxial**), la solución preferida que están instalando los proveedores de TV por cable y que se ilustra en la figura 2-23(b). La propuesta es la siguiente. Los cables coaxiales actuales de 300 a 450 MHz serán sustituidos por cables coaxiales de 750 MHz, elevando la capacidad de 50 a 75 canales de 6 MHz a 125 canales de 6 MHz. Setenta y cinco de los 125 canales se usarán para la transmisión de televisión analógica.

Los 50 canales nuevos se modularán individualmente usando QAM-526, que proporciona unos 40 Mbps por canal, dando un total de 2 Gbps de ancho de banda nuevo. Los *head-ends* se moverán más hacia el interior de los vecindarios, de modo que cada cable pase por sólo 500 casas. Una división sencilla indica que puede asignarse a cada casa un canal dedicado de 4 Mbps, capaz de transportar alguna combinación de programas MPEG-1, MPEG-2, datos ascendentes, telefonía analógica y digital, etcétera.

Aunque esto suena maravilloso, requiere que los proveedores de cable reemplacen todos los cables existentes con coaxial de 750 MHz, instalen nuevos *head-ends* y eliminen todos los amplificadores de una vía; en pocas palabras, que reemplacen el sistema completo de TV por cable. En consecuencia, la cantidad de infraestructura nueva es comparable con la que necesitan las compañías telefónicas para FTTC. En ambos casos, el proveedor local de red tiene que tender fibra a los barrios residenciales. También en ambos casos, la fibra termina en un convertidor optoeléctrico. En FTTC, el segmento final es un lazo local punto a punto que usa pares trenzados. En HFC, el segmento final es cable coaxial compartido. Técnicamente, estos sistemas no son tan diferentes, como suelen asegurar sus patrocinadores.

No obstante, hay una diferencia real que es importante indicar. HFC usa un medio compartido sin conmutación ni enrutamiento. Cualquier información que se ponga en el cable puede ser retirada por cualquier suscriptor sin mayores trámites. FTTC, que es completamente conmutado, no tiene esta propiedad. Como resultado, los operadores HFC quieren que los servidores transmitan cadenas cifradas, de modo que los clientes que no hayan pagado una película no puedan verla. A los operadores FTTC no les interesa el cifrado, puesto que agrega complejidad, disminuye el desempeño y no proporciona seguridad adicional a su sistema. Desde el punto de vista de la compañía que opera un servidor de vídeo, ¿es buena idea cifrar? Un servidor operado por una compañía telefónica o una de sus subsidiarias o socios podría intencionalmente decidir no cifrar sus vídeos, indicando como razón la eficiencia, pero en realidad para provocar pérdidas a sus competidores HFC.

Con todas estas redes locales de distribución es probable que se equipe a cada vecindario con uno o más servidores de *spool*. Éstos son, de hecho, simplemente versiones más pequeñas de los servidores de vídeo que estudiamos antes. La gran ventaja de estos servidores locales es que, dado que las redes de distribución local son pequeñas y generalmente no son conmutadas, no introducen fluctuaciones como lo haría una red de *backbone* ATM.

Dichos servidores locales pueden precargarse con películas de manera dinámica o por reservación. Por ejemplo, al seleccionar un usuario una película, podría transmitirse el primer minuto al servidor local en menos de 2 segundos con OC-3. Tras 55 segundos, el siguiente minuto podría enviarse al servidor local en 2 segundos, etc. De esta manera, el tráfico a través del *backbone* ATM ya no tiene que estar libre de fluctuaciones, haciendo posible el uso del servicio ABR en lugar de CBR, más costoso.

Si la gente indica con suficiente adelanto al proveedor las películas que desea ver, pueden bajarse al servidor local durante horas no pico, obteniéndose aún más ahorros. Esta observación probablemente llevará a los operadores de red a contratar ejecutivos de aviación para diseñar las tarifas. Pueden preverse tarifas en las que las películas ordenadas con 24 a 72 horas de adelanto para ser vistas la tarde de un martes o jueves antes de las 6 p.m. o después de las 11 p.m. tendrán 27% de descuento. Las películas ordenadas el primer domingo del mes antes de las 8 a.m. para ser vistas un miércoles por la tarde de un día cuya fecha sea un número primo tendrán 43% de descuento, etcétera.

La selección del grupo de protocolos para vídeo a solicitud aún está en el aire. ATM claramente es la tecnología a escoger, pero ¿qué protocolo de adaptación debe usarse? El AAL 1 se diseñó para vídeo, por lo que es un candidato fuerte, pero corresponde a la categoría de servicio CBR. Dedicar el máximo ancho de banda que podría necesitarse es caro, especialmente dado que el MPEG es inherentemente tráfico VBR, por lo que el circuito virtual tendrá que sobredimensionarse.

El AAL 2 no está acabado (y probablemente nunca lo estará), y el AAL 3/4 es demasiado torpe, así que el AAL 5 es el único contendiente que queda. AAL 5 no está atado a un servicio CBR, y el envío de un bloque grande de MPEG en cada mensaje sería extremadamente eficiente, pudiéndose dedicar casi 100% del ancho de banda de usuario para la corriente de vídeo. Por el lado negativo, el AAL 5 hace detección de errores. El que se descarte un bloque completo debido a un error de un bit es muy poco atractivo, especialmente dado que la mayoría de los errores son de un solo bit a la mitad de los datos. Como consecuencia, hay quienes proponen cambiar el AAL 5 de modo que permita que las aplicaciones soliciten todos los bloques, junto con un bit que indique si la cifra de comprobación fue correcta o no.

La pila de protocolos de vídeo a solicitud que acabamos de esbozar se ilustra en la figura 7-93. Por encima de la capa AAL, vemos el programa MPEG y la capa de transporte de corrientes. Luego vienen la codificación y decodificación de audio y vídeo MPEG, respectivamente. Por último, tenemos hasta arriba la aplicación.

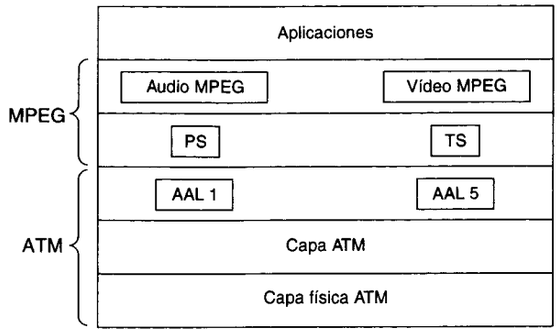


Figura 7-93. Una pila de protocolos de vídeo a solicitud.

Cajas de control

Todos los métodos de distribución local anteriores llevan en última instancia una o más corrientes MPEG a la casa. Para decodificarlas y verlas, se requiere una interfaz de red, un decodificador MPEG y otros componentes electrónicos. Son posibles dos enfoques.

En el primer enfoque, la gente usa sus computadoras personales para decodificar y ver películas. Para ello sería preciso comprar una tarjeta especial que contenga unos cuantos *chips* especiales y un conector para hacer interfaz con la red local de distribución. Las películas aparecen en el monitor de la computadora, posiblemente en una ventana. Este enfoque es más barato (todo lo que se requiere es una tarjeta y el *software*), usa un monitor no entrelazado de alta definición, tiene una interfaz de usuario manejada por ratón, y puede integrarse fácilmente con la WWW y otras fuentes de información y entretenimiento orientadas a computadora. Por otra parte, las PC generalmente tienen pantallas pequeñas, se ubican en estudios o en cuartos pequeños en lugar de salas y tradicionalmente las usa una sola persona a la vez. También emiten bastante menos luz que los aparatos de televisión.

En el segundo enfoque, el operador de red local renta o vende al usuario una **caja de control**, a la que se conectan la red y el aparato de televisión. Este enfoque tiene la ventaja de que todo mundo tiene una televisión y no todos una PC, y muchas de las PC son viejas, peculiares, o no son adecuadas para la decodificación MPEG. Además, la televisión normalmente está ubicada en un cuarto adecuado para un grupo de gente.

Por el lado negativo, el monitor tiene un cinescopio entrelazado de baja definición (lo que lo hace inadecuado para material orientado a texto, como la WWW); además, tiene una interfaz de usuario horrenda (el control remoto), lo que hace virtualmente imposible que el usuario haga otra cosa que no sea seleccionar opciones de menús sencillos. Incluso escribir el nombre de una película es un asunto laborioso, no se diga entablar un diálogo solicitando al servidor que busque todas las películas de cierto actor, director o compañía productora durante cierto periodo de

tiempo. Por último, no es fácil producir las cajas de control con el desempeño requerido a un precio aceptable (que se piensa será de unos cuantos cientos de dólares).

Considerando todos estos factores, la mayoría de los sistemas de vídeo a solicitud han optado por el modelo de caja de control, principalmente porque los estrategas de mercadeo de masas odian excluir a cualquier cliente potencial (gente sin PC). También, puede haber ganancias en la renta o venta de cajas de control. No obstante, el mercado para las tarjetas de PC es bastante grande, por lo que sin duda se producirán también estas tarjetas.

Las funciones primarias de la caja de control son la interfaz con la red de distribución local, la decodificación de la señal MPEG, la sincronización de las corrientes de audio y vídeo, la producción de una señal compuesta NTSC, PAL o SECAM para el aparato televisor, la recepción de señales del control remoto, y el manejo de la interfaz de usuario. Algunas funciones adicionales podrían incluir interfaces con equipos de sonido estereofónico, teléfonos y otros dispositivos. Hay actualmente una gran batalla en la industria sobre la funcionalidad que debe incluirse en la caja de control y la que debe estar en la red. El resultado está por verse.

Una arquitectura posible de una caja de control sencilla se muestra en la figura 7-94. El dispositivo consta en CPU, ROM, RAM, controlador de E/S, decodificador MPEG e interfaz de red. Opcionalmente puede agregarse un *chip* de seguridad para el descifrado de películas y el cifrado de menajes de salida (números de tarjeta de crédito para compras desde casa, etcétera).

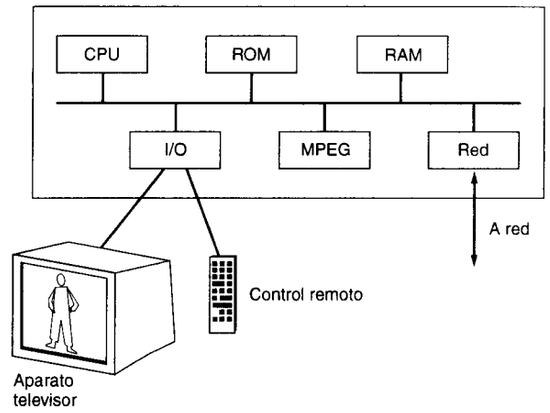


Figura 7-94. Arquitectura de *hardware* de una caja de control sencilla.

Un problema importante del vídeo a solicitud es la sincronización de audio/vídeo y el manejo de la fluctuación. La adición de unos 500 kB de RAM permite contar con un segundo de *buffer* de MPEG-2, pero con un costo adicional en un dispositivo que los fabricantes esperan vender en unos cuantos cientos de dólares, cuando mucho.

Puesto que la caja de control es sólo una computadora, necesitará *software*, probablemente un sistema operativo de tiempo real basado en micronúcleo, almacenado en ROM. Para proporcionar flexibilidad y adaptabilidad, probablemente sería conveniente permitir la carga de otro *software* desde la red. Esta posibilidad genera un problema: ¿qué ocurre cuando el dueño de una caja de control basada en MIPS quiere jugar un juego escrito para una caja de control basada en SPARC? El uso de un lenguaje interpretado como Java resuelve el problema de la compatibilidad, pero disminuye severamente el desempeño en un entorno de tiempo real en el que es crucial el alto desempeño.

Estándares

No puede ignorarse el aspecto económico del vídeo a solicitud. Un servidor grande de vídeo fácilmente puede costar más que un *mainframe*, ciertamente unos 10 millones de dólares. Supóngase que el servidor da servicio a 100,000 hogares, cada uno de los cuales ha rentado una caja de control de 300 dólares. Ahora agréguense 10 millones de dólares en equipo de red con un periodo de depreciación de cuatro años, y el sistema tiene que generar 10 dólares por casa por mes. A cinco dólares/película, todos tendrían que comprar dos películas al mes, solamente para que el operador recupere la inversión (sin tomar en cuenta salarios, mercadeo y todos los demás costos). El que realmente vaya a ocurrir esto, no se puede saber.

Las cifras indicadas antes pueden redistribuirse de varias maneras (por ejemplo, cargando 6 dólares por renta mensual de la caja de control y 2 dólares por película), y los costos cambian todo el tiempo, pero debe quedar claro que, sin un mercado masivo, no hay modo de que el vídeo a solicitud tenga sentido desde el punto de vista económico. Para crear un mercado masivo, es esencial que se estandaricen todas las partes del sistema. Si cada proveedor de vídeo, operador de red y fabricante de cajas de control diseña su propia interfaz, nada interactuará con el resto del sistema. Hasta ahora, el único estándar en el que todos están de acuerdo es con el uso de MPEG-2 para la codificación de vídeo. Todo lo demás está abierto. En la figura 7-95 se listan algunas de las muchas preguntas que tendrán que responderse antes de que pueda construirse un sistema nacional.

Si pueden estandarizarse todas estas áreas, es fácil imaginar la producción de equipos que consistan en una caja con un conector telefónico, un monitor, teclado y ratón que puedan usarse para ver vídeos, efectuar cómputo, o tal vez ambas cosas a la vez. Entonces será realidad la muy mencionada convergencia de las industrias de cómputo, comunicación y entretenimiento.

7.7.5. MBone — *Backbone* de multidifusión

Mientras todas estas industrias hacen grandes (y muy publicitados) planes para el vídeo digital (inter)nacional a solicitud, la comunidad de Internet ha estado implementando calladamente su propio sistema digital de multimedia, **MBone** (*Multicast* Backbone, ***backbone* de multidifusión**). En esta sección haremos un repaso breve a sus características y su funcionamiento. Para un libro completo sobre MBone, véase (Kumar, 1996). Para artículos sobre MBone, véase (Eriksson, 1994, y Macedonia y Brutzman, 1994).

¿Qué tecnología usará el *backbone* (SONET, ATM, SONET + ATM)?
¿A qué velocidad operará el *backbone* (OC-3, OC-12)?
¿Cómo se efectuará la distribución local (HFC, FTTC)?
¿Cuál será el ancho de banda ascendente (16 kbps, 1.5 Mbps)?
¿Se cifrarán las películas? De ser así, ¿cómo?
¿Habrá corrección de errores (obligatoria, opcional, ausente)?
¿Quién será el dueño de la caja de control (usuario, operador de red)?
¿Será la telefonía parte del sistema (analógica, N-ISDN)?
¿Se manejarán aplicaciones de hipertexto de alta resolución (por ejemplo, WWW)?

Figura 7-95. Algunas de las áreas en las que se necesitan estándares.

Puede pensarse en el MBone como una radio y televisión de Internet. A diferencia del vídeo a solicitud, donde o más importante es solicitar y ver películas precomprimidas almacenadas en un servidor, MBone se usa para difundir audio y vídeo en vivo de forma digital por todo el mundo a través de Internet. MBone ha estado en operación desde comienzos de 1992. Se han difundido muchas conferencias científicas, especialmente las reuniones del IETF, así como eventos científicos de importancia noticiosa, como varios lanzamientos del transbordador espacial. Alguna vez se difundió un concierto de los Rolling Stones por MBone. El que esto tenga un interés científico está abierto a debate. También hay *software* disponible para aquellos que deseen grabar digitalmente una difusión MBone (Holfelder, 1995).

La mayor parte de las investigaciones acerca del MBone se han orientado a la manera de multitransmitir eficientemente a través de Internet (orientada a datagramas). Se ha hecho poco acerca de la codificación de audio o vídeo. Las fuentes MBone están en libertad de experimentar con MPEG o cualquier otra tecnología que deseen. No hay estándares de Internet respecto al contenido o la codificación.

Desde el aspecto técnico, MBone es una red virtual sobrepuesta a Internet. Consiste en islas capaces de multitransmisión conectadas mediante túneles, como se muestra en la figura 7-96. En esta figura, el MBone consiste en seis islas, *A* a *F*, conectadas mediante siete túneles. Cada isla (por lo común, una LAN o grupo de LAN interconectadas) apoya la multitransmisión por *hardware* a sus *hosts*. Los túneles propagan paquetes MBone entre las islas. En el futuro, cuando todos los enrutadores sean capaces de manejar directamente tráfico de multitransmisión, dejará de necesitarse esta superestructura, pero por el momento lleva a cabo el trabajo.

Cada isla contiene uno o más enrutadores especiales, llamados **enrutadores m** o *mrouters* (**enrutadores de multitransmisión**). Algunos de éstos son en realidad enrutadores normales, pero otros sencillamente son estaciones de trabajo UNIX que ejecutan *software* especial de nivel de usuario (pero como la raíz). Los enrutadores m se conectan lógicamente mediante túneles. En el pasado, los paquetes MBone se enviaban por un túnel de enrutador m a enrutador m (general-

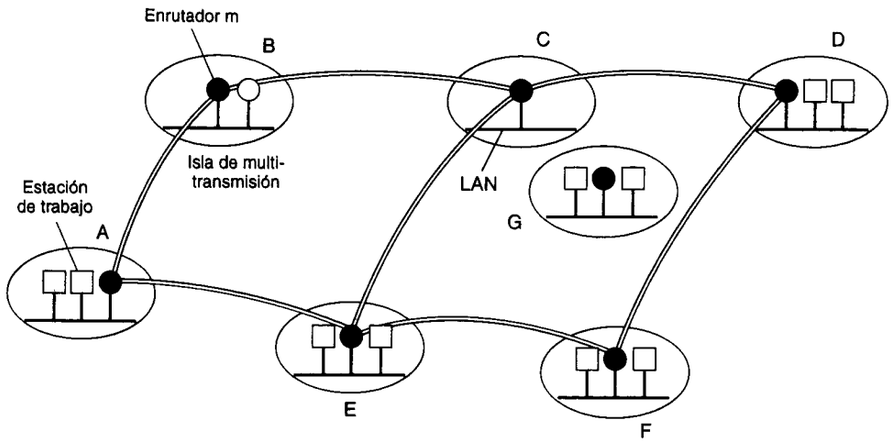


Figura 7-96. El MBone consiste en islas de multitransmisión conectadas mediante túneles.

mente a través de uno o más enrutadores que no entendían de MBone), usando enrutamiento no estricto desde el origen. Hoy día, los paquetes MBone se encapsulan en paquetes IP y se envían como paquetes de unitransmisión normales a la dirección IP del enrutador m del destino. Sin embargo, si todos los enrutadores que intervienen apoyan la multitransmisión, no se necesita el proceso de túnel.

Los túneles se configuran de manera manual. Por lo general, un túnel corre por una trayectoria para la que existe una conexión física, pero esto no es un requisito. Si por accidente se cae la trayectoria física subyacente de un túnel, los enrutadores m que usan el túnel ni siquiera lo notarán, puesto que la Internet reenrutará automáticamente todo el tráfico IP entre ellos por otras líneas.

Al aparecer una isla nueva que desea unirse al MBone, como en el caso de *G* de la figura 7-96, su administrador envía un mensaje anunciando su existencia a la lista de correo de MBone. Los administradores de las instalaciones cercanas entonces se ponen en contacto con ella para acordar el establecimiento de túneles. A veces se reordenan los túneles existentes para aprovechar la nueva isla y optimizar la topología. A fin de cuentas, los túneles no existen físicamente, se definen mediante tablas en los enrutadores m y pueden agregarse, eliminarse o moverse con sólo cambiar estas tablas. Por lo común, cada país en el MBone tiene un *backbone* cor islas regionales conectadas a él. Normalmente se configura el MBone con uno o dos túneles que cruzan el Océano Atlántico y el Pacífico, haciendo al MBone de escala global.

Por tanto, en cualquier momento, el MBone consiste en una topología específica de túneles e islas, independiente de la cantidad de direcciones de multitransmisión en uso en el momento y de quiénes las están escuchando u observando. Esta situación es muy parecida a una subred normal (física), por lo que se aplican los mismos algoritmos de enrutamiento. En consecuencia, el MBone inicialmente usaba un algoritmo de enrutamiento, el **DVMRP** (*Distance Vector Multicast Routing Protocol*, **protocolo de enrutamiento multitransmisión por vector de distancia**),

basado en el algoritmo de vector de distancia Bellman-Ford. Por ejemplo, en la figura 7-96, la isla *C* puede enrutar a *A* vía *B* o vía *E* (o tal vez vía *D*). Esto lo decide *C* tomando los valores que le dan los nodos sobre sus respectivas distancias a *A*, y sumando después su distancia a ellos. De esta manera, cada isla determina la mejor ruta a todas las demás islas. Sin embargo, las rutas en realidad no se usan de esta manera, como veremos en breve.

Consideremos ahora la manera en que ocurre la multitransmisión. Para multitransmitir un programa de audio o vídeo, una fuente primero debe adquirir una dirección de multitransmisión clase D, que actúa como la frecuencia o el número de canal de una estación. Las direcciones clase D se reservan usando un programa que busca direcciones de multitransmisión libres en una base de datos. Pueden ocurrir simultáneamente muchas multitransmisiones, y un *host* puede "sintonizarse" con aquella en la que está interesado si escucha en la dirección de multitransmisión adecuada.

Periódicamente, cada enrutador m envía un paquete de difusión IGMP limitado a su isla, preguntando quién está interesado en qué canal. Los *hosts* que desean (continuar) recibiendo uno o más canales envían como respuesta otro paquete IGMP. Estas respuestas se reparten en el tiempo, para evitar sobrecargar la LAN local. Cada enrutador m mantiene una tabla de los canales que debe poner en sus LAN, para evitar el desperdicio de ancho de banda al multitransmitir canales que nadie quiere.

Las multitransmisiones se propagan por el MBone según sigue. Cuando una fuente de audio o vídeo genera un paquete nuevo, lo multitransmite a su isla local usando el recurso de multitransmisión del *hardware*. Este paquete lo recoge el enrutador m local, que entonces lo copia en todos los túneles a los que está conectado.

Cada enrutador m que recibe uno de tales paquetes por medio de un túnel lo revisa para ver si llegó por la mejor ruta, es decir, la ruta que, según su tabla, debe usarse para llegar al origen (como si fuera el destino). Si el paquete llegó por la mejor ruta, el enrutador m copia el paquete en todos sus otros túneles. Si el paquete llegó por una ruta subóptima, se descarta. Por ejemplo, en la figura 7-96, si las tablas de *C* le indican que use *B* para llegar a *A*, entonces cuando un paquete multitransmisión de *A* a *C* llega vía *B*, se copia de ahí a *D* y a *E*. Sin embargo, cuando un paquete de multitransmisión de *A* a *C* llega vía *E* (que no es la mejor trayectoria), simplemente se descarta. Este algoritmo sólo es el de reenvío por trayectoria invertida que vimos en el capítulo 5; aunque no es perfecto, es bastante bueno y muy sencillo de implementar.

Además de usar reenvío por trayectoria invertida para evitar inundar la Internet, también se usa el campo *tiempo de vida* del IP para limitar el alcance de la multitransmisión. Cada paquete comienza con algún valor (determinado por el origen). A cada túnel se le asigna un peso. Un paquete sólo pasa a través de un túnel si tiene suficiente peso; de otra manera, se descarta. Por ejemplo, los túneles transoceánicos se configuran normalmente con un peso de 128, por lo que los paquetes pueden limitarse al continente de origen dándoles un *tiempo de vida* inicial de 127 o menos. Tras pasar por el túnel, se resta el peso del túnel del campo de *tiempo de vida*.

Aunque el algoritmo de enrutamiento de MBone funciona, se han realizado muchas investigaciones para mejorarlo. Una propuesta mantiene la idea del enrutamiento por vector de distancia, pero hace jerárquico el algoritmo, agrupando en regiones las instalaciones MBone y enrutando primero hacia ellas (Thyagarajan y Deering, 1995).

Otra propuesta es usar una forma modificada del enrutamiento por estado de enlace en lugar de enrutamiento por vector de distancia. En particular, un grupo de trabajo del IETF está modificado el OSPF para adecuarlo a la multitransmisión dentro de un solo sistema autónomo. El OSPF de multitransmisión resultante se llama **MOSPF** (Moy, 1994). Lo que hacen las modificaciones es que el mapa completo construido por el MOSPF lleva el registro de las islas y túneles de multitransmisión, además de la información normal de enrutamiento. Si se conoce la topología completa, es directo el cálculo de la mejor trayectoria desde cualquier isla a cualquier otra usando los túneles. Por ejemplo, puede usarse el algoritmo de Dijkstra.

Una segunda área de investigación es el enrutamiento entre los AS. Aquí, otro grupo de trabajo del IETF está desarrollando un algoritmo llamado **PIM** (*Protocol Independent Multicast*, **multitransmisión independiente del protocolo**) (Huitema, 1995). El PIM viene en dos versiones, dependiendo si las islas son densas (casi todos quieren ver) o dispersas (casi nadie quiere ver). Ambas versiones usan las tablas de enrutamiento de unitransmisión estándar, en lugar de crear una topología superpuesta como lo hacen DVMRP y MOSPF.

En el PIM denso, la idea es recortar las trayectorias inútiles. El recortado funciona como sigue. Cuando llega un paquete multitransmisión por el túnel "equivocado", se devuelve un paquete de recorte por el túnel, indicando al transmisor que cese el envío de paquetes desde la fuente en cuestión. Al llegar un paquete por el túnel "correcto", se copia en todos los demás túneles que no han sido previamente recortados. Si todos los demás túneles se han recortado y no hay interés por el canal en la isla local, el enrutador m devuelve un mensaje de recorte por el canal "correcto". De esta manera, la multitransmisión se adapta automáticamente y sólo va a donde se le quiere.

El PIM disperso funciona de manera diferente. Aquí la idea es evitar la saturación de Internet porque tres personas de Berkeley quieren realizar una llamada en conferencia por una dirección clase D. El PIM disperso opera estableciendo puntos de reunión. Cada una de las fuentes de un grupo de multitransmisión PIM disperso envía sus paquetes a los puntos de reunión. Cualquier instalación interesada en unirse solicita a uno de los puntos de reunión que establezca un túnel a ella. De esta manera, todo el tráfico PIM disperso se transporta por unitransmisión en lugar de multitransmisión.

En resumen, los multimedia son un campo emocionante y de rápido movimiento. Diariamente se anuncian tecnologías y aplicaciones nuevas, pero el área en conjunto sin duda será importante durante las siguientes décadas.

7.8. RESUMEN

Las redes de cómputo son inherentemente inseguras. Para mantener la información en secreto, ésta debe cifrarse. Los protocolos de cifrado pertenecen a dos clases generales: de clave secreta (por ejemplo, DES, IDEA) y de clave pública (por ejemplo, RSA). El uso de estos protocolos es directo; la parte difícil es la administración de claves.

Además de proporcionar secreto, los protocolos criptográficos también pueden proporcionar validación de identificación, de modo que cuando Alicia piensa que se está comunicando con Benjamín, en realidad se está comunicando con Benjamín, y no con Trudy. Por último, la

criptografía también puede servir para permitir la firma de mensajes de manera que el transmisor no pueda repudiarlos una vez enviados.

Los nombres en Internet usan un sistema distribuido de base de datos, el DNS. El DNS contiene registros con las direcciones IP, centrales de correo y otra información. Mediante consultas al servidor DNS, un proceso puede relacionar un nombre de dominio de Internet con la dirección IP usada para comunicarse con ese dominio.

A medida que crecen las redes, se vuelven más difíciles de administrar. Por esta razón, se han desarrollado sistemas y protocolos especiales de administración de redes, de los cuales el más común es el SNMP. Este protocolo permite a los administradores comunicarse con agentes que están dentro de los dispositivos para leer su estado y emitirles comandos.

Cuatro de las aplicaciones principales de las redes son el correo electrónico, las noticias USENET, la *World Wide Web* y los multimedia (vídeo a solicitud y MBone). La mayoría de los sistemas de correo electrónico usan el sistema de correo definido en los RFC 821 y 822. Los mensajes enviados por este sistema usan cabeceras de sistema en ASCII para definir las propiedades del mensaje. Estos mensajes se envían usando SMTP. Existen dos sistemas para hacer seguro el correo electrónico, el PGP y el PEM.

Las noticias USENET consisten en miles de grupos de noticias sobre todos los temas. La gente puede suscribirse a los grupos de noticias de manera local, y entonces puede publicar mensajes en todo el mundo usando el protocolo NNTP, que tiene algún parecido con el SMTP.

La *World Wide Web* (red mundial) es un sistema para vincular documentos de hipertexto. Cada documento es una página escrita en HTML, posiblemente con hipervínculos con otros documentos. Un visualizador puede presentar en pantalla un documento estableciendo una conexión TCP con su servidor, solicitando el documento y cerrando después la conexión. Al seleccionar el usuario un hipervínculo, puede traerse por la misma vía ese documento. De este modo, los documentos de todo el mundo se vinculan para formar una red (*web*) inmensa.

Los multimedia son la nueva estrella en el firmamento de las redes; permiten la digitalización de audio y vídeo, y su transporte electrónico para exhibirse. La mayoría de los proyectos multimedia usa los estándares MPEG y transmite los datos por conexiones ATM. El MBone es un servicio experimental de radio y televisión digital de alcance mundial que opera en Internet.

PROBLEMAS

1. Descifre el siguiente cifrado monoalfabético. El texto normal, que consiste sólo en letras, es un fragmento bien conocido de un poema de Lewis Carroll, que está en inglés.

kfd ktbd fzm eubd kfd pzyiom mztx ku kzyg ur bzha kfthcm
ur mfudm zhx mftnm zhx mdzythc pzq ur ezsszcdm zhx gthcm
zhx pfa kfd mdz tm sutythc fuk zhx pfdkfdi ntcm fzld pthcm
sok pztk z stk kfd uamkdim eitdx sdruid pd fzld uoi efzk
rui mubd ur om zid uok ur sidzkf zhx zyy ur om zid rzk
hu foiia mztx kfd ezindhkdi kfda kfzhgdx ftb boef rui kfzk

2. Descifre el siguiente cifrado por transposición columnar. El texto normal (en inglés) se tomó de un libro de texto sobre computación bastante conocido, por lo que "computer" es una palabra probable. El texto normal consiste por entero en letras (sin espacios). El texto cifrado se dividió en bloques de cinco caracteres para hacerlo más legible.

aauan cvlre rurnn dltme aeepb ytust iceat npmey iicgo gorch srsoc
nntii imiha oofpa gsivt tpsit lbolr otoex

3. En la figura 7-4, las cajas P y las cajas S se alternan. Aunque este arreglo es atractivo estéticamente, ¿es más seguro que tener primero todas las cajas P y luego todas las cajas S?

4. Suponga que se ha cifrado un mensaje usando DES en modo de encadenamiento de bloque cifrado. Un bit de texto cifrado del bloque C_i cambia accidentalmente de 0 a 1 durante la transmisión. ¿Cuánto texto normal se alterará como resultado?

5. Considere nuevamente el encadenamiento de bloque cifrado. En lugar de transformarse un solo bit 0 en bit 1, se inserta un bit 0 extra en la corriente de texto cifrado después del bloque C_i. ¿Cuánto texto normal se alterará como resultado?

6. Diseñe un ataque al DES con base en el conocimiento de que el texto cifrado consiste exclusivamente en letras ASCII mayúsculas más espacios, comas, puntos, punto y coma, retornos de carro y saltos de línea. No se sabe nada sobre los bits de paridad del texto normal.

7. Compare el encadenamiento de bloque cifrado con el modo de realimentación de cifrado en términos de la cantidad de operaciones de cifrado necesarias para transmitir un archivo grande. ¿Cuál es más eficiente, y por cuánto?

8. Usando el criptosistema RSA de clave pública, con $a = 1$, $b = 2$, etc.,
(a) Si $p = 7$ y $q = 11$, liste cinco valores legales de d.
(b) Si $p = 13$, $q = 31$ y $d = 7$, encuentre e.
(c) Usando $p = 5$, $q = 11$ y $d = 27$, encuentre e y cifre "abcdefghij".

9. Se está usando el intercambio de claves Diffie-Hellman para establecer una clave secreta entre Alicia y Benjamín. Alicia envía a Benjamín (719, 3, 191). Benjamín responde con (543). El número secreto de Alicia, x, es 16. ¿Cuál es la clave secreta?

10. Modifique ligeramente un mensaje del protocolo de la figura 7-14 para hacerlo resistente al ataque por reflexión. Explique por qué funciona su cambio.

11. En el protocolo de rana de boca amplia, ¿por qué se envía A en texto normal junto con la clave de sesión cifrada?

12. En el protocolo de rana de boca amplia indicamos que es un riesgo de seguridad iniciar cada mensaje de texto normal con 32 bits cero. Supóngase que cada mensaje comienza con un número aleatorio por usuario, de hecho una segunda clave secreta conocida sólo por su usuario y el KDC. ¿Impide esto el ataque por texto normal conocido?

13. En el protocolo Needham-Schroeder, Alicia genera dos retos, R_A y R_{A2}. Esto parece demasiado. ¿Habría bastado con uno solo?

14. En el protocolo de validación de identificación de clave pública de la figura 7-21, en el mensaje 3, R_B se cifra con K_S. ¿Es necesario este cifrado, o habría sido adecuado devolver R_B en texto normal?

15. El protocolo de firma de la figura 7-22 tiene la siguiente debilidad. Si Benjamín se cae, puede perder el contenido de su RAM. ¿Qué problemas causa esto y qué puede hacerse para evitarlos?

16. Tras la confesión de Elena a Marilyn respecto al engaño en el asunto de la cátedra de Tomás, Marilyn decidió evitar este problema dictando sus mensajes futuros a una grabadora y haciendo que su nueva secretaria sólo los mecanografíe. Marilyn planeaba examinar los mensajes en su terminal, una vez mecanografiados, para asegurarse de que contuvieran sus palabras exactas. ¿Puede la nueva secretaria usar todavía el ataque de cumpleaños para falsificar un mensaje, y de ser así, cómo? *Sugerencia*: Sí puede.

17. Las terminales de punto de venta que usan tarjetas con tira magnética y códigos PIN tienen una falla mortal: un comerciante sin escrúpulos puede modificar su lector de tarjetas para capturar y almacenar toda la información sobre la tarjeta al igual que el código PIN, a fin de efectuar transacciones adicionales (falsas) en un futuro. La siguiente generación de terminales de punto de venta usará tarjetas con una CPU completa, teclado y pantalla miniatura en la tarjeta. Diseñe un protocolo para este sistema que no puedan violar los comerciantes sin escrúpulos.

18. De acuerdo con la información dada en la figura 7-27, ¿está *little-sister.cs.vu.nl* en una red clase A, B o C?

19. En la figura 7-27 no hay un punto después de *rowboat*. ¿Por qué no?

20. ¿Cuál es el *OBJECT IDENTIFIER* del objeto tcp?

21. Es necesario transmitir un entero SNMP cuyo valor es 200. Muestre la representación binaria de los bits enviados en la sintaxis de transferencia ASN.1.

22. ¿Cuál es la representación de la cadena de 11 bits binarios '11100001111' en la sintaxis de transferencia ASN.1?

23. Suponga que ha sido contratado por un proveedor de puentes para escribir código que se ajuste al SNMP para uno de sus puentes. Usted lee todos los RFC y aún tiene preguntas, por lo que sugiere al IAB que se especifique en un solo lugar una gramática formal y completa del lenguaje empleado para describir las variables SNMP. La reacción del IAB es estar de acuerdo y encargarle a usted la tarea. ¿Debe agregarse la gramática al RFC 1442 o al RFC 1213? ¿Por qué? *Sugerencia*: No necesita buscar en los RFC; se da bastante información en el texto.

24. Algunos sistemas de correo electrónico reconocen un campo de cabecera *Content Return:*, que especifica si el cuerpo de un mensaje debe devolverse en el caso de no poderse entregar. ¿Pertenece este campo al sobre o a la cabecera?

25. Los sistemas de correo electrónico necesitan directorios para poder buscar las direcciones de correo electrónico de la gente. Para construir tales directorios, deben dividirse los nombres en componentes estándar (por ejemplo, nombre y apellido), a fin de hacer posible la búsqueda. Explique algunos de los problemas que deben resolverse para que sea aceptable un estándar mundial.

26. Un archivo binario tiene 3072 bytes de longitud. ¿Qué tan grande será si se codifica usando codificación base64, con la inserción de un par CR+LF tras cada 80 bytes enviados y al final?

27. Considere el esquema de codificación MIME entrecomillado-imprimible. Mencione un problema que no se haya analizado en el texto y proponga una solución.

28. Dé *dos* razones por las que el PGP comprime los mensajes.

29. Suponga que alguien establece un *daemon* de vacaciones y luego envía un mensaje justo antes de terminar la sesión. Desafortunadamente, el receptor ha estado de vacaciones durante una semana y también tiene activo un *daemon* de vacaciones. ¿Qué ocurre después? ¿Continuarán yendo y viniendo los mensajes enlatados hasta que alguien regrese?

30. Suponiendo que todos en la Internet usaran PGP, ¿podría enviarse un mensaje PGP a una dirección Internet arbitraria y ser decodificado correctamente por todos los interesados? Explique su respuesta.

31. PGP no reconoce la canonización como lo hace PEM. ¿Por qué no?

32. Adivine lo que podría significar la carita :–X (a veces escrita como :–#).

33. ¿Cuánto tiempo se requiere para distribuir el equivalente a un día de noticias a través de un canal satelital de 50 Mbps?

34. ¿Cuáles de los comandos listados en la figura 7-56 son redundantes en teoría?

35. Una red grande consiste en una matriz de máquinas $n \times n$. Todos los nodos internos tienen cuatro vecinos; los de las orillas (esquinas) tienen tres (dos) vecinos. Si se publica un artículo de m bytes en alguna máquina mediante NNTP, ¿cuántos bytes de ancho de banda se consumen para hacerlo llegar a todas las demás máquinas (ignorando la carga extra NNTP y contando sólo los bytes de mensaje)?

36. Repita el problema anterior, pero calcule ahora el ancho de banda aproximado que se necesitará para distribuir el mensaje usando una lista de correo. ¿Qué tanto mayor es respecto del problema anterior?

37. Al enviarse páginas de *Web*, tienen como prefijo cabeceras MIME. ¿Por qué?

38. ¿Cuándo se requieren visores externos? ¿Cómo sabe el visualizador cuál debe usar?

39. Imagine que alguien del departamento de informática de Stanford acaba de escribir un programa nuevo que quiere distribuir mediante FTP. Pone el programa en el directorio FTP *ftp/pub/freebies/newprog.c*. ¿Cómo será el URL de este programa?

40. En la figura 7-60, se asigna 1 al parámetro *ALT* de la etiqueta <IMG>. ¿En qué condiciones lo usa el visualizador, y cómo?

41. ¿Cómo se confiere en HTML la posibilidad de hacer clic en una imagen? Dé un ejemplo.

42. Muestre la etiqueta <A> que se necesita para hacer que la cadena "ACM" sea un hipervínculo a *http://www.acm.org*.

43. Diseñe una forma para una compañía nueva, Interburger, que permita ordenar hamburguesas vía Internet. La forma debe incluir el nombre del cliente, su dirección y ciudad, así como una selección de tamaños (gigante o inmensa) y una opción de queso. Las hamburguesas se pagarán al contado a la entrega, por lo que no se requiere información de tarjeta de crédito.

44. Java no tiene estructuras como las de C ni registros como los de Pascal. ¿Hay alguna otra manera de lograr el mismo efecto de agrupar un conjunto de variables diferentes para formar un solo tipo de datos? ¿De haberla, cuál es?

45. Usando las estructuras de datos de la figura 7-75, liste los pasos exactos necesarios para examinar un URL nuevo y comprobar si ya está en *url_table*.

46. Supóngase que, en su esfuerzo por orientarse más al mercado, la KGB se vuelve comercial y contrata a una agencia de publicidad que le diseña una página de *Web*. Su compañía ha sido contratada como consultor externo para implementarla. Escriba el HTML para producir la siguiente página de *Web*.

BIENVENIDOS A LA PÁGINA BASE DE WWW DE LA KGB

En virtud de su reciente privatización, la KGB se complace en anunciar la disponibilidad comercial de muchos artículos y servicios de calidad previamente disponibles sólo para los grandes gobiernos.

¡Precios competitivos! ¡Servicio discreto asegurado!

- Productos
  - Armamento nuclear (pequeño, mediano, grande, jumbo)
  - Satélites espía (vigile a sus vecinos)
  - Aeronaves supersónicas de bajo perfil de radar (sobrevuele las casas de sus amigos sin ser visto)
- Servicios
  - Colocación de espías en la organización que desee
  - Golpes de estado (corporativos y gubernamentales)
  - Asistencia para establecer su propio laboratorio de armas bacteriológicas
- Especiales con descuento
  - Obras seleccionadas de Felix Dzerzhinsky (edición limitada)
  - Fotografías aéreas de Afganistán (1984)
  - Tanques búlgaros de calidad (95% de descuento)

Webmaster@kgb.ru

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECO
MONTEVIDEO - URUGUAY

47. En C y C++, el tamaño de un entero no está especificado por el lenguaje. En Java, sí lo está. Cite un argumento a favor del enfoque de C y otro a favor del enfoque de Java.

48. Suponga que la *Web* contiene 10 millones de páginas, cada una con un promedio de 10 hipervínculos. Para traer una página se requieren 100 mseg en promedio. ¿Cuál es el tiempo máximo para indizar la *Web* completa?

49. Un disco compacto contiene 650 MB de datos. ¿Se usa compresión en los CD de audio? Explique su razonamiento.

50. ¿Cuál es la tasa de bits para transmitir color VGA sin compresión con 8 bits/píxel a 40 marcos/seg?

51. En la figura 7-76(c) ocurre ruido de cuantización debido al uso de muestras de 3 bits. La primera muestra, en 0, es exacta, pero las siguientes no. ¿Cuál es el porcentaje de error para las muestras en 1/32, 2/32 y 3/32 del periodo?

52. Un error de 1 bit en un marco MPEG, ¿puede tener efectos que alcancen más allá del marco en el que ocurrió el error? Explique su respuesta.

53. Considere el ejemplo del servidor de vídeo con 100,000 clientes dado en el texto. Suponga que la mitad de todas las películas se sirven de 8 p.m a 10 p.m. ¿Cuántas películas tiene que transmitir el

servidor a la vez durante este periodo de tiempo? Si cada película requiere 4 Mbps, ¿cuántas conexiones OC-12 a la red necesita el servidor?

54. Suponga que la ley de Zipf se cumple para accesos a un servidor de vídeo de 10,000 películas. Si el servidor contiene las 1000 películas más populares en disco magnético y las 9000 restantes en disco óptico, escriba una expresión para la fracción de las referencias que estarán en disco magnético. Escriba un programa pequeño para evaluar numéricamente esta expresión.

55. Los paquetes PES del MPEG contienen un campo que indica el estado de derechos de autor de la transmisión actual. ¿Para qué podría servir este campo?

8

LISTA DE LECTURAS Y BIBLIOGRAFÍA

Hemos terminado nuestro estudio de las redes de computación, pero éste es sólo el comienzo. No se han tratado muchos temas interesantes con el detalle que merecen, y se han omitido otros por falta de espacio. En este capítulo ofrecemos algunas sugerencias de lectura y una bibliografía, para el beneficio de los lectores interesados en continuar sus estudios de las redes de cómputo.

8.1. SUGERENCIAS PARA LECTURA ADICIONAL

Hay una extensa literatura sobre todos los aspectos de las redes de computación y los sistemas distribuidos. Cuatro publicaciones que con frecuencia publican artículos sobre esta área son *IEEE Transactions on Communications*, *IEEE Journal on Selected Areas in Communications*, *Computer Communication Review* y *Computer Networks and ISDN Systems*. Muchas otras publicaciones también contienen artículos ocasionales sobre el tema.

El IEEE también publica dos revistas, *IEEE Network Magazine* y *IEEE Communications Magazine*, que contienen investigaciones, tutoriales y estudios de casos sobre las redes. La primera está orientada hacia la arquitectura, los estándares y el *software*, y la última tiende hacia la tecnología de comunicaciones (fibra óptica, satélites, etcétera).

Además, hay varias conferencias anuales o bienales que con frecuencia atraen ponencias sobre redes y sistemas distribuidos, en particular, *SIGCOMM '9x*, *The International Conference on Distributed Computer Systems*, *The Symposium on Operating Systems Principles* y *The N-th*

Data Communications Symposium. Además, el IEEE ha publicado varios volúmenes de reimpresiones de investigaciones sobre redes en formato accesible en rústica.

A continuación se presenta una lista de sugerencias para lectura adicional, ordenada según los capítulos de este libro con el que se relacionan.

8.1.1. Introducción y obras generales

Jabbari *et al.*, " Network Issues for Wireless Communication"

Esta introducción a los sistemas de radio celular cubre control de llamadas, enrutamiento, señalización y otros aspectos de los sistemas modernos de comunicación móvil.

Comer, *The Internet Book*

Cualquiera que desee una introducción fácil a Internet debe buscarla aquí. Comer describe la historia, crecimiento, tecnología, protocolos y servicios de Internet en términos que pueden entender los principiantes, pero gran parte del material cubierto por este libro también es de interés para los lectores más técnicos.

Kwok, "A Vision for Residential Broadband Service"

Si desea conocer cómo piensa Microsoft que debe organizarse el vídeo a solicitud, este artículo es para usted. En él, el arquitecto principal de ATM de Microsoft explica el punto de vista de su compañía. En resumen, la idea de Microsoft es: el camino a seguir es ATM a los hogares. Olvidemos todas las soluciones "realistas" (es decir, *ad hoc*), como el ADSL, y hagámoslo bien.

Le Boudec, "The Asynchronous Transfer Mode: A tutorial"

El ATM es una tecnología nueva en crecimiento, y este documento es una introducción detallada a él. Se cubren la capa física, la capa ATM y la capa AAL. Además, la sección final analiza el debate sobre el ATM.

Pahlavan *et al.*, "Trends in Local Wireless Networks"

Las LAN inalámbricas sin duda serán cada vez más importantes en el futuro. En este artículo, los autores estudian lo más avanzado y las tendencias en el uso del espectro y las tecnologías para las LAN inalámbricas.

Siu and Jain, "A Brief Overview of ATM"

Se cubren muchas de las características de los sistemas ATM en este documento introductorio, pero el enfoque es hacia la simulación de LAN y la administración de tráfico; también sirve como introducción a una edición especial de *Computer Communication Review* dedicada a la tecnología ATM.

Stallings, *Data and Computer Communications*, 4th ed.

En lugar de cubrir sólo los modelos de referencia OSI y TCP/IP, Stallings incluye un tercero, el SNA, por si no basta con los otros. Los tres se describen en el capítulo 10.

8.1.2. La capa física

Awdeh and Mouftah, "Survey of ATM Switch Architectures"

Cualquiera interesado en aprender más sobre el diseño de los conmutadores ATM debe buscar aquí. Tras una introducción a los conmutadores en general y a las estrategias de *buffer*, los autores estudian muchos tipos de conmutadores de barras cruzadas, trayectoria disjunta y banianos. El artículo también proporciona más de 200 referencias a otros artículos.

Bellamy, *Digital Telephony*

Todo lo que siempre quiso saber sobre el sistema telefónico y más, contenido en este libro de referencia. Particularmente interesantes son los capítulos sobre transmisión y multiplexión, conmutación digital, fibra óptica e ISDN.

De Prycker, *Asynchronous Transfer Mode*, 2nd ed.

El capítulo 4 contiene abundante información sobre los conmutadores ATM. Los principios se ilustran mediante numerosos ejemplos, incluidos los conmutadores por *knockout*, Roxanne, Coprin y Athena.

Held, *The Complete Modem Reference*, 2nd ed.

Aquí está todo lo que concebiblemente puede querer saber sobre los módems, desde las reglas de estandarización de los gobiernos de Estados Unidos y Canadá, a las técnicas y estándares de modulación, y la manera de identificar problemas en un módem enfermo.

IEEE Communications Mag., Jan. 1995, "Wireless Personal Communications"

Esta edición especial contiene siete artículos sobre diferentes aspectos de la comunicación personal inalámbrica. En conjunto, los artículos cubren propagación, métodos de acceso, principios de los receptores, aspectos de los sistemas y asuntos sobre redes.

Metcalfe, "Computer/Network Interface Design: Lessons from Arpanet & Ethernet"

Aunque los ingenieros han estado construyendo interfaces de red durante décadas, con frecuencia nos preguntamos si han aprendido algo de toda esta experiencia. En este artículo, el diseñador de Ethernet describe la manera de construir una interfaz de red, y lo que se debe hacer una vez construida. No hace concesiones, indicando lo que hizo mal y lo que hizo bien.

Padgett *et al.*, "Overview of Wireless Personal Communications"

Introducción a los sistemas celulares e inalámbricos de comunicación, y comparación entre ambos. Se cubre el estándar de Estados Unidos y el europeo.

Palais, *Fiber Optic Communication*, 3rd ed.

Los libros sobre fibra óptica tienden a estar orientados al especialista, pero éste es más accesible que la mayoría; cubre las guías de ondas, fuentes de luz, detectores de luz, acopladores, modulación, ruido y muchos otros temas.

Pandya, "Emerging Mobile and Personal Communications Systems"

Para una introducción corta y agradable a los sistemas de comunicación personales, vale la pena ver este artículo. Una de las nueve páginas contiene una lista de 70 siglas usadas en las otras ocho páginas.

Partridge, *Gigabit Networking*

Además de describir varios tipos de conmutadores ATM, el capítulo 5 también compara *buffers* de entrada y *buffers* de salida, y deriva fórmulas para el desempeño de ambos.

Spragins *et al.*, *Telecommunications Protocols and Design*

El capítulo 2 contiene una buena introducción a la tecnología de transmisión, incluidos alambres de cobre, fibra óptica, radio celular y satélites. También tiene estudios extensos de los límites de Nyquist y de Shannon, y comenta sus implicaciones.

8.1.3. La capa de enlace de datos

Black, *Data Link Protocols*

Éste es un libro completo dedicado a la capa de enlace de datos; tiene un enfoque práctico, con una gran cantidad de material sobre HDLC, LLC, PPP y otros protocolos comercialmente importantes.

Holzmann, *Design and Validation of Computer Protocols*

Los lectores interesados en los aspectos más formales de los protocolos de enlace de datos (y similares) deben leer este libro. En él se cubren la especificación, modelado, diseño correcto y pruebas de tales protocolos.

Spragins *et al.*, *Telecommunications Protocols and Design*

Los lectores interesados en aprender más sobre códigos de detección de errores y corrección de errores deben ver el capítulo 6 de este libro. La obra también cubre los principios de los protocolos elementales de enlace de datos más o menos en el mismo nivel que nuestro libro. El capítulo 7 continúa el estudio y analiza con detalle varios protocolos de enlace de datos.

Walrand, *Communication Networks: A First Course*

El capítulo 4 cubre los protocolos de enlace de datos, con énfasis en el análisis de desempeño. También se tratan los enfoques de máquina de estados finitos y de red de Petri para el diseño correcto de protocolos.

8.1.4. La subcapa de acceso al medio

Abeysundara and Kamal, "High-Speed Local Area Networks and Their Performance"

Dado que las LAN de alta velocidad son de interés precisamente por su alta velocidad, es bienvenido un artículo en el que se estudia y analiza el desempeño. En éste, el enfoque es en los

diferentes tipos de LAN de *bus*, anillo, árbol y estrella, así como sus características de retardo y de utilización.

Jain, *FDDI Handbook—High-Speed Networking Using Fiber and other Media*

Para un tratamiento detallado del FDDI (incluidos agradables tutoriales sobre fibra óptica y SONET), este libro es una buena decisión. Además de secciones grandes sobre el *hardware* y *software* de FDDI, el libro tiene una sección sobre el desempeño e incluso consejos para la compra de cables de fibra óptica.

Perlman, *Interconnections: Bridges and Routers*

El libro de Perlman es el lugar para buscar un informado pero entretenido tratamiento de los puentes (y enrutadores). El autor diseñó los algoritmos usados en el árbol de extensión IEEE 802 al igual que los algoritmos de enrutamiento del DECnet, y evidentemente es un experto en el tema.

Stallings, *Local and Metropolitan Area Nerworks*, 4th ed.

Las tres LAN IEEE 802 forman el núcleo de este libro, pero también hay material sobre otras LAN y MAN.

Walrand, *Communication Networks: A First Course*

Al igual que el libro de Stallings mencionado antes, el capítulo 5 de éste cubre el material 802 básico, más FDDI y DQDB. La orientación es hacia el análisis del desempeño de los protocolos.

8.1.5. La capa de red

Comer, *Internetworking with TCP/IP*, Vol. 1, 3rd ed.

Comer ha escrito el libro definitivo sobre la serie de protocolos TCP/IP. Los capítulos 4 a 11 tratan el IP y protocolos relacionados de la capa de red. Los otros capítulos se encargan principalmente de las capas superiores, y también valen la pena.

Huitema, *Routing in the Internet*

Si quiere saber todo lo que hay por saber sobre el enrutamiento en Internet, éste es el libro para usted. Se tratan con lujo de detalle tanto los algoritmos pronunciables (por ejemplo, RIP, CIDR y MBONE), como los impronunciables (por ejemplo, OSPF, IGRP, EGP y BGP). También están aquí las características nuevas, como multitransmisión, IP móvil y reservación de recursos.

Perlman, *Interconnections: Bridges and Routers*

En el capítulo 9, Perlman describe muchos de los asuntos implicados en diseño de algoritmos de enrutamiento unitransmisión y multitransmisión, tanto en las WAN como en las redes de varias LAN, así como su implementación en varios dispositivos. Es claro que al autor le importa el tema, habiendo dado a la sección 9.13.10 el título "Mi opinión sobre la multitransmisión de capa de red estilo IP".

Sterbenz *et al.*, "Report on the IEEE ComSoc Gigabit Networking Workshop"

Antes de que sean utilizables las redes de gigabits, deben resolverse cuestiones básicas. Una que es crucial es si estas redes usarán ATM, TCP/IP o ambos. Para entender mejor estos temas, el IEEE organizó un taller de trabajo en abril de 1995, del que se presenta un resumen aquí. Vale la pena leer la crítica del ATM que hace Schulzrinne si usted cree que el ATM es la solución a los problemas de telecomunicación del mundo.

Stevens, *TCP/IP Illustrated*, Vol. 1

Los capítulos 3 a 10 son un estudio detallado del IP y protocolos relacionados (ARP, RARP e ICMP), ilustrados con ejemplos.

Yang and Reddy, "A Taxonomy for Congestion Control Algorithms in Packet Switching Networks"

Los autores han diseñado una taxonomía para los algoritmos de control de congestionamiento. Las categorías principales son ciclo abierto con control de origen, ciclo abierto con control de destino, ciclo abierto con realimentación explícita y ciclo cerrado con realimentación implícita. Los autores usan esta taxonomía para describir y clasificar 23 algoritmos existentes.

8.1.6. La capa de transporte

Comer, *Internetworking with TCP/IP*, Vol. 1, 3rd ed.

Como se mencionó antes, Comer ha escrito el trabajo definitivo sobre la serie de protocolos TCP/IP.

Mogul, "IP Network Performance"

A pesar del título de este artículo, trata más el desempeño del TCP y las redes en general, que el desempeño del IP en particular; está lleno de pautas y reglas empíricas útiles.

Stallings, *Data and Computer Communications*, 4th ed.

El capítulo 12 trata los protocolos de transporte y cubre los servicios y mecanismos de manera abstracta, al igual que, con detalle, los protocolos de transporte OSI y TCP.

Stevens, *TCP/IP Illustrated*, Vol. 1

Los capítulos 17-24 dan un tratamiento detallado del TCP, ilustrado con ejemplos.

8.1.7. La capa de aplicación

Anderson, R., "Why Cryptosystems Fail"

Según Anderson, la seguridad de los sistemas bancarios es mala, pero no debido a que intrusos astutos violen el DES desde sus PC. Los problemas reales van desde los empleados

deshonestos (un oficinista que cambia la dirección de correo de un cliente a la suya propia para interceptar el número de tarjeta bancaria y PIN) a los errores de programación (dar a todos los clientes el mismo número de PIN). Lo que es especialmente interesante es la respuesta que dan los bancos al confrontar un error: nuestros sistemas son perfectos y, por tanto, todos los errores deben ser causados por errores del cliente o un fraude.

Berghel, "The Client Side of the Web"

Introducción informal a los visualizadores de la *Web* y a las características que pueden apoyar. Los temas principales son manejo de HTML/HTTP, desempeño, capacidad de reconfiguración, integración con los equipos de escritorio y herramientas de navegación. Se hace la comparación de estos aspectos entre nueve visualizadores populares.

Berners-Lee *et al.*, "The World Wide Web"

Una perspectiva de la *Web* y hacia dónde va, desde el punto de vista de la persona que la inventó. El artículo se enfoca hacia la arquitectura de la *Web*, HTTP y HTML, así como las direcciones futuras.

Carl-Mitchell and Quarterman, *Practical Internetworking with TCP/IP and UNIX*

El capítulo 5 presenta una buena introducción a los nombres y el DNS, incluidas autoridades de nombres, arquitectura operativa y base de datos DNS.

Choudbury *et al.*, "Copyright Protection for Electronic Publishing on Computer Networks"

Aunque numerosos libros y artículos describen los algoritmos criptográficos, pocos describen la manera en que pueden usarse para evitar que los usuarios distribuyan más los documentos que tienen permitido descifrar. Este artículo describe una variedad de mecanismos que podrían ayudar a proteger los derechos de autor en el área electrónica.

Furht *et al.*, "Design Issues for Interactive Television Systems"

El vídeo a solicitud presenta muchos problemas técnicos complicados que se relacionan con arquitectura del sistema, topología de red, diseño de servidores y diseño de cajas de control. En este artículo, los autores presentan un tutorial sobre algunos de los problemas clave y algunas soluciones que se están investigando.

Handley and Crowcroft, *The World Wide Web—Beneath the Surf*

Aunque el 99% de los libros sobre la WWW simplemente dicen cómo usar un visualizador o listar algunos URL interesantes, éste explica el funcionamiento interno de la *Web*. El lado del cliente, el del servidor y el HTML se explican sin indigestar al lector.

Kaufman *et al.*, *Network Security*

Este informado y frecuentemente ingenioso libro es el primer lugar para buscar más información sobre seguridad en las redes. Se explican con detalle algoritmos y protocolos de clave secreta y

pública, parcialización de mensajes, validación de identificación, Kerberos y correo electrónico. Las mejores partes son los debates entre los autores (e inclusive con ellos mismos), etiquetados con subíndices, como: "Yo$_2$ no pudo hacer que yo$_1$ fuera más específico..."

Kumar, *MBone: Interactive Multimedia on the Internet*

La cubierta de este libro dice: "Descubra la manera de difundir, anunciar y presentar sus productos en Internet". Afortunadamente, el tema no se menciona en ningún otro lugar del libro. Lo que sí se cubre es la arquitectura e implementación del MBone, incluyendo mucho material sobre su funcionamiento y su uso.

Memeth *et al.*, *UNIX System Administration Handbook*

El capítulo 16 es una larga introducción al DNS. Aborda todos los detalles escabrosos, ilustrando los diferentes archivos y registros de recursos con numerosos ejemplos. También se cubren con algún detalle los programas y otras herramientas usados para administrar un servidor DNS.

Rose, *The Internet Message*

Si le gusta su correo electrónico servido con una pizca de iconoclastia, este libro es una buena apuesta. El autor no duda en levantar la voz de vez en cuando para anunciar lo que está mal en el mundo. Cuando uno logra entrar en el tema, su gusto no resulta malo.

Schneier, *Applied Cryptography*, 2nd ed.

Este compendio monumental es la peor pesadilla de la NSA: un solo libro que describe todos los algoritmos criptográficos conocidos. Para empeorar (o mejorar, según su punto de vista) las cosas, el libro contiene la mayoría de los algoritmos como programas ejecutables (en C). Además, se proporcionan más de 1600 referencias a la literatura criptográfica. Si *en realidad* quiere mantener secretos sus archivos, lea este libro.

Steinmetz and Nahrstedt, *Multimedia: Computing, Communications and Applications*

Aunque algo caótico, este libro cubre mucho terreno de los multimedia. Los temas tratados con detalle incluyen audio, imágenes fijas, imágenes en movimiento, compresión, almacenamiento óptico, sistemas operativos multimedia, redes, hipertexto, sincronización de corrientes y aplicaciones multimedia.

Van der Linden, *Just Java*

Cuando el capítulo 1 de un libro se titula "Entra en mi estancia, le dijo la araña a la mosca", muy probablemente se trata de un cuento de hadas o un libro sobre la WWW. Éste es sobre la *Web*, específicamente sobre el lenguaje Java y su entorno. Para la gente que quiere jugar con Java, el libro incluye el sistema Java completo en CD-ROM.

8.2. BIBLIOGRAFÍA ALFABÉTICA

ABEYSUNDARA, B.W., and KAMAL, A.E.: "High-Speed Local Area Networks and Their Performance" *Computing Surveys*, vol. 23, pp. 221-264, June 1991.

ABRAMSON, N.: "Development of the ALOHANET," *IEEE Trans. on Information Theory*, vol. IT-31, pp. 119-123, March 1985.

ADAM, J.A.: "Privacy and Computers," *IEEE Spectrum*, vol. 32, pp. 46-52, Dec. 1995.

ADAMS, N., GOLD, R., SCHILIT, B.N., TSO, M.M., and WANT, R.: "An Infrared Network for Mobile Computers," *Proc. USENIX Mobile and Location-Independent Computing Symposium*, USENIX, pp. 41-51, 1993.

ANDERSON, R.J.: "Why Cryptosystems Fail," *Commun. of the ACM*, vol. 37, pp. 32-40, Nov. 1994.

ARMBRUSTER, H.: "The Flexibility of ATM: Supporting Future Multimedia and Mobile Communications," *IEEE Commun. Magazine*, vol. 33, pp. 76-84, Feb. 1995.

ARMITAGE, G.J., and ADAMS, K.M.: "How Efficient is IP over ATM Anyway?" *IEEE Network Magazine*, vol. 9, pp. 18-26, Jan./Feb. 1995.

ARNOLD, K., and GOSLING, J.: *The Java Programming Language*, Reading, MA: Addison-Wesley, 1996.

AT&T and BELLCORE: "Observations of Error Characteristics of Fiber Optic Transmission Systems," CCITT SG XVIII, San Diego, Jan. 1989.

AWDEH, R.Y., and MOUFTAH, H.T.: "Survey of ATM Switch Architectures," *Computer Networks and ISDN Systems*, vol. 27, pp. 1567-1613, Nov. 1995.

BAKNE, A., and BADRINATH, B.R.: "I-TCP: Indirect TCP for Mobile Hosts," *Proc. Fifteenth Int'l. Conf. on Distr. Computer Systems*, IEEE, pp. 136-143, 1995.

BALAKRISHNAN, H., SESHAN, S, and KATZ, R.H.: "Improving Reliable Transport and Handoff Performance in Cellular Wireless Networks," *Proc. ACM Mobile Computing and Networking Conf.*, ACM, pp. 2-11, 1995.

BALLARDIE, T., FRANCIS, P., and CROWCROFT, J.: "Core Based Trees (CBT)," *Proc. SIGCOMM '93 Conf.*, ACM, pp. 85-95, 1993.

BANTZ, D.F., and BAUCHOT, F.J.: "Wireless LAN Design Alternatives," *IEEE Network Magazine*, vol. 8, pp. 43-53, March/April, 1994.

BARANSEL, C., DOBOSIEWICZ, W., and GBURZYNSKI, P.: "Routing in Multihop Packet Switching Networks: Gb/s Challenge," *IEEE Network Magazine*, vol. 9, pp. 38-61, May/June, 1995.

BARLOW, J.P.: "Property and Speech: Who Owns What You Say in Cyberspace," *Commun. of the ACM*, vol. 38, pp. 19-22, Dec. 1995.

BATCHER, K.E.: "Sorting Networks and Their Applications," *Proc. AFIPS Spring Joint Computer Conf.*, vol. 32, pp. 307-315, 1968.

BATES, R.J.: *Wireless Networked Communications*, New York: McGraw-Hill, 1994.

BERGHEL, H.L.: "The Client Side of the Web," *Commun. of the ACM*, vol. 39, pp. 33-40, Jan. 1996.

BELLAMY, J.: *Digital Telephony*, New York: John Wiley, 1991.

BELLMAN, R.E.: *Dynamic Programming*, Princeton, NJ: Princeton University Press, 1957.

BELSNES, D.: "Flow Control in the Packet Switching Networks," *Communications Networks*, Uxbridge, England: Online, pp. 349-361, 1975.

BERNERS-LEE, T., CAILLAU, A., LOUTONEN, A., NIELSEN, H.F., and SECRET, A.: "The World Wide Web," *Commun. of the ACM*, vol. 37, pp. 76-82, Aug. 1994.

BERTSEKAS, D., and GALLAGER, R.: *Data Networks*, 2nd ed., Englewood Cliffs, NJ: Prentice Hall, 1992.

BHARGHAVAN, V., DEMERS, A., SHENKER, S., and ZHANG, L.: "MACAW: A Media Access Protocol for Wireless LANs," *Proc. SIGCOMM '94 Conf.*, ACM, pp. 212-225, 1994.

BIHAM, E., and SHAMIR, A.: *Differential Cryptanalysis of the Data Encryption Standard*, New York: Springer-Verlag, 1993.

BINDER, R.: "A Dynamic Packet Switching System for Satellite Broadcast Channels," *Proc. Int'l. Conf. on Commun.*, pp. 41-1 to 41-5a, 1975.

BLACK, U.D.: *TCP/IP and Related Protocols*, New York: McGraw-Hill, 1995.

BLACK, U.D.: *Emerging Communications Technologies*, Englewood Cliffs, NJ: Prentice Hall, 1994.

BLACK, U.D.: *Data Link Protocols*, Englewood Cliffs, NJ: Prentice Hall, 1993.

BLAZE, M.: "Protocol Failure in the Escrowed Encryption Standard," *Proc. Second ACM Conf. on Computer and Commun. Security*, ACM, pp. 59-67, 1994.

BOGINENI, K., and SIVALINGAM, K.M.: "Low-Complexity Multiple Access Protocols for Wavelength-Division Multiplexed Photonic Networks," *IEEE Journal on Selected Areas in Commun.*, vol. 11, pp. 590-604, May 1993.

BONOMI, F., and FENDICK, K.W.: "The Rate-Based Flow Control Framework for the Available Bit-rate ATM Service," *IEEE Network Magazine*, vol. 9, pp. 25-39, March/April 1995.

BOWMAN, C.M., DANZIG, P.B., HARDY, D.R., MANBER, U., and SCHWARTZ, M.F.: "The Harvest Information Discovery and Access System," *Computer Networks and ISDN Systems*, vol. 28, pp. 119-125, Dec. 1995.

BOWMAN, C.M., DANZIG, P.B., MANBER, U., and SCHWARTZ, M.F.: "Scalable Internet Resource Discovery: Research Problems and Approaches," *Commun. of the ACM*, vol. 37, pp. 98-107, Aug. 1994.

BRAKMO, L.S., O'MALLEY, S.W., and PETERSON, L.L.: "TCP Vegas: New Techn. for Congestion Detection and Avoidance," *Proc. SIGCOMM '94 Conf.*, ACM, pp. 24-35, 1994.

BROADHEAD, M.A. and OWEN, C.B.: "Direct Manipulation of MPEG Compressed Digital Audio," *Proc. of ACM Multimedia '95*, ACM, pp. 499-507, 1995.

BROWN, L., KWAN, M., PIEPRZYK, J., and SEBERRY, J.: "Improving Resistance to Differential Cryptanalysis and the Redesign of LOKI," *ASIACRYPT '91 Abstracts*, pp. 25-30, 1991.

BUFORD, J.F.K. (Ed.): *Multimedia Systems*, Reading, MA: Addison-Wesley, 1994.

BURROWS, M., ABADI, M, and NEEDHAM, R.M.: "A Logic of Authentication," DEC System Research Center Report, Feb. 1990.

CAMPBELL, A., COULSON, G., and HUTCHISON, D.: "A Quality of Service Architecture," *Computer Commun. Rev.*, vol. 24, pp. 6-27, April 1994.

CAMPIONE, M., and WALRATH, K.: *The Java Language Tutorial: Object-Oriented Programming for the Internet*, Reading, MA: Addison-Wesley, 1996.

CAPETANAKIS, J.I.: "Tree Algorithms for Packet Broadcast Channels," *IEEE Trans. on Information Theory*, vol. IT-25, pp. 505-515, Sept. 1979.

CARL-MITCHELL, S., and QUARTERMAN, J.S.: *Practical Internetworking with TCP/IP and UNIX*, Reading, MA: Addison-Wesley, 1993.

CATLETT, C.E.: "In Search of Gigabit Applications," *IEEE Commun. Magazine*, vol. 30, pp. 42-51, April 1992.

CERF, V., and KAHN, R.: "A Protocol for Packet Network Interconnection," *IEEE Trans. on Commun.*, vol. COM-22, pp. 637-648, May 1974.

CHANDRANMENON, G.P., and VARGHESE, G.: *"Trading Packet Headers for Packet Processing," Proc. SIGCOMM '95 Conf., ACM, pp. 162-173, 1995.*

CHANG, Y.-H., COGGINS, D., PITT, D., SKELLERN, D., THAPAR, M., and VENKATRAMAN, C.: "An Open-System Approach to Video on Demand," *IEEE Commun. Magazine*, vol. 32, pp. 68-80, May 1994.

CHAO, J.J., GHOSAL, D., SAHA, D., and TRIPATHI, S.K.: "IP on ATM Local Area Networks," *IEEE Commun. Magazine*, vol. 32, pp. 52-59, Aug. 1994.

CHAPMAN, D.E., and ZWICKY, E.D.: *Building Internet Firewalls*, Sebastopol, CA: O'Reilly, 1995.

CHEN, K.-C.: "Medium Access Control of Wireless LANs for Mobile Computing," *IEEE Network Magazine*, vol. 8, pp. 50-63, Sept./Oct. 1994.

CHEN, M., and YUM, T.-S.: "A Conflict-Free Protocol for Optical WDMA Networks," *Proc. Globecom '91*, pp. 1276-1281, 1991.

CHEN, W.Y., and WARING, D.L.: "Applicability of ADSL to Support Video Dial Tone in the Copper Loop," *IEEE Commun. Magazine*, vol. 32, pp. 102-106, May 1994.

CHERITON, D., and WILLIAMSON, C.: "VMTP as the Transport Layer for High-Performance Distributed Systems," *IEEE Commun. Magazine*, vol. 27, pp. 37-44, June 1989.

CHERVENAK, A.L.: *Tertiary Storage: An Evaluation of New Applications*, Ph.D. thesis, CSD, Univ. of California at Berkeley, 1994.

CHERVENAK, A.L., PATTERSON, D.A., and KATZ, R.H.: "Choosing the Best Storage System for Video Service," *Proc. of ACM Multimedia '95*, ACM, pp. 109-119, 1995.

CHESSON, G.L.: "XTP/PE Design Considerations," *IFIP Workshop on Protocols for High-Speed Networks*, IFIP, pp. 27-33, 1989.

CHESWICK, W.R. and BELLOVIN, S.M.: *Firewalls and Interwalls—Repelling the Wily Hacker*, Reading, MA: Addison-Wesley, 1994.

CHOUDBURY, A.K., MAXEMCHUK, N.F., PAUL, S., and SCHULZRINNE, H.G.: "Copyright Protection for Electronic Publishing on Computer Networks," *IEEE Network Magazine*, vol. 9, pp. 12-20, May/June, 1995.

CLARK, D.D.: "The Design Philosophy of the DARPA Internet Protocols," *Proc. SIGCOMM '88 Conf.*, ACM, pp. 106-114, 1988.

CLARK, D.D.: "NETBLT: A Bulk Data Transfer Protocol," RFC 998, 1987.

CLARK, D.D.: "Window and Acknowledgement Strategy in TCP," RFC 813, July 1982.

CLARK, D.D., DAVIE, B.S., FARBER, D.J., GOPAL, I.S., KADABA, B.K., SINCOSKIE, W.D., SMITH, J.M., and TENNENHOUSE, D.L.: "The Aurora Gigabit Testbed," *Computer Networks and ISDN Systems*, vol. 25, pp. 599-621, Jan. 1993.

CLARK, D.D., JACOBSON, V., ROMKEY, J., and SALWEN, H.: "An Analysis of TCP Processing Overhead," *IEEE Commun. Magazine*, vol. 27, pp. 23-29, June 1989.

CLARK, D.D., LAMBERT, M., and ZHANG, L.: "NETBLT: A High Throughput Transport Protocol," *Proc. SIGCOMM '87 Conf.*, ACM, pp. 353-359, 1987.

CLOS, C.: "A Study of Non-Blocking Switching Networks," *Bell System Tech. J.*, vol. 32, pp. 406-424, March 1953.

COMER, D.E.: *The Internet Book*, Englewood Cliffs, NJ: Prentice Hall, 1995.

COMER, D.E.: *Internetworking with TCP/IP*, vol. 1, 3rd ed., Englewood Cliffs, NJ: Prentice Hall, 1995.

COOK, A., and STERN, J.: "Optical Fiber Access—Perspectives Toward the 21st Century," *IEEE Commun. Magazine*, vol. 32, pp. 78-86, Feb. 1994.

COOPER, E.: *Broadband Network Technology*, Englewood Cliffs, NJ: Prentice Hall, 1986.

COULOURIS, G.F., DOLLIMORE, J., and KINDBERG, T.: *Distributed Systems Concepts and Design*, 2nd ed. Reading, MA: Addison-Wesley, 1994.

CRESPO, P.M., HONIG, M.L., and SALEHI, J.A.: "Spread-Time Code-Division Multiple Access," *IEEE Trans. on Commun.*, vol. 43, pp. 2139-2148, June 1995.

CRONIN, W.J., HUTCHINSON, J.D., RAMAKRISHNAN, K.K., and YANG, H.: "A Comparison of High Speed LANs," *Proc. Nineteenth Conf. on Local Computer Networks*, IEEE, pp. 40-49, 1994.

CROWCROFT, J., WANG, Z., SMITH, A., and ADAMS, J.: "A Rough Comparison of the IETF and ATM Service Models," *IEEE Network Magazine*, vol. 9, pp. 12-16, Nov./Dec. 1995.

CROWTHER, W., RETTBERG, R., WALDEN, D., ORNSTEIN, S., and HEART, F.: "A System for Broadcast Communication: Reservation-Aloha," *Proc. Sixth Hawaii Int. Conf. System Sci.*, pp. 371-374, 1973.

CUSICK, T.W., and WOOD, M.C.: "The REDOC-II Cryptosystem," *Advances in Cryptology—CRYPTO '90 Proceedings*, NY: Springer-Verlag, pp. 545-563, 1991.

DAGDEVIREN, N., NEWELL, J.A., SPINDEL, L.A., and STEFANICK, M.J.: "Global Networking with ISDN," *IEEE Commun. Magazine*, vol. 32, pp. 26-32, June 1994.

DANSKIN, J.M., DAVIS, G.M., and SONG, X.: "Fast Lossy Internet Image Transmission," *Proc. of ACM Multimedia '95*, ACM, pp. 321-332, 1995.

DANTHINE, A.A.S.: "Protocol Representation with Finite-State Models," *IEEE Trans. on Commun.*, vol. COM-28, pp. 632-643, April 1980.

DAVIS, P.T., and McGUFFIN, C.R.: *Wireless Local Area Networks*, New York: McGraw-Hill, 1995.

DAY, J.D.: "The (Un)Revised OSI Reference Model," *Computer Commun. Rev.*, vol. 25, pp. 39-55, Oct. 1995.

DAY, J.D., and ZIMMERMANN, H.: "The OSI Reference Model," *Proc. of the IEEE*, vol. 71, pp. 1334-1340, Dec. 1983.

DE JONGE, W., and CHAUM, D.: "Some Variations on RSA Signatures and Their Security," in *Advances in Cryptology—CRYPTO '86 Proceedings*, Odlyzko, A.M. (Ed.), New York: Springer Verlag, 1987.

DE PRYCKER, M.: *Asynchronous Transfer Mode*, 2nd. ed., New York: Ellis Horwood, 1993.

DEAN, D., and WALLACH, D.S.: "Security Flaws in the HotJava Web Browser," Technical Report 502, Dept. of Computer Science, Princeton Univ., 1995.

DEERING, S.E.: "SIP: Simple Internet Protocol," *IEEE Network Magazine*, vol. 7, pp. 16-28, May/June 1993.

DEERING, S.E., and CHERITON, D.R.: "Multicast Routing in Datagram Internetworks and Extended LANs," *ACM Trans. on Computer Systems*, vol. 8, pp. 85-110, May 1990.

DEERING, S.E., ESTRIN, D., FARINACCI, D., JACOBSON, V., LIU, C.-G., and WEI, L.: "An Architecture for Wide-Area Multicast Routing," *Proc. SIGCOMM '94 Conf.*, ACM, pp. 126-135, 1994.

DELODDERE, D., VERBIEST, W., and VERHILLE, H.: "Interactive Video on Demand," *IEEE Commun. Magazine*, vol. 32, pp. 82-88, May 1994.

DEMERS, A., KESHAV, S., and SHENKER, S.: "Analysis and Simulation of a Fair Queueing Algorithm," *Internetwork: Research and Experience*, vol. 1, pp. 3-26, Sept. 1990.

DENNING, D.E., and SACCO, G.M.: "Timestamps in Key Distribution Protocols," *Commun. of the ACM*, vol. 24, pp. 533-536, Aug. 1981.

DIFFIE, W., and HELLMAN, M.E.: "Exhaustive Cryptanalysis of the NBS Data Encryption Standard," *IEEE Computer Magazine*, vol. 10, pp. 74-84, June 1977.

DIFFIE, W., and HELLMAN, M.E.: "New Directions in Cryptography," *IEEE Trans. on Information Theory*, vol. IT-22, pp. 644-654, Nov. 1976.

DIJKSTRA, E.W.: "A Note on Two Problems in Connexion with Graphs," *Numer. Math.*, vol. 1, pp. 269-271, Oct. 1959.

DIRVIN, R.A., and MILLER, A.R.: "The MC68824 Token Bus Controller: VLSI for the Factory LAN," *IEEE Micro Magazine*, vol. 6, pp. 15-25, June 1986.

DIXIT, S., and SKELLY, P.: "MPEG-2 over ATM for Video Dial Tone Network," *IEEE Network Magazine*, vol. 9, pp. 30-40, Sept./Oct. 1995.

DIXON, R.C.: "Lore of the Token Ring," *IEEE Network Magazine*, vol. 1, pp. 11-18, Jan./Feb. 1987.

DOERINGER, W.A., DYKEMAN, D., KAISERSWERTH, M., MEISTER, B.W., RUDIN, H., and WILLIAMSON, R.: "A Survey of Light-Weight Transport Protocols for High-Speed Networks," *IEEE Trans. on Commun.*, vol. 38, pp. 2025-2039, Nov. 1990.

DORFMAN, R.: "Detection of Defective Members of a Large Population," *Annals Math. Statistics*, vol. 14, pp. 436-440, 1943.

ECKBERG, A.E.: "B-ISDN/ATM Traffic and Congestion Control," *IEEE Network Magazine*, vol. 6, pp. 28-37, Sept./Oct. 1992.

ECKBERG, A.E., DOSHI, B.T., and ZOCCOLILLO, R.: "Controlling Congestion in B-ISDN/ATM: Issues and Strategies," *IEEE Commun. Magazine*, vol. 29, pp. 64-70, Sept. 1991.

EDWARDS, A., and MUIR, S.: "Experience Implementing a High-Performance TCP in User-Space," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 197-205, 1995.

EL GAMAL, T.: "A Public-Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms," *IEEE Trans. on Information Theory*, vol. IT-31, pp. 469-472, July 1985.

ERIKSSON, H.: "MBone: The Multicast Backbone," *Commun. of the ACM*, vol. 37, pp. 54-60, Aug. 1994.

ESTRIN, D., REKHTER, Y., and HOTZ, S.: "Scalable Inter-Domain Routing Architecture," *Proc. SIGCOMM '92 Conf.*, ACM, pp. 40-52, 1992.

FEIG, E., and WINOGRAD, S.: "Fast Algorithms for Discrete Cosine Transformations," *IEEE Trans. on Signal Processing*, vol. 40, Sept. 1992.

FEIT, S.: *SNMP—A Guide to Network Management*, New York: McGraw-Hill, 1995.

FIORINI, D., CHIANI, M., TRALLI, V., and SALATI, C.: "Problems with HDLC," *Computer Commun. Rev.*, vol. 25, pp. 61-80, Oct. 1995.

FISCHER, W., WALLMEIER, E., WORSTER, T., DAVIS, S.P., HAYTER, A.: "Data Communications Using ATM: Architectures, Protocols, and Resource Management," *IEEE Commun. Magazine*, vol. 32, pp. 24-33, Aug. 1994.

FLOYD, S., and JACOBSON, V.: "Random Early Detection for Congestion Avoidance," *IEEE/ACM Trans. on Networking*, vol. 1, pp. 397-413, Aug. 1993.

FLUCKIGER, F.: *Understanding Networked Multimedia*, Englewood Cliffs, NJ: Prentice Hall, 1995.

FORD, L.R., Jr., and FULKERSON, D.R.: *Flows in Networks*, Princeton, NJ: Princeton University Press, 1962.

FORD, P.S., REKHTER, Y., and BRAUN, H.-W.: "Improving the Routing and Addressing of IP," *IEEE Network Magazine*, vol. 7, pp. 10-15, May/June 1993.

FORMAN, G.H., and ZAHORJAN, J.: "The Challenges of Mobile Computing," *IEEE Computer Magazine*, vol. 27, pp. 38-47, April 1994.

FRANCIS, P.: "A Near-Term Architecture for Deploying Pip," *IEEE Network Magazine*, vol. 7, pp. 30-37, May/June 1993.

FRASER, A.G.: "Early Experiments with Asynchronous Time Division Networks," *IEEE Network Magazine*, vol. 7, pp. 12-27, Jan./Feb. 1993.

FRASER, A.G.: "Towards a Universal Data Transport System," in *Advances in Local Area Networks*, Kummerle, K., Tobagi, F., and Limb, J.O. (Eds.), New York: IEEE Press, 1987.

FURHT, B., KALRA, D., KITSON, F.L., RODRIGUEZ, and WALL, W.E.: "Design Issues for Interactive Televisions Systems," *IEEE Computer Magazine*, vol. 28, pp. 25-39, May 1995.

GARCIA-HARO, J., and JAJSZCZYK, A.: "ATM Shared-Memory Switching Architectures," *IEEE Network Magazine*, vol. 8., pp. 18-26, July/Aug. 1994.

GARG, V., and WILKES, J.E.: *Wireless and Personal Communication Systems*, Englewood Cliffs, NJ: Prentice Hall, 1996.

GASMAN, L.: *Broadband Networking*, New York: Van Nostrand Reinhold, 1994.

GIACOPELLI, J.N., HICKEY, J.J., MARCUS, W.S., SINCOSKIE, W.D., and LITTLEWOOD, M.: "Sunshine: A High-Performance Self-Routing Broadband Packet Switch Architecture," *IEEE Journal on Selected Areas in Commun.*, vol. 9, pp. 1289-1298, Oct. 1991.

GOODMAN, D.J.: "Trends in Cellular and Cordless Communications," *IEEE Commun. Magazine*, vol. 29, pp. 31-40, June 1991.

GORALSKI, W.J.: *Introduction to ATM Networking*, New York: McGraw-Hill, 1995.

GOSLING, J., JOY, B., and STEELE, G.: *The Java Language Specification*, Reading, MA: Addison-Wesley, 1996.

GREEN, P.E., Jr.: *Fiber Optic Networks*, Englewood Cliffs, NJ: Prentice Hall, 1993.

HAC, ANNA: "Wireless and Cellular Architecture and Services," *IEEE Commun. Magazine*, vol. 33, pp. 98-104, Nov. 1995.

HAFNER, K., and MARKOFF, J.: *Cyberpunk*, New York: Simon and Schuster, 1991.

HAMMING, R.W.: "Error Detecting and Error Correcting Codes," *Bell System Tech. J.*, vol. 29, pp. 147-160, April 1950.

HANDEL, R., HUBER, M.N., and SCHRODER, S.: *ATM Concepts, Protocols, and Applications*, 2nd ed., Reading, MA: Addison-Wesley, 1994.

HANDLEY, M., and CROWCROFT, J.: *The World Wide Web—Beneath the Surf*, London: UCL Press, 1994.

HAWLEY, G.T.: "Historical Perspectives on the U.S. Telephone System," *IEEE Commun. Magazine*, vol. 29, pp. 24-28, March 1991.

HEIN, M., and GRIFFITHS, D.: *SNMP*, London: Thompson, 1995.

HELD, G.: *The Complete Modem Reference*, 2nd ed., New York: John Wiley, 1994.

HELLMAN, M.E.: "A Cryptanalytic Time-Memory Tradeoff," *IEEE Trans. on Information Theory*, vol. IT-26, pp. 401-406, July 1980.

HENDERSON, T.R.: "Design Principles and Performance Analysis of SSCOP: A New ATM Adaptation Layer Protocol," *Computer Commun. Review*, vol. 25, pp. 47-59, April 1995.

HOARE, C.A.R.: "Monitors, An Operating System Structuring Concept," *Commun. of the ACM*, vol. 17, pp. 549-557, Oct. 1974; Erratum in *Commun. of the ACM*, vol. 18, p. 95, Feb. 1975.

HODGE, W.W.: *Interactive Television*, New York: McGraw-Hill, 1995.

HODGE, W.W., Martin, S., POWERS, J.T., Jr.: "Video on Demand: Architectures, Systems, and Applications," *Society of Motion Picture and Television Engineers Journal*, vol. 102, pp. 791-803, Sept. 1993.

HOFFMAN, L.J. (ed.): *Building in Big Brother: The Cryptographic Policy Debate*, New York: Springer-Verlag, 1995.

HOLFELDER, W.: "MBone VCR—Video Conference Recording on the MBone," *Proc. of ACM Multimedia '95*, ACM, pp. 237-238, 1995.

HOLZMANN, G.J.: *Design and Validation of Computer Protocols*, Englewood Cliffs, NJ: Prentice Hall, 1991.

HONG, D., and SUDA, T.: "Congestion Control and Prevention in ATM Networks," *IEEE Network Magazine*, vol. 5, pp. 10-16, July/Aug. 1991.

HUANG, A., and KNAUER, S.: "Starlite: A Wideband Digital Switch," *Proc. Globecom '84*, pp. 121-125, 1984.

HUGHES, J.P., and FRANTA, W.R.: "Geographic Extension of HIPPI Channels," *IEEE Network Magazine*, vol. 8, pp. 42-53, May/June 1994.

HUI, J.: "A Broadband Packet Switch for Multi-rate Services," *Proc. Int'l. Conf. on Communications*, IEEE, pp. 782-788, 1987.

HUITEMA, C.: *IPv6: The New Internet Protocol*, Englewood Cliffs, NJ: Prentice Hall, 1996.

HUITEMA, C.: *Routing in the Internet*, Englewood Cliffs, NJ: Prentice Hall, 1995.

HUMBLET, P.A., RAMASWAMI, R., and SIVARAJAN, K.N.: "An Efficient Communication Protocol for High-Speed Packet-Switched Multichannel Networks," *Proc. SIGCOMM '92 Conf.*, ACM, pp. 2-13, 1992.

IEEE: *Communications Magazine*, vol. 33, Jan. 1995.

IEEE: *802.3: Carrier Sense Multiple Access with Collision Detection*, New York: IEEE, 1985a.

IEEE: *802.4: Token-Passing Bus Access Method*, New York: IEEE, 1985b.

IEEE: *802.5: Token Ring Access Method*, New York: IEEE, 1985c.

IOANNIDIS, J., and MAQUIRE, G.Q., Jr.: "The Design and Implementation of a Mobile Internetworking Architecture," *Proc. Winter USENIX Conf.*, USENIX, pp. 491-502, Jan. 1993.

IRMER, T.: "Shaping Future Telecommunications: The Challenge of Global Standardization," *IEEE Commun. Magazine*, vol. 32, pp. 20-28, Jan. 1994.

IVANCIC, W.D., SHALKHAUSER, M.J., and QUINTANA, J.A.: "A Network Architecture for a Geostationary Communication Satellite," *IEEE Commun. Magazine*, vol. 32, pp. 72-84, July 1994.

JABBARI, B., COLOMBO, G., NAKAJIMA, A., and KULKARNI, J. "Network Issues for Wireless Communications," *IEEE Commun. Magazine*, vol. 33, pp. 88-98, Jan. 1995.

JACOBSON, V.: "Congestion Avoidance and Control," *Proc. SIGCOMM '88 Conf.*, ACM, pp. 314-329, 1988.

JAIN, R.: "Congestion Control and Traffic Management in ATM Networks: Recent Advances and a Survey," *Computer Networks and ISDN Systems*, vol. 27, Nov. 1995.

JAIN, R.: *FDDI Handbook—High-Speed Networking Using Fiber and other Media*, Reading, MA: Addison-Wesley, 1994.

JAIN, R.: *The Art of Computer Systems Performance Analysis*, New York: John Wiley, 1991.

JAIN, R.: "Congestion Control in Computer Networks: Issues and Trends," *IEEE Network Magazine*, vol. 4, pp. 24-30, May/June 1990.

JIA, F., and MUKHERJEE, B.: "The Receiver Collision Avoidance (RCA) Protocol for a Single-Hop WDM Lightwave Network," *Journal of Lightwave Technology*, vol. 11, pp. 1053-1065, May/June 1993.

JOHNSON, D.B.: "Scalable Support for Transparent Mobile Host Internetworking," *Wireless Networks*, vol. 1, pp. 311-321, Oct. 1995.

JOHNSON, H.W.: *Fast Ethernet—Dawn of a New Network*, Englewood Cliffs, NJ: Prentice Hall, 1996.

KAHN, D.: "Cryptology Goes Public," *IEEE Commun. Magazine*, vol. 18, pp. 19-28, March 1980.

KAHN, D.: *The Codebreakers*, New York: Macmillan, 1967.

KALISKI, B.S., and ROBSHAW, M.J.B.: "Fast Block Cipher Proposal," *Proc. Cambridge Security Workshop*, Springer-Verlag, pp. 26-39, 1994.

KAMOUN, F., and KLEINROCK, L.: "Stochastic Performance Evaluation of Hierarchical Routing for Large Networks," *Computer Networks*, vol. 3, pp. 337-353, Nov. 1979.

KARN, P.: "MACA—A New Channel Access Protocol for Packet Radio," *ARRL/CRRL Amateur Radio Ninth Computer Networking Conf.*, pp. 134-140, 1990.

KAROL, M.J., HLUCHYJ, M.G., and MORGAN, S.P.: "Input Versus Output Queueing on a Space-Division Packet Switch," *IEEE Trans. on Commun.*, vol. 35, pp. 1347-1356, Dec. 1987.

KARSHMER, A.I., and THOMAS, J.N.: "Computer Networking on Cable TV Plants," *IEEE Commun. Magazine*, vol. 30, pp. 32-40, Nov. 1992.

KATZ, D., and FORD, P.S.: "TUBA: Replacing IP with CLNP," *IEEE Network Magazine*, vol. 7, pp. 38-47, May/June 1993.

KATZ, E.D., BUTLER, M., and McGRATH, R.: "A Scalable HTTP Server: The NCSA Prototype," *Computer Networks and ISDN Systems*, vol. 27, pp. 155-164, Nov. 1994.

KAUFMAN, C., PERLMAN, R., and SPECINER, M.: *Network Security*, Englewood Cliffs, NJ: Prentice Hall, 1995.

KAVAK, N.: "Data Communication in ATM Networks," *IEEE Network Magazine*, vol. 9, pp. 28-37, May/June 1995.

KENT, C.A., and MOGUL, J.C.: "Fragmentation Considered Harmful," *Proc. SIGCOMM '87 Conf.*, ACM, pp. 390-401, 1987.

KENT, S.T.: "Internet Privacy Enhanced Mail," *Commun. of the ACM*, vol. 36, pp. 48-60, Aug. 1993.

KESSLER, G.C.: *ISDN*, 2nd ed., New York: McGraw-Hill, 1993.

KESSLER, G.C., and TRAIN, D.: *Metropolitan Area Networks: Concepts, Standards, and Services*, New York: McGraw-Hill, 1992.

KIM, J.B., SUDA, T., and YOSHIMURA, M.: "International Standardization of B-ISDN," *Computer Networks and ISDN Systems*, vol. 27, pp. 5-27, Oct. 1994.

KLEINROCK, L., and TOBAGI, F.: "Random Access Techniques for Data Transmission over Packet-Switched Radio Channels," *Proc. Nat. Computer Conf.*, pp. 187-201, 1975.

KOHNO, R., MEIDAN, R., and MILSTEIN, L.B.: "Spread Spectrum Access Methods for Wireless Communication," *IEEE Commun. Magazine*, vol. 33, pp. 58-67, Jan. 1995.

KUMAR, V.: *MBone: Interactive Multimedia on the Internet*, Indianapolis, IN: New Riders, 1996.

KUNG, H.T., and MORRIS, R.: "Credit-Based Flow Control for ATM Networks," *IEEE Network Magazine*, vol. 9, pp. 40-48, March/April 1995.

KWAN, T.T., McGRATH, R.E., and REED, D.A.: "NCSA's WWW Server: Design and Performance," *IEEE Computer Magazine*, vol. 28, pp. 68-74, Nov. 1995.

KWOK, T.: "A Vision for Residential Broadband Service: ATM to the Home," *IEEE Network Magazine*, vol. 9, pp. 14-28, Sept./Oct. 1995.

KYAS, O.: *ATM Networks*, London: International Thomson Publishing, 1995.

LAI, X.: *On the Design and Security of Block Ciphers*, Konstanz, Germany: Hartung-Gorre, 1992.

LAI, X., and MASSEY, J.: "A Proposal for a New Block Encryption Standard," *Advances in Cryptology—Eurocrypt '90 Proceedings*, New York: Springer-Verlag, pp. 389-404, 1990.

LAMPSON, B.W.: "A Note on the Confinement Problem," *Commun. of the ACM*, vol. 10, pp. 613-615, Oct. 1973.

LANDAU, S.: "Zero-Knowledge and the Department of Defense," *Notices of the American Mathematical Society*, vol. 35, pp. 5-12, Jan. 1988.

LANGSFORD, A.: "The Open System User's Programming Interfaces," *Computer Networks*, vol. 8, pp. 3-12, 1984.

LA PORTA, T.F., VEERARAGHAVAN, M., AYANOGLU, E., KAROL, M., and GITLIN, R.D.: "B-ISDN: A Technological Discontinuity," *IEEE Commun. Magazine*, vol. 32, pp. 84-97, Oct. 1994.

LATIF, A., ROWLANCE, E.J., and ADAMS, R.H.: "The IBM 8209 LAN Bridge," *IEEE Network Magazine*, vol. 6, pp. 28-37, May/June 1992.

LAUDON, K.C.: "Ethical Concepts and Information Technology," *Commun. of the ACM*, vol. 38, pp. 33-39, Dec. 1995.

LE BOUDEC, J.-Y.: "The Asynchronous Transfer Mode: A Tutorial," *Computer Networks and ISDN Systems*, vol. 24, pp. 279-309, May 1992.

LEINER, B.M., COLE, R., POSTEL, J., and MILLS, D.: "The DARPA Internet Protocol Suite," *IEEE Commun. Magazine*, vol. 23, pp. 29-34, March 1985.

LEVINE, D.A., and AKYILDIZ, I.A.: "PROTON: A Media Access Control Protocol for Optical Networks with Star Topology," *IEEE/ACM Trans. on Networking*, vol. 3, pp. 158-168, April 1995.

LEVY, S.: "Crypto Rebels," *Wired*, pp. 54-61, May/June 1993.

LIN, F., CHU, P., and LIU, M.: "Protocol Verification Using Reachability Analysis: The State Space Explosion Problem and Relief Strategies," *Proc. SIGCOMM '87 Conf.*, ACM, pp. 126-135, 1987.

LIPPER, E.H., and RUMSEWICZ, M.P.: "Teletraffic Considerations for Widespread Deployment of PCS," *IEEE Network Magazine*, vol. 8, pp. 40-49, Sept./Oct. 1994.

LITTLE, T.D.C., and VENKATESH, D.: "Prospects for Interactive Video on Demand," *IEEE Multimedia Magazine*, vol. 1, pp. 14-24, Fall 1994.

LIU, C.L., and LAYLAND, J.W.: "Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment," *Journal of the ACM*, vol. 20, pp. 46-61, Jan. 1973.

LUOTONEN, A., and ALTIS, K.: "World Wide Web Proxies," *Computer Networks and ISDN Systems*, vol. 27, pp. 147-154, Nov. 1994.

MACARIO, R.C.V.: *Cellular Radio—Principles and Design*, New York: McGraw-Hill, 1993.

MACEDONIA, M.R., and BRUTZMAN, D.P.: "MBone Provides Audio and Video Across the Internet," *IEEE Computer Magazine*, vol. 27, pp. 30-36, April 1994.

MASSEY, J.L.: "SAFER K-64: A Byte-Oriented Block Ciphering Algorithm," *Proc. Cambridge Security Workshop*, Springer-Verlag, pp. 1-17, 1994.

MATSUI, M.: "Linear Cryptanalysis Method for DES Cipher," *Advances in Cryptology—Eurocrypt '93 Proceedings*, New York: Springer-Verlag, pp. 386-397, 1994.

McBRYAN, O.: "GENVL and WWWW: Tools for Taming the Web," *Proc. First Int'l. WWW Conference*, pp. 79-90, 1994.

McDYSAN, D.E., and SPOHN, D.L.: *ATM—Theory and Application*, NY: McGraw-Hill, 1995.

McKENNEY, P.E., and DOVE, K.F.: "Efficient Demultiplexing of Incoming TCP Packets," *Proc. SIGCOMM '92 Conf.*, ACM, pp. 269-279, 1992.

MENEZES, A.J., and VANSTONE, S.A.: "Elliptic Curve Cryptosystems and Their Implementation," *Journal of Cryptology*, vol. 6, pp. 209-224, 1993.

MERKLE, R.C.: "Fast Software Encryption Functions," *Advances in Cryptology—CRYPTO '90 Proceedings*, New York: Springer-Verlag, pp. 476-501, 1991.

MERKLE, R.C., and HELLMAN, M.: "On the Security of Multiple Encryption," *Commun. of the ACM*, vol. 24, pp. 465-467, July 1981.

MERKLE, R.C., and HELLMAN, M.: "Hiding and Signatures in Trapdoor Knapsacks," *IEEE Trans. on Information Theory*, vol. IT-24, pp. 525-530, Sept. 1978.

METCALFE, R.M.: "On Mobile Computing," *Byte*, vol. 20, p. 110, Sept. 1995.

METCALFE, R.M.: "Computer/Network Interface Design: Lessons from Arpanet and Ethernet," *IEEE Journal on Selected Areas in Commun.*, vol. 11, pp. 173-179, Feb. 1993.

METCALFE, R.M., and BOGGS, D.R.: "Ethernet: Distributed Packet Switching for Local Computer Networks," *Commun. of the ACM*, vol. 19, pp. 395-404, July 1976.

MIKI, T.: "The Potential of Photonic Networks," *IEEE Commun. Magazine*, vol. 32, pp. 23-27, Dec. 1994a.

MIKI, T.: "Toward the Service-Rich Era," *IEEE Commun. Magazine*, vol. 32, pp. 34-39, Feb. 1994b.

MINOLI, D.: *Video Dialtone Technology*, New York: McGraw-Hill, 1995

MINOLI, D., and VITELLA, M.: *ATM & Cell Relay for Corporate Environments*, New York: McGraw-Hill, 1994.

MIRCHANDANI, S., and KHANNA, R. (eds): *FDDI Technologies and Applications*, New York: John Wiley, 1993.

MISHRA, P.P. and KANAKIA, H.: "A Hop by Hop Rate-Based Congestion Control Scheme," *Proc. SIGCOMM '92 Conf.*, ACM, pp. 112-123, 1992.

MOCHIDA, Y.: "Technologies for Local-Access Fibering," *IEEE Commun. Magazine*, vol. 32, pp. 64-73, Feb. 1994.

MOGUL, J.C.: "The Case for Persistent-Connection HTTP," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 299-314, 1995.

MOGUL, J.C.: "IP Network Performance," in *Internet System Handbook*, Lynch, D.C. and Rose, M.T. (eds.), Reading, MA: Addison-Wesley, pp. 575-675, 1993.

MOK, A.K., and WARD, S.A.: "Distributed Broadcast Channel Access," *Computer Networks*, vol. 3, pp. 327-335, Nov. 1979.

MORALES, J., PATKA, A., CHOA, P., and KUI, J.: "Video Dial Tone Sessions," *IEEE Network Magazine*, vol. 9, pp. 42-47, Sept./Oct. 1995.

MOY, J.: "Multicast Routing Extensions," *Commun. of the ACM*, vol. 37, pp. 61-66, Aug. 1994.

MULLENDER, S.J. (ed.): *Distributed Systems*, 2nd ed., New York: ACM Press, 1993.

MYLES, A., and SKELLERN, D.: "Comparison of Mobile Host Protocols for IP," *Computer Networks and ISDN Systems*, vol. 26, pp. 349-355, Dec. 1993.

NAGLE, J.: "On Packet Switches with Infinite Storage," *IEEE Trans. on Commun.*, vol. COM-35, pp. 435-438, April 1987.

NAGLE, J.: "Congestion Control in TCP/IP Internetworks," *Computer Commun. Rev.*, vol. 14, pp. 11-17, Oct. 1984.

NEEDHAM, R.M., and SCHROEDER, M.D.: "Authentication Revisited," *Operating Systems Rev.*, vol. 21, p. 7, Jan. 1987.

NEEDHAM, R.M., and SCHROEDER, M.D.: "Using Encryption for Authentication in Large Networks of Computers," *Commun. of the ACM*, vol. 21, pp. 993-999, Dec. 1978.

NELSON, M.N., and LINTON, M.: "A Highly Available, Scalable ITV System," *Proc. Fifteenth Symp. on Operating Systems Prin.*, ACM, pp. 54-67, 1995.

NEMETH, E., SNYDER, G., SEEBASS, S., and HEIN, T.R.: *UNIX System Administration Handbook*, Englewood Cliffs, NJ: Prentice Hall, 1995.

NEMZOW, M.: *Implementing Wireless Networks*, New York: McGraw-Hill, 1995.

NEUMAN, B.C., and TS'O, T.: "Kerberos: An Authentication Service for Computer Networks," *IEEE Commun. Magazine*, vol. 32, pp. 33-38, Sept. 1994.

NEWMAN, P.: "Traffic Management for ATM Local Area Networks," *IEEE Commun. Magazine*, vol. 32, pp. 44-50, Aug. 1994.

NEWMAN, P.: "ATM Local Area Networks," *IEEE Commun. Magazine*, vol. 32, pp. 86-98, March 1994.

NIST: "Secure Hash Algorithm," U.S. Government Federal Information Processing Standard 180, 1993.

OMIDYAR, C.G., and ALDRIDGE, A.: "Introduction to SDH/SONET," *IEEE Commun. Magazine*, vol. 31, pp. 30-33, Sept. 1993.

OTWAY, D., and REES, O.: "Efficient and Timely Mutual Authentication," *Operating Systems Rev.*, pp. 8-10, Jan. 1987.

PADGETT, J.E., GUNTHER, C.G., and HATTORI, T.: "Overview of Wireless Personal Communications," *IEEE Commun. Magazine*, vol. 33, pp. 28-41, Jan. 1995.

PAFF, A.: "Hybrid Fiber/Coax in the Public Telecommunications Infrastructure," *IEEE Commun. Magazine*, vol. 33, pp. 40-45, April 1995.

PAHLAVAN, K., PROBERT, T.H., and CHASE, M.E.: "Trends in Local Wireless Networks," *IEEE Commun. Magazine*, vol. 33, pp. 88-95, March 1995.

PALAIS, J.C.: *Fiber Optic Commun.*, 3rd ed., Englewood Cliffs, NJ: Prentice Hall, 1992.

PALMER, L.C., and WHITE, L.W.: "Demand Assignment in-the ACTS LBR System," *IEEE Trans. on Commun.*, vol. 38, pp. 684-692, May 1990.

PAN, D.: "A Tutorial on MPEG/Audio Compression," *IEEE Multimedia Magazine*, vol. 2, pp.60-74, Summer 1995.

PANCHA, P., and EL ZARKI, M.: "MPEG Coding for Variable Bit Rate Video Transmission," *IEEE Commun. Magazine*, vol. 32, pp. 54-66, May 1994.

PANDYA, R.: "Emerging Mobile and Personal Communication Systems," *IEEE Commun. Magazine*, vol. 33, pp. 44-52, June 1995.

PARTRIDGE, C.: *Gigabit Networking*, Reading, MA: Addison-Wesley, 1994.

PARTRIDGE, C.: "A Proposed Flow Specification," Internet RFC 1363, Sept. 1992.

PARTRIDGE, C., HUGHES, J., and STONE, J.: "Performance of Checksums and CRCs over Real Data," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 68-76, 1995.

PARULKAR, G., SCHMIDT, D.C., and TURNER, J.S.: "AITPM: A Strategy for Integrating IP with ATM," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 49-58, 1995.

PAXSON, V.: "Growth Trends in Wide-Area TCP Connections," *IEEE Network Magazine*, vol. 8, pp. 8-17, July/Aug. 1994.

PAXSON, V., and FLOYD, S.: "Wide-Area Traffic: The Failure of Poisson Modeling," *Proc. SIGCOMM '94 Conf.*, ACM, pp. 257-268, 1995.

PERKINS, C.: "Providing Continuous Network Access to Mobile Hosts Using TCP/IP," *Computer Networks and ISDN Systems*, vol. 26, pp. 357-370, Nov. 1993.

PERLMAN, R.: *Interconnections: Bridges and Routers*, Reading, MA: Addison-Wesley, 1992.

PERLMAN, R.: *Network Layer Protocols with Byzantine Robustness*, Ph.D. thesis, M.I.T., 1988.

PERRY, T.S., and ADAM, J.A.: "E-Mail: Pervasive and Persuasive," *IEEE Spectrum*, vol. 29, pp. 22-28, Oct. 1992.

PETERSON, W.W., and BROWN, D.T.: "Cyclic Codes for Error Detection," *Proc. IRE*, vol. 49, pp. 228-235, Jan. 1961.

PICKHOLTZ, R.L., SCHILLING, D.L., and MILSTEIN, L.B.: "Theory of Spread Spectrum Communication—A Tutorial," *IEEE Trans. on Commun.*, vol. COM-30, pp. 855-884, May 1982.

PIERCE, J.: "How Far Can Data Loops Go?" *IEEE Trans. on Commun.*, vol. COM-20, pp. 527-530, June 1972.

PINKERTON, B.: "Finding What People Want: Experiences with the WebCrawler," *Proc. First Int'l. WorldWide Web Conference*, 1994.

PISCITELLO, D.M., and CHAPIN, A.L.: *Open Systems Networking: TCP/IP and OSI*, Reading, MA: Addison-Wesley, 1993.

PITT, D.A.: "Bridging—The Double Standard," *IEEE Network Magazine*, vol. 2, pp. 94-95, Jan. 1988.

QUICK, R. F., Jr., and BALACHANDRAN, K.: "An Overview of the Cellular Digital Packet Data (CDPD) System," *Fourth Int'l. Symp. on Personal, Indoor, and Mobile Radio Commun.*, pp. 338-343, 1993.

QUISQUATER, J.-J., and GIRAULT., M.: "Chinese Lotto as an Exhaustive Code-Breaking Machine," *IEEE Computer Magazine*, vol. 24, pp. 14-22, Nov. 1991.

RABIN, M.O.: "Digital Signatures and Public-Key Functions as Intractable as Factorization," Technical Report LCS-TR-212, M.I.T., Jan 1979.

RAHNEMA, M.: "Overview of the GSM System and Protocol Architecture," *IEEE Commun. Magazine*, vol. 31, pp. 92-100, April 1993.

RAJAGOPALAN, B.: "Reliability and Scaling Issues in Multicast Communication," *Proc. SIGCOMM '92 Conf.*, ACM, pp. 188-198, 1992.

RANSOM, M.N.: "The VISTAnet Gigabit Network Testbed," *Journal of High Speed Networks*, vol. 1, pp. 49-60, 1992.

RAO, S.K., and HATAMIAN, M.: "The ATM Physical Layer," *Computer Commun. Rev.*, vol. 25, pp. 73-81, April 1995.

RIVEST, R.L.: "The MD5 Message-Digest Algorithm," RFC 1320, April 1992.

RIVEST, R.L., and SHAMIR, A.: "How to Expose an Eavesdropper," *Commun. of the ACM*, vol. 27, pp. 393-395, April 1984.

RIVEST, R.L., SHAMIR, A., and ADLEMAN, L.: "On a Method for Obtaining Digital Signatures and Public Key Cryptosystems," *Commun. of the ACM*, vol. 21, pp. 120-126, Feb. 1978.

ROBERTS, L.: "Dynamic Allocation of Satellite Capacity through Packet Reservation," *Proc. NCC*, AFIPS, pp. 711-716, 1973.

ROBERTS, L.: "Extensions of Packet Communication Technology to a Hand Held Personal Terminal," *Proc. Spring Joint Computer Conference*, AFIPS, pp. 295-298, 1972.

ROMANOW, A., and FLOYD, S.: "Dynamics of TCP Traffic over ATM Networks," *Proc. SIGCOMM '84 Conf.*, ACM, pp. 79-88, 1994.

ROSE, M.T.: *The Simple Book*, Englewood Cliffs, NJ: Prentice Hall, 1994.

ROSE, M.T.: *The Internet Message*, Englewood Cliffs, NJ: Prentice Hall, 1993.

ROSE, M.T., and McCLOGHRIE, K.: *How to Manage Your Network Using SNMP*, Englewood Cliffs, NJ: Prentice Hall, 1995.

ROSS, F.E., and HAMSTRA, J.R.: "Forging FDDI," *IEEE Journal on Selected Areas in Commun.*, vol. 11, pp. 181-190, Feb. 1993.

SADIKU, M.N.O., and ARVIND, A.S.: "Annotated Bibliography on Distributed Queue Dual Bus (DQDB)," *Computer Commun. Rev.*, vol. 24, pp. 21-36, Jan. 1994.

SALTZER, J.H., POGRAN, K.T., and CLARK, D.D.: "Why a Ring?" *Computer Networks*, vol. 7, pp. 223-230, Aug. 1983.

SALTZER, J.H., REED, D.P., and CLARK, D.D.: "End-to-End Arguments in System Design," *ACM Trans. on Computer Systems*, vol. 2, pp. 277-288, Nov. 1984.

SANDERSON, D.W., and DOUGHERTY, D.: *Smileys*, Sebastopol, CA: O'Reilly, 1993.

SANTIFALLER, M.: "TCP/IP and ONC/NFS," Reading, MA: Addison-Wesley, 1994.

SCHNEIER, B.: *Applied Cryptography*, 2nd ed., New York: John Wiley, 1996.

SCHNEIER, B.: *E-Mail Security*, New York: John Wiley, 1995.

SCHNEIER, B.: "Description of a New Variable-Length Key, 64-Bit Block Cipher [Blowfish]," *Proc. of the Cambridge Security Workshop*, Springer-Verlag, pp. 191-204, 1994.

SCHNORR, C.P.: "Efficient Signature Generation for Smart Cards," *Journal of Cryptology*, vol. 4, pp. 161-174, 1991.

SCHOLTZ, R.A.: "The Origins of Spread-Spectrum Communications," *IEEE Trans. on Commun.*, vol. COM-30, pp. 822-854, May 1982.

SCOTT, R.: "Wide Open Encryption Design Offers Flexible Implementations," *Cryptologia*, vol. 9, pp. 75-90, Jan. 1985.

SELFRIDGE, O.G., and SCHWARTZ, R.T.: "Telephone Technology and Privacy," *Technology Rev.*, vol. 82, pp. 56-65, May 1980.

SEYBOLD, A.M.: *Using Wireless Communications in Business*, New York: Van Nostrand Reinhold, 1994.

SHACHAM, N., and McKENNEY, P.: "Packet Recovery in High-Speed Networks Using Coding and Buffer Management," *Proc. INFOCOM '90*, IEEE, pp. 124-130, 1990.

SHAH, A., and RAMAKRISHNAN, G.: *FDDI—A High Speed Network*, Englewood Cliffs, NJ: Prentice Hall, 1994.

SHANNON, C.: "A Mathematical Theory of Communication," *Bell System Tech. J.*, vol. 27, pp. 379-423, July 1948; and pp. 623-656, Oct. 1948.

SHEN, B., and SETHI, I.K.: "Inner-Block Operations on Compressed Images," *Proc. of ACM Multimedia '95*, ACM, pp. 489-498, 1995.

SHIMIZU, A., and MIYAGUCHI, S.: "Fast Data Encipherment Algorithm FEAL," *Advances in Cryptology—Eurocrypt '87 Proceedings*, NY: Springer-Verlag, pp. 267-278, 1988.

SHREEDHAR, M., and VARGHESE, G.: "Efficient Fair Queueing Using Deficit Round Robin," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 231-243, 1995.

SINGLETON, A.: "Wired on the Web," *Byte*, vol. 21, pp. 77-80, Jan. 1996.

SIPIOR, J.C., and WARD, B.T.: "The Ethical and Legal Quandary of Email Privacy," *Commun. of the ACM*, vol. 38, pp. 48-54, Dec. 1995.

SIU, K.-Y., and JAIN, R.: "A Brief Overview of ATM: Protocol Layers, LAN Emulation, and Traffic Management," *Computer Commun. Rev.*, vol. 25, pp. 6-20. April 1995.

SMITH, P.: *Frame Relay*, Reading, MA: Addison-Wesley, 1993.

SOHA, M., and PERLMAN, R.: "Comparison of Two LAN Bridge Approaches," *IEEE Network Magazine*, vol. 2, pp. 37-43, Jan./Feb. 1988.

SPAFFORD, E.H.: "The Internet Worm: Crisis and Aftermath," *Commun. of the ACM*, vol. 32, pp. 678-687, June 1989.

SPRAGINS, J.D., with HAMMOND, J.L., and PAWLIKOWSKI, K.: *Telecommunications Protocols and Design*, Reading, MA: Addison-Wesley, 1991.

STALLINGS, W.: *ISDN and Broadband ISDN with Frame Relay and ATM*, Englewood Cliffs, NJ: Prentice Hall, 1995a.

STALLINGS, W.: *Network and Internetwork Security*, Englewood Cliffs, NJ: Prentice Hall, 1995b.

STALLINGS, W.: *Protect Your Privacy: The PGP User's Guide*, Englewood Cliffs, NJ: Prentice Hall, 1995c.

STALLINGS, W.: *Data and Computer Communications*, 4th ed., New York: Macmillan, 1994.

STALLINGS, W.: *SNMP, SNMPv2, and CMIP*, Reading, MA: Addison-Wesley, 1993a

STALLINGS, W.: *Local and Metropolitan Area Networks*, 4th ed., New York: Macmillan, 1993b.

STEELE, R., WHITEHEAD, J., and WONG, W.C.: "System Aspects of Cellular Radio," *IEEE Commun. Magazine*, vol. 33, pp. 80-86, Jan. 1995a.

STEELE, R., WILLIAMS, J., CHANDLER, D., DEHGHAN, S., and COLLARD, A.: "Teletraffic Performance of GSM900/DCS1800 in Street Microcells," *IEEE Commun. Magazine*, vol. 33, pp. 102-108, March 1995b.

STEINER, J.G., NEUMAN, B.C., and SCHILLER, J.I.: "Kerberos: An Authentication Service for Open Network Systems," *Proc. Winter USENIX Conf.*, USENIX, pp. 191-201, 1988.

STEINMETZ, R., and NAHRSTEDT, K.: *Multimedia: Computing, Communications and Applications*, Englewood Cliffs, NJ: Prentice Hall, 1995.

STEPHENS, W.E., and BANWELL, T.C.: "155.52 Mb/s Data Transmission on Category 5 Cable Plant," *IEEE Commun. Magazine*, vol. 33, pp. 62-69, April 1995.

STERBENZ, J.P.G., SCHULZRINNE, H.G., and TOUCH, J.D.: "Report and Discussion of the IEEE ComSoc TCGN Gigabit Networking Workshop 1995," *IEEE Network Magazine*, vol. 9, pp. 9-29, July/Aug. 1995.

STEVENS, W.R.: *TCP/IP Illustrated*, Vol. 1, Reading, MA: Addison-Wesley, 1994.

STILLER, B.: "A Survey of UNI Signaling Systems and Protocols," *Computer Commun. Rev.*, vol. 25, pp. 21-33, April 1995.

STINSON, D.R.: *Cryptography Theory and Practice*, Boca Raton, FL: CRC Press, 1995.

SUNSHINE, C.A., and DALAL, Y.K.: "Connection Management in Transport Protocols," *Computer Networks*, vol. 2, pp. 454-473, 1978.

SUZUKI, T.: "ATM Adaptation Layer Protocol," *IEEE Commun. Magazine*, vol. 32., pp. 80-83, April 1994.

TANENBAUM, A.S.: *Distributed Operating Systems*, Englewood Cliffs, NJ: Prentice Hall, 1995.

TANENBAUM, A.S.: *Modern Operating Systems*, Englewood Cliffs, NJ: Prentice Hall, 1992.

TERAOKA, F., YOKTE, Y., and TOKORO, M.: "Host Migration Transparency in IP Networks," *Computer Commun. Rev.*, vol. 23, pp. 45-65, Jan. 1993.

THYAGARAJAN, A.S., and DEERING, S.E.: "Hierarchical Distance-Vector Multicast Routing for the MBone," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 60-66, 1995.

TOKORO, M., and TAMARU, K.: "Acknowledging Ethernet," *Compcon*, IEEE, pp. 320-325, Fall 1977.

TOLMIE, D.E.: "Gigabit LAN Issues—HIPPI, Fibre Channel, and ATM," in *Proc. High-Performance Computing and Networking*, Hertzberger, B., and Serazzi, G. (Eds.), Berlin: Springer Verlag, pp. 45-53, 1995.

TOLMIE, D.E.: "Gigabit Networking," *IEEE LTS*, vol. 3, pp. 28-36, May 1992.

TOLMIE, D.E., and RENWICK, J.: "HIPPI: Simplicity Yields Success," *IEEE Network Magazine*, vol. 7, pp. 28-32, Jan./Feb. 1993.

TOMLINSON, R.S.: "Selecting Sequence Numbers," *Proc. SIGCOMM/SIGOPS Interprocess Commun. Workshop*, ACM, pp. 11-23, 1975.

TOUCH, J.D.: "Performance Analysis of MD5," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 77-86, 1995.

TRUONG, H.L., ELLINGTON, W.W. Jr., LE BOUDEC, J.-Y., MEIER, A.X., and PACE, J.W.: "LAN Emulation on an ATM Network," *IEEE Commun. Magazine*, vol. 33, pp. 70-85, May 1995.

TUCHMAN, W.: "Hellman Presents No Shortcut Solutions to DES," *IEEE Spectrum*, vol. 16, pp. 40-41, July 1979.

TURNER, J.S.: "New Directions in Communications (or Which Way to the Information Age)," *IEEE Commun. Magazine*, vol. 24, pp. 8-15, Oct. 1986.

VAN DER LINDEN, P.: *Just Java*, Englewood Cliffs, NJ: Prentice Hall, 1996.

VAN OORSCHOT, P.C., and WIENER, M.J.: "A Known-Plaintext Attack on Two-Key Triple Encryption," *Advances in Cryptology—CRYPTO '88 Proceedings*, New York: Springer-Verlag, pp. 119-131, 1988.

VAN RENESSE, R., VAN STAVEREN, H., and TANENBAUM, A.S.: "Performance of the World's Fastest Distributed Operating System," *Operating Systems Rev.*, vol. 22, pp. 25-34, Oct. 1988.

VARGHESE, G., and LAUCK, T.: "Hashed and Hierarchical Timing Wheels: Data Structures for the Efficient Implementation of a Timer Facility," *Proc. Eleventh Symp. on Operating Systems Prin.*, ACM, pp. 25-38, 1987.

VENKATRAMANI, C., and CHIUEH, T.: "Design, Implementation, and Evaluation of a Software-Based Real-Time Ethernet Protocol," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 27-37, 1995.

VETTER, R.J., SPELL, C., and WARD, C.: "Mosaic and the World-Wide Web," *IEEE Computer Magazine*, vol. 27, pp. 49-57, Oct. 1994.

VILLAMIZAN, C., and SONG, C.: "High Performance TCP in ANSNET," *Computer Commun. Rev.*, vol. 25, pp. 45-60, Oct. 1995.

VITERBI, A.J.: *CDMA Principles of Spread Spectrum Communication*, Reading, MA: Addison-Wesley, 1995.

WADA, H., YOZAWA, T., OHNISHI, T., and TANAKA, Y.: "Mobile Computing Environment Based on Internet Packet Forwarding," *Proc. Winter USENIX Conf.*, USENIX, pp. 503-517, Jan. 1993.

WALRAND, J.: *Communication Networks: A First Course*, Homewood, IL: Irwin, 1991.

WATSON, R.W.: "Timer-Based Mechanisms in Reliable Transport Protocol Connection Management," *Computer Networks*, vol. 5, pp. 47-56, Feb. 1981.

WAYNER, P.: "Picking the Crypto Lock," *Byte*, pp. 77,80, Oct. 1995.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

WEISBAND, S.P., and REINIG, B.A.: "Managing User Perceptions of Email Privacy," *Commun. of the ACM*, vol. 38, pp. 40-47, Dec. 1995.

WIENER, M.J.: "Efficient DES Key Search," Technical Report TR-244, School of Computer Science, Carleton Univ., Ottawa, 1994.

WILLIAMS, K.A., DAM, T.Q., and DU, D.H.-C.: "A Media Access Protocol for Time and Wavelength-Division Multiplexed Passive Star Networks," *IEEE Journal on Selected Areas in Commun.*, vol. 11, pp. 560-567, May 1993.

WILLINGER, W., TAQQU, M.S., SHERMAN, R., and WILSON, D.V.: "Self-Similarity through High Variability: Statistical Analysis of Ethernet LAN Traffic at the Source Level," *Proc. SIGCOMM '95 Conf.*, ACM, pp. 100-113, 1995.

WOLTER, M.S.: "Fiber Distributed Data Interface—A Tutorial," *ConneXions*, pp. 16-26, Oct. 1990.

YANG, C.-Q., and REDDY, A.V.S.: "A Taxonomy for Congestion Control Algorithms in Packet Switching Networks," *IEEE Network Magazine*, vol. 9, pp. 34-45, July/Aug. 1995.

YEH, Y.-S., HLUCHYJ, M.G., and ACAMPORA, A.S.: "The Knockout Switch: A Simple, Modular Architecture for High-Performance Packet Switching," *IEEE Journal on Selected Areas in Commun.*, vol. 5, pp. 1274-1283, Oct. 1987.

YOUSSEF, A.M., KALMAN, E., BENZONI, L.: "Technico-Economic Methods of Radio Spectrum Assignment," *IEEE Commun. Magazine*, vol. 33, pp. 88-94, June 1995.

YUVAL, G.: "How to Swindle Rabin," *Cryptologia*, vol. 3, pp. 187-190, July 1979.

ZHANG, L.: "Comparison of Two Bridge Routing Approaches," *IEEE Network Magazine*, vol. 2, pp. 44-48, Jan./Feb. 1988.

ZHANG, L.: "RSVP A New Resource ReSerVation Protocol," *IEEE Network Magazine*, vol. 7, pp. 8-18, Sept./Oct. 1993.

ZIMMERMANN, P.R.: *The Official PGP User's Guide*, Cambridge, MA: M.I.T. Press, 1995a.

ZIMMERMANN, P.R.: *PGP: Source Code and Internals*, Cambridge, MA: M.I.T. Press, 1995b.

ZIPF, G.K.: *Human Behavior and the Principle of Least Effort: An Introduction to Human Ecology*, Cambridge, MA: Addison-Wesley, 1949.

ZIV, J., and LEMPEL, Z.: "A Universal Algorithm for Sequential Data Compression," *IEEE Trans. on Information Theory*, vol. IT-23, pp. 337-343, May 1977.

ÍNDICE

A

AAL (*véase* capa de adaptación de ATM)
AAL 1, 547-549, 753
AAL 2, 549-550, 753
AAL 3/4, 550-552, 753
AAL 5, 552-554, 753
ABR (*véase* servicio de tasa de bits disponible)
abrir primero la trayectoria más corta, 424-429
acceso múltiple con detección digital, 270-271
acceso múltiple con prevención de colisiones, 264-265
acceso múltiple por división de código, 271-275, 333
acceso múltiple por división en longitud de onda, 260-262
ACR (*véase* tasa de células real)
ACTS (*véase* satélite de tecnología de comunicaciones avanzada)
acuerdo de tres vías, 496-498
acuse negativo, 215
ADC (*véase* convertidor analógico a digital)
ADCCP (*véase* procedimiento avanzado de control de comunicación de datos)
adenda, 42, 322
administración de fichas, 32
administración postal, telegráfica y telefónica, 67
ADSL (*véase* línea asimétrica digital de suscriptor)
agente de base, 368
agente de transferencia de mensajes, 645
agente foráneo, 368
agente SNMP, 631
agente usuario, 645, 646-648
alambre de cobre, comparación con fibra, 92-94
algoritmo de aprendizaje en reversa, 311-312
algoritmo de arranque lento, 538-539
algoritmo de arranque lento de Jacobson, 538-539
algoritmo de control de congestionamientos, 374-395
  ATM, 467-471
  basado en tasa, 469-471
  cubeta con fichas, 381-384
  cubeta con fugas, 380-381
  en redes de circuitos virtuales, 386-387
  manejo de colas justo ponderado, 388-389
  multitransmisión, 393-395
  paquetes de estrangulamiento, 387-391
  principios generales, 376-378
  TCP, 536-539
algoritmo de cubeta con fichas, 381-384
algoritmo de cubeta con fugas, 380-381
algoritmo de enrutamiento, 345-374
  adaptable, 347
  basado en flujo, 353-355
  difusión, 370-372
  estado de enlace, 359-365
  host móvil, 367-370
  interred, 405-406
  inundación, 351
  jerárquico, 365-367
  multitransmisión, 372-374
  no adaptable, 347
  reenvío por trayectoria invertida, 371-372
  trayectoria más corta, 349
  vector de distancia, 355-359
algoritmo de Karn, 541
algoritmo de Nagle, 534-535
algoritmo de planificación virtual, 466
algoritmo genérico de tasa de células, 463-466
algoritmo internacional de cifrado de datos, 596-597
algoritmo RSA, 598-600, 665-666
algoritmo seguro de parcialización, 618
alias, correo electrónico, 647-648
alimentación de noticias, 675

ALOHA, 246-250
por satélite, 329
puro, 247-249
ranurado, 249-250
American National Standards Institute, 70
AMPS (*véase* sistema de telefonía móvil avanzada)
análisis de asequibilidad, 20
análisis de Fourier, 78
anillo de claves privadas, pgp, 666
anillo de claves públicas, 667
anillo en forma de estrella, 295
ANSI (*véase American National Standards Institute*)
ANSNET, 51
anuncios, IP móvil, 433
aplicación ayudante, 684
aplicaciones Internet
correo electrónico, 643-670
MBone, 756-760
noticias de red, 669-680
World Wide Web, 681-723
apocalipsis de los dos elefantes, 40-41
applet, 707-709
araña, 720
árbol basado en núcleo, 374
árbol de extensión, 371
archivo de eliminación, 672
área local de acceso y transporte, 106
área OSPF, 425
armadura ASCII, 654
ARP (*véase* protocolo de resolución de direcciones)
ARP apoderado, 423
ARP gratuito, 433
ARPANET, 35, 47-50, 71, 355, 569, 622
ARQ (*véase* solicitud de repetición automática)
arquitectura de red, 18
arquitectura de red de sistemas, 41
arreglo redundante de discos baratos, 748
artículo de noticias, ejemplo, 676
asignación de canales en las LAN, 244-246
ASN.1 (*véase* Notación de Sintaxis Abstracta-1)
aspectos sociales, 6-7
relacionados a la criptografía, 620-622
ataque de brigada de cubetas, 606
ataque de cumpleaños, 618-620
ataque de encuentro a la mitad, 594
ataque de sólo texto cifrado, 582
ataque de texto normal conocido, 582
ataque de texto normal escogido, 582
ataque del hombre a la mitad, 606
ataque por reflexión, 603
ataque por repetición, 608
atenuación, 109
en fibra, 89
ATM (*véase* modo de transferencia asíncrona)
audio digital, 724-726
Aurora, 55
autoridad certificadora, 668
autoridad certificadora de políticas, 668
Autoridad de Registro de Políticas de Internet, 668

B

backbone de multitransmisión, 756-760
backbone de OSPF, 425
banda de frecuencia, 95
banda de muy alta frecuencia, 95, 97-98
banda de muy baja frecuencia, 95, 97
banda industrial/científica/médica, 99
banda ISM (*véase* banda industrial/científica/médica)
banda VHF (*véase* banda de muy alta frecuencia)
banda VLF (*véase* banda de muy baja frecuencia)
base64, 654
base de información de administración, 632, 641-642
baud, 79
Bell Operating Company, 106
Bell System, 103
BGP (*véase* protocolo de pasarela exterior)
bit de paridad, 185
BITNET, 53
Blanca, 56
bloqueo de inicio de línea, 149
BOC (*véase Bell Operating Company*)
BOOTP, 424
BUS (*véase* servidor de difusión/desconocido)
bus doble de colas distribuidas, 11, 301-303
buzón, 645

C

cabecera, 19
correo electrónico, 646
marco, 192
cabecera de noticias, 676-677

cable coaxial, 84-86
de banda ancha, 85-86
de banda base, 84-85
cable de banda ancha, 85-86
cable de banda base, 85, 86, 276, 277, 280
cable de fibra, 90-91
cable dividido a la mitad, 86
cable subdividido, 85
caja de control, 754-756
caja P, 587-588
caja S, 587-588
calidad de servicio, 23, 460-463, 481-483
ATM, 460-463
campo de vídeo, 728
canal, 11
canal de acceso aleatorio, 243
canal de fibra, 326-327
canal disimulado, 719-720
canal multiacceso, 243
canal virtual, ATM, 450
cancelador de eco, 113
capa, 17
capa AAL, 64, 545-555
capa ATM, 449-473, 63
capa de adapatación sencilla y eficiente, 552-554
capa de adaptación de ATM, 545-555
capa de aplicación, 33-34, 37, 577-766
administración de red, 630-643
correo electrónico, 643-669
multimedia, 723-760
noticias de red, 669-680
seguridad de la red, 577-622
sistema de nombres de dominio, 622-630
World Wide Web, 681-723
capa de enlace de datos, 175-242
ATM, 235-239
control de errores, 182-183
control de flujo, 183
cuestiones de diseño, 176-183
enmarcado, 179-182
LLC, 275, 302-304
OSI, 30
protocolo HDLC, 225-228
protocolos de ejemplo, 225-239
protocolos de ventana corrediza, 202-219
protocolos elementales, 190-202
relleno de bits, 181
capa de enlace de datos (*cont.*)
relleno de caracteres, 180-181
servicios proporcionados, 176-179
capa de interred, 35-36
capa de presentación, 33
capa de red, 31, 35-36, 339-478
algoritmos de enrutamiento, 345-374
control de congestionamiento, 374-395
Internet, 412-449
interredes, 396-412
organización interna, 342-345
OSI, 31
problemas de diseño, 339-345
redes ATM, 449-473
servicios proporcionados, 340-342
capa de sesión, OSI, 32-33
capa de transporte, 31-32, 36-37, 479-576
AAL de ATM, 545-555
desempeño de red, 555-572
ejemplo, 510-521
elementos de protocolo, 488-510
Internet, 521-545
OSI, 31
servicio proporcionado, 479-487
capa física, 29-30, 77-174
medios de transmisión, 82-94
OSI, 29-30
radio celular, 155-163
satélites de comunicación, 163-170
sistema telefónico, 102-163
transmisión inalámbrica, 94-102
capa *host* a red, 38
carita, 674
Carnegie-Mellon University, 7
CASA, 56
casi vídeo a solicitud, 744
CBR (*véase* servicio de tasa de bits constante)
CCITT, 68, 119, 121, 122, 124, 142, 644
CD de audio, 724-725
CDMA (*véase* acceso múltiple por división de código)
CDPD (*véase* paquete celular digital de datos)
CDV (*véase* variación de retardo de células)
celda de radio celular, 158
Célula
ATM, 62
HTML, 701
célula de administración de recursos, 470

célula de operación y mantenimiento, 236
célula OAM (*véase* célula de operación y mantenimiento)
célula RM (*véase* célula de administración de recursos)
censura
  de CompuServe, 676
  del CMU, 7
central de conmutación de datos, 12
central de ramal privado, 142
centro de conmutación móvil, 159
centro de distribución de claves, 607-610
centro de información de redes, 417
CER (*véase* tasa de error de células)
CGI (*véase* interfaz de pasarela común)
Chapulín Colorado, 745
chip, 272
CIDR (*véase* enrutamiento interdominios sin clases)
cifrado
  de bloque, 585, 590, 595-596
  de César, 582-583
  por sustitución, 582-583
  por transposición, 583-585
cifrado (*véase* criptografía)
cifrado de enlace, 579
cifrado por sustitución monoalfabética, 582-583
circuito, 11
circuito virtual, 342-345
  comparación con datagrama, 344-345
circuito virtual conmutado, 60, 145-146
circuito virtual permanente, 60, 145-146
circuitos virtuales concatenados, 401-402
clave criptográfica, 580
clave de sesión, 602
CLR (*véase* tasa de pérdida de células)
CLUT (*véase* tabla de búsqueda de colores)
CMR (*véase* tasa de mala inserción de células)
codec, 121
codificación, 730
  entrópica, 731-732
  por fuente, 732-734
codificación de enrejado, 112
codificación diferencial, 732
codificación entrecomillada imprimible, 654
codificación estadística, 731-732
codificación Manchester, 279-280
codificación Manchester diferencial, 279-280
codificación por longitud de serie, 731
codificación por transformación, 732-733
codificación predictiva, 124
código de corrección de errores, 184
código de detección de errores, 184, 186-190
código de redundancia cíclica, 187
código polinómico, 187
Comisión Federal de Comunicaciones, 96, 100, 167
comparación de las LAN, 299-301
compendio de mensaje, 617-618
compresión de datos, 730-744
  CLUT, 732
  codificación de origen, 732-734
  codificación diferencial, 732
  codificación entrópica, 731-732
  codificación estadística, 731-732
  codificación por longitud de serie, 731
  codificación por transformación, 732
  con pérdidas, 732-734
  cuantización vectorial, 733
  sin pérdidas, 731-732
  transformación por coseno discreto, 733
CompuServe, 676
computadora *big endian*, 413
computadora *little endian*, 413
comunicación dúplex integral, 21, 113
comunicación semidúplex, 21, 113
comunicación símplex, 21
concertador de satélite, 165
Conferencia Mundial Administrativa de Radio, 95
confirmación, 25-27
conformación de tráfico, 379-380, 463-468
conmutación
  de circuitos, 130-134
  de mensajes, 131-133
  de paquetes, 133-134
  por almacenamiento y reenvío, 133
  telefónica, 130-139
conmutador
  barras cruzadas, 135-138
  por división de espacio, 136-138
  por división de tiempo, 138-139
conmutador ATM, 147-155
conmutador Batcher-baniano, 151-155
conmutador de barras cruzadas, 135-138
conmutador de punto de cruce, 135-138
conmutador por división de espacio, 136-138
conmutador por división en el tiempo, 138-139

conmutador por *knockout*, 150-151
Consejo de Arquitectura de Internet, 71
consulta recursiva, 630
conteo binario descendente, 255-256
control de admisión, 386, 468
control de congestionamiento basado en tasa, 469-471
control de enlace de alto nivel, 225-228
control de enlace lógico, 275, 302-304
control de errores de cabecera, ATM, 235-238
control de errores, 182-190
control de fluctuación, 392-393
control de flujo, 183, 502-506
control síncrono de enlace de datos, 226-227
convertidor analógico a digital, 725
corrección de errores, 184-190
correo con confidencialidad mejorada, 667-669
  comparación con pgp, 669-670
correo electrónico, 5, 53, 643-670
  agente de transferencia de mensaje, 645
  agente de usuario, 645, 646-648
  arquitectura y servicios, 645-646
  cabecera, 646
  comandos de usuario, 648-650
  confidencialidad, 663-669
  cuerpo, 646
  entrega final, 662-663
  envío, 646-648
  filtro, 662
  formato de mensaje, 650-658
  formato MIME, 653-657
  formato RFC 822, 651-653
  funciones, 645
  lectura, 648-650
  pasarela, 659-661
  primeros sistemas, 644
  sobre, 646
  transferencia de mensaje, 657-663
corriente de programa, MPEG, 743
corriente de transporte, 743
corriente elemental empacada, 743
criptoanálisis, 581
criptoanálisis diferencial, 595
criptoanálisis lineal, 595
criptografía, 577-622
  de clave pública, 597-601
  de clave secreta, 587-597
  tradicional, 580-585
criptografía de clave simétrica, 598
criptología, 581
crominancia, 728-729
CSMA (*véase* protocolo de acceso múltiple con detección de portadora)
CSMA/CD, 252-254
CSNET, 50
CTD (*véase* retardo de transferencia de células)
cuantización de vector, 733
cuenta de *shell*, 229
cuerpo, correo electrónico, 646
CVDT (*véase* tolerancia de variación de retardo de células)

D

daemon de vacaciones, 663
datagrama, 342
  comparación con circuito virtual, 344-345
datos urgentes, 524
DCE (*véase* equipo de terminación de datos)
DCS 1800, 266
decibel, 81, 724
decodificación, 730
DES (*véase* estándar de cifrado de datos)
descriptor de tráfico, 461
desempeño de red, 555-572
  IEEE 802.3, 283-285
desempeño, aspectos de, 555-572
desprendimiento de carga, 390-392
desvanecimiento multitrayectoria, 99
detección de errores, 183-184
difusión, 8
digrama, 583
dirección, 492
  de transporte, 489-492
dirección de difusión, 280
dirección de encargado, 433
dirección de multitransmisión, 280
dirección IP, 416-419
dirección jerárquica, 492
dirección plana, 492
directiva HTML, 696-699
dispersión, en fibra, 89
distancia de Hamming, 184
distorsión por retardo, 109

división, 748
división de discos, 748
DMSP (*véase* protocolo de sistema de correo distribuido)
DNS (*véase* sistema de nombres de dominio)
dominio, 623
DQDB (*véase bus* doble de colas distribuidas)
DS1, 121
DSMA (*véase* acceso múltiple con detección digital)
DSS (*véase* estándar de firmas digitales)
DTE (*véase* equipo de terminación de circuito de datos)
DVMRP (*véase* protocolo de enrutamiento de multitransmisión por vector de distancia)

E

EARN, 53
elefantes, apocalipsis de los dos, 40
email (*véase* correo electrónico)
encadenamiento de bloque cifrado, 590-591
encapsulado, Java, 713
encolamiento justo ponderado, 388-389
enmarcado, 179-182
enrutador de multitransmisión, 757-758
enrutador designado, 428
enrutador m (*véase* enrutamiento de multitransmisión)
enrutador multiprotocolo, 398
enrutamiento adaptable, 347
enrutamiento basado en flujo, 353-355
enrutamiento Bellman-Ford, 355
enrutamiento de multitransmisión, 372-374
enrutamiento de sesión, 346
enrutamiento desde el origen, 415-416
enrutamiento entre redes, 405-406
enrutamiento Ford-Fulkerson, 355
enrutamiento interdominios sin clases, 434-437
enrutamiento IS-IS, 365
enrutamiento jerárquico, 365-367
enrutamiento multidestino, 370
enrutamiento no adaptable, 347
enrutamiento por difusión, 370-372
enrutamiento por estado de enlace, 359-365
enrutamiento por trayectoria más corta, 348-352
enrutamiento por vector de distancia, 355-359
ensamblador desensamblador de paquetes, 60
entidad, 22
entidad de transporte, 480
entidad igual, 22
entrega celular, 169
envoltura síncrona de carga útil, 126-127
equipo de terminación de circuito de datos, 114
equipo de terminación de datos, 114
ER (*véase* tasa explícita)
especificación de flujo, 384-386
espectro electromagnético, 94-97
espectro extendido, 96
  secuencia directa, 96
establecer modo de respuesta normal extendido, 228
establecer modo extendido balanceado asíncrono, 228
establecimiento de conexión, 493-498
  TCP, 529-530
estación administradora, 631
estación de referencia, 329
estándar de cifrado de datos, 588-595
  ataque, 592-595
  controversia, 593
  encadenamiento, 589-592
estándar *de facto*, 67
estándar de firmas digitales, 616
estándar *de jure*, 67
estándar internacional, 70
estándar JPEG, 734-738
estándar MPEG, 738-744, 753-754
  corrientes, 743
  Marco B, 742
  Marco I, 740
  Marco P, 740-741
  macrobloque, 740-741
  MPEG-1, 738-742
  MPEG-2, 742-744
  perfiles, 742
estándares ISO
  ISO 3166, 623
  ISO 8802, 70, 275
  ISO 8859-1, 696
estandarización
  Internet, 70-72
  ISO, 69-70
  red, 66-72
  telecomunicaciones, 67-69
  vídeo a solicitud, 756-757
estandarización de redes, 66-72

estandarización de telecomunicaciones, 67-69
estrella pasiva, 92
estructura de información de administración, 639-641
éter luminífero, 276
Ethernet, 10, 276 (*véase también* IEEE 802.3)
Ethernet conmutado, 285-287
Ethernet delgado, 277
Ethernet rápido, 322-324
etiqueta, HTML, 696-699
extensiones multipropósito de correo de Internet, 653-657

F

factor de trabajo, 581
FAQ (*véase* preguntas comunes)
FCC (*véase* Comisión Federal de Comunicaciones)
FDDI (*véase* interfaz de datos distribuidos por fibra)
FDM (*véase* multiplexión por división en frecuencia)
fibra hasta la acera, 116-118, 120, 751-752
fibra hasta la casa, 116-118, 752-753
fibra multimodo, 88
fibra óptica, 87-94
  atenuación, 88-89
  comparación con el cobre, 92-94
  comparación con satélite, 168-170
  componentes de sistema, 86
  dispersión, 89
  FDDI, 319-322
  multimodo, 88
  principios básicos, 87-88
  SONET, 125-130
  unimodo, 88
  WDM, 119-121
fibra unimodo, 88
fibras ópticas (*véase también* fibra óptica)
  multimodo, 88
  unimodo, 88
ficha, 287-288, 293
filtro de paquetes, 411
firma digital, 613-620
  de clave pública, 615-616
  de clave secreta, 614-615
fluctuación, 385, 724
forma, HTML, 701-706
Foro ATM, 65
fragmentación interred, 406-409
frecuencia, 94
FTP (*véase* protocolo de transferencia de archivos)
FTTC (*véase* fibra hasta la acera)
FTTH (*véase* fibra hasta la casa)
Fuerza de Trabajo de Ingeniería de Internet, 71
funcionamiento en cascada, 209
fuzzball, 50

G

gabinete de alambrado, 83
GCRA (*véase* algoritmo genérico de tasa de células)
Gopher, 693
granja de discos, 748
grupo, 119
grupo de noticias
  creación, 674-675
  ejemplo, 673
grupo de noticias moderado, 673
grupo maestro, 119
GSM (*véase* sistema global de comunicación móvil))
guerra de flamas, 672
gusano, 720

H

haz localizador, 165
HDLC (*véase* control de enlace de datos de alto nivel)
HDTV (*véase* televisión de alta definición)
head-end, cable de, 85-86
HEC (*véase* control de errores de cabecera, ATM)
HEPNET, 53
herencia, Java, 713
herramientas de ventana abstracta, 717-718
HFC (*véase* híbrido fibra/coaxial)
híbrido fibra/coaxial, 752
hipermedia, 684
hipertexto, 682
hipervínculo, 682
HIPPI (*véase* interfaz paralela de alto desempeño)
hoja de estilos, 698
horizonte dividido, 358-359
host, 11
host móvil, algoritmo de enrutamiento, 367-370

HTML (*véase* lenguaje de marcación de hipertexto)
HTTP (*véase* protocolo de transferencia de hipertexto)

I

IAB (*véase* Consejo de Arquitectura de Internet)
IBM, 41, 226, 307, 588, 593-594
ICMP (*véase* protocolo de control de mensajes del Internet)
iconos de emoción, 674
IDEA (*véase* algoritmo internacional de cifrado de datos)
identificador universal de recursos, 695
IDU (*véase* unidad de datos de interfaz)
IEEE, 70
IEEE 802, 275-301
  comparación de LAN, 299-301
IEEE 802.2, 302-304
IEEE 802.3, 276-279
  cableado, 276-279
  codificación de señales, 279-280
  conmutado, 285-287
  desempeño, 283-285
  Ethernet rápido, 322-324
  formato de marco, 281
  protocolo, 280-283
IEEE 802.3u, 322-324
IEEE 802.4, 287-292
  mantenimiento de anillo, 290-292
  protocolo, 288-290
IEEE 802.5, 292-299
  mantenimiento de anillo, 298-299
  protocolo, 296-298
IEEE 802.6, 301-303
IETF (*véase* Fuerza de Trabajo de Ingeniería del Internet)
IGMP (*véase* protocolo de mantenimiento de grupos del Internet)
igual, 17
IMAP (*véase* protocolo interactivo de acceso de correo)
IMP (*véase* procesador de mensajes de interfaz)
IMTS (*véase* servicio telefónico móvil mejorado)
incorporación, 202-203
indicación, 25-27
ingreso remoto, 53
interbloqueo de protocolos, 222
intercambio de claves Diffie-Hellman, 605-606
interconexión de redes, 396-412
  orientada a conexiones, 401-402
  por qué es necesaria, 399-400
  sin conexiones, 401-402
interfaz de datos distribuidos por fibra, 319-322
interfaz de pasarela común, 705-706
interfaz digital de instrumentos musicales, 726
interfaz entre capas, 18
interfaz paralela de alto desempeño, 325-326
interferómetro
  Fabry-Perot, 91, 261
  Mach-Zehnder, 91, 261
Internet, 16
  administración de conexiones, 529-533
  capa de enlace de datos, 229-235
  capa de interred, 35-36, 412-449
  CIDR, 434-437
  historia, 52-54
  IP, 36, 412-419
  IP móvil, 432-434
  IPv6, 437-449
  multitransmisión, 431-432
  protocolos de enrutamiento, 424-431
  TCP, 36-37, 521-542
interred, 16
interrupción, SNMP, 632
intruso, 580
inundación, 351
inundación selectiva, 351
IP (*véase* protocolos de Internet)
IP de línea en serie, 229-230, 685
IP móvil, 432-434
IPRA (*véase* Autoridad de Registro de Políticas de Internet)
IPv4, 413-419
IPv5, 438
IPv6, 437-449
  cabecera de extensión, 443-446
  cabecera principal, 439-443
  controversias, 447-449
  direcciones, 441
  jumbograma, 445
IPX, 46

ISDN (*véase* red digital de servicios integrados)
ISDN de banda ancha, 61-65, 144-155
ISDN de banda angosta, 139-144
ISO (*véase* Organización Internacional de Estándares)
ITU (*véase* Unión Internacional de Telecomunicaciones)
ITU-R, 68
ITU-T, 68
IXC (*véase* portadora entre centrales)

J

Java, 706-720
  API, 716-718
  clase, 713-716
  descripción del lenguaje, 709-718
  herramientas de ventana abstracta, 717-718
  orientación a objetos, 712-716
  polimorfismo, 715
  seguridad, 718-720
jerarquía de almacenamiento, 746-747
jerarquía de conmutadores, teléfono, 134-135
jerarquía de protocolo, 17-20
jerarquía digital síncrona, 125-130
jerarquías USENET, 671
juicio final modificado, 104
jumbograma, 445

K

KDC (*véase* centro de distribución de claves)
Kerberos, 610-612
knowbot, 720

L

LAN (*véase* red de área local)
LAN de alta velocidad, 318-327
LAN de ATM, 471-473
LAN *token bus* (*véase* IEEE 802.4)
LAN *token ring*, 292-299
LAN voladora, 15
LAP (*véase* procedimiento de acceso de enlace)
LATA (*véase* área local de acceso y transporte)
lazo local, 104, 108-118
  de fibra, 115-118
LCP (*véase* protocolo de control de enlace)
LEC (*véase* portadora de central local)
lenguaje de marcación, 695
lenguaje de marcación de hipertexto, 691-706
  formas, 701-706
  versiones, 699-701
lenguaje de marcación estándar generalizado, 695
LES (*véase* servidor de simulación de LAN)
ley de Zipf, 746
liberación de conexión, 498-502, 530-533
liberación de una conexión, 498-502
  TCP, 530-533
límite de Nyquist, 81
límite de Shannon, 81-82
línea asimétrica digital de suscriptor, 751
línea de fases alternantes, 728-729
línea de grado de voz, 79
línea, SONET, 126
LIS (*véase* subred lógica IP)
lista de correo, 645-646
LLC (*véase* control de enlace lógico)
localizador uniforme de recursos, 692-695
  esquemas, 692-693
longitud de onda, 94
lotería china, 593
luminancia, 728-729

M

MACA (*véase* acceso múltiple con prevención de colisiones)
MACAW, 265
macrobloque, 740-741
mailto, 693
MAN (*véase* red de área metropolitana)
mapa activo, 684
máquina de búsqueda, *World Wide Web*, 720-723
máquina de estados finitos, 219-223, 519-521
máquina de flujo de datos, 8
marco
  acuse, 30
  de datos, 30
  de vídeo, 727
Marco B, MPEG, 742

marco de información, 226-227
marco de supervisión, 226-228
Marco I, MPEG, 740
marco no numerado, 226-228
Marco P, MPEG, 740-741
máscara de subred, 419
MBone (*véase backbone* de multitransmisión)
MCR (*véase* tasa de células mínima)
MD5, 618, 665
media pasarela, 398
medio físico, 18
medios continuos, 724
medios de transmisión, 82-94
mensaje de crédito, 519
método, HTTP, 690
MIB (*véase* base de información de administración)
MIDI (*véase* interfaz digital de instrumentos musicales)
MILNET, 50
MIME (*véase* extensiones multipropósito de correo Internet)
MNP 5, 112
modelo cliente-servidor, 3-4
modelo de red de Petri, 223-224
modelo de referencia, 28-44
  B-ISDN, 63-65
  comparación de OSI y TCP/IP, 38-39
  OSI, 28-35
  TCP/IP, 35-38
modelo de referencia TCP/IP, 35-38, 43-44
  comparación con OSI, 38-39
módem, 109-113
módem nulo, 114
modo de libro de código electrónico, 590
modo de realimentación de cifrado, 591-592
modo de realimentación de salida, 593
modo de referencia OSI, 28-35
  comparación con TCP/IP, 38-39
  crítica, 40-43
modo de transferencia asíncrona, 61-65
  calidad de servicio, 460-463
  canal virtual, 450
  capa de enlace de datos, 235-239
  categorías de servicio, 458-460
  conformación de tráfico, 463-468
  control de congestionamientos, 467-471
  cubeta con fugas, 466
modo de transferencia asíncrona (*cont.*)
  enrutamiento y conmutación, 455-458
  establecimiento de conexión, 452-455
  formato de célula, 450-452
  NNI, 450-451
  perspectiva, 65
  plano de control, 64
  plano de usuario, 64
  subcapa CS, 65
  subcapa PMD, 64
  subcapa SAR, 65
  subcapa TC, 64-65
  trayectoria virtual, 450
  UNI, 450-451
modo promiscuo, 306
modulación, 110-112
  amplitud, 110
  fase, 110
  frecuencia, 110
modulación de amplitud en cuadratura, 111
modulación delta, 123-124
modulación por codificación de pulsos, 121
Mosaic, 696
MOSPF (*véase* OSPF de multitransmisión)
MOTIS, 644
MSC (*véase* centro de conmutación móvil)
MTSO (*véase* oficina de conmutación de telefonía móvil)
MTU (*véase* unidad máxima de transferencia)
multicomputadora, 8
multimedia, 723-760
  audio, 724-726
  compresión de datos, 730-744
  MBone, 756-760
  vídeo, 727-730
  vídeo a solicitud, 744-756
multiplexión, 118-130, 506-508
  ascendente, 506
  descendente, 507
multiplexión por división en el tiempo, 118, 121-124, 330-333
multiplexión por división en frecuencia, 118-121, 330
multiplexión por división en longitud de onda, 119-121
multitransmisión, 8, 372, 393-395
  Internet, 431-432
multitransmisión independiente del protocolo, 760

N

N-ISDN (*véase* ISDN de banda angosta)
NAK (*véase* acuse negativo)
NAP (*véase* punto de acceso a la red)
National Institute of Standards and Technology, 70
National Security Agency, 593
National Television Standards Committee, 728-729
NCP (*véase* protocolo de control de red)
NCP (*véase* protocolo de núcleo de red)
Néctar, 56
negociación, 26
negociación de opciones, 483
NETBLT, 572
NetWare, Novell, 45-47
NIC (*véase* centro de información de redes)
NIST (*véase* National Institute of Standards and Technology)
NNTP (*véase* protocolo de transferencia de noticias de red)
nodo de conmutación de paquetes, 12
Notación de Sintaxis Abstracta 1, 633-636
notación decimal con puntos, 417
noticias, 53, 669-680, 693, 694
noticias de red (*véase* USENET)
Novell NetWare, 45-47
NREN, 51-52
NSA (*véase* National Security Agency)
NSAP (*véase* punto de acceso a servicio de red)
NSFNET, 50-52
NT1, 140-142
NT2, 141-142
NTSC (*véase* National Television Standards Committee)
núnico, 608

O

objeto
  Java, 713
  SNMP, 632, 641-642
OC-*n* (*véase* portadora óptica)
oficina central, 104
oficina central local, 104
oficina de cargo, 104
oficina de cinta rota, 133
oficina de conmutación de telefonía móvil, 159
oficina tándem, 104
oficina terminal, 104
onda milimétrica, 100
ONU (*véase* unidad de red óptica)
Organización Internacional de Estándares, 69
oruga, 720
Oryctolagus cuniculus, 18
OSPF (*véase* abrir primero la trayectoria más corta)
OSPF de multitransmisión, 760

P

PAD (*véase* ensamblador desensamblador de paquetes)
página de *Web*, 682, 683, 697
PAL (*véase* línea de fases alternantes)
paquete, 7
paquete celular digital de datos, 15, 269-271
paquete de estrangulamiento, 387-391
paquete, Java, 713
par trenzado, 83-84
  categoría 3, 83
  categoría 5, 84
par trenzado sin blindaje, 84
pared cortafuego, 410-412
pasarela, 16
pasarela de aplicación, 398, 411
pasarela de transporte, 398
patrón de constelación, 111-112
PBX (*véase* central de ramal privado)
PCA (*véase* autoridad certificadora de políticas)
PCM (*véase* modulación por codificación de pulsos)
PCM diferencial, 123
PCN (*véase* red personal de comunicaciones)
PCR (*véase* tasa de células pico)
PCS (*véase* servicios personales de comunicación)
PDU (*véase* unidad de datos de protocolo)
PEM (*véase* correo con confidencialidad mejorada)
perfil de usuario, 648
PES (*véase* corriente elemental empacada)
PGP (*véase* confidencialidad bastante buena)
pila de protocolos, 18
PIM (*véase* multitransmisión independiente del protocolo)
píxel, 729
plano de control ATM, 64

plano de usuario, ATM, 64
polimorfismo, Java, 715
polinomio generador, 187
política, 43
política de "leche", 390
política de sangrado, 390
POP (*véase* punto de presencia)
POP3 (*véase* protocolo de oficina postal-3)
portadora
  común, 67, 119
  módem, 110-111, 114
portadora de central local, 106-107
portadora de lado A, 160
portadora de lado B, 160
portadora de línea alámbrica, 160
portadora entre centrales, 106-107
portadora óptica, 128-129
portadora T1, 121-122
portadora T2, 124
portadora T3, 124
portadora T4, 124
POTS (*véase* servicio telefónico tradicional)
PPP (*véase* protocolo punto a punto)
predicción de cabecera, 567
preguntas comunes, 674
prevención de congestionamientos, 378-379
primitiva de servicio, 25-27
  ejemplo, 510-512
primitivas de servicio de transporte, 483-486
principal, 601
principio de optimalidad, 347-348
privacidad bastante buena, 664-667
  comparación con PEM, 669-670
problema de conteo a infinito, 357-358
problema de estación expuesta, 264
problema de estación oculta, 264
problema de los dos ejércitos, 499-500
procedimiento avanzado de control de comunicación de datos, 226
procedimiento de acceso a enlace, 226
procesador de interfaz de terminal, 48
procesador de mensajes de interfaz, 47
procesamiento rápido de TPDU, 565-568
proceso en túnel, 404-405
producto ancho de banda-retardo, 557
proporción de bloques de células con errores severos, 463
protocolo, 17
  802.5, 296-298
  AAL, 545-555
  AAL de ATM, 547-554
  acceso múltiple, 246-275
  ADCCP, 226
  ARP, 420-423
  ARQ, 200-202
  BGP, 429-431
  BOOTP, 424
  canal ruidoso, 197-200
  contención limitada, 256-259
  conteo descendente binario, 255-256
  CSMA, 250-254
  de 1 bit, 205-207
  DMSP, 662
  DSMA, 270-271
  DVMRP, 758-759
  elemental de enlace de datos, 190-202
  HDLC, 225-228
  HTTP, 689-691
  ICMP, 419-420
  IEEE 802.3, 280-283
  IEEE 802.4, 288-290
  IEEE 802.5, 296-298
  IGMP, 431-432
  IMAP, 662
  IP, 36, 412-419
  IPX, 46
  LAN inalámbrica, 262-265
  LAP, 226
  LAPB, 226
  LCP, 231
  libre de colisiones, 254-256
  MACA, 264-265
  MACAW, 265
  mapa de bits, 254-255
  NCP (protocolo de control de red), 231
  NCP (protocolo de núcleo de red), 46
  Needham-Schroeder, 608-609
  NNTP, 677-680
  Otway-Rees, 609-610
  PAR, 200-202
  pasarela exterior, 405-406
  pasarela interior, 405-406
  PIM, 760
  POP3, 662

protocolo (*cont.*)
  PPP, 231-235
  Q.2931, 453
  RARP, 423-424
  recorrido de árbol, 258-259
  red de gigabits, 568-572
  regresar n, 207-213
  repetición selectiva, 213-219
  reto-respuesta, 602-604
  RSVP, 394-396
  SDLC, 226-227
  símplex sin restricciones, 195-197
  SLIP, 229-230
  SMTP, 658-660
  SNMP, 642-643
  SSCOP, 555
  TCP, 36-37, 521-542
  UDP, 37, 542-544
  validación de identificación, 601-613
  ventana corrediza, 202-219
  WDMA, 260-262
protocolo de acceso múltiple con detección de portadora, 250-254
protocolo de administración de grupos de Internet, 431-432, 759
protocolo de conexión inicial, 490
protocolo de control de enlace, 231
protocolo de control de mensajes de Internet, 419-420
protocolo de control de red, 231
protocolo de control de transmisión, 36-37, 521-542, 658, 678, 685
  administración de conexión, 529-533
  administración de temporizadores, 539-542
  algoritmo de Karn, 541
  algoritmo de Nagle, 534-535
  cabecera de segmento, 526-529
  control de congestionamientos, 536-539
  modelo de servicio, 523-524
  política de transmisión, 533-536
  redes inalámbricas, 543-545
  síndrome de ventana boba, 534-535
protocolo de datagramas de usuario, 37, 542-544
protocolo de enrutamiento de multitransmisión por vector de distancia, 758-759
protocolo de mapa de bits, 254-255
protocolo de núcleo de red, 46
protocolo de oficina postal-3, 662
protocolo de pasarela exterior, 405-406, 424, 429-431
protocolo de pasarela interior, 405-406, 424-429
protocolo de recorrido de árbol, 258-259
protocolo de regresar n, 207-213
protocolo de repetición selectiva, 213-219
protocolo de reservación de recursos, 394-395
protocolo de resolución de direcciones, 420-423
  ARP gratuito, 433
protocolo de resolución de direcciones en reversa, 423-424
protocolo de reto-respuesta, 602-604
protocolo de sapo de boca ancha, 607
protocolo de servicio específico orientado a conexiones, 555
protocolo de sistema de correo distribuido, 662
protocolo de transferencia de archivos, 693
protocolo de transferencia de hipertexto, 689-691
protocolo de transporte, 488
  control de flujo, 502-506
  de Internet, 521-545
  direccionamiento, 489-492
  elementos, 488-510
  multiplexión, 506-508
protocolo de validación de identificación, 601-613
  clave pública, 612-613
  Kerberos, 610-612
  usando KDC, 607-620
protocolo de ventana corrediza, 202-219
  de 1 bit, 205-207
protocolo interactivo de acceso al correo, 662
protocolo libre de colisiones, 254-256
protocolo Needham-Schroeder, 608-609
protocolo Otway-Rees, 609-610
protocolo punto a punto, 231-235, 685
protocolo sencillo de administración de redes, 630-643
protocolo sencillo de transferencia de correo, 658-660
protocolo simple de Internet mejorado, 438
protocolo SNMP, 642-643
protocolos AAL, comparación, 554-555
protocolos de acceso múltiple, 246-275
protocolos de Internet
  ARP, 420-423, 433
  BGP, 429-431
  DVMRP, 758-759
  HTTP, 689-691
  ICMP, 419-420
  IGMP, 431-432, 759

protocolos de Internet (*cont.*)
IP, 36, 412-419
NNTP, 677-680
OSPF, 424-429
PIM, 760
PPP, 231-235, 685
RARP, 423-424
RSVP, 394-395
SLIP, 229-230, 685
SMPT, 658-660
TCP, 36-37, 521-542, 658, 678, 685
UDP, 37, 542-544
protocolos de transferencia de noticias de red, 677-680
proveedor de servicio, 22
proveedor de servicio de transporte, 481
proveedor de servicio Internet, 229
PSTN (*véase* red telefónica pública conmutada)
PTT (*véase* administración postal, telegráfica y telefónica)
publicaciones cruzadas, 672
puente, 304-318, 398
árbol de extensión, 310-313
de enrutamiento desde el origen, 314-316
entre LAN IEEE 802, 307-310
remoto, 317-318
transparente, 310-313
puerto bien conocido, 523
puerto, TCP, 523
bien conocido, 523
punto de acceso a servicio, 22
punto de acceso a servicio de red, 489
punto de acceso al servicio de transporte, 489
punto de acceso de red, 52
punto de cruce, 136
punto de presencia, 107
punto de referencia, ISDN, 142
PVC (*véase* circuito virtual permanente)

Q

Q.2931, 453
QAM (*véase* modulación de amplitud en cuadratura)
QoS (*véase* calidad de servicio)

R

radio celular, 155-163
digital, 266-275
RAID (*véase* arreglo redundante de discos baratos)
RARP (*véase* protocolo de resolución de direcciones en reversa)
Recorte, 760
recuperación de caídas, 508-510
red ATM, 144-155
red de área amplia, 11-13
red de área local, 9-10
ATM, 471-473
de alta velocidad, 318-327
Ethernet, 10, 276-287
Ethernet rápido, 322-324
IEEE 802, 275-304
reparto de canales, 244-246
token bus, 287-292
token ring, 292-299
red de área metropolitana, 10-11
red de computadoras, 2
uso, 3-7
red de difusión, 7-8
red de distribución, 750-754
red de fibra óptica, 91-94
red de gigabits, 54-56, 568-572
red de punta, 430
red de tránsito, 430
red digital de servicios integrados, 61, 139-155
red multiacceso, 425
red personal de comunicaciones, 162-163
red punto a punto, 8
red satelital, 327-333
comparación con fibra, 168-170
comunicación, 163-179
de órbita baja, 167-170
geosíncrona, 164-167
red síncrona óptica, 125-130
red telefónica pública conmutada, 102
redes inalámbricas, 13-15
hosts móviles, 367-370, 432-434
LAN inalámbricas, 262-265
ondas electromagnéticas, 94-101
radio analógica, 155-163
radio digital, 266-275
TCP inalámbrico, 543-545

reenviador anónimo, 674
reenvío por trayectoria invertida, 371-372
reflectometría de dominio de tiempo, 277
registro autorizado, 629
registro de recursos, 624-628
reino, Kerberos, 611
relación de señal a ruido, 81
relevador de células, 62
relevo de marco, 60-61
relleno de bits, 181
relleno de caracteres, 180-181
relleno de una sola vez, 585
repetición selectiva, 209
repetidor, 91-94, 279, 398
repetidor activo, 91
repetidor, satélite, 164
reservación de recursos, 468-469
resolvedor, DNS, 622
respuesta, 25-27
retardo de transferencia de células, 462
retroceso exponencial binario, 282-283
RFC (*véase también* solicitud de comentario)
RFC 768, 542
RFC 792, 420
RFC 793, 522
RFC 821, 651, 659, 761, 644
RFC 822, 644, 650, 651-653, 655, 660, 661, 665, 667, 676, 677, 688, 689, 690, 691, 761
RFC 826, 422
RFC 903, 423
RFC 951, 424
RFC 977, 677
RFC 1028, 630
RFC 1034, 622
RFC 1035, 622
RFC 1036, 676
RFC 1048, 424
RFC 1055, 229
RFC 1056, 662
RFC 1064, 622
RFC 1067, 630
RFC 1084, 424
RFC 1106, 528, 529
RFC 1112, 432
RFC 1122, 522
RFC 1144, 230
RFC (*cont.*)
RFC 1155, 630
RFC 1157, 630
RFC 1213, 642
RFC 1225, 662
RFC 1247, 424
RFC 1268, 431
RFC 1323, 522, 528
RFC 1421, 667
RFC 1422, 667
RFC 1423, 667
RFC 1424, 667
RFC 1425, 659
RFC 1441, 630
RFC 1442, 630, 639
RFC 1443, 630
RFC 1444, 630
RFC 1445, 630
RFC 1446, 630
RFC 1447, 630
RFC 1448, 630, 643
RFC 1449, 630
RFC 1450, 630
RFC 1451, 630
RFC 1452, 630
RFC 1483, 473, 554
RFC 1519, 435
RFC 1521, 653, 654, 655
RFC 1550, 437
RFC 1577, 342, 473, 554
RFC 1654, 431
RFC 1661, 231, 234
RFC 1662, 231
RFC 1663, 231
RFC 1700, 415, 523
RFC 1715, 443
RFC 1883, 438
RFC 1884, 438
RFC 1885, 438
RFC 1886, 438
RFC 1887, 438
rock and roll, relación de señal a ruido, 739
RS-232, 114-116
RS-422-A, 115
RS-423-A, 115
RS-449, 115-116

RSVP (*véase* protocolo de reservación de recursos)
rueda de temporización, 567-568
ruido, 109
ruido de cuantización, 725

S

SABME (*véase* establecer modo extendido balanceado asíncrono)
SAP (*véase* punto de acceso a servicio)
satélite de comunicaciones, 163-170
satélite de órbita baja, 167-170
satélite de tecnología de comunicación avanzada, 331
satélite geosíncrono, 164-167
SCR (*véase* tasa de células sostenida)
SDH (*véase* jerarquía digital síncrona)
SDLC (*véase* control síncrono de enlace de datos)
SDU (*véase* unidad de datos de servicio)
SEAL (*véase* capa de adaptación sencilla y eficiente)
SECAM (*véase sequentiel couleur avec memoire*)
SECBR (*véase* proporción de bloques de células con errores severos)
sección, SONET, 126
secuencia de *chips*, 272
segmento, TCP, 525
seguridad
  Java, 718-720
  teléfono celular, 161
seguridad en las redes, 577-622
señal síncrona de transporte-1, 126
señalamiento asociado a canal, 122
señalamiento en banda, 113
señalización de canal común, 122
sequentiel couleur avec memoire, 728-729
servicio
  de datagramas, 24-25
  de solicitud-respuesta, 24-25
  orientado a conexiones, 23-25
  sin conexiones, 23-25
servicio confirmado, 26-27
servicio de datos conmutado multimegabit, 57-59
servicio de tasa constante de bits, 458-459
servicio de tasa de bits variable, 459
servicio de tasa de datos disponible, 459-460
servicio de tasa de datos no especificada, 460
servicio no confirmado, 26-27
servicio telefónico móvil mejorado, 157
servicio telefónico tradicional, 142
servicios personales de comunicación, 162-163
servidor apoderado, 688
servidor ATMARP, 473
servidor de archivos, 3
servidor de difusión/desconocido, 472
servidor de directorios, 491
servidor de nombres, 491, 628-630
servidor de procesos, 491
servidor de simulación de LAN, 472
servidor de vídeo, 745-750
  software, 747-749
SGML (*véase* lenguaje de marcación estándar generalizado)
SHA (*véase* algoritmo seguro de parcialización)
sincronización, 33
síndrome de ventana boba, 534-535
sintaxis de transferencia ASN.1, 637-638
SIPP (*véase* protocolo simple de Internet mejorado)
sistema autónomo, 406, 412
sistema de avisos, 155-156
sistema de contención, 246-247, 252-258
sistema de nombres de dominio, 421, 622-630
sistema de telefonía móvil avanzada, 158-161
sistema distribuido, 2
sistema global de comunicación móvil, 266-275
sistema intermedio, 12
sistema presione para hablar, 157
sistema telefónico, 102-163
  conmutación, 130-139
  lazo local, 108-118
  política, 106-108
  portadora T1, 121-122
  SONET, 125-130
  troncales y multiplexión, 118-130
sistema terminal, 11
SLIP (*véase* IP de línea en serie)
SMDS (*véase* servicio de datos conmutado multimegabit)
SMI (*véase* estructura de información de administración)
SMTP (*véase* protocolo sencillo de transferencia de correo)
SMTP extendido, 659
SNA (*véase* arquitectura de red de sistemas)

SNMP (*véase* protocolo sencillo de administración de redes)
SNRME (*véase* establecer modo de respuesta normal extendido)
sobre de correo electrónico, 646
sobrecarga, Java, 715
Sociedad Internet, 53, 71
socket, 486-487
software de red, 16-28
solicitud, 25-27
solicitud automática de repetición, 200-202
solicitud de comentario, 71
Soliton, 89
sondeo, 328
sondeo dirigido por interrupción, 632
SONET (*véase* red óptica síncrona)
SPADE, 330
SPAN, 53
SPE (*véase* envoltura síncrona de carga útil)
SPX, 46
SSCOP (*véase* protocolo de servicio específico orientado a conexiones)
STS-1 (*véase* señal síncrona de transporte-1)
subcapa cs, atm, 65
subcapa de acceso al medio, 243-335
subcapa de convergencia AAL, 546
subcapa de línea, SONET, 129-130
subcapa de sección, SONET, 129-130
subcapa de trayectoria, SONET, 129-130
subcapa fotónica, SONET, 129
subcapa MAC (*véase* subcapa de control de acceso al medio)
subcapa PMD, ATM, 64, 147, 235-239
subcapa SAR, ATM, 65, 546
subcapa TC, ATM, 64-65, 235-239
subclase, Java, 713
subred, 11
  Internet, 417-419
subred de almacenamiento y reenvío, 12
subred de comunicaciones, 11
subred de conmutación de paquetes, 12
subred IP lógica, 473
subred punto a punto, 12
suma de comprobación, 179, 182, 187-191, 235
superclase, Java, 713
supergrupo, 119
supresor de eco, 112-113

T

tabla de búsqueda de colores, 732
tarifa, 67
tasa básica ISDN, 142-143
tasa de células mínima, 461
tasa de células pico, 461
tasa de células real, 471
tasa de células sostenida, 461
tasa de error de células, 463
tasa de mala inserción de células, 463
tasa de pérdida de células, 462
tasa explícita, 471
tasa primaria, ISDN, 142-143
TCP (*véase* protocolo de control de transmisión)
TCP indirecto, 543-544
TDM (multiplexión por división en el tiempo)
teléfono celular, 157-163
  administración de llamadas, 160-161
  AMPS, 158-161
  analógico, 157-161
  digital, 162
  seguridad, 161
teléfono inalámbrico, 157
televisión
  analógica, 727-729
  digital, 729-730
televisión de alta definición, 729
Telnet, 686-687, 693, 694
temporización, ficha, 321
temporizador de manutención de vida, TCP, 542
temporizador de persistencia, tcp, 542
temporizador de retransmisiones, TCP, 539-540
terminal de abertura muy pequeña, 165
terminal virtual de red, 33
texto cifrado, 580
texto normal, 580
tiempo de retención de ficha, 296
TIP (*véase* procesador de interfaz de terminal)
tolerancia de variación de retardo de células, 462
tormenta de difusión, 557
tostador en un poste, 168
TPDU (*véase* unidad de datos de protocolo de transporte)
trama de conmutación, 148
transceptor, 277
transferencia de archivos, 53

transformación por coseno discreto, 733
transmisión
de onda de luz, 100-102
infrarroja, 100
transmisión a cualquiera, 442
transmisión balanceada, 115
transmisión de radio, 97-98
transmisión no balanceada, 115
transmisión por microondas, 98-99
trayectoria virtual, ATM, 450
trayectoria, SONET, 126
tributario, SONET, 127
trigrama, 583
triple X, 60
troncal, 11, 118-130
troncal con conexión de cargo, 104
troncal interoficinas, 104
troncal intertarifas, 104
TSAP (*véase* punto de acceso al servicio de transporte)
tubo de bits, 140
TV por cable, 85-86, 107, 144, 172

U

UBR (*véase* servicio de tasa de datos no especificada)
UDP (*véase* protocolo de datagramas de usuario)
umbral de congestionamiento, 538
unidad de datos de protocolo, 22-23
unidad de datos de protocolo de transporte, 484
unidad de datos de servicio, 22
unidad de interfaz de datos, 22
unidad de red óptica, 751-752
unidad máxima de transferencia, 525
Unión Internacional de Telecomunicaciones, 68, 453, 471, 545, 634, 668, 734
URI (*véase* identificador universal de recursos)
URL (*véase* localizador uniforme de recursos)
USENET, 669-680, 693
implementación, 675-680
relación con la Internet, 669
vista de usuario, 670-675
usuario de servicio, 22
usuario de servicio de transporte, 481
UTP (*véase* par trenzado sin blindaje)
UUCP, 669

V

V.24, 114
V.32, 111
V.32 bis, 111
V.34, 111
V.42 bis, 112
variación de retardo de células, 462
VBR (*véase* servicio de tasa de bits variable)
vector de inicialización, 590
velocidad de la luz, 94
ventana de congestionamiento, 537-538
ventana receptora, 203
ventana transmisora, 203
verificación de protocolo, 219-224
vídeo, 727-730
analógico, 727-729
digital, 729-730
entrelazado, 728
progresivo, 728
vídeo a solicitud, 744-756
caja de control, 754-756
red de distribución, 750-754
servidor, 745-750
vídeo compuesto, 728
videoconferencia, 5
vigilancia del tráfico, 379-380
visor externo, 684
VISTAnet, 56
visualizador de la *World Wide Web*, 682
VSAT (*véase* terminal de abertura muy pequeña)
VTMP, 572

W

WAN (*véase* red de área amplia)
WARC (*véase* Conferencia Mundial Administrativa de Radio)
WDM (*véase* multiplexión por división en longitud de onda)
WDMA (*véase* acceso múltiple por división de longitud de onda)
Web (*véase World Wide Web*)
World Wide Web, 54, 681-723
CGI, 705-706

World Wide Web (*cont.*)
hipermedia, 684
hipertexto, 682
hipervínculo, 682
Java, 706-720
lenguaje HTML, 691-706
máquina de búsqueda, 720-723
obtención de una página, 685-687
protocolo HTTP, 689-691
servidor, 685-689
URL, 692-695
visor externo, 684
visualizador, 682
WWV, 494
WWW (*véase World Wide Web*)
WYSIWYG, 695

X

X.3, 60
X.21, 59
X.25, 59-60
X.28, 60
X.29, 60
X.400, 644, 661
X.509, 668-669
XTP, 572

Z

zona, DNS, 628